

ゼロからの

OS自作入門

o p e r a t i n g s y s t e m

内田公太 [著]
@uchan_nos

「MikanOS」を
手作りしながら
OSのしくみを
学べる!

すべて
自分で作る
OSの世界。

C++ による記述

UEFI BIOS 起動

Intel 64モード

USB3.0 ドライバ

ページングを用いた
メモリ管理

プリエンプティブ
マルチタスク

ウィンドウシステム

ファイルシステム

"知識ゼロ"からはじめて、 オリジナルOS 「MikanOS」を自作します。

・MikanOSはUEFI BIOSにより起動してIntel 64モードで動作、ページングを用いてメモリ管理を行い、USB3.0ドライバを搭載。ウィンドウシステム、プリエンプティブマルチタスク、ファイルシステムを持つOSです。専門用語の意味もOSを作りながら解説。

・MikanOSは C++ で記述します。本文中で必要となる説明を入れており、ある程度のプログラミング経験があれば無理なく読み進められます。

・パソコンの電源を入れOS本体を呼び出すところから様々なアプリを動かせるようになるまで。OS作りを一通り体験！



内田公太 [著]
@uchan_nos



下記のサイトにて本書のサポートを行います。質疑応答、正誤表の提供、読者同士のコミュニケーションを主な目的としています。

「ゼロからの OS 自作入門」サポートサイト：

<https://zero.osdev.jp/>

この本で作るMikanOS をビルドしたり、起動したりする方法（＝開発環境を整える方法）については、下記のGitHubリポジトリにまとめています。

<https://github.com/uchan-nos/mikanos-build>

Linux ディストリビューションの1 つであるUbuntu で開発するのが標準ですが、Windows のWSL上に用意したUbuntu を使っても開発可能です。詳細については本書「付録A」を参照してください。

マイナビ出版サポートページ：

<https://book.mynavi.jp/supportsite/detail/9784839975869.html>

- 本書は2021年2月段階での情報に基づいて執筆されています。本書に登場する製品やソフトウェア、サービスのバージョン、画面、機能、URL、製品のスペックなどの情報は、すべてその原稿執筆時点でのものです。執筆以降に変更されている可能性がありますので、ご了承ください。
- 本書に記載された内容は、情報の提供のみを目的としております。したがって、本書を用いての運用はすべてお客様自身の責任と判断において行ってください。
- 本書の制作にあたっては正確な記述につとめました。が、著者や出版社のいずれも、本書の内容に関してなんらかの保証をするものではなく、内容に関するいかなる運用結果についてもいっさいの責任を負いません。あらかじめご了承ください。
- 本書に記載されている会社名・製品名等は、一般に各社の登録商標または商標です。本文中では ©、®、TM 等の表示は省略しています。

はじめに

この本は **OS** を手作りする本です。OS 作りに関する知識が無いところから始めて、30 章で簡単な機能を持つ OS、名付けて「MikanOS」を作ります。30 章が終わった後には画面のようになります。この画面にあるものすべて、この本に書かれている内容を元に手作りしてるんですよ。もちろん、これらの機能は、他の OS の力を一切借りずに自力で動いています。

MikanOS の 30 日後の姿



この画面は MikanOS でいくつかアプリを起動させた様子です。左上では日本語の MikanOS 取扱説明書を表示しています。左下ではカラフルな立方体がぐるぐる回っています。中央には黒い背景のターミナルが起動していて、右下は富士山の JPEG 写真です。その他にもいくつか小物アプリが起動しています。右上には OS 起動時からの経過時間（0.01 秒刻み）を示すウィンドウと、その上にマウスポインタがありますね。全体的になかなか本格的な見た目じゃないでしょうか？（笑）

OS とはオペレーティングシステム（Operating System）の略です。OS はコンピュータの中で基本となるソフトウェアで、Windows、macOS、Linux などが有名です。人間がコンピュータを利用する際に OS は重要な役割を果たします。Web ブラウザやワープロなどのアプリが共通に利用する機能を提供することでアプリ開発を支援したり、計算資源を分配して複数のアプリを同時に使えるようにしたり、全体で統一した操作方法を提供したりすることで、OS はコンピュータを高度かつ使いやすくします。

Windows などの他人が作った OS の上で動くアプリではない、OS の力を借りずに動く「自作 OS」を作るのがこの本の目的です。他の OS の力を借りないということは、例えばマウスを操作したときに画面上のマウスポインタ（矢印）を移動させる処理を自分で書くということです。また、キーボードでコマンド入力し、Enter キーを押したらコマンドを起動する処理も自分で作ります。パソコンに搭載されたメモリ量を把握してメモリを管理する機能も作らないといけません。およそすべての処理を自分で作るのが「OS を手作りする」ということです。ワクワクしませんか？

OS 自作というのは一見するととても無駄なことです。すでに高性能な OS がいくつも存在するのに、

それを後追いで作ろうというわけですからね。しかし、OS 自作は私たちに貴重な経験を与えてくれます。コンピュータシステムがどう動くのかを探索することは知的好奇心を大いに刺激します。実用面でも、パソコンのハードウェアや OS の処理内容に関する知識はソフトウェアエンジニアの仕事の幅を広げてくれます。OS の動作に目が向くようになり、効率的に動作するアプリの作り方を探索できるようになるでしょう。あるいはシステムの障害原因調査において、OS をデバッグした経験をもとに深いところまで原因追求ができるでしょう。いざ Linux カーネルのコードを読む必要に迫られても、OS を作った経験が有ると無いとでは読みやすさが格段に違います。

専門用語を使って MikanOS の特徴を紹介すると、MikanOS は UEFI BIOS により起動して 64 ビットモードで動作し、プリエンティブマルタスク、ウィンドウシステム、ページングによるメモリ管理、システムコールなどの機能を持った OS です。これらの専門用語の意味が分からなくても問題ありません。実際に物を作りながら意味を説明するのがこの本の役目ですからね。

対象読者

いまや個人で高性能な Web サービスや VR 対応のゲームを作れる時代になりました。IoT 向けの小型コンピュータを買えば、一昔前は想像できなかったような高度な電子工作をすぐ始められます。コンパイラや CPU の作り方を説明する本も出版されています。

いろいろなものが簡単に作れるようになりましたが、OS については逆に作りにくい時代になりました。コンピュータの高度化に対応して OS が進化した結果です。この本は、間に隠されつつある OS 自作に光を当て、手作りの楽しさを途絶えさせないために書いています。筆者と一緒に、OS を手作りしてみませんか？

この本は 2006 年に出版された「30 日でできる！ OS 自作入門」(参考文献 [1]) の流れを汲んでいます。2021 年の今になっても絶版になっていない非常に素晴らしい本なのですが、出版当時からパソコンの中身は大きく進化し、内容の一部（とくにハードウェア関係）の情報は古くなり過ぎました。そこで当時の雰囲気大事にしつつも、まったく新しい本を書き下ろすことにしたのです*¹。ということで、この本の内容は執筆時点で市販されているパソコンで試せるように工夫しています（ただし、すべての機種でちゃんと動くことを保証するわけではありません。その点はご了承ください）。

この本の対象読者は、簡単なプログラムを書いたことのある人を想定しています。数百行程度のプログラミング経験があると心配なく読み進められると思いますが、プログラムをちっとも書いたことがないと苦勞すると思います。この本が初めてのプログラミング経験である、という方がいましたら、ぜひ筆者に読んだ感想を教えてください。

筆者の趣味により、この本で作る MikanOS は C++ で書きました。C++ は C 言語を拡張して作られたプログラミング言語で、OS 作りにも活かせる便利な機能が豊富です。C++ にそれほど詳しくない方のために本文中で多少の説明を入れています。C++ をさらに詳しく勉強するには、この本で出てきた言葉を頼りに、入門書やウェブサイトを読んでみてください。もしかすると、OS 開発をする前に C++ を完璧に勉強しておこうと意気込むかもしれませんが、OS を作りながら必要に応じて勉強の方がやる気を維持しやすいのではないかと筆者は考えています。気張らずに前に進んでください*²。

* 1 筆者が高校生のときに「30 日でできる！ OS 自作入門」の校正作業に参加しました。その当時、まさか、その流れを汲む本を執筆することになるとは夢にも思っていませんでした。

* 2 編集注：「OS 自作」に興味を持ったら、ぜひ「第 0 章 OS って個人で作れるの？」も読んでみてください。

目次

はじめに.....	3
対象読者.....	4
第0章 OSって個人で作れるの?.....	13
0.1 OSの作り方.....	14
0.2 そもそもOSって何?	15
コラム0.1 OSの仕様とPOSIX	17
0.3 OS自作の手順.....	18
0.4 OS自作の楽しみ方	19
0.5 OS自作の全体像.....	20
第1章 PCの仕組みとハローワールド.....	27
1.1 ハローワールド.....	28
1.2 USBメモリのデバイス名の探し方	32
1.3 WSLでのやり方	33
1.4 エミュレータでのやり方	35
1.5 結局、何をやったのか?	36
1.6 とにかく手を動かそう.....	40
1.7 UEFI BIOSによる起動.....	40
1.8 OSを作る道具	41
1.9 C言語でハローワールド	42
コラム1.1 PEとCOFFとELF	44
第2章 EDK II入門とメモリマップ.....	47
2.1 EDK II入門	48
2.2 EDK IIでハローワールド (osbook_day02a)	49
コラム2.1 インクルード	52
2.3 メインメモリ.....	53
2.4 メモリマップ.....	54
2.5 メモリマップの取得 (osbook_day02b)	55
2.6 メモリマップのファイルへの保存.....	58
2.7 メモリマップの確認.....	61
2.8 ポインタ入門 (1) : アドレスとポインタ	62
2.9 ポインタとアロー演算子	64
コラム2.2 ポインタのポインタ.....	65

第3章 画面表示の練習とブートローダ	67
3.1 QEMUモニター	68
3.2 レジスタ	70
3.3 初めてのカーネル (osbook_day03a)	72
コラム3.1 レッドゾーン	81
3.4 ブートローダからピクセルを描く (osbook_day03b)	82
3.5 カーネルからピクセルを描く (osbook_day03c)	83
3.6 エラー処理をしよう (osbook_day03d)	86
コラム3.2 ポインタのキャスト	87
3.7 ポインタ入門(2): ポインタとアセンブリ言語	89
 第4章 ピクセル描画とmake入門	93
4.1 make入門 (osbook_day04a)	94
4.2 ピクセルを自在に描く (osbook_day04b)	97
コラム4.1 ABI	101
4.3 C++の機能を使って書き直す (osbook_day04c)	102
コラム4.2 コンパイルエラーはお友達	107
4.4 vtable	108
4.5 ロードを改良する (osbook_day04d)	109
 第5章 文字表示とコンソールクラス	117
5.1 文字を書いてみる (osbook_day05a)	118
コラム5.1 参照とポインタ	121
5.2 分割コンパイル (osbook_day05b)	122
5.3 フォントを増やそう (osbook_day05c)	124
5.4 文字列描画とsprintf() (osbook_day05d)	127
5.5 コンソールクラス (osbook_day05e)	129
5.6 printf() (osbook_day05f)	132
 第6章 マウス入力とPCI	135
6.1 マウスカーソル (osbook_day06a)	136
6.2 USBホストドライバ	139
6.3 PCIデバイスの探索 (osbook_day06b)	141
6.4 ポーリングでマウス入力 (osbook_day06c)	151
コラム6.1 ログ関数	159
コラム6.2 static_cast<uint64_t>(0xf)の謎	159

第7章 割り込みとFIFO	161
7.1 割り込み (osbook_day07a)	162
7.2 割り込みハンドラ	163
7.3 割り込みベクタ	164
7.4 割り込み記述子の設定	167
7.5 MSI割り込み	169
7.6 割り込みのまとめ	170
7.7 割り込みハンドラ的高速化 (osbook_day07b)	172
7.8 FIFOとFILO	172
7.9 キューの実装	174
7.10 キューを使った割り込み高速化	177
 第8章 メモリ管理	 181
8.1 メモリ管理	182
8.2 UEFIメモリマップ (osbook_day08a)	182
8.3 データ構造の移動 (osbook_day08b)	185
8.4 スタック領域の移動	186
8.5 セグメンテーションの設定	187
8.6 ページングの設定	194
8.7 メモリ管理に挑戦 (osbook_day08c)	198
 第9章 重ね合わせ処理	 205
9.1 重ね合わせ処理 (osbook_day09a)	206
9.2 new 演算子	206
9.3 重ね合わせ処理の原理	209
コラム9.1 スマートポインタ	225
9.4 重ね合わせ処理の時間計測 (osbook_day09b)	226
9.5 重ね合わせ処理の高速化 (osbook_day09c)	230
9.6 スクロール処理の時間計測 (osbook_day09d)	240
9.7 スクロール処理の高速化 (osbook_day09e)	241
 第10章 ウィンドウ	 247
10.1 もっとマウス (osbook_day10a)	248
10.2 はじめてのウィンドウ (osbook_day10b)	249
10.3 高速カウンタ (osbook_day10c)	252
10.4 チラチラ解消 (osbook_day10d)	253
10.5 バックバッファ (osbook_day10e)	259
10.6 ウィンドウのドラッグ移動 (osbook_day10f)	261
10.7 ウィンドウだけドラッグ移動 (osbook_day10g)	266

第11章 タイマとACPI	269
11.1 ソースコード整理 (osbook_day11a)	270
11.2 タイマ割り込み (osbook_day11b)	271
11.3 細かく時間を計る (osbook_day11c)	273
コラム11.1 volatileの必要性	277
11.4 複数のタイマとタイムアウト通知 (osbook_day11d)	277
11.5 ACPI PMタイマとRSDP (osbook_day11e)	282
第12章 キー入力	289
12.1 FADTを探す (osbook_day12a)	290
12.2 ACPI PMタイマを使う (osbook_day12b)	293
12.3 USBキーボードドライバ (osbook_day12c)	295
12.4 モディファイアキー (osbook_day12d)	298
12.5 テキストボックス (osbook_day12e)	303
12.6 カーソル (osbook_day12f)	306
第13章 マルチタスク(1)	309
13.1 マルチタスクとコンテキスト	310
13.2 コンテキストの切り替えに挑戦 (osbook_day13a)	311
コラム13.1 x86-64アーキテクチャとスタックのアライメント制約	320
13.3 コンテキストスイッチの自動化 (osbook_day13b)	321
13.4 マルチタスクの検証 (osbook_day13c)	325
13.5 タスクを増やす (osbook_day13d)	326
第14章 マルチタスク(2)	333
14.1 スリープしてみる (osbook_day14a)	334
14.2 イベントが来たら起床する (osbook_day14b)	340
14.3 性能測定	344
14.4 タスクに優先度を付ける (osbook_day14c)	344
14.5 アイドルタスク (osbook_day14d)	352
第15章 ターミナル	355
15.1 ウィンドウ描画はメインスレッドで (osbook_day15a)	356
15.2 アクティブウィンドウ (osbook_day15b)	360
コラム15.1 タイトルにstd::stringを使う理由	368
15.3 ターミナルウィンドウ (osbook_day15c)	369
15.4 描画の高速化 (osbook_day15d)	374

第 16 章	コマンド	379
16.1	ターミナルでキー入力 (osbook_day16a)	380
16.2	echo コマンド (osbook_day16b)	384
16.3	clear コマンド (osbook_day16c)	388
16.4	lspci コマンド (osbook_day16d)	389
16.5	コマンド履歴 (osbook_day16e)	390
16.6	省電力化 (osbook_day16f)	393
第 17 章	ファイルシステム	395
17.1	ファイルとファイルシステム	396
17.2	BIOS パラメータブロック	400
17.3	ディレクトリエントリ	402
17.4	ボリュームを読み出す (osbook_day17a)	403
	コラム 17.1 ボリュームの読み込みは 16MiB で足りるか	409
17.5	ls コマンド (osbook_day17b)	410
第 18 章	アプリケーション	415
18.1	ファイルアロケーションテーブル (osbook_day18a)	416
18.2	初アプリ (osbook_day18b)	422
18.3	C++ で計算機 (osbook_day18c)	428
18.4	標準ライブラリ (osbook_day18d)	435
第 19 章	ページング	437
19.1	実行ファイルとメモリアドレス	438
19.2	アドレス変換	440
	コラム 19.1 事実上のアドレス	442
19.3	アプリのロードと実行 (osbook_day19a)	442
19.4	仮想アドレスと 4 階層ページング	442
19.5	アプリケーションを後半で動かす	446
19.6	アプリケーションのロード	447
19.7	階層ページング構造の設定	449
19.8	階層ページング構造の片付け	453
	コラム 19.2 構造化束縛	456

第20章 システムコール	457
20.1 アプリがOSの機能を使う方法 (osbook_day20a)	458
20.2 OSを守ろう (1) (osbook_day20b)	460
20.3 TSSを設定しよう (osbook_day20c)	467
20.4 バグ発見を手伝おう (osbook_day20d)	476
20.5 システムコール (osbook_day20e)	479
20.6 システムコールの登録処理	481
20.7 システムコールの本体	483
第21章 アプリからウィンドウ	487
21.1 ISTを設定しよう (osbook_day21a)	488
21.2 文字列表示システムコール (osbook_day21b)	490
21.3 システムコールの作成	492
21.4 write()の作成	496
21.5 終了システムコール (osbook_day21c)	498
21.6 スタックポインタの復帰	502
21.7 コード整理 (osbook_day21d)	504
21.8 ウィンドウを開く (osbook_day21e)	506
21.9 ウィンドウに文字を書く (osbook_day21f)	508
第22章 グラフィックとイベント (1)	511
22.1 exit()を使う (osbook_day22a)	512
22.2 点を描く (osbook_day22b)	514
22.3 タイマ値の取得 (osbook_day22c)	518
22.4 ウィンドウ描画の最適化 (osbook_day22d)	520
22.5 線を引く (osbook_day22e)	522
22.6 ウィンドウのクローズ (osbook_day22f)	526
22.7 キー入力待つ (osbook_day22g)	528
第23章 グラフィックとイベント (2)	537
23.1 マウス入力 (osbook_day23a)	538
23.2 お絵描きソフト (osbook_day23b)	543
23.3 タイマコマンド (osbook_day23c)	547
23.4 アニメーション (osbook_day23d)	552
23.5 ブロック崩しゲーム (osbook_day23e)	554

第 24 章	複数のターミナル	559
24.1	ターミナルを増やす (osbook_day24a)	560
24.2	カーソル点滅を自分で (osbook_day24b)	561
24.3	複数アプリの同時起動 (osbook_day24c)	564
24.4	ウィンドウの重なりりのバグ修正 (osbook_day24d)	569
24.5	ターミナル無しのアプリ起動 (osbook_day24e)	571
24.6	OSをフリーズさせるアプリ (osbook_day24f)	575
24.7	OSを守ろう (2) (osbook_day24g)	577
第 25 章	アプリでファイル読み込み	581
25.1	ディレクトリ対応 (osbook_day25a)	582
25.2	ファイル読み込み (osbook_day25b)	588
25.3	正規表現検索 (osbook_day25c)	597
第 26 章	アプリでファイル書き込み	601
26.1	標準入力 (osbook_day26a)	602
26.2	ファイルディスクリプタの抽象化	602
26.3	キーボード入力を受け取る	604
26.4	EOFとEOT (osbook_day26b)	607
26.5	ファイル書き込み (1) (osbook_day26c)	609
26.6	ファイル書き込み (2) (osbook_day26d)	616
第 27 章	アプリのメモリ管理	621
27.1	デマンドページング (osbook_day27a)	622
27.2	メモリマップトファイル (osbook_day27b)	628
27.3	メモリ使用量を測ろう (osbook_day27c)	637
27.4	コピーオンライト (osbook_day27d)	638
第 28 章	日本語表示とリダイレクト	649
28.1	日本語と文字コード (osbook_day28a)	650
28.2	日本語フォント (osbook_day28b)	657
28.3	リダイレクト (osbook_day28c)	662

第29章 アプリ間通信	669
29.1 終了コード (osbook_day29a)	670
29.2 パイプ (osbook_day29b)	673
29.3 コマンドラインの解析とタスクの起動	676
29.4 パイプ処理の本体PipeDescriptor	677
29.5 ターミナルの起動と終了	680
29.6 タスクの終了	682
29.7 sortコマンド (osbook_day29c)	685
29.8 ターミナルのバグ修正 (osbook_day29d)	687
29.9 共有メモリ	690
第30章 おまけアプリ	691
30.1 アプリにパスを通す (osbook_day30a)	692
30.2 moreコマンド (osbook_day30b)	693
30.3 catを入力に対応させる (osbook_day30c)	696
30.4 閉じるボタン (osbook_day30d)	698
30.5 テキストビューア (osbook_day30e)	703
30.6 画像ビューア (osbook_day30f)	706
第31章 これからの道	711
付録A 開発環境のインストール	716
A.01 WSLのインストール	716
A.02 WSLでQEMUを使う準備	717
付録B MikanOSの入手	722
B.01 MikanOSのバージョン間の差分確認	723
B.02 ソースコードの検索	724
付録C EDK IIのファイル説明	725
付録D C++のテンプレート	728
付録E iPX E	729
E.01 iPX Eのビルドとインストール	729
E.02 HTTPサーバの起動	730
E.03 ネットワーク起動の実践	731
付録F ASCIIコード表	732
参考文献	736
謝辞	738
索引	739

第 0 章

OSって 個人で作れるの？

皆さんが普段使っているであろう Windows、macOS、Linux はいずれも巨大な OS です。例えば Linux のバージョン 4.10 (2017 年リリース) は約 2100 万行もあります。他の 2 つはソースコードが公開されていないので分かりませんが、数千万行はあると思います。もちろん、こんなに大きなソフトウェアを個人で作ることはできません。

それらの OS が最初から巨大だったかというところでもありません。Linux の作者リーナス・トーバルズ氏が作った最初のバージョン 0.01 (1991 年リリース) は、たったの 1 万 239 行だそうです。バージョン 4.10 に比べるととても小さいですが、OS としての基本的な機能は備えていました。

また「30 日でできる! OS 自作入門」で作成することになる「はりぼて OS」は 4249 行で、コンパイル後の大きさは 40KB 程度です。このくらいの大きさであれば個人で作れそうな気がしてきませんか? そう、作れちゃうのです!

本書で作る OS はシンプルなものですが、それでも GUI^{*1} があってウィンドウがポコポコ表示され、マルチタスク^{*2} で動くような、それなりにちゃんとしたものです。OS 本体は最終的に 1 万 1071 行になります。知っている人向けに書くと、UEFI で起動し、Intel 64 モードで動き、ページングを用いてメモリ管理を行い、USB3.0 ドライバを搭載し、ファイルシステムを持つような OS を作ります。

0.1 OS の作り方

「OS を作る」と言ったときに大きく分けて 2 通りの作り方があります。既存の OS を改造するか、全部自分で作るかです。

既存の OS、例えば Linux を基にして、必要な機能を足して不要な機能を取り除けばオリジナルな OS を作ることができます。実用的な OS を作るにはこのやり方が一番早いです。macOS や Android は基にした OS こそ違うものの、このやり方で作ってあります。ただこの方法では OS の全体を見渡すことは難しく、OS 作りのごく一部しか体験できないのです。せっかくなら全体をやってみたいと思いませんか?

本書では、OS を全部自分で作ってみることにします。パソコンの電源を入れて OS 本体を呼び出すところから、いろいろなアプリを動かせるようになるまで OS づくりを一通り体験します。読み進めるにつれて、今までブラックボックスだと思っていたパソコンの中身、OS の仕組みが分かってくるでしょう。

OS を作るにはコツがあります。それは、最初から完璧に作ろうとしないことです。最初から完璧を目指す手が止まってしまって全然前に進まなくなります。そうではなく、最初は OS っぽく見えるおもちゃを作ることしましょう。おもちゃを作りこんでいくと、最初はおもちゃだったものがだんだんと本物に近づいていくのです。

いざ OS を作るというと、やれモノリシックにするかマイクロカーネルにするかとか、メモリ管理はどうするんだとか、リアルタイム性は担保するのかとか、詳しい人からいろいろ言われるかもしれませんが。そういった質問にちゃんと答えるにはそれなりに勉強しなくてははいけません。OS 理論を学ぶことは、それはそれで大切なことです。

しかし、いろいろ勉強してからでなければ OS を書いてはいけない、なんてことはありません。むしろ、理論を知らないまま OS を作り始めた方が良くさえ筆者は思います。なぜなら下手に勉強してしまうと先人が残したすごい理論に打ち負かされてしまい、OS 作りを楽しめなくなるからです (それで楽しめる人もいるとは思いますが)。OS 作りはとってもワクワクする創造的活動なのです。最初くらいは好き勝手に作って楽しむ方が、その後の勉強意欲も駆り立てられるというものです。

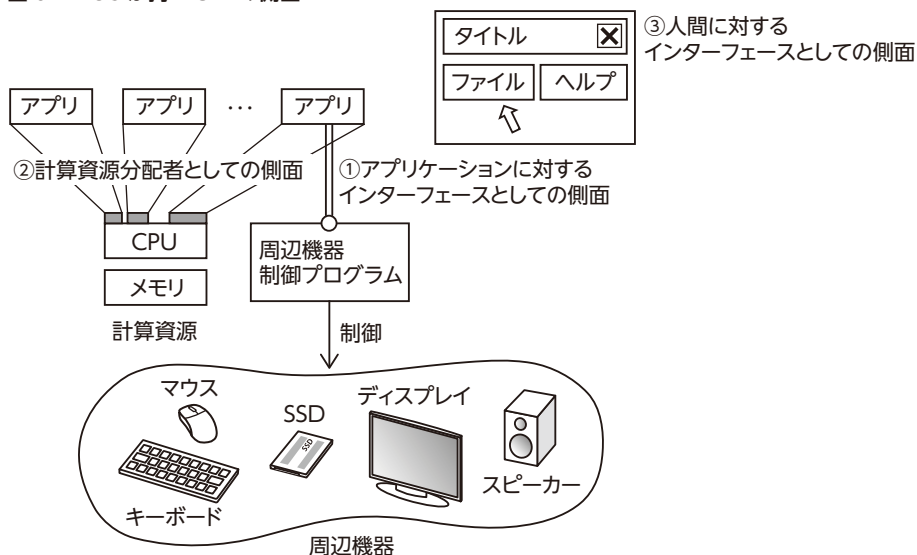
0.2 そもそもOSって何？

ここまで何となく OS という言葉を使ってきましたが、そもそも OS とは何でしょうか？ 何を実現すれば OS と言えるのでしょうか？

実は人によって何を OS と呼ぶかはばらつきがあり、バシッと定義することは難しそうです。「30 日 できる！ OS 自作入門」にはこうあります。「さまざまな OS を比較してみたところ、この機能が共通点、といえそうなものを見つけられませんでした。結局のところ、それぞれの作者が『これは OS なんだ』と言い張って、周囲の人も『まあそうかな』と思えばどんなソフトでも OS なんですよ」。確かに そんな気もします。なぜなら、世界にはとてもシンプルな機能しか持たないけど「OS」と呼ばれるソフトウェアがたくさんあるからです。全部に共通する機能を探すのは大変です。

ただ、そこで思考停止せずに、よく使われる OS にどんな機能があるかを考えるのは有意義だと思います。すべての OS とまでは言わず、Windows、macOS、Linux など、読者の皆さんが普段使っているであろう OS を観察すると 3 つの側面が見えてきます (図 0.1)。

図 0.1 OS が持つ 3 つの側面



この図に登場するインターフェースとは 2 つの物の接合点を意味します。アプリと OS が接する点、あるいは人間とコンピュータが接する点ですね。日常生活で例えると、電子レンジや洗濯機のボタンは人間と機械が接する点ですのでインターフェースです。あるいは、家庭用ゲーム機のコントローラは人間とゲームをつなぐインターフェースです。細かく見ると、コントローラのボタンは人間に対するインターフェース、ゲーム機にあるコントローラ接続口はコントローラに対するインターフェースと言えます。

- * 1 Graphical User Interface. ボタンやチェックボックスのような図形パーツを用いた視覚的な操作方法。
- * 2 複数のタスク (仕事) を同時に実行すること。今となってはマルチタスクは当たり前ですが、昔の OS にはマルチタスクができないものがありました。

す。

インターフェースは操作の窓口となります。インターフェースの良し悪しは製品の使いやすさに直結します。OS に当てはめると、アプリが OS の機能を使うときや人間がコンピュータを使うときに OS が提供するインターフェースを利用します。使いやすいインターフェースを提供できるかが OS 作者の腕の見せ所です。

コンピュータにはいろいろな周辺機器が接続されています。HDD や SSD などの 2 次記憶装置、マウスやキーボード、ディスプレイ、NIC^{*3}、カメラ、スピーカーなどなど。OS が無かった時代は、各アプリがそれぞれの周辺機器を制御していました。OS は各アプリがやっていた周辺機器の制御を肩代わりします。通常、周辺機器（ハードウェア）の制御は複雑になりがちですが、その複雑さを OS が吸収してくれるため、アプリは単純化されたインターフェースを介して周辺機器を活用できます。

OS が提供する単純化されたインターフェースを使うことで、アプリは簡単に周辺機器を使えるようになります。加えて、アプリから周辺機器の詳細を隠すことで異なる（同種の）周辺機器を同じインターフェースで使えるようになります。例えばストレージ（HDD や SSD など）は製品により接続規格や速度が異なりますが、OS がその違いを吸収することで、アプリはファイルに対して read/write という単純化されたインターフェースを使いさえすればいいのです。このように、詳細を隠して単純なインターフェースを提供することを**抽象化**といいます。抽象化は本来複雑であるコンピュータの世界を人間の手に負える程度に簡単に保つために必須の仕組みです。

抽象化は、周辺機器の制御だけでなくネットワーク通信などにも応用されます。インターネット通信では TCP や UDP がよく使われますが、通常、その下にある伝送方法は OS に隠蔽されていてアプリからは見えません。もしかしたらマザーボードに接続された LAN ケーブルの Ethernet かもしれませんし、USB 経由で WiFi アンテナが刺さっているかもしれません。それぞれ、物理的には異なる伝送形式ですが、OS が抽象化したインターフェースを提供するためアプリは気にしなくて済みます。

今まで見たように「アプリケーションに対するインターフェースとしての側面」とは、プログラマがアプリを作る際にお世話になることが多い部分です。対して、これから説明する「計算資源分配者としての側面」は、アプリを使う側のお話です。

計算資源というのは CPU が持つ計算能力、メモリの記憶、ストレージの読み書き、NIC の送受信など、アプリの処理に必要なあらゆる資源（リソース）のことです。これらの資源は有限ですから、1 つのアプリが独占してしまうと他のアプリが実行できません。さらに、これらの資源を必要とするのはアプリだけではなく、マウスカーソルの移動処理など、OS が活動するためにも資源が必要です。複数のアプリや OS が並行に動けるよう、適切に計算資源を分配するのが OS の役目です。各資源をどう分割して分配するかは OS 作者の腕の見せ所です。

資源の分配は突き詰めると奥が深い話題です。例えば、動画のエンコードのような CPU をがんがん使うアプリを動かしている最中にマウスをちょっと動かしたとします。動画エンコードが終わるまでそのアプリが CPU を独占するような単純な作りの OS だとしたら、動画エンコードが終わるまでマウスが止まったままです。これでは、操作している人はパソコンがフリーズして壊れたと思うでしょう。動画のエンコードは CPU を大量に使うものの、一瞬なら処理が中断しても問題ありません。一方でマウスの移動処理は、平均すると CPU をほとんど使わないものの、マウスを動かした瞬間にはなるべく早く処理したいのです。普及している OS はどれも、このような場合でもちゃんと動くように工夫して作られています。

今まで見てきた 2 つの切り口は、主にプログラマ的な話でした。プログラムを作りやすくしたり、複数のプログラムがうまく協調動作するようにしたり。最後の「人間に対するインターフェースとしての側面」は、人間がコンピュータを扱うのを OS が支援するというお話です。

パソコンを使うとき、各アプリに統一されたメニューがあると操作しやすいですね。どのアプリにも同じような見方でメインメニューがあり、「ファイル」メニューをクリックすると「上書き保存」や「名前を付けて保存」を選べる、という具合に。また、Ctrl-C でコピーして Ctrl-V で貼り付ける、というキーボードショートカットも、アプリ間で共通してほしいですね。このように、アプリ横断で共通のインターフェースを提供するのも OS^{*4} の役割です。

3つの切り口で OS の役割を見てきました。これらがすべて備わっていないと OS とは呼べない、というわけではありませんが、普及している OS にはそれらの機能が入っているということです。本書でも既存の OS にならい、多かれ少なかれそのような機能を持つように作っていかうと思います。

コラム 0.1 OS の仕様と POSIX

インターフェースというキーワードが出てきました。OS とアプリ、あるいは人間はインターフェースを介してやり取りします。そのため、インターフェースの利用方法や挙動がどのようなものであるか、つまりインターフェースの仕様が重要です。アプリからみたインターフェースの仕様といえば、例えば関数名や引数の型、関数の動作、戻り値などです。人間からみたインターフェースの仕様は、マウスをクリックしたときに何が起きるか、アプリの起動方法、アプリの結果をファイルに保存する方法、ウィンドウの「閉じる」ボタンの位置、などになりますね。OS のインターフェースがカバーする範囲は非常に広いです。

OS に限った話ではありませんが、インターフェースが同じであれば実装が異なっても同じように利用できます。同じインターフェースに準拠する複数の OS があったとすると、1つのアプリをどの OS でも動かせるようになり、アプリの作者や利用者は大変便利です。OS の作者にとってもこれは嬉しいことです。OS の対象ハードウェアを x86 パソコンにしようが、Arm スマホにしようが、自作 CPU を採用したコンピュータにしようが、共通のインターフェースさえ提供できれば既存のアプリを動かせることになりますからね。

OS のインターフェースとしては POSIX (Portable Operating System Interface) が有名です。これは UNIX 系の OS が広く採用しているインターフェースで、C 言語の関数やファイルシステム、プロセスなど、広い範囲をカバーします。本書で採用する標準 C ライブラリである Newlib は POSIX を前提として作られているため、MikanOS を作る際にも POSIX に関連した話がときどき登場します。

ただ、MikanOS 自体は既存のどのインターフェース仕様にも準拠しません。既存のインターフェース仕様を紹介するのが本書の目的ではないからです。むしろ、既存の仕様にとらわれず、作りたいものを作りやすい方法で実現しようと思います。POSIX やその他の既存インターフェース仕様と互換性のある OS を自作するのも非常に面白いと思いますから、読者の皆さんがそれに挑戦することは止めません。応援します！

*3 Network Interface Card. ネットワークの送受信をする周辺機器。元々は拡張カードとして売られていたのでカードと呼ばれますが、現在はパソコン用マザーボードに標準で組み込まれています。

*4 いやいや、それは OS (カーネル) ではなく GUI フレームワークの役目だ、という指摘はあり得ます。それはその通り。ここでは OS をカーネルに限定せず、もう少し広く捉えて説明しています。

0.3 OS自作の手順

普通のアプリケーションであれば、パソコンでソースコードを書いてコンパイルすれば実行可能ファイルが出来上がります。そのファイルを起動させれば（ダブルクリックするとか、ターミナルでファイル名を打ち込むとか）実行されますよね。Python などのインタプリタ言語であれば、ソースコードをそのまま実行することができます。

しかし、そうやって実行できるのも OS のお陰なのです。私たちが作ろうとしているのは OS そのものですから、他の OS に頼らずに動作する特殊なプログラムを作らねばなりません。次に示す手順で作っていきます。

1. 開発用パソコンで OS のソースコードを書き、コンパイルする
2. 生成された実行ファイルを USB メモリに書き込む
3. 試験用パソコンに USB メモリを接続し、電源を入れ、実行する

開発用パソコンと試験用パソコンは同じでも構いませんが、その場合はいちいち再起動が必要となりますので面倒です。そこで、筆者は試験用パソコンとして GPD MicroPC という小型パソコンを使っています。5 万円程度で買って機能が充実していますので、自作 OS を試験するにはぴったりです。そのほか、AMD の FX-8800P という CPU が搭載された A10N-8800E というマザーボードで組み立てた自作 PC も試験機としてたまに使っています。中古のパソコンを買ってもいいでしょう。2012 年以降に発売された機種であれば、本書の内容がそのまま動作すると思います。保証はできませんが。

試験用パソコンの代わりにエミュレータを使って自作 OS を試すこともできます。エミュレータというのは、パソコンの中に仮想的にパソコンを再現するソフトウェアです。エミュレータを使うことで、試験用パソコンを用意できない場合でも安心して本書の内容を試していただけます。エミュレータを使うと USB メモリの準備などをせず手軽に試せるので、試験用パソコンを持っていたとしてもエミュレータは活躍するはずですが。筆者は MikanOS の開発のために QEMU というエミュレータをよく使っています。「1.9 C 言語でハローワールド」で QEMU を使った方法を説明します。

本書は Linux で開発を行う想定で書かれています。「30 日でできる! OS 自作入門」は Windows で開発することになっていましたが、その時代に比べると Linux も十分使いやすくなりましたし、WSL^{*5} という機能により Windows 上でも Linux が動きます。Windows よりも Linux の方が開発環境を整えやすいため^{*6}、もしまだ Linux を使ったことがない方は、これを機に Linux へ入門してみてもいいでしょうか？ 正確に言えば Linux のディストリビューションの 1 種である Ubuntu 18.04 で動作を検証しています。WSL 上の Ubuntu と操作に違いがある場合は本文で説明しますので、Windows メインの方も一緒に楽しめます！

手順 1 でソースコードを書くテキストエディタは好きなもので構いません。筆者は Vim が好きですが、とにかく、最終的に目的の実行ファイルを得られれば道具は何でもいいのです。ソースコードを実行ファイルに変換するにはコンパイラというソフトウェアを使います。コンパイラにもいろいろありますが、本書では Clang (+LLVM) を使います^{*7}。Clang のほかに GCC という有名なコンパイラがありまして、そちらに挑戦してみるのもまた面白いかもしれません。

手順 2 で使う USB メモリは、壊れても大丈夫のように安い USB メモリを買ってくることをお勧めします。容量は小さくて大丈夫です。また、試験用パソコンのファームウェアが対応しているメディアであれば USB メモリである必要はありません。内蔵や外付けの HDD や、ネットワーク経由の起動も可能です。筆者はいちいち USB メモリを抜き差しするのが嫌でしたので、USB メモリにはネットワーク経

由で OS を読み込むソフトウェア (iPXE) だけを入れておき、OS 本体のファイルは開発用パソコンに置いておく、というやり方で OS 開発をしています。こうしておけば、USB メモリはずっと挿したままで良くなります。

0.4 OS自作の楽しみ方

OS 自作に正解はありません。「0.5 OS 自作の全体像」では本書の流れを大まかに紹介しますが、単なる一例にすぎないのです。読者の皆さんはぜひ、迷うことを恐れず横道に逸れてみてください。独自の改造をしたり、本書で扱っていない機能を実装したりしてみてください。本書の流れをそのまま再現するだけでも OS や低レイヤについてそれなりの知識を得られると思いますが、横道を探検することでさらに奥深い経験を得られるでしょう。

本の内容を深く理解したい場合は「写経」をお勧めします。写経とはもともと、仏教の経典を書き写すことです。本書でいう写経は、サンプルコードをコピペせず自分で入力することを意味します。それに何の意味があるのだ、と思うかもしれませんが。コピペの方が圧倒的に素早く学習を進められますよね。

筆者の経験では、写経には細部を理解しながらコードを読めるという利点があります。コピペしたコードをただ眺めるより、1文字ずつ自分で打ち込む方が細かい点に気付くからです。実際、筆者は「30日できる! OS自作入門」を写経しながら読み、深く理解できたと感じています。もちろん写経にはそれなりの時間がかかりますが、もしあなたが時間に余裕のある学習者であれば、写経にチャレンジしてみてもいいでしょうか。

本書を読み進める中で内容について疑問が出てくるかもしれません。あるいは、独自の改造に挑戦しようとなると調べても分からないことがきつと湧いてくるでしょう。そんなときは筆者や、OS を自作している人のコミュニティに質問してみましょう。本書の内容に直接関連した質問であれば、本書のサポートサイト <https://zero.osdev.jp/> が最適です。サポートサイトの中にある GitHub Issues で質問を投稿すると、筆者や他の読者の方が質問に答えます。また、本書の勉強メモを記録したり、独自の改造を発表したりする場として GitHub Wiki をご活用ください。サポートサイトには本書の読者が集まるわけですから、読書仲間を探すこともできます。ただし、GitHub の利用規約により 13 歳未満の読者の方は Issues や Wiki を使えませんので、後述する osdev-jp のメーリングリストに加入すると良いと思います。

本書で説明していない発展的な内容 (例えば自作 OS から USB メモリを直接読み書きする方法など) に関する質問や議論は、筆者が運営している自作 OS コミュニティ「osdev-jp」が最適でしょう。osdev-jp にはパソコン用 OS だけでなく、組み込み機器向けの OS を作るようなメンバーもいます。

- * 5 Windows Subsystem for Linux。Windows 10 に搭載されている機能で、Windows と Ubuntu などの各種 Linux ディストリビューションを共存させることができます。
- * 6 大抵の Linux ディストリビューションには開発ツールが付属していたり、簡単にインストールできるようになっていたりします。本書の範囲を超えて、例えば Git を使ってソースコードのバージョン管理をしてみようとか、違うコンパイラを試してみよう等と思ったら、Linux の方が便利でしょう。
- * 7 Clang を選んだ理由はいくつかあります。GCC より警告メッセージが読みやすいこと、オプションでビルドターゲットを指定できることなどです。ただ、これは好みの問題なので、GCC が好きな人はそちらを使っても良いです。

osdev-jp に加入する方法は <https://osdev.jp/joinus.html> を参照してください。osdev-jp のメンバーリストであれば、13 歳未満の方も加入できます。

本書で作る MikanOS は、すべてのパソコンで正常に動作するとは限りません。筆者は手元のパソコンで起動することを確認してはいますが、読者の皆さんが使っているパソコンで起動しない可能性は大いにあり得ます。そこが OS 開発の難しいところであり、また、楽しいところでもあります。アプリであれば、ハードウェアの些細な違いで動かなくなることは考えづらいですが、OS はハードウェアを直接扱いますから、個々のハードウェアの違いに大きく左右されるのです。もし MikanOS が動かない機種があったら、なぜ動かないのかを究明し、動くように修正を加えるのは皆さんに任されています。

0.5 OS自作の全体像

第1章からは具体的に OS の自作を進めていきます。かなり細かい内容を扱いますので、まるで森を探検するような感覚になると思います。ともすると森の中で迷ってしまうかもしれません。なるべく迷わないために、本編に入る前に全体を見渡しておきましょう。ただし、本編が濃密すぎて全体像を見渡すだけでもかなり長くなってしまいました。ごめんなさい。後で立ち返って来る際は目次が鳥瞰図^{*8}としての役目を果たすでしょう。

第1章から第3章にかけて、ブートローダを作ります。ブートローダとは OS をメインメモリに読み込み、起動させるプログラムのことです。パソコンの仕組みからくる制約により、実行するプログラムはメインメモリに配置しなければなりません。しかし、一般的なメインメモリ (DDR-SDRAM) はパソコンの電源を落とすと内容が消えてしまいます。そこで、電源を切っても内容が消えないストレージ (HDD や SSD など) に OS を記録しておき、ブートローダを用いてストレージからメインメモリに読み出すのです。

本書では UEFI BIOS で動くブートローダを作ります。UEFI にはブートローダを作るための支援機能が沢山あります。例えば、UEFI はストレージを読み書きする機能を持っているため、ブートローダの作者が自分でストレージ装置の制御プログラム (デバイスドライバ) を作る必要がありません。UEFI の機能呼び出しだけでいいのです。これで、かなり楽にブートローダを作ることができます。

ブートローダの後はいよいよ OS 作成を始めます。まずは図形や文字を画面に表示するのが目標です。パソコンの画面 (ディスプレイやモニタと呼ぶ) はよく見ると小さな四角 (ピクセル) が縦横に沢山ならんだ構造になっていて、それぞれの小さな四角に任意の色を表示できます。適切なピクセルに色を塗っていけば図形や文字が表示できます。長方形や直線なら計算によって塗るべきピクセルの位置をはじき出せます。文字の場合はフォントという文字の形をデータ化したものを使って、どのピクセルを塗るかを決めます。ここまで来ると図 0.2 のように、好きな色で長方形を描いたり、文字列を表示したりできるようになります。

*8 鳥瞰 (ちょうかん) とは鳥の視点のこと。高いところからは全体を見渡せるため、森の中での自分の居場所を把握しやすくなります。

図 0.2 フォント大集合



第6章、第7章ではマウスを使えるようにします。マウスが使えるようになるとマウスカーソルが画面内を移動できるようになります。これには割り込みという仕組みを使います。割り込みは、通常の処理に割り込んで処理を行う仕組みのことです。割り込みを使わずにマウスを制御しようとすると、一定間隔（例えば 0.01 秒ごと）で「マウスが動いたか？」というのをマウスに問い合わせる必要があります。平均してみるとマウスはほとんど止まっていて、問い合わせは無駄に終わります。かといって問い合わせ頻度を下げると「マウスがカクカクするなあ」と感じてしまうでしょう。割り込みを使えば、マウスが動いたときに割り込み処理が自動的に動作し、マウスカーソルが動くようになります。マウスを動かす処理は割り込みに任せるのが効率的なのです。

第8章ではメモリ管理の仕組みを作ります。メモリ管理とは、パソコンに搭載されているメインメモリの中で、どこが使用中でどこが空いているかを把握し、メモリを必要とするアプリやプログラムに対してメモリ領域を払い出すことです。そのためにはまず、メモリの容量と初期状態を知る必要があります。メモリが4GBなのか32GBなのかを知らなければならなりませんし、OS以外のプログラム（UEFI BIOS 自体やブートローダなど）が使っているメモリ領域を把握しなければ、正しいメモリ管理はできません。

その後続く2章でウィンドウを表示できるようにします。ウィンドウの表示は文字の表示と同じで、基本的にはピクセルに絵を描くだけなのですが、重ね合わせの処理がちっと難しいです。背景、ウィンドウ、マウスカーソルなどの重なりを考慮して描画しなければなりません。基本的には重なりのもっとも下にある絵から順に描くことで重ね合わせを表現すれば良いです。しかし、描画処理は大量のピクセルに色を描くため、単純なやり方では無駄に処理が重くなりがちです。マウスカーソルやウィンドウをスムーズに動かせるように高速化にも挑戦します。

第11章はタイマに対応します。タイマはパソコンに内蔵された時間を計るハードウェアのことです。時間を計れるということは一定時間毎に何らかの処理をするという仕組みが実現できるということです。複数のタスクを一定時間毎に切り替え、複数のタスクを並行に動かすプリエンティブマルチタス

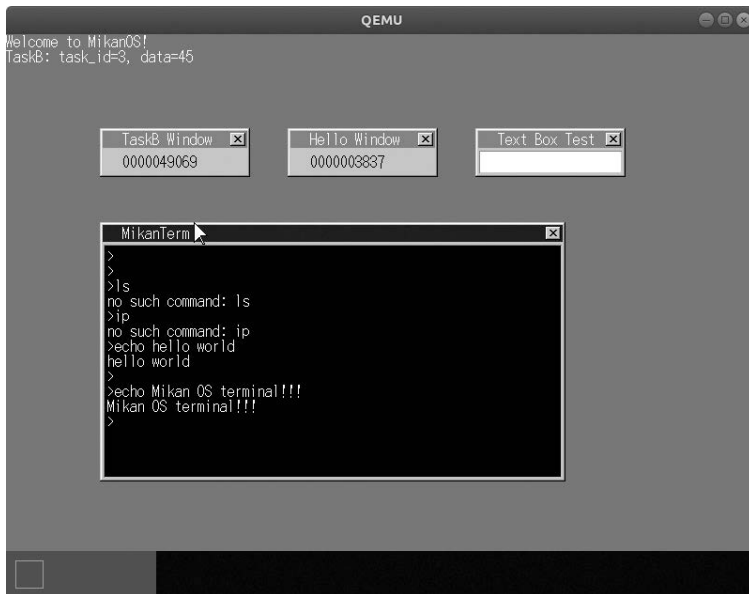
クの実現にはタイマが必須です。そのほか、ゲームや動画といった、時間をきちんと計る必要があるアプリケーションの実現にもタイマが活躍します。

第12章はキーボードに対応し、文字入力をできるようにします。キーボードの各キーにはキーコードと呼ばれる数値が振られていて、いずれかのキーを押すたびにキーコードが送られてきます。それを割り込み処理で受け取ります。キーをよく見ると複数の文字が印字されていますよね？ 例えば日本語キーボードの「W」の上にある「2」キーには、2のほかに「"」も印字されているはずです。キーコードはキーに対して振られた数値ですので、2のつもりで押しても"のつもりで押しても同じ数値が送られてきます。OSは、Shiftキーが同時に押されていたかどうかを考慮し、キーコードを適切な文字に変換します。

第13章、第14章ではマルチタスクという、複数のタスクを並行に動作させる仕組みに挑戦します。アプリはタスクの1種です。パソコンで作業するとき、音楽アプリでBGMを流しつつウェブブラウザで文献を調べ、ワープロアプリでレポートを書く、などと複数のアプリを同時に使いますよね。このようなことができるのはマルチタスクの仕組みのおかげなのです。マルチタスク機能がないOSでは、1つのアプリを閉じるまで他のアプリを使えません。

マルチタスクの仕組みは、パソコンの処理性能を最大限生かすためにも重要です。例えば秒間10フレームで動くゲームを作るとします。フレームとフレームの間は0.1秒だけ時間があり、1フレーム目の処理が0.02秒で終わったとします。2フレーム目までの0.08秒の空き時間を他のタスクに割り振ることができれば、その分だけ多くの処理をこなせます。人間は0.08秒の時間を与られても何もできませんが、CPUはとても速いのでそのくらい短時間でも有効活用できるのです。マルチタスクの仕組みが無かったら、ほかにやるべき処理があるのにパソコンが暇な状態になってしまいます。

図 0.3 ターミナルの echo コマンドで遊ぶ様子



第15章、第16章ではターミナルとコマンドを実装します。ターミナルというのは黒い背景に白い文字が表示される、あの画面です。ターミナルは文字を使って命令したり結果を表示するための仕組みで

あり、CLI^{*9}と呼ばれます。CLIでは、アプリは文字列だけを扱えば良いため、ウィンドウへのボタン配置やイベント駆動のプログラミングを必要とするGUIアプリに比較して、とても楽に作れます。ですので、特にOS開発初期にはターミナルはぴったりなのです。それに、ターミナルに命令を入力してエンターキーで実行するのはハッカーみたいでかっこいいですからね！ 第16章の途中までいくと図0.3のような見た目になります。

第17章ではファイルシステムを作ります。現代のパソコンはバイトを基本にしてデータを扱います。バイトは0から255の数値を表現できる^{*10}小さな記憶の単位です。英数字なら1バイトで1文字を表現できます。HDDやSSDなどのストレージは非常に大きなバイトの列だと思えることができます。どのくらい大きいかというと、1TBで広辞苑約2万冊分^{*11}の文章を収録できます。2万冊の広辞苑が積まれた様子を思い浮かべてください。特定のページを探すのさえ大変ですよ。非常に扱いづらそうです。

ファイルシステムは、ストレージの広大なバイト列を小さく切り出し、「ファイル」として名前を付ける機能だと言えます。ファイルシステムは、ファイル名とバイト列の位置を対応付ける表を管理していて、名前前で指定されたファイルがバイト列のどこにあるかを返したり、新規にファイルを作るときにバイト列の空き領域を探し、ファイル名と位置の対応表を更新したりします。バイト列の管理の仕方によって様々なファイルシステムがありますが、本書ではFATを採用します。

第18章から第20章にかけてアプリケーションを作れるようにします。今まではOS本体の機能を作っていましたが、ここからはOSの上で動くアプリを作っていきます。前節で紹介したOSの3つの切り口は、いずれもアプリがあってこそです。そもそも「アプリケーション」は「応用」という意味の英単語で、現実世界への応用、ユーザに何らかの価値をもたらすもの、という意味合いです。アプリがないOSは何の役にも立ちません。アプリが動くようになるのは、OS自作にとって大きな節目なのです。

特に第20章に作る**システムコール**の仕組みは、1つ目の切り口「アプリケーションに対するインターフェースとしての側面」を実現する大変重要な仕組みです。システムコールというのはアプリがOSの機能を呼び出すための窓口のことです。アプリはOSの様々な機能を活用して処理を進めていきますが、OSに含まれたプログラムを自由に呼び出せてしまえばセキュリティが保てません。自分で作ったアプリしか動かさないのであればいいですが、普通は第3者が作ったアプリを入手して使いますよね。もし悪い人が作ったアプリだったらシステム全体が乗っ取られ、他のアプリが持つ重要なデータを盗まれてしまいます。アプリとOSの接点をシステムコールに限定することで、セキュリティを保ったままOSの機能をアプリに提供できるようになります。OSを自作する大半の方は、他人のアプリを自分のOSで動かすなんてことはしないでしょう（してくれたら筆者としてはとても嬉しいですが！）から、システムコールなんてなくてもいいのです。でも、システムコールは一般的なOSには必ず備わる仕組みですので、本書でも作ってみようというわけです。

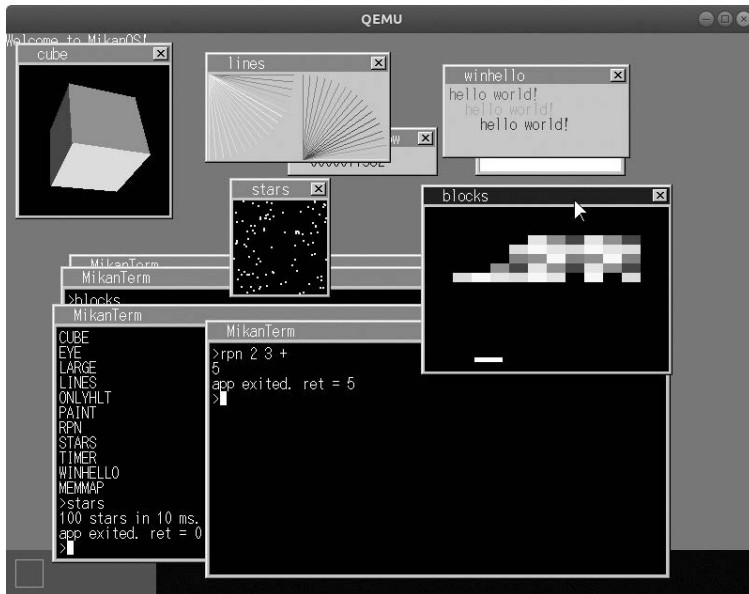
第21章から第23章は、ひたすらシステムコールを拡充していきます。ウィンドウを開いたり、絵を描いたり、時間を計ったり、キー入力したり。システムコールが増えるにつれ、作れるアプリの幅が広がっていきます。アプリをたくさん作るこの期間はとても楽しいと筆者は思います。OSの基礎的な仕組みを作るのも楽しいのですが、アプリが増えていくのは目に見えてモチベーションが上がりますよ。

*9 Command Line Interface。コマンドラインとはターミナルに打ち込む命令のこと。CLIアプリというと、文字で命令し文字で結果を表示するようなアプリを意味します。

*10 1バイトが何ビットであるかは決まっていますが、本書で扱うx86-64アーキテクチャでは1バイト=8ビットです。8ビットで0から255までの数値を扱えます。

*11 広辞苑1冊で約1500万字だそうです。1文字あたり平均3バイトだとすると1500万字で45MB。1TB/45MB ≈ 22222。ちなみに、原稿執筆時に家電量販店でSSDを調べると、500GBから1TB程度の製品が多く売られていました。

図 0.4 複数のアプリを起動した様子



第24章ではターミナルを複数起動できるようにします。本書で作るOSの設計では1ターミナル=1アプリですので、複数のターミナルが開けるということは複数のアプリを同時に使えるということです。ここまで来ると図0.4のような見た目になります。ずいぶん本格的な感じがしませんか？

第25章、第26章では、アプリがファイルを読み書きする仕組みを作ります。アプリはファイルディスクリプタ番号という整数値を使ってファイル进行操作します。ファイルを開くと新たなファイルディスクリプタ番号がOSによって割り当てられ、以降アプリはその番号を使ってファイルの読み書きなどを行います。OSは番号とファイルディスクリプタの対応表を持ち、アプリが開いたファイルの一覧、それぞれのファイルの読み込み、書き込み位置を管理します。整数値でファイルを指定するというやり方が必須なわけではありませんが、広く採用されている方法ですので本書でも真似しました。同じ仕組みにしておけば、他の技術者と会話するときに話が通じやすいですからね。アプリからファイルの読み書きができるようになると、作れるアプリの幅が一気に広がります。

第27章ではアプリが大量のメモリを確保したり、ファイルをメモリに見せかけたりする仕組みを作ります。ここで作る仕組みは地味ですが高度で、「OSを裏から支える仕組みを作っているぞ！」という感覚になるでしょう。作れるアプリの幅はほとんど広がりますが、技術的には興味深い内容となっています。

第28章の日本語表示は、OSの教科書としてはおまけみたいなものですが、でもやっぱり日本語に対応すると見た目が豪華になって楽しいので、ここで挑戦することにしました。日本語フォントは、本質的には英数字のフォントと同じです。文字数の多い日本語を表示するためにたくさんの文字の形が収録されているというだけのことです。FreeTypeというライブラリの力を借りて、TrueTypeフォントをえるようにします。読者の皆さんの好きなフォントに変更して遊んでみてください。

第29章で挑戦するアプリ間通信^{*12}（パイプ）は、複数のアプリが互いに情報交換するための仕組みです。今まで各アプリは単独で起動し、何らかの処理をして終了する、というものでした。アプリ間通信によって、あるアプリの出力結果を他のアプリに入力して処理させることができます。小さなアプリ

を組み合わせることで複雑な仕事をさせられるようになり、一段とアプリの応用範囲が広がりますよ。

第30章は、おまけの機能やアプリを作ります。テキストファイルを大画面で見るためのテキストビューア、画像ファイルを表示する画像ビューアを作ると、画面が一気に華やかになります。3枚の画像を表示した様子を図0.5に示します。素敵でしょ？

図0.5 画像ビューアで3つの画像を表示した



* 12 一般にはプロセス間通信と呼ばれます。

第 1 章

PCの仕組みと ハローワールド

本章では OS 作りの一歩目を踏み出すために、他の OS が無い状態でコンピュータの画面にメッセージを表示してみます。OS の力を借りないということは、コンピュータの生の機能を使うということです。コンピュータはデジタル回路で作られていて 2 進数しか扱えないのに、どうして文字が表示できるのでしょうか？バイナリエディタを使ったプログラミングを通して、仕組みを簡単に説明します。本章の最後に、C 言語を使ってメッセージの表示をやり直してみます。

1.1 ハローワールド

さて、いよいよ OS 作りを始めていきましょう。手始めに、他の OS の力を借りずに起動して、画面にメッセージを表示する簡単なプログラムを作ってみます。普通はこういうとき、プログラミング言語を使ってプログラムを書きますが、本書では最初の一步としてプログラミング言語を使わずにやってみます。まずは普通の (WSL ではない) Ubuntu でのやり方を紹介し、その後で WSL での違いを示します。

作成したプログラムの実行は、もし試験用パソコンがあればそれでやってみてください。試験用パソコンが無くてもエミュレータでの実行方法を紹介しますので、安心してください。

ここでは、プログラムの作成にバイナリエディタというものを使います。バイナリエディタはどんなファイルも作れる万能のツールです。これから作るのは画面にメッセージを表示するプログラムですが、これも 1 種のファイルです。もちろんバイナリエディタで作れちゃうわけです。

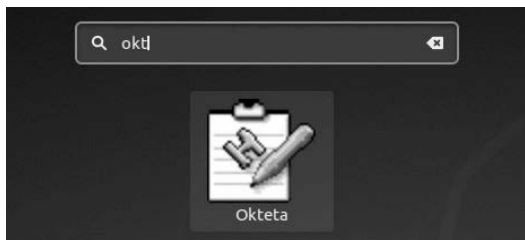
いろんなバイナリエディタがありますが、バイナリエディタであればどれも基本的な機能は一緒です。お好みで選んでいいのですが、何があるか分からんという方も多いでしょう。Linux であれば「Okteta」が完成度が高く使いやすいと思います。これは無料のツールですから気軽に試せます。Ubuntu では「Ubuntu ソフトウェア」を使うか、次のコマンドラインを使えばインストールできます。

```
$ sudo apt install okteta
```

先頭の \$ 記号は、この行がコマンドラインであることを示します。皆さんが入力する必要はありません。入力するのは sudo 以降の文字列です。

前記のコマンド実行が成功したら Okteta のインストールは完了です。次は Okteta を起動しましょう。Ubuntu で Windows キー*¹を押すと出てくる画面で「okteta」と入力し、Okteta を探します。出てきたアイコンをクリックすると Okteta が起動します (図 1.1)。

図 1.1 「okteta」を検索し Okteta を起動する



それではバイナリエディタを起動させ、図 1.2 に示す通りに数値を打ち込んでいきましょう。といってもほとんどが「0」ですが。1 行すべてが 0 となる区間は「*」で省略しています。ここで入力した数値が何を意味するのかは後ほど説明します。

* 1 Windows マークが書かれたキー。Mac ではコマンドキー、Linux ではスーパーキーと呼ばれます。

図 1.2 バイナリエディタで入力する謎の数値列

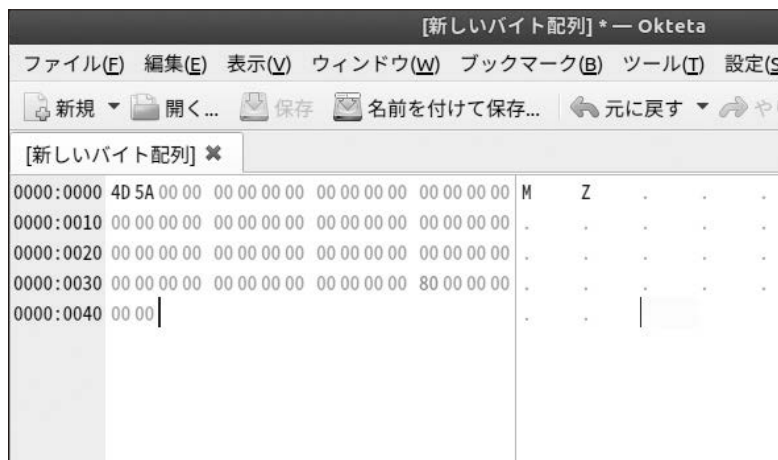
```

00000000 4d 5a 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |MZ.....|
*
00000030 00 00 00 00 00 00 00 00 00 00 00 00 80 00 00 00 |.....|
*
00000080 50 45 00 00 64 86 02 00 00 00 00 00 00 00 00 00 |PE..d.....|
00000090 00 00 00 00 f0 00 22 02 0b 02 00 00 00 02 00 00 |.....".|
000000a0 00 02 00 00 00 00 00 00 00 10 00 00 00 10 00 00 |.....|
000000b0 00 00 00 40 01 00 00 00 00 10 00 00 00 02 00 00 |...@.....|
000000c0 00 00 00 00 00 00 00 00 06 00 00 00 00 00 00 00 |.....|
000000d0 00 30 00 00 00 02 00 00 00 00 00 00 0a 00 60 81 |.0.....\|
000000e0 00 00 10 00 00 00 00 00 00 10 00 00 00 00 00 00 |.....|
000000f0 00 00 10 00 00 00 00 00 00 10 00 00 00 00 00 00 |.....|
00000100 00 00 00 00 10 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
00000180 00 00 00 00 00 00 00 00 2e 74 65 78 74 00 00 00 |.....text...|
00000190 14 00 00 00 00 10 00 00 00 02 00 00 00 02 00 00 |.....|
000001a0 00 00 00 00 00 00 00 00 00 00 00 20 00 50 60 |.....P^|
000001b0 2e 72 64 61 74 61 00 00 1c 00 00 00 00 20 00 00 |.rdata.....|
000001c0 00 02 00 00 00 04 00 00 00 00 00 00 00 00 00 00 |.....|
000001d0 00 00 00 00 40 00 50 40 00 00 00 00 00 00 00 00 |....@.P@.....|
*
00000200 48 83 ec 28 48 8b 4a 40 48 8d 15 f1 0f 00 00 ff |H..(H.JeH.....|
00000210 51 08 eb fe 00 00 00 00 00 00 00 00 00 00 00 00 |Q.....|
*
00000400 48 00 65 00 6c 00 6c 00 6f 00 2c 00 20 00 77 00 |H.e.l.l.o.,.w.|
00000410 6f 00 72 00 6c 00 64 00 21 00 00 00 00 00 00 00 |o.r.l.d.!.....|
*
00000600

```

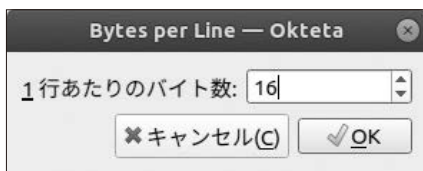
筆者が Okteta でこの数値列を入力している様子を図 1.3 に示します。

図 1.3 Okteta で数値列を入力している様子



Okteta は初期設定ではちょっと違った表示になっているかもしれません。「表示」メニューの「1 行あたりのバイト数を設定」で「16」を設定すると筆者と同じような表示になると思います (図 1.4)。

図 1.4 Okteta での表示を調整する様子



数値といいつつ「4d」のように文字が混じっています。これは、前記の数値が 16 進数で書いてあるからです。16 進数については後ほど説明します。今のところは、図の通りに入力してみてください。入力し終えたら、それを「BOOTX64.EFI」というファイル名で保存します。保存する場所はどこか分かりやすいところにおきます。

BOOTX64.EFI の内容が正しいかどうか、前記の数値列とにらめっこして間違いを探すのは至難の業です。そこでチェックサムという技術を紹介します。チェックサムは専門用語で言うと誤り検出符号というもので、チェックサムの値を比べるだけでファイルの内容が等しいか、異なっているかを（高い確率で）判定できます。チェックサムを計算するには sum コマンドを使います。

```
$ sum BOOTX64.EFI
12430      2
```

バイナリエディタで間違いなく入力できていれば、BOOTX64.EFI のチェックサムは「12430」となるはずですが。入力ミスがあるとチェックサムが違う値になります。ただ、チェックサムだけでは誤りの場所までは分かりませんので、最終的には目視で頑張って誤りを探してください。それが面倒な方は、本書のサポートサイトにあるファイル^{*2}を利用しても良いですよ。

ここからは、作ったプログラムをパソコンで実行する方法を説明します。実機ではなくエミュレータで実行する方法は「**1.4 エミュレータでの実行**」で紹介します。

実機で実行する場合は作ったプログラムが入った USB メモリを準備する必要があります。それにはまず USB メモリを FAT 形式でフォーマットし、次に先ほど作った BOOTX64.EFI を USB メモリの /EFI/BOOT ディレクトリ^{*3}にコピーします。USB メモリをパソコンに挿したときのデバイス名を /dev/sdb とすると、具体的なコマンド列は次のようになります。実際のデバイス名を探す方法は「**1.2 USB メモリのデバイス名の探し方**」で説明します。

```
$ sudo umount /dev/sdb1
$ sudo mkfs.fat /dev/sdb1
$ sudo mkdir -p /mnt/usbmem
$ sudo mount /dev/sdb1 /mnt/usbmem
$ sudo mkdir -p /mnt/usbmem/EFI/BOOT
$ sudo cp BOOTX64.EFI /mnt/usbmem/EFI/BOOT
$ sudo umount /mnt/usbmem
```

* 2 <https://github.com/uchan-nos/mikanos-build/blob/master/day01/bin/hello.efi>

* 3 ディレクトリとは複数のファイルをしまっておく場所のこと。詳しくは「25.1 ディレクトリ対応」を参照。

BOOTX64 をコピーしたら USB メモリを試験用パソコンに挿し替えて、試験用パソコンの電源を入れます。すると画面に「Hello, world!」と表示されるはずです（開発用パソコンで動作確認もするのなら、USB メモリを挿したまま再起動すれば OK です）。

図 1.5 GPD Micro PC に USB メモリを挿して起動してみた

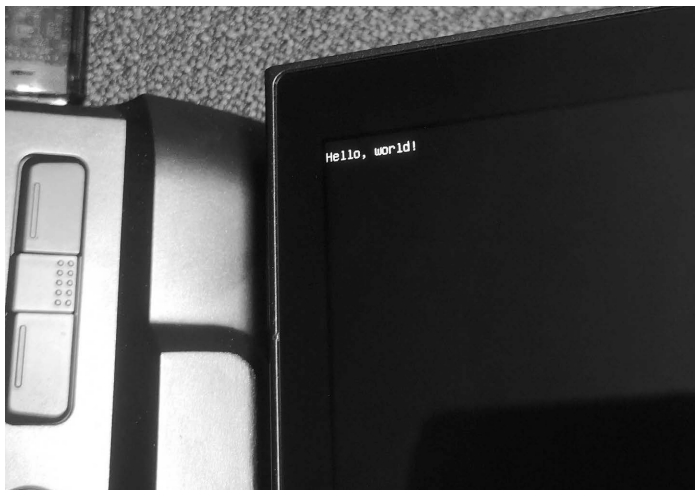
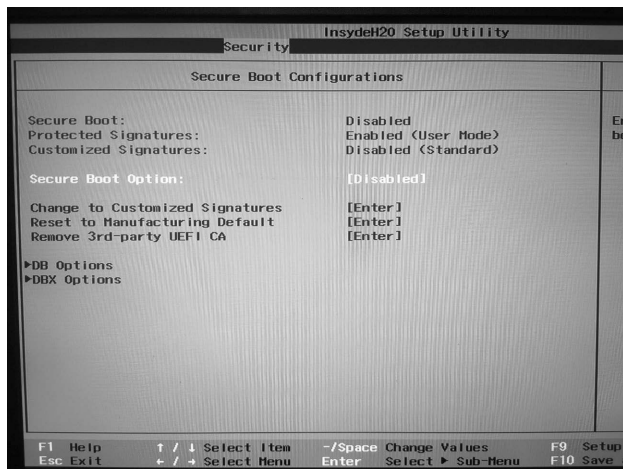


図 1.5 は筆者が MikanOS の実験に使っている小型 PC 「GPD Micro PC」に USB メモリを挿入し、起動してみた様子です。非常に小さいですが「Hello, world!」と表示されています。

もしかすると、みなさんのパソコンでは BOOTX64.EFI が起動しないかもしれません。よくある原因の 1 つはセキュアブートと呼ばれる機能が有効になっていることです。本来は改ざんされた不正な OS を起動させないための機能なのですが、自作 OS の起動も妨害されてしまいますので、無効にしておかないといけません。BIOS 設定画面にてセキュアブートを無効化します（図 1.6）。

図 1.6 セキュアブートを無効化する



BIOS 設定画面の出し方はパソコンの機種によって異なりますが、例えば、Delete キーか F2 キーを押しながらパソコンの電源を入れることで出すことができます（パソコンのマニュアルを確認してください）。

セキュアブートの設定項目がある場所も機種により異なります。筆者のパソコンでは「Security」メニューの中に「Secure Boot Option」という項目がありました。これを「Disabled」（無効）に設定します。設定を変えたら保存して再起動します。保存して再起動するには「Save Changes and Reset」のようなメニューを実行します。

BIOS の設定変更は慣れていないと難しいです。少し間違えるとパソコンが起動しなくなることもある、危険と隣り合わせの作業です。実験にしようと思っているパソコンが家族と共用であるなど、起動しなくなると困るものなら、BIOS の設定変更はかなり慎重にしてください。後で紹介するエミュレータを使った実験方法では BIOS の設定変更が不要ですから、心配ならその方法で実験してください。とは言っても、BIOS の設定変更や実機での実験は良い経験になりますから、壊れても大丈夫なパソコンをぜひ手に入れてほしいと筆者は思っています。

1.2 USBメモリのデバイス名の探し方

前記のコマンド例では USB メモリが `/dev/sdb` と認識されると仮定しました。実際にどんな名前になっているかは、USB メモリを挿入した後に `dmesg` コマンドを実行すれば分かります。筆者のパソコンに BUFFALO の USB メモリを挿入した直後の `dmesg` の出力は次のようになりました。

```
$ dmesg
...
[573264.472481] usb 2-2: new SuperSpeed Gen 1 USB device number 5 using xhci_hcd
[573264.493232] usb 2-2: New USB device found, idVendor=0411, idProduct=01dd,
bcdDevice= a.00
[573264.493233] usb 2-2: New USB device strings: Mfr=1, Product=2, SerialNumber=3
[573264.493234] usb 2-2: Product: USB Flash Disk
[573264.493235] usb 2-2: Manufacturer: BUFFALO
[573264.493236] usb 2-2: SerialNumber: 0902000000CE30204B00009026
[573264.494355] usb-storage 2-2:1.0: USB Mass Storage device detected
[573264.494589] scsi host0: usb-storage 2-2:1.0
[573265.516972] scsi 0:0:0:0: Direct-Access    BUFFALO  USB Flash Disk
1.00 PQ: 0 ANSI: 5
[573265.517314] sd 0:0:0:0: Attached scsi generic sg0 type 0
[573265.517608] sd 0:0:0:0: [sda] 30162944 512-byte logical blocks: (15.4 GB/14.4 GiB)
[573265.517722] sd 0:0:0:0: [sda] Write Protect is off
[573265.517724] sd 0:0:0:0: [sda] Mode Sense: 23 00 00 00
[573265.517841] sd 0:0:0:0: [sda] Write cache: disabled, read cache: disabled,
doesn't support DPO or FUA
[573265.519160]  sda: sda1
[573265.520001] sd 0:0:0:0: [sda] Attached SCSI removable disk
$
```

この出力からは、BUFFALO の USB メモリ（USB Flash Disk）が挿入され、`sda` という名前で認識されたことが分かります。この場合、前記のコマンド列の `/dev/sdb` を `/dev/sda` に置き換えて実行します。

1.3 WSLでのやり方

WSL (Windows Subsystem for Linux) 上の Ubuntu で行う場合、操作の流れは普通の Ubuntu とほぼ同じですが、いくつか異なる点があります。

まずはバイナリエディタです。普通の Ubuntu では Okteta を使いましたが、WSL で作業する際は Windows 用のバイナリエディタを使う方が良いでしょう。筆者が思うに、Windows 向けのバイナリエディタの方が完成度が高いものが多いです。お勧めは「Binary Editor Bz」です。このエディタは「30日でできる! OS自作入門」でも紹介されている歴史のあるもので、オリジナルの作者 c.mos さんの後を tamachan さんが継いで開発が続いているようです*⁴。

さて、次は WSL における USB メモリのフォーマットと BOOTX64.EFI の書き込み方を説明します。普通の Ubuntu ではどちらもコマンド操作で行っていましたが、WSL ではコマンドで USB メモリのフォーマットができません。そこで、フォーマットは Windows 側で行い、その後 WSL のコマンドでファイルをコピーすることにします。

USB メモ리를 Windows パソコンに挿入すると自動的に認識され、1 文字のドライブレターが付与されます。筆者の環境では「F」になりました。この USB メモ리를 フォーマットするには、エクスプローラー上で USB メモ리의 アイコンを右クリックし「フォーマット」を選びます (図 1.7)。

図 1.7 USB メモ리를 Windows でフォーマットする



図 1.8 ファイルシステムは exFAT を選択する



* 4 公式サイトは「<https://gitlab.com/devill.tamachan/binaryeditorbz/>」です (2021 年 1 月現在)。

すると、**図 1.8** のような画面が出てきます。ファイルシステムを「exFAT」に設定してフォーマットします。BOOTX64.EFI を動作させるためには NTFS にしてはいけません^{*5}。ボリュームラベルは英字で 11 文字以内の適当な名前にしておきます。ここでやったフォーマット作業は `mkfs.fat` コマンドの代わりとなります。

図 1.9 スタートメニューから Ubuntu を起動する



フォーマットが終わったら、次は BOOTX64.EFI を USB メモリにコピーします。この作業は WSL の Ubuntu 上で行います。WSL は**図 1.9** のようにスタートメニューから起動できます。なお、このメニュー項目は WSL のインストールを完了しないと出ません。WSL のインストール方法は「**付録 A.01 WSL のインストール**」を参照してください。

WSL が起動したら、USB メモリを `/mnt/usbmem` にマウントし、ファイルをコピーします。

```
$ sudo mkdir -p /mnt/usbmem
$ sudo mount -t drvfs F: /mnt/usbmem
$ sudo mkdir -p /mnt/usbmem/EFI/BOOT
$ sudo cp BOOTX64.EFI /mnt/usbmem/EFI/BOOT
$ sudo umount /mnt/usbmem
```

WSL と普通の Ubuntu との違いは `mount` の行だけです。WSL では `mount` コマンドのオプションとして `/dev/sdb1` の代わりに「`-t drvfs F:`」を指定します。もちろん「`F:`」の部分は皆さんの環境に合わせ、適切なドライブレターに変えてください。このやり方は WSL 1 でも WSL 2 でも同じです。

*5 なぜなら UEFI の仕様がそうになっているから。BOOTX64.EFI を配置する EFI システムパーティションのフォーマットは FAT でなければなりません。

1.4 エミュレータでのやり方

パソコンの代わりに、ソフトウェアで仮想的なパソコンを再現できる「QEMU」というエミュレータを使う方法を説明します。ここで説明する手順を実行するには「開発環境」をインストールしておく必要があります。「付録 A 開発環境のインストール」を参照してインストールしてください。WSL 上の Ubuntu で QEMU を使うためには追加の準備が必要ですから、「付録 A.02 WSL で QEMU を使う準備」を参照し、X サーバのインストールもしてください。

QEMU で BOOTX64.EFI を起動するには 2 段階の処理が必要です。まずは BOOTX64.EFI を含んだディスクイメージを作成し、次に QEMU にそのディスクイメージを読み込ませて起動します。

ディスクイメージを作成するコマンドは次の通りです。

```
$ qemu-img create -f raw disk.img 200M
$ mkfs.fat -n 'MIKAN OS' -s 2 -f 2 -R 32 -F 32 disk.img
$ mkdir -p mnt
$ sudo mount -o loop disk.img mnt
$ sudo mkdir -p mnt/EFI/BOOT
$ sudo cp BOOTX64.EFI mnt/EFI/BOOT/BOOTX64.EFI
$ sudo umount mnt
```

qemu-img コマンドで 200MB の空のファイルを作って FAT 形式でフォーマットし、そこに BOOTX64.EFI ファイルを書き込む、という流れです。最終的に BOOTX64.EFI を含んだ disk.img というファイルが作成されます。

sudo の実行には特権が必要なため、パスワードを聞かれるでしょう。WSL のインストール時に設定したパスワードを入力してください。1 度パスワードを入力するとしばらくの間はパスワード無しで sudo を実行できるようになります。

作ったディスクイメージを QEMU で起動するコマンドは次の通りです。

```
$ qemu-system-x86_64 \
  -drive if=pflash,file=$HOME/osbook/devenv/OVMF_CODE.fd \
  -drive if=pflash,file=$HOME/osbook/devenv/OVMF_VARS.fd \
  -hda disk.img
```

2 行目と 3 行目の長いオプションは、QEMU を UEFI モードで起動させるために必要です。これらのオプションを指定しないと、QEMU はレガシー BIOS モードで起動してしまいます。私たちが作った BOOTX64.EFI は UEFI モードが前提のプログラムで、レガシー BIOS モードではうまく動作しません。UEFI については「1.7 UEFI BIOS による起動」で詳しく説明します。

ディスクイメージを作って QEMU で起動するという操作はよく使うので、それらをまとめて実行するスクリプト「run_qemu.sh」を開発環境に同梱しました。そのスクリプトを使うと前記の長い手順は次の 1 行になります。

```
$ $HOME/osbook/devenv/run_qemu.sh BOOTX64.EFI
```

ね、便利でしょ？

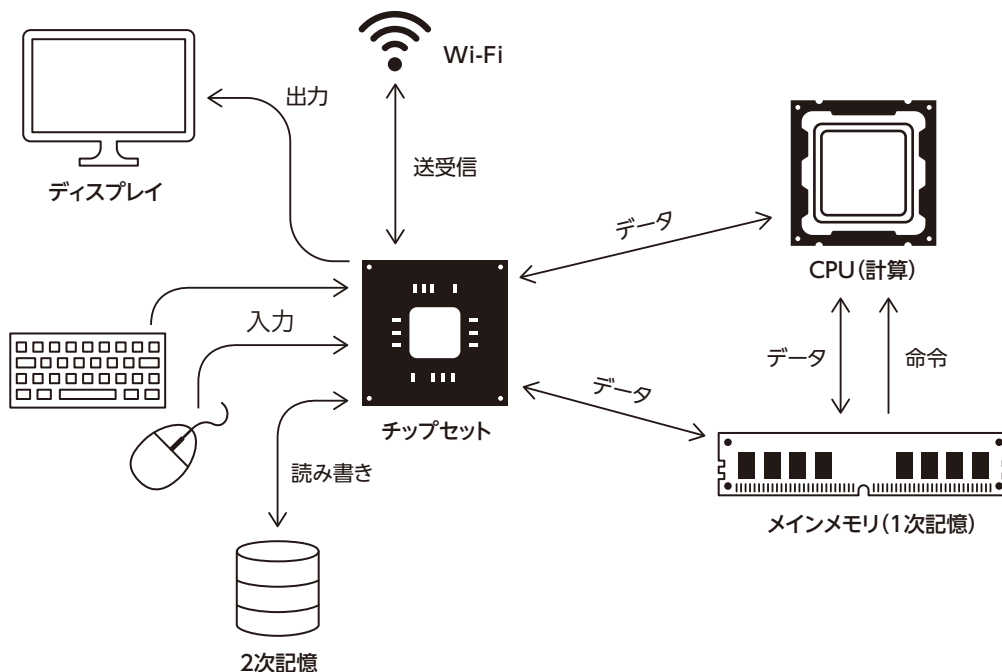
1.5 結局、何をやったのか？

ここまでで、何だか良くわからないファイルを作り、それを USB メモリの中に特定の名前で配置し、パソコンの電源をつけました。そうすると画面にメッセージが表示されました。この一連の流れは、コンピュータが起動して動作する様子を端的に表しています。OS 作りの第一歩となる重要な内容ですから、順を追って説明します。

まず最初は、バイナリエディタで作った BOOTX64.EFI ファイルについて。このファイルは実行可能ファイルと呼ばれるものです。実行可能ファイルとは、CPU が実行可能な**機械語**を含んだファイル、の意味です。

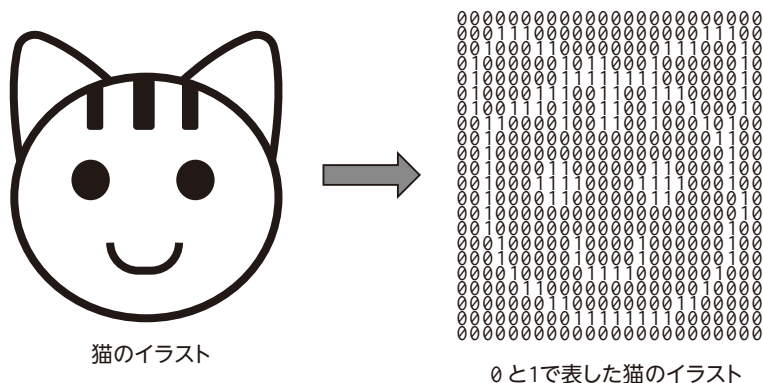
図 1.10 に示す通り、みなさんが使っているコンピュータというものは命令を実行する CPU、CPU が処理する命令やデータを一時的に置いておくメインメモリ、それらを永続的に保存するストレージ(HDD や SSD)、人間とコンピュータを繋ぐための入出力装置(マウス、キーボード、ディスプレイ)などから構成されています。CPU はメインメモリから命令やデータを取ってきて実行し、結果をメインメモリに書き戻す、というような処理をずっと繰り返します。

図 1.10 コンピュータの構成図



CPU はデジタル回路というもので構成されています。デジタル回路というのは電圧が高いか、低いか、という 2 つの値で動作するような回路です。2 種類の値しか扱えないと言うと、ほとんど何もできなさそうですが、そんなことはないのです。仮に、電圧が高いことを 1、低いことを 0 と表すとすると、0 と 1 でお絵描きをすることができます (図 1.11)。

図 1.11 2 値のお絵かき



また、0 と 1 を扱えるデジタル回路をいくつか組み合わせれば複数の 0 と 1 を扱えるようになります。例えば 00、01、10、11 というような 2 桁の数を扱うデジタル回路を作ることができます。00 を 0、01 を 1、10 を 2、11 を 3 だと思うことにすれば、0 から 3 までの数値を扱える回路だと考えることができますね。桁をどんどん増やしていけば、大きな整数を扱えるようになります。

私たちが普段使う数の表し方は **10 進数** と呼びます。10 進数の世界では、1 つの桁は 0 から 9 までの 10 種類の数値で表します。9 の次は桁が繰り上がって 10 となります。

一方、上で出てきた 00、01、10、11 のような数の表し方を **2 進数** と呼びます。2 進数の世界では、1 つの桁は 0 と 1 で表し、1 の次は繰り上がって 10 となります。0 から順に数えると、0、1、10、11、100、……というようになります。ちなみに、本来の 2 進数では先頭の 0 は書きません。10 進数の世界で 42 のことを 042 と書かないのと同じです。でも、デジタル回路の世界では、扱える桁数を明示するために 001 のように書くことがあります。この場合、3 桁まで扱える回路だな、というのが分かります。

困ったことに「100」と書いたときに、それが 10 進数なのか 2 進数なのか見分けがつきません。そこで C++ では、10 進数はそのまま、2 進数は 0b100 などと先頭に「0b」を書くことで、両者を区別することになっています。本書でも、どちらか紛らわしいときにはこの書き方を採用することにします。

デジタル回路の世界では、2 進数の 1 桁のことを **ビット (bit)** と呼びます。1 ビットは 0 か 1 のどちらかの状態を取ります。複数のビットをまとめて扱うことで、大きな桁の数値を表現することができます。典型的には、8 ビットをまとめたものを **バイト (byte)** と呼びます。

2 進数はデジタル回路とすごく相性が良いのですが、すぐに桁が増えてしまうという不便さがあります。そこで、2 進数の代わりに 16 進数を使うことがよくあります。本書でもときどき登場しますので、ついでに紹介しておきます。

10 進数が 10 個の数字を使い、2 進数が 2 個の数字を使うのと同様に、16 進数では 1 つの桁を **16 種類** の「数字」で表します。といっても、私たちが知っている数字は 0 から 9 の 10 種類だけ。16 種類を表すには足りません。そこで 0 から 9 に加えて、A から F の 6 つの文字を使います。つまり、0、1、……、9、**A**、**B**、……、**F** となり、その次は桁が繰り上がって 10 となります。10 進数と 16 進数を区別したいときは 16 進数の先頭に「0x」を書きます。16 進数は 2 進数と相性が良くて便利なのですが、どう相性が良いかは少し後に説明します。

2 進数、10 進数、16 進数の対応関係を表 1.1 に示します。

表 1.1 2 進数、10 進数、16 進数の対応関係

10 進数	16 進数	2 進数	10 進数	16 進数	2 進数
0	0x0	0b0000	16	0x10	0b10000
1	0x1	0b0001	17	0x11	0b10001
2	0x2	0b0010	18	0x12	0b10010
3	0x3	0b0011	19	0x13	0b10011
4	0x4	0b0100	20	0x14	0b10100
5	0x5	0b0101	21	0x15	0b10101
6	0x6	0b0110	22	0x16	0b10110
7	0x7	0b0111	23	0x17	0b10111
8	0x8	0b1000	24	0x18	0b11000
9	0x9	0b1001	25	0x19	0b11001
10	0xA	0b1010	26	0x1A	0b11010
11	0xB	0b1011	27	0x1B	0b11011
12	0xC	0b1100	28	0x1C	0b11100
13	0xD	0b1101	29	0x1D	0b11101
14	0xE	0b1110	30	0x1E	0b11110
15	0xF	0b1111	31	0x1F	0b11111

この表を見ると、「16 進数の 1 桁」と「2 進数の 4 桁」がちょうど 1 対 1 に対応していることが分かります。2 進数を 4 桁ずつに区切って 16 進数で表すことで、どんなに桁数が大きくても 2 進数と 16 進数を簡単に変換できちゃうのです。10 進数と 2 進数、あるいは 10 進数と 16 進数の変換はこんなに楽ではありません。0b1111101011001110 を 16 進数に変換する例を示します。

```
0b1111101011001110
= 1111 1010 1100 1110
=   F   A   C   E
= 0xFACE
```

少し脱線してしまいました。話を戻すと、CPU はデジタル回路であり、デジタル回路は電圧の高低、すなわち 2 進数を扱うのだ、というのが言いたかったことです。2 進数で表さなければ CPU には扱えない、とも言えます。現在主流のパソコンは 64 ビット CPU を搭載している、というのを聞いたことがあるかもしれませんが、それは 1 回の計算で 64 桁の 2 進数を扱えるということです。CPU は 10 進数の計算はできませんが、10 進数を 2 進数で表して計算することで、10 進数の世界でできる計算ならどんなものでも計算できます。要するに、CPU は任意の数値を扱うことができるわけです。

コンピュータで扱う数値は大きく 2 つに分けられます。CPU が実行するための命令「機械語命令」か、それ以外かです。皆さんがバイナリエディタで打ち込んだ BOOTX64.EFI の中には、それら両方が含まれています。機械語命令としては文字列を画面に表示する命令が該当します。機械語命令以外では、画

面に表示するための文字列「Hello, world!」とメタデータ^{*6}ですね。

画面に表示するための文字列の場所は、BOOTX64.EFI の 0x0400 から 0x041b あたりに埋め込まれています。バイナリエディタに「Hello, world!」と表示されているのでパッと見て分かるはずです。ここで注目したいのは、それぞれの文字を数値として表す方法です。文字は数字ではないので、そのまま数値で表わせと言われても難しいわけです。そこで、コンピュータの世界では文字に数値を対応付けて扱うことにしています。例えば A から Z を 0x41 から 0x5A、a から z を 0x61 から 0x7A に対応させるとすると、H は 0x48、o は 0x6f となります。

このような文字と数値（バイト列）の対応付け規則のことを「文字コード」と呼びます。文字コードはたくさん種類がありまして、有名なものに「ASCII コード」や「Unicode」、「Shift-JIS」などがあります。ASCII コードは英語のアルファベットを表すための文字コードとして非常に有名です。「付録 F ASCII コード表」に、ASCII コードで決まっているすべての文字とバイトの対応を載せました。文字コードは一般に文字をバイト列に対応付けるものですが、ASCII コードは非常に単純で、1 文字が 1 バイトに対応します。

BOOTX64.EFI の中で文字を表すには Unicode の一種である UCS-2 を使います^{*7}。UCS-2 は 1 文字を 2 バイト（16 ビット）で表す文字コードです。ASCII コードとは異なる文字コードではありますが、英字の範囲では ASCII コードと同じ値に 1 バイトの 0 を付加した 2 バイトの列となります。BOOTX64.EFI をバイナリエディタで見ると各文字の間に 0x00 が挟まっていますよね。これが付加された 1 バイトです。例えば H は ASCII において 0x48 ですから、UCS-2 においては 0x0048 です。バイト単位で表すと「0x48 0x00」となります。

ちなみに、多バイトの数値をどのように並べるかで複数の方法が考えられます。0x0102 という 2 バイトの数値を下位バイトから「0x02 0x01」と並べるやり方をリトルエンディアン、上位バイトから「0x01 0x02」と並べるやり方をビッグエンディアンと呼びます。本書で対象にしている x86-64 アーキテクチャはリトルエンディアンを採用していますので、バイナリエディタで入力するときは下位バイトを先に入力します。

話を戻します。画面に表示するための文字列の場所は、BOOTX64.EFI の 0x0400 から 0x041b あたりにあることが分かりました。一方で、機械語命令は 0x0200 から 0x022f あたりにあります。CPU が読むための命令ですから人間にはちょっと分かりにくいですがね。こんな風に、実行可能ファイルは CPU が実行するための命令列（今回は、文字列を画面に出すという命令）とその命令列によって利用されるデータ（今回は「Hello, world!」という文字列）、それからメタデータを含んでいます。

そうやって作った実行可能ファイルを USB メモリに指定された名前で保存することで、パソコンに内蔵されている UEFI BIOS がそのファイルを読み取って実行します。UEFI BIOS については少し後で説明します。

*6 ファイルのフォーマットや大きさ、どんな CPU 向けの機械語が入っているかなど、データ本体を説明するための付加情報をメタデータと呼びます。

*7 なぜなら BOOTX64.EFI が動作する UEFI の世界では UCS-2 を使うことが決まっているからです。

1.6 とにかく手を動かそう

ここまでの説明は、OS 自作初心者にとってかなり難しい話でしょう。簡単に伝えられないのは筆者の力不足という面もありますが、本質的に難しい話題なのは確かだろうと思っています。

読者のみなさんはきっと、きっちり理解してから先に進みたいと思っているでしょう。でも、挫折せずに本書を読み進めるには、説明が分からなくてもとりあえず手を動かし、先に進んでほしいと思っています。手を動かしてプログラムを書いたり、実行したりすることで、本文を読むだけに比べて理解が深まります。また、ちょっと先に進んでから振り返ると理解できることもあるかもしれません。

本書のコンセプトは、筆者が OS を作る過程を細かく刻んで紹介するというものです。OS に関する理論を体系的に説明することは目指していません。理論的な教科書の章末にある練習問題だけを抽出したような本だとも言えます。そのような事情もあり、手を動かすことで理解するのがお勧めです。筆者の OS 作りを追体験してみてください。

本書で紹介している改造だけでなく、ぜひ、独自の改造に挑戦してください。独自の改造をするためには内容の深い理解が必要です。そのために紙面を戻って読み直したりする必要があるでしょう。最終的に独自の改造ができたとき、それは内容を深く理解した証となります。

プログラムのソースコードを手で写してみる「写経」はお勧めの学習方法です。筆者の経験では、付属のサンプルコードをコピーして実行するだけの場合に比べ、細かい点まで注意が向くために理解が捗ります。ただ、本文中に完全な差分が載っているわけではないので注意してください。本文で紹介された変更をすべて写経したとしても改造が不十分のままになる可能性があります。サンプルコードの差分を「git diff」コマンドで表示することで、紙面で紹介できていない差分まで含めて完全な差分が分かります。詳しくは「付録 B.01 MikanOS のバージョン間の差分確認」を参照してください。

1.7 UEFI BIOS による起動

BOOTX64.EFI を USB メモリに保存し、その USB メモリをパソコンに挿して電源を入れると、画面に「Hello, world!」が表示されました。CPU が BOOTX64.EFI の中にある機械語を実行したからです。ただ、CPU が USB メモリの内容を直接読んで実行しているかという点、実は違います。パソコンに内蔵されている BIOS がその役を担っているのです。

BIOS は、コンピュータの電源を入れて最初に行われるファームウェアと呼ばれるプログラムです。「Basic Input Output System」の略で、基本的な入出力機能、特に OS 起動前にコンピュータ内部を初期化したり、OS（ブートローダ）をストレージから読み出す機能を提供します。パソコンの起動時に Delete キーか F2 キーを押下することで設定画面に入ることができるアレです。

UEFI^{*8}という標準仕様に仕様が作られた BIOS を「UEFI BIOS」と言います。歴史的にはレガシー BIOS という古いファームウェアが使われてきましたが、最近ではすっかり UEFI BIOS で置き換えられています。「30 日のできる！ OS 自作入門」(参考文献 [1]) が出版された 2006 年は、レガシー BIOS から UEFI BIOS へちょうど切り替わりつつある時期でした。本書で BIOS と言ったとき、特に断らないかぎりには UEFI BIOS のことを意味します。

USB メモリに保存した実行可能ファイルが実行されるまでの大雑把な流れは次の通りです。

まず、コンピュータの電源を入れると CPU は BIOS の実行を開始します。BIOS を実行する、というのは、CPU が BIOS に格納された機械語プログラムを実行するということです。BIOS（に格納された機械語プログラム）はコンピュータを初期化した後、接続されているストレージを探索するように作ら

れています。ストレージの中に実行可能ファイルを見つけると、BIOSはそのファイルをメインメモリへ読み出します。その後、CPUはBIOSの実行を中断し、読み出したファイルの実行を開始します。各段階をもう少し詳しく見てみます。

パソコンの電源が切れるとメインメモリの内容は消えます^{*9}から、次にパソコンの電源を入れたとき、メインメモリには何もデータがありません。CPUはメインメモリに置いてあるプログラムしか実行できない^{*10}ため、それでは困ってしまいます。そこで、メインメモリと同様に扱えて電源を切っても消えないメモリ（ROM）にBIOSを書き込んでおき、CPUがそのままBIOSを実行できる仕組みがパソコンに備わっています。

そのようにして実行を開始したBIOSは、まずパソコン本体や周辺機器を初期化します。具体的にはCPUの動作モードを設定したり、PCIデバイスを検出して設定したりします。その後、パソコンに接続されたストレージ（HDD、SSD、USBメモリ、DVDなど）を検索して実行可能ファイルを探します。BOOTX64.EFIのようなファイルが特定のディレクトリに存在するのを見つけると、そのファイルをメインメモリに読み込みます。次にBIOSは自分自身の実行を中断し、読み込んだプログラムの実行を開始します。

UEFI BIOSが実行してくれるプログラム（今回の例ではBOOTX64.EFI）のことをUEFIアプリケーションと呼びます。つまり、皆さんはバイナリエディタでUEFIアプリケーションを作ったということになります。UEFIアプリケーションはUEFI BIOSの機能を使うことができます。例えば、画面に文字列を表示したり、キーボードから文字を入力したり、ストレージからファイルを読み込んだり。UEFIアプリケーションとして作れるものは幅広いですが、本書ではもっぱらOSをメインメモリに読み込んで起動させるためのブートローダを作っていきます。

1.8 OSを作る道具

バイナリエディタは2進数で表せるものなら何でも作成できる万能のツールです。感動的な文章や音楽のファイルだって、結局は2進数のかたまりなのでバイナリエディタで作れます（作詞作曲能力があるかどうか、という別の問題はありますけど）。あるいはBOOTX64.EFIやコンピュータゲームのようなプログラムだって2進数で作られていますのでバイナリエディタで作成できます。

ただ、バイナリエディタで何でも作れるとはいっても、決して「作りやすい」わけではありません。それぞれのファイルを作るために専用の道具を使った方が断然作りやすいです。バイナリエディタは、いわばまっさらな紙です。万能ですが最適というわけではありません。文章作成なら原稿用紙、作曲なら五線譜など、専用紙を使う方が便利です。OS自作において原稿用紙や五線譜に当たるものは、何らかのプログラミング言語とテキストエディタです。

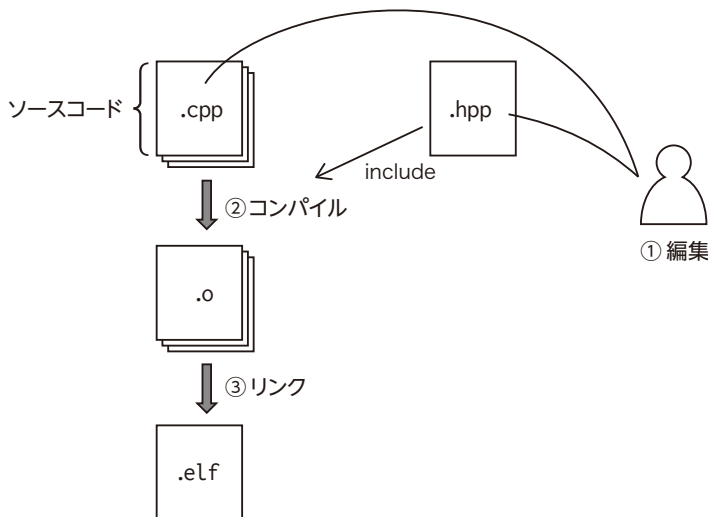
*8 Unified Extensible Firmware Interface。直訳すると「統一された、拡張可能なファームウェアインタフェース」となります。統一と拡張可能なニュアンスを筆者が推測してみると、「各社がばらばらに開発するのではなく1つの仕様に従う」という意味で統一されていて、「パソコンがもつ機能が増えても柔軟に対応できる」という意味で拡張可能であるということでしょう。

*9 最近ではDIMMスロットに挿せる不揮発性メモリNVDIMM（Non-Volatile DIMM）が実用化されており、電源が切れてもメインメモリの内容が消えないようなパソコンを作ろうと思えば作ることができます。近い将来、NVDIMMを前提とした新しい仕組みのコンピュータが普及するかもしれません。

*10 CPUは、メモリアドレス空間にマップされた領域からしか命令を読み取れません。

ということで、本書ではOSをプログラミング言語を使って作ることにします。バイナリエディタで全部作ると思っていた皆さん、ごめんなさい。プログラミング言語はいろいろありますが、本書ではCとC++を使います。C/C++を使った実行可能ファイルの作り方は図1.12のようになります。

図 1.12 実行可能ファイルを作成する流れ



この流れの開始点にあるソースコードは、プログラミング言語を使って書かれたテキストファイルです。テキストファイルの実体は、文字コードによって変換されたバイト列です。HDDやSSDもCPUと同様、数値（2進数）しか扱えませんから文字コードの列として記録するわけですね。

コンパイル処理を行うソフトウェアをコンパイラ（compiler）と呼びます。コンパイルによってソースコードからオブジェクトファイルを作ります。オブジェクトファイルはCPUが直接実行できる機械語命令や、その機械語命令が利用するデータなどを含んだファイルです。1つのソースコードは1つのオブジェクトファイルになります。最後にリンクという作業を行って、すべてのオブジェクトファイルをまとめて実行可能ファイルにします。

1.9 C言語でハローワールド

皆さんにバイナリエディタで打ち込んでもらったバイナリデータは、もともとはC言語プログラムを元にしたものです。C言語でメッセージを表示するプログラムを書き、コンパイル＆リンクしたファイルを参考にして、なるべく小さなサイズでメッセージが表示できるようにしてあります。

これからOS開発を進めていくにあたり、バイナリエディタでやり続けるのは辛いのでC言語でプログラムを作っていきたいです。ということで、バイナリエディタで打ち込んだ機械語プログラムと同じ動きをするC言語プログラムを見ていくことにします。すでに開発環境がインストールされていれば\$HOME/osbook/day01/c/hello.cにソースコードがあるはずです。開発環境のインストールについては「A.1 開発環境のインストール」を参照してください。

このファイルの中身は少しごちゃごちゃしていますが、プログラムの本体と言えるのは最後の方にある `EfiMain()` 関数だけです (リスト 1.1)。

この関数は、普通のプログラミングで登場する `main()` 関数と同じように、プログラムが起動したときに最初に行われる関数です。UEFI アプリケーションだということを分かりやすくするためにこの名前にしてみました。

リスト 1.1 EfiMain() の実装 (hello.c)

```
EFI_STATUS EfiMain(EFI_HANDLE ImageHandle,
                   EFI_SYSTEM_TABLE *SystemTable) {
    SystemTable->ConOut->OutputString(SystemTable->ConOut, L"Hello, world!\n");
    while (1);
    return 0;
}
```

この関数の中でも「`SystemTable->ConOut...`」という 1 行が、「Hello, world!」と画面に出力するための心臓部です。特に「`OutputString()`」が重要で、この関数によってメッセージが画面に出力されます。`EfiMain()` の 2 つの引数 `ImageHandle` と `SystemTable` は UEFI BIOS が値を設定してくれています。

このプログラムはこの第 1 章でしか登場しませんが、プログラム全体の解説はしないことにします。一応、このプログラムをコンパイル & リンクする手順は簡単に説明しておきます。前記の C 言語プログラムを「Clang」と「LLD」を使ってコンパイル & リンクする手順は次の通りです。この手順はすでに開発環境がインストールされていることを前提としています。

```
$ cd $HOME/osbook/day01/c
$ clang -target x86_64-pc-win32-coff \
    -mno-red-zone -fno-stack-protector -fshort-wchar -Wall -c hello.c
$ lld-link /subsystem:efi_application /entry:EfiMain /out:hello.efi hello.o
```

UEFI アプリケーションの実体は、Windows の標準的な実行可能ファイルの形式である PE 形式（詳しくは「**コラム 1.1 : PE と COFF と ELF**」を参照）です。そのため、Windows 用アプリを作る手順を流用して UEFI アプリを作ることができます。

2 行目の `clang` は Linux 用のコンパイラです。このコンパイラは通常は ELF 形式でファイルを出力するのですが、「`-target x86_64-pc-win32-coff`」を指定することで Windows 向けの COFF 形式で出力するようになります。次に実行する `lld-link` が COFF 形式を要求するため、このオプションを指定する必要があります。コンパイルが完了すると `hello.o` という COFF 形式のファイルが生成されます。

`lld-link` は PE 形式の実行可能ファイルを作るためのリンカです。ここでは `hello.o` をリンクして `hello.efi` を得るのに利用します。このリンカの主目的は Windows 用の PE ファイルを生成することですが、ありがたいことに、`/subsystem:efi_application` というオプションを指定すると UEFI 用の PE ファイルを生成してくれます。このオプションは、Windows 用の PE ファイルと UEFI 用の PE ファイルで若干異なる部分を考慮してファイルを作ってくれます。

`hello.efi` が C 言語版のハローワールドアプリです。このファイルを USB メモリの `/EFI/BOOT` ディレクトリに `BOOTX64.EFI` という名前でコピーし、その USB メモリをパソコンに挿して起動させると画面に「Hello, world!」と表示されるはずです。

やっぱりパソコンを操るためのOSですから本物のパソコンで動かすのが楽しいと思いますが、開発中に何度もUSBメモリへの書き込みとパソコンの再起動をするのは面倒ですね。C言語で作ったBOOTX64.EFIもバイナリエディタで作ったものと同じ手順で実行できます。

```
$ $HOME/osbook/devenv/run_qemu.sh hello.efi
```

こうするとQEMUが立ち上がり、しばらくするとhello.efiが実行されます(図1.13)。run_qemu.shの実行中にパスワードを聞かれたら、指示に従ってパスワードを入力してください。run_qemu.shが中でやっていることは「1.4 エミュレータでのやり方」で説明しています。

図 1.13 hello.efi を QEMU で実行



コラム 1.1 PE と COFF と ELF

皆さんはパソコン上でいろいろなアプリを使うと思います。ウェブブラウザ、テレビ会議、ワープロなど。この章で使ったバイナリエディタやコンパイラ、リンカなどもアプリですね。それぞれのアプリは実行可能ファイルや設定ファイルなどを組み合わせて構成されています。

それらのファイルの中で、実行可能ファイルはパソコンに指示するためのプログラムと、プログラムが動作するのに必要なデータを含みます。CPUは機械語だけを実行できる、という話をしました。実行可能ファイルにはいくつかの種類があります。1つはスクリプトと呼ばれるタイプの実行可能ファイルです。これは典型的にはプログラミング言語を使って書かれ、インタプリタと呼ばれるアプリがスクリプトの中身を読み取って実行します。例えば、シェルスクリプトは次のようなファイルです。

```
#!/bin/bash
echo "hello, world"
```

1行目の「#!/bin/bash」は、/bin/bashで指定されたインタプリタ（ここではBash）がこのスクリプトファイルを読み取って実行することを意味します。この行のことを「シェバン」(shebang、シバン)と呼びます。2行目以降にはBashが解釈できる形式でプログラムを書きます。このスクリプトを実行するように命ぜられたOS（シェル）は、Bashを起動し、2行目以降の内容をBashに入力します。

もう1つの実行可能ファイルの種類は機械語プログラムを中心とするものです。スクリプトとは異なり、こちらはCPUが直接実行できる機械語をファイルの中に含んでいます。この種類のファイルを実行するように命ぜられたOSは、その機械語をCPUが直接実行するように諸々の設定を行います。「**第18章 アプリケーション**」では、そのような機械語プログラムからなる実行可能ファイルを起動する機能をMikanOSに加えます。楽しみにしてください。

コラムの主題であるPE形式は、機械語プログラムからなる実行可能ファイルのためのファイル形式で、Windowsでの標準です。通常はPE形式のファイルの拡張子を「.exe」とします。これに対し、LinuxではELF形式が標準で使われます。

COFF形式はオブジェクトファイル形式の一種です。PE形式のファイルを作る過程の中間ファイル形式としてCOFF形式を使うことが多いです。COFF形式のオブジェクトファイルをリンクしてPE形式の実行可能ファイルを作る、という流れが一般的です。hello.cからhello.efiを生成するときにもそういう手順でやりましたよね。

実は、ELF形式はオブジェクトファイルの形式としても使えます。PE/COFFが実行可能ファイルとオブジェクトファイルの形式として分かれているのに対し、ELFはどちらも表現できるということです。ELFはExecutable and Linkable Format（実行可能かつリンク可能フォーマット）の略で、その名の通り実行可能ファイルもリンク可能ファイル（＝オブジェクトファイル）も表せる優れたものです。

PE、COFF、ELFはいずれも機械語を格納するファイル形式という点で共通しています。これらの形式の関係は、Word、PDF、HTMLの違いに似ています。いずれも文章や文字修飾の情報、画像などを含むことができる点で共通していますが、ファイル内部での表現方法が異なります。PE、COFF、ELFはいずれもx86-64 CPU用の機械語を含むことができますが、ファイル内部での表現（メタデータの付け方やデータの配置方法など）が違います。

歴史的にはa.outやCOM形式など、いろいろな実行可能ファイルの形式がありましたが、今はPEとELFがほとんどです。ちなみに「**はりぼてOS**」([参考文献 \[1\]](#))はHRBという独自の実行可能ファイル形式を採用しています。HRBはPEやELFと互換性がありません。

第 2 章

EDK II 入門と メモリマップ

このあと本格的に開発を進めるために、EDK II という UEFI 用の開発キットを使えるようにします。手始めに、1 日目に作ったハローワールドプログラムを EDK II を用いて再実装してみます。次に、EDK II の機能を活用してパソコンのメモリマップを取得し、それをファイルに保存してみます。

2.1 EDK II 入門

EDK II^{*1} は、もともと Intel が UEFI とその周辺のパログラムを実装し、それが後にオープンソースとして公開されたものです。EDK II は UEFI BIOS 自体の開発にも、UEFI BIOS 上で動くアプリケーションの開発にも使うことができる開発キットになっています。

何はともあれ、まずは EDK II を入手してみます。といっても、開発環境のインストールが完了していれば、すでに \$HOME/edk2 にダウンロードされているはずです。EDK II の公式リポジトリは <https://github.com/tianocore/edk2> にあります。

EDK II にどんなファイルがあるかを軽く見ておくことにします。ls コマンドを使ってファイルの配置を見てみると次のような感じになっていることが分かります。

edk2/	
edksetup.sh	環境変数設定用スクリプト
Conf/	
target.txt	ビルド設定
tools_def.txt	ツールチェーンの設定
MdePkg/	EDK の中心的ライブラリのパッケージディレクトリ
...Pkg/	その他のパッケージディレクトリ

edksetup.sh は、EDK II のビルドコマンドが動くようにするための準備用スクリプトです。ビルドの前に使うということだけ分かればよく、中身を知る必要はありません。気になる人は中を読んでみてほしいと思いますが。

Conf ディレクトリには、何をビルドするかを設定する target.txt と、ビルドに用いるコンパイラを設定する tools_def.txt を配置します。初期状態ではこの 2 つのファイルは存在しませんが、edksetup.sh を初めて実行したときに、ひな形が生成されます。

HogePkg ディレクトリ^{*2}には各種のプログラムがパッケージ単位で収められています。代表的なパッケージをいくつか紹介します。

MdePkg は、他のプログラムからよく利用される基本ライブラリです。「ライブラリ」は図書館という意味がありますが、プログラミングの世界では「プログラムの部品を集めたもの」を意味します。ライブラリを利用することで、自分で開発するプログラムの量を減らすことができるので、目的のプログラムを早く完成させることができます。MdePkg 以外のパッケージはすべて MdePkg に含まれる部品を利用して作られています。

AppPkg は UEFI アプリケーションをいくつか含んでいます。オリジナルの UEFI アプリケーションを書く際の参考になるでしょう。

OvmfPkg は UEFI BIOS のオープンソース実装である OVMF が収められています。EDK II は UEFI BIOS 上で動くアプリケーションだけではなく、UEFI BIOS そのものを開発するためにも使われるのです。

*1 「EDK」は「EFI Development Kit」に由来すると思われますが、公式ページでは EDK としか書かれていないため、それが正式名称なのでしょう。

*2 「hoge」は「何らかの任意の名前」を表すための意味のない名前です。「HogePkg」と書いたら、それは「AppPkg」とか「MdePkg」とかを表します。このような意味のない名前には他に「fuga」や「piyo」があります。英語圏では「foo」や「bar」を用いられます。これらは、メタ構文変数とも呼ばれます。

2.2 EDK II でハローワールド (osbook_day02a)

EDK II のファイル構成が何となく分かったところで、さっそくアプリケーションを作っていきます。手始めに第 1 章で作った C 言語のハローワールドを EDK II のライブラリを使って書き直してみます。

何はともあれ、このアプリケーションの名前を決めます。今後、このアプリをどんどん拡張して、USB メモリからメインメモリに OS を読み込むのに使う**ブートローダ**に進化させようという思いを込めて「MikanLoader」という名前にしました。本格的に使うには機能が未完成なので「未完」、OS を読み込むので「ローダ」、です。ソースコードは MikanOS リポジトリの osbook_day02a タグです。ファイル構成は次のようになっています。

```
$HOME/workspace/mikanos/MikanLoaderPkg/ (Git タグ: osbook_day02a)
MikanLoaderPkg.dec      パッケージ宣言ファイル
MikanLoaderPkg.dsc      パッケージ記述ファイル
Loader.inf             コンポーネント定義ファイル
Main.c                 ソースコード
```

パッケージ宣言ファイル、パッケージ記述ファイル、コンポーネント定義ファイルは EDK II でプログラムを書く際には必ず作る必要があるファイルですが、内容はある程度コピー＆ペーストで作るようなつまらないものです。全体的な説明は「**付録 C EDK II のファイル説明**」に譲るとして、ここでは特筆すべき部分のみ取り上げて説明します。

リスト 2.1 Loader.inf のエントリポイント設定 (Loader.inf)

```
[Defines]
< 中略 >
ENTRY_POINT                = UefiMain
```

特に見てほしいのが「Loader.inf」の ENTRY_POINT 設定です(リスト 2.1)。ENTRY_POINT には、この UEFI アプリケーションのエントリポイント名を書きます。**エントリポイント**は聞き慣れない言葉かもしれませんが、通常の C/C++ プログラムにおける main() に相当する、起動時に最初に実行される関数のことです。通常の C/C++ プログラムでは main() という名前は固定ですが、EDK II では UEFI アプリケーションごとに自由にエントリポイント名を指定することができます。

リスト 2.2 EDK II を使ったハローワールドアプリ (Main.c)

```
#include <Uefi.h>
#include <Library/UefiLib.h>

EFI_STATUS EFIAPI UefiMain(
    EFI_HANDLE image_handle,
    EFI_SYSTEM_TABLE *system_table) {
    Print(L"Hello, Mikan World!\n");
    while (1);
    return EFI_SUCCESS;
}
```

ハローワールドプログラムの本体はMain.c ファイルです。**リスト 2.2**にその全体を示します。ぱっと見て「第1章 PCの仕組みとハローワールド」で作ったhello.cよりすっきりしていることが分かります。なぜなら、hello.cで書いたものの大部分はEDK IIのライブラリが提供しているため、Main.cでは#includeでそれを取り込むだけで良くなったからです。

#includeとは、指定したファイルの内容をそこに読み込む（インクルードする）ことをコンパイラへ指示します。「#include <Uefi.h>」と書くと「Uefi.h」というファイルの中身をその場所に展開することになります。Uefi.hはEDK IIに含まれるヘッダファイルで、実体は\$HOME/edk2/MdePkg/Include/Uefi.hにあります。

とにかく、2行の#includeを書くことでEFI_STATUSやEFI_SYSTEM_TABLE、Print()などのC言語の標準ではない型や関数が使えるようになるのだ、という理解で十分です。が、興味のある読者のために、それらが使えるようになる仕組みを「コラム：インクルード」で少し詳しく説明します。

Print()関数はC言語標準のprintf()と似た、文字列の表示関数です。ハローワールドの例には登場しませんが%dや%sなどを使った書式指定も可能です。printf()と違うのは引数にワイド文字を渡さなければならぬところです。文字列の前にLと書いてあるのに気づいたでしょうか？ これはワイド文字から構成された文字列を表現するためのやり方です。UEFIで文字表示するにはワイド文字にする、と覚えておけば十分です*3。

さて、ソースコードの中身が分かったので、これをビルドして起動してみましょう。ビルドに先立ちいくつかの設定が必要です。MikanOSのリポジトリでビルドしたいバージョンを呼び出しておくこと、MikanLoaderPkgにシンボリックリンクを張ること、edksetup.shを読み込んでおくことです。順番にやっていきます。

```
$ cd $HOME/workspace/mikanos
$ git checkout osbook_day02a
```

git checkoutは、指定したバージョン（タグ）のソースコードを呼び出し、ファイルとして配置するコマンドです。これで、\$HOME/workspace/mikanos 以下には day02a バージョンのソースコードが配置された状態になります。

```
$ cd $HOME/edk2
$ ln -s $HOME/workspace/mikanos/MikanLoaderPkg ./
```

「ln -s」はシンボリックリンク、Windows でいえばショートカットのようなものを作るコマンドです。このコマンドにより、\$HOME/edk2の中に\$HOME/workspace/mikanos/MikanLoaderPkgを指すシンボリックリンクが作成されます*4。

*3 UEFIでは、表示する文字がUCS2（16ビット幅のUnicode表現）でエンコードされていることを要求します。C言語の規格ではワイド文字はcharと同等かそれ以上のビット幅であるという決まりしかありませんが、UEFIアプリケーションを作るときは、ワイド文字をUCS2で扱ってくれるコンパイラが必要となります。

*4 シンボリックリンクが正しく作成されたかどうかは、\$HOME/edk2の中で「ls -l」を実行すれば確かめられます。


```
$ source edksetup.sh
```

source コマンドで edksetup.sh ファイルを読み込むと、Conf/target.txt ファイルが（存在しない場合に）自動的に生成されます。そして、作ったハローワールドプログラムをビルドするためには、生成された Conf/target.txt で MikanLoaderPkg をビルド対象として指定する必要があります。ここでは、表 2.1 のように設定します。

表 2.1 EDK II で MikanLoaderPkg をビルドする設定

設定項目	設定値
ACTIVE_PLATFORM	MikanLoaderPkg/MikanLoaderPkg.dsc
TARGET	DEBUG
TARGET_ARCH	X64
TOOL_CHAIN_TAG	CLANG38

設定が完了したら、EDK II 付属の build コマンドを用いてビルドします*⁵。

```
$ cd $HOME/edk2
$ build
```

ビルドが完了すると \$HOME/edk2/Build/MikanLoaderX64/DEBUG_CLANG38/X64/Loader.efi に目的のファイルが出力されます。例のごとく、このファイルを USB メモリの /EFI/BOOT/BOOTX64.EFI にコピーして起動させると、画面に「Hello, Mikan World!」と表示されるはずです。

* 5 build コマンドは一見パスが通った普通のコマンドに見えますが、edksetup.sh を読み込むことで使えるようになっています。

コラム 2.1 インクルード

このコラムでは、リスト 2.2 の中で、Uefi.h をインクルードすることにより EFI_STATUS 型が使えるようになる仕組みを説明します。

Uefi.h の中身は、前半のコメント部分を省略すると **リスト 2.3** のようになっています。

リスト 2.3 EDK II の Uefi.h

```
#ifndef __PI_UEFI_H__
#define __PI_UEFI_H__

#include <Uefi/UefiBaseType.h>
#include <Uefi/UefiSpec.h>

#endif
```

この中の #ifndef と #define、最後の #endif の 3 行は、インクルードガードと呼ばれる技法です。同じファイルが何度もインクルードされてしまうことを防ぐためにあります。本書では詳しくは扱いませんが、意図しないコンパイルエラーを防ぐためにヘッダファイルにはインクルードガードを書くものだ、と思ってください。

インクルードガードを除くと 2 行の #include がありますね。#include 命令は、その場所に指定したファイルを読み込んで展開しろ、とコンパイラに指示するものです。例えば 1 行目であれば Uefi/UefiBaseType.h をその場を読み込むことになります。このファイルの実体は \$HOME/edk2/MdePkg/Include/Uefi/UefiBaseType.h にあります。一部を抜き出すと **リスト 2.4** のようになっています。

リスト 2.4 EDK II の UefiBaseType.h

```
#ifndef __UEFI_BASETYPE_H__
#define __UEFI_BASETYPE_H__

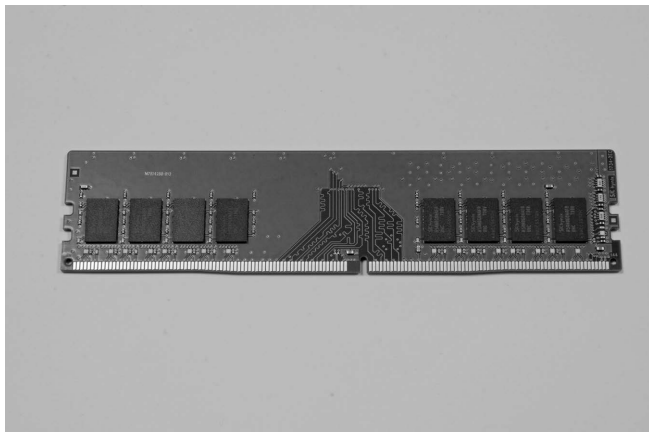
#include <Base.h>
...
typedef RETURN_STATUS          EFI_STATUS;
...
#endif
```

EFI_STATUS という型は、ここで定義されていることが分かりますね。ちなみに RETURN_STATUS は Base.h の中で符号なし整数型として定義されています。ここまでで、「#include <Uefi.h>」をすることで EFI_STATUS 型が使えるようになる一連の流れを見てきました。EFI_SYSTEM_TABLE 型や Print() 関数についても、このようにインクルードの連鎖を丁寧に追っていくと見つけることができますので、興味があったらやってみてください。

2.3 メインメモリ

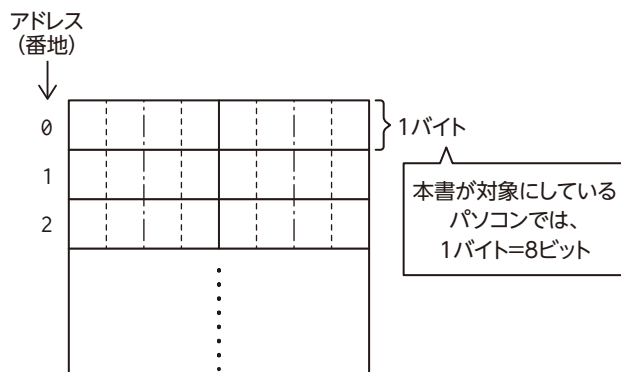
パソコンを構成する大切な要素の1つである**メインメモリ**は、デスクトップPC向けには図2.1のような形の部品として売られています。1枚のメモリ基盤には通常、複数のメモリチップ（写真の中で規則正しく並んでいる四角い黒い部品）が搭載されていて、それらが全体としてまとまって大きな容量を実現しています。ノートPCでは小型化を狙って、メモリチップがマザーボードに直接取り付けられることも多いです。

図 2.1 DDR4-SDRAM と呼ばれるメモリ部品



ハードウェアの視点からは複数のメモリチップから構成されるメインメモリですが、ソフトウェアの視点からは図2.2のように多数のバイトが隙間なく直線状に整列しているかのように見えます。これはCPUに搭載されているメモリコントローラのおかげです。それぞれのバイトには0から始まる連番が振られていて、CPUはこの連番を用いることで、メインメモリのデータをバイト単位で読み書きすることができます。

図 2.2 メインメモリは多数のバイトが並んでいる



各バイトに振られた連番のことを番地やアドレスと呼んだりします。一般には「アドレス」の方がよく使われます。CPU はアドレスを使って機械語命令を読んだり、データを読み書きしたりするのです。

2.4 メモリマップ

道路、建物、地名などが載っている地図 (map) は、不慣れな街で目的の場所に行くのには欠かせない道具です。地図無しで街を歩いてどこに何があるかは分かりません。筆者は、駅に着くとまず案内図をよく見てどっちに歩くかを決めることが良くあります。

コンピュータの世界でもメモリマップと呼ばれる地図があります。メモリマップはメインメモリのどの部分がどんな用途で使われているかが載っている地図です。街の地図は 2 次元の広がりがありますが、メモリマップは表 2.2 のようになっていて、1 次元方向にしか広がっていません。そういう意味で、地図というより数直線と言ったほうが伝わりやすいかもしれません。

表 2.2 メモリマップの例

PhysicalStart	Type	NumberOfPages
0x00000000	EfiBootServicesCode	0x1
0x00001000	EfiConventionalMemory	0x9F
0x00100000	EfiConventionalMemory	0x700
0x00800000	EfiACPIMemoryNVS	0x8
⋮	⋮	⋮

日本では住所は「X 丁目 Y 番地 Z」のような構造になっていることが多いですが、アメリカでは「X 番地 Y 通り」といった感じの住所が一般的です。例えばサンタクララにあるインテルミュージアム^{*6}の住所は「2200 Mission College Blvd」ですが、この中の 2200 が番地、「Mission College Blvd」が道路の名前です。日本式の住所は数値の規則が分かりづらいですが、アメリカ式の住所ではその道路に沿って連続的に番号が振られているため、道路さえ発見できればすぐに目的の住所にたどり着けます。

どうしてアメリカの話をしているかというと、メモリマップはアメリカ式の住所の付け方をしているからです。メモリマップにおいて、住所 (英語で Address) に当たる部分は表の「PhysicalStart」列にある数値です。もし世界に道路が 1 本しかない街があって、その街の地図をアメリカ人が作ったとすると、きっとそんな感じになると思います。現実の世界との違いは、番地の範囲がとて大きい (32 ビット CPU では 0 から 4 億程度までである) ことでしょうか。

メモリマップの番地、すなわち PhysicalStart 列の数値はバイト単位の値です。表の 1 行目は最も先頭にあるメモリ領域のことを、表の 2 行目は 0x00001000=4096 バイト目から存在するメモリ領域のことを意味しているのです。Type 列はその領域が何に使われているか (あるいは、使われていない空き領域か) を示していて、それぞれの意味は表 2.3 の通りです。

*6 インテル CPU の歴史が分かる博物館。シリコンインゴットの実物や 4004 のマスクが展示してあったりして、CPU が好きならとても楽しめます。

表 2.3 メモリマップの例

Type 値	Type 名	意味
1	EfiLoaderCode	UEFI アプリケーションの実行コード
2	EfiLoaderData	UEFI アプリケーションが使うデータ領域
3	EfiBootServicesCode	ブートサービスドライバの実行コード
4	EfiBootServicesData	ブートサービスドライバが使うデータ領域
7	EfiConventionalMemory	空き領域

現実の街では建物の大きさに差があるように、それぞれのメモリ領域の大きさもばらばらです。メモリマップの中の `NumberOfPages` はメモリ領域の大きさをページ単位で表した数値です。1 ページの大きさは文脈によって違いますが^{*7}、UEFI のメモリマップにおける 1 ページの大きさは 4KiB^{*8} です。UEFI のメモリマップでは、メモリ領域の大きさはすべてページ単位で表せるのに、なぜか番地はバイト単位で記されています。まったくチグハグですよね。どちらかに統一してほしいものです。

上記のメモリマップの例ではそれぞれのメモリ領域が隣接していますが、現実には歯抜けがあり得ます。すなわち、`PhysicalStart` に `NumberOfPages` × 4KiB を足しても次の行の `PhysicalStart` にならない場合がある、ということです。メモリマップを読み取るプログラムを作るときはこのことに注意が必要です。

さて、OS が正しく動くためにはメインメモリの様子をきちんと把握しなければなりません。例えば CPU の設定情報が書かれたメモリ領域を上書きしてしまうと CPU が誤動作するかもしれません。UEFI が使用中のメモリを破壊するのも避けねばなりません。メモリを使うなら空き地を探す必要があるのです。

そこで、OS 作りに先立って、UEFI の機能を使ってメモリマップを取得するプログラムを作ります。最終的には、取得したメモリマップは OS に渡すこととなりますが、今はとりあえずファイルに保存することを目標にします。

2.5 メモリマップの取得 (osbook_day02b)

UEFI の機能を使ってメモリマップを取得するプログラムは `$HOME/workspace/mikanos/MikanLoaderPkg/Main.c` (Git タグ: `osbook_day02b`) にあります。リスト 2.5 に `Main.c` の中で実際にメモリマップを取得している部分を示します。

*7 例えば x86_64 アーキテクチャの CPU では、ページングという機能があり、ページサイズは 4KiB、2MiB、4MiB、1GiB のいずれかに設定できます。

*8 KiB はキビバイトと読む。1KiB は 1024 バイト。コンピュータの分野では 2 の 10 乗 = 1024 を単位として数えることが多いため「キロ」では 1000 倍なのか 1024 倍なのか紛らわしくなります。「キビ」といえば 1024 倍を表すことが明確になります。さらに 1024 倍していくと MiB (メビバイト)、GiB (ギビバイト)、TiB (テビバイト) となります。

リスト 2.5 GetMemoryMap() はメモリマップを取得する (Main.c)

```
EFI_STATUS GetMemoryMap(struct MemoryMap* map) {
    if (map->buffer == NULL) {
        return EFI_BUFFER_TOO_SMALL;
    }

    map->map_size = map->buffer_size;
    return gBS->GetMemoryMap(
        &map->map_size,
        (EFI_MEMORY_DESCRIPTOR*)map->buffer,
        &map->map_key,
        &map->descriptor_size,
        &map->descriptor_version);
}
```

UEFI は大きく分けて、OS を起動するために必要な機能を提供するブートサービスと、OS 起動前、起動後どちらでも使える機能を提供するランタイムサービスから構成されています。メモリ管理関連の機能はブートサービスに含まれるので、ブートサービスを表すグローバル変数 `gBS`^{*9} を使用しています。ランタイムサービスに含まれる機能を使う場合は `gRT` というグローバル変数を使いますが、本書には登場しません。

UEFI の規格書 (参考文献 [2]) によると `gBS->GetMemoryMap()` の引数と戻り値の型 (関数プロトタイプ) はリスト 2.6 の通りです。

リスト 2.6 gBS->GetMemoryMap() の仕様

```
EFI_STATUS GetMemoryMap(
    IN OUT UINTN *MemoryMapSize,
    IN OUT EFI_MEMORY_DESCRIPTOR *MemoryMap,
    OUT UINTN *MapKey,
    OUT UINTN *DescriptorSize,
    OUT UINT32 *DescriptorVersion);
```

引数リストにある IN や OUT は C 言語としては変な書き方です。これらは EDK II 独自のマクロで、引数がどのように使われるかをプログラマに伝えるためだけにあります。コンパイル時に空文字列に置き換えられるため、コンパイル結果には何も影響を与えません。IN、OUT はそれぞれ関数への入力用、関数からの出力用であることを意味します。入力用引数には関数を呼び出す前に有効な値を書いておく必要があります。出力用引数は、関数がある内容を書き換えます。値を書き換えるためにポインタで渡すことになっています^{*10}。IN OUT の指定は、その引数の値が入力値として使用されたあと、出力にも使われることを表します。

`gBS->GetMemoryMap()` は、関数呼び出し時点のメモリマップを取得し、引数 `MemoryMap` で指定されたメモリ領域に書き込みます。正常にメモリマップが取得できると `EFI_SUCCESS` を返します。メモリ領域が小さくてメモリマップすべてを収めることができない場合は `EFI_BUFFER_TOO_SMALL` を返します。その他のエラーを返すこともあります。

1 番目の引数 `MemoryMapSize` は、入力としてはメモリマップ書き込み用のメモリ領域の大きさ（バイト数）を設定します。出力としては実際のメモリマップの大きさが設定されます。

2 番目の引数 `MemoryMap` は、メモリマップ書き込み用のメモリ領域の先頭ポインタを設定します。メモリ領域の先頭ポインタを入力するという意味で `IN` が、メモリマップが書き込まれるという意味で `OUT` が指定されています。書き込まれるメモリマップのデータ構造は後ほど詳しく紹介します。

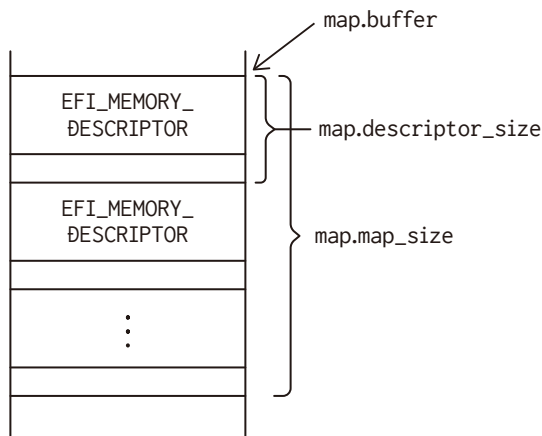
3 番目の引数 `MapKey` は、メモリマップを識別するための値を書き込む変数を指定します。メモリマップはプログラムの処理や UEFI 自体の処理によって変化していきます。2 つの時点で取得した `MapKey` の値が同じなら、その時間内ではメモリマップに変化が無いということを意味します。この値は後で `gBS->ExitBootServices()` を呼び出すときに必要になります。

4 番目の引数 `DescriptorSize` は、メモリマップの個々の行を表すメモリディスクリプタのバイト数を表します。メモリディスクリプタの大きさは `sizeof(EFI_MEMORY_DESCRIPTOR)` で計算できるのではと思うかもしれませんが、UEFI の実装によっては構造体が拡張されていたりしてバイト数が異なる可能性があるので、ここで取得した値を使うのが正確です。

5 番目の引数 `DescriptorVersion` は、メモリディスクリプタの構造体のバージョン番号を表します。本書ではこの値は使いません。

`MemoryMap` で指し示されるメモリ領域に書き込まれるデータの構造は、図 2.3 に示すとおり `EFI_MEMORY_DESCRIPTOR` 構造体の配列となっています。メモリマップ全体の大きさは `MemoryMapSize` バイトで、要素は `DescriptorSize` バイト間隔で配置されます。

図 2.3 メモリマップのデータ構造



- * 9 EDK II では、グローバル変数の名前には先頭に `g` を付けることになっているようです。
- * 10 C 言語では戻り値は 1 つしか指定できないので、複数の値を関数から返したいときにはポインタを使うのが常套手段です。EDK II の多くの関数は戻り値が `EFI_STATUS` であるため、その他に出力がある場合は、ポインタ引数を使うことになります。

EFI_MEMORY_DESCRIPTOR 構造体の定義を表 2.4 に示します。この定義は UEFI の規格書 (参考文献[2]) から持ってきたものです。EDK II の MdePkg/Include/Uefi/UefiSpec.h というヘッダファイルの中に、規格に沿って書かれた構造体定義があります。

表 2.4 メモリディスクリプタの構造

フィールド名	型	説明
Type	UINT32	メモリ領域の種別
PhysicalStart	EFI_PHYSICAL_ADDRESS	メモリ領域先頭の物理メモリアドレス
VirtualStart	EFI_VIRTUAL_ADDRESS	メモリ領域先頭の仮想メモリアドレス
NumberOfPages	UINT64	メモリ領域の大きさ (4KiB ページ単位)
Attribute	UINT64	メモリ領域が使える用途を示すビット集合

さて、ここまでで UEFI の規格で定められた gBS->GetMemoryMap() とメモリディスクリプタの仕様を説明しました。これらに加え、サンプルプログラムでは MemoryMap という構造体を独自に定義して使っています。この構造体の定義はリスト 2.7 のようになっていて、メモリディスクリプタを書き込むためのバッファ全体のサイズや、gBS->GetMemoryMap() で取得したディスクリプタサイズなどを記録できるようになっています。

リスト 2.7 メモリマップ構造体 (Main.c)

```
struct MemoryMap {
    UINTN buffer_size;
    VOID* buffer;
    UINTN map_size;
    UINTN map_key;
    UINTN descriptor_size;
    UINT32 descriptor_version;
};
```

2.6 メモリマップのファイルへの保存

GetMemoryMap() で取得したメモリマップをファイルへ保存するプログラムは UefiMain() にあります。該当部分を抜き出してリスト 2.8 に示します。

リスト 2.8 メイン関数の更新部分 (Main.c)

```
CHAR8 memmap_buf[4096 * 4];
struct MemoryMap memmap = {sizeof(memmap_buf), memmap_buf, 0, 0, 0, 0};
GetMemoryMap(&memmap);

EFI_FILE_PROTOCOL* root_dir;
OpenRootDir(image_handle, &root_dir);

EFI_FILE_PROTOCOL* memmap_file;
```



```

root_dir->Open(
    root_dir, &memmap_file, L"\\memmap",
    EFI_FILE_MODE_READ | EFI_FILE_MODE_WRITE | EFI_FILE_MODE_CREATE, 0);

SaveMemoryMap(&memmap, memmap_file);
memmap_file->Close(memmap_file);

```

このプログラムに出てくる `OpenRootDir()` や `root_dir->Open()` の2つの関数は深くは説明しませんが、簡単に言えば書き込み先のファイルを開く関数です。この2つの関数を実行すると、`memmap` というファイルが書き込みモードで開かれます（存在しなければ新規作成されます）。開いたファイルを `SaveMemoryMap()` に渡し、そこへ先ほど取得したメモリマップを保存します。

`OpenRootDir()` と `root_dir->Open()` では、「ポインタのポインタ」というテクニックが使われています。具体的には `&root_dir` や `&memmap_file` のように、ポインタ変数に対してアドレス演算子 `&` を適用することです。UEFI アプリを作る際に多用されるやり方ですから「**コラム2.2：ポインタのポインタ**」で詳しく説明します。

メモリマップはかなり大きくなることがあります。筆者の環境では8KiBで足りないことがありましたので、`memmap_buf` は16KiBにしてあります。読者の皆さんの環境でうまくいかないことがあったら、もっと余裕のある大きさに増やしてみるといいかもしれません。

リスト 2.9 SaveMemoryMap() はメモリマップをファイルに保存する (Main.c)

```

EFI_STATUS SaveMemoryMap(struct MemoryMap* map, EFI_FILE_PROTOCOL* file) {
    CHAR8 buf[256];
    UINTN len;

    CHAR8* header =
        "Index, Type, Type(name), PhysicalStart, NumberOfPages, Attribute\n";
    len = AsciiStrLen(header);
    file->Write(file, &len, header);

    Print(L"map->buffer = %08lx, map->map_size = %08lx\n",
        map->buffer, map->map_size);

    EFI_PHYSICAL_ADDRESS iter;
    int i;
    for (iter = (EFI_PHYSICAL_ADDRESS)map->buffer, i = 0;
        iter < (EFI_PHYSICAL_ADDRESS)map->buffer + map->map_size;
        iter += map->descriptor_size, i++) {
        EFI_MEMORY_DESCRIPTOR* desc = (EFI_MEMORY_DESCRIPTOR*)iter;
        len = AsciiSPrint(
            buf, sizeof(buf),
            "%u, %x, %-1s, %08lx, %lx, %lx\n",
            i, desc->Type, GetMemoryTypeUnicode(desc->Type),
            desc->PhysicalStart, desc->NumberOfPages,
            desc->Attribute & 0xffffflu);
        file->Write(file, &len, buf);
    }

    return EFI_SUCCESS;
}

```

SaveMemoryMap() の実装を **リスト 2.9** に示します。この関数は、引数で与えられたメモリマップ情報を CSV^{*11} 形式でファイルに書き出します。処理の前半にある file->Write() によってヘッダ行を出力します。これをしておくと、CSV ファイルを開いたときに列の意味が分かりやすくなります（いちいちソースコードを読んで「ああ、この列はこの値だったな」と思い出す必要がなくなります）。後半にある for 文では、メモリマップの各行をカンマ区切りにして出力しています。

この for 文を少し詳しく説明します。繰り返し処理に関係する変数は 2 つ、i と iter です。i はメモリマップの行番号を表すカウンタです。iter はメモリマップの各要素（メモリディスクリプタ）のアドレスを表します。

iter は Iterator（イテレータ）という英単語の省略形のつもりです。イテレータは、配列やリストのような複数の要素を持つデータ構造で、要素 1 つずつに対して処理をするときに、各要素を指し示すものです。日本語では反復子と呼ぶこともありますが、イテレータという呼び方の方が一般的です。iter の初期値はメモリマップの先頭、すなわち先頭のメモリディスクリプタを指していて、更新式によって隣接するメモリディスクリプタを次々と指します。

リスト 2.10 イテレータ変数のキャスト

```
EFI_MEMORY_DESCRIPTOR* desc = (EFI_MEMORY_DESCRIPTOR*)iter;
```

for 文の中身で最初に行っているのは整数からポインタへのキャスト（型変換）です（**リスト 2.10**）。整数型である iter を、ポインタ型 EFI_MEMORY_DESCRIPTOR* へと変換しています。整数をポインタにキャストすると、その整数をアドレスとするポインタに変換されます。ポインタまわりは難しいところですから分からなくても心配いりません。「**2.8 ポインタ入門 (1)：アドレスとポインタ**」を読んでみてください。

リスト 2.11 メモリディスクリプタの内容を文字列に変換

```
len = AsciiSprintf(
    buf, sizeof(buf),
    "%u, %x, %-1s, %08lx, %lx, %lx\n",
    i, desc->Type, GetMemoryTypeUnicode(desc->Type),
    desc->PhysicalStart, desc->NumberOfPages,
    desc->Attribute & 0xffffflu);
```

キャストした desc を利用して、メモリディスクリプタの値を文字列に変換します（**リスト 2.11**）。AsciiSprintf() は EDK II に用意されているライブラリ関数で、指定した char 配列に整形した文字列を書き込んでくれます。C 言語標準関数の sprintf() とほぼ同様の使い方です。GetMemoryTypeUnicode() は Main.c で定義している関数で、メモリディスクリプタのタイプ値からタイプ名を取得して返します。

* 11 Comma Separated Values。表形式のデータを表す方法の 1 つで、カンマ (,) を使って 1 行の中で値を区切ります。

リスト 2.12 文字列をファイルに書き出し

```
file->Write(file, &len, buf);
```

最後に EFI_FILE_PROTOCOL が提供する Write() を使って、ファイルに文字列を書き出します。Write() の第2引数は、入力としては文字列のバイト数を指定します。出力としては実際にファイルへ書き出されたバイト数が返されます。本当は、指定した文字列がすべて出力しきれなかった場合に残りを出力するようなプログラムを書くべきですが、今回は省略しています。

2.7 メモリマップの確認

実際にメモリマップ取得プログラムを動かし、メモリマップを確認してみます。まずは MikanLoaderPkg をビルドします。

```
$ cd $HOME/workspace/mikanos
$ git checkout osbook_day02b
$ cd $HOME/edk2
$ source edksetup.sh
$ build
```

ハローワールドプログラムのビルドとやり方は同じですね。ビルドしてできた Loader.efi ファイルを USB メモリにコピーして実機で実行することもできますが、まずは QEMU での方法を紹介します。次のコマンドを実行すると、QEMU で Loader.efi を起動できます。

```
$ $HOME/osbook/devenv/run_qemu.sh Build/MikanLoaderX64/DEBUG_CLANG38/X64/Loader.efi
```

run_qemu.sh スクリプトを使って QEMU を起動させると、カレントディレクトリに disk.img というファイルが作られます。これは USB メモリの内容を1つのファイルにしたディスクイメージと呼ばれるファイルで、マウント^{*12}することで中身を見ることができます。

```
$ mkdir -p mnt # -p を付けると既に mnt があってもエラーにならない
$ sudo mount -o loop disk.img mnt
$ ls mnt
```

ls mnt するとディスクイメージの中身が表示されるはずですが、EFI ディレクトリの他に memmap というファイルがあるはずですが、これは Loader.efi によりファイルに保存されたメモリマップファイルです。中身を見てみましょう。

```
$ cat mnt/memmap
```

* 12 マウント (mount) は Linux の基本操作の1つで、ファイルシステムを開いて使えるようにする操作です。USB メモリや CD-ROM、ネットワークドライブなど、外部記憶装置を Linux で使うときに良くする操作ですが、今回のようにディスクイメージを開いて読み書きするような用途にも使えます。

cat はファイルの中身をターミナルに表示するコマンドです。memmap は CSV 形式のファイルなので表計算ソフトで開くと綺麗に見えますが、簡単に確認するには cat の方が手軽だと思います。上記のコマンドを実行すると、メモリマップが何行も表示されると思います。一通り確認が済んだら、ディスクイメージをアンマウントしておきましょう。

```
$ sudo umount mnt
```

QEMU を使わず、試験用パソコンで実験することもできます。ハローワールドのときと同じように、Loader.efi を USB メモリにコピーして試験用パソコンに挿して一度起動させます。その後 USB メモリを開発用パソコンに差し替えて中を見てみると memmap というファイルが保存されているはずです。これを cat コマンドや表計算ソフトで開いてみると、試験用パソコンのメモリマップを見ることができます。

2.8 ポインタ入門 (1) : アドレスとポインタ

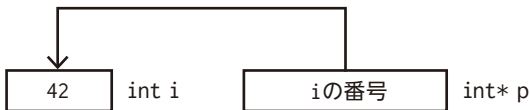
ポインタ (Pointer) は、指す人とか指し棒とかの意味を持つ英単語です。獲物の位置を人間に知らせる犬種の名前でもあります。「マウスポインタ」は画面上の 1 点を指す矢印ですよね。これが C++ の文脈になると「変数を指す物」という意味になります。

以下は、ポインタの基本となる例です。

```
int i = 42;
int* p = &i;
```

1 行目で int 型変数を定義し、初期値を 42 とします。2 行目でポインタ変数 p を定義し、初期値を i へのポインタとします。これで p は i を指すポインタ変数となります。

図 2.4 ポインタ変数 p が変数 i を指す様子



これ以降、p を用いて i の値を読んだり、変更したりできます。上記の 2 行に続けて下記のコードを実行すると、r1 の値は 42、r2 の値は 1 となります。

```
int r1 = *p;
*p = 1;
int r2 = i;
```

「*p = 1;」の * は、ポインタの指す先を取ってくる (参照する) ための演算子です。これを間接演算子と呼ぶのでした。この行の時点では p は i を指していますから、*p は i と等しくなります。

C++ の規格では、ポインタをどのように実現するかをほとんど規定していません。ただ、ポインタと整数は相互に変換可能である、ということは決まっています。そして、異なる変数へのポインタを整数に変換したものは、互いに異なる整数になる必要があります。言い換えると、ポインタはそれぞれの変数に割り振られた固有の番号を内包します。ポインタとは、変数固有の番号と型情報を組み合わせたものと言えます。

```
uintptr_t addr = reinterpret_cast<uintptr_t>(p);
int* q = reinterpret_cast<int*>(addr);
```

この1行目は、ポインタから整数への変換です。あらゆるポインタの精度を失わずに変換が可能な整数型として `uintptr_t` (あるいは `intptr_t`) が標準で用意されていますので、それを使います。ポインタから整数への変換は、ポインタから型情報を削除することに対応します。変換して得られた値 `addr` は、変数 `i` に割り振られた固有の番号となります。

2行目は整数からポインタへの変換です。この変換は変数の番号に変数の型情報を加えることに対応します。結果として得られる `q` は、変数 `i` を指すポインタとなります。

変数に割り振る固有の番号を具体的にどんな値にするかは C++ の規格で規定されておらず、処理系に任されています。本書で使う Clang を含む多くのコンパイラは、変数が配置されたメモリアドレスを使います。メモリアドレスはバイト単位の連番になっていますので、変数固有の番号でなければならないという制約を満たします。

図 2.5 x86-64 アーキテクチャにおけるポインタの具体例

アドレス	メインメモリ	
0xffc8		
	0xffec	int* q
0xffd0		
	0xffec	uintptr_t addr
0xffd8	1	int r2
0xffdc	42	int r1
0xffe0		
	0xffec	int* p
0xffe8		
0xffec		
	1	int i
0xffff0		

上記のプログラムを x86-64 アーキテクチャにおいて実行した結果の例を図 2.5 に示します。これは 1 つの例にすぎませんが、ポインタがどのように実装されるかの理解には役立ちます。

図の左側に示したメモリアドレスの絶対値に意味はなく、相対的な値にこそ意味があります。0xffec は 0xffff0 から 4 だけずれた場所にあるなあ、とかそういうことです。

それから、この図に登場する変数の個数や配置は最適化を行わないでコンパイルしたときに実現されるであろう配置を示しました。最適化を有効にしてコンパイルすると異なる変数の配置になるかもしれませんし、そもそもいずれの変数も生成されないかもしれません。

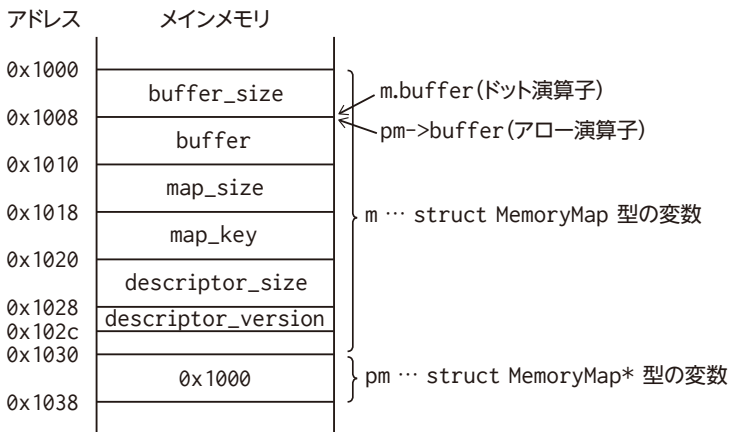
2.9 ポインタとアロー演算子

C 言語の構造体は、メンバが宣言順に並んだメモリ構造となります。例えば MemoryMap 構造体のメンバ `buffer_size` は構造体の最も先頭に、メンバ `buffer` はその直後に配置されます。構造体の先頭からメンバ先頭までの距離（バイト数）のことをオフセットと呼びます。`buffer_size` のオフセットは 0 バイト、`buffer` のオフセットは 8 バイト（64 ビット環境では `UINTN` は 8 バイトです）ということになります。

```
struct MemoryMap m;
struct MemoryMap* pm = &m;
```

構造体型の変数を定義すると、メモリ上には構造体の大きさ分のメモリ領域が確保されます。上記の例では `m` という名前の変数を 1 つ定義しており、この変数は `sizeof(struct MemoryMap)` 分の大きさのメモリ領域を持つことになります。また、ポインタ型の変数 `pm` も定義しています。これら 2 つの変数の関係を図に表すと次のようになります（図 2.6）。この図は、ポインタがメモリアドレスを用いて実現されている前提で描きました。

図 2.6 2 つの変数 `m` と `pm` の関係



先ほどの例と同様に、ポインタは変数固有の番号と型情報を合わせたものとなっています。ここで注目したいのは、指す先の型が `int`（4 バイト）だろうと `MemoryMap`（48 バイト）だろうと、ポインタ変数は同じ大きさ（8 バイト）だということです。これは、ポインタ変数はあくまでも変数のアドレスを保持するものだからです。アドレスを表現するのに要する大きさは、指す先の変数の型に依らず一定です。

`pm` を介して `m` の各メンバを読み書きするには「`->`」（アロー演算子）を使うか、「`*`」（間接演算子）と「`.`」（ドット演算子）を使う方法があります。`m.map_size` に 0 を代入する例を示します。

```
pm->map_size = 0;
(*pm).map_size = 0;
```

上記の2行はまったく同じ処理になります。でもアロー演算子を使った方が楽に書けますから、構造化型のポインタを介してメンバにアクセスする際はアロー演算子を使うのが普通です。このように、意味を保ったままより簡単に書けるように用意された文法を糖衣構文と呼んだりします。

コラム 2.2 ポインタのポインタ

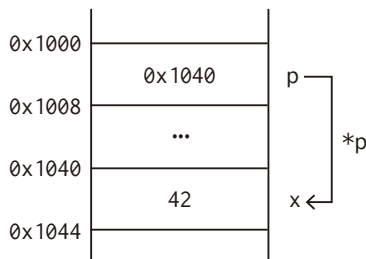
「ポインタのポインタ」は、ポインタ変数にアドレス演算子&を適用することです。UEFI アプリの作成ではよく登場するテクニックです。変数がメモリ上にどのように配置されるかを知ることによってポインタのポインタを理解できます。このコラムに登場する数値やメモリ配置は特定のCPUアーキテクチャやコンパイラに依存した話ですが、具体的なイメージを持つことで一般論の理解も捗るでしょう。

まず、普通のポインタについて簡単に復習します。ポインタは、変数の値を呼び出した関数の中から書き換えるときに使うことができます。

```
void f(int* p) {
    *p = 42;
}
int g() {
    int x = 1;
    int* p = &x;
    f(p);
    return x;
}
```

このコードにおいて、関数g()を実行すると戻り値は42となります。変数xの初期値は1ですが、関数f()の中で変数xの内容が42に書き換わったからです(図2.7)。これがポインタの効果ですね。

図 2.7 変数とポインタの関連



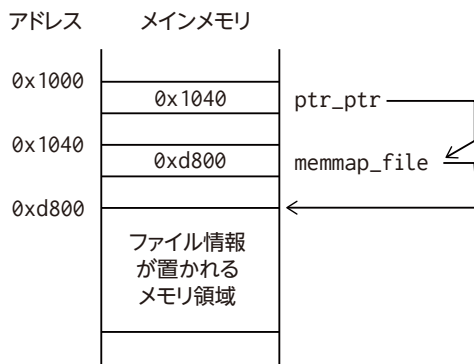
pがxを指す(p points to x)

ポインタのポインタはUEFIプログラミングで頻繁に用いられるテクニックです。本文中ではOpenRootDir()やroot_dir->Open()を使う際に出てきました。後者についてのコードを次に示します(説明のためにポインタのポインタを変数に入れるように改造してあります)。

ポインタのポインタは一見難しいように思うかもしれませんが、ポインタ変数は「アドレスという整数値を格納するための変数だ」ということが分ると理解しやすいでしょう。ポインタ変数 `memmap_file` と、そのポインタ `ptr_ptr` の関係を図で表すと図 2.8 のようになります。

```
EFI_FILE_PROTOCOL* memmap_file;
EFI_FILE_PROTOCOL** ptr_ptr = &memmap_file;
root_dir->Open(
    root_dir, ptr_ptr, L"\\memmap",
    EFI_FILE_MODE_READ | EFI_FILE_MODE_WRITE | EFI_FILE_MODE_CREATE, 0);
```

図 2.8 ポインタのポインタと普通のポインタの関係



`root_dir->Open()` に `memmap_file` ではなく `ptr_ptr` を渡しているのは、`root_dir->Open()` が `memmap_file` の中身 (0xd800) を書き換えられるようにするためです。`root_dir->Open()` は指定されたファイルを開き、そのファイルに関する情報へのポインタを返す関数です。本来なら「`memmap_file = root_dir->Open(...);`」などと、戻り値としてポインタを返したいところですが、戻り値は `EFI_STATUS` を表すために使われてしまっています。仕方なく引数としてポインタのポインタを受け取って、それが指す先の変数を書き換えることによってファイル情報へのポインタを関数の呼び出し元に返しているわけです。

複数の戻り値を持つ関数を作る場合、ポインタを引数で受け取るのは C 言語において常套手段です。ポインタのポインタは、戻り値の型がポインタだった場合に仕方なく出現してしまうだけであり、普通のポインタと考え方は一緒だ、ということが分かっていただけたでしょうか。

第 3 章

画面表示の練習と ブートローダ

EDK II での開発に慣れたので、いよいよ OS 本体を起動するためのブートローダの開発を始めます。この章では画面全体を塗りつぶすだけの簡単な OS 本体を作り、それをブートローダから起動するまでをやってみます。

3.1 QEMU モニタ

作ったプログラムに不具合があったら皆さんはどうやってデバッグしますか？ まずはソースコードの適当な場所に `Print()` の呼び出しを追加して変数の値を確認してみたり、ファイルに値を書き出してみたりしてデバッグすると思います。

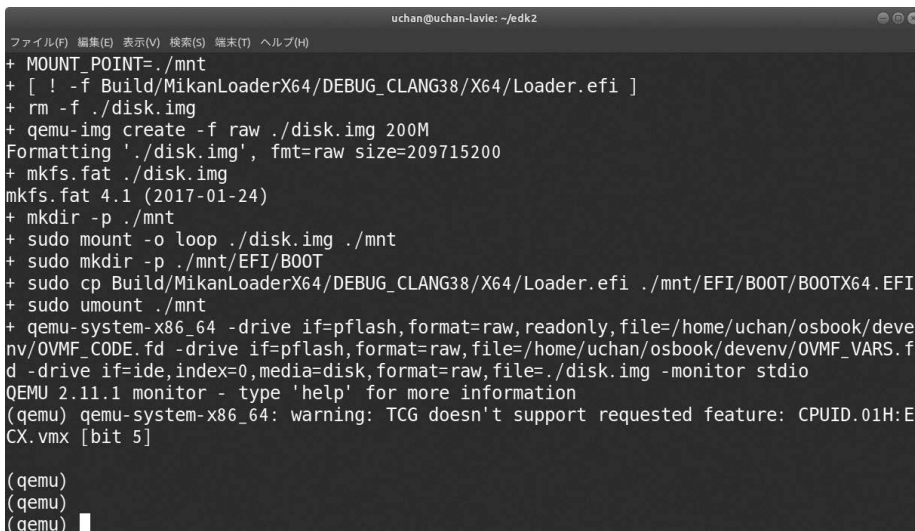
それらはぜひやるべきですが、ときにはその方法だと不具合を特定できなかったり、デバッグコードを仕込むことが影響して不具合が再現しなかったり症状が変わることがあります。OS 上で動くアプリケーションと比べて、ブートローダや OS の開発中はそのような場合が多いです。

例えばメモリ破壊を起こしていたり、CPU の設定が意図したものになっていなかったりすると、ちょっとプログラムが書き換わるだけで症状が変化することがよくあります。「`Print()` 呼び出しをコメントアウトするとなぜか動く」みたいな不思議なことが、低レイヤの開発にはつきものなのです。

そこで、ブートローダ作りを始める前に QEMU モニタを使ってデバッグする方法を紹介したいと思います。QEMU モニタは QEMU の標準機能で、CPU の設定を表示したりメモリの中身を読み書きしたりできます。自作 OS を実機で動かしてしまうとそういうことができませんが、エミュレータなら簡単です。メモリマップ取得プログラムを例にやり方を紹介します。

QEMU モニタを使うには、QEMU が起動した後に `run_qemu.sh` を実行したターミナルに戻ります。`run_qemu.sh` の中で、QEMU モニタをターミナルから使えるようにしてある^{*1}ので、図 3.1 のような状態になっているはずです。(qemu) が QEMU モニタのプロンプト^{*2}です。

図 3.1 QEMU モニタの初期画面



```

uchan@uchan-lavie: ~/edk2
+ MOUNT_POINT=./mnt
+ [ ! -f Build/MikanLoaderX64/DEBUG_CLANG38/X64/Loader.efi ]
+ rm -f ./disk.img
+ qemu-img create -f raw ./disk.img 200M
Formatting './disk.img', fmt=raw size=209715200
+ mkfs.fat ./disk.img
mkfs.fat 4.1 (2017-01-24)
+ mkdir -p ./mnt
+ sudo mount -o loop ./disk.img ./mnt
+ sudo mkdir -p ./mnt/EFI/BOOT
+ sudo cp Build/MikanLoaderX64/DEBUG_CLANG38/X64/Loader.efi ./mnt/EFI/BOOT/BOOTX64.EFI
+ sudo umount ./mnt
+ qemu-system-x86_64 -drive if=pflash,format=raw,readonly,file=/home/uchan/osbook/deve
nv/OVMF_CODE.fd -drive if=pflash,format=raw,file=/home/uchan/osbook/devenv/OVMF_VARS.f
d -drive if=ide,index=0,media=disk,format=raw,file=./disk.img -monitor stdio
QEMU 2.11.1 monitor - type 'help' for more information
(qemu) qemu-system-x86_64: warning: TCG doesn't support requested feature: CPUID.01H:E
CX.vmx [bit 5]

(qemu)
(qemu)
(qemu)
  
```

まずは CPU のレジスタ値を確認してみます。QEMU モニタで `info registers` コマンドを実行します。すると、次のように CPU の各レジスタの現在の値が表示されます。

- *1 QEMU の起動オプションに指定した「`-monitor stdio`」の効果によりターミナルで QEMU モニタが使えます。
- *2 プロンプト (prompt) はユーザからの入力待ちを示す文字列のこと。Bash なら標準は `$`、Windows のコマンドプロンプトなら `>` です。

```
(qemu) info registers
RAX=0000000000000000 RBX=0000000000000001 RCX=0000000007b7b1c0 RDX=0000000000000002
RSI=0000000000000400 RDI=0000000007eab2d0 RBP=000000000000002e RSP=0000000007eaa8a0
R8 =00000000000000af R9 =0000000000000288 R10=0000000000000050 R11=0000000000000000
R12=00000000067ae8d0 R13=000000000000ffff R14=0000000007eaa930 R15=000000000722a920
RIP=00000000067ae4c4 RFL=00000202 [-----] CPL=0 II=0 A20=1 SMM=0 HLT=0
ES =0030 0000000000000000 ffffffff 00cf9300 DPL=0 DS [-WA]
CS =0038 0000000000000000 ffffffff 00af9a00 DPL=0 CS64 [-R-]
```

レジスタ (register) は英語で「登録」という意味があります。CPU のレジスタは、その名の通り値を登録しておく記憶領域のことです。値を記憶しておくという点ではメインメモリと同じですが、レジスタは CPU の内部にあり、メインメモリは CPU の外側にある点が大きく違います。レジスタについては「**3.2 レジスタ**」で詳しく説明しますが、簡単に説明すると上記の出力例にある RAX や ES などはいずれもレジスタです。特に RIP レジスタはデバッグ時に良く使うレジスタで、次に実行される予定の機械語命令の位置を示しています。この情報と、次に紹介するメモリダンプを組み合わせると、現在実行中の機械語命令を確認することができます。

次に、メインメモリの中で指定したアドレス付近の値を表示 (メモリダンプ) してみます。メモリダンプするには x コマンドを使います。コマンドの指定方法は QEMU モニタで help コマンド名で調べられます。x コマンドの書式は次の通りです。

```
x /fmt addr
```

/fmt に指定された書式にしたがって、addr を先頭とするメモリ領域の値を表示します。/fmt は [個数][フォーマット][サイズ] と分解できます。「個数」は何個分表示するかを指定します。「フォーマット」は 16 進数表示の x、10 進数表示の d、機械語命令を逆アセンブルして表示する i のいずれかが指定できます。「サイズ」は何バイトを 1 単位として解釈するかを指定します。1 バイトは b、2 バイトは h、4 バイトは w、8 バイトは g を指定します。試しに、0x067ae4c4 から 4 バイトを 16 進数で表示してみます。

```
(qemu) x /4xb 0x067ae4c4
00000000067ae4c4: 0xeb 0xfe 0x66 0x90
```

おそらくこれは機械語命令だと思われます*³。試しに 2 命令分を逆アセンブルして表示してみましょう。

```
(qemu) x /2i 0x067ae4c4
0x00000000067ae4c4: jmp     0x67ae4c4
0x00000000067ae4c6: xchg   %ax,%ax
```

* 3 次に実行する機械語命令を指す RIP 付近を表示しているというのと、0xeb はジャンプ命令だという筆者の知識によって、これらが機械語命令だと推測しました。

`jmp 0x67ae4c4` は「`0x67ae4c4` にジャンプする」という命令ですが、`0x67ae4c4` はその命令がある場所そのものですから、結局は同じ場所をぐるぐると回ることになります。実は、このアセンブリ命令は `while (1);` をコンパイルした結果です。

とても簡単にですが QEMU モニタの使い方を説明しました。もっと詳しい使い方は Wikibooks の QEMU モニタのページ (参考文献 [4]) にあります。QEMU モニタについての知識と、実行ファイル (EFI ファイルや ELF ファイル) についての知識、アセンブリ言語の知識を組み合わせると、深く、効率よくデバッグすることができます (この本ではアセンブリ言語の話はあまり登場しませんが……)。

3.2 レジスタ

ここでは CPU に内蔵された記憶領域であるレジスタについて、少し詳しく説明します。CPU には一般に、汎用レジスタと特殊レジスタが搭載されています。汎用レジスタは一般の演算に使えるレジスタです。一方で特殊レジスタには CPU の設定を行うためのものやタイマなどの CPU に内蔵された機能を制御するためのものがあります。

汎用レジスタの主目的は値を記憶することです。その点で CPU の外部にあるメインメモリと役割は似ています。しかし、容量と読み書き速度の点で大きく異なります。メインメモリと対照的に、レジスタは容量が小さく読み書きが速いのが特徴です。メインメモリは例えば 16GB (2 の 34 乗バイト) 程度あるのに対し、x86-64 アーキテクチャの汎用レジスタは 128B (2 の 7 乗バイト) しかありません。メインメモリ (DDR4-SDRAM) は読み書きにだいたい 100 ナノ秒くらい要しますが、レジスタは待ち時間なく読み書き可能です。2GHz で動く CPU であれば 0.5 ナノ秒程度ということになります。

x86-64 の汎用レジスタは次の 16 個です。RAX、RBX、RCX、RDX、RBP、RSI、RDI、RSP、R8 ~ R15。これらの汎用レジスタは CPU の演算対象に指定できます。例えば加算命令 `add` には次のように 2 つのレジスタを指定できます。

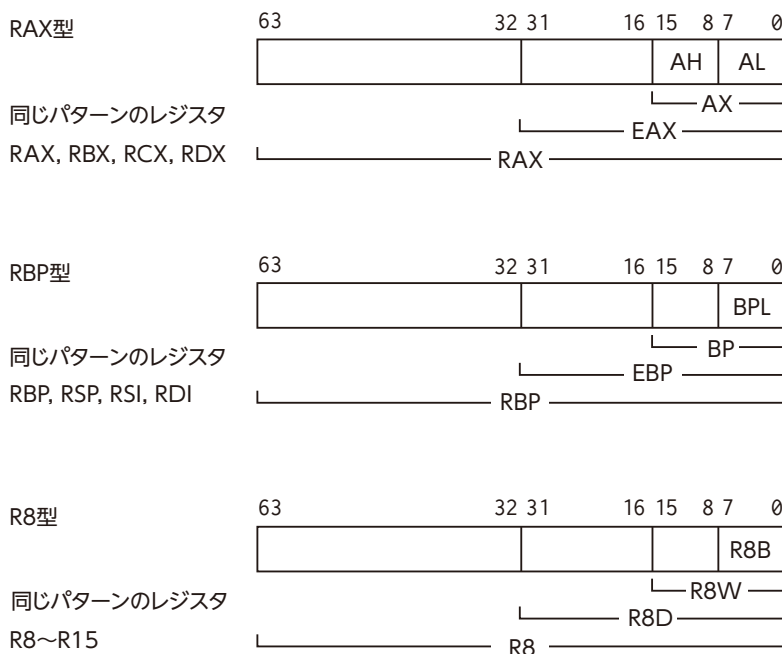
```
add rax, rbx
; オペコード オペランド 1, オペランド 2
```

一般に x86-64 の演算命令は 2 つのオペランド (引数) を取り、左側が書き込み先、右側が読み込み元となります。上記の命令では RAX に RBX の値を加えるという意味になります。C++ 風に書けば `rax += rbx;` ということですね。読み書きが高速な汎用レジスタは、CPU が計算を行う際になくはならない存在です。ちなみに、`add` の部分に来るアセンブリ命令の名前のことをオペコードと呼びます。

x86-64 の汎用レジスタはすべて 8 バイト (=64 ビット) です。でも、場合によってはそれより小さなレジスタが欲しい場合があります。C++ で `char` や `uint16_t` といった、8 バイトより小さい型を使う場面は良くありますよね。そのような小さな型の変数 (変数はメインメモリに配置される) をレジスタに読み出したり、演算したり、結果をメモリに書き戻したりしたいわけです。そのために、汎用レジスタの一部を小さなレジスタとしてアクセスできるようになっています (図 3.2)。例えば AX レジスタは RAX レジスタの下位 16 ビットを表す名前になっていて、AX を読み書きすることで RAX の下位 16 ビットを読み書きできます*4。

*4 Intel SDM (参考文献 [3]) では R8B は R8L という名前になっていますが、Nasm アセンブラは R8L という名前を認識できないようです。

図 3.2 汎用レジスタと下位バイトの別名



特殊レジスタは汎用レジスタに比べ、とても多くの種類があります。特殊レジスタは汎用レジスタ同様に値を記憶する機能はもちろんです。汎用レジスタにはない特徴も持ちます。それは、値を読み書きするという操作自体に意味があったり、各ビットの役割が異なる場合があることです。代表的なものをいくつか紹介します。

- **RIP**: CPU が次に実行する命令のメモリアドレスを保持するレジスタです。
- **RFLAGS**: 命令の実行結果によって変化するフラグを集めたレジスタです。
- **CR0**: CPU の重要な設定を集めたレジスタです。

RIP は次に実行する命令のメモリアドレスを保持していて、命令の実行にともなって変化します。IP は Instruction Pointer (命令を指すもの) の略です。演算系の命令を実行した場合は単に次の命令を指すように増えるだけですが、`jmp` や `call` のような分岐命令の場合はオペランドで指定されたアドレスが RIP に書き込まれます。例えば `jmp label` は、`label` で表されるアドレスへジャンプする命令ですが、内部的には `mov rip, label` という処理が行われているイメージです。

RFLAGS はその名の通りフラグを集めたレジスタで、各ビット毎に異なる役割を持ちます。例えばビット 0 がキャリーフラグ (CF)、ビット 6 がゼロフラグ (ZF) です。これらのフラグは様々な命令の実行にともなって変化します。例えば命令の実行結果が 0 になると ZF が 1 になります。加算がオーバーフローすると CF が 1 になります。RIP が CPU によって自動更新されるのと同様、RFLAGS も命令実行にともなって内容が自動更新されるのです。

フラグレジスタの普通の使い方は、演算系の命令や `cmp` などのフラグレジスタに影響を与える命令

の直後に、`jz` や `cmovz` などのフラグレジスタの内容によって動作が変わる命令を配置します。例えば次のプログラムは RAX から 1 を引いた結果が 0 でない間、ぐるぐるとループします。

```
loop:
    dec    rax
    jnz    loop
```

`dec` 命令はレジスタの値を 1 だけ減らします。命令実行の結果、RAX がちょうど 0 になると ZF が 1 になります。直後にある `jnz` は Jump if Not Zero の意味で、ZF が 0 の場合にのみジャンプする、という命令です。RFLAGS レジスタは上記プログラムに明示的には登場しませんが、2 つの命令の内部で暗黙に利用されるのです。

CR0 のビット 0 (PE) に 1 を書くと CPU は保護モードに遷移します。ビット 31 (PG) に 1 を書くとページングが有効になります。

このように、特殊レジスタは単に値を記録するだけでなく、値を書く行為によって何らかの動作が引き起こされる仕掛けになっていることがあります。同様に、値を読むことが動作の引き金になるレジスタも存在します。さらには 1 を書くことによって 0 に初期化されるビットを持つレジスタなどもあり、レジスタの世界は奥が深いのです。

3.3 初めてのカーネル (osbook_day03a)

ブートローダと OS 本体（カーネル）の開発方法はいろいろ考えられますが、この本ではブートローダは UEFI アプリとして、カーネルは ELF バイナリとして別々のファイルとして開発し、ブートローダからカーネルを呼び出す形式にしようと思います。UEFI アプリは UEFI の規格で決められた通りに作らなければなりません。ファイルを分けることで、カーネルはその制約を気にしないで自由に作ることができます。まずカーネルのもととなる小さな実行ファイルを作ります。

記念すべき一番最初のカーネルはどんなものにしましょうか？

ここではとりあえず、**リスト 3.1** に示す通り、何もしないで永久ループするプログラムとします。今の段階ではカーネルから文字表示などができないため、かろうじて動作が分かるものといえば、その場で永久ループするようなプログラムなのです。ここで定義する関数 `KernelMain()` がブートローダから呼び出されるのですが、このような関数のことをエントリポイントと呼びます。

リスト 3.1 一番最初のカーネル (main.cpp)

```
extern "C" void KernelMain() {
    while (1) __asm__("hlt");
}
```

関数定義で特徴的なのは `extern "C"` でしょう。これは C 言語形式で関数を定義することを意味します。C++ の通常関数定義では、引数の個数や型が異なる同名の関数を定義できる^{*5}ように、裏では関数名に引数の情報が結合された名前が付きまします。この変換のことを名前修飾（マングリング）と呼びます。例えば `int foo()` は `_Z3foov` に、`int foo(int, int)` は `_Z3fooui` に変換されます。C 言語のプログラムから C++ で定義した関数を呼ぶには修飾された名前を使う必要がありますが、難し

い名前になるため非現実的です。そんなとき、`extern "C"` を関数定義の先頭に付けて名前修飾を防ぐ^{*6} のが良くあるやり方なのです。

関数 `KernelMain()` が呼び出されると、すぐに永久ループに入ります。永久ループの中には `__asm__("hlt");` という見慣れない文がありますので説明します。`__asm__()` はインラインアセンブラのための記法で、C 言語プログラムの中にアセンブリ言語の命令を埋め込むときに使います。アセンブリ言語は機械語にとっても近い言語で、機械語と基本的には 1 対 1 に対応しており、C 言語では表すことができない命令でもアセンブリ言語なら表現できます。アセンブリ言語でしか表現できない命令をその場所に埋め込みたいときにインラインアセンブラを使うと便利です^{*7}。

今回のプログラムでは `hlt` 命令を埋め込むのに使っています。`hlt` 命令は CPU を停止させる命令で、これを実行すると CPU が省電力な状態になります。`hlt` をせずに永久ループすると CPU が 100% の使用率になり電気を無駄にしますし熱も出ますので、積極的に `hlt` していくことにします。`hlt` により CPU は省電力状態となり動作が止まりますが、割り込みがあると動作が再開します。割り込みとは、例えばキーボードが押されたとか、ネットワークでデータを受信したとか、そういうものです。ただし、現在は何も割り込みの設定をしていませんから割り込みは一切発生せず、CPU はずっと止まったままになります。

このソースコードからカーネルファイルを作るには、次のようにコンパイルとリンクを行います。

```
$ cd $HOME/workspace/mikanos/kernel
$ git checkout osbook_day03a
$ clang++ -O2 -Wall -g --target=x86_64-elf -ffreestanding -mno-red-zone \
  -fno-exceptions -fno-rtti -std=c++17 -c main.cpp
$ ld.lld --entry KernelMain -z norelro --image-base 0x1000000 --static \
  -o kernel.elf main.o
```

1 行目では、`clang++` コマンドでソースコードをコンパイルし、オブジェクトファイル `main.o` を作ります。オブジェクトファイルとは、ソースコードをコンパイルした結果の機械語命令を含んだファイルです。コンパイラにとっては機械語命令を出力することが「オブジェクト (目的)」なわけです。コンパイラに指定したオプションの意味を表 3.1 にて簡単に説明します。

- *5 関数のオーバーロードと呼ばれる機能。引数の個数や型が異なる関数をいくつも定義できます。
- *6 副作用として、引数の個数や型が異なるだけの同名の関数を定義できなくなります。あくまで関数名に関することであり、関数の本体で C++ 固有の機能を使うことは制限されません。
- *7 「30 日のできる! OS 自作入門」(参考文献 [1]) ではインラインアセンブラは使わず、アセンブラで作った関数を C 言語から呼び出す方法を使っています。本書では、筆者の好みでインラインアセンブラを活用することにしました。

表 3.1 コンパイラのオプション

-O2	レベル2の最適化を行う
-Wall	警告をたくさん出す。
-g	デバッグ情報付きでコンパイルする。
--target=x86_64-elf	x86_64 向けの機械語を生成する。出力ファイルの形式を ELF とする。
-ffreestanding	フリースタンディング環境向けにコンパイルする。
-mno-red-zone	Red Zone 機能を無効にする。
-fno-exceptions	C++ の例外機能を使わない。
-fno-rtti	C++ の動的型情報を使わない。
-std=c++17	C++ のバージョンを C++17 とする。
-c	コンパイルのみする。リンクはしない。

`-ffreestanding` はフリースタンディング環境向けにコンパイルを行うための指定です。C++ の動作環境は大きく 2 種類、**ホスト環境** (hosted environment) と **フリースタンディング環境** (freestanding environment) が規定されています。ホスト環境は OS の上で動くプログラムのための環境、フリースタンディング環境は OS がいない環境のことです。OS 自体は OS がいない環境で動くプログラムですから、このオプションを付けます。`-mno-red-zone`、`-fno-exceptions`、`-fno-rtti` は OS を作る時にはとりあえず付けておくというオプション群です。`-mno-red-zone` については「**コラム 3.1 レッドゾーン**」で説明します。その他の 2 つのオプションは、OS からのサポートが必要な C++ の言語機能を無効にするものです。OS を作る時には OS のサポートを必要とする言語機能は使えないのです。

次にリンカ `ld`、`ld.lld` によってオブジェクトファイルから実行可能ファイルを作ります。オブジェクトファイルは機械語命令を含んでいますから、それを CPU がそのまま実行すればいいじゃないかと思うかもしれませんが、それはできません。なぜなら、オブジェクトファイルは他のオブジェクトファイルとリンク (結合) することが前提の中間形式であり、それ単体で実行することが考慮されていないからです。今回はたまたまオブジェクトファイルが 1 個でしたが、今後は増えていく予定です。リンカに指定したオプションの意味を表 3.2 に簡単に記載しました。

表 3.2 リンカのオプション

--entry KernelMain	KernelMain() をエントリポイントとする。
-z norelro	リロケーション情報を読み込み専用にする機能を使わない。
--image-base 0x1000000	出力されたバイナリのベースアドレスを 0x1000000 番地とする。
-o kernel.elf	出力ファイル名を kernel.elf とする。
--static	静的リンクを行う。

さて、コンパイルとリンクを行うと `kernel.elf` というファイルが生成されます。これがカーネルファイルです。おめでとうございます！ とても小さいですが、ブートローダではない、正真正銘の OS 本体を作ったということです。やったね！

喜びもつかの間、このファイルをブートローダから起動させるためには、ブートローダがなんとかしてこのファイルをメインメモリに読み出す必要があります。やり方はいろいろ考えられますが、この本

では、ブートローダの実行ファイル Loader.efi とカーネルの実行ファイル kernel.elf を両方とも USB メモリに書き込んでおき、UEFI の機能を使ってブートローダからカーネルを読み出して起動することにします*⁸。

次はブートローダでカーネルファイルを読み込む部分を作ります。ファイルの読み込みは、ファイルを開き、ファイル全体を格納できる十分なメモリを確保し、ファイルの内容を読み取るという流れになります。ブートローダの UefiMain() でカーネルファイルを読み取る部分を **リスト 3.2** に示します。

リスト 3.2：メイン関数でカーネルファイルを読み込む部分 (Main.c)

```
EFI_FILE_PROTOCOL* kernel_file;
root_dir->Open(
    root_dir, &kernel_file, L"\\kernel.elf",
    EFI_FILE_MODE_READ, 0);

UINTN file_info_size = sizeof(EFI_FILE_INFO) + sizeof(CHAR16) * 12;
UINT8 file_info_buffer[file_info_size];
kernel_file->GetInfo(
    kernel_file, &gEfiFileInfoGuid,
    &file_info_size, file_info_buffer);

EFI_FILE_INFO* file_info = (EFI_FILE_INFO*)file_info_buffer;
UINTN kernel_file_size = file_info->FileSize;

EFI_PHYSICAL_ADDRESS kernel_base_addr = 0x100000;
gBS->AllocatePages(
    AllocateAddress, EfiLoaderData,
    (kernel_file_size + 0xfff) / 0x1000, &kernel_base_addr);
kernel_file->Read(kernel_file, &kernel_file_size, (VOID*)kernel_base_addr);
Print(L"Kernel: 0x%0lx (%lu bytes)\n", kernel_base_addr, kernel_file_size);
```

カーネルファイルを開く処理は、メモリマップを書き込むためのファイルを開くのと似ています。トップディレクトリにある kernel.elf という名前のファイルを読み込み専用 (EFI_FILE_MODE_READ) で開きます。

次に、開いたファイル全体を読み込むためのメモリを確保します。そのためにはファイルの大きさを知る必要がありますので、kernel_file->GetInfo() を使ってファイル情報を取得します。この関数の第4引数には EFI_FILE_INFO 型を十分格納できる大きさのメモリ領域を指定する必要があります。ここでは sizeof(CHAR16)*12 バイトだけ大きめに確保したメモリ領域を指定していますが、その理由を説明します。

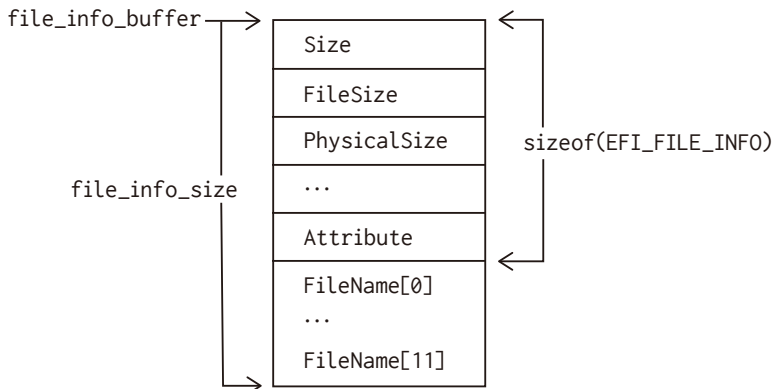
* 8 一方、Linux にはカーネルファイルの先頭部分を細工して EFI ファイルと認識されるようにし、ブートローダなしで起動するための「EFI boot stub」と呼ぶ仕組みがあります。この方法は EFI ファイルのヘッダ構造についての知識が必要となるので、本書では採用しませんでした。

リスト 3.3 ファイル情報の構造体

```
typedef struct {
    UINT64 Size, FileSize, PhysicalSize;
    EFI_TIME CreateTime, LastAccessTime, ModificationTime;
    UINT64 Attribute;
    CHAR16 FileName[];
} EFI_FILE_INFO;
```

EFI_FILE_INFO 型の定義をリスト 3.3 に示します。最後のメンバ FileName の定義に注目すると、配列の要素数が書かれていないことがわかりますね。これは、文字数が変わりうる文字列を扱うための技法の 1 つで、C 言語だとしばしば使われます。要素数を指定しない場合、そのメンバの大きさは 0 と計算されるため、sizeof(EFI_FILE_INFO) はメンバ Attribute までの大きさと等しくなります。図 3.3 のようなメモリレイアウトとなります。

図 3.3 EFI_FILE_INFO のメモリレイアウト



FileName の要素数を省略していることで、file_info_buffer のバイト数を増減させることにより任意の文字数を格納させられるのです。今回は \kernel.elf という 12 文字（ヌル文字含む）を格納するために、EFI_FILE_INFO の大きさより sizeof(CHAR16)*12 バイトだけ大きい領域を確保したというわけです。

kernel_file->GetInfo() の実行が終わると、file_info_buffer には EFI_FILE_INFO 型のデータが書かれた状態になります。UINT8 へのポインタである file_info_buffer を EFI_FILE_INFO へのポインタとしては扱うためにキャストを行い、キャストの結果を使って FileSize を読み取っています。キャストについて詳しくは「**コラム 3.2 ポインタのキャスト**」で説明します。

カーネルファイルの大きさが分かったら、gBS->AllocatePages() を使ってファイルを格納できる十分な大きさのメモリ領域を確保します。この関数は第 1 引数にメモリの確保の仕方を、第 2 引数に確保するメモリ領域の種別を、第 3 引数に大きさを、第 4 引数には確保したメモリ領域のアドレスを書き込む変数を指定します。

メモリの確保の仕方は 3 通りから選べます。どこでもいいから空いてる場所に確保する (AllocateAnyPages) か、指定したアドレス以下で空いてる場所に確保する (AllocateMaxAddress) か、指定したアドレスに確保する (AllocateAddress) かです。カー

ネルファイルは 0x100000 番地に配置して動作させる前提で作っており (ld.lld のオプション `--image-base` で指定しています)、それ以外の場所に配置してしまうと動きません。そのため、`AllocateAddress` を指定して確実に 0x100000 番地にメモリを確保するようにしています。

なお、実行する機種によっては 0x100000 番地のメモリが空き領域ではないことがあるかもしれません。そんなときはメモリマップを確認して `EfiConventionalMemory` となっている十分な大きさの領域を探し、その先頭アドレスを `kernel_base_addr` 変数に設定してください。

メモリ領域の種別は、ブートローダが使うための領域なら普通は `EfiLoaderData` を指定しておけば大丈夫です。第 4 引数は `kernel_base_addr` 変数のポインタを渡しています。ポインタを渡しているのは、メモリ確保モードが `AllocateAnyPages` か `AllocateMaxAddress` の場合に、`gBS->AllocatePages()` は関数の呼び出し側に対し、実際に確保されたメモリ領域の先頭アドレスを知らせる必要があるからです。ポインタとして渡さないと変数の値を関数内から書き換えることができません^{*9}。しかし、今回は `AllocateAddress` モードですので変数の値が書き換えられることはありません。

`gBS->AllocatePages()` に渡すメモリ領域の大きさはページ単位で指定しますので、`kernel_file_size` が表すバイト単位の値をページ単位の値に変換する必要があります。UEFI における 1 ページの大きさは 4KiB=0x1000 バイトですから、変換の式は次のようになります。

```
ページ数 = (kernel_file_size + 0xfff) / 0x1000
```

0xfff を足しているのはページサイズへと値を切り上げるためです。`kernel_file_size` がちょうどページサイズの倍数になっていれば、0x1000 で割るだけでぴったりの値になります。しかし多くの場合は端数があるはずですから、単に 0x1000 で割るだけだと端数が切り捨てられてしまいます (C 言語の整数同士の割り算は小数点以下を切り捨てます)。確保されるメモリ領域が、端数の分だけちょっと小さくなり、ファイル全体を読み込むことができません。なぜ 0xfff を足すとうまくいくかは説明しませんが、気になる人は手元で計算してみてください。

メモリ領域が確保できたら `kernel_file->Read()` を使ってファイル全体を読み込みます。ここは特に難しいこともないでしょう。

さて、カーネルがメモリ上に読み込めたら後はカーネルを起動するだけ、と思うかもしれませんが、その前にやっておくべき処理があります。それは今まで動いていた UEFI BIOS のブートサービスを停止することです。ブートサービスは裏でひっそり動いて様々な処理してくれているのですが、OS にとっては邪魔になってしまいますので止めておきます。**リスト 3.4** にそのためのコードを示します。

* 9 `gBS->AllocatePages()` の戻り値はメモリ確保が成功したか失敗したかを表すステータスコードとして使われてしまっており、確保したメモリ領域のアドレスを返すためにポインタを引数として受け取ります。このように C 言語では、2 つ以上の値を呼び出し元に返すためにポインタを引数に取るのは、よく使われる手法です。

リスト 3.4 カーネル起動前にブートサービスを停止させる (Main.c)

```
EFI_STATUS status;
status = gBS->ExitBootServices(image_handle, memmap.map_key);
if (EFI_ERROR(status)) {
    status = GetMemoryMap(&memmap);
    if (EFI_ERROR(status)) {
        Print(L"failed to get memory map: %r\n", status);
        while (1);
    }
    status = gBS->ExitBootServices(image_handle, memmap.map_key);
    if (EFI_ERROR(status)) {
        Print(L"Could not exit boot service: %r\n", status);
        while (1);
    }
}
}
```

ちょっと長いですが、まずは2行目の `gBS->ExitBootServices()` を呼び出している部分に注目します。この関数がまさにブートサービスを停止するための関数です。この関数を実行し、成功したら、その後はブートサービスの機能 (`Print()` やファイルやメモリ関連の機能など) を使うことができなくなります。

`gBS->ExitBootServices()` は、その呼び出し時点で最新のメモリマップのマップキーを要求します。マップキーとは関数の引数に指定している `memmap.map_key` のことで、メモリマップを識別するための値です。メモリマップの内容はブートサービスの利用などによって時々刻々と変化していきます。マップキーはメモリマップと紐付いていて、メモリマップが変化するとマップキーも変化します。`gBS->ExitBootServices()` は指定されたマップキーが最近のメモリマップに紐付くマップキーでない場合、実行に失敗します。

`gBS->ExitBootServices()` が失敗したことは戻り値で分かります。失敗したら再度メモリマップを取得し、そのマップキーを使って再実行します。初回のメモリマップ取得のときからこの関数を呼び出す間にブートサービスの機能をいろいろ使っているため、おそらく1回目の実行は必ず失敗するでしょう。2回目の実行に失敗することは基本的にはありませんが、もし2回目も失敗してしまったら重大なエラーでしょうから、とりあえず永久ループにより停止することにします。

ブートサービスを停止することができたら、いよいよ最後にカーネルを起動する部分を見てみます。

リスト 3.5 メイン関数でカーネルを起動する部分 (Main.c)

```
UINT64 entry_addr = *(UINT64*)(kernel_base_addr + 24);

typedef void EntryPointType(void);
EntryPointType* entry_point = (EntryPointType*)entry_addr;
entry_point();
```

読み込んだカーネルを起動する部分を**リスト 3.5**に示します。このコードは、メモリ上で**エントリポイント**が置いてある場所を計算してエントリポイントと呼び出します。エントリポイントとは、あるプログラムの入り口（エントリ）となる場所のことで、一般的なC言語プログラムでは`main()`、今回は`KernelMain()`がそれです。メモリ上のどこに`KernelMain()`の実体があるかを特定し、それを呼

び出すことこそがブートローダの最大の目的です。コードを理解するにはカーネルファイルの内部構造についての知識が重要なので、少し説明します。

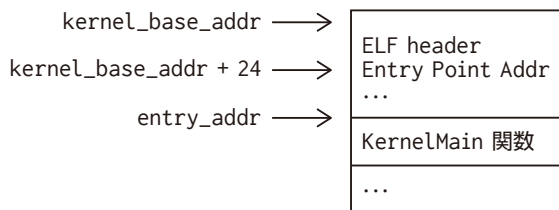
カーネルファイル `kernel.elf` は ELF 形式のファイルです。次のように `readelf` コマンドを使うと詳しい情報が出てきます。

```
$ cd $HOME/workspace/mikanos/kernel
$ readelf -h kernel.elf
ELF ヘッダ:
マジック:  7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00
クラス:                                ELF64
...
型:                                EXEC (実行可能ファイル)
...
エントリポイントアドレス:          0x101000
...
```

`readelf` のオプション `-h` は、指定した ELF ファイルのヘッダ^{*10}を表示するオプションです。`kernel.elf` が 64 ビット用の ELF ファイルであること、種別が EXEC であること、エントリポイントアドレスの値が `0x101000` であることが分かります。その他のヘッダ情報もずらっと表示されますが、今はあまり重要ではありません。

ELF 形式の仕様(参考文献[5])によれば、64 ビット用の ELF のエントリポイントアドレスは、オフセット 24 バイトの位置から 8 バイト整数として書かれることになっています。したがって、上記のコードの 1 行目でその値を読み取り、変数 `entry_addr` に設定しています。種別が EXEC の場合、読み取ったエントリポイントアドレスの値は、`KernelMain()` の実体が置かれたアドレスとなります(図 3.4)。

図 3.4 エントリポイントアドレスとファイル位置との関係



メモリ上でのエントリポイントの場所が `entry_addr` 変数に設定されていますので、最後にその値を関数ポインタにキャストし、呼び出します。この部分が少し複雑なのは、エントリポイントを C 言語の関数として呼び出すための準備が必要だからです。`entry_addr` に設定された値は、エントリポイントが存在する場所のアドレスを表す整数値です。関数の場所が分かれば後は呼び出すだけ、と思うかもしれませんが、しかし、関数定義のメモリ上の位置情報だけでは C 言語の関数として呼ぶのに十分ではなく、さらに引数と戻り値の型情報(関数プロトタイプ)が必要です。

* 10 Header. 構造化されたデータを含むファイルに付加されるもので、ファイルのどこにどんな構造でどんなデータが置かれているかを説明するもの。ファイルの先頭(ヘッド)に書かれたそのような情報をヘッダと呼びます。

`typedef void EntryPointType(void);` は見慣れない書き方かもしれませんが「引数と戻り値がどちらも void 型であるような関数」を表す `EntryPointType` という型を新しく作っています。新しく作った型を使って、`EntryPointType* entry_point` というようにポインタ変数 `entry_point` を定義しています。初期値は `entry_addr` としています。このように、関数の先頭アドレスに引数と戻り値の型情報を組み合わせることによって、C 言語の関数として呼び出す (`entry_point()`) ことが可能になります。

ちなみに、ポインタ変数を新しく作らずとも、`((EntryPointType*)entry_addr)();` のように呼び出すことが可能です。上記のコードは分かりやすさのためにポインタ変数を経由する方法を使いました。

さて、ブートローダをビルドして、カーネルを起動してみましょう。次のコマンドを実行すると QEMU で起動できます。

```
$ cd $HOME/workspace/mikanos
$ git checkout osbook_day03a
$ cd $HOME/edk2
$ build
$ $HOME/osbook/devenv/run_qemu.sh Build/MikanLoaderX64/DEBUG_CLANG38/X64/Loader.efi
efi $HOME/workspace/mikanos/kernel/kernel.elf
```

起動すると、カーネルの動作が永久ループで止まり、最後の `Print(L"All done\n");` が実行されないはずですが。All done と表示される手前で動作が止まれば成功です。

本当でしょうか？ もしかして、変な場所にジャンプしてしまい CPU が暴走している、ということはないでしょうか。こんな時に役立つのが QEMU モニタです。info registers コマンドを使って RIP の値を確認します (図 3.5)。何度か実行し、値が変わっていなければ永久ループしていると考えられます。また、その RIP の値付近のメインメモリ内容を確認して、hlt 命令があるかどうかを見てみるといいでしょう。実際にメインメモリの内容を確認した様子を図 3.6 に示します。まずは RIP が指すメモリ領域を表示したところ jmp 命令がありました。次にそのジャンプ先アドレス 0x101010 が指すメモリ領域を表示してみました。するとそこに hlt 命令が書かれていることが分かりました。

図 3.5 RIP の値を確認

```
(qemu) info registers
RAX=0000000000100000 RBX=
RSI=0000000003feaca08 RDI=
R8 =0000000003fea87c4 R9 =
R12=0000000003effe920 R13=
RIP=0000000000101011 RFL=
```

図 3.6 メインメモリの内容を確認

```
(qemu) x /2i 0x101011
0x0000000000101011: jmp 0x101010
0x0000000000101013: int3
(qemu) x /2i 0x101010
0x0000000000101010: hlt
0x0000000000101011: jmp 0x101010
(qemu) █
```

USB メモリに書き込んで試験用パソコンで実行するには、次のようなディレクトリ構成にします。

```
/kernel.elf
/EFI/BOOT/BOOTX64.EFI <--Loader.efi
```

コラム 3.1 レッドゾーン

カーネルのコンパイルでは `-mno-red-zone` というオプションを付けました。レッドゾーンとは何か、このオプションを付けたらどうなるかを紹介します。現時点ではちょっと難しい話ですので、初回は読み飛ばし、後で戻ってくるのを勧めます。

レッドゾーン (red zone) はスタックポインタを少し超えたスタック領域のことです。System V AMD64 ABI(参考文献[6])では RSP の手前 128 バイトの領域がレッドゾーンと規定されています。この領域は実行中の関数により予約されていて、割り込みハンドラが勝手に変更してはならない、と決められています。

すなわち、他の関数を呼び出さない関数 (leaf function) は RSP の値を調整することなくレッドゾーンを使って良いということです。通常、スタックを一時領域として使うには RSP から使いたい大きさだけ数値を引きますが、欲しい領域の大きさが 128 バイト以下であれば RSP の値を調整せずにレッドゾーンを利用できるのです。

レッドゾーンの機能が機械語にどのような影響を与えるかを紹介します。次の C++ プログラムをレッドゾーン有効なままコンパイルしてみます。

```
int g(int index) {
    int a[16] = {1, 1};
    for (int i = 2; i < 16; ++i) {
        a[i] = a[i - 2] + a[i - 1];
    }
    return a[index];
}
```

上記のプログラムを `clang++ -O1 -mno-sse -c g.cpp` としてコンパイルしました。結果を逆アセンブルすると次のような機械語を得られました。

```
0000000000000000 <g(int)>:
0:  48 c7 44 24 f0 00 00 00 00    mov     QWORD PTR [rsp-0x10],0x0
9:  48 c7 44 24 e8 00 00 00 00    mov     QWORD PTR [rsp-0x18],0x0
...
```

`rsp-0x10` や `rsp-0x18` などのスタックポインタを超えたところにあるメモリ領域に書き込んでいます。これがレッドゾーンの効果です。RSP の値を調整しないことで少し高速化が望めるでしょう。

レッドゾーンを利用した関数の実行中に割り込みが発生するとデータが壊れます。いつ割り込みが発生しても壊れないようにするには、割り込みハンドラにきちんと IST を設定するなどしなければなりません。OS 開発中はレッドゾーンを無効にしておくのが良いでしょう。

3.4 ブートローダからピクセルを描く (osbook_day03b)

何もしないカーネルではつまらないので、カーネルから何か目に見える処理をさせてみようと思います。そうですね、まずは画面をいろんな色で塗りつぶすのを最初の目標とします。

今まで画面表示といえば `Print()` を使った文字列表示だけでした。今後作っていく OS では、ウィンドウやマウスカーソルなど、文字列以外も表示したいと思います。そのために、ピクセル単位^{*11}で描画する方法を知る必要があります。UEFI にある GOP (Graphics Output Protocol) という機能によりピクセル単位で描画するのに必要な情報を得ることができます。

ピクセル描画に必要な情報 (主なもの) は次の通りです。

- ・フレームバッファの先頭アドレス。フレームバッファ (Frame Buffer) とはピクセル (に描画するための値) を敷き詰めたメモリ領域のことです。フレームバッファの各点に値を書き込むと、それがディスプレイのピクセルに反映される仕組みになっています。
- ・フレームバッファの表示領域の幅と高さ。解像度とも言います。
- ・フレームバッファの非表示領域を含めた幅。フレームバッファには、表示領域の右側に表示されない余分な横幅が存在することがあります。
- ・1 ピクセルのデータ形式。フレームバッファの中で 1 ピクセルが何バイトで表現されているか、RGB の 3 色が何ビットずつどんな順番で並んでいるかという情報です。1 ピクセルが 8 ビットであれば 256 色、各色 8 ビットであれば約 1677 万色の表示が可能です。

これらの情報を取得し、それに基づいて適当に模様を描くプログラムを **リスト 3.6** に示します。

リスト 3.6 GOP を取得して画面描画する (Main.c)

```
EFI_GRAPHICS_OUTPUT_PROTOCOL* gop;
OpenGOP(image_handle, &gop);
Print(L"Resolution: %ux%u, Pixel Format: %s, %u pixels/line\n",
      gop->Mode->Info->HorizontalResolution,
      gop->Mode->Info->VerticalResolution,
      GetPixelFormatUnicode(gop->Mode->Info->PixelFormat),
      gop->Mode->Info->PixelsPerScanLine);
Print(L"Frame Buffer: 0x%0lx - 0x%0lx, Size: %lu bytes\n",
      gop->Mode->FrameBufferBase,
      gop->Mode->FrameBufferBase + gop->Mode->FrameBufferSize,
      gop->Mode->FrameBufferSize);

UINT8* frame_buffer = (UINT8*)gop->Mode->FrameBufferBase;
for (UINTN i = 0; i < gop->Mode->FrameBufferSize; ++i) {
    frame_buffer[i] = 255;
}
```

* 11 Pixel。画面を構成する各点のこと。通常のディスプレイでは RGB (赤緑青) を組み合わせて 1 つのピクセルを構成しています。

このプログラムではまず、`OpenGOP()` を使って GOP (Graphics Output Protocol) を取得します。この関数の中身はあまり本質的ではないので説明を省きますが、関数の実行が成功するとポインタ変数 `gop` に値が設定されます。その後、取得した `gop` の中の主な項目について、値を画面に出力しています。

次に、フレームバッファの先頭アドレス (`gop->Mode->FrameBufferBase`) と全体サイズ (`gop->Mode->FrameBufferSize`) を使って画面を塗りつぶしています。自分の好きな色を描画するためにはピクセルのデータ形式 (`gop->Mode->Info->PixelFormat`) をきちんと解釈して、正しく RGB 値を書き込む必要がありますが、今は面倒なので白で塗りつぶすことにします。

このプログラムを QEMU で実行すると、図 3.7 のように画面が白で塗りつぶされました。ピクセルのデータ形式を考慮せずに値を書いています、すべてのバイトに 255 を書く、すなわちすべてのビットを 1 にする^{* 12} と白になるのはどのデータ形式の場合でも同じです。

図 3.7 フレームバッファを白で塗りつぶす



3.5 カーネルからピクセルを描く (osbook_day03c)

さて、ブートローダからピクセルを描くことには成功しました。同じことをカーネルからやってみようと思います。そのためには、描画に必要な情報をブートローダからカーネルに渡す必要があります。やり方はいろいろありますが、今回は `KernelMain()` の引数に指定することになります。改造後のカーネル側のプログラムをリスト 3.7 に示します。

* 12 255=0xff=0b11111111。8 ビットすべてが 1 となります

リスト 3.7: フレームバッファの情報を受け取ってピクセルを描く (main.cpp)

```
#include <cstdint>

extern "C" void KernelMain(uint64_t frame_buffer_base,
                           uint64_t frame_buffer_size) {
    uint8_t* frame_buffer = reinterpret_cast<uint8_t*>(frame_buffer_base);
    for (uint64_t i = 0; i < frame_buffer_size; ++i) {
        frame_buffer[i] = i % 256;
    }
    while (1) __asm__("hlt");
}
```

KernelMain() の引数に 2 つの 64 ビット整数が追加されたことに注目してください。それぞれの引数にはブートローダからフレームバッファの先頭アドレスとサイズが渡される予定です。渡された引数を使って、フレームバッファを塗りつぶすプログラムとなっています。

関数の中で最初に書いてある reinterpret_cast は初めて見るかもしれません。これは C 言語にはない C++ 特有の記法で、キャストの一種です。C 言語で (uint8_t*)frame_buffer_base と書くのと効果はまったく一緒に、単にアドレスを表す整数値をポインタに変換しているだけです、reinterpret_cast を使うことで「整数とポインタの変換であるから気をつけて取り扱うべし」ということが分かりやすくなります。その他はブートローダで塗りつぶしたときとほぼ一緒のプログラムです。画面全体が同じ色だとつまらないので、塗りつぶす色をちょっと工夫して、ピクセルの位置によって色を変えて模様を出すようにしてみました。

<cstdint> をインクルードすることで uintX_t という整数型 (X はビット数) を使えるようにしています。C や C++ の規格では short や int のビット数が決まっていないため、ビット数が重要な場合はこれらのビット数固定の整数型が便利に使えます。フレームバッファの先頭アドレスは 64 ビット値なので、64 ビット (以上の) 変数で受け取ることがバグを産まないためには大切なのです。自作 OS で <cstdint> を使うためには <cstdint> のありかを Clang に伝える必要があります。それを手軽に行えるように buildenv.sh というファイルを準備しました。このファイルは source コマンドで読み込みます。

```
$ source $HOME/osbook/devenv/buildenv.sh
```

buildenv.sh はビルド用に環境変数を整えるためのファイルです。CPPFLAGS と LDFLAGS という環境変数に標準ライブラリへのパスやその他必要な値を設定してくれます。表示して確認してみましょう。

```
$ echo $CPPFLAGS
-I/home/uchan/osbook/devenv/x86_64-elf/include/c++/v1 -I/home/uchan/osbook/devenv/
x86_64-elf/include -nostdlibinc -D__ELF__ -D_LDBL_EQ_DBL -D_GNU_SOURCE -D_POSIX_TIMERS
```

CPPFLAGS という変数にもろもろのオプションが設定されているのが分かります。<cstdint> を利用するために重要なのは 1 つ目のオプション -I<略>/include/c++/v1 です。-I オプションは #include <...> で読み込むファイルの検索対象として指定したディレクトリを追加せよ、という意

味があります。<stdint>は実際には<略>/include/c++/v1/cstdintに有りますので、そのディレクトリ部分を-Iで指定すればコンパイラが適切にファイルを見つけられるようになります。

CPPFLAGSとLDFLAGSを用いてコンパイルとリンクをすればエラーなく完了するはずです。

```
$ clang++ $CPPFLAGS -O2 --target=x86_64-elf -fno-exceptions -ffreestanding -c main.cpp
$ ld.lld $LDFLAGS --entry KernelMain -z norelro --image-base 0x100000 --static -o
kernel.elf main.o
```

さて、カーネル側の改造は終わったのでブートローダ側のプログラムを見てみます。変更点を**リスト3.8**に示します。

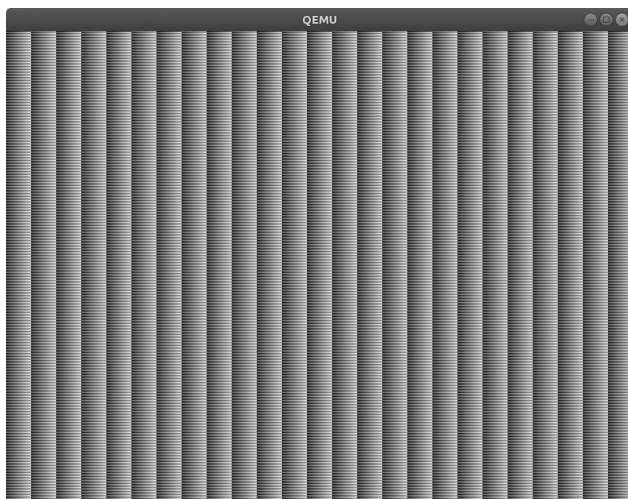
リスト 3.8 フレームバッファの情報をカーネルに渡す (Main.c)

```
typedef void EntryPointType(UINT64, UINT64);
EntryPointType* entry_point = (EntryPointType*)entry_addr;
entry_point(gop->Mode->FrameBufferBase, gop->Mode->FrameBufferSize);
```

KernelMain()の変更に合わせてentry_pointに2つの引数を渡すようにしました。EntryPointType型の定義では引数の型しかなく、引数の名前(frame_buffer_baseなど)は書いていませんが、関数プロトタイプには名前が無くても良いのです。

改造を終えたブートローダで新しいカーネルを起動すると**図3.8**のようになりました。ピクセルのデータ形式を考慮せずに値を書いているので、機種によっては違う模様、色になるかもしれません。

図 3.8 適当な模様をフレームバッファに描く



3.6 エラー処理をしよう (osbook_day03d)

エラー処理とは、呼び出した関数や何らかの機能が失敗したことを検出して対処することです。エラー処理をしなくても正常に動く範囲では何も問題がないように振る舞いますが、それを外れた例外的な状態（メモリが足りないとか、ディスクへの書き込みが失敗したとか、入力された文字列が想定よりずっと長いとか、とにかくいろいろなエラー状態）に陥ると、プログラムはいとも簡単に壊れてしまいます。書き捨てのアプリであればエラー処理は必要ないかもしれませんが、安定して動いてほしいプログラム（OS はまさにその部類ですね）ではエラー処理は必須です。また、エラー処理をしなかったことによる未定義の動作（ヌルポインタアクセスとかバッファオーバーフローとか）が発生すると、それはそのまま脆弱性となり、セキュリティを脅かす存在となります。

今までブートローダの中でエラー処理をほとんどやっていませんでしたが、ここで練習としてエラー処理の追加をやってみます。例えば `gBS->AllocatePages()` は動作が成功すると `EFI_SUCCESS` を、失敗するとそれ以外の値（例えば、要求された大きさのメモリ領域を確保できない場合は `EFI_OUT_OF_RESOURCES`）を返すことになっています。したがって、戻り値が失敗を表す値だったらカーネルの起動処理を中断する、というようなプログラムにします^{*13}。追加したエラー処理とその前後をリスト 3.9 に示します。

リスト 3.9 `gBS->AllocatePages()` のエラー処理 (Main.c)

```
EFI_PHYSICAL_ADDRESS kernel_base_addr = 0x100000;
status = gBS->AllocatePages(
    AllocateAddress, EfiLoaderData,
    (kernel_file_size + 0xffff) / 0x1000, &kernel_base_addr);
if (EFI_ERROR(status)) {
    Print(L"failed to allocate pages: %r", status);
    Halt();
}
```

エラーが検出されたら、そのことを画面に表示します。`Print()` で使っている `%r` 書式指定子は、`EFI_STATUS` の値をエラーメッセージに変換して出力してくれます。`EFI_STATUS` は単なる整数値なので、数値のまま表示しても分かりづらいです。UEFI の機能の多くが `EFI_STATUS` を返すようになっているので、どれも同じようにエラー処理を書けます。

リスト 3.10 `Halt()` はプログラムの実行を無限ループにより止める (Main.c)

```
void Halt(void) {
    while (1) __asm__("hlt");
}
```

エラー処理の中で呼び出している `Halt()` は、リスト 3.10 に示す通り単に `hlt` しつつ無限ループするだけです。この関数を呼ばずに `UefiMain` から `return` するのもいいのですが、そうすると UEFI

* 13 エラー処理の方針は大きく 2 つあります。処理を中断するか、エラー状態を回復して処理を続行するか。一般に、エラー状態を回復させるのは中断するよりも難しいです。ブートローダのエラー処理は、処理を中断する方針を一貫して採用しました。

はこのブートローダの実行を諦めて起動処理を続けようとするため、画面が更新されてエラーメッセージが消えてしまうことがあります。それが嫌な場合、このように無限ループして実行を止めるのがお勧めです。

コラム 3.2 ポインタのキャスト

C/C++ のキャストとは、値の型を変換する操作のことです。整数と浮動小数点数を相互に変換したり、値の精度を変換したり（例えば `int` と `long` の変換など）できます。ポインタと整数、あるいはポインタとポインタの相互変換も可能です。OS 作りではポインタが絡むキャストを多用しますので、このコラムで詳しく説明します。

例えば、`0x1000` にある `int` 型の変数を指し示すポインタを作るには次のようにします。この例に登場する「`(int*)`」がキャストです。

```
(int*)0x1000
```

ポインタとは「指し示す先のアドレスと型を組み合わせたもの」と言えます。整数からポインタへのキャストは「入力された整数（`0x1000`）に対し、指し示す先の型情報（`int`）を付与する操作」なのです。メモリの `0x1000` に置かれたのが 8 ビット整数なのか 32 ビット整数なのか、浮動小数点数なのか構造体なのか、知っているのはプログラマだけです。キャストによってそれらをコンパイラに指示することで、コンピュータは正しく値を読み書きできます。

ポインタ同士をキャストすることもできます。試しに `unsigned long` を指すポインタから `float` を指すポインタにキャストしてみます。

```
#include <iostream>
int main() {
    unsigned long long_var = 0x40200000;
    float* float_ptr = (float*)&long_var;
    std::cout << *float_ptr << std::endl;

    *float_ptr = 1.0;
    std::cout << std::hex << long_var << std::endl;
    return 0;
}
```

このプログラムをコンパイルして実行すると次のような出力となります。なお、このプログラムは C++ の仕様に違反している^{*14}ため、最適化処理によっては異なる結果になる可能性があります。以降の説明は、コンパイラによる最適化が行われないことを前提にしています。

* 14 `float` にキャストしたポインタを経由して `long_var` を読み書きしている箇所が Strict aliasing rule という仕様に違反します。

```
$ clang++ main.cpp && ./a.out
2.5
3f800000
```

ポインタ `&long_var` が指すメモリアドレス (変数 `long_var` が割り当てられているアドレス) と、**キャスト後のポインタが指すメモリアドレスは同じで**、指す先の型だけが `unsigned long` から `float` に変更されます。キャスト前後のどちらのポインタを使って読み書きするかは、単に、メモリ上のビット列をどちらの型の値だと思って読み書きするかの違いです。上記の例では整数 `0x40200000` と浮動小数点数 `2.5` のビット表現、整数 `0x3f800000` と浮動小数点数 `1.0` のビット表現が互いに同じであることが分かります。

「3.3 初めてのカーネル」では「8 ビット符号なし整数を指すポインタ」から「`EFI_FILE_INFO` を指すポインタ」へのキャストが登場しました。キャスト後のポインタを利用して `FileSize` を読み取ります。

```
UINT8 file_info_buffer[file_info_size];
...
EFI_FILE_INFO* file_info = (EFI_FILE_INFO*)file_info_buffer;
UINTN kernel_file_size = file_info->FileSize;
```

`file_info_buffer` は 8 ビット整数の配列ですが、キャスト後のポインタを利用することで、同じ場所にあたかも `EFI_FILE_INFO` 構造体があるかのように読み書きすることができます。このように、8 ビット整数型 (`UINT8` とか) の配列として適切な大きさのメモリ領域を確保しておき、後でそれを異なる型のポインタにキャストして使う、というのは OS 自作ではよく使うテクニックです。

この方法を使うときはキャスト前後の型のアライメント (alignment) に気をつける必要があります。アライメントは変数を配置するメモリアドレスの制約であり、「N バイトアライメント」は N バイトの倍数のアドレスに変数を置くことを意味します。`int` や構造体などの 1 バイトより大きい型はアライメント制約を持っている一方で、`UINT8` などの 1 バイト型は普通はアライメント制約を持ちません (1 バイトアライメント制約があるとも言えます)。そのため、`file_info_buffer` が `EFI_FILE_INFO` 型のアライメント制約を満たさないようなアドレスに割り当てられてしまった場合、キャスト後のポインタを使った読み書きがうまく行かない (変な値が読めたり、CPU が例外を発生させたり) ことがあります。本文では説明を避けるために省略してしまいましたが (そして、今回は運良くそれで動きましたが)、本来は次のようにアライメント制約をきちんと指定する方が良いでしょう。`alignas` と `alignof` は、C 言語では `<stdalign.h>` をインクルードすると使えます。

```
alignas(alignof(EFI_FILE_INFO)) UINT8 file_info_buffer[file_info_size];
```

なお、ポインタやアドレス、キャストなどの話題は C++ の規格で厳密に決まっておらず、処理系依存となっていることが多いです。このコラムの内容 (に限らずこの本の内容すべて) はこの本で対象としている x86-64 向けのコンパイラが前提となっています。他の処理系では違う挙動になるかもしれませんのでご注意ください。

3.7 ポインタ入門 (2) : ポインタとアセンブリ言語

ポインタがどのように実現されるかをアセンブリ言語の世界で見ると理解が深まります。アセンブリ言語の世界ではポインタの実現にアドレスが直接に使われ、ポインタの動作を詳細に観察できるからです。

```
void foo() {
    int i = 42;
    int* p = &i;
    int r1 = *p;
    *p = 1;
    int r2 = i;
    uintptr_t addr = reinterpret_cast<uintptr_t>(p);
    int* q = reinterpret_cast<int*>(addr);
}
```

上記の C++ プログラムを Clang で（最適化をかけずに）コンパイルすると次のようになりました。まずは関数の前半部分から。

```
_Z3foov:
    push    rbp
    mov     rbp, rsp

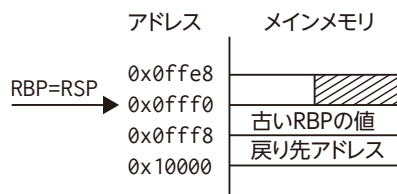
    ; int i = 42; (; から始まる行はコメント)
    mov     dword ptr [rbp - 4], 42
```

1行目の「_Z3foov」は、関数 `foo` の、アセンブリ言語の世界における名前です。C++ の名前修飾により、このように異なる名前に変換されます。名前修飾について詳しくは「3.3 初めてのカーネル」を参照してください。

2行目、3行目は関数の入口における典型的な処理です。2行目の `push` 命令は、RSP レジスタの値を8だけ減らしてから RSP レジスタが指すメモリ領域に指定された値を書き込みます。すなわち、RBP レジスタの値がスタックの末尾に保存されます。C++ のポインタとは違いますが、RSP はスタックポインタと呼ばれる一種のポインタです。スタックの末尾となるメインメモリ上の位置を指します。

3行目の `mov` 命令は右から左方向への値のコピーです。C++ 風に書けば `rbp = rsp;` という処理ですね。RSP の値は一般にどんどん変化してしまうため、後で基準点として使えるように、ある瞬間の値をコピーしておくのです。3行目まで終わるとメモリとレジスタの状態は図 3.9 のようになります。

図 3.9 関数の入り口処理が終わった時点でのスタック



この状態で6行目のmov命令が実行されるとどうなるか、詳しく見てみます。このmovも右から左へのコピーですが、先ほどと違うのは左側の値が[と]で囲まれていることです。x86 アセンブラは[xxx]をアドレスxxxへのメモリアクセスだと認識します*15。今回は、メモリアドレスがRBP-4、すなわち図の斜線部へのメモリアクセスとなります。この部分に42が書き込まれます。

[]の前に書いてあるdwordは、メモリアクセスがdwordの大きさ(4バイト)であることを意味します。今回使ったコンパイラのint型が4バイトであることと対応します。その後ろにあるptrは[]がメモリアクセスだと明示するものですが、今回は省いても構いません。

このmov命令をC++に対応させて解釈すると、RBP-4を先頭とする4バイトのメモリ領域が変数iに対応し、そこに42を書き込む、ということになります。関数内で宣言したローカル変数は、このようにスタック領域に置かれるのですね。

さて、いよいよC++でのポインタがアセンブリ言語でどうなるかを見ていきます。皆さんはすでに、次のアセンブリ言語コードを理解するのに必要な知識の多くを持っています。

```
; int* p = &i;
lea  rax, [rbp - 4]
mov  qword ptr [rbp - 16], rax

; int r1 = *p;
mov  rax, qword ptr [rbp - 16]
mov  ecx, dword ptr [rax]
mov  dword ptr [rbp - 20], ecx

; *p = 1;
mov  rax, qword ptr [rbp - 16]
mov  dword ptr [rax], 1

; int r2 = i;
mov  ecx, dword ptr [rbp - 4]
mov  dword ptr [rbp - 24], ecx

; uintptr_t addr = reinterpret_cast<uintptr_t>(p);
mov  rax, qword ptr [rbp - 16]
mov  qword ptr [rbp - 32], rax

; int* q = reinterpret_cast<int*>(addr);
mov  rax, qword ptr [rbp - 32]
mov  qword ptr [rbp - 40], rax

pop  rbp
ret
```

* 15 []がメモリアクセスを表すのはインテル記法の文法。AT&T記法はまた異なる文法体系を持ちます。

2行目の `lea *16` は初出ですが、理解は簡単です。leaq 命令は、実際のメモリアクセスは行わずにメモリアドレスを計算した結果をレジスタに書き込む命令です。要するに `rax = rbp - 4;` ということです。これは変数 `i` のアドレスを取得することに他なりません。つまり `&i` を計算しているんですね。3行目で、そのアドレス値を RBP-16 のメモリ領域に書き込みます。qword は 8 バイトという意味です。RBP-16 を先頭とする 8 バイトのメモリ領域が変数 `p` に対応し、そこに変数 `i` のアドレスを書きます。

変数 `i` に 42 を書く処理と変数 `p` にアドレスを書く処理がとても似ていると思いませんか？ 書き込むサイズは dword、qword という違いがありますが、両方とも mov 命令であり、第 1 引数に `[]` を使っていることが共通しています。これは単なる偶然ではありません。i も p もメモリ上に確保された変数であることに変わりはないのです。

さらに読み進めます。6行目からは変数 `p` が指す先の値を読み、変数 `r1` に書く操作です。6行目は変数 `p` に書かれた値を読み、RAX に覚えておきます。ここで読まれる値は 3行目で書いた値、すなわち変数 `i` のメモリアドレスです。7行目は RAX が指すメモリ領域（変数 `i`）から 4 バイトを読みます。8行目で、その値を変数 `r1` に書きます。

どうでしょうか？ `int r1 = *p;` という、ポインタ変数を使ったメモリ読み込みの動作をアセンブリ言語の視点で見てみましたが、とても簡単ではないでしょうか。アセンブリ言語の視点では**ポインタ変数と普通の変数に扱いの差はなく**、どちらも mov 命令で読み書きするだけです。`[]` を使ったメモリアクセスを 2 回やるか、1 回やるかが違うだけです。

ポインタ変数に「他の変数を指すアドレス」が格納されていることを知っているのは C++ コンパイラです。C++ コンパイラの気持ちになって、`int r1 = *p;` という間接演算子によるメモリアクセスに対応するアセンブリ命令を考えてみます。

- `p` は int 型へのポインタだな。ってことは `*p` はポインタが指す先から int 型の値を 1 つ、読めばいいんだ。
- まず `p` が保持する値を一旦レジスタに読み出そう。`p` はポインタだから qword でいいな。
→ `mov rax, qword ptr [rbp - 16]`
- 読んだ値はアドレス値だから、それが指す先の値を読もう。指す先の型は int だから dword だ。
→ `mov ecx, dword ptr [rax]`
- 最後に、読んだ値を `r1` に書いておしまいだ！ `r1` は int だから dword だ。→ `mov dword ptr [rbp - 20], ecx`

ここまで分かれば残りの処理も怖くありません。読者の皆さんの宿題としますので、理解を試みてください。

最後の 2 行は関数の出口の処理です。関数の入り口にあった処理（push と mov）と対となる、典型的な処理です。pop で RBP の値を関数呼び出し前の値に復元し、ret で関数の呼び出し元に戻ります。

* 16 Load Effective Address。Effective address とは、セグメンテーションによるアドレス変換やスケールリング、ディスプレースメントの加算を終えた後のリニアアドレスのこと。

第 4 章

ピクセル描画と make入門

この章では OS 本体から画面に絵を描く方法を学びます。画面の表示を自由自在に操れることは、コンピュータをユーザが操作できるようにするためにはとても重要です。後半ではプログラムを正しく動かすのに重要な ELF ファイルのロード方法を学びます。

4.1 make 入門 (osbook_day04a)

make とはコンパイルやリンクなどの作業を自動化するツールです。今までカーネルをコンパイルするときは clang++ や ld.lld を 1 つ 1 つ手で入力していましたが、make を使うとそれらを自動化することができます。覚えておくと便利なツールですので、ここで説明することにします。なお、より詳しく知りたい人は「GNU Make 第 3 版」(参考文献 [7]) を参照してください。

make を構成するのは make コマンドと、指示書である Makefile です。まずは「第 3 章 画面表示の練習とブートローダ」で登場したカーネルのコンパイルとリンクを Makefile で表現してみます(リスト 4.1)。コマンドラインが書いてある 3 つの行 (rm、ld.lld、clang++) の先頭の空白は、スペースではなくタブ文字です。

リスト 4.1 初めての Makefile

```
TARGET = kernel.elf
OBSJS = main.o

CXXFLAGS += -O2 -Wall -g --target=x86_64-elf -ffreestanding -mno-red-zone \
            -fno-exceptions -fno-rtti -std=c++17
LDFLAGS += --entry KernelMain -z norelro --image-base 0x100000 --static

.PHONY: all
all: $(TARGET)

.PHONY: clean
clean:
    rm -rf *.o

kernel.elf: $(OBSJS) Makefile
    ld.lld $(LDFLAGS) -o kernel.elf $(OBSJS)

%.o: %.cpp Makefile
    clang++ $(CPPFLAGS) $(CXXFLAGS) -c <
```

この Makefile の中身を説明する前に、とりあえず make を使ったビルドがどのように動くのか見てみましょう。Makefile が置いてあるディレクトリに cd で移動してから make を実行します。

```
$ cd $HOME/workspace/mikanos/kernel
$ git checkout osbook_day04a
$ make
clang++ -I/home/uchan/osbook/devenv/x86_64-elf/include/c++/v1 -I/home/uchan/osbook/devenv/x86_64-elf/include -I/home/uchan/osbook/devenv/x86_64-elf/include/freetype2 -nostdlibinc -D__ELF__ -D_LDBL_EQ_DBL -D_GNU_SOURCE -D_POSIX_TIMERS -O2 -Wall -g --target=x86_64-elf -ffreestanding -mno-red-zone -fno-exceptions -fno-rtti -std=c++17 -c main.cpp
ld.lld -L/home/uchan/osbook/devenv/x86_64-elf/lib --entry KernelMain -z norelro --image-base 0x100000 --static -o kernel.elf main.o
```

make の出力を見てみると clang++ と ld.lld が順に実行されていることが分かります。これは Makefile に記述した依存関係のルールが正しく機能している証拠です。以下、ファイルの中身を詳しく見ていきます。

Makefile は前半が変数定義、後半がルールの定義になっています。変数は自由に定義して使えますので、ここでは表 4.1 に示す 4 つの変数を定義してみました。

表 4.1 初めての Makefile の変数

変数名	意味
TARGET	この Makefile が生成する最終成果物。
OBJS	TARGET を作るのに必要なオブジェクトファイル群。
CXXFLAGS	コンパイルオプション。
LDFLAGS	リンクオプション。

変数定義の後にはルールが続きます。Makefile において「ルール」とは、ターゲットとその前提となる必須項目、それから、必須項目からターゲットを生成するレシピ（コマンド列）を集めたものです。1 つのルールは次のような書き方をします。「ターゲット」を作り出すために必要となるファイル群が「必須項目」です。必須項目からターゲットを作る実際の手順がレシピです。レシピの行は必ず先頭をタブ文字で始める必要があります。

ターゲット：必須項目
レシピ

上記の Makefile には 4 つのターゲットがあります。all、clean、kernel.elf、%.o です。ターゲット名は make コマンドの引数に指定することができます。試しに make clean とすると main.o が削除されるはずです。make を引数なしで実行した場合は Makefile の最初に現れるターゲット、すなわち all が指定されたのと同じ動きになります。

ターゲットのうち、all と clean は実際のファイル名ではなく、単にルールを表す名前として使っています。デフォルトのビルドを行う all、ビルドの中間ファイルを削除する clean といったように。これらのターゲットのことを、本当のファイル名ではないという意味で偽物のターゲット（phony target）と呼び、.PHONY 宣言を行います^{*1}。

さて、Makefile の要となるそれぞれの「ルール」は次のように再帰的に処理されます。

- ・まず、すべての必須項目に対して、それぞれをターゲットとするルールを実行する
- ・その後、次の規則にしたがってレシピを処理する
 - ターゲットが必須項目より新しい場合、何もしない
 - それ以外（ターゲットが実在しないか必須項目より古い）なら、レシピを実行する

all から始まって再帰的にルールが検索され実行される様子を図示すると次のようになります。

*1 Phony 宣言は、all や clean といったファイルが実在する場合に効果を発揮します。それらのファイルがなければ、Phony 宣言しなくても問題が起きることはありません。

```
all: kernel.elf
--> 必須項目 (kernel.elf) のルール (kernel.elf: main.o Makefile) を実行。
--> 必須項目 (main.o) のルール (main.o: main.cpp Makefile) を実行。
--> 必須項目 (main.cpp) のルールを実行しようとするが、ないので何もしない。
--> 必須項目 (Makefile) のルールを実行しようとするが、ないので何もしない。
--> レシピを実行。clang++ で main.o を生成。
--> 必須項目 (Makefile) のルールを実行しようとするが、ないので何もしない。
--> レシピを実行。ld.lld で kernel.elf を生成。
--> レシピなし。何もしない。
```

動作が再帰的なので分かりにくいかもしれません。正確に理解しなくても本書の残りの部分を読むのに支障はありませんので、もし分からなくても立ち止まらずに読み進めることをお勧めします。もし、何故かレシピがスキップされて必要なコマンドが実行されないことがあったら、魔法のコマンド `make -B` を実行するといいでしょう。-B オプションは、ファイルが新しいとか古いとかを無視して、すべてのレシピを再実行させるオプションです。

`%o` や `%cpp` は具体的なファイル名ではなく、ファイル名のパターンです。パターンを用いたルールは、対象となるファイルが多くなる場合に便利です。OBJS に含まれるオブジェクトファイルは将来的に増えていきますが、各ファイルに対するルールを `main.o: main.cpp Makefile` などと具体的に記述するより、パターンルールで統一した方が楽ですね。`%` に当てはまる部分のことを「幹 (stem)」と呼びます。

さて、`%o` のレシピには特殊な変数 `$<` がありますね。この変数は `make` が自動的に定義してくれるもので、値は必須項目の先頭 1 つのファイル名となります。今回の場合では `main.cpp` です。この他にも自動的に定義される変数があります。主なものを表 4.2 に示します。

表 4.2 make が自動的に定義する変数

変数名	説明
<code>\$<</code>	必須項目の先頭 1 つ
<code>^</code>	必須項目すべてをスペース区切りで並べたもの
<code>\$@</code>	ターゲット (拡張子含む)
<code>\$*</code>	パターンルールにおける幹 (stem)

`$*` の説明が分かりにくいかもしれません。いくつか実例を示します (表 4.3)。

表 4.3 幹の例

ターゲットパターン	実際のターゲット	<code>\$*</code> の値 (幹)
<code>%o</code>	<code>foo.o</code>	<code>foo</code>
<code>a.%.b</code>	<code>dir/a.foo.b</code>	<code>dir/foo</code>

ところで、`%o` や `kernel.elf` のルールの必須項目には `Makefile` を指定していますがレシピ内では使っていません。レシピで使うファイルだけを必須項目に書くのが基本ですが、これだけ例外になっています。その理由は、`Makefile` の内容が更新された場合にはビルドし直すべきだという考えがあるからです。`Makefile` を必須項目として登録しておけば、更新に合わせてレシピが再実行されます。

4.2 ピクセルを自在に描く (osbook_day04b)

前章までで、カーネルからピクセルを塗りつぶせるようになりましたが、ピクセルのデータ形式をまったく考慮しておらず、機種によってどんな色になるか分かりません。また、画面上の位置を指定して特定のピクセルだけ塗るというのでもできません。ウィンドウを描くにも、文字を表示するにも、好きな位置に好きな色を描画できることが必要になります。この節ではそういう機能を開発していきます。

リスト 4.2 フレームバッファの構成情報を表す構造体 (frame_buffer_config.hpp)

```
#pragma once

#include <stdint.h>

enum PixelFormat {
    kPixelRGBResv8BitPerColor,
    kPixelBGRResv8BitPerColor,
};

struct FrameBufferConfig {
    uint8_t* frame_buffer;
    uint32_t pixels_per_scan_line;
    uint32_t horizontal_resolution;
    uint32_t vertical_resolution;
    enum PixelFormat pixel_format;
};
```

ピクセル描画に必要な情報をまとめるための FrameBufferConfig 構造体の定義を **リスト 4.2** に示します。フレームバッファ領域へのポインタ、フレームバッファの余白を含めた横方向のピクセル数、水平と垂直の解像度、ピクセルのデータ形式を保持できます。

UEFI の規格ではピクセルのデータ形式は次の 4 種類があります (EFI_GRAPHICS_PIXEL_FORMAT 列挙体で定義された値です)。

- ・ PixelRedGreenBlueReserved8BitPerColor
- ・ PixelBlueGreenRedReserved8BitPerColor
- ・ PixelBitMask
- ・ PixelBltOnly

皆さんは光の 3 原色を知っているでしょうか。赤・緑・青を混ぜることで多くの色を表現できるというやつです。赤・緑・青の英語の頭文字を取って RGB と呼ぶこともあります。この原理はパソコンの画面にも利用されていて、画面上に無数に並ぶピクセルの 1 つ 1 つが 3 原色の光を出せるようになっています。遠くから見ると色が混ざって見えるため、様々な色の絵を描けるというわけです。

ですから、画面に絵を描くには、各ピクセルの各色をどれだけの強さで光らせるか、というのを指定することになります。光る強さのことを **階調** と呼び、各色の階調により表現できる色数が決まります。例えば赤を 8 階調 (3 ビット)、緑を 4 階調 (2 ビット)、青を 8 階調 (3 ビット) で色を表現したとすると、全部で 256 色を表現できます。フレームバッファは 3 原色の光る強さを整数値で並べたものだ、ということができます。フレームバッファにどのような色の順番で、それぞれ何ビットで並べるか、というのが

PixelFormat が表しているものです。

本書では4種類のうち最初の2種類だけをサポートすることにします。多くのパソコンで2種類のどちらかはサポートされていると思いますので、描画プログラムが複雑になる PixelBitMask の実装はサボりたかったのです。また、PixelBltOnly はピクセル単位で描画することができず、メモリに描いた絵を一気にコピーすることにより画面描画する方式です。さらに描画プログラムが複雑になってしまいますので、本書の対象外とします。

リスト 4.3 ブートローダは OS 本体に描画に必要な情報を渡す (Main.c)

```

struct FrameBufferConfig config = {
    (UINT8*)gop->Mode->FrameBufferBase,
    gop->Mode->Info->PixelsPerScanLine,
    gop->Mode->Info->HorizontalResolution,
    gop->Mode->Info->VerticalResolution,
    0
};
switch (gop->Mode->Info->PixelFormat) {
    case PixelRedGreenBlueReserved8BitPerColor:
        config.pixel_format = kPixelRGBResv8BitPerColor;
        break;
    case PixelBlueGreenRedReserved8BitPerColor:
        config.pixel_format = kPixelBGRResv8BitPerColor;
        break;
    default:
        Print(L"Unimplemented pixel format: %d\n", gop->Mode->Info->PixelFormat);
        Halt();
}

typedef void EntryPointType(const struct FrameBufferConfig*);
EntryPointType* entry_point = (EntryPointType*)entry_addr;
entry_point(&config);

```

ブートローダ側はリスト 4.3 のように改造しました。UEFI の GOP から取得した情報を、先ほど作った構造体にコピーします。そして、その構造体へのポインタを KernelMain() の第 1 引数に渡します。

リスト 4.4 WritePixel() を使って画面を描画する (main.cpp)

```

extern "C" void KernelMain(const FrameBufferConfig& frame_buffer_config) {
    for (int x = 0; x < frame_buffer_config.horizontal_resolution; ++x) {
        for (int y = 0; y < frame_buffer_config.vertical_resolution; ++y) {
            WritePixel(frame_buffer_config, x, y, {255, 255, 255});
        }
    }
    for (int x = 0; x < 200; ++x) {
        for (int y = 0; y < 100; ++y) {
            WritePixel(frame_buffer_config, 100 + x, 100 + y, {0, 255, 0});
        }
    }
    while (1) __asm__("hlt");
}

```

KernelMain() 側の改造をリスト 4.4 に示します。注目してほしいのは 2 点、構造体のポインタを参照 (const FrameBufferConfig&) で受け取っていることと、関数本体で WritePixel() なる関数と呼んでいることです。

参照型は C++ 固有の文法で、C 言語にはありません。参照型の引数を持つ関数を C 言語から呼び出すには、参照の代わりにポインタを指定すれば大丈夫です。これは C++ 自体の仕様ではありませんが、私たちが使っているコンパイラの仕様である System V AMD64 ABI (参考文献 [6]) で決まっています。ABI についてはコラム 4.1 で少し詳しく説明します。

リスト 4.5 WritePixel() の実装 (main.cpp)

```
struct PixelColor {
    uint8_t r, g, b;
};

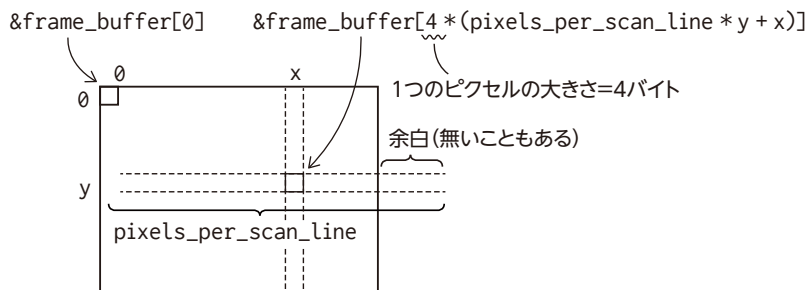
/** WritePixel は 1 つの点を描画します.
 * @retval 0 成功
 * @retval 非 0 失敗
 */
int WritePixel(const FrameBufferConfig& config,
               int x, int y, const PixelColor& c) {
    const int pixel_position = config.pixels_per_scan_line * y + x;
    if (config.pixel_format == kPixelRGBResv8BitPerColor) {
        uint8_t* p = &config.frame_buffer[4 * pixel_position];
        p[0] = c.r;
        p[1] = c.g;
        p[2] = c.b;
    } else if (config.pixel_format == kPixelBGRResv8BitPerColor) {
        uint8_t* p = &config.frame_buffer[4 * pixel_position];
        p[0] = c.b;
        p[1] = c.g;
        p[2] = c.r;
    } else {
        return -1;
    }
    return 0;
}
```

2 つ目の注目ポイントである WritePixel() の実装をリスト 4.5 に紹介します。この関数は指定したピクセル座標 (x と y) に、指定した色 (c) を描画する関数です。ピクセルのデータ形式 (config.pixel_format) に基づいて、光の 3 原色をフレームバッファへ書き込みます。

pixel_position にはピクセルの座標をフレームバッファ先頭からの位置に変換した値が設定されます*2。座標をフレームバッファ先頭からの位置に変換するには「余白を含めた横方向のピクセル数 * y + x」という式を用います。ピクセル座標とフレームバッファ先頭からの位置の関係は図 4.1 のようになります。

* 2 const はその変数を定数化します。const 宣言された変数は初期化時にしか値を設定できず、その後で代入しようとするとコンパイルエラーになります。

図 4.1 ピクセル座標とフレームバッファ先頭からの位置の関係



2つのデータ形式 `PixelRGBResv8BitPerColor` と `PixelBGRResv8BitPerColor` は両方とも各色を8ビットで表しますが、その並び順が違います。名前の通り、前者は赤、緑、青と並び、最後に8ビットの予約領域が続きます。後者は青、緑、赤、予約領域の順です。予約領域は、原色の3色だけだと24ビットと中途半端になるところ、8ビットを追加することで32ビットに揃える役目があります。1ピクセルは32ビットすなわち4バイトですから、あるピクセルのフレームバッファ先頭からのバイト位置を計算するには、ピクセルとしての位置 (`pixel_position`) に4を掛けます。

`KernelMain()` では `WritePixel()` を使って、まず画面全体を白 (255、255、255) で塗りつぶし、その上に 200×100 の緑の四角形を描画しています。このプログラムを実行すると図 4.2 のような図形が表示されるはずです。

図 4.2 `WritePixel()` を使って緑色の四角を描いた

ここまでのやり方でとりあえずピクセルの描画は自由にできるようになりました。しかし、上記のプログラムには動作効率の点で良くないことがあります。それは `WritePixel()` の中でピクセルのデータ形式の判定と分岐をしていることです。ピクセルのデータ形式は画面描画の最中に変化することはないのに、`WritePixel()` が呼び出されるたびに判定するのは無駄なのです。 `WritePixel()` は `for` 文の中で呼ばれますから、画面の解像度が 800×600 だとすると、 $800 \times 600 + 200 \times 100 = 50$ 万回も呼び出されます。50万回も無駄な判定をしたくないですね*³。次節では、C++ の言語機能を利用してピクセル描画プログラムを改良してみます。

コラム 4.1 ABI

ABI (Application Binary Interface) は、プログラムと CPU の相互作用を機械語レベルで規定します。プログラムが動作するにあたり必要なレジスタやメモリの使い方が ABI によって決まるのです。コンパイラは ABI に従って機械語を生成するため、OS 内部の仕組みも ABI を考慮しなければなりません。

本書では全体的に System V AMD64 ABI を前提にプログラミングを行います。この ABI は x86-64 向けのもので、Linux で標準的に使われています。x86-64 向けの ABI でもう 1 つ有名なのは Microsoft x64 ABI です。こちらは Windows で使われています。2 つの ABI は関数呼び出し規約等が異なり、互いに互換性がありません。異なる ABI を用いてコンパイルされた関数をリンクしても正常に動作しないということです。

ABI の中の重要な項目の 1 つが**呼出規約** (Calling Convention) です。呼出規約は関数を呼び出す際の決まり事の中で、引数をどのように渡すか、戻り値をどのように返すか、関数の中で変更して良いレジスタはどれか、などを定めています。「**コラム 3.1 レッドゾーン**」で紹介したレッドゾーンも ABI で規定されているルール of the 1 つです。

まずは引数です。System V AMD64 ABI では、64 ビットまでの整数やポインタを引数にする場合、先頭から RDI、RSI、RDX、RCX、R8、R9 の順でレジスタを割り当てます。x86 の 32 ビットモードで標準的に使われる ABI では、引数はレジスタではなくスタックに積むことになっていました。レジスタを使うことで関数呼び出しが高速になります。

戻り値は RAX、RDX をこの順で使います。64 ビットまでの整数やポインタを戻り値にする場合は RAX だけを使います。RAX に戻り値を設定した状態で `ret` することで、関数の呼び出し側に値を返せます。

関数呼び出しの前後で RSP、RBP、RBX、R12 から R15 の計 7 個のレジスタの値が保たれていなければなりません。すなわち、呼び出された側の関数がこれらのレジスタを使うのであれば、レジスタを変更する前にスタックなどに値を退避し、関数から抜ける前にレジスタの値を復帰させます。これら 7 個以外のレジスタは呼び出された側の関数が自由に書き換えて良いことになっています。

この他のケース、例えば浮動小数点数や 64 ビットを超えるデータを引数や戻り値にする規則や、汎用レジスタ以外の保存と退避の規則などは System V AMD64 ABI ([参考文献 \[5\]](#)) を参照してください。ただ、本書に登場するプログラムを理解するには、このコラムで紹介した知識で十分です。

C++ で関数を書いたなら、コンパイラが自動的に ABI の制約を守って機械語を作ります。アセンブリ言語で関数を書くなら、ABI の制約を守るのはプログラマの責任です。本書ではいくつかの関数をアセンブリ言語で実装しますから、そのたびに ABI の話が登場します。

* 3 CPU には投機実行という仕組みがあり、実際は 50 万回のうちのほとんどは判定コストがかからない可能性があります。ただ、ループの外側でできる処理はなるべくループの外側でやる、というのは速いプログラムを書くための良い心がけです。

4.3 C++の機能を使って書き直す (osbook_day04c)

前節で作った `WritePixel()` には、関数内部でピクセルのデータ形式を判定してしまっているという問題がありました。そこで、ここではC++の言語機能である**仮想関数**を使って、`WritePixel()` 相当の機能を持ちつつ、関数の外側でデータ形式を判定するように書いてみます。

まず基本的な設計はクラスを使うことにします。C++における**クラス**はデータとその操作をひとまとめにしたもので、大雑把に言えば、C言語の構造体に操作用の関数を追加したものです。C++のクラスのように、データとそれを操作する手続きを合わせたものを**抽象データ型**と呼びます。抽象データ型は外部のユーザからデータの中身（構造や内容）の詳細を隠蔽し、外部に公開された手続きによる操作を強制することで、インタフェースと実装を分離できるという特徴があります。これは大きなプログラムを構成しやすくするモジュール分割を行うための基本的な考え方の1つです。

うんちくを語ってしまいましたが、ここではクラスを使うことで、ピクセルのデータ形式に依存しない「ピクセル描画インタフェース」と「ピクセルのデータ形式にしたがって実際に描画する実装」を分離するのに使っています。まずはインタフェースの部分を、次に実装の部分を説明します。

リスト 4.6 PixelWriter クラスの実装 (main.cpp)

```
class PixelWriter {
public:
    PixelWriter(const FrameBufferConfig& config) : config_{config} {
    }
    virtual ~PixelWriter() = default;
    virtual void Write(int x, int y, const PixelColor& c) = 0;

protected:
    uint8_t* PixelAt(int x, int y) {
        return config_.frame_buffer + 4 * (config_.pixels_per_scan_line * y + x);
    }

private:
    const FrameBufferConfig& config_;
};
```

リスト 4.6 はインタフェースにあたる部分のソースコードです。PixelWriter クラスを1つ定義しています。このクラスはピクセルを描画するための `Write()` 関数を含んでいます。この関数のプロトタイプ宣言の後ろに `= 0` とあるのは、この関数が純粋仮想関数であることを示します。純粋仮想関数は実装を持っていないことが特徴であり、後で複数の実装を与えて動的に実装を切り替えて使うことができます。純粋仮想関数は、実際の処理の内容は決まっていないけど、戻り値と引数の仕様、それから関数名は決まっている関数、つまりインタフェースを表現できます。

戻り値の型が何も書いてない、クラス名と同じ名前の `PixelWriter()` 関数はコンストラクタと呼ばひ、頭にチルダがある `~PixelWriter()` 関数はデストラクタと呼ばひます。クラスは構造体とほぼ同じものですから、クラスそのものがメモリ上に存在するわけではありません。クラス型の変数を定義して初めて、クラスの定義にしたがってインスタンスがメモリ上に生成されます^{*4}。インスタンスをメモリ上に構築 (construct) するために呼ばれるのがコンストラクタ、インスタンスを破棄 (destruct) するために呼ばれるのがデストラクタです。

コンストラクタ `PixelWriter::PixelWriter` はフレームバッファの構成情報を受け取

り、クラスのメンバ変数 `config_` にコピーします（コピーと言っても参照型のコピーですから、`FramebufferConfig` 構造体の中身がコピーされるのではなく、その参照（実体はポインタ）がコピーされるだけです）。構成情報をコンストラクタで受け取っておくことで、後で `PixelWriter::Write()` を呼び出す際に構成情報を渡す必要が無くなります。クラスを使わない場合は `WritePixel()` のように1ピクセルを描画するたびに構成情報を渡す必要がありますが、クラスを使うとその辺りが綺麗に書けますね。

リスト 4.7 PixelWriter クラスを継承したクラス群 (main.cpp)

```
class RGBResv8BitPerColorPixelWriter : public PixelWriter {
public:
    using PixelWriter::PixelWriter;

    virtual void Write(int x, int y, const PixelColor& c) override {
        auto p = PixelAt(x, y);
        p[0] = c.r;
        p[1] = c.g;
        p[2] = c.b;
    }
};

class BGRResv8BitPerColorPixelWriter : public PixelWriter {
public:
    using PixelWriter::PixelWriter;

    virtual void Write(int x, int y, const PixelColor& c) override {
        auto p = PixelAt(x, y);
        p[0] = c.b;
        p[1] = c.g;
        p[2] = c.r;
    }
};
```

リスト：4.7 はインタフェースに対する実装に相当する部分のソースコードです。インタフェースである `PixelWriter` クラスを継承して、2種類のピクセル形式それぞれに対応する実装を行った `RGBResv8BitPerColorPixelWriter` クラスと `BGRResv8BitPerColorPixelWriter` クラスを定義しています。継承というのは、あるクラスを基にして機能の差分を書くやり方の1つです。今回は、基となる `PixelWriter` クラスの `Write()` に実装を与えるために継承しています。継承の基となるクラスを親クラスとかベースクラスと呼び、継承してできた新しいクラスを子クラスとかサブクラスと呼びます。このあたりの知識はC++の教科書に詳しく載っているので必要に応じて参照してください。

親クラスの関数を子クラスで上書きすることを**オーバーライド** (override) と呼びます。純粋仮想関数に対して実装を与えることもオーバーライドの一種です（上の例はこのケース）。オーバーライドするときは、同じ名前、同じ引数、同じ戻り値を持つ関数を定義すれば良いのですが、どれか1つでも間違えるとオーバーライドではなく新規の関数が作られてしまうので注意してください。override キー

* 4 クラスが設計図だとすると、インスタンスは設計図に基づいて作った制作物ということになります。C/C++ では設計図はメモリ上に置かれるものではなくコンパイラがコンパイル時に使うものですが、プログラミング言語によっては設計図をメモリ上に配置することもあります。

ワードを書きおくことで、オーバーライドしたつもりが新規の関数定義になってしまったとき、コンパイルがエラーになって気付けます (**コラム 4.2 コンパイルエラーはお友達**)。

2つの子クラスには一見するとコンストラクタの定義がありませんが、実は `using PixelWriter::PixelWriter;` がコンストラクタ定義の代わりになっています。この `using` 宣言によって、親クラスのコンストラクタをそのまま子クラスのコンストラクタとして使うことができます。この効果は子クラスのインスタンスを生成するときに実感できます。

リスト 4.8 PixelWriter クラスの使い方 (main.cpp)

```
extern "C" void KernelMain(const FrameBufferConfig& frame_buffer_config) {
    switch (frame_buffer_config.pixel_format) {
        case kPixelRGBResv8BitPerColor:
            pixel_writer = new(pixel_writer_buf)
                RGBResv8BitPerColorPixelWriter{frame_buffer_config};
            break;
        case kPixelBGRResv8BitPerColor:
            pixel_writer = new(pixel_writer_buf)
                BGRResv8BitPerColorPixelWriter{frame_buffer_config};
            break;
    }

    for (int x = 0; x < frame_buffer_config.horizontal_resolution; ++x) {
        for (int y = 0; y < frame_buffer_config.vertical_resolution; ++y) {
            pixel_writer->Write(x, y, {255, 255, 255});
        }
    }
    for (int x = 0; x < 200; ++x) {
        for (int y = 0; y < 100; ++y) {
            pixel_writer->Write(x, y, {0, 255, 0});
        }
    }
    while (1) __asm__("hlt");
}
```

リスト 4.9 pixel_writer ポインタ変数の定義 (main.cpp)

```
char pixel_writer_buf[sizeof(RGBResv8BitPerColorPixelWriter)];
PixelWriter* pixel_writer;

extern "C" void KernelMain(const FrameBufferConfig& frame_buffer_config) {
    ...
}
```

リスト 4.8 は定義したクラスを実際に使っている部分です。この中で登場する `pixel_writer` と `pixel_writer_buf` は **リスト 4.9** のように定義されたグローバル変数です。このプログラムの前半では、ピクセルのデータ形式 `frame_buffer_config.pixel_format` に基づいて2つの子クラスの適する方のインスタンスを生成し、そのインスタンスへのポインタを `pixel_writer` 変数に設定します。後半では `pixel_writer` を使って実際に画面を塗りつぶしたあと、 200×100 の緑の四角を描いています。

ここで**配置 new** の話をしなければいけません。2つのインスタンス生成で使っている `new` 演算子に

注目すると、一般的な使い方とは異なり new が引数を取っています。これが配置 new です。一般の new を説明した後、配置 new を説明することにします。

一般の new の使い方は new クラス名です。引数は取りません。一般の new は指定したクラスのインスタンスを**ヒープ領域**に生成します。ヒープ領域とは関数の実行が終了しても破棄されない領域であり、関数を抜けても生存する必要があるデータは new 演算子(C 言語では malloc())で確保することになっています。ちなみに、new 演算子や malloc() を使わず普通に定義した変数はスタック領域に配置されます。スタック領域に配置された変数は、その変数が定義されたブロックを抜けるときに破棄されます。

new 演算子と malloc() 関数の大きな違いはクラスのコンストラクタが呼び出されるかどうかです。malloc() でメモリを確保した場合はコンストラクタが呼ばれず、中身にはゴミデータが入った状態のメモリ領域が返されます。一方、new 演算子はメモリ領域を確保した後にコンストラクタを呼び出します。正確に言うと、コンパイラが new 演算子を発見すると、その部分にコンストラクタを呼び出すための命令を自動的に挿入するという仕組みです。クラスのコンストラクタをプログラマが明示的に呼び出す方法はないため、クラスのインスタンスを作るには new 演算子を使うしかありません。

一般の new は変数をヒープ領域に確保すると言いました。実は、これは OS がメモリを管理できるようになって初めて可能です。new 演算子がメモリを確保する際は OS に対してメモリ確保要求を出すのですが、我々の OS はまだメモリの管理ができていませんからその要求に答えられません。しかし、先に述べたように、クラスのコンストラクタを呼び出すには new 演算子を使うほかありません。メモリ管理ができるようになるまで C++ のクラスは使えないのでしょうか？ いいえ、そんなことはありません。

メモリ管理がないときでもクラスのインスタンスを作りたい人の強い味方、それが**配置 new**です。配置 new は一般の new とは違ってメモリ領域の確保を行いません。その代わり、引数に指定したメモリ領域の上にインスタンスを生成します。そのメモリ領域に対してコンストラクタを呼び出すわけです。OS にメモリ管理機能が無くても、プログラミング言語にもともと備わっている「配列」を使えば好きな大きさのメモリ領域を確保することができますから、配列と配置 new を組み合わせることでクラスのインスタンス生成が可能になるのです！

リスト 4.10 配置 new の演算子の定義 (main.cpp)

```
void* operator new(size_t size, void* buf) {
    return buf;
}

void operator delete(void* obj) noexcept {
}
```

配置 new は <new> をインクルードするか自分で定義する必要があります。配置 new の実装がいかに簡単かを知ってもらいたかったので**リスト 4.10**のように自分で定義しました。C++ では operator キーワードを使うと演算子を定義することができます。一般の new の引数は size だけなのですが、配置 new では追加の引数があり、そこにメモリ領域へのポインタが渡されます。配置 new はメモリ領域自体を確保する必要はありませんから、受け取ったメモリ領域へのポインタ buf をそのまま返すだけで完了です。

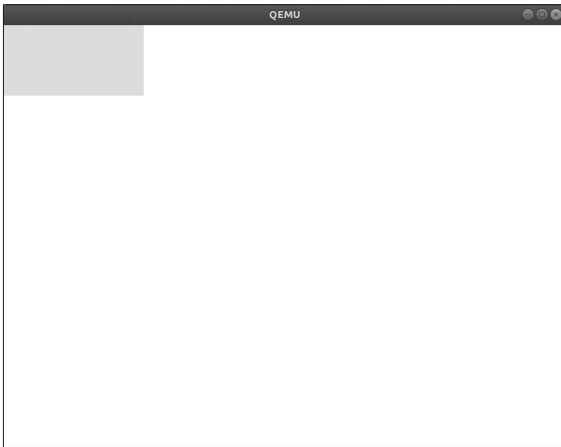
operator delete の方は配置 new とは関係ない、普通の delete 演算子なのですが、これがな

いとリンク時にエラーとなるので仕方なく定義しています。~PixelWriter() がこの演算子を要求するようです。pixel_writer_buf に対して new こすれど、delete はしませんので定義不要かなと思ったのですが……。C++ 難しい。

さて、そんな便利な配置 new を使って生成した子クラスのインスタンスへのポインタを、親クラス PixelWriter を指すポインタ型である pixel_writer 変数に代入します。継承関係がある2つのクラスでは、子クラスのポインタを親クラスのポインタに代入することで、あたかも親クラスであるかのように子クラスを操作することができます。今回の例では、親クラスで定義されたインタフェース PixelWriter::Write() を介して子クラスで実装された機能呼び出します。この仕組みを実現する vtable という仕組みについては「**4.4 vtable**」で紹介します。

後半では pixel_writer を使って画面を塗りつぶします。まずは白で全体を塗りつぶし、次に緑で四角を描きます。WritePixel() のときと違い、pixel_writer->Write() の内部には条件分岐がないことに注目してください。50万回の条件分岐は、KernelMain() 中にあるたった1回の switch 文に姿を変えたのです。ソースコード行数は少し増えてしまいましたが、筆者は意味的にきれいなプログラムが好きなので、わざわざページを割いて今のやり方を紹介しました。

図 4.3 PixelWriter クラスを使って緑色の四角を描いた



改造したプログラムを実行してみましょう。いつものように QEMU で実行してみると画面に緑の四角が表示されるはずです (図 4.3)。

コラム 4.2 コンパイラエラーはお友達

コンパイラがエラーや警告 (warning) を出すとき、皆さんはどんなことを思うでしょうか。怖いかな嬉しいかというと、怖いと感じる人もいでしょう。なんだかよく分からないけどコンパイラに怒られるというのは気持ちよくないですね。エラーメッセージは英語ですから、それの意味が分かりにくくなっている一因ですね。

でも筆者は、コンパイラのエラーや警告とはぜひ仲良くしてほしいと思っています。なぜなら、それらはプログラムの間違いを教えてくれるありがたい存在だからです。コンパイル時に間違いに気づけることは、いざ実行した時にエラーが出てうまく行かないことより嬉しいのです。バグはなるべく早い段階で見つけたほうが直すのが簡単ですから。

```
$ clang++-10 compiler_error.cpp
compiler_error.cpp:3:9: error: cannot initialize a variable of type 'char *'
with an lvalue of type 'char'
  char* p = arr[7];
          ^~~~~~
```

上記のエラーメッセージの1行目は `char *` 型の変数を `char` 型の値で初期化できないという意味の英文です。1行目の先頭にある `compiler_error.cpp:3:9:` は、エラーが検出されたファイルの名前、行番号、列番号が並んでいます。2行目はエラーが検出された行の実際の内容を示します。

コンパイラのエラーや警告と仲良くなるには、その内容をちゃんと読んで理解しようとするのが大切です。といっても、「英語だから読めない!」という方も多いかと思います。残念ながら、プログラミングの世界では英語が標準になっており、基本的には英語で読むしかありません (いくつかのコンパイラの特定のバージョンでは日本語メッセージに対応しているものもありますが)。幸いなことにファイル名、行番号、列番号が分かれば、英語が理解できずとも指摘されている箇所を見つけるのは簡単です。

問題の箇所を少し眺めてもエラーの原因が分からなければ、いよいよ英文を読解する必要があります。これは骨の折れる作業ですが、エラーメッセージは一般的な英文と違ってパターン化されているため、通常の英語学習ほどは難しくありません。上記のエラー例では `'char *'` と `'char'` の部分が変わるだけでその他は固定だということです。さらに、普段良く会えるエラーは限られていますから、出会ったエラーから1つずつ意味を覚えていけばいいのです。意味が分からないエラーに出会ったら、インターネットで検索したり、知人や本書のサポートサイトに質問するといいでしょう。

エラーや警告が「よく分からない怖いもの」ではなく「間違いを理路整然と報告する親切なやつ」だと思えるようになれば、開発の強力な味方になります。コンパイラが言っていることをよく聞き、理解し、楽しい開発生活を送ってください!

4.4 vtable

前節の例では2つの子クラスのどちらかのインスタンスを親クラスのポインタに代入し、親クラスのポインタを介して子クラスの機能呼び出ししました。この、親クラスのポインタを介して子クラスの機能呼び出す仕組みの実現方法はコンパイラに任されていますが、我々が使っている Clang では vtable を用いて実現しています (vtable 方式が最もよくあるやり方だと思います)。vtable は一言で表せば仮想関数 (virtual function) のポインタ表 (table) のことです。vtable はクラス毎に存在するもので、インスタンス毎ではありません。

リスト 4.11 クラス階層

```
class Base {
public:
    virtual ~Base();
    void Func();
    virtual void VFunc1();
    virtual void VFunc2();
};

class Sub : public Base {
public:
    ~Sub();
    void Func(); // override キーワードを付けるとエラー
    void VFunc1() override;
private:
    int x;
};
```

リスト 4.11 に vtable を説明するための例となるクラス階層を示します。メンバ関数の定義は載せていませんが、適当に定義されていると思ってください。

クラスのメンバ関数を定義する際に virtual キーワードを付けることで、その関数は仮想関数ということになり、vtable に追加されます。一度 vtable に追加されると、その子クラスでは特に virtual キーワードを付けなくてもその関数は仮想関数として扱われます (オーバーライドしたとしても)。したがって Base クラスとその子クラス Sub の vtable は表 4.4、表 4.5 のようになります。virtual キーワードを親クラスで付加していない Base::Func は仮想関数になりません。したがって子クラスで同じ名前と同じ引数の関数を定義したとしても、それは親クラスの関数をオーバーライドすることにはなりません。オーバーライドはあくまで仮想関数専用の機能なのです。したがって Sub::Func に override キーワードを付けるとエラーとなります。

表 4.4 Base クラスの vtable

関数名	値
~Base	Base::~~Base
VFunc1	Base::VFunc1
VFunc2	Base::VFunc2

表 4.5 Sub クラスの vtable

関数名	値
~Sub	Sub::~~Sub
VFunc1	Sub::VFunc1
VFunc2	Base::VFunc2

仮想関数を 1 つ以上含むクラスのインスタンスの先頭には vtable へのポインタが埋め込まれます。すなわち Base クラスのインスタンスには Base::vtable へのポインタが、Sub クラスのインスタンスには Sub::vtable へのポインタが埋め込まれることになります。埋め込まれたポインタはプログラマからは利用できませんが、コンパイラが関数呼び出しのときに参照します。次のように Sub のインスタンスを Base として扱う場合でも、vtable 自体は Sub::vtable が利用されるため、意図した関数が呼び出されることになります。

リスト 4.12 vtable 経由での関数呼び出し

```
Base* base_ptr = new Sub;
base_ptr->Func(); // vtable を使わない呼び出し。Base::Func が呼ばれる。
base_ptr->VFunc1(); // vtable 経由での呼び出し。Sub::VFunc1 が呼ばれる。
base_ptr->VFunc2(); // vtable 経由での呼び出し。Base::VFunc2 が呼ばれる。
delete base_ptr; // vtable 経由での呼び出し。Sub::~~Sub が呼ばれる。
```

4.5 ローダを改良する (osbook_day04d)

仮想関数を使った描画プログラムは一見するとうまく動いているようですが、実はカーネルの読み込み処理にバグがあります。カーネルを読み込むためのメモリを確保する部分で、メモリの大きさを計算する処理が間違っているのです。

本来、ローダはカーネルファイルに記録された情報を元にメモリの大きさを決める必要があります。kernel.elf のフォーマットである ELF 形式では、メモリへの読み込みに関する情報はプログラムヘッダという部分に記載されています。プログラムヘッダを確認するには readelf -l コマンドを使います。

```
$ cd $HOME/workspace/mikanos/kernel
$ readelf -l kernel.elf
```

```
Elf ファイルタイプは EXEC (実行可能ファイル) です
Entry point 0x101020
There are 5 program headers, starting at offset 64
```

プログラムヘッダ：

タイプ	オフセット ファイルサイズ	仮想 Addr メモリサイズ	物理 Addr フラグ 整理
PHDR	0x0000000000000040 0x0000000000000118	0x0000000000100040 0x0000000000000118	R 0x8
LOAD	0x0000000000000000 0x00000000000001b0	0x0000000000100000 0x00000000000001b0	R 0x1000
LOAD	0x0000000000000100 0x0000000000000199	0x0000000000101000 0x0000000000000199	R E 0x1000
LOAD	0x0000000000000200 0x0000000000000000	0x0000000000102000 0x0000000000000018	RW 0x1000
GNU_STACK	0x0000000000000000 0x0000000000000000	0x0000000000000000 0x0000000000000000	RW 0x0

セグメントマッピングへのセクション：

セグメントセクション ...

```
00
01      .rodata
02      .text
03      .bss
04
```

プログラムヘッダに書いてある PHDR や LOAD、GNU_STACK などとはセグメントと呼ばれるまとまりです。セグメントは ELF 形式のファイルの一部分のことで、セグメントごとにファイル上でのオフセットと大きさ、メモリ上でのオフセットと大きさ、さらにメモリ属性を持ちます。これらのセグメント情報を基に、ローダはファイルをメモリ上に読み込みます。

ローダが見るべきなのは主に LOAD というセグメントです。LOAD セグメントはその名の通り、ローダが読み込み処理のために参照するセグメントです。ローダは、LOAD セグメントに記載された情報に従って、ファイルのデータをメモリにコピーします。このような処理をロード (load) といいます。

表 4.6 3つの LOAD セグメントの情報

オフセット	仮想 Addr	ファイルサイズ	メモリサイズ	フラグ
0x0000	0x100000	0x01b0	0x01b0	R
0x1000	0x101000	0x0199	0x0199	R E
0x2000	0x102000	0x0000	0x0018	RW

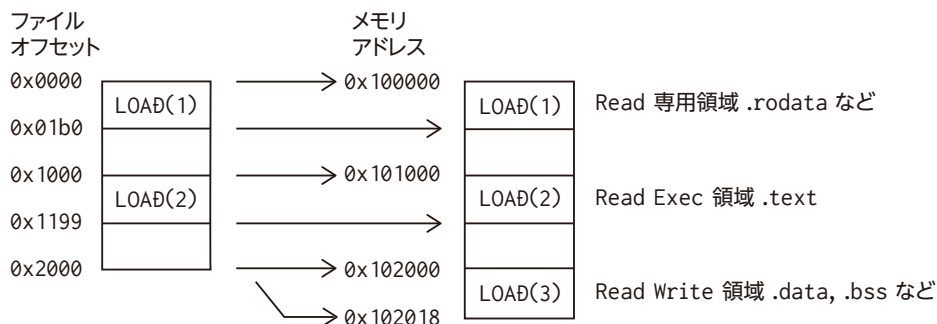
3つの LOAD セグメントの情報をまとめて表 4.6 に示します。表の中の「仮想 Addr」は単なるメモリアドレスだと思ってください。「第 19 章 ページング」でページングを学ぶと「仮想」の意味、それから仮想と物理の違いが分かるようになります。各フラグは R=Readable(読み取り可)、W=Writable(書き込み可)、E=Executable(実行可)の意味です。主にセキュリティを向上させる目的で、メモリに属性を付けて管理するための情報です。しかし、今は無視で大丈夫です。

この表を見ると、仮想 Addr は 0x100000 から始まっていること、3つめの LOAD セグメントだけはファイル上のサイズとメモリ上のサイズが違うこと、などが分かります。仮想 Addr が 0x100000 から始まるのは、もちろん ld, lld のオプションに `--image-base 0x100000` を指定したからです。では、3つめの LOAD セグメントだけ、ファイルとメモリのサイズが違うのはなぜでしょうか？

それは3つめの LOAD セグメントが .bss セクションを含んでいるからです。.bss セクションとは、通常、初期値なしのグローバル変数が配置されるセクションです。例えば main.cpp で定義したグローバル変数 pixel_writer_buf や pixel_writer は初期値がありませんから、.bss セクションに配置されることになります。初期値がないということは、初期値をファイルに記録しておく必要がないということです。そのため、ファイル上での大きさは 0 で、メモリ上では大きさを持つ変数となります。これが、LOAD セグメントのファイルサイズよりメモリサイズが大きくなる理由です。

3つの LOAD セグメントの情報を合わせて考えると、図 4.4 のようにメモリ上へ読み込めば良いことが分かります。

図 4.4 LOAD セグメントをコピーする



さて、ここまででプログラムヘッダに記録された LOAD セグメントについての数値の意味が分かりました。次に、ローダが `kernel.elf` ファイルの中に書かれたプログラムヘッダを探し出す方法を説明します。そのために、ELF 形式について少し詳しく説明します。もっと詳しく知りたい場合は「リンク・ローダ実践開発テクニック」(参考文献 [8]) が参考になるでしょう。

図 4.5 ELF ファイルの構造

ファイルヘッダ
プログラムヘッダ
セクション本体 .text .data .rodata .bss .dynamic など
セクションヘッダ

ELF 形式のファイルは大きく分けてファイルヘッダ、プログラムヘッダ、セクション本体、セクションヘッダに分けられます (図 4.5)。多くの ELF ではこの順で並んでいることが多いようです。ファイルヘッダはファイル全体の情報、例えば ELF ファイルのビット数、対象アーキテクチャ、プログラムヘッダやセクションヘッダの開始位置やサイズなどが記載されています。プログラムヘッダは先程見たように、ローダ向けの情報が書いてあります。セクションヘッダはリンカ向けの情報で、各セクションのファイル中での位置とサイズ、セクションの属性などが書いてあります。

リスト 4.13 64ビット ELF のファイルヘッダ (elf.hpp)

```
#define EI_NIDENT 16

typedef struct {
    unsigned char e_ident[EI_NIDENT];
    Elf64_Half    e_type;
    Elf64_Half    e_machine;
    Elf64_Word    e_version;
    Elf64_Addr    e_entry;
    Elf64_Off     e_phoff;
    Elf64_Off     e_shoff;
```

```

Elf64_Word    e_flags;
Elf64_Half    e_ehsize;
Elf64_Half    e_phentsize;
Elf64_Half    e_phnum;
Elf64_Half    e_shentsize;
Elf64_Half    e_shnum;
Elf64_Half    e_shstrndx;
} Elf64_Ehdr;

```

64 ビット用 ELF のファイルヘッダは**リスト 4.13** に示す構造になっています。この中で `e_phoff` がプログラムヘッダのファイルオフセットを表す項目です。ここで示されるファイル領域を読めばプログラムヘッダを取得することができます。プログラムヘッダは配列になっていて、`e_phentsize` は要素 1 つの大きさを、`e_phnum` は要素数を表します。

リスト 4.14 64 ビット ELF のプログラムヘッダの要素 (elf.hpp)

```

typedef struct {
    Elf64_Word  p_type;    // PHDR、LOAD などのセグメント種別
    Elf64_Word  p_flags;   // フラグ
    Elf64_Off   p_offset;  // オフセット
    Elf64_Addr  p_vaddr;   // 仮想 Addr
    Elf64_Addr  p_paddr;
    Elf64_Xword p_filesz;  // ファイルサイズ
    Elf64_Xword p_memsz;   // メモリサイズ
    Elf64_Xword p_align;
} Elf64_Phdr;

```

プログラムヘッダの各要素は**リスト 4.14** に示す構造体になっています。**表 4.6** の項目名との対応を、コメントで書いておきました。

さて、ここまで情報が集まれば実際のローダの改造を始めることができます。改造の方針は次のようにしてみます。

1. カーネルファイル `kernel.elf` をいきなり最終目的地に読み込むのではなく、一時領域に読み込むようにする。
2. 一時領域に読み込んだカーネルファイルのプログラムヘッダを読み、最終目的地の番地の範囲を取得する。
3. 一時領域から最終目的地へ、LOAD セグメントをコピーし、一時領域を削除する。

最終目的地は `ld.lld` の `--image-base` オプションで指定した値で、本書の中では一貫して `0x100000` です。ですから、2 番目の処理は不要に思えるかもしれませんが、実は意味がある処理です。1 つはコピーすべきバイト数を計算するため、もう 1 つは `--image-base` の値を変えてもちゃんと動くようにするためです。

さて、この方針に沿って実際にコードを書きます。まずはカーネルファイルを一時領域に読み込む処理を**リスト 4.15** に示します。

リスト 4.15 カーネルファイルの読み込み (Main.c)

```
EFI_FILE_INFO* file_info = (EFI_FILE_INFO*)file_info_buffer;
UINTN kernel_file_size = file_info->FileSize;

VOID* kernel_buffer;
status = gBS->AllocatePool(EfiLoaderData, kernel_file_size, &kernel_buffer);
if (EFI_ERROR(status)) {
    Print(L"failed to allocate pool: %r\n", status);
    Halt();
}
status = kernel_file->Read(kernel_file, &kernel_file_size, kernel_buffer);
if (EFI_ERROR(status)) {
    Print(L"error: %r", status);
    Halt();
}
}
```

`gBS->AllocatePool()` は初めて登場する関数で、ここではカーネルファイルを読み込むための一時領域を確保するのに使っています。この関数は `gBS->AllocatePages()` と同様にメモリ領域を確保する関数ですが、ページ単位ではなくバイト単位でメモリを確保できます。その代わり、確保する場所を指定するような機能はありません。一時的な領域にカーネルファイルを読み込みたいだけです、位置指定の機能は不要です。

`gBS->AllocatePool()` が成功すると、`kernel_buffer` には確保されたメモリ領域の先頭アドレスが格納されます。そのアドレスを `kernel_file->Read` に指定することで、カーネルファイルの内容をすべて一時領域へ読み込みます。

リスト 4.16 : コピー先のメモリ領域の確保 (Main.c)

```
Elf64_Ehdr* kernel_ehdr = (Elf64_Ehdr*)kernel_buffer;
UINT64 kernel_first_addr, kernel_last_addr;
CalcLoadAddressRange(kernel_ehdr, &kernel_first_addr, &kernel_last_addr);

UINTN num_pages = (kernel_last_addr - kernel_first_addr + 0xffff) / 0x1000;
status = gBS->AllocatePages(AllocateAddress, EfiLoaderData,
                           num_pages, &kernel_first_addr);
if (EFI_ERROR(status)) {
    Print(L"failed to allocate pages: %r\n", status);
    Halt();
}
}
```

次に、最終目的地の番地の範囲を取得し、それに応じて最終目的地のメモリ領域を確保する処理を **リスト 4.16** に示します。最終目的地の番地の範囲とは、具体的には `0x100000` から始まるアドレスの範囲のことです。`CalcLoadAddressRange()` が範囲を計算し、`kernel_first_addr` 変数に開始アドレスを、`kernel_last_addr` 変数に終了アドレスを設定します。それらを使って必要なメモリ領域の大きさ (`num_pages`) を計算し、メモリを確保します。

リスト 4.17 CalcLoadAddressRange() の実装 (Main.c)

```

void CalcLoadAddressRange(Elf64_Ehdr* ehdr, UINT64* first, UINT64* last) {
    Elf64_Phdr* phdr = (Elf64_Phdr*)((UINT64)ehdr + ehdr->e_phoff);
    *first = MAX_UINT64;
    *last = 0;
    for (Elf64_Half i = 0; i < ehdr->e_phnum; ++i) {
        if (phdr[i].p_type != PT_LOAD) continue;
        *first = MIN(*first, phdr[i].p_vaddr);
        *last = MAX(*last, phdr[i].p_vaddr + phdr[i].p_memsz);
    }
}

```

CalcLoadAddressRange() の実装をリスト 4.17 に示します。この関数はカーネルファイル内のすべての LOAD セグメント (p_type が PT_LOAD であるセグメント) を順にめぐり、アドレス範囲 (first、last) を更新していきます。

first の計算は、要するに複数の値の中から最小値を見つける計算です。複数の値 (a1、a2、……、aN とする) から最小値を見つけるには次のようなやり方があります。最大値を見つけるのもほとんど同じやり方です。

1. 計算用の変数 (x とする) を準備し、a1……aN には絶対に現れない大きな値で初期化する。
2. a1……aN を 1 つずつ見て (ai とする)、ai と x の値を比べる。
3. ai と x の値の小さい方を x に代入する。

phdr はプログラムヘッダの配列を指すポインタです。phdr[i] は i 番目のプログラムヘッダを表します。p_type はそのセグメントの種別を表しますから、それが LOAD セグメントではないときは処理をスキップします。

この関数の処理が終わると、*first の値は先頭にある LOAD セグメントの p_vaddr の値になります。*last の値は末尾にある LOAD セグメントの p_vaddr+p_memsz の値になります。すべての LOAD セグメントを 1 つのかたまりとみなして、その先頭と末尾のアドレスを求めている、とも言えます。

リスト 4.18 LOAD セグメントのコピー (Main.c)

```

CopyLoadSegments(kernel_ehdr);
Print(L"Kernel: 0x%0lx - 0x%0lx\n", kernel_first_addr, kernel_last_addr);

status = gBS->FreePool(kernel_buffer);
if (EFI_ERROR(status)) {
    Print(L"failed to free pool: %r\n", status);
    Halt();
}

```

さて、ここまでの説明で、最終目的地の番地の範囲を計算し、実際にメモリ領域を確保することができました。次は一時領域から最終目的地へ LOAD セグメントをコピーします。リスト 4.18 がその処理をしている部分です。といっても、ただ CopyLoadSegments() を呼び出しているだけですが。その後に行っている gBS->FreePool() は一時領域を解放するためのものです。

リスト 4.19 CopyLoadSegments() の実装 (Main.c)

```

void CopyLoadSegments(Elf64_Ehdr* ehdr) {
    Elf64_Phdr* phdr = (Elf64_Phdr*)((UINT64)ehdr + ehdr->e_phoff);
    for (Elf64_Half i = 0; i < ehdr->e_phnum; ++i) {
        if (phdr[i].p_type != PT_LOAD) continue;

        UINT64 segm_in_file = (UINT64)ehdr + phdr[i].p_offset;
        CopyMem((VOID*)phdr[i].p_vaddr, (VOID*)segm_in_file, phdr[i].p_filesz);

        UINTN remain_bytes = phdr[i].p_memsz - phdr[i].p_filesz;
        SetMem((VOID*)(phdr[i].p_vaddr + phdr[i].p_filesz), remain_bytes, 0);
    }
}

```

CopyLoadSegments() の中身を**リスト 4.19** に示します。この関数は、p_type == PT_LOAD であるセグメントに対して、次の2つの処理を行います。

1. segm_in_file が指す一時領域から p_vaddr が指す最終目的地へデータをコピーする (CopyMem())。
2. セグメントのメモリ上のサイズがファイル上のサイズより大きい場合 (remain_bytes > 0)、残りを0で埋める (SetMem())。

最終目的地へすべての LOAD セグメントをコピーし終わったら、コピー先からエントリポイントを取得します (**リスト 4.20**)。これでローダの改良は完了です。

リスト 4.20 エントリポイントの取得 (Main.c)

```

UINT64 entry_addr = *(UINT64*)(kernel_first_addr + 24);

```

第 5 章

文字表示と コンソールクラス

この章では OS 本体から画面に文字を表示する方法を学びます。
画面に文字が表示できるようになるといろいろな情報を出せるよ
うになり、OS 開発がしやすくなります。

5.1 文字を書いている (osbook_day05a)

Linux は起動時にコンソールにたくさんの起動メッセージが出ます (図 5.1)。筆者はこのメッセージを眺めるのが割と好きです。眺めていると、OS はたくさんの部品から構成されているんだなあと感じられるからです。画面いっぱい大きな画像 (スプラッシュと呼ぶ) を表示して起動メッセージを隠するのが最近の流行りですが、でもやっぱり OS の起動と言えばメッセージを垂れ流すことに筆者は憧れます。皆さんはどうでしょうか。

図 5.1 Ubuntu の起動メッセージ

```
[ 5.329910] EXT4-fs (dm-0): re-mounted. Opts: errors=remount-ro
[ OK ] Started Remount Root and Kernel File Systems.
      Starting Load/Save Random Seed...
[ 5.379505] iscsi: registered transport (iser)
[ 5.422304] RPC: Registered named UNIX socket transport module.
[ 5.422704] RPC: Registered udp transport module.
[ 5.423106] RPC: Registered tcp transport module.
[ 5.423461] RPC: Registered tcp NFSv4.1 backchannel transport module.
[ OK ] Started Journal Service.
[ OK ] Started Load Kernel Modules.
[ OK ] Started Create list of required static device nodes for the current kernel
.
[ OK ] Mounted RPC Pipe File System.
[ OK ] Started Load/Save Random Seed.
[ OK ] Started LVM2 metadata daemon.
      Starting Create Static Device Nodes in /dev...
      Starting Apply Kernel Variables...
      Mounting Kernel Configuration File System...
      Mounting FUSE Control File System...
      Starting Flush Journal to Persistent Storage...
[ OK ] Mounted Kernel Configuration File System.
[ OK ] Mounted FUSE Control File System.
[ OK ] Started Create Static Device Nodes in /dev.
      Starting udev Kernel Device Manager...
```

ピクセルに色を描く方法は前章までで分かりました。今は適当に塗りつぶしているだけですが、頑張ると文字の形をした絵を描くこともできるはず。文字を画面に表示するには、文字の線の部分だけを塗りそれ以外を塗らない (あるいは背景色で塗る) という処理をします。これは簡単にできるので実際にやってみましょう。まずは「A」を描くために黒く塗る部分を 1、塗らない部分を 0 としたデータを用意します (リスト 5.1)。

リスト 5.1 A のフォントデータ (main.cpp)

```
const uint8_t kFontA[16] = {
    0b00000000, //
    0b00011000, // **
    0b00011000, // **
    0b00011000, // **
    0b00011000, // **
    0b00100100, // * *
    0b00100100, // * *
    0b00100100, // * *
    0b00100100, // * *
    0b01111110, // *****
}
```

```

0b01000010, // * *
0b01000010, // * *
0b01000010, // * *
0b11100111, // *** ***
0b00000000, //
0b00000000, //
};

```

このような字形を表すデータを**フォント**と呼びます。プログラミング的にはフォントの大きさが2のべき乗だと扱いやすいため、横8ピクセル、縦16ピクセルでフォントを用意してみました。1ピクセルに3バイトが必要な色情報とは異なり、フォントは塗るか塗らないかを表せば十分なため、1ピクセルは1ビットで表せます。すなわち16個の8ビット整数で1文字を表せることになりますから、フォントは1文字あたり `uint8_t` の16要素の配列としました。ちなみに、`0b11100111` などの先頭に `0b` が付いた数値は2進数を意味します（「**1.2 結局何をやったのか**」を参照）。

リスト 5.2 フォント描画関数 (main.cpp)

```

void WriteAscii(PixelWriter& writer, int x, int y, char c, const PixelColor& color) {
    if (c != 'A') {
        return;
    }
    for (int dy = 0; dy < 16; ++dy) {
        for (int dx = 0; dx < 8; ++dx) {
            if ((kFontA[dy] << dx) & 0x80u) {
                writer.Write(x + dx, y + dy, color);
            }
        }
    }
}

```

リスト 5.2 は、作成したフォントデータを利用して1文字を描画するための `WriteAscii()` です。この関数は引数として ASCII コードを受け取り、その文字に対応するフォントを指定された位置に描画します。

外側の `for` 文ではまず縦方向について回し、内側の `for` 文では横方向について回しています。したがって、最初に一番上のピクセルを横方向に描画し、次に2本目のピクセルを横方向に描画し、次に3本目の……というように、横方向に8ピクセルを描画することを16回繰り返してフォントを描画します。

フォントデータの横1行分のデータは8ビット整数です。8ビット整数の最大ビット（ビット7）が文字の左端に、最小ビット（ビット0）が文字の右端に該当します。ループ変数の `dx` が0のときを左端、7のときを右端に対応させることにして、フォントデータの該当するビットが1かどうかを検査する式（1だったときに真になる式）は `(kFontA[dy] << dx) & 0x80u` ということになります。ビット演算が少し分かりにくいかもしれませんが、`kFontA[dy]` が仮に `0b11000001=0xc1` の場合のビット演算の動作を表 **5.1** に示します。

表 5.1 0xc1 に対するビット演算

dx	0xc1u << dx	(0xc1u << dx) & 0x80u	真偽
0	00000000 11000001	0x80	真 (塗る)
1	00000001 10000010	0x80	真 (塗る)
2	00000011 00000100	0	偽 (塗らない)
⋮	⋮	⋮	⋮
7	01100000 10000000	0x80	真 (塗る)

C++ では 0 が偽、非 0 が真を表しますから、表の $(0xc1u \ll dx) \& 0x80u$ の値が 0 になるとき `writer.Write()` は実行されず (= ピクセルは塗られず)、0x80 になるとき実行される (= ピクセルが塗られる) ことになります。引数 `color` で塗る色を指定できます。

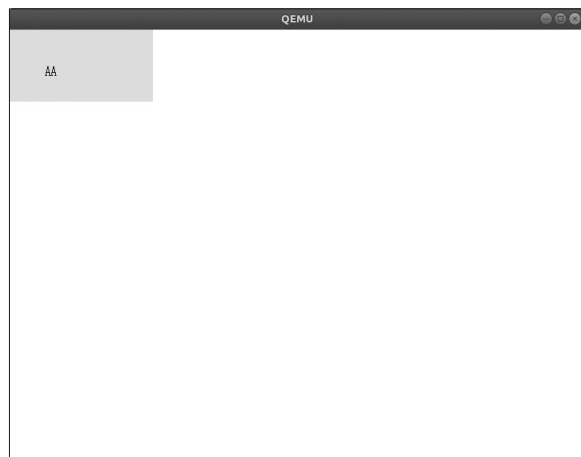
リスト 5.3 フォント描画関数を使う (main.cpp)

```
WriteAscii(*pixel_writer, 50, 50, 'A', {0, 0, 0});
WriteAscii(*pixel_writer, 58, 50, 'A', {0, 0, 0});
```

このフォント描画関数を使うコードを `KernelMain()` に付け足しました (リスト 5.3)。`WriteAscii()` は内部でピクセルを描画する必要があるため、引数として `PixelWriter` クラスへの参照を要求します。ここはポインタを受け取るようにもできますが、そうではなく参照にしたのは理由があります。参照とポインタの使い分けは「[コラム 5.1 参照とポインタ](#)」にまとめました。

`WriteAscii()` は `RGBResv8BitPerColorPixelWriter` などの実装ではなく `PixelWriter` というインターフェースに依存することになります。インターフェースと実装を分離させ、インターフェースを介してモジュール間を結合させるやり方は、大きなプログラムを作るためのモジュール分割の基本です。`WriteAscii()` の実装はピクセルの具体的な描画方法とは切り離されるため、ピクセルの描画方法を変えても `WriteAscii()` 側を変更する必要がありません。

図 5.2 フォントを描画してみた



さて、フォント描画機能を追加したカーネルを QEMU で起動すると図 5.2 のようになりました。文字が増えれば楽しいことができそうですね。

コラム 5.1 参照とポインタ

C の時代からあるポインタと C++ になって導入された参照は、他のオブジェクト（変数など）を指し示すという点で本質的に同じ機能です。例えば次のプログラムで、変数 a と b の最終的な値は両方とも 42 になります。

```
int a = 1, b = 2;
int& ra = a; // ra は a を指し示す参照
int* pb = &b; // pb は b を指し示すポインタ
ra = 42;
*pb = 42;
```

コンパイラが出力する機械語としても参照とポインタは多くの場合同じになります。結局のところ、参照もポインタもメモリアドレスを用いた表現になることには変わりがないのです。

では何が違うかというと、ヌルポインタの作りやすさです。ポインタの場合は単に `nullptr` とか 0 と書くだけでヌルポインタを表現できてしまいます。一方で参照の場合はヌルポインタ（ヌル参照）を作ることはかなり面倒です。無理やり作ってみると次のようになるでしょうか。

```
int* null_ptr = nullptr;
int& null_ref = *null_ptr;
```

ヌルポインタの作りやすさに比べるとヌル参照は非常に作りにくいです。というより作れないと言ってもいいでしょう。なぜなら、ヌル参照を作るにはこの例のようにヌルポインタをデリファレンス（`*null_ptr` のように間接演算子で参照を外すこと）するという未定義の動作を利用するしかないからです。未定義の動作は重大なバグですから、バグのないプログラムにするためにはヌル参照はあってはならないのです。

ヌル参照の作りにくさによって「ヌルを渡されたくない」という意思を示すことができます。ポインタの場合、ヌルポインタを含めて正常値とするかヌルポインタは異常値とするか、どちらもあり得ます。一方で参照の場合、ヌル参照は必ず異常値であり、ヌル参照が渡されることはないという仮定をしまして問題ありません。WriteAscii() は PixelWriter へのポインタではなく参照を受け取っていますが、これにより PixelWriter は必ず存在するものを渡してほしいという意思を明示しているわけです。

5.2 分割コンパイル (osbook_day05b)

開発が進み、気がついたら `main.cpp` は 130 行になっていました。まだ大きすぎるということはありませんが、画面描画に関するコード、フォントに関するコード、メイン関数など、複数のものが 1 つのファイルに入り込んでしまっていて少し混乱気味です。

大きなプログラムを開発する際は、1 つのファイルはなるべく 1 つの関心事に特化させておくことで開発しやすくなります。ファイルを分割することで関心事がそれぞれまとまって把握しやすくなる効果のほかに、変更があったファイルだけをコンパイルし直すことでコンパイル時間を削減できるという利点もあるからです。プログラムが大きくなるにつれてコンパイル時間は延びますから、ファイル分割の効果がその分出やすくなります。ファイル分割のやり方を学ぶのにちょうどいい頃ですのでこの節で説明します。

C++ では、関数やクラスといった単位でファイルに分割することができます。分割したファイルは個別にコンパイルしてオブジェクトファイルに変換^{*1}した後、オブジェクトファイルすべてをリンクすることで 1 つの実行可能ファイル(あるいは共有オブジェクトファイル)を作ります。コンパイルは `.cpp` ファイルの単位で行われますから、ファイルを分割しておくに変更のあったファイルだけをコンパイルした後で全体をリンクしなおせば済むということです。通常、コンパイルの方がリンクよりも時間がかかりますので、ファイル分割をしておくで全体にかかる時間が減ります。

ファイル分割の方法には `.cpp` を分ける以外にもヘッダファイルとして分割しておいて `.cpp` からインクルードする方法もあります。例えば `elf.hpp` や `frame_buffer_config.hpp` のように。ただ、この方法では 1 回のコンパイル時にすべてインクルードされ、まとめてコンパイルされることになります。したがって分割コンパイルによって全体のコンパイル時間を削減するという目的は満たせないことになります。

ということで複数の `.cpp` ファイルに分割することを目指します。現状の `main.cpp` に書かれているコードを次のように分割することにしました。

`graphics.cpp`

画像処理に関するコードを書くファイルです。PixelWriter クラスなどはここに格納します。

`font.cpp`

フォントに関するコードを書くファイルです。WriteAscii() などはここに格納します。

`main.cpp`

メイン関数を格納するファイルです。

ファイルを分割することは利点だけでなく、残念ながら欠点もあります。それはクラス名や関数名などが見える範囲が分かれてしまうことに起因する問題です。ファイルを分割すると他方のファイルに書かれたクラス名や関数名などが見えず、それらを使うことができなくなります。そこで、普通はヘッダファイルに宣言を書いておき、必要な `.cpp` ファイルからヘッダファイルをインクルードするという方法を使います。そうすることで、定義をそれぞれの `.cpp` ファイルに分割させつつも、ファイルをまたがって機能を利用することができるようになります。具体例を見たほうが理解しやすいと思いますので、`graphics.cpp` に関するファイル分割の例を紹介します。

*1 それぞれのコンパイル処理を**翻訳単位**と呼びます。通常、ある翻訳単位は 1 つの `.cpp` ファイルとそこからインクルードされるヘッダファイルから構成されます。

リスト 5.4 graphics.hpp には構造体やクラスの定義を書く

```
#include "frame_buffer_config.hpp"

struct PixelColor {
    uint8_t r, g, b;
};

class PixelWriter {
    ...
};

class RGBResv8BitPerColorPixelWriter : public PixelWriter {
public:
    using PixelWriter::PixelWriter;
    virtual void Write(int x, int y, const PixelColor& c) override;
};

class BGRResv8BitPerColorPixelWriter : public PixelWriter {
public:
    using PixelWriter::PixelWriter;
    virtual void Write(int x, int y, const PixelColor& c) override;
};
```

リスト 5.4 に新しく作ったヘッダファイルを示します。もともと main.cpp にあった PixelColor 構造体や PixelWriter クラス、およびそのサブクラスをこちらに移動してきました。このように、複数の .cpp ファイルで使う可能性があるプログラム部品はヘッダファイルに書いておきます。

RGBResv8BitPerColorPixelWriter のようにメンバ関数を持つクラス^{*2}の場合は、ヘッダファイルにはメンバ関数を含むクラス定義全体を書きます。ただしメンバ関数の本体はヘッダファイルに書いておらず、プロトタイプ宣言だけであることに注目してください。メンバ関数の本体は graphics.cpp の方にあります。メンバ関数の本体をヘッダファイルに書くのが良いか、今回のように .cpp ファイルに書くのが良いかは場合によりけりですが、分割コンパイルの利点を最大に生かすには関数本体はなるべく .cpp ファイルに書くのが良いです。ヘッダファイルに関数本体を書いてしまうと、関数本体を修正するたびにそのヘッダファイルをインクルードしているすべての .cpp ファイルをコンパイルし直す必要があるからです。

リスト 5.5 graphics.cpp に移動したメンバ関数定義

```
#include "graphics.hpp"

void RGBResv8BitPerColorPixelWriter::Write(int x, int y, const PixelColor& c) {
    auto p = PixelAt(x, y);
    p[0] = c.r;
    p[1] = c.g;
    p[2] = c.b;
}
```

*2 C++ における構造体とクラスは、デフォルトの可視属性 (public と private) が異なることを除いて同じものです。しかし、筆者はメンバ関数を持つ場合にクラス、それ以外の場合に構造体として使い分けています。

リスト 5.5に graphics.cpp ファイルの内容の一部を示します。分かりやすさの観点から、ヘッダファイルと .cpp ファイルは組にして作っています。ファイルの内容にある通り、クラス定義の外側でメンバ関数の定義を書くにはクラス名::メンバ名のようにクラス名を前置する必要があります。クラス名を書かず void Write(...) { と書いたのでは、メンバ関数ではなくグローバル関数の定義になってしまいますからね。

リスト 5.6 main.cpp で分割したヘッダファイルを読み込む様子

```
#include "frame_buffer_config.hpp"
#include "graphics.hpp"
#include "font.hpp"
```

リスト 5.6 には、main.cpp で `FramebufferConfig` と `PixelWriter` と `WriteAscii()` を使うために 3 つのヘッダファイルを読み込んでいる様子を示します。ファイル分割の結果それぞれの定義が各ファイルに分散したため、個別に `#include` する必要が生じたのです。

これに合わせて Makefile も改造しましたが、本題とはずれるので本文では説明しません。気になる方はサンプルコード `$HOME/workspace/mikanos/kernel/Makefile` (Git タグ: `osbook_day05b`) を読んでください。

5.3) フォントを増やそう (osbook_day05c)

これまでに A という文字の表示ができました。後はフォントを増やしていけばどんな文字も同じように描けます。ただ、英数字と記号だけでも 100 文字くらいありますからすべて自分で作るのは大変です。OS を作りただけでフォントを作りたいわけではありませんので、ここは既存のものを使うことにします。自由に使えるフォントの 1 つに「東雲フォント」*³ というものがあり、ちょうどいいことに 8 × 16 の英数字フォントが含まれていますのでそれを使うことにします。東雲フォントのデータを加工して扱いやすくしたものを `hankaku.txt` として追加しました。このファイルには **リスト 5.7** のようなデータが 256 文字分入っています。

リスト 5.7 hankaku.txt の中身の一部

```
0x41 'A'
... ..
..@...
..@...
..@@...
..@@...
..@@...
..@...@...
..@...@...
..@...@...
.....@@@@
@.....@
@.....@
@.....@
@.....@
```

さて、hankaku.txt としてフォントファイルが準備できましたのでこれを OS から使えるようにします。やり方はいろいろあります。例えば OS にファイル読み込み機能をつけてフォントファイルを読み込むなんていう方法が考えられます。ただ、これは現時点ではかなり難しい方法です。まずファイル読み込み機能を作るのに必要な前提機能がいろいろ足りないため、それらを含む大規模な開発が必要です。仮にファイル読み込みができるようになったとしても、フォントファイルが読み込めなかった際のエラーメッセージが表示できないことになります。ということでカーネルファイルに直接組み込むことにします。関数の機械語や変数の初期値が kernel.elf に含まれているのと同様にフォントデータも含ませてしまうのです。そうすれば、OS の起動直後からフォントデータを利用することができます。

hankaku.txt のままではプログラムから扱いにくいので、前回用意した kFontA 配列と同じような形式に事前に変換してから埋め込みます。つまり 1 文字あたり 16 バイトのバイナリデータにします。全部で 256 文字分のフォントがありますので $16 \times 256 = 4096$ バイトのデータとなります。変換するためのツールは筆者が作って \$HOME/workspace/mikanos/tools/makefont.py に置きました。これを使うと hankaku.txt を変換して 4096 バイトのバイナリファイルを得られます。

```
$ ../tools/makefont.py -o hankaku.bin hankaku.txt
```

生成された hankaku.bin はリンクするための「のりしろ」となる情報が一切付いてないバイナリ（ベタバイナリとかフラットバイナリと呼びます）ですから、カーネルファイルに組み込むためにはさらに工夫が必要です。objcopy というコマンドを使ってベタバイナリにのりしろを付けることができます。

```
$ objcopy -I binary -O elf64-x86-64 -B i386:x86-64 hankaku.bin hankaku.o
```

これで hankaku.o という、リンクに必要なのりしろ付きのオブジェクトファイルが手に入ります。このファイルを main.o などと一緒にリンクすることで、プログラムの中から変数として見えるようになります。

リスト 5.8 hankaku.o のデータを変数として参照する (font.cpp)

```
extern const uint8_t _binary_hankaku_bin_start;
extern const uint8_t _binary_hankaku_bin_end;
extern const uint8_t _binary_hankaku_bin_size;

const uint8_t* GetFont(char c) {
    auto index = 16 * static_cast<unsigned int>(c);
    if (index >= reinterpret_cast<uintptr_t>(&_binary_hankaku_bin_size)) {
        return nullptr;
    }
    return &_binary_hankaku_bin_start + index;
}
```

* 3 <http://openlab.ring.gr.jp/efont/shinonome/>

リスト 5.8 に、組み込んだフォントデータをプログラムから参照するやり方を示します。_binary から始まる 3 つの変数名は objcopy コマンドにより決められた名前ですから、自由に命名することはできません。ここで定義した GetFont() は、ASCII コードに対応したフォントデータ (1 文字あたり 16 バイト) の先頭アドレスを返します。

3 つの変数宣言に extern が付いていることに注目してください。extern は、どこか他のオブジェクトファイルにある変数を参照しますとコンパイラに伝える効果があります。3 つの変数を font.o の中に生成したいわけではなく、hankaku.o にあるものを参照したいだけですから extern を付けるようにしましょう。(extern 関連の話がよく分からなくても大した問題ではありませんので、気にせず先に進んでくださいね。)

リスト 5.9 hankaku.o のデータを使うように改造した (font.cpp)

```
void WriteAscii(PixelWriter& writer, int x, int y, char c, const PixelColor& color) {
    const uint8_t* font = GetFont(c);
    if (font == nullptr) {
        return;
    }
    for (int dy = 0; dy < 16; ++dy) {
        for (int dx = 0; dx < 8; ++dx) {
            if ((font[dy] << dx) & 0x80u) {
                writer.Write(x + dx, y + dy, color);
            }
        }
    }
}
```

リスト 5.9 に、組み込んだフォントデータを使うように改造した WriteAscii() を示します。GetFont() により文字 c に対応するフォントを取得して描画します。主な改造はフォントの取得方法だけで、for 文の中身は kFontA を font に変更した以外は以前と同じです。

ASCII コードの文字が一通り出力できるようになりましたので記念撮影をしました (図 5.3)。

図 5.3 フォント大集合



5.4 文字列描画と sprintf() (osbook_day05d)

ここまで来ると 1 文字ずつではなく文字列で、欲を言えば printf のように書式付きの文字列で表示したくなってきました。そんなに大変ではありませんので、順を追って実装していきます。

リスト 5.10 文字列描画関数 (font.cpp)

```
void WriteString(PixelWriter& writer, int x, int y, const char* s, const PixelColor& color) {
    for (int i = 0; s[i] != '\0'; ++i) {
        WriteAscii(writer, x + 8 * i, y, s[i], color);
    }
}
```

まずは書式なしの文字列表示に対応させます。これは本当に簡単で、単に for 文で繰り返し WriteAscii() を呼べばいいだけです。リスト 5.10 に文字列を描画するための関数 WriteString() を示します。

リスト 5.11 文字列描画関数を使ってハローワールド (main.cpp)

```
int i = 0;
for (char c = '!'; c <= '~'; ++c, ++i) {
    WriteAscii(*pixel_writer, 8 * i, 50, c, {0, 0, 0});
}
WriteString(*pixel_writer, 0, 66, "Hello, world!", {0, 0, 255});
```

リスト 5.11 にメイン関数から WriteString() を使う例を示します。これを実行すると画面に青で「Hello, world!」と表示されるはずです。

次に書式整形付きの文字列表示に挑戦しましょう。これをやるには printf のように %d などを解釈できる関数を準備する必要があります。簡易的なものであれば自作してもいいかもしれませんが、やはり標準ライブラリに含まれている関数を使う方が楽です。

本書では Newlib という標準 C ライブラリを使うことにします。Newlib は OS が入っていない組み込み機器などでも簡単に使えるように移植性が高い作りになっており、いま作っている OS にも簡単に導入できます。ライブラリは本書で用意した開発環境に含まれているため、すでに皆さんのパソコンの \$HOME/osbook/devenv/x86_64-elf 以下に配置されているはずです。include ディレクトリに各種のヘッダファイルが、lib ディレクトリに各種のライブラリファイルがあります。

標準 C ライブラリに含まれる sprintf() を使って書式付き文字列の表示をやってみます。といっても非常に簡単で、<stdio.h> をインクルードした上でリスト 5.12 のように書くだけです。

リスト 5.12 sprintf() で書式整形 (main.cpp)

```
char buf[128];
sprintf(buf, "1 + 2 = %d", 1 + 2);
WriteString(*pixel_writer, 0, 82, buf, {0, 0, 0});
```

Newlib はできるだけ OS に依存しないように実装されていますが、しかし一部のライブラリ関数、例えば `malloc` や `printf` などはどうしても OS の機能を使う必要があります。動的メモリ管理や入出力がそういった機能の代表ですね。Newlib では OS に依存する部分は `malloc` や `printf` などの標準関数から別の関数に切り出されていて、Newlib を使う人が独自に実装することになっています。したがって、Newlib を使うためにはそれらの切り出された関数群を実装する必要があります。現段階では OS がほとんど機能を持っていないため、それらの関数群のほとんどは中身が空で定義せざるを得ませんが、それでも `strlen` や `sprintf` など、メモリ上の操作だけで完結するような関数については問題なく使用できます^{*4}。

ということで `sprintf` を使う際に定義しなければならない関数群を `newlib_support.c` にまとめて定義しました。だいたい関数は空のため見ても面白くありませんので本文中では紹介しませんが、興味のある方はソースコードを読んでください。OS に機能が充実していくにつれてこれらの関数に実装を足していけば、より多くの標準ライブラリ関数を使うことができるようになります。

さて、`newlib_support.c` に関数群を実装したら `sprintf` が使えるようになりますので、ビルドして起動を試みましょう。

```
$ source $HOME/osbook/devenv/buildenv.sh
$ make
```

ビルドが完了したら QEMU で起動してみます。図 5.4 のように、%d に値が埋め込まれて表示されれば成功です。

図 5.4 sprintf() で計算結果を表示



※ 4 厳密には `sprintf` は内部で動的メモリ確保を行うことがありますが、`%d` などのシンプルな書式指定子を使う限りは動的メモリ確保が実際に行われることはありません。

5.5 コンソールクラス (osbook_day05e)

文字列が表示できるようになったらぜひ実装したい機能として**コンソール**があります。コンソールという言葉はもともとはコンピュータの制御卓のことを表します。典型的には画面と入力装置が付いていて、キーボードで打ち込んでコンピュータを制御する装置のことです。しかし、ここでは「画面にどばどばメッセージを表示する機能」程度の意味で使っています。まだキーボードから入力する機能などを実装していませんから出力専用のコンソールになりますね。

メッセージをどばどば表示するためには改行文字で改行する機能と、行が画面の下端に達したときに1行ずつスクロールする機能がほしいです。改行文字で改行するのは簡単そうです。文字列を先頭から見て行って改行文字に出会ったらX座標を0に戻し、Y座標を16だけ増やせば良いでしょう。一方でスクロールは少し複雑です。スクロールするためには画面を一度塗りつぶしてメッセージを消し、改めて1行ずらしてメッセージを描画する必要があります。したがって、それまでに表示されたメッセージを最低でも1画面分は記憶しておく必要もあります。

リスト 5.13 コンソールクラスの定義 (console.hpp)

```
#pragma once

#include "graphics.hpp"

class Console {
public:
    static const int kRows = 25, kColumns = 80;

    Console(PixelWriter& writer,
            const PixelColor& fg_color, const PixelColor& bg_color);
    void PutString(const char* s);

private:
    void Newline();

    PixelWriter& writer_;
    const PixelColor fg_color_, bg_color_;
    char buffer_[kRows][kColumns + 1];
    int cursor_row_, cursor_column_;
};
```

ということで作成したコンソールクラスの定義を**リスト 5.13**に示します。要点を簡単に説明します。

kRows と kColumns はコンソールの表示領域の大きさを文字数で表したものです。縦方向の行数が kRows、横方向の列数が kColumns です。クラス内部に static で定義した変数は静的変数となり、OS がメモリにロードされてから終了するまでずっと存在する変数となります。static const は C++ で定数を定義する一般的な書き方です。

PutString() はクラスのユーザに公開された唯一のインターフェースで（コンストラクタを除いて）、与えられた文字列をコンソールに出力するというコンソールクラスの主目的となる機能を提供します。一般のコンソールにはカーソルと呼ばれるものがあって、そのカーソル位置から文字列が描画されます。このクラスにも cursor_row_ と cursor_column_ があって、それぞれカーソルの縦方向と横方向の位置を表します。文字列は基本的にカーソル位置から右方向に描画されていきますが、もし

文字列中に改行文字があれば改行処理を行います。文字列を描画する色は `fg_color_` を使います (`fg` は `foreground= 前景`)。

`Newline()` は改行処理を行います。改行処理では必ず `cursor_column_` を 0 に戻します。`cursor_row_` については 1 増やすか、もしコンソール表示領域の最下行に到達してしまっていたら代わりにスクロール処理を行います。

`buffer_` はコンソール表示領域に表示されている文字列を保存しているバッファです。行末にヌル文字 ('`\0`') を書くために列数より 1 だけ大きく確保してあります。このバッファのデータはスクロール処理のときに表示領域全体を再描画するのに使われます。スクロール処理のときに表示領域を塗りつぶす色は `bg_color_` を使います (`bg` は `background= 背景`)。

リスト 5.14 コンソールクラスのコンストラクタの定義 (console.cpp)

```
Console::Console(PixelWriter& writer,
    const PixelColor& fg_color, const PixelColor& bg_color)
    : writer_{writer}, fg_color_{fg_color}, bg_color_{bg_color},
      buffer_{}, cursor_row_{0}, cursor_column_{0} {
}
```

リスト 5.14 は何の変哲もないコンストラクタです。クラスの動作に必要な情報を受け取ってメンバ変数へセットします。表示領域の文字列を格納するための `buffer_` は一旦ヌル文字で初期化しておき、文字列が空である状態にしておきます。

リスト 5.15 唯一の公開インターフェースである `PutString()` の定義 (console.cpp)

```
void Console::PutString(const char* s) {
    while (*s) {
        if (*s == '\n') {
            Newline();
        } else if (cursor_column_ < kColumns - 1) {
            WriteAscii(writer_, 8 * cursor_column_, 16 * cursor_row_, *s, fg_color_);
            buffer_[cursor_row_][cursor_column_] = *s;
            ++cursor_column_;
        }
        ++s;
    }
}
```

リスト 5.15 は `PutString()` の定義です。与えられた文字列を先頭から 1 文字ずつ処理します。改行文字なら `Newline()` に処理を委譲し、それ以外の普通の文字ならそのまま描画します。

今のところ改行文字にしか対応していませんが、ASCII コードではそれ以外の制御文字もたくさん定義されています。例えばタブ文字 ('`\t`') は改行文字に並んでよく使われると思いますが、今のままではタブが普通の 1 文字として扱われてしまいます。本来なら 8 桁ずつに揃えるように空白を出すなど工夫したいところです。また、少しマイナーなベル文字 ('`\a`') というものもあります。ベル文字が来たら音を鳴らしたりアイコンを点滅させたりすると本格的です。

リスト 5.16 改行処理を行う Newline() の定義 (console.cpp)

```

void Console::Newline() {
    cursor_column_ = 0;
    if (cursor_row_ < kRows - 1) {
        ++cursor_row_;
    } else {
        for (int y = 0; y < 16 * kRows; ++y) {
            for (int x = 0; x < 8 * kColumns; ++x) {
                writer_.Write(x, y, bg_color_);
            }
        }
        for (int row = 0; row < kRows - 1; ++row) {
            memcpy(buffer_[row], buffer_[row + 1], kColumns * 1);
            WriteString(writer_, 0, 16 * row, buffer_[row], fg_color_);
        }
        memset(buffer_[kRows - 1], 0, kColumns * 1);
    }
}

```

リスト 5.16 は Newline() の定義です。この関数は改行文字に対する処理を行います。現在のカーソル位置がまだ最下行に達していないなら単にカーソルを 1 行進めるだけです。

カーソルが最下行にあるときはカーソルを進める代わりに表示領域全体を 1 行ずらすスクロール処理をする必要があります。最初の 2 重 for 文で表示領域を塗りつぶし、次の for 文で 1 行ずつ buffer_ の中身をずらしながら再描画していきます。

memcpy() は標準 C ライブラリの関数で、memcpy(dest, src, size) と書くと src の先頭から size バイトを dest にコピーするという意味になります。今回は row + 1 行目を row 行目にコピーするのに使っています。

memset() も標準 C ライブラリの関数で、指定した配列を指定した値で埋めることができます。最下行をヌル文字で埋めるのに使っています。

図 5.5 コンソールに 27 行分出力した様子



コンソールクラスができたので、そこに 25 行を少し超える行を書き込んでみてスクロールされるかを確認してみます (図 5.5)。ふむふむ、先頭の方が見えなくなっているのだからちゃんとスクロールされてそうですね。ということでコンソールは取り敢えず完成です。今後の開発ではここにメッセージを垂れ流していきます。

5.6 printk() (osbook_day05f)

Linux にはカーネル内部からメッセージを出すための `printk()` という関数があります。この関数はカーネル内のどこからでも使える上、`printf()` と同様に書式整形の機能を持つためカーネルを改造する際にとても便利に使えます。そのような機能をぜひ MikanOS にも搭載したいと思います。改造の方針は、まずコンソールクラスをグローバル変数として参照できるようにし、そのコンソールを出力先とする `printk()` を作ることにします。

リスト 5.17 コンソールクラスのバッファをグローバル変数として定義 (main.cpp)

```
char console_buf[sizeof(Console)];
Console* console;
...
extern "C" void KernelMain(const FrameBufferConfig& frame_buffer_config) {
    ...
    console = new(console_buf) Console(*pixel_writer, {0, 0, 0}, {255, 255, 255});
    ...
}
```

コンソールクラスをグローバル変数にするには `pixel_writer` と同じ方法を採用すれば良いでしょう。すなわちグローバル領域に `Console` の大きさ分のメモリ領域を `char` の配列として確保しておき、配置 `new` でインスタンスを生成するのです。実際のコードはリスト 5.17 のようになります。

リスト 5.18 `printk()` の定義 (main.cpp)

```
int printk(const char* format, ...) {
    va_list ap;
    int result;
    char s[1024];

    va_start(ap, format);
    result = vsprintf(s, format, ap);
    va_end(ap);

    console->PutString(s);
    return result;
}
```

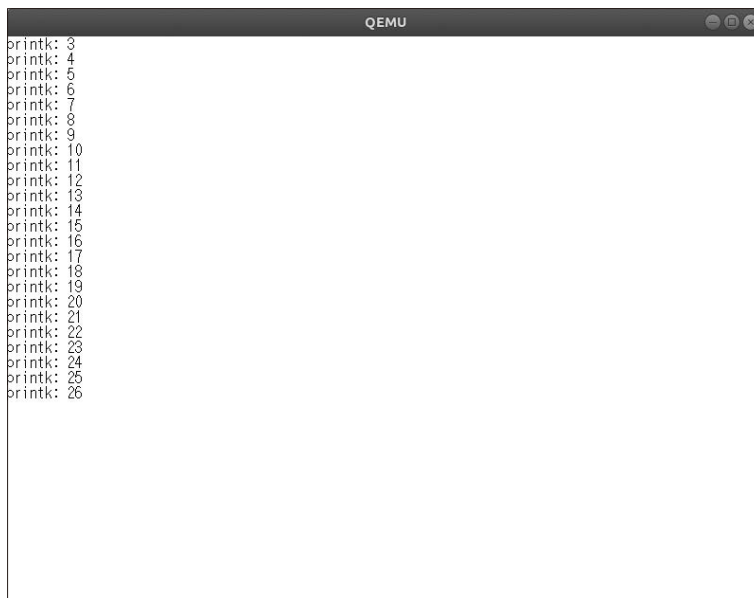
リスト 5.19 printk() の使用方法 (main.cpp)

```
for (int i = 0; i < 27; ++i) {  
    printk("printk: %d\n", i);  
}
```

リスト 5.18 に `printk()` の実装を、リスト 5.19 に使い方を示します。何といてもこの関数は可変長引数を取るというのが最大の特徴です。可変長引数を受け取るには可変長引数を ... (ドット 3 つ) で宣言しつつ、関数の中で `va_list` を使って受け取ります。実際の書式整形処理は `vsprintf()` に丸投げしています。 `vsprintf()` は `sprintf()` の親戚で、可変長引数の代わりに `va_list` 型の変数を受け取ることができますから、書式整形処理を移譲するのに大変便利です。作成した `printk()` は内部で 1024 バイトの固定配列を取っているため最大文字数に制限がありますが、それ以外は `printf()` と同様に使えます。

図 5.6 にリスト 5.19 の実行結果を示します。 `printk()` によって書式整形されていることが確認できます。 いちいち `sprintf()` と `console->PutString()` を組み合わせなくてもコンソールに書付きメッセージを表示できるようになって、今後の OS 開発が捗ること間違いなし！

図 5.6 `printk()` を使ってコンソールに表示してみる



第 6 章

マウス入力と PCI

ここまでで改行機能付きのメッセージの表示ができるようになり、見た目だけはだんだんと OS っぽさが出てきました。ただ、まだユーザーからの入力は一切受け付けないため、コンパイル時に仕込んだメッセージを表示するだけの機械となっていていす。この章ではマウス入力を受け付けられるようにし、ユーザーがコンピュータを操作できるようにします。


```
}  
}
```

リスト 6.2 にマウスカーソルの描画プログラムを示します。やっていることは単純で、先程定義したカーソルの形の定義に従ってピクセルを塗るだけです。'@' は縁なので黒 {0, 0, 0}、'.' は白 {255, 255, 255} で塗ります。

もっと OS っぽい見た目にするために、デスクトップ画面の絵を描いてみようと思います。筆者のイメージでは、背景は青っぽい色で、画面の下の方にはタスクバーのような領域があるといいかなと思いました。そのためには長方形 (Rectangle) をいくつか描画したいです。マウスカーソルと同様に 2 重 for ループで描画するのでもいいのですが、何回も使う処理ですので関数にしちゃおうと思います。

リスト 6.3 FillRectangle() の実装 (graphics.cpp)

```
void FillRectangle(PixelWriter& writer, const Vector2D<int>& pos,  
                  const Vector2D<int>& size, const PixelColor& c) {  
    for (int dy = 0; dy < size.y; ++dy) {  
        for (int dx = 0; dx < size.x; ++dx) {  
            writer.Write(pos.x + dx, pos.y + dy, c);  
        }  
    }  
}
```

リスト 6.3 に示す FillRectangle() は長方形を塗りつぶす関数です。第 2 引数に長方形の左上の座標、第 3 引数に長方形の大きさ、第 4 引数に描画色を指定します。引数で使っている Vector2D は 2 次元ベクトル (X、Y 座標の組) を表す構造体で、**リスト 6.4** のように定義しました。

リスト 6.4 2次元ベクトル Vector2D (osbook_day06c の graphics.hpp)

```
template <typename T>  
struct Vector2D {  
    T x, y;  
  
    template <typename U>  
    Vector2D<T>& operator +=(const Vector2D<U>& rhs) {  
        x += rhs.x;  
        y += rhs.y;  
        return *this;  
    }  
};
```

この構造体はテンプレートを用いて実装されていて、いろいろな型について 2 次元ベクトルを表現できます。1 行目の template <typename T> がテンプレートの目印で、この後の構造体定義において T を何らかの型として使いますよ、という意味です。構造体定義に現れる T は、構造体の使用時に具体的な型に置き換えられます。詳しくは「**付録 C++ のテンプレート**」を参照してください。

Vector2D は 2 つのメンバ変数に加え、演算子 operator += を持っています。この演算子のおかげで、Vector2D 型の 2 つの変数 a、b に対して a+=b というような、あるベクトルにもう 1 つのベクトルを足し込む操作が可能になっています。この操作はすこし後で使う予定です。このように演算子を独自に

定義することを C++ の用語で演算子のオーバーロードと言います。

リスト 6.5 DrawRectangle() の実装 (graphics.cpp)

```
void DrawRectangle(PixelWriter& writer, const Vector2D<int>& pos,
                  const Vector2D<int>& size, const PixelColor& c) {
    for (int dx = 0; dx < size.x; ++dx) {
        writer.Write(pos.x + dx, pos.y, c);
        writer.Write(pos.x + dx, pos.y + size.y - 1, c);
    }
    for (int dy = 1; dy < size.y - 1; ++dy) {
        writer.Write(pos.x, pos.y + dy, c);
        writer.Write(pos.x + size.x - 1, pos.y + dy, c);
    }
}
```

もう1つ、長方形の枠だけを描く関数 DrawRectangle() を作りました (リスト 6.5)。FillRectangle() と引数はまったく同じですが、長方形の中身は塗りつぶしません。

リスト 6.6 デスクトップの描画 (main.cpp)

```
FillRectangle(*pixel_writer,
              {0, 0},
              {kFrameWidth, kFrameHeight - 50},
              kDesktopBGColor);
FillRectangle(*pixel_writer,
              {0, kFrameHeight - 50},
              {kFrameWidth, 50},
              {1, 8, 17});
FillRectangle(*pixel_writer,
              {0, kFrameHeight - 50},
              {kFrameWidth / 5, 50},
              {80, 80, 80});
DrawRectangle(*pixel_writer,
              {10, kFrameHeight - 40},
              {30, 30},
              {160, 160, 160});

console = new(console_buf) Console{
    *pixel_writer, kDesktopFGColor, kDesktopBGColor
};
printk("Welcome to MikanOS!\n");
```

作成した2つの関数を使ってデスクトップ背景を描くプログラムをリスト 6.6 に示します。まず、デスクトップのほぼ全面を青っぽい色 (kDesktopBGColor) で塗りつぶし、その後黒っぽい色 ({1, 8, 17}) と灰色 ({80, 80, 80}) で小さな長方形を描きます。最後に明るい灰色で 30 × 30 の小さな枠を描きます。

起動して改造結果をチェックしてみましょう。改造後のデザインは図 6.1 のようになりました。なんだか一気に OS っぽい見た目になったと思いませんか？ 読者の皆さんはぜひオリジナルのデザインに挑戦してみてください。

図 6.1 デスクトップのデザインを整えてみた



6.2 USB ホストドライバ

マウスカーソルを描画することができました。そうすると、次はそれを動かしたくなりますよね。この章では最終的にマウスカーソルを実際のマウスの動きに連動して動かすことを目標とします。

さて、皆さんはどんなマウスを使っているでしょうか？ USB 接続の外付けマウスですか？ ノートパソコンのタッチパッドでしょうか？ Bluetooth で無線接続するマウスをお使いの方もいるかもしれませんね。

USB が普及するまではマウス（およびキーボード）の接続規格といえば PS/2 が主流でした。デスクトップパソコンにもノートパソコンにも PS/2 ポートが付いているのが普通でした。USB マウスが主流になってからも、BIOS が USB マウスを PS/2 マウスに見せかけるエミュレーション機能を持っていたため、PS/2 にしか対応していない OS でも USB マウスを使うことができました。USB よりも PS/2 の方が、圧倒的に制御が楽なため、すでに USB マウスが普及していた時代に出版された「30 日のできる！ OS 自作入門」（参考文献【1】）でも PS/2 マウスを前提にしていました。

しかし現代の UEFI BIOS には PS/2 のエミュレーション機能を持たないものも多くあります。筆者が自作 OS の動作確認に使っているボードコンピュータも PS/2 のエミュレーション機能がない機種です。そこで、本書では USB マウスをエミュレーションなしに扱うことにしました。すなわち、PS/2 としてではなく、きちんと USB 規格に準拠して通信できるようにするのです*¹。

USB とは Universal Serial Bus の略で、その名の通りシリアルバスの規格です。**バス**はコンピュータ分野の一般的な言葉で、語源は乗り物のバスと同じです。街を走る乗り合いバスに様々な人が乗り込むのと似ていて、コンピュータのバスには様々な機器からの信号が乗ります。コンピュータのバスは銅で

* 1 USB デバイスにきちんと対応するというのが、本書の最大のウリと言っても過言ではない、かもしれません。

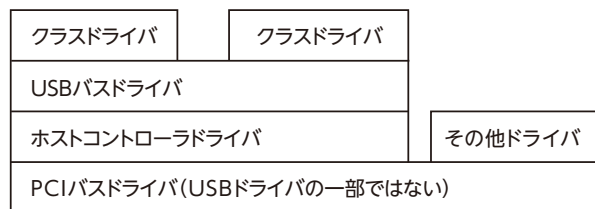
作られた電気信号のためのものもあれば、光信号を乗せるガラス製のバスもあります。1対1で接続する専用線ではなく、様々な機器が相乗りするのがバスの特徴です。USBもバス的一种で、様々な機器(キーボードやマウス、スピーカー、USBメモリ、Webカメラなどなど)が相乗りしていますよね。

シリアルの意味はOSを作るときには知る必要はありませんが、気になる人のために一応説明しておきます。シリアル(通信)とは1本の信号線を使って信号が1ビットずつ送られるという意味です。対義語にパラレルというものがあります。パラレル(通信)は複数の信号線を使って複数のビットを一気に送受信します。シリアルの方が信号線を少なくできるため、細いケーブルで機器を接続できる特徴があります。しかも、パラレルよりシリアルの方が(ケーブルが長くなった際に)高速な通信がしやすいため、周辺機器の接続はシリアルが主流です。後で出てくるPCIeもシリアル通信規格の1つです。

パソコンにはUSB機器とソフトウェア(OS)を橋渡しするための**USBホストコントローラ**という制御チップが搭載されています。OSはそのコントローラを制御することでUSB機器と通信できるようになります。このように、主として制御チップのようなハードウェアを扱うためのソフトウェアのことを**ドライバ(driver)**と呼びます*²。乗り物の運転手のこともドライバと呼びますね。あれと同じです。人間の運転手は乗り物を操作し、コンピュータのドライバは対象のハードウェアを操作します。ドライバはOSの一部です。

USB関連のドライバは、個々のUSB機器(USBターゲット)に搭載するターゲットドライバと、中心となるコンピュータ(USBホスト)に搭載するホストドライバに分かれます。もちろん本書で扱うのはホストドライバの方です。ホストドライバの作り方はいろいろ考えられますが、筆者は図6.2のような3階層(とPCIバスドライバの層)に分けて作りました。ホストコントローラを制御するホストコントローラドライバ、ホストコントローラの詳細を隠してUSB規格で定められたAPIを提供するUSBバスドライバ、USBターゲットの種類ごとに用意するクラスドライバです。

図 6.2 USBドライバの階層



ホストコントローラドライバはホストコントローラの規格別に作る必要があります。執筆時点でのパソコン向けの主なコントローラの規格としてUSB1.1用の**OHCI**と**UHCI**、USB2.0用の**EHCI**、USB3.x用の**xHCI**があります。4つに対応するのはとても大変ですので、本書ではxHCIだけを考えることにします。最近のパソコンはほぼ全てUSB3.xポートを搭載していてxHCI規格に準拠したコントローラが搭載されているはずですが、xHCI非対応のパソコンしか持っていない方は申し訳ありませんが新しいパソコンを買うか*³QEMUで我慢してください。ちなみに、xHCIはeXtended Host Controller Interfaceの略で、ホストコントローラのインターフェースを定めた規格*⁴です。xHCIに準拠したホストコントローラのことを**xHC**と呼びます。

* 2 元来、ドライバはハードウェアを操作するためのものですが、中にはハードウェアを操作しない仮想ドライバと呼ばれるものもあります。例えば、本物のプリンタではなくPDFとして保存するための仮想プリンタドライバなどです。

バスドライバはホストコントローラドライバの機能を利用し、USB 規格で定められた API を提供します。**API** (Application Programming Interface) とは、ソフトウェアが他のソフトウェアと相互にやり取りするのに使うインターフェースのことで、ここでは USB 規格で定められた各種の関数や定数などを意味します。例えば USB ターゲットがどんな機器かを識別するために発行する GET_DESCRIPTOR メッセージや、その引数として指定する DEVICE や CONFIGURATION といった定数などが API に含まれます。クラスドライバに対してそれらの API を提供するのが USB バスドライバの仕事です。

クラスドライバは USB ターゲットの種類毎に作ります。ここでいうクラスとは「種別」という意味があり、USB 規格で定められた標準クラスとしてキーボードやマウスが属する HID^{*5} クラス、オーディオ機器が属するオーディオクラス、記憶装置が属するマスストレージクラスなどがあります。クラスごとに動作がまったく違うため、クラスドライバはどうしてもクラス別に作る必要があります。バスドライバが提供する USB 規格に従った API を用いてクラスドライバを実装することになります。

さっそく USB ホストコントローラを制御するための USB ホストコントローラドライバと、それを利用してマウスを制御するクラスドライバを開発します。と言いたいところですが、USB 関連のドライバは本書で説明するには高度に複雑すぎますので、筆者が開発したドライバを使う方法を説明するだけにします。作り方を学べると思っていた方にはごめんなさい。作り方は説明しませんがソースコードを読むことはできます。\$HOME/workspace/mikanos/kernel/usb 以下を見てください。

6.3 PCI デバイスの探索 (osbook_day06b)

この章の残りの部分を使い、USB ドライバを使ってマウスからデータを入力する方法を説明します。大まかな流れは次のようになります。

- PCI バスに接続された PCI デバイスをすべて列挙する
- 列挙されたデバイスの一覧から xHC を探す
- xHC を初期化する
- USB バス上でマウスを探す
- マウスを初期化する
- マウスからデータを受信する

本書が対象とする xHCI 規格では、xHC は PCI デバイスであると定められています。**PCI** (Peripheral Component Interconnect) とは部品とマザーボードを繋ぐための規格であり、現代のパソコンの中心的技术です。xHC のほかにも NVMe SSD、高精度タイマ、ネットワークカード、GPU といったパソコンを構成する主要な部品が PCI、およびその後継規格である PCIe により接続されています。ですから、PCI および PCIe を扱えるようになることは、現代のパソコン用 OS を作るうえでは必須です。幸いな

* 3 筆者が動作確認に使った GPD MicroPC や MinnowBoard Turbot に対応しています。

* 4 こんなところでもインターフェースと実装が分離されています。ドライバを作る人は xHCI 規格にだけ依存すべきで、実装固有の挙動には依存しないようにすべきです。

* 5 Human Interface Device。人間がコンピュータとつながるための機器。典型的にはキーボードとマウスが HID クラスに属します。

ことに PCI と PCIe はソフトウェア互換性がありますから、PCI を扱えるようになれば十分です。

この節ではまず PCI デバイスを探索する方法から説明します。後のことも考えて、xHC だけでなく PCI バス（PCI デバイスが複数接続されるバス）に接続されている PCI デバイスをすべて列挙することになります。その後、列挙した中から xHC を改めて探します。

PCI 規格に準拠した機器はそれぞれ 256 バイトの **PCI コンフィギュレーション空間**を持っています。この空間には PCI デバイスのベンダ ID（製作会社ごとに割り振られた ID）やクラスコード（PCI デバイスの種類を表す数値）など、PCI デバイスに関する基礎的な情報が書かれています。PCI デバイスの列挙とは、詰まるところすべての PCI デバイスの PCI コンフィギュレーション空間を順に読み、ベンダ ID やクラスコードなどを調べていく作業です。

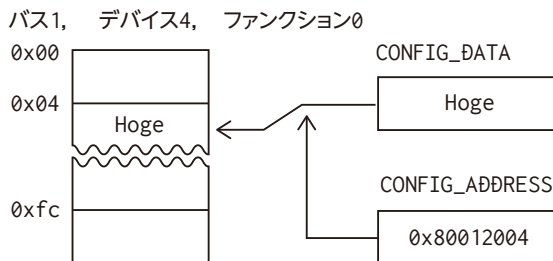
図 6.3 PCI コンフィギュレーション空間

31	24 23	16 15	8 7	0	
Device ID			Vendor ID		03-00h
Status			Command		07-04h
Base Class	Sub Class		Interface	Revision ID	0B-08h
BIST	Header Type		Latency Timer	Cacheline Size	0F-0Ch
Base Address Register 0					13-10h
Base Address Register 1					17-14h
Base Address Register 2					1B-18h
Base Address Register 3					1F-1Ch
Base Address Register 4					23-20h
Base Address Register 5					27-24h
Cardbus CIS Pointer					2B-28h
Subsystem ID			Subsystem Vendor ID		2F-2Ch
Expansion ROM Base Address					33-30h
Reserved				Capabilities Pointer	37-34h
Reserved					3B-38h
Max_lat	Min_Gnt		Interrupt Pin	Interrupt Line	3F-3Ch
Device dependent region					40h
...					FFh

図 6.3 に PCI コンフィギュレーション空間の構造を示します。PCI コンフィギュレーション空間は 256 バイト（0x00 から 0xff）あり、先頭の 0x40 バイトはどの PCI デバイスでも共通です。各項目の詳しい意味は後ほど説明することにして、今はこの空間をどうやって読むのかを説明します。

PCI コンフィギュレーション空間を読むには CONFIG_ADDRESS レジスタと CONFIG_DATA レジスタを使います。それぞれ IO アドレス空間の 0x0cf8 と 0x0cfc にある 32 ビットのレジスタです（具体的な読み書き方法は後で説明します）。CONFIG_ADDRESS レジスタに読み書きしたい PCI コンフィギュレーション空間の位置を設定してから CONFIG_DATA を読み書きすることで、PCI コンフィギュレーション空間を読み書きすることができます（図 6.4）。

図 6.4 CONFIG_ADDRESS レジスタからコンフィギュレーション空間を覗き見る



CONFIG_ADDRESS レジスタは表 6.1 に示すビット構造になっています。

表 6.1 CONFIG_ADDRESS レジスタの構造

ビット位置	内容
31	Enable ビット。1 にすると CONFIG_DATA の読み書きが PCI コンフィギュレーション空間に転送されます。
30:24	予約領域。0 にしておきます。
23:16	バス番号 (0-255)。
15:11	デバイス番号 (0-31)。
10:8	ファンクション番号 (0-7)。
7:0	レジスタオフセット (0-255)。4 バイト単位のオフセットを指定します。

PCI バスは複数あり、1 つのバスについて最大 32 個の PCI デバイスが接続されます。1 つの PCI デバイスは最大 8 つまでのファンクション（機能）を持つことができます。デバイスは必ずファンクション 0 を持ちますが、その他のファンクションはないかもしれません。ファンクションが複数ある場合も 1 から順に番号が振られているとは限りません。虫食い状態でも良いことになっています。実装されていないファンクション番号かどうかは、そのファンクション番号に対するベンダ ID を読み取って 0xffff（無効なベンダ ID）かどうかで判別できます。

Enable ビットを 1 にした状態で CONFIG_DATA レジスタを読み書きすることで任意のレジスタを読み書きすることができます。各種の番号から CONFIG_ADDRESS レジスタに書き込むための 32 ビット値を生成する関数をリスト 6.7 に示します。

リスト 6.7 MakeAddress() の実装 (pci.cpp)

```

/** @brief CONFIG_ADDRESS 用の 32 ビット整数を生成する */
uint32_t MakeAddress(uint8_t bus, uint8_t device,
                    uint8_t function, uint8_t reg_addr) {
    auto shl = [](uint32_t x, unsigned int bits) {
        return x << bits;
    };

    return shl(1, 31) // enable bit
        | shl(bus, 16)
        | shl(device, 11)

```

```

    | shl(function, 8)
    | (reg_addr & 0xfc);
}

```

MakeAddress() は指定されたバス番号、デバイス番号、ファンクション番号、レジスタオフセットを適切に組み合わせ、Enable ビットを 1 にした 32 ビット値を作ります。関数定義が少し見慣れない形をしていると思いますので詳しく説明します。

4 行目から始まる [](uint32_t x, unsigned int bits) {……} はラムダ式を作る書き方です。ラムダ式は無名関数とも呼ばれ、要するに普通の関数から名前を無くしたものです。ラムダ式の典型的な構文は [キャプチャ](引数リスト){本体} となっていて、今回はキャプチャが空の形です。キャプチャがあるラムダ式は「**第 9 章 重ね合わせ処理**」で登場します。

ここでは 2 つの引数 x と bits を引数にとり、x を左に bits 桁だけビットシフトした値を返すラムダ式を作っています。4 行目ではさらに、作ったラムダ式を後で利用するために、変数 shl にセットしています。言い換えるとラムダ式に shl という名前を付けた(せっかくの無名関数なのに!)わけですが、なぜわざわざ普通の関数ではなくラムダ式にしたのでしょうか? それは、普通の関数は他の関数の内部には定義できませんが、ラムダ式は関数の内部で定義できるからです。ある関数を実装するときに必要な小さな部品をラムダ式としてその場で定義することで、部品を実装する場所と部品を利用する場所が近くなり、ソースコードが理解しやすくなることがあります (濫用すると逆に読みにくくなりますから、ほどほどに)。

ラムダ式は普通の関数と同様に呼び出せます。例えば shl(1, 31) はラムダ式の呼び出しであり、結果は 1 を 31 桁だけ左にずらした値となります。ちょうど Enable ビットの位置に 1 が来ることになりますね。同様に bus、device、function の位置を CONFIG_ADDRESS レジスタの構造に合うようにずらし、それらを | で OR 結合します。レジスタオフセットは必ず必要ありませんのでそのまま結合します (ただし下位 2 ビットが 0 である必要があるためビットマスクをかけています)。

リスト 6.8 IO ポートアドレスの定義 (pci.hpp)

```

/** @brief CONFIG_ADDRESS レジスタの IO ポートアドレス */
const uint16_t kConfigAddress = 0x0cf8;
/** @brief CONFIG_DATA レジスタの IO ポートアドレス */
const uint16_t kConfigData = 0x0cfc;

```

さて、次に CONFIG_ADDRESS レジスタと CONFIG_DATA レジスタを読み書きする方法を説明します。これら 2 つのレジスタは IO アドレス空間 (後述) の 0xcf8 と 0xcfc に存在することになっています。ということでこれらを定数として定義しておきます (**リスト 6.8**)。

リスト 6.9 IO ポートを読み書きするアセンブラ関数 (asmfunc.asm)

```

; asmfunc.asm
;
; System V AMD64 Calling Convention
; Registers: RDI, RSI, RDX, RCX, R8, R9

bits 64
section .text

```

```

global IoOut32 ; void IoOut32(uint16_t addr, uint32_t data);
IoOut32:
    mov dx, di      ; dx = addr
    mov eax, esi    ; eax = data
    out dx, eax
    ret

global IoIn32 ; uint32_t IoIn32(uint16_t addr);
IoIn32:
    mov dx, di      ; dx = addr
    in  eax, dx
    ret

```

IO アドレス空間はメモリアドレス空間とは完全に別のアドレス空間です。名前の通り、メモリアドレス空間はメインメモリ用、IO アドレス空間は周辺機器用、という使い分けになっていて、PCI コンフィギュレーション空間は周辺機器なので IO アドレス空間に繋がっているというわけです。2つのアドレス空間を区別するために、IO アドレス空間の読み書きには専用の IO 命令（in と out）を使う必要があります。これらの IO 命令は特殊なので純粋な C++ プログラムとしては表現できません。仕方がないので、この部分だけアセンブリ言語で書き、それを C++ から呼び出して使うことにします。**リスト 6.9**に、C++ プログラムから呼び出せるように実装した 2つの関数 IoOut32() と IoIn32() を示します。

IoOut32() は引数で指定された IO ポートアドレス addr に対して 32 ビット整数 data を出力します。System V AMD64 ABI の仕様^{*6}より、引数 addr は RDI レジスタに、data は RSI レジスタに設定された状態で関数の処理が始まります。IoOut32() の中で主となるのは次の 1 行です。

```
out dx, eax
```

この 1 行は DX に設定された IO ポートアドレスに対して EAX に設定された 32 ビット整数を出力します。DX と EAX の値を調整するのがその前の 2 行の役割です。

```
mov dx, di      ; dx = addr
mov eax, esi    ; eax = data

```

この 2 行は、DX に DI の値を、EAX に ESI の値を代入しています。「**第 3 章 レジスタ**」によれば、DI は RDI の下位 16 ビット、ESI は RSI の下位 32 ビットのことから、結局、DX に引数 addr の値が、EAX に引数 data の値が書き込まれることになります。アセンブリ言語ではセミコロン以降にコメントを書けますので、分かりやすいようにそのことを書いておきました。

IoIn32() は指定した IO ポートアドレスから 32 ビット整数を入力して返す関数です。主となるのは次の 1 行です。

```
in  eax, dx
```

*6 「コラム 4.1 ABI」を参照。引数は RDI、RSI、RDX、RCX、R8、R9 の順で割り当てられます。

この1行はIOに設定されたIOポートアドレスから32ビット整数を入力してEAXに設定します。System V AMD64 ABIの仕様より、RAXレジスタに設定した値が戻り値になるのです。そのためinを実行した直後にそのままretすれば、IOポートから入力された値が適切に関数の呼び出し元へ返されます。

リスト 6.10 PCI レジスタ操作の関数群 (pci.cpp)

```
void WriteAddress(uint32_t address) {
    IoOut32(kConfigAddress, address);
}

void WriteData(uint32_t value) {
    IoOut32(kConfigData, value);
}

uint32_t ReadData() {
    return IoIn32(kConfigData);
}

uint16_t ReadVendorId(uint8_t bus, uint8_t device, uint8_t function) {
    WriteAddress(MakeAddress(bus, device, function, 0x00));
    return ReadData() & 0xffff;
}
```

リスト 6.10 に、アセンブリ言語で作った関数を用いて CONFIG_ADDRESS と CONFIG_DATA を読み書きする関数群と、それを使って実際にコンフィギュレーション空間からベンダ ID を読み取る ReadVendorId() の実装を示します。WriteAddress() でバス番号、デバイス番号、ファンクション番号、レジスタオフセットを CONFIG_ADDRESS に設定してから WriteData() および ReadData() を使って CONFIG_DATA を読み書きする、というのが典型的な使い方です。ベンダ ID はコンフィギュレーション空間の先頭にありますから、レジスタオフセットは 0 を指定しています。ReadVendorId() 以外にも同様の関数をいくつか作りましたが、ほとんど同じ形なので本文では紹介しません。

リスト 6.11 PCI デバイスを登録する配列をグローバル変数として定義 (pci.hpp)

```
/** @brief ScanAllBus() により発見された PCI デバイスの一覧 */
inline std::array<Device, 32> devices;
/** @brief devices の有効な要素の数 */
inline int num_device;
/** @brief PCI デバイスをすべて探索し devices に格納する
 *
 * バス 0 から再帰的に PCI デバイスを探索し、devices の先頭から詰めて書き込む。
 * 発見したデバイスの数を num_devices に設定する。
 */
Error ScanAllBus();
```

さて、必要な事前知識が一通り揃いましたので、ついに PCI デバイスを探索する関数に取り掛かることにします（ここまで長い道のりでした……）。まずは探索によって発見した PCI デバイスを登録するための 2 つのグローバル変数 pci::devices、pci::num_device の定義をリスト 6.11 に示します。

これらの変数がどのように使われるかは後で詳しく見ます。

グローバル変数をヘッダファイル（今回の場合は `pci.hpp`）に定義する際は `inline` を使ってインライン変数とします。古い C++ ではヘッダファイルでは `extern` を付け、どこか 1 つの `.cpp` に実体を定義するというのが普通でした^{*7}。

リスト 6.12 PCI バスに繋がったデバイスを再帰的に探索する関数 (`pci.cpp`)

```
Error ScanAllBus() {
    num_device = 0;

    auto header_type = ReadHeaderType(0, 0, 0);
    if (IsSingleFunctionDevice(header_type)) {
        return ScanBus(0);
    }

    for (uint8_t function = 1; function < 8; ++function) {
        if (ReadVendorId(0, 0, function) == 0xfffffu) {
            continue;
        }
        if (auto err = ScanBus(function)) {
            return err;
        }
    }
    return Error::kSuccess;
}
```

リスト 6.12 に探索の開始点となる `pci::ScanAllBus()` を示します。この関数は PCI バスをくまなく探索し、発見した PCI デバイスを `devices` 配列の先頭から書き込み、発見した PCI デバイスの総数を `num_device` に設定します。

`ScanAllBus()` を少し詳しく見てみると、最初にバス 0、デバイス 0、ファンクション 0 の PCI コンフィギュレーション空間からヘッダタイプを読み取っていることが分かります。バス 0 上のデバイス 0 は必ず**ホストブリッジ**ですから、ここではホストブリッジのヘッダタイプを読み取っていることになります。ホストブリッジはホスト側と PCI バス側を橋渡しする部品で、CPU と PCI デバイスの間の通信はここを必ず通過します。

ヘッダタイプは 8 ビット整数で、ビット 7 が 1 の場合はマルチファンクションデバイス、つまりファンクション 0 以外にも機能を持っている PCI デバイスであることを意味します。ファンクション 0 のホストブリッジがマルチファンクションデバイスではない（単機能デバイスである）場合、そのホストブリッジはバス 0 を担当するホストブリッジです。マルチファンクションデバイスである場合、ホストブリッジが複数存在することを意味します。その場合はファンクション 0 のホストブリッジはバス 0 を担当し、ファンクション 1 のホストブリッジはバス 1 を担当する、というように、ファンクション番号が担当バス番号を表します。各バスの探索は `ScanBus()` に任せます。

*7 本書には従来の方法を用いたグローバル変数の定義が残っている箇所があります。多くの場合、執筆の都合で残ってしまっているだけです。

リスト 6.13 指定されたバス上のデバイスを探索する関数 (pci.cpp)

```

/** @brief 指定のバス番号の各デバイスをスキャンする.
 * 有効なデバイスを見つけたら ScanDevice を実行する.
 */
Error ScanBus(uint8_t bus) {
    for (uint8_t device = 0; device < 32; ++device) {
        if (ReadVendorId(bus, device, 0) == 0xffffu) {
            continue;
        }
        if (auto err = ScanDevice(bus, device)) {
            return err;
        }
    }
    return Error::kSuccess;
}

```

リスト 6.13 に `pci::ScanBus()` を示します。ScanAllBus() から呼ばれた ScanBus() はそのバス上のすべてのデバイスを探索します。あるデバイス番号の位置に実際のデバイスがあるかどうかは、各デバイスのファンクション 0 のベンダ ID を調べれば分かります。ベンダ ID が 0xffff (無効値) 以外なら実際のデバイスがあることを意味します。ベンダ ID が有効な値であるデバイスを見つけたら ScanDevice() に処理を引き継ぎます。

リスト 6.14 指定されたデバイスのファンクションを探索する関数 (pci.cpp)

```

/** @brief 指定のデバイス番号の各ファンクションをスキャンする.
 * 有効なファンクションを見つけたら ScanFunction を実行する.
 */
Error ScanDevice(uint8_t bus, uint8_t device) {
    if (auto err = ScanFunction(bus, device, 0)) {
        return err;
    }
    if (IsSingleFunctionDevice(ReadHeaderType(bus, device, 0))) {
        return Error::kSuccess;
    }

    for (uint8_t function = 1; function < 8; ++function) {
        if (ReadVendorId(bus, device, function) == 0xffffu) {
            continue;
        }
        if (auto err = ScanFunction(bus, device, function)) {
            return err;
        }
    }
    return Error::kSuccess;
}

```

リスト 6.14 に `pci::ScanDevice()` を示します。指定された PCI デバイスがマルチファンクションかどうかで処理を切り替える必要があるため ScanBus() より複雑ですが、上から読めば何をやっているかは分かると思います。有効なファンクションを探し、見つけたら ScanFunction() に処理を引き継ぎます。

リスト 6.15 指定されたファンクションを調べる関数 (pci.cpp)

```

/** @brief 指定のファンクションを devices に追加する.
 * もし PCI-PCI ブリッジなら、セカンダリバスに対し ScanBus を実行する
 */
Error ScanFunction(uint8_t bus, uint8_t device, uint8_t function) {
    auto header_type = ReadHeaderType(bus, device, function);
    if (auto err = AddDevice(bus, device, function, header_type)) {
        return err;
    }

    auto class_code = ReadClassCode(bus, device, function);
    uint8_t base = (class_code >> 24) & 0xffu;
    uint8_t sub = (class_code >> 16) & 0xffu;

    if (base == 0x06u && sub == 0x04u) {
        // standard PCI-PCI bridge
        auto bus_numbers = ReadBusNumbers(bus, device, function);
        uint8_t secondary_bus = (bus_numbers >> 8) & 0xffu;
        return ScanBus(secondary_bus);
    }

    return Error::kSuccess;
}

```

リスト 6.15 に `pci::ScanFunction()` を示します。この関数の前半では、指定されたバス番号、デバイス番号、ファンクション番号の組を `devices` に登録する作業を行います。 `AddDevice()` は `devices` 配列の末尾に指定された番号の組を追記する関数です（実装は後ほど紹介します）。もし PCI デバイスの数が多すぎて `devices` に入り切らない場合、 `AddDevice()` は `Error::kFull` を返してエラーを報告します。そうすると PCI デバイスの探索を続行しても意味がないので `return err;` を実行して探索を中止します。

登場する 2 つの関数を説明します。 `pci::ReadHeaderType()` は PCI コンフィギュレーション空間（図 6.3）から Header Type を読み取ります。PCI コンフィギュレーション空間は先頭 16 バイトが共通で、それ以降の構造は Header Type により異なります。一般的な PCI デバイスはすべて Header Type=0x00 になっていますから、本書ではヘッダ種別を特に気にしません。ただし Header Type のビット 7 はこの PCI デバイスがマルチファンクションデバイスであることを示すビットとして使われます。

`pci::ReadClassCode()` は PCI コンフィギュレーション空間からクラスコード（0Bh-08h の 4 バイト）を読み取ります。クラスコードは 8 ビット単位で意味が分かれています。ビット 31:24 はベースクラスといって大雑把なデバイス種別を表し、ビット 23:16 はサブクラスといって細かいデバイス種別を表します。例えば、ベースクラス 0x0c はシリアル通信のためのコントローラであり、その中でサブクラス 0x03 は USB であることを表します。ビット 15:8 は Programming Interface といって、レジスタレベルの仕様を表します。例えば 0x20 は USB 2.0 (EHCI)、0x30 は USB 3.0 (xHCI) を表します。API はソフトウェア同士のインターフェースで、Programming Interface はハードウェアに関するインターフェースといえます。

関数の後半ではこのファンクションが PCI-to-PCI ブリッジだった場合の処理を行います。PCI-to-PCI ブリッジは PCI デバイスの一種であり、2 つの PCI バス同士をつなぐものです。ベースクラス

0x06、サブクラス 0x04 であれば PCI-to-PCI ブリッジです。ブリッジの上流側をプライマリバス、下流側をセカンダリバスと呼び、PCI コンフィギュレーション空間の 0x18 にそれらのバス番号が記録されています。ブリッジを発見したらセカンダリバスの番号を取得し ScanBus() を呼び出すことで、セカンダリバスに接続された PCI デバイスも探索するようにします。

リスト 6.16 発見した PCI デバイスを devices に追加する関数 (pci.cpp)

```

/** @brief devices[num_device] に情報を書き込み num_device をインクリメントする. */
Error AddDevice(uint8_t bus, uint8_t device,
                uint8_t function, uint8_t header_type) {
    if (num_device == devices.size()) {
        return Error::kFull;
    }

    devices[num_device] = Device{bus, device, function, header_type};
    ++num_device;
    return Error::kSuccess;
}

```

リスト 6.16 に AddDevice() の実装を示します。関数の先頭では num_device の値をチェックしています。num_device は devices 配列に現在書き込まれている要素の数を表す変数ですので、それが devices 配列の容量 (devices.size()) と等しい場合はもう配列に空きがないことを意味します。その場合は Error::kFull を返してエラーを伝えます。まだ配列に空きがあるなら末尾に書き込んで num_device をインクリメントします。

リスト 6.17 PCI デバイスを列挙する (main.cpp)

```

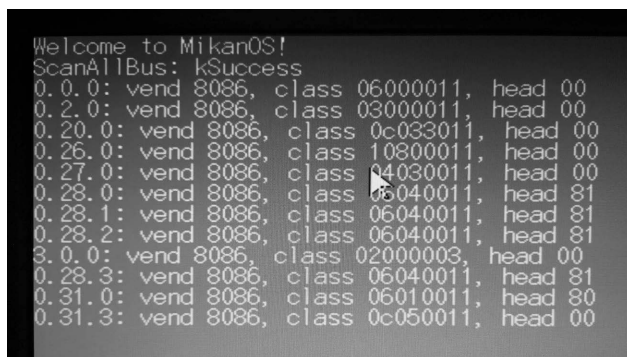
auto err = pci::ScanAllBus();
printf("ScanAllBus: %s\n", err.Name());

for (int i = 0; i < pci::num_device; ++i) {
    const auto& dev = pci::devices[i];
    auto vendor_id = pci::ReadVendorId(dev.bus, dev.device, dev.function);
    auto class_code = pci::ReadClassCode(dev.bus, dev.device, dev.function);
    printf("%d.%d.%d: vend %04x, class %08x, head %02x\n",
        dev.bus, dev.device, dev.function,
        vendor_id, class_code, dev.header_type);
}

```

作った関数を呼び出すためにメイン関数に書き加えたコードをリスト 6.17 に示します。まずは ScanAllBus() を呼び出して PCI デバイスをすべて検出し、次に検出された PCI デバイスの一覧を表示します。昨日作った printf が大活躍ですね。筆者が持っているボードコンピュータ「MinnowBoard Turbo」で実行したら図 6.5 のようになりました。QEMU や皆さんがお持ちの実機で実行するとまた違う結果になって面白いと思います。

図 6.5 MinnowBoard Turbot で PCI デバイス一覧を表示した様子



6.4 ポーリングでマウス入力 (osbook_day06c)

さて、この節では、列挙した PCI デバイスから xHC を探して初期化し、USB マウスを使えるようにしてみます。この節の最後には、マウスの動きに応じてマウスカーソルが動くようになります。

まずは xHC の探索です。これは簡単で、pci::devices からクラスコードが 0x0c、0x03、0x30 のものを探すだけです。ベースクラス 0x0c は USB に限らずシリアルバスのコントローラ全体を表します*⁸。その中でサブクラス 0x03 は USB コントローラを、インターフェース 0x30 は xHCI を表します。

リスト 6.18 PCI バスから xHC を探す (main.cpp)

```

// Intel 製を優先して xHC を探す
pci::Device* xhc_dev = nullptr;
for (int i = 0; i < pci::num_device; ++i) {
    if (pci::devices[i].class_code.Match(0x0cu, 0x03u, 0x30u)) {
        xhc_dev = &pci::devices[i];

        if (0x8086 == pci::ReadVendorId(*xhc_dev)) {
            break;
        }
    }
}

if (xhc_dev) {
    Log(kInfo, "xHC has been found: %d.%d.%d\n",
        xhc_dev->bus, xhc_dev->device, xhc_dev->function);
}

```

* 8 ベースクラス 0x0c には USB のほかに IEEE 1394 FireWire や InfiniBand などが属します。PCI クラスコードの一覧は、「The PCI ID Repository」(参考文献 [10]) のようなクラスコード収集サイトが参考になります。

リスト 6.18 に探索プログラムを示します。このプログラムはパソコンに複数の xHC が搭載されている場合でも 1 つだけを選びます。1 つの xHC だけに制限することで、USB ドライバ全体の仕組みを単純に保てるからです。複数の中から 1 つを選ぶときは Intel 製を優先して採用します。筆者の経験上、Intel 製のがメインのコントローラである可能性が高いです。

`ClassCode::Match()` は自身が持つクラスコードと引数で指定されたコードを比較し、3 つが一致すれば真となります。xHC を発見したらポインタを `xhc_dev` に記録します。for 文を抜けたときに `xhc_dev` がヌルでなければ xHC が存在するということになります。Intel 製の PCI デバイスは共通して 0x8086 というベンダ ID を持っていますので、それを見つけたら即座に break します。これで Intel 製が優先されますね。

ベンダ ID の取得はよく書く処理です。そのため **リスト 6.10** で作ったバス番号、デバイス番号、ファンクション番号を引数に取る 3 引数の `ReadVendorId()` を使って、`Device` 構造体を直接渡せる 1 引数の `ReadVendorId()` を作りました。**リスト 6.18** の探索プログラムでは新たに作った関数を使っています。

`Log()` は初めて出てきました。これは改良版の `printf()` みたいなものです。筆者が USB ドライバを開発する過程で、メッセージの表示を改良するために作った関数です。詳しい説明は「**コラム 6.1 ログ関数**」を参照してください。

xHC を発見したら、次は xHC のレジスタ群が配置されているメモリアドレスを取得します。実は、メモリアドレス空間にはメインメモリだけでなく、メモリのように扱えるメモリではない部品がいろいろ接続されています。その代表的なものが MMIO (メモリマップト IO) です。MMIO はメモリと同じように読み書きできるレジスタです。「**3.2 レジスタ**」で紹介した CPU 内蔵のレジスタはメモリアドレスではなく RAX のような名前で読み書きします。一方で MMIO にはメモリアドレスが付与されていて、そのアドレスを使って読み書きするのです。ただ、MMIO はあくまでもレジスタですから、そのほかの特徴 (値を記憶するだけでなく、読み書きそのものに意味があったりする) は CPU 内蔵のレジスタと同じです。

xHCI の仕様では、xHC を制御するレジスタ群は MMIO となっています。MMIO ですから、メモリアドレス空間のどこかにレジスタ群が存在するのですが、その場所は機種や搭載しているメインメモリの量などにより変わります。MMIO アドレスは PCI コンフィギュレーション空間にある BAR0 (Base Address Register 0) に記録されていることになっていますので、その値を読み出して利用しましょう。

リスト 6.19 BAR0 レジスタの読み取り (main.cpp)

```
const WithError<uint64_t> xhc_bar = pci::ReadBar(*xhc_dev, 0);
Log(kDebug, "ReadBar: %s\n", xhc_bar.error.Name());
const uint64_t xhc_mmio_base = xhc_bar.value & ~static_cast<uint64_t>(0xf);
Log(kDebug, "xHC mmio_base = %08lx\n", xhc_mmio_base);
```

リスト 6.19 に `pci::ReadBar()` を利用して BAR0 を読み取るプログラムを示します。`pci::ReadBar()` は `pci.cpp` で定義されていますので、中身が気になる方はソースコードを参照してください。

PCI コンフィギュレーション空間 (**図 6.3**) の中には BAR0 から BAR5 までの 6 つの BAR があります。それぞれの BAR は 32 ビット幅ですので、64 ビットのアドレスを表現する際は連続した 2 つの BAR を使うことになっています。`pci::ReadBar()` は指定された BAR と後続の BAR を読み、それらを結合したアドレスを返します。最終的に変数 `xhc_bar` には xHC の MMIO を指す 64 ビットのアドレスが設

定されます。

`xhc_bar.value` は xHC のレジスタ群の先頭を指すアドレスを表しますが、この下位 4 ビットは BAR のフラグが存在します。ですので、下位 4 ビットをマスクした値を真の MMIO ベースアドレスとして `xhc_mmio_base` に設定します。ここで出てくる `~static_cast<uint64_t>(0xf)` という式については「**コラム 6.2 static_cast<uint64_t>(0xf) の謎**」で説明します。

リスト 6.20 エラーを返す可能性のある戻り値を表す (error.hpp)

```
template <class T>
struct WithError {
    T value;
    Error error;
};
```

`pci::ReadBar()` は `WithError<uint64_t>` という構造体を返します。この構造体の元となる構造体テンプレートは `error.hpp` の中で **リスト 6.20** のように定義されています。この構造体テンプレートには `value` と `error` という 2 つのメンバが含まれていて、何らかの値とエラーコードを組にして表すことができます。C++ では関数の戻り値は 1 つだけですから、複数の値を返したい^{*9} 場合はこのように構造体に包むテクニックを良く使います。

リスト 6.21 xHC の初期化と起動 (main.cpp)

```
usb::xhci::Controller xhc{xhc_mmio_base};

if (0x8086 == pci::ReadVendorId(*xhc_dev)) {
    SwitchEhci2Xhci(*xhc_dev);
}
{
    auto err = xhc.Initialize();
    Log(kDebug, "xhc.Initialize: %s\n", err.Name());
}

Log(kInfo, "xHC starting\n");
xhc.Run();
```

BAR0 の値を使って xHC を初期化します (**リスト 6.21**)。 `usb::xhci::Controller` は筆者が作った USB ドライバプログラムの一部で、xHCI 規格にしたがったホストコントローラを制御するためのクラスです。このクラスのコンストラクタに BAR0 の値を指定してインスタンスを生成します。そのインスタンスに対して `Initialize()` メソッドを実行することで、xHC がリセットされた後、動作に必要な設定が行われます (`Initialize` は初期化という意味の英単語)。

Intel 製の xHC だった場合のみ、xHC の初期化をする前に `SwitchEhci2Xhci()` を呼び出しています。この関数は、2012 年に発売された Intel Panther Point チップセットに内蔵されている USB ホストコントローラのための特別対応をします。Panther Point チップセットには USB2.0 用の規格である EHCI

*9 「return」の訳には「戻す」と「返す」の 2 つがあります。明確な使い分けがあるわけではありませんが、筆者の好みにより名詞では「戻り値」、動詞では「値を返す」と表現しています。

コントローラと USB3.0 用の規格である xHCI コントローラが両方搭載されていて、初期状態では USB ポートを EHCI で制御する設定になっています^{*10}。SwitchEhci2Xhci() は特殊な設定を行うことで、xHCI で制御する設定に切り替えます。

リスト 6.22 SwitchEhci2Xhci() は USB ポートの制御モードを切り替える (main.cpp)

```
void SwitchEhci2Xhci(const pci::Device& xhc_dev) {
    bool intel_ehc_exist = false;
    for (int i = 0; i < pci::num_device; ++i) {
        if (pci::devices[i].class_code.Match(0x03u, 0x03u, 0x20u) /* EHCI */ &&
            0x8086 == pci::ReadVendorId(pci::devices[i])) {
            intel_ehc_exist = true;
            break;
        }
    }
    if (!intel_ehc_exist) {
        return;
    }

    uint32_t superspeed_ports = pci::ReadConfReg(xhc_dev, 0xdc); // USB3PRM
    pci::WriteConfReg(xhc_dev, 0xd8, superspeed_ports); // USB3_PSSSEN
    uint32_t ehci2xhci_ports = pci::ReadConfReg(xhc_dev, 0xd4); // XUSB2PRM
    pci::WriteConfReg(xhc_dev, 0xd0, ehci2xhci_ports); // XUSB2PR
    Log(kDebug, "SwitchEhci2Xhci: SS = %02, xHCI = %02x\n",
        superspeed_ports, ehci2xhci_ports);
}
```

SwitchEhci2Xhci() の中身を詳しく知りたい方のために実装を説明します (リスト 6.22)。興味がない方は読み飛ばして構いません。この関数はまず、パソコンに EHCI 準拠で Intel 製のコントローラが存在することを確認めます。そのような EHC が搭載されていれば、いよいよ EHCI から xHCI へのモード変更を行います。その処理には 4 種類のレジスタが関係します。それらのレジスタの仕様は Intel 7 シリーズチップセットのデータシート (参考文献 [9]) の 17.1.33 から 17.1.36 に載っています。

XUSB2PR (xHC USB 2.0 Port Routing) レジスタの各ビットは USB ポート 1 つに対応していて、ビットに 1 を書くとそのポートが xHCI モードになります。USB3_PSSSEN (USB 3.0 Port SuperSpeed Enable) レジスタの各ビットも USB ポートに対応していて、1 を書くとそのポートに対して SuperSpeed が有効になります。この設定自体は EHCI から xHCI へのモード切り替えには関係ありませんが、xHCI モードにした後で USB3.0 本来のスピードを発揮するために必要な設定です。これら 2 つのレジスタは、レジスタ中の全ビットが有効というわけではありません。2 つのマスクレジスタ USB3PRM と XUSB2PRM が有効なビットを示しています。そのため、マスクレジスタに記載された値をそのまま XUSB2PR および USB3_PSSSEN に書くようにします。

* 10 このように面倒なことになっている理由を推測してみます。2012 年というのは、USB3.0 対応のチップセットが普及し始めた時期で、まだ xHCI に対応していない OS も多く使われていました。そのような古い OS では EHCI しか認識できないため、初期状態で EHCI モードとしておくことで互換性を高めようとしたのでしょう。xHCI を認識できる新しい OS は当然 EHCI にも対応していることが期待されるため、それで問題ないと判断されても納得です。筆者は MikanOS で USB マウスが使えない機種があるとの報告を受けてバグ調査を頑張り、最終的に Linux のデバイスドライバを読んで Panther Point の特殊仕様を知りました。

USB ポートの xHCI モード設定と xHC の初期化が終わったら、次は xHC の Run() をメソッドを呼び xHC の動作を開始させます。動作を開始した xHC は、パソコンに接続された USB 機器の認識などを順次進めます。

リスト 6.23 USB ポートを調べて接続済みポートの設定を行う (main.cpp)

```
usb::HIDMouseDriver::default_observer = MouseObserver;

for (int i = 1; i <= xhc.MaxPorts(); ++i) {
    auto port = xhc.PortAt(i);
    Log(kDebug, "Port %d: IsConnected=%d\n", i, port.IsConnected());

    if (port.IsConnected()) {
        if (auto err = ConfigurePort(xhc, port)) {
            Log(kError, "failed to configure port: %s at %s:%d\n",
                err.Name(), err.File(), err.Line());
            continue;
        }
    }
}
```

xHC を起動させたら、いよいよ接続された USB 機器の中からマウスを探す作業を行います。**リスト 6.23** に、すべての USB ポートを探索し、何かが接続されているポートの設定を行うプログラムを示します。port.IsConnected() が真となっているポートには何らかの機器が接続されていますから、そのようなポートを見つけたら usb::xhci::ConfigurePort() を呼び出します。この関数も筆者が作ったドライバの一部で、ポートのリセットや xHC の内部設定、クラスドライバの生成などを行います。あるポートに USB マウスが接続されていた場合、usb::HIDMouseDriver::default_observer に設定した関数とその USB マウスからのデータを受信する関数として、USB マウス用のクラスドライバに登録されます。MouseObserver() については後で説明します。

ConfigurePort() は usb::xhci という名前空間に定義されているのに、メイン関数から名前空間名を付けずに呼び出しています。なぜでしょうか？ それは ADL (実引数依存の名前探索) という C++ の機能のおかげです。引数に指定した xhc の型が usb::xhci::Controller であることから、usb::xhci の中に定義された関数が自動的に探索されるという仕組みです。

リスト 6.24 xHC に溜まったイベントを処理する (main.cpp)

```
while (1) {
    if (auto err = ProcessEvent(xhc)) {
        Log(kError, "Error while ProcessEvent: %s at %s:%d\n",
            err.Name(), err.File(), err.Line());
    }
}
```

実際にマウスを動かしたときのイベントを処理する部分を**リスト 6.24** に示します。マウスなどの USB 機器の動作にともなって処理すべきデータが発生します。マウスが動けば「上下、左右にどれほど動いたか」というデータが発生しますし、キーボードなら押されたキーに関するデータが発生します。それらはイベントという形で xHC に溜まっていきます。このプログラムでは、while 文の中で繰り返し ProcessEvent() を呼び出すことで、xHC に対して溜まったイベントを処理するように命令します。

このように、イベントの有無を能動的に調べる方法を**ポーリング** (Polling) と呼びます。USB ホストコントローラに対し「処理すべきイベントはありますか？」と問い合わせ (Poll)、あれば処理し、無ければ何もしないという方法です。ポーリング方式は、問い合わせるたびに高確率でイベントがある (つまり、イベントの発生頻度が高い) 場合は効率が良い方法です。しかし、マウス入力やキーボード入力はイベントの発生が散発的^{* 11} ですので、イベント発生を問い合わせるとイベントがないことの方が多いわけです。かと言って、ポーリングを 10 秒に 1 回のような低い頻度にしてしまうと、最悪の場合はキーを押下してから OS が認識するまでに 10 秒かかることになり、とても使いづらい OS になってしまいます。

イベントが散発的に発生するものについては、ポーリングではなく割り込み方式が適しています。ポーリングの方が簡単なのでここで紹介しましたが、次章では割り込みを用いたマウスの処理をやってみようと思います。お楽しみに！

リスト 6.25 MouseObserver() の定義 (main.cpp)

```
char mouse_cursor_buf[sizeof(MouseCursor)];
MouseCursor* mouse_cursor;

void MouseObserver(int8_t displacement_x, int8_t displacement_y) {
    mouse_cursor->MoveRelative({displacement_x, displacement_y});
}
```

リスト 6.25 に、説明を後回しにしていた MouseObserver() の実装を示します。といってもとても短い関数で、たった 1 行、mouse_cursor->MoveRelative() を呼び出すだけです。具体的な処理内容を知るために MouseCursor クラスの実装を見てみましょう。

リスト 6.26 MouseCursor クラスの定義 (mouse.hpp)

```
#include "graphics.hpp"

class MouseCursor {
public:
    MouseCursor(PixelWriter* writer, PixelColor erase_color,
                Vector2D<int> initial_position);
    void MoveRelative(Vector2D<int> displacement);

private:
    PixelWriter* pixel_writer_ = nullptr;
    PixelColor erase_color_;
    Vector2D<int> position_;
};
```

リスト 6.26 に MouseCursor クラスの定義のうち、ヘッダファイル側 (mouse.hpp) を示します。このクラスはマウスカーソルの描画と移動に責任を持つクラスです。3 つの引数を取るコンストラクタと、1 つの引数を取るメンバ関数を持つことが分かります。

* 11 マウス (やその他の入力機器) は常に一定間隔で使うわけではなく、大部分の時間は止まっていて、ときどき動かすことが良くあります。

リスト 6.27 MouseCursor クラスのインスタンス生成 (main.cpp)

```
mouse_cursor = new(mouse_cursor_buf) MouseCursor{
    pixel_writer, kDesktopBGColor, {300, 200}
};
```

MouseCursor クラスのインスタンスを生成している部分をリスト 6.27 に示します。配置 new よりコンストラクタ呼び出しを行っていることが分かります。コンストラクタにピクセル描画クラス writer、マウスカーソルを消す際の色 erase_color、初期位置 initial_position を与えると、指定した初期位置にマウスカーソルが描画されます。ここでは、初期位置を 300, 200 としています。画面の左上から右に 300 ピクセル、下に 200 ピクセルの位置にカーソルが描画されるはずです。

リスト 6.28 MouseCursor クラスのコンストラクタとメンバ関数 (mouse.cpp)

```
MouseCursor::MouseCursor(PixelWriter* writer, PixelColor erase_color,
                          Vector2D<int> initial_position)
    : pixel_writer_{writer},
      erase_color_{erase_color},
      position_{initial_position} {
    DrawMouseCursor(pixel_writer_, position_);
}

void MouseCursor::MoveRelative(Vector2D<int> displacement) {
    EraseMouseCursor(pixel_writer_, position_, erase_color_);
    position_ += displacement;
    DrawMouseCursor(pixel_writer_, position_);
}
```

さて、MouseCursor クラスのコンストラクタとメンバ関数の具体的な実装を見てみましょう。リスト 6.28 に実装を示します。メンバ関数はヘッダファイルにプロトタイプ宣言だけを書き、対応する.cpp ファイルに定義を書くのでしたね。その方針にならって、この実装は mouse.cpp に書いています。

コンストラクタは、3 つのメンバ変数を初期化した後、1 回だけマウスカーソルを描画します。描画する位置には引数で与えられた initial_position を使います。DrawMouseCursor() は、「6.1 マウスカーソル」で紹介したマウスカーソル描画プログラム（リスト 6.2）を少し改造して関数化したものです。「6.1 マウスカーソル」のバージョンでは位置が 200, 100 に固定でしたが、DrawMouseCursor() は引数で位置を指定できるように改造してあります。

MoveRelative() メンバ関数はマウスカーソルを指定された方向に移動させます。移動の基準点は現在位置なので Relative（相対）という名前にしてあります。引数名に使っている Displacement という英単語は「変位」や「ずれ」という意味があり、ソフトウェアの世界でよく使われます。ここでは「マウスカーソルの位置のずれ」を意味します。例えば、displacement に Vector2D<int>{3, -4} を指定すると、マウスカーソルが右に 3 ピクセル、上に 4 ピクセルだけ移動します。

MoveRelative() から呼び出しているEraseMouseCursor() は、マウスカーソルを消す(Erase) ための関数です。画面の背景色を使ってマウスカーソルの形を塗りつぶせばマウスカーソルを消せる、という発想で実装されています。中身は DrawMouseCursor() とほぼ一緒ですが、DrawMouseCursor() は白と黒を固定的に使うのに対し、EraseMouseCursor() は指定した色を描画に使うところが違います。

長い説明でしたが、ここまできたら実際のマウスの動きに反応して画面上のマウスカーソルが動くはずです。さっそく動かしてみました (図 6.6)。

図 6.6 マウスカーソルをぐりぐり動かす



あ！なんとマウスカーソルを画面の下の方に移動させたら、折角カッコよく描画したステータスバーの絵が壊れてしまいました。これは、マウスがどこにあっても `EraseMouseCursor()` が同じ色に塗りつぶしてしまうのが原因です。これを解決するのはちょっと大変なので、しばらくはこの状態のまま我慢してください。とりあえずマウスカーソルが動いて、めでたし、めでたし。

コラム 6.1 ログ関数

`Log()` は画面にメッセージを表示するための関数です。`printf()` と似ていますが、最大の違いは条件に応じてメッセージを表示したり、表示しなかったりできることです。`Log()` の第 1 引数で**ログレベル**という、メッセージの重要度を表す値を指定します。指定したログレベルは事前に `SetLogLevel()` で設定した閾値（しきいち）と比較されて、閾値以上であれば画面に表示されるという仕組みです。

ログレベルは優先度が高い順に次の 4 種類が定義されています：`kError`、`kWarn`、`kInfo`、`kDebug`。例えば `kInfo` でログを書くと、`SetLogLevel()` で `kInfo` または `kDebug` に設定してあるときだけ、画面にメッセージが出力されます。

`printf()` では無条件にすべてのデバッグメッセージが表示されてしまいますが、`Log()` を使えば閾値を調整するだけでログの詳しさを調整できます。また、特定の箇所を `SetLogLevel(kDebug)` と `SetLogLevel(kWarn)` で囲むことで、その部分だけログを詳しく表示する、というような使い方も可能です。段々と機能が増えてきてメッセージの量が多くなってきたので、`day06b` と `day06c` の間で、`printf()` を使っていたほとんどの箇所を `Log()` に置き換えました。そのおかげで画面表示が随分すっきりしたのが分かると思います。

コラム 6.2 `static_cast<uint64_t>(0xf)` の謎

本文中で次のようなプログラムが登場しました。

```
const uint64_t xhc_mmio_base = xhc_bar.value & ~static_cast<uint64_t>(0xf);
```

この中の式 `~static_cast<uint64_t>(0xf)` がやろうとしていることは、下位 4 ビットだけが 0 である 64 ビット整数を作ることです。つまり `0xffffffffffffffff0` という整数が欲しいのです。この整数と `xhc_bar.value` のビット AND を計算することで、`xhc_bar.value` の下位 4 ビットを無視した（0 にした）値を得ようとしています。ちなみに、このような「特定の値だけが 0 であるような整数とビット AND を計算する操作」をビットマスクと呼びます。

プログラム中に `0xffffffffffffffff0` という値をそのまま埋め込んでもいいのですが、`f` が多すぎて読みにくく、書き間違いそうです。あくまでやりたいのは「下位 4 ビットのマスク」であって、`f` の数を気にしたくはありません。そこで `~0xf` あるいは `~0b1111` というように、下位 4 ビットが全部 1 である整数をビット反転する戦略を取ろうと思いました。この戦略を使えば、下位 4 ビットだけが 0 でそれ以外が 1 となる整数をミスなく作れます。

しかしこの戦略には罠があります。`0xf` が `int` と解釈されてしまうのです。私たちが使うコンパイラは `int` を 32 ビット整数として扱いますから、`~0xf` は 32 ビットの `0xffffffff0` という値になってしまい、`f` がまったく足りません。これを防ぐために、あらかじめ `0xf` を 64 ビットに拡張した後でビット反転する必要があり、最終的に `~static_cast<uint64_t>(0xf)` というコードにたどり着きました。

なお、別解として `~0xfull` や `~UINT64_C(0xf)` という書き方もあります。こちらの方が短いのですが、ぱっと見たときに分かりやすいだろうということで `static_cast` を使った方法を採用してみました。

第 7 章

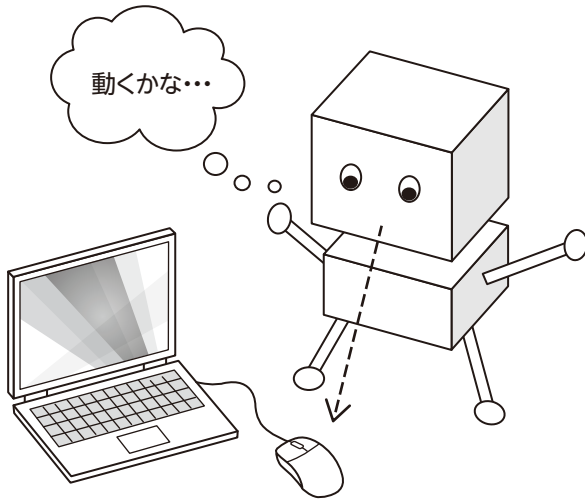
割り込みとFIFO

前章では USB マウスに対応し、初めてユーザーからの入力を受けて動作を変化させることができましたようになりました。ただメッセージを表示するだけの機械から脱却し、ユーザーとの相互作用ができるようになった点で大きな一歩です。しかし、マウス入力をポーリングするのは非効率です。この章では割り込みを使ってマウス入力を処理できるようにします。

7.1 割り込み (osbook_day07a)

ポーリング方式は、イベントが来ているかどうかを問い合わせる方式のことでした。対して**割り込み**方式は、イベントが到着したときにハードウェアが OS に通知する方式です。割り込みを用いると、何らかのイベントが発生したときに通知を受けて処理を行うことができ、マウスやキーボード入力などに対してとても効率的に処理が可能です。OS は普段、マウスやキーボードのことなんてまったく気にせず、ほかの処理をしていて良いのです。割り込みを例えると、動きがあるまで見張っていてくれる頼りがいのあるロボットのようなものです (図 7.1)。

図 7.1 マウスが動いたかどうかの監視を割り込みに任せる



この章では、x86-64 アーキテクチャで割り込みを扱う方法を説明します。ちょっと説明が長くなってしまのですが、頑張って付いてきてください。割り込みができるようになると、マウス入力以外にもいろいろな処理が書きやすくなりますので。例えば、**リスト 6.24** で紹介した無限ループでは今のところマウスの処理しかしていませんが、そのほかの処理をここに追加しようとするとかかなり難しくなります。後の章で出てくるタイマ処理では、タイマが一定時間を刻むたびに処理を行う必要がありますが、無限ループに「一定時間毎に特定の処理を行う」という機能を追加するのは至難の業でしょう。割り込みを使うと、一定時間毎に割り込みを発生させるだけでそれを実現できます。

本書で対象としている x86-64 アーキテクチャでは、割り込みは次のように実現されます (x86-64 に限らず、どの CPU でも基本的には同じような処理の流れですが)。

・事前準備

- イベント発生時に実行する割り込みハンドラを準備します。
- 割り込みハンドラを、IDT (割り込み記述子テーブル) に登録します。

・イベント発生時

- ハードウェア (場合によってはソフトウェア) がイベントを CPU に通知します。

- CPU は現在の処理を中断し、イベントの種類に応じて登録された割り込みハンドラに処理を移します。
- 割り込みハンドラの処理が終わると、中断していた処理を再開します。

ここで主役となるのは**割り込みハンドラ** (Interrupt Handler) と呼ばれるものです。ハンドラとは、何かをハンドル (扱う) する者という意味がある言葉で、コンピュータの分野で良く使われます。割り込みハンドラは割り込み発生時に実行される関数のことを意味します。見張り役のロボットが何か変化を検知した際に実行する処理、それが割り込みハンドラです。

7.2 割り込みハンドラ

割り込みハンドラの中身を見ていきます。USB ホストコントローラ (xHCI) ドライバ用の割り込みハンドラの実装をリスト 7.1 に示します。

リスト 7.1 xHCI 用割り込みハンドラの定義 (main.cpp)

```
usb::xhci::Controller* xhc;

__attribute__((interrupt))
void IntHandlerXHCI(InterruptFrame* frame) {
    while (xhc->PrimaryEventRing()->HasFront()) {
        if (auto err = ProcessEvent(*xhc)) {
            Log(kError, "Error while ProcessEvent: %s at %s:%d\n",
                err.Name(), err.File(), err.Line());
        }
    }
    NotifyEndOfInterrupt();
}
```

まず目につくのが `__attribute__((interrupt))` です。これは、直後に定義される関数が純粋な C++ の関数ではなく、割り込みハンドラであることをコンパイラに伝える役割があります。この 1 行を書いておくと、コンパイラが割り込みハンドラに必要な前処理と後処理を挿入してくれます^{*1}。

中身は、`NotifyEndOfInterrupt()` の呼び出しを除き、これまで `KernelMain()` の最後の方に書いてあった処理を移してただけです。USB ホストドライバに、溜まっているイベントを処理するように指示しています。処理すべきイベントが着信すると割り込みが発生し、割り込みハンドラの中からその処理をドライバに指示する、という流れです。

リスト 7.2 `NotifyEndOfInterrupt()` の定義 (interrupt.cpp)

```
void NotifyEndOfInterrupt() {
    volatile auto end_of_interrupt = reinterpret_cast<uint32_t*>(0xfe000b0);
    *end_of_interrupt = 0;
}
```

*1 挿入される前処理と後処理は、コンテキストの保存と復帰処理です。コンテキストの保存と復帰の概念は「13.2 コンテキストの切り替えに挑戦」を参照。

割り込みハンドラの最後で `NotifyEndOfInterrupt()` を呼んでいます。この関数の実装をリスト 7.2 に示します。やっているのは単純で、`0xfee00b0` 番地 (End of Interrupt レジスタ) に 0 を書き込むだけです。書き込む値は何でも良いのですが、とにかく、この番地に値を書くことで割り込み処理の終了を CPU に伝えられるようになっています。割り込み終了を伝えておかないと、次に同じ割り込みが発生しても割り込みハンドラが呼び出されなくなってしまう、マウスをグリグリと動かし続けられなくなります。

なぜ、メインメモリに値を書くだけで CPU に伝わる仕組みになっているのかと疑問に思うかもしれません。普通、メインメモリに値を書くという処理は単に値を書き込むだけであり、それ以外の効果はないはずだからです。その理由は、実は `0xfee00000` から `0xfee00400` までの 1024 バイトの範囲にはメインメモリは配置されておらず、CPU のレジスタが置かれているということです*²。メインメモリと違い、レジスタの場合は「書き込み動作」自体を何らかの処理の引き金とすることができ*³、今回の場合は割り込み処理の終了を CPU に伝える処理が実施されます。

`end_of_interrupt` に `volatile` 修飾子を付けることで、この変数は揮発性であることを C++ コンパイラに伝えています。これが必要なのは `0xfee00b0` 番地への書き込みを確実に行うためです。`volatile` を付けた変数の読み書きは最適化の対象になりません。`volatile` を付けない場合、C++ コンパイラは `*end_of_interrupt = 0;` によって書き込んだ値が他の場所で利用されないため、この書き込みは無駄だと考えてメモリ書き込み命令を省くかもしれません。確かに書き込んだ値を読み出すコードはありませんが、書き込むこと自体が重要ですので、書き込み命令が省かれてしまっては困るのです。さらに詳しい説明はコラム 11.1 を参照してください。

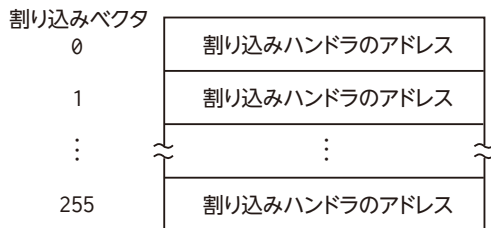
7.3 割り込みベクタ

割り込みには様々な種類があります。マウスやキーボードが動くなどして USB ホストコントローラがイベントを検知したときに発生する割り込み、タイマが一定時間を刻んだときに発生する割り込み、ディスク装置とメインメモリの間でのデータ転送が完了したときの割り込み、0 で割り算を実行しようとしたときの割り込み、などなど。すべての割り込みが同じ割り込みとして扱われると大変不便ですので、これら割り込みの種類毎に異なる**割り込み要因番号**、またの名を**割り込みベクタ**というものを割り振って管理する仕組みがあります。

割り込みベクタには固定で番号が割り振られているものもあるし、OS の作者が勝手に番号を決めて良いものもあります。いずれにしても、x86-64 アーキテクチャでは 0 から 255 のいずれかの数値になります。例えば、0 除算割り込みは CPU の仕様によって 0 番と決まっています。USB に関する割り込み番号は OS の作者が自由に割り当てます。いずれかの割り込みが発生したとき、CPU は現在実行中の処理を中断し、割り込みベクタに応じた割り込みハンドラに処理を切り替える、という動きをします。というわけで、CPU はそれぞれの割り込みベクタ N に対し「N 番の割り込みが発生したときに実行すべき割り込みハンドラ」を知っている必要があります。以降ではこれを設定する表について説明していきます。

- *² メモリアドレスを解釈してメインメモリを読み書きするのは CPU 自身なので、メモリアドレスの一部を横取りして自分のレジスタに振り向けるなんてことは、造作もないことなのです。
- *³ そのようにハードウェアを作ることができる、ということ。CPU の EOI レジスタは、そう作られており、値を書き込むという動作自体に意味があります。

図 7.2 IDT の構造



IDT（割り込み記述子テーブル：Interrupt Descriptor Table）とは、割り込み要因番号と割り込みハンドラを対応付けるためのテーブル（表）のことです。**図 7.2** に示す構造であり、メインメモリ上に配置することになっています。0 から 255 の割り込み要因番号に対し、それぞれ異なる割り込みハンドラを対応付けることができます。割り込みハンドラを IDT に登録することで、実際に割り込みが発生した際に登録されたハンドラが呼び出されるという仕組みです。

リスト 7.3 IDT の定義 (interrupt.cpp)

```
std::array<InterruptDescriptor, 256> idt;
```

IDT 全体の定義を**リスト 7.3** に示します。割り込み記述子を 256 個並べた配列になっており、0 から 255 までの任意の割り込みベクタを扱えます。std::array は C++ で固定長配列を作るためのクラスで、C の固定長配列と実態は同じです。以降では、要素の型である InterruptDescriptor を詳しく説明していきます。

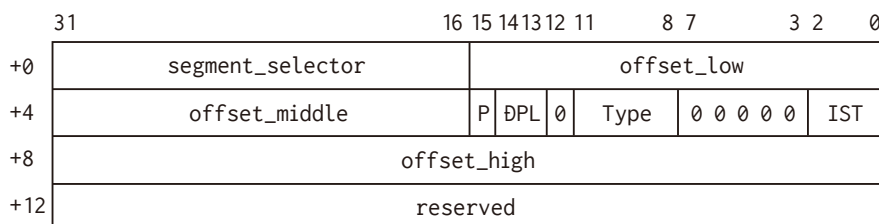
リスト 7.4 InterruptDescriptor 構造体の定義 (interrupt.hpp)

```
union InterruptDescriptorAttribute {
    uint16_t data;
    struct {
        uint16_t interrupt_stack_table : 3;
        uint16_t : 5;
        DescriptorType type : 4;
        uint16_t : 1;
        uint16_t descriptor_privilege_level : 2;
        uint16_t present : 1;
    } __attribute__((packed)) bits;
} __attribute__((packed));

struct InterruptDescriptor {
    uint16_t offset_low;
    uint16_t segment_selector;
    InterruptDescriptorAttribute attr;
    uint16_t offset_middle;
    uint32_t offset_high;
    uint32_t reserved;
} __attribute__((packed));
```

IDTの各要素、すなわち割り込み記述子の属性を表す共用体 `InterruptDescriptorAttribute` と、割り込み記述子そのものを表す構造体 `InterruptDescriptor` を **リスト 7.4** に示します。この構造体を図にすると **図 7.3** のようになります。

図 7.3 割り込み記述子の構造



図の中の P、DPL、Type、IST などはビットフィールドという C++ の機能を使っていますので、簡単に説明します。これらのフィールドは `attr.bits` に含まれます。この型の定義は `InterruptDescriptorAttribute` の中にある無名構造体です。各行が型 フィールド名: ビット幅となっており、これがビットフィールドの書き方です。構造体の通常のフィールドは 1 バイト単位でしか名前を付けられませんが、ビットフィールドを用いるとビット単位で名前が付けられます。各行に書いてある型はあまり重要ではなく、ビット幅以上の幅を持つ符号なし整数を指定すれば OK です。

ビットフィールドを使う際は `idt[...].attr.bits.type = 14;` のように書きます。この例は Type フィールド (オフセット 4 のビット 11:8) に対し整数の 14 を代入します。C++ コンパイラは、ビットの AND や OR といった命令を用いて、他のビットを変更することなく Type に 14 が代入されるような機械語を生成します。

割り込み記述子の各フィールドの意味を説明します。offset_low、offset_middle、offset_high は割り込みハンドラのアドレスを設定するフィールドで、3 つを組み合わせると 64 ビットアドレスを指定できます。x86 アーキテクチャは互換性を保ちつつ 16 ビット、32 ビット、64 ビットと進化してきたため、このように分断されてしまっています。segment_selector はこの割り込みハンドラを実行する際のコードセグメントを指定します。

コードセグメントは実行可能コードが置いてあるメモリ区画 (セグメント) という意味です。メモリを区画に分けて管理するためのセグメンテーションはメモリ管理のための機能で、メモリアドレス空間の区画 (セグメント) に対し、実行可能、読み書き可能などの属性を設定できます。うまく使うと便利はありますが、現代で広く使われている OS ではセグメンテーション機能は使われていません。だからなのかは分かりませんが、x86-64 アーキテクチャではセグメンテーションの機能が大きく縮小されました。本書が対象とする x86-64 の 64 ビットモードでは、コードセグメントと言いつつメモリを区画に分ける機能はもはや無効化されており、単にメモリ全体の属性を指定するだけの機能になってしまいました。セグメンテーションについては「**第 8 章 メモリ管理**」で改めて説明します。

`attr` は割り込み記述子の属性を設定します。属性のうち重要なのは `type` と `descriptor_privilege_level` (DPL) です。type は記述子の種別を設定します。設定可能な値は 14 (Interrupt Gate) または 15 (Trap Gate) のみです。通常の割り込みであれば 14 を設定します。DPL は割り込みハンドラの実行権限を設定します。詳しくは「**20.2 OS を守ろう (1)**」で説明しますが、割り込み記述子の DPL にはほとんどの場合に 0 を設定しておけば大丈夫です。interrupt_stack_table は本書では常に 0 です。present は記述子が有効であることを示すフラグで、もちろん 1 を設定します。

割り込み記述子の構造体の説明の最後に、3箇所使われている `__attribute__((packed))` を紹介しましょう。これは、構造体の各フィールドを詰めて配置するためのコンパイラ拡張です。コンパイラは何も指定されない場合、変数のアライメントを守るためにフィールド間に隙間（パディング）を挿入します。しかし、ハードウェアの仕様で定まったデータ構造を構造体として表現するには、コンパイラが勝手に隙間を挿入してはまずいわけです。そのため、このコンパイラ拡張を指定し、隙間の挿入を防止します。

7.4 割り込み記述子の設定

さて、割り込み記述子は 16 バイトの構造体であることとその詳しい中身の構造が分かりました。次に、この構造体に値を設定する関数 `SetIDTEntry()` を紹介します。関数の実装をリスト 7.5 に示します。

リスト 7.5 `SetIDTEntry()` の実装 (`interrupt.cpp`)

```
void SetIDTEntry(InterruptDescriptor& desc,
                InterruptDescriptorAttribute attr,
                uint64_t offset,
                uint16_t segment_selector) {
    desc.attr = attr;
    desc.offset_low = offset & 0xffff;
    desc.offset_middle = (offset >> 16) & 0xffff;
    desc.offset_high = offset >> 32;
    desc.segment_selector = segment_selector;
}
```

`SetIDTEntry()` は引数 `desc` で指定された割り込み記述子に各種の設定を行う関数です。記述子の属性、割り込みハンドラのアドレス、それから割り込みハンドラが置かれたコードセグメントのセクタ値を設定します。割り込みハンドラのアドレスは 3 つに分割して設定しなければならず面倒ですが、やっていることは単純です。

リスト 7.6 割り込みベクタ 0x40 を設定して IDT を CPU に登録する (`main.cpp`)

```
const uint16_t cs = GetCS();
SetIDTEntry(idt[InterruptVector::kXHCI], MakeIDTAttr(DescriptorType::kInterruptGate, 0),
            reinterpret_cast<uint64_t>(IntHandlerXHCI), cs);
LoadIDT(sizeof(idt) - 1, reinterpret_cast<uintptr_t>(&idt[0]));
```

`SetIDTEntry()` の使い方はリスト 7.6 のようになります。IDT の xHCI 用の割り込みベクタ^{*4}に対し、xHCI 用の割り込みハンドラ (`IntHandlerXHCI()`) を登録しています。割り込み記述子の種別には `Interrupt Gate` を、DPL には 0 を指定しています。セグメントセクタには、`GetCS()` で取得した現在のコードセグメントのセクタ値を指定します。筆者の手元で試したところ、`GetCS()` の戻り値は 0x38 でした。

*4 `InterruptVector::kXHCI` は `interrupt.hpp` で 0x40 と定義されています。

リスト 7.7 LoadIDT() の定義 (asmfunc.asm)

```

global LoadIDT ; void LoadIDT(uint16_t limit, uint64_t offset);
LoadIDT:
    push rbp
    mov rbp, rsp
    sub rsp, 10
    mov [rsp], di ; limit
    mov [rsp + 2], rsi ; offset
    lidt [rsp]
    mov rsp, rbp
    pop rbp
    ret

```

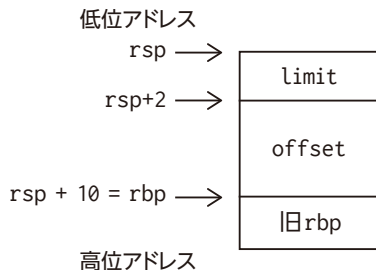
IDT の設定が終わったら IDT の場所を CPU に教える必要があります。そのための関数が LoadIDT() です。この関数は IDT の大きさと IDT が配置されているメインメモリのアドレスを受け取り、lidt 命令で CPU に登録します。この命令は IDT の大きさとアドレスを直接指定するのではなく、2 つの値を書き込んだ 10 バイトのメモリ領域を指定する必要があります。このメモリ領域の構造を表 7.1 に示します。

表 7.1 lidt 命令に指定するメモリ領域の構造

オフセット	型	意味
0	uint16_t	IDT のサイズ-1
2	uint64_t	IDT の先頭アドレス

LoadIDT() はアセンブリ言語で作った関数です。関数の引数は RDI、RSI、RDX、RCX、R8、R9 の順に格納されるのでした（「[コラム 4.1 ABI](#)」参照）。したがって第 1 引数 limit は RDI、第 2 引数 offset は RSI という対応関係になります。limit は 16 ビットの値ですから、実際には RDI の下位 16 ビットを表す DI レジスタを用います。RSP から 10 を引くことでスタック上に確保した 10 バイトのメモリ領域にこれら 2 つの値を書き込みます。lidt 命令を実行する直前のスタック領域の様子は図 7.4 のようになります。

図 7.4 lidt 命令実行時のスタック領域の状態



7.5 MSI 割り込み

ここまでの設定で、割り込みが発生してから割り込みハンドラが実行されるまでの設定は説明しました。残る説明は、割り込みを発生させる部分の設定です。割り込みハンドラの設定が完了していても肝心の割り込みが発生しなければ意味がありませんからね。xHCI では割り込みの発生方法として、PCI 規格が定める MSI を採用しています。本節では MSI 割り込みの仕組みの概略と、設定方法を説明します。

PCI 機器から CPU に割り込みを通知する方法にはいくつかあり、最も原始的なのは割り込み信号線の電圧レベルを変化させる方式です。この方式では PCI 機器の数だけ信号線が必要になりますが、PCI 規格では INT#A から INT#D の 4 本の割り込み信号線しか規定しておらず、複数の PCI 機器が 1 本の割り込み信号線を共有することになっていました。2000 年頃に PCI デバイスを使っていた方は IRQ の共有で苦労した方も多いのではないのでしょうか。

MSI（および拡張規格の MSI-X）は Message Signaled Interrupts の略で、PCI 規格で定められた比較的新しい割り込み方式です。信号線は使わず、メモリバスへの書き込み動作により割り込みを発生させます。End of Interrupt レジスタへ何らかの値を書き込むと割り込み終了を伝えられる、あの仕組みと似ており、特定のメモリアドレスへの書き込みにより割り込みを CPU へ通知することができます。これにより割り込み信号線が PCI 機器間で共有されることはなく、また同一の PCI 機器から複数種類の割り込みを CPU へ通知できるようになり、割り込み事情が大幅に改良されました。

MSI および MSI-X 割り込みでは、特定のメモリアドレス（Message Address と言います）に対して 32 ビットの値（Message Data と言います）を書き込むことで割り込みを発生させます。メモリアドレスと値のフォーマットは PCI 規格の外側、つまり CPU 側の仕様で規定されています。x86-64 の場合は **図 7.5** と **図 7.6** に示すフォーマットになっています。

図 7.5 Message Address レジスタのフォーマット

31	20 19	12 11	4 3	2	1	0
0xfee	Destination ID	予約	RH	DM	XX	

RH: Redirection Hint
DM: Destination Mode

図 7.6 Message Data レジスタのフォーマット

31	16 15	14	13	11 10	8 7	0
予約	TM	LV	予約	DM	Vector	

TM: Trigger Mode
LV: Level
DM: Delivery Mode

Message Address の各フィールドのうち、重要なのが Destination ID です。ここには割り込みを通知する CPU コアの番号（Local APIC ID。詳しくは後述）を設定します。現代の CPU は普通、コアが複数搭載されており、どのコアに割り込みを配送するかを指定できるようになっているのです。Redirection Hint は本書では 0 にして使います。ここを 1 に設定すると Destination Mode との組み合わせで柔軟な割り込みの配送ができるようになります。

Message Data の各フィールドのうち、重要なのが Vector です。ここには割り込みベクタ番号を設定します。マウスが動かされるなどして割り込みが発生する状況になると、ここに設定した割り込みベクタに対して割り込み処理が発生します。割り込みハンドラ `IntHandlerXHCI()` を設定した割り込み記述子の番号 (`InterruptVector::kXHCI`) と同じ番号を Vector フィールドに設定しておくことで、マウスが動くなどした際に `IntHandlerXHCI()` が呼ばれるようになります。

リスト 7.8 MSI 割り込みを有効化する (main.cpp)

```
const uint8_t bsp_local_apic_id =
    *reinterpret_cast<const uint32_t*>(&0xf0000000) >> 24;
pci::ConfigureMSIFixedDestination(
    *xhc_dev, bsp_local_apic_id,
    pci::MSITriggerMode::kLevel, pci::MSIDeliveryMode::kFixed,
    InterruptVector::kXHCI, 0);
```

xHC に対して MSI 割り込みを設定するプログラムをリスト 7.8 に示します。核となるのは `pci::ConfigureMSIFixedDestination()` です。これが xHC に対して MSI 割り込みを有効化するための設定を行います。この関数の第 2 引数に渡している `bsp_local_apic_id` が Destination ID フィールドに、第 5 引数に渡している `InterruptVector::kXHCI` が Vector フィールドに設定される値となります。すなわち、`bsp_local_apic_id` で指定される CPU コアに対し、`InterruptVector::kXHCI` で指定される割り込みを発生させるという設定になります。

マルチコア CPU ではコア毎に固有の番号 (Local APIC ID) が振られています。Local APIC ID は割り込みをどの CPU コアに通知するかを指定するために必要なものです。0xf0000000 番地のビット 31:24^{*5} を読み取ることで、そのプログラムが動作しているコアの Local APIC ID を取得できます。

マルチコア CPU では電源投入時には唯一のコアだけが起動し、他のコアは明示的に起動されるまでは停止しています。最初に動くコアのことを BSP (Bootstrap Processor) と呼びます。リスト 7.8 の時点では BSP のみが動いている状態ですから、0xf0000000 を読み取って得られる Local APIC ID は BSP のものということになります。すなわち、変数 `bsp_local_apic_id` は BSP の Local APIC ID を表します。残念ながら本書では BSP 以外のコアは無効にしたまま OS を作成していきます。そのため、BSP 以外のコアに割り込みを発生させることはありません。

7.6 割り込みのまとめ

さて、ここまでで x86-64 アーキテクチャにおける割り込みの仕組みと、その中心となる割り込み記述子と IDT の具体的な設定方法、MSI 割り込みの概要について説明しました。割り込みが発生してから割り込みハンドラが実行されるまでの流れをまとめます。

- 割り込みを扱うためには、割り込みハンドラ、割り込み記述子、そして割り込みの発生源の設定が必要です。
- 割り込みハンドラは `__attribute__((interrupt))` を付け、処理の最後に End Of Interrupt レジスタへの値の書き込みを行います。
- 割り込み記述子はメインメモリ上に作成した IDT という配列構造の 1 つの要素で、割り込みハンドラのアドレスや各種属性を設定します。

- IDT（割り込み記述子テーブル）は最大で 256 個の要素を持つ配列で、それぞれの要素は 0 から 255 の割り込みベクタに対応します。
- IDT の先頭アドレス、および大きさを `lidt` 命令により CPU に登録します。
- xHCI では MSI（または MSI-X）という方式で割り込みを発生させることになっているため、Message Address および Message Data の設定をします。

割り込み方式でマウスを動かしてみました（図 7.7）。といっても、見た目は何も変わりませんね。

図 7.7 割り込みの力でマウスカーソルを動かす



* 5 N:M と書いて、ビット N からビット M までの範囲を表します。通常、左側（N）に大きい数字を書きます。

7.7 割り込みハンドラの高速化 (osbook_day07b)

前節で割り込み処理ができるようになりましたが、1つ大きな問題があります。それは割り込みハンドラの処理に時間がかかることがある、ということです。割り込みハンドラの実装を改めて見てみましょう (リスト 7.9)。

リスト 7.9 xHCI 用割り込みハンドラの定義 (再掲) (main.cpp)

```
usb::xhci::Controller* xhc;

__attribute__((interrupt))
void IntHandlerXHCI(InterruptFrame* frame) {
    while (xhc->PrimaryEventRing()->HasFront()) {
        if (auto err = ProcessEvent(*xhc)) {
            Log(kError, "Error while ProcessEvent: %s at %s:%d\n",
                err.Name(), err.File(), err.Line());
        }
    }
    NotifyEndOfInterrupt();
}
```

マウスが動くなどして割り込みが発生するとこの割り込みハンドラの処理が始まります。すると、xHC に処理すべきイベントが溜まっている間はずっと `ProcessEvent()` を呼び出し続けます。`ProcessEvent()` のたった 1 回の実行を抜き出してみても多くの処理を実行します。USB 機器から受信したデータを解釈し、マウスからの入力なら `MouseObserver()` を呼び出します。この関数の中では、元のマウスカーソルを背景色で塗りつぶし、新しい場所に描画し直します。マウスカーソルの透明でない部分は 173 ピクセルですから、毎回 173×2 回のメモリ (VRAM) 書き込みが発生します。これは (CPU から見たら) かなり時間のかかる処理です。

割り込みハンドラの処理に時間がかかると何がいけないのでしょうか？ それは、割り込み処理をしている間は他の割り込みを受け付けることができない、というのが大きな理由です。割り込み処理中に新たな割り込みが集中して発生すると、新しい割り込みを取りこぼしてしまうことがあるのです。マウスが少しカクカクする程度ならまだしも、ネットワークパケットを取りこぼしてしまったりすると大変です。そんなわけで、この節では割り込みハンドラの処理を高速化することを考えます。

割り込みハンドラの処理を短時間にする基本戦略は、割り込みが来たことを記録だけしておき、本格的な処理は後でゆっくりと行うことです。割り込みが来たことを記録する方法は幾つもありますが、ここでは FIFO というデータ構造を使った方法を紹介します。

7.8 FIFO と FILO

FIFO は First In First Out の略です。日本語では先入れ先出しとなります。FIFO は図 7.8 に示すように (概念的には) 筒状のデータ構造になっていて、片側がデータを入れる側、もう片方がデータを取り出す側という構造になっています。

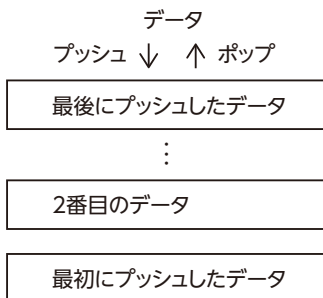
図 7.8 FIFO 構造



筒に入れる操作を**プッシュ** (push)、筒から取り出す操作を**ポップ** (pop) と呼びます^{*6}。筒の中ではデータの順序は入れ替わらないように作られているため、最初にプッシュしたデータが最初にポップされます。ここではまだ概念の話しかしておらず具体的なことを何も書いていません。例えば「データ」は4バイトの整数かもしれないし、メール文面かもしれないし、動画データかもしれない。とにかく何らかのデータをいくつか貯めておくことができ、プッシュした順番でポップされるようなデータ構造を FIFO と呼びます。

割り込みの高速化には関係ありませんが、ついでに FILO 型のデータ構造についても紹介します。**FILO** は First In Last Out の略です。日本語では先入れ後出しとなります。LIFO (Last In First Out)、後入れ先出しと言う場合もありますが同じ意味です。FILO は図 7.9 に示すように本を積んだようなデータ構造になっていて、上から積んで、上から取り出します。

図 7.9 FILO 構造



FIFO や FILO にはそれぞれ別名が付いています。FIFO はキュー (queue)、FILO はスタック (stack) です。コンピュータの教科書では別名の方をよく見るかもしれません。

さて、ここでは自作 OS 向けに FIFO 型のデータ構造を実装します。実はキューやスタックはとても良く使うので C++ の標準ライブラリに含まれている^{*7} のですが、それらは動的メモリ管理が必要になるため、メモリ管理を実装していない現段階の OS では利用できません。仕方がないので、キューを自作することにします。

*6 FIFO に対するプッシュをエンキュー (enqueue)、ポップをデキュー (dequeue) と呼ぶこともあります。

*7 `std::deque` と `std::list` は FIFO、FILO どちらとしても使えます。`std::vector` は FILO として使えます。また、既存のコンテナクラスをラップして `push` と `pop` メソッドを利用できるようにするコンテナアダプタとして `std::queue` と `std::stack` があります。

7.9 キューの実装

ここでは、C++ のテンプレート機能を用いてキューを実装してみようと思います。少し複雑に見えるかもしれませんが、C++ の練習にちょうどいい課題だと思いますので詳しく解説していきます。

まずキュークラス `ArrayQueue` の全体の形を **リスト 7.10** に示します。メンバ関数のプロトタイプ宣言とデータメンバが定義されているのが分かります。

リスト 7.10 `ArrayQueue` クラスの全景 (`queue.hpp`)

```
template <typename T>
class ArrayQueue {
public:
    template <size_t N>
    ArrayQueue(std::array<T, N>& buf);
    ArrayQueue(T* buf, size_t size);
    Error Push(const T& value);
    Error Pop();
    size_t Count() const;
    size_t Capacity() const;
    const T& Front() const;

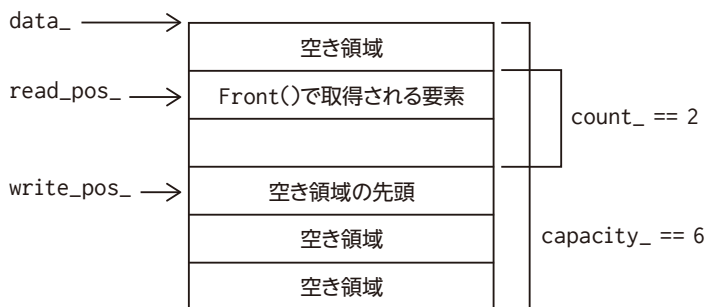
private:
    T* data_;
    size_t read_pos_, write_pos_, count_;
    /*
     * read_pos_ points to an element to be read.
     * write_pos_ points to a blank position.
     * count_ is the number of elements available.
     */
    const size_t capacity_;
};
```

クラス名の `ArrayQueue` は、このクラスが配列 (array) を実装に用いたキュー (queue) であることからこのように命名しました。

1 行目の `template <typename T>` は、以下に続く `ArrayQueue` クラスを **テンプレートクラス** とするための記述です。この記述のおかげで、クラス内部では `T` を何らかのデータ型を表す文字として扱うようになります。`T` は、このクラスを実際に使うときに `int` などの具体的な型に置き換わります。

まず、メンバ変数の説明をします。それぞれのメンバ変数がどんな値を表すかを **図 7.10** に示します。

図 7.10 ArrayQueue クラスのメンバ変数の関連



`data_` はキューに格納されるデータを実際に保管するための配列を指します。`capacity_` はその配列の要素数を表します。`capacity_` が `const` 宣言されていることから分かる通り、このキューは固定長のキューです。プッシュし続けてバッファが満杯になっても容量が自動拡張されたりはしない、ということです。

`read_pos_` はキューに格納されている先頭のデータを指します。このデータは、言い換えれば次にポップされる対象のデータであり、`Front()` で取得できるデータです。`write_pos_` はキューに格納されている末尾のデータの直後、すなわち空き領域の先頭を指します。ここは次にプッシュによってデータが書かれる場所です。

ここまでで、メンバ変数の値がどう変化するか、何を表すかを説明しました。もちろん、変数の値が勝手に変化するなんてことはありません。説明したことが実現するように頑張ってプログラムを書いていくのです。以降はプログラムを説明していきますので、変数の値の変化に注目して解説を読んでみてください。

リスト 7.11 ArrayQueue クラスのコンストラクタ (queue.hpp)

```
template <typename T>
template <size_t N>
ArrayQueue<T>::ArrayQueue(std::array<T, N>& buf) : ArrayQueue(buf.data(), N) {}

template <typename T>
ArrayQueue<T>::ArrayQueue(T* buf, size_t size)
    : data_{buf}, read_pos_{0}, write_pos_{0}, count_{0}, capacity_{size}
{}

```

リスト 7.11 に `ArrayQueue` クラスのコンストラクタを示します。コンストラクタは、クラス型の変数を定義する (= 実体化する、インスタンス化する) ときに呼び出される初期化用の関数です。2 つのコンストラクタのうち、2 引数の方をまずは見ていきます。引数 `buf` にはキューのデータを格納する場所となる配列を指定します。引数 `size` はその配列の要素数を指定します。受け取った 2 つの引数を使ってメンバ変数 `data_` と `capacity_` を初期化しているのが分かりますね。その他のメンバ変数の初期値は 0 としています。

1 引数コンストラクタは 2 引数コンストラクタに処理を移譲しています。1 引数コンストラクタは、その唯一の引数 `buf` から、配列の先頭ポインタと要素数という 2 つの情報を得て 2 引数コンストラクタ

タに渡しているわけですね。std::array は型そのものに要素数が組み込まれているため、引数として要素数を受け取る必要がありません。

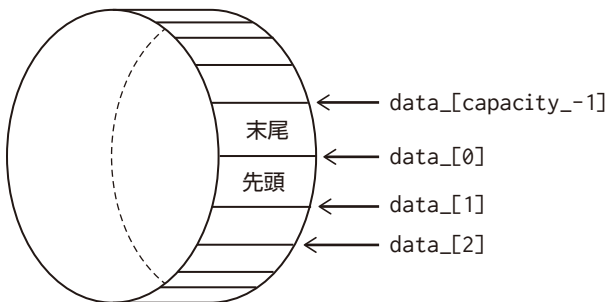
リスト 7.12 要素を末尾に追加する Push() (queue.hpp)

```
template <typename T>
Error ArrayQueue<T>::Push(const T& value) {
    if (count_ == capacity_) {
        return MAKE_ERROR(Error::kFull);
    }

    data_[write_pos_] = value;
    ++count_;
    ++write_pos_;
    if (write_pos_ == capacity_) {
        write_pos_ = 0;
    }
    return MAKE_ERROR(Error::kSuccess);
}
```

キューにデータをプッシュする関数 Push() をリスト 7.12 に示します。この関数はキューが満杯であればエラー Error::kFull を返して終了します。空いていれば、引数で与えられた値 value を write_pos_ が指す位置に書き込み、write_pos_ をインクリメントします。そしてここが最大のポイントですが、write_pos_ が配列の末尾を超えたら 0 にリセットします。この動作によって配列の末尾と先頭が繋がり、リングのように扱えるようになります (図 7.11)。

図 7.11 配列の末尾と先頭が繋がっている様子



リスト 7.13 先頭要素を削除する Pop() (queue.hpp)

```
template <typename T>
Error ArrayQueue<T>::Pop() {
    if (count_ == 0) {
        return MAKE_ERROR(Error::kEmpty);
    }

    --count_;
    ++read_pos_;
    if (read_pos_ == capacity_) {

```

```

    read_pos_ = 0;
}
return MAKE_ERROR(Error::kSuccess);
}

```

キューからデータをポップする関数 `Pop()` をリスト 7.13 に示します。この関数はキューが空であればエラー `Error::kEmpty` を返して終了します。そうでなければ先頭要素を削除します。削除と言っても `read_pos_` をインクリメントするだけです。インクリメントした結果、配列の末尾を超えたら 0 にリセットします。 `write_pos_` と同じですね。

リスト 7.14 先頭要素を取得する `Front()` (`queue.hpp`)

```

template <typename T>
const T& ArrayQueue<T>::Front() const {
    return data_[read_pos_];
}

```

`Pop()` はポップされた値を返しません。キューの先頭のデータを得るには `Front()` を使います (リスト 7.14)。 `Front()` は単に `read_pos_` が指すデータへの参照を返します。なぜ `Pop()` が値を返さないかは C++ 特有の事情があるのですが、OS 自作にはあまり関係ない話なのでここでは触れません。

関数名とプレース { の間にある `const` は、この関数 (ここでは `Front()`) はメンバ変数の値を変化させませんよ、ということを表します。 `Push()` や `Pop()` と違い、 `Front()` を呼び出すだけならキューの中身が変化しないということです。 `ArrayQueue` が使われる場面によってはこの `const` が効果を発揮します。

7.10 キューを使った割り込み高速化

実装したキューを使って割り込みハンドラの高速度を行います。キューを使うように改造した割り込みハンドラをリスト 7.15 に示します。

リスト 7.15 割り込みハンドラからメッセージをプッシュする (`main.cpp`)

```

struct Message {
    enum Type {
        kInterruptXHCI,
    } type;
};

ArrayQueue<Message>* main_queue;

__attribute__((interrupt))
void IntHandlerXHCI(InterruptFrame* frame) {
    main_queue->Push(Message{Message::kInterruptXHCI});
    NotifyEndOfInterrupt();
}

```

キューが扱うデータ型として、専用の Message 構造体を定義することにしました。割り込みハンドラからメイン関数に対して送信するメッセージという意味を含めています。この構造体はメッセージの種類を判別する type 値を持ちます。今はまだ xHCI からの割り込みでしか使わないので type の種類は 1 種類ですが、今後増えていく予定です。

割り込みハンドラの中では Message 構造体の値を生成してキューにプッシュするだけになりました。今まではマウスカーソルを動かす処理までやっていましたから、劇的に処理が軽くなりました^{*8}。理想的な割り込みハンドラです。

リスト 7.16 メッセージを繰り返し処理するイベントループ構造 (main.cpp)

```
while (true) {
    __asm__("cli");
    if (main_queue.Count() == 0) {
        __asm__("sti\n\thlt");
        continue;
    }

    Message msg = main_queue.Front();
    main_queue.Pop();
    __asm__("sti");

    switch (msg.type) {
    case Message::kInterruptXHCI:
        while (xhc.PrimaryEventRing()->HasFront()) {
            if (auto err = ProcessEvent(xhc)) {
                Log(kError, "Error while ProcessEvent: %s at %s:%d\n",
                    err.Name(), err.File(), err.Line());
            }
        }
        break;
    default:
        Log(kError, "Unknown message type: %d\n", msg.type);
    }
}
```

キューにメッセージを追加するだけではマウスカーソルは動きません。キューからメッセージを取り出して処理する必要があります。KernelMain() に実装した処理をリスト 7.16 に示します。ちょっと複雑ですが、大枠としてはキューからメッセージを 1 つ取り出し、その内容に応じて処理を行う仕組みになっています。今はまだ xHCI の処理しかありません。

リスト 7.17 メッセージをキューから取り出す処理 (main.cpp)

```
__asm__("cli");
if (main_queue.Count() == 0) {
    __asm__("sti\n\thlt");
    continue;
}

Message msg = main_queue.Front();
main_queue.Pop();
__asm__("sti");
```

キューからメッセージを取り出す部分だけを抜き出してリスト 7.17 に示します。前半部分でキューの要素数を確認し、空なら continue します。空でなければ先頭要素を取得して msg 変数に格納し、その後先頭要素をポップします。

1 行目では「3.3 初めてのカーネル」で紹介したインラインアセンブラの書き方を利用して cli 命令^{*9}を埋め込んでいます。cli 命令は CPU の割り込みフラグ (IF: Interrupt Flag) を 0 にする命令です。割り込みフラグは CPU 内の RFLAGS レジスタにあるフラグです。このフラグが 0 のとき、CPU は外部割り込み (xHCI からの割り込みとか) を受け取らなくなります。すなわち、割り込みハンドラ IntHandlerXHCI() は実行されなくなります。

なぜ cli 命令を実行するかというと、キューの操作中に割り込みがあると困ったことになるからです。専門用語で「データ競合」とか「並行性バグ」とかいうものです。割り込みはどんなタイミングで来るか分かりません。例えば Pop() の実行中に割り込みが発生し、割り込みハンドラから Push() が呼ばれることもあり得ます。これがどうしてダメなのか、少し詳しく説明します。

Pop() では count_ 変数をデクリメントします。デクリメント処理が次のように実現される場合を考えます (実際にどうなるかはコンパイラが出力するアセンブリ次第です)。

1. count_ 変数を RAX に読み込む
2. RAX から 1 を引く
3. RAX の値を count_ 変数に書き込む

処理 1 と処理 2 の間で割り込みが発生したとしましょう。割り込み発生時点では count_ の値は変化していません。次に割り込みハンドラの中で Push() が実行され count_ がインクリメントされます。割り込みから復帰すると、CPU は中断していた処理を再開します。つまり処理 2 と処理 3 が実行されます。RAX レジスタにはインクリメントされる前の count_ の値が格納されていますから、処理 2 では結果的に割り込みハンドラによる count_ のインクリメントが無視されることになります。最終的に、count_ には本来期待されるより 1 だけ小さい値が書かれることになります。

以上のシナリオは、あり得るデータ競合のごく一部にすぎません。現代の高度に複雑化した CPU やコンパイラ最適化のもとでは、思いもよらないデータ競合が発生する可能性があります。サンプルコードでは cli 命令で割り込みを禁止することでキュー操作にまつわるデータ競合を防ごうとしています。それとて十分とは限りません。ただ、本書はデータ競合を完璧に防ぐ手法が主題ではありませんので、簡単に効果の高いと思われる方法だけ紹介することにしました。データ競合について詳しくは「C++ Concurrency in Action」(参考文献 [11])などを参照してください。

さて、次は cli 命令で割り込みを受け取らない設定にした後の処理を説明します。最初にキューが空でない場合の処理を追います。キューが空でない (Count() が 0 でない) 場合、キューの先頭要素を取得して msg 変数に格納します。その後、キューの先頭要素をポップします。注目したいのはその直後の sti 命令^{*10}です。この命令は cli と対になる命令で、割り込みフラグ (IF) を 1 にする効果があります。割り込みフラグが 1 のとき、CPU は外部割り込みを受け取るようになります。キューの操作が終わったら忘れずに sti を実行しておかないと、いつまでも割り込みが発生せずマウスカーソルが動かなくなってしまいます。

* 8 本来は処理にかかる時間を計測して改善度合いを見るべきですが、まだその手段がないので、ここでは計測しないで済ませています。

* 9 Clear Interrupt Flag。

* 10 Set Interrupt Flag。

キューが空のときに実行される `__asm__("sti\n\thlt")` は `sti` 命令と `hlt` 命令を並べたものです*¹¹。インラインアセンブラでは `"\n\t"` で区切って複数の命令を並べることができるのです。`hlt` は CPU を省電力モードに切り替える命令でした。`hlt` を実行すると CPU が省電力モードになり、次に割り込みが発生するまで命令の実行が停止します。もし IF が 0 のときに `hlt` してしまうと省電力モードから復帰できなくなりますので、`hlt` の前に `sti` を実行する必要があります。

割り込みが発生すると CPU は省電力モードから復帰し、その割り込みに対応する割り込みハンドラを実行します。割り込みハンドラの処理が完了すると `hlt` の次の命令から処理が再開されます。`hlt` の次にあるのは `continue;` ですから、`while` ループの先頭に戻ります。この時、割り込みによりキューには 1 つ以上のメッセージが入っているはずですから、今度は `hlt` することなく処理が継続されるでしょう。このようにして、キューからメッセージを 1 つ取り出し、その内容に応じて処理を行うというループが回り続けます。この繰り返し処理のことをイベントループと呼びます。

話は少しずれますが、OS のメイン関数が呼ばれるときの IF フラグの状態はどうなっているでしょう。UEFI の仕様ではブートサービス実行中、すなわちブートローダが開始してから `gBS->ExitBootServices()` を呼ぶまでの間、割り込みが許可状態であると規定されています。`gBS->ExitBootServices()` が割り込みを禁止したりはしませんから、OS のメイン関数が呼ばれるときには IF=1 になっていると考えられます。実際、筆者の手元のパソコンはそうなっていました。メイン関数の開始からイベントループに突入する間に `sti` 命令を実行せずとも、最初から割り込みは有効な状態になっているため、うまくマウスが動いたのです。

この章では割り込みというものの紹介とキューを用いた割り込みハンドラの高速化を説明しました。高速化しただけで機能は変わっていませんから退屈な章だったと思いますが、今後の開発のために大事なことが盛りだくさんでした。明日からは新機能の開発を再開していきたいと思います。

* 11 `sti` 命令と直後の 1 命令の間では割り込みが起きない、という仕様を活用するために、隙間なく並べることが重要なのです。

第 8 章

メモリ管理

パソコンに搭載されたメモリの空き領域を把握し、アプリケーションの求めに応じて割り当てるメモリ管理機能は、OS の基本的な機能の 1 つです。この章では物理メモリの中の空き領域を探す方法と、空き領域と割り当て済み領域を管理する方法を紹介します。章の最後には malloc のような、動的にメモリを確保できる仕組みが整います。

8.1 メモリ管理

OSが動作するにしたがい、メモリが必要になったり不要になったりすることがよくあります。例えばアプリケーションを起動したりファイルを開いたりといった様々な処理で一時的なメモリ領域が必要になります。こういった一時領域はずっと必要なわけではなく、アプリケーションが終了したりファイルを閉じたりすればそのメモリ領域は不要になります。起動するアプリケーションの数や開くファイルの数は事前には分かりませんから、必要になったときに必要なだけメモリを確保し解放する機能が必要です。

こういった機能を**メモリ管理**と呼び、OSが備えるべき基本的な機能の1つです。メモリ管理の主要な機能（の1つ）はメモリ領域の確保と解放です。メモリを必要とする人に必要な量だけ専用のメモリ領域を確保し割り当て、不要になったメモリ領域が返却されたら解放を行います。これらを実現するにはパソコンに搭載されたメインメモリの中で、どの領域は未使用でどの領域が使用中なのかを管理することが必要です。管理しなかったら、同じメモリ領域を複数のアプリケーションが読み書きしてデータが壊れてしまったりします。

この章ではまず、OS起動直後のメインメモリの未使用領域を把握する方法を紹介します。そのためにUEFIで取得したメモリマップを使います。次に簡易なメモリ管理手法を紹介し、実装してみます。

8.2 UEFI メモリマップ (osbook_day08a)

メモリ管理をするにあたり、パソコンに搭載されたメインメモリの状況を把握しなければなりません。メインメモリのうち、どの領域が使用中で、どこが空いているのかを知らなければメモリ管理はできません。さて、メインメモリの状況はどうやったら分かるのでしょうか？ 実は「**第2章 EDK II 入門とメモリマップ**」で登場したメモリマップがまさにその情報を含んでいるのです。ということで、何れともあれUEFI BIOSの機能で取得したメモリマップをKernelMain()に渡すように改造しようと思います。

リスト 8.1 メモリマップ構造体のポインタをKernelMain()に渡す (Main.c)

```
typedef void EntryPointType(const struct FrameBufferConfig*,
                           const struct MemoryMap*);
EntryPointType* entry_point = (EntryPointType*)entry_addr;
entry_point(&config, &memmap);
```

実は、我々のブートローダはすでにUEFI BIOS側が持っている一部の情報をKernelMain()の引数として渡しています。それを真似て引数を追加するだけで作業完了です。改造部分のコードを**リスト 8.1**に示します。

リスト 8.2 メモリマップ構造体のポインタを受け取る (main.cpp)

```
extern "C" void KernelMain(const FrameBufferConfig& frame_buffer_config,
                          const MemoryMap& memory_map) {
```

リスト 8.2 に受け側の変更を示します。ブートローダ側の変更に合わせて引数を追加しただけです。

MemoryMap 構造体はもともと MikanLoaderPkg/Main.c で定義していました。ただ、今後は kernel/main.cpp でも同じ構造体定義を利用する必要があります。ということで、構造体の定義をまるごと memory_map.hpp に抜き出し、Main.c と main.cpp からインクルードするように改造しました。

リスト 8.3 MemoryMap 構造体と関連する型の定義 (memory_map.hpp)

```
#pragma once

#include <stdint.h>

struct MemoryMap {
    unsigned long long buffer_size;
    void* buffer;
    unsigned long long map_size;
    unsigned long long map_key;
    unsigned long long descriptor_size;
    uint32_t descriptor_version;
};

struct MemoryDescriptor {
    uint32_t type;
    uintptr_t physical_start;
    uintptr_t virtual_start;
    uint64_t number_of_pages;
    uint64_t attribute;
};

#ifdef __cplusplus
enum class MemoryType {
    kEfiReservedMemoryType,
    kEfiLoaderCode,
    kEfiLoaderData,
    kEfiBootServicesCode,
    kEfiBootServicesData,
    kEfiRuntimeServicesCode,
    kEfiRuntimeServicesData,
    kEfiConventionalMemory,
    kEfiUnusableMemory,
    kEfiACPIReclaimMemory,
    kEfiACPIMemoryNVS,
    kEfiMemoryMappedIO,
    kEfiMemoryMappedIOPortSpace,
    kEfiPalCode,
    kEfiPersistentMemory,
    kEfiMaxMemoryType
};

inline bool operator==(uint32_t lhs, MemoryType rhs) {
    return lhs == static_cast<uint32_t>(rhs);
}

inline bool operator==(MemoryType lhs, uint32_t rhs) {
```

```

    return rhs == lhs;
}

```

リスト 8.3に memory_map.hpp の中身を示します。このファイルは Main.c と main.cpp から利用されますので、C としても C++ としても利用できるようにしておく必要があります。enum class は C++ 特有の機能ですので、C としてはコンパイルできません。そのため、C++ 特有の機能は #ifdef __cplusplus で囲み、Main.c から読み込んだときにはその部分が有効にならないようにします。条件的に有効、無効を切り替えることを条件コンパイルと呼びます。

KernelMain() の中で、受け取ったメモリマップの中身を表示してみようと思います。

リスト 8.4 メモリマップ構造体のポインタを受け取る (main.cpp)

```

extern "C" void KernelMain(const FrameBufferConfig& frame_buffer_config,
                           const MemoryMap& memory_map) {

    《中略》

    printk("memory_map: %p\n", &memory_map);
    for (uintptr_t iter = reinterpret_cast<uintptr_t>(memory_map.buffer);
         iter < reinterpret_cast<uintptr_t>(memory_map.buffer) + memory_map.map_size;
         iter += memory_map.descriptor_size) {
        auto desc = reinterpret_cast<MemoryDescriptor*>(iter);
        for (int i = 0; i < available_memory_types.size(); ++i) {
            if (desc->type == available_memory_types[i]) {
                printk("type = %u, phys = %08lx - %08lx, pages = %lu, attr = %08lx\n",
                       desc->type,
                       desc->physical_start,
                       desc->physical_start + desc->number_of_pages * 4096 - 1,
                       desc->number_of_pages,
                       desc->attribute);
            }
        }
    }
}

```

リスト 8.4にメモリマップを表示するプログラムを示します。メモリマップはメモリディスクリプタの配列になっていますから、それを順番に表示するというプログラムにしています。注意したいのは、パソコンに搭載されている UEFI BIOS のバージョンによっては、MemoryDescriptor の定義と実際に取得されるメモリディスクリプタの構造が異なる可能性があることです。より新しい UEFI バージョンでは、もしかしたらメモリディスクリプタにメンバ変数が追加されるかもしれないわけです。

そういう可能性に備え、UEFI ではメモリマップそのものに加え、要素の大きさ（バイト数）も取得できるようになっています。その大きさを記録した変数が memory_map.descriptor_size です。変数 iter は、メモリマップの先頭アドレスから開始し、memory_map.descriptor_size ずつずらしながらメモリマップを一通り辿ります。iter という変数名は英語の iterator（反復子）から来ています。覚えていないかもしれませんが、この辺のプログラムは第 2 章で登場したのとはほぼ同じです。

iter は配列の要素の先頭アドレスを表しますから、それを MemoryDescriptor へのポインタヘキャストすれば、配列の要素を構造体として参照できるようになります。

8.3 データ構造の移動 (osbook_day08b)

UEFI から渡されたメモリマップを参照して空いているメモリ領域を探し、メモリ割り当て要求に応じて必要なメモリ領域を払い出すというのが主にやりたいことです。空きメモリ領域はメモリマップの type から簡単に判別できますから、後はその情報を使って空き領域と使用中の領域をきちんと管理すればいいだけです。指定されたメモリタイプ (リスト 8.4 における desc->type のこと) が空き領域かどうかを判定する IsAvailable() の実装をリスト 8.5 に示します。

リスト 8.5 IsAvailable() は空き領域を判定する (memory_map.hpp)

```
inline bool IsAvailable(MemoryType memory_type) {
    return
        memory_type == MemoryType::kEfiBootServicesCode ||
        memory_type == MemoryType::kEfiBootServicesData ||
        memory_type == MemoryType::kEfiConventionalMemory;
}

const int kUEFIPageSize = 4096;
```

UEFI 規格ではメモリ区画の用途に応じてメモリタイプをいくつか定義しています。UEFI を抜けた後^{*1}は空き領域として扱って良いことになっているメモリタイプがリスト 8.5 の 3 種類というわけです。IsAvailable() が真を返すメモリ領域は OS が自由に使って良いことになっているため、後で紹介するメモリ管理クラス BitmapMemoryManager でもその通り空き領域として扱います。

では早速メモリマップを参照してメモリ管理しましょう。と言いたいところですが、その前にちょっとした準備が必要です。その作業とはいくつかのデータ構造を OS が所有するメモリ領域 (IsAvailable() が未使用と判定しないメモリ領域) へと移動することです。UEFI は起動時に、x86-64 の 64 ビットモードで動作するために必要となるデータ構造を作成します。作成する場所はブートローダ専用データ領域 (kEfiBootServicesData) です。この領域は IsAvailable() が未使用と判定してしまう領域ですので、将来のメモリ割り当て要求に対してこの領域が払い出される可能性があります。UEFI が作成したデータ構造を別の場所へ移動しておかないと、他のデータで上書きされて CPU が誤動作する原因となってしまうのです。

移動すべきデータ構造は 3 つです。1 つ目はスタック領域です。スタック領域はプログラムの実行に欠かせない大事な領域ですが、UEFI から OS に処理が移ってきた直後は UEFI が準備したスタック領域を使ってしまっているため、移動の対象です。2 つ目は GDT^{*2} といって、セグメンテーションの設定を集めたものです。セグメンテーション (segmentation) はメモリを区画に分けるという意味が込められた言葉ですが、x86-64 の 64 ビットモードではもはや区画に分ける機能はなくなり、事実上無効化されました。ただし最低限の設定はしておかなければなりません。3 つ目はページテーブルといって、ページングという機能のための設定情報を集めたものです。ページングはメモリを固定長の区画 (ページ) に分割して管理する機能で、セグメンテーションに代わって現代のメモリ管理の主役になっています。ページングは「第 19 章 ページング」で詳しく扱いますので、この章では最低限の設定だけを紹介합니다。

* 1 正確には ExitBootServices() を呼び出した後。

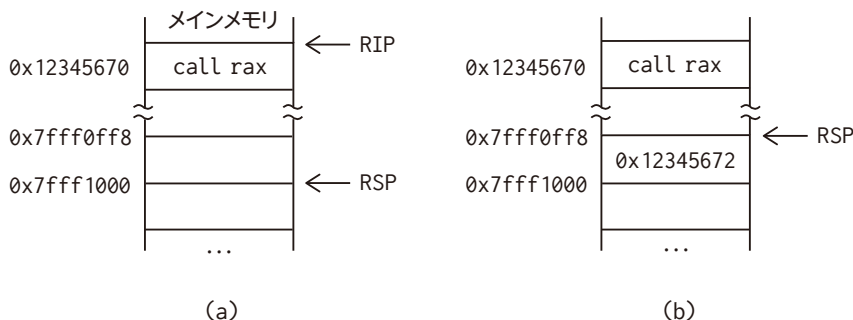
* 2 Global Descriptor Table

8.4 スタック領域の移動

手始めにスタック領域を UEFI 管理のものから OS 管理のものへと移し替えます。スタック領域の仕組みが分かれば後は単純な話ですので、まずはスタック領域の説明をします。

スタックは一般的には先入れ後出しのデータ構造のようですが、ここで扱うスタックは CPU がプログラムを実行する際に利用するメモリ領域のことで、CPU は `call` 命令で関数を呼び出す際に暗黙的にスタックに戻り先アドレスを保存します。また、C++ コンパイラは一時変数をスタックに確保します。このスタックの実体は RSP レジスタです。

図 8.1 スタック領域とスタックポインタ



CPU のレジスタの 1 つである RSP はメモリアドレスを保持しています。図 8.1 の a に、例として RSP に `0x7fff1000` が設定されている場合を示します。この状態で `call rax` 命令を実行すると、CPU は RSP の値を 8 減らし、そのメモリ領域へ現時点での RIP の値を書き込みます (図の b)。その後 RAX に格納されたアドレスにジャンプします。次に `ret` 命令を実行したとき、CPU は `call` とは逆の操作を行います。すなわち RSP が指すメモリ領域から 8 バイトを読み取り、RSP を 8 増やします。その後読み取ったアドレスにジャンプします。

スタック領域を移し替えるとは、すなわち RSP の値を書き換えることです。それだけです。分かっ
てしまえば簡単ですね。注意したいのは、図 8.1 のようにスタックはアドレス 0 の方向へ成長しますから、RSP の初期値はメモリ領域の末尾にしておく必要があるということです。

リスト 8.6 KernelMainNewStack() へと名前を変えた旧エントリポイント (main.cpp)

```
alignas(16) uint8_t kernel_main_stack[1024 * 1024];

extern "C" void KernelMainNewStack(
    const FrameBufferConfig& frame_buffer_config_ref,
    const MemoryMap& memory_map_ref) {
    FrameBufferConfig frame_buffer_config{frame_buffer_config_ref};
    MemoryMap memory_map{memory_map_ref};
```

リスト 8.6 に `main.cpp` の変更点を示します。主な変更は、新しいスタック領域となるメモリ領域 `kernel_main_stack` を定義し、`KernelMain()` の名前を `KernelMainNewStack()` という名前に変えました。また、引数で渡された 2 つのデータ構造を新しいスタック領域へとコピーするために 2

行を付け足しています。

リスト 8.7 新しいエントリポイント (asmfunc.asm)

```
extern kernel_main_stack
extern KernelMainNewStack

global KernelMain
KernelMain:
    mov rsp, kernel_main_stack + 1024 * 1024
    call KernelMainNewStack
.fin:
    hlt
    jmp .fin
```

リスト 8.7 にスタックを切り替えつつ `KernelMainNewStack()` を呼び出すプログラムを示します。関数名が `KernelMain()` となっていることに注目してください。こうすることで、UEFI はこれから C++ の関数ではなく、新しくアセンブリで書いた関数を呼び出してくれるようになります。

新しい `KernelMain()` では 1 行目で RSP を設定しています。設定している値は `main.cpp` で用意したメモリ領域 `kernel_main_stack` の末尾のアドレスとしています。RSP を設定した後でもととのメイン関数 `KernelMainNewStack()` を呼び出します。`.fin:` からの 3 行は念の為のプログラムです。`KernelMainNewStack()` からは戻って来ないはずですが、もし戻って来てしまった場合に備えて永久ループするようにしてあります。なぜなら、新しいスタック領域には `KernelMain()` からの戻り先アドレスが書かれていないからです。

8.5 セグメンテーションの設定

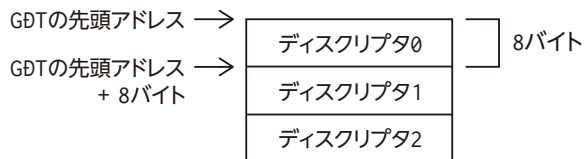
次はセグメンテーション用のデータの移動に取り掛かりましょう。やるべきことは GDT を OS 側の領域に再構築することです。

本書で対象としている x86-64 の 64 ビットモードでは、セグメンテーションは CPU の動作権限を決めるための機能といえます。CPU は現在の動作権限 (0 から 3 のいずれか) を持っていて、その権限で命令を実行していきます。0 を特権モードとかカーネルモード、3 をユーザーモードなどと呼ぶことがあります。セグメンテーションの仕組みを使うと CPU の動作権限を設定することができ、種々のアクセス制御に活用できます。特権モードでしか実行できない機械語命令 (特権命令) があったり、ページングと組み合わせることで特権モードでしかアクセスできないメモリ領域を設定することができるようになります。

セグメンテーションの設定に必要なデータ構造は **GDT** (グローバルディスクリプタテーブル) です。ディスクリプタと呼ばれる 8 バイトのデータ構造をいくつも並べた配列になっています (図 8.2)。ディスクリプタは「記述子」という意味の一般的な言葉です^{*3} が、この節では特に断らない限りは GDT の要素として登場するデータ構造のことをディスクリプタと呼んでいます。

*3 POSIX で登場するファイルディスクリプタや、USB デバイスに備わるデバイスディスクリプタなどもディスクリプタの一種。

図 8.2 3つのディスクリプタを持つ GDT の外観



リスト 8.8 グローバルディスクリプタテーブルの実体定義 (segment.cpp)

```
namespace {
    std::array<SegmentDescriptor, 3> gdt;
}
```

GDTの実体は `segment.cpp` の中で定義されたグローバル変数 `gdt` です。リスト 8.8 に変数定義をしている部分を示します。 `SegmentDescriptor` を要素として 3 つ持つ配列であることが分かりますね。変数 `gdt` は `segment.cpp` の中だけで参照できれば良いため、無名名前空間という C++ の機能を使って定義しています。無名名前空間の内側で定義した識別子はそのファイル外から見えなくなります。

リスト 8.9 セグメントディスクリプタの型定義 (segment.hpp)

```
union SegmentDescriptor {
    uint64_t data;
    struct {
        uint64_t limit_low : 16;
        uint64_t base_low : 16;
        uint64_t base_middle : 8;
        DescriptorType type : 4;
        uint64_t system_segment : 1;
        uint64_t descriptor_privilege_level : 2;
        uint64_t present : 1;
        uint64_t limit_high : 4;
        uint64_t available : 1;
        uint64_t long_mode : 1;
        uint64_t default_operation_size : 1;
        uint64_t granularity : 1;
        uint64_t base_high : 8;
    } __attribute__((packed)) bits;
} __attribute__((packed));
```

リスト 8.9 に GDT の個々の要素である `SegmentDescriptor` 構造体の定義を示します。ビットフィールドとして 8 バイトのデータ構造を定義しています。それぞれのフィールドの意味は後ほど紹介します。

リスト 8.10 セグメンテーションの設定 (main.cpp)

```
SetupSegments();

const uint16_t kernel_cs = 1 << 3;
const uint16_t kernel_ss = 2 << 3;
SetDSAll(0);
SetCSSS(kernel_cs, kernel_ss);

SetupIdentityPageTable();
```

GDT を再構築 (変数 `gdt` に値を設定) し、再構築した GDT の内容を CPU に反映させるプログラムを **リスト 8.10** に示します。このプログラムは `KernelMain()` の前の方に追加しましたが、実際にメモリを割り当てる処理を行う前ならどこに追加しても大丈夫だと思います。 `SetupSegments()` が GDT を再構築する処理です。その後の `SetDSAll()` と `SetCSSS()` により、再構築した GDT の内容が CPU (のセグメントレジスタ) に反映されます。最後の `SetupIdentityPageTable()` については「**8.6 ページングの設定**」で説明します。順番に見ていきましょう。

リスト 8.11 GDT を再構築するプログラム (segment.cpp)

```
void SetCodeSegment(SegmentDescriptor& desc,
                   DescriptorType type,
                   unsigned int descriptor_privilege_level,
                   uint32_t base,
                   uint32_t limit) {
    desc.data = 0;

    desc.bits.base_low = base & 0xfffffu;
    desc.bits.base_middle = (base >> 16) & 0xfffu;
    desc.bits.base_high = (base >> 24) & 0xfffu;

    desc.bits.limit_low = limit & 0xfffffu;
    desc.bits.limit_high = (limit >> 16) & 0xfu;

    desc.bits.type = type;
    desc.bits.system_segment = 1; // 1: code & data segment
    desc.bits.descriptor_privilege_level = descriptor_privilege_level;
    desc.bits.present = 1;
    desc.bits.available = 0;
    desc.bits.long_mode = 1;
    desc.bits.default_operation_size = 0; // should be 0 when long_mode == 1
    desc.bits.granularity = 1;
}

void SetDataSegment(SegmentDescriptor& desc,
                   DescriptorType type,
                   unsigned int descriptor_privilege_level,
                   uint32_t base,
                   uint32_t limit) {
    SetCodeSegment(desc, type, descriptor_privilege_level, base, limit);
    desc.bits.long_mode = 0;
    desc.bits.default_operation_size = 1; // 32-bit stack segment
}
```

```

void SetupSegments() {
    gdt[0].data = 0;
    SetCodeSegment(gdt[1], DescriptorType::kExecuteRead, 0, 0, 0xfffff);
    SetDataSegment(gdt[2], DescriptorType::kReadWrite, 0, 0, 0xfffff);
    LoadGDT(sizeof(gdt) - 1, reinterpret_cast<uintptr_t>(&gdt[0]));
}

```

リスト 8.11 に GDT を再構築する関数 `SetupSegments()` と、そこから利用される 2 つの関数 `SetCodeSegment()` と `SetDataSegment()` の実装を示します。

`SetupSegments()` では GDT にある 3 つのディスクリプタに値を設定しています。0 番目は 8 バイトをすべて 0 にしています。1 番目はコードセグメントディスクリプタとして、2 番目はデータセグメントディスクリプタとして設定しています。3 つのディスクリプタを設定したら、最後に `LoadGDT()` を呼び出すことで、変数 `gdt` を正式な GDT として CPU に登録します。この処理が終わると、CPU は UEFI が作った古い GDT を見なくなります。`LoadGDT()` は C++ では書けない命令を必要とするためアセンブリ言語で書いてあります。後ほど実装を紹介します。

GDT の 0 番目にあるディスクリプタをヌルディスクリプタ (null descriptor) と呼びます。GDT の 0 番目は使われないことになっているので 8 バイト全部を 0 で埋めておきます。

GDT の要素として設定できるディスクリプタにはいろいろな種類がありますが、今のところヌルディスクリプタ以外は**セグメントディスクリプタ**のみを扱います。指定したディスクリプタをセグメントディスクリプタとして設定するための関数が `SetCodeSegment()` および `SetDataSegment()` です。

この処理を理解するためにディスクリプタの各フィールドの意味を説明します (**表 8.1**)。

表 8.1 セグメントディスクリプタのフィールド

フィールド	意味
<code>base_...</code>	セグメントの開始アドレス
<code>limit_...</code>	セグメントのバイト数 -1
<code>type</code>	ディスクリプタタイプ
<code>system_segment</code>	1 ならコードまたはデータセグメント
<code>descriptor_privilege_level</code>	ディスクリプタの権限レベル
<code>present</code>	1 ならディスクリプタが有効
<code>available</code>	OS が自由に使って良いビット
<code>long_mode</code>	1 なら 64 ビットモード用のコードセグメント
<code>default_operation_size</code>	<code>long_mode</code> が 1 なら必ず 0 とする
<code>granularity</code>	1 ならリミットを 4KiB 単位として解釈する

セグメンテーションはメモリを区画に分けて管理する機能であるため、メモリ区画の開始アドレス (`base`) と区画の大きさ (`limit`) という属性を持っています。`base` は 32 ビット、`limit` は 20 ビットで設定するのですが、歴史的な経緯により設定すべきフィールドが飛び飛びになっています。64 ビットモードではそもそも `base` と `limit` は無視されるため値を設定する必要はありませんが、なんとなく値を設定できるようにしておきました。

`system_segment` フィールドと `type` フィールドを組み合わせでディスクリプタのタイプが決

定されます。system_segment を 1、type を 10 とすると、読み込み可能、実行可能なセグメントという意味になります。実行可能なセグメントのことをコードセグメントと呼びます。system_segment を 1、type を 2 とすると、読み書き可能、実行不能なセグメントという意味になります。実行不能なセグメントのことをデータセグメントと呼びます。x86_descriptor.hpp の中で DescriptorType::kExecuteRead は 10、DescriptorType::kReadWrite は 2 と定義されています。

実はこの type フィールドは、「第 7 章 割り込みと FIFO」で登場した割り込み記述子 (InterruptDescriptor 構造体) でも共通している項目です。割り込み記述子には system_segment フィールドがありませんが、構造体の定義をよく見ると system_segment があるべき位置が 1 ビットの無名フィールドになっていることが分かります*⁴。この無名フィールドの値は構造体の初期化時に 0 になるため、CPU から見ると system_segment が 0、type が 14、つまり割り込みゲート (interruptgate) ディスクリプタだということが分かるようになっていきます。

descriptor_privilege_level は略して DPL と呼ばれ、CPU の動作権限のレベルを 0 から 3 の数値で指定します。DPL の値の意味はディスクリプタの種類に応じて少し変わります。今回のようなコードセグメントディスクリプタに対する DPL の場合は、CPU の現在の権限レベル (CPL *⁵) を設定する役割があります。後述する SetCS() で CPU が参照するコードセグメントを切り替えますが、そのときに DPL の値が CPU の権限レベルとしてセットされます。CPU の権限レベルは特権命令の使用可否やページングでアクセス制限をするときなどに影響を及ぼします。今は権限レベル 0 (最も高い権限) しか使いません。

present は有効なディスクリプタでは必ず 1 にします。ここが 0 のディスクリプタを利用しようとすると CPU が例外を発生させます。available は CPU からは利用されないフィールドです。OS が好きな目的に使うことができますが、本書では特に利用しません。

long_mode はコードセグメント特有のフィールドで、64 ビット用コードかどうかを示します。1 であれば 64 ビットコードという意味です。本書で対象としている CPU の動作モードは IA-32e モード*⁶ですが、実はそのモードの中にはサブモードとして 64 ビットモードと互換モード (compatibility mode) があります。64 ビットモードでは 64 ビットコードが動き、互換モードでは 32 ビットおよび 16 ビットのコードが動きます。long_mode を 0 にすると互換モード用のセグメントであることを意味しますが、本書では互換モードを使いませんので long_mode は 1 とします。long_mode が 1 のときは default_operation_size を 0 にしなければなりません。

データセグメントでは long_mode フィールドは予約フィールドになっていますので、必ず 0 を設定しておきます。CPU が 64 ビットモードで動作中はデータセグメントの default_operation_size の設定は無視されるのですが、「第 20 章 システムコール」で出てくる syscall 命令と整合性を取るために 1 を設定しておきます。

granularity を 1 にすると limit が 4KiB 単位だと解釈されます。しかし 64 ビットモードではそもそも limit が無視されますからこのビットは意味がありません (サンプルコードではなんとなく 1 を設定しています)。

さて、ここまでで SetCodeSegment() および SetDataSegment() の処理の意味を簡単に説明しました。最終的に、GDT は 1 つのヌルディスクリプタと 2 つのセグメントディスクリプタが並んだ状

* 4 uint16_t : 1; の行。

* 5 Current Privilege Level

* 6 AMD ではロングモードと呼びます。

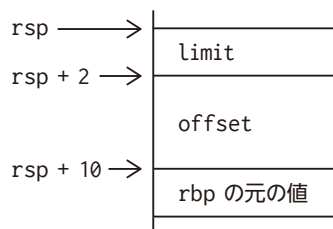
態になります。ここまで来れば GDT のデータ構造としては完成です。最後に LoadGDT() を呼び出して GDT の場所とサイズを CPU に登録する作業を行います。

リスト 8.12 LoadGDT() は GDT を GDTR に登録する (asmfunc.asm)

```
global LoadGDT ; void LoadGDT(uint16_t limit, uint64_t offset);
LoadGDT:
    push rbp
    mov rbp, rsp
    sub rsp, 10
    mov [rsp], di ; limit
    mov [rsp + 2], rsi ; offset
    lgdt [rsp]
    mov rsp, rbp
    pop rbp
    ret
```

リスト 8.12 に LoadGDT() の実装を示しました。C++ では記述できない lgdt 命令を使うためにアセンブリで書いてあります。この関数は 2 つの引数 limit と offset を取ります。limit は GDT のバイト数（から 1 を引いた数）、offset は GDT の先頭アドレスです。lgdt 命令はその 2 つの値を GDTR というレジスタに設定します。GDTR は GDT Register の略で、GDT の大きさや場所を記憶する CPU 内に存在する 80 ビット（10 バイト）幅のレジスタです。

図 8.3 lgdt 命令に渡す 10 バイトのメモリ領域の構造



lgdt 命令は limit と offset を 2 つのレジスタで受け取るのではなく、limit と offset を書き込んだ 10 バイトのメモリ領域を受け取る仕様です。したがって、LoadGDT() ではスタックに 10 バイトの空き領域を確保し、そこに limit と offset を書き込み、それを lgdt 命令に渡しています。10 バイトの領域の構造は図 8.3 のようになります。もちろん、x86-64 はリトルエンディアンですから、limit も offset もリトルエンディアンで書き込む必要があります。何も考えずに mov 命令を使っておけば自然とそうなるんですけどね。

sub rsp, 10 がスタックに空き領域を確保する命令です。こうすることでスタックポインタが 10 だけ動き、スタック上に 10 バイトの空き領域が確保されます。

mov [rsp], di はスタックポインタが指すアドレスから 2 バイトの領域に DI レジスタの内容をコピーする命令です。LoadGDT() の第 1 引数である limit は RDI レジスタにて渡されますので、そのレジスタの下位 16 ビットである DI レジスタに欲しい値が入っているというわけです。同様に mov [rsp + 2], rsi は limit の直後の 8 バイトの領域に RSI レジスタの内容をコピーします。第 2 引数の offset は RSI レジスタにて渡されますからこれでうまくいくはずですが。

さて、ここまでで GDT の設定と GDTR への登録が完了しました。**リスト 8.10** を見てこの後の処理を思い出すと、どうやら `SetDSAll()` と `SetCSSS()` という関数を呼び出していますね。これら 2 つの実装を見てみます。まずは `SetDSAll()` から。

リスト 8.13 `SetDSAll()` は DS や SS などのレジスタを設定する (asmfunc.asm)

```
global SetDSAll ; void SetDSAll(uint16_t value);
SetDSAll:
    mov ds, di
    mov es, di
    mov fs, di
    mov gs, di
    ret
```

リスト 8.13 に `SetDSAll()` の実装を示します。4 つのセグメントレジスタに引数をコピーするだけの単純な処理です。

`mov` 命令の第 1 オペランドに書いてある `ds` などは、セグメントレジスタと呼ばれる 16 ビットのレジスタです。セグメントレジスタは GDT のインデックス番号を保持し、メモリの読み書きの際にその番号のディスクリプタの属性に基づいてメモリ保護が行われます。ただ、x86-64 の 64 ビットモードでは DS、ES は利用されませんし、FS と GS も明示的にプログラマが指定しない限り利用されません。また、C++ コンパイラが知らないうちに FS や GS を利用するようなコードを生成することもあります。そこで `SetDSAll()` では引数に 0 を渡してやって、4 つのセグメントレジスタがヌルディスクリプタを指すようにしておきます。

SS (スタックセグメントレジスタ) も実質的にはまったく利用されないのですが、「**第 20 章 システムコール**」で出てくる `syscall` 命令との互換性を考えて適当なデータセグメントを設定しておくことにしました。実際の設定は、後で紹介する `SetCSSS()` でやっています。

例外は CS (コードセグメントレジスタ) です。CS は 64 ビットモードでも有効に作用し (ただし `limit` と `base` は無視されます)、すべての命令は実行時に CS が指すディスクリプタの設定内容に基づいてアクセス権の検査が行われます。例えば `SetDSAll()` の 4 つの `mov` 命令や `ret` 命令も、その実行時に CS の設定値の影響を受けます。

あれ? 何かおかしいですね。`SetDSAll()` を呼び出す時点ですでに GDTR は新しい GDT を指すように切り替わっているわけです。しかし CS の内容は `SetDSAll()` の後で更新しています。CS には UEFI が設定した値が入ったままですから、GDTR の再設定から CS を更新するまでの間は CS が意図しないディスクリプタを指すか、もしかしたら GDT の範囲外を指してしまっているかもしれません。

実はすべてのセグメントディスクリプタにはプログラマがアクセスできない隠し領域^{*7}があり、先の問題を解決しています。隠し領域はセグメントディスクリプタと同様の内容 (リミット、ベース、属性) を保持できるようになっています。`mov` 命令などによってセグメントレジスタが更新されると、GDT から対応するディスクリプタが読み取られて隠し領域に設定されます。次にセグメントレジスタが更新されるまでの間、この隠し領域の情報が利用されるため、GDT の内容が書き換わったり GDTR が再設定されたりしても、その影響が直ちに現れることを防いでくれるのです。GDTR の再設定から CS を更新するまでの間、プログラムが動き続けられるのは隠し領域のおかげだったのです。

*7 ディスクリプタキャッシュやシャドウレジスタと呼ばれることもあります。

リスト 8.14 SetCSSS() は CS レジスタを設定する (asmfunc.asm)

```
global SetCSSS ; void SetCSSS(uint16_t cs, uint16_t ss);
SetCSSS:
    push rbp
    mov rbp, rsp
    mov ss, si
    mov rax, .next
    push rdi ; CS
    push rax ; RIP
    o64 retf
.next:
    mov rsp, rbp
    pop rbp
    ret
```

ということで安心して SetCSSS() の説明に移れます (リスト 8.14)。この関数は CS と SS に値を設定するための関数です。SS は DS などと同様に mov 命令で設定できるのですが、CS だけは mov 命令による書き換えができません。代わりに retf 命令を使うことにしました。

retf は far return のことで、本来は far call で呼び出された関数から復帰するときに使う命令です。far call はセグメント間ジャンプの一種で、異なるコードセグメントにある関数へジャンプするときに使います。far ではない call 命令による普通の関数呼び出しでは、スタックには戻り先アドレスだけを積みます。一方で far call では戻り先アドレスに加えて現在の CS の値もスタックに積みます。そして、far return により復帰するときにはその逆の操作、すなわち、スタックから値を取ってきて CS と RIP に設定します。

CS を書き換えるために retf の動作を応用します。あたかも far call で関数が呼び出された直後かのようなスタックの状態を再現してから retf を呼ぶことで CS を希望の値に変更するわけです。retf はデフォルトで 32 ビットの値をスタックから取ってきてしまいます。64 ビットの値を取ってきてほしいので、オペランドサイズを変更するために o64 を指定します*⁸。

スタックに積んだ RIP の値は retf にとっては戻り先のアドレスです。この部分には SetCSSS() 内のラベル .next の値を設定していますから、retf を実行すると .next にジャンプすることになります。 .next にジャンプしたら、後は関数のお片付けをしてから普通の ret 命令によって関数を終了し、SetCSSS() の呼び出し元に戻ります。

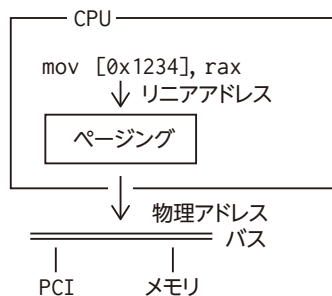
SetCSSS() により CS には 1 << 3 が、SS には 2 << 3 が設定されます。すなわち CS は gdt[1] を、SS は gdt[2] を指した状態になります。これで UEFI が設定した GDT から完全に脱却できました。

8.6 ページングの設定

ページングとはメモリアドレス空間をページ単位で管理する仕組みです。ページはメモリ空間を固定長に区切ったもので、x86-64 においては 4KiB、2MiB、1GiB のページサイズが規定されています。最もよく使われるのが 4KiB のページです。x86-64 の 64 ビットモードではページングの利用が必須（無効にできない）ですので、ページングに関する設定も OS 用の領域に移動する必要があります。

* 8 o64 は NASM の独自の書き方。

図 8.4 リニアアドレスから物理アドレスへの変換



ページングは「第 19 章 ページング」で詳しく扱いますが、ページングの機能を一言でいうと**リニアアドレス** (linear address) を物理アドレス (physical address) に変換する機能です。リニアアドレスはソフトウェアが指定するアドレス、物理アドレスは CPU がメモリ^{*9}を読み書きするときに使うアドレスです。C++ とかアセンブリ言語のプログラムの中でメモリの読み書きをするとき (例えば CPU が mov 命令を実行するとき) に指定するアドレスはリニアアドレス^{*10}です。CPU はページテーブルを参照し、リニアアドレスを物理アドレスに変換してからメモリの読み書きを行います (図 8.4)。

ページングを用いるとページ単位でアドレス変換を設定することができます。そのため入り組んだ複雑な変換を行うこともできるのですが、現時点では特にそうする意味がありませんので、リニアアドレスと物理アドレスを一致させておくことにします。プログラム上でのアドレス表記と実際にアクセスされるアドレスが等しくなる設定を**アイデンティティマッピング**と呼びます。この章ではページングの詳しい説明は省き、とりあえずアイデンティティマッピングができる最低限の設定方法だけを説明します。

リスト 8.15 アイデンティティマッピング用の階層ページング構造の作成 (paging.cpp)

```
namespace {
    const uint64_t kPageSize4K = 4096;
    const uint64_t kPageSize2M = 512 * kPageSize4K;
    const uint64_t kPageSize1G = 512 * kPageSize2M;

    alignas(kPageSize4K) std::array<uint64_t, 512> pml4_table;
    alignas(kPageSize4K) std::array<uint64_t, 512> pdp_table;
    alignas(kPageSize4K)
        std::array<std::array<uint64_t, 512>, kPageDirectoryCount> page_directory;
}

void SetupIdentityPageTable() {
    pml4_table[0] = reinterpret_cast<uint64_t>(&pdp_table[0]) | 0x003;
    for (int i_pdpt = 0; i_pdpt < page_directory.size(); ++i_pdpt) {
        pdp_table[i_pdpt] = reinterpret_cast<uint64_t>(&page_directory[i_pdpt]) | 0x003;
    }
}
```

*9 メインメモリはもちろん、ほかにも PCI バスなどのメモリアドレス空間に接続された各種デバイスのアクセスで物理アドレスが用いられます。

*10 正確に言えばソフトウェアが指定するアドレスは論理アドレス (logical address) で、セグメンテーションにより論理アドレスが変換されてリニアアドレスとなります。しかし 64 ビットモードでは、セグメンテーションによるアドレス変換は行われないため、論理アドレス = リニアアドレスとなります。

```

    for (int i_pd = 0; i_pd < 512; ++i_pd) {
        page_directory[i_pdpt][i_pd] = i_pdpt * kPageSize1G + i_pd * kPageSize2M | 0x083;
    }
}

SetCR3(reinterpret_cast<uint64_t>(&pml4_table[0]));
}

```

リスト 8.15 にアイデンティティマッピングの設定を行う関数 `SetupIdentityPageTable()` の実装を示します。64 ビットモードにおけるページングの設定は、次に示す 4 階層の構造になっています。

- ページマップレベル 4 テーブル (PML4 table)
- ページディレクトリポインタテーブル (PDP table)
- ページディレクトリ
- ページテーブル

ページングが最初に登場したときはページディレクトリとページテーブルの 2 階層だけがありました。ページディレクトリはページテーブルへのポインタの配列です。この関係は、ファイルシステムにおけるディレクトリがファイルのリストになっているのと似ています。かなり分かりやすいネーミングだと筆者は思います。しかし、x86-64 の 64 ビットモードで 4 レベルページング^{*11}に拡張され、ディレクトリより上の階層が登場したために分かりにくい名前になってしまいました。4 レベルページングではページマップレベル 4 テーブルが最上位の階層です。

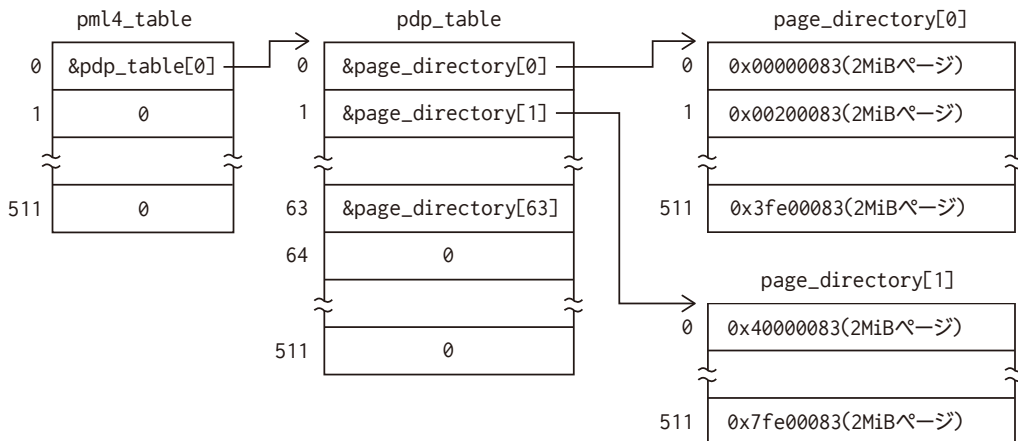
ページテーブルという言葉は、狭義には最下層の構造のことを指しますが、広義には階層的な構造全体を指すことがあります。ちょっと紛らわしいですね。**参考文献 [3]** には階層構造全体を指す言葉として階層ページング構造 (hierarchical paging structure) と書かれていますので、本書ではそう呼ぶことにします。

`SetupIdentityPageTable()` ではこの 4 階層の変換テーブルをそれぞれ設定しているのが分かります。関数の最初の行では PML4 テーブルの先頭要素 `pml4_table[0]` に、PDP テーブルの先頭アドレスを設定しています。2 行目の `for` 文では PDP テーブルの各要素 `pdp_table[i_pdpt]` に、ページディレクトリの先頭アドレスを設定しています。内側の `for` 文ではページディレクトリの各要素を設定しています。

`kPageDirectoryCount` は定数で、値は 64 です。作成するページディレクトリの個数を表しています。1 つのページディレクトリで表せるメモリ領域は $2\text{MiB} \times 512 = 1\text{GiB}$ です。したがって 64GiB までの領域をアイデンティティマップすることになります。

* 11 ここでいうレベル (level) とは階層という意味。

図 8.5 構築した階層ページング構造の全体像



`SetupIdentityPageTable()` が最終的に形作る階層ページング構造は図 8.5 のようになります。4 レベルページングといいつつ、すべて 2MiB ページで構成してあるので 3 階層になっていますが。ページディレクトリの各要素に書き込む値に `0x083` というようにビット和を行っていますね。ページディレクトリの各要素のビット 7 を 1 にすることで 2MiB ページになるのです。アイデンティティマッピングでは細かい単位でページを組み替えたりはしませんので、なるべく大きなページを採用することで作成すべきデータ構造を小さくできます。本来なら最大のページである 1GiB ページを採用したいところですが、1GiB ページをサポートしない CPU があるようですので、互換性を考えて 2MiB ページを採用しました。

リスト 8.16 `SetCR3()` は CR3 レジスタに値を書く (`asmfunc.asm`)

```
global SetCR3 ; void SetCR3(uint64_t value);
SetCR3:
    mov cr3, rdi
    ret
```

`SetupIdentityPageTable()` が最後に実行している `SetCR3()` の実装をリスト 8.16 に示します。この関数は PML4 テーブルの物理アドレスを CR3 レジスタに設定します。CR3 を書き換えた直後から CPU は新しい階層ページング構造を使ってアドレス変換をするようになります。 `SetCR3()` の直前までは UEFI が用意した階層ページング構造が使われていましたが、これからは自分で用意したものが使われるようになります。やったね！

8.7 メモリ管理に挑戦 (osbook_day08c)

3つのデータ構造を kEfiBootServicesData から OS 管理下の領域へ移動させることができました。やっと本来やりたかったメモリ管理ができそうです。この節ではビットマップを用いた簡易なメモリマネージャ BitmapMemoryManager を作り、UEFI から受け取ったメモリマップを使ってメモリマネージャを初期化します。メモリマネージャ (memory manager) とは、メモリを管理 (manage) する者、という意味です。

これから作成する BitmapMemoryManager はメモリ領域を 4KiB ページフレーム単位で管理するメモリマネージャにしようと思います。このようにしておくで後々のページングと相性が良いからです。ページフレーム単位で、あるページフレームは使用中、あるページフレームは未使用、というのが分かるようになれば目標達成です。

なお、ページではなくページフレームと書いているのは意味があります。ページはリニアアドレス上の区画(4KiB ページなら 4KiB)を表し、ページフレームは物理アドレス上の区画を表すのです。すなわち、これから作ろうとしているメモリマネージャは物理アドレスの使用状況を管理する機能であって、リニアアドレスの使用状況は管理しませんよ、という気持ちが込められています。この辺の話はページングをきちんと学んでからでないと理解しにくいですし、今のところはリニアアドレスと物理アドレスが一致していますので、ページとページフレームの違いは意識しなくて大丈夫です。なぜこんな話を書いたかという、以降で紹介するプログラムに frame という単語が登場するので困惑しないように、ということです。

ページフレーム単位で管理するやり方はいくつか考えられます。今回採用する方法は1ページフレームにつき1ビットを使い、ビットが1なら使用中、0なら未使用ということにして管理する方法です。このような、ビットによって状態を表すものをビットマップと呼びます。この方式のいいところはページフレーム番号とビット位置が1対1に紐付き、処理がとても分かりやすいことです。準備としてページフレーム番号を表すデータ型を作成しました。

リスト 8.17 単位演算子とページフレーム ID を表す型の定義 (memory_manager.hpp)

```
namespace {
    constexpr unsigned long long operator""_KiB(unsigned long long kib) {
        return kib * 1024;
    }

    constexpr unsigned long long operator""_MiB(unsigned long long mib) {
        return mib * 1024_KiB;
    }

    constexpr unsigned long long operator""_GiB(unsigned long long gib) {
        return gib * 1024_MiB;
    }
}

/** @brief 物理メモリフレーム1つの大きさ (バイト) */
static const auto kBytesPerFrame{4_KiB};

class FrameID {
public:
    explicit FrameID(size_t id) : id_{id} {}
    size_t ID() const { return id_; }
```

```
void* Frame() const { return reinterpret_cast<void*>(id_ * kBytesPerFrame); }

private:
    size_t id_;
};

static const FrameID kNullFrame{std::numeric_limits<size_t>::max()};
```

リスト 8.17 にページフレーム番号を表す型 `FrameID` と、補助的な関数群の定義を示します。まず `FrameID` の定義を見てみると、これは内部に `size_t` 型の変数を 1 つもつクラスだと分かります。ページフレーム番号は単なる整数値なのですが、だからといって `size_t` をそのまま使っていると「これはページ番号なのか？ ページフレーム番号なのか？ もしかしたらページの大きさかも？」などと混乱するかもしれません。単なる整数値であってもクラス（または構造体）でラップすることで混乱を避けることができます。

`FrameID` の定義の中に `kBytesPerFrame` という定数が登場します。この定数は `FrameID` の定義の直前で定義されていて、初期値は `4_KiB` となっています。この書き方はちょっと見たことない方も多かもしれません。先頭が数字だから識別子^{*12}ではないようですし、後ろに `_KiB` と続いているので単なる整数リテラル^{*13}にも見えません。

実はこの書き方は C++11 から導入されたユーザー定義リテラルというものです。ユーザー定義リテラルは `operator""` サフィックスという形の演算子定義をオーバーロードして作成します。**リスト 8.17** のコード例にはサフィックスとして `_KiB`、`_MiB`、`_GiB` の 3 つのバージョンを定義しています。このように定義した演算子を使って `4_KiB` と書くと、`operator""_KiB` の引数 `kib` に 4 が渡され、その実行結果 4096 が返されます。すなわち `kBytesPerFrame` は `unsigned long long` 型の 4096 を値として持つ定数になります。

最後に「未定義のページフレーム番号」を表す定数として `kNullFrame` を定義しています。この定数は、例えば特定のページフレームを探す関数でページフレームが見つからなかった際の戻り値などで使おうと思っています。

リスト 8.18 メモリマネージャに UEFI のメモリマップを伝える (main.cpp)

```
::memory_manager = new(memory_manager_buf) BitmapMemoryManager;

const auto memory_map_base = reinterpret_cast<uintptr_t>(memory_map.buffer);
uintptr_t available_end = 0;
for (uintptr_t iter = memory_map_base;
     iter < memory_map_base + memory_map.map_size;
     iter += memory_map.descriptor_size) {
    auto desc = reinterpret_cast<const MemoryDescriptor*>(iter);
    if (available_end < desc->physical_start) {
        memory_manager->MarkAllocated(
            FrameID{available_end / kBytesPerFrame},
```

*12 識別子 (identifier) とは、変数名や関数名のこと。識別子の先頭に数字は使えません。

*13 リテラル (= 即値) とは、ソースコード中に埋め込まれた数値や文字列のこと。例えば `char c="abc"[1];` というコードの `"abc"` は文字列リテラル、`1` は整数リテラルです。

```

        (desc->physical_start - available_end) / kBytesPerFrame);
    }

    const auto physical_end =
        desc->physical_start + desc->number_of_pages * kUEFIPageSize;
    if (IsAvailable(static_cast<MemoryType>(desc->type))) {
        available_end = physical_end;
    } else {
        memory_manager->MarkAllocated(
            FrameID{desc->physical_start / kBytesPerFrame},
            desc->number_of_pages * kUEFIPageSize / kBytesPerFrame);
    }
}
memory_manager->SetMemoryRange(FrameID{1}, FrameID{available_end / kBytesPerFrame});

```

メモリマネージャの実装を紹介する前に、その利用方法を見えます。**リスト 8.18** にメモリマネージャに対して使用中領域を設定するプログラムを示します。初期状態ではメモリマネージャはメモリ領域全体を未使用だと思っているので、MarkAllocated() メソッドを通じて使用中領域を教えてやりえます。IsAvailable() が偽を返す領域、および、メモリマップ上で歯抜けになっている領域は使用中ということです。そのような領域を見つけたら、領域の先頭ページフレーム番号と領域の大きさ（ページフレーム単位）を計算してMarkAllocated() メソッドに渡します。

desc->physical_start はメモリ領域の先頭の物理アドレスが書かれていますので、物理アドレスをページフレームの大きさで割ればページフレーム番号になります。アドレス 0 から 4095 がページフレーム 0、アドレス 4096 から 8192 がページフレーム 1 という具合です。

領域の大きさの計算は少々複雑です。なぜなら、UEFI 規格におけるページサイズと、今から作ろうとしているメモリマネージャが管理対象とするページフレームのサイズが一致するとは限らないからです。desc->number_of_pages は UEFI 規格のページサイズを基準としたページ数が書かれていますから、メモリマネージャで扱わせるためには単位変換が必要です。kUEFIPageSize を掛け算することでバイト単位に変換し、さらに kBytesPerFrame で割ってページフレーム単位に変換します^{*14}。

使用中領域のマーキングが済んだら、最後にメモリマネージャの SetMemoryRange() メソッドを使い、物理メモリの大きさを設定します。これをしないとメモリマネージャは際限なく大きな物理メモリが積まれていると勘違いしたままになってしまいます。available_end 変数は「最後の未使用領域の末尾のアドレス」を記録しています。通常、この値はパソコンに積まれたメインメモリの量とほぼ一致するはずです。

リスト 8.19 ビットマップ方式でメモリを管理する BitmapMemoryManager クラス (memory_manager.hpp)

```

class BitmapMemoryManager {
public:
    /** @brief このメモリ管理クラスで扱える最大の物理メモリ量 (バイト) */
    static const auto kMaxPhysicalMemoryBytes{128_GiB};
    /** @brief kMaxPhysicalMemoryBytes までの物理メモリを扱うために必要なフレーム数 */
    static const auto kFrameCount{kMaxPhysicalMemoryBytes / kBytesPerFrame};

    /** @brief ビットマップ配列の要素型 */
    using MapLineType = unsigned long;
    /** @brief ビットマップ配列の 1 つの要素のビット数 == フレーム数 */

```

```

static const size_t kBitsPerMapLine{8 * sizeof(MapLineType)};

/** @brief インスタンスを初期化する. */
BitmapMemoryManager();

/** @brief 要求されたフレーム数の領域を確保して先頭のフレーム ID を返す */
WithError<FrameID> Allocate(size_t num_frames);
Error Free(FrameID start_frame, size_t num_frames);
void MarkAllocated(FrameID start_frame, size_t num_frames);

/** @brief このメモリマネージャで扱うメモリ範囲を設定する.
 * この呼び出し以降, Allocate によるメモリ割り当ては設定された範囲内でのみ行われる.
 *
 * @param range_begin_ メモリ範囲の始点
 * @param range_end_   メモリ範囲の終点. 最終フレームの次のフレーム.
 */
void SetMemoryRange(FrameID range_begin, FrameID range_end);

private:
std::array<MapLineType, kFrameCount / kBitsPerMapLine> alloc_map_;
/** @brief このメモリマネージャで扱うメモリ範囲の始点. */
FrameID range_begin_;
/** @brief このメモリマネージャで扱うメモリ範囲の終点. 最終フレームの次のフレーム. */
FrameID range_end_;

bool GetBit(FrameID frame) const;
void SetBit(FrameID frame, bool allocated);
};

```

リスト 8.19 にメモリマネージャを表すクラス `BitmapMemoryManager` の全体像を示します。このクラスの最も核となるデータ構造は `alloc_map_` です。これは 1 ページフレームを 1 ビットで表したビットマップです。必要なビット数、すなわちフレーム数は物理アドレスの最大値 / フレームサイズとなり、これを `kFrameCount` という定数で表します。

管理領域の無駄をなくすために 1 バイトに 8 ビットを詰め込んで管理することにしました。`alloc_map_` の要素型を `unsigned long` として、`kFrameCount` を `unsigned long` のビット数で割れば、必要な配列の要素数を求めることができますね。

2 つのメンバ変数 `range_begin_` と `range_end_` は、このメモリマネージャが扱う、つまり将来的にメモリ割り当てを行うメモリの範囲を表します。`SetMemoryRange()` がこれらの変数を設定します。

リスト 8.20 `MarkAllocated()` は使用中領域を設定する (`memory_manager.cpp`)

```

void BitmapMemoryManager::MarkAllocated(FrameID start_frame, size_t num_frames) {
    for (size_t i = 0; i < num_frames; ++i) {
        SetBit(FrameID{start_frame.ID() + i}, true);
    }
}

```

* 14 実は UEFI ページは 4KiB と決まっていますので、`kBytesPerFrame` が 4KiB である限り単位変換は不要です。将来的にページフレームの大きさを変えて遊ぶときにバグとならないように、念のため単位変換を行っています。

さて、クラス定義の大枠が分かったので、個々のメソッド定義を見ていきます。**リスト 8.20** に `MarkAllocated()` の実装を示します。このメソッドは引数に使用中領域の先頭ページフレーム番号と領域の大きさ（ページフレーム単位）を受け取ります。処理は非常に単純で、ビットマップの中で、引数で指定された範囲に対応するビットを `true` に設定するだけです。`SetBit()` は後で紹介しますが、`alloc_map_` の指定されたビットに指定された値を書き込むだけの関数です。

リスト 8.21 `SetMemoryRange()` はメモリマネージャが扱うアドレス範囲を設定する (`memory_manager.cpp`)

```
void BitmapMemoryManager::SetMemoryRange(FrameID range_begin, FrameID range_end) {
    range_begin_ = range_begin;
    range_end_ = range_end;
}
```

リスト 8.21 に `SetMemoryRange()` の実装を示します。引数で指定された物理アドレスの範囲をメンバ変数に設定しているだけの超単純なメソッドです。

リスト 8.22 ビットを読み書きする 2 つのメソッドの定義 (`memory_manager.cpp`)

```
bool BitmapMemoryManager::GetBit(FrameID frame) const {
    auto line_index = frame.ID() / kBitsPerMapLine;
    auto bit_index = frame.ID() % kBitsPerMapLine;

    return (alloc_map_[line_index] & (static_cast<MapLineType>(1) << bit_index)) != 0;
}

void BitmapMemoryManager::SetBit(FrameID frame, bool allocated) {
    auto line_index = frame.ID() / kBitsPerMapLine;
    auto bit_index = frame.ID() % kBitsPerMapLine;

    if (allocated) {
        alloc_map_[line_index] |= (static_cast<MapLineType>(1) << bit_index);
    } else {
        alloc_map_[line_index] &= ~(static_cast<MapLineType>(1) << bit_index);
    }
}
```

リスト 8.22 に `GetBit()` と `SetBit()` の定義を示します。どちらのメソッドも冒頭で引数 `frame` で指定されたページフレーム番号をビットマップ `alloc_map_` におけるビット位置に変換しています。`line_index` が `alloc_map_` の添字、`bit_index` がその添字で指定された要素の中でのビット位置を表します。

図 8.6 ビットマップ中でのビット位置計算

6 6 6	...	2 1 0	
3 2 1	...		

<code>alloc_map_[0]</code>	0 0 0	0 0 x	← フレーム0: <code>line_index=0</code> , <code>bit_index=0</code>
<code>alloc_map_[1]</code>	0 0 0	y 0 0	← フレーム66: <code>line_index=1</code> , <code>bit_index=2</code>

line_indexとbit_indexの関係を図8.6に示します。ビット番号をalloc_map_の1要素のビット数kBitsPerMapLineで割ることでalloc_map_の添字を得られます。また、割り算の余りが1要素の中でのビット位置を表します。

ここまででメモリマネージャに使用中領域を教え、物理メモリの範囲を設定する機能を説明しました。これらはメモリマネージャを使うための準備段階でした。ここからはメモリマネージャの最も大切な機能である、メモリ割り当てと解放の処理を見ていきましょう。

メモリの割り当ての基本は、指定された大きさ以上の未使用領域を探し、そのような領域があれば使用中マークを付け、その領域の位置を返すことです。未使用領域の探し方や、複数の候補があった場合の領域を割り当て方などによっていろいろなやり方があります。ここではなるべく単純な方法でやってみたいと思います。これから紹介するアルゴリズムは、専門用語でファーストフィット (first fit) と呼ばれるものです。

リスト 8.23 Allocate() は指定したフレーム数のメモリ領域を確保する (memory_manager.cpp)

```
WithError<FrameID> BitmapMemoryManager::Allocate(size_t num_frames) {
    size_t start_frame_id = range_begin_.ID();
    while (true) {
        size_t i = 0;
        for (; i < num_frames; ++i) {
            if (start_frame_id + i >= range_end_.ID()) {
                return {kNullFrame, MAKE_ERROR(Error::kNoEnoughMemory)};
            }
            if (GetBit(FrameID{start_frame_id + i})) {
                // "start_frame_id + i" にあるフレームは割り当て済み
                break;
            }
        }
        if (i == num_frames) {
            // num_frames 分の空きが見つかった
            MarkAllocated(FrameID{start_frame_id}, num_frames);
            return {
                FrameID{start_frame_id},
                MAKE_ERROR(Error::kSuccess),
            };
        }
        // 次のフレームから再検索
        start_frame_id += i + 1;
    }
}
```

リスト 8.23 に指定したサイズのメモリ領域を割り当てる Allocate() メソッドを示します。引数に割り当てたいページフレームの数を渡すと、連続でその大きさ以上の空きがある領域を探して割り当てます。

探し方は次の通りです。このメモリマネージャが担当するメモリ領域の先頭 (range_begin_) から探索を開始します。まず、連続で num_frames 分の未使用フレームがあるかを検査します。for 文の内側にある GetBit(FrameID{start_frame_id + i}) が一度も真にならないとき、すなわち num_frames 個のフレームが連続して未使用であるとき、break; は実行されません。その場合にのみ for 文の次にある if (i == num_frames) が成り立ち、未使用フレームが num_frames 個連続していることが分かります。次に今しがた発見した num_frames 個の未使用ページに使用中マークを

付けます。最後に領域先頭のフレーム番号 (start_frame_id) を返して終了です。

リスト 8.24 Free() は指定されたメモリ領域を未使用状態にする (memory_manager.cpp)

```
Error BitmapMemoryManager::Free(FrameID start_frame, size_t num_frames) {
    for (size_t i = 0; i < num_frames; ++i) {
        SetBit(FrameID{start_frame.ID() + i}, false);
    }
    return MAKE_ERROR(Error::kSuccess);
}
```

確保した領域が不要になり、返却されることがあります。**リスト 8.24** にメモリ領域を返却するための Free() メソッドを示します。確保に比べ非常に単純で、引数で指定されたフレームの範囲について未使用状態にするだけです。

malloc と free を知っている方はこう思うかもしれません。「free はポインタ(領域の先頭アドレス)を渡すだけでいいのに、Free() は先頭のフレーム番号と領域の大きさを指定するようになっているのはなぜだろう？」と。この理由はメモリマネージャの作りやすさのためです。先頭のフレーム番号さえ渡せば返却できるようにするためには、確保時にサイズをどこかに覚えておく必要がありますが、今回、それをやりたくなかったのです。

ここまででメモリマネージャの説明が完了したので、本章は終了です。お疲れ様でした！

第 9 章

重ね合わせ処理

マウ斯卡ーソルをスタートメニューに重ねると背景色で塗りつぶされてしまい、スタートメニューが壊れてしまいます。この章ではマウ斯卡ーソルを重ねても裏にある画像が壊れないように、重ね合わせの処理を作っていきます。まずは単純なアルゴリズムで実装し、その後高速化を試みます。この章が終わると、マウスをぐりぐり動かしてもスタートメニューや文字表示が壊れないようになります。

9.1 重ね合わせ処理 (osbook_day09a)

マウスカーソルを動かす処理は、もともとの位置をデスクトップの背景色で塗りつぶした後、新しい位置にマウスカーソルを描画するという流れになっています。そのため、背景色ではないところをマウスが通過してしまうと画像が壊れてしまいます。この節では画像が崩れないように重ね合わせの処理を実装してみます。パソコンで画像の加工をしたり絵を書いたりする方にはレイヤと言えば伝わりやすいでしょうか。

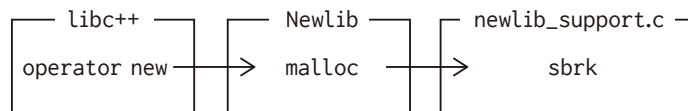
重ね合わせの処理を実装するために new によるメモリ割り当てができると都合が良いので、以降ではまず new を使えるようにしてから重ね合わせの処理を実装することにします。

9.2 new 演算子

せっかくメモリ管理を実装したので、C++ の new 演算子を使えるようにしたいと思います。今まで登場した new 演算子のもつぱら配置 new として使っていました。配置 new は動的にメモリを割り当てる代わりに、プログラマが指定したメモリ領域にクラスのインスタンスを構築することができる機能でした。動的にメモリを確保する機能がない間は配置 new を使うしかありませんでしたが、今は動的メモリ管理機能があるわけですから配置 new から脱却したいです。

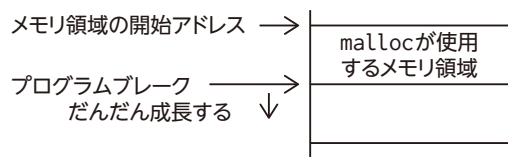
本書で採用している C++ ライブラリである libc++ では、new 演算子はメモリを確保する際に malloc() を呼び出す実装となっています。したがって普通の new ができるようになるには malloc() が使えるようになれば良いことになります。本書で採用している C ライブラリである Newlib は malloc() を提供していますので、あらためて自分で定義する必要はありません。ただし、Newlib の malloc() が適切に動くためには sbrk() という関数をプログラマが実装する必要がありますので、以降でそれを説明していきます。依存関係をまとめると図 9.1 のようになります。

図 9.1 new と malloc() と sbrk() の関係



普通の new を使えるようにするために私たちがやるべきは sbrk() を適切に実装することです。sbrk() はプログラムブレーク (program break) を設定する関数です。プログラムブレークというのは、もともとは Unix システムにおいて各プロセスが使えるメモリ領域の末尾を指すアドレスを意味します。プロセスはプログラムブレークを後ろにずらすことで新たにメモリを確保するわけです。本書では malloc() が利用するメモリ領域を確保するためにプログラムブレークを利用します (図 9.2)。

図 9.2 プログラムブレークはプログラムが使えるメモリ領域の末尾を指す



それでは `sbrk()` を実装していきましょう。まず `sbrk()` が満たすべき仕様を確認してみる^{*1} と次の通りであることが分かります。

- ・関数プロトタイプは `caddr_t sbrk(int incr);`
- ・`sbrk()` はプログラムブレークを `incr` バイトだけ増減させる。
- ・処理が成功したら、増加させる前のプログラムブレークを返す。
- ・処理が失敗したら `errno` を `ENOMEM` に設定し、`(caddr_t)-1` を返す。

さて、この仕様を満たす関数を書きたいのですが、まだ情報が足りません。それは、そもそもプログラムブレークの初期値 (= `malloc()` が使用できる領域の先頭) と上限値 (= `malloc()` が使用できる領域の末尾) をどうするかということです。これらは `sbrk` を実装する人が決めなければなりません。ということで決めました。「第8章 メモリ管理」で作ったメモリマネージャを利用して割り当てたメモリ領域を利用することにします。

リスト 9.1 `sbrk` は指定されたバイト数だけプログラムブレークを増減させる (`newlib_support.c`)

```
caddr_t program_break, program_break_end;

caddr_t sbrk(int incr) {
    if (program_break == 0 || program_break + incr >= program_break_end) {
        errno = ENOMEM;
        return (caddr_t)-1;
    }

    caddr_t prev_break = program_break;
    program_break += incr;
    return prev_break;
}
```

仕様に合わせて実装してみたものをリスト 9.1 に示します。まず目に付くのはグローバル変数として定義してある `program_break` と `program_break_end` でしょう。これらはそれぞれプログラムブレークの初期値と末尾を表す変数のつもりです。定義時点では値が設定されていませんが、`sbrk` はこれらの値が適切に設定されていることを前提として動作します。したがって `sbrk` を初めて使う前にこれら 2 つの変数を初期化する必要があります。

次に関数本体を見ていきます。最初に `program_break` 変数がちゃんと設定されていることと、`mem`

*1 `sbrk()` の仕様は Newlib のドキュメント (<https://sourceware.org/newlib/libc.html#index-sbrk>) や `man sbrk`などを参照。

メモリ領域が十分に空いていることを確認します。program_break に値が設定されていない場合は暗黙の初期値0になっているはずですので、0のときはエラーにします。また、仮に引数 incr で指定されたバイト数だけプログラムブレークを増加させたとして、プログラムブレークの上限を超えてしまうならメモリが足りません。その場合もエラーとします。sbrk の処理が失敗したら、仕様通りに errno 変数に ENOMEM を設定して (caddr_t)-1 を返します。

program_break 変数がきちんと設定されていてメモリが十分に空いているなら実際のメモリ確保処理を行います。プログラムブレークを incr バイトだけ増減させ、変化させる前の値 (prev_break) を返します。

これで sbrk の実装が完了しました。次に program_break と program_break_end の初期値を設定するプログラムを書きます。

リスト 9.2 InitializeHeap() はプログラムブレークの初期値を設定する (memory_manager.cpp)

```
extern "C" caddr_t program_break, program_break_end;

Error InitializeHeap(BitmapMemoryManager& memory_manager) {
    const int kHeapFrames = 64 * 512;
    const auto heap_start = memory_manager.Allocate(kHeapFrames);
    if (heap_start.error) {
        return heap_start.error;
    }

    program_break = reinterpret_cast<caddr_t>(heap_start.value.ID() * kBytesPerFrame);
    program_break_end = program_break + kHeapFrames * kBytesPerFrame;
    return MAKE_ERROR(Error::kSuccess);
}
```

リスト 9.2 に 2 つのグローバル変数に値を設定する関数 InitializeHeap() の実装を示します。この関数はまず、メモリマネージャから 64 × 512 フレームだけメモリを割り当ててもらいます。64 × 512 フレーム (128MiB) もあれば当面は足りると思いますが、もし後で new や malloc が失敗するようになったらこの数を増やすことにしましょう。memory_manager.Allocate はメモリ不足などの際にエラーを返す可能性があるため、エラーチェックを行っています。

正常にフレームを確保できたら、フレーム番号からアドレスを計算します。フレーム番号に kBytesPerFrame をかければフレームの先頭アドレスになります。確保したメモリ領域の先頭のフレーム番号は heap_start.value.ID() ですから、その値をアドレスに変換すればプログラムブレークの初期値となります。メモリ領域の末尾のアドレスは program_break の値にメモリ領域のバイト数を足したアドレスとなります。

リスト 9.3 InitializeHeap() を呼び出す (main.cpp)

```
memory_manager->SetMemoryRange(FrameID{1}, FrameID{available_end / kBytesPerFrame});

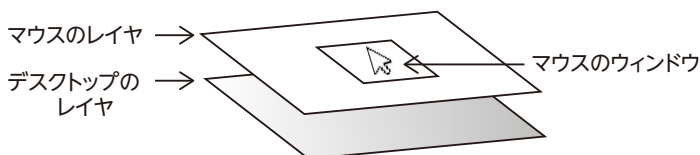
if (auto err = InitializeHeap(*memory_manager)) {
    Log(kError, "failed to allocate pages: %s at %s:%d\n",
        err.Name(), err.File(), err.Line());
    exit(1);
}
```

先ほど定義した `InitializeHeap()` を呼び出す処理をメイン関数に加えましょう。**リスト 9.3** に追加した呼び出し処理を示します。メモリマネージャの `SetMemoryRange()` を呼び出してメモリマネージャが使用可能になった直後に加えてみました。

9.3 重ね合わせ処理の原理

重ね合わせの処理を実装するには「デスクトップ」と「マウスカーソル」に対応する 2 つのレイヤに予めデスクトップとマウスカーソルの図形を準備しておき、マウスが動くたびに下にあるレイヤから順に描画すれば良さそうです。考え方を **図 9.3** に示します。

図 9.3 レイヤによる画面描画



レイヤは無限に広がる平面だと考えることができます。レイヤは原点の座標と重なり順のみを属性として持ちます。レイヤは横と縦の大きさを持ちません。一方、マウスカーソルやデスクトップの背景はどちらも有限の大きさを持つ描画領域です。マウスカーソルは四角形ではない複雑な形ですが、隙間を透過色で埋めてある四角形だと思っても良いわけです。そうすると、マウスカーソルもデスクトップの背景も、横と縦の大きさを持つ四角形として扱えます。このような、任意の大きさの四角形の描画領域を**ウィンドウ**と呼ぶことにします。

ということで、早速レイヤとウィンドウを作っていきます。レイヤの中にウィンドウがあるという構造ですので、先にウィンドウを作ることになります。

リスト 9.4 Window クラスの定義 (window.hpp)

```
class Window {
public:
    /** @brief WindowWriter は Window と関連付けられた PixelWriter を提供する。
     */
    class WindowWriter : public PixelWriter {
    < 中略 >
    };

    /** @brief 指定されたピクセル数の平面描画領域を作成する。 */
    Window(int width, int height);
    ~Window() = default;
    Window(const Window& rhs) = delete;
    Window& operator=(const Window& rhs) = delete;

    /** @brief 与えられた PixelWriter にこのウィンドウの表示領域を描画する。
     *
     * @param writer 描画先
     */
};
```

```

    * @param position  writer の左上を基準とした描画位置
    */
void DrawTo(PixelWriter& writer, Vector2D<int> position);
/** @brief 透過色を設定する。 */
void SetTransparentColor(std::optional<PixelColor> c);
/** @brief このインスタンスに紐付いた WindowWriter を取得する。 */
WindowWriter* Writer();

/** @brief 指定した位置のピクセルを返す。 */
PixelColor& At(int x, int y);
/** @brief 指定した位置のピクセルを返す。 */
const PixelColor& At(int x, int y) const;

/** @brief 平面描画領域の横幅をピクセル単位で返す。 */
int Width() const;
/** @brief 平面描画領域の高さをピクセル単位で返す。 */
int Height() const;

private:
    int width_, height_;
    std::vector<std::vector<PixelColor>> data_{};
    WindowWriter writer_{*this};
    std::optional<PixelColor> transparent_color_{std::nullopt};
};

```

リスト 9.4 にウィンドウクラス Window の定義を示します。このクラスのメンバ変数は幅と高さを持つ `width_` と `height_`、ピクセルの配列 `data_`、ピクセルへの書き込み機能を提供する `writer_`、透過色を持つ `transparent_color_` があります。`std::optional` は初めて見る方もいるでしょう。これは「値を持たない」という状態を明示的に表現できる型を作るラップクラス^{*2}です。`transparent_color_` は「PixelColor 型の値を持つ」か「値を何も持たない」かのどちらかを表すことができます。初期値は `std::nullopt` という特殊な値が設定されていて、値を持たない、すなわちどの色も透過しない状態になります。

ウィンドウクラスの先頭には WindowWriter クラスの定義があります。このクラスはウィンドウクラス専用の書き込み機能を提供します。WindowWriter は PixelWriter を継承していますので、PixelWriter を要求する場所で代わりに使用することができます。クラス定義の詳しい中身は後述します。

ウィンドウクラスのコンストラクタを見えます。指定した大きさを持つウィンドウを生成するコンストラクタ `Window(int width, int height)` を持っているのが分かります。**リスト 9.5** にコンストラクタの定義を示します。

リスト 9.5 指定した大きさのウィンドウを生成するコンストラクタ (window.cpp)

```

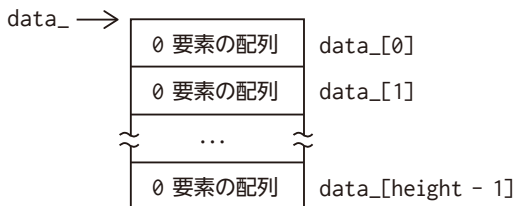
Window::Window(int width, int height) : width_{width}, height_{height} {
    data_.resize(height);
    for (int y = 0; y < height; ++y) {
        data_[y].resize(width);
    }
}

```

*2 Wrapper、包む物。ここでは、PixelColor クラスを包んだ新しいクラスを生成します。

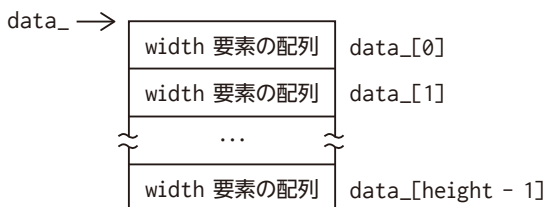
コンストラクタでは指定された幅と高さを持つ2次元配列を生成しています。data_ の型を見てみると `std::vector<std::vector<PixelColor>>` となっていて、配列の配列となっていることが分かります。コンストラクタの1行目では、`resize` メソッドを使って1次元目の配列の要素数を `height` 個に拡大します。1行目が実行されると data_ は図9.4のような形の配列になります。

図9.4 data_.resize(height) を実行した後の様子



要素が0個の(つまり、空の)配列が `height` 個並ぶ形になるのです。コンストラクタの2行目以降では、`for` 文を使ってそれぞれの配列の要素数を `width` 個に拡大していきます。結局、コンストラクタを実行し終わると data_ は図9.5に示す形になります*3。

図9.5 コンストラクタを実行した後の様子



`DrawTo()` は指定された `PixelWriter` を使ってウィンドウの内容を描画するメソッドです。ウィンドウの内容を画面に描画するためのメソッドとして作りました。引数 `writer` には画面に関連付けられた `PixelWriter` が渡される予定です。引数 `position` には画面の中でそのウィンドウを表示すべき座標が渡される予定です。

リスト9.6 DrawTo() はウィンドウの内容を指定された描画先へ描画する (window.cpp)

```
void Window::DrawTo(PixelWriter& writer, Vector2D<int> position) {
    if (!transparent_color_) {
        for (int y = 0; y < Height(); ++y) {
            for (int x = 0; x < Width(); ++x) {
                writer.Write(position.x + x, position.y + y, At(x, y));
            }
        }
    }
}
```

*3 コンストラクタの初期化子として `data_(height, std::vector<PixelColor>(width, PixelColor{}))` と書くだけでも2次元配列を拡大することはできますが、説明を分かりやすくするために、あえて `for` 文を使った方法を採用しました。

```

    return;
}

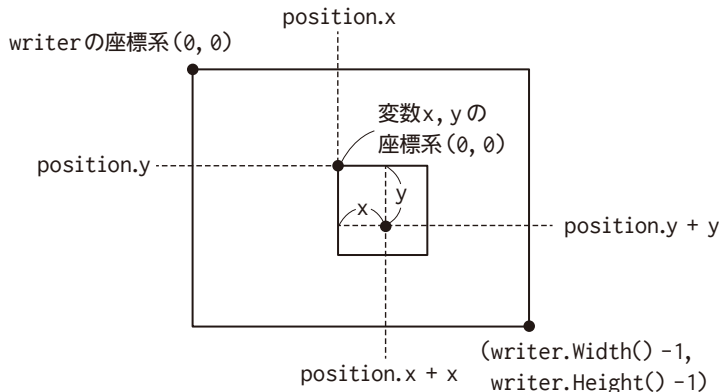
const auto tc = transparent_color_.value();
for (int y = 0; y < Height(); ++y) {
    for (int x = 0; x < Width(); ++x) {
        const auto c = At(x, y);
        if (c != tc) {
            writer.Write(position.x + x, position.y + y, c);
        }
    }
}
}
}

```

リスト 9.6 に `DrawTo()` メソッドの実装を示します。透過色が設定されているかどうかによって処理が2通りに分かれています。透過色が設定されていない場合、`!transparent_color_` は真となり `if` 文の中身が実行されます。`transparent_color_` の型である `std::optional` は、`if` 文の中などの真偽値が求められる場面で、有効な値が設定されている場合は真が、そうでない場合 (`std::nullopt` が設定されている場合) は偽が返されます。これが真となる場合、`transparent_color_.value()` によって設定されている値を取得できます。

どちらの処理にせよやっていることはほぼ一緒です。このウィンドウの全ピクセルデータを、引数で指定された `writer` を用いて描画します。描画する位置は `position` で指定されます。描画の様子を図にすると図 9.6 のようになります。

図 9.6 ウィンドウを `PixelWriter` を用いて描画する



`DrawTo()` に対し、フレームバッファに関連付けられた `PixelWriter`、すなわち `main.cpp` で定義されている `pixel_writer` を渡すことで、このウィンドウの内容をフレームバッファに描画することができます。`Window::DrawTo()`^{*4} は、後で紹介する `Layer` クラスの `DrawTo()` メソッドから呼び出されます。

*4 `::` はネームスペースやクラスの内部を参照する書き方で、ここでは `Window` クラス内に定義された `DrawTo()` メソッドを参照しています。

リスト 9.7 SetTransparentColor() はウィンドウの透過色を設定する (window.cpp)

```
void Window::SetTransparentColor(std::optional<PixelColor> c) {
    transparent_color_ = c;
}
```

リスト 9.7 に `transparent_color_` に値を設定する関数 `SetTransparentColor()` を示します。やっていることはめっちゃ単純ですね。この関数に `PixelColor` 型の値を渡す、例えば `window.SetTransparentColor(PixelColor{1,1,1})`; などとすると、その色を透過色として設定できます。また、この関数に `std::nullopt` を渡せば透過色の設定を無効化できます。

リスト 9.8 WindowWriter クラスの定義 (window.hpp)

```
class WindowWriter : public PixelWriter {
public:
    WindowWriter(Window& window) : window_{window} {}
    /** @brief 指定された位置に指定された色を描く */
    virtual void Write(int x, int y, const PixelColor& c) override {
        window_.At(x, y) = c;
    }
    /** @brief Width は関連付けられた Window の横幅をピクセル単位で返す。 */
    virtual int Width() const override { return window_.Width(); }
    /** @brief Height は関連付けられた Window の高さをピクセル単位で返す。 */
    virtual int Height() const override { return window_.Height(); }

private:
    Window& window_;
};
```

さて、Window クラス本体の説明はだいたいできたので、残っていた WindowWriter クラスの説明を続けます (**リスト 9.8**)。このクラスはウィンドウに関連付けられた書き込み機能を提供します。

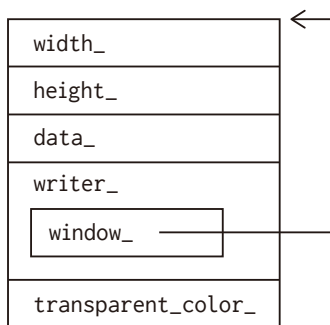
このクラスの特徴はメンバ変数として Window への参照を持っていることです。これがあるおかげで `WindowWriter::Write()` の中からウィンドウに対して描画できるようになっています。この参照が初期化される過程は少し難しいかもしれませんが、詳しくに説明します。

`window_` は `WindowWriter` のコンストラクタで初期化されます。**リスト 9.8** の `window_{window}` が初期化の部分です。初期値はコンストラクタの引数に指定された Window クラスのインスタンスが指定されています。

ではこのコンストラクタがどこで呼ばれているのかというと、Window クラスのメンバ変数の初期化部分です。すなわち **リスト 9.4** 中の `WindowWriter writer_{*this}`; です。ここでメンバ変数 `writer_` の初期化をしていて、その初期値として `*this` が指定されています。`this` はそれが使用された場所により異なるものを指す特殊な変数で、ここでは Window クラスのインスタンスを指しています。すなわち Window 型の変数を生成する際、その変数への先頭ポインタが `this` となり、さらに `writer_` のコンストラクタの引数にそのポインタが渡されるわけですね。

まとめると、Window クラスのインスタンスのメンバ変数 `writer_` は、そのインスタンスへの参照を持つことになります。この関係を図にすると **図 9.7** に示す通りです。

図 9.7 Window クラスと WindowWriter クラスの関係



クラスや構造体に含まれるメンバ変数が親のクラスや構造体を参照する、という状況は割とよくあります。解決法の一つは今回のように内部クラスに親クラスへのポインタまたは参照を持たせる方法です。Linux カーネルでは違う方法が採用されていて、これより少しだけメモリ容量を節約できるやり方となっています。興味がある方は `container_of` マクロを調べてみてください。

リスト 9.9 重ね合わせを実現する Layer クラス (layer.hpp)

```

class Layer {
public:
    /** @brief 指定された ID を持つレイヤーを生成する。 */
    Layer(unsigned int id = 0);
    /** @brief このインスタンスの ID を返す。 */
    unsigned int ID() const;

    /** @brief ウィンドウを設定する。既存のウィンドウはこのレイヤーから外れる。 */
    Layer& SetWindow(const std::shared_ptr<Window>& window);
    /** @brief 設定されたウィンドウを返す。 */
    std::shared_ptr<Window> GetWindow() const;

    /** @brief レイヤーの位置情報を指定された絶対座標へと更新する。再描画はしない。 */
    Layer& Move(Vector2D<int> pos);
    /** @brief レイヤーの位置情報を指定された相対座標へと更新する。再描画はしない。 */
    Layer& MoveRelative(Vector2D<int> pos_diff);

    /** @brief writer に現在設定されているウィンドウの内容を描画する。 */
    void DrawTo(PixelWriter& writer) const;

private:
    unsigned int id_;
    Vector2D<int> pos_;
    std::shared_ptr<Window> window_;
};
  
```

次はレイヤを作っていきます。レイヤを表す Layer クラスの定義をリスト 9.9 に示します。メンバ変数としては、レイヤを識別するための ID、原点座標、ウィンドウへのポインタを持たせました。std::shared_ptr はスマートポインタ（賢いポインタ）の一種です。詳しくは「**コラム 9.1 スマートポインタ**」で説明します。とりあえず、window_ は Window* 型のポインタ変数だと思ってください。

レイヤクラスで重要なのは、移動と描画のためのメソッドです。中身を見ていきます。

リスト 9.10 Move() はレイヤを指定した位置に動かす (layer.cpp)

```
Layer& Layer::Move(Vector2D<int> pos) {
    pos_ = pos;
    return *this;
}

Layer& Layer::MoveRelative(Vector2D<int> pos_diff) {
    pos_ += pos_diff;
    return *this;
}
```

リスト 9.10 にレイヤを移動するためのメソッドを示します。Move() は絶対座標、MoveRelative() は相対座標を用いての移動を行います。特に難しいことはないと思います。MoveRelative() の中で pos_ += pos_diff; のようなコードが書けるのは、Vector2D クラスが operator+= を実装しているからです。

リスト 9.11 レイヤを pos_ に設定された位置に描画する (layer.cpp)

```
void Layer::DrawTo(PixelWriter& writer) const {
    if (window_) {
        window_->DrawTo(writer, pos_);
    }
}
```

リスト 9.11 にレイヤを描画するためのメソッド DrawTo() を示します。このメソッドはウィンドウへのポインタ window_ が設定されていれば、そのウィンドウの DrawTo() メソッドを呼び出します。ウィンドウ自身は平面上での位置を記憶せず、レイヤが位置を管理するという構造にしています。概念的には、重なりを管理するレイヤと表示領域を表すウィンドウは別物です。両者を分けたが、まとめてしまってもプログラムの読みやすさは大差ないかもしれません。

さて、レイヤは単体では意味がありません。重ね合わせの処理というくらいですから複数のレイヤの重なりを表現する必要があります。そのためのクラスとして LayerManager を作ってみます。

リスト 9.12 LayerManager クラスはレイヤを複数管理する (layer.hpp)

```
class LayerManager {
public:
    /** @brief Draw メソッドなどで描画する際の描画先を設定する。 */
    void SetWriter(PixelWriter* writer);
    /** @brief 新しいレイヤーを生成して参照を返す。
     *
     * 新しく生成されたレイヤーの実体は LayerManager 内部のコンテナで保持される。
     */
    Layer& NewLayer();

    /** @brief 現在表示状態にあるレイヤーを描画する。 */
    void Draw() const;

    /** @brief レイヤーの位置情報を指定された絶対座標へと更新する。再描画はしない。 */
```

```

void Move(unsigned int id, Vector2D<int> new_position);
/** @brief レイヤーの位置情報を指定された相対座標へと更新する。再描画はしない。 */
void MoveRelative(unsigned int id, Vector2D<int> pos_diff);

/** @brief レイヤーの高さ方向の位置を指定された位置に移動する。
 *
 * * new_height に負の高さを指定するとレイヤーは非表示となり、
 * * 0 以上を指定するとその高さとなる。
 * * 現在のレイヤー数以上の数値を指定した場合は最前面のレイヤーとなる。
 * */
void UpDown(unsigned int id, int new_height);
/** @brief レイヤーを非表示とする。 */
void Hide(unsigned int id);

private:
    PixelWriter* writer_{nullptr};
    std::vector<std::unique_ptr<Layer>> layers_{};
    std::vector<Layer*> layer_stack_{};
    unsigned int latest_id_{0};

    Layer* FindLayer(unsigned int id);
};

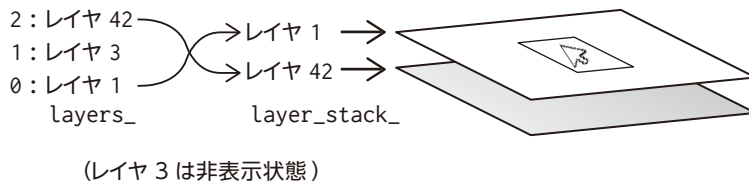
extern LayerManager* layer_manager;

```

リスト 9.12 に LayerManager の定義を示します。まずはメンバ変数 `layers_` に注目します。この変数は動的配列 `std::vector` になっていて、生成したレイヤのインスタンスを生成した順に格納することにします。レイヤは非表示状態になることもありますが、非表示になっているものも含め、存在するすべてのレイヤを格納します。

`layers_` の要素の型 `std::unique_ptr` は馴染みがないかもしれませんが。この型は `std::shared_ptr` と同じくスマートポインタの一種ですが、`std::shared_ptr` がポインタの共有を許す一方、`std::unique_ptr` はポインタの共有ができないという特徴があります。この特徴のおかげで、あるオブジェクトが誰かに「所有」されているという関係を明確に示すことができます。レイヤはレイヤマネージャに所有されているという設計にしたいので、まさにこの特徴がうってつけでした。

図 9.8 レイヤスタックとレイヤマップの関係



さて、もう 1 つ大事なメンバ変数があります。 `layer_stack_` です。レイヤを積む、という意味で `stack` (積む) という名前にしてみました。この変数はレイヤの積み重ねを表現するもので、配列先頭の要素を最背面レイヤ、そこから順に積んでいって、末尾を最前面とするつもりです。非表示になっているレイヤは含みません。**図 9.8** に `layers_` と `layer_stack_` の関係を示します。現時点ではレイヤはマウスカーソル用と背景用の 2 枚だけですからレイヤの順序が入れ替わることがありませんが、将

来的にはアプリケーションのウィンドウなどのためにレイヤが増え、順番が入れ替わるようになる予定です。

主要なデータ構造を決めましたので、各メソッドの実装に取り掛かりましょう。

リスト 9.13 NewLayer() は新しいレイヤを生成する (layer.cpp)

```
Layer& LayerManager::NewLayer() {
    ++latest_id_;
    return *layers_.emplace_back(new Layer{latest_id_});
}
```

リスト 9.13 に NewLayer() の実装を示します。このメソッドは、固有の ID を持つレイヤインスタンスを 1 つ生成して layers_ に格納します。1 行目はメンバ変数 latest_id_ の値を 1 増やすことで ID を生成しているというのはいすぐ分かります。2 行目はちょっと難しいと思いますので詳しくに説明します。

layers_.emplace_back() は指定した値を配列末尾に追加するメソッドです。ここでは値に new Layer{latest_id_} を渡しています。したがって、2 行目の実行が終わった時点の layers_ は、その時点の latest_id_ を ID として持つレイヤインスタンスが追加された状態になります。

NewLayer() を呼び出す側では、新たに生成されたレイヤインスタンスにいくつかの設定をする必要がありますから、NewLayer() は生成したレイヤインスタンスへの参照 (Layer&) を返すことにします。こうしておけば、呼び出し側では NewLayer().SetWindow(...) のように手軽に追加設定を加えられます。

layers_.emplace_back() は追加された要素の参照を戻り値として返します。すなわち std::unique_ptr<Layer>& という型の値が返ってきます。これはポインタ型なので、これをそのまま戻り値にすれば良いように感じますよね。しかし、std::unique_ptr は共有ができないポインタのため、これをそのまま返すことができないのです。そこで * 演算子を使って Layer& 型に変換し、戻り値を作り出しています。

リスト 9.14 FindLayer() は指定された ID のレイヤを探す (layer.cpp)

```
Layer* LayerManager::FindLayer(unsigned int id) {
    auto pred = [id](const std::unique_ptr<Layer>& elem) {
        return elem->ID() == id;
    };
    auto it = std::find_if(layers_.begin(), layers_.end(), pred);
    if (it == layers_.end()) {
        return nullptr;
    }
    return it->get();
}
```

リスト 9.14 に layers_ から指定された ID を持つレイヤインスタンスを探し出す FindLayer() メソッドの実装を示します。このメソッドは、指定された ID のレイヤがあればそのインスタンスへのポインタを返し、見つからなければ nullptr を返します。

この実装で用いられている std::find_if() は std::find_if(begin, end, pred) というように 3 つの引数を取ります。begin から end までを順に調べ、最初に pred が真を返す要素を探し

出します。この `pred` は一般に述語 (predicate) と呼ばれるものです。**リスト 9.14** では述語をラムダ式で定義しています。

ラムダ式は「**6.3 PCI デバイスの探索**」で出てきましたが、今回は `[id]` のようにキャプチャに変数名が書いてあることが大きな違いです。キャプチャは、ラムダ式の外側にあるローカル変数をラムダ式の内側で使うための仕組みです。ここでは変数 `id` を内部で利用するためにキャプチャしています。

このラムダ式は `layers_` の要素を 1 つ受け取って、`id` と同じレイヤ ID を持っているかをチェックします。`id` と等しい ID を持つレイヤであれば真を返します。このように、述語とは「真か偽で判定できる文」のことを言います。例えば整数 `x` に対して「`x` は偶数である」という文は述語です。`x` が与えられたときに真か偽を判定できるからです。

`std::find_if()` に与える述語は引数が 1 つの関数 (または関数のように呼び出せる何か) でなければなりません。また、その引数の型はコンテナの要素を受け取れる型でなければなりません。述語は `begin` から `end` まで、要素毎に呼び出され、その要素について何らかの条件が成り立つか、成り立たないかを真偽値で返します。`std::find_if()` は最初に述語が真を返した (条件が成り立った) 要素を返します。

要素を返すと書きましたが、`std::find_if()` が実際に返すのは要素を指すイテレータです。要素が見つからない場合はイテレータの値は `layers_.end()` となりますから、その場合は `nullptr` を返すことにします。要素が見つかった場合、イテレータ `it` は `std::unique_ptr<Layer>` を指します。そこから生ポインタ `Layer*` を取り出すには `it->get()` となるわけです。

`FindLayer()` が定義できたので、それを使って他のメソッドを実装していきます。

リスト 9.15 `Move()` はレイヤを指定位置まで移動させる (`layer.cpp`)

```
void LayerManager::Move(unsigned int id, Vector2D<int> new_position) {
    FindLayer(id)->Move(new_position);
}

void LayerManager::MoveRelative(unsigned int id, Vector2D<int> pos_diff) {
    FindLayer(id)->MoveRelative(pos_diff);
}
```

リスト 9.15 にレイヤを移動させるメソッド 2 つの実装を示します。`Move()` を見てみると、1 行目で指定された ID のレイヤを探してきてそのレイヤの `Move()` を呼び出します。レイヤの `Move()` は絶対座標を指定するメソッドでしたね。このメソッドは再描画をしないので、移動を画面に反映するには後で実装する `Draw()` メソッドを呼ぶ必要があります。`MoveRelative()` は相対座標バージョンです。

2 つのメソッドの実装で早速 `FindLayer()` が役に立っています。もし指定した ID を持つレイヤがない場合に `FindLayer()` は検索に失敗しますが、有効な ID を指定することを呼び出し側の責任としてしまいましょう。ID の有効性の確認を呼び出し側の責任とすることで、`Move()` 内でのヌルチェックを省けます。

リスト 9.16 Draw() はレイヤを画面に描画する (layer.cpp)

```
void LayerManager::Draw() const {
    for (auto layer : layer_stack_) {
        layer->DrawTo(*writer_);
    }
}
```

リスト 9.16 に Draw() の実装を示します。このメソッドは最も下にあるレイヤから上に向かって描画します。layer_stack_ は配列先頭が最背面、末尾が最前面となっていますので、先頭から順に描画すれば自然とそうなります。下から順に描画することでレイヤの重なりを表現できます。描画先 writer_ は SetWriter() で設定できます。典型的にはフレームバッファと関連付けられた PixelWriter を描画先とすることが多いです。

ここで出てきた for 文は範囲 for 文と呼ばれるものです。範囲 for 文は std::vector 等のコンテナの各要素について処理するループを簡潔に書けます。for (型変数: コンテナ) と書くと、コンテナの .begin() から順に変数に設定された状態で for の本体が実行されます。

リスト 9.17 Hide() は指定したレイヤを非表示にする (layer.cpp)

```
void LayerManager::Hide(unsigned int id) {
    auto layer = FindLayer(id);
    auto pos = std::find(layer_stack_.begin(), layer_stack_.end(), layer);
    if (pos != layer_stack_.end()) {
        layer_stack_.erase(pos);
    }
}
```

指定したレイヤを非表示にするメソッド Hide() を実装します (**リスト 9.17**)。このメソッドは表示中レイヤを重なり順に保持している layer_stack_ からレイヤを取り除くことでそのレイヤを非表示にします。

std::find() は指定した値を持つ要素を探す関数です。第3引数に述語を渡す std::find_if() とは異なり、述語ではなく目的の値をそのまま指定します。目的の値の位置をイテレータとして返すのは std::find() も std::find_if() も同じです。目的のレイヤが見つかったら layer_stack_.erase() を使ってその要素を削除します。

リスト 9.18 UpDown() はレイヤを指定の高さに移動する (layer.cpp)

```
void LayerManager::UpDown(unsigned int id, int new_height) {
    if (new_height < 0) {
        Hide(id);
        return;
    }
    if (new_height > layer_stack_.size()) {
        new_height = layer_stack_.size();
    }

    auto layer = FindLayer(id);
    auto old_pos = std::find(layer_stack_.begin(), layer_stack_.end(), layer);
```

```

auto new_pos = layer_stack_.begin() + new_height;

if (old_pos == layer_stack_.end()) {
    layer_stack_.insert(new_pos, layer);
    return;
}

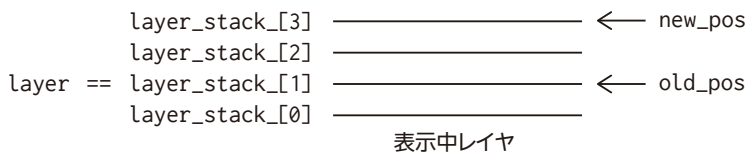
if (new_pos == layer_stack_.end()) {
    --new_pos;
}
layer_stack_.erase(old_pos);
layer_stack_.insert(new_pos, layer);
}

```

リスト 9.18 は指定したレイヤの重なり順を変更する `UpDown()` の定義です。このメソッドは、指定したレイヤ ID を指定した高さに変更します。new_height には layer_stack_ の中での位置を指定します。0 を指定すると最背面に、layer_stack_.size() 以上の値を指定すると最前面に移動するという意味になります。負の値を指定するとそのレイヤを非表示にします。

処理が複雑なので詳しく追ってみます。最初の if 文では new_height が負の場合の処理をして関数を抜けます。次の if 文では new_height の値が大きすぎる場合に layer_stack_.size() を上限に調整します。この 2 つの if 文を抜けたところでは、new_height の値が 0 から layer_stack_.size() の範囲に収まっていることが保証されます。

図 9.9 UpDown() メソッド中で定義される変数の関係



new_height の値の調整が終わったら必要な変数をいくつか定義します。指定された ID を持つレイヤインスタンス layer、そのレイヤの現在の位置を指すイテレータ old_pos、new_height の位置を指すイテレータ new_pos の 3 つです。これらの変数の関係を図 9.9 に示します。図では、指定されたレイヤが「高さ = 1」で表示中となっていて、かつ新しく「高さ = 3」に設定しようとしています。

変数の定義が終わったらそれらを使って処理を行います。処理の基本戦略は old_pos からレイヤを削除してから new_pos にレイヤを挿入する、としようかと思います。まず、指定されたレイヤが非表示だった場合 (old_pos == layer_stack_.end()) の処理です。このときはレイヤを削除する必要はありませんから、new_pos が指す位置にレイヤを挿入して完了です。

残りはレイヤが現在表示中の場合の処理です。このケースでは layer_stack_ から一旦レイヤを削除する際に layer_stack_ の要素が 1 つだけ少なくなります。new_pos が末尾を指している場合 (new_pos == layer_stack_.end())、要素が 1 つ少なくなると末尾を飛び越えてしまい困ったことになります。したがって new_pos を 1 だけ手前にずらしておきます。もろもろの調整が終わったら現在の表示位置からレイヤを削除し、新しい位置へレイヤを挿入します。

ここまででウィンドウとレイヤの準備が終わりました。やっと本来やりたかったことに手を付けられます。マウスカーソルを動かしても背景が壊れないようにしたかったのです。

リスト 9.19 マウスのイベントハンドラでレイヤを使う (main.cpp)

```

unsigned int mouse_layer_id;

void MouseObserver(int8_t displacement_x, int8_t displacement_y) {
    layer_manager->MoveRelative(mouse_layer_id, {displacement_x, displacement_y});
    layer_manager->Draw();
}

```

リスト 9.19 にマウスを動かしたときに呼ばれる関数 `MouseObserver()` の新しい実装を示します。今までは `mouse_cursor->MoveRelative()` を使ってマウスを動かしていましたが、レイヤの仕組みを使うように変更しました。グローバル変数 `mouse_layer_id` はマウスカーソルと関連付けられたレイヤの ID を表します。メイン関数の中で値が設定されます (**リスト 9.20**)。

リスト 9.20 2 枚のレイヤを生成する (main.cpp)

```

const int kFrameWidth = frame_buffer_config.horizontal_resolution;
const int kFrameHeight = frame_buffer_config.vertical_resolution;

auto bgwindow = std::make_shared<Window>(kFrameWidth, kFrameHeight);
auto bgwriter = bgwindow->Writer();

DrawDesktop(*bgwriter);
console->SetWriter(bgwriter);

auto mouse_window = std::make_shared<Window>(
    kMouseCursorWidth, kMouseCursorHeight);
mouse_window->SetTransparentColor(kMouseTransparentColor);
DrawMouseCursor(mouse_window->Writer(), {0, 0});

layer_manager = new LayerManager;
layer_manager->SetWriter(pixel_writer);

auto bglayer_id = layer_manager->NewLayer()
    .SetWindow(bgwindow)
    .Move({0, 0})
    .ID();
mouse_layer_id = layer_manager->NewLayer()
    .SetWindow(mouse_window)
    .Move({200, 200})
    .ID();

layer_manager->UpDown(bglayer_id, 0);
layer_manager->UpDown(mouse_layer_id, 1);
layer_manager->Draw();

```

リスト 9.20 にメイン関数の改造箇所を示します。上からざっと説明すると、まず画面の縦と横の大きさを持つ `Window` のインスタンス `bgwindow` を作成します。変数名は `background` (背景) を表すウィンドウという意味です。次に `DrawDesktop()` を用いて `bgwindow` に背景画像を描画します。その後 `console` の描画先をそのウィンドウに切り替えます (詳しくはちょっと後で説明します)。ここまでで背景用ウィンドウの準備は完了です。

次にマウスカーソル用の Window インスタンスを生成します。マウスは長方形ではない複雑な形をしているので、透過色の設定が必要です。透過色は `kMouseTransparentColor` という値にします。この定数は `mouse.hpp` で `const PixelColor kMouseTransparentColor{0, 0, 1};` と定義されています。この色はマウスカーソル本体には使われない色なので透過色に設定しても問題がありません。次に `DrawMouseCursor()` を用いてマウスのウィンドウにマウスカーソルの絵を書きます。

中盤ではレイヤマネージャを生成して、レイヤマネージャが用いる `PixelWriter` として `pixel_writer` をセットします。これで、今後レイヤマネージャの `Draw()` を呼び出した際、設定されたレイヤが `pixel_writer` に書き出されます。

後半ではレイヤマネージャを使って、背景用とマウス用の 2 つのレイヤを生成しています。レイヤの生成、ウィンドウの設定、レイヤの移動、ID の取得がすべて連なっているこの書き方はメソッドチェーンと呼ばれます。`Layer()` クラスのいくつかのメソッドは戻り値として自分自身への参照 (`Layer&`) を返すため、このように連なった書き方が可能なのです。生成したレイヤの ID をそれぞれ `bglayer_id` と `mouse_layer_id` という変数に書き込みます。`mouse_layer_id` は `MouseObserver()` の中で使うためグローバル変数として定義してあります (**リスト 9.19**)。

生成したレイヤは初期状態では非表示 (`layer_stack_` に未登録) ですので、`UpDown()` メソッドを使って表示状態にする必要があります。必要なレイヤを表示状態にできたら、最後に `Draw()` で画面へ描画します。`Draw()` はレイヤマネージャに事前に設定してある `pixel_writer` に対して描画を行います。`pixel_writer` はフレームバッファと関連付けられているため、`Draw()` の書き込みは最終的にフレームバッファへの書き込みとなり、画面に出力されるという仕組みです。

ここまでで重ね合わせの処理は大体完了なのですが、やり残したことが 1 つあります。それはデバッグ用のメッセージを表示するためのコンソールクラスの改造です。コンソールクラスはメモリマネージャやレイヤマネージャなどより前に準備すべきものです。なぜなら、それらを準備する間のデバッグ情報もコンソールに表示したいからです。必然的にコンソールクラスはレイヤの仕組みが整う前に使われ始め、レイヤの仕組みが整ったらその仕組みに載せ替えられる必要があります。

リスト 9.21 コンソールの `SetWriter()` を呼び出す (`main.cpp`)

```
DrawDesktop(*pixel_writer);

console = new(console_buf) Console{
    kDesktopFGColor, kDesktopBGColor
};
console->SetWriter(pixel_writer);
printfk("Welcome to MikanOS!\n");
SetLogLevel(kWarn);
```

リスト 9.21 にコンソールクラスの生成処理を示します。改造によって `console->SetWriter(pixel_writer);` という行が追加されているのが分かります。今まではコンソールクラスのコンストラクタに `PixelWriter` を渡していましたが、後から再設定できるように設定用メソッドを増設しました。

リスト 9.22 SetWriter() はコンソール文字列の書き込み先を設定する (console.cpp)

```
void Console::SetWriter(PixelWriter* writer) {
    if (writer == writer_) {
        return;
    }
    writer_ = writer;
    Refresh();
}
```

リスト 9.22 に増設した SetWriter() メソッドの実装を示します。このメソッドはコンソールクラスの書き込み先となる PixelWriter のインスタンスを設定します。その後、バッファに溜まっている文字列を新しい書き込み先に対して書くために Refresh() を実行します。これをしないと、書き込み先を切り替えたときに、それまでに出力された文字列が見えなくなってしまうです。

リスト 9.23 Refresh() はバッファをもとに描画する (console.cpp)

```
void Console::Refresh() {
    for (int row = 0; row < kRows; ++row) {
        WriteString(*writer_, 0, 16 * row, buffer_[row], fg_color_);
    }
}
```

リスト 9.23 に Refresh() メソッドの実装を示します。やっていることは単純で、コンソールクラスが持つバッファ buffer_ の内容を現在設定されている書き込み先 writer_ にひたすら描画します。

リスト 9.24 PutString() が画面を再描画するようにする (console.cpp)

```
void Console::PutString(const char* s) {
    while (*s) {
        if (*s == '\n') {
            Newline();
        } else if (cursor_column_ < kColumns - 1) {
            WriteAscii(*writer_, 8 * cursor_column_, 16 * cursor_row_, *s, fg_color_);
            buffer_[cursor_row_][cursor_column_] = *s;
            ++cursor_column_;
        }
        ++s;
    }
    if (layer_manager) {
        layer_manager->Draw();
    }
}
```

最後に、文字列表示を行うたびに画面を再描画するように改造しておきましょう (**リスト 9.24**)。描画先をフレームバッファ本体からコンソールウィンドウに切り替えた後は、明示的に layer_manager->Draw() を呼び出しておかないと画面が更新されません。

レイヤ機能が作れたので、本当にマウスを重ねても絵が壊れないか試してみましょう (**図 9.10**)。

図 9.10 マウスカーソルを重ねても背景が壊れない



（マウスグリグリ）おお、背景画像が壊れなくなっていますね！ これは嬉しい！ しかし、マウスカーソルを動かすととても遅いし画面がチラチラするようになってしまいました。写真だと分かりませんが。これは次節で対応することにししょう。

コラム 9.1 スマートポインタ

スマートポインタは普通のポインタを多機能にした、賢いポインタの総称です。普通のポインタは他の変数を指し示す機能があります。もちろんスマートポインタも他の変数を指し示す機能は基本として持っていますが、それに加え、スマートポインタの種類によって、変数が不要になった際に自動的に破棄したり、同じ変数を指すポインタを複数作れなくしたりといった機能を持っています。

本文中に登場した `std::shared_ptr` は、指し示す先の変数が不要になったときに自動的に破棄することができるポインタです。これには参照カウントという技術が使われています。`std::shared_ptr` はその内部で参照カウントを保持しており、ポインタがコピーされるたびに参照カウントを増やします。逆にポインタが破棄される際に参照カウントを減らしていき、参照カウントが0になった時点で指す先の変数を破棄します。

```
#include <iostream>
#include <memory>

struct A {
    A(int a, int b) : a{a}, b{b} {}
    int a, b;
};

void Test() {
    std::shared_ptr<A> p = std::make_shared<A>(3, 7); // ポインタを生成 (参照カウント=1)
    {
        std::shared_ptr<A> p2 = p;                    // ポインタをコピー (参照カウント→2)
        std::cout << p2->a << ", " << p2->b << std::endl;
    }                                                  // ポインタを破棄 (参照カウント→1)
    std::cout << p->a << ", " << p->b << std::endl;
}
```

`Test()` の先頭で `std::shared_ptr<A>` のインスタンスが1つ生成されます。A 型の変数を指すスマートポインタ、という意味ですね。`std::make_shared<A>()` は内部で `new` 演算子を使って変数を生成し、受け取った引数を使って初期化します。そして、生成した変数へのポインタを返します。普通のポインタを使うと `A* p = new A(3, 7);` という感じになるでしょうか。

ブロックの内側で `p` をコピーして `p2` を生成しています。`std::shared_ptr` のコピーコンストラクタは参照カウントを1増やすため、この時点で `p` と `p2` は両方とも参照カウント2を持った状態になります。

ブロックの最後で `p2` が破棄されます。`std::shared_ptr` のデストラクタは参照カウントを1減らすため、この時点で `p` と `p2` の参照カウントは両方とも1になります。まだ参照カウントが0より大きいので、ポインタが指す先の変数（`new` により生成された変数）は破棄されません。

`Test()` の最後に `p` が破棄されます。デストラクタによって `p` の参照カウントが1減り、0になります。すると、`p` が指す先の変数が `delete` 演算子によって破棄されます。

スマートポインタを使えば、適切に変数の生成と破棄が行われます。手で `new/delete` を書く必要がなくなり、破棄し忘れる、すでに破棄された変数を使ってしまう、複数回破棄してしまう、というミスを防げます。ということで、MikanOS ではいろいろなところにポインタがコピーされる可能性があるウィンドウを、スマートポインタを用いて管理することにしました。

9.4 重ね合わせ処理の時間計測 (osbook_day09b)

マウスカーソルを動かす処理がとても遅いことが判明しました。ということでこの処理の高速化を目指します。

これは OS の自作に限った話ではなく、何かの処理を高速化（最適化）する際は改善作業の効果が分かる数値を計測することをお勧めします。計測は自動でも手動でもいいですが、計測無しに改善しようとしても効果がよく分からず、結局、ほとんど良くなってないのにソースコードが無駄に複雑化してしまった、という悪い結果になってしまうかもしれません。

この節ではマウスカーソルの移動処理を高速化する準備として、処理にかかっている時間を計ります。時間を計るのにはタイマというものを使います。タイマは一般に数値を記憶するための「カウンタ」と呼ばれるレジスタを持ち、一定時間ごとにカウンタの数値を増やすか減らしていきます。処理の前後でカウンタの値を読み、その差を計算すると処理にかかった時間が分かるという仕組みです。

いくつかの種類のタイマがパソコンに搭載されています。今回はその中でも簡単に使える Local APIC タイマを使ってみたいと思います。Local APIC タイマはカウンタの値を秒に変換するのが面倒ですが、今回は高速化の度合いを計りたいだけです。秒に変換できる必要がありません。

リスト 9.25 Local APIC タイマ操作の関数群 (timer.cpp)

```
#include "timer.hpp"

namespace {
    const uint32_t kCountMax = 0xffffffffu;
    volatile uint32_t& lvt_timer = *reinterpret_cast<uint32_t*>(0xfe00320);
    volatile uint32_t& initial_count = *reinterpret_cast<uint32_t*>(0xfe00380);
    volatile uint32_t& current_count = *reinterpret_cast<uint32_t*>(0xfe00390);
    volatile uint32_t& divide_config = *reinterpret_cast<uint32_t*>(0xfe003e0);
}

void InitializeLAPICTimer() {
    divide_config = 0b1011; // divide 1:1
    lvt_timer = (0b001 << 16) | 32; // masked, one-shot
}

void StartLAPICTimer() {
    initial_count = kCountMax;
}

uint32_t LAPICTimerElapsed() {
    return kCountMax - current_count;
}

void StopLAPICTimer() {
    initial_count = 0;
}
```

リスト 9.25 に Local APIC タイマを操作するための関数群を定義した timer.cpp ファイルを示します。ファイルの先頭では 1 つの定数と、Local APIC タイマに関係する 4 つのレジスタを定義しています。レジスタの意味を表 9.1 に示します（参考文献 [3] Vol.3 「10.5.4 APIC Timer」参照）。

表 9.1 Local APIC タイマのレジスタ

レジスタ名	メモリアドレス	意味
LVT Timer	0xfe00320	割り込みの発生方法の設定など
Initial Count	0xfe00380	カウンタの初期値
Current Count	0xfe00390	カウンタの現在値
Divide Configuration	0xfe003e0	カウンタの減少スピードの設定

`InitializeLAPICTimer()` の実装を見てみます。この関数はタイマの設定を行っています。Local APIC タイマはある周期のクロックに従ってカウンタの値を 1 ずつ減らします。Local APIC タイマにはそのクロックを分周する回路が備わっていき、Divide Configuration レジスタで分周比を設定できます。分周とはクロックを n 分の 1 にすることです。分周回路のおかげでカウンタが減るスピードを抑えられ、長時間の時間を計れるようになります。

表 9.2 Local APIC タイマの分周比設定

Divide Configuration ビット 3,1,0	分周比
000	2
001	4
⋮	⋮
110	128
111	1

Divide Configuration レジスタによる分周比の設定を表 9.2 に示します。分周比を大きくするほどカウンタの減りがゆっくりになります。今回は短時間の計測なので分周は必要ないと判断し、`InitializeLAPICTimer()` では分周比を 1 に設定しています (0b1011 は 2 進数リテラルで、ビット 3、1、0 が 1 であるような数値を表します)。

LVT Timer (Local Vector Table Timer) レジスタのフィールド構造を表 9.3 に示します。このレジスタでは主に割り込みに関する設定を行います。Local APIC タイマは設定した時間が経過したときに割り込みを起こすことができます。LVT Timer レジスタでは割り込みの許可、不許可と割り込みベクタ番号を設定します。今回は割り込みを使わないため、Mask に 1 を設定して割り込みが発生しないようにしておきます。

表 9.3 LVT Timer レジスタのフィールド

ビット位置	フィールド名	意味
0:7	Vector	割り込みベクタの番号
12	Delivery Status	割り込みの配送状況 (0= 空き、1= 配送待ち)
16	Mask	割り込みマスク (1= 割り込み不許可)
17:18	Timer Mode	タイマ動作モード (0= 単発、1= 周期)

タイマの動作モードを選択するのもこの LVT Timer レジスタです。タイマの動作モードは次の 2 通り*⁵を選択できます。今回は 1 回だけ時間を計れば十分ですので単発モードを選択します。

- 0: 単発 (oneshot) モード。1 回タイムアウトしたらタイマ動作が終了する。
- 1: 周期 (periodic) モード。タイムアウト時に初期値を再度読み込み、継続動作する。

単発モードの場合、Initial Count レジスタに値を書き込むことで Local APIC タイマの動作が始まります。動作が始まるとまず Initial Count レジスタの値が Current Count レジスタにコピーされます。その後 1 ずつ値が減っていき、0 に到達すると動作が止まります。このとき Mask=0 であれば、Vector に設定した番号に対して割り込みが発生します。

ということで、Initial Count レジスタに大きな値を書き込んでおいて何らかの処理を実行し、直後に Initial Count レジスタの値を読み出すことで経過時間を計算できます。StartLAPICTimer() ではできるだけ長い時間を計れるように、32 ビット幅の Initial Count レジスタの最大値である 0xffffffff を書き込みます。そして LAPICTimerElapsed() では初期値 0xffffffff と現在値の差を計算し、処理にかかった時間を算出します。

単発モードでも周期モードでも、タイマの動作中に Initial Count レジスタに 0 を書くことでタイマの動作を停止させることができます。

リスト 9.26 timer.hpp

```
#pragma once

#include <stdint>

void InitializeLAPICTimer();
void StartLAPICTimer();
uint32_t LAPICTimerElapsed();
void StopLAPICTimer();
```

Local APIC タイマの操作用の関数群を他のファイルから呼び出せるようにヘッダファイルも作りました (リスト 9.26)。

リスト 9.27 timer.hpp をインクルードする (main.cpp)

```
#include "timer.hpp"
```

これらの関数群を使ってマウスカーソルの移動処理の時間を計ってみます。まずは main.cpp の中で関数を使えるようにするために timer.hpp ファイルをインクルードします (リスト 9.27)。

* 5 CPUID.01H:ECX.TSC_Deadline が 1 の場合、Timer Mode=2 とすることで TSC デッドラインモードを選択できます (このモードの説明は割愛)。

リスト 9.28 Local APIC タイマを初期化する (main.cpp)

```
printk("Welcome to MikanOS!\n");
SetLogLevel(kWarn);

InitializeLAPICTimer();
```

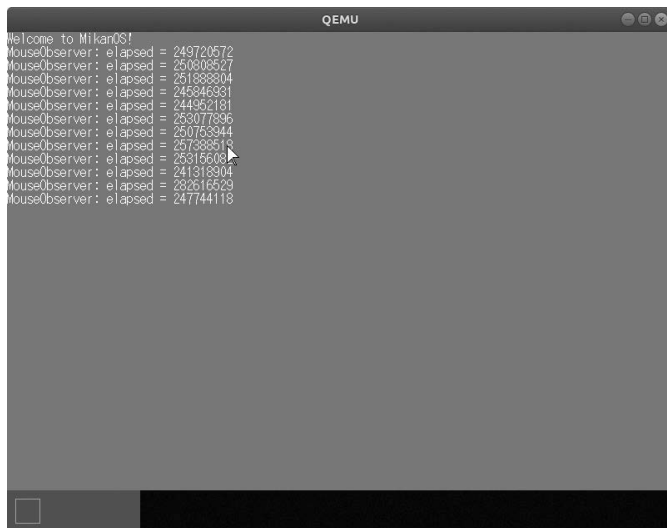
時間を計る前に Local APIC タイマの設定をする必要があります。**リスト 9.28**のように、メイン関数の途中に初期化関数の呼び出し処理を追加しました。マウスを有効化する前の段階ならどこで初期化しても問題ありませんが、タイマをマウス以外の時間測定に使いたくなるかもしれませんから、早い段階で初期化することにしました。

リスト 9.29 マウスの移動処理の時間を計る (main.cpp)

```
void MouseObserver(int8_t displacement_x, int8_t displacement_y) {
    layer_manager->MoveRelative(mouse_layer_id, {displacement_x, displacement_y});
    StartLAPICTimer();
    layer_manager->Draw();
    auto elapsed = LAPICTimerElapsed();
    StopLAPICTimer();
    printk("MouseObserver: elapsed = %u\n", elapsed);
}
```

初期化が終わったらいよいよ時間を計ります。**リスト 9.29**に時間を計るように改造した `MouseObserver()` を示します。今回高速化しようと思っているのはマウスの移動に伴う画面の再描画処理ですので、`layer_manager->Draw()` の時間を計ることにします。この行の直前でタイマを開始 (`StartLAPICTimer()`) し、直後に経過時間を取得 (`LAPICTimerElapsed()`) します。最後に処理にかかった時間を表示します。

図 9.11 マウスカーソルの移動時間を計っている様子



さて、ちゃんと時間が計れているでしょうか？ 実行して確かめてみます (図 9.11)。いいですね。ちゃんと計れているようです。マウスを動かすたびにおおよそ 2.5 億程度の数値が表示されます。単位は分かりませんが、この数値が小さくなるように高速化していけばいいはず。数値は実行する環境、バッテリーなのか電源に繋がっているのか、QEMU で実行している場合は同時に実行している他のソフトウェアの負荷状況などによって変わりますので、計測する際はなるべく環境を揃えてください。

9.5 重ね合わせ処理の高速化 (osbook_day09c)

マウスカーソルの移動処理の時間を計れるようになりました。これできちんと効果を見ながら高速化を目指せます。

高速化するために改造すべき場所を見つけるために、まずはマウスカーソルの移動処理で実施される主な処理を書き出してみます。

1. マウスを動かすと xHC にデータが届き、最終的に MouseObserver() に到達します。
2. MouseObserver() はマウスレイヤの座標を更新し、LayerManager::Draw() を呼び出します。
3. LayerManager::Draw() は最下層のレイヤから順に Layer::DrawTo() を呼び出します。
4. Layer::DrawTo() は Window::DrawTo() を呼び出します。
5. Window::DrawTo() は透過色を考慮しつつ、PixelWriter::Write() を使ってウィンドウの中身を描画します。
6. PixelWriter::Write() は指定された色を指定された座標に描画します。

この中で最も時間がかかるのは 5 と 6 です。なぜなら PixelWriter::Write() がウィンドウのピクセル数だけ繰り返し呼ばれるからです。最も大きなウィンドウである「背景ウィンドウ」の場合、画面の解像度が 1920x1080 だとすると約 207 万ピクセルもあります。関数呼び出し 1 回の時間はそれほどではありませんが、それが 207 万回も繰り返されるとかなりの時間になってしまいます。

5 と 6 の高速化の方針は次の 2 つが考えられるでしょう。

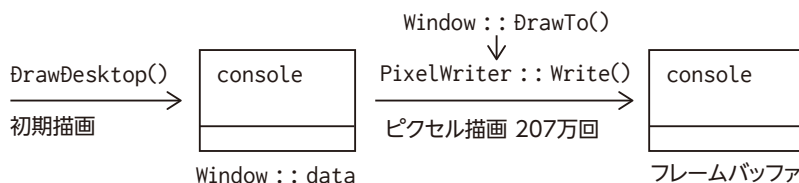
- ウィンドウ画像をフレームバッファに書き込む処理の高速化
- 書き込むべきピクセルの数の削減

1 つ目は毎回 207 万ピクセルをフレームバッファへ書き込む処理を高速化します。207 万ピクセルを書き込むこと自体は変えませんが、それを全体的に速くします。2 つ目はそもそもフレームバッファへ転送するピクセルの数を減らします。マウスを移動したときに更新しなければならない画面の領域は一部ですから、再描画する領域を限定することで再描画が速くなるはず。まずは 1 つ目の手法からやってみようと思います。

PixelWriter::Write() の役目を思い出してみます。それは、引数で指定された PixelColor 型の値を、実際のフレームバッファのデータ形式に変換してフレームバッファへ書き込む、というものでした。フレームバッファのデータ形式がどうであれ、プログラマは PixelColor という共通の型を使えば良く、とても便利な仕組みです。一方でいちいちデータ形式の変換をするため、単に値をメモリに書き込むよりも重たい処理になってしまっています。

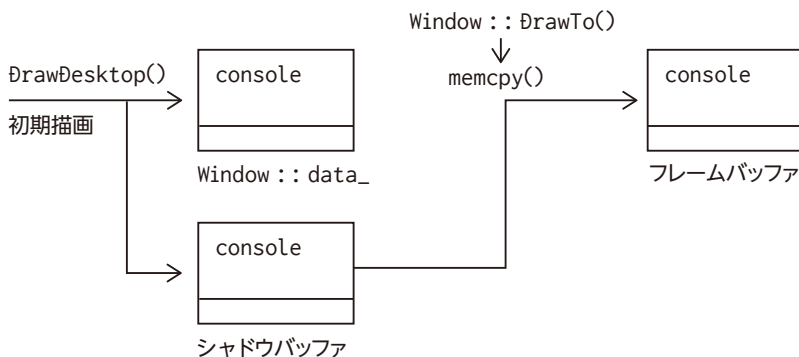
再描画のたびに `PixelWriter::Write()` をピクセルの数だけ呼び出すのは無駄です。なぜなら、マウスが移動している間に背景ウィンドウやマウスウィンドウの画像が変化するとは限らないからです。今はマウスを移動するたびに移動処理の時間を表示するので背景ウィンドウの内容は変化しますが、それでも背景全体の中からすると小さい部分が変わるだけです。また、時間計測しない場合はマウスを動かしたただけでは背景は何も変化しません。それなのにマウスをちょっと動かすたびに背景全体をフレームバッファに書き直す処理がもったいないです。今の描画処理の流れを図示すると図 9.12 のようになります。

図 9.12 ウィンドウをフレームバッファに描画する処理の流れ



`PixelWriter::Write()` を毎回呼び出すのではなく、メモリ領域を `memcpy` で転送するようにしたいです。`Window::data_` の内容を `memcpy` により直接フレームバッファへコピーすることができればかなりの高速化になるはずですが。なぜなら関数呼び出しの回数を大幅に削減できますし、一般的に `memcpy` は高度に最適化されたコピー命令を使用するため高速だからです。しかし、`Window::data_` は `PixelColor` 型の配列なので、それをフレームバッファにそのまま転送するわけにはいきません。たまたまフレームバッファのデータ形式が `PixelColor` と同じメモリ配置であれば可能ですが、どの機種でもそうとは限りません。

図 9.13 シャドウバッファを使った描画処理の流れ



高速化の方法として、図 9.13 に示すようにシャドウバッファというものを考えてみました。シャドウバッファは縦横の大きさは `Window::data_` と同じですが、ピクセルのデータ形式はフレームバッファと合わせてあるメモリ領域です。プログラマからは存在が隠されているためシャドウ (shadow=影) バッファと名付けました。プログラマが `Window::data_` へピクセルを書き込むとき、自動的にシャドウバッファにもデータ形式を変換して書き込みます。

シャドウバッファのデータ形式はフレームバッファと合わせてあるため、画面の再描画の際にはシャドウバッファの内容を `memcpy` でフレームバッファへとコピーすることができます。これで毎回 `PixelWriter::Write()` を呼び出す必要がなくなります。

この方式のデメリットはシャドウバッファ分のメモリ領域が必要なことです。フレームバッファの1ピクセルが4バイトとすると、1920x1080のシャドウバッファは約8MiBです。最近のパソコンは数GB以上のメインメモリが乗っていますので、高速化のためならこの程度のメモリは惜しくないかと筆者は判断しました。

リスト 9.30 Window クラスにシャドウバッファを持たせる (window.hpp)

```
private:
    int width_, height_;
    std::vector<std::vector<PixelColor>> data_{};
    WindowWriter writer_{*this};
    std::optional<PixelColor> transparent_color_{std::nullopt};

    FrameBuffer shadow_buffer_{};
```

リスト 9.30 のように Window クラスにシャドウバッファを持たせるようにしました。FrameBuffer の実装は後でやることにして、まずはこのクラスの使い方から見ていきます。

リスト 9.31 シャドウバッファの初期化 (window.cpp)

```
Window::Window(int width, int height, PixelFormat shadow_format) : width_{width}, height_{height} {
    data_.resize(height);
    for (int y = 0; y < height; ++y) {
        data_[y].resize(width);
    }

    FrameBufferConfig config{};
    config.frame_buffer = nullptr;
    config.horizontal_resolution = width;
    config.vertical_resolution = height;
    config.pixel_format = shadow_format;

    if (auto err = shadow_buffer_.Initialize(config)) {
        Log(kError, "failed to initialize shadow buffer: %s at %s:%d\n",
            err.Name(), err.File(), err.Line());
    }
}
```

リスト 9.31 にシャドウバッファを初期化する処理を書きました。Window クラスを生成するときに自動的にシャドウバッファを初期化します。シャドウバッファには縦横の大きさでピクセルのデータ形式を渡す必要があります。ちょうどいいので FrameBufferConfig 構造体を利用することにしました。config.frame_buffer に nullptr を入れておくことで、shadow_buffer_.Initialize が内部で自動的にメモリ領域を確保する、という仕様にするつもりです。

shadow_buffer_.Initialize() がエラーの際にログを出しているのが不思議かもしれませんが。OS をビルドする際に C++ の例外機能を無効にしておき Window クラスのコンストラクタから呼び出し

元にエラーを返す方法がありません。そのためエラーを報告する方法がログを出すくらいしかないのです。通常の関数の呼び出しであればエラーを戻り値として返せますが、コンストラクタには戻り値がありませんから、コンストラクタの呼び出し元にエラーを返すには例外を使いたいところです。例外が無効になっている環境ではコンストラクタはちょっと制限が厳しいですね……。

リスト 9.32 Write() メソッドはシャドウバッファにも書き込む (window.cpp)

```
const PixelColor& Window::At(Vector2D<int> pos) const{
    return data_[pos.y][pos.x];
}

void Window::Write(Vector2D<int> pos, PixelColor c) {
    data_[pos.y][pos.x] = c;
    shadow_buffer_.Writer().Write(pos, c);
}
```

デスクトップやマウスカーソルの絵を Window に描く際に、今までは Window::data_ にのみ描けば良かったのですが、これからはシャドウバッファにも同じ内容を描く必要があります。ということで **リスト 9.32** のように Write() メソッドを新設しました。これまでは window_.At(x, y) = c; として、非 const 版の At() メソッドの戻り値に代入するという方法でピクセルを描いていましたが、それだと Window::data_ に描くと自動的に Window::shadow_buffer_ にも描かれる、という挙動を実現できません。

リスト 9.33 WindowWriter は新設された Write() でピクセルを描く (window.hpp)

```
/** @brief 指定された位置に指定された色を描く */
virtual void Write(Vector2D<int> pos, const PixelColor& c) override {
    window_.Write(pos, c);
}
```

リスト 9.33 に新設した Write() を使うように改造した WindowWriter の実装を示します。なんか無駄な感じがしますよね、PixelWriter::Write() が Window::Write() を同じ引数で呼び出すなんて。こんなコードを書くくらいなら Window クラス自体を PixelWriter としても使えるようにした方が良かったかもしれません。そのあたりの改造は読者の皆さんにおまかせします。

リスト 9.34 DrawTo() はウィンドウの内容を指定されたフレームバッファへ転送する (window.cpp)

```
void Window::DrawTo(FrameBuffer& dst, Vector2D<int> position) {
    if (!transparent_color_) {
        dst.Copy(position, shadow_buffer_);
        return;
    }

    const auto tc = transparent_color_.value();
    auto& writer = dst.Writer();
    for (int y = 0; y < Height(); ++y) {
        for (int x = 0; x < Width(); ++x) {
            const auto c = At(Vector2D<int>{x, y});
```

```

        if (c != tc) {
            writer.Write(position + Vector2D<int>{x, y}, c);
        }
    }
}
}

```

リスト 9.34 に改造した `DrawTo()` メソッドの実装を示します。改造したのは透過色が設定されていない場合の処理です。透過色がない場合はシャドウバッファの内容を単に目的のフレームバッファへとコピーするだけで良いです。透過色が設定されている場合はシャドウバッファをコピーしてしまうと透過色も一緒にコピーされてしまい、透明ではなくなってしまう。この場合は仕方ないので地道に `writer.Write()` を呼び出して描画します。いまのところ透過色を使っているのがマウスカーソルだけなので、面積としては非常に小さく、大きな影響はないはずです。

リスト 9.35 シャドウバッファを実現する `FrameBuffer` クラス (`frame_buffer.hpp`)

```

#pragma once

#include <vector>
#include <memory>

#include "frame_buffer_config.hpp"
#include "graphics.hpp"
#include "error.hpp"

class FrameBuffer {
public:
    Error Initialize(const FrameBufferConfig& config);
    Error Copy(Vector2D<int> pos, const FrameBuffer& src);

    FrameBufferWriter& Writer() { return *writer_; }

private:
    FrameBufferConfig config_{};
    std::vector<uint8_t> buffer_{};
    std::unique_ptr<FrameBufferWriter> writer_{};
};

int BitsPerPixel(PixelFormat format);

```

シャドウバッファを利用する側について完成したので、今度はシャドウバッファのクラス `FrameBuffer` を実装していきます。まずは作成するクラスの全体像を **リスト 9.35** のように定義しました。

「フレームバッファ」というと普通はディスプレイと接続された（そこに描いた内容がディスプレイに表示される特殊な）メモリ領域のことを意味します。VRAM（Video RAM）と呼ぶこともあります。対してシャドウバッファは単なるメモリ領域で、そこに描いた絵が自動的にディスプレイに表示されることはありませんが、あえて `FrameBuffer` という名前にしました。その理由は、フレームバッファと同じように縦と横のサイズを持ち、フレームバッファと同じようにピクセルのデータ形式が決まっているメモリ領域であることを表現したかったからです。

Framebuffer クラスのメンバ変数を紹介します。config_ は描画領域の縦横サイズ、ピクセルのデータ形式など、描画領域に関する構成情報を保持します。buffer_ はピクセルの配列で、描画領域の本体です。Window::data_ が PixelColor の配列であるのとは異なり、こちらは uint8_t の配列です。ピクセルのデータ形式は機種によって様々ですから何でも格納できる型にしておく必要があります。writer_ はこの描画領域と関連付けられた PixelWriter のインスタンスを格納します。std::shared_ptr ではなく std::unique_ptr を使っているのは、writer_ が指すインスタンスの所有権が FrameBuffer にあるからです。

これから各メソッドを実装していきます。

リスト 9.36 Initialize() は指定された設定でバッファを確保する (frame_buffer.cpp)

```
Error FrameBuffer::Initialize(const FrameBufferConfig& config) {
    config_ = config;

    const auto bits_per_pixel = BitsPerPixel(config_.pixel_format);
    if (bits_per_pixel <= 0) {
        return MAKE_ERROR(Error::kUnknownPixelFormat);
    }

    if (config_.frame_buffer) {
        buffer_.resize(0);
    } else {
        buffer_.resize(
            ((bits_per_pixel + 7) / 8)
            * config_.horizontal_resolution * config_.vertical_resolution);
        config_.frame_buffer = buffer_.data();
        config_.pixels_per_scan_line = config_.horizontal_resolution;
    }

    switch (config_.pixel_format) {
        case kPixelRGBResv8BitPerColor:
            writer_ = std::make_unique<RGBResv8BitPerColorPixelWriter>(config_);
            break;
        case kPixelBGRResv8BitPerColor:
            writer_ = std::make_unique<BGRResv8BitPerColorPixelWriter>(config_);
            break;
        default:
            return MAKE_ERROR(Error::kUnknownPixelFormat);
    }

    return MAKE_ERROR(Error::kSuccess);
}
```

リスト 9.36 に初期化メソッド Initialize() を示します。このメソッドは引数 config の設定にしたがって描画領域を準備します。メソッドの冒頭で 1 ピクセルが占めるビット数を BitsPerPixel() を用いて計算します*6。今のところはあり得ませんが、BitsPerPixel() が対応していないデータ形式の場合にエラーを返すようにしています。

*6 現在のところピクセル形式はすべて 32 ビットなので、わざわざ計算する必要はありません。機種や設定によってはビット数が変わる可能性があることを明示するために、関数で計算しています。

リスト 9.37 BitsPerPixel() は 1 ピクセルのビット数を返す (frame_buffer.cpp)

```
int BitsPerPixel(PixelFormat format) {
    switch (format) {
        case kPixelRGBResv8BitPerColor: return 32;
        case kPixelBGRResv8BitPerColor: return 32;
    }
    return -1;
}
```

参考までに**リスト 9.37**に BitsPerPixel() の実装を紹介します。今は 2 種類しかデータ形式を定義していないため、あまり意味がある実装になっていませんね……。

1 ピクセルあたりのビット数を取得した後は config_.frame_buffer が nullptr かどうかで処理が分かります。特別な場合を除き config_.frame_buffer は nullptr になっていて、else 節が実行されることを想定しています。else 節では動的にメモリ領域を確保します。計算で得たビット数を用いて描画領域に必要なバイト数、すなわち「1 ピクセルのバイト数×横サイズ×縦サイズ」を求めます。buffer_.resize() メソッドに要素数を指定すると、その要素数を格納するに十分な大きさまで配列が拡大されます^{*7}。動的に確保したメモリ領域の先頭ポインタを config_.frame_buffer に設定しておきます。

config_.frame_buffer が nullptr でない場合、すなわち何らかのポインタが設定されている場合は、そのポインタが指すメモリ領域を描画領域として使います。UEFI で取得したフレームバッファ（本来のフレームバッファ）を FrameBuffer クラスとして扱えるように工夫したのです。この場合は動的なメモリ確保は不要ですので buffer_.resize(0) として内部バッファを 0 要素に縮小します。

Initialize() メソッドの後半では writer_ の設定を行います。config_.pixel_format に応じて適切な PixelWriter を生成して writer_ に設定します。ちなみに std::make_unique は初期化済みの std::unique_ptr を生成するための関数で、次の 2 行は同じ意味になります。

```
std::unique_ptr<A>(new A(config_))
std::make_unique<A>(config_)
```

リスト 9.38 Copy() は指定されたバッファを自身のバッファへコピーする (frame_buffer.cpp)

```
Error FrameBuffer::Copy(Vector2D<int> pos, const FrameBuffer& src) {
    if (config_.pixel_format != src.config_.pixel_format) {
        return MAKE_ERROR(Error::kUnknownPixelFormat);
    }

    const auto bits_per_pixel = BitsPerPixel(config_.pixel_format);
    if (bits_per_pixel <= 0) {
        return MAKE_ERROR(Error::kUnknownPixelFormat);
    }
    const auto dst_width = config_.horizontal_resolution;
    const auto dst_height = config_.vertical_resolution;
    const auto src_width = src.config_.horizontal_resolution;
    const auto src_height = src.config_.vertical_resolution;
```

*7 std::vector::resize() は new 演算子を用いてメモリを確保します。少し前に実装した sbrk が役に立っています！

```

const int copy_start_dst_x = std::max(pos.x, 0);
const int copy_start_dst_y = std::max(pos.y, 0);
const int copy_end_dst_x = std::min(pos.x + src_width, dst_width);
const int copy_end_dst_y = std::min(pos.y + src_height, dst_height);

const auto bytes_per_pixel = (bits_per_pixel + 7) / 8;
const auto bytes_per_copy_line =
    bytes_per_pixel * (copy_end_dst_x - copy_start_dst_x);

uint8_t* dst_buf = config_.frame_buffer + bytes_per_pixel *
    (config_.pixels_per_scan_line * copy_start_dst_y + copy_start_dst_x);
const uint8_t* src_buf = src.config_.frame_buffer;

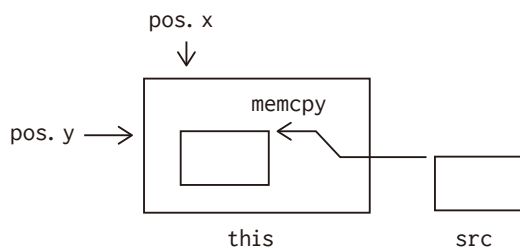
for (int dy = 0; dy < copy_end_dst_y - copy_start_dst_y; ++dy) {
    memcpy(dst_buf, src_buf, bytes_per_copy_line);
    dst_buf += bytes_per_pixel * config_.pixels_per_scan_line;
    src_buf += bytes_per_pixel * src.config_.pixels_per_scan_line;
}

return MAKE_ERROR(Error::kSuccess);
}

```

リスト 9.38 は FrameBuffer クラスの一番の目玉機能、バッファ同士をコピーするための Copy() メソッドです。PixelWriter() で 1 ピクセル毎にデータ変換を繰り返すのではなく、バッファを memcpy することで描画するための中核となるメソッドがこの Copy() です。このメソッドは図 9.14 で示すように、引数 src で指定されたバッファの内容を、引数 pos で指定された位置へとコピーします。

図 9.14 FrameBuffer::Copy() の動作の様子



ちょっと長いですが、上から見ていきましょう。

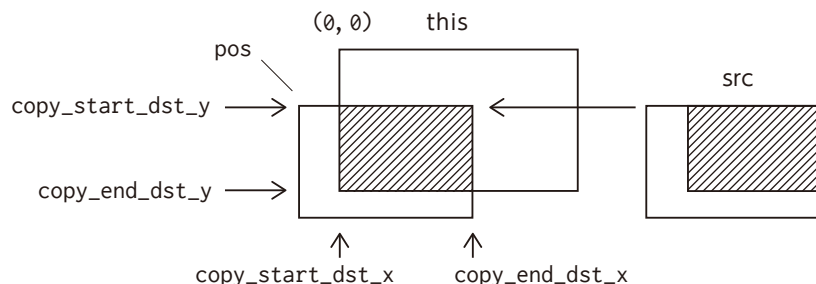
メソッドの先頭でコピー元 (src) とコピー先 (this) のピクセルのデータ形式が同じかどうかを調べます。データ形式が異なるバッファを memcpy してしまうと色が変化してしまったりするので禁止します。

次に 1 ピクセルあたりのビット数を計算しているのは Initialize() と同じです。

その次に 4 行の変数定義があり、いろいろな値を dst_width 等の短い名前の変数に設定しています。これは後半のプログラムを読みやすくするための工夫です。

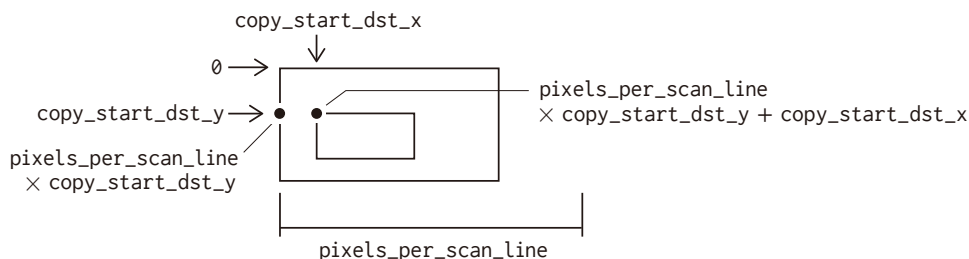
その次の 4 行で copy_start_dst_x 等の変数を定義していますが、ここはちょっと難しいかもしれません。この 4 行は pos で指定した位置が範囲外だったり、src のサイズが大きくてはみ出したりする場合に対応するための計算です。

図 9.15 FrameBuffer::Copy() で範囲外だった場合の動作



pos.x が範囲外にあるパターンを図 9.15 に示します。この場合、コピーすべきは図中の斜線で示した範囲です。その範囲を求めるのが先の 4 行の計算というわけです。

`bytes_per_copy_line` 変数は 1 回あたりのコピーすべきバイト数を表します。図 9.15 で言えば斜線部の横 1 列をコピーするのに必要なバイト数と言えます。

図 9.16 `dst_buf` の初期値の意味

`dst_buf` 変数の初期値はコピー先アドレスになっています(図 9.16)。結構複雑な式ですが、実は「4.2 ピクセルを自在に描く」でも同様の計算が登場しています。「余白を含めた横方向のピクセル数 $\times y + x$ 」という式です。 $y = \text{copy_start_dst_y}$ 、 $x = \text{copy_start_dst_x}$ とすると、括弧の中はこの式と一緒になりますね。これに 1 ピクセルあたりのバイト数をかけて先頭アドレス `config.frame_buffer` を足せば、それが求めたいアドレスになります。

`src_buf` 変数の初期値はコピー元アドレスです。こちらは簡単ですね。

最後にある `for` 文でいよいよデータのコピーを行います。`src_buf` から `dst_buf` へコピーする、というのを列の数だけ繰り返します。コピーする毎に `src_buf` と `dst_buf` を 1 列分だけ増加させ、次のコピーに備えます。

リスト 9.39 LayerManager が FrameBuffer を使うように変更 (layer.hpp)

```
private:
    FrameBuffer* screen_{nullptr};
    std::vector<std::unique_ptr<Layer>> layers_{};
    std::vector<Layer*> layer_stack_{};
    unsigned int latest_id_{0};
```

リスト 9.39 に LayerManager の主な改造部分を紹介します。今までは PixelWriter* writer_{nullptr}; としていたところを FrameBuffer* screen_{nullptr}; に変更しました。画面への描画は FrameBuffer を使ってやるぞ、ということですね。これに伴って、いくつか PixelWriter を使っている部分を、FrameBuffer を使うように書き換えました。

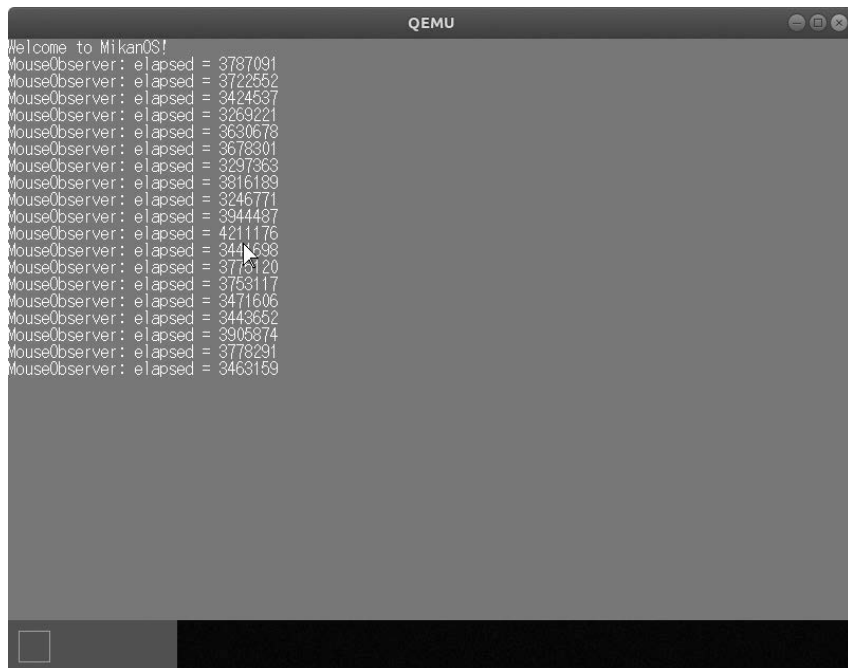
リスト 9.40 本物のフレームバッファを表す FrameBuffer のインスタンスを作成 (main.cpp)

```
FrameBuffer screen;
if (auto err = screen.Initialize(frame_buffer_config)) {
    Log(kError, "failed to initialize frame buffer: %s at %s:%d\n",
        err.Name(), err.File(), err.Line());
}

layer_manager = new LayerManager;
layer_manager->SetWriter(&screen);
```

メイン関数内ではリスト 9.40 に示すように FrameBuffer のインスタンスを作っています。この Initialize() に渡している frame_buffer_config は、実際のディスプレイに関連付けられたフレームバッファを表す構造体です。frame_buffer_config.frame_buffer にはフレームバッファへの先頭アドレスが設定されています。そうです、これが Initialize() メソッドの if 文 if (config_.frame_buffer) が真になる唯一の場合です。

図 9.17 memcpy 版のマウスカーソルの移動時間



さて、改造が済んだのでマウスを動かして時間を計ってみます(図9.17)。大体370万くらいの値になっていますね。改造前が2.5億程度だったので、約67倍の高速化ということになります。これはすごいですよ！

マウスをグリグリしているととても快適になったことが分かると思います。しかし、最初のほうは快適でもコンソールのスクロールが発生するとガクガクし始めるように感じます。コンソールのスクロール処理がもしかしたら遅いのでしょうか？ 次節ではその部分の高速化をやってみたいと思います。

9.6 スクロール処理の時間計測 (osbook_day09d)

高速化は正しい測定から、ということで、コンソールのスクロール処理にかかっている時間を計ります。リスト9.41に示すようにprintkの中に時間計測プログラムを追加し、1行を表示するのにかかる時間を計るようにしてみました。

リスト 9.41 1 行の表示にかかる時間を計る (main.cpp)

```
int printk(const char* format, ...) {
    va_list ap;
    int result;
    char s[1024];

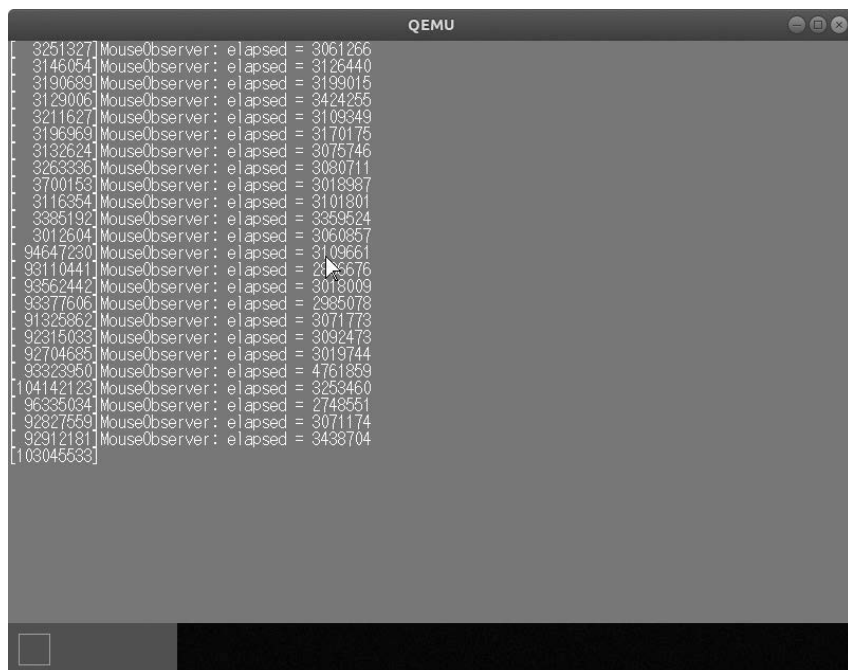
    va_start(ap, format);
    result = vsprintf(s, format, ap);
    va_end(ap);

    StartLAPICTimer();
    console->PutString(s);
    auto elapsed = LAPICTimerElapsed();
    StopLAPICTimer();

    sprintf(s, "[%9d]", elapsed);
    console->PutString(s);
    return result;
}
```

実際に計ってみます(図9.18)。おお、やっぱりスクロール処理が発生するとガクッと処理時間が増えていますね。スクロールが発生していないときが300万程度、スクロールが発生すると1億程度になっています。33倍程度遅くなるようです。これはいくら何でも遅くなりすぎです。

図 9.18 スクロールが発生すると処理時間が増える



9.7 スクロール処理の高速化 (osbook_day09e)

スクロール処理に時間がかかる理由を考えてみます。スクロール処理を担当している `Console::Newline()` の処理をリスト 9.42 に再掲します。このソースコードの初出はリスト 5.16 です。

リスト 9.42 改行処理を行う `Newline()` の定義 (改良前)

```
void Console::Newline() {
    cursor_column_ = 0;
    if (cursor_row_ < kRows - 1) {
        ++cursor_row_;
    } else {
        for (int y = 0; y < 16 * kRows; ++y) {
            for (int x = 0; x < 8 * kColumns; ++x) {
                writer_.Write(x, y, bg_color_);
            }
        }
        for (int row = 0; row < kRows - 1; ++row) {
            memcpy(buffer_[row], buffer_[row + 1], kColumns + 1);
            WriteString(writer_, 0, 16 * row, buffer_[row], fg_color_);
        }
        memset(buffer_[kRows - 1], 0, kColumns + 1);
    }
}
```

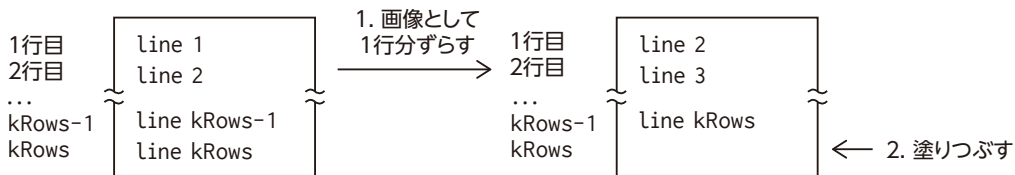
```

    }
}

```

スクロール処理が発生するのは if 文が偽のとき（`cursor_row_` が最終行に到達したとき）です。if 文が偽のときの処理では、まず最初にコンソール全体を背景色で塗りつぶしています。塗りつぶし処理はコンソールの大きさだけ `writer_.Write()` を呼び出しているのですが、おそらくこれが遅いのでしょう。画面の再描画でも `PixelWriter::Write()` の呼び出しが遅くなっている原因でした。`WriteString()` も内部で `PixelWriter::Write()` を用いてフォント描画を行っているため、これも遅くなっている原因ですね。

図 9.19 フレームバッファのピクセル移動を用いたスクロール処理の想像図



これを高速化するのに筆者が考えた方法はフレームバッファ内でのピクセル移動を用いた方法です。スクロール処理はいまのところ「コンソール全体を塗りつぶし、1 行目から `kRows-1` 行目までの文字列を改めて描画する」というやり方になっています。これを「2 行目から最終行までを**画像として** 1 行分ずらし、最終行だけ背景色で塗りつぶしてから文字列を描画する」というやり方に変えるのです（図 9.19）。これで `PixelWriter::Write()` の呼び出し回数を大きく減らせるはずです。

リスト 9.43 コンソールの改行に `Window::Move()` を用いる (`console.cpp`)

```

void Console::Newline() {
    cursor_column_ = 0;
    if (cursor_row_ < kRows - 1) {
        ++cursor_row_;
        return;
    }

    if (window_) {
        Rectangle<int> move_src{{0, 16}, {8 * kColumns, 16 * (kRows - 1)}};
        window_>Move({0, 0}, move_src);
        FillRectangle(*writer_, {0, 16 * (kRows - 1)}, {8 * kColumns, 16}, bg_color_);
    } else {
        FillRectangle(*writer_, {0, 0}, {8 * kColumns, 16 * kRows}, bg_color_);
        for (int row = 0; row < kRows - 1; ++row) {
            memcpy(buffer_[row], buffer_[row + 1], kColumns + 1);
            WriteString(*writer_, Vector2D<int>{0, 16 * row}, buffer_[row], fg_color_);
        }
        memset(buffer_[kRows - 1], 0, kColumns + 1);
    }
}

```


リスト 9.43 にピクセル移動 (`Window::Move()`) を用いたスクロール処理の実装を示します。ピクセル移動メソッドは後で実装しますが、仕様としては `Window::Move(移動先座標, 移動元矩形)` と書くことで移動元矩形で指定された長方形を、移動先座標に移動することとしましょう。コンソールをスクロールするには、2 行目から最終行までの長方形領域を 1 行上にずらせばいいわけですので、移動元の基準座標が (0, 16)、移動先の座標が (0, 0)、長方形の大きさが (8*kColumns, 16*(kRows - 1))、とすればいいでしょう。移動して空いた最終行は `FillRectangle()` で塗りつぶしておきます。

`Rectangle` クラスは `graphics.hpp` の中に新しく作った型で、1 つの長方形 (矩形) を表します。左上の点の座標 (`Vector2D pos`) と縦横の大きさ (`Vector2D size`) を持っています。

ピクセル移動によるスクロール処理ができるようになると、もはや文字列を格納するための `buffer_` は使いみちがなくなります。スクロール時に 1 行ずらす処理をやめてみました。また、ソースコードは掲載していませんが、前記の改造に合わせて `Console` クラスのメンバ変数として `std::shared_ptr<Window> window_;` を加えました。

リスト 9.44 SetWindow() の実装 (console.cpp)

```
void Console::SetWindow(const std::shared_ptr<Window>& window) {
    if (window == window_) {
        return;
    }
    window_ = window;
    writer_ = window->Writer();
    Refresh();
}
```

コンソールが `PixelWriter` の代わりに `Window` を使うようになるタイミングで適切に切りかえができるように `SetWindow()` メソッドを作りました (**リスト 9.44**)。特に難しいところはないと思います。最後に `Refresh()` を呼び出し、コンソール全体を再描画しておくのがポイントです。

リスト 9.45 画面内の移動処理はシャドウバッファに委譲する (window.cpp)

```
void Window::Move(Vector2D<int> dst_pos, const Rectangle<int>& src) {
    shadow_buffer_.Move(dst_pos, src);
}
```

リスト 9.45 にこの節の本題である `Window::Move()` の実装を示します。と言っても `Framebuffer::Move()` に処理を委譲しているだけです。

リスト 9.46 指定された範囲を移動させる (frame_buffer.cpp)

```
void FrameBuffer::Move(Vector2D<int> dst_pos, const Rectangle<int>& src) {
    const auto bytes_per_pixel = BytesPerPixel(config_.pixel_format);
    const auto bytes_per_scan_line = BytesPerScanLine(config_);

    if (dst_pos.y < src.pos.y) { // move up
        uint8_t* dst_buf = FrameAddrAt(dst_pos, config_);
        const uint8_t* src_buf = FrameAddrAt(src.pos, config_);
        for (int y = 0; y < src.size.y; ++y) {
```

```

        memcpy(dst_buf, src_buf, bytes_per_pixel * src.size.x);
        dst_buf += bytes_per_scan_line;
        src_buf += bytes_per_scan_line;
    }
} else { // move down
    uint8_t* dst_buf = FrameAddrAt(dst_pos + Vector2D<int>{0, src.size.y - 1}, config_);
    const uint8_t* src_buf = FrameAddrAt(src_pos + Vector2D<int>{0, src.size.y - 1},
                                         config_);
    for (int y = 0; y < src.size.y; ++y) {
        memcpy(dst_buf, src_buf, bytes_per_pixel * src.size.x);
        dst_buf -= bytes_per_scan_line;
        src_buf -= bytes_per_scan_line;
    }
}
}
}

```

リスト 9.46 が `FrameBuffer::Move()` の本体です。このメソッドでは移動方向が上か下かによって処理を分けています。移動する際は横方向の1行ずつピクセルのコピーをしていきますので、移動元データを上書きしてしまわないように場合分けが必要なのです。

リスト 9.47 ユーティリティ関数群 (frame_buffer.cpp)

```

namespace {
    int BytesPerPixel(PixelFormat format) {
        switch (format) {
            case kPixelRGBResv8BitPerColor: return 4;
            case kPixelBGRResv8BitPerColor: return 4;
        }
        return -1;
    }

    uint8_t* FrameAddrAt(Vector2D<int> pos, const FrameBufferConfig& config) {
        return config.frame_buffer + BytesPerPixel(config.pixel_format) *
            (config.pixels_per_scan_line * pos.y + pos.x);
    }

    int BytesPerScanLine(const FrameBufferConfig& config) {
        return BytesPerPixel(config.pixel_format) * config.pixels_per_scan_line;
    }

    Vector2D<int> FrameBufferSize(const FrameBufferConfig& config) {
        return {static_cast<int>(config.horizontal_resolution),
                static_cast<int>(config.vertical_resolution)};
    }
}

```

移動処理は `memcpy` を使ってやります。その他いくつかの便利関数を定義して使っています。主な便利関数について**リスト 9.47** に定義を載せました。`BytesPerPixel()` は1ピクセルあたりのバイト数を求める関数です。今まで `BitsPerPixel()` を使っていましたが、バイト単位で求めた方が便利なので作り直しました。`FrameAddrAt()` は指定された座標に対応するメモリアドレスを求めます。残り2つは実装を見れば意味が分かります。これら便利関数を使っていくつかの関数の実装を改

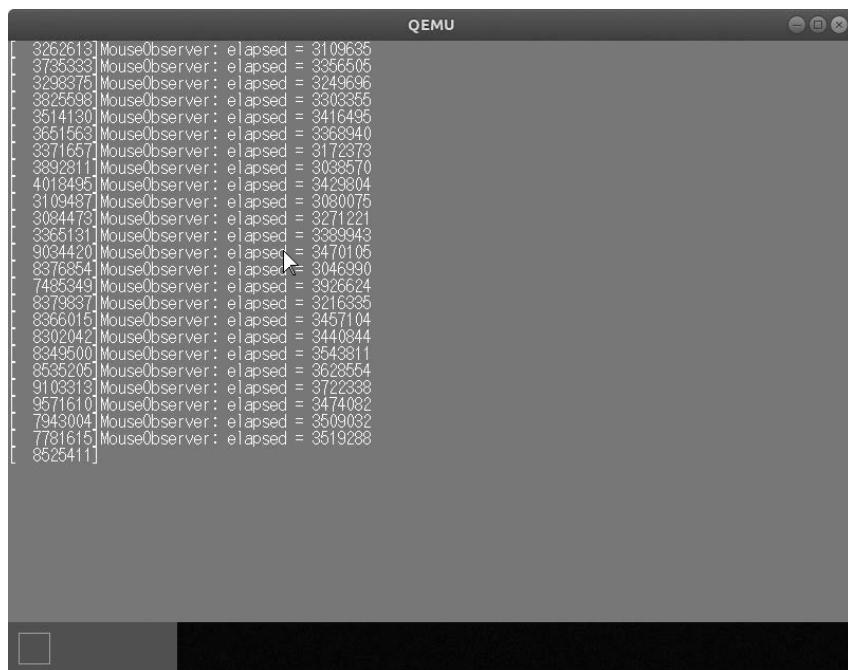
良しました。差分はそれらは添付のソースコードをご覧ください。

リスト 9.48 コンソールの描画先ウィンドウを設定する (main.cpp)

```
DrawDesktop(*bgwriter);  
console->SetWindow(bgwindow);
```

最後に、ウィンドウの準備ができたらリスト 9.48 に示すようにコンソールに設定します。これでコンソールはウィンドウを用いた描画およびスクロール処理を行ってくれるようになります。

図 9.20 スクロールの処理も高速に



改造の効果を見てみます (図 9.20)。スクロールが発生する前の処理時間はほとんど変わらない一方、スクロール処理が発生すると 850 万程度のカウントになっています。遅くなっているといっても 3 倍弱の増加で済んでいます。Window::Move() の実装前に比べると 11 倍の高速化ということになります。筆者の環境ではマウスの動きはまったくカクカクしなくなりました。十分な高速化ができたのではないのでしょうか。

第 10 章

ウィンドウ

この章ではそれっぽい見た目のウィンドウを出してみます。ウィンドウが出ると見た目が GUI を搭載した OS に近づきます。ウィンドウが出るだけでは少し味気ないので、ウィンドウにタイマのカウント値を表示させたりして遊んでみます。

10.1 もっとマウス (osbook_day10a)

皆さんはお気づきでしょうか？ マウスを画面の左右の端より先に動かすと反対側から出てきてしまうことを。普通の OS ではマウスを画面外に動かそうとしても画面の端で止まるようになっているので私たちもそれを真似することにしましょう。

リスト 10.1 Window::DrawTo() での座標計算バグの修正 (window.cpp)

```
for (int y = std::max(0, 0 - position.y);
     y < std::min(Height(), writer.Height() - position.y);
     ++y) {
    for (int x = std::max(0, 0 - position.x);
         x < std::min(Width(), writer.Width() - position.x);
         ++x) {
```

まずはマウスが画面の端から飛び出たときに逆側から出てきてしまうバグを修正します。Window::DrawTo() メソッドの透過色がある場合の処理にバグがあります。リスト 10.1 に示すように、描画範囲を画面端で打ち切るように修正しました。

この修正によりマウスカーソルが画面端を飛び越えて反対側から出てくるバグは治りますが、画面端を超えてどこまでも移動してしまう問題は解決しません。マウスカーソルが迷子にならないように、画面端で止まるように改造してみます。

リスト 10.2 マウスの座標を画面に収まるよう調整する (main.cpp)

```
unsigned int mouse_layer_id;
Vector2D<int> screen_size;
Vector2D<int> mouse_position;

void MouseObserver(int8_t displacement_x, int8_t displacement_y) {
    auto newpos = mouse_position + Vector2D<int>{displacement_x, displacement_y};
    newpos = ElementMin(newpos, screen_size + Vector2D<int>{-1, -1});
    mouse_position = ElementMax(newpos, {0, 0});

    layer_manager->Move(mouse_layer_id, mouse_position);
    layer_manager->Draw();
}
```

リスト 10.2 に修正した MouseObserver() を示します。まずはマウスカーソルの座標を動かしてみ、その結果を画面内に収まるように修正をしています。ElementMin() は 2 つのベクトルを要素毎に比較し、それぞれ小さい方を採用する関数です。次の 1 行

```
newpos = ElementMin(newpos, screen_size + Vector2D<int>{-1, -1});
```

は次の if 文と同じ効果となります。

```
if (newpos.x > screen_size.x - 1) { newpos.x = screen_size.x - 1; }
if (newpos.y > screen_size.y - 1) { newpos.y = screen_size.y - 1; }
```

つまり座標の上限を (screen_size.x-1,screen_size.y-1) に制限するということですね。同様に ElementMax() は 2 つのベクトルを要素毎に比較し、それぞれ大きい方を採用します。座標の下限を (0,0) に制限します。

リスト 10.3 screen_size に画面の大きさを設定する (main.cpp)

```
screen_size.x = frame_buffer_config.horizontal_resolution;  
screen_size.y = frame_buffer_config.vertical_resolution;
```

screen_size の値は main.cpp で設定するようにします (リスト 10.3)。さて、これでうまくマウスカーソルが画面から飛び出ないようになるでしょうか？ 写真で示してもよく分からないので、実際に手元で試してみてください。ちなみに、day10a のコードからは時間計測のコードを削除してあります。今は不要なので。

10.2 はじめてのウィンドウ (osbook_day10b)

マウスカーソルだけでは味気ないですから、そろそろウィンドウっぽい画面を出してみたいです。今まで作り込んできた重ね合わせの処理を活用してさくっと作ってみましょう。

リスト 10.4 小さな表示領域を生成してウィンドウの絵を描く (main.cpp)

```
auto main_window = std::make_shared<Window>(  
    160, 68, frame_buffer_config.pixel_format);  
DrawWindow(*main_window->Writer(), "Hello Window");  
WriteString(*main_window->Writer(), {24, 28}, "Welcome to", {0, 0, 0});  
WriteString(*main_window->Writer(), {24, 44}, " MikanOS world!", {0, 0, 0});
```

リスト 10.4 に表示領域を生成してウィンドウの絵を描く処理を示します。Window クラスは「表示領域」を表す一方、ここで言う「ウィンドウ」は閉じるボタンなどが付いた部品のことなのでちょっと紛らわしいですが、いい言葉が思いつかなかったなのでこのまま進めます。ここでは 160 × 68 の大きさを持つ表示領域を作成し、そこに DrawWinow() を使ってウィンドウの枠を描いています。この関数は後で実装します。次に WriteString() を使ってウィンドウ内に表示するメッセージを書きます。

リスト 10.5 表示領域と関連付けられたレイヤを生成して登録する (main.cpp)

```
auto main_window_layer_id = layer_manager->NewLayer()  
    .SetWindow(main_window)  
    .Move({300, 100})  
    .ID();  
  
layer_manager->UpDown(bglayer_id, 0);  
layer_manager->UpDown(mouse_layer_id, 1);  
layer_manager->UpDown(main_window_layer_id, 1);  
layer_manager->Draw();
```

作成したウィンドウを表示するためのレイヤを生成する処理は**リスト 10.5** のようになります。レイヤはマウスより下の高さにしておきましょう。

リスト 10.6 DrawWindow() はウィンドウのタイトルバーと背景を描画する (window.cpp)

```
void DrawWindow(PixelWriter& writer, const char* title) {
    auto fill_rect = [&writer](Vector2D<int> pos, Vector2D<int> size, uint32_t c) {
        FillRectangle(writer, pos, size, ToColor(c));
    };
    const auto win_w = writer.Width();
    const auto win_h = writer.Height();

    fill_rect({0, 0}, {win_w, 1}, 0xc6c6c6);
    fill_rect({1, 1}, {win_w - 2, 1}, 0xffffffff);
    fill_rect({0, 0}, {1, win_h}, 0xc6c6c6);
    fill_rect({1, 1}, {1, win_h - 2}, 0xffffffff);
    fill_rect({win_w - 2, 1}, {1, win_h - 2}, 0x848484);
    fill_rect({win_w - 1, 0}, {1, win_h}, 0x000000);
    fill_rect({2, 2}, {win_w - 4, win_h - 4}, 0xc6c6c6);
    fill_rect({3, 3}, {win_w - 6, 18}, 0x000084);
    fill_rect({1, win_h - 2}, {win_w - 2, 1}, 0x848484);
    fill_rect({0, win_h - 1}, {win_w, 1}, 0x000000);

    WriteString(writer, {24, 4}, title, ToColor(0xffffffff));

    for (int y = 0; y < kCloseButtonHeight; ++y) {
        for (int x = 0; x < kCloseButtonWidth; ++x) {
            PixelColor c = ToColor(0xffffffff);
            if (close_button[y][x] == '@') {
                c = ToColor(0x000000);
            } else if (close_button[y][x] == '$') {
                c = ToColor(0x848484);
            } else if (close_button[y][x] == ':') {
                c = ToColor(0xc6c6c6);
            }
            writer.Write({win_w - 5 - kCloseButtonWidth + x, 5 + y}, c);
        }
    }
}
```

DrawWindow() の実装を**リスト 10.6** に示します。この関数ではウィンドウのタイトルバー（タイトルを表示する横長の領域）と閉じるボタン、それからウィンドウの背景の絵を描いています。

関数の1行目にあるラムダ式 fill_rect は単に FillRectangle() を呼び出すだけのものです。FillRectangle() という名前よりちょっと短くて、さらに毎回 writer を指定する必要がなく便利です。キャプチャには &writer と指定しています。先頭の & は、この変数を参照型でキャプチャしますよ、という意味があります。PixelWriter クラスは抽象クラスなので値のコピーでは受け取れず、参照キャプチャするしかありません。

関数の中ほどでは先ほど作ったラムダ式 fill_rect を利用してタイトルバーと背景を描画します。fill_rect は左上の座標、長方形のサイズ、色を渡すと、その形の長方形を指定した色で描画します。描画に使う色は16進数6桁のカラーコードで表し、それを fill_rect の中で PixelColor に変換して使っています。カラーコードは左から赤、緑、青の順で2桁ずつ指定します。Web の分野ではよ

図 10.1 ウィンドウを 1 つ表示した様子



10.3 高速カウンタ (osbook_day10c)

ウィンドウに固定のメッセージではなく何か動きのあるものを表示してみたいです。何がいいでしょうか？ あ、そうだ。メイン関数のループの回数を数えて表示させることにします。

リスト 10.8 ウィンドウを小さなサイズで作る (main.cpp)

```
auto main_window = std::make_shared<Window>(
    160, 52, frame_buffer_config.pixel_format);
DrawWindow(*main_window->Writer(), "Hello Window");
```

ループの回数は 1 行で表示できるので、とりあえずウィンドウのサイズを 1 行分 (16 ピクセル) だけ小さくしました (リスト 10.8)。表示していた文字列も消しました。

リスト 10.9 カウンタ変数を作る (main.cpp)

```
char str[128];
unsigned int count = 0;
```

数字を数えるためのカウンタ変数 `count` を初期化しておきます (リスト 10.9)。カウント値を表示するときに使う文字列バッファ `str` も定義しておきます。

リスト 10.10 カウント変数の値を表示する (main.cpp)

```

++count;
sprintf(str, "%010u", count);
FillRectangle(*main_window->Writer(), {24, 28}, {8 * 10, 16}, {0xc6, 0xc6, 0xc6});
WriteString(*main_window->Writer(), {24, 28}, str, {0, 0, 0});
layer_manager->Draw();

__asm__("cli");
if (main_queue.Count() == 0) {
    __asm__("sti");
    continue;
}

```

リスト 10.10 がカウンタ変数でループ回数を数える処理と、それをウィンドウに表示する処理です。数え方はいたって単純で、count をインクリメントしているだけです。その値を sprintf() で文字列に変換し、WriteString() によりウィンドウに描画します。

main_queue.Count() == 0 が真のとき、今までは sti に続けて hlt を実行していました。hlt することで割り込みが来るまで CPU を待機させ、消費電力を下げていました。しかし今回は割り込みが来ないときでも全速力でループを回してカウンタをインクリメントさせたいので hlt をやめました。

図 10.2 カウント値を表示している様子



実行すると図 10.2 のようになりました。カウンタがすごい速度で増えていくのが分かると思います。写真では伝わりませんが、画面がとってもちらつくのが気になります。これは、1 増やす毎に layer_manager->Draw() を呼び出し、画面全体を書き換えてしまっているからです。次の節で対処することにししょう。

10.4 チラチラ解消 (osbook_day10d)

高速カウンタがチラチラするのは画面を再描画する際に全体を描画し直しているからです。背景を描いてからウィンドウを描いていますので、ウィンドウが表示されていない時間帯が存在するのです。存在するというより、ウィンドウが表示されていない時間の方が長いかもしれません。これではちらつくのも当然でしょう。

さて、これを解消しつつ画面描画の高速化もできちゃう方法があります。それは描画範囲を制限する方法です。ウィンドウだけを再描画するにすれば、ウィンドウが常に画面に表示されるようになってチラチラしなくなるはずです。しかも描画するピクセル数が大幅に減るので描画処理もかなり速くな

るはずです。早速改造してみます。

リスト 10.11 範囲またはレイヤを指定して描画する (layer.cpp)

```
void LayerManager::Draw(const Rectangle<int>& area) const {
    for (auto layer : layer_stack_) {
        layer->DrawTo(*screen_, area);
    }
}

void LayerManager::Draw(unsigned int id) const {
    bool draw = false;
    Rectangle<int> window_area;
    for (auto layer : layer_stack_) {
        if (layer->ID() == id) {
            window_area.size = layer->GetWindow()->Size();
            window_area.pos = layer->GetPosition();
            draw = true;
        }
        if (draw) {
            layer->DrawTo(*screen_, window_area);
        }
    }
}
```

まずは `LayerManager::Draw()` の改造を行います (**リスト 10.11**)。今までこのメソッドは何も引数を取らず、画面全体を描画し直していました。今回の改造では描画する範囲またはレイヤ ID を指定するようにして、特定の範囲だけを再描画するようにしました。引数が異なるだけの同じ名前の関数、メソッドを複数作れるのは C++ の関数のオーバーロード (overload) という仕組みのおかげです。

描画範囲を引数に取る方の `Draw(const Rectangle<int>& area)` は最下層のレイヤから順に、その範囲に絞って描画します。

レイヤ ID を引数に取る方の `Draw(unsigned int id)` は、指定されたレイヤとそれより上にあるレイヤだけを描画します。指定されたレイヤより下のレイヤはどうせ上書きされてしまいますから、再描画する意义がありません。 `layer_stack_` を最下層から探して行って、指定されたレイヤが見つかったら描画範囲を決定し、 `draw` フラグをセットします。以後、 `draw` フラグはずっと `true` になるため、描画処理が実行されていきます。

リスト 10.12 コンソールは自身のレイヤを指定して再描画する (console.cpp)

```
if (layer_manager) {
    layer_manager->Draw(layer_id_);
}
```

`LayerManager::Draw()` の引数が変わったので、呼び出し部分を改造する必要があります。まずはコンソールの描画処理の改造部分を **リスト 10.12** に示します。 `layer_id_` というメンバ変数にコンソール用のレイヤ ID を設定するようにして、それを描画に用いるようにしました。

リスト 10.13 初回の描画は画面全体を対象にする (main.cpp)

```
layer_manager->UpDown(bglayer_id, 0);
layer_manager->UpDown(console->LayerID(), 1);
layer_manager->UpDown(main_window_layer_id, 2);
layer_manager->UpDown(mouse_layer_id, 3);
layer_manager->Draw({{0, 0}, screen_size});
```

リスト 10.14 カウンタの表示はメインウィンドウだけを再描画する (main.cpp)

```
++count;
sprintf(str, "%010u", count);
FillRectangle(*main_window->Writer(), {24, 28}, {8 * 10, 16}, {0xc6, 0xc6, 0xc6});
WriteString(*main_window->Writer(), {24, 28}, str, {0, 0, 0});
layer_manager->Draw(main_window_layer_id);
```

メイン関数の中でレイヤを生成した後の1回目の描画処理(リスト 10.13)とカウンタの表示処理(リスト 10.14)も改造する必要があります。1回目の描画処理では画面全体を書き直す必要があるのですが、レイヤ ID ではなく描画範囲を指定する方の `Draw()` を呼び出すようにしました。でもよく考えるとレイヤ ID を指定する方を使って `layer_manager->Draw(bglayer_id)` としても同じでしたね……。カウンタの表示処理ではメインウィンドウの ID を指定して再描画を行います。こうすることで、余分な部分が再描画されず、チラチラしなくなるはずです。

リスト 10.15 `Layer::DrawTo()` が描画範囲を受け取るようにする (layer.cpp)

```
void Layer::DrawTo(FrameBuffer& screen, const Rectangle<int>& area) const {
    if (window_) {
        window_->DrawTo(screen, pos_, area);
    }
}
```

リスト 10.16 `Window::DrawTo()` が描画範囲を受け取るようにする (window.cpp)

```
void Window::DrawTo(FrameBuffer& dst, Vector2D<int> pos, const Rectangle<int>& area) {
    if (!transparent_color_) {
        Rectangle<int> window_area(pos, Size());
        Rectangle<int> intersection = area & window_area;
        dst.Copy(intersection.pos, shadow_buffer_, {intersection.pos - pos, intersection.size});
        return;
    }
}
```

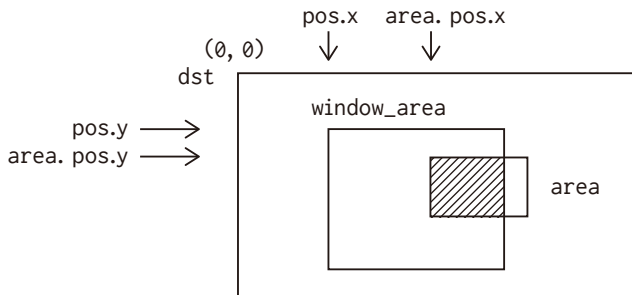
`LayerManager::Draw()` は内部で `Layer::Draw()` に対して描画範囲を指定するようになったので、`Layer::Draw()` 側を改造する必要があります。ということでリスト 10.15、リスト 10.16 に改造箇所を示します。`Layer::DrawTo()` はウィンドウに処理を委譲しているだけですから、本質的な変更は `Window::DrawTo()` だけです。

`Window::DrawTo()` の引数の意味を再確認しておきます。`dst` はウィンドウの絵を描画すべきバッファ、`pos` はそのバッファの左上を基準とするウィンドウの左上の座標です。そして今回追加された引数 `area` は**バッファの左上を基準とする描画範囲**です。ウィンドウ内での座標ではなく、バッファを基

準とした座標であることを意識してください。

このメソッドの中で実行している `area & window_area` が特徴的です。矩形同士の `&` 演算は共通部分を表します（そうなるように後で実装します）。共通部分は数学の用語ですが、ここでは矩形の重なった部分を意味します。このプログラムでは `area` と `window_area` の重なった部分を計算しています。

図 10.3 ウィンドウと描画範囲の共通部分



例えば図 10.3 において、斜線の部分が `area` と `window_area` の共通部分となります。考え方はとても簡単ですね。実際にこれを計算する `&` 演算子を実装します。

リスト 10.17 矩形に対する `operator&()` は共通部分を計算する (graphics.hpp)

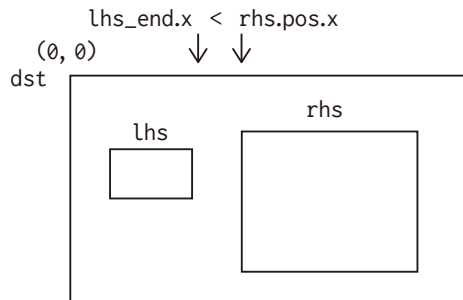
```
template <typename T, typename U>
Rectangle<T> operator&(const Rectangle<T>& lhs, const Rectangle<U>& rhs) {
    const auto lhs_end = lhs.pos + lhs.size;
    const auto rhs_end = rhs.pos + rhs.size;
    if (lhs_end.x < rhs.pos.x || lhs_end.y < rhs.pos.y ||
        rhs_end.x < lhs.pos.x || rhs_end.y < lhs.pos.y) {
        return {{0, 0}, {0, 0}};
    }

    auto new_pos = ElementMax(lhs.pos, rhs.pos);
    auto new_size = ElementMin(lhs_end, rhs_end) - new_pos;
    return {new_pos, new_size};
}
```

リスト 10.17 に示すように `&` 演算子をオーバーロードしました。引数の `lhs` と `rhs` はそれぞれ左辺、右辺を表す英語 Left Hand Side、Right Hand Side の頭文字を取ったものです。 `lhs` & `rhs` と書いたときに `lhs` は左側に、`rhs` は右側に来ますよね。

演算子の前半では 2 つの矩形 `lhs` と `rhs` が重なっていない場合の処理をしています。 `lhs_end` と `rhs_end` はそれぞれの矩形の右下の座標を意味します。したがって `lhs_end.x < rhs.pos.x` という式は、矩形 `rhs` の左端より矩形 `lhs` の右端の方が小さい（＝左にある）ときに真となります。その時はどう頑張っても 2 つの矩形は重なりませんね（図 10.4）。

図 10.4 lhs_end.x < rhs.pos.x が真となるとき の位置関係



同様に Y 軸についても調べ、さらに 2 つの矩形が左右上下で入れ替わったパターンを調べます。2 つの矩形が重ならない場合は `operator&()` は面積が 0 の矩形を返すことにします。

リスト 10.18 Copy() にコピー元の範囲を指定できるようにする (frame_buffer.cpp)

```
Error FrameBuffer::Copy(Vector2D<int> dst_pos, const FrameBuffer& src,
                        const Rectangle<int>& src_area) {
    if (config_.pixel_format != src.config_.pixel_format) {
        return MAKE_ERROR(Error::kUnknownPixelFormat);
    }

    const auto bytes_per_pixel = BytesPerPixel(config_.pixel_format);
    if (bytes_per_pixel <= 0) {
        return MAKE_ERROR(Error::kUnknownPixelFormat);
    }

    const Rectangle<int> src_area_shifted(dst_pos, src_area.size);
    const Rectangle<int> src_outline{dst_pos - src_area.pos, FrameBufferSize(src.config_)};
    const Rectangle<int> dst_outline{{0, 0}, FrameBufferSize(config_)};
    const auto copy_area = dst_outline & src_outline & src_area_shifted;
    const auto src_start_pos = copy_area.pos - (dst_pos - src_area.pos);

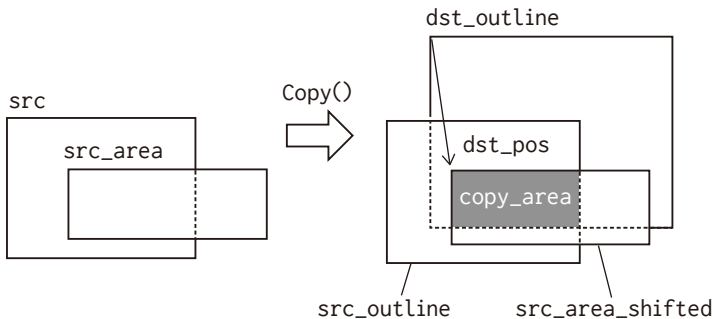
    uint8_t* dst_buf = FrameAddrAt(copy_area.pos, config_);
    const uint8_t* src_buf = FrameAddrAt(src_start_pos, src.config_);

    for (int y = 0; y < copy_area.size.y; ++y) {
        memcpy(dst_buf, src_buf, bytes_per_pixel * copy_area.size.x);
        dst_buf += BytesPerScanLine(config_);
        src_buf += BytesPerScanLine(src.config_);
    }

    return MAKE_ERROR(Error::kSuccess);
}
```

`Window::DrawTo()` が呼び出す `FrameBuffer::Copy()` にもコピー元の矩形を指定するために引数を追加しました (リスト 10.18)。今までは `src` で指定したバッファの全体をコピーしようとしていましたが、今回の修正により、矩形 `src_area` でコピー元の範囲を指定できるようになりました。

図 10.5 FrameBuffer::Copy() に登場する矩形の関係



3つの矩形 `src_area_shifted`、`src_outline`、`dst_outline` が登場して少し複雑なので図にしてみました (図 10.5)。これらの複雑な計算の目的は `copy_area` を求めることです。 `copy_area` は実際にピクセルをコピーする範囲で、それはコピー元の範囲やコピー先の範囲をはみ出ないように慎重に計算しなければなりません。いずれかをはみ出してしまうとメモリ破壊につながります。

リスト 10.19 Move() はレイヤを移動して描画し直す (layer.cpp)

```
void LayerManager::Move(unsigned int id, Vector2D<int> new_pos) {
    auto layer = FindLayer(id);
    const auto window_size = layer->GetWindow()->Size();
    const auto old_pos = layer->GetPosition();
    layer->Move(new_pos);
    Draw({old_pos, window_size});
    Draw(id);
}
```

今回の主な変更点の1つは `LayerManager::Draw()` が範囲またはレイヤを指定して描画するようにした点です。もう1つ大きな変更点があります。それは `LayerManager::Move()` が自動的に画面を再描画するようにした点です。変更した `LayerManager::Move()` の実装をリスト 10.19 に示します。このメソッドは指定されたレイヤを指定した位置に移動した後、そのレイヤの移動前後の範囲を描画します。

リスト 10.20 コンソール用のウィンドウを生成する (main.cpp)

```
auto console_window = std::make_shared<Window>(
    Console::kColumns * 8, Console::kRows * 16, frame_buffer_config.pixel_format);
console->SetWindow(console_window);
```

リスト 10.21 コンソール用のレイヤを生成する (main.cpp)

```
console->SetLayerID(layer_manager->NewLayer()
    .SetWindow(console_window)
    .Move({0, 0})
    .ID());
```

リスト 10.22 SetLayerID() はコンソールにレイヤ ID を設定する (console.cpp)

```
void Console::SetLayerID(unsigned int layer_id) {
    layer_id_ = layer_id;
}

unsigned int Console::LayerID() const {
    return layer_id_;
}
```

特定のレイヤ（が持つウィンドウ）だけに限定して描画することで画面の再描画が早くなったはずで
す。ということは、コンソールもレイヤに分けておけばスクロールが効率的になるはずで
す。さくっと
やりました（リスト 10.20、リスト 10.21、リスト 10.22）。

さて、ここまで改造したら実行してみます。ふむふむ、チラチラしなくなりました。これは快適で
す。もちろんマウスも動きますよ。ぐるぐるー。あ、マウスカーソルをウィンドウの上に重ねるとまだチラ
チラする気がします。スクリーンショットを何枚か撮ってみたら運良く 図 10.6 が撮れました。マウス
カーソルの再描画中のようです。

図 10.6 マウスをカウンタに重ねるとチラチラする



このチラチラ問題は次節で対応しましょう。ひとまずチラチラ問題を無視してループの時間を計って
みました。プログラムは紹介しませんが、Local APIC タイマを使ってループが一定数（例えば 100 回とか）
回るとにかかった時間を計ってみると、「10.3 高速カウンタ」で作ったものに比べて、ここで作った
ものは 7 倍程度高速化していました。やったー。

10.5 バックバッファ (osbook_day10e)

マウスカーソルをカウンタの上に重ねるとちらつくのは、カウンタを描画した後にマウスを描いてい
るため、マウスカーソルが表示されていない期間があるからです。この現象は画面全体を描画してい
たときも発生していましたね。今は描画範囲を絞っているものの、その描画範囲にマウスカーソルが入っ
てきてしまうと同様の現象が起きるというわけです。

これを完全に解決するにはバックバッファという仕組みを使えば良さそうです。バックバッファとい
うのはフレームバッファと同じ大きさを持つバッファのことです。バックバッファに描画を行った後で
バックバッファをフレームバッファに転送することにより、マウスカーソルが表示されていない期間を
完全になくすことができます。

リスト 10.23 バックバッファを LayerManager のメンバ変数とする (layer.hpp)

```
private:
    FrameBuffer* screen_{nullptr};
    mutable FrameBuffer back_buffer_{};
```

ということで早速バックバッファを実装していきます。まずはリスト 10.23 のようにバックバッファ用のメンバ変数 `back_buffer_` を追加します。`mutable` は初登場のキーワードです。`mutable` として宣言されたメンバ変数は `const` なメソッド内からでも変更することができるようになります。`LayerManager::Draw()` は `const` 宣言されているため、その中で `back_buffer_` を書き換えるためにはメソッドから `const` を外すか、`back_buffer_` を `mutable` にするかしかありません。

`mutable` の濫用はよくないですが、今回のような使い方であればむしろ `mutable` にするのが自然だと筆者は思います。`back_buffer_` は `Draw()` メソッドのローカル変数とすることもできますが、毎回メモリを確保、破棄することになって遅くなってしまいます。その最適化のためにメンバ変数として定義してあるにすぎないのです。`back_buffer_` に対する変更は他のメソッドに影響を及ぼしませんから、`mutable` 宣言が妥当でしょう。

リスト 10.24 バックバッファを初期化する (layer.cpp)

```
void LayerManager::SetWriter(FrameBuffer* screen) {
    screen_ = screen;

    FrameBufferConfig back_config = screen->Config();
    back_config.frame_buffer = nullptr;
    back_buffer_.Initialize(back_config);
}
```

リスト 10.24 に改造した `LayerManager::SetWriter()` を示します^{*2}。今までは 1 行目だけでしたが、その後ろにバックバッファの初期化処理を付け足しました。バックバッファは与えられた `screen` と同じサイズ、同じデータ形式で作らねばなりませんから、`back_config.frame_buffer` 以外のパラメータは `screen->Config()` からコピーしたものを使います。

リスト 10.25 範囲を指定して全レイヤを描画する (layer.cpp)

```
void LayerManager::Draw(const Rectangle<int>& area) const {
    for (auto layer : layer_stack_) {
        layer->DrawTo(back_buffer_, area);
    }
    screen_->Copy(area.pos, back_buffer_, area);
}
```

*2 引数の型が `FrameBuffer` に変わった際にメソッド名も `SetScreen()` などに変えれば良かったかもしれませんが、サボってそのままにしています。

back_buffer_ を用いて描画するように改造した LayerManager::Draw() の実装を **リスト 10.25** に示します。今までは layer->DrawTo で直接 screen_ に描画していたのを back_buffer_ に描画するように変更し、すべてのレイヤの描画が終わった後に back_buffer_ を screen_ にコピーするようにしました。

リスト 10.26 指定のレイヤから上のレイヤを描画する (layer.cpp)

```
void LayerManager::Draw(unsigned int id) const {
    bool draw = false;
    Rectangle<int> window_area;
    for (auto layer : layer_stack_) {
        if (layer->ID() == id) {
            window_area.size = layer->GetWindow()->Size();
            window_area.pos = layer->GetPosition();
            draw = true;
        }
        if (draw) {
            layer->DrawTo(back_buffer_, window_area);
        }
    }
    screen_->Copy(window_area.pos, back_buffer_, window_area);
}
```

レイヤを指定する方の Draw() も同様に改造します(**リスト 10.26**)。back_buffer_ に描画し終わった後に screen_ にコピーするという流れは先ほどと同じです。

ぜひ手元で実行してみてください。スクリーンショットは変化がないので載せませんが、マウスカーソルをカウンタに重ねても全然ちらつかなかったのが分かると思います。とても快適ですね！ ちなみに、筆者が性能を測ってみたところ、約 0.7 倍の高速化、つまり 30% ほど遅くなってしまいました……。バックバッファからフレームバッファへのコピーが毎回発生するので当然ではありますが、少し悲しいです。まあでも、ちらつかなかったので良しとしましょう。

10.6 ウィンドウのドラッグ移動 (osbook_day10f)

ウィンドウの描画をかなり頑張っていい感じになりましたが、機能としてはほとんど進歩がなくて退屈です。気づいてしまったのですが、せっかくマウスが動くのに何も役に立っていませんよね。ウィンドウのちらつきの原因になったりして、むしろ邪魔しているとも言えます。マウスに役立ってもらうために、この節ではマウスでウィンドウを移動できるようにしたいと思います。

ウィンドウのドラッグ移動というのは、ウィンドウの上でマウスの左ボタンをクリックし、ボタンを押したままマウスを動かすことでウィンドウを移動させることです。これをやるためには、何はともあれマウスのボタンクリックを取得できるように USB ドライバを改良する必要があります。いまのところ、USB ドライバはマウスが動いたときに MouseObserver() を呼び出してくれますが、このときに取得できる情報は座標の差分だけです。MouseObserver() の引数としてボタンの押下状態が渡されるようにしたいです。

ということで USB ドライバの中のマウス情報を扱う部分の改造を行います。USB ドライバは筆者が作ったものを皆さんは使っているだけですので、その実装はあまり分からないでしょう。修正点をさらっ

と説明していきますが、分からなくても気にせず読み飛ばして大丈夫です。

リスト 10.27 マウスから受信したデータを解釈する (usb/classdriver/mouse.cpp)

```
Error HIDMouseDriver::OnDataReceived() {
    uint8_t buttons = Buffer()[0];
    int8_t displacement_x = Buffer()[1];
    int8_t displacement_y = Buffer()[2];
    NotifyMouseMove(buttons, displacement_x, displacement_y);
    Log(kDebug, "%02x,%3d,%3d\n", buttons, displacement_x, displacement_y);
    return MAKE_ERROR(Error::kSuccess);
}
```

リスト 10.27 に USB のマウスドライバがマウスからデータを受信したときに呼び出される関数 `OnDataReceived()` を示します。この関数の主目的は単なるバイト列である受信データを解析して意味付けし、`NotifyMouseMove()` を解析済みの値とともに呼び出すことです。今まで `NotifyMouseMove()` には座標の差分しか渡していませんでしたが、今回の改造でその時点のボタンの押下状況 `buttons` も渡すことにしました。

リスト 10.28 登録されたオブザーバを呼ぶ (usb/classdriver/mouse.cpp)

```
void HIDMouseDriver::NotifyMouseMove(
    uint8_t buttons, int8_t displacement_x, int8_t displacement_y) {
    for (int i = 0; i < num_observers_; ++i) {
        observers_[i](buttons, displacement_x, displacement_y);
    }
}
```

リスト 10.28 に `NotifyMouseMove()` の実装を示します。この関数は `observers_` に登録されたすべての関数を呼び出します。その際の引数に `buttons` を加えたのが主な変更点です。

リスト 10.29 マウスの移動量に従ってカーソルを動かす (main.cpp)

```
void MouseObserver(uint8_t buttons, int8_t displacement_x, int8_t displacement_y) {
    static unsigned int mouse_drag_layer_id = 0;
    static uint8_t previous_buttons = 0;

    const auto oldpos = mouse_position;
    auto newpos = mouse_position + Vector2D<int>{displacement_x, displacement_y};
    newpos = ElementMin(newpos, screen_size + Vector2D<int>{-1, -1});
    mouse_position = ElementMax(newpos, {0, 0});

    const auto posdiff = mouse_position - oldpos;

    layer_manager->Move(mouse_layer_id, mouse_position);

    const bool previous_left_pressed = (previous_buttons & 0x01);
    const bool left_pressed = (buttons & 0x01);
    if (!previous_left_pressed && left_pressed) {
        auto layer = layer_manager->FindLayerByPosition(mouse_position, mouse_layer_id);
```

```
    if (layer) {  
        mouse_drag_layer_id = layer->ID();  
    }  
    } else if (previous_left_pressed && left_pressed) {  
        if (mouse_drag_layer_id > 0) {  
            layer_manager->MoveRelative(mouse_drag_layer_id, posdiff);  
        }  
    } else if (previous_left_pressed && !left_pressed) {  
        mouse_drag_layer_id = 0;  
    }  
  
    previous_buttons = buttons;  
}
```

NotifyMouseMove() の変更に合わせて MouseObserver() を改造します (リスト 10.29)。第 1 引数で buttons を受け取るようにしました。buttons 引数はビットごとにマウスのボタンの押下状況を表します。ボタンが離れていれば 0、押されていれば 1 となります。一般的なマウスでは左ボタンがビット 0、右ボタンがビット 1、中央ボタンがビット 2 となるようです。

先頭で定義されている 2 つの静的変数を説明します。mouse_drag_layer_id はマウスでドラッグ移動しているレイヤを記憶します。どのレイヤも移動していない場合に 0 にしておくことにします (レイヤ ID は 1 以上の整数ですので 0 を無効値としても問題ありません)。previous_buttons は 1 回前のボタンの押下状況を記憶します。USB ドライバからは「現在押されているかどうか」の情報しか来ません。この変数を使って「今押されたのか、前から押されているのか」の判別をします。

MouseObserver() の前半では今までどおりマウスカーソルの移動処理を行います。変数 posdiff は重要で、マウスカーソルの移動量を表します。移動量は displacement_x, displacement_y じゃないのかと思うかもしれませんが、画面の端で移動量が修正されることがありますので、修正後の移動量を改めて計算しているということです。

MouseObserver() の後半ではウィンドウの移動処理を行います。if 文の上から順に、左ボタンを押した瞬間、左ボタンを押している間、左ボタンを離れた瞬間で条件分岐しています。左ボタンが押されたとき、layer_manager->FindLayerByPosition() を使ってマウスカーソルの座標にあるウィンドウを持つレイヤを取得します。レイヤが見つかったら mouse_drag_layer_id にそのレイヤの ID を設定しておきます。以後、このレイヤがドラッグ移動の対象となります。

左ボタンを押している間は layer_manager->MoveRelative() を使ってレイヤを移動させます。このとき、先ほど計算しておいた posdiff が活躍します。左ボタンを離れたら mouse_drag_layer_id を 0 にリセットしておきます。

リスト 10.30 指定した座標にある最前面レイヤを探す (layer.cpp)

```

Layer* LayerManager::FindLayerByPosition(Vector2D<int> pos, unsigned int exclude_id) const {
    auto pred = [pos, exclude_id](Layer* layer) {
        if (layer->ID() == exclude_id) {
            return false;
        }
        const auto& win = layer->GetWindow();
        if (!win) {
            return false;
        }
        const auto win_pos = layer->GetPosition();
        const auto win_end_pos = win_pos + win->Size();
        return win_pos.x <= pos.x && pos.x < win_end_pos.x &&
            win_pos.y <= pos.y && pos.y < win_end_pos.y;
    };
    auto it = std::find_if(layer_stack_.rbegin(), layer_stack_.rend(), pred);
    if (it == layer_stack_.rend()) {
        return nullptr;
    }
    return *it;
}

```

リスト 10.30 に `LayerManager::FindLayerByPosition()` の実装を示します。この関数は指定された座標 `pos` にウィンドウを持つレイヤを、表示順に探します。ただし、単純に探してしまうと常にマウスカーソルがその条件に当てはまってしまうため、`exclude_id` で検索対象から除くレイヤを指定できるようにしてあります。

述語 `pred` はレイヤを 1 つ受け取り、そのレイヤが持つウィンドウの表示範囲内に `pos` が含まれるなら真を返します。そのレイヤが `exclude_id` で指定されたレイヤか、ウィンドウを持たないレイヤなら偽を返します。

`std::find_if()` に渡している開始と終了の範囲は `layer_stack_.rbegin()` と `layer_stack_.rend()` です。順序が定義されているデータ構造では、`begin()` はそのデータ構造の先頭から後ろに向かって進むイテレータを、`rbegin()` は末尾から前に向かって進むイテレータを生成します。Reverse の `r` と覚えるといいですね。 `layer_stack_` は、表示中のレイヤを下から順に格納したデータ構造ですので、逆順に辿ることで目的のレイヤを探せるというわけです。

図 10.7 ウィンドウを動かしている様子



さて実行してみます（図 10.7）。マウスでウィンドウを掴み、好きなだけぐりぐり動かすことができます。ウィンドウが画面外に飛び出しても誤動作しませんね。共通部分の計算処理がちゃんと働いている証拠です。すばらしい！

図 10.8 コンソールも動かせてしまう……



ウィンドウを動かして遊んでいたら突然コンソールがズレてしまいました！（**図 10.8**） ああ、そうか。コンソールもウィンドウの 1 種だから動かしてしまうんですね。ということは、背景画像でさえ動かせるということでしょうか……？ はい、動かせました。これはちょっと格好悪いです。本章の仕上げとして、マウスで動かせるものとそうでないものを区別する仕組みを作ることにしましょう。

10.7 ウィンドウだけドラッグ移動 (osbook_day10g)

余計なウィンドウがドラッグ操作によって移動されないようにするには、ドラッグ可能かどうかの属性をレイヤに持たせ、それを `MouseObserver()` の中でチェックすれば良いでしょう。簡単そうですね。やってみます。

リスト 10.31 ドラッグ可能を表す `draggable_` フラグを追加 (layer.hpp)

```
private:
    unsigned int id_;
    Vector2D<int> pos_{};
    std::shared_ptr<Window> window_{};
    bool draggable_{false};
```

リスト 10.31 に `Layer` クラスのメンバ変数の変更点を示します。 `draggable_` という `bool` 型のメンバ変数を追加しました。その初期値を偽としておきます。初期値は偽でも真でも良かったのですが、いまのところドラッグ不可能なレイヤの方が多いので初期値を偽とした方がコードの変更は少なくて済みます。そういえば `pos_` と `window_` の初期化をしていないことに気づいたので、念の為初期化するようにしておきました。

リスト 10.32 `draggable_` フラグを設定、取得するメソッド (layer.cpp)

```
Layer& Layer::SetDraggable(bool draggable) {
    draggable_ = draggable;
    return *this;
}

bool Layer::IsDraggable() const {
    return draggable_;
}
```

リスト 10.32 に `Layer` クラスに追加した 2 つのメソッドを示します。増設した `draggable_` に値を設定したり、取得したりするメソッドです。簡単ですね。

リスト 10.33 MouseObserver() 内でドラッグ可能フラグをチェックする (main.cpp)

```
if (!previous_left_pressed && left_pressed) {  
    auto layer = layer_manager->FindLayerByPosition(mouse_position, mouse_layer_id);  
    if (layer && layer->IsDraggable()) {  
        mouse_drag_layer_id = layer->ID();  
    }  
}
```

MouseObserver() 内では先ほど追加した `Layer::IsDraggable()` を使ってドラッグ可能性を調べます (リスト 10.33)。左ボタンを押したときに取得したレイヤがドラッグ可能である場合のみ `mouse_drag_layer_id` に値を設定します。ドラッグ不可能なレイヤ上で左ボタンを押しても `mouse_drag_layer_id` は 0 のままです。

リスト 10.34 メインウィンドウだけをドラッグ可能にする (main.cpp)

```
auto main_window_layer_id = layer_manager->NewLayer()  
    .SetWindow(main_window)  
    .SetDraggable(true)  
    .Move({300, 100})  
    .ID();
```

`Layer::SetDraggable()` メソッドを使って、メインウィンドウを含むレイヤだけをドラッグ可能に設定しました (リスト 10.34)。これで実行するとコンソールや背景レイヤはドラッグ移動できないようになっているはずです。

第 11 章

タイマとACPI

OS の基本的な機能に時間を計るというものがあります。前章までは Local APIC タイマをポーリングで読み取って高速化のための時間計測を行っていました。この章では複数のタイマを使って気軽に時間を計れる仕組みを作ろうと思います。なぜなら時間を計る処理は今後たくさん必要となるからです。

11.1 ソースコード整理 (osbook_day11a)

タイマの話をしようと思って改めて main.cpp を見るとメイン関数がごちゃごちゃしていてひどいです。1つの関数は1画面に収まるくらいに短くするのが理想なのですが、筆者のパソコンで5画面程度もある長い関数になってしまっています。また、割り込みハンドラやマウスのイベントハンドラのような、名前に interrupt.cpp や mouse.cpp にありそうな機能が main.cpp にあるのも気持ちが悪いです。ということで main.cpp の整理をしたいと思います。

リスト 11.1 KernelMainNewStack() を整理した様子 (main.cpp)

```
extern "C" void KernelMainNewStack(
    const FrameBufferConfig& frame_buffer_config_ref,
    const MemoryMap& memory_map_ref) {
    MemoryMap memory_map{memory_map_ref};

    InitializeGraphics(frame_buffer_config_ref);
    InitializeConsole();

    printk("Welcome to MikanOS!\n");
    SetLogLevel(kWarn);

    InitializeSegmentation();
    InitializePaging();
    InitializeMemoryManager(memory_map);
    ::main_queue = new std::deque<Message>(32);
    InitializeInterrupt(main_queue);

    InitializePCI();
    usb::xhci::Initialize();

    InitializeLayer();
    InitializeMainWindow();
    InitializeMouse();
    layer_manager->Draw({{0, 0}, ScreenSize()});
```

いろいろ整理をしました。メイン関数のループより前を抜き出してリスト 11.1 に示します。以前に比べて大分すっきりした見た目になりました。基本的にはメイン関数にあったコードを他のファイルの InitializeXXX() という関数に移動しただけですが、ちょっとした変更も行いました。例えば main_queue は今まで ArrayQueue という自作のキューでしたが、標準ライブラリのキューに切り替えました。ArrayQueue はメモリ管理が使える今はもはや不要なのです。

その他、変更点は大量にあるもののコードの移動が主なので全部を解説することはしません。興味のある方は付属のソースコードで確かめてみてください。

11.2 タイマ割り込み (osbook_day11b)

「第9章 重ね合わせ処理」や「第10章 ウィンドウ」では Local APIC タイマをポーリングにより読み取り、関数の実行時間を計っていました。しかし、この方法では1度に1系統の時間計測しかできず不便です。今後はコンソールに入力機能を付けて入力カーソルを点滅させるためにタイマを使いつつ、並行して関数の実行時間を計り、さらにアプリを作るようになったら指定した秒数だけスリープするようなこともしたいです。これらをやるためには複数のタイマを同時に作動させる必要があります。

Local APIC タイマは1つのCPUコアに1つしか搭載されていません。複数のタイマを同時並行で動かすにはどうすればいいのでしょうか？ 例えば、短い時間で Local APIC タイマを周期動作させて、その回数を数えて時間を計れば良さそうです。早速このアイデアを実装していくことにしましょう。

まずは Local APIC タイマを周期モードで実行させ、時間が来るたびに割り込みを発生させてみます。これはかなり簡単です。

リスト 11.2 タイマを周期モードに設定して割り込みを許可する (timer.cpp)

```
void InitializeLAPICTimer() {
    divide_config = 0b1011; // divide 1:1
    lvt_timer = (0b010 << 16) | InterruptVector::kLAPICTimer; // not-masked, periodic
    initial_count = kCountMax;
}
```

まずは **リスト 11.2** に示すように、InitializeLAPICTimer() が Local APIC タイマを周期モードに設定するようにしました。加えて、割り込みを許可し、割り込みベクタ番号は InterruptVector::kLAPICTimer を設定するようにしました。これで Current Count レジスタの値が0になるたびに指定したベクタ番号で割り込みが生成されるようになります。ここで設定している LVT Timer レジスタのフィールドの定義は **表 9.3** を参照してください。

リスト 11.3 割り込みベクタ番号の定義を追加する (interrupt.hpp)

```
class InterruptVector {
public:
    enum Number {
        kXHCI = 0x40,
        kLAPICTimer = 0x41,
    };
};
```

割り込みベクタ番号 InterruptVector::kLAPICTimer は interrupt.hpp で定義しています (**リスト 11.3**)。0x21 以上 0xff 以下で使えない値にしておく必要があります。

リスト 11.4 メイン関数で InitializeLAPICTimer() を呼び出す (main.cpp)

```
InitializeLAPICTimer();
```

改造した InitializeLAPICTimer() をメイン関数から呼び出すようにします (リスト 11.4)。

リスト 11.5 Local APIC タイマ用の割り込みハンドラを登録する (interrupt.cpp)

```
void InitializeInterrupt(std::deque<Message>* msg_queue) {
    ::msg_queue = msg_queue;

    SetIDTEntry(idt[InterruptVector::kXHCI],
                MakeIDTAttr(DescriptorType::kInterruptGate, 0),
                reinterpret_cast<uint64_t>(IntHandlerXHCI),
                kKernelCS);
    SetIDTEntry(idt[InterruptVector::kLAPICTimer],
                MakeIDTAttr(DescriptorType::kInterruptGate, 0),
                reinterpret_cast<uint64_t>(IntHandlerLAPICTimer),
                kKernelCS);
    LoadIDT(sizeof(idt) - 1, reinterpret_cast<uintptr_t>(&idt[0]));
}
```

割り込みを生成する側 (Local APIC タイマ) の設定が完了したら、割り込みを受け取る側の設定を行います。リスト 11.5 に割り込みハンドラの登録処理を示します。InitializeInterrupt() はメイン関数から呼び出され、IDT ハンドラを登録し、IDT を IDTR レジスタに登録します。引数の msg_queue にはメイン関数内で生成した main_queue が渡されてきます。

リスト 11.6 割り込みハンドラの本体 (interrupt.cpp)

```
namespace {
    std::deque<Message>* msg_queue;

    __attribute__((interrupt))
    void IntHandlerXHCI(InterruptFrame* frame) {
        msg_queue->push_back(Message{Message::kInterruptXHCI});
        NotifyEndOfInterrupt();
    }

    __attribute__((interrupt))
    void IntHandlerLAPICTimer(InterruptFrame* frame) {
        msg_queue->push_back(Message{Message::kInterruptLAPICTimer});
        NotifyEndOfInterrupt();
    }
}
```

リスト 11.6 に割り込みハンドラの定義を示します。msg_queue にはメイン関数で生成された main_queue が設定されていますので、そこに対して割り込みが発生したことを示すメッセージを追加 (push_back) します。割り込みの最後に NotifyEndOfInterrupt() するのを忘れると、2 回目以降の割り込みが届かなくなってしまいます。

リスト 11.7 メイン関数内でタイマイベントを処理する (main.cpp)

```

switch (msg.type) {
case Message::kInterruptXHCI:
    usb::xhci::ProcessEvents();
    break;
case Message::kInterruptLAPICTimer:
    printk("Timer interrupt\n");
    break;
}

```

今回はとりあえずタイマ割り込みを発生させることが目的ですから、**リスト 11.7**のように割り込みがきたことを単に印字してみます。一定周期で「Timer interrupt」と表示されれば成功です。実行してみましょう (**図 11.1**)。おお！ 起動してしばらく待つとちゃんとタイマ割り込みが発生しているようです。めでたし、めでたし。

図 11.1 タイマ割り込みが発生した様子



11.3 細かく時間を計る (osbook_day11c)

今のタイマ割り込みは筆者の環境だと数秒に1度のペースで発生するようです（機種によってはもっと速かったり遅かったりするでしょう）。今の状態でも割り込みの回数を数えることで数秒単位での時間計測ができますから、キッチンタイマみたいな用途では十分実用になるでしょう。しかし、OSの機能としてタイマを使いたいの、数秒単位では遅すぎます。1ミリ秒くらいの分解能がほしいところです。

ということで、Local APIC タイマをもっと短い周期で動作させつつ、その割り込み回数を数えることで短い時間も長い時間も計れるようにしてみようと思います。やるべきは、Initial Count レジスタに書き込む値を小さくすること、割り込み回数を数える仕組みを作ることです。さっそく実装してみます。

リスト 11.8 TimerManager クラスはタイマの割り込み回数を数える (timer.hpp)

```
class TimerManager {
public:
    void Tick();
    unsigned long CurrentTick() const { return tick_; }

private:
    volatile unsigned long tick_{0};
};

extern TimerManager* timer_manager;
```

割り込みの回数を数えるために TimerManager クラスを作ることになりました (リスト 11.8)。tick_ というメンバ変数が割り込みの回数を記憶します。Tick() メソッドは割り込み回数を 1 だけ増やし、CurrentTick() メソッドは現在の割り込み回数を返します。

tick_ に volatile 修飾子を付けることで、tick_ の読み書きに関連した最適化を抑制します。これが必要なのは、tick_ は割り込みハンドラの中で変更され、割り込みハンドラの外から参照するからです。C++ コンパイラは割り込みハンドラが実際に呼び出されることを認識しないため、最適化によって CurrentTick() が返す値が固定されてしまうことがあります。詳しくは **コラム 11.1** に書きました。

リスト 11.9 割り込みのたびに Tick() を呼び出す (timer.cpp)

```
void TimerManager::Tick() {
    ++tick_;
}

TimerManager* timer_manager;

void LAPICTimerOnInterrupt() {
    timer_manager->Tick();
}
```

リスト 11.9 は TimerManager::Tick() の実装と、それを呼び出す関数 LAPICTimerOnInterrupt() です。この関数は後でタイマ割り込みのハンドラの中から呼び出すようにする予定です。

リスト 11.10 タイマ割り込みの周期を設定する (timer.cpp)

```
void InitializeLAPICTimer() {
    timer_manager = new TimerManager;

    divide_config = 0b1011; // divide 1:1
    lvt_timer = (0b010 << 16) | InterruptVector::kLAPICTimer; // not-masked, periodic
    initial_count = 0x1000000u;
}
```

リスト 11.10 に改造した `InitializeLAPICTimer()` を示します。冒頭で `TimerManager` クラスのインスタンスを生成するようにしたこと、Initial Count レジスタに設定する値を小さくしたことが主な変更点です。

リスト 11.11 タイマ割り込みのハンドラ (`interrupt.cpp`)

```
__attribute__((interrupt))
void IntHandlerLAPICTimer(InterruptFrame* frame) {
    LAPICTimerOnInterrupt();
    NotifyEndOfInterrupt();
}
```

作成したタイマ割り込みのハンドラをリスト 11.11 に示します。やっていることは単純ですね。`LAPICTimerOnInterrupt()` を呼び出し、最後に割り込みの完了を LocalAPIC に通知するだけです。

さて、ここまで実装するとタイマ割り込みのたびに `timer_manager->Tick()` が呼び出され、`tick_` が 1 ずつ増えていく状態になります。メインループでは今まで、`hlt` せず全速力でループ回数を数えていました。それを変更し、割り込みが発生するたびに現在の `tick_` の値を表示するように改造してみます。

リスト 11.12 メインループでタイマの値を表示する (`main.cpp`)

```
__asm__("cli");
const auto tick = timer_manager->CurrentTick();
__asm__("sti");

sprintf(str, "%010lu", tick);
FillRectangle(*main_window->Writer(), {24, 28}, {8 * 10, 16}, {0xc6, 0xc6, 0xc6});
WriteString(*main_window->Writer(), {24, 28}, str, {0, 0, 0});
layer_manager->Draw(main_window_layer_id);

__asm__("cli");
if (main_queue->size() == 0) {
    __asm__("sti\n\thlt");
    continue;
}
```

リスト 11.12 にメインループの改造箇所を示します。主な変更点は `count` 変数を削除したこと、メインウィンドウに `timer_manager->CurrentTick()` の値を表示するようにしたこと、`sti` の後に `hlt` するように戻したことです。

`timer_manager->CurrentTick()` を実行する前後を `cli` と `sti` で囲んでいるのは、ここで読み取る `tick_` 変数が割り込みハンドラの中で変更される可能性があるからです。割り込みはどんなタイミングでも発生し得るため、`tick_` 変数を読み取ろうとしたまさにそのときに割り込みが発生し、`tick_` 変数を書き換えてしまう可能性があります。これは思わぬ誤動作の原因となるため避けたいです。そこで前後を `cli` と `sti` で囲むことで割り込みを発生させないようにしました。このあたり（非同期環境における同期処理）はかなり難しい話ですので本書では詳しくは扱いませんが、とにかく、割り込みハンドラで書き換えられる可能性のある変数を割り込みハンドラの外で読み出すときは、その瞬間に割り込みが発生しないように保護しておくのだな、と覚えておいてください。

図 11.2 タイマ割り込みの回数を表示する



実行してみると図 11.2 のようになりました。写真ではあまり違いが分かりませんが、筆者の QEMU 上で動かすとカウンタがゆっくり動くようになりました。そして、全速力でループを回さなくなったため CPU 使用率が大幅に下がりました。成功です！

コラム 11.1 volatile の必要性

次に示すような、ループの中で CurrentTick() を繰り返し呼んで時間経過を待つプログラムを書くとしてみます。

```
const auto begin = timer_manager->CurrentTick();
while (begin + 10 > timer_manager->CurrentTick());
```

CurrentTick() は tick_ 変数の値を読み取ります。ループに入る前に現在値を読み取っておいて、10 カウントするまで待つことを意図したプログラムです。CurrentTick() の値がだんだん増えていき、begin + 10 に到達したらループを抜けます。特に問題がないように思えます。

このループの中には Tick() の呼び出しなどがいないため、C++ コンパイラは tick_ 変数の値がループ中は不変であると判断する可能性があります。その場合、tick_ が変化しないのに CurrentTick() を毎回呼び出すのは無駄ですから、代わりに固定値を埋め込むでしょう。するとこの while ループは、実質的に無限ループになってしまいます。C++ コンパイラは現在のコンテキストの中だけでプログラムが動作すると仮定して最適化を行うため、本当はタイマ割り込みで tick_ の値が変化するという事実は無視されます。

tick_ を volatile 修飾することで、この変数は揮発性であるとコンパイラに伝えることができます。揮発性とは値がいつでも変化する可能性があるという性質のことで、ハードウェアのレジスタが代表的な揮発性変数です。コンパイラは揮発性変数の読み書きは最適化しませんので、このプログラムの while ループが無限ループになることなく、意図通りに動くようになります。

11.4 複数のタイマとタイムアウト通知 (osbook_day11d)

タイムアウトというのはタイマに設定した時間が経過することです。Local APIC タイマの Current Count レジスタが 0 になるということもタイムアウトと言えます。Local APIC タイマはタイムアウトすると割り込みによって CPU に通知します。

この節では TimerManager::tick_ を基準として動作する論理的なタイマを生成し、タイムアウトしたときに通知する機能を実装してみようと思います。Local APIC タイマは 1 カウントが何秒なのかは不明ですから秒を計るのには使えませんが、「Local APIC タイマ時間」とでもいいでしょうか、Local APIC タイマの 1 カウントを基準とした時間の計測はできます。

リスト 11.13 論理タイマを表す Timer クラス (timer.hpp)

```
class Timer {
public:
    Timer(unsigned long timeout, int value);
    unsigned long Timeout() const { return timeout_; }
    int Value() const { return value_; }

private:
```

```

    unsigned long timeout_;
    int value_;
};

```

論理的なタイマを表す Timer クラスを作成しました (**リスト 11.13**)。このクラスはタイムアウト時刻を表す timeout_ 変数とタイムアウト時に送信する値を格納する value_ 変数を持ちます。とても単純なクラスですね。タイムアウト時刻が TimerManager::tick_ を過ぎたらタイムアウトした、ということにしようと思います。

リスト 11.14 TimerManager が複数の論理タイマを持てるようにした (timer.hpp)

```

class TimerManager {
public:
    TimerManager(std::deque<Message>& msg_queue);
    void AddTimer(const Timer& timer);
    void Tick();
    unsigned long CurrentTick() const { return tick_; }

private:
    volatile unsigned long tick_{0};
    std::priority_queue<Timer> timers_{};
    std::deque<Message>& msg_queue_;
};

```

リスト 11.14 に改造した TimerManager クラスの定義を示します。複数の論理タイマを保持するための timers_ 変数と、タイマのタイムアウトを通知するためのメッセージキューを指す msg_queue_ 参照変数を作りました。std::priority_queue は初めて登場したので説明します。

std::priority_queue は優先度付きキューと呼ばれるデータ構造です。普通のキューは FIFO と呼ばれるように、最初に入れたデータが最初に取り出されるようなデータ構造でした。優先度付きキューは「キュー」と名付けられてはいますが FIFO ではありません。入れた順序は関係なく、そのときに格納されているデータの中から最も優先度の高いものを取り出されるのです。

std::priority_queue の場合、優先度は less 演算子 (<) を使って決められます。すなわち、less 演算子で比較して大きい方が優先度が高い、ということになります。例えば整数の 2 と 3 であれば 2<3 ですから 3 の優先度は 2 より高いと判断されます。timers_ の要素は Timer です。2 つのタイマ t1 と t2 があったときに t1<t2 であればタイマ t2 の優先度が高い、ということになります。ただし、標準では Timer 同士を比較できる less 演算子は提供されていませんから自分で作る必要があります。

リスト 11.15 タイマの優先度を比較するための less 演算子 (timer.hpp)

```

/** @brief タイマー優先度を比較する。タイムアウトが遠いほど優先度低。 */
inline bool operator<(const Timer& lhs, const Timer& rhs) {
    return lhs.Timeout() > rhs.Timeout();
}

```

ということで作りました (**リスト 11.15**)。この less 演算子は 2 つのタイマの優先度を比較し、優先度が lhs<rhs である場合に真を返します。ところでタイマの「優先度」とは何でしょうか？ いろいろな決め方があると思いますが、今回はタイムアウトの処理に適した優先度を考えます。

タイムアウト処理は大雑把にいうと `timers_` の中からタイムアウトしたタイマを探して `timers_` から取り除く、ということをやります。すなわち、それぞれのタイマに対し、`timeout_` 変数が `timer_manager->tick_` の値以下かどうかを調べ、そうであれば `timer_manager->timers_` から取り除くということです。タイマの中には3カウント後にタイムアウトするようなものや1万カウント後にタイムアウトするものもあるかもしれませんが、タイムアウト処理の中で注目すべきは3カウント後にタイムアウトするタイマです。ということは、タイマの優先度を「タイムアウトが近いほど高くなる」ようにすると良さそうです。優先度をそのように決めておけば、`timer_manager->timers_` からタイムアウトが近い順にタイマを取り出すことができます。それをコードで表現したものが**リスト 11.15** というわけです。

リスト 11.16 AddTimer() はタイマを優先度付きキューに挿入する (timer.cpp)

```

TimerManager::TimerManager(std::deque<Message>& msg_queue)
    : msg_queue_{msg_queue} {
    timers_.push(Timer{std::numeric_limits<unsigned long>::max(), -1});
}

void TimerManager::AddTimer(const Timer& timer) {
    timers_.push(timer);
}

```

リスト 11.16 に `TimerManager` クラスのコンストラクタと `AddTimer()` メソッドの実装を示します。コンストラクタの引数 `msg_queue` にはメイン関数で生成した `main_queue` が渡されてきます。`msg_queue_` はそのメッセージキューを指す参照となります。コンストラクタの本文では `timers_` 変数にタイマを1つ挿入しています。このタイマはタイムアウト値 (`unsigned long`) として表せる最大の値を持っており、他のタイマよりも優先度が低くなります。これは番兵と呼ばれるテクニックです。後で詳しく説明します。

`AddTimer()` は指定されたタイマを `timers_` に追加します。もちろん `timers_` は優先度付きキューですから、タイマは末尾に追加されるわけではなくタイムアウト値の小さい順に並びます。

リスト 11.17 Tick() するたびにタイムアウト処理を行う (timer.cpp)

```

void TimerManager::Tick() {
    ++tick_;
    while (true) {
        const auto& t = timers_.top();
        if (t.Timeout() > tick_) {
            break;
        }

        Message m{Message::kTimerTimeout};
        m.arg.timer.timeout = t.Timeout();
        m.arg.timer.value = t.Value();
        msg_queue_.push_back(m);

        timers_.pop();
    }
}

```

リスト 11.17 に `TimerManager::Tick()` の実装を示します。前回までは `++tick_;` するだけのメソッドでしたが、今回はタイムアウト処理を追加したため立派な見た目になりましたね。思い出すために書いておくと、この `Tick()` メソッドは Local APIC タイマの割り込みハンドラから呼ばれた `LAPICTimerOnInterrupt()` から呼ばれるのです。

`while` ループの先頭では、`timers_` から最も優先度が高いタイマを取ってきて、それを参照変数 `t` に設定します。次の行では、取ってきたタイマ `t` のタイムアウト時刻を調べ、まだタイムアウトしていなければ `while` ループを抜けます。`t` はその時点で最も優先度が高いタイマ、すなわちタイムアウトが最も近いタイマですから、そのタイマがまだタイムアウトしていないのであれば、それ以上 `timers_` の残りのタイマを調べる意味はありません。

タイマ `t` がタイムアウトしているなら `while` ループを抜けず、その次の処理に移ります。タイマのタイムアウト時刻 (`t.Timeout()`) と通知用値 (`t.Value()`) を使って通知用メッセージ `m` を作り、それをメッセージキュー `msg_queue_` に追加します。こうすることで、タイマがタイムアウトしたことをメイン関数側で感知することができるようになります。

リスト 11.18 Message 構造体の定義 (message.hpp)

```
#pragma once

struct Message {
    enum Type {
        kInterruptXHCI,
        kTimerTimeout,
    } type;

    union {
        struct {
            unsigned long timeout;
            int value;
        } timer;
    } arg;
};
```

メッセージキューの要素の型である `Message` 構造体を **リスト 11.18** のように改造しました。今までは通知メッセージの種類を表すメンバ変数 `type` しかありませんでしたが、今回はタイムアウト時刻と通知用値を表すために `arg` というメンバ変数を追加しました。この `Message` 構造体は今後いろいろな通知に使うため、`arg` は様々な種類のデータを扱える必要があります。そこで共用体として定義してみました。`type` が `kTimerTimeout` となっているメッセージでは `arg.timer` を利用するようにします。

リスト 11.19 メイン関数内でタイマを 2 つ生成する (main.cpp)

```
InitializeLAPICTimer(*main_queue);

timer_manager->AddTimer(Timer(200, 2));
timer_manager->AddTimer(Timer(600, -1));
```

タイマの実装ができたので実際にいくつかのタイマを生成して遊んでみたいと思います。**リスト 11.19**にタイマの生成と登録の例を示します。それぞれ 200 カウントと 600 カウント後にタイムアウトする 2 つのタイマを生成して timer_manager に登録します。

リスト 11.20 タイムアウト通知を表示し、タイマを追加登録する (main.cpp)

```
case Message::kTimerTimeout:
    printk("Timer: timeout = %lu, value = %d\n",
           msg.arg.timer.timeout, msg.arg.timer.value);
    if (msg.arg.timer.value > 0) {
        timer_manager->AddTimer(Timer(
            msg.arg.timer.timeout + 100, msg.arg.timer.value + 1));
    }
    break;
```

タイマがタイムアウトした際に main_queue に送られてくる通知メッセージを処理する部分を書きました (**リスト 11.20**)。タイマのタイムアウト時刻と通知用値をコンソールに表示します。情報を表示した後、通知用値が正の値であれば 100 カウント後にタイムアウトするタイマを追加登録します。こうすることで、100 カウント周期でタイムアウトするタイマを表現できます。追加登録の際に通知用値を 1 だけ増やしていることに深い意味はありません。1 ずつ値が増えたら見ていて分かりやすいかなー、という程度です。ずっと同じ値でも問題ありません。

図 11.3 2 つのタイマを動作させている様



図 11.3 に実行した様子を載せました。100 カウント周期のタイマと 600 カウントの単発タイマが動作している様子が分かります。ということで大成功ですね。

11.5 ACPI PM タイマと RSDP (osbook_day11e)

これまで扱ってきた Local APIC タイマは 1 カウントが何秒かわからないタイマです。しかも機種によって動作速度にかなりの差(筆者の環境だと、QEMU では 100 カウントが数秒、実機だと数十秒)があったりしてそのままでは扱いにくいです。timer.cpp の中で initial_count に代入する値を調整すると tick_ がインクリメントする時間を変化させることが可能ですから、なんとかしてこれを自動調整する方法を考えます。

動作周波数が分かっているタイマを使って Local APIC タイマの周期を測定すれば良さそうです。実は現代のパソコンには Local APIC タイマのほかにもいくつかのタイマが搭載されています。それらの中で動作周波数が分かるタイマを使えば、Local APIC タイマの 1 カウントが何秒なのかを計算することができます。この節では ACPI PM タイマを使って Local APIC タイマの周期を測る機能を作ります。

ACPI (Advanced Configuration and Power Interface) はコンピュータの構成や電源を管理するための規格です。現代のパソコンはこの ACPI 規格に基づいて作られていて、パソコンに搭載されている部品の構成情報をここから取得することができます。ACPI PM (Power Management) タイマはその名の通り ACPI の電源管理のために搭載されるタイマですが、電源管理以外の目的でも使うことができます。筆者の経験上 ACPI PM タイマはほぼすべての機種に搭載されています。ACPI PM タイマが搭載されていない機種を使っている場合は以降で紹介するプログラムは動作しません。ごめんなさい……。

さて、ACPI PM タイマを使うには何が必要かというと、タイマを制御するためのレジスタ群の IO ポート番号とレジスタの中身についての知識です。まずは IO ポート番号を得ることを目指します。ACPI PM タイマのレジスタ群のポート番号はメモリアドレス空間にある **FADT** というテーブルに記載されています。この FADT の場所 (メモリアドレス) を知るには **XSDT** (Extended System Descriptor Table) というこれまたメモリアドレス空間にあるテーブルを調べる必要があります。そして、XSDT の場所は **RSDP** (Root System Description Pointer) に記載されています。ですので、まずは RSDP を取得することから始めます。

RSDP はポインタという名前が付いていますが、C++ のポインタのような単純なものではなく構造体になっています。RSDP の構造体もメモリアドレス空間に置かれていて、その場所は UEFI BIOS が知っています。そこで、ブートローダを改造して UEFI BIOS からその情報を取得し、OS 側に渡すようにします。

リスト 11.21 UEFI のメイン関数

```
EFI_STATUS EFIAPI UefiMain(
    EFI_HANDLE image_handle,
    EFI_SYSTEM_TABLE* system_table) {
    ...
}
```

ブートローダのメイン関数 UefiMain() の定義は **リスト 11.21** のようになっています。見ての通り、メイン関数の第 2 引数には EFI_SYSTEM_TABLE 構造体へのポインタが渡されます。今までは system_table をまったく使ってきませんでしたが、RSDP の場所を知るには system_table を調べる必要があります。EFI_SYSTEM_TABLE 構造体のメンバ変数の中で RSDP の取得に必要なものを **表 11.1** に示します。

表 11.1 EFI_SYSTEM_TABLE 構造体のメンバ変数 (抜粋)

メンバ名	型
NumberOfTableEntries	UINTN
ConfigurationTable	EFI_CONFIGURATION_TABLE*

ConfigurationTable は構成情報の配列を指すポインタで、配列の要素数は NumberOfTableEntries で表されます。配列の要素は EFI_CONFIGURATION_TABLE 構造体で、**表 11.2** に示すメンバを持ちます。EFI_SYSTEM_TABLE 構造体や EFI_CONFIGURATION_TABLE 構造体の定義は UEFI の仕様書 ([参考文献 \[2\]](#)) に書いてあります。

表 11.2 EFI_CONFIGURATION_TABLE 構造体のメンバ

メンバ名	型	説明
VendorGuid	EFI_GUID	システム設定テーブルを識別する GUID。
VendorTable	VOID*	システム設定テーブルへのポインタ。

このテーブルから VendorGuid が ACPI を表す要素を探します。そのときの VendorTable が目的の値、すなわち、RSDP 構造体へのポインタです。

リスト 11.22 RSDP 構造体へのポインタを取得する UEFI プログラム (Main.c)

```

VOID* acpi_table = NULL;
for (UINTN i = 0; i < system_table->NumberOfTableEntries; ++i) {
    if (CompareGuid(&gEfiAcpiTableGuid,
                    &system_table->ConfigurationTable[i].VendorGuid)) {
        acpi_table = system_table->ConfigurationTable[i].VendorTable;
        break;
    }
}

typedef void EntryPointType(const struct FrameBufferConfig*,
                           const struct MemoryMap*,
                           const VOID*);
EntryPointType* entry_point = (EntryPointType*)entry_addr;
entry_point(&config, &memmap, acpi_table);

```

ということで RSDP へのポインタを取得するプログラムは **リスト 11.22** のようになります。

プログラム中で使っている CompareGuid() は 2 つの GUID が等しいときに真を返す関数で、<Library/BaseMemoryLib.h> をインクルードすることで使えます。比較対象の gEfiAcpiTableGuid は EDK II により定義された GUID で、ACPI 2.0 以上を表す定数です。ループを抜けると変数 acpi_table に RSDP へのポインタが設定された状態になっているはずです。これをメイン関数に渡せるように引数に const VOID* を追加しました。

リスト 11.23 gEfiAcpiTableGuid を Guids リストに追加する (Loader.inf)

```
[Guids]
gEfiFileInfoGuid
gEfiAcpiTableGuid
```

gEfiAcpiTableGuid を使うには**リスト 11.23**のように Loader.inf の Guids にその GUID を追加する必要があります。

リスト 11.24 メイン関数の引数で RSDP への参照を受け取る (main.cpp)

```
extern "C" void KernelMainNewStack(
    const FrameBufferConfig& frame_buffer_config_ref,
    const MemoryMap& memory_map_ref,
    const acpi::RSDP& acpi_table) {
```

ブートローダの方でメイン関数の引数を追加しましたので本体側も同様に変更します(**リスト 11.24**)。ブートローダ側では const VOID* としていましたが、本体側では const acpi::RSDP& としました。型が違うのは C++ プログラムとしては行儀が悪いのですが、結局はメモリアドレスが来るだけなので動きはします。acpi::RSDP は acpi.hpp で新たに定義した構造体です。後で紹介します。

リスト 11.25 ACPI ライブラリの初期化関数を呼び出す (main.cpp)

```
acpi::Initialize(acpi_table);
InitializeLAPICTimer(*main_queue);
```

acpi::RSDP 構造体を受け取っていろいろな処理を行う acpi::Initialize() をメイン関数から呼び出すようにしました(**リスト 11.25**)。

リスト 11.26 ACPI の初期化関数では RSDP の整合性チェックを行う (acpi.cpp)

```
void Initialize(const RSDP& rsdp) {
    if (!rsdp.IsValid()) {
        Log(kError, "RSDP is not valid\n");
        exit(1);
    }
}
```

acpi::Initialize() の実装を**リスト 11.26**に示します。rsdp.IsValid() は引数で指定された RSDP 構造体が正しいかどうかを検証するメソッドです。つまり acpi::Initialize() は現時点ではエラーチェックだけしかやっておらずなんとも無駄な関数ですが、これから拡充していく予定です。

リスト 11.27 RSDP のデータ構造を表す RSDP 構造体の定義 (acpi.hpp)

```

/**
 * @file acpi.hpp
 *
 * ACPI テーブル定義や操作用プログラムを集めたファイル。
 */

#pragma once

#include <cstdint>

namespace acpi {

struct RSDP {
    char signature[8];
    uint8_t checksum;
    char oem_id[6];
    uint8_t revision;
    uint32_t rsdt_address;
    uint32_t length;
    uint64_t xsdt_address;
    uint8_t extended_checksum;
    char reserved[3];

    bool IsValid() const;
} __attribute__((packed));

void Initialize(const RSDP& rsdp);

} // namespace acpi

```

リスト 11.27 は RSDP 構造体の定義を含む `acpi.hpp` ファイルの内容です。この構造体のメンバ変数の意味は ACPI の仕様（参考文献 [12]）で決められていて、表 11.3 のようになっています。

表 11.3 RSDP 構造体のメンバ変数の意味

メンバ名	型	offset	説明
signature	char[8]	0	RSDP のシグネチャ。「RSD 〇 PTR 〇」の 8 文字。
checksum	uint8_t	8	前半 20 バイトのためのチェックサム。
oem_id	char[6]	9	OEM の名前。
revision	uint8_t	15	RSDP 構造体のバージョン番号。ACPI 1.0 の場合は 0。6.2 では 2。
rsdt_address	uint32_t	16	RSDT を指す 32 ビットの物理アドレス。
length	uint32_t	20	RSDP 全体のバイト数。
xsdt_address	uint64_t	24	XSDT を指す 64 ビットの物理アドレス。
extended_checksum	uint8_t	32	拡張領域を含めた RSDP 全体のチェックサム。
reserved	char[3]	33	予約領域。

RSDP の構造が壊れていないことを確かめるには、signature が正しい値であること、checksum と extended_checksum を用いたチェックサム計算が正しいことを調べる必要があります。チェックサムとは誤り検出の方法の 1 つで、データを一定のビット数で区切ってすべて足し合わせ、0 になることを確認する方法です。RSDP の場合は 1 バイトずつの合計値が 0 になることを調べます。

リスト 11.28 IsValid() は RSDP の整合性を検査する (acpi.cpp)

```
bool RSDP::IsValid() const {
    if (strcmp(this->signature, "RSD PTR ", 8) != 0) {
        Log(kDebug, "invalid signature: %.8s\n", this->signature);
        return false;
    }
    if (this->revision != 2) {
        Log(kDebug, "ACPI revision must be 2: %d\n", this->revision);
        return false;
    }
    if (auto sum = SumBytes(this, 20); sum != 0) {
        Log(kDebug, "sum of 20 bytes must be 0: %d\n", sum);
        return false;
    }
    if (auto sum = SumBytes(this, 36); sum != 0) {
        Log(kDebug, "sum of 36 bytes must be 0: %d\n", sum);
        return false;
    }
    return true;
}
```

リスト 11.28 に RSDP::IsValid() の実装を示します。まずはシグネチャが正しい値であることを確認します。signature は NUL 文字で終端されていませので strcmp で比較してはいけません。かならず文字数が指定できる比較関数を使ってください。

シグネチャの次はリビジョンの検査です。リビジョンが 0 の場合は ACPI 1.0 で決められた RSDP 構造体であることを意味しており、前半 20 バイトだけ存在します。リビジョンが 2 (執筆時点での最新) の場合は 36 バイトの大きさの構造体であることを意味します。最終的に欲しい値は XSDT のアドレス (xsdt_address) ですから、これを読むためにはリビジョン 2 でなければならないのです。

次に前半 20 バイトおよび 36 バイトのチェックサムを計算します。前半 20 バイトのチェックサムの計算には checksum を使います。使いますといっても明示的に何かするわけではなく、総和に含めるというだけのことです。SumBytes() はリスト 11.29 で定義しています。この関数は与えられたメモリ領域を 1 バイトずつ足し合わせます。20 バイトの領域を 1 バイトずつ足すと結果は 1 バイトに収まらずオーバーフローするわけですが、それでいいのです。チェックサムの計算は 1 バイトずつ足し合わせるたびに下位 1 バイトだけを残すというやり方なのです。前半 20 バイトの総和を取ると、自動的に checksum が計算に含まれることになり、結果として正しくチェックサムを計算できます。

リスト 11.29 SumBytes() は与えられたメモリ領域をバイト単位で総和する (acpi.cpp)

```

namespace {

template <typename T>
uint8_t SumBytes(const T* data, size_t bytes) {
    return SumBytes(reinterpret_cast<const uint8_t*>(data), bytes);
}

template <>
uint8_t SumBytes<uint8_t>(const uint8_t* data, size_t bytes) {
    uint8_t sum = 0;
    for (size_t i = 0; i < bytes; ++i) {
        sum += data[i];
    }
    return sum;
}

} // namespace

```

リスト 11.29 では C++ のテンプレートの機能であるテンプレートの特殊化を用いています。実現したいのは「バイト配列とそのサイズを取って、バイト単位の総和を計算する」という機能です。ですからバイト配列とサイズを取る関数を定義すればいいのですが、その関数を使ってバイト配列以外の変数、今回なら RSDP 構造体の総和を取るためにはキャストが必要になってしまいます。テンプレートの特殊化を使うとキャストを関数のユーザーから隠蔽できます。

2 つの SumBytes() の定義のうち前者は一般的な定義で、テンプレート引数 T によりどんな型にも対応します。一方で後者は uint8_t に特殊化された定義です。前者において T が uint8_t である場合に、前者の定義の代わりに後者の定義が使われます。言い換えると、uint8_t 以外の型が来たら uint8_t にキャストして特殊化された定義を使うようにする、という仕組みになりますね。この仕組みのおかげで、関数を使う側ではキャストすることなく SumBytes(&rsdp, 20) と呼び出してチェックサムを計算できます。素晴らしい。

閑話休題。前半 20 バイトのチェックサム計算まで説明しました。次は RSDP 全体、36 バイトのチェックサムを計算します。このチェックサムの計算には extended_checksum が関係します。といってもやはり総和に含めるというだけのことなのですが、36 バイトの総和が 0 であればチェックサムが正しいということになります。

ここまでの検査にすべて合格すれば RSDP のデータ構造が壊れていないと思って良いでしょう。検査に合格したということで、IsValid() は真を返します。

これで RSDP の取得は完了です。次は XSDT の取得をやりたいのですが、だいぶ疲れたので次章に回すことにします。

第 12 章

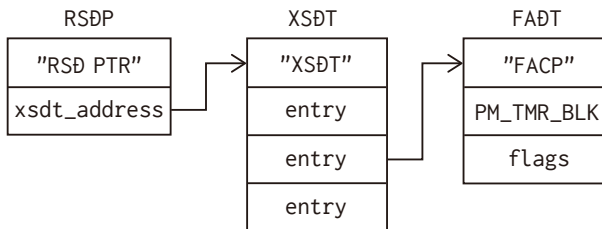
キー入力

前章ではマウスとタイマが使えるようになって OS っぽさが出てきました。次に何をしたら OS っぽさが増すかなと考えて、この章ではキーボード入力をできるようにしようと思います。実はキーボードもマウスと同じく USB HID という種類の機器ですので、ほとんど同じように制御することができます。この章を終えると、キーボードから文字列を入力してテキストボックスに表示できるようになります。

12.1 FADT を探す (osbook_day12a)

USB キーボードに取り掛かる前に、前章でやり残した作業をやってしまいましょう。ACPI PM タイマを使って Local APIC タイマの 1 カウントの秒数を計る作業です。そのためには FADT が必要で、その場所を知るために XSDT が必要で、XSDT の場所を知るために RSDP を取得したのが前章の最後の仕事でしたね。これらの関係を図にすると図 12.1 のようになっています。

図 12.1 RSDP、XSDT、FADT の関連



XSDT は ACPI の中核となるデータ構造ですので、まずは XSDT を表す構造体を定義しようと思います。

リスト 12.1 記述ヘッダと XSDT を表す構造体の定義 (acpi.hpp)

```

struct DescriptionHeader {
    char signature[4];
    uint32_t length;
    uint8_t revision;
    uint8_t checksum;
    char oem_id[6];
    char oem_table_id[8];
    uint32_t oem_revision;
    uint32_t creator_id;
    uint32_t creator_revision;

    bool IsValid(const char* expected_signature) const;
} __attribute__((packed));

struct XSDT {
    DescriptionHeader header;

    const DescriptionHeader& operator[](size_t i) const;
    size_t Count() const;
} __attribute__((packed));
  
```

リスト 12.1 に `acpi::XSDT` 構造体と、その中で使っている `acpi::DescriptionHeader` 構造体の定義を示します。ACPI に登場する構造体で特徴的なのは、XSDT それ自身や XSDT から指される他のデータ構造の先頭に共通のヘッダである**記述ヘッダ**が付いていることです。DescriptionHeader はそのヘッダを表す構造体です。XSDT は共通の記述ヘッダの後に各データ構造へのアドレスが並ぶ構造になっています。

リスト 12.2 記述ヘッダの IsValid() はデータ構造全体の整合性を調べる (acpi.cpp)

```
bool DescriptionHeader::IsValid(const char* expected_signature) const {
    if (strcmp(this->signature, expected_signature, 4) != 0) {
        Log(kDebug, "invalid signature: %.4s\n", this->signature);
        return false;
    }
    if (auto sum = SumBytes(this, this->length); sum != 0) {
        Log(kDebug, "sum of %u bytes must be 0: %d\n", this->length, sum);
        return false;
    }
    return true;
}
```

リスト 12.2 に記述ヘッダを用いてそのデータ構造の整合性を確認する IsValid() メソッドを示します。記述ヘッダにある 4 バイトのシグネチャを確認し、チェックサムを計算します。記述ヘッダのチェックサムは、記述ヘッダだけではなくそのデータ構造全体（length に書かれた大きさ）について和を計算することになっています。

リスト 12.3 XSDT のエントリにアクセスする添字演算子 (acpi.cpp)

```
const DescriptionHeader& XSDT::operator[](size_t i) const {
    auto entries = reinterpret_cast<const uint64_t*>(&this->header + 1);
    return *reinterpret_cast<const DescriptionHeader*>(entries[i]);
}

size_t XSDT::Count() const {
    return (this->header.length - sizeof(DescriptionHeader)) / sizeof(uint64_t);
}
```

XSDT は記述ヘッダの後に各データ構造へのアドレスが並ぶ構造になっています。それらのアドレスを得るために**リスト 12.3** のような添字演算子（operator[]）を定義しました。添字演算子では [] の中に指定された数値が引数 i として渡ってきますから、XSDT の中の指定された添字のアドレスを取得し、そのアドレスを const DescriptionHeader& 型に変換して返します。Count() は XSDT が保持するデータ構造のアドレスの個数を表します。

リスト 12.4 XSDT が持つアドレス配列から FADT を検索する (acpi.cpp)

```
void Initialize(const RSDP& rsdp) {
    if (!rsdp.IsValid()) {
        Log(kError, "RSDP is not valid\n");
        exit(1);
    }

    const XSDT& xsdt = *reinterpret_cast<const XSDT*>(rsdp.xsdt_address);
    if (!xsdt.header.IsValid("XSDT")) {
        Log(kError, "XSDT is not valid\n");
        exit(1);
    }

    fadt = nullptr;
}
```

```

for (int i = 0; i < xsdt.Count(); ++i) {
    const auto& entry = xsdt[i];
    if (entry.IsValid("FACP")) { // FACP is the signature of FADT
        fadt = reinterpret_cast<const FADT*>(&entry);
        break;
    }
}

if (fadt == nullptr) {
    Log(kError, "FADT is not found\n");
    exit(1);
}
}

```

先ほど作った `XSdt::operator[]()` や `XSdt::Count()` を使って FADT を探すプログラムを **リスト 12.4** に示します。XSDT のデータ構造は `rsdp.xsdt_address` に書いてある物理アドレスに置かれています。そのアドレスを XSdt 構造体のポインタに変換して使います。参照変数 `xsdt` はメモリ上にある XSdt データ構造を指し示すことになります。

for 文で FADT を検索します。XSdt の記述ヘッダの後ろに並んでいるアドレス配列を先頭からたどり、シグネチャが「FACP」となっているものを探します。歴史的な事情で FADT のシグネチャはデータ構造の名前と異なっています。「FACP」は間違いではないのです。

`xsdt[i]` とすると、XSdt の記述ヘッダの後ろに並んでいるアドレスの *i* 番目を取得できます。といってもアドレスを表す整数値ではなく `DescriptionHeader` 構造体への参照が返されます。それを `entry` 参照変数で受け取って使います。無事に FADT を見つけたら FADT へのポインタを `acpi::fadt` というグローバル変数に設定して for ループを抜けます。

リスト 12.5 FADT を表す構造体の定義 (acpi.hpp)

```

struct FADT {
    DescriptionHeader header;

    char reserved1[76 - sizeof(header)];
    uint32_t pm_tmr_blk;
    char reserved2[112 - 80];
    uint32_t flags;
    char reserved3[276 - 116];
} __attribute__((packed));

extern const FADT* fadt;

```

FADT を表す FADT 構造体の定義を **リスト 12.5** に示します。FADT は全体で 276 バイトもある大きな構造体ですが、ACPI PM タイマの制御に必要なメンバ変数は 2 つしかありません。不要な部分は `reserved1` のようにして飛ばすことで定義を簡略化しました。

12.2 ACPI PM タイマを使う (osbook_day12b)

FADT が取得できたら ACPI PM タイマを使う準備は整っています。この節では ACPI PM タイマを使って Local APIC タイマの 1 カウントが何秒なのかを計ります。

ACPI PM タイマは 3.579545MHz で常に動き続けているタイマです。カウンタは 24 ビット、環境によっては 32 ビットあります。24 ビットカウンタなら約 4.7 秒で 1 周して 0 に戻ります。このタイマを使ってどうやって Local APIC タイマの 1 カウントを計りましょうか？ 筆者が考えた方法は、ACPI PM タイマが 1 秒進むのを待って、その前後で Local APIC タイマのカウンタを読む方法です。これで、実世界の 1 秒と Local APIC タイマの周期の対応関係が分かります。1 秒待つ処理や Local APIC タイマのカウンタを読む処理にわずかに時間がかかるため少し誤差が出ますが、それは我慢することにしましよう。

リスト 12.6 WaitMilliseconds() は指定したミリ秒が経過するのを待つ (acpi.cpp)

```
void WaitMilliseconds(unsigned long msec) {
    const bool pm_timer_32 = (fadt->flags >> 8) & 1;
    const uint32_t start = IoIn32(fadt->pm_tmr_blk);
    uint32_t end = start + kPMTimerFreq * msec / 1000;
    if (!pm_timer_32) {
        end &= 0x00ffffffu;
    }

    if (end < start) { // overflow
        while (IoIn32(fadt->pm_tmr_blk) >= start);
    }
    while (IoIn32(fadt->pm_tmr_blk) < end);
}
```

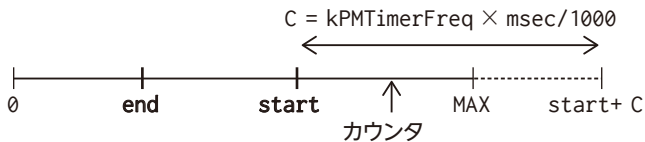
リスト 12.6 のように指定したミリ秒だけ待つ WaitMilliseconds() を実装しました。1 行目で定義している変数 pm_timer_32 は fadt->flags のビット 8 を取り出したもので、ACPI PM タイマのカウンタが 32 ビット幅の場合に真、24 ビット幅の場合に偽になります。

ACPI PM タイマは特に何もせずとも常にカウントを続けています。カウンタは fadt->pm_tmr_blk が表す IO ポートに存在しますから、IoIn32(fadt->pm_tmr_blk) とすればその時点のカウント値を読み取れます。関数の 2 行目でその時のカウント値を読んで start 変数に記憶しておき、その値を使って指定されたミリ秒後のカウント値を計算して end 変数に設定します。kPMTimerFreq は acpi.hpp で 3579545 という値の定数として定義しました。

4 行目の if 文でカウンタのビット幅を調べ、24 ビットカウンタの場合 (pm_timer_32 が偽の場合) に end の値を 24 ビットに制限します。この処理をすることで、カウンタの幅が 24 ビットでも 32 ビットでも後の処理を共通化できます。

後半の if 文はほとんどの場合に実行されず、そのまま最後の while 文が実行されます。この while 文は ACPI PM タイマが指定したミリ秒だけ経過するのを待ちます。タイマのカウント値が end になるまで待つというとても単純な処理ですね。

図 12.2 end がオーバーフローした様子



では後半の if 文はどんなときに実行されるかというと、end がオーバーフローして start より小さくなった場合です(図 12.2)。このとき、すぐに最後の while 文を実行してしまうとまったく待たずにループを抜けてしまいます。そこで、カウンタがオーバーフローして 0 に戻るまで待つということを if の中にある while 文でやっているわけです。

リスト 12.7 Local APIC タイマの周波数と論理タイマの周波数 (timer.hpp)

```
extern TimerManager* timer_manager;
extern unsigned long lapic_timer_freq;
const int kTimerFreq = 100;
```

WaitMilliseconds() を使って Local APIC タイマの 1 カウントの時間を計り、その結果を記憶しておくグローバル変数を定義したいです。ただ、1 カウントあたりの秒数は非常に小さい値になってしまうため整数変数では扱いづらいです。ですので、代わりに 1 秒あたりのカウント数(タイマの周波数)を記憶しておくことにします。これならかなり大きな値になるため、整数変数でも高い精度での表現が可能です。ということで lapic_timer_freq というグローバル変数を用意しました(リスト 12.7)。

その下で定義している定数 kTimerFreq は、1 秒間に TimerManager::Tick() が呼ばれる回数を表します。100 としておくと 1 秒間に 100 回、すなわち 10 ミリ秒毎に tick_ がインクリメントされていきます。これだけの精度があればとりあえず十分でしょう。kTimerFreq を増やすほど細かく時間を計れるようになりますが、それだけタイムアウト処理にかかる時間が増え、OS 全体の処理性能が落ちてしまいます。バランスが大事です。

リスト 12.8 LAPIC タイマの周波数を測定する (timer.cpp)

```
void InitializeLAPICTimer(std::deque<Message>& msg_queue) {
    timer_manager = new TimerManager{msg_queue};

    divide_config = 0b1011; // divide 1:1
    lvt_timer = 0b001 << 16; // masked, one-shot

    StartLAPICTimer();
    acpi::WaitMilliseconds(100);
    const auto elapsed = LAPICTimerElapsed();
    StopLAPICTimer();

    lapic_timer_freq = static_cast<unsigned long>(elapsed) * 10;

    divide_config = 0b1011; // divide 1:1
    lvt_timer = (0b010 << 16) | InterruptVector::kLAPICTimer; // not-masked, periodic
    initial_count = lapic_timer_freq / kTimerFreq;
}
```

リスト 12.8 に `WaitMilliseconds()` を使って Local APIC タイマの周波数を計るプログラムを示します。100 ミリ秒 (0.1 秒) だけ待ち、その前後での Local APIC タイマのカウンタの変化量を調べます。その変化量を 10 倍すると Local APIC タイマの 1 秒あたりのカウント、すなわち周波数を得られます。

始めは 1 秒間の計測をしようと思っていましたが、ちょっと待ちすぎかなと思い 0.1 秒に短縮しました。OS の起動処理が 0.1 秒伸びるくらいなら許容範囲でしょう。

計った周波数を元に `initial_count` 変数の値を計算します。以降はここに設定した時間間隔で割り込みが発生することになります。どんな機種で実行しても早すぎず遅すぎず、約 10 ミリ秒毎に割り込みが発生するようになるはずです。

図 12.3 ちゃんと 1 秒ごと論理タイマのタイムアウトが発生している



ビルドして起動してみると図 12.3 のようになりました。今回も写真だと伝わりませんが、100 カウントでタイムアウトするようにした周期タイマが、ちゃんと 1 秒間隔でタイムアウトしているようです。QEMU でも実機でもタイムアウト間隔が一定です。大成功ですね！

12.3 USB キーボードドライバ (osbook_day12c)

ここまでタイマを作り込んできました。一段落しましたので、ここからは今日の本題であるキー入力に取り掛かろうと思います。キーボードを叩いたら文字やアルファベットや記号類が入力される、という状態を目指します。

本書ではキー入力の装置として USB キーボードを対象にします。キーボードの接続の種類はいくつかありますが、最も入手しやすいのが USB キーボードでしょう。USB キーボードはマウスと同じ HID というクラスの機器でして、マウスとほぼ同様に制御できます。筆者が作った USB ドライバに実はキーボード用のドライバも含まれているため、簡単にキーボード対応ができます (自分でキーボードドライ

バを作る方法を勉強できると思った皆さん、ごめんなさい……)。

リスト 12.9 キー押下ハンドラを USB ドライバに登録する (keyboard.cpp)

```
void InitializeKeyboard(std::deque<Message>& msg_queue) {
    usb::HIDKeyboardDriver::default_observer =
        [&msg_queue](uint8_t keycode) {
            Message msg{Message::kKeyPush};
            msg.arg.keyboard.keycode = keycode;
            msg.arg.keyboard.ascii = keycode_map[keycode];
            msg_queue.push_back(msg);
        };
}
```

リスト 12.9 にキー押下イベントを処理するハンドラをラムダ式として作成して USB ドライバに登録する関数 `InitializeKeyboard()` の定義を示します。このハンドラ（ラムダ式）はいずれかのキーが押されるたびに呼び出されます。押されたキーのキーコードが引数 `keycode` に渡されてきますので、その値と、それを ASCII コードに変換した値を `msg_queue` に登録します。

キーコードとは各キーに割り当てられた数値のことです。アルファベットや記号類のキーにはもちろん、ESC や F1、バックスペース、エンターキーなどにもキーコードが割り当てられていて、どのキーが押されたかを判別するのに使えます。各キーのキーコードは HID Usage Table（参考文献 [13]）という規格の「10 Keyboard/Keypad Page (0x07)」に掲載されています。

リスト 12.10 キーコードを ASCII コードに変換する表 (keyboard.cpp)

```
const char keycode_map[256] = {
    0, 0, 0, 0, 'a', 'b', 'c', 'd', // 0
    'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', // 8
    'm', 'n', 'o', 'p', 'q', 'r', 's', 't', // 16
    'u', 'v', 'w', 'x', 'y', 'z', '1', '2', // 24
    '3', '4', '5', '6', '7', '8', '9', '0', // 32
    '\n', '\b', 0x08, '\t', ' ', '-', '=', '[', // 40
    ']', '\\', '#', ';', '\'', '`', '.', // 48
    '/', 0, 0, 0, 0, 0, 0, 0, // 56
    0, 0, 0, 0, 0, 0, 0, 0, // 64
    0, 0, 0, 0, 0, 0, 0, 0, // 72
    0, 0, 0, 0, '/', '*', '-', '+', // 80
    '\n', '1', '2', '3', '4', '5', '6', '7', // 88
    '8', '9', '0', '.', '\\', 0, 0, '=', // 96
};
```

キーコードから英数字や記号に変換するためにはキーコードを ASCII コードに変換する必要があります。**リスト 12.10** に変換表の定義を示します。このような変換表のことを一般にマップと呼びますから、キーコードを ASCII にマップするという意味でキーコードマップ、と名付けました。配列 `keycode_map` の添字としてキーコードを与えると対応する ASCII コードが得られます。例えば、キーコード 4 に対応する ASCII コードは `keycode_map[4]` で、それは英字「a」の ASCII コードに対応します。

このような変換が必要な理由の 1 つとしては、同じ文字が印字されたキーでも Shift キーを押してい

るかどうかで入力文字が変わるという事情があります。例えば「a」のキーを Shift を押しながら叩くと「A」が入力されるようにしたいのですが、「a」キー自体は Shift に関係なく同一キーなのでキーコードは変わりません。Shift の状態を考慮してキーボード側で ASCII に変換して送ってほしいなと思うかもしれませんが、もろもろの理由により（気になる人は考えてみてください）、キーボードは ASCII ではなくキーコードを送ってくるので、頑張って変換するしかありません。

リスト 12.11 メイン関数から初期化関数を呼ぶ (main.cpp)

```
InitializeKeyboard(*main_queue);
```

メイン関数のループに入る前に初期化関数を呼び出すようにします（リスト 12.11）。キー押下イベントハンドラが USB キーボードドライバに登録されます。

リスト 12.12 Message 構造体にキー押下イベント用のフィールドを追加 (message.hpp)

```
#pragma once

struct Message {
    enum Type {
        kInterruptXHCI,
        kTimerTimeout,
        kKeyPush,
    } type;

    union {
        struct {
            unsigned long timeout;
            int value;
        } timer;

        struct {
            uint8_t keycode;
            char ascii;
        } keyboard;
    } arg;
};
```

キーの押下イベントをメイン関数に通知するために Message 構造体を改造しました（リスト 12.12）。type の種類としてキー押下イベントを表す kKeyPush を追加し、arg に keyboard を追加しました。キーが押された時点のキーコードと、それを ASCII に変換した値（変換できなければ 0）を設定できます。ちなみに arg.timer と arg.keyboard は共用体の中に入っているためメモリ領域を共有します。したがって Message 構造体のサイズが増えることはありません。

リスト 12.13 メイン関数でキー押下イベントに反応する (main.cpp)

```
case Message::kKeyPush:
    if (msg.arg.keyboard.ascii != 0) {
        printk("%c", msg.arg.keyboard.ascii);
    }
    break;
```

メイン関数のループの中にキー押下イベントに反応する case を追加します(リスト 12.13)。USB キーボードのキーを押すたびにこの case が実行され、コンソールに入力されたキーに対応する文字が表示されるはずです。実行してみましょう(図 12.4)。ああ、そういえば、タイマはもう十分だと思いまして、メイン関数の中でタイマを生成したりタイムアウトしたりする処理は削除しておきましたよ。

図 12.4 キーボードからの入力を試しているところ



最後にピクリマーク「!!」を入力しようとしたのですが、まだ Shift キーに対応していないため「11」と入力されてしまいました。次節ではこれを直してみようと思います。

12.4 モディファイアキー (osbook_day12d)

現状では、キーが押されたときに呼ばれるイベントハンドラ (InitializeKeyboard()) の中で作っているラムダ式) が 1 つのキーコードしか受け取れないため、モディファイアキーの状態を知ることができません。どうやってモディファイアキーの押下状態を得ましょうか？ 筆者はイベントハンドラに 1 つ引数を追加して、その時点のモディファイアキーの状態も送ればいかなと考えました。モディファイアキー単体を押してもイベントを通知せず、モディファイアキー以外のキーを押したときにモディ

ファイアキーの状態をセットにして通知する、ということです。

リスト 12.14 キー押下イベントハンドラに引数を追加する (keyboard.cpp)

```
void InitializeKeyboard(std::deque<Message>& msg_queue) {
    usb::HIDKeyboardDriver::default_observer =
        [&msg_queue](uint8_t modifier, uint8_t keycode) {
            const bool shift = (modifier & (kLShiftBitMask | kRShiftBitMask)) != 0;
            char ascii = keycode_map[keycode];
            if (shift) {
                ascii = keycode_map_shifted[keycode];
            }
            Message msg{Message::kKeyPush};
            msg.arg.keyboard.modifier = modifier;
            msg.arg.keyboard.keycode = keycode;
            msg.arg.keyboard.ascii = ascii;
            msg_queue.push_back(msg);
        };
}
```

リスト 12.14 に改造したイベントハンドラを示します。modifier という引数を追加して、その引数から Shift キーの押下状態を判定してキーコードマップを切り替えるようにしたのが主な変更点です。keycode は 1 つのキーのキーコードが入ってきますが、modifier は 8 つのモディファイアキーの押下状態をビット毎に表現しています。どのビットがどのキーかは HID 規格（参考文献 [14]）の「8.3 Report Format for Array Items」に書いてあり、リスト 12.15 のようになっています。

リスト 12.15 モディファイアキーのビット位置の定義 (keyboard.cpp)

```
const int kLControlBitMask = 0b00000001u;
const int kLShiftBitMask   = 0b00000010u;
const int kLAltBitMask     = 0b00000100u;
const int kLGUIBitMask     = 0b00001000u;
const int kRControlBitMask = 0b00010000u;
const int kRShiftBitMask   = 0b00100000u;
const int kRAltBitMask     = 0b01000000u;
const int kRGUIBitMask     = 0b10000000u;
```

モディファイアキーはいくつも組み合わせて押されることがあります。例えば左 Control と右 Shift を同時に押した状態で「a」キーを押下すると modifier は 0x41、keycode は 4 となります。今回は Shift を押した状態でキーコードのマップを切り替えたかったため、左 Shift または右 Shift のどちらかが押されていたら keycode_map_shifted（リスト 12.16）を使うというプログラムにしました。

リスト 12.16 Shift を押した状態のキーコードマップ (keyboard.cpp)

```
const char keycode_map_shifted[256] = {
    0,    0,    0,    0,    'A',  'B',  'C',  'D', // 0
    'E',  'F',  'G',  'H',  'I',  'J',  'K',  'L', // 8
    'M',  'N',  'O',  'P',  'Q',  'R',  'S',  'T', // 16
    'U',  'V',  'W',  'X',  'Y',  'Z',  '!',  '@', // 24
    '#',  '$',  '%',  '^',  '&',  '*',  '(',  ')', // 32
    '\n',  '\b',  0x08,  '\t',  ' ',  '-',  '+',  '[', // 40
    '}',  '|',  '~',  ':',  '"',  '~',  '<',  '>', // 48
    '?',  0,    0,    0,    0,    0,    0,    0, // 56
    0,    0,    0,    0,    0,    0,    0,    0, // 64
    0,    0,    0,    0,    0,    0,    0,    0, // 72
    0,    0,    0,    0,    '/',  '*',  '-',  '+', // 80
    '\n',  '1',  '2',  '3',  '4',  '5',  '6',  '7', // 88
    '8',  '9',  '0',  '.',  '\\',  0,    0,    '=', // 96
};
```

さて、イベントハンドラがモディファイアキーを受け取るように改造しましたので、それに合わせて USB ドライバ側の改造も必要です。モディファイアキーを取得してイベントハンドラに渡すようにしなければなりません。そのために、まずは USB キーボードがどのような形式でデータを送ってくるのかを知ることにしましょう。

キーボードやマウスなどの HID クラスに属する機器は、一般には区別なく統一的に扱われます。キーボードとマウスは完全に違う機器なのですが、例えばたくさんボタンが付いたマウスやタッチパッド付きキーボードなど、その中間に位置するような機器もあります。また、スピーカーに付いているボリューム調整つまみなどのキーボードともマウスとも似つかない HID 機器もあるでしょう。とにかく、人間とコンピュータをつなぐための機器は HID として実装されることが多いため、HID 規格ではすべてをひっくるめて扱える規格になっているのです。そのため、通常はキーボードもマウスも区別されず、HID 機器としてだけ認識されます。そのような多様な機器に対応するため、送受信されるデータの構造を記載したレポートディスクリプタ^{*1}を HID 機器自身に内蔵させておき、OS はそれを読み取って適切にデータを送受信する仕組みになっています。

筆者が作った USB ドライバでは、キーボードやマウスを**ブートインターフェース**として使うことにしています。ブートインターフェースはキーボードとマウスだけに存在します。ブートインターフェースでは規格でデータ構造が決められていて、レポートディスクリプタを読まずに機器を制御することができます。ブートインターフェースは主にブートローダや UEFI BIOS など、あまり高度なことをしたくないソフトウェアでも最低限キーボードとマウスを使えるように用意されています。なるべく簡単な USB ドライバにするために、筆者の USB ドライバにはレポートディスクリプタの解釈をするプログラムを入れず、ブートインターフェースを使うことにしています。

*1 Report Descriptor. レポートディスクリプタを解釈することで、ボタンの数やデータの構造などが分かります。

表 12.1 USB キーボードのデータ構造

バイト位置	値の意味
0	モディファイアキー
1	予約
2	キーコード 1
3	キーコード 2
4	キーコード 3
5	キーコード 4
6	キーコード 5
7	キーコード 6

ブートインターフェースの場合にキーボードから送られてくるデータ構造を表 12.1 に載せました。これによれば、バイト 0 にモディファイアキーの情報が来ることが分かります。キーコード 1 からキーコード 6 までは、その時押されているすべてのキーが列挙されます。その時押されたキーだけでなく、押されているキーすべてというのに気をつけてください。

表 12.2 abc キーを順に押した際のデータ列の変化の様子

モディファイアバイト	キーコード 1	キーコード 2	キーコード 3	キーボード操作
0b00000000	0x00	0x00	0x00	初期状態（何も押してない）
0b00000000	0x04	0x00	0x00	A キーを押す
0b00000010	0x04	0x00	0x00	Shift キーを押す
0b00000010	0x04	0x1b	0x00	X キーを押す
0b00000010	0x04	0x05	0x1b	B キーを押す
0b00000010	0x05	0x1b	0x00	A キーを離す

いくつかのキーを押したり離したりした場合にキーボードから送られてくるデータ列の例を表 12.2 に示します。押されたキーを太字にしました。表の中で B キーを押したときに注目してください。表ではキーコードの小さい順に表示していますが、HID の規格ではキーコード 1 から 6 の順番は特に決められていないことに注意が必要です。

リスト 12.17 モディファイアバイトを取得してハンドラに渡す (usb/classdriver/keyboard.cpp)

```
Error HIDKeyboardDriver::OnDataReceived() {
    for (int i = 2; i < 8; ++i) {
        const uint8_t key = Buffer()[i];
        if (key == 0) {
            continue;
        }
        const auto& prev_buf = PreviousBuffer();
        if (std::find(prev_buf.begin() + 2, prev_buf.end(), key) != prev_buf.end()) {
            continue;
        }
        NotifyKeyPush(Buffer()[0], key);
    }
}
```

```

    }
    return MAKE_ERROR(Error::kSuccess);
}

```

ここまでの知識を使って USB キーボードドライバを改造していきます。**リスト 12.17** にキーボードからデータが送られてきたときに呼び出される関数 `OnDataReceived()` の改造後のソースコードを示します。主に改造したのは `NotifyKeyPush(Buffer()[0], key);` というように第1引数にモディファイアキーのバイト `Buffer()[0]` を渡すようにしたことです。

後は些細なバグ修正を1つしました。修正点は `std::find()` の第1引数を `prev_buf.begin()` から `prev_buf.begin() + 2` に変えただけです。この行がやっているのは、前回キーボードから送られてきた8バイトのデータと今回送られてきたデータの差分から、新たに押されたキーを見つけることです。差分を取りたいのはキーコード1から6までであり、バイト0と1を読み飛ばさないといけないのに読み飛ばしていませんでした。モディファイアキーの組み合わせによっては今押されたキーのキーコードと偶然一致してしまい、キーを押しても入力できないバグがありました（修正前は、例えば左Altを押したままだとAキーを入力できませんでした）。

リスト 12.18 登録されたオブザーバを呼ぶ (usb/classdriver/keyboard.cpp)

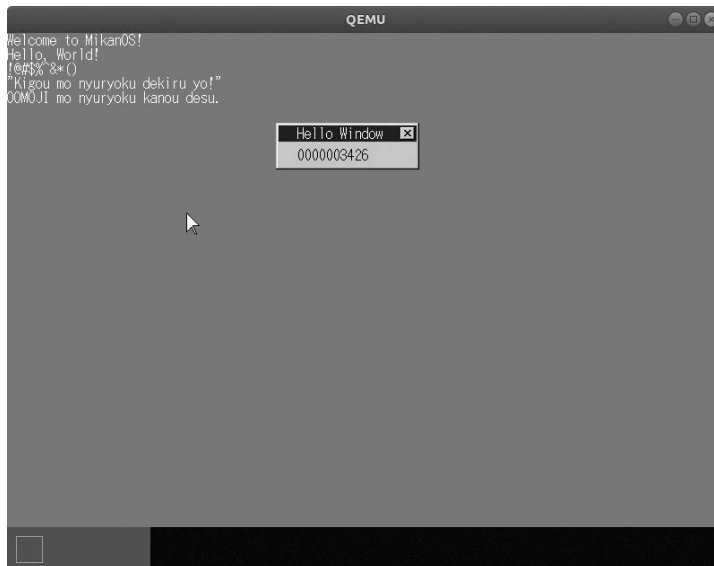
```

void HIDKeyboardDriver::NotifyKeyPush(uint8_t modifier, uint8_t keycode) {
    for (int i = 0; i < num_observers_; ++i) {
        observers_[i](modifier, keycode);
    }
}

```

`NotifyKeyPush()` が2つの引数を受け取るように改造します（**リスト 12.18**）。何も難しいところはありません。

図 12.5 Shift キーを使って大文字や記号を入力している様子



さて、改造が済んだら実行してみましょう (図 12.5)。ちゃんと Shift キーが効いていますね。大文字や記号類が入力できるようになっています。

12.5 テキストボックス (osbook_day12e)

キー入力ができるようになったのでもう少し遊んでみたいになりました。今は単にコンソールに文字が出るだけですが、より OS っぽくするためにウィンドウの中に設置したテキストボックスに文字が入力できたらカッコいい気がします。やってみます。

リスト 12.19 テキストボックス付きのウィンドウを作る (main.cpp)

```
std::shared_ptr<Window> text_window;
unsigned int text_window_layer_id;
void InitializeTextWindow() {
    const int win_w = 160;
    const int win_h = 52;

    text_window = std::make_shared<Window>(
        win_w, win_h, screen_config.pixel_format);
    DrawWindow(*text_window->Writer(), "Text Box Test");
    DrawTextbox(*text_window->Writer(), {4, 24}, {win_w - 8, win_h - 24 - 4});

    text_window_layer_id = layer_manager->NewLayer()
        .SetWindow(text_window)
        .SetDraggable(true)
        .Move({350, 200})
        .ID();

    layer_manager->UpDown(text_window_layer_id, std::numeric_limits<int>::max());
}
```

まずはテキストボックスを備えたウィンドウを作成することにしました。リスト 12.19 に新しいウィンドウを作るプログラムを示します。main_window を作る時とほとんど同じですが、今回は DrawTextbox() を呼び出しているところが違います。この関数はその名の通りテキストボックスの絵を描くための関数です。

リスト 12.20 DrawTextbox() はテキストボックスを描画する (window.cpp)

```
void DrawTextbox(PixelWriter& writer, Vector2D<int> pos, Vector2D<int> size) {
    auto fill_rect =
        [&writer](Vector2D<int> pos, Vector2D<int> size, uint32_t c) {
            FillRectangle(writer, pos, size, ToColor(c));
        };

    // fill main box
    fill_rect(pos + Vector2D<int>{1, 1}, size - Vector2D<int>{2, 2}, 0xffffffff);

    // draw border lines
    fill_rect(pos, {size.x, 1}, 0x848484);
```

```

fill_rect(pos,                                {1, size.y}, 0x848484);
fill_rect(pos + Vector2D<int>{0, size.y}, {size.x, 1}, 0xc6c6c6);
fill_rect(pos + Vector2D<int>{size.x, 0}, {1, size.y}, 0xc6c6c6);
}

```

リスト 12.20 に `DrawTextbox()` の実装を示します。この関数では、テキストボックスの背景を白で塗りつぶし、さらに外枠を描きます。関数の第1引数 `pos` はテキストボックスの左上の座標（ウィンドウの左上を基準とした座標）、第2引数 `size` はテキストボックスのピクセル単位の外形サイズです。

リスト 12.21 ウィンドウを表示する (main.cpp)

```

InitializeLayer();
InitializeMainWindow();
InitializeTextWindow();
InitializeMouse();
layer_manager->Draw({{0, 0}, ScreenSize()});

```

リスト 12.21 に示すように、先ほど定義した `InitializeTextWindow()` を呼び出します。これで実際にウィンドウが生成され、画面に表示されるようになります。

リスト 12.22 キー押下イベントが来たら文字を入力 (main.cpp)

```

case Message::kKeyPush:
    InputTextWindow(msg.arg.keyboard.ascii);
    break;

```

テキストボックス付きのウィンドウが表示されるようになったので、次はキー押下イベントが発生したらそのテキストボックスに文字列を表示するプログラムを作っていきます。**リスト 12.22** にキー押下イベントを処理している箇所を示します。以前は `printk()` でコンソールに入力された文字を送っていましたが、今回は `InputTextWindow()` に送るようにしました。この関数はまだないので、これから作っていきます。

リスト 12.23 受け取った文字をテキストボックス末尾に追加 (main.cpp)

```

int text_window_index;
void InputTextWindow(char c) {
    if (c == 0) {
        return;
    }

    auto pos = []() { return Vector2D<int>{8 + 8*text_window_index, 24 + 6}; };

    const int max_chars = (text_window->Width() - 16) / 8;
    if (c == '\b' && text_window_index > 0) {
        --text_window_index;
        FillRectangle(*text_window->Writer(), pos(), {8, 16}, ToColor(0xffffffff));
    } else if (c >= ' ' && text_window_index < max_chars) {
        WriteAscii(*text_window->Writer(), pos(), c, ToColor(0));
        ++text_window_index;
    }
}

```

```
layer_manager->Draw(text_window_layer_id);  
}
```

リスト 12.23 に `InputTextWindow()` の実装を示します。この関数は 1 文字を受け取り、それをテキストボックスに表示するための関数です。普通の文字であればテキストボックスの先頭からどんどん追加されていきます。入力文字がバックスペース ('`\b`') だったら文字を追加するのではなく削除します。

グローバル変数 `text_window_index` はテキストボックスに現在表示している文字数を記録します。バックスペースが入力された際は、表示している文字が 1 文字以上あれば削除処理を行います。削除処理と言っても、最後の文字を白で塗りつぶすだけです。バックスペース以外が入力されたら、表示している文字数がテキストボックスに表示できる最大文字数 `max_chars` 未満であるかを調べます。1 文字以上の余裕があれば末尾に入力文字を追加します。

リスト 12.24 32 ビット整数から `PixelColor` 構造体を作る (`graphics.hpp`)

```
constexpr PixelColor ToColor(uint32_t c) {  
    return {  
        static_cast<uint8_t>((c >> 16) & 0xff),  
        static_cast<uint8_t>((c >> 8) & 0xff),  
        static_cast<uint8_t>(c & 0xff)  
    };  
}
```

`ToColor()` を `main.cpp` から使えるように、関数の定義を `window.cpp` から `graphics.hpp` に移動しました (**リスト 12.24**)。普通の関数ならヘッダファイルにはプロトタイプ宣言だけを書いて `.cpp` ファイルで本体の定義を行います。が、`constexpr` 指定をした関数の場合はヘッダファイルに本体の定義まで書く必要があります。

図 12.6 テキストボックス付きウィンドウに文字入力している様子

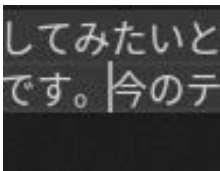


改造した OS を動作させた様子を図 12.6 に示します。きちんとテキストボックスに文字列が表示できています。18 文字を超えて入力しようとしても入力できないこと、バックスペースできちんと消せることも確認できました。完璧ですね！

12.6 カーソル (osbook_day12f)

今日の仕上げとして、テキストボックスにカーソルを表示してみたいと思います。カーソルというのはテキストエディタなどで点滅して表示される長方形や棒状のマークです。例えば Visual Studio Code のカーソルは図 12.7 のような形をしています（「今」の左側にある縦棒がそれです）。

図 12.7 Visual Studio Code のカーソル



テキストボックスにカーソルを表示することによって、そこに入力できることがはっきりとユーザーに伝わります。今のカーソルの表示がないテキストボックスだと「本当にそこに入力されるのだろうか？」と不安になります。キーボードで文字が入力されるときに少し違和感を感じませんか？ 筆者は感じました。ということで、入力されている文字列の後ろにカーソルを表示するようにします。

リスト 12.25 カーソルを文字入力にともなって動かす (main.cpp)

```
void InputTextWindow(char c) {
    if (c == 0) {
        return;
    }

    auto pos = [](int) { return Vector2D<int>{8 + 8*text_window_index, 24 + 6}; };

    const int max_chars = (text_window->Width() - 16) / 8 - 1;
    if (c == '\b' && text_window_index > 0) {
        DrawTextCursor(false);
        --text_window_index;
        FillRectangle(*text_window->Writer(), pos(), {8, 16}, ToColor(0xffffffff));
        DrawTextCursor(true);
    } else if (c >= ' ' && text_window_index < max_chars) {
        DrawTextCursor(false);
        WriteAscii(*text_window->Writer(), pos(), c, ToColor(0));
        ++text_window_index;
        DrawTextCursor(true);
    }

    layer_manager->Draw(text_window_layer_id);
}
```


リスト 12.25 にカーソルを表示するように改造した `InputTextWindow()` を示します。`DrawTextCursor()` は後ほど実装する関数で、引数に真を渡すとカーソルを表示し、偽を渡すとカーソルを消します。カーソルを描いたり消したりする位置は `text_window_index` 変数で決まります。この関数を使って、バックスペースが入力されたときはカーソルを左に移動し、その他の文字が入力されたときは右に移動するようにしました。

リスト 12.26 `DrawTextCursor()` はカーソルを描いたり消したりする (main.cpp)

```
void DrawTextCursor(bool visible) {
    const auto color = visible ? ToColor(0) : ToColor(0xffffffff);
    const auto pos = Vector2D<int>{8 + 8*text_window_index, 24 + 5};
    FillRectangle(*text_window->Writer(), pos, {7, 15}, color);
}
```

リスト 12.26 に載せた `DrawTextCursor()` は、`text_window_index` 変数で表される位置にカーソルを表示したり、または消したりします。消すといっても白で塗りつぶすということですが、引数 `visible` を真にするとカーソルを表示し、偽にすると消します。

さて、もうカーソルが表示できるようにはなっているはずですが、でも、カーソルはやっぱり点滅させたいですね？ 今のままだと、ただずっと黒い長方形が表示されっぱなしになり、カーソルっぽさが足りません。ということで点滅機能も実装することにします。方針としては、0.5 秒くらいでタイムアウトする周期タイマを作り、タイムアウトするたびにカーソルを描いたり消したりすれば良いと思います。早速やってみます。一生懸命タイマを実装したおかげで、0.5 秒の周期タイマはとても簡単に実現できるようになっています。

リスト 12.27 0.5 秒でタイムアウトするタイマを作る (main.cpp)

```
const int kTextboxCursorTimer = 1;
const int kTimer05Sec = static_cast<int>(kTimerFreq * 0.5);
__asm__("cli");
timer_manager->AddTimer(Timer{kTimer05Sec, kTextboxCursorTimer});
__asm__("sti");
bool textbox_cursor_visible = false;
```

リスト 12.27 のようにタイマを作り、最初のタイムアウト時刻を 0.5 秒とします。`kTimerFreq` は `timer.hpp` で定義された定数で、1 秒間に `tick_` が増える数を表します。`kTimerFreq` に好きな秒数をかけることで、その秒数に対応する `tick_` の変化量を計算できるというわけです。

`timer_manager->AddTimer()` の前後で割り込みを禁止、許可している理由は「11.3 細かく時間を計る」で説明したのと同じです。`timer_manager` はタイマ割り込みハンドラの中で使われますから、割り込みハンドラ外から操作するときは一時的に割り込みを禁止する必要があるのです。

割り込みを禁止しないと何が起こるか、復習を兼ねて詳しく考えてみます。`timer_manager->AddTimer()` が `timers_` 変数に対してタイマを追加しようとしているまさにそのときにタイマ割り込みが発生した場合を想像してください。タイマ割り込みハンドラは `timer_manager->Tick()` を呼び出し、その中で `timers_` を読み取ります。そうなると中途半端に変更された `timers_` を読むことになり、未定義の動作を引き起こします。更に悪いことに、タイムアウトしたタイマを見つけたら

timers_ 変数を書き換えようとするでしょう。中途半端に更新された timers_ 変数を書き換えようものなら中身がぐちゃぐちゃになってしまいます。お怖いです。

リスト 12.28 カーソル用タイマがタイムアウトしたときの処理 (main.cpp)

```
case Message::kTimerTimeout:
    if (msg.arg.timer.value == kTextboxCursorTimer) {
        __asm__("cli");
        timer_manager->AddTimer(
            Timer{msg.arg.timer.timeout + kTimer05Sec, kTextboxCursorTimer});
        __asm__("sti");
        textbox_cursor_visible = !textbox_cursor_visible;
        DrawTextCursor(textbox_cursor_visible);
        layer_manager->Draw(text_window_layer_id);
    }
    break;
```

タイムアウトのイベントを処理するプログラムを作りました (リスト 12.28)。msg.arg.timer.value が期待したものであることを確認して、そうであれば新しく 0.5 秒後にタイムアウトするタイマを追加し、カーソルの点滅処理を行います。点滅処理は textbox_cursor_visible の真偽を反転させつつ、その値を使ってカーソルの表示、非表示を切り替えます。これで 0.5 秒毎にカーソルの状態が切り替わって点滅しているように見えるはずですが。ここでも timer_manager->AddTimer() の実行中は割り込みを禁止しておきます。

図 12.8 文字の入力にともなってカーソルが動き、点滅もしている



動かしてみると図 12.8 のようになりました。写真だと点滅が分かりませんが、筆者の環境ではちゃんと点滅しています。もちろん文字を入力するとカーソルが左右に動きます。テキストボックスとして全然違和感がありません。大成功です。

第 13 章

マルチタスク(1)

キー入力もできてますます OS のような見た目になってきました。しかし、OS として重要な「アプリケーションを動かす」という機能はまだ実現できていません。この章ではその第一歩となる、カーネル空間でのマルチタスクを実装していきます。カーネル空間でマルチタスクが実現できるようになるとそれを応用してアプリケーションの起動ができるようになります。この章を終える頃には複数のタスクを順次切り替えて実行することができるようになります。

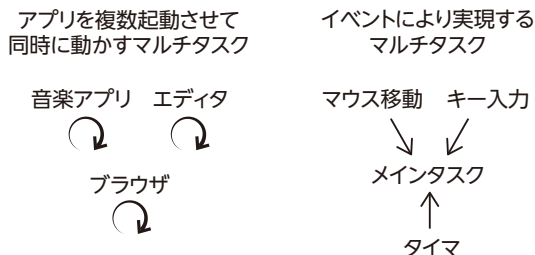
13.1 マルチタスクとコンテキスト

マルチタスク (Multi Task) とはその名の通り複数の仕事を「同時に」することをいいます。本当に同時ではなくても、ある仕事をしつつ、たまにもう 1 つの仕事をある程度進め、また元の仕事に戻る、みたいな場合もマルチタスクと呼びます。皆さんは裏で音楽を再生しつつブラウザでドキュメントを表示し、それを見ながらエディタでプログラミングを行う——といったことをした経験があると思います。この場合、ブラウザやエディタ、音楽再生アプリが同時に動いているわけです。そういうのをマルチタスクといいます。

MikanOS もある意味マルチタスクができています。USB 機器やタイマからのイベントを 1 つずつ処理することで、マウスの移動、キー入力、カーソルの点滅といった処理が同時に動いていると言えます。

しかし、アプリを複数起動させて同時に動かすマルチタスクと、イベントにより実現するマルチタスクは毛色がかなり異なりますね (図 13.1)。前者では各アプリは自分の処理だけを考えていれば十分です。エディタはユーザーの入力を受け取って画面に表示したりファイルに保存したりする処理をすればよく、ドキュメントの表示や音楽の再生のやり方は考えなくていいのです。一方で、後者はマウスの移動、キー入力、カーソルの点滅など、すべての処理を一手に引き受けれます。

図 13.1 イベント駆動のマルチタスクではイベントの種類に応じて処理を分ける



また、後者では各処理に時間がかかってしまうと他の処理が遅れるという特徴もあります。例えば、A キーが押されたら何らかのファイルを暗号化する、という処理をイベント駆動でやるとします。そのファイルが小さいうちはいいかもしれませんが、少し大きくなると暗号化処理に時間がかかるようになり、処理の間はマウスカーソルが動かず、テキストボックスに何も文字を入力できない、ということになります。そうならないよう、各イベントの処理は短時間で終わるようにしておく必要があります。アプリを複数起動するマルチタスクでは、1 つのアプリをちょっと実行したら強制的に次のアプリに切り替えることで、あるアプリが重い処理をしていても他のアプリが実行できなくなることがありません^{*1}。

このような差は**コンテキスト** (Context) という言葉で説明できます。コンテキストとは日本語では文脈と訳されることが多いです。コンテキストとは、ある作業をするために必要なデータや変数をまとめたもの、と言えます。例えば、OS 上で動く C 言語アプリを考えます。このアプリのコンテキストとは、そのアプリの実行バイナリ、コマンドライン引数、環境変数、スタックメモリ、各レジスタの値などをまとめたもののことです。同じアプリを複数回同じように起動させると、実行バイナリやコマンドライン引数、環境変数は同じでもスタックメモリやレジスタ値は別々です。したがって、それぞれ別のコンテキストということになります。

アプリを複数起動するタイプのマルチタスクでは、それぞれのアプリが独自のコンテキストを持ち、その中で動作をします。あるコンテキストの中では他のコンテキストを考慮することなくプログラムが

動作します。そのため、各アプリは独立して動作できるのです。一方でイベント駆動のマルチタスクでは、イベント処理の部分が1つのコンテキストにならざるを得ず、1つのコンテキストで複数の異なった種類のタスクを処理することになります。

この章でやりたいのは別々のコンテキストを持つタスクを同時に動かすことです。実現方法は大きく2つあります。1つは各コンテキストを別々のCPUコアで動作させることです。現代のパソコンに搭載されているCPUはほとんどがマルチコアCPUといって、独立して動作するCPUコアが複数搭載されています。それぞれのCPUコアは自分自身のレジスタを持っていますので、1つのCPUコアは1つのコンテキストを持てます。8コアのCPUを使えば8つのコンテキストを同時に動かすことができるということです。複数のコンテキストが真に同時に動くことを**並列処理**と呼びます。

もう1つはCPUコアで実行するコンテキストを時分割で切り替えていく方法です。ある瞬間を見るとどれか1つのコンテキストしか動いていませんが、平均して見るとそれぞれのコンテキストがまんべんなく実行されているため、パソコンを使っている人間にはすべてのコンテキスト（タスク）が同時に動いているように見えます。このような処理方式を**並行処理**と呼びます。

実際にはこれら2つのどちらかだけを選ぶというより、両者を組み合わせるのが一般的です。コアが余らないように最大の並列化を行うと効率的な処理が行なえますが、コア数以上に実行すべきタスクがあるならコア内でコンテキストを切り替える必要があります。本書で扱うにはマルチコアの難易度がちょっと高すぎるため、1つのCPUコアのみで実現できる並行処理だけを実装することにします。

13.2 コンテキストの切り替えに挑戦 (osbook_day13a)

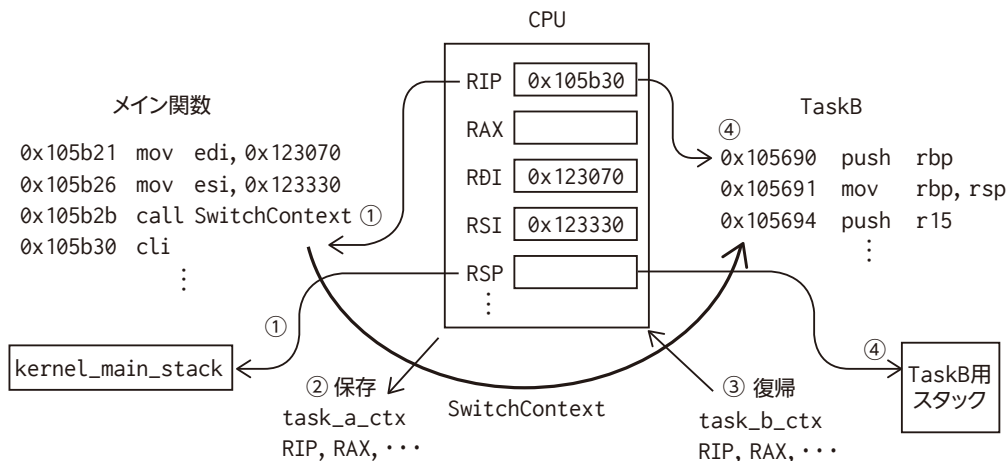
タスクの切り替えと書いてもいいのですが、より正確なタイトルにしてみました。この節では2つのコンテキストを用意してそれらを切り替える、すなわち**コンテキストスイッチ**を実装してみます。コンテキストスイッチは複数のアプリを同時に動かすための第一歩となります。

コンテキストの1つはメイン関数を、もう1つはTaskB()という関数を実行するようなコンテキストとします。2つのコンテキストを交互に切り替えて実行し、メイン関数の処理とTaskB()の処理が交互にちょっとずつ進むようにします。

詳しく見てみましょう。今、CPUはメイン関数を実行しています。RIPレジスタは現在実行中の命令の次の命令を指していて、それはメイン関数内のどこかです。RSPレジスタはスタックの先頭を指していて、それはkernel_main_stackのどこかです。汎用レジスタ(RAXなど)を使って次々にメイン関数内の命令が実行されていきます。この様子を図にすると図13.2の①のようになります。

* 1 アプリがCPU以外のリソースを使い切ってしまう、そのリソースを必要とする他のアプリがまともに動かない、というのはあり得ます。ここではCPU時間というリソースに限った話をしています。

図 13.2 TaskB() 用のコンテキストに切り替える様子



ここで、もう1つのコンテキスト(TaskB()のコンテキスト)に切り替えるのに必要な処理を考えます。最低限やらなくちゃいけないのはRIPをTaskB()の先頭アドレスに書き換えること、TaskB()の2つの引数をRDI、RSIに設定することです。これさえやれば、TaskB()に切り替えるだけのことはできるでしょう。ただ、それだけでは片道切符です。後でメイン関数のコンテキストに戻ってくるためにはいくつかのレジスタを保存しておく必要があります。少なくともRIPを保存しておかないとどこに戻ってきたらいいか分からなくなってしまいますし、RSPの値を元通りにする必要もあります。そのほかにもいくつか保存しておくべきレジスタがありますが、詳しくは後で説明します。

CPUはあくまでも、レジスタの値を参照して機械語を1つずつ実行しているだけ、というのが大切なところ。RIPが指すメモリ領域から機械語を読み取り、RIPを次の命令を指すように更新し、機械語を実行する、という動作をただ繰り返します。機械語の実行にともなって各レジスタの値が更新されたり、メモリの読み書きが行われたりします。この単純さを利用し、うまくレジスタの値を変更してやることでCPUはだまされて、実行するコンテキストを切り替えることができます。レジスタの値の入れ替えこそがコンテキストスイッチの本質なのです。

リスト 13.1 TaskB() が何かを表示するためのウィンドウを作る (main.cpp)

```
std::shared_ptr<Window> task_b_window;
unsigned int task_b_window_layer_id;
void InitializeTaskBWindow() {
    task_b_window = std::make_shared<Window>(
        160, 52, screen_config.pixel_format);
    DrawWindow(*task_b_window->Writer(), "TaskB Window");

    task_b_window_layer_id = layer_manager->NewLayer()
        .SetWindow(task_b_window)
        .SetDraggable(true)
        .Move({100, 100})
        .ID();

    layer_manager->UpDown(task_b_window_layer_id, std::numeric_limits<int>::max());
}
```

ここから、実際に 2 つのタスクを切り替える処理を作ってみます。まずは TaskB() がちゃんと動いているというのが分かるように、情報を表示するためのウィンドウ task_b_window を作ります (**リスト 13.1**)。これは今までのウィンドウの表示と同じやり方ですから特に難しいことはないはずです。ウィンドウを作る処理はコンテキストスイッチとはまったく無関係です。

リスト 13.2 タスクのコンテキストを保存するための構造体 (main.cpp)

```
struct TaskContext {
    uint64_t cr3, rip, rflags, reserved1; // offset 0x00
    uint64_t cs, ss, fs, gs; // offset 0x20
    uint64_t rax, rbx, rcx, rdx, rdi, rsi, rsp, rbp; // offset 0x40
    uint64_t r8, r9, r10, r11, r12, r13, r14, r15; // offset 0x80
    std::array<uint8_t, 512> fxsave_area; // offset 0xc0
} __attribute__((packed));

alignas(16) TaskContext task_b_ctx, task_a_ctx;
```

コンテキストを保存するための構造体 TaskContext の定義を **リスト 13.2** に示します。この構造体は、コンテキストを切り替えるときに値の保存と復帰に必要なレジスタをすべて含んでいます。

リスト 13.3 TaskB() ではループのたびにコンテキストを切り替える (main.cpp)

```
void TaskB(int task_id, int data) {
    printf("TaskB: task_id=%d, data=%d\n", task_id, data);
    char str[128];
    int count = 0;
    while (true) {
        ++count;
        sprintf(str, "%010d", count);
        FillRectangle(*task_b_window->Writer(), {24, 28}, {8 * 10, 16}, {0xc6, 0xc6, 0xc6});
        WriteString(*task_b_window->Writer(), {24, 28}, str, {0, 0, 0});
        layer_manager->Draw(task_b_window_layer_id);

        SwitchContext(&task_a_ctx, &task_b_ctx);
    }
}
```

リスト 13.3 に TaskB() の定義を示します。この関数が一旦起動されると無限ループに入り、ループ回数を数えてウィンドウに表示します。この無限ループが普通の無限ループと違うのは、1 周するたびに SwitchContext() という謎の関数を呼び出していることです。この関数は 2 つのコンテキストを切り替えるための関数で、ここでは TaskB() を実行するコンテキスト task_b_ctx から、メイン関数を実行するコンテキスト task_a_ctx へと切り替えています。

SwitchContext() は後で実装しますが、レジスタを操作してコンテキストを切り替える関数になる予定です。具体的には、レジスタ (RIP や RSP など) の元の値を第 2 引数が指すメモリ領域 task_b_ctx に保存し、第 1 引数で指定されたメモリ領域 task_a_ctx からレジスタの値を復帰させます。

レジスタをいい具合に操作してコンテキストを切り替えるという処理は C++ の規格には登場しない概念です。C++ コンパイラは、TaskB() を実行中にいきなりメイン関数に実行が移るなんていうことはまったく想定していません。SwitchContext() は普通の関数であり、中で何らかの処理を実行し

たら帰ってくる、と思っています。System V AMD64 ABI では、関数の呼び出し前後でいくつかのレジスタは値が保存されることが決まっています。C++ コンパイラは SwitchContext() の呼び出しの前後でその規則が守られているはずだと信じています。

しかし、実際は SwitchContext() を呼び出してから関数が終了するまでの間に、一時的にメイン関数コンテキストに切り替わり、メイン関数の処理がしばらく実行されます。メイン関数の処理では、本来保存されていなければならないレジスタの値が変更されるかもしれません。そのため、SwitchContext() の呼び出しの前後でレジスタの値を保存、復帰する必要があります。TaskContext 構造体がたくさんのレジスタを持つ理由の 1 つはこれです。

リスト 13.4 メインループの中でもコンテキストを切り替える (main.cpp)

```
__asm__("cli");
if (main_queue->size() == 0) {
    __asm__("sti");
    SwitchContext(&task_b_ctx, &task_a_ctx);
    continue;
}
```

リスト 13.4 にメインループの改造箇所を示します。メイン関数用のコンテキストから TaskB() 用のコンテキスト（以降では TaskB コンテキストと呼びます）へと切り替えます。

リスト 13.5 TaskB() 用のコンテキストを作成する (main.cpp)

```
std::vector<uint64_t> task_b_stack(1024);
uint64_t task_b_stack_end = reinterpret_cast<uint64_t>(&task_b_stack[1024]);

memset(&task_b_ctx, 0, sizeof(task_b_ctx));
task_b_ctx.rip = reinterpret_cast<uint64_t>(TaskB);
task_b_ctx.rdi = 1;
task_b_ctx.rsi = 42;

task_b_ctx.cr3 = GetCR3();
task_b_ctx.rflags = 0x202;
task_b_ctx.cs = kKernelCS;
task_b_ctx.ss = kKernelSS;
task_b_ctx.rsp = (task_b_stack_end & ~0xflu) - 8;

// MXCSR のすべての例外をマスクする
*reinterpret_cast<uint32_t*>(&task_b_ctx.fxsave_area[24]) = 0x1f80;
```

TaskB コンテキストの初期化プログラムをリスト 13.5 に示します。1 行目でこのコンテキスト用のスタックとなる適当な大きさのメモリ領域（ここではとりあえず 8KiB としてみました）task_b_stack を確保します。

その後、コンテキストの初期値を設定します。コンテキスト領域全体をまず 0 でクリアした後、必要な値を個別に設定する方法としました。ここで設定した値は、後で作る SwitchContext() の中で利用されることになります。task_b_ctx.rip には TaskB() の先頭アドレスを入れていきます。SwitchContext() を最初に実行したとき、ここで設定したアドレスへとジャンプすることになります。task_b_ctx.rdi と rsi には TaskB() の引数となる値を書いておきます。詳しくは

SwitchContext() を実装した後で説明することになります。

リスト 13.6 GetCR3() は CR3 レジスタの値を取得する (asmfunc.asm)

```
global GetCR3 ; uint64_t GetCR3();
GetCR3:
    mov rax, cr3
    ret
```

task_b_ctx.cr3 には現在 CR3 に設定されている値をコピーします。GetCR3() の実装を **リスト 13.6** に示します。CR3 には PML4 テーブルのアドレスが設定されていますから、その値をコピーすることは、TaskB() 実行中にも同じ PML4 テーブルを参照することになります。

task_b_ctx.rflags は TaskB() を実行する際の RFLAGS の値を指定します。設定している 0x202 という値は、ビット 1 と 9 だけが 1 でその他のビットは 0 です。ビット 9 は IF (割り込みフラグ) で、ここに 1 を設定すると割り込みが許可されます。IF は cli や sti で操作される、おなじみのビットですね。ビット 1 には 0 と 1 のどちらを設定しても差はありません。なぜなら RFLAGS レジスタのビット 1 はハードウェア的に 1 に固定されているからです。0 を書いても読み出すと 1 になります。

task_b_ctx.cs と ss にはメイン関数を実行するときと同じセグメントセクタを設定しておきます。

task_b_ctx.rsp にはスタックポインタの初期値を設定します。この計算がちょっと複雑なのは、x86-64 アーキテクチャ特有のアライメント制約を満たすためです。詳しくは「**コラム 13.1 x86-64 アーキテクチャとスタックのアライメント制約**」で説明します。その制約を満たすため、task_b_ctx.rsp の値の下位 4 ビットが 8 になるように調整します。

task_b_ctx.fxsave_area のオフセット 24 から 27 は MXCSR レジスタに対応します。このレジスタは浮動小数点数計算の設定をしたり、ステータスを表したりするためのレジスタです。C++ コンパイラは、浮動小数点数の計算をしないプログラムであっても浮動小数点数計算用の命令を出力することがあります。そのときに、MXCSR のビット 12:7 がすべて 1 になっていないと正しい処理ができませんので、初期値として 0x1f80 を設定しておきます。MXCSR については Intel SDM (参考文献 [3]) Vol.1 「10.2.3 MXCSR Control and Status Register」を参照してください。

リスト 13.7 SwitchContext() は 2 つのコンテキストを切り替える (asmfunc.asm)

```

global SwitchContext
SwitchContext:  ; void SwitchContext(void* next_ctx, void* current_ctx);
    mov [rsi + 0x40], rax
    mov [rsi + 0x48], rbx
    mov [rsi + 0x50], rcx
    mov [rsi + 0x58], rdx
    mov [rsi + 0x60], rdi
    mov [rsi + 0x68], rsi

    lea rax, [rsp + 8]
    mov [rsi + 0x70], rax ; RSP
    mov [rsi + 0x78], rbp

    mov [rsi + 0x80], r8
    mov [rsi + 0x88], r9
    mov [rsi + 0x90], r10
    mov [rsi + 0x98], r11
    mov [rsi + 0xa0], r12
    mov [rsi + 0xa8], r13
    mov [rsi + 0xb0], r14
    mov [rsi + 0xb8], r15

    mov rax, cr3
    mov [rsi + 0x00], rax ; CR3
    mov rax, [rsp]
    mov [rsi + 0x08], rax ; RIP
    pushfq
    pop qword [rsi + 0x10] ; RFLAGS

    mov ax, cs
    mov [rsi + 0x20], rax
    mov bx, ss
    mov [rsi + 0x28], rbx
    mov cx, fs
    mov [rsi + 0x30], rcx
    mov dx, gs
    mov [rsi + 0x38], rdx

    fxsave [rsi + 0xc0]

    ; iret 用のスタックフレーム
    push qword [rdi + 0x28] ; SS
    push qword [rdi + 0x70] ; RSP
    push qword [rdi + 0x10] ; RFLAGS
    push qword [rdi + 0x20] ; CS
    push qword [rdi + 0x08] ; RIP

    ; コンテキストの復帰
    fxrstor [rdi + 0xc0]

    mov rax, [rdi + 0x00]
    mov cr3, rax
    mov rax, [rdi + 0x30]
    mov fs, ax

```

```

mov rax, [rdi + 0x38]
mov gs, ax

mov rax, [rdi + 0x40]
mov rbx, [rdi + 0x48]
mov rcx, [rdi + 0x50]
mov rdx, [rdi + 0x58]
mov rsi, [rdi + 0x68]
mov rbp, [rdi + 0x78]
mov r8, [rdi + 0x80]
mov r9, [rdi + 0x88]
mov r10, [rdi + 0x90]
mov r11, [rdi + 0x98]
mov r12, [rdi + 0xa0]
mov r13, [rdi + 0xa8]
mov r14, [rdi + 0xb0]
mov r15, [rdi + 0xb8]

mov rdi, [rdi + 0x60]

o64 iret

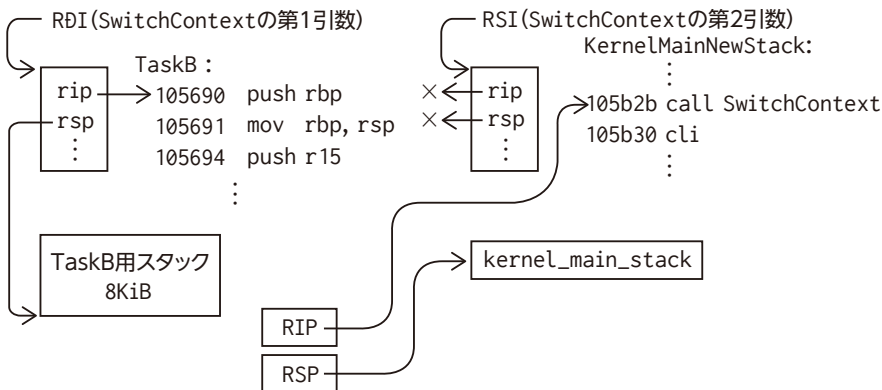
```

いよいよコンテキストスイッチの心臓部となる `SwitchContext()` を実装します (**リスト 13.7**)。この関数の主目的はコンテキストの保存と復帰です。現在実行中のコンテキストを第2引数 (RSI) が指すメモリ領域に保存し、第1引数 (RDI) が指すメモリ領域から CPU のレジスタ群を復帰します。だいぶ長いですがやっていることは単純で、各レジスタをコンテキスト構造体の対応するフィールドにコピーしたり、逆にコンテキスト構造体から各レジスタに値をコピーするだけです。とても愚直な実装です。この中で `fxsave/fxrstor` という命令を使っていますね。これらは浮動小数点数用のレジスタ群 (XMM0 など) をまとめて保存、復帰するための命令です。 `TaskContext::fxsave_area` を読み書きします。

`SwitchContext()` は、今のところ保存と復帰が不要なレジスタまで対象にしていますが、これは将来的にやろうと思っているコンテキストスイッチに備えるためです。具体的には CR3、セグメントレジスタ (CS など)、RBP、RBX、R12 から R15 を除く汎用レジスタは保存と復帰が不要です。なぜなら、CR3 とセグメントレジスタは2つのコンテキストで同じ値が使われるからです。また、この6個以外の汎用レジスタは関数呼び出しの前後で値が維持されている必要がないからです。後でプリエンティブマルチタスクというものをやるときに、それらのレジスタを保存、復帰した意味が出てきます。

`SwitchContext()` が最初に呼ばれるのは **リスト 13.4** の部分に到達したときです。一通りイベント処理を終えるとここに到達します。最初に `SwitchContext()` を呼び出す直前の2つのコンテキスト構造体の様子は **図 13.3** のようになっています。 `task_b_ctx` には `TaskB()` 用のコンテキストの初期値が入っていて、 `task_a_ctx` は0クリアされています。

図 13.3 SwitchContext() 実行直前のコンテキスト構造体

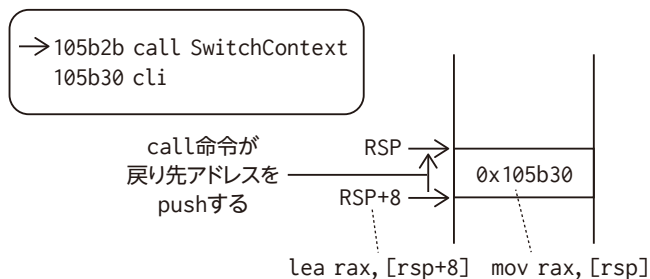


`SwitchContext()` は第 2 引数 `RSI` が指すコンテキスト構造体 `task_a_ctx` に、現在のレジスタ値を保存していきます。汎用レジスタは単に `mov` だけで保存可能ですが、いくつかのレジスタ値の保存は工夫が必要です。最も分かりにくいのが `RIP` と `RSP` です。

コンテキスト構造体に保存した値は、次にそのコンテキストを復帰するときにレジスタに書き戻されます。そのため、`SwitchContext()` の実行直後となるような `RIP` と `RSP` の値をコンテキスト構造体に記録しておきたいです。そのような `RIP` と `RSP` の値はどうやって取得したらいいかと考えて、**リスト 13.7** のようなプログラムになっています。

メイン関数は `call` 命令によって `SwitchContext()` を呼び出します。`call` 命令はスタックに戻り先アドレスを積んでから対象の関数にジャンプする、という命令です。したがって、`SwitchContext()` の先頭にジャンプした直後、`RSP` が指すスタック領域には `call` 命令の次の命令のアドレスが記録されています (図 13.4)。

図 13.4 RIP と RSP の計算



そこで、`mov rax, [rsp]` とすることで `RAX` レジスタに戻り先アドレスを取得できます。この値こそが `task_a_ctx.rip` に記録すべき値ですね。また、そのときの `RSP` の値に 8 を加えた値が `task_a_ctx.rsp` に記録すべき値となります。これは `lea rax, [rsp+8]` により計算できます (`mov` と `add` を組み合わせても計算できますが `lea` なら 1 命令です)。

そのほか、特殊な方法でしか取得できないのは `RFLAGS` レジスタです。その他のレジスタのように

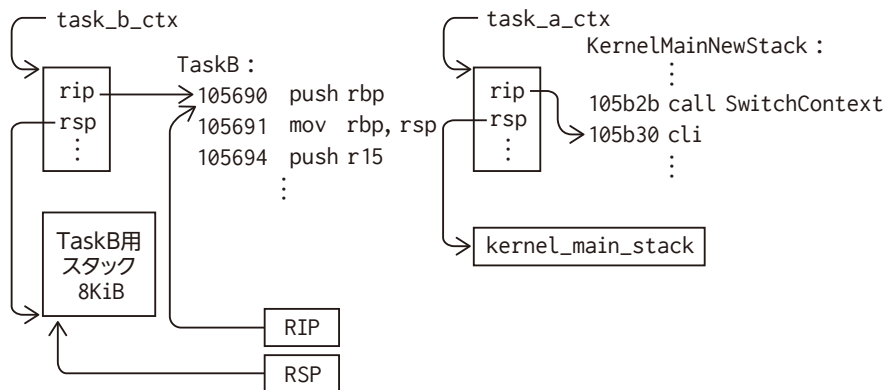
mov 命令では扱えず、pushfq という専用の命令を使います。pushfq は RFLAGS の値をスタックに積みます（このための専用命令があるなんて面白いですね）。この命令を使ってスタックに積んだ値を pop で取り出します。

fxsave まで終わったらコンテキストの保存は完了です。次は第 1 引数 RDI が指すコンテキスト構造体 task_b_ctx からコンテキストの復帰を行います。ほとんどのレジスタは mov 命令を使って復帰できますが、いくつかのレジスタは特殊な方法で復帰させます。

各レジスタの復帰に先立ち、iret 用のスタックフレームを構築します。これは SwitchContext() の最後に iret 命令を実行するのに必要なものです。普通、関数から戻るときは ret 命令を使って、スタックから戻り先アドレスだけを取り出し、戻ります。しかし、コンテキストを切り替えるには通常の ret では不十分です。なぜなら、コンテキストを切り替えるには CS や RSP などと同時に切り替える必要があるからです。そのために iret 命令が便利に使えます。

iret 用のスタックフレームを構築し終わったら、iret で復帰される以外のレジスタを復帰していきます。愚直にやるだけなのですが、1 つ注意点があり、RDI は最後に復帰しなければなりません。RDI を変更してしまうとコンテキスト構造体のアドレスが分からなくなってしまうからです。

図 13.5 SwitchContext() 実行直後のコンテキスト構造体



iret 命令が実行された直後のコンテキスト構造体の様子を図 13.5 に示します。task_a_ctx に SwitchContext() を呼び出した時点の各レジスタの値が記録され、task_b_ctx に設定されていた値が各レジスタに読み込まれます。CPU は、iret 命令を実行した後は、iret 命令によって RIP に設定されたアドレスが指すメモリ領域から機械語を読み取って実行を続けていきます。

以上で SwitchContext() の実装が完了です。この関数をメイン関数と TaskB() で相互に実行しあうことで、2 つのコンテキストが高速で切り替わりながら並行に動作していくはずですが。

図 13.6 2つのコンテキストが並行に実行されている様子



図 13.6 に実行の様子を示します。2つのコンテキストが並行して動いていることが分かります。2つのコンテキストを切り替えつつ少しずつ処理をすすめていく実験は成功です！

コラム 13.1 x86-64 アーキテクチャとスタックのアライメント制約

System V AMD64 ABI では、ある関数から他の関数を呼ぶときはスタックポインタの値を 16 バイト境界に揃えておくことが定められています。これは、x86-64 の一部の命令 (movaps など) に与えるメモリアドレスが 16 バイトアライメントを満たさねばならないからです。16 バイトの倍数ではないメモリアドレスをそれらの命令に与えると GP 例外 (一般保護例外) が発生してしまいます。

ある関数が他の関数を呼ぶ直前、すなわち call 命令を実行する直前にスタックポインタが 16 の倍数になっている必要があります。16 進数で書いた場合、0xffff0 のように下 1 桁が 0 になることを意味します。call 命令は 8 バイトの戻り先アドレスをスタックに積むため、call 命令の直後、すなわち関数の先頭ではスタックポインタの下位 4 ビットの値が必ず 8 になります。16 進数で書いた場合、0xffe8 のように下 1 桁がかならず 8 になるということです。C++ コンパイラはこの制約が満たされていることを仮定して変数をスタックに配置します。仮定が破られると GP 例外の原因となります。

リスト 13.5 では、次のように TaskB のスタックポインタの下位 4 ビットの値を 8 に調整しています。

```
task_b_ctx.rsp = (task_b_stack_end & ~0xfllu) - 8;
```

まず `task_b_stack_end & ~0xfllu` によってアドレス値の下位 4 ビットを切り捨て、16 の倍数に調整します。その値からさらに 8 を引けば、下位 4 ビットが必ず 8 になるわけです。あれ？ スタックポインタの値が 16 の倍数でなければならないのに、わざと 8 だけずらすのはなぜでしょうか？

それは `TaskB()` があたかも `call` 命令で呼び出されたかのように見せかける必要があるからです。`TaskB` コンテキストの `RIP` の初期値には `TaskB()` の先頭アドレスを書いてあります。したがって、最初にメインタスクから `TaskB` へ切り替わるとき、`TaskB()` の先頭から処理が始まります。`C++` コンパイラは関数が `call` 命令で呼び出される、すなわち、`TaskB()` の先頭ではスタックポインタの下位 4 ビットが 8 であると仮定し、機械語を出力します。これが、`TaskB` コンテキストの `RSP` の初期値をわざと 8 だけずらしている理由です。

13.3 コンテキストスイッチの自動化 (osbook_day13b)

2 つのタスクが `SwitchContext()` を互いに呼び合うことで切り替えを行うことに成功しました。こういう方式を**協調的マルチタスク**^{*2}と呼びます。1 つのタスクが CPU を独占することなく、適切なタイミングでちゃんと `SwitchContext()` を呼び出すということに同意しているので「協調的」と言います。処理にバグがあって `SwitchContext()` を呼び出すことなく処理を続けてしまうタスクが存在すると、協調的マルチタスクは容易に破綻してしまいます。

OS に内蔵された関数をタスクとして実行するなら協調的マルチタスクでも十分実用的です。ただ、将来的にアプリケーションのマルチタスクをやる場合には協調的マルチタスクだと不十分です。もし 1 つでもお行儀の悪い、もしくはバグがあるアプリがあって、`SwitchContext()` を呼ばずにずっと処理を続けるとコンテキストスイッチされずに OS 全体がフリーズしてしまいます。

この節では協調的マルチタスクをやめて**プリエンプティブマルチタスク** (preemptive multitasking) を実現してみます。プリエンプティブマルチタスクでは CPU の割り込み処理 (通常はタイマ割り込み) を使って強制的にタスクを切り替えます。タスク自身はタスク切り替えを気にしなくてよく、OS が勝手にタスクを切り替えて順次実行していきます。

リスト 13.8 メインループでは `SwitchContext()` を呼ばず `hlt` する (main.cpp)

```
__asm__("cli");
if (main_queue->size() == 0) {
    __asm__("sti\n\thlt");
    continue;
}
```

*2 Cooperative multitasking。ノンプリエンプティブマルチタスクとも呼ばれます。

協調的マルチタスクをやめるので、とりえず TaskB() とメインループから SwitchContext() の呼び出しを削除しておきます。TaskB() からは単に SwitchContext() を呼び出している行を削除しただけですのでソースコードは紹介しません。メインループではリスト 13.8 に示す通り、SwitchContext() を呼ばなくしたことに加えて hlt するようにしました。協調的マルチタスクの場合では何もすることがなくなったら即座に TaskB() に切り替えればよかったのですが、プリエンプティブマルチタスクではそうはいきません。タスク切り替え用タイマがタイムアウトするまで hlt して待ちます。

リスト 13.9 タスク切り替え用タイマの値の定義 (timer.hpp)

```
const int kTaskTimerPeriod = static_cast<int>(kTimerFreq * 0.02);
const int kTaskTimerValue = std::numeric_limits<int>::min();
```

タスク切り替え用のタイマを作るために、タイマの周期とタイムアウトしたときの値を定義しました (リスト 13.9)。タイマは 0.02 秒 (20 ミリ秒) 周期でタイムアウトさせることにしました。1 秒間に 50 回のコンテキスト切り替えが起こることになります。それだけ切り替われば十分でしょう。タイムアウトしたときの値は、なるべく他のタイマで使いたくない値ということで、int 型が取れる最小の値を設定することにしました。

リスト 13.10 InitializeTask() はマルチタスク機能を初期化する (task.cpp)

```
void InitializeTask() {
    current_task = &task_a_ctx;

    __asm__("cli");
    timer_manager->AddTimer(
        Timer{timer_manager->CurrentTick() + kTaskTimerPeriod, kTaskTimerValue});
    __asm__("sti");
}
```

リスト 13.10 にマルチタスク関連の機能を初期化するための InitializeTask() の定義を示します。この関数を呼び出したコンテキストを現在実行中のコンテキストとして current_task に設定します。また、タスク切り替え用タイマの登録を行います。

リスト 13.11 InitializeTask() をメイン関数から呼び出す (main.cpp)

```
InitializeTask();
```

作成した初期化関数はいつもの通りメイン関数から呼び出します (リスト 13.10)。初期化関数を呼ぶとその直後にタスク切り替えが発生し得ますので、TaskB コンテキストの初期化やその他の初期化処理が終わった後に呼び出すのが良いです。ということでメインループの直前に呼ぶようにしました。

タスク切り替え用タイマはその他のタイマとは異なる処理をする必要があります。他のタイマと同じように、タスク切り替え用タイマのタイムアウトを、メイン関数側のメッセージキューに通知するのだとうまく動かないのです。なぜなら、メイン関数がイベントを処理するためには TaskB() の実行を中断してメイン関数コンテキストに切り替えねばなりません、TaskB() はそのイベントなんてお構い

なしに動き続けてしまうからです。これでは一向にタスク切り替え用タイマのタイムアウトイベントが処理されません。

リスト 13.12 Tick() はタスク切り替え用タイマを特別に扱う (timer.cpp)

```
bool TimerManager::Tick() {
    ++tick_;

    bool task_timer_timeout = false;
    while (true) {
        const auto& t = timers_.top();
        if (t.Timeout() > tick_) {
            break;
        }

        if (t.Value() == kTaskTimerValue) {
            task_timer_timeout = true;
            timers_.pop();
            timers_.push(Timer{tick_ + kTaskTimerPeriod, kTaskTimerValue});
            continue;
        }

        Message m{Message::kTimerTimeout};
        m.arg.timer.timeout = t.Timeout();
        m.arg.timer.value = t.Value();
        msg_queue_.push_back(m);

        timers_.pop();
    }

    return task_timer_timeout;
}
```

リスト 13.12 に改造した `TimerManager::Tick()` を示します。まず大きく変わったのは `Tick()` の戻り値です。今までは `void` でしたが `bool` に変えました。タスク切り替え用タイマがタイムアウトした場合は真を返すということにします。

関数の中ほどではタスク切り替え用タイマがタイムアウトした場合の特別な処理を行います。メッセージキューに通知を送るのではなく、`task_timer_timeout` 変数を真に設定します。その後タスク切り替え用タイマを再登録します。この再登録によりタスク切り替え用タイマは周期タイマとして振る舞います。すべてのタイマの処理が終わったら `task_timer_timeout` を返します。

リスト 13.13 タイマ割り込みの処理をして、必要ならタスクを切り替える (timer.cpp)

```
void LAPICTimerOnInterrupt() {
    const bool task_timer_timeout = timer_manager->Tick();
    NotifyEndOfInterrupt();

    if (task_timer_timeout) {
        SwitchTask();
    }
}
```

リスト 13.13 に改造したタイマ割り込みハンドラの実装を示します。この関数はタイマ割り込みが発生するたびに呼ばれます。1 行目で `timer_manager->Tick()` を呼び出すのは今まで通りですが、その戻り値を受け取るようにしました。その戻り値が真であれば、その後 `SwitchTask()` を呼び出してタスク切り替えを行います。`NotifyEndOfInterrupt()` を関数の最後ではなく、`SwitchTask()` をする前に呼び出しているのが気になるかもしれません。

普通、`NotifyEndOfInterrupt()` は割り込みハンドラとなるべく最後に呼ぶのが良いのですが、ここでは最後ではなく中盤で呼び出しています。それは `SwitchTask()` を呼び出してしまうとタスクが切り替わってしまい、`NotifyEndOfInterrupt()` を呼び出さないままになってしまうからです。`NotifyEndOfInterrupt()` を呼ばないとタイマ割り込みが来なくなってしまう、以降ずっとタスクが切り替わらないままになってしまいます。

リスト 13.14 SwitchTask() はタスクを切り替える (task.cpp)

```
alignas(16) TaskContext task_b_ctx, task_a_ctx;

namespace {
    TaskContext* current_task;
}

void SwitchTask() {
    TaskContext* old_current_task = current_task;
    if (current_task == &task_a_ctx) {
        current_task = &task_b_ctx;
    } else {
        current_task = &task_a_ctx;
    }
    SwitchContext(current_task, old_current_task);
}
```

リスト 13.14 に `SwitchTask()` の実装を示します。この関数は次に切り替えるべきタスクを決定し、そのタスクのコンテキストに切り替えます。次のタスクを決めるために、現在実行中のタスクを `current_task` 変数に記憶しておくことにしました。`task_a` と `task_b` のうち、`current_task` と一致する方が現在実行中のタスクということになりますから、残っている方のコンテキストへと切り替えを行います。

リスト 13.15 TaskB() の引数の値を 43 にする (main.cpp)

```
memset(&task_b_ctx, 0, sizeof(task_b_ctx));
task_b_ctx.rip = reinterpret_cast<uint64_t>(TaskB);
task_b_ctx.rdi = 1;
task_b_ctx.rsi = 43;
```

これまでの改造で主な変更は完了です。改造したことが分かりやすいように、TaskB() の引数 data を 42 から 43 に変えておきました (リスト 13.15)。

タイマによるプリエンプティブマルチタスクが実装できたので動かしてみます (図 13.7)。おお、ちゃんと切り替わっていますね。よしよし、とりあえず成功のようです。

図 13.7 プリエンプティブマルチタスクで 2 つのタスクが交互に実行される様子



13.4 マルチタスクの検証 (osbook_day13c)

筆者はちょっと不安になりました。本当にプリエンプティブマルチタスクが実現できているのだろうか。画面の見た目は協調的マルチタスクのときと変わっていないので、本当にタイマによるタスク切り替えなのか検証してみたいです。検証は簡単にできます。タスク切り替え用タイマの周期を目に見えるくらい大きくすればいいのです。

```
const int kTaskTimerPeriod = static_cast<int>(kTimerFreq * 1.0);
```

ということで timer.hpp の kTaskTimerPeriod 定数を 1 秒にしてみます。これだけです。ビルドして実験してみましょう。

図 13.8 1 秒ごとにタスクが切り替わる様子



実験の様子を図 13.8 に示します。Hello World ウィンドウの数値が 899 になっているのは偶然ではなく、1 秒 (100 カウント) ごとにタスクが切り替わるので 99、299、499、699、899 と増えては止まるのを繰り返している証拠です。マウスやキーボードを操作した結果が画面に反映されるのも 1 秒ごとになっています。ちゃんとプリエンティブマルチタスクになっていますね！

ということで検証は成功しました。この後、タイマの周期を元に戻しておきました。

13.5 タスクを増やす (osbook_day13d)

これまで 2 つのタスクでコンテキストスイッチを実験していました。今の状態でタスクを増やそうとすると、タスクの数だけ task_c_ctx などを作ることになってしまい不便ですし、途中でタスクを増やしたり減らしたりするのも困難です。タスクを好きなときに好きなだけ作れるような仕組みを作りたいと思います。

リスト 13.16 1 つのタスクを表す Task クラス (task.hpp)

```
using TaskFunc = void (uint64_t, int64_t);

class Task {
public:
    static const size_t kDefaultStackBytes = 4096;
```

```
Task(uint64_t id);
Task& InitContext(TaskFunc* f, int64_t data);
TaskContext& Context();

private:
    uint64_t id_;
    std::vector<uint64_t> stack_;
    alignas(16) TaskContext context_;
};
```

1つのタスクを表現するのに必要な情報はスタック領域とコンテキスト構造体で足りそうです。ということで**リスト 13.16**に示すように1つのタスクを表すTaskクラスを作りました。タスクは固有のIDとスタック領域、コンテキスト構造体を持ちます。

InitContext()の第1引数の型TaskFuncはぱっと見では関数ポインタに見えませんが、関数ポインタならvoid (*f)(int, int)みたいな複雑な形になるはずだからです。

実はTaskFuncは**using宣言**を利用し、関数型に付けた別名です。**リスト 13.16**のプログラムの1行目にあるusing TaskFunc = ...がそれですね。void (uint64_t, int64_t)は64ビット整数型の引数を2つ取って、戻り値はvoidであるような関数の型を表す書き方です。このように関数の型に別名を付けておくと、TaskFunc* fooのように、あたかも変数へのポインタのような感覚で関数ポインタを作ることができます。もし別名を使わずに関数ポインタを書くvoid (*foo)(uint64_t, int64_t)となります。別名を使ったポインタ定義の方が分かりやすくないですか？

そうそう、今までTaskB()はint型の引数を受け取るようにしていました。今回からTaskFunc型に合わせて引数の型を変えました。関数の引数はどうせRDIやRSIなどの64ビットレジスタで渡しますから、intのような64ビットより小さい(小さい可能性がある)型にしておく理由がありませんからね。

リスト 13.17 Taskクラスのコンストラクタ (task.cpp)

```
Task::Task(uint64_t id) : id_{id} {
}
```

リスト 13.17にTaskクラスのコンストラクタを示します。指定されたタスクIDをid_に設定するだけのシンプルなコンストラクタです。stack_やcontext_に値を設定するのは、次に紹介するInitContext()の役目です。

リスト 13.18 InitContext()はコンテキストの初期値を設定する (task.cpp)

```
Task& Task::InitContext(TaskFunc* f, int64_t data) {
    const size_t stack_size = kDefaultStackBytes / sizeof(stack_[0]);
    stack_.resize(stack_size);
    uint64_t stack_end = reinterpret_cast<uint64_t>(&stack_[stack_size]);

    memset(&context_, 0, sizeof(context_));
    context_.cr3 = GetCR3();
    context_.rflags = 0x202;
    context_.cs = kKernelCS;
    context_.ss = kKernelSS;
    context_.rsp = (stack_end & ~0xflu) - 8;
```

```

context_.rip = reinterpret_cast<uint64_t>(f);
context_.rdi = id_;
context_.rsi = data;

// MXCSR のすべての例外をマスクする
*reinterpret_cast<uint32_t*>(&context_.fxsave_area[24]) = 0x1f80;

return *this;
}

```

タスク用のコンテキスト構造体に初期値を設定する関数 `InitContext()` を実装しました (**リスト 13.18**)。この関数で設定した値は、このタスクが最初に `SwitchContext()` によって起動されたときに読み取られます。今までメイン関数でやっていた処理をほとんどそのまま持ってきていただけですから難しくはないでしょう。

タスク用スタックの大きさは `Task::kDefaultStackBytes` で調整できるようにしました。今のところ 4096 バイトとしています。スタックのアライメント制約を満たすために、RSP の初期値 (`context_.rsp`) はわざと 16 バイト境界から 8 バイトだけずれた位置に調整しておきます。

リスト 13.19 `Context()` はコンテキスト構造体への参照を返す (`task.cpp`)

```

TaskContext& Task::Context() {
    return context_;
}

```

リスト 13.19 に `Context()` の実装を示します。この関数はタスクのコンテキスト構造体への参照を返します。値ではなく参照であることが大事です。なぜなら、コンテキスト構造体のアドレスを `SwitchContext()` に渡す必要があるからです。`SwitchContext()` はコンテキストを切り替える際、そのアドレスが指すメモリ領域にレジスタ値を保存するのです。

リスト 13.20 `TaskManager` クラスは複数のタスクを管理する (`task.hpp`)

```

class TaskManager {
public:
    TaskManager();
    Task& NewTask();
    void SwitchTask();

private:
    std::vector<std::unique_ptr<Task>> tasks_{};
    uint64_t latest_id_{0};
    size_t current_task_index_{0};
};

extern TaskManager* task_manager;

```

1 つのタスクを表すクラスを実装したので、次は複数のタスクを管理する `TaskManager` クラスを実装していきます (**リスト 13.20**)。Task と `TaskManager` の関係は `Layer` と `LayerManager` の関係に似ています。TaskManager が Task を生成する役目と、生成した Task のインスタンスを保存して

おく役目を持ちます。また、TaskManager にタスク切り替えの機能 (SwitchTask()) を持たせることにします。

リスト 13.21 コンストラクタはタスクを 1 つだけ生成する (task.cpp)

```
TaskManager::TaskManager() {
    NewTask();
}
```

リスト 13.21 に TaskManager のコンストラクタを示します。コンストラクタに明示してはいませんが、3 つのメンバ変数 tasks_、latest_id_、current_task_index_ はヘッダファイルで指定した初期値を使って初期化されます。これらの初期化に続き、コンストラクタは NewTask() メソッドを呼び出してタスクを 1 つ生成します。このタスクは現在実行中のコンテキスト、すなわち NewTask() をまさに今実行しようとしているコンテキストに対応したタスクとなります。

リスト 13.22 NewTask() は tasks_ の末尾に新たなタスクを追加する (task.cpp)

```
Task& TaskManager::NewTask() {
    ++latest_id_;
    return *tasks_.emplace_back(new Task{latest_id_});
}
```

リスト 13.22 に NewTask() メソッドの実装を示します。最新のタスク ID を表す latest_id_ をインクリメントし、その値を使って Task クラスのインスタンスを生成し、tasks_ の末尾に追加します。このメソッドの主機能はタスクを生成して tasks_ に登録することですが、ついでに生成したタスクへの参照を返すようにしました。参照を返すことで NewTask().InitContext(...) のようにメソッドチェーンで書いて便利です。

リスト 13.23 SwitchTask() は次のタスクへと切り替える (task.cpp)

```
void TaskManager::SwitchTask() {
    size_t next_task_index = current_task_index_ + 1;
    if (next_task_index >= tasks_.size()) {
        next_task_index = 0;
    }

    Task& current_task = *tasks_[current_task_index_];
    Task& next_task = *tasks_[next_task_index];
    current_task_index_ = next_task_index;

    SwitchContext(&next_task.Context(), &current_task.Context());
}
```

タスクを生成するだけでは足りません。タスクを切り替える機能が必要です。**リスト 13.23** にタスクを切り替える SwitchTask() メソッドの実装を示します。このメソッドは、現在実行中のタスクとその次のタスクを取得し、次のタスクが持つコンテキストへと実行を切り替えます。

現在実行中のタスクは tasks_[current_task_index_] で表すことにして、タスクを切り替え

るたびに `current_task_index_` をインクリメントします。こうしておけば `SwitchTask()` を呼ぶたびに次々にタスクを切り替えていくことができます。

リスト 13.24 TaskManager のインスタンスを生成する (task.cpp)

```
void InitializeTask() {
    task_manager = new TaskManager;

    __asm__("cli");
    timer_manager->AddTimer(
        Timer{timer_manager->CurrentTick() + kTaskTimerPeriod, kTaskTimerValue});
    __asm__("sti");
}
```

`InitializeTask()` の中で `TaskManager` のインスタンスを生成し、グローバル変数 `task_manager` に設定するようにしました (リスト 13.24)。`InitializeTask()` を呼び出した後から `task_manager` を使えるようになります。

リスト 13.25 マルチタスク機能を初期化して 3 つのタスクを生成する (main.cpp)

```
InitializeTask();
task_manager->NewTask().InitContext(TaskB, 45);
task_manager->NewTask().InitContext(TaskIdle, 0xdeadbeef);
task_manager->NewTask().InitContext(TaskIdle, 0xcafebabe);
```

リスト 13.25 にメイン関数側の改造を示します。今までここに書いていた `TaskB` コンテキストに関するプログラムを削除して、その代わりに `task_manager->NewTask()` を呼び出すようにしました。タスクの管理を `std::vector` でやることにしたおかげで、タスクを好きなだけ作れるようになりました。試しに `TaskIdle()` という関数を新たに作り、その関数を持つタスクを 2 つ生成してみます。

同じ関数を実行するタスクが 2 つ生成され、これらは異なるコンテキストを持ちます。同じ関数を実行するのになぜコンテキストが異なるかという、2 つのタスクがそれぞれ異なるスタックとコンテキスト構造体を持つからです。実行する機械語はまったく同じですが、実行する際のスタックやレジスタ値は分かれていますのでそれぞれ独立して動作します。これはよくあることです。例えば同じテキストエディタを 2 つ起動させて、それぞれ異なるファイルを開くことがありますよね。その場合も実行ファイルは共通でもコンテキストが異なるため、それぞれ独立してファイルを編集できるわけです。

リスト 13.26 TaskIdle() はずっと hlt するだけ (main.cpp)

```
void TaskIdle(uint64_t task_id, int64_t data) {
    printk("TaskIdle: task_id=%lu, data=%lx\n", task_id, data);
    while (true) __asm__("hlt");
}
```

`TaskIdle()` はリスト 13.26 のような関数です。Idle とは「暇な」という意味があり、その名の通りタスク ID とデータを印字した後はずっと `hlt` し続けます。なんとも無駄な関数ですが、実験用なのでなるべくシンプルにしました。

図 13.9 4つのタスクが並行して動いている様子



改造が終わったらビルドして実行してみます (図 13.9)。どうでしょうか？ 左上には「TaskIdle」という表示が2行出ていますし、TaskBWindow の表示もちゃんと更新されていますし、マウスやキーボードも使えます。ということで4つのタスクがしっかりと動いているようです。

マウスを動かしているとなんとなく引っかかりを感じませんか？ 筆者は感じました。よく分からない場合はタスクをもっと増やして (TaskIdle を実行するタスクをたくさん作れば良いです) 実験してみるとはっきり体感できるでしょう。この現象は、各タスクが0.02秒ずつCPU時間を消費するため、タスクの増加とともにメインタスクまで処理が回ってくるのに時間がかかるようになり、マウスのイベント処理が遅れるのが原因です。マウスを動かすという重要な処理をしなければならないときに悠長にTaskIdle()の実行などとしていられないのですが、現状ではそうなってしまっています。次章では、この問題に対処してみようと思います。

第 14 章

マルチタスク (2)

カーネル空間で複数のタスクが切り替わるようになり、OS の主要な機能である「CPU 時間の分配」ができるようになりました。ただし各タスクに強制的に CPU 時間が割り振られてしまい、すべきことがないタスクは CPU 時間を浪費するしかない状況でした。この章ではタスクがやるべきことがない場合にスリープする機能や、タスクの優先順位を付ける機能を作ります。最後まで行くと、タスクが増えても無駄に CPU 時間が消費されなくなり、マウスがカクカクしにくくなります。

14.1 スリープしてみる (osbook_day14a)

前章では、タスクをいくつでも生成できるようにして `hlt` するだけの簡単なタスクを増やしたところ、マウスがカクカクするという問題が発生しました。これは、各タスクに平等に 0.02 秒ずつ CPU 時間が割り当てられてしまい、マウスのイベントを処理するメインタスク（タスク ID=1）がたまにしか実行されなくなってしまったからです。これを解決するために 2 つのやり方があります。

- ・やるべきことがなくなったらタスクをスリープ状態にして、CPU 時間が割り当てられないようにする
- ・マウスのイベントを処理するタスクの優先度を上げて、マウスが動いたら優先的にそのタスクを実行する

前者は「CPU という資源を無駄にしない」ということ、後者は「素早く実施すべき処理は素早く実施する」ということです。前者はいいとして後者をちょっと詳しく説明します。OS がやるべき処理を、時間がかかるがゆっくりやれば良い大きな処理と、時間はかからないが今すぐやるべき処理に分けます。ゆっくりやれば良い処理は CPU が暇をしているときに少しずつ処理をすることにします。今すぐやるべき処理が発生したら、ゆっくりやれば良い処理を中断して今すぐやるべき処理に着手します。これは、通常のプログラムを実行中に割り込みが発生する、という状況と似ています。これをタスク管理のレベルでやるのです。

ちなみに「時間がかかるのに今すぐやるべき処理」が発生してしまうとどうにもならなくなります。昔々、Java のガベージコレクションが発動するとプログラムが数秒から数十秒止まってしまうことがありました。ガベージコレクションは、もうこれ以上メモリの空きがないというときに発動するため、回収すべきメモリが大量にあり（＝時間がかかる）、しかも今すぐやらねばならない処理のため、仕方なく Java プログラムを止めるという最悪の選択がされてしまうのです。

すみません。話が脱線しました。2 つの手法を紹介しましたが、普通は両方の手法を組み合わせます。本書では、まずはスリープ機能から実現していこうと思います。

スリープ状態とはタスクの状態の 1 つです。タスクは生成されると実行待ちの列（ランキューと呼ばれます）に追加され、**実行可能状態**となります。その待機列にあるタスクは順次実行されていきます。CPU が待機列からタスクを取り出して実行しているとき、そのタスクは**実行状態**です。0.02 秒経ってコンテキストが切り替わり、また待機列に戻って実行可能状態になります。このように、タスクは実行可能状態と実行状態を頻繁に移り替わって少しずつ処理が進みます。

実行可能状態または実行状態にあるタスクを待機列から外すことをスリープと呼び、待機列から外れている状態を**スリープ状態**と言います。スリープ状態にあるタスクは何らかの操作で待機列に再び戻るまでは CPU によって実行されることがありません。このような仕組みを実装してみたいと思います。作るべきは待機列となるランキュー、タスクをスリープさせる（ランキューからタスクを削除する）機能、タスクを起こす（スリープ状態だったタスクをランキューに追加し直す）機能です。

リスト 14.1 TaskManager にランキューとメソッドを追加した (task.hpp)

```
class TaskManager {
public:
    TaskManager();
    Task& NewTask();
    void SwitchTask(bool current_sleep = false);

    void Sleep(Task* task);
    Error Sleep(uint64_t id);
    void Wakeup(Task* task);
    Error Wakeup(uint64_t id);

private:
    std::vector<std::unique_ptr<Task>> tasks_{};
    uint64_t latest_id{0};
    std::deque<Task*> running_{};
};
```

リスト 14.1 に改造した TaskManager の定義を示します。まず、Sleep() と Wakeup() メソッドが増えました。それぞれ 2 つずつあるのは、タスク ID でもタスク変数のポインタでも指定できるようにするためです。後で実装を見ればやりたかったことが分かるかと思います。メンバ変数から current_task_index_ を削除して、代わりに running_ を増やしました。今までは tasks_[current_task_index_] が現在実行中のタスクということにしていますが、これからは running_[0] を現在実行中のタスクということにします。

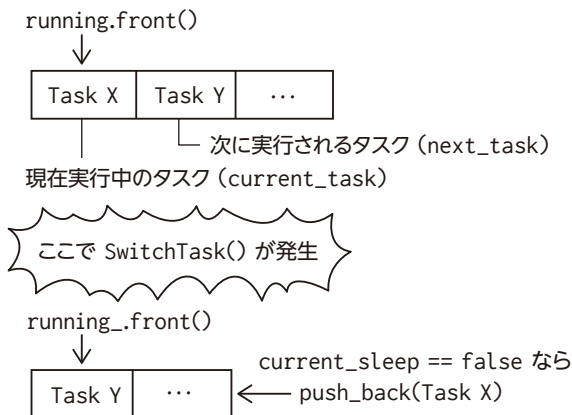
リスト 14.2 SwitchTask() はランキュー上のタスクを順に切り替える (task.cpp)

```
void TaskManager::SwitchTask(bool current_sleep) {
    Task* current_task = running_.front();
    running_.pop_front();
    if (!current_sleep) {
        running_.push_back(current_task);
    }
    Task* next_task = running_.front();

    SwitchContext(&next_task->Context(), &current_task->Context());
}
```

メンバ変数を変更したのに合わせて SwitchTask() を改造しました (リスト 14.2)。まず大きく変わったのは、今まで tasks_ 上のタスクを順に切り替えていたのをやめて、running_ 上にあるタスクを扱うようにした点です。tasks_ はすべての状態のタスクが格納されている配列で、running_ はランキュー、すなわち実行可能状態のタスクだけが格納されたキュー (FIFO) です。実行可能状態にあるタスクはすべて running_ に格納されています (そうなるように今から作っていきます) から、順に取り出して切り替えていけばいいのです。

図 14.1 SwitchTask() によるランキュー (running_) の変化



`SwitchTask()` は `running_` の先頭にあるタスクを取り出して末尾に改めて追加します。その様子は図 14.1 のようになります。ランキューの先頭のタスクが TaskX だとすると、TaskX はランキューの先頭から削除された後、`SwitchTask()` の引数 `current_sleep` が偽であれば末尾に追加されます。`current_sleep` が真の場合、TaskX はランキューの末尾に追加されません。すなわち、その後何回 `SwitchTask()` が呼ばれても TaskX は実行されなくなります。TaskX はスリープするということです。

ちなみに引数 `current_sleep` は、`task.hpp` の中でデフォルト値が与えられています。 `void SwitchTask(bool current_sleep = false);` というように。C++ では関数の引数にデフォルト値を与えられますので、今まで `task_manager->SwitchTask();` と無引数で呼び出していた部分を修正せずに新しい引数を追加できました。コードの修正が少ないって素晴らしい。

リスト 14.3 Sleep() は指定されたタスクをランキューから外す (task.cpp)

```
void TaskManager::Sleep(Task* task) {
    auto it = std::find(running_.begin(), running_.end(), task);

    if (it == running_.begin()) {
        SwitchTask(true);
        return;
    }

    if (it == running_.end()) {
        return;
    }

    running_.erase(it);
}
```

リスト 14.3 に指定したタスクをスリープさせる `Sleep()` の実装を示します。この関数はまずランキュー `running_` から指定されたタスクを探します。タスクがランキューの先頭にある場合 (`it == running_.begin()`)、そのタスクは現在実行中ということです (タスクが自分自身をスリープさせようとする場合にそうなります)。現在実行中のタスクをスリープさせるには、タスク切り替えをしなければなりません。ということで、先ほど改造した `SwitchTask()` を使ってタスクを切り替えます。

SwitchTask() の引数に真を渡しているため、そのタスクはランキューから外れてスリープ状態になります。

指定されたタスクが見つからない場合 (it == running_.end()) は、そのタスクはすでにランキューから外れている (= スリープ状態) を意味しますので何もせず終了します。それ以外のパターンではタスクはランキューの先頭以外に存在しますので、running_.erase(it) としてタスクをランキューから外します。この場合、タスク切り替えは不要ですね。

リスト 14.4 Wakeup() は指定されたタスクをランキューに追加する (task.cpp)

```
void TaskManager::Wakeup(Task* task) {
    auto it = std::find(running_.begin(), running_.end(), task);
    if (it == running_.end()) {
        running_.push_back(task);
    }
}
```

リスト 14.4 に指定したタスクを起こす (実行可能状態にする) Wakeup() を示します。この関数は指定されたタスクがまだランキューに存在しなければ末尾に追加します。非常に簡単ですね。末尾に追加しておけばそのうち実行されますから、タスク切り替えなどを明示的にする必要はありません。

リスト 14.5 ランキューに現在のコンテキストを表すタスクを追加 (task.cpp)

```
TaskManager::TaskManager() {
    running_.push_back(&NewTask());
}
```

初期状態でランキューにはどんなタスクも存在しませんので、タイマ割り込みによって SwitchTask() が呼ばれると困ったことになります。ということで、リスト 14.5 のように TaskManager の初期化と同時にランキューに最初のタスクを追加しておくことにします。ここで NewTask() で作成したタスクは TaskManager のコンストラクタを呼び出したコンテキスト、つまりメイン関数を実行しているコンテキストに対応するタスクを表すのでしたね。コンストラクタを実行し終わった時点で、ランキューにはタスクが 1 つだけ追加された状態になります。

リスト 14.6 タスク ID で指定できる Sleep() (task.cpp)

```
Error TaskManager::Sleep(uint64_t id) {
    auto it = std::find_if(tasks_.begin(), tasks_.end(),
        [id](const auto& t){ return t->ID() == id; });
    if (it == tasks_.end()) {
        return MAKE_ERROR(Error::kNoSuchTask);
    }

    Sleep(it->get());
    return MAKE_ERROR(Error::kSuccess);
}
```

リスト 14.7 タスク ID で指定できる Wakeup() (task.cpp)

```
Error TaskManager::Wakeup(uint64_t id) {
    auto it = std::find_if(tasks_.begin(), tasks_.end(),
        [id](const auto& t){ return t->ID() == id; });
    if (it == tasks_.end()) {
        return MAKE_ERROR(Error::kNoSuchTask);
    }

    Wakeup(it->get());
    return MAKE_ERROR(Error::kSuccess);
}
```

リスト 14.6 と **リスト 14.7** に、タスク ID で指定できるバージョンの Sleep() と Wakeup() の実装を示します。両者ともほとんど同じ処理になっていて、まず tasks_ をタスク ID で検索してタスクインスタンスを得て、それを使って Sleep() または Wakeup() を呼び出します。

リスト 14.8 新たに Task クラスに追加したメソッド群 (task.cpp)

```
uint64_t Task::ID() const {
    return id_;
}

Task& Task::Sleep() {
    task_manager->Sleep(this);
    return *this;
}

Task& Task::Wakeup() {
    task_manager->Wakeup(this);
    return *this;
}
```

毎回 task_manager->Sleep(task) と書くのは面倒です。もっと簡潔に task->Sleep() と書けるようにしたいです。ということで **リスト 14.8** のようにいくつかのメソッドを Task クラスに追加しました。

リスト 14.9 生成したタスクを実行可能状態にする (main.cpp)

```
InitializeTask();
const uint64_t taskb_id = task_manager->NewTask()
    .InitContext(TaskB, 45)
    .Wakeup()
    .ID();
task_manager->NewTask().InitContext(TaskIdle, 0xdeadbeef).Wakeup();
task_manager->NewTask().InitContext(TaskIdle, 0xcafebabe).Wakeup();
```

TaskManager::NewTask() の中で、タスクを生成するだけでなくランキューに追加するように改造することもできましたが、あえてそうしませんでした。深い理由はありませんが。ということで **リスト 14.9** のように、NewTask() で生成したタスクに対し Wakeup() してやることにします。そうしな

いと、ただタスクが生成されるだけで実行されません。

リスト 14.10 TaskB を眠らせたり起こしたりする (main.cpp)

```
case Message::kKeyPush:
    InputTextWindow(msg.arg.keyboard.ascii);
    if (msg.arg.keyboard.ascii == 's') {
        printk("sleep TaskB: %s\n", task_manager->Sleep(taskb_id).Name());
    } else if (msg.arg.keyboard.ascii == 'w') {
        printk("wakeup TaskB: %s\n", task_manager->Wakeup(taskb_id).Name());
    }
    break;
```

最後にキー押下イベントの処理部分にプログラムを書き加えて、S キーを押すと TaskB を眠らせ、W キーを押すと起こすようにしてみました (リスト 14.10)。処理は単純です。さて、これで実行してみるとどうなるでしょうか？ おお、S と W を押すと図 14.2 のように TaskB が止まったり動いたりします！ 成功です！

図 14.2 S キーで TaskB をスリープさせている様子



14.2 イベントが来たら起床する (osbook_day14b)

マウスの移動を高速化するために前の節で紹介した2つの方法の1つを終えました。以降ではもう片方の「マウスのイベントを処理するタスクの優先度を上げて、マウスが動いたら優先的にそのタスクを実行する」というのをやってみます。ただ、マウスだけを特別視するのではなく「優先度の高い処理が発生したら先にそっちを処理する」というような、より一般的な仕組みを作ろうと思います。

実現の方法にはいろいろあります。例えば優先度が高いタスクがより多くの実行時間をもらえる仕組みはどうでしょうか？ 優先度 10 (最高) のタスクは 0.1 秒、優先度 1 のタスクは 0.01 秒、という感じで。この方法は重たい処理がいくつかあるときに、それぞれの処理をどのくらいの割合でこなすかを調整するには適しています。しかし、今やりたいのはマウスやウィンドウを動かすという比較的軽い処理にできるだけ早く着手することでカクカクを防ぐことです。

それでは、優先度ごとにタスクを分類して、高優先のタスクがある間はそれより低い優先度のタスクが実行されないようにしたらどうでしょう。マウスイベントを処理するタスク（つまりメインタスク）は他のタスクより優先度は高いけれども普段はスリープ状態になっているため TaskB() や TaskIdle() が実行されます。ひとたびマウスが動きマウスイベントが発生すると、メインタスクが起こされて実行可能状態となります。実行可能状態の中でメインタスクの優先度が最も高いため、他のタスクはメインタスクがスリープするまでは実行が中断されたままとなります。

この方法は良さそうな感じがします。マウスイベントの処理は比較的短時間ですので、メインタスクが起床しっぱなしになって他のタスクが動かない、ということはないはずです。もちろん、メインタスクが重たい処理をするようになってはいけません。高優先のタスクは短時間で処理を終える責任があります。

タスクを優先度ごとに分類して普段は高優先のタスクをスリープさせておき、イベントが発生したら起こすということを実現するためには、イベントが発生した際にスリープしているタスクを起こす機能が必要です。具体的に表現すると、main_queue にメッセージを追加したときにメインタスクがスリープしていたら起こす、という機能です。この機能はメインタスクに限らず、他のタスクを起こすのにも使えて便利だと思います。ということでタスクの標準機能として実装してみようと思います。

リスト 14.11 Task クラスにメッセージキューを追加した (task.hpp)

```
private:
    uint64_t id_;
    std::vector<uint64_t> stack_;
    alignas(16) TaskContext context_;
    std::deque<Message> msgs_;
```

まず Task クラスにメッセージキュー msgs_ を付け足しました（リスト 14.11）。Task クラスに追加しましたので、メインタスクだけでなくすべてのタスクに main_queue と同様の機能が備わることになります。これに合わせて、main.cpp から main_queue の定義を削除しました。

リスト 14.12 メッセージキューにメッセージを追加し、寝ていれば起こす (task.cpp)

```
void Task::SendMessage(const Message& msg) {
    msgs_.push_back(msg);
    Wakeup();
}
```

タスクのメッセージキューにメッセージを追加する `SendMessage()` メソッドを **リスト 14.12** に示します。このメソッドはメッセージをキューに追加した後 `Wakeup()` を呼び出すことで、タスクが寝ていた場合に起こします。`Wakeup()` はタスクが起きていたときに呼び出しても害はありません。このメソッドを使ってタスク間でメッセージを送り合うようにすれば、メッセージがないときに安心してスリープできます。

リスト 14.13 メッセージキューからメッセージを 1 つ取り出す (task.cpp)

```
std::optional<Message> Task::ReceiveMessage() {
    if (msgs_.empty()) {
        return std::nullopt;
    }

    auto m = msgs_.front();
    msgs_.pop_front();
    return m;
}
```

リスト 14.13 にメッセージを取得する `ReceiveMessage()` メソッドを示します。このメソッドの戻り値の型 `std::optional<Message>` は無効値付きの `Message` 型です。無効値とは無効であることを明示的に表す値のことです。例えば `int` 型を返す関数で `-1` を無効値にすることがありますが、`-1` がエラーを表すのか有効な値なのかは `int` という型からは分からず、その関数の仕様書を読まなければなりません。`std::optional` を使うと好きな型を無効値 `std::nullopt` 付きの型に拡張することができます。ということで、`ReceiveMessage()` はメッセージキューが空の場合に無効値を、そうでなければメッセージキューの先頭の値を返す関数となります。`std::optional` は「**9.3 重ね合わせ処理の原理**」でも登場しました。

リスト 14.14 メインタスクのメッセージキューが空ならスリープする (main.cpp)

```
__asm__("cli");
auto msg = main_task.ReceiveMessage();
if (!msg) {
    main_task.Sleep();
    __asm__("sti");
    continue;
}
```

リスト 14.14 にメインループ内でメッセージを受信する処理を示します。先ほど作った `ReceiveMessage()` を使ってメッセージを 1 つ取り出してきました。メッセージキューが空のとき `msg` は無効値になりますから、それを `if (!msg)` というように検査します。無効値だったらスリープします。ちなみに自分自身をスリープさせますから `main_task.Sleep()` から戻ってこないまま次のタスクに切り替わります。

リスト 14.15 `main_task` に現在のタスクを取得して設定する (main.cpp)

```
InitializeTask();
Task& main_task = task_manager->CurrentTask();
```

リスト 14.14 のプログラムで使っている `main_task` 変数は、**リスト 14.15** のようにメインループに入る前に作っている参照変数です。`task_manager->CurrentTask()` は現在実行中の（すなわち、このメソッドを実行している）タスクを返すメソッドです。

リスト 14.16 `CurrentTask()` は現在実行中のタスクを返す (task.cpp)

```
Task& TaskManager::CurrentTask() {
    return *running_.front();
}
```

リスト 14.16 に `CurrentTask()` の実装を示します。ランキューの先頭を返すだけの非常に単純なメソッドです。ランキューの先頭には現在実行中のタスク、すなわち `CurrentTask()` を呼び出したタスク（`Task` へのポインタ）が設定されているのでしたね。

リスト 14.17 `SendMessage()` を使ってメッセージを送る (timer.cpp)

```
if (t.Value() == kTaskTimerValue) {
    task_timer_timeout = true;
    timers_.pop();
    timers_.push(Timer{tick_ + kTaskTimerPeriod, kTaskTimerValue});
    continue;
}

Message m{Message::kTimerTimeout};
m.arg.timer.timeout = t.Timeout();
m.arg.timer.value = t.Value();
task_manager->SendMessage(1, m);

timers_.pop();
```

`main.cpp` から `main_queue` を削除しましたので、これまで `main_queue` に直接メッセージを追加していたプログラムを修正する必要があります。例えば `TimerManager::Tick()` の中で `main_queue.push_back()` を呼び出していた部分を **リスト 14.17** のように改造しました。`task_manager->SendMessage(1, m)` はタスク ID が 1 のタスクにメッセージ `m` を送信し、寝ていれば起こすためのメソッドです。

リスト 14.18 `SendMessage()` は指定されたタスクにメッセージを送る (task.cpp)

```
Error TaskManager::SendMessage(uint64_t id, const Message& msg) {
    auto it = std::find_if(tasks_.begin(), tasks_.end(),
                          [id](const auto& t){ return t->ID() == id; });
    if (it == tasks_.end()) {
        return MAKE_ERROR(Error::kNoSuchTask);
    }

    (*it)->SendMessage(msg);
    return MAKE_ERROR(Error::kSuccess);
}
```

`TaskManager::SendMessage()` の実装をリスト 14.18 に示します。指定されたタスク ID を持つタスクを `tasks_` から探してきて、そのタスクの `SendMessage()` を呼び出します。メインタスクは常にタスク ID が 1 ですので、メインタスクにメッセージを送るときに大変便利に使えます。

リスト 14.19 `SendMessage()` を使ってメッセージを送る—— その 2 (`interrupt.cpp`)

```
__attribute__((interrupt))
void IntHandlerXHCI(InterruptFrame* frame) {
    task_manager->SendMessage(1, Message{Message::kInterruptXHCI});
    NotifyEndOfInterrupt();
}
```

`TaskManager::SendMessage()` を使うように改造した例をもう 1 つリスト 14.19 に示します。今までここは `msg_queue->push_back(...)` と書いてありましたが、`task_manager->SendMessage()` を使うように変えました。ほかにも `main_queue` へのポインタを受け取る箇所がありました。同様に修正しました。

リスト 14.20 タスク機能の初期化後に xHCI を初期化する (`main.cpp`)

```
usb::xhci::Initialize();
InitializeKeyboard();
InitializeMouse();

char str[128];

while (true) {
```

リスト 14.20 に示すように、xHCI の初期化処理を `InitializeTask()` の実行の後ろに移動させました。なぜなら `usb::xhci::Initialize()` を実行するとすぐに割り込みが発生するからです。割り込みハンドラ `IntHandlerXHCI()` の中でグローバル変数 `task_manager` を使うため、先にタスク機能が初期化されている必要があるのです。

さて、必要な改造は終わりました。実行してみます。スクリーンショットでは全然変化が伝わらないため載せませんが、動いている様子を見るとメインウィンドウのカウンタが 50 ずつ増えるようになりました。さらに、増えるタイミングがテキストボックスのカーソルの点滅と同期されているように見えます。それはそのはずで、メインタスクは 50 カウント周期のカーソルタイマのタイムアウトによって起こされ、表示処理の後すぐスリープするようになったため、先ほどのような変化がおきたのです。

試しにマウスを動かしたりキーを押したりすると、そのタイミングでメインウィンドウのカウンタが更新されるのが分かります。マウス移動やキー押下のイベントが `SendMessage()` によりメインタスクに通知されるたびにメインタスクが起こされ、メインループが 1 周している証拠です。

14.3 性能測定

メインタスクが暇なときにスリープするようにしたため、スリープしない場合にメインタスクに割り当てられるはずだった CPU 時間が他のタスクに分配されているはずです。ということは、TaskB のカウンタがより速く回ると考えられます。ちょっと測ってみましょう。

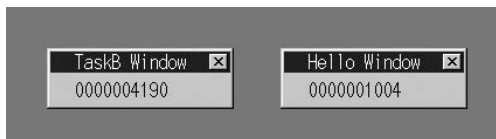
測り方はとても原始的なやり方でやってみました。メインウィンドウのカウンタが 1000 (=10 秒) 程度になったときにスクリーンショットを撮り、目で見るというやり方です。

図 14.3 メインタスクを常に動作させたとき (osbook_day14a の内容)



まず、メインタスクが常に動作しているバージョン（つまり osbook_day14a）が図 14.3 です。メインウィンドウのカウンタが 1000 を示すころ、TaskB のカウンタは 2991 となっています。

図 14.4 メインタスクをスリープさせたとき (osbook_day14b の内容)



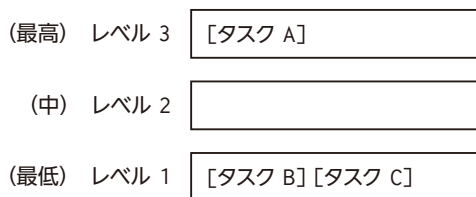
一方、メインタスクをスリープさせるバージョン (osbook_day14b) が図 14.4 です。メインのカウンタが 1004 を示すころ、TaskB のカウンタは 4190 となりました。1000 と 1004 で多少ずれがあるもの、誤差だと思うと TaskB のカウンタの性能は $4190/2991 \approx 1.40$ 倍だけ向上したことになりますね。

day14a では常に 4 つのタスクが動いており、各タスクに均等に時間が割り当てられているため TaskB は 10 秒のうち 2.5 秒だけ CPU を使いました。day14b ではメインタスクはほとんどの期間でスリープしているため、仮にメインタスクが消費する CPU 時間を 0 秒とすると、残り 3 つのタスクは 3.33 秒ずつ CPU を使ったことになります。 $3.33/2.5 \approx 1.33$ 倍です。先ほど計算した 1.40 倍とちょっと違いますが、まあこんなものだと思うことにしましょう。メインタスクをスリープしたことで、TaskB が処理できる時間が増えたのは確実ですね。

14.4 タスクに優先度を付ける (osbook_day14c)

イベントが来たらスリープ中のタスクを起こすことができるようになりました。いよいよ本来やりたかった、優先度ごとにタスクを分類して、高優先のタスクがある間はそれより低い優先度のタスクが実行されない仕組みを作っていきます。優先度と言うといろいろな解釈がありますので、ここでは「レベル」と呼ぶことにします。

図 14.5 タスクをレベルに分ける



レベルを何段階に分けるかは自由ですがとりあえず 3 段階の場合を例に説明します。図 14.5 は 3 段階にレベルを分けた場合の概念図です。各レベルはそれぞれがランキューになっていて、実行可能タスクが登録されています。3 つの実行可能タスクのうち、タスク A が最も高いレベルになっています。したがって、これら 3 つのタスクのうち実行されるのはタスク A だけです。タスク B と C はレベルがタスク A より低いため、タスク A がスリープするかレベルが 1 に下がるまでは実行されません。

通常、レベルが高いタスクは短時間で処理を終えてスリープすることが期待されます。レベルの高いタスクがずっと動き続けているとその他のタスクが一切実行されなくなるからです。仮にタスク A がスリープしたとすると、3 つのタスクのうちタスク B と C が最も高いレベル 1 となります。したがってタスク B と C が 0.02 秒ずつ切り替わって実行されるようになります。

リスト 14.21 TaskManager クラスにレベル別ランキューを導入 (task.hpp)

```
class TaskManager {
public:
    // level: 0 = lowest, kMaxLevel = highest
    static const int kMaxLevel = 3;

    TaskManager();
    Task& NewTask();
    void SwitchTask(bool current_sleep = false);

    void Sleep(Task* task);
    Error Sleep(uint64_t id);
    void Wakeup(Task* task, int level = -1);
    Error Wakeup(uint64_t id, int level = -1);
    Error SendMessage(uint64_t id, const Message& msg);
    Task& CurrentTask();

private:
    std::vector<std::unique_ptr<Task>> tasks_{};
    uint64_t latest_id_{0};
    std::array<std::deque<Task*>, kMaxLevel + 1> running_{};
    int current_level_{kMaxLevel};
    bool level_changed_{false};

    void ChangeLevelRunning(Task* task, int level);
};
```

レベルの概念を説明したところで、さっそくリスト 14.21 に示すように TaskManager クラスにレベルを実装してみました。まず注目するのは running_ 変数です。これは今まで1つのキューでしたが、今はレベルの数だけ要素を持つ配列にしてみました。レベルは 0 から kMaxLevel まで作ることにしました。現状では kMaxLevel は 3 なので、4 段階のレベルがあることになります。足りなくなったら増やしましょう。

current_level_ 変数は現在実行中のタスクが属するレベルを表します。初期値を kMaxLevel としているのはメインタスクを最高レベルとするためです。TaskManager のコンストラクタの中でこの値を利用します。図 14.5 に示す状況である場合、current_level_ は 3 となります（タスク A が実行中のため）。

level_changed_ 変数は、次のタスク切り替え時に実行レベルを見直す必要があることを知らせるために使うつもりです。この変数を真にしておくと、次に SwitchTask() を実行した際に current_level_ の見直しが行われます。詳しくは SwitchTask() を改造するときに説明します。

リスト 14.22 Task クラスはレベルと実行状態を記憶する (task.hpp)

```
private:
    uint64_t id_;
    std::vector<uint64_t> stack_;
    alignas(16) TaskContext context_;
    std::deque<Message> msgs_;
    unsigned int level_{kDefaultLevel};
    bool running_{false};

    Task& SetLevel(int level) { level_ = level; return *this; }
    Task& SetRunning(bool running) { running_ = running; return *this; }

    friend TaskManager;
```

リスト 14.22 に示すように、Task クラスにそのタスクの現在のレベルを表す変数 level_ と実行状態を表す変数 running_ を加えました。running_ はタスクが実行状態または実行可能状態であれば真となります。

これら 2 つの変数は主として TaskManager がタスクの管理をしやすいするために利用するものですので、書き換えるのは TaskManager だけに許可したいです。そこで書き換えるためのメソッド SetLevel() と SetRunning() を非公開 (private) とし、friend TaskManager; と宣言しておきます。こうすることで、Task クラス以外では TaskManager クラスだけが 2 つのメソッドを利用できるようにします。

リスト 14.23 メインタスクのレベルは current_level_ の値を使う (task.cpp)

```
TaskManager::TaskManager() {
    Task& task = NewTask()
        .SetLevel(current_level_)
        .SetRunning(true);
    running_[current_level_].push_back(&task);
}
```

TaskManager のコンストラクタをリスト 14.23 のように改造しました。コンストラクタを呼び出しているコンテキスト（メイン関数を実行しているコンテキスト）に対応するタスクを生成して、そのタスクの初期レベルを `current_level_` の値に設定（`SetLevel()`）します。生成したタスクを当該レベルのランキューに追加すれば初期設定は完了です。`current_level_` は最も高いレベルに初期化されますから、メインタスクは最高の優先度を持つことになり、マウスイベントに素早く反応できるようになるはずです。

`Sleep()`、`Wakeup()`、`SwitchTask()` をレベルの仕組みに対応させなければなりません。まずは一番簡単な `Sleep()` の改造をしていきます。リスト 14.24 にレベル別ランキューに対応させた `Sleep()` を載せました。

リスト 14.24 `Sleep()` をレベルに対応させる (task.cpp)

```
void TaskManager::Sleep(Task* task) {
    if (!task->Running()) {
        return;
    }

    task->SetRunning(false);

    if (task == running_[current_level_].front()) {
        SwitchTask(true);
        return;
    }

    Erase(running_[task->Level()], task);
}
```

やっていることは単純ですね。タスクが実行可能状態かどうかをタスクの `running` フラグによって判定します。後で `Wakeup()` を改造するときこのフラグを立てる（`true` にする）ことにしましょう。タスクが実行可能状態であれば `running` フラグを下げます。

その後の `if` 文では、`task` が現在実行中であるか、すなわち自分自身をスリープさせようとしているかを判定します。`current_level_` は現在実行中のタスクが属するレベルを表しています（そうなるように実装を進めていきます）。したがって、そのレベルのランキューの先頭が `task` であるということは `task` が現在実行中のタスクであることを意味します。自分自身をスリープさせる場合はタスク切り替えが必要となりますので `SwitchTask(true);` を実行します。引数の `true` は、現在実行中のタスクをスリープさせて欲しいことを `SwitchTask()` に伝える役目があります。

`task` が現在実行中のタスクでない場合、すなわち他のタスクをスリープさせようとしている場合は単にそのタスクが属するレベルのランキューからそのタスクを削除すれば完了です。`std::deque` から要素を削除するには自作した `Erase()`（リスト 14.25）を使います。この関数は、指定した値を持つ要素すべてを `std::deque` から削除します。

リスト 14.25 Erase() は指定した要素をキューから削除する (task.cpp)

```
namespace {
    template <class T, class U>
    void Erase(T& c, const U& value) {
        auto it = std::remove(c.begin(), c.end(), value);
        c.erase(it, c.end());
    }
} // namespace
```

Erase() の実装はたったの2行です。1行目の `std::remove()` は、指定した値と等しい要素を末尾に集めます。名前から考えると要素を削除してくれそうなイメージですが、本当の削除は行いません。あくまで削除する要素を末尾に集めるだけです。`std::remove()` は削除する要素の先頭を返してくれますので、その位置から後ろを `c.erase()` を使って削除します。これで本当に削除されます。

リスト 14.26 SwitchTask() をレベルに対応させる (task.cpp)

```
void TaskManager::SwitchTask(bool current_sleep) {
    auto& level_queue = running_[current_level_];
    Task* current_task = level_queue.front();
    level_queue.pop_front();
    if (!current_sleep) {
        level_queue.push_back(current_task);
    }
    if (level_queue.empty()) {
        level_changed_ = true;
    }

    if (level_changed_) {
        level_changed_ = false;
        for (int lv = kMaxLevel; lv >= 0; --lv) {
            if (!running_[lv].empty()) {
                current_level_ = lv;
                break;
            }
        }
    }

    Task* next_task = running_[current_level_].front();

    SwitchContext(&next_task->Context(), &current_task->Context());
}
```

次に SwitchTask() を改造します。**リスト 14.26** に改造後の関数を示します。関数の前半では現在実行中のレベル `current_level_` のランキューを1要素だけ回転させます。レベルがなかったときとやっていることは同じです。引数 `current_sleep` が真の場合、現在実行中のタスク (`current_task`) をスリープさせます。といっても難しいことはなく、ランキューに追加しなければいいだけです。ということで、`current_sleep` が偽の場合のみ、ランキューの末尾にタスクを追加します。

次の if 文では現在実行中のレベルのランキューが空になったときの処理を書いています。この場合、実行するレベルを更新する必要があります。level_changed_ に真を設定し、その後の if 文が実行されるようにしておきます。

関数の中盤では `level_changed_` が真だった場合に実行中レベルを見直す処理を行います。タスクが存在する最も高いレベルを見つけます。具体的には高いレベルのランキューから順に見ていって、タスクが1つ以上登録されているランキューを見つけたらそのレベルを `current_level_` に設定します。図 14.5 の状況であれば `current_level_` は3になり、もしタスクAがスリープしていたら `current_level_` は1になります。

`level_changed_` が真になるタイミングはいくつかあります。そのうちの1つは直前の if 文です。ほかには `Wakeup()` の中で `level_changed_` が真になることがあります。

お気づきの方がいるかもしれませんが、この部分の処理はどのレベルにもタスクが存在しない場合を考慮していません。すべてのタスクがスリープしてしまったら `current_level_` の更新が行われず、その後の処理の結果は未定義となってしまいます。この問題には後で対処することにして、ここでは必ず1つ以上のタスクが実行可能状態になっていると仮定することにします。

`SwitchTask()` は最後に `current_level_` が指すランキューの先頭から1つタスクを取ってきて、そのタスクへ切り替えます。この時点で `current_level_` はタスクが存在するレベルの中で最も高いレベルになっているはずです。だから、そのレベルのランキューの先頭のタスクに切り替えれば良いのです。

リスト 14.27 `Wakeup()` に動作中タスクのレベルを変える機能を追加 (task.cpp)

```
void TaskManager::Wakeup(Task* task, int level) {
    if (task->Running()) {
        ChangeLevelRunning(task, level);
        return;
    }

    if (level < 0) {
        level = task->Level();
    }

    task->SetLevel(level);
    task->SetRunning(true);

    running_[level].push_back(task);
    if (level > current_level_) {
        level_changed_ = true;
    }
    return;
}
```

リスト 14.27 に `Wakeup()` の実装を示します。今までこの関数は、指定されたタスクがランキューに存在しなければ (= スリープ中であれば) ランキューの末尾に追加する、という単純な処理を行うものでした。今回はレベル別ランキューを導入したため機能が大きく拡張されました。

`Wakeup()` は大きく分けて2つの役割を持ちます。1つはスリープ中のタスクを起こす機能、もう1つは動作中タスクのレベルを変える機能です。後者は `wakeup` なのかという感じもありますが、`Wakeup()` を「指定したタスクの現在の状態にかかわらず、とにかく指定したレベルで動作させる」関数にするためには後者の機能も必要だったのです。

関数の冒頭でタスクが実行可能状態かどうかを調べます。実行可能状態なのであれば動作中のレベル変更ということで `ChangeLevelRunning()` メソッドに処理を委譲します。このメソッドの実装は後

ほど紹介します。ここではまず、タスクがスリープ中であった場合の処理を説明します。

タスクがスリープ中であれば先頭の if 文は実行されず、後ろの処理に進みます。まずは引数 level が負の場合の処理です。level でタスクの実行レベルを指定しますが、タスクのレベルを変更せず、前回スリープする直前に動作していたレベルを引き継ぐときは負の値を指定することにします。次にタスクの状態記憶用の変数を更新しておきます。その後、タスクを目的のレベルのランキューに追加します。

最後の if 文はちょっと分かりにくいかもしれませんが。この if 文の意図は、次のタスク切り替え時に current_level_ を更新すべきかを判断することです。現在実行中のタスクよりレベルの高いタスクが新たに実行可能状態になった場合、次のタスク切り替えではレベルの高い方のタスクを実行しなければなりません。そこで level_changed_ を真に設定し、タスク切り替え時にレベルの見直しをするように要求します。

リスト 14.28 ChangeLevelRunning() は動作中のタスクのレベルを変える (task.cpp)

```
void TaskManager::ChangeLevelRunning(Task* task, int level) {
    if (level < 0 || level == task->Level()) {
        return;
    }

    if (task != running_[current_level_].front()) {
        // change level of other task
        Erase(running_[task->Level()], task);
        running_[level].push_back(task);
        task->SetLevel(level);
        if (level > current_level_) {
            level_changed_ = true;
        }
        return;
    }

    // change level myself
    running_[current_level_].pop_front();
    running_[level].push_front(task);
    task->SetLevel(level);
    if (level >= current_level_) {
        current_level_ = level;
    } else {
        current_level_ = level;
        level_changed_ = true;
    }
}
```

リスト 14.28 に動作中のタスクのレベルを変える ChangeLevelRunning() の実装を示します。この関数はタスクが実行可能状態である場合に Wakeup() の中から呼び出されます。

関数冒頭の if 文では実行レベルが変化しない場合に関数を終了させます。単なる最適化ですね。

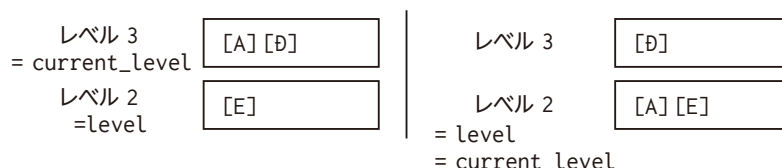
2 つ目の if 文では指定されたタスクが現在実行中のタスクかどうかを調べます。あるタスクが自分自身のレベルを変更する場合と、他のタスクのレベルを変更する場合で処理を変えるためです。この if 文は他のタスクのレベルを変更する場合を担当します。

他のタスクの実行レベルを変えるのは比較的単純です。そのタスクが現在属しているレベルのラン

キューから削除し、目的のレベルのランキューに追加し直すだけです。目的のレベルが現在の動作レベルより高ければ次回のタスク切り替え時にレベル見直しが必要です。level_changed_ を真にしておきます。

次に 2 つ目の if 文が偽、すなわちあるタスクが自分自身のレベルを変更しようとしている場合の処理を見ていきます。まずはランキューの調整を行います。そのタスクを現在のランキューから取り除き (pop_front())、目的のレベルのランキューに追加します (push_front(task))。その後、current_level_ を目的のレベルに更新しておきます。

図 14.6 タスク A が自分自身の実行レベルを 3 から 2 に変更する場合



タスクを追加する場所が末尾ではなく先頭であり、current_level_ の更新も行う理由は次回のタスク切り替えを正常に行うためです。図 14.6 に、タスク A が自分自身の実行レベルを 3 から 2 に変更しようとする様子を示します。タスク切り替えを担当する SwitchTask() は「current_level_ のランキューの先頭にあるタスク」を現在実行中のタスクだと認識します。そのため、Wakeup() の処理が完了した時点で「タスク A が current_level_ のランキューの先頭にある」という状態になっている必要があります。ランキューの先頭にタスクを追加する理由、その後 current_level_ の書き換えを行う理由はその状態を作るためだったのです。

さて、現在実行中のタスクのレベルが current_level_ より高くなる場合はランキューの調整だけでいいのですが、低いレベルになる場合には次回のタスク切り替え時に current_level_ の見直しをしなければなりません。図 14.6 がまさにその状況を表しています。タスク A のレベルを下げたことにより、次回のタスク切り替えでタスク D を実行すべき状況になりました。しかし、タスク A のレベルを下げた影響で current_level_ も下がってしまっていますから、level_changed_ が偽のままだとタスク D が無視されてレベル 2 のタスク群が実行されてしまいます。次回のタスク切り替えでレベルを見直すことで、きちんとタスク D が実行されるようにします。

図 14.7 メインタスクが優先されるのでマウスがスムーズに動く



改造が終わったら動作させてみましょう (図 14.7)。相変わらず写真だと伝わりにくいですが、マウスを動かしたときの動きが非常にスムーズになりました。そして、メインタスクのウィンドウに表示される値が今までは 1304 のように 4 だけずれていたのですが、1300 などのキリの良い数字が表示されるようになりました。テキストボックスのカーソル点滅用タイマがタイムアウトしたときにメインタスクが起こされ、他のタスクより優先して動作することで、タイムアウトからイベント処理までの時間が短くなった証拠です。ということで、レベル別ランキューの導入は成功です。

14.5 アイドルタスク (osbook_day14d)

タスクのスリープができるようになりましたので、メインタスクだけでなく、すべてのタスクで暇になったらスリープするようにしましょう。と思ったのですが、1つ潜在的な問題が残っていました。TaskManager::SwitchTask()の実装が、すべてのタスクがスリープしてランキューが空になった場合を考慮していないのです。

「ランキューが空の場合」に対処する特別なコードを追加しても良いのですが、実はもっと素敵な解決策があります。それは、絶対にスリープしない「ずっと hlt するだけのタスク」を1つ追加する方法です。このようなタスクを**アイドルタスク** (idle task) と呼びます*¹。アイドルタスクを最も低いレベルに1つ入れて置くことでランキューが空になることがなくなりますので、今のプログラムを変更することなく問題を回避できます。

* 1 アイドル (idle) は「仕事のない、暇な」という意味。歌ったり踊ったりするアイドル (idol) とは関係ありません。

リスト 14.29 TaskManager のコンストラクタでアイドルタスクを登録 (task.cpp)

```
TaskManager::TaskManager() {
    Task& task = NewTask()
        .SetLevel(current_level_)
        .SetRunning(true);
    running_[current_level_].push_back(&task);

    Task& idle = NewTask()
        .InitContext(TaskIdle, 0)
        .SetLevel(0)
        .SetRunning(true);
    running_[0].push_back(&idle);
}
```

ということで早速実装していきます。リスト 14.29 に TaskManager クラスのコンストラクタを示します。メインタスクと同時にアイドルタスクも生成して、最低のレベルのランキューに登録することにした。今後はレベル 0 をアイドルタスク専用ということにして、他のタスクはレベル 1 から kMaxLevel までのレベルを使うことにしましょう。

リスト 14.30 シンプルになったメイン関数のタスク生成処理 (main.cpp)

```
InitializeTask();
Task& main_task = task_manager->CurrentTask();
const uint64_t taskb_id = task_manager->NewTask()
    .InitContext(TaskB, 45)
    .Wakeup()
    .ID();
```

TaskManager のコンストラクタでアイドルタスクを登録するようにしたので、メイン関数で改めて登録する必要はありません。メイン関数のタスク生成部はリスト 14.30 に示すようにかなりシンプルになりました。

リスト 14.31 TaskIdle() は hlt するだけ (task.cpp)

```
void TaskIdle(uint64_t task_id, int64_t data) {
    while (true) __asm__("hlt");
}
} // namespace
```

TaskIdle() はもともと main.cpp にありましたが、もうメイン関数で使うことはありませんので task.cpp に移設しました (リスト 14.31)。ついでにアイドルタスク起動時のメッセージも出ないようにしました。アイドルタスクは影に身を潜め、表舞台で出てこなくなります。

図 14.8 アイドルタスクがランキューが空の状況をうまく防ぐ



図 14.8 に実行した様子を示します。S キーを押して TaskB をスリープさせるとメインタスクと TaskB の両方がスリープすることになりますが、アイドルタスクがいるおかげで変な動作にならず正常に実行が続きます。また、S キーを押さずに TaskB のカウンタの増え方を観察していると、以前より 4 倍程度速く増えることに気づきました。アイドルタスクが TaskB より低いレベルで登録されているため、アイドルタスクが TaskB の実行を邪魔しないのです。素晴らしい！

第 15 章

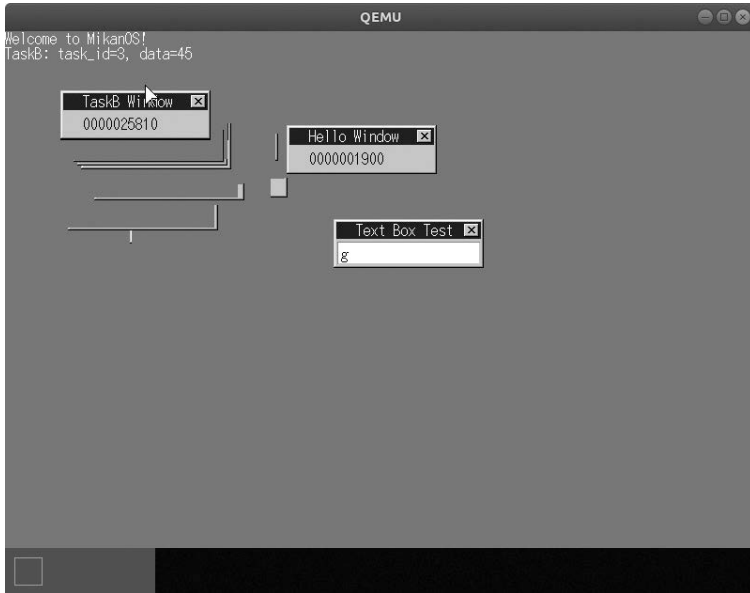
ターミナル

やっぱり OS といえば黒い画面にコマンドを入力して操作したいですね。ターミナル（端末）とはコマンドを入力する窓のことで、Linux であれば `gnome-terminal` などのターミナル、Windows であれば MS-DOS プロンプトや Windows Terminalなどを指します。この章では OS っぽさをさらに高めるために、コマンドを入力するためのターミナル画面を作ってみることにします。

15.1 ウィンドウ描画はメインスレッドで (osbook_day15a)

早速ターミナルを作っていこうと思ったのですが、前章で作った OS で遊んでいたらある問題に気づきました。TaskB のウィンドウをマウスで移動させると図 15.1 のようにゴミが残ってしまうのです。

図 15.1 TaskB のウィンドウを移動するとゴミが残る



これは TaskB の中で `layer_manager->Draw()` を呼び出す部分と、マウスで TaskB のウィンドウを移動させるために `layer_manager->MoveRelative()` を呼び出す部分がデータ競合を起こすためです。この 2 つのメソッドはともに、内部でスクリーンのバックバッファやフレームバッファの書き換えをします。

TaskB のウィンドウを再描画するための `Draw()` は TaskB コンテキストで行われる一方、マウス移動に伴う `MoveRelative()` はメインコンテキストで実行されます。そのため TaskB がカウンタをインクリメントして `Draw()` を実行している最中にマウスイベントが発生すると、メインタスクに切り替わり `MoveRelative()` の実行が開始されます。2 つのメソッドは `layer_manager` のメンバ変数の内容を変更しますから、これはデータ競合となります。データ競合があると何が起きるか予測ができません。今回は単に画面の表示が乱れる程度で済んでいますが……。

直し方は 2 つあります。1 つは `Draw()` と `MoveRelative()` の実行中に割り込みを禁止することです。cli と sti で重要な処理を挟むやり方は今まで何回か登場しました。それらと同様にすることでデータ競合を防げます。もう 1 つの方法は、ウィンドウの描画や移動の処理をすべて専用タスクが引き受ける方法です。ウィンドウの操作がしたくなったタスクは、その専用タスクのメッセージキューに要求を投げます。専用タスクは要求が来るとスリープから復帰し、要求に従ってウィンドウの操作を行います。

前者のやり方はとてもシンプルです。しかし、画面描画というのは大量のピクセル書き換えが発生するため比較的重い処理です。その重い処理の間、ずっと割り込みを禁止してしまうのは良くありま

せん。割り込み禁止期間が長いほど割り込みを取りこぼす可能性が高くなるからです。低い優先度であるはずのウィンドウの再描画処理によって優先度の高い割り込み処理が阻害されることになります。これを**優先順位の逆転**と言って、OS のタスクスケジューリングにおける有名な問題の 1 つです。

後者のやり方は多少複雑になりますが、ウィンドウの操作の間もずっと割り込み許可状態のため上記の問題は発生しません。ということで、この節では後者の方法を実装していくことにします。専用タスクを特別に作るのではなく、メインタスクがその責務を負うことにします。

リスト 15.1 Message の種別にレイヤ操作タイプを加える (message.hpp)

```
struct Message {
    enum Type {
        kInterruptXHCI,
        kTimerTimeout,
        kKeyPush,
        kLayer,
        kLayerFinish,
    } type;

    uint64_t src_task;
```

リスト 15.1 に示すように、TaskB からメインタスクへウィンドウ描画などのメッセージを投げるために、`Message::Type` に `kLayer` を加えました。ウィンドウの描画はレイヤの機能ですのでそういう名前にしてみました。描画や移動が終わったことを送信元に知らせるためのタイプ `kLayerFinish` と、送信元タスクの ID を表す `src_task` も加えました。

リスト 15.2 kLayer 用の引数 (message.hpp)

```
struct {
    LayerOperation op;
    unsigned int layer_id;
    int x, y;
} layer;
```

リスト 15.3 LayerOperation はレイヤ操作の種類を表す (message.hpp)

```
enum class LayerOperation {
    Move, MoveRelative, Draw
};
```

`kLayer` に対応する引数として `arg.layer` を追加しました (リスト 15.2)。`op` はレイヤ操作の種類を表す列挙体で、リスト 15.3 のような定義になっています。`layer_id` は操作対象のレイヤを表す ID、`x` と `y` は移動系のレイヤ操作の場合に使用する位置情報です。ここで `Vector2D<int>` を使わなかったのは、`message.hpp` が他の非標準ヘッダファイルに依存しないように保ちたい気分がなんとなくあるからです。

リスト 15.4 レイヤ操作要求に対処する (main.cpp)

```
case Message::kLayer:
    ProcessLayerMessage(*msg);
    __asm__("cli");
    task_manager->SendMessage(msg->src_task, Message{Message::kLayerFinish});
    __asm__("sti");
    break;
```

レイヤ操作要求に対応する処理をメインループ内に増やしました (**リスト 15.4**)。msg->type が kLayer であるようなメッセージを受信したら、メッセージの引数に基づいて処理した後、送信元タスクへ処理の完了を通知してあげます。処理の完了を通知する際に Message 構造体を増やした src_task 変数が役立ちます。

リスト 15.5 ProcessLayerMessage() はレイヤ操作要求を実際に処理する (layer.cpp)

```
void ProcessLayerMessage(const Message& msg) {
    const auto& arg = msg.arg.layer;
    switch (arg.op) {
    case LayerOperation::Move:
        layer_manager->Move(arg.layer_id, {arg.x, arg.y});
        break;
    case LayerOperation::MoveRelative:
        layer_manager->MoveRelative(arg.layer_id, {arg.x, arg.y});
        break;
    case LayerOperation::Draw:
        layer_manager->Draw(arg.layer_id);
        break;
    }
}
```

リスト 15.5 に、ProcessLayerMessage() の実装を示します。メッセージの引数にしたがって要求された操作を実行します。操作によっては x と y は使われないことがあります。

リスト 15.6 TaskB からメインタスクへレイヤ操作要求を出す (main.cpp)

```
void TaskB(uint64_t task_id, int64_t data) {
    printk("TaskB: task_id=%lu, data=%lu\n", task_id, data);
    char str[128];
    int count = 0;

    __asm__("cli");
    Task& task = task_manager->CurrentTask();
    __asm__("sti");

    while (true) {
        ++count;
        sprintf(str, "%010d", count);
        FillRectangle(*task_b_window->Writer(), {24, 28}, {8 * 10, 16}, {0xc6, 0xc6, 0xc6});
        WriteString(*task_b_window->Writer(), {24, 28}, str, {0, 0, 0});
    }
```

```

Message msg{Message::kLayer, task_id};
msg.arg.layer.layer_id = task_b_window_layer_id;
msg.arg.layer.op = LayerOperation::Draw;
__asm__("cli");
task_manager->SendMessage(1, msg);
__asm__("sti");

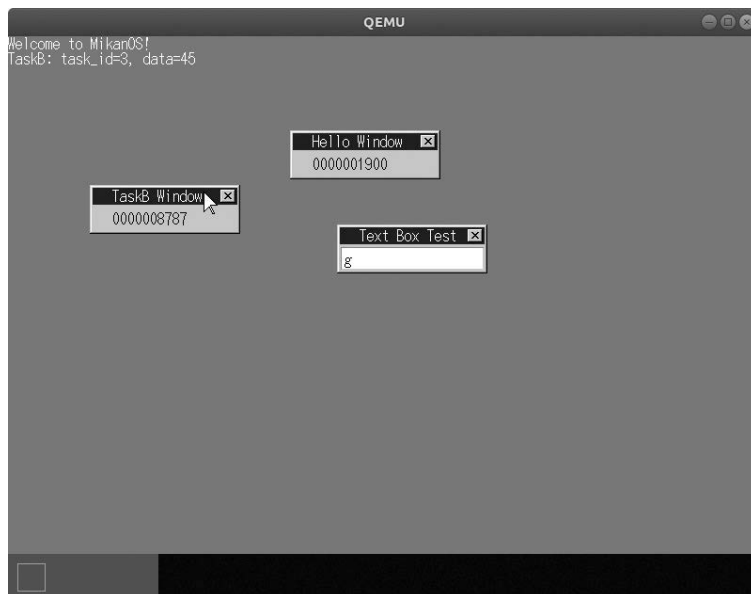
while (true) {
    __asm__("cli");
    auto msg = task.ReceiveMessage();
    if (!msg) {
        task.Sleep();
        __asm__("sti");
        continue;
    }

    if (msg->type == Message::kLayerFinish) {
        break;
    }
}
}
}
}

```

リスト 15.6 に、ここまでで作ったレイヤ操作要求と処理の仕組みを利用するように改造した TaskB() を示します。大きく変わったのは、layer_manager->Draw() を直接呼ぶ代わりに kLayer タイプのメッセージをメインタスクへ投げたようにしたこと、メインタスクから操作完了の通知が来るのを待つために while ループを加えたことです。while ループの中身は一見すると複雑ですが、実はメインループと構造は同じですので理解はしやすいでしょう。

図 15.2 TaskB のウィンドウを移動してもゴミが残らない



改造が完了しました。実行してみると先ほどのようなゴミが出なくなっているのが分かります (図 15.2)。ただし、TaskB のカウンタの進みが改造前の 1/3 ほどの早さになってしまいました。TaskB とメインタスクで頻繁にタスク切り替えが発生するため、その切り替えのオーバーヘッドが大きいのでしょう。今回はプログラムの速さを犠牲にして安全性が高まりましたので、それでよしとしましょう。

15.2 アクティブウィンドウ (osbook_day15b)

1 つバグを解消したので気分が良いです。この調子でターミナルを作っていきますよ。まずはそうですね、ターミナルっぽいウィンドウを作って、とりあえず文字入力ができるようにしてみたいと思います。と思ってふと OS の画面を見るとなんだか違和感を感じました。複数のウィンドウが同時にアクティブになっているように見えるのです。

アクティブな状態とは、そのウィンドウが選択状態になっていて最前面にきていることを意味します。普通の OS ではアクティブになっているウィンドウとそうでないウィンドウはタイトル色が違って見分けが付くようになっています。また、キーボードの入力はアクティブウィンドウに送られます。

しかし MikanOS にはまだウィンドウのアクティブ、非アクティブを切り替える仕組みはありません。すべてのウィンドウのタイトルバーが同じ色です。キーボード入力はすべて「Text Box Test」のウィンドウに送信されます。これは困ります。ターミナルウィンドウをせっかく作っても、キーボード入力がターミナルに送信されなかったらコマンド入力できません。ということで、ウィンドウを選択するとアクティブになる仕組みを導入します。

リスト 15.7 ActiveLayer クラスはウィンドウをアクティブ化する (layer.hpp)

```
class ActiveLayer {
public:
    ActiveLayer(LayerManager& manager);
    void SetMouseLayer(unsigned int mouse_layer);
    void Activate(unsigned int layer_id);
    unsigned int GetActive() const { return active_layer_; }

private:
    LayerManager& manager_;
    unsigned int active_layer_{0};
    unsigned int mouse_layer_{0};
};

extern ActiveLayer* active_layer;
```

マウスで選択したウィンドウをアクティブにするための準備として、指定したウィンドウ（を含むレイヤ）を最前面に持ってくるために ActiveLayer クラスを作ることになりました (リスト 15.7)。マウスレイヤは常に最前面にする必要があるため、マウスレイヤの ID を登録する SetMouseLayer() メソッドがあります。最も重要なメソッドは Activate() です。これは指定したレイヤを最前面(といいつつ、マウスレイヤよりは下)に引っ張り上げ、タイトルバーの色を変えます。

リスト 15.8 ActiveLayer クラスのコンストラクタと SetMouseLayer() (layer.cpp)

```
ActiveLayer::ActiveLayer(LayerManager& manager) : manager_{manager} {  
}  
  
void ActiveLayer::SetMouseLayer(unsigned int mouse_layer) {  
    mouse_layer_ = mouse_layer;  
}
```

リスト 15.8 に ActiveLayer クラスのコンストラクタと SetMouseLayer() メソッドの実装を示します。コンストラクタでは、この ActiveLayer クラスのインスタンスが用いる LayerManager を設定します。わざわざ引数で受け取らず、グローバル変数の layer_manager を使ってもいいのですが、わずかな手間ですべてグローバル変数を使わないようにできるならそうしておいて損はないでしょう。

SetMouseLayer() はマウスレイヤの ID を設定します。InitializeMouse() の中から呼び出すようにしました (**リスト 15.9**)。

リスト 15.9 マウスレイヤを active_layer に登録する (mouse.cpp)

```
active_layer->SetMouseLayer(mouse_layer_id);
```

Activate() の実装の方針を考えます。このメソッドが呼ばれたらすべきことは、指定されたレイヤを最前面に持ってきつつ、そのレイヤが持つウィンドウのタイトルバーの色を変えることです。それと同時に、それまで最前面にあったレイヤが持つウィンドウのタイトルバーの色を元に戻す必要がありますね。ウィンドウの色を変えるのはレイヤの機能ではなくウィンドウの機能ですから Window クラス側にそのためのメソッドを持たせることにします。

リスト 15.10 Activate() は指定したレイヤを最前面に上げる (layer.cpp)

```
void ActiveLayer::Activate(unsigned int layer_id) {  
    if (active_layer_ == layer_id) {  
        return;  
    }  
  
    if (active_layer_ > 0) {  
        Layer* layer = manager_.FindLayer(active_layer_);  
        layer->GetWindow()->Deactivate();  
        manager_.Draw(active_layer_);  
    }  
  
    active_layer_ = layer_id;  
    if (active_layer_ > 0) {  
        Layer* layer = manager_.FindLayer(active_layer_);  
        layer->GetWindow()->Activate();  
        manager_.UpDown(active_layer_, manager_.GetHeight(mouse_layer_) - 1);  
        manager_.Draw(active_layer_);  
    }  
}
```

リスト 15.10 に Activate() の実装を示します。やっていることは大きく 2 つです。1 つは今までアクティブだったレイヤ (active_layer_) が持つウィンドウを非アクティブ化することで

す。もう1つは指定されたレイヤを最前面に持ってきつつ、そのレイヤが持つウィンドウをアクティブ化することです。ウィンドウのアクティブ、非アクティブの切り替えのために、Window クラスに `Activate()` と `Deactivate()` を新しく作ることにしました。

ここでちょっと Window クラスについて考察しておきます。このクラスは縦横の大きさを持つ単なる表示領域を表すクラスです。タイトルバーを持つウィンドウだけでなく、コンソールやマウスカーソルも Window クラスです。ですから、Window クラス自身はタイトルバーの存在なんて感知していません。私たちがタイトルバーだと思っているものは、ウィンドウの描画領域に描かれている単なる絵です。この状態では「`Window::Activate()` を呼び出したらタイトルバーの色を変える」を実現するのは難しいです。タイトルバーを描画し直そうと思っても、Window クラスはタイトルバーの文字列を知らないため再描画できないのです。

そこで、Window クラスを継承してタイトルバー付きウィンドウを表すクラスを作ることになります。名前は迷いましたが `ToplevelWindow` とします。

リスト 15.11 Window クラスを継承できるようにする (window.hpp)

```
/** @brief 指定されたピクセル数の平面描画領域を作成する。 */
Window(int width, int height, PixelFormat shadow_format);
virtual ~Window() = default;
Window(const Window& rhs) = delete;
Window& operator=(const Window& rhs) = delete;
```

Window クラスを継承できるようにするために、**リスト 15.11** のようにデストラクタを `virtual` にしました。継承の基底となるクラスのデストラクタは仮想デストラクタにしておかないといけないのです*¹。

リスト 15.12 Window クラスにアクティブ化メソッドを増やす (window.hpp)

```
virtual void Activate() {}
virtual void Deactivate() {}
```

リスト 15.12 のように、`ActiveLayer::Activate()` から呼び出すための2つのメソッドを増やします。中身は空で良いのですが、 subclasses でオーバーライドできるように `virtual` にしておきます。

リスト 15.13 ToplevelWindow はタイトルバーを持つウィンドウを表す (window.hpp)

```
class ToplevelWindow : public Window {
public:
    static constexpr Vector2D<int> kTopLeftMargin{4, 24};
    static constexpr Vector2D<int> kBottomRightMargin{4, 4};

    class InnerAreaWriter : public PixelWriter {
    public:
        InnerAreaWriter(ToplevelWindow& window) : window_{window} {}
        virtual void Write(Vector2D<int> pos, const PixelColor& c) override {
            window_.Write(pos + kTopLeftMargin, c);
        }
        virtual int Width() const override {
            return window_.Width() - kTopLeftMargin.x - kBottomRightMargin.x;
        }
        virtual int Height() const override {
```

```

        return window_.Height() - kTopLeftMargin.y - kBottomRightMargin.y; }

private:
    ToplevelWindow& window_;
};

ToplevelWindow(int width, int height, PixelFormat shadow_format,
               const std::string& title);

virtual void Activate() override;
virtual void Deactivate() override;

InnerAreaWriter* InnerWriter() { return &inner_writer_; }
Vector2D<int> InnerSize() const;

private:
    std::string title_;
    InnerAreaWriter inner_writer_{*this};
};

```

Window クラスを継承する準備が整いましたので、タイトルバーを持つウィンドウを表すクラス `ToplevelWindow` を定義します (**リスト 15.13**)。class `ToplevelWindow` : public `Window` として、`Window` クラスを `public` 継承しています。`public` 継承とは is-a 関係 (`ToplevelWindow` is a `Window`) を満たす継承です。具体的には親クラスの `public` なメンバすべてが子クラスでも `public` となる継承です。すなわち `ToplevelWindow` も `Window` と同様、`DrawTo()` や `Write()` といった `public` メソッドを持つことになります。`public` 継承にしたため、`ToplevelWindow` を `Window` として扱えるようになります。

`title_` というメンバ変数に注目します。これはタイトルバーに表示する文字列を記憶するための変数です。この変数を持つことが `Window` クラスに比した `ToplevelWindow` の最大の特徴と言ってもいいかもしれません。この変数のおかげでタイトルバーの再描画が可能になります。変数の型として `std::string` を使っている理由は「**コラム 15.1 タイトルに `std::string` を使う理由**」にまとめました。

もう1つ `ToplevelWindow` クラスの重要な機能に、ウィンドウの枠内専用の描画ツール `InnerAreaWriter` があります。ウィンドウの中身を書き換える際、タイトルバーやウィンドウの枠には触らずに枠内だけを書き換えたいということが多いでしょう。そういうときに「ウィンドウの左右 4 ピクセル、上から 24 ピクセルは余白だから避けて……」のようなことを毎回考えるのは面倒です。そこで、ウィンドウの枠内でお絵描きをするための道具を提供することにしました。`InnerWriter()` はこのウィンドウ用の描画ツールを返します。この戻り値を用いて絵を描くと、座標 (0,0) がウィンドウ本体の (4,24) となるように調整されます。

* 1 本書では理由までは説明しませんが、デストラクタを仮想にしなければならないのは C++ のルールです。気になる人は C++ の教科書を参照してください

リスト 15.14 ToplevelWindow のメソッド定義 (window.cpp)

```

ToplevelWindow::ToplevelWindow(int width, int height, PixelFormat shadow_format,
                                const std::string& title)
    : Window(width, height, shadow_format}, title_{title} {
    DrawWindow(*Writer(), title_.c_str());
}

void ToplevelWindow::Activate() {
    Window::Activate();
    DrawWindowTitle(*Writer(), title_.c_str(), true);
}

void ToplevelWindow::Deactivate() {
    Window::Deactivate();
    DrawWindowTitle(*Writer(), title_.c_str(), false);
}

Vector2D<int> ToplevelWindow::InnerSize() const {
    return Size() - kTopLeftMargin - kBottomRightMargin;
}

```

リスト 15.14 に `ToplevelWindow` の各メソッドの実装を示します。コンストラクタでは親クラス `Window` のコンストラクタを呼び出すと同時に、`ToplevelWindow` クラスの独自のメンバ変数である `title_` の初期化も行います。また、そのタイトルを使ってタイトルバーとウィンドウの枠を `DrawWindow()` を用いて描画します。

`Activate()` はウィンドウのアクティブ化を担当します。やっていることは単純で、`DrawWindowTitle()` を `active=true` で呼び出すだけです。これによりタイトルバーがアクティブ用の色（青系）で描画されます。`DrawWindowTitle()` は後で実装します。

ちなみに、親クラスのメソッド `Window::Activate()` を呼び出していますが実際には何の効果もありません。定義が空だからです（**リスト 15.12** を参照）。ただ、メソッドをオーバーライドするときには親クラスの同名メソッドを呼び出しておくのが良い場合があります。将来、もしかしたら `Window::Activate()` の機能が拡張されて何か意味のあることをするようになるかもしれません。そんなとき、子クラスから呼び出すのを忘れていると、せっかくの改造が無効化されてしまいます。だから、今は意味がなかったとしても、念の為呼び出しているというわけです。

`Deactivate()` は `Activate()` の逆の処理です。`DrawWindowTitle()` を `active=false` で呼び出し、タイトルバーを非アクティブ用の色（灰系）で描画します。

`InnerSize()` はウィンドウ枠内の大きさを計算します。ウィンドウ全体の大きさから上下左右の余白を引くと枠内の大きさとなります。

リスト 15.15 DrawWindowTitle() はタイトルバーを描画する (window.cpp)

```

void DrawWindowTitle(PixelWriter& writer, const char* title, bool active) {
    const auto win_w = writer.Width();
    uint32_t bgcolor = 0x848484;
    if (active) {
        bgcolor = 0x000084;
    }
}

```

```

FillRectangle(writer, {3, 3}, {win_w - 6, 18}, ToColor(bgcolor));
WriteString(writer, {24, 4}, title, ToColor(0xffffffff));

for (int y = 0; y < kCloseButtonHeight; ++y) {
    for (int x = 0; x < kCloseButtonWidth; ++x) {
        PixelColor c = ToColor(0xffffffff);
        if (close_button[y][x] == '@') {
            c = ToColor(0x000000);
        } else if (close_button[y][x] == '$') {
            c = ToColor(0x848484);
        } else if (close_button[y][x] == ':') {
            c = ToColor(0xc6c6c6);
        }
        writer.Write({win_w - 5 - kCloseButtonWidth + x, 5 + y}, c);
    }
}
}
}

```

リスト 15.15 に DrawWindowTitle() の実装を示します。今まで DrawWindow() の中にあったタイトルバーの描画処理を抜き出してきて、色を変える機能を付けただけです。

さて、これで ToplevelWindow クラスが完成しました。ActiveLayer::Activate() が呼び出されると、指定されたウィンドウの Activate() と、今までアクティブだったウィンドウの Deactivate() が呼ばれることになります。しかし、まだ肝心の ActiveLayer::Activate() を呼び出す処理を書いていませんでした。マウスでウィンドウをクリックしたときにこれを呼び出すようにしたいと思います。

リスト 15.16 マウスでウィンドウをクリックしたら Activate() する (mouse.cpp)

```

const bool previous_left_pressed = (previous_buttons_ & 0x01);
const bool left_pressed = (buttons & 0x01);
if (!previous_left_pressed && left_pressed) {
    auto layer = layer_manager->FindLayerByPosition(position_, layer_id_);
    if (layer && layer->IsDraggable()) {
        drag_layer_id_ = layer->ID();
        active_layer->Activate(layer->ID());
    } else {
        active_layer->Activate(0);
    }
} else if (previous_left_pressed && left_pressed) {

```

リスト 15.16 に、Mouse::OnInterrupt() の改造箇所を抜粋しました。改造したのはマウスの左ボタンをクリックしたときの処理です。これまでは左ボタンをクリックするとウィンドウのドラッグ移動を開始するだけでしたが、今回の改造ではウィンドウのアクティブ化も行うようにしました。具体的には active_layer->Activate(layer->ID()); という行を加えました。クリックした位置がドラッグ不可能なレイヤ（背景とか、コンソールウィンドウとか）だったら、active_layer->Activate(0); としてすべてのウィンドウを非アクティブにします。

これで、マウスでウィンドウのアクティブ状態を切り替えることができるようになりました。他の OS だと Alt+Tab キーでもアクティブウィンドウを切り替えられますよね。本書ではキーボードによる切り替え機能は実装しませんが、興味のある方はチャレンジしてみてもいいと思います。

リスト 15.17 アクティブウィンドウによりキーの処理を変える (main.cpp)

```

case Message::kKeyPush:
    if (auto act = active_layer->GetActive(); act == text_window_layer_id) {
        InputTextWindow(msg->arg.keyboard.ascii);
    } else if (act == task_b_window_layer_id) {
        if (msg->arg.keyboard.ascii == 's') {
            printk("sleep TaskB: %s\n", task_manager->Sleep(taskb_id).Name());
        } else if (msg->arg.keyboard.ascii == 'w') {
            printk("wakeup TaskB: %s\n", task_manager->Wakeup(taskb_id).Name());
        }
    } else {
        printk("key push not handled: keycode %02x, ascii %02x\n",
            msg->arg.keyboard.keycode,
            msg->arg.keyboard.ascii);
    }
    break;

```

リスト 15.17 にメインループの中でのキー入力処理を示します。アクティブウィンドウに応じて処理を変えるようにしました。テキストボックスがあるウィンドウ (`text_window_layer_id`) がアクティブならテキストボックスにキーを入力しますし、TaskB 用のウィンドウがアクティブなら S キーと W キーでスリープ、ウェイクアップを行います。その他のウィンドウがアクティブ、またはどのウィンドウもアクティブでない場合はコンソールに「key push not handled」というログを出します。

リスト 15.18 ToplevelWindow を使うように修正 (main.cpp)

```

std::shared_ptr<ToplevelWindow> text_window;
unsigned int text_window_layer_id;
void InitializeTextWindow() {
    const int win_w = 160;
    const int win_h = 52;

    text_window = std::make_shared<ToplevelWindow>(
        win_w, win_h, screen_config.pixel_format, "Text Box Test");
    DrawTextbox(*text_window->InnerWriter(), {0, 0}, text_window->InnerSize());
}

```

最後の仕上げとして、タイトルバーを持つウィンドウを Window から ToplevelWindow に切り替える必要があります。修正すべき箇所はたくさんあります。例として、テキストボックスを持つウィンドウに関する修正を **リスト 15.18** に示します。std::shared_ptr<Window> を使っていた場所を std::shared_ptr<ToplevelWindow> に切り替え、std::make_shared の引数の最後にタイトル「Text Box Test」を追加し、DrawWindow() の呼び出し行を削除するというのが主な変更点です。

細かい変更として DrawTextbox() の中で使っていた text_window->Writer() を text_window->InnerWriter() に切り替え、それに合わせて座標 {4, 24} を {0, 0} に変えました。また、サイズもウィンドウの枠内にピッタリさせたいので text_window->InnerSize() を使うようにしました。

リスト 15.19 なるべく InnerWriter() を使う (main.cpp)

```
void DrawTextCursor(bool visible) {  
    const auto color = visible ? ToColor(0) : ToColor(0xffffffff);  
    const auto pos = Vector2D<int>{4 + 8*text_window_index, 5};  
    FillRectangle(*text_window->InnerWriter(), pos, {7, 15}, color);  
}
```

そのほか、`ToplevelWindow::Writer()` を使っている箇所を `ToplevelWindow::InnerWriter()` を使うように修正しました。例えば `DrawTextCursor()` は **リスト 15.19** のように修正しました。`InnerWriter()` を使うと、座標の指定でタイトルバーやウィンドウ枠の太さを気にする必要がなくなつてスッキリします。

図 15.3 アクティブウィンドウにキーが送られている様子



必要な修正が終わったはずなので実行してみます (図 15.3)。どれどれ……おお！ ちゃんとクリックしたウィンドウがアクティブになり、アクティブなウィンドウにキーが入力できるじゃないですか！ ということで成功です。ちなみに、コンソールに出ているログは Hello Window に A キーと B キーを送ったときのログです (ASCII コード 0x61 と 0x62 は a と b に対応しています)。その後テキストボックスを持つウィンドウをアクティブにしてキーを押下すると、ちゃんとテキストボックスに入力できました。素晴らしい！

コラム 15.1 タイトルに `std::string` を使う理由

なぜ `const char* title_` としなかったのでしょうか？ それは、将来的に動的に生成した文字列をタイトルとして設定できるようにするためです。タイトルは今は文字列リテラルしか設定されませんから `const char*` でも問題ありませんが、将来的に、例えば `sprintf()` などで作った文字列を設定しようと思うとそれではだめなのです。`std::string` は文字列全体のコピーを格納する一方、`const char*` は先頭アドレスしか格納できないというのが理由です。

`sprintf()` で動的に生成した文字列をタイトルにしようと思うと、ぱっと思いつく方法は、ローカル変数として作った配列に文字列を書き出し、その配列を `title_` に設定することでしょう。

```
std::shared_ptr<ToplevelWindow> CreateTickWindow() {
    char win_title[32];
    sprintf(win_title, "tick = %lu", timer_manager->CurrentTick());
    return std::make_shared<ToplevelWindow>(
        160, 52, screen_config.pixel_format, win_title);
}
```

`CreateTickWindow()` はタイマの現在値をタイトルに含むようなウィンドウを生成して返します。このプログラムの問題点は、`win_title` がローカル変数となってしまうことです。ローカル変数は関数を抜けるときに破棄されます。一方、戻り値となっているウィンドウ自体は関数を抜けた後も有効です。

仮にタイトルを保持するメンバ変数を `const char* title_` とした場合を考えてみます。この場合、`title_` には `win_title` 配列へのポインタが格納されます。しかし、`win_title` は関数を抜けると破棄されますから、`title_` は**破棄された配列へのポインタ**を保持することになります。生成したウィンドウを画面に描画する際にこのポインタを経由して破棄された配列が参照されます。これは重大なバグです。

このバグを解決するには、`win_title` をローカル変数ではなく、グローバル変数あるいは `malloc()` で動的に生成した変数とする必要があります。`title_` の型を `std::string` にしておけば、内部で `malloc()` 相当の機能によりメモリを動的に確保し、そこに文字列をコピーして保持してくれるため、手軽に欲しい機能を実現できるのです。

15.3 ターミナルウィンドウ (osbook_day15c)

改造してかなりかっこいい感じになってきました。これくらいの機能があればターミナルを作っていけそうです。ターミナルはメインタスクとは別の、専用タスクとして作っていいと思います。そうすればターミナルを増やしたいときにタスクを増やすだけで簡単に増やせるようになります。この節ではターミナル専用タスクとそれっぽい見た目のウィンドウを作ることを目指します。

リスト 15.20 Terminal クラスはレイヤとウィンドウを内包する (terminal.hpp)

```
class Terminal {
public:
    static const int kRows = 15, kColumns = 60;

    Terminal();
    unsigned int LayerID() const { return layer_id_; }
    void BlinkCursor();

private:
    std::shared_ptr<ToplevelWindow> window_;
    unsigned int layer_id_;

    Vector2D<int> cursor_{0, 0};
    bool cursor_visible_{false};
    void DrawCursor(bool visible);
};

void TaskTerminal(uint64_t task_id, int64_t data);
```

リスト 15.20 に新しく作った Terminal クラスを示します。このクラスはターミナル用のウィンドウとレイヤを内部に持ち、コンストラクタでそれらを生成します。このクラスはカーソルの点滅を切り替える BlinkCursor() メソッドを備えます。カーソルの位置は cursor_ 変数で表すつもりです。このクラスに文字列を入力する機能はまだありません。次の節ではそのあたりを拡充していいと思います。

リスト 15.21 Terminal のコンストラクタではウィンドウを生成する (terminal.cpp)

```
Terminal::Terminal() {
    window_ = std::make_shared<ToplevelWindow>(
        kColumns * 8 + 8 + ToplevelWindow::kMarginX,
        kRows * 16 + 8 + ToplevelWindow::kMarginY,
        screen_config.pixel_format,
        "MikanTerm");
    DrawTerminal(*window_>InnerWriter(), {0, 0}, window_>InnerSize());

    layer_id_ = layer_manager->NewLayer()
        .SetWindow(window_)
        .SetDraggable(true)
        .ID();
}
```

リスト 15.21 に Terminal クラスのコンストラクタを示します。コンストラクタでは「MikanTerm」というタイトルのウィンドウと、そのウィンドウを保持するレイヤを生成します。レイヤの移動や高さの調整などは Terminal クラスの外側でやるようにしようかと思います（そんなに深い理由はありませんがその方が分かりやすいかと思ひまして）。

TopLevelWindow::kMarginX は今回増やした定数で、ウィンドウの左右の余白（枠の幅）の合計を表します。同様に TopLevelWindow::kMarginY はウィンドウの上下の枠とタイトルバーの幅の合計を表します。ここに 8 や 28 のような数値そのもの（リテラル）を書いてしまうと、後で OS のデザインを変えようとしてウィンドウの余白を変更したくなったときに変え忘れる可能性が高まります。このように名前付き定数を使えば、その定数の定義を 1 箇所変えるだけで、定数を使っている箇所すべてが自動的に修正されます。

DrawTerminal() は後で作る予定の関数です。ターミナルといえばやっぱり黒い背景で白い文字の画面ですね。この関数は指定した大きさと黒い背景のテキストボックスを描画します。

リスト 15.22 BlinkCursor() はカーソルの点滅を行う (terminal.cpp)

```
void Terminal::BlinkCursor() {
    cursor_visible_ = !cursor_visible_;
    DrawCursor(cursor_visible_);
}

void Terminal::DrawCursor(bool visible) {
    const auto color = visible ? ToColor(0xffffffff) : ToColor(0);
    const auto pos = Vector2D<int>{4 + 8*cursor_.x, 5 + 16*cursor_.y};
    FillRectangle(*window_>InnerWriter(), pos, {7, 15}, color);
}
```

リスト 15.22 にカーソルの点滅処理を担当する BlinkCursor() の実装を示します。後ほどタイマを使って 0.5 秒毎にこのメソッドを呼び出すようにする予定です。カーソルの表示、非表示を表す cursor_visible_ の値を反転させて、その状態に応じたカーソルを DrawCursor() により描画します。

DrawCursor() では、カーソルが表示状態のときは白で、非表示状態のときは黒でカーソルを描画します。ターミナルの背景色が黒ですので非表示 = 黒となります。通常であれば最後に layer_manager->Draw(layer_id_) などと画面を更新する必要があります。そうしないと画面上に FillRectangle() の結果が反映されません。しかし、DrawCursor() はメインタスクとは別タスクであるターミナル用タスク内で実行するつもりです。データ競合を避けるためにメインタスクへ画面更新依頼を送信する必要があります。その処理は後でタスクを実装するときに合わせて書こうと思ひます。

リスト 15.23 ターミナル専用タスク TaskTerminal() (terminal.cpp)

```
void TaskTerminal(uint64_t task_id, int64_t data) {
    __asm__("cli");
    Task& task = task_manager->CurrentTask();
    Terminal* terminal = new Terminal;
    layer_manager->Move(terminal->LayerID(), {100, 200});
    active_layer->Activate(terminal->LayerID());
    __asm__("sti");
}
```

```

while (true) {
    __asm__("cli");
    auto msg = task.ReceiveMessage();
    if (!msg) {
        task.Sleep();
        __asm__("sti");
        continue;
    }

    switch (msg->type) {
    case Message::kTimerTimeout:
        terminal->BlinkCursor();

        {
            Message msg{Message::kLayer, task_id};
            msg.arg.layer.layer_id = terminal->LayerID();
            msg.arg.layer.op = LayerOperation::Draw;
            __asm__("cli");
            task_manager->SendMessage(1, msg);
            __asm__("sti");
        }
        break;
    default:
        break;
    }
}
}
}

```

リスト 15.23 に TaskTerminal() の実装を示します。関数の冒頭では先ほど作った Terminal クラスをインスタンス化します。Terminal クラスのコンストラクタが生成したレイヤの ID を使って、ターミナルのウィンドウを適当な場所に移動し、アクティブ化をします。layer_manager->UpDown() ではなく active_layer->Activate() を使っているのは、そちらの方がマウスカーソルを考慮してくれて便利だからです。UpDown() ではマウスカーソルを考慮しないため、大きすぎる高さ値を指定するとマウスカーソルよりも前面に来てしまい、ウィンドウの下をマウスカーソルがくぐり抜けるという変な状態になります。

関数冒頭の初期化の部分はグローバル変数をたくさん使うので全体的に割り込みを禁止しておくことにします。初期化が終わったら割り込み禁止を解除します。

ループ内ではタイマのタイムアウトメッセージを受け取る処理を書きました。このタイマは 0.5 秒間隔でタイムアウトするカーソル点滅用タイマのつもりです。メッセージを受け取ったらカーソルの点滅用メソッド BlinkCursor() を呼び出します。実際にタイムアウトメッセージを TaskTerminal() に対して送る部分は後で作ります。

リスト 15.24 ターミナル用タスクを生成して起床させる (main.cpp)

```

const uint64_t task_terminal_id = task_manager->NewTask()
    .InitContext(TaskTerminal, 0)
    .Wakeup()
    .ID();

```

リスト 15.24 にターミナル用のタスクを生成する処理を載せました。TaskB を生成している箇所の直後に追加すれば良いでしょう。

リスト 15.25 カーソル用タイマのタイムアウトをターミナルにも通知する (main.cpp)

```
case Message::kTimerTimeout:
    if (msg->arg.timer.value == kTextboxCursorTimer) {
        __asm__("cli");
        timer_manager->AddTimer(
            Timer{msg->arg.timer.timeout + kTimer05Sec, kTextboxCursorTimer});
        __asm__("sti");
        textbox_cursor_visible = !textbox_cursor_visible;
        DrawTextCursor(textbox_cursor_visible);
        layer_manager->Draw(text_window_layer_id);

        __asm__("cli");
        task_manager->SendMessage(task_terminal_id, *msg);
        __asm__("sti");
    }
    break;
```

メインループの中でカーソル用タイマがタイムアウトした場合の処理を少し改造しました (**リスト 15.25**)。テキストボックスのカーソルを点滅させる従来からの処理に加えて、ターミナル用タスクにも同じメッセージを送ってあげることにします。

リスト 15.26 DrawTerminal() は黒いテキストボックスを描画する (window.cpp)

```
void DrawTextbox(PixelWriter& writer, Vector2D<int> pos, Vector2D<int> size) {
    DrawTextbox(writer, pos, size,
        ToColor(0xffffffff), ToColor(0xc6c6c6), ToColor(0x848484));
}

void DrawTerminal(PixelWriter& writer, Vector2D<int> pos, Vector2D<int> size) {
    DrawTextbox(writer, pos, size,
        ToColor(0x000000), ToColor(0xc6c6c6), ToColor(0x848484));
}
```

リスト 15.26 に DrawTerminal() の実装を示します。この関数はターミナル画面内部のテキストボックス、つまり黒い背景のテキストボックスを描画します。

DrawTerminal() でやりたいことは、色の違いを除くと **リスト 12.20** で定義した DrawTextbox() と一緒です。ですので、DrawTextbox() をコピーして DrawTerminal() を作ることもできますが、コードが重複するのが嫌です。ここでは共通の処理を新しい関数として切り出してコードが重複しないように実装したいと思います。

色情報を受け取るバージョンの DrawTextbox() を新しく作り、大部分の処理をそちらに移動することにしました。DrawTerminal() は新しく作った DrawTextbox() を、背景色を黒として呼び出すだけの簡単な実装になりました。そして、もともとの DrawTextbox() は背景色を白として新しい DrawTextbox() を呼び出すだけの簡単な中身になりました。

リスト 15.27 色を指定できるバージョンの DrawTextbox() (window.cpp)

```

namespace {
    void DrawTextbox(PixelWriter& writer, Vector2D<int> pos, Vector2D<int> size,
                    const PixelColor& background,
                    const PixelColor& border_light,
                    const PixelColor& border_dark) {
        auto fill_rect =
            [&writer](Vector2D<int> pos, Vector2D<int> size, const PixelColor& c) {
                FillRectangle(writer, pos, size, c);
            };

        // fill main box
        fill_rect(pos + Vector2D<int>{1, 1}, size - Vector2D<int>{2, 2}, background);
    }
}

```

リスト 15.27 に色情報を受け取るバージョンの DrawTextbox() を示します。引数で背景と枠の色を指定することができる点を除くと、やっていることは従来の DrawTextbox() と同じです。C++ ではこのように同じ名前でも引数の個数や型が異なる関数を定義できます。これを関数のオーバーロードと呼ぶのでした（「3.3 初めてのカーネル」参照）。

図 15.4 ターミナルのウィンドウにカーソルが表示された



ここまで作ったら実行をしてみます（図 15.4）。ターミナルのようなウィンドウが表示され、カーソルが点滅しています。ターミナル画面がでるととっても OS っぽさが増したと思いませんか？ 筆者はとてもワクワクしています。

15.4 描画の高速化 (osbook_day15d)

無事にターミナルが表示できたわけですが、1つ気になるところがあります。それはカーソルを点滅させるためにターミナル画面全体を再描画していることです。たった7×15ピクセルだけ再描画すれば良いはずなのに、大きな画面を再描画していてかなり無駄ですね。この節では無駄な再描画をしないように改造しておきます。

リスト 15.28 レイヤ ID に加えて範囲を指定できる Draw() を追加 (layer.cpp)

```
void LayerManager::Draw(unsigned int id) const {
    Draw(id, {{0, 0}}, {-1, -1});
}

void LayerManager::Draw(unsigned int id, Rectangle<int> area) const {
    bool draw = false;
    Rectangle<int> window_area;
    for (auto layer : layer_stack_) {
        if (layer->ID() == id) {
            window_area.size = layer->GetWindow()->Size();
            window_area.pos = layer->GetPosition();
            if (area.size.x >= 0 || area.size.y >= 0) {
                area.pos = area.pos + window_area.pos;
                window_area = window_area & area;
            }
            draw = true;
        }
        if (draw) {
            layer->DrawTo(back_buffer_, window_area);
        }
    }
    screen_->Copy(window_area.pos, back_buffer_, window_area);
}
```

手始めにレイヤ ID に加えて再描画範囲を指定できる Draw() メソッドを LayerManager クラスに追加しました (リスト 15.28)。新しく追加したメソッドでは、ウィンドウ全体の範囲 (window_area) と指定された描画範囲 (area) の共通部分を計算し、その部分に限定して再描画を行います。描画範囲 area はウィンドウの左上を基準とした座標になっている一方、window_area は画面の左上を基準とした座標です。座標系を合わせるために area.pos に window_area.pos を足してから共通部分を計算しましょう。

今まであったレイヤ ID だけを指定するバージョンのメソッドは、新しく追加したメソッドを特殊な引数で呼び出すだけになりました。

リスト 15.29 LayerOperation に種類を追加 (message.hpp)

```
enum class LayerOperation {
    Move, MoveRelative, Draw, DrawArea
};
```

メインタスク以外からメインタスクへ範囲付きの再描画を要求できるように、メッセージ構造体に指定できる `LayerOperation` の種類を増やします (リスト 15.29)。今まであった範囲なしの描画要求 `Draw` に加え、範囲指定ありのバージョン `DrawArea` を増やしたのが分かります。

リスト 15.30 描画範囲を設定するフィールドを追加 (message.hpp)

```
struct {
    LayerOperation op;
    unsigned int layer_id;
    int x, y;
    int w, h;
} layer;
```

リスト 15.30 に示すように、メッセージ構造体に描画範囲を指定するためのフィールド `int w, h` を追加しました。変数名はそれぞれ `width` (幅) と `height` (高さ) の頭文字です。

リスト 15.31 範囲付き再描画リクエストを追加する (layer.cpp)

```
case LayerOperation::DrawArea:
    layer_manager->Draw(arg.layer_id, {{arg.x, arg.y}, {arg.w, arg.h}});
    break;
```

`ProcessLayerMessage()` の `switch` 文に `DrawArea` に対応するケースを追加しました (リスト 15.31)。

リスト 15.32 再描画要求を範囲付きバージョンに切り替える (terminal.cpp)

```
case Message::kTimerTimeout:
{
    const auto area = terminal->BlinkCursor();
    Message msg = MakeLayerMessage(
        task_id, terminal->LayerID(), LayerOperation::DrawArea, area);
    __asm__("cli");
    task_manager->SendMessage(1, msg);
    __asm__("sti");
}
break;
```

リスト 15.32 に示すように、追加した描画範囲付きのバージョンを使うように `TaskTerminal()` を改造します。 `terminal->BlinkCursor()` は今まで何も値を返していませんでしたが、これからは再描画すべき範囲を返すことにします。その情報を使ってメッセージ構造体を作りメインタスクへ送信します。 `MakeLayerMessage()` はメッセージ構造体を作るための関数です。後ほど作ります。

リスト 15.33 BlinkCursor() は描画範囲を返す (terminal.cpp)

```
Rectangle<int> Terminal::BlinkCursor() {
    cursor_visible_ = !cursor_visible_;
    DrawCursor(cursor_visible_);

    return {TopLevelWindow::kTopLeftMargin +
            Vector2D<int>{4 + 8*cursor_.x, 5 + 16*cursor_.y},
            {7, 15}};
}
```

リスト 15.33 に示すように、再描画すべき範囲を `Terminal::BlinkCursor()` が返すようにしました。再描画範囲の座標系はウィンドウの左上を基準にしなければなりません。そのためウィンドウ枠内の座標に `kTopLeftMargin` を足して、ウィンドウ左上からの座標に変換します。

リスト 15.34 MakeLayerMessage() はレイヤ操作要求メッセージを作る (layer.hpp)

```
constexpr Message MakeLayerMessage(
    uint64_t task_id, unsigned int layer_id,
    LayerOperation op, const Rectangle<int>& area) {
    Message msg{Message::kLayer, task_id};
    msg.arg.layer.layer_id = layer_id;
    msg.arg.layer.op = op;
    msg.arg.layer.x = area.pos.x;
    msg.arg.layer.y = area.pos.y;
    msg.arg.layer.w = area.size.x;
    msg.arg.layer.h = area.size.y;
    return msg;
}
```

リスト 15.34 に `MakeLayerMessage()` の実装を示します。与えられた要求の種類 `op` や座標情報などからメッセージ構造体を作ります。この関数を使うと何行も書いていた箇所をすっきりさせられますね。

さて、この改造による改善度合いを Local APIC タイマで計ってみます。性能測定は久々ですね。次のような計測プログラムを仕込んで、範囲指定の有無でそれぞれ時間を計ります。

```
case LayerOperation::Draw:
    if (arg.layer_id == 7) {
        auto start = LAPICTimerElapsed();
        layer_manager->Draw(arg.layer_id);
        auto elapsed = LAPICTimerElapsed() - start;
        Log(kWarn, "draw layer 7: elapsed = %u\n", elapsed);
        break;
    }
    layer_manager->Draw(arg.layer_id);
    break;
case LayerOperation::DrawArea:
    if (arg.layer_id == 7) {
        auto start = LAPICTimerElapsed();
        layer_manager->Draw(arg.layer_id, {{arg.x, arg.y}, {arg.w, arg.h}});
```

```

    auto elapsed = LAPICTimerElapsed() - start;
    Log(kWarn, "draw layer 7: elapsed = %u\n", elapsed);
    break;
}
layer_manager->Draw(arg.layer_id, {{arg.x, arg.y}, {arg.w, arg.h}});
break;

```

計測の結果を紹介します。図 15.5 は範囲を指定しないバージョン (LayerOperation::Draw) で描画した場合の計測結果です。平均 240 万カウントほどの時間がかかっています。

図 15.5 描画範囲を制限しなかった場合の性能

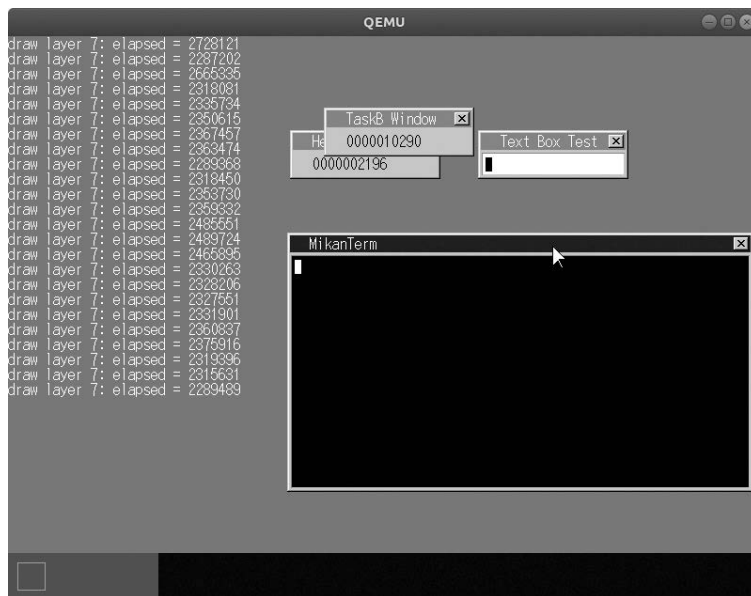
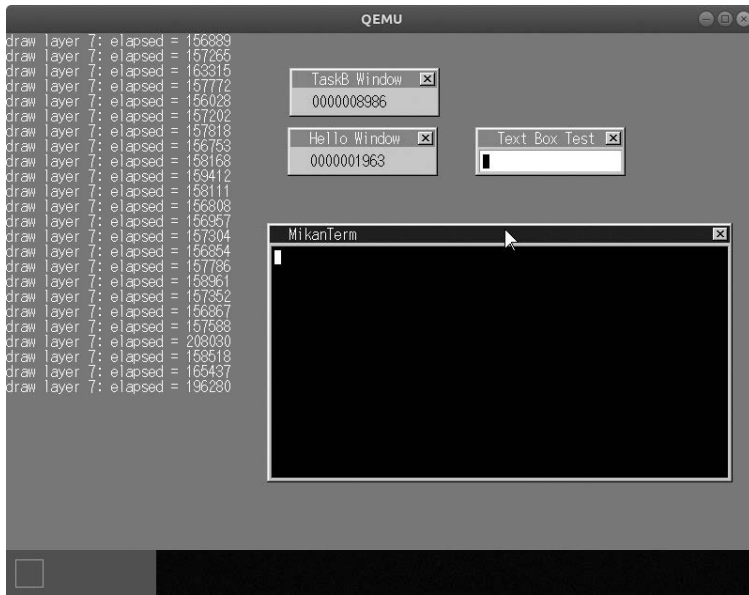


図 15.6 は範囲を指定するバージョン (LayerOperation::DrawArea) で描画した場合の計測結果です。平均 15 万カウントほどまで短縮できました。およそ 16 倍の高速化です。高速化の甲斐が十分にありましたね！

図 15.6 描画範囲をカーソルの 7×15 に限定したときの性能

高速化ができたのでいよいよターミナルに文字入力機能を付けよう、と思ったのですが、もう今日は大分疲れてしまいました。ということで、文字入力は次章でやろうと思います。

第 16 章

コマンド

この章ではまずターミナルに文字を入力できるようにします。キーを打ってターミナルに文字が入力できるとかなりかっこよくなります。その後、echo という単に引数を表示するだけの簡単なコマンドや、画面をまっさらな状態に戻す clear コマンド、PCI デバイス一覧を表示する lspci コマンドを作ります。それに加え、過去に実行したコマンドを呼び出せるコマンド履歴機能を実装します。この章が終わるころには、前章まで単なる黒い画面だったターミナルで 3 つのコマンドを実行できるようになります。もうそれは OS じゃないかと思う程度に OS っぽくなりますよ。

16.1 ターミナルでキー入力 (osbook_day16a)

前章でやり残したターミナルのキー入力機能を作っていきます。ターミナル画面がアクティブになっているときにキーを入力すると、ターミナル上に白い文字が入力できるようになることを目指します。まずは Terminal クラスにキー入力のためのメソッドを作り、次にそのメソッドを使ってキー情報を送るようにしたいと思います。

リスト 16.1 Terminal クラスにラインバッファを追加 (terminal.hpp)

```
private:
    std::shared_ptr<ToplevelWindow> window_;
    unsigned int layer_id_;

    Vector2D<int> cursor_{0, 0};
    bool cursor_visible_{false};
    void DrawCursor(bool visible);
    Vector2D<int> CalcCursorPos() const;

    int linebuf_index_{0};
    std::array<char, kLineMax> linebuf_{};
    void Scroll1();
```

リスト 16.1 のように、キー入力を 1 行分ためておくバッファ linebuf_ を追加しました。入力された文字を画面に表示するだけなら不要なのですが、入力した文字列を使って何らかの処理をするためには必要になります。そういう処理はすぐにしたくなるはずなので、ここでついでに追加しておきます。Scroll1() は、入力がターミナルの最終行にきたときに 1 行だけ画面を上にならずメソッドです。メソッド名の 1 は 1 行という意味です。

リスト 16.2 kLineMax はラインバッファの最大文字数を表す (terminal.hpp)

```
static const int kLineMax = 128;
```

linebuf_ の大きさを表す kLineMax は、**リスト 16.2** のように Terminal クラスの中で定義した定数です。

リスト 16.3 CalcCursorPos() はカーソルのピクセル座標を計算する (terminal.cpp)

```
Vector2D<int> Terminal::CalcCursorPos() const {
    return ToplevelWindow::kTopLeftMargin +
        Vector2D<int>{4 + 8 * cursor_.x, 4 + 16 * cursor_.y};
}
```

CalcCursorPos() はカーソルの文字単位の位置を表す cursor_ 変数からウィンドウの左上を基準としたピクセル座標を計算するメソッドです。**リスト 16.3** のような簡単な関数です。キー入力処理を作るときにこの計算を複数の場所でやる必要があるため、メソッドとして切り出しました。ソースコードは載せませんが、BlinkCursor() や DrawCursor() もこのメソッドを使うように修正しました。

リスト 16.4 InputKey() はキー入力を受け付ける (terminal.cpp)

```

Rectangle<int> Terminal::InputKey(
    uint8_t modifier, uint8_t keycode, char ascii) {
    DrawCursor(false);

    Rectangle<int> draw_area{CalcCursorPos(), {8*2, 16}};

    if (ascii == '\n') {
        linebuf_[linebuf_index_] = 0;
        linebuf_index_ = 0;
        cursor_.x = 0;
        Log(kWarn, "line: %s\n", &linebuf_[0]);
        if (cursor_.y < kRows - 1) {
            ++cursor_.y;
        } else {
            Scroll1();
        }
        draw_area.pos = ToplevelWindow::kTopLeftMargin;
        draw_area.size = window_->InnerSize();
    } else if (ascii == '\b') {
        if (cursor_.x > 0) {
            --cursor_.x;
            FillRectangle(*window_->Writer(), CalcCursorPos(), {8, 16}, {0, 0, 0});
            draw_area.pos = CalcCursorPos();

            if (linebuf_index_ > 0) {
                --linebuf_index_;
            }
        }
    } else if (ascii != 0) {
        if (cursor_.x < kColumns - 1 && linebuf_index_ < kLineMax - 1) {
            linebuf_[linebuf_index_] = ascii;
            ++linebuf_index_;
            WriteAscii(*window_->Writer(), CalcCursorPos(), ascii, {255, 255, 255});
            ++cursor_.x;
        }
    }

    DrawCursor(true);

    return draw_area;
}

```

リスト 16.4 に InputKey() の実装を示します。このメソッドは 1 つのキー入力を受け付けて文字入力や改行の処理をします。戻り値は BlinkCursor() と同様、再描画すべき範囲を返します。

現在のところ、改行 (\n) とバックスペース (\b) は特別扱いをしています。改行文字であれば linebuf_ の内容をコンソールに表示し、次の行にカーソルを進めます。もしカーソルがすでに最終行にある場合は 1 行だけスクロールします。バックスペースであれば、カーソルの位置にある文字を 1 つ消してカーソルを左に動かします。その他の ASCII コードであれば linebuf_ の末尾に追加し、画面にその文字を表示します。改行とバックスペース以外にも特別視すべき文字はありますが（例えばタブ文字など）、これ以上複雑にしないために今は無視します。

リスト 16.5 Scroll1() は 1 行だけ表示をずらす (terminal.cpp)

```
void Terminal::Scroll1() {
    Rectangle<int> move_src{
        ToplevelWindow::kTopLeftMargin + Vector2D<int>{4, 4 + 16},
        {8*kColumns, 16*(kRows - 1)}}
    };
    window->Move(ToplevelWindow::kTopLeftMargin + Vector2D<int>{4, 4}, move_src);
    FillRectangle(*window->InnerWriter(),
        {4, 4 + 16*cursor_.y}, {8*kColumns, 16}, {0, 0, 0});
}
```

リスト 16.5 に Scroll1() の実装を示します。このメソッドは画面の 2 行目から最終行までを 1 行だけ上に移動させ、最終行を黒で塗りつぶします。似たような処理を「**9.7 スクロール処理の高速化**」でもやりましたね。

ここまで来たら Terminal クラス内の修正は完了です。次に、作成した InputKey() メソッドを利用してターミナルに文字を入力する処理を作ります。ターミナルはメインタスクとは独立したタスクとして作ってありますから、ターミナルにキーを入力するためにはターミナルタスクのメッセージキューを利用することにしましょう。今のところターミナルタスクのメッセージキューにはカーソル点滅のためのタイムアウトメッセージだけが来ますが、そこにキー入力メッセージも加えるようにします。

リスト 16.6 TaskTerminal() のループでキーイベントを処理する (terminal.cpp)

```
case Message::kKeyPush:
{
    const auto area = terminal->InputKey(msg->arg.keyboard.modifier,
                                         msg->arg.keyboard.keycode,
                                         msg->arg.keyboard.ascii);

    Message msg = MakeLayerMessage(
        task_id, terminal->LayerID(), LayerOperation::DrawArea, area);
    __asm__("cli");
    task_manager->SendMessage(1, msg);
    __asm__("sti");
}
break;
```

早速、TaskTerminal() のメッセージループに kKeyPush に対応するケースを追加しました (**リスト 16.6**)。キー入力メッセージが来たら、先ほど作った InputKey() を呼び出し、ターミナル画面を再描画します。再描画はもちろんメインループに再描画要求を投げることで行います。

これで、キー入力メッセージが来れば処理ができる状態になりました。問題は、どうやってキー入力メッセージをターミナルタスクに送るかです。やりたいのはアクティブなウィンドウに対してキー入力メッセージを送ることです。アクティブなウィンドウを持つタスクを探してきて、そのタスクのメッセージキューに kKeyPush を送ればいいですね。しかし今のところウィンドウとタスクを関連付けるものはありませんので、作る必要があります。

リスト 16.7 layer_task_map はレイヤ ID とタスクを関連付ける (layer.cpp)

```
ActiveLayer* active_layer;
std::map<unsigned int, uint64_t>* layer_task_map;
```

リスト 16.7 のようにレイヤ ID とタスクを関連付けるマップ layer_task_map を定義しました。先ほどはウィンドウとタスクを関連付けると書きましたが、アクティブ、非アクティブの管理にレイヤ ID を使っている都合で、ここでもレイヤ ID とタスクを関連付けることにします。std::map を使い、レイヤ ID からタスク ID を検索できるようにしました。

リスト 16.8 ターミナルタスクを検索表に登録する (terminal.cpp)

```
layer_task_map->insert(std::make_pair(terminal->LayerID(), task_id));
__asm__("sti");
```

layer_task_map にターミナル画面とターミナルタスクを登録するプログラムをリスト 16.8 に示します。TaskTerminal() の初期化部分からの抜粋です。

リスト 16.9 検索表からタスクを取得してキー入力メッセージを投げる (main.cpp)

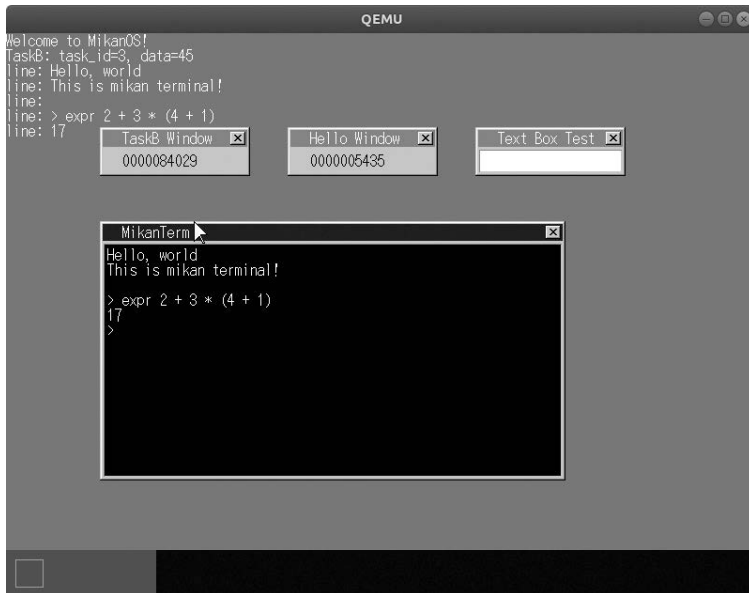
```
case Message::kKeyPush:
    if (auto act = active_layer->GetActive(); act == text_window_layer_id) {
        InputTextWindow(msg->arg.keyboard.ascii);
    } else if (act == task_b_window_layer_id) {
        if (msg->arg.keyboard.ascii == 's') {
            printf("sleep TaskB: %s\n", task_manager->Sleep(taskb_id).Name());
        } else if (msg->arg.keyboard.ascii == 'w') {
            printf("wakeup TaskB: %s\n", task_manager->Wakeup(taskb_id).Name());
        }
    } else {
        __asm__("cli");
        auto task_it = layer_task_map->find(act);
        __asm__("sti");
        if (task_it != layer_task_map->end()) {
            __asm__("cli");
            task_manager->SendMessage(task_it->second, *msg);
            __asm__("sti");
        } else {
            printf("key push not handled: keycode %02x, ascii %02x\n",
                msg->arg.keyboard.keycode,
                msg->arg.keyboard.ascii);
        }
    }
    break;
```

リスト 16.9 にメインループ内のキー入力メッセージの処理部分を示します。外側の if 文ではアクティブになっているウィンドウによって場合分けをしています。アクティブなウィンドウが text_window_layer_id でも task_b_window_layer_id でもない場合、今までは「key push not handled」というログを出すだけでした。そこを改造して、検索表からアクティブなレイヤ ID に対応

するタスク ID を探し、見つければそのタスクに対してキー入力メッセージを送信します。

`layer_task_map->find(act)` はアクティブなレイヤの ID をキーとして検索します。そのキーが登録されていた場合、`find()` メソッドはキー値ペアを指すイテレータを返します。すなわち `task_it->first` は `act` であり、`task_it->second` はタスク ID を表します。`find()` がキーを見つけられなかった場合は `layer_task_map->end()` を返します。

図 16.1 ターミナルにキー入力をしている様子



作り終わったら実行してみます。起動してキーをポチポチっと……(図 16.1)。なんということでしょう！ ターミナルに文字が入力できましたよ！ 記号も入力できますし、エンターキーを押すとその行に入力した文字列がログとして出ます。完璧です！

16.2 echo コマンド (osbook_day16b)

ターミナルが完成したので何かコマンドを作ってみたくになりました。最初は簡単に echo コマンドでも作ってみましょうか。echo コマンドとは、引数をそのまま画面に出力するコマンドです。次のように使います。

```
> echo this is a pen
this is a pen
```

1 行目の左にある `>` は**プロンプト** (prompt) です。プロンプトという英単語は「ユーザーに行動を促す」という意味がありまして、ユーザーにコマンド入力を促すような表示であることからそう呼ばれます。Linux でよく使われるプロンプトには `$` や `#` がありますね。いずれにせよ、システムがコマンド

入力を受け付けられる状態にあることを表すためにプロンプトが用いられます。

リスト 16.10 コンストラクタでプロンプトを表示する (terminal.cpp)

```
Terminal::Terminal() {
    window_ = std::make_shared<TopLevelWindow>(
        kColumns * 8 + 8 + TopLevelWindow::kMarginX,
        kRows * 16 + 8 + TopLevelWindow::kMarginY,
        screen_config.pixel_format,
        "MikanTerm");
    DrawTerminal(*window_>InnerWriter(), {0, 0}, window_>InnerSize());

    layer_id_ = layer_manager->NewLayer()
        .SetWindow(window_)
        .SetDraggable(true)
        .ID();

    Print(">");
}
```

プロンプトを表示するためにリスト 16.10 のように Terminal クラスのコンストラクタの最後に `Print(">");` を追加しました。Print() はこれから実装する関数で、指定された文字列を画面に表示します。

リスト 16.11 Print() は指定された文字列をターミナルへ表示する (terminal.cpp)

```
void Terminal::Print(const char* s) {
    DrawCursor(false);

    auto newline = [this]() {
        cursor_.x = 0;
        if (cursor_.y < kRows - 1) {
            ++cursor_.y;
        } else {
            Scroll1();
        }
    };

    while (*s) {
        if (*s == '\n') {
            newline();
        } else {
            WriteAscii(*window_>Writer(), CalcCursorPos(), *s, {255, 255, 255});
            if (cursor_.x == kColumns - 1) {
                newline();
            } else {
                ++cursor_.x;
            }
        }
        ++s;
    }
}
```

```
DrawCursor(true);
}
```

`Print()` の実装をリスト 16.11 に示します。この関数は引数で指定された文字列をターミナルに表示する関数です。プロンプトを表示する際や、後で作る `echo` コマンドが引数を表示する際に利用する予定です。`InputKey()` とは異なり、表示する際に `linebuf_` や `linebuf_index_` を変更しないことが特徴です。

関数の初めの方でラムダ式を定義しています。これは改行文字が来たときにカーソルを 1 行下にずらすか、カーソルがすでに最下行に来ていたら 1 行だけスクロールします。`while` ループの中 2 箇所を使うのでラムダ式として共通化しました。ラムダ式を使うと、メソッドや関数として抜き出すのは適さないようなコード片でも手軽に部品化できて便利です。

`if (cursor_.x == kColumns - 1)` の `if` 文に注目してください。この条件が成り立つということはカーソルがターミナルの右端に来ているということです。このときに `newline()` を実行します。これにより、1 行に収まらない長い文字列を `Print()` しても、画面右端で折り返して表示できる機能を実現しています。`echo` コマンドの場合は引数を単に表示するだけですから、引数が 1 行分しか入力できない以上、この折り返し機能が実際に役立つことはありませんけど……。

リスト 16.12 エンターキーが押されたら `ExecuteLine()` を呼ぶ (terminal.cpp)

```
if (ascii == '\n') {
    linebuf_[linebuf_index_] = 0;
    linebuf_index_ = 0;
    cursor_.x = 0;
    if (cursor_.y < kRows - 1) {
        ++cursor_.y;
    } else {
        Scroll1();
    }
    ExecuteLine();
    Print(">");
    draw_area.pos = ToplevelWindow::kTopLeftMargin;
    draw_area.size = window_>InnerSize();
} else if (ascii == '\b') {
```

リスト 16.12 にエンターキーが押されたときの処理を示します。今までは単に `Log()` を使ってコンソールに `linebuf_` の内容を表示するだけでした。それをやめて `ExecuteLine()` を呼び出すようにしたのが主な変更です。その後、再度プロンプトを表示しておきます。

リスト 16.13 `ExecuteLine()` はコマンドを実行する (terminal.cpp)

```
void Terminal::ExecuteLine() {
    char* command = &linebuf_[0];
    char* first_arg = strchr(&linebuf_[0], ' ');
    if (first_arg) {
        *first_arg = 0;
        ++first_arg;
    }
}
```



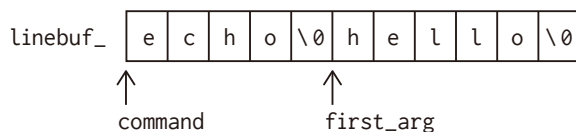
```

if (strcmp(command, "echo") == 0) {
    if (first_arg) {
        Print(first_arg);
    }
    Print("\n");
} else if (command[0] != 0) {
    Print("no such command: ");
    Print(command);
    Print("\n");
}
}

```

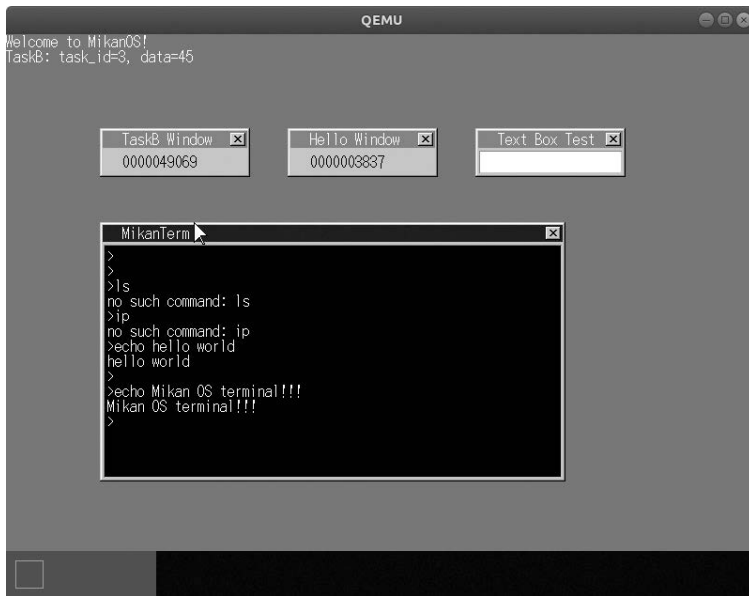
リスト 16.13 に ExecuteLine() の実装を示します。このメソッドは linebuf_ の内容をもとにコマンドを実行します。linebuf_ の先頭から最初の空白文字、または NUL 文字までをコマンド名だと認識し、そのコマンドを実行します。

図 16.2 linebuf_ をコマンド名と引数に分割する様子



コマンド名の後には引数がある場合があります。関数の 2 行目ではコマンド名の直後の空白の位置を strchr によって検索しています。空白があれば first_arg にはその位置が、空白が見つからなければヌルポインタが設定されます。その直後の if 文では、空白が見つかった場合にその位置へ NUL 文字を書き込み、first_arg を 1 つ前に進めます。「echo hello」というコマンドを実行した場合に linebuf_ がどうなるかを図 16.2 に示しました。

図 16.3 echo コマンドで遊ぶ



ここまで改造したら実行してみます（図 16.3）。ちゃんと echo コマンドが使えます。そして存在しないコマンドもちゃんと「no such command」と表示されますね。よしよし、うまく行っているようです。

16.3 clear コマンド (osbook_day16c)

echo コマンドで遊んでいたらターミナルがごちゃごちゃしてきたので、ターミナル上の文字を消す clear コマンドを実装しようと思います。

リスト 16.14 clear コマンドを追加する (terminal.cpp)

```
if (strcmp(command, "echo") == 0) {
    if (first_arg) {
        Print(first_arg);
    }
    Print("\n");
} else if (strcmp(command, "clear") == 0) {
    FillRectangle(*window_ -> InnerWriter(),
        {4, 4}, {8*kColumns, 16*kRows}, {0, 0, 0});
    cursor_.y = 0;
} else if (command[0] != 0) {
```

やってみたらとっても簡単に実装できてしまいました（リスト 16.14）。なんと、たったの 4 行の追加だけです。今までいろいろ下準備をしてきた成果が出ていますね！

図 16.4 clear コマンドを実行した様子



clear コマンドを実行したら図 16.4 のようにターミナルの表示が全部消えて、カーソルが左上に戻りました。起動直後の写真じゃないかって？ いや、本当に clear を実行したんですってば！

16.4 lspci コマンド (osbook_day16d)

もう少し実用的なコマンドを作りたくなりました。何にしましょうか……。ああ、そうだ。パソコンに接続されている PCI デバイスのリストを表示できるようにしてみましょう。Linux にある lspci コマンドと同じように、各 PCI デバイスのバス番号やクラス ID を一覧表示します。

リスト 16.15 lspci コマンドを追加する (terminal.cpp)

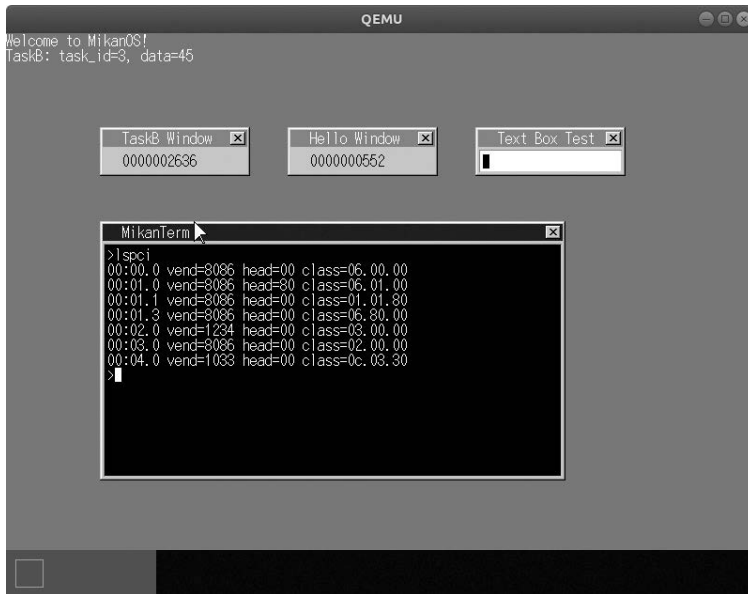
```

} else if (strcmp(command, "lspci") == 0) {
    char s[64];
    for (int i = 0; i < pci::num_device; ++i) {
        const auto& dev = pci::devices[i];
        auto vendor_id = pci::ReadVendorId(dev.bus, dev.device, dev.function);
        sprintf(s, "%02x:%02x.%d vend=%04x head=%02x class=%02x.%02x.%02x\n",
            dev.bus, dev.device, dev.function, vendor_id, dev.header_type,
            dev.class_code.base, dev.class_code.sub, dev.class_code.interface);
        Print(s);
    }
} else if (command[0] != 0) {

```

リスト 16.15 に lspci コマンドの実装を示します。「6.3 PCI デバイスの探索」で登場した pci::devices 配列を 1 つずつ表示するだけで、特に難しいことはないと思います。

図 16.5 lspci コマンドを実行した様子



QEMU 上で `lspci` を実行してみたところ図 16.5 のようになりました。class=0c.03.30 が xHCI コントローラのクラス ID です。そのほかにもいろいろな部品が搭載されているなあということが分かります。ところで、自作 OS で `lspci` コマンドが実行できるって、かなり感動しませんか？ 筆者は感動しました。

16.5 コマンド履歴 (osbook_day16e)

Linux や Windows など、他の OS のターミナルでは矢印キーの上を押すと以前に入力したコマンドを呼び出すことができます。これはコマンド履歴と呼ばれる機能です。かっこいいので MikanOS にも搭載してみたくまりました。そんなに難しくない気がするのでやってみます。

リスト 16.16 コマンド履歴を保存するキュー `cmd_history_` 変数 (terminal.hpp)

```
std::deque<std::array<char, kLineMax>> cmd_history_{};
int cmd_history_index_{-1};
Rectangle<int> HistoryUpDown(int direction);
```

まずはコマンドの履歴を保存しておくための場所となる変数を作ります。Terminal クラスのメンバ変数として `cmd_history_` を追加しました（リスト 16.16）。この変数はリング状になっていて、各要素は 1 つのコマンド文字列（最大 `kLineMax` 文字）を保持します。最も新しいコマンドが先頭（インデックス 0）に来るようにしようと思います。矢印キーの上下でコマンド履歴を 1 つずつ辿っていく際に、現在の場所を記憶する変数が `cmd_history_index_` です。-1 は、履歴を辿っていない状態を表します。

リスト 16.17 コマンド履歴は最大 8 個 (terminal.cpp)

```
Print(">");  
cmd_history_.resize(8);
```

Terminal クラスのコンストラクタで cmd_history_ の要素数を決めます (リスト 16.17)。ここで決めた数の履歴を保存することができます。

リスト 16.18 エンターキーが押されたらコマンド履歴を登録する (terminal.cpp)

```
if (ascii == '\n') {  
    linebuf_[linebuf_index_] = 0;  
    if (linebuf_index_ > 0) {  
        cmd_history_.pop_back();  
        cmd_history_.push_front(linebuf_);  
    }  
    linebuf_index_ = 0;  
    cmd_history_index_ = -1;
```

リスト 16.18 に示すように、エンターキーが押されたときにコマンド履歴を登録するようにしました。linebuf_ に 1 文字以上の入力があるときに cmd_history_ の先頭に履歴を登録します。コンストラクタで設定した履歴の最大数を超えないように、cmd_history_ の末尾から 1 つ削除しておくことも忘れずにやっておきます。

コマンド履歴を上下キーでたどり、希望するコマンドを発見したらエンターキーで実行するというのが典型的な使い方です。エンターキーを押したら cmd_history_index_ を -1 にリセットし、コマンド履歴の探索モードを解除します。

リスト 16.19 上下キーでコマンド履歴をたどる (terminal.cpp)

```
} else if (keycode == 0x51) { // down arrow  
    draw_area = HistoryUpDown(-1);  
} else if (keycode == 0x52) { // up arrow  
    draw_area = HistoryUpDown(1);  
}
```

リスト 16.19 のように、上下キーを押した場合の if 文を追加しました。上下キーは ASCII コードを割り当てられていないキーですからキーコードで判定します。上下キーのキーコードは参考文献 [13] の「Table 12: Keyboard/Keypad Page」から持ってきました。

リスト 16.20 コマンド履歴を上下にたどる (terminal.cpp)

```
Rectangle<int> Terminal::HistoryUpDown(int direction) {  
    if (direction == -1 && cmd_history_index_ >= 0) {  
        --cmd_history_index_;  
    } else if (direction == 1 && cmd_history_index_ + 1 < cmd_history_.size()) {  
        ++cmd_history_index_;  
    }  
  
    cursor_.x = 1;
```

```

const auto first_pos = CalcCursorPos();

Rectangle<int> draw_area{first_pos, {8*(kColumns - 1), 16}};
FillRectangle(*window_>Writer(), draw_area.pos, draw_area.size, {0, 0, 0});

const char* history = "";
if (cmd_history_index_ >= 0) {
    history = &cmd_history_[cmd_history_index_][0];
}

strcpy(&linebuf_[0], history);
linebuf_index_ = strlen(history);

WriteString(*window_>Writer(), first_pos, history, {255, 255, 255});
cursor_.x = linebuf_index_ + 1;
return draw_area;
}

```

HistoryUpDown() の定義をリスト 16.20 に示します。このメソッドがコマンド履歴機能の中核で、引数で与えられた方向にコマンド履歴をたどります。1 でより古い履歴、-1 でより新しい履歴です。メソッドが実行された時点の cmd_history_index_ 変数の値を基準として、指定された方向に 1 つ進んだ場所にあるコマンド履歴を linebuf_ にコピーし、ターミナルに表示します。

メソッドの冒頭では、引数の値に応じて cmd_history_index_ を変化させます。上矢印(↑)キーを押すと direction は 1 となりますから、cmd_history_index_ は 1 ずつ増え、古くなる方向へと履歴をたどります。下キーを押すと direction は -1 となり、cmd_history_index_ が 1 ずつ減ることで新しい方へとたどります。

direction が 1 の場合の条件式の右辺に注目します。&& 演算子の右辺は、次の 2 行のうち上の書き方をしています。一見すると下の書き方でも良いように見えますが、実はうまくいかないのです。C++ 力を養う良い教材だと思いますので、少し詳しく理由を説明します。

```

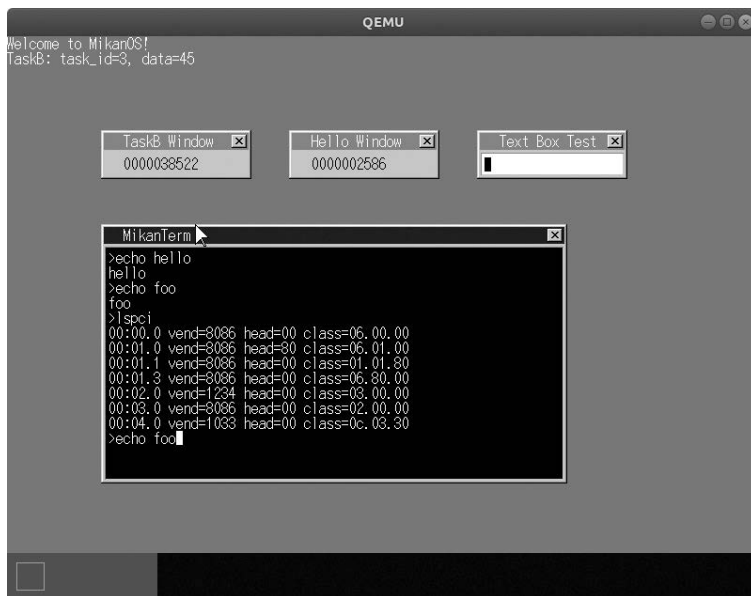
cmd_history_index_ + 1 < cmd_history_.size() // 実際の書き方
cmd_history_index_ < cmd_history_.size() - 1 // だめな書き方

```

下の書き方がだめなのは、cmd_history_index_ が -1 の場合に無符号整数型である size_t に変換されてしまうからです。比較演算子の左辺にある cmd_history_index_ は符号付き整数(int)です。対して右辺にある cmd_history_.size() は無符号整数 (size_t) です。MikanOS の開発環境で採用している処理系では size_t の方が int より大きい型^{*1}ですので、int は size_t に変換されてから左右が比較されることになります。

下の書き方では、cmd_history_index_ が -1 であるときに size_t へ変換されてしまいます。-1 を無符号整数型に変換すると、その型が表せる最大の数になります。今回であれば size_t が表せる最大の数になります。そのため、比較演算子 < の左辺が右辺より大きな数になってしまい、本来は真になるべき条件式が偽になってしまうのです。一方、上の書き方をすれば、cmd_history_index_ + 1 の結果が無符号整数に変換されることになりますから、たとえ cmd_history_index_ が -1 であっても問題になりません。

図 16.6 コマンド履歴をたどっている様子



OSを実行してコマンド履歴の機能を使ってみます（図 16.6）。いくつかコマンドを入力してから上矢印（↑）キーを押してみます。すると、新しいコマンドから古いコマンドへと1つずつ履歴をさかのぼっていくことができました。成功です。

16.6 省電力化（osbook_day16f）

筆者は MikanOS を起動するたびに CPU の冷却ファンの回転数が上がるのが気になっています。なぜかと考えてみると、TaskB が全力でカウントし続けていることを思い出しました。今となっては役に立っていないどころか、電気を無駄に使っています。特に筆者は、ノートパソコンをバッテリーで動かしているときも自作 OS 開発をすることがありまして、なるべく消費電力を抑えたいのです。ということで TaskB を削除しておくことにします。

TaskB 関連のソースコードを削除しただけですから変更点の説明はしませんが、Git のタグは打っておきました。修正点を確認したい方は osbook_day16e と osbook_day16f の差分をご覧ください。

* 1 「大きい型」とは、正確に言えば整数変換の順位（integer conversion rank）が高い型ということです。

図 16.7 TaskB 関連のソースコードを削除



起動してみると TaskB のウィンドウがなくなったことが分かります (図 16.7)。また、Linux 上で `top` コマンドなどを使って CPU 使用率を確認してみると、QEMU の CPU 使用率が大幅に小さくなっているはず。筆者の環境では、osbook_day16e の時点では QEMU の CPU 使用率が常時 100% を超えていましたが、今は最大で数 % 程度にとどまっています。これで、電池駆動のときも安心して OS の動作確認ができるようになりました。めでたし、めでたし。

第 17 章

ファイルシステム

この章では MikanOS がファイルを扱えるようにします。まずはファイルを扱う際に重要なファイルシステムについて紹介します。本書では、UEFI BIOS と親和性の高い FAT ファイルシステムを扱うことにします。ファイルシステムをざっと説明したら、次は実際に起動メディア（OS 本体である `kernel.elf` が入っているメディア）上のファイルを読み込む機能を実装していきます。この章が終わる頃には起動メディアに入っているファイルを一覧できるようになります。

17.1 ファイルとファイルシステム

何気なくファイルと言っていますが、ファイルとは何でしょうか？ 一般的に、ファイルは任意のバイト列に名前を付けたものです。例えば MikanOS の本体である `kernel.elf` は実行可能バイナリに「`kernel.elf`」という名前を付けたものですね。名前以外にも様々なメタデータが付くことがあります。作成日時、変更日時、所有者、権限などなど。

ファイルは今では様々なデータを記憶する用途に幅広く用いられますが、データの記録にはファイルが必須というわけではありません。例えば SSD や USB メモリなどはデータ領域が固定の大きさを持つブロックに分割されていて、ブロック単位でデータを読み書きします（このような記憶装置のことを**ブロックデバイス**と呼びます）。1 ブロックで足りないデータは必要な個数のブロックを繋げればいいだけです。この方法はそれぞれのデータの大きさがあまり変化しない場合はうまく機能します。初期の大きさを基準として、少し余裕をもたせたブロック数を割り当てておけば良いのです。また、データにアクセスする際は開始ブロック番号を与えればよく、データの場所の管理を考えなくて済みます。この方法の欠点は、データを指定するときに名前ではなく番号が必要な点と、データのサイズが大きくなりすぎると区画に収まらず、小さくなりすぎると空き領域が無駄になる点です。

ファイル方式ではデータに名前を付けて管理します。そのために名前とブロック番号の対応表を使います。この対応表を工夫することによって、あるファイルが複数のブロックにとびとびに記録される状況をも表現できます。その表現力のおかげで、データが大きくなったら動的にブロックを割り当てたり、小さくなれば不要なブロックを回収したりすることができ、無駄になる領域が小さくて済みます。名前とブロック番号の対応表を管理する機能を中心とした仕組みをファイルシステムと呼びます。

管理データの置き方やブロックの割り当て方式などの違いによって様々なファイルシステムがあります。Linux では `ext4` や `btrfs` などが代表的でしょう。Windows では従来は `FAT` が、現在では `NTFS` がよく使われます。macOS は `APFS` です。UEFI 規格では `FAT` が標準的に用いられます。特に UEFI アプリケーションを格納するための `EFI System Partition` には `FAT` を使うことが決められています。USB メモリなどの取り外し可能なストレージでは `FAT` が用いられることが多いです。本書では、試しやすさや規格が公開されていて実装しやすいなどの理由で `FAT` を扱う方法を説明します。

`FAT` がどのようなものか知るために、まずは `FAT` でフォーマットされたボリュームイメージを用意し、中身を覗いてみます。ボリュームイメージとは物理的なブロックデバイスではなく、ブロックデバイスの中身を記録したデータのことです。実際のブロックデバイスから読み出して作ることもありますが、まったく新規に作ることもできます。今回は新規に作っていきます。

```
$ dd if=/dev/zero of=fat_disk bs=1M count=128
$ mkfs.fat -n 'MIKAN OS' -s 2 -f 2 -R 32 -F 32 fat_disk
```

`dd` コマンドは `if` (input file) で指定したファイルから `of` (output file) で指定したファイルへとコピーします。`if` に指定した `/dev/zero` はゼロが無限に読み出せる特殊なデバイスファイルです。`of` にはボリュームイメージとなるファイル名を指定します。

`dd` は `cp` コマンドよりもコピーの動作を細かく指定でき、特定の大きさの空ファイルを作るような場合によく使われます。今回は 128MiB のゼロで埋め尽くされたファイルを作るのに使いました。`bs` は `dd` コマンドが一度にコピーするデータの大きさ（ブロックサイズ）を指定します。`M` は 1024×1024 を意味する接尾辞です。`count` はコピーするブロック数を指定します。結局のところ、 $bs \times count = 1\text{MiB} \times 128 = 128\text{MiB}$ だけ `if` から `of` へコピーされます。`bs=1`、`count=128M` などとしても良い

のですが、bs にはある程度大きな値を指定すると高速にコピーできます。

mkfs.fat コマンドは指定したブロックデバイスを FAT 形式でフォーマットするためのコマンドですが、本物のデバイスではなくボリュームイメージを指定することもできます。その機能を使い、先ほど作成した 128MiB のゼロで埋められたファイルを FAT 形式でフォーマットします。オプションの意味は表 17.1 の通りです。

表 17.1 mkfs.fat コマンドのオプション

オプション	意味
-s 2	1 クラスタあたりのセクタ数を 2 とする
-f 2	FAT の個数を 2 とする
-R 32	予約セクタ数を 32 とする
-F 32	FAT タイプを FAT32 とする

オプションの意味を理解するには FAT の仕組みを知らなければなりません。それはおいおい説明することにして、まずはボリュームイメージの中身を探検してみることにします。メタデータやファイルがどこに配置されるかを確認してみましょう。今のところ fat_disk は FAT 形式でフォーマットした空の状態になっているはずですが。空のボリュームイメージはどのような中身になっているのでしょうか？

```
$ hexdump -C -s 16k fat_disk
00004000 f8 ff ff 0f ff ff ff 0f f8 ff ff 0f 00 00 00 00 |.....|
00004010 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
00083000 f8 ff ff 0f ff ff ff 0f f8 ff ff 0f 00 00 00 00 |.....|
00083010 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
00102000 4d 49 4b 41 4e 20 4f 53 20 20 20 08 00 00 12 4f |MIKAN OS ....|
00102010 44 4f 44 4f 00 00 12 4f 44 4f 00 00 00 00 00 00 |DODO...ODO....|
00102020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
08000000
```

hexdump はファイルの中身を 16 進数で確認するためのコマンドです。-C オプションは 16 進数表示の隣に ASCII コードを表示するオプションです。ファイルの中に文字列が埋め込まれている場合、ひと目で分かるので大変便利です。実際、mkfs.fat でボリューム名として指定した「MIKAN OS」という文字列が見えますね。その後にある「ODOODO...ODO」は、文字列ではない値がたまたま O や D の ASCII コードと一致したため表示されています。

-s 16k はファイルの先頭から 16KiB だけ飛ばした位置から表示するという意味のオプションです。紙面の都合で余分なデータが表示されないために指定しています（読者の皆さんはこのオプション無しの表示もぜひ見てみてください）。mkfs.fat で予約セクタ数を 32 と指定したので（-R 32）、先頭から 32×512 バイト = 16KiB の位置から FAT の各種データが記録されます。

ボリュームイメージにファイルを 1 つ書き込んでみて、それが FAT にどう反映されるのかを見てみましょう。ボリュームイメージ fat_disk にファイル cafe.txt を追加する手順は次のようになります。

```
$ mkdir mnt
$ sudo mount -o loop fat_disk mnt
$ ls -a mnt
.  ..
$ echo deadbeef > cafe.txt
$ sudo cp cafe.txt mnt/cafe.txt
$ ls -a mnt
.  ..  cafe.txt
$ sudo umount mnt
```

ボリュームイメージをループバックモードでマウントすればボリュームイメージを通常の記憶装置と同じように操作できます。1行目はマウントポイントとなるディレクトリを作ります。名前は何でもいいですが、ここではmntとしました。2行目で、今作ったマウントポイントにボリュームイメージfat_diskをマウントします。ボリュームイメージをマウントする際は-o loopとし、ループバックモードを指定します。マウント操作はrootユーザーでしかできないのでsudoを使いましょう。

マウントが完了すると、mntディレクトリを介してボリュームイメージを操作できるようになります。ここではcafe.txtというファイルを作り、内容をdeadbeefとします。マウント操作をrootユーザーとして実施したため、mnt内に変更を加えるにはrootユーザーになる必要があります。ですので、ファイルを一般ユーザーで作った後、mnt/cafe.txtにコピーする操作はrootでやっています。

mnt内の操作が終わったらアンマウント(umount)します。アンマウントすることで、ボリュームイメージに書かれていない(メモリ上だけに存在する)変更がすべてボリュームイメージに書き込まれます。アンマウントしないでボリュームイメージを操作すると壊れるかもれませんので、アンマウントはした方がいいですね。

```
$ hexdump -C -s 16k fat_disk
00004000 f8 ff ff 0f ff ff ff 0f f8 ff ff 0f ff ff ff 0f |.....|
00004010 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
00083000 f8 ff ff 0f ff ff ff 0f f8 ff ff 0f ff ff ff 0f |.....|
00083010 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
00102000 4d 49 4b 41 4e 20 4f 53 20 20 20 08 00 00 12 4f |MIKAN OS ....0|
00102010 44 4f 44 4f 00 00 12 4f 44 4f 00 00 00 00 00 00 |DODO...ODO.....|
00102020 41 63 00 61 00 66 00 65 00 2e 00 0f 00 dc 74 00 |Ac.a.f.e.....t.|
00102030 78 00 74 00 00 00 ff ff ff ff 00 00 ff ff ff ff |x.t.....|
00102040 43 41 46 45 20 20 20 20 54 58 54 20 00 7a 90 09 |CAFE TXT .z..|
00102050 44 4f 44 4f 00 00 90 09 44 4f 03 00 09 00 00 00 |DODO...DO.....|
00102060 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
00102400 64 65 61 64 62 65 65 66 0a 00 00 00 00 00 00 00 |deadbeef.....|
00102410 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
08000000
```

ファイルを追加した後のボリュームイメージを表示してみました。よく見るとファイル名(cafe.txt) ばい文字列が2箇所にあることが分かります。1箇所目は「c.a.f.e.....t.x.t.」です。何故か1文字飛ばしになっていますが、2箇所目は「CAFE TXT」です。大文字になっていますが、確かにファイル名でしょう。さらにオフセット00102400からはdeadbeefという文字列も見えます。これはcafe.txtの内容で間違いありません。

ということで、FATでフォーマットしたボリュームイメージにファイルを追加すると、ファイル名とファイルの内容が記録されるということが分かりました。もう1つファイルを追加してみ様子を見てみます。

```
$ sudo mount -o loop fat_disk mnt
$ echo CafeBabe > HelloWorld.data
$ sudo cp HelloWorld.data mnt/HelloWorld.data
$ sudo umount mnt
```

HelloWorld.dataというファイルを追加してみました。ファイル名にわざと大文字と小文字を混ぜてみました。さらに、拡張子を除くファイル名が8文字を超えるようにしてみました。cafe.txtが「CAFE TXT」となっていたように、ファイル名の長さが8文字に制限されているように見えたからです。

```
$ hexdump -C -s 16k fat_disk
00004000 f8 ff ff 0f ff ff ff 0f f8 ff ff 0f ff ff ff 0f |.....|
00004010 ff ff ff 0f 00 00 00 00 00 00 00 00 00 00 00 |.....|
00004020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
00083000 f8 ff ff 0f ff ff ff 0f f8 ff ff 0f ff ff ff 0f |.....|
00083010 ff ff ff 0f 00 00 00 00 00 00 00 00 00 00 00 |.....|
00083020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
00102000 4d 49 4b 41 4e 20 4f 53 20 20 20 08 00 00 10 56 |MIKAN OS ...V|
00102010 44 4f 44 4f 00 00 10 56 44 4f 00 00 00 00 00 00 |DOBO...VDO....|
00102020 41 63 00 61 00 66 00 65 00 2e 00 0f 00 dc 74 00 |Ac.a.f.e.....t.|
00102030 78 00 74 00 00 00 ff ff ff ff 00 00 ff ff ff ff |x.t.....|
00102040 43 41 46 45 20 20 20 20 54 58 54 20 00 90 20 0e |CAFE TXT .. |
00102050 44 4f 44 4f 00 00 20 0e 44 4f 03 00 09 00 00 00 |DOBO...DO.....|
00102060 42 61 00 00 00 ff ff ff ff ff ff 0f 0f e9 ff ff |Ba.....|
00102070 ff ff ff ff ff ff ff ff ff ff 00 00 ff ff ff ff |.....|
00102080 01 48 00 65 00 6c 00 6f 00 57 00 0f 00 e9 6f 00 |.H.e.l.o.W....o.|
00102090 72 00 6c 00 64 00 2e 00 64 00 00 00 61 00 74 00 |r.l.d...d...a.t.|
001020a0 48 45 4c 4f 57 4f 7e 31 44 41 54 20 00 53 34 16 |HELOWO~1DAT .S4.|
001020b0 46 4f 46 4f 00 00 34 16 46 4f 04 00 09 00 00 00 |FOFO..4.FO.....|
001020c0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
00102400 64 65 61 64 62 65 65 66 0a 00 00 00 00 00 00 00 |deadbeef.....|
00102410 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
00102800 43 61 66 65 42 61 62 65 0a 00 00 00 00 00 00 00 |CafeBabe.....|
00102810 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
08000000
```

2つ目のファイルを追加したボリュームイメージを見ると次のことが分かります。

- 8文字を超えるファイル名は、ちょっと見にくいけど「.H.e.l.o.W....o.r.l.d...d...a.t.」のように記録されているようだ。
- もう1箇所には「HELOWO~1DAT」というように大文字化され、さらに省略された名前が入る。
- ファイルの内容は大文字小文字がきちんと保たれる。

以降では、ボリュームの内容を OS からきちんと読み取れるように詳しい仕様を紹介していきます。

17.2 BIOS パラメータブロック

これまでに作成したボリュームイメージは 1 つのパーティションを表します。パーティションとはブロックデバイスを複数に分割した 1 つの領域のことです。ただ、USB メモリなどのリムーバブルメディアでは通常は記憶領域全体を 1 つのパーティションとして扱うことになっていますから、ボリュームイメージは USB メモリ全体をファイルにしたものということになります。

パーティションの先頭 1 ブロックのことを **PBR** (Partition Boot Record) と呼びます。ここにはそのパーティションの大きさ (ブロック数) や FAT のデータ構造が何ブロック目から始まるかなどの情報が書かれています。PBR を解釈することで正しくパーティションを認識して FAT を操作できます。特に、PBR の中でも **BPB** (BIOS Parameter Block) と呼ばれる領域に重要な情報が書いてあります。BPB の構造を表 17.2 に紹介します。なお、この表は FAT の仕様書 (参考文献 [15]) をもとに作成しました。FAT の仕様書ではブロックのことをセクタ (sector) と呼んでいるため、フィールド名に登場する Sec はブロックのことだと思ってください。

表 17.2 BPB の構造

フィールド名	オフセット	バイト数	fat_disk での値	意味
BS_jumpBoot	0	3	eb 58 90	プログラムへのジャンプ命令
BS_OEMName	3	8	mkfs.fat	任意の 8 バイト文字列
BPB_BytsPerSec	11	2	0x0200	ブロックあたりのバイト数
BPB_SecPerClus	13	1	0x02	クラスタあたりのブロック数
BPB_RsvdSecCnt	14	2	0x0020	ボリューム先頭からの予約ブロック数
BPB_NumFATs	16	1	0x0002	FAT の数
BPB_RootEntCnt	17	2	0x0000	※ルートディレクトリエントリ数
BPB_TotSec16	19	2	0x0000	※ボリューム全体の総ブロック数
BPB_Media	21	1	0xf8	メディア種別 (今では使われない)
BPB_FATSz16	22	2	0x0000	※ FAT あたりのブロック数
BPB_SecPerTrk	24	2	0x0020	トラックあたりのブロック数
BPB_NumHeads	26	2	0x0040	ヘッド数
BPB_HiddSec	28	4	0x00000000	隠しブロック数
BPB_TotSec32	32	4	0x00040000	ボリューム全体の総ブロック数
BPB_FATSz32	36	4	0x000003f8	FAT あたりのブロック数
BPB_ExtFlags	40	2	0x0000	FAT の冗長化に関するフラグ
BPB_FSVer	42	2	0x0000	ファイルシステムバージョン (最新は 0.0)
BPB_RootClus	44	4	0x00000002	ルートディレクトリの開始クラスタ
BPB_FSInfo	48	2	0x0001	FSINFO 構造体の開始ブロック番号

BPB_BkBootSec	50	2	0x0006	ブートセクタのコピーが置かれているブロック番号
BPB_Reserved	52	12	.	予約領域
BS_DrvNum	64	1	0x80	BIOS の INT 0x13 で使うドライブ番号
BS_Reserved1	65	1	0x00	予約領域
BS_BootSig	66	1	0x29	拡張シグネチャ
BS_VolID	67	4	0x4f0c572f	ボリュームのシリアルナンバー
BS_VolLab	71	11	MIKAN OS	ボリュームラベル
BS_FilSysType	82	8	FAT32	ファイルシステム種別

FAT フォーマットには FAT32 のほかに FAT12 と FAT16 があります。この表のオフセット 0 から 35 までは FAT12/16/32 で共通です。オフセット 36 からは FAT32 特有のフィールドです。意味欄で「※」を付けた項目は FAT32 では常に 0 です。

BPB の内容のうち、特に重要なものをいくつか詳しく説明します。まずは BPB_BytsPerSec です。これは 1 ブロックのバイト数を表します。先ほど作ったボリュームイメージでは 0x0200=512 となっていて、1 ブロックが 512 バイトであることが分かります。FAT の仕様では BPB_BytsPerSec の値として 512、1024、2048、4096 のいずれかが許されています。現在使われているブロックデバイスの多くは、1 ブロックが 512 バイトまたは 4096 バイトの製品が多いです。

BPB_SecPerClus は 1 クラスタあたりのブロック数を表します。**クラスタ**は FAT ファイルシステムの重要な考え方で、いくつかのブロックをまとめたものです。FAT はクラスタを基本単位として管理します。ブロック単位ではない本当の理由を筆者は知りませんが、クラスタ単位で管理することで、大きなボリュームも効率よく管理できるという効果があります。

BPB_TotSec32 はボリューム全体のブロック数を表します。fat_disk は 128MiB ですから、ブロック数は 128MiB/512B=256Ki 個となります。256Ki=0x40000 ですので、BPB に記録されている値と一致しますね。同じ目的の BPB_TotSec16 は 2 バイトしかなく、大きなボリュームでは不足します。BPB_TotSec32 を使う際は BPB_TotSec16=0 としておくことになっています。

BPB_RootClus はルートディレクトリの開始クラスタ番号です。ルートディレクトリとはファイルシステムの最も上位にあるディレクトリ（フォルダともいいます）のことで、作成したボリュームイメージの中で cafe.txt や HelloWorld.data を入れたディレクトリです。詳しい説明はまた後でしますが、次の計算をするとルートディレクトリのブロック番号を得られます。

$$\text{BPB_RsvdSecCnt} + \text{BPB_NumFATs} \times \text{BPB_FATSz32} + (\text{BPB_RootClus} - 2) \times \text{BPB_SecPerClus} = 2064$$

このほかにも大切な項目がありますが、それは次節以降で説明します。

17.3 ディレクトリエントリ

ディレクトリは複数のファイルやディレクトリを格納することができます。ファイルシステムによってディレクトリの表現方法は様々ですが、FAT ファイルシステムではディレクトリは特殊なファイルとして表現されます。ファイルの内容は任意のバイト列である一方、ディレクトリの内容は**ディレクトリエントリ** (directory entry) という 32 バイトのデータ構造が配列状に並んだ構造になっています。fat_disk の 0x102000 から始まるデータはディレクトリエントリの配列です。ディレクトリエントリの構造が分かれば、この部分を読解できるようになります。

表 17.3 FAT ファイルシステムのディレクトリエントリの構造

フィールド名	オフセット	バイト数	cafe.txt の値	意味
DIR_Name	0	11	CAFE _ _ _ _ TXT	ファイルの短名
DIR_Attr	11	1	0x20	ファイル属性
DIR_NTRes	12	1	0x00	Windows NT 用の予約領域
DIR_CrtTimeTenth	13	1	0x90	ファイル作成時刻のミリ秒部分
DIR_CrtTime	14	2	0x0e20	ファイル作成時刻
DIR_CrtDate	16	2	0x4f44	ファイル作成日
DIR_LstAccDate	18	2	0x4f44	最終アクセス日
DIR_FstClusHI	20	2	0x0000	開始クラスタ番号の上位 2 バイト
DIR_WrtTime	22	2	0x0e20	最終書き込み時刻
DIR_WrtDate	24	2	0x4f44	最終書き込み日
DIR_FstClusLO	26	2	0x0003	開始クラスタ番号の下位 2 バイト
DIR_FileSize	28	4	0x00000009	ファイルのバイト数

表 17.3 にディレクトリエントリの構造を示します。cafe.txt ファイルに関するディレクトリエントリ (ボリュームイメージの 0x102040 から始まる 32 バイト) の値を例として載せました。

DIR_Name はファイルの短名です。短名とは 8+3 形式の名前のことです。8 文字の文字列と、それに続くドットと 3 文字までの拡張子を扱えます。昔から使われるファイルの拡張子の多くが 3 文字なのはこの制約から来ているのではないのでしょうか (筆者の推測に過ぎませんが)。短名と言うからにはもちろん長名も存在します。例えば 0x102020 や 0x102080 から始まるディレクトリエントリは長名を保持する専用のエントリです。長名を扱うのは面倒なので、本書では無視することにします。

DIR_Attr はファイルの属性を表す 1 バイトの値です。ファイルの属性は表 17.4 に示す種類があります。この中からいくつか紹介します。

表 17.4 FAT ファイルシステムのファイル属性

属性名	値	意味
ATTR_READ_ONLY	0x01	読み込み専用ファイル
ATTR_HIDDEN	0x02	隠しファイル
ATTR_SYSTEM	0x04	システムファイル
ATTR_VOLUME_ID	0x08	ボリューム名
ATTR_DIRECTORY	0x10	ディレクトリ
ATTR_ARCHIVE	0x20	バックアップすべきフラグ
ATTR_LONG_NAME	0x0f	長名エントリ

ATTR_READ_ONLY はまあ分かりやすいと思います。そのファイルが読み込み専用であることを表す属性です。ただ、この属性が付いているからといってボリューム上のそのファイルへの書き込みが物理的にできないわけではありません。OS が、この属性が付いたファイルへの書き込みをさせないように保護する責任があります。

ATTR_VOLUME_ID は特殊な属性です。この属性を持ったファイルは普通のファイルではなく、ファイルの内容というものを持ちません。しかも、ルートディレクトリに 1 つだけしか存在できません。このファイルの名前は、実際にはボリューム名を表します。

ATTR_DIRECTORY はそのファイルがディレクトリであることを表します。FAT ファイルシステムでは、ディレクトリとはディレクトリエントリの配列を中身として持つファイルです。本章ではルートディレクトリだけに対応することにして、サブディレクトリは無視します。

ATTR_LONG_NAME は、そのエントリがファイルそのものではなく、関連付けられたファイルの長名を保持するエントリであることを示します。本書では長名エントリは無視します。

17.4 ボリュームを読み出す (osbook_day17a)

ここまでの説明で、ボリュームイメージに含まれるファイル一覧を得る方法が分かりました。BPB の値からルートディレクトリの位置を取得し、その中身をディレクトリエントリの構造に従って読めば良いのです。読み取った内容を表示すれば、ls コマンド^{*1}の簡易版を作ることができそうです。

しかし、まだ足りない知識があります。それはボリュームイメージをメモリに読み出す方法です。具体的には、どうやって USB メモリなどのブロックデバイスからデータを読み出すかということです。

一般的な OS であれば USB メモリや SSD などのブロックデバイスそれぞれに対してドライバが準備されており、OS が自力でブロックデバイスを読み書きします。USB メモリであれば USB ドライバと SCSI プロトコルスタックを記述しますし、NVMe 接続の SSD であれば NVMe 用のドライバを準備すれば読み書きできるようになります。もちろん Windows、macOS、Linux などはそれらのドライバを搭載し、自由自在にブロックデバイスを読み書きしています。ただ、それらのドライバを開発するのはかなり大変^{*2}で、本書が余計に分厚くなってしまいます。そのため、もっと楽な方法を探します。

レガシー BIOS にはフロッピーディスクやハードディスクを読み書きする機能がありました。だから

*1 ls コマンドは POSIX 系の OS でファイルを一覧するのに使うコマンド。MS-DOS では dir コマンドが相当します。

UEFI BIOS にも同様の機能があるはずだと思って UEFI の仕様書を探したら、ありました。Block I/O Protocol です。これを使って OS 本体が起動する前にブロックデバイスのデータをまとめてメモリ上に読み出すことにします。OS からはブロックデバイスを直接制御してデータを読むのではなく、ブートローダがメモリ上に読み出したデータを読むことにします。

リスト 17.1 ブートローダでボリュームを読み込む (Main.c)

```
VOID* volume_image;

EFI_FILE_PROTOCOL* volume_file;
status = root_dir->Open(
    root_dir, &volume_file, L"\\fat_disk",
    EFI_FILE_MODE_READ, 0);
if (status == EFI_SUCCESS) {
    status = ReadFile(volume_file, &volume_image);
    if (EFI_ERROR(status)) {
        Print(L"failed to read volume file: %r", status);
        Halt();
    }
} else {
    EFI_BLOCK_IO_PROTOCOL* block_io;
    status = OpenBlockIoProtocolForLoadedImage(image_handle, &block_io);
    if (EFI_ERROR(status)) {
        Print(L"failed to open Block I/O Protocol: %r\n", status);
        Halt();
    }

    EFI_BLOCK_IO_MEDIA* media = block_io->Media;
    UINTN volume_bytes = (UINTN)media->BlockSize * (media->LastBlock + 1);
    if (volume_bytes > 16 * 1024 * 1024) {
        volume_bytes = 16 * 1024 * 1024;
    }

    Print(L"Reading %lu bytes (Present %d, BlockSize %u, LastBlock %u)\n",
        volume_bytes, media->MediaPresent, media->BlockSize, media->LastBlock);

    status = ReadBlocks(block_io, media->MediaId, volume_bytes, &volume_image);
    if (EFI_ERROR(status)) {
        Print(L"failed to read blocks: %r\n", status);
        Halt();
    }
}
```

リスト 17.1 にボリュームイメージを読み取る処理を示します。これはブートローダに追加した処理です。前半部分では起動メディア（USB メモリなどの、ブートローダを入れている記憶装置）に `fat_disk` という名のファイルがあるかを調べ、あればそれをボリュームイメージだと思って読み込みます。そのようなファイルがなければ起動メディア全体を読み取ります。このような 2 段階の構成にしているのは柔軟に対応できるようにするためです。開発時には内容が分かっているファイルを `fat_disk` として配置することもできるし、実際の USB メモリで試したくなくても対応できます。

* 2 USB メモリや NVMe SSD などのためのドライバを書くのはかなり大変です。筆者が USB ドライバを書くのでさえ 1 年以上もかかりました。しかし、ドライバ開発は楽しい領域でもあるので、興味がある方はぜひ挑戦してみてください。

ファイルを読み込む処理は `kernel.elf` の読み込みと同じです。コードをコピペするのは嫌なので `ReadFile()` という関数に切り出して共通化しました。この関数に読みたいファイルを渡すと内部でファイルの大きさのメモリ領域を割り当て、そこにファイル内容を読み込みます。割り当てたメモリ領域は第 2 引数に指定したポインタ変数（ここでは `volume_image`）に書き戻されます。

`fat_disk` という名のファイルが無い場合は `else` 節が実行されます。`else` 節では Block I/O Protocol を用いて起動メディアの先頭ブロックから 16MiB 分を読み込みます。処理がちょっと難しいかもしれませんが順を追って説明します。

まずは `OpenBlockIoProtocolForLoadedImage()` を使って、起動メディアに関連付けられた Block I/O Protocol を取得し、`block_io` 変数に設定します。Block I/O Protocol を使うとブロックデバイスに関する情報を得たり、ブロック単位でデータを読み書きしたりできます。この関数の実装は後ほど紹介します。

`block_io->Media` には、その Block I/O Protocol が関連付けられているブロックデバイスの各種情報が記録されています。それらの情報のうち、1 ブロックあたりのバイト数 `BlockSize` と、最大ブロック番号 `LastBlock` を使って、ブロックデバイスの全体サイズをバイト単位で求めます。

全体サイズ `volume_bytes` の上限値を 16MiB とします。この理由は、ブロックデバイスの全体を読み込もうとしても、それが収まるだけのメインメモリが搭載されていない可能性があることと、全体を読もうとするとかなり時間がかかり、OS の起動が遅くなることです。FAT ファイルシステムでは普通、ブロックデバイスの先頭の方から順にデータが詰め込まれていくので、先頭部分だけを読み込んでおけば十分なのです。足りなくなったらこの上限値をだんだん引き上げていきましょう。

最後に `ReadBlocks()` を用いてブロックデバイスからデータを読み込みます。この関数は内部で `volume_bytes` だけメモリ領域を確保し、そこにデータを読み込みます。確保したメモリ領域は第 4 引数（ここでは `volume_image`）に書き戻されます。この関数の実装も後ほど紹介します。

リスト 17.2 Block I/O Protocol を開く (Main.c)

```
EFI_STATUS OpenBlockIoProtocolForLoadedImage(
    EFI_HANDLE image_handle, EFI_BLOCK_IO_PROTOCOL** block_io) {
    EFI_STATUS status;
    EFI_LOADED_IMAGE_PROTOCOL* loaded_image;

    status = gBS->OpenProtocol(
        image_handle,
        &gEfiLoadedImageProtocolGuid,
        (VOID**)&loaded_image,
        image_handle,
        NULL,
        EFI_OPEN_PROTOCOL_BY_HANDLE_PROTOCOL);
    if (EFI_ERROR(status)) {
        return status;
    }

    status = gBS->OpenProtocol(
        loaded_image->DeviceHandle,
        &gEfiBlockIoProtocolGuid,
        (VOID**)block_io,
        image_handle, // agent handle
        NULL,
        EFI_OPEN_PROTOCOL_BY_HANDLE_PROTOCOL);
```

```

    return status;
}

```

リスト 17.2 に `OpenBlockIoProtocolForLoadedImage()` の実装を示します。この関数はブートローダが格納されているブロックデバイス（起動メディア）に対応する Block I/O Protocol を開きます。そのための手順は 2 段階です。まずブートローダに紐付いた Loaded Image Protocol を開きます。次に、Loaded Image Protocol から得た値を使って Block I/O Protocol を開きます。

`UefiMain()` の第 1 引数である `image_handle` には、ブートローダである `B00TX64.EFI` を表すハンドルが渡されます（UEFI の世界では UEFI アプリケーションのことをイメージと呼びます）。ブートローダへのハンドルを `OpenProtocol()` の第 1 引数に指定することで、ブートローダに紐付いた Loaded Image Protocol を得られます。開いたプロトコルは `loaded_image` 変数に書き戻されます。

`loaded_image->DeviceHandle` には、そのイメージ（ここではブートローダ）が格納されている記憶装置を表すハンドルが設定されています。そのハンドルを `OpenProtocol()` の第 1 引数に指定することで、その記憶装置に紐付いた Block I/O Protocol を得られます。ちなみに `OpenProtocol()` の第 1 引数に他の記憶装置を指定することで、UEFI が対応する記憶装置ならどれでも Block I/O Protocol を開けます。他の記憶装置を表すハンドルを取得する方法は本書では説明しません。

リスト 17.3 ブロックデバイスからデータを読み込む (Main.c)

```

EFI_STATUS ReadBlocks(
    EFI_BLOCK_IO_PROTOCOL* block_io, UINT32 media_id,
    UINTN read_bytes, VOID** buffer) {
    EFI_STATUS status;

    status = gBS->AllocatePool(EfiLoaderData, read_bytes, buffer);
    if (EFI_ERROR(status)) {
        return status;
    }

    status = block_io->ReadBlocks(
        block_io,
        media_id,
        0, // start LBA
        read_bytes,
        *buffer);

    return status;
}

```

リスト 17.3 に `ReadBlocks()` の実装を示します。先ほど開いた Block I/O Protocol を使ってデータを読み取ります。読み取るデータの大きさは引数 `read_bytes` にバイト単位で指定します。指定された大きさのメモリ領域を `gBS->AllocatePool()` で割り当て、そこへブロックデバイスから読んだデータを書き込みます。

リスト 17.4 ボリュームイメージを OS に渡す (Main.c)

```
typedef void EntryPointType(const struct FrameBufferConfig*,
                           const struct MemoryMap*,
                           const VOID*,
                           VOID*);

EntryPointType* entry_point = (EntryPointType*)entry_addr;
entry_point(&config, &memmap, acpi_table, volume_image);
```

リスト 17.4 に示すように、ボリュームイメージが置いてあるメモリ領域を指すポインタ変数 `volume_image` を OS のメイン関数に渡すように改造しました。OS のメイン関数の第 4 引数として渡します。

リスト 17.5 Block I/O Protocol のための GUID を追加する (Loader.inf)

```
[Protocols]
gEfiLoadedImageProtocolGuid
gEfiLoadFileProtocolGuid
gEfiSimpleFileSystemProtocolGuid
gEfiBlockIoProtocolGuid
```

Block I/O Protocol は新しく登場したプロトコルですので、`Loader.inf` に GUID を追加する必要があります (リスト 17.5)。これを追加して EDK II で `build` コマンドを叩けば新しいブートローダの完成です。ブートローダ側ができたなら、次に OS 側を改造していきます。

リスト 17.6 OS のメイン関数に引数を追加する (main.cpp)

```
extern "C" void KernelMainNewStack(
    const FrameBufferConfig& frame_buffer_config_ref,
    const MemoryMap& memory_map_ref,
    const acpi::RSDP& acpi_table,
    void* volume_image) {
```

ブートローダの変更に合わせて OS のメイン関数に引数を追加します (リスト 17.6)。第 4 引数として `void* volume_image` を追加しました。これで、`volume_image` には `fat_disk`、またはブロックデバイスから読んだデータが渡されてくることになります。

リスト 17.7 ボリュームイメージを表示する (main.cpp)

```
uint8_t* p = reinterpret_cast<uint8_t*>(volume_image);
printf("Volume Image:\n");
for (int i = 0; i < 16; ++i) {
    printf("%04x:", i * 16);
    for (int j = 0; j < 8; ++j) {
        printf(" %02x", *p);
        ++p;
    }
    printf(" ");
    for (int j = 0; j < 8; ++j) {
        printf(" %02x", *p);
        ++p;
    }
```

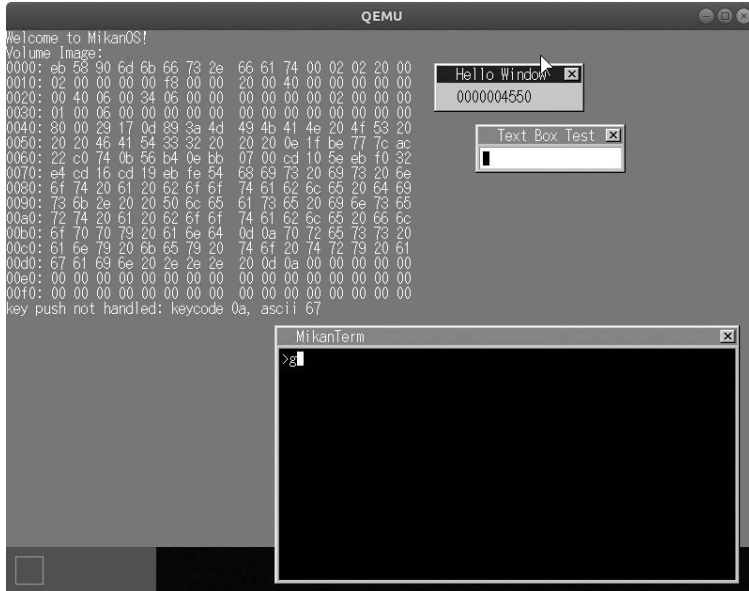
```

}
printf("\n");
}

```

OS にボリュームイメージが渡されただけでは見た目が何も変化しません。そこで、ボリュームイメージの内容を 16 進数で表示するようにしてみました (リスト 17.7)。ボリュームイメージの先頭 256 バイトを 16 進数で表示します。

図 17.1 ボリュームイメージを 16 進数で表示した様子



改造した OS をビルドして、新しいブートローダによって起動させると図 17.1 のようになりました。なんだかそれっぽい値が表示されている気がしますね。でも本当に、これは読み込んだボリュームイメージなのでしょうか？ 確かめてみましょう。QEMU で実行している場合は \$HOME/edk2/disk.img が起動メディアになっていますので、それを 16 進数で表示してみます。

```

$ cd $HOME/edk2
$ hexdump -n 256 -C disk.img
00000000 eb 58 90 6d 6b 66 73 2e 66 61 74 00 02 02 20 00 |.X.mkfs.fat... |
00000010 02 00 00 00 00 f8 00 00 20 00 40 00 00 00 00 00 |.....@.....|
00000020 00 40 06 00 34 06 00 00 00 00 00 00 02 00 00 00 |.@..4.....|
00000030 01 00 06 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
00000040 80 00 29 17 0d 89 3a 4d 49 4b 41 4e 20 4f 53 20 |...)...MIKAN OS |
00000050 20 20 46 41 54 33 32 20 20 20 0e 1f be 77 7c ac | FAT32 ...w|. |
00000060 22 c0 74 0b 56 b4 0e bb 07 00 cd 10 5e eb f0 32 |".t.V.....^..2|
00000070 e4 cd 16 cd 19 eb fe 54 68 69 73 20 69 73 20 6e |.....This is n|
00000080 6f 74 20 61 20 62 6f 6f 74 61 62 6c 65 20 64 69 |ot a bootable di|
00000090 73 6b 2e 20 20 50 6c 65 61 73 65 20 69 6e 73 65 |sk. Please inse|
000000a0 72 74 20 61 20 62 6f 6f 74 61 62 6c 65 20 66 6c |rt a bootable fl|
000000b0 6f 70 70 79 20 61 6e 64 0d 0a 70 72 65 73 73 20 |oppy and..press |

```

```
000000c0 61 6e 79 20 6b 65 79 20 74 6f 20 74 72 79 20 61 |any key to try a|
000000d0 67 61 69 6e 20 2e 2e 2e 20 0d 0a 00 00 00 00 00 |gain ... .....|
000000e0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
00000100
```

おお、完全に一致しています。ブートローダが、起動メディアからデータをちゃんと読み出して、それを OS に渡せていることが分かりました。めでたし、めでたし。

コラム 17.1 ボリュームの読み込みは 16MiB で足りるか

UEFI の Block I/O Protocol を使った読み込み速度が遅いので、ボリュームの先頭 16MiB だけを読み取ることにしています。本当に 16MiB で足りるのでしょうか？ それを調べるのに、Linux では `fatcat` というコマンドが便利です。Ubuntu であれば `sudo apt install fatcat` としてインストールできます。`fatcat` を使って、筆者が使っている USB メモリの FAT ファイルシステム情報を見てみます。

```
$ sudo fatcat -i /dev/sda1
```

皆さんのパソコンで USB メモリがどのように認識されるかはシステムによって異なります。ここでは USB メモリが `/dev/sda` として認識された前提で説明します。`/dev/sda1` はこの USB メモリの第 1 パーティションを表します。

`fatcat -i` の出力のうち Data start address の値は `0xe5c000` となっていました。この値はボリュームの中でデータ領域が始まるアドレスを意味します。ボリュームの先頭には予約ブロックや BPB、FAT などの管理用データ構造があり、その直後からデータ領域が始まるのでしたね。

`0xe5c000` は約 14.4MiB。筆者が使っている USB メモリの管理用データ構造は 14.4MiB 程度であることが分かりました。これは大問題です。ボリュームの先頭 16MiB しか読み取ってないのに、その大部分が管理用データ構造で埋め尽くされていて、肝心のファイルの内容が読み取れないからです。

14.4MiB のうちほとんどを FAT が占めます（1 つの FAT の大きさを表す Fat size は約 7.1MiB。FAT は 2 個あります）。ボリュームが大きくなるほど、クラスタが小さくなるほど FAT は大きくなります。そのため、128MiB のボリュームイメージ `fat_disk` では大した問題ではなかった FAT の大きさが、15GiB 程度の容量がある筆者の USB メモリではとても大きくなってしまったわけです。

これは困りました。簡単な解決策は Block I/O Protocol でもっとたくさん読むことです。今回であれば 2 倍にすれば足りそうです。ですが、USB メモリの容量やクラスタサイズによってはもっと増やす必要があります。読み取るデータを増やすほど OS の起動に時間がかかってイライラします。

お勧めの解決方法は、もともと容量が小さい USB メモリを使うか、大容量の USB メモリの第 1 パーティションを小さくすることです。パーティションさえ小さくしてしまえば小容量の USB メモリと同様に扱えます。Ubuntu や Windows には任意の大きさのパーティションを作るツールが

付属しています。Ubuntu であれば「GNOME ディスクユーティリティ」、Windows であれば「コンピュータの管理」にある「ディスクの管理」機能です。

図 17.2 GNOME ディスクユーティリティで 1GB のパーティションを作った



試しに 1GB 程度でパーティションを作り直してみました (図 17.2)。FAT がかなり小さくなり、16MiB だけの読み込みでも問題がなくなりました。めでたし、めでたし。

17.5 ls コマンド (osbook_day17b)

OS がボリュームイメージを受け取れるようになりましたので、ls コマンドが作れるようになりました。このコマンドはボリュームに含まれるファイルの一覧を表示します。ルートディレクトリエントリを解釈し、ファイル名を表示すれば良いのです。やってみましょう。

リスト 17.8 FAT モジュールの初期化関数 (fat.cpp)

```
BPB* boot_volume_image;

void Initialize(void* volume_image) {
    boot_volume_image = reinterpret_cast<fat::BPB*>(volume_image);
}
```

リスト 17.8 に FAT モジュールの初期化用関数 `fat::Initialize()` の実装を示します。メイン関数からボリュームイメージのポインタを受け取り、それをグローバル変数に設定するだけです。簡単ですね。ここで登場する BPB 構造体は表 17.2 の定義に基づいた構造になっていて、`fat.hpp` に定義されています。各要素の名前をちょっと変えてあるだけで、構造自体は表の定義通りですので構造体のソースコードは割愛します。

リスト 17.9 メイン関数でFAT モジュールを初期化する (main.cpp)

```
fat::Initialize(volume_image);
InitializePCI();
```

作った関数をメイン関数から呼び出します。リスト 17.9 のように、割り込みを初期化した後、PCI を初期化する前に置いてみました。どこに置いてもいいのですが、なんとなくこの辺がしっくりきました。

リスト 17.10 ExecuteLine() に追加した ls コマンド (terminal.cpp)

```
} else if (strcmp(command, "ls") == 0) {
    auto root_dir_entries = fat::GetSectorByCluster<fat::DirectoryEntry>(
        fat::boot_volume_image->root_cluster);
    auto entries_per_cluster =
        fat::boot_volume_image->bytes_per_sector / sizeof(fat::DirectoryEntry)
        * fat::boot_volume_image->sectors_per_cluster;
    char base[9], ext[4];
    char s[64];
    for (int i = 0; i < entries_per_cluster; ++i) {
        ReadName(root_dir_entries[i], base, ext);
        if (base[0] == 0x00) {
            break;
        } else if (static_cast<uint8_t>(base[0]) == 0xe5) {
            continue;
        } else if (root_dir_entries[i].attr == fat::Attribute::kLongName) {
            continue;
        }

        if (ext[0]) {
            sprintf(s, "%s.%s\n", base, ext);
        } else {
            sprintf(s, "%s\n", base);
        }
        Print(s);
    }
} else if (command[0] != 0) {
```

リスト 17.10 に示すように、ls コマンドを Terminal::ExecuteLine() に追加しました。このプログラムは冒頭でルートディレクトリの先頭ポインタを取得し、後半の for 文でルートディレクトリの中身を先頭から順に表示しています。

ルートディレクトリは BPB_RootClus が表すクラスタに存在します。BPB_RootClus の値は fat::boot_volume_image->root_cluster で取得でき、その値を fat::GetSectorByCluster() という関数に与えるとメインメモリ上でのクラスタの位置を取得できます。この関数は後ほど実装します。

ルートディレクトリの中身は表 17.3 に示すディレクトリエントリの配列になっています。この表に基づいて定義した構造体が fat::DirectoryEntry です。ルートディレクトリの先頭を指すポインタをこの構造体へのポインタに変換したものを root_dir_entries という変数に設定しています。

entries_per_cluster という変数は 1 クラスタあたりのディレクトリエントリ数を表します。こ

の数を最大として for ループを回します。例えば 1 クラスタ = 2 ブロック、1 ブロック = 512 バイトであれば 1 クラスタには 32 個のディレクトリエントリが入ります。といっても、ルートディレクトリに 32 個のファイルやディレクトリしか格納できないわけではありません。32 個より多いファイルやディレクトリを格納する場合は、ルートディレクトリが複数のクラスタにまたがって存在することになります。次の章で紹介する FAT を解釈できるようになると、複数のクラスタにまたがった場合にも対応できるようになります。今はとりあえず 1 クラスタに格納できる数だけに対応することとしましょう。

for 文の中ではディレクトリエントリの配列を先頭から調べ、順に表示していきます。ループの先頭では、後ほど実装予定の `fat::ReadName()` という関数を使ってファイルの短名を取得します。この関数を使うと 8+3 形式のファイル名を別々に取得できます。ファイル名の拡張子を除く部分を `base`、拡張子を `ext` とします。

FAT の仕様によれば `DIR_Name` の先頭 1 バイト、すなわち `base[0]` は次に示す特別な意味を持ちます。

- ・先頭バイトが 0xE5：このディレクトリエントリは空いています。どんなファイルやディレクトリも表しません。
- ・先頭バイトが 0x00：このディレクトリエントリは空いています。さらに、これより後ろには有効なディレクトリエントリがありません。
- ・先頭バイトが 0x05：ファイル名の先頭 1 バイトが本来は 0xE5 であることを意味します。Shift-JIS で日本語混じりのファイル名を付けると先頭 1 バイトが 0xE5 になることがあります。

for ループの中で、この仕様に従って場合分けをしています。0xE5 の場合はそのエントリを無視するために `continue` し、0x00 の場合は `break` でループを終了させます。MikanOS では日本語ファイル名に対応しないつもりですので、最後のケースは考慮しないことにしました。

エントリが有効であることが分かったら、次は `DIR_Attr` を検査します。FAT の仕様によれば、この値が `ATTR_LONG_NAME` のときはエントリの構造が `fat::DirectoryEntry` とは異なる、長名専用の構造になっているそうです。ですので、`DIR_Attr` がその値のときは処理をしないように `continue` します。

`DIR_Attr` が `ATTR_LONG_NAME` でなければ短名の表示を行います。拡張子がある場合にはドットを挟んで基本名と拡張子を表示します。拡張子が無い場合は基本名だけを表示します。

リスト 17.11 `GetSectorByCluster()` はクラスタ番号をブロック先頭へのポインタに変換する (`fat.hpp`)

```
template <class T>
T* GetSectorByCluster(unsigned long cluster) {
    return reinterpret_cast<T*>(GetClusterAddr(cluster));
}
```

後回しにしていた `fat::GetSectorByCluster()` の実装をリスト 17.11 に示します。この関数は希望する型へのポインタにキャストすることしかしておらず、実質的な処理は `GetClusterAddr()` に委譲しています。そちらを見ていきましょう。

リスト 17.12 GetClusterAddr() はクラスタ番号をブロック位置へ変換する (fat.cpp)

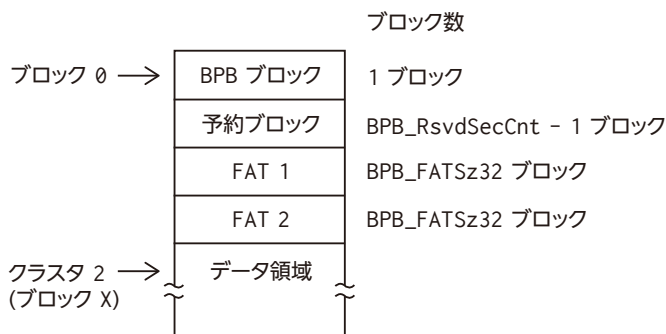
```

uintptr_t GetClusterAddr(unsigned long cluster) {
    unsigned long sector_num =
        boot_volume_image->reserved_sector_count +
        boot_volume_image->num_fats * boot_volume_image->fat_size_32 +
        (cluster - 2) * boot_volume_image->sectors_per_cluster;
    uintptr_t offset = sector_num * boot_volume_image->bytes_per_sector;
    return reinterpret_cast<uintptr_t>(boot_volume_image) + offset;
}

```

リスト 17.12 に、実質的な処理を行っている `fat::GetClusterAddr()` の実装を示します。この関数は、指定されたクラスタ番号に対応するブロックのアドレスを返します。クラスタは BPB_SecPerClus 個のブロックが集まったものです。この関数が返すのはクラスタの先頭ブロックのアドレスです。

図 17.3 FAT ボリュームの構造



`GetClusterAddr()` はまず、指定されたクラスタ番号からブロック番号を計算します。FAT ファイルシステムでフォーマットされたボリュームは図 17.3 のような構造になります。ブロック番号はボリュームイメージ (`boot_volume_image`) の先頭を 0 とした連番、クラスタ番号はデータ領域の先頭を 2 とした連番です。クラスタ番号 `cluster` から求めたブロック番号を `sector_num` 変数に設定します。FAT の用語に合わせて変数名に `sector` を使ってみました。

次に、求めたブロック番号からボリュームイメージ先頭からのバイトオフセットを計算します。単に 1 ブロックあたりのバイト数をブロック番号にかけるだけです。そうして求めたバイトオフセットをボリュームイメージの先頭アドレスに加えれば、最終的に求めたいクラスタの先頭ブロックが存在するアドレスとなります。

リスト 17.13 ReadName() はディレクトリエントリから短名を得る (fat.cpp)

```

void ReadName(const DirectoryEntry& entry, char* base, char* ext) {
    memcpy(base, &entry.name[0], 8);
    base[8] = 0;
    for (int i = 7; i >= 0 && base[i] == 0x20; --i) {
        base[i] = 0;
    }
}

```

```

memcpy(ext, &entry.name[8], 3);
ext[3] = 0;
for (int i = 2; i >= 0 && ext[i] == 0x20; --i) {
    ext[i] = 0;
}
}
}

```

`fat::ReadName()` はリスト 17.13 のように実装しました。8+3 形式の名前のうち、8 文字の方を `base` に、3 文字の方を `ext` にコピーし、末尾の空白文字を削除します。ヌル文字で終端するために引数 `base` は 9 バイト以上、`ext` は 4 バイト以上の配列でなければなりません。

図 17.4 QEMU で起動した OS で `ls` コマンドを実行してみた



ここまで改造が終われば `ls` コマンドが実行できるようになっているはずです。動かしてみます (図 17.4)。4 つのファイルが表示されました。ELF はブートローダを格納したディレクトリ、`KERNEL.ELF` は OS 本体、`MEMMAP` はブートローダの中で取得したメモリマップを保存したファイルですね。

では、一体 `MIKAN OS` は何者でしょう？ 実はこのファイルは普通のファイルではありません。ボリューム名です。ルートディレクトリの中に 1 つだけ存在することを許された特別なエントリで、ボリュームの名前を表します。`ls` コマンドで表示すべきでないかもしれませんが。困るようなことがあれば表示されないように修正することにします。

第 18 章

アプリケーション

前の章では起動メディアに入っているファイルの一覧を見られるようになりました。この章では実際にファイルを開き、中身を扱えるようにしていきます。まずは大きなファイルを正しく扱えるように FAT 構造を理解できるようにし、その機能を使ってファイルの中身をターミナルに表示する `cat` コマンドを作ります。そしていよいよ、実行可能ファイルを開いてアプリを実行する仕組みを作ってみます。C++ で書いたアプリが起動すると感動間違いなしです。

18.1 ファイルアロケーションテーブル (osbook_day18a)

FAT はファイルアロケーションテーブル (file allocation table) の頭文字を取ったもので、FAT ファイルシステムの中核を成すデータ構造です。FAT はファイルがどのクラスタに割り当てられているか (allocated) を記録しています。FAT を解釈できるようになると、1 クラスタを超えて複数のクラスタにまたがって保存されるファイルをうまく扱えるようになります。

表 18.1 FAT クラスタチェーンの例

クラスタ番号	次のクラスタ番号	ファイル
0	0x0FFFFFFF8	
1	0x0FFFFFFF	
2	0x0FFFFFFF8	/
3	0x0FFFFFFF	/EFI
4	0x0FFFFFFF	/EFI/BOOT
5	6	/EFI/BOOT/BOOTX64.EFI
6	7	/EFI/BOOT/BOOTX64.EFI
⋮	⋮	⋮
15	16	/EFI/BOOT/BOOTX64.EFI
16	0x0FFFFFFF	/EFI/BOOT/BOOTX64.EFI
17	18	/KERNEL.ELF
18	19	/KERNEL.ELF
⋮	⋮	⋮
2542	2543	/KERNEL.ELF
2543	0x0FFFFFFF	/KERNEL.ELF
2544	2545	/MEMMAP
2545	2546	/MEMMAP
2546	0x0FFFFFFF	/MEMMAP

FAT はクラスタチェーン (クラスタの鎖) の構造になっています。鎖をたどっていくことで、あるクラスタ番号の次のクラスタが何番なのかを調べることができます。具体例を見てみます。QEMU の起動に使っているボリュームイメージ `disk.img` のクラスタチェーンを表 18.1 に示します。クラスタ 2 から始まるルートディレクトリの次のクラスタを見ると 0x0FFFFFFF8 になっています。これはクラスタチェーンの末尾を表す値で、ルートディレクトリはここで終わりだということが分かります。クラスタ 3 からは EFI というディレクトリが始まります。これも 1 つのクラスタで終わりです。

複数クラスタをまたがる初めてのファイルは BOOTX64.EFI です。このファイルはクラスタ 5 から始まります。クラスタチェーンをたどてみると 5 → 6 → 7 → …… → 15 → 16 となります。disk.img では 1 クラスタは 1KiB ですの、約 12 × 1KiB = 12KiB 程度の大きさだろう、ということが分かります。実際に BOOTX64.EFI の大きさを調べると約 11.8KiB でした。このように、FAT を利用すると複数クラスタにまたがるファイルをきちんと扱えます。

リスト 18.1 NextCluster() はクラスタチェーンの次のクラスタ番号を得る (fat.cpp)

```

unsigned long NextCluster(unsigned long cluster) {
    uintptr_t fat_offset =
        boot_volume_image->reserved_sector_count *
        boot_volume_image->bytes_per_sector;
    uint32_t* fat = reinterpret_cast<uint32_t*>(
        reinterpret_cast<uintptr_t>(boot_volume_image) + fat_offset);
    uint32_t next = fat[cluster];
    if (next >= 0xfffffffful) {
        return kEndOfClusterchain;
    }
    return next;
}

```

リスト 18.1 に、クラスタチェーンを順にたどるための関数 `fat::NextCluster()` の実装を示します。初めに FAT データ構造の位置を計算しています。FAT データ構造は大切なデータですので、1つのファイルシステムの中に典型的には同じものが2つ存在します（図 17.3）。この関数ではそのうち1つ目の FAT を使うことにします。本来は2つの FAT に差がないことを調べた方がいいのですが、実装が面倒なのでさぼります。

ボリューム先頭から FAT までのバイトオフセットを計算して `fat_offset` という変数に設定します。そのオフセット値を `boot_volume_image` のアドレスに加えれば、メモリ上で FAT が存在するアドレスを得られます。FAT32 ファイルシステムでは FAT は 32 ビット整数の配列ですので、得られたアドレスを `uint32_t*` 型のポインタにキャストしています。

FAT の先頭を指すポインタ `fat` の添字にクラスタ番号を与えるとクラスタチェーンを順にたどれます。`fat[cluster]` というコード片と表 18.1 の関係を説明します。表の「クラスタ番号」列が添字 `cluster`、「次のクラスタ番号」列が要素の値 `fat[cluster]`、という対応関係になっています。すなわち `cluster → fat[cluster] → fat[fat[cluster]] → ……` というようにクラスタチェーンをたどることができます。

リスト 18.2 クラスタチェーンの末尾を表す特別なクラスタ番号 (fat.hpp)

```

static const unsigned long kEndOfClusterchain = 0xfffffffflu;

```

`0xffffffff` 以上の値はクラスタチェーンの末尾を表します。なぜ1つの値にしなかったのか疑問ですが、とにかくそれ以上の値が末尾を表すと仕様に書いてあるので仕方ありません。実際のボリュームイメージを調べてみると、確かに `0xffffffff` だったり `0xffffffff` だったりとはらつきがあるようです。MikanOS ではクラスタチェーンの末尾を表す定数 `kEndOfClusterchain` を作ることにしました（リスト 18.2）。

リスト 18.3 FindFile() は指定された名前のファイルを探す (fat.cpp)

```

DirectoryEntry* FindFile(const char* name, unsigned long directory_cluster) {
    if (directory_cluster == 0) {
        directory_cluster = boot_volume_image->root_cluster;
    }
}

```

```

while (directory_cluster != kEndOfClusterchain) {
    auto dir = GetSectorByCluster<DirectoryEntry>(directory_cluster);
    for (int i = 0; i < bytes_per_cluster / sizeof(DirectoryEntry); ++i) {
        if (NameIsEqual(dir[i], name)) {
            return &dir[i];
        }
    }

    directory_cluster = NextCluster(directory_cluster);
}

return nullptr;
}

```

ファイル検索関数 `fat::FindFile()` を実装してみます (**リスト 18.3**)。先ほど実装した `NextCluster()` を使うことで、ファイルがたくさんあり、ディレクトリ(ディレクトリエントリの配列)が複数クラスタにまたがっていても対応できるようになっています。

このファイル検索関数は指定されたディレクトリの中を指定された名前で検索し、一致するファイルを返します。ディレクトリの指定はディレクトリの開始クラスタ番号により行います。引数 `directory_cluster` にクラスタ番号を渡します。

`fat.hpp` を見ると分かりますが、引数 `directory_cluster` にはデフォルト値として 0 が設定されています。したがって `FindFile()` を第 1 引数のみで呼び出すと冒頭の `if` 文が実行され、ルートディレクトリのクラスタ番号が `directory_cluster` に設定されることになります。すなわち、この関数は第 1 引数のみを指定することで、ルートディレクトリからファイルを探してくれます。

リスト 18.4 `bytes_per_cluster` の値を計算する (`fat.cpp`)

```

BPB* boot_volume_image;
unsigned long bytes_per_cluster;

void Initialize(void* volume_image) {
    boot_volume_image = reinterpret_cast<fat::BPB*>(volume_image);
    bytes_per_cluster =
        static_cast<unsigned long>(boot_volume_image->bytes_per_sector) *
        boot_volume_image->sectors_per_cluster;
}

```

`for` 文で使っている `bytes_per_cluster` は **リスト 18.4** のように初期化関数内で設定しています。よく使う定数ですから、いちいち計算式を書かなくて済むように最初に計算しておきます。

リスト 18.5 `NameIsEqual()` はファイル名が一致するかを確認する (`fat.cpp`)

```

bool NameIsEqual(const DirectoryEntry& entry, const char* name) {
    unsigned char name83[11];
    memset(name83, 0x20, sizeof(name83));

    int i = 0;
    int i83 = 0;
    for (; name[i] != 0 && i83 < sizeof(name83); ++i, ++i83) {

```



```

    if (name[i] == '.') {
        i83 = 7;
        continue;
    }
    name83[i83] = toupper(name[i]);
}

return memcmp(entry.name, name83, sizeof(name83)) == 0;
}

```

ファイル名の比較に使っている `fat::NameIsEqual()` の実装を **リスト 18.5** に示します。この関数は `entry.name` と `name` を比較し、等しければ真を返します。文字列の単純な比較ではなく 8+3 形式に基づいて比較する必要があるため複雑なプログラムになっています。

内部の処理を詳しく見てみます。関数冒頭では 11 バイトの配列 `name83` を用意し、すべて空白文字で埋めます。次の `for` 文では、その配列に引数 `name` の内容をコピーして 8+3 形式に変換します。コピーする際は 1 文字ずつ大文字に変換します。2 つのカウンタ変数 `i`、`i83` はそれぞれ `name`、`name83` の添字として使っています。

関数の最後では、今しがた構築した 8+3 形式の文字列 `name83` とディレクトリエントリの `name` を比較します。一致すれば真を返します。

リスト 18.6 cat コマンドの実装 (terminal.cpp)

```

} else if (strcmp(command, "cat") == 0) {
    char s[64];

    auto file_entry = fat::FindFile(first_arg);
    if (!file_entry) {
        sprintf(s, "no such file: %s\n", first_arg);
        Print(s);
    } else {
        auto cluster = file_entry->FirstCluster();
        auto remain_bytes = file_entry->file_size;

        DrawCursor(false);
        while (cluster != 0 && cluster != fat::kEndOfClusterchain) {
            char* p = fat::GetSectorByCluster<char>(cluster);

            int i = 0;
            for (; i < fat::bytes_per_cluster && i < remain_bytes; ++i) {
                Print(*p);
                ++p;
            }
            remain_bytes -= i;
            cluster = fat::NextCluster(cluster);
        }
        DrawCursor(true);
    }
} else if (command[0] != 0) {

```

いよいよ `cat` コマンドを実装します。`cat` コマンドとは POSIX 系の OS で使われるコマンドで、典型的にはファイルの内容を画面（標準出力）に出す用途で使われます。今回作るコマンドもファイル名

を1つ受け取って、その内容を画面に出力するようにします。**リスト 18.6**に示すように、その他のコマンドと同じく `Terminal::ExecuteLine()` 内に追加しました。実装を上から見ていきましょう。

まず `FindFile()` を使って、引数で指定されたファイルを探します。第2引数 `directory_cluster` に値を指定していませんから、ルートディレクトリからファイルを検索することになります。ファイルが見つからない場合、変数 `file_entry` はヌル値になりますのでエラーメッセージを出して終了します。

ファイルが見つければ、そのファイルの内容の表示に取り掛かります。`file_entry->FirstCluster()` は**リスト 18.7**に示すように、とびとびに存在する2つのフィールドをつなぎ合わせた値を返すメソッドです。FAT32のクラスタ番号は32ビット幅ですが、16ビットずつ分かれて存在するためビット演算でつなぎ合わせます。

リスト 18.7 ディレクトリエントリを表す構造体 (fat.hpp)

```
struct DirectoryEntry {
    unsigned char name[11];
    Attribute attr;
    uint8_t ntres;
    uint8_t create_time_tenth;
    uint16_t create_time;
    uint16_t create_date;
    uint16_t last_access_date;
    uint16_t first_cluster_high;
    uint16_t write_time;
    uint16_t write_date;
    uint16_t first_cluster_low;
    uint32_t file_size;

    uint32_t FirstCluster() const {
        return first_cluster_low |
            (static_cast<uint32_t>(first_cluster_high) << 16);
    }
} __attribute__((packed));
```

ファイルのクラスタ番号が取得できたら、`fat::GetSectorByCluster()` を使ってメモリ上のブロックを取得し、内容を表示します。表示に使っている `Print()` は今回新しく追加した、1文字表示用のメソッドです。後で実装を紹介します。この関数を使って1バイトずつファイル内容を表示していきますが、ファイルが1クラスタ(QEMUで使っているボリュームイメージでは1KiB)より大きい場合は表示を中断し、クラスタチェーンをたどらなければなりません。

クラスタチェーンをたどる処理は `cluster = fat::NextCluster(cluster);` の部分です。この処理によって、ファイルがとびとびのクラスタに散らばっていても正しくファイルの続きを読むことができます。クラスタチェーンの末尾に到達すると `fat::NextCluster(cluster)` は `kEndOfClusterchain` を返しますから、そうになったらループを終了します。

リスト 18.8 Print(char) は画面に 1 文字だけ出力する (terminal.cpp)

```

void Terminal::Print(char c) {
    auto newline = [this]() {
        cursor_.x = 0;
        if (cursor_.y < kRows - 1) {
            ++cursor_.y;
        } else {
            Scroll1();
        }
    };

    if (c == '\n') {
        newline();
    } else {
        WriteAscii(*window_>Writer(), CalcCursorPos(), c, {255, 255, 255});
        if (cursor_.x == kColumns - 1) {
            newline();
        } else {
            ++cursor_.x;
        }
    }
}

```

リスト 18.8 に 1 文字表示用の Print(char) の実装を示します。引数で指定された文字が改行文字なら改行し、そうでなければ画面に 1 文字出力します。画面の右端に到達したら折り返して表示を行います（ワードラップ処理）。この処理は見覚えがあると思います。そうです。今までも文字列表示をする Print() という同じ名前の関数がありました。

リスト 18.9 Print(const char*) は画面に文字列を出力する (terminal.cpp)

```

void Terminal::Print(const char* s) {
    DrawCursor(false);

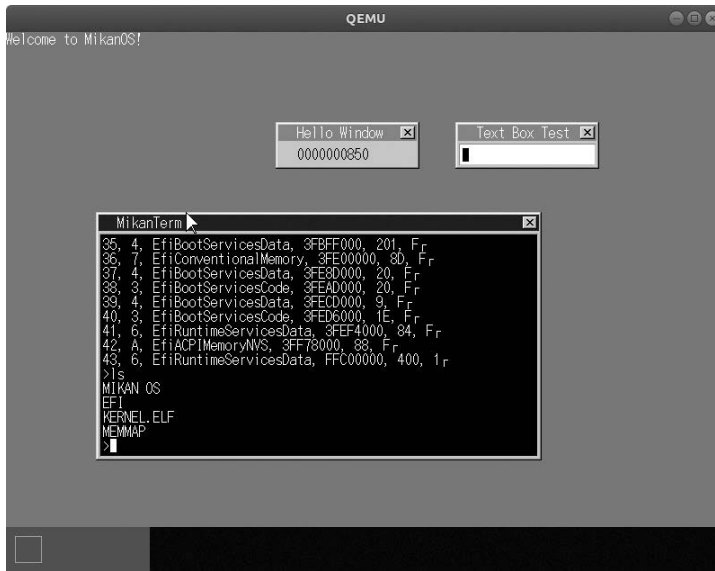
    while (*s) {
        Print(*s);
        ++s;
    }

    DrawCursor(true);
}

```

従来からある Print(const char*) は、今回追加した 1 文字表示用関数を使って**リスト 18.9** のようにシンプルになりました。1 文字表示用関数の中ではいちいちカーソルの再描画は行わず、その外側でやることで効率化しています。

図 18.1 cat コマンドで MEMMAP ファイルを表示する



実装が一通り完了しましたので cat コマンドを実行してみます。試しに、ブートローダで書き出したメモリマップファイル MEMMAP を表示してみます (図 18.1)。ふむふむ、メモリマップらしき文字列がターミナルに表示されています。良さそうです。ちなみに、行末にかきかっこのような記号が表示されていますが、これは UEFI でファイルを書き込んだときに自動的に追加された CR (0x0d) が表示されているのです。バグではありません。

18.2 初アプリ (osbook_day18b)

ボリューム上のファイルを読み込んで表示することができました。この技術を応用すれば実行可能ファイルを読み込んで起動するというのもできそうな気がします。OS 本体とは別のファイルとして作ったアプリケーションを読み込んで実行する機能は、まさに OS だなあという感じがしてワクワクしませんか？

リスト 18.10 初めてのアプリは hlt するだけ (onlyhlt.asm)

```
bits 64
section .text

loop:
    hlt
    jmp loop
```

記念すべき初めてのアプリの実装をリスト 18.10 に示します。このプログラムは見ての通り、ただ hlt し続けるだけの本当に単純なものです。画面にメッセージを表示することさえしません。というより、アプリから画面に文字列を表示するための機能がまだ OS になく、このくらい単純なアプリしか作

れないという方が正しいのですが。

リスト 18.11 アプリをビルドしてフラットバイナリにする (Makefile)

```
TARGET = onlyhlt

.PHONY: all
all: $(TARGET)

onlyhlt: onlyhlt.asm Makefile
        nasm -f bin -o $@ $<
```

リスト 18.11 にアプリケーションをビルドするための Makefile を示します。この中で最も大事なのは `nasm` を実行している行です。オプション `-f bin` はフラットバイナリを生成するオプションです。**フラットバイナリ**とは、ファイルの先頭から純粋に機械語だけが詰めて配置されたファイルのことです。OS 本体のファイル形式である ELF などは構造化されていて、ファイル先頭にはヘッダがあり、実行コードやデータがそれぞれ切り出せるように配置されていますが、フラットバイナリはベタ書きの機械語になっています。百聞は一見にしかず、実際にビルドして生成されたファイルの中身を見てみます。

```
$ source $HOME/osbook/devenv/buildenv.sh
$ cd $HOME/workspace/mikanos/apps/onlyhlt
$ git checkout osbook_day18b
$ make
nasm -f bin -o onlyhlt onlyhlt.asm
$ hexdump -C onlyhlt
00000000 f4 eb fd |...|
00000003
```

ビルドすると `onlyhlt` というファイルが生成されます。これを 16 進数で表示してみると、どうやら 3 バイトの中身を持つファイルのようです。この 3 バイトは機械語のはずですから逆アセンブルしてみます。

```
$ objdump -D -m i386:x86-64 -b binary onlyhlt
< 中略 >
0000000000000000 <.data>:
   0: f4                hlt
   1: eb fd            jmp     0x0
```

ふむふむ。先ほどの 3 バイトは `hlt` と `jmp 0` に対応しているようです。`onlyhlt.asm` に書いたプログラムが、そのまま機械語に変換されてファイルの先頭から書かれているということが確かめられました。あれ？ `jmp 0x0` とありますが、実際に書いたのは `jmp loop` のはず。0 番地とは書いていません。これは一体どういうことでしょうか？

実は機械語の `eb fd` は**相対ショートジャンプ**と呼ばれるもので、その時点の RIP レジスタの値を基準として相対位置にジャンプします。相対ショートジャンプ自体を表すのが `0xEB` で、ジャンプ先を表すのが `0xFD` です。`0xFD` を符号付き整数として解釈すると -3 となりますので、RIP より 3 バイト手前にジャンプすることになります。x86-64 アーキテクチャでは、ある命令を実行中の RIP の値は次の命

令を表すことになっているため、この相対ショートジャンプは `jmp 0x0` の次の命令から 3 バイト手前、すなわち `hlt` までジャンプすることになります。ということで、この相対ショートジャンプはたった 2 バイトで無限ループを実現しています。

さて、生成された `onlyhlt` というバイナリファイルを実行するにはどうすればいいのでしょうか？これは OS 本体をどうやって起動したかを思い出せば簡単です。OS 本体は、ブートローダがメモリ上に読み込み、そのエントリポイントを関数だと思って呼び出すことで実現していました。同じようにすればいいはずです。ターミナルに `onlyhlt` と入力したらその処理が実行されるようにします。

リスト 18.12 打ち込まれた名前が組み込みコマンド以外ならファイルを探す (terminal.cpp)

```

} else if (command[0] != 0) {
    auto file_entry = fat::FindFile(command);
    if (!file_entry) {
        Print("no such command: ");
        Print(command);
        Print("\n");
    } else {
        ExecuteFile(*file_entry);
    }
}

```

リスト 18.12 にターミナル側の改造を示します。ターミナルに入力したコマンド名が組み込みのコマンド (`ls` や `cat` など) のいずれとも一致しない場合の処理です。その名前のファイルを探し、見つけたら `ExecuteFile()` を呼び出します。

リスト 18.13 `ExecuteFile()` はファイルを読み込んで実行する (terminal.cpp)

```

void Terminal::ExecuteFile(const fat::DirectoryEntry& file_entry) {
    auto cluster = file_entry.FirstCluster();
    auto remain_bytes = file_entry.file_size;

    std::vector<uint8_t> file_buf(remain_bytes);
    auto p = &file_buf[0];

    while (cluster != 0 && cluster != fat::kEndOfClusterchain) {
        const auto copy_bytes = fat::bytes_per_cluster < remain_bytes ?
            fat::bytes_per_cluster : remain_bytes;
        memcpy(p, fat::GetSectorByCluster<uint8_t>(cluster), copy_bytes);

        remain_bytes -= copy_bytes;
        p += copy_bytes;
        cluster = fat::NextCluster(cluster);
    }

    using Func = void ();
    auto f = reinterpret_cast<Func*>(&file_buf[0]);
    f();
}

```

`ExecuteFile()` の実装を **リスト 18.13** に示します。与えられたファイルをメモリ上に読み込んで、その先頭には関数があると思って呼び出します。ファイルを読み込む処理は `cat` コマンドとほとんど

同じですから説明は不要でしょう。読み込んだバイト列を画面に表示するのではなく、配列 `file_buf` に読み込むという違いくらいしかありません。

配列にファイルの内容を読み込み終わったら、ファイルの先頭から実行可能な機械語が配置されているはずだと信じて関数呼び出しを行います。onlyhlt ファイルの場合は先頭にある hlt の無限ループが実行されるはずで、一度呼び出したらずっと戻ってこないため、ターミナルがフリーズしたように見えるでしょう。ターミナルがフリーズすれば成功、しなかったら失敗です。

リスト 18.14 メッセージ取得後に sti しておく (terminal.cpp)

```
__asm__("cli");
auto msg = task.ReceiveMessage();
if (!msg) {
    task.Sleep();
    __asm__("sti");
    continue;
}
__asm__("sti");
```

本題からはそれなのですが、筆者が改造後の OS で実行したところ、ターミナルだけでなく OS 全体がフリーズすることに気づきました。メインタスクのカーソルの点滅が止まり、マウスが動かなくなりました。どうしてかなと探っていたら、TaskTerminal() の中でキューからメッセージを取得した後、sti を実行していないことに気づきました。リスト 18.14 の最終行の sti がなかったということです。この sti を忘れると、ターミナルは割り込み禁止のまま onlyhlt の実行を始めてしまいます。割り込み禁止状態で hlt を実行すると、タスク切り替えタイマのタイムアウト割り込みさえ発生しなくなるため OS 全体がフリーズするのです。ということで sti を加えて改造は完了です。

いよいよ実行してみたいのですが、1 つ問題があります。onlyhlt ファイルをボリュームイメージに追加する必要があるのですが、今までの方法 (run_qemu.sh を使った方法) ではブートローダ (/EFI/BOOT/BOOTX64.EFI) と OS 本体 (/kernel.elf) しか追加されないからです。そこで新たにスクリプトを作りました (リスト 18.15)。このファイルは \$HOME/osbook/devenv/run_mikanos.sh にあります。

リスト 18.15 MikanOS 起動用スクリプト (run_mikanos.sh)

```
#!/bin/sh -ex

DEVENV_DIR=$(dirname "$0")
DISK_IMG=./disk.img

DISK_IMG=$DISK_IMG $DEVENV_DIR/make_mikanos_image.sh
$DEVENV_DIR/run_image.sh $DISK_IMG
```

スクリプトではまず onlyhlt ファイルを含んだ起動用ボリュームイメージを作ります。その後、作成したイメージを起動します。ボリュームイメージを作る make_mikanos_image.sh が重要なので、リスト 18.16 に紹介します。

リスト 18.16 MikanOS 起動用スクリプト (make_mikanos_image.sh)

```
#!/bin/sh -ex

DEVENV_DIR=$(dirname "$0")
MOUNT_POINT=./mnt

if [ "$DISK_IMG" = "" ]
then
    DISK_IMG=./mikanos.img
fi

if [ "$MIKANOS_DIR" = "" ]
then
    if [ $# -lt 1 ]
    then
        echo "Usage: $0 <day>"
        exit 1
    fi
    MIKANOS_DIR="$HOME/osbook/$1"
fi

LOADER_EFI="$HOME/edk2/Build/MikanLoaderX64/DEBUG_CLANG38/X64/Loader.efi"
KERNEL_ELF="$MIKANOS_DIR/kernel/kernel.elf"

$DEVENV_DIR/make_image.sh $DISK_IMG $MOUNT_POINT $LOADER_EFI $KERNEL_ELF
$DEVENV_DIR/mount_image.sh $DISK_IMG $MOUNT_POINT

if [ "$APPS_DIR" != "" ]
then
    sudo mkdir $MOUNT_POINT/$APPS_DIR
fi

for APP in $(ls "$MIKANOS_DIR/apps")
do
    if [ -f $MIKANOS_DIR/apps/$APP/$APP ]
    then
        sudo cp "$MIKANOS_DIR/apps/$APP/$APP" $MOUNT_POINT/$APPS_DIR
    fi
done

if [ "$RESOURCE_DIR" != "" ]
then
    sudo cp $MIKANOS_DIR/$RESOURCE_DIR/* $MOUNT_POINT/
fi

sleep 0.5
sudo umount $MOUNT_POINT
```

make_mikanos_image.sh の処理を上から順に説明します。

スクリプトではまず各種変数を定義しています。変数 DEVENV_DIR は、このスクリプトが置いてあるディレクトリへのパスになります。MOUNT_POINT はボリュームイメージをマウントするディレクトリ名、DISK_IMG はボリュームイメージの名前を表します。変数 MIKANOS_DIR の処理は少々複雑です。この if 文でやりたいのは、変数が定義されていればその値を、定義されていない（値が空）ならスクリ

プトの第1引数を変数の値として使います。

変数の定義が終わったら、make_image.shを使ってBOOTX64.EFIとkernel.elfを含んだボリュームイメージdisk.imgを作ります。make_image.shは最後にイメージをアンマウントしてしましますが、さらに変更を加えるためにmount_image.shを使ってイメージをマウントし直します。

マウントしたらアプリをイメージにコピーします。\$MIKANOS_DIR/appsからアプリを検索し、見つけたものをすべて\$MOUNT_POINT/\$APPS_DIR以下にコピーします。変数APPS_DIRを指定しなければ空文字となり、アプリは\$MOUNT_POINT/にコピーされることになります。現段階では\$MIKANOS_DIR/apps/onlyhlt/onlyhltをmnt/onlyhltとしてコピーすることになります。最後にイメージをアンマウントし、完成したイメージをrun_image.shによって起動します。

リスト 18.17 OSとアプリ全体をビルドするスクリプト (build.sh)

```
#!/bin/sh -eu

make ${MAKE_OPTS:-} -C kernel kernel.elf

for MK in $(ls apps/*/Makefile)
do
    APP_DIR=$(dirname $MK)
    APP=$(basename $APP_DIR)
    make ${MAKE_OPTS:-} -C $APP_DIR $APP
done

if [ "${1:-}" = "run" ]
then
    MIKANOS_DIR=$PWD $HOME/osbook/devenv/run_mikanos.sh
fi
```

もう1つ、OS本体とアプリすべてをビルドするためのスクリプトも追加しました。\$HOME/workspace/mikanos/build.sh (Gitタグ: osbook_day18b) です (リスト 18.17)。このスクリプトは引数無しで実行すると全体をビルドします。引数runを加えて実行すると、ビルドしたあとでQEMUを起動させます。このスクリプトを使って実行するときは次のようにします。

```
$ cd $HOME/workspace/mikanos
$ git checkout osbook_day18b
$ ./build.sh run
```

こうすると全体がビルドされ、先ほど作ったrun_mikanos.shによってQEMUが起動します。

図 18.2 onlyhlt を起動したらターミナルがフリーズした



cat memmap コマンドに続いて onlyhlt コマンドを実行してみたのが図 18.2 です。スクリーンショットだと分かりにくいですが、onlyhlt と入力してからエンターキーを押したらカーソルの点滅が止まりました。マウスカーソルを動かしたり、「Text Box Test」の方にキー入力したりできますが、ターミナルは一切反応しなくなりました。どうやら onlyhlt が期待通りに動いたみたいですね。

18.3 C++ で計算機 (osbook_day18c)

ここまで来ると、皆さんとしては「アプリを書いて起動できたから、さっそく面白いアプリを作りまくるぞ!」と意気込むかもしれません。しかし、アセンブリ言語でしかアプリを作れないとなると大変ですので、C++ で書けるようにしておきたいところです。この節では C++ でアプリを作って、それを起動させることに挑戦します。

アセンブリ言語で onlyhlt コマンドはフラットバイナリとして作りましたが、本格的なプログラムを作るにはフラットバイナリでは表現力が足りません。本格的なプログラムではグローバル変数などの静的変数、文字列等の固定データ、機械語などが組み合わさって機能を実現しますが、フラットバイナリでは機械語の表現しかできません。静的変数をおくためのメモリ領域が何バイト分必要だとか、ファイルの中でどこに固定データが配置してあるかなど、機械語以外に表現しなければならない事柄があります。

今まで作ってきた OS 本体のファイル形式が ELF 形式だったことを思い出してください。実は ELF はプログラムに必要な各種の情報を表現できる力を持っており、OS 本体だって扱えちゃうファイル形式です。そのパワフルな ELF でアプリを作り、起動できるように `Terminal::ExecuteFile()` を改良するという方針にします。

さて、とりあえず何か少し本格的なアプリを作りたいと思います。何が良いかなと思ったのです

が、ここでは逆ポーランド記法電卓を作ってみることにします。**逆ポーランド記法**^{*1}とは数式の書き方の1種で、演算子を後ろに置きます。そのため後置記法と呼ばれることもあります。私たちが普段良く使う中置記法と比較すると表 18.2 のようになります。

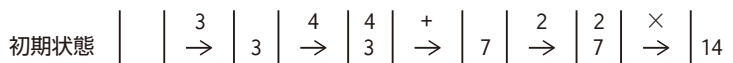
表 18.2 演算子の配置場所による違い

中置記法	逆ポーランド記法 (後置記法)	逆ポーランド記法の異なる表現
2+3	2 3 +	
(3+4) × 2	3 4 + 2 ×	2 3 4 + ×

逆ポーランド記法の大きな特徴として、括弧を使わずとも計算の順序を紛れなく表現できます。この特徴によって計算プログラムを簡単に書くことができます。逆ポーランド記法で書かれた数式は、数値用のスタックを使って次の手順で計算することができます。

1. 数値を格納するための空のスタックを用意する。
2. 数式の左から数値または演算子を 1 個ずつ読み取る。
3. 読み取ったものが数値であればスタックにプッシュする。
4. 読み取ったものが 2 項演算子であれば、スタックから数値を 2 つポップし、計算結果をスタックにプッシュする。
5. 数式の末尾に到達するまで 2 ～ 4 を繰り返す。
6. スタックに残っている数値が計算結果を表す。

図 18.3 逆ポーランド記法「3 4 + 2 ×」を計算する際のスタックの様子



このアルゴリズムに基づいて「3 4 + 2 ×」を計算するときのスタックの変化の様子を図 18.3 に示します。手順とスタックの変化の様子を見比べれば難しいことはないと思います。最後にスタックに残った 14 が計算結果ということになります。正しそうです。

リスト 18.18 逆ポーランド記法計算機の実装 (rpn.cpp)

```
int strcmp(const char* a, const char* b) {
    int i = 0;
    for (; a[i] != 0 && b[i] != 0; ++i) {
        if (a[i] != b[i]) {
            return a[i] - b[i];
        }
    }
    return a[i] - b[i];
}
```

```
long atol(const char* s) {
    long v = 0;
```

*1 RPN : Reverse Polish Notation

```

    for (int i = 0; s[i] != 0; ++i) {
        v = v * 10 + (s[i] - '0');
    }
    return v;
}

int stack_ptr;
long stack[100];

long Pop() {
    long value = stack[stack_ptr];
    --stack_ptr;
    return value;
}

void Push(long value) {
    ++stack_ptr;
    stack[stack_ptr] = value;
}

extern "C" int main(int argc, char** argv) {
    stack_ptr = -1;

    for (int i = 1; i < argc; ++i) {
        if (strcmp(argv[i], "+") == 0) {
            long b = Pop();
            long a = Pop();
            Push(a + b);
        } else if (strcmp(argv[i], "-") == 0) {
            long b = Pop();
            long a = Pop();
            Push(a - b);
        } else {
            long a = atol(argv[i]);
            Push(a);
        }
    }
    if (stack_ptr < 0) {
        return 0;
    }
    return static_cast<int>(Pop());
}

```

逆ポーランド記法の計算手順に基づいて計算するプログラムを作りました。**リスト 18.18** です。このプログラムはコマンドライン引数として受け取った数式を計算し、結果をプログラムの終了コードとして出力します。実は、このプログラムは MikanOS のアプリとしてだけでなく Linux 上でも実行できます。Linux で実行するために、まずは Clang を用いてビルドします。

```
$ clang++ rpn.cpp
```

すると実行ファイル `a.out` が生成されるはずですので、コマンドライン引数に計算式を渡して計算させます。

```
$ ./a.out 2 3 +
$ echo $?
5
```

コマンドの終了コードは \$? という特殊な変数に設定されますので、コマンドが終了した直後に echo \$? とすると表示できます。どうやら正しい答えが得られたようですね。

Linux 上での実行方法が分かりましたので、次はプログラムに簡単な説明を加えます。まず、**リスト 18.18**を見ると 1 つも #include が無いことに違和感を感じませんか？ 印刷のミスではないのです。インクルードをしていないのは、MikanOS ではまだアプリ用の標準ライブラリを用意していないからです。Newlib や libc++ といった標準ライブラリを使うにはいくつかの OS 依存の関数群が必要です。OS 用の関数群は newlib_support.c や libcxx_support.cpp などに定義していますが、アプリ用のものはまだ用意していませんでした。

ということで標準ライブラリに含まれる関数である strcmp() や atol() を自分で定義することにしました。strcmp() は 2 つの文字列を比較し、一致していれば 0 を返します。atol() は 10 進数表現の文字列を数値に変換します。

次に逆ポーランド記法の計算の核であるスタックの定義を行います。stack がスタック本体となる配列で、stack_ptr が配列の添字としてスタックトップの位置を表します。Pop() と Push() はその名の通りポップ操作とプッシュ操作を実現します。

main() が数式の計算の主要部分を実現しています。for 文の中でコマンドライン引数 argv から 1 つずつ数値や演算子を読み取って処理を進めます。すべての引数を消費した後にスタックに残った数値を終了コードとして返します。for 文の中を見ると分かりますが演算子は + と - だけ対応しています。増やしたい人は改造してみると良いと思います。

リスト 18.19 MikanOS 向け実行ファイルのためのビルドスクリプト (Makefile)

```
TARGET = rpnp

CPPFLAGS += -I.
CFLAGS += -O2 -Wall -g --target=x86_64-elf -ffreestanding
CXXFLAGS += -O2 -Wall -g --target=x86_64-elf -ffreestanding \
            -fno-exceptions -fno-rtti -std=c++17
LDFLAGS += --entry main -z norelro --image-base 0 --static

.PHONY: all
all: $(TARGET)

rpn: rpn.o Makefile
    ld.lld $(LDFLAGS) -o rpn rpn.o

%.o: %.cpp Makefile
    clang++ $(CPPFLAGS) $(CXXFLAGS) -c $< -o $@
```

rpn.cpp を MikanOS 用にビルドするための Makefile を **リスト 18.19** に示します。通常、OS 上で動くアプリケーションはホスト環境で動くようにビルドします。一方、OS そのものは OS がない環境であるフリースタANDING環境を対象にビルドします。逆ポーランド記法計算アプリは MikanOS という OS 上で動くアプリですからホスト環境かなと思いきや、今はまだホスト環境のための支援機能^{*2}が MikanOS には備わっていないため、アプリもフリースタANDING環境としてビルドする必要があります。したがって `-ffreestanding` を付けてコンパイルします。

リンカ `ld.lld` は出力形式を指定しなければ ELF 形式で出力します。したがって、この Makefile が生成する実行ファイル `rpn` は ELF 形式となります。`--image-base 0` を指定していますので、実行ファイルはメモリアドレス 0 から配置される前提でリンクされます。後で ELF ファイルを読み込む処理を作りますが、その時に読み込む先のメモリアドレスを固定することができず、アドレス調整が必要となります。`--image-base` を 0 としておくことで、アドレス調整の計算を少し楽にできます。

`rpn` の実装は完了です。次は ELF 形式のアプリを起動できるように、ターミナルのアプリ起動部分を拡張します。`rpn` はコマンドライン引数 `argv` に数式を渡す必要がありますから、そのための仕組みも実装します。

リスト 18.20 ExecuteFile() にコマンドライン引数を渡す (terminal.cpp)

```
auto file_entry = fat::FindFile(command);
if (!file_entry) {
    Print("no such command: ");
    Print(command);
    Print("\n");
} else {
    ExecuteFile(*file_entry, command, first_arg);
}
```

リスト 18.20 に示すように `ExecuteFile()` に引数を 2 つ増やしました。コマンド名とコマンドライン引数です。例えば `rpn 2 3 +` とターミナルに打ち込むと `command` は `"rpn"` に、`first_arg` は `"2 3 +"` になるはずですが。

リスト 18.21 ELF 形式かどうかでアプリ起動処理を分ける (terminal.cpp)

```
auto elf_header = reinterpret_cast<Elf64_Ehdr*>(&file_buf[0]);
if (memcmp(elf_header->e_ident, "\x7f" "ELF", 4) != 0) {
    using Func = void ();
    auto f = reinterpret_cast<Func*>(&file_buf[0]);
    f();
    return;
}

auto argv = MakeArgVector(command, first_arg);

auto entry_addr = elf_header->e_entry;
entry_addr += reinterpret_cast<uintptr_t>(&file_buf[0]);
```

*2 ホスト環境を謳うには、マルチスレッドや標準入出力、ファイルシステム操作などをサポートする必要があります。

```

using Func = int (int, char**);
auto f = reinterpret_cast<Func*>(entry_addr);
auto ret = f(argv.size(), &argv[0]);

char s[64];
sprintf(s, "app exited. ret = %d\n", ret);
Print(s);

```

リスト 18.21 に `ExecuteFile()` 内部の改造箇所を示します。もともとはフラットバイナリのための起動処理だけでしたが、今回からは ELF 形式かどうかで条件分岐するようにしました。ELF 形式は先頭 4 バイトが「0x7f」「E」「L」「F」であると決まっていますので、`memcmp()` を使って 4 バイトの比較をしています。ELF 形式でないと判定された場合は、従来通りのフラットバイナリの起動処理を行います。

"\x7f" "ELF" という書き方が不思議に思うかもしれません。これは、2 つの文字列リテラルを連続して書くと自動的に結合するという C/C++ の仕様を利用した書き方です。すなわち "\x7f" "ELF" は結合されて 5 バイト (4 バイト + ナル文字) の文字列リテラルとなります。わざわざ分割しているのは、"\x7fELF" と書いてしまうと \x7fe まだが 16 進数だと解釈されてしまう (挙句のはてに、0x7fe が 1 バイトの範囲外のためコンパイルエラーになる) からです。

ELF 形式と判定されたら新しく追加した起動処理に移ります。まず `MakeArgVector()` でコマンドライン引数を空白で区切って配列にします。その後、ELF ヘッダに基づいてエントリポイントのアドレスを取得し、`&file_buf[0]` を足してから呼び出します。ELF ヘッダに記録されているエントリポイントアドレスは ELF ファイルをアドレス 0 に置くことを前提とした数値ですので、実際に配置されるアドレスを足す必要があるのですね。

`f(argv.size(), &argv[0])` がエントリポイント、すなわちアプリの `main()` を実行している部分です。エントリポイントに 2 つの引数を渡して実行します。エントリポイントの実行が完了するとここに戻ってきて、エントリポイントの戻り値が `ret` 変数に書き込まれます。アプリが終了したことに加え、その値を画面に表示するようにしました。逆ポーランド記法アプリは `main()` の戻り値として計算結果を出力しますから、上記のように戻り値を表示しなければ計算結果が分かりません。

リスト 18.22 `MakeArgVector()` はコマンドライン引数を空白で区切る (terminal.cpp)

```

namespace {

std::vector<char*> MakeArgVector(char* command, char* first_arg) {
    std::vector<char*> argv;
    argv.push_back(command);

    char* p = first_arg;
    while (true) {
        while (isspace(p[0])) {
            ++p;
        }
        if (p[0] == 0) {
            break;
        }
        argv.push_back(p);

        while (p[0] != 0 && !isspace(p[0])) {

```

```

    ++p;
}
if (p[0] == 0) {
    break;
}
p[0] = 0;
++p;
}

return argv;
}

} // namespace

```

MakeArgVector()の実装をリスト 18.22 に示します。まずコマンド名を argv[0] に格納します。while ループの中でコマンドライン引数を空白で区切り、argv[1] から順に格納します。isspace() は C 標準ライブラリの関数で、指定した文字が空白文字（スペースやタブなど）であれば真を返します。

図 18.4 rpnn コマンドを使って計算を行う様子



逆ポーランド記法アプリを使っている様子を図 18.4 に示します。いくつか計算をしてみましたが生しい答えを返しているようですね。しかも Linux 上と同じように実行できて、筆者は地味に感動しています。随分と本格的な OS になってきましたよ。

18.4 標準ライブラリ (osbook_day18d)

逆ポーランド記法アプリを作るときに `strcmp()` と `atol()` を自作したのはなんだか負けた気がします。標準ライブラリにある関数は自分で作りたくありません。なんとかして標準ライブラリをアプリでも使いたいです。

幸いにも OS とアプリは同じ CPU 上で動きますので、標準ライブラリをアプリ用にビルドし直す必要はありません。アプリ用に `newlib_support.c` や `libcxx_support.cpp` を用意するだけで大丈夫なはずです。というより、実は `strcmp()` と `atol()` だけならそれらのファイルの用意さえ不要なので、単に標準ライブラリをリンクしさえすれば完了です。

リスト 18.23 標準ライブラリをリンクする (rpn/Makefile)

```
rpn: rpn.o Makefile
    ld.lld $(LDFLAGS) -o rpn rpn.o -lc -lc++ -lc++abi
```

ということで `apps/rpn/Makefile` を変更して標準ライブラリをリンクするようにしました (リスト 18.23)。`-lc` が C 標準ライブラリ、`-lc++` と `-lc++abi` が C++ 標準ライブラリです。

リスト 18.24 標準ライブラリをインクルードして使う (rpn/rpn.cpp)

```
#include <cstring>
#include <cstdlib>

int stack_ptr;
long stack[100];
```

rpn.cpp の方も標準ライブラリを利用するように書き換えます（リスト 18.24）。特に難しいことはないかなあとします。

図 18.5 標準ライブラリを使うように改造した rpn コマンド



改造が完了したらビルドして実行させてみます。QEMUで動かした様子を図 18.5 に示します。うーん、おかしいですね。計算すると「5」になるはずが「0」と表示されてしまっています。この謎は次章で調査することとします。

第 19 章

ページング

個別のファイルとして作ったアプリケーションを実行できるようになり、さらに本物の OS に近づいてきました。しかし、前の章の最後で作ったアプリはなぜかうまく動作しませんでした。この章ではアプリとメモリアドレスの関係を紐解きつつ、ページングという CPU の機能を用いてアプリが正常に動作するようにします。この章を読み終わる頃にはページングの仕組みと実際の設定方法が分かります。

19.1 実行ファイルとメモリアドレス

標準ライブラリを使うようにした `rpn` コマンドが誤動作してしまう原因を解明します。最も怪しいのはエントリポイントを呼び出す前にアドレスを調整している部分です。**リスト 18.21** を思い出してください。

リンカは実行可能ファイルを作るために、そのファイルがどのアドレスに配置されるかを知らなければなりません^{*1}。リンカはデフォルトで `0x200000` とか `0x400000` とかいう番地（リンカの種類により異なります）に配置する前提でリンクを行います。`rpn` コマンドをリンクする際に指定した `--image-base 0` は配置アドレスを 0 番地に変更するものです。本来は 0 ではなく、`rpn` が実際に読み込まれるメモリアドレスを指定したいところですが、そのアドレスは OS の実行時にしか決まらないためリンカに指定できません。ですので、苦肉の策としてリンカには 0 番地に配置されるとっておいて、実行時に実際のアドレスを足しているのです。

生成された `rpn` ファイルを逆アセンブルして、`--image-base` の指定がどう影響しているかを確認します。次に示すコマンドで逆アセンブルを行います。

```
$ objdump -d -M intel -C -S apps/rpn/rpn
```

逆アセンブルの結果から `main()` の一部を抜粋して**リスト 19.1** に載せます。

リスト 19.1 `rpn` ファイルの逆アセンブル結果（抜粋）

```
.....
0000000000001040 <main>:

extern "C" int main(int argc, char** argv) {
.....
if (strcmp(argv[i], "+") == 0) {
    1070:    49 8b 3c dc          mov     rdi,QWORD PTR [r12+rbx*8]
    1074:    be 62 01 00 00      mov     esi,0x162
    1079:    e8 e2 00 00 00      call   1160 <strcmp>
    107e:    85 c0               test    eax,eax
    1080:    74 3e              je      10c0 <main+0x80>
        long b = Pop();
        long a = Pop();
        Push(a + b);
} else if (strcmp(argv[i], "-") == 0) {
    1082:    49 8b 3c dc          mov     rdi,QWORD PTR [r12+rbx*8]
    1086:    be 60 01 00 00      mov     esi,0x160
    108b:    e8 d0 00 00 00      call   1160 <strcmp>
    1090:    85 c0               test    eax,eax
    1092:    74 4c              je      10e0 <main+0xa0>
        long b = Pop();
.....
```

*1 リンカは「位置独立実行可能コード」（PIE：Position Independent Executable）という、どこに配置しても動くファイルを作ることができます。でも本書では、ページングを説明したいという思いがあり、PIE ではないファイルを動作させるというストーリーにしました（PIE ファイルを実行するにはリロケーションが必要となり難易度が高くなるという事情もあります）。

掲載した逆アセンブル結果の前半は `strcmp(argv[i], "+")` に対応する部分です。関数の呼び出しルールは第 1 引数を RDI レジスタに、第 2 引数を RSI レジスタに設定するのです。という知識を思い出してから逆アセンブル結果と C++ のコードを比較すると、第 1 引数 `argv[i]` が `QWORD PTR [r12+rbx*8]` に、第 2 引数 `"+"` が `0x162` に対応していることが分かります。

第 1 引数はちょっと複雑ですが、今注目すべきなのは第 2 引数の方です。第 2 引数をレジスタに設定している命令は `mov esi, 0x162` で、対応する機械語を見ると `be 62 01 00 00` となっています。Intel SDM (参考文献 [3]) によれば `be` は 32 ビットリテラルから 32 ビットレジスタ ESI への `mov` だそうです。ということで後ろに続く 4 バイト `62 01 00 00` は 32 ビットリテラルを表すはずで、`x86-64` はリトルエンディアンですから、この 4 バイトは `0x00000162` と読めます。

`0x162` という数値が機械語レベルで埋め込まれていることが分かりました。この数値は C++ のコード上では `strcmp()` の第 2 引数 `"+"` に対応します。`"+"` は文字列リテラルです。式 (expression) に出てきた文字列リテラルは文字列へのポインタに変換されますので、`0x162` は文字列が置いてある (置かれる予定の) メモリアドレスということになります。リンクは `--image-base 0` の指定にしたがって、実行ファイルをメモリの先頭から配置する前提でリンクします。そのため `0x162` はファイルの先頭から `0x162` の位置に対応するはずで、

逆アセンブル結果の後半は `strcmp(argv[i], "-")` に対応します。この第 2 引数を設定している機械語は `be 60 01 00 00` となっていて、先ほどとほぼ同じですが、RSI レジスタに設定する 32 ビット値が 2 だけずれた `0x00000160` となっています。`0x160` はファイルの先頭から `0x160` の位置に対応するはずで、ファイルの先頭から `0x160` および `0x162` の位置にあるデータを、`hexdump` コマンドを用いて確かめてみます。

```
$ hexdump -C -s 0x160 -n 4 apps/rpn/rpn
00000160 2d 00 2b 00                |-.+.|
00000164
```

`-s 0x160` はファイル先頭から `0x160` バイト以降を読む、`-n 4` は 4 バイトだけ読むという意味があります。コマンドの実行結果を見ると `0x160` から `-`、ヌル文字、`+`、ヌル文字と並んでいることが分かります。これはまさに、ヌル文字で終端された 2 つの文字列 `"-"` と `"+"` です。

ここまでの話をまとめると、実行ファイル `rpn` にはメモリアドレスを表す数値 (`0x162` など) がそのまま埋め込まれていて、そのアドレスは `--image-base` に指定したアドレスを基準に計算されたものであることが分かりました。

一方、MikanOS 上では `rpn` が読み込まれるアドレスは 0 ではありません。そのためエントリポイントのアドレスを調整する必要があったわけです。しかし、エントリポイントのアドレスを調整しただけでは機械語に含まれる `0x162` などの数値までは調整されません。そのため、プログラムは `mov esi, 0x160` をそのまま実行してしまい、`strcmp()` の第 2 引数にその値が渡されてしまいます。もちろん `0x162` には `"+"` という文字列が配置されているわけがありません (奇跡的に同じ値がメモリに書き込まれている可能性はありますが、それは単に偶然です)。その結果、プログラムが誤動作します。以上が、標準ライブラリを使うようにした逆ポーランド記法アプリが MikanOS で正しく動かなかった理由です。

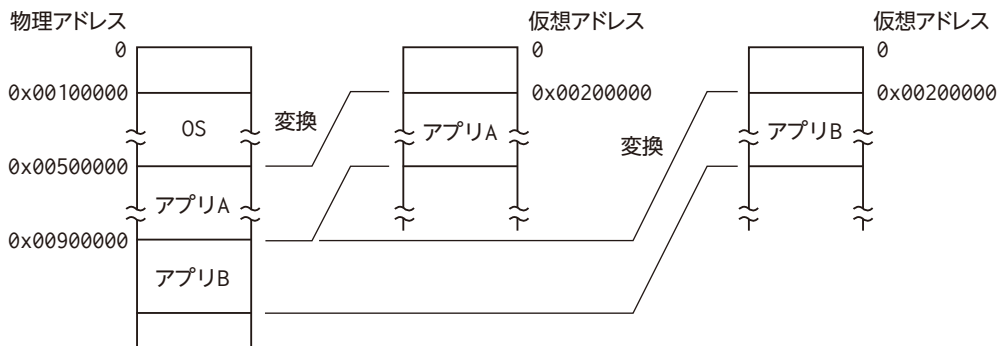
19.2 アドレス変換

このような問題に対処する方法はいくつかあります。1つは実行ファイルを配置するアドレスを予め決めておくことです。一般に複数のアプリがあるはずですので、互いに被らないように配置アドレスを決めます。もちろん OS 本体とも被らないように決める必要があります。この方法はメインメモリの容量が固定で動かすアプリも決まっている、組み込みシステムのようなものでは手軽で有用な方法です。ですが、一般的なパソコンではメインメモリのどの部分が空き領域かが機種によって違いますから、いろいろな機種で動かせる OS を作ろうと思うと、機種ごとにアドレスを求めてリンクしなおさねばなりません。面倒ですね。

2つ目は実行ファイルを再配置（リロケーション）する方法です。実行ファイルの先頭アドレスを仮の値でリンクしておいて、実行ファイルをメモリに配置した後で値を修正することを再配置と呼びます。この方法では意図的に配置アドレスを変えることができますので、実行ファイルを毎回ランダムなアドレスに配置することでセキュリティ攻撃をしにくくする ASLR（address space layout randomization）という技術に応用されたりします。再配置の処理は結構難しいため、本書ではこの方法は採用しませんでした。

3つ目はアプリから見えるメモリアドレスを物理的なアドレスに変換する方法です。アプリそれぞれに対して専用のアドレス変換表を準備しておき、アプリ A とアプリ B が同じアドレス、例えば 0x200000 を参照しても、それぞれ異なる物理アドレスに変換するのです（図 19.1）。ちなみに、ここでアプリ A とアプリ B を 0x200000 に配置するのは、OS 本体が配置されている 0x100000 と重ならないようにするためです。詳しい話は複雑すぎるので省略しますが、OS からアプリに実行を切り替える処理を簡単にするためにずらしておく必要があるのです。

図 19.1 アプリケーションのアドレス変換



アプリ A の動作を開始させるとき、すなわち CPU がアプリ A のコンテキストを実行しようとするときに、OS は事前にアプリ A 用のアドレス変換表を CPU に設定します。CPU はこの変換表を使って、アプリ A が mov 命令などでメモリの読み書きをするときにアドレスを変換します。アプリ A が実行しようとした mov 命令に指定されたアドレスを**仮想アドレス**と呼びます（筆者は「事実上のアドレス」と訳す方が良いと思っている、という思いを「**コラム 19.1 事実上のアドレス**」に書きました）。

このようなアドレスを変換することを「アドレスを**マップ**（map）する」とも言います。マップという土地図というような意味もあるのですが、それとはちょっと違います。ある数値を異なる数値に変換すること、またはその対応表自体をマップと呼ぶのです。C++ 標準ライブラリにある `std::map<A,B>`

というクラスもこの意味のマッピングです。型 A の数値を型 B に変換する、という感覚ですね。C の配列もマッピングの 1 つです。添え字という数値を要素の値へと変換するわけですから。まさしくマッピングですね。

仮想アドレスを物理アドレスにマッピングする機能があると、アプリ A とアプリ B、あるいは OS の機械語や変数をおくためのメモリ領域を被らないように物理メモリ上に配置しつつも、それぞれのアプリが実行する機械語には同じメモリアドレスを含むことができるようになります。これは非常に便利で、多くの CPU に同様の機能が搭載されています。

中でも x86 系の CPU はアドレスマッピングのための機能を 2 種類も持っています。セグメンテーションとページングです。悲しいことに x86-64 アーキテクチャの 64 ビットモードではセグメンテーションがほとんど無効化されてしまいましたので、本書ではページングだけを説明します。セグメンテーションが無効になった本当の理由は分かりませんが、きっとその機能がほとんど使われなかったからじゃないかと思います。

ページング (paging) はメモリアドレス空間を固定長の領域 = ページに分け、ページごとにアドレスをマッピングする方式です。x86-64 アーキテクチャの 64 ビットモードではページの大きさは 4KiB、2MiB、1GiB から選ぶことができます (機種によっては 1GiB ページが使えません)。**階層ページング構造**と呼ばれる、ページングの仕組みで用いるアドレスマッピングにおいて、例えば 0x00200000 から始まる 4KiB のページを物理アドレスの 0x00500000 にマッピングしておきます。そうすると、アプリ A が 0x00200000 から始まる 4KiB の領域のどこかを読み書きしようとするアドレスが変換され^{*2}、物理メモリの 0x00500000 から始まる 4KiB の領域のオフセットが同じ場所に対して読み書きが行われます。

アプリ A をしばらく実行した後、タイマ割り込みが発生してアプリ B のコンテキストに切り替わるとします。OS はアプリ B 用の階層ページング構造 (アドレスマッピング) を CPU に設定し直します。この階層ページング構造には仮想アドレスの 0x00200000 から始まる 4KiB ページを物理アドレスの 0x00900000 から始まるページへマッピングしておきます。すると、アプリ B が 0x00200000 から始まる 4KiB の領域のどこかを読み書きしようとする、物理メモリの 0x00900000 から始まる 4KiB の領域のオフセットが同じ場所に対して読み書きが行われます。

このように、アプリ A とアプリ B が同じ仮想アドレスに対して読み書きをしても、ページングにより異なる物理アドレスの読み書きに変換されます。

ここまではアプリを 0x200000 に配置するとして話を進めましたが、本当にそれで良いのでしょうか？ いえ、そうとは限りません。アプリは OS と絶対に重複しない仮想アドレスに配置したいですが、OS が 0x200000 を使わない保証はありません。ではどこに配置するのが良いのでしょうか？ 以降では、MikanOS で採用する 4 階層ページングを前提としてアプリを配置する仮想アドレスを検討します。

*2 実際にアドレス変換を行うハードウェアを MMU (memory management unit) と呼びます。MMU は CPU に内蔵されていて、アドレス変換のほかにはメモリ権限の検査なども担当します。

コラム 19.1 事実上のアドレス

「仮想」は英語の virtual の訳語として一般に使われますが、仮想アドレスの「仮想」に関しては「事実上の」と訳した方が良く筆者は思っています。すなわち、アプリが mov 命令などに指定するメモリアドレスは、そのアプリにとっての「事実上のアドレス」ということです。

virtual のニュアンスは「名目上はそうではないが、事実上は（あるいは実質的には）そうである」という感じです。virtual address は「CPU の外側の世界で使われる『本物の』アドレスではないけれども、アプリにとっては実質的にそれが本物のアドレス」なのです。「仮想」というと想像上の何か、実体がないもの、という印象を受けてしまいます。仮想アドレスは物理アドレスではないのは確かですが、アプリにとっては仮想アドレスこそがアドレスですし、CPU が実際に解釈する実体のある数値であることは確かなのです。

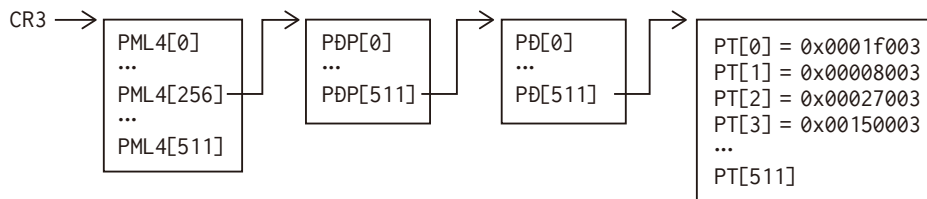
「事実上のアドレス」と書くのと長いのと、一般的な訳語として「仮想アドレス」が使われるのは確かですので、本書では「仮想アドレス」と書きます。が、読者の皆さんはぜひ「仮想アドレス」を上記のような意味で捉えてくださいな。

19.3 アプリのロードと実行 (osbook_day19a)

ページングで実現したいことはなんとなく伝わりましたか？ ページングでは、4KiB や 2MiB などの固定長に区切ったページ単位でアドレス変換を行います。「8.6 ページングの設定」で紹介しましたが、階層ページング構造はその名の通り階層的な構造になっています。CPU は階層ページング構造を使って、仮想アドレスを物理アドレスに変換してからメインメモリを読み書きします。階層の深さはページングのモードによりいくつか種類があります。MikanOS で採用する x86-64 の 64 ビットモードでは 4 階層ページングを使うことになっていますので、以降ではそれを説明します。

19.4 仮想アドレスと 4 階層ページング

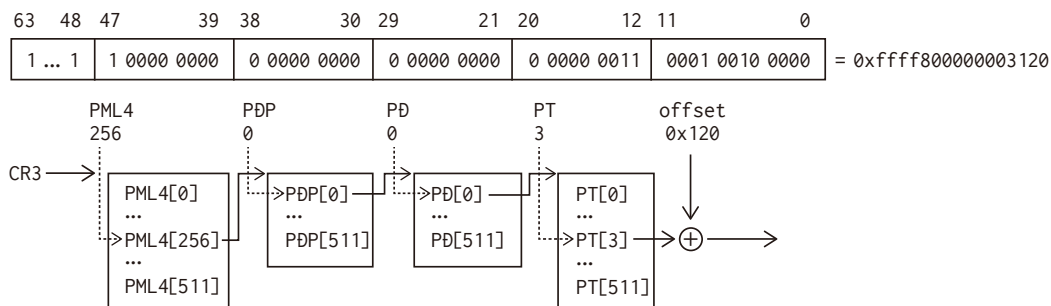
図 19.2 アプリケーション向けの階層ページング構造の例



ある仮想アドレス（アプリにとっての事実上のアドレス）がどの物理アドレスに変換されるかを説明します。前提として、CPU の CR3 レジスタに図 19.2 に示す階層ページング構造を設定してあるとします。CPU が仮想アドレス 0xffff800000003120 にアクセスしようとするとき、どのようなアドレス変換

が行われるのでしょうか？

図 19.3 仮想アドレスの構成



4 階層ページングでは、CPU は仮想アドレスを図 19.3 のように 6 つの部分に分解して扱います。ビット 47:12 は 9 ビットずつに分かれ、それぞれ各階層での配列の添字として用いられます。この配列の要素（例えば PML4[0]）のことを「ページング構造のエントリ」と呼ぶことにします。1 つのエントリは図 19.4 に示すような構造になっています。

図 19.4 ページング構造のエントリの構造



40 ビット幅^{*3}の物理アドレスフィールドには、物理アドレスを右に 12 ビットシフトした値を入れます。すなわち PML4[0] & 0x000fffffffff000ull という計算で得た値がそのまま有効な物理アドレスとなるようにします。この物理アドレスは 1 つ下位の階層ページング構造の先頭を指します。

仮想アドレス 0xffff800000003120 が最終的にどんな物理アドレスに変換されるかは、階層ページング構造の最終段（PT）のエントリに設定された物理アドレスの値で決まります。図 19.2 によると PT[3]=0x150003 ですので、このページエントリが指す物理フレームの先頭は 0x150000 です。ここに仮想アドレスのオフセット 0x120 を足せば物理アドレスが求まります。結局、0xffff800000003120 は 0x150120 に変換されます。

*3 40 ビットのうち何ビットが有効であるかは CPU の機種に依存します。基本的には新しい機種ほど有効なビット幅が増えます。

リスト 19.2 ページング構造のエントリの型 (paging.hpp)

```
union PageMapEntry {
    uint64_t data;

    struct {
        uint64_t present : 1;
        uint64_t writable : 1;
        uint64_t user : 1;
        uint64_t write_through : 1;
        uint64_t cache_disable : 1;
        uint64_t accessed : 1;
        uint64_t dirty : 1;
        uint64_t huge_page : 1;
        uint64_t global : 1;
        uint64_t : 3;

        uint64_t addr : 40;
        uint64_t : 12;
    } __attribute__((packed)) bits;

    PageMapEntry* Pointer() const {
        return reinterpret_cast<PageMapEntry*>(bits.addr << 12);
    }

    void SetPointer(PageMapEntry* p) {
        bits.addr = reinterpret_cast<uint64_t>(p) >> 12;
    }
};
```

ページング構造のエントリの構造をソースコードで表現したものが**リスト 19.2**です。40 ビットの `addr` フィールドを本当のアドレスに変換するために 12 ビットのシフトをしていることが分かりますね。 `present` 等のフラグは必要に応じて説明を加えます。

図 19.3の説明に戻ります。CPU は CR3 レジスタから出発し、4 つの添字を使って階層ページング構造をたどり、最終的にページテーブルの要素 `PT[3]` を得ます。ページテーブルの要素は物理アドレス空間での 4KiB フレームを指しますから、そこにビット 11:0 の値を加えてアクセスすべき物理アドレスを得ます。

ビット 11:0 は 4KiB ページ内のオフセット位置です。ビット 47:12 を用いて決定した 4KiB フレームの先頭アドレスにオフセット値を加えることで、実際に読み書きするメモリアドレスが求められます。

さて、アドレス変換が実際に CPU によってどのように処理されるのか、例を見てみます。次の 2 行のアセンブリ言語プログラムがあったとします。

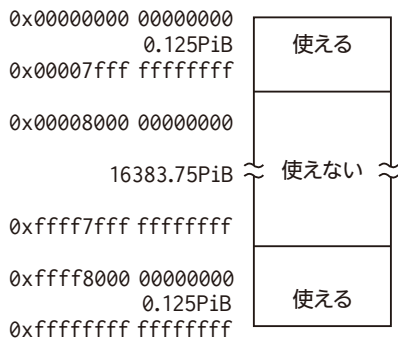
```
mov rbx, 0xc0200042
mov [rbx], rax
```

CPU の気持ちになって、2 行目で発生するアドレス変換の様子を想像してみます。RBX が指す仮想アドレスはどんな物理アドレスに変換されるのでしょうか？

1. `mov rbx, 0xc0200042` は簡単だ。RBX に `0xc0200042` を書き込もう。
2. 次は `mov [rbx], rax` だな。メモリへの書き込みか。RBX の値は仮想アドレスだから、まず物理アドレスに変換しよう。
3. CR3 の値は A だ。PML4 テーブルの場所は A ということが分かったぞ。
4. RBX の 47:39 は 0 か。じゃあ A[0] を読もう。PDP テーブルの場所は B ということが分かったぞ。
5. RBX の 38:30 は 3 か。じゃあ B[3] を読もう。ページディレクトリの場所は C ということが分かったぞ。
6. RBX の 29:21 は 1 か。じゃあ C[1] を読もう。ページテーブルの場所は D ということが分かったぞ。
7. RBX の 20:12 は 0 か。じゃあ D[0] を読もう。アドレス E を得た。
8. RBX の 11:0 は `0x42` か。じゃあ最終的な物理アドレスは `E+0x42` だな。

仮想アドレスのビット 63:48 は 4 階層ページングでは使われません。使われないからといって自由な値で良いわけではありません。63:48 の全ビットをビット 47 と同じ内容にしておかないとメモリ読み書き時にエラー（CPU 例外）が発生します。つまり、ビット 63:48 はすべて 0 になるか、すべて 1 になるかのどちらかです。この制約を満たしたアドレスを**カノニカルアドレス**と呼びます。

図 19.5 プログラマが使える仮想アドレス空間はほんのわずか



ビット 63:47 がすべて 0、またはすべて 1 のアドレスだけが有効なアドレスということは、64 ビットアドレス空間の全体から見ると実際に使えるのは両端のほんのわずかな部分だということです（図 19.5）。使える領域が分断されているため、どちらかの領域に OS を、逆側にアプリを配置するのが自然な発想でしょう。

19.5 アプリケーションを後半で動かす

MikanOS では OS を低位アドレス側で、アプリを高位アドレス側で動かすことにします*⁴。すなわち、アプリは 0xffff800000000000 から 0xffffffffffffff のどこかに配置します。分かりやすさのために 0xffff800000000000 から配置することにしましょう。そのために rpn コマンドをリンクする際に、リンクオプションで `--image-base 0xffff800000000000` と指定することになります（リスト 19.3）。

リスト 19.3 アプリ共通の Makefile

```
CXXFLAGS += -O2 -Wall -g --target=x86_64-elf -ffreestanding -mcmodel=large \
            -fno-exceptions -fno-rtti -std=c++17
LD_FLAGS += --entry main -z norelro --image-base 0xffff800000000000 --static
```

`--image-base` で 32 ビットを超えるアドレスを指定する場合、コンパイル時に `-mcmodel=large` を指定しておかないとリンク時エラーになります。このオプションはコンパイラに対し、関数やグローバル変数などの配置アドレスを 64 ビットとして出力するように指示します。このオプション無しではコンパイラはアドレス幅を 32 ビットと仮定して機械語を出力しますから、0xffff800000000000 のような 32 ビットで収まらないアドレスに対しリンクできません。

以上の設定でコンパイル & リンクした rpn ファイルのプログラムヘッダを確認してみます。readelf -l rpn コマンドの結果を見ると LOAD セグメントは表 19.1 のようになりました。

表 19.1 rpn ファイルの LOAD セグメント一覧

オフセット	仮想 Addr	ファイルサイズ	メモリサイズ	フラグ
0x0000	0xffff800000000000	0x04ec	0x04ec	R
0x1000	0xffff800000001000	0x0460	0x0460	R E
0x2000	0xffff800000002000	0x0768	0x0aa0	RW

仮想 Addr がうまくアドレス空間の後半に配置されたようです。ちなみにエントリポイントは 0xffff800000001060 となっていて、LOAD セグメントの場所とちゃんと対応しているようです。OS はアプリを実行するために、これらの LOAD セグメントを指示どおりに配置しなければなりません。

rpn コマンドをちゃんと動かすためには、大きく次の 2 つの作業が必要です。

- ・ ELF ファイルのプログラムヘッダに基づいて階層ページング構造を設定する
- ・ LOAD セグメントをメモリに読み込む

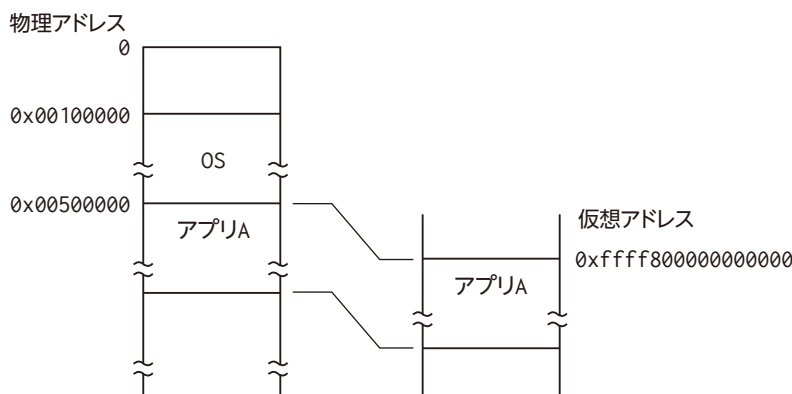
プログラムヘッダや LOAD セグメントはすでに「4.5 ローダを改良する」で説明しました。rpn ファイルに含まれるプログラムヘッダや LOAD セグメントもそれと同類のものです。プログラムヘッダの設

* 4 普及している OS では、OS を後半、アプリを前半で動作させることが多いです。筆者が推測するに、そうする主な理由は、将来的に使える仮想アドレス空間が増えなくてもアプリをビルドし直すことなく広大なメモリ空間が自然に使えるようになるから、あるいは前半に置くほうが効率の良い機械語になるからでしょう。MikanOS では、OS の作りやすさを優先して、OS を前半に置くことにしました。

定に基づいて LOAD セグメントを最終目的地へとコピーします。これらの作業をロードと呼ぶのでしたね。

OS のロード処理とアプリケーションのロード処理で大きく異なるのは、アプリケーションでは LOAD セグメントの仮想 Addr が物理メモリの容量を大きく超えた値（具体的には 0xffff800000000000）になっていることです。「8.3 データ構造の移動」で紹介した SetupIdentityPageTable() により設定されたアイデンティティマッピングでは、先頭 64GiB の仮想アドレス空間、すなわち仮想アドレス 0 から 0xffffffff までが物理アドレス 0 から 0xffffffff までに対応している状態です。そのため、階層ページング構造を変更せずに LOAD セグメントをコピーしようとする、まだマッピングされていないページに対して書き込もうとするため、ページフォルトという例外が発生してしまいます。

図 19.6 アプリケーションのアドレスマッピング例



19.6 アプリケーションのロード

そこで、LOAD セグメントの最終目的地へのコピーに先立ち、必要なページを設定する必要があります。具体的には仮想アドレス 0xffff800000000000 から始まる仮想アドレス範囲を、適当な物理アドレス範囲に対応するように階層ページング構造を設定します（図 19.6）。このとき、仮想アドレス空間ではページが連続している必要がありますが、物理アドレス空間ではフレーム単位で不連続でも構いません。

リスト 19.4 LoadELF() は ELF ファイル中の LOAD セグメントを適切に配置する

```
Error LoadELF(Elf64_Ehdr* ehdr) {
    if (ehdr->e_type != ET_EXEC) {
        return MAKE_ERROR(Error::kInvalidFormat);
    }

    const auto addr_first = GetFirstLoadAddress(ehdr);
    if (addr_first < 0xffff'8000'0000'0000) {
        return MAKE_ERROR(Error::kInvalidFormat);
    }

    if (auto err = CopyLoadSegments(ehdr)) {
```

```

    return err;
}

return MAKE_ERROR(Error::kSuccess);
}

```

リスト 19.4 に、アプリのロードを行う関数 `LoadELF()` の実装を示します。この関数は階層ページング構造の設定と `LOAD` セグメントのコピーの両方を行い、アプリが動作できる状態にします。この関数はまず、与えられた `ELF` ファイルが実行可能ファイルであることを確かめます（最初の `if` 文）。次に 1 つ目の `LOAD` セグメントの仮想アドレスがカノニカルアドレスの後半領域になっていることを確かめます（2 つ目の `if` 文）。C++ では数値リテラルの途中にシングルクォートを挿入することができますから、64 ビットアドレスを 4 桁ずつに区切ってみました。多少見やすいかな？

2 つの検査に合格したらいよいよロード処理を実行します。ロード処理の本体は `CopyLoadSegments()` という関数に切り出してありますから、そちらの実装を見てみます。

リスト 19.5 `CopyLoadSegments()` は `LOAD` セグメントを最終目的地にコピーする (`terminal.cpp`)

```

Error CopyLoadSegments(Elf64_Ehdr* ehdr) {
    auto phdr = GetProgramHeader(ehdr);
    for (int i = 0; i < ehdr->e_phnum; ++i) {
        if (phdr[i].p_type != PT_LOAD) continue;

        LinearAddress4Level dest_addr;
        dest_addr.value = phdr[i].p_vaddr;
        const auto num_4kpages = (phdr[i].p_memsz + 4095) / 4096;

        if (auto err = SetupPageMaps(dest_addr, num_4kpages)) {
            return err;
        }

        const auto src = reinterpret_cast<uint8_t*>(ehdr) + phdr[i].p_offset;
        const auto dst = reinterpret_cast<uint8_t*>(phdr[i].p_vaddr);
        memcpy(dst, src, phdr[i].p_filesz);
        memset(dst + phdr[i].p_filesz, 0, phdr[i].p_memsz - phdr[i].p_filesz);
    }
    return MAKE_ERROR(Error::kSuccess);
}

```

リスト 19.5 に `CopyLoadSegments()` の実装を示します。OS 本体をロードするために同名の関数を「**4.5 ロードを改良する**」で実装していて、両者とも `LOAD` セグメントを最終目的地へとコピーするという目的も一緒です。今回の実装では、`LOAD` セグメントのコピーに先立ち、階層ページング構造の設定をしているのが主な違いです。

19.7 階層ページング構造の設定

階層ページング構造の設定は `SetupPageMaps()` に切り出しました。この関数の引数には、LOAD セグメントを配置する先頭アドレス (`dest_addr`) と 4KiB ページ単位のセグメントの大きさ (`num_4kpages`) を渡します。指定した先頭アドレスから指定したページ数分だけ、階層ページング構造を設定します。

リスト 19.6 `SetupPageMaps()` は階層ページング構造全体を設定する (`terminal.cpp`)

```
Error SetupPageMaps(LinearAddress4Level addr, size_t num_4kpages) {
    auto pml4_table = reinterpret_cast<PageMapEntry*>(GetCR3());
    return SetupPageMap(pml4_table, 4, addr, num_4kpages).error;
}
```

`SetupPageMaps()` の実装を **リスト 19.6** に示します。処理はとても短いですね。CR3 レジスタには階層ページング構造の最も上位の構造である PML4 の物理アドレスが格納されています。PML4 の物理アドレスを取得したら、具体的な処理を `SetupPageMap()` に委譲します。

リスト 19.7 `SetupPageMap()` は指定された階層の設定をする (`terminal.cpp`)

```
WithError<size_t> SetupPageMap(
    PageMapEntry* page_map, int page_map_level, LinearAddress4Level addr, size_t
    num_4kpages) {
    while (num_4kpages > 0) {
        const auto entry_index = addr.Part(page_map_level);

        auto [ child_map, err ] = SetNewPageMapIfNotPresent(page_map[entry_index]);
        if (err) {
            return { num_4kpages, err };
        }
        page_map[entry_index].bits.writable = 1;

        if (page_map_level == 1) {
            --num_4kpages;
        } else {
            auto [ num_remain_pages, err ] =
                SetupPageMap(child_map, page_map_level - 1, addr, num_4kpages);
            if (err) {
                return { num_4kpages, err };
            }
            num_4kpages = num_remain_pages;
        }

        if (entry_index == 511) {
            break;
        }

        addr.SetPart(page_map_level, entry_index + 1);
        for (int level = page_map_level - 1; level >= 1; --level) {
            addr.SetPart(level, 0);
        }
    }
}
```

```

}

return { num_4kpages, MAKE_ERROR(Error::kSuccess) };
}

```

階層ページング構造を設定する中心的な役割の `SetupPageMap()` はリスト 19.7 のように実装しました。全体構成としては**再帰呼び出し**により実装してあります。

階層ページング構造には各階層の構造がほとんど同じという特徴があります。各階層のデータ構造は 64 ビットのエン트리 (物理アドレスとフラグ) が 512 個並ぶ配列です。すなわち配列の大きさは 64 ビット × 512 = 4KiB です。PML4、PDP、PD では、エントリーはいずれも 1 つ下位のページング構造を指すものとなっていて、処理が共通化できます。最下層の構造である PT は、エントリーが指す先がページング構造ではなく単なる物理メモリ領域である点が違いますが、512 個の 64 ビット値が並んでいるという構造は共通しています。

`SetupPageMap()` は次の 4 引数を取ります。階層を示す値 (`page_map_level`: 4=PML4、1=PT)、その階層のページング構造 (`page_map`)、設定対象の仮想アドレス領域の先頭 `addr`、仮想アドレス領域の大きさ `num_4kpages`。この関数は、指定された階層において、`addr` と `num_4kpages` で指定された仮想アドレス領域をカバーするのに十分な範囲にページを割り当てます。

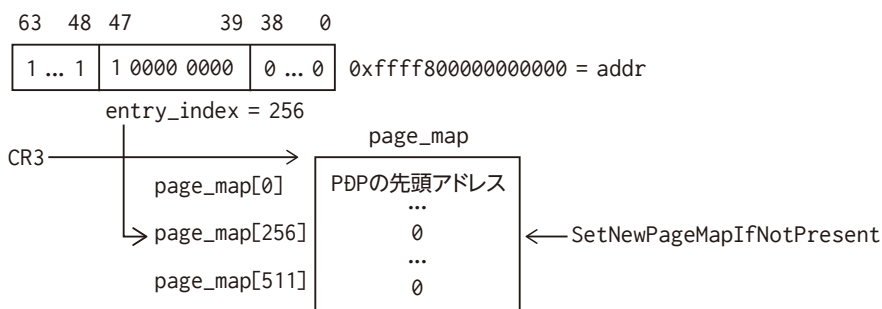
`rpn` コマンドの LOAD セグメントのうち、先頭の LOAD セグメントをロードする場合を例に説明します。`SetupPageMaps()` が `SetupPageMap()` を呼び出すとき、および再帰呼び出しされる際の引数の値は表 19.2 のようになります。

表 19.2 `SetupPageMap()` の引数の例

	page_map	page_map_level	addr	num_4kpages
1 回目	CR3 の値	4 (=PML4)	0xffff800000000000	1
2 回目	0x1000	3 (=PDP)	0xffff800000000000	1
3 回目	0x2000	2 (=PD)	0xffff800000000000	1
4 回目	0x3000	1 (=PT)	0xffff800000000000	1

1 回目の呼び出しについて詳しく処理を追ってみます。`SetupPageMap()` は、呼び出されるとすぐに `while` ループに突入します。ループの先頭では `addr.Part(page_map_level)` により、仮想アドレスの指定した階層の値を取り出します。1 回目の呼び出しでは `page_map_level` が 4 ですので、PML4 の 9 ビット値を取り出すことになります。すなわち `entry_index` は 256 となります (図 19.7)。紙面の都合で `addr` の型である `LinearAddress4Level` の定義は紹介しません。簡単な定義ですからソースコードを参照してください。

図 19.7 addr と entry_index の関係



entry_index の取得が終わったら SetNewPageMapIfNotPresent(page_map[entry_index]) という処理を行います。この関数は新たなページング構造を生成してエントリに設定する関数です。中身を見てみます。

リスト 19.8 SetNewPageMapIfNotPresent() は必要に応じて新たなページング構造を生成して設定する (terminal.cpp)

```
WithError<PageMapEntry*> SetNewPageMapIfNotPresent(PageMapEntry& entry) {
    if (entry.bits.present) {
        return { entry.Pointer(), MAKE_ERROR(Error::kSuccess) };
    }

    auto [ child_map, err ] = NewPageMap();
    if (err) {
        return { nullptr, err };
    }

    entry.SetPointer(child_map);
    entry.bits.present = 1;

    return { child_map, MAKE_ERROR(Error::kSuccess) };
}
```

SetNewPageMapIfNotPresent() の実装をリスト 19.8 に示します。この関数は、引数にページング構造のエントリを取ります。エントリの present フラグが 1 であれば、このエントリはすでに有効な値が設定済みだと認識して何もせず終了します。フラグが 0 なら、新しいページング構造を生成 (NewPageMap()) してエントリの addr フィールドに設定 (SetPointer()) します。SetupPageMap() の 1 回目の呼び出しでは page_map[256] の present フラグは 0 となっているはずですから、この生成処理が実行されるはずですが。

present フラグはエントリの有効、無効を表します。フラグが 0 であればエントリは無効であり、有効な物理アドレスが設定されていないことを意味します。無効なエントリが表す仮想アドレス範囲に対して読み書きしようすると、CPU はページフォルトを発生させます。例えば PML4[256] の present フラグが 0 のときに 0xfffff80000000000 から ffff807ffffffffff の範囲に対して mov 命令を実行するとページフォルトが発生します。

さて、SetupPageMap() の while ループの説明に戻ります。今見たように、

SetNewPageMapIfNotPresent(page_map[entry_index]) の処理が終わった段階で、page_map[entry_index] の addr フィールドには1つ下位のページング構造を指すアドレスが設定された状態になっています。この関数の戻り値を受け取っている部分が auto [...] という見慣れない形になっています。これは構造化束縛という C++ の機能で、戻り値が構造体の場合に各フィールドを個別の変数で受け取れます。詳しくは「[コラム 19.2](#)」に書きました。

次に page_map[entry_index].bits.writable = 1; という処理により、writable フラグを 1 にします。このフラグは名前の通り、エントリが表す仮想アドレス範囲への書き込みを許可するためのものです。

次に page_map_level の値で場合分けします。最初は page_map_level が 4 ですから else 節が実行されます。else 節では、先ほど生成した1つ下位のページング構造 child_map の設定を行います。呼び出している関数は SetupPageMap() です。このように自分自身を呼び出すことを再帰呼び出しと言います。

再帰呼び出しの引数を詳しく見てみます。SetupPageMap(child_map, page_map_level - 1, addr, num_4kpages) となっています。第1引数 child_map は while ループの最初の方で生成した、1つ下位のページング構造です。第2引数 page_map_level - 1 は1つ下位の階層レベルを表します。2つの引数はいずれも、1つ下位の階層を表す値になっているというのが共通していますね。

再帰呼び出しするプログラムを書く際は再帰関数の引数と戻り値の意味を決めてしまうのがポイントです。引数は決まったので戻り値を考えます。そうですね、SetupPageMap() は未処理ページの数进行を返すことにします。引数 num_4kpages で処理すべきページ数を渡されますが、1度にすべてのページをマップできるとは限りません。page_map_level が1のときは1ページずつしかマップできませんし、page_map_level が2のときは、最大で 512 ページしかマップできません。そのため、引数 num_4kpages からマップできたページ数を引いた数を戻り値とするのが良さそうです。

```
addr.SetPart(page_map_level, entry_index + 1);
for (int level = page_map_level - 1; level >= 1; --level) {
    addr.SetPart(level, 0);
}
```

while ループの最後の処理を再掲しました。1行目では仮想アドレスの中の、page_map_level に相当するフィールドに entry_index + 1 を設定しています。結果的に、次のループの開始時に取得する entry_index が1だけインクリメントされた状態になり、ページング構造のエントリを次々に設定することになります。続く for 文では下位のフィールドにすべて 0 を設定しています。これは要するに桁の繰り上がり処理ですね。

ということで SetupPageMap() のほとんどを説明しました。再帰処理のときに決めたように、SetupPageMap() は最後に未処理ページの数进行を返す必要があります。num_4kpages は未処理ページの数进行を保持していますので、それを返して処理完了です。

さて、話を CopyLoadSegments() に戻します。この関数は LOAD セグメントを適切な位置へ配置するのでした。プログラムヘッダを1つずつ見て、種別が LOAD セグメントであれば p_vaddr で指定された仮想アドレスに対して適切に階層ページング構造を設定し、そこへ LOAD セグメントの中身をコピーします。先ほど階層ページング構造を設定する関数 SetupPageMaps() を説明しました。残るは LOAD セグメントの中身のコピー処理ですが、[リスト 4.17](#) とほぼ同じ処理ですので説明は省略します。

19.8 階層ページング構造の片付け

ここまででアプリケーションを起動するために必要な階層ページング構造の設定と LOAD セグメントのコピーを説明しました。アプリケーションはいつかは終了しますので、終了時の片付けも行う必要があります。生成したページング構造や LOAD セグメントをコピーするための物理メモリ領域を解放するというのが主な後片付けです。

リスト 19.9 CleanPageMaps() はアプリ用のページング構造を破棄する (terminal.cpp)

```
Error CleanPageMaps(LinearAddress4Level addr) {
    auto pml4_table = reinterpret_cast<PageMapEntry*>(GetCR3());
    auto pdp_table = pml4_table[addr.parts.pml4].Pointer();
    pml4_table[addr.parts.pml4].data = 0;
    if (auto err = CleanPageMap(pdp_table, 3)) {
        return err;
    }

    const auto pdp_addr = reinterpret_cast<uintptr_t>(pdp_table);
    const FrameID pdp_frame{pdp_addr / kBytesPerFrame};
    return memory_manager->Free(pdp_frame, 1);
}
```

リスト 19.9 に後片付け用の関数 CleanPageMaps() の実装を示します。この関数は SetupPageMaps() により確保された 4KiB ページをすべて解放するのが仕事です。実装を簡単にするために PML4 テーブルには 1 つしかエントリがないことを仮定しています。PML4 テーブルで複数のエントリが必要なほどの大きなメモリを必要とするアプリは作りませんので問題ありません。

リスト 19.10 CleanPageMap() は指定されたページング構造内のエントリをすべて削除する (terminal.cpp)

```
Error CleanPageMap(PageMapEntry* page_map, int page_map_level) {
    for (int i = 0; i < 512; ++i) {
        auto entry = page_map[i];
        if (!entry.bits.present) {
            continue;
        }

        if (page_map_level > 1) {
            if (auto err = CleanPageMap(entry.Pointer(), page_map_level - 1)) {
                return err;
            }
        }

        const auto entry_addr = reinterpret_cast<uintptr_t>(entry.Pointer());
        const FrameID map_frame{entry_addr / kBytesPerFrame};
        if (auto err = memory_manager->Free(map_frame, 1)) {
            return err;
        }
        page_map[i].data = 0;
    }
}
```

```
    return MAKE_ERROR(Error::kSuccess);
}
```

リスト 19.10 にページング構造のエントリをすべて削除するための `CleanPageMap()` を示します。`present` ビットが立っている = 有効なエントリを問答無用で削除します。もちろん、再帰的に削除しなければなりませんので `CleanPageMap()` を再帰的に呼び出します。

さて、ここまでで階層ページング構造の設定をするための主要な関数群を説明し終わりました。長い旅でした。後少し、説明しておくべきプログラムが残っています。

リスト 19.11 `LoadFile()` はファイル内容をバッファに読み込む (`fat.cpp`)

```
size_t LoadFile(void* buf, size_t len, const DirectoryEntry& entry) {
    auto is_valid_cluster = [](uint32_t c) {
        return c != 0 && c != fat::kEndOfClusterchain;
    };
    auto cluster = entry.FirstCluster();

    const auto buf_uint8 = reinterpret_cast<uint8_t*>(buf);
    const auto buf_end = buf_uint8 + len;
    auto p = buf_uint8;

    while (is_valid_cluster(cluster)) {
        if (bytes_per_cluster >= buf_end - p) {
            memcpy(p, GetSectorByCluster<uint8_t>(cluster), buf_end - p);
            return len;
        }
        memcpy(p, GetSectorByCluster<uint8_t>(cluster), bytes_per_cluster);
        p += bytes_per_cluster;
        cluster = NextCluster(cluster);
    }
    return p - buf_uint8;
}
```

クラスタチェーンをたどってファイルを読み込む処理はいろいろな場所で必要となります。そのたびにクラスタチェーンを読み取る `while` ループを書くのは無駄なので関数化しました (**リスト 19.11**)。 `LoadFile()` はディレクトリエントリによって指定されたファイルを、指定されたバッファ `buf` に読み込みます。バッファの大きさ (バイト単位) を `len` に指定します。以前出てきたコードとほとんど同じですから内部的な動作の説明は省略します。

リスト 19.12 アプリを読み込んで実行するまでの流れ (terminal.cpp)

```

auto argv = MakeArgVector(command, first_arg);
if (auto err = LoadELF(elf_header)) {
    return err;
}

auto entry_addr = elf_header->e_entry;
using Func = int (int, char**);
auto f = reinterpret_cast<Func*>(entry_addr);
auto ret = f(argv.size(), &argv[0]);

char s[64];
sprintf(s, "app exited. ret = %d\n", ret);
Print(s);

const auto addr_first = GetFirstLoadAddress(elf_header);
if (auto err = CleanPageMaps(LinearAddress4Level{addr_first})) {
    return err;
}

```

アプリを読み込んで実行する処理をリスト 19.12 のように実装しました。先ほど作った LoadELF() を使ってアプリの LOAD セグメントを適切な仮想アドレスへ読み込んだ後、ELF ヘッダからエントリポイントのアドレスを取ってきてジャンプします。以前はエントリポイントのアドレス entry_addr を実際のメモリアドレスに変換するために値を調整していましたが、今はページングがきちんと設定できているため調整不要です。アプリの実行が終わったら終了コードを表示し、階層ページング構造を削除して処理完了です。

図 19.8 標準ライブラリを使うように改造した rpn コマンド (再挑戦)



今度こそ、ちゃんと正しい計算結果が表示されています！(図 19.8) アプリケーションがきちんと仮想アドレスの後半領域で動いている証拠です。ページング作戦、大成功です！

コラム 19.2 構造化束縛

関数の戻り値が構造体（クラス）である場合、そのメンバを個別の変数で受け取ることができる機能です。本文中では次のように使っています。

```
auto [ child_map, err ] = SetNewPageMapIfNotPresent(page_map[entry_index]);
```

`SetNewPageMapIfNotPresent()` の戻り値は `WithError<PageMapEntry*>` という、2つのメンバを持つ構造体です。上記のように書くと、先頭のメンバが `child_map` に、2つ目のメンバが `err` に束縛されます。これが構造化束縛です。構造化束縛をうまく使って読みやすいプログラムを目指しましょう！

構造化束縛では変数の型を必ず `auto` とする必要があります。個々の変数に対して個別に型を指定することはできません。また、使わないメンバもすべて受け取る必要があり、他のプログラミング言語のように `_` を使って読み捨てる機能はありません。次のように書けばうまく読み捨てられるように思うかもしれません。

```
auto [ child_map, _ ] = SetNewPageMapIfNotPresent(page_map[entry_index]);
```

しかし、これは単に `_` という名前の変数で値を受け取っているだけです。そう、C++ では `_` も識別子として有効なのです（`_` で始まる識別子は予約名なので本番では避けましょう）。

第20章

システムコール

アプリケーションの入出力の方法が、今のところコマンドライン引数と終了コードしかありません。他の OS だったらアプリから文字列を出力したり絵を描いたりできるのに。この章では MikanOS にシステムコールを搭載し、アプリが OS に対しいろいろな命令できるようにしてみます。この章が終わる頃にはアプリからターミナルに文字列を表示できるようになります。

20.1 アプリが OS の機能を使う方法 (osbook_day20a)

rpn コマンドが結果を出力する方法が main 関数の戻り値しかないのは寂しいです。普通のプログラミング環境なら `printf()` みたいな関数で文字列を出力できますし、もう少しリッチな処理系であれば簡単に絵を描けたりもします。MikanOS のアプリケーションでもそういうことをやりたいです。

よく考えると、OS 本体では文字列も絵も画面に描いているわけです。アプリケーションからそれらの機能呼び出すことができれば、結構簡単に実現できるのではないのでしょうか？ 使いたい OS の機能が関数なのであれば、その関数が置いてあるアドレスが分かれば呼び出せるはずです。OS に含まれる関数もアプリ内の関数と同様、メモリ上に置かれているのですから。

関数が配置される仮想アドレスを調べるのは簡単です。nm という Linux のコマンドを使うとファイル中のシンボル名とアドレスの一覧を得られます。そこから目的のシンボルを探せば良いです。試しに `printk` のアドレスを探してみます。

```
$ nm -C kernel/kernel.elf | grep printk
00000000010b000 T printk(char const*, ...)
```

`printk` の仮想アドレスが `0x10b000` であることが分かりました。お手軽ですね。nm に与えている `-C` オプションはシンボルをデマングル (demangle) して表示するためのものです。C++ の関数シンボルは引数の型が名前の一部となるようにマングル (mangle) されていますので、デマングルすることで読みやすい表示になります。

リスト 20.1 必要なシンボルのアドレスを定義する (rpn/rpn.cpp)

```
#include <cstring>
#include <cstdlib>
#include "../kernel/graphics.hpp"

auto& printk = *reinterpret_cast<int (*)>(const char*, ...)>(0x00000000010b000);
auto& fill_rect = *reinterpret_cast<decltype(FillRectangle)*>(0x00000000010c1c0);
auto& scrn_writer = *reinterpret_cast<decltype(screen_writer)*>(0x00000000024d078);
```

そのようにして取得したアドレスを用いて、**リスト 20.1** のようにいくつかの関数およびグローバル変数への参照を定義しました。`reinterpret_cast<型*>(アドレス)` を使うと、任意の整数をポインタ型に変換することができます。

「型」の部分に指定している `decltype(...)` がよく分からないかもしれません。`decltype(式)` は「式」の型を得ることができます。すなわち `decltype(FillRectangle)` は `FillRectangle()` 関数の型、`decltype(screen_writer)` は `screen_writer` 変数の型となります。`decltype` を使わずに型を書き下しても良いのですが、特に `FillRectangle()` の型は長いので手で書きたくありませんでした。

リスト 20.2 OS の機能を直接使って文字列や絵を描く (rpn/rpn.cpp)

```
extern "C" int main(int argc, char** argv) {
    stack_ptr = -1;

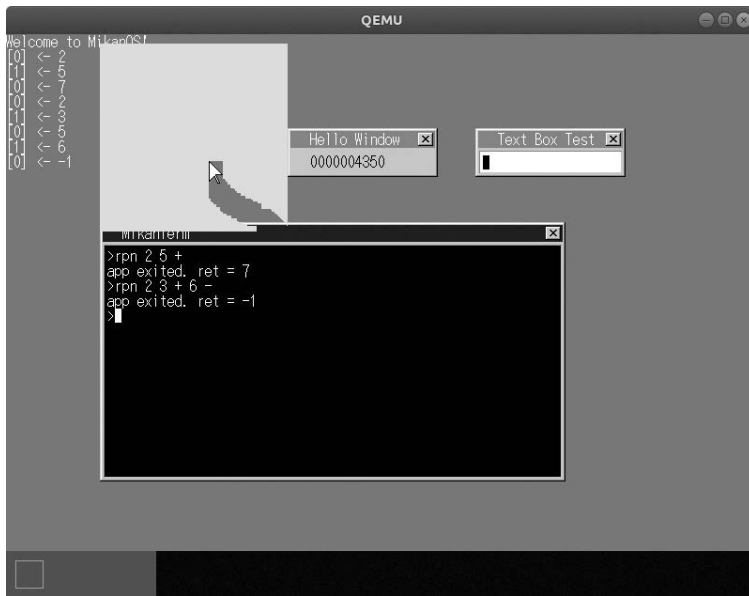
    for (int i = 1; i < argc; ++i) {
        if (strcmp(argv[i], "+") == 0) {
            long b = Pop();
            long a = Pop();
            Push(a + b);
            printk("[%d] <- %ld\n", stack_ptr, a + b);
        } else if (strcmp(argv[i], "-") == 0) {
            long b = Pop();
            long a = Pop();
            Push(a - b);
            printk("[%d] <- %ld\n", stack_ptr, a - b);
        } else {
            long a = atol(argv[i]);
            Push(a);
            printk("[%d] <- %ld\n", stack_ptr, a);
        }
    }

    fill_rect(*scrn_writer, Vector2D<int>{100, 10}, Vector2D<int>{200, 200},
    ToColor(0x00ff00));

    if (stack_ptr < 0) {
        return 0;
    }
    return static_cast<int>(Pop());
}
```

定義したシンボル群を使って文字列や絵を描くコードを書き足したのがリスト 20.2 です。計算にともなうスタックの変化をコンソールに表示し、最後に適当な緑色の四角を描いてみることにしました。さて、これをビルドして実行してみます (図 20.1)。

図 20.1 rpnn コマンドが OS の機能を使って描画する様子



計算は今まで通り正しく実行されているようです。加えてコンソールにスタックの変化が表示されたり、緑の四角が登場したりしました。成功です。このようにすれば、今まで作ってきた OS の関数を自由に呼んで、様々なアプリを作れるようになります！

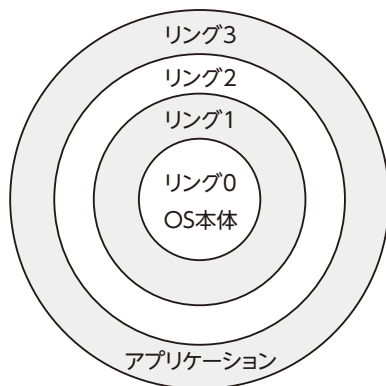
20.2 OSを守ろう(1) (osbook_day20b)

ちょっと考えてみると、アプリから OS の関数を自由に呼び出せるのはセキュリティ的にはまったく好ましくありません。関数が呼べるだけではなく、任意のメモリ領域を読み書きすることもできてしまいます。悪意を持った人が作った攻撃用アプリを起動させてしまったらシステム全体を破壊されたり、重要なデータを盗まれたりしてしまいます。自分が作ったアプリだけを使うなら良いかなと思うかもしれませんが、アプリのバグが OS に悪影響を与える可能性があります。OS が不安定になりシステムが強制終了してしまうと、原因を探ることもままならず、デバッグが非常に困難です。

ということでアプリから OS の関数や変数を使えなくします。具体的には x86-64 に備わる保護の仕組みを使って、アプリをユーザーモードで動作させることにします。仕組みの中心的な役割を果たすのが、コードセグメントに設定する DPL です。DPL は「7.3 割り込みベクタ」で出てきた言葉で、ここでは割り込みハンドラの実行権限を設定するために DPL を使う、という説明をしました。実は DPL は割り込みハンドラ以外にも x86-64 のいろいろな場所に登場する、実行権限を決める大事な概念なのです。

x86-64 アーキテクチャには 0 から 3 の 4 つの権限レベルがあります。0 が最も権限が高く、一般的に OS 本体（カーネル）が動作するレベルとして使われます。3 が最低の権限で、アプリケーションを動かすのに使うことが多いです。図 20.2 のように同心円（リング）状になっていると考えることができ、権限レベル x のことをリング x と呼ぶことがあります。

図 20.2 保護のリング



リング0ではすべての命令が実行できますが、リング3では例えばhlt命令は実行できません。hltが実行できてしまうとアプリが勝手にCPUを止めることができてしまいます。CPUを休ませるか、ほかにやるべき処理に割り当てるかはOSが管理することですので、アプリが勝手に止めてはいけません。4つの権限レベルのうち2つだけ使う場合はリング0と3を使いなさい、とIntel SDMに書いてありますので、MikanOSでもリング0でOS本体を、リング3でアプリを動かすことにします。

リスト 20.3 アプリ用のセグメントディスクリプタを作成する (segment.cpp)

```
void SetDataSegment(SegmentDescriptor& desc,
                    DescriptorType type,
                    unsigned int descriptor_privilege_level,
                    uint32_t base,
                    uint32_t limit) {
    SetCodeSegment(desc, type, descriptor_privilege_level, base, limit);
    desc.bits.long_mode = 0;
    desc.bits.default_operation_size = 1; // 32-bit stack segment
}

void SetupSegments() {
    gdt[0].data = 0;
    SetCodeSegment(gdt[1], DescriptorType::kExecuteRead, 0, 0, 0xffffffff);
    SetDataSegment(gdt[2], DescriptorType::kReadWrite, 0, 0, 0xffffffff);
    SetCodeSegment(gdt[3], DescriptorType::kExecuteRead, 3, 0, 0xffffffff);
    SetDataSegment(gdt[4], DescriptorType::kReadWrite, 3, 0, 0xffffffff);
    LoadGDT(sizeof(gdt) - 1, reinterpret_cast<uintptr_t>(&gdt[0]));
}
```

アプリをリング3で動かすためには、何はともあれアプリ用のセグメントディスクリプタを作成する必要があります。リスト 20.3 に示すように DPL=3 を持つコードセグメント (gdt[3]) とデータセグメント (gdt[4]) を増やしました。この2つのセグメントを使ってアプリを動かすことで、アプリが低い権限レベルで動作するようになります。

2つのセグメントを使ってアプリを動かす、とはどういうことでしょうか？ これはずばり、CS レジスタと SS レジスタがそれぞれのセグメントを指すことです。セグメントレジスタは図 20.3 に示す構造になっています。この中の Index フィールドに GDT の添字を設定することで、セグメントレジス

タがそのセグメントを指す状態になります。

図 20.3 セグメントレジスタの構造

15	3	2	1	0
Index	TI	RPL		

CS の Index に 3 を設定すると、CS がアプリ用コードセグメント gdt[3] を指す状態になります。同様に SS の Index に 4 を設定すると、SS がアプリ用データセグメント gdt[4] を指します。これは「**第 8 章 メモリ管理**」で作成した SetCSSS() と同じ処理でできそうです。

リスト 20.4 CallApp() は指定されたアプリを指定された環境で呼び出す (asmfunc.asm)

```
global CallApp
CallApp:  ; void CallApp(int argc, char** argv, uint16_t cs, uint16_t ss, uint64_t rip,
uint64_t rsp);
    push rbp
    mov rbp, rsp
    push rcx ; SS
    push r9  ; RSP
    push rdx ; CS
    push r8  ; RIP
    o64 retf
    ; アプリケーションが終了してもここには来ない
```

ということで、CS と SS をアプリ用のセグメントを指すようにしてアプリを起動する関数 CallApp() を実装してみました（**リスト 20.4**）。SetCSSS() と処理は似ているのですが、明らかに違うのが SS の設定方法です。mov 命令で設定するのではなくスタックに値を置いています。これはどういうことでしょうか？

実は、DPL が今より大きい（権限が今より低い）コードセグメントへの far return の場合、CS と RIP だけでなく、SS と RSP もスタックから取り出されることになっているのです。その機能を使って CS と SS を同時に設定してしまおうということです。RIP と RSP としてアプリのエントリポイントのアドレスとアプリ用のスタック領域のアドレスを設定しておくことで OS からアプリへと実行が移るはずで

図 20.3 に示すように、セグメントレジスタには RPL (Request Privilege Level) フィールドがあります。このフィールドは多くの場合、Index が指すセグメントの DPL と同じ値にする必要があります。ですので、アプリ用コードおよびデータセグメント（DPL=3）を Index に設定するときは RPL=3 とします。

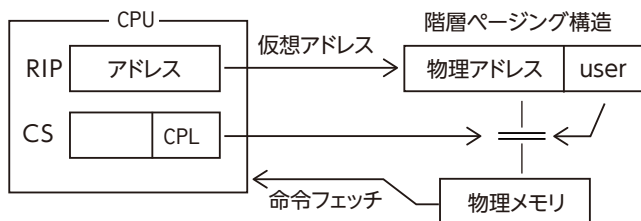
CS の RPL フィールドは特別に **CPL** (Current Privilege Level) と呼びます。Current という名が示す通り、CPL は CPU の現在の動作権限レベルを意味します。CPU は CS レジスタが更新されると、その直後から設定された CPL で動作します。すなわち far return 命令による CS の書き換えは、CPU の動作モードを変えるマーカーのような働きをします。すなわち、CS を書き換えることで「今から OS モードで処理をします！」「ここからはアプリモードになるので特権命令は実行できません！」みたいに、パッと切り替えることができるのです。

ちなみに、CPL が 0 より大きな数値のときにソフトウェア割り込みを発生させたらどうなるでしょうか？ 割り込み発生時、CPL は割り込み記述子に書かれた DPL と比較され、 $CPL \leq DPL$ である場合に

割り込みが正常に実行できます。MikanOS では DPL=0 を設定していますので $CPL \leq DPL$ は成り立ちません。すると一般保護例外が発生します。これは、低い権限（数字的に大きな CPL）で動くアプリから、高い権限（数字的に小さい DPL）の割り込みハンドラを起動できないようにするためです。

話を戻します。アプリ用のコードセグメントとデータセグメントを用意し、それらに切り替えてアプリを起動する関数（CallApp）を実装できました。アプリを DPL=3 で動かすために、次にすべきことは階層ページング構造の設定変更です。**リスト 19.2** に示したページング構造のエントリを見てみると user というフラグがありますね。これが重要です。

図 20.4 命令フェッチ



CPU は RIP で示された仮想アドレスから命令を読み込んで動作します。これを命令フェッチと呼び、**図 20.4** のようなイメージです。命令を読み込みもうとするとときに RIP に対応するページの user フラグがチェックされます。user フラグが 0 の場合、CPU が $CPL < 3$ で動作中のときだけメモリアクセスが許可されます。user フラグが 1 の場合、CPL がどんな値でもメモリアクセスが許可されます。今のところ、アプリを配置した $0xffff800000000000$ 以降の仮想アドレスに対応するエントリでは user フラグが 0 になっていますので、CS をアプリ用コードセグメントに切り替えて $CPL=3$ になった途端、命令フェッチが禁止されてページフォルトが発生してしまいます。

リスト 20.5 アプリを配置したページの user ビットを 1 にする (terminal.cpp)

```
auto [ child_map, err ] = SetNewPageMapIfNotPresent(page_map[entry_index]);
if (err) {
    return { num_4kpages, err };
}
page_map[entry_index].bits.writable = 1;
page_map[entry_index].bits.user = 1;
```

リスト 20.5 に示すように、アプリの ELF ファイルを配置したページの user ビットを 1 にするようにしました。こうしておけば $CPL=3$ になった後も命令フェッチが可能になります。OS 本体が配置されているページ（64 ビットアドレス空間の前半部分）の user ビットは変更していませんので、アプリが OS 領域を触ろうとするとページフォルトが発生します。実は、これこそがやりたかった OS の保護です。OS 領域の user ビットを 0、アプリ領域の user ビットを 1 にしてから $CPL=3$ に切り替えることで、アプリが OS の関数や変数を読み書きできないように保護することができました。

リスト 20.6 専用のスタックを指定してアプリを起動する (terminal.cpp)

```

LinearAddress4Level stack_frame_addr{0xffff'ffff'ffff'e000};
if (auto err = SetupPageMaps(stack_frame_addr, 1)) {
    return err;
}

auto entry_addr = elf_header->e_entry;
CallApp(argc.value, argv, 3 << 3 | 3, 4 << 3 | 3, entry_addr,
        stack_frame_addr.value + 4096 - 8);

/*
char s[64];
sprintf(s, "app exited. ret = %d\n", ret);
Print(s);
*/

```

リスト 20.6 にアプリの起動処理を示します。今までは `auto ret = f(argv.size(), &argv[0]);` というように、アプリのエントリポイントを普通の関数と同じ方法で呼び出していました。これからは、アプリを DPL=3 のセグメントで動かすために `CallApp()` を用いた起動になります。

`CallApp()` は引数を 6 つ取ります。コマンドライン引数の個数と文字列 (`argc` と `argv`)、アプリ用のコードおよびスタックセグメントのセレクタ値、それから RIP と RSP の初期値。第 3 引数の `3 << 3 | 3` は同じ数字が連続していて分かりにくいかもしれませんが。これはまず `3 << 3` が計算されて `0b11000` となり、その結果に `3` が OR されて `0b11011` となります。セグメントレジスタの Index フィールドに合わせるためにビットシフト (`<<`) を使い、RPL に値を設定するためにビット OR (`|`) を使っているのです。

`CallApp()` の実装 (**リスト 20.4**) をよく見ると RDI と RSI を使っていないように見えます。この 2 つのレジスタには第 1、第 2 引数 (`argc` と `argv`) が渡されるはずですが。これらを無視して良いのでしょうか？ はい、良いのです。これら 2 つのレジスタの値を変更することなく `retf` 命令を実行することで、アプリのエントリポイントとなる関数の第 1、第 2 引数にそのまま値を引き継ぐことを意図しています。

さて、これでうまくアプリが動くかということ、まだ考慮が足りていない部分があります。それは `argv` の置き場所です。今まで `auto argv = MakeArgVector(command, first_arg);` という処理で `argv` を生成していました。`argv` の型は `std::vector<char*>` です。`std::vector` は内部で `new` 演算子を呼び出してメモリを確保します。`new` 演算子は最終的に `sbrk()` によって割り当てられた領域、つまり OS 用のアドレス空間に存在するメモリ領域を使います*¹。したがって、アプリが `argv` を読み取ろうとした瞬間にページフォルトが発生してしまうのです。`argv` をアプリ用の領域 (user ビットを 1 としたページ) に用意しなければなりません。

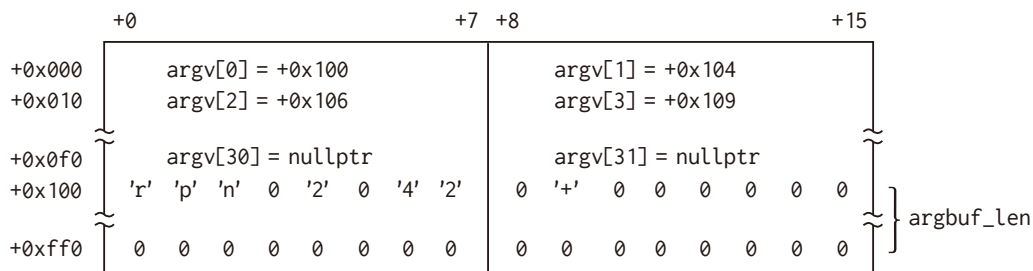
*¹ C++ のコンテナ (`std::vector` など) に備わるアロケータという仕組みを使って、独自のメモリ確保を行うことも可能です。少し複雑なので以降の説明では他の方法を採用しました。

リスト 20.7 アプリ用のページ内に argv を構築する (terminal.cpp)

```
LinearAddress4Level args_frame_addr{0xffff'ffff'ffff'f000};
if (auto err = SetupPageMaps(args_frame_addr, 1)) {
    return err;
}
auto argv = reinterpret_cast<char*>(args_frame_addr.value);
int argv_len = 32; // argv = 8x32 = 256 bytes
auto argbuf = reinterpret_cast<char*>(args_frame_addr.value + sizeof(char*) * argv_len);
int argbuf_len = 4096 - sizeof(char*) * argv_len;
auto argc = MakeArgVector(command, first_arg, argv, argv_len, argbuf, argbuf_len);
if (argc.error) {
    return argc.error;
}
```

アプリ用に 1 ページを確保し、その中に argv を構築するように改造しました (リスト 20.7)。rpn 2 42 + とアプリを起動した場合、改造した MakeArgVector() によって図 20.5 に示すようなメモリ構造が作成されます。1 ページの先頭部分に argv 配列を置き、その後ろに文字列データを置くことにしました。argv はポインタの配列ですので、各要素はその文字列データの位置を指すことになります。

図 20.5 ページの中の argv の構造



argv_len は argv の要素数です。実際の引数の数を動的に求めるのは面倒なので決め打ちで 32 としました。コマンドライン引数は 32 個もあれば当面は十分でしょう……。argbuf というポインタ変数は文字列データの開始位置 (図の +0x100 の位置) を指します。argbuf_len は文字列データの最大バイト数を表す変数で、1 ページの大きさから argv 配列の大きさ (0x100) を引いた値となります。

リスト 20.8 改造した MakeArgVector() はメモリ上に argv を構築する (terminal.cpp)

```
WithError<int> MakeArgVector(char* command, char* first_arg,
    char** argv, int argv_len, char* argbuf, int argbuf_len) {
    int argc = 0;
    int argbuf_index = 0;

    auto push_to_argv = [&](const char* s) {
        if (argc >= argv_len || argbuf_index >= argbuf_len) {
            return MAKE_ERROR(Error::kFull);
        }

        argv[argc] = &argbuf[argbuf_index];
```

```

    ++argc;
    strcpy(&argbuf[argbuf_index], s);
    argbuf_index += strlen(s) + 1;
    return MAKE_ERROR(Error::kSuccess);
};

if (auto err = push_to_argv(command)) {
    return { argc, err };
}
if (!first_arg) {
    return { argc, MAKE_ERROR(Error::kSuccess) };
}

```

MakeArgVector() を改造し、与えられたメモリ領域に上図のようなデータ構造を作成するようにしました。改造した関数の先頭部分を **リスト 20.8** に示します。修正した点で重要なのは関数内で定義した push_to_argv というラムダ式です。このラムダ式がやっているのは、ラムダ式への引数で与えられた文字列 s を argbuf が指す文字列データ領域にコピーすることと、コピー先の文字列へのポインタを argv の末尾に追加することです。このラムダ式を使って、コマンドライン引数を先頭から空白で区切りつつ argv を構築していきます。

argv を構築するページは SetupPageMaps() で確保しますから user ビットが 1 になります。したがって、アプリから argv を参照してもページフォルトが発生しないでしょう。これでアプリが動く準備が整ったはずです。

ごめんなさい。まだ準備が整っていませんでした。最後に 1 つだけ仕上げをします。それは **リスト 20.4** の最後の行にあるコメント「アプリケーションが終了してもここには来ない」に関連しています。

CallApp() は retf 命令でアプリにジャンプします。call 命令でアプリを呼び出すわけではありませんから、アプリから ret 命令で戻ってくることができません。それでは、アプリ側で無理やりスタックに OS 用の CS と戻り先アドレスを積んで retf を実行したらどうなるでしょうか？ 実は権限が高くなる（DPL が小さくなる）方向への far return はできないことになっていて、無理に実行しようすると一般保護例外（GP）が発生することでしょう。苦肉の策ですが、アプリの最後で無限ループするようにしておきます（**リスト 20.9**）。

リスト 20.9 far return ができないので無限ループしておく (rpn/rpn.cpp)

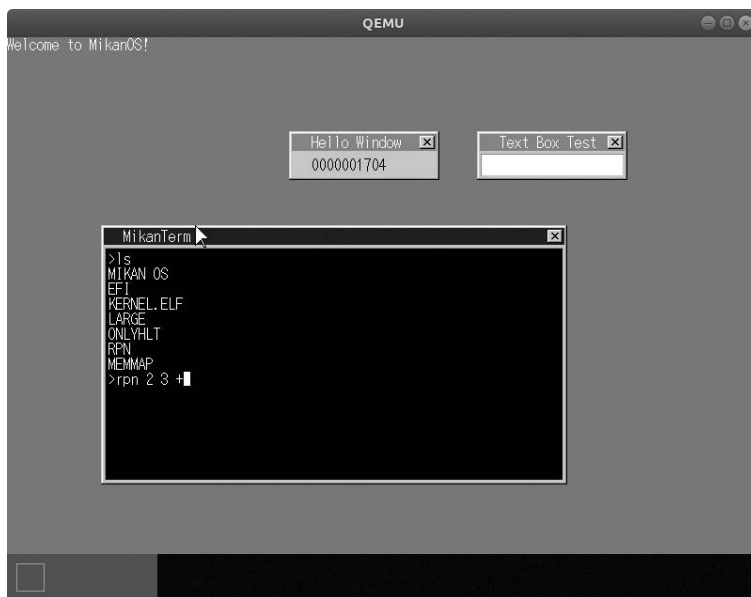
```

if (stack_ptr < 0) {
    return 0;
}
while (1);
//return static_cast<int>(Pop());

```

そういえば printk() や fill_rect() はもはやアプリから呼び出せませんので、それらの行は削除しましたよ。さて、これで本当に準備が整ったはずですから、アプリを起動してみましょう。いつもの通り ./build.sh run してみます。OS が起動したら rpn コマンドを叩いてみます（**図 20.6**）。

図 20.6 リング 3 で rpn コマンドを起動できた



rpn コマンドが本当にリング 3 で動いているのか、OS がフリーズしたのか非常に分かりづらいですが、よく見ると「Text Box Test」ウィンドウのカーソルが点滅を続けていますから OS 自体は生きています。そしてしばらく待っても OS が強制的に再起動してしまうようなことはないようです。セグメントやページの設定が足りなければ CPU の例外が発生して OS 自体が再起動するはずですから、それが発生していないんだな、ということが分かります。意図通りの動きになっているので成功と言っていいと思います。

でも、筆者は悲しいです。アプリをリング 3 で動かす代償としてアプリが OS 側に結果を返すことができなくなってしまいました。これでは何のためのアプリなのでしょう。ただ CPU を無駄に消費しているだけではありませんか。この章の残りでは、アプリが結果を出力できるようにし、処理が終わった後で終了できる仕組みを作ろうと思います。

20.3 TSSを設定しよう (osbook_day20c)

rpn コマンドが正常にリング 3 で動いているように見えますが、実は目に見えない重大なバグが残っています。これは CPU の仕様を知らないと感じきようがないバグで、rpn 実行時に割り込みが発生すると仮想アドレスの末尾付近のメモリが静かに破壊されるというものです。

CPU の仕様で、CPL=3 でプログラムを実行中に割り込みが発生して CPL=0 に切り替わる際、TSS.RSP0 の値が RSP レジスタに設定されます。**TSS** はメインメモリにある task-state segment という構造体のことです (表 20.1)。TSS.RSP0 は、TSS が持つ 64 ビット幅のフィールドのことです。割り込み発生により TSS.RSP0 の値が RSP レジスタに設定されるという動作は無効にすることができません。rpn コマンドを実行中にタイマ割り込みが発生した際、この仕組みが意図せず発動しているのが現状です。

表 20.1 TSS の構造

オフセット	サイズ	内容
0	4	予約 (0)
4	8	RSP0
12	8	RSP1
20	8	RSP2
28	8	予約 (0)
36	8	IST1
44 - 84		IST2 - IST7
92	10	予約 (0)
102	2	I/O マップベースアドレス

TSS は TR レジスタが指す 108 バイトのメモリ領域（セグメント）にあります。ただ、今まで TR を一切設定してきませんでしたから TR レジスタは 0 となっており、TSS がアドレス 0 に置いてあることになっているのです。そのため、rpn コマンドを実行中（すなわち CPL=3 で CPU が動作中）にタイマ割り込みが発生すると、メモリのアドレス 4 からの 8 バイト領域に書いてある値が RSP レジスタに設定され、割り込みハンドラ `IntHandlerLAPICTimer()` が実行されます。アドレス 4 に書いてある値は場合によって違うでしょうが、QEMU で実行したところ 0 でした。そのため、割り込みハンドラ実行直前の RSP の値は 0 となります。

CPU は割り込み発生時にスタックに 40 バイト（`InterruptFrame` 型）のスタックフレームを積みます。RSP=0 のときにスタックフレームが積まれると、スタックポインタがマイナス方向にオーバーフローして `0xffffffffd8 ~ 0xffffffff` の範囲にスタックフレームが書き込まれることになります。そして、割り込みハンドラの処理が進むにつれ、さらにスタックにデータが書き込まれ、メモリ破壊が進行します。

普通、変な仮想アドレスへの書き込みがあるとページフォルトが起き、OS が強制的に再起動するため、すぐエラーに気づけます。しかし rpn コマンドは一見すると問題なく実行できています。なぜでしょう？ その理由は、`0xffffffffd8 ~ 0xffffffff` の範囲は偶然にもアプリのコマンドライン引数 (argv) を格納する 4KiB の領域の末尾と重なっていて、物理フレームが割り当てられた状態になっているからです。物理フレームが（書き込み可の設定で）割り当てられたページにデータを書き込んでもページフォルトは起きません。加えて "rpn 2 3 +" 程度の短いコマンドラインでは 4KiB の領域のうち後ろの方は使われません。そのため、割り込みによって値が書き換えられても rpn の動作には影響が出ないのです。これらの事情によって、一見うまくいっているように見える裏側では静かにメモリ破壊が進行しています。

メモリ破壊を防ぐには TSS を適切に設定する必要があります。適当な大きさのスタック領域を確保し、その末尾のアドレスを TSS.RSP0 に記載しておきます。そうすれば CPL=3 でプログラム実行中に割り込みが発生した際、RSP レジスタが適切に設定されます。さくっとやっちゃいましょう。

リスト 20.10 TSS 用の領域を定義 (segment.cpp)

```
namespace {
    std::array<SegmentDescriptor, 7> gdt;
    std::array<uint32_t, 26> tss;

    static_assert((kTSS >> 3) + 1 < gdt.size());
}
```

リスト 20.10 に示すように、TSS 用の 108 バイトの領域として `tss` を定義しました。そして、GDT の大きさも 2 増やして 7 としました。これは TSS 用のセグメントを GDT に設定しなければならないからです。TSS 用のセグメントは GDT の 2 要素分を消費します。その下にある `static_assert` は、GDT の大きさが十分かどうかをコンパイル時に検査するものです。プログラムの動作には影響しませんから無視しても大丈夫です。

リスト 20.11 TSS は GDT[5] に置くことにする (segment.hpp)

```
const uint16_t kKernelCS = 1 << 3;
const uint16_t kKernelSS = 2 << 3;
const uint16_t kKernelDS = 0;
const uint16_t kTSS = 5 << 3;
```

GDT の 1 から 4 は OS やアプリ用のコード / データセグメントが設定されています。そこで、未使用の GDT[5] を TSS 用にします (**リスト 20.11**)。セグメントセクタは図 20.3 に示したように、GDT の添え字を 3 ビットシフトした値にするのでしたよね。

リスト 20.12 TSS を初期化して GDT に設定する (segment.cpp)

```
void InitializeTSS() {
    const int kRSP0Frames = 8;
    auto [ stack0, err ] = memory_manager->Allocate(kRSP0Frames);
    if (err) {
        Log(kError, "failed to allocate rsp0: %s\n", err.Name());
        exit(1);
    }
    uint64_t rsp0 =
        reinterpret_cast<uint64_t>(stack0.Frame()) + kRSP0Frames * 4096;
    tss[1] = rsp0 & 0xffffffff;
    tss[2] = rsp0 >> 32;

    uint64_t tss_addr = reinterpret_cast<uint64_t>(&tss[0]);
    SetSystemSegment(gdt[kTSS >> 3], DescriptorType::kTSSAvailable, 0,
        tss_addr & 0xffffffff, sizeof(tss)-1);
    gdt[(kTSS >> 3) + 1].data = tss_addr >> 32;

    LoadTR(kTSS);
}
```

リスト 20.12 に TSS を初期化して GDT へ設定する関数 `InitializeTSS()` の実装を示します。この関数はまず、TSS.RSP0 に設定するスタック領域を確保します。大きさは 8 フレーム = 32KiB としておきます。これだけあれば、割り込みハンドラを実行するには十分な大きさでしょう。

次に、確保したスタック領域を TSS.RSP0 に書き込みます。TSS.RSP0 は 8 バイトの大きさを持つにも関わらず TSS の先頭から 4 バイト目から始まります。普通、8 バイトの大きさのデータは 8 バイト境界に揃っていることが多いのですが、TSS は歴史的な事情により、各フィールドが 4 バイトずれて配置されていて少し面倒です。このため、上位 4 バイトと下位 4 バイトを分けて書き込みます。どうせ上位 4 バイトは 0 だから書く必要はないと言えないのですけどね。

TSS 自体の設定が完了したら、TSS の先頭アドレスを GDT[5] および GDT[6] に設定します。GDT のエントリ 1 つだけでは 32 ビットまでのアドレスしか設定できません。64 ビットモードにおいては 2 つのエントリを用いて TSS のアドレスを指定することになっています。SetSystemSegment() は GDT[5] だけを設定する関数です。GDT[6] はその後、追加で設定しています。

リスト 20.13 SetSystemSegment() はシステムセグメントを設定する (segment.cpp)

```
void SetSystemSegment(SegmentDescriptor& desc,
                     DescriptorType type,
                     unsigned int descriptor_privilege_level,
                     uint32_t base,
                     uint32_t limit) {
    SetCodeSegment(desc, type, descriptor_privilege_level, base, limit);
    desc.bits.system_segment = 0;
    desc.bits.long_mode = 0;
}
```

リスト 20.13 に SetSystemSegment() の実装を示します。この関数はシステムセグメント、つまり system segment ビットが 0 であるセグメントを設定します。セグメントディスクリプタの構造自体はコードセグメント（およびデータセグメント）と共通ですので、大部分の処理を SetCodeSegment() に委譲しています。

SetSystemSegment() を用いて GDT に TSS を登録し終わったら、TR レジスタに TSS のセグメントセクタ値を設定します。CPU は、割り込みが発生して TSS の値を読む必要が発生すると、TR レジスタが指す GDT エントリを参照して TSS を取得します。TR レジスタに値を書き込むには `ltr` (Load TR) という命令を使う必要があります。これは C++ では書けませんのでアセンブリ言語で実装した LoadTR() (**リスト 20.14**) を利用します。

リスト 20.14 LoadTR() は 16 ビット値を TR レジスタに設定する (asmfunc.asm)

```
global LoadTR
LoadTR: ; void LoadTR(uint16_t sel);
        ltr di
        ret
```

これで InitializeTSS() の説明が終わりました。まとめると、この関数は CPU が正しい TSS を参照できるようにします。108 バイトの TSS 領域の TSS.RSP0 にスタック領域の末尾アドレスを書きます。TSS の先頭アドレスを GDT[5]、GDT[6] に設定します。最後に GDT[5] を指すセグメントセクタ値を TR レジスタにロードします。

リスト 20.15 InitializeTSS() をメイン関数から呼び出す (main.cpp)

```
InitializeMemoryManager(memory_map);
InitializeTSS();
InitializeInterrupt();
```

作った InitializeTSS() を呼び出す必要があります。InitializeTSS() の中でメモリマネージャを使いますので、メモリマネージャを初期化した後に呼び出すようにします (リスト 20.15)。TSS の設定は、メモリマネージャの初期化後、かつ、CPL=3 から CPL=0 になるような割り込みが発生する前 (rpn コマンドを呼び出す前) であればいつでもいいはずです。

TSS の設定が完了しましたので、rpn コマンド実行中の割り込みが正しいスタックで処理されるようになり、メモリ破壊をしなくなったはずですが、ただ、今回の改造によって新たに問題が発生します。それはコンテキストスイッチで保存する RSP の値の計算に関係があります。

図 20.7 TaskContext::rsp に保存する値の変更

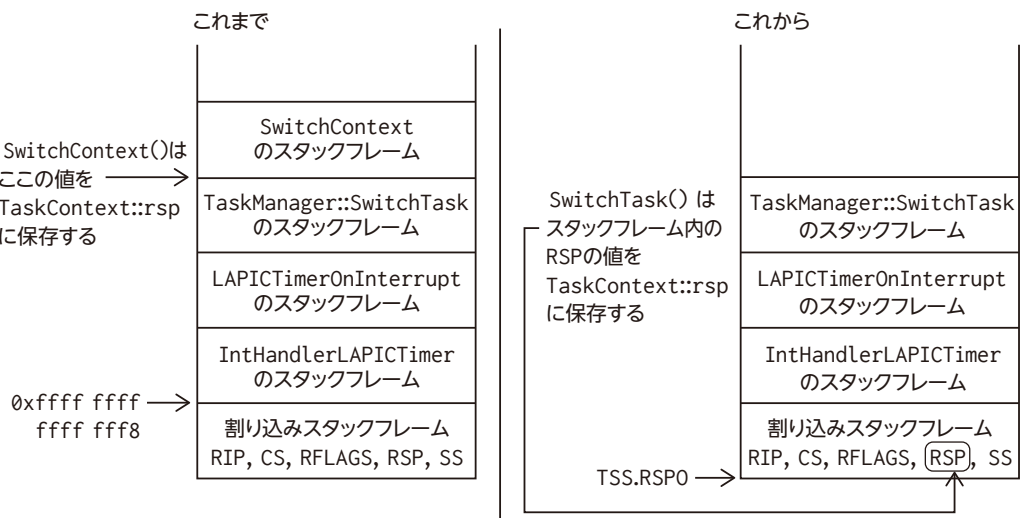


図 20.7 の左にこれまでの仕組みを示します。タイマ割り込みが発生すると CPU は RIP や RSP などの値を含む 40 バイトのデータ (割り込みフレーム) をスタックに積んでから IntHandlerLAPICTimer() を実行します。この割り込みハンドラは LAPICTimerOnInterrupt() を呼び出し、その中でさらに task_manager->SwitchTask() が呼ばれます。最終的に SwitchContext() が呼ばれ、その中でコンテキストが保存されます。TaskContext::rsp に保存するための値は SwitchContext() が呼ばれる直前の RSP レジスタの値となります。

この仕組みは、スタックがすべてのコンテキストで別々であれば問題ありません。保存されたコンテキストが次に復帰するまでの間に、スタックの中身が書き換わることがないからです。次にコンテキストが復帰したとき、割り込みハンドラの途中 (SwitchContext() を呼び出す call 命令の次の命令) から処理が再開することになります。スタックフレームを辿ってレジスタ値の復帰やいくつかの ret 命令が実行され、最終的に iret 命令にたどり着いて割り込みが終了します。

しかし先ほど加えた変更によって、ユーザーモード (CPL=3) で動くコンテキストは同じスタック (TSS.

RSP0 が指すスタック) を使って割り込み処理を行うようになりました。同時に動くアプリの数は、今は高々 1 つですが、将来は複数のアプリを同時に起動できるようにしていくつもりです。ユーザーモードで動くコンテキストが複数ある場合、割り込み処理によって TSS.RSP0 が指すスタックの中身が書き換えられてしまうため、これまでの方式は使えません。

そこで、**図 20.7** の右側のように、割り込みフレームに含まれる値をコンテキスト構造体に保存するように改造します。この改造により、次にコンテキストが復帰する際、割り込みハンドラの途中からではなく、割り込みハンドラの終了直後の場所から処理が再開されます。TSS.RSP0 が指すスタックは一切参照されませんから、スタックが上書きされていても大丈夫です。

リスト 20.16 割り込みフレームを上位の関数に流す (timer.cpp)

```
extern "C" void LAPICTimerOnInterrupt(const TaskContext& ctx_stack) {
    const bool task_timer_timeout = timer_manager->Tick();
    NotifyEndOfInterrupt();

    if (task_timer_timeout) {
        task_manager->SwitchTask(ctx_stack);
    }
}
```

割り込みフレームが必要なのは TaskManager::SwitchTask() ですが、割り込みフレームを取得できるのは割り込みハンドラの中だけです。したがって、割り込みハンドラで割り込みフレーム内の値を取得して SwitchTask() にまで伝播させなければなりません。**リスト 20.16** に示すように、LAPICTimerOnInterrupt() が引数として割り込みフレームの情報 (を使って構築したコンテキスト構造体) を受け取って SwitchTask() に渡すようにしました。割り込みフレーム自体ではなく、割り込みフレームの情報を使って構築したコンテキスト構造体を渡すようにしたのは、SwitchTask() に渡す情報が割り込みフレームの値だけでは足りなかったからです。後で詳しく説明します。

LAPICTimerOnInterrupt() は割り込みハンドラ IntHandlerLAPICTimer() から呼び出されます。これまで、この割り込みハンドラは C++ コンパイラの __attribute__((interrupt)) の拡張機能を利用して作っていましたが、**リスト 20.17** のようにアセンブリ言語で実装しなおしました。C++ では書けない特殊な処理を書きたかったからです。

リスト 20.17 割り込みハンドラ内でコンテキスト構造体を構築する (asmfunc.asm)

```
extern LAPICTimerOnInterrupt
; void LAPICTimerOnInterrupt(const TaskContext& ctx_stack);

global IntHandlerLAPICTimer
IntHandlerLAPICTimer: ; void IntHandlerLAPICTimer();
    push rbp
    mov rbp, rsp

    ; スタック上に TaskContext 型の構造を構築する
    sub rsp, 512
    fxsave [rsp]
    push r15
    push r14
    push r13
```

```
push r12
push r11
push r10
push r9
push r8
push qword [rbp]      ; RBP
push qword [rbp + 0x20] ; RSP
push rsi
push rdi
push rdx
push rcx
push rbx
push rax

mov ax, fs
mov bx, gs
mov rcx, cr3

push rbx              ; GS
push rax              ; FS
push qword [rbp + 0x28] ; SS
push qword [rbp + 0x10] ; CS
push rbp              ; reserved1
push qword [rbp + 0x18] ; RFLAGS
push qword [rbp + 0x08] ; RIP
push rcx              ; CR3

mov rdi, rsp
call LAPICTimerOnInterrupt

add rsp, 8*8 ; CR3 から GS までを無視
pop rax
pop rbx
pop rcx
pop rdx
pop rdi
pop rsi
add rsp, 16 ; RSP, RBP を無視
pop r8
pop r9
pop r10
pop r11
pop r12
pop r13
pop r14
pop r15
fxrstor [rsp]

mov rsp, rbp
pop rbp
iretq
```

実装しなおした `IntHandlerLAPICTimer()` の主要な役目は、スタック上にコンテキスト構造体を構築し、それを `LAPICTimerOnInterrupt()` に渡すことです。コンテキスト構造体の末尾からスタック上に積み上げていくことでコンテキスト構造体を構築します。

コンテキスト構造体に保存する値は、次にこのコンテキストを復帰したときに割り込み処理がちょうど終わった状態となる値にしておきたいです。RIP や RSP など、5 つのレジスタの値は割り込みフレームから取得できます。ただ、その他のレジスタの値は割り込みフレームには記録されていませんから、レジスタ値が変化してしまう前に保存しなければなりません。レジスタ値が変化する前に保存する、というシビアな処理のために、割り込みハンドラをアセンブリ言語で実装する必要があったというわけです。

CR3 の値をスタックに積み終わった時点 (`push rcx` を実行した時点) で完全なコンテキスト構造体がスタック上に構築されます。そのときの RSP レジスタがコンテキスト構造体の先頭を指した状態になっていますので、RSP の値を第 1 引数のためのレジスタである RDI にコピーしてから `LAPICTimerOnInterrupt()` を呼び出します。こうして `LAPICTimerOnInterrupt()` はスタック上のコンテキスト構造体を引数で受け取ることができます。

リスト 20.18 スタック上のコンテキスト構造体を受け取る (task.cpp)

```
void TaskManager::SwitchTask(const TaskContext& current_ctx) {
    TaskContext& task_ctx = task_manager->CurrentTask().Context();
    memcpy(&task_ctx, &current_ctx, sizeof(TaskContext));
    Task* current_task = RotateCurrentRunQueue(false);
    if (&CurrentTask() != current_task) {
        RestoreContext(&CurrentTask().Context());
    }
}
```

リスト 20.18 に `TaskManager::SwitchTask()` 側の改造を示します。第 1 引数でコンテキスト構造体を受け取るようにしました。受け取ったコンテキスト構造体を現在動作中のタスクのコンテキスト構造体にコピーし、次のタスクのコンテキストへと切り替えます。`RotateCurrentRunQueue()` は、ランキューの先頭要素をランキューの末尾に移動する関数です。もともと `SwitchTask()` にあった処理を関数に切り出したただけですので説明は省きます。

ランキューを操作した後、今までは `SwitchContext()` を使ってコンテキストを切り替えていました。しかし、この関数は内部でコンテキストを保存してしまうので、もはや使えません。そこで、新しく `RestoreContext()` という関数を作りました。コンテキストの保存はせず、与えられたコンテキストを復帰するだけの関数です。

リスト 20.19 `RestoreContext()` はコンテキストを復帰する (asmfunc.asm)

```
global SwitchContext
SwitchContext: ; void SwitchContext(void* next_ctx, void* current_ctx);
    mov [rsi + 0x40], rax

    < 中略 >

    fxsave [rsi + 0xc0]
    ; fall through to RestoreContext
```

```

global RestoreContext
RestoreContext: ; void RestoreContext(void* task_context);
; iret 用のスタックフレーム
push qword [rdi + 0x28] ; SS

< 中略 >

o64 iret

```

リスト 20.19 に `RestoreContext()` の実装を示します。じっくり見ると分かると思いますが、実はこの関数、`SwitchContext()` の処理の後半部分に名前を付けただけです。`RestoreContext()` を呼び出しても `SwitchContext()` を呼び出しても、最終的には `RestoreContext()` の処理が実行されます。この関数は、与えられたコンテキスト構造体の値を各レジスタに復帰させ、最後は `iret` 命令を使って次のコンテキストへと処理を移します。

リスト 20.20 `Sleep()` は引き続き `SwitchContext()` を使う (task.cpp)

```

void TaskManager::Sleep(Task* task) {
    if (!task->Running()) {
        return;
    }

    task->SetRunning(false);

    if (task == running_[current_level_].front()) {
        Task* current_task = RotateCurrentRunQueue(true);
        SwitchContext(&CurrentTask().Context(), &current_task->Context());
        return;
    }

    Erase(running_[task->Level()], task);
}

```

`Sleep()` は今まで通り `SwitchContext()` を使ってコンテキストを切り替えます (**リスト 20.20**)。なぜなら、`Sleep()` は割り込みではない通常の処理で使われる関数だからです。通常の処理ではスタックが複数のコンテキストで共用されることが無く、処理を変える必要がありませんでした。

GDB を接続した QEMU で MikanOS を起動させ、タイマ割り込みハンドラ `IntHandlerLAPICTimer()` の先頭で処理を止めて様子を見てみました。

```

$ gdb
.....
Breakpoint 1, 0x000000000010e8ae in IntHandlerLAPICTimer ()
(gdb) p $rsp
$1 = (void *) 0x8fd8
(gdb)

```

改造前は RSP の値が `0xffffffffffffd8` になっていましたが、改造後は `0x8fd8` となりました。`Log()` を使って `TSS.RSP0` の値を出力してみたところ `0x9000` となっていて、意図通りであることが確認できました。

20.4 バグ発見を手伝おう (osbook_day20d)

セグメンテーションやページングといった CPU の機能を使ったプログラミングをしていると、頻繁に CPU 例外を発生させてしまい OS 全体が強制的に再起動することがよくあります。これでは開発がとてもしにくいです。この節では、CPU 例外を捉えて例外発生時の状況をプログラマが分かるように表示する機能を付けてみたいと思います。皆さんのためというよりは、筆者が MikanOS を開発していく中で欲しくなったからというのが主な理由です。読者の皆さんは筆者が頑張ってデバッグした後の例外が発生しないプログラムしか見ませんが、筆者は本書に掲載するソースコードに辿り着くまでにたくさんの例外をくぐり抜ける必要があるのです。

リスト 20.21 CPU 例外に対応する割り込みハンドラ群 (interrupt.cpp)

```
#define FaultHandlerWithError(fault_name) \
    __attribute__((interrupt)) \
    void IntHandler ## fault_name (InterruptFrame* frame, uint64_t error_code) { \
        PrintFrame(frame, "#" #fault_name); \
        WriteString(*screen_writer, {500, 16*4}, "ERR", {0, 0, 0}); \
        PrintHex(error_code, 16, {500 + 8*4, 16*4}); \
        while (true) __asm__("hlt"); \
    }

#define FaultHandlerNoError(fault_name) \
    __attribute__((interrupt)) \
    void IntHandler ## fault_name (InterruptFrame* frame) { \
        PrintFrame(frame, "#" #fault_name); \
        while (true) __asm__("hlt"); \
    }

FaultHandlerNoError(DE)
FaultHandlerNoError(DB)
FaultHandlerNoError(BP)
FaultHandlerNoError(OF)
FaultHandlerNoError(BR)
FaultHandlerNoError(UD)
FaultHandlerNoError(NM)
FaultHandlerWithError(DF)
FaultHandlerWithError(TS)
FaultHandlerWithError(NP)
FaultHandlerWithError(SS)
FaultHandlerWithError(GP)
FaultHandlerWithError(PF)
FaultHandlerNoError(MF)
FaultHandlerWithError(AC)
FaultHandlerNoError(MC)
FaultHandlerNoError(XM)
FaultHandlerNoError(VE)
```

リスト 20.21 に示すように、CPU 例外に対応する割り込みハンドラをたくさん定義しました。よく経験する例外からマイナーな例外までありますが、とりえず発生する可能性のあるものは全部定義しておきました。それぞれの例外の意味は Intel SDM (参考文献 [3]) Vol3 「6.15 Exception and Interrupt Reference」を参照してください。

例外ハンドラの実装は共通していますのでマクロ化しています。FaultHandlerWithError() マクロはエラーコード付きの例外ハンドラを定義します。エラーコードは例外発生時の原因を示す整数値で、いくつかの例外がエラーコード付きになっています。FaultHandlerNoError() マクロはエラーコード無しの例外用で、エラーコードを表示するための行 PrintHex(error_code,...) がありませんね。

リスト 20.22 PrintFrame() はスタックフレームの情報を表示する (interrupt.cpp)

```
void PrintHex(uint64_t value, int width, Vector2D<int> pos) {
    for (int i = 0; i < width; ++i) {
        int x = (value >> 4 * (width - i - 1)) & 0xfu;
        if (x >= 10) {
            x += 'a' - 10;
        } else {
            x += '0';
        }
        WriteAscii(*screen_writer, pos + Vector2D<int>{8 * i, 0}, x, {0, 0, 0});
    }
}

void PrintFrame(InterruptFrame* frame, const char* exp_name) {
    WriteString(*screen_writer, {500, 16*0}, exp_name, {0, 0, 0});
    WriteString(*screen_writer, {500, 16*1}, "CS:RIP", {0, 0, 0});
    PrintHex(frame->cs, 4, {500 + 8*7, 16*1});
    PrintHex(frame->rip, 16, {500 + 8*12, 16*1});
    WriteString(*screen_writer, {500, 16*2}, "RFLAGS", {0, 0, 0});
    PrintHex(frame->rflags, 16, {500 + 8*7, 16*2});
    WriteString(*screen_writer, {500, 16*3}, "SS:RSP", {0, 0, 0});
    PrintHex(frame->ss, 4, {500 + 8*7, 16*3});
    PrintHex(frame->rsp, 16, {500 + 8*12, 16*3});
}
```

PrintHex() は与えられた整数値を指定された桁数の 16 進数として画面に表示します (リスト 20.22)。例外発生時は通常とは違う厳しい状況のことがありますから、処理の複雑な sprintf() などの標準ライブラリ関数は使わず、単純な実装を心がけます。また、コンソールのスクロール処理が発生するのも避けたいと思い、printk() で表示するのではなく画面のフレームバッファにフォントを直書きすることにしました。

PrintFrame() は画面に例外発生時のスタックフレームを表示する関数です。PrintHex() を活用し、スタックフレームに含まれる各種レジスタ値を表示します。スタックフレームに含まれないレジスタ値 (RAX や CS など) も表示してあげるとさらに良いかもしれませんね。必要になったら付け加えることにしましょう。

リスト 20.23 例外ハンドラを IDT に登録する (interrupt.cpp)

```
void InitializeInterrupt() {
    auto set_idt_entry = [](int irq, auto handler) {
        SetIDTEntry(idt[irq],
            MakeIDTAttr(DescriptorType::kInterruptGate, 0),
            reinterpret_cast<uint64_t>(handler),
            kKernelCS);
    };
}
```

```

};
set_idt_entry(InterruptVector::kXHCI, IntHandlerXHCI);
set_idt_entry(InterruptVector::kLAPICTimer, IntHandlerLAPICTimer);
set_idt_entry(0, IntHandlerDE);
set_idt_entry(1, IntHandlerDB);
set_idt_entry(3, IntHandlerBP);
set_idt_entry(4, IntHandlerOF);
set_idt_entry(5, IntHandlerBR);
set_idt_entry(6, IntHandlerUD);
set_idt_entry(7, IntHandlerNM);
set_idt_entry(8, IntHandlerDF);
set_idt_entry(10, IntHandlerTS);
set_idt_entry(11, IntHandlerNP);
set_idt_entry(12, IntHandlerSS);
set_idt_entry(13, IntHandlerGP);
set_idt_entry(14, IntHandlerPF);
set_idt_entry(16, IntHandlerMF);
set_idt_entry(17, IntHandlerAC);
set_idt_entry(18, IntHandlerMC);
set_idt_entry(19, IntHandlerXM);
set_idt_entry(20, IntHandlerVE);
LoadIDT(sizeof(idt) - 1, reinterpret_cast<uintptr_t>(&idt[0]));
}

```

リスト 20.23 に例外ハンドラを IDT に登録するプログラムを示します。何も難しいことはなく、CPU の仕様で決められた割り込み番号に先ほど定義したハンドラを登録するだけです。この中で**一般保護例外** (GP=13) と**ページフォルト** (PF=14) の名前と番号は覚えておいてもいいかもしれません。いろいろな場所で見かけると思います。

図 20.8 アプリから printk() を呼び出すと例外が発生する



図 20.8 は、リスト 20.1 と同じように `printk()` を呼び出すように改造した `rpn` コマンドを実行した様子です。`printk()` が呼び出された瞬間にページフォルトが発生していることが分かります。例外ハンドラを追加したおかげでどの例外がどこで発生したかを知ることができ、デバッグが一気にやりやすくなると思います。

試し終わったら `rpn` コマンドから `printk()` の呼び出しを削除しておきましょう。

20.5 システムコール (osbook_day20e)

今のままではアプリケーションから OS の機能呼び出せません。アプリができるのはメモリ上の計算だけです。権限が低いアプリにとって、OS の機能を使わなければ画面に文字や数字を出力することさえできないのです。表示できないままでは何も計算していないのと変わらないどころか、ただ CPU を無駄遣いしていることになります。悲しいですね。

アプリケーションから安全に OS の機能呼び出す仕組みのことを**システムコール**と呼びます。システム (OS) をコール (呼び出す) という意味ですね。名前だけ聞いたことがあるという方も多いのではないのでしょうか。なんと、この節ではシステムコールを自分の手で作っちゃいます。

システムコールの実現方式はいくつもありますが、x86-64 アーキテクチャの 64 ビットモードでは `syscall` 命令を使った方式が一般的でしょう。その他の方法には「30 日でできる！ OS 自作入門」で採用されている割り込みを用いた方法^{*2}、Linux に `io_uring` という名前で導入されたメモリ上のキューを用いた方法などがあります。権限レベルの概念がない CPU で採用される、OS の関数を直接呼び出す方法もシステムコールの一種と言えるでしょう。本書では最もオーソドックスな `syscall` を用いた方法を説明し、実装します。

`syscall` 命令は低い権限で動くプログラムから、高い権限でしか動作できないプログラムを呼び出す仕組みです。アプリと OS の橋渡しをする関数を 1 つ事前登録しておくことで、安全にアプリから OS の機能呼び出すことができます。リング 3 で動くアプリが自由に OS の関数や変数を使ってしまうと危険ですが、呼び出す関数を限定することで安全に OS の機能呼び出せるのです。

リスト 20.24 システムコールを呼び出す小さなコード (rpn/syscall.asm)

```
bits 64
section .text

global SyscallLogString
SyscallLogString:
    mov eax, 0x80000000
    mov r10, rcx
    syscall
    ret
```

*2 「30 日でできる！ OS 自作入門」では API と呼んでいます。API はシステムを呼び出すこと以外も含む広い概念です。

`syscall` 命令の使い方を **リスト 20.24** に示します。`syscall` 命令は C++ では書けないのでアセンブリ言語の関数として用意します。`SyscallLogString()` のプログラムはとても単純で、EAX レジスタに固定値を設定し RCX を R10 にコピーしたうえで `syscall` を実行します。これだけです。

`SyscallLogString()` は、OS の `Log()` を使ってコンソールに文字列を表示するシステムコールを呼び出します。後で詳しく説明しますが、EAX に設定する番号でシステムコールの機能を選択します。番号と機能の対応付けは OS 作者の自由です。うーん、そうですね、0x80000000 がコンソール表示機能ということにしましょうか。システムコールを増やすときはこの番号をインクリメントします。なぜ 0x80000000 にしたかという、Linux のシステムコール番号 (0 始まり) と被らないようにしたかったからです。Linux 向けのアプリを無変更で動かしたくなったときに楽できる、かもしれません。

システムコールの受け口となる OS 側の関数に、どうにかしてシステムコールの種類を表す番号やシステムコールの処理に必要な引数を渡す必要があります。渡し方はいくつかあり得ます。最も楽なのは、通常の間数呼び出しと同様にレジスタで引数を渡す方法です。レジスタの使い方はシステムコールを呼び出す側と呼び出される側で合わせればよく、特定の ABI を採用する必然性はありません。しかし、あえて異なるルールを採用する意味がありませんので、MikanOS では基本的には System V AMD64 ABI に合わせることにしました。

System V AMD64 ABI に従うことにすると、引数をシステムコールに渡すのはとても楽です。なぜなら、`SyscallLogString()` が C++ から呼び出されたとき、引数はすでに RDI、RSI、RDX、RCX、R8、R9 の順に格納されているため、基本的には何もしなくてよいからです。そのまま `syscall` 命令を実行すればレジスタの値が OS 側にそのまま伝わります。

`syscall` はほとんどの汎用レジスタの値を保持しますが、2 つだけ例外があります。`syscall` 命令は RCX に RIP の値を、R11 に RFLAGS の値を保存してしまうのです。いやまあ、保存してくれるのはいいのですが、レジスタの値が壊されてしまうので注意が必要です。普通の `call` 命令はスタック上に戻り先アドレスを保存するのですが、`syscall` 命令はレジスタに保存する仕様になっています。

この例外のために、システムコールの第 4 引数レジスタとして RCX が使えません。Linux ではシステムコールの第 4 引数を R10 で渡すことでこの問題に対処しています。MikanOS でもそれに合わせることにしました。`mov r10, rcx` はそのための調整をしているのですね。冷静に考えると `SyscallLogString()` の引数は 2 つだけですから調整は不要でした。でも、きっと 4 つ以上の引数を持つシステムコールが今後増えるでしょう。そのときに RCX が上書きされるということを忘れないために、無駄ですが残しておきます。

さて、`syscall` を実行すると事前に登録してある OS 側の関数が呼び出されます。その中でシステムコールの処理が行われ、終わると `syscall` の次の行に戻ってきます。そこには `ret` 命令がありますので `SyscallLogString()` の呼び出し元に戻ります。EAX に設定している値の意味を理解するため、`syscall` で呼び出される関数の定義と、その関数を登録する処理を見えます。

20.6 システムコールの登録処理

まずは `syscall` が呼び出す関数を登録する処理を **リスト 20.25** に示します。いずれの行も `WriteMSR()` という関数を実行していますね。この関数は第 1 引数で指定されたモデル固有レジスタ^{*3} に第 2 引数の値を書き込みます。実装は後で紹介します。

リスト 20.25 `syscall` で呼び出される関数を登録する (`syscall.cpp`)

```
void InitializeSyscall() {
    WriteMSR(kIA32_EFER, 0x0501u);
    WriteMSR(kIA32_LSTAR, reinterpret_cast<uint64_t>(SyscallEntry));
    WriteMSR(kIA32_STAR, static_cast<uint64_t>(8) << 32 |
        static_cast<uint64_t>(16 | 3) << 48);
    WriteMSR(kIA32_FMASK, 0);
}
```

1 行目は `syscall` を使えるようにする設定です。IA32_EFER レジスタのビット 0 は SCE (Syscall Enable) と呼び、ここを 1 にすると `syscall` が使えるようになります。ビット 8 (LME) とビット 10 (LMA) は 64 ビットモードを使うときは 1 にしておく必要がありますので、合わせて 0x501 をレジスタに書き込みます。

2 行目は IA32_LSTAR レジスタに `SyscallEntry()` という関数の先頭アドレスを書き込みます。そう、IA32_LSTAR レジスタこそが、`syscall` が実行されたときに呼び出される関数を登録するための場所です。アプリが `SyscallLogString()` (の中の `syscall`) を実行すると、ここに登録した関数 (`SyscallEntry()`) が起動することになります。

`syscall` は CS と SS を OS 用に設定してから登録された関数を実行します。でないと CPL=3 のまま OS の関数を実行することになり、ページフォルトが発生してしまいます。3 行目の設定は、これら 2 つのセグメントレジスタの値に影響を与えます。ここに設定した値の使われ方を **表 20.2** にまとめます。

表 20.2 `syscall` と `sysret` がセグメントレジスタに書く値

レジスタ	<code>syscall</code>	<code>sysret</code>
CS	IA32_STAR[47:32]	IA32_STAR[63:48]+16
SS	IA32_STAR[47:32]+8	IA32_STAR[63:48]+8

表にある `sysret` は `syscall` と対応する命令です。通常の `call/ret` と同じような関係の命令です。`sysret` は CS と SS をアプリ用に設定してから `syscall` の呼び出し元 (アプリ) に戻ります。

表の項目は、それぞれの命令を実行したときに 2 つのセグメントレジスタへロードされる値を示します。例えば `syscall` を実行すると IA32_STAR[47:32] に設定された 16 ビットの値が CS へ書き込まれます。また、その値に 8 を足した値が SS に書き込まれます。いずれも OS 用のセグメントセクタですので、下位 2 ビット (RPL) は 0 にしておきます。

`sysret` を実行すると IA32_STAR[63:48] の値に 16 を足した値が CS に、8 を足した値が SS に書き込まれます。これら 2 つはアプリ用のセグメントセクタですので下位 2 ビットは 3 にします (16

* 3 MSR : Model Specific Register

！ 3 のように 3 をビット OR しているのはそのためです)。

CS と SS の値を個別に指定するのではなく、1 つの値から計算されるというのがちょっと分かりにくいかもしれませんが。それでも `syscall` の方は比較的シンプルですが。MikanOS の GDT では OS 用のコードセグメントが `gdt[1]`、スタックセグメントが `gdt[2]` ですので、CS には $8(1 \ll 3)$ を、SS には $16(2 \ll 3)$ を設定したいです。IA32_STAR[47:32] に 8 を設定しておけば良さそうです。

一方、`sysret` はもっと複雑です。コードセグメントは IA32_STAR[63:48] の設定値 +16、スタックセグメントは設定値 +8 の値になってしまいますから、コードセグメントをスタックセグメントの後に置く必要があります。しかし、今の設定では `gdt[3]` がアプリ用コードセグメント、`gdt[4]` がアプリ用スタックセグメントとなっていて、順序が逆です。

リスト 20.26 セグメントディスクリプタの順序を調整する (segment.cpp)

```
void SetupSegments() {
    gdt[0].data = 0;
    SetCodeSegment(gdt[1], DescriptorType::kExecuteRead, 0, 0, 0xffffffff);
    SetDataSegment(gdt[2], DescriptorType::kReadWrite, 0, 0, 0xffffffff);
    SetDataSegment(gdt[3], DescriptorType::kReadWrite, 3, 0, 0xffffffff);
    SetCodeSegment(gdt[4], DescriptorType::kExecuteRead, 3, 0, 0xffffffff);
    LoadGDT(sizeof(gdt) - 1, reinterpret_cast<uintptr_t>(&gdt[0]));
}
```

ということで**リスト 20.26**に示すように GDT の設定を変えました。具体的には `gdt[3]` と `gdt[4]` を入れ替えただけです。この状態で IA32_STAR[63:48] に 19 ($16 \ll 3$) を設定しておくことで、`sysret` によって CS に 35 ($4 \ll 3$) が、SS に 27 ($3 \ll 3$) が設定され、希望通りアプリ用のセグメントに切り替わります。この修正に合わせて `CallApp()` の引数の順序も入れ替えておきました。

リスト 20.27 WriteMSR() は指定されたモデル固有レジスタに値を設定する (asmfunc.asm)

```
global WriteMSR
WriteMSR: ; void WriteMSR(uint32_t msr, uint64_t value);
    mov rdx, rsi
    shr rdx, 32
    mov eax, esi
    mov ecx, edi
    wrmsr
    ret
```

リスト 20.27に `WriteMSR()` の実装を示します。第 1 引数で指定されたモデル固有レジスタ (MSR) に、第 2 引数で指定された値を書き込みます。汎用レジスタは RAX のような名前で指定できますが、モデル固有レジスタは ECX にレジスタ番号を設定することによって指定します。wrmsr 命令は ECX で指定されたレジスタへ、EDX:EAX の 64 ビット値を書き込みます。RSI に渡された第 2 引数の上位 32 ビットを EDX にコピーするために、右シフト命令 `shr` を使って 32 ビットのシフトを行っています。

20.7 システムコールの本体

システムコールの登録処理が分かりました。次に `syscall` によって呼び出される関数 `SyscallEntry()` の実装を示します (リスト 20.28)。アプリから OS の機能呼び出す唯一の口となる関数です。WriteMSR() に引き続きアセンブリ言語による実装でごめんなさい。アセンブリ言語でしか書けないため仕方ないのです。

リスト 20.28 `syscall` で呼び出される OS 側の関数 (`asmfunc.asm`)

```
extern syscall_table
global SyscallEntry
SyscallEntry: ; void SyscallEntry(void);
    push rbp
    push rcx ; original RIP
    push r11 ; original RFLAGS

    mov rcx, r10
    and eax, 0xffffffff
    mov rbp, rsp
    and rsp, 0xfffffffffffffffff0

    call [syscall_table + 8 * eax]
    ; rbx, r12-r15 は callee-saved なので呼び出し側で保存しない
    ; rax は戻り値用なので呼び出し側で保存しない

    mov rsp, rbp

    pop r11
    pop rcx
    pop rbp
    o64 sysret
```

この関数の最重要な行は真ん中にある `call` 命令です。この `call` は、`syscall_table` という名の関数ポインタ表からシステムコール番号に応じた関数ポインタを取ってきて、それを呼び出します。

`call` の前後は必要な調整をしています。先頭から見ていきます。まず最初に RBP を保存しているのは、このレジスタの値が関数呼び出しの前後で変化してはいけないからです。後で RBP を RSP の一時的な保存のために使おうと思っていますので、もともとの値をスタックに保存しておきます。

System V AMD64 ABI によると、関数呼び出し前後で RBP の値が保存されている必要はあるものの、RCX と R11 はその対象ではありません。ではなぜ、RBP に続けてこれら 2 つのレジスタも保存しているのでしょうか？

RCX と R11 を保存しているのは、これらの値を `sysret` 命令が利用するからです。`syscall` 命令はこれら 2 つのレジスタに重要な値を保存します。それは `syscall` 実行時の RIP と RFLAGS の値です。`sysret` は逆に RCX と R11 の値を RIP と RFLAGS に書き戻すことで、`syscall` の呼び出し元に戻ることができます。`call` による関数呼び出しで RCX と R11 が変更されることに備え、`call` の前後で保存と復帰をしているというわけです。

3 つのレジスタを保存した後、`mov rcx, r10` を実行しています。これはリスト 20.24 で RCX を R10 にコピーした操作をもとに戻しているだけです。`syscall` 命令の仕様により RCX を引数の受け渡しに使用できないためにこのような変換が必要なのですね。

and eax, 0x7fffffff はシステムコール番号の最上位ビットをマスクします。システムコール番号を syscall_table の添字として使いたいのですが、0x80000000 という大きな数値のシステムコール番号をそのまま添字として使うと、先頭に 0x80000000 個の使わない要素を持つ巨大な配列が必要になってしまいます。最上位ビットを 0 にすることで、添字に適した 0 始まりの番号へと変換しています。

次の 2 行 (RSP の保存と値の調整) は次の call 命令への備えです。System V AMD64 ABI では関数を呼び出す際のスタックポインタが 16 の倍数になっている必要があると決まっています。そのための調整をしています。

その次には最初に紹介した call があって、その後は保存した値を復帰する処理が続き、最後に sysret を実行して syscall の呼び出し元に戻ります。

リスト 20.29 関数ポインタ表 syscall_table の定義 (syscall.cpp)

```
namespace syscall {

#define SYSCALL(name) \
    int64_t name( \
        uint64_t arg1, uint64_t arg2, uint64_t arg3, \
        uint64_t arg4, uint64_t arg5, uint64_t arg6)

SYSCALL(LogString) {
    if (arg1 != kError && arg1 != kWarn && arg1 != kInfo && arg1 != kDebug) {
        return -1;
    }
    const char* s = reinterpret_cast<const char*>(arg2);
    if (strlen(s) > 1024) {
        return -1;
    }
    Log(static_cast<LogLevel>(arg1), "%s", s);
    return 0;
}

#undef SYSCALL

} // namespace syscall

using SyscallFuncType = int64_t (uint64_t, uint64_t, uint64_t,
                                uint64_t, uint64_t, uint64_t);
extern "C" std::array<SyscallFuncType*, 1> syscall_table{
    /* 0x00 */ syscall::LogString,
};
}
```

リスト 20.29 に syscall_table の定義を示します。添字 0 の位置に LogString() のアドレスを登録しています。この添字に 0x80000000 を足した値がシステムコール番号となります。

syscall::LogString() の処理は単純です。まず引数の検査をし、不正な値でなければコンソールへ与えられた文字列を出力します。システムコールでは引数の検査は重要です。信頼できないアプリを動かそうとするときには特に。攻撃しようという目的で作られたアプリではどんな不正な値が引数に指定されるかわかりません。OS のセキュリティを向上するために引数の検査は欠かせません。

リスト 20.30 アプリからシステムコールを呼び出す (rpn/rpn.cpp)

```
extern "C" int64_t SyscallLogString(LogLevel, const char*);

extern "C" int main(int argc, char** argv) {
    stack_ptr = -1;

    for (int i = 1; i < argc; ++i) {
        if (strcmp(argv[i], "+") == 0) {
            long b = Pop();
            long a = Pop();
            Push(a + b);
            SyscallLogString(kWarn, "+");
        } else if (strcmp(argv[i], "-") == 0) {
            long b = Pop();
            long a = Pop();
            Push(a - b);
            SyscallLogString(kWarn, "-");
        } else {
            long a = atol(argv[i]);
            Push(a);
            SyscallLogString(kWarn, "#");
        }
    }
    if (stack_ptr < 0) {
        return 0;
    }
    SyscallLogString(kWarn, "\\nhello, this is rpn\\n");
    while (1);
    //return static_cast<int>(Pop());
}
```

さて、システムコールの準備は整いましたので、rpn コマンドを改造してシステムコールを使ってみます（リスト 20.30）。アセンブリで実装した SyscallLogString() を C++ のプログラムから呼び出すだけですから何も難しいことはありません。何らかの文字列が表示できれば動作確認としては十分です。試しに、数字が入力されたら # を、演算子が入力されたらその演算子を表示することにしました。例えば rpn 2 5 + と入力したら ##+ と表示されてほしいです。うまくいくでしょうか……。

図 20.9 システムコールによってコンソールに文字列が表示される様子



図 20.9 に `rpn` コマンドを実行した様子を示します。ターミナル画面（黒い画面）は相変わらず変化しませんが、左上のコンソールに `##+` と表示されるようになりました。システムコールをきちんと呼び出すことができた証拠です。大成功！

第 21 章

アプリからウィンドウ

今のところアプリケーションはコンソールに文字列を表示することしかできません。もっとグラフィカルな楽しいアプリを作ってみたくなってきました。ということでアプリケーションに絵を描かせてみたいのですが、何はともあれ専用のウィンドウを作れるようにする必要があります。この章ではアプリケーションがウィンドウを開く仕組みを整えます。

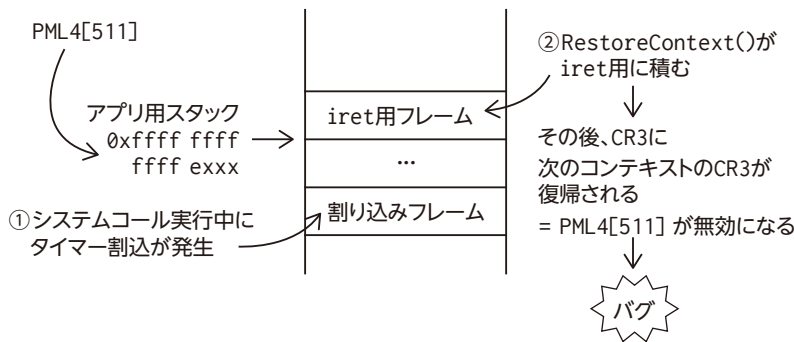
21.1 ISTを設定しよう (osbook_day21a)

コンソールに文字列を表示するシステムコールだけ作りました。うまく動いているのですが、実は重大なバグが残っています。通常はこのバグは潜伏していて、運が悪いと発動してOSのクラッシュに繋がる、とても発見にくいバグです。筆者はこの見つけにくいバグをどうやって見つけたのでしょうか？実はMikanOSを30日目まで作り進めているときに、とあるアプリがうまく動かないという現象から発見に繋がりました。前章の「20.3 TSSを設定しよう」も、この時に発見したバグへ対処するため、後から付け足した節だったりします。

このバグは、アプリがシステムコールを実行中にタイマ割り込みが発生してコンテキストスイッチが起きると発動します。システムコールは `syscall` 命令により呼び出されます。`syscall` 命令はCSとSSをOS用に設定しますが、RSPは変化させません。そのため、システムコールの関数（今は `syscall::LogString()`）しかありませんね……）はRSPがアプリのスタックを指した状態で処理されていきます。ここでタイマ割り込みが発生するとどうなるのでしょうか？

システムコール実行中にタイマ割り込みが発生したとします。このとき、CSがOS用に設定されており、CPUはCPL=0で動作しています。ですので、割り込み発生時点ではCPLが変化せず、TSS.RSP0に設定したスタックは使用されません。すなわち、タイマ割り込みの処理がアプリのスタック上で実行されるのです（図21.1）。これがバグを引き起こします。

図 21.1 アプリスタックでコンテキストスイッチが起こる様子



アプリのスタックは仮想アドレスの後半部分に設定しています。`RestoreContext()`がアプリのコンテキストから次のコンテキストに切り替える処理の一環でCR3レジスタを更新します。CR3は階層ページング構造（PML4）を指しているため、その時点でページマッピングが次のタスク用に更新されるのです。OS用領域（PML4[0]からPML4[255]まで）のマッピングはすべてのコンテキストで共通ですが、アプリ用領域（PML4[256]以降）のマッピングは各コンテキストに固有です。すなわち、CR3の更新後はスタック領域が載っているページが無効化（presentフラグが0となる）されます。その状態で `iret` 命令を実行すると、スタックへのアクセスが発生してページフォルトが起き、OSがクラッシュします。

これがなぜ普段は発生しないのかというと、システムコールの処理がすぐに終わるからです。システムコールを呼び出してから処理が終わるまでの間にタイマ割り込みが発生しなければこのバグは発動しません。このバグは、重たい処理を行うシステムコールを作って利用するほど発動確率が高まります。

このバグを直すには、OS領域に確保したスタック上でタイマ割り込みが処理されるようになればよ

さそうです。筆者が思い付いた方法は 2 つあります。1 つは、システムコール開始時にスタックを入れ替える方法、もう 1 つは、CPU の機能を使ってタイマ割り込み時に自動的にスタックを入れ替える方法です。

システムコール開始時にスタックを入れ替えるには、SyscallEntry() の先頭で RSP を更新してからシステムコール表 (syscall_table) に登録された関数を呼び出す、とすれば実現できそうです。ただ、この方法ではちょっと問題が残っています。それは、SyscallEntry() に処理が移ってから RSP を更新するまでの間にタイマ割り込みが発生した場合を考慮できていないことです。非常に短い時間ですから、その間にタイマ割り込みが起こる確率はとても低いでしょうが、もし起きれば動作がおかしくなります。

CPU の機能を使ってタイマ割り込み時に自動的にスタックを入れ替える方法であれば、確実にタイマ割り込み発生時にスタックを入れ替えることができます。このために備わる IST (interrupt stack table) という仕組みがあります。IST は割り込みハンドラを実行する際に、事前に設定しておいたスタックを必ず使うようにする仕組みです。x86-64 の 64 ビットモードだけで使えます。

IST は TSS に含まれています。TSS といえば前章で、TSS.RSP0 を設定しましたね。IST も RSP0 と非常に似た設定を行うことで有効化できます。システムコール開始時にスタックを入れ替える改造よりもずっと楽です。楽で確実な方法ですから、もちろんこっちを採用することにしました。

リスト 21.1 TSS.IST1 を設定する (segment.cpp)

```
void InitializeTSS() {
    SetTSS(1, AllocateStackArea(8));
    SetTSS(7 + 2 * kISTForTimer, AllocateStackArea(8));

    uint64_t tss_addr = reinterpret_cast<uint64_t>(&tss[0]);
    SetSystemSegment(gdt[kTSS >> 3], DescriptorType::kTSSAvailable, 0,
                    tss_addr & 0xffffffff, sizeof(tss)-1);
    gdt[(kTSS >> 3) + 1].data = tss_addr >> 32;

    LoadTR(kTSS);
}
```

リスト 21.1 に示すように InitializeTSS() を改造して TSS にある IST の設定もするようにしました。「表 20.1 TSS の構造」に示すように IST1 から IST7 まであり、どの割り込みハンドラでどの IST を使うかを選ぶ仕組みになっています。どれを選んでもいいです。ここでは定数 kISTForTimer で希望の IST 番号を表すことにしました。

リスト 21.2 TSS 設定用の便利関数 (segment.cpp)

```
void SetTSS(int index, uint64_t value) {
    tss[index] = value & 0xffffffff;
    tss[index + 1] = value >> 32;
}

uint64_t AllocateStackArea(int num_4kframes) {
    auto [ stk, err ] = memory_manager->Allocate(num_4kframes);
    if (err) {
        Log(kError, "failed to allocate stack area: %s\n", err.Name());
        exit(1);
    }
}
```

```

    }
    return reinterpret_cast<uint64_t>(stk.Frame()) + num_4kframes * 4096;
}

```

TSSの設定で使っているSetTSS()とAllocateStackArea()の実装をリスト21.2に載せます。今までInitializeTSS()に直接書いていたプログラムを関数として切り出しただけです。

リスト 21.3 タイマ割り込みでIST1を使う (interrupt.hpp)

```
cconst int kISTForTimer = 1; // index of the interrupt stack table
```

リスト21.3に示す通り、kISTForTimerは1としています。タイマ割り込みではIST1を使うよ、という意図です。

リスト 21.4 タイマ割り込みでISTを使うように設定する (interrupt.cpp)

```

SetIDTEntry(idt[InterruptVector::kLAPICTimer],
            MakeIDTAttr(DescriptorType::kInterruptGate, 0 /* DPL */,
                        true /* present */, kISTForTimer /* IST */,
                        reinterpret_cast<uint64_t>(IntHandlerLAPICTimer),
                        kKernelCS);

```

割り込み時にISTの仕組みを使うには、割り込みハンドラをIDTへ登録するときにIST番号を設定する必要があります。リスト21.4に設定プログラムを示します。MakeIDTAttr()の4番目にIST番号を設定できるようになっていますので、そこに先ほどの定数を指定します。これで、IDTエントリのISTフィールドに1が設定され、タイマ割り込み発生時にTSS.IST1に設定したスタックが使われるようになります。

この節でやりたかったバグ修正は完了です。これで、システムコール実行中にタイマ割り込みが起きてもバグらなくなったはずですが、そもそもバグを起こすのが難しいため、本当に治ったかどうかは検証できませんが……。修正したMikanOSは今まで通りに動くようですから完成ということにしておきましょう。

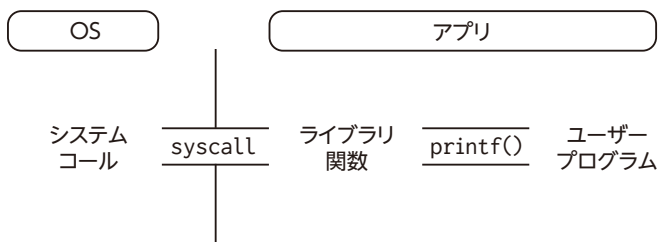
21.2 文字列表示システムコール (osbook_day21b)

絵を描く仕組みを作る前に最低限作っておきたいシステムコールが2つあります。それはターミナルに対する文字列表示とアプリの終了です。まずはターミナルへ文字列を表示するシステムコールを作ってみます。

いまのところの唯一のシステムコールは、ターミナルとは関係のないグローバルな表示領域であるコンソールに文字列を出力するものです。これはこれで便利なのですが、ターミナルで起動したアプリがそのターミナルへメッセージを表示する機能もぜひ欲しいところです。C/C++のプログラムでprintf()を使うとターミナルへ文字列が出力されますよね。同じことをMikanOSでもできるようにします。

実装に入る前にprintf()とシステムコールの関係を説明しておきましょう。両者を整理しておかないと、これからの作業が何をやっているのか理解しにくいからです。図21.2に両者の大まかな関係を示しました。

図 21.2 システムコールとライブラリ関数

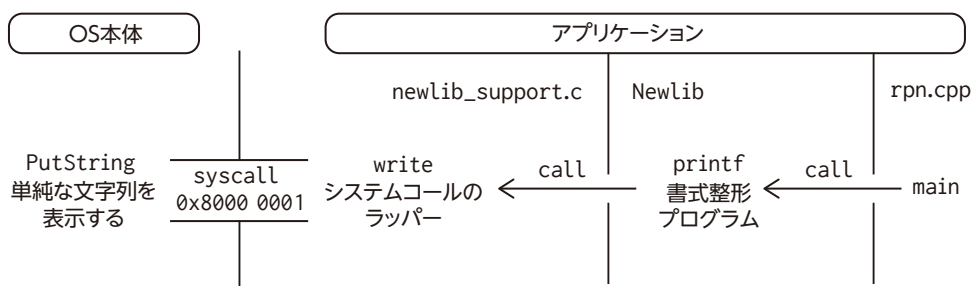


システムコールは OS 本体に備わる機能です。アプリからは `syscall` 命令を通じて呼び出します。システムコールの処理は OS 本体の実行ファイルに含まれます。一方で `printf()` は C 言語の標準ライブラリ関数です。標準ライブラリの関数はいずれもアプリにリンクされて 1 つの実行ファイルを構成します。つまり同じ実行ファイルにアプリの `main()` と `printf()` の両方が含まれるということです。

C の処理系は C のプログラムから `printf()` を呼び出せることを保証しなければなりません。逆にいえば、C のプログラムから `printf()` が呼び出せる限り実現方法は特に決まっていません。`printf()` は文字列に数字などを埋め込む書式整形機能をもちますが、その機能をシステムコール内で実現するのか、C 標準ライブラリ側に持たせるのかは自由なのです。前者の実現方法であれば、`printf()` はシステムコールの薄いラップラとなつて、実質的な処理はすべてシステムコールが担当します。

Linux では、システムコールとして単純な文字列表示だけを提供し、書式整形機能はライブラリ側に持たせる作りになっています。この方式は Linux 以外でも広く用いられていて、本書で採用している Newlib も同様になっています。MikanOS でも同じ仕組みを採用しようと思います (図 21.3)。

図 21.3 Newlib とシステムコールの役割分担



printf() 自体の実装は Newlib に入っています。したがって、私たちが作る必要がある部分は文字列を表示するためのシステムコールと、システムコールと printf() をつなぐための write() です。また、Newlib の printf() を使おうとするといくつかの OS 依存の関数を定義する必要があります。といっても、write() を除いて、単にエラーを返すだけの関数を用意すれば大丈夫です。「第 5 章 文字表示とコンソールクラス」で newlib_support.c を作りましたが、それをアプリ向けにも作るということです。

21.3 システムコールの作成

システムコールとライブラリ関数の役割分担が整理できました。早速システムコールを作ります。そのためにまず、ターミナルへの文字列表示をするために何をすることが必要があるか考えてみます。

システムコールが `syscall` 命令で呼び出されたときを考えます。このとき、システムコールの入り口は1つしかありませんから、そのままではシステムコールを呼び出したアプリを特定できません。アプリを特定できないと表示先のターミナルも特定できません。いやまあ、今はターミナルが1つしかありませんから、決め打ちでそのターミナルへ表示するのも可能です。ただ、将来的にターミナルを複数出したいからでしょうから、複数のターミナルに対応した仕組みを考えることにします。

システムコールからターミナルへ文字列を表示するには、表示対象のターミナルを特定する必要があります。そのためには、アプリとターミナルの対応表を準備しておき、システムコール内でどうにかして呼び出したアプリを特定し、その対応表を検索すればうまくいきそうです。

さて、アプリを識別するにはどうすればいいでしょう。よく考えてみるとタスク ID によってアプリを識別できることが分かります。なぜなら、アプリは個別のタスクとして起動したターミナル上で起動するからです。ある時点で見たとき、ターミナル上で動くアプリは高々1つです。そのため、システムコールを呼び出したタスクの ID が分かればターミナルを特定できるのです。

リスト 21.5 タスク ID とターミナルの対応表 (terminal.cpp)

```
std::map<uint64_t, Terminal*> terminals;

void TaskTerminal(uint64_t task_id, int64_t data) {
    __asm__("cli");
    Task& task = task_manager->CurrentTask();
    Terminal* terminal = new Terminal{task_id};
    layer_manager->Move(terminal->LayerID(), {100, 200});
    active_layer->Activate(terminal->LayerID());
    layer_task_map->insert(std::make_pair(terminal->LayerID(), task_id));
    (*terminals)[task_id] = terminal;
    __asm__("sti");
}
```

タスク ID からターミナルを検索するための対応表 `terminals` をリスト 21.5 のように定義しました。ターミナルのタスクが起動するときに自分自身を対応表へ登録します (`(*terminals)[task_id] = terminal;`)。 `terminals` は様々なタスクから読み書きされる変数ですので、競合しないように `cli/sti` で囲んだ中で変更します。

リスト 21.6 `terminals` の初期化はターミナルを生成する前に (main.cpp)

```
terminals = new std::map<uint64_t, Terminal*>;
const uint64_t task_terminal_id = task_manager->NewTask()
    .InitContext(TaskTerminal, 0)
    .Wakeup()
    .ID();
```

リスト 21.6 に `terminals` の初期化処理を示します。この初期化処理は、1 つめのターミナルタスクが起動する前に置く必要があることに注意してください。

リスト 21.7 ターミナルへの文字列表示システムコール (syscall.cpp)

```
SYSCALL(PutString) {
    const auto fd = arg1;
    const char* s = reinterpret_cast<const char*>(arg2);
    const auto len = arg3;
    if (len > 1024) {
        return { 0, E2BIG };
    }

    if (fd == 1) {
        const auto task_id = task_manager->CurrentTask().ID();
        (*terminals)[task_id]->Print(s, len);
        return { len, 0 };
    }
    return { 0, EBADF };
}
```

リスト 21.7 にこの節の本題となるシステムコール `syscall::PutString()` の実装を示します。このシステムコールが取る 3 つの引数を表 21.1 に掲載します。

表 21.1 `PutString()` システムコールの引数

引数名	値の例	説明
fd	1	ファイルディスクリプタ番号
s	foo bar	表示したい文字列へのポインタ
len	7	NUL 文字を含まないバイト数

fd は文字列を表示する対象を指定する番号です。1 がターミナルを表す番号ということにします。将来的にアプリからファイルを扱えるようになったら、fd に指定する値を変えることでターミナルではなくファイルに文字列を書き込めるように拡張したいなあ、という気持ちです。s と len で文字列へのポインタとバイト数を指定します。NUL 文字で終端されていない文字列にも対応できるようにバイト数を指定する設計にしました。

文字列が長すぎるときにはエラー E2BIG を返すことにします。E2BIG は <cerrno> ヘッダで定義された、Linux などの POSIX 系システムで使われる標準的なエラー番号です。あ、そうそう。結果の値とエラー値を両方返せるように、システムコールの戻り値の型を変えましたよ (リスト 21.8)。

リスト 21.8 システムコールの戻り値の型を変えた (syscall.cpp)

```
namespace syscall {
    struct Result {
        uint64_t value;
        int error;
    };

#define SYSCALL(name) \
    Result name( \
        uint64_t arg1, uint64_t arg2, uint64_t arg3, \
        uint64_t arg4, uint64_t arg5, uint64_t arg6)
```

戻り値の型を表す `syscall::Result` は、システムコールの結果（例えば `PutString()` が書き込んだバイト数）を表す `value` とエラー番号を表す `error` を持った構造体です。エラー番号が 0 以外のときにエラーがあるということにします。この構造体は全体で 128 ビットに収まるため、System V AMD64 ABI の仕様によりシステムコールの戻り値が `RAX:RDX` に乗ります。もし 128 ビットより大きいとスタック上に乗るため、面倒なことになるところでした。システムコールの戻り値がレジスタに収まることで、何の工夫もなくアプリに値を返せます。素晴らしい！

リスト 21.9 システムコールの関数テーブルの型も変えた (`syscall.cpp`)

```
using SyscallFuncType = syscall::Result (uint64_t, uint64_t, uint64_t,
                                           uint64_t, uint64_t, uint64_t);
extern "C" std::array<SyscallFuncType*, 2> syscall_table{
    /* 0x00 */ syscall::LogString,
    /* 0x01 */ syscall::PutString,
};
```

戻り値の型を変えたので `syscall_table` の型も変更しました（リスト 21.9）。それから、既存のシステムコールである `syscall::LogString()` の実装もそれに合わせて改修しました。エラー値を返すようにしただけです。ソースコードは割愛します。

話をもとに戻して `fd` が 1 のときの処理を説明します。この数行の処理が `PutString()` システムコールの本質です。

まず現在実行中のタスクの ID を取得します。このシステムコールはアプリから呼び出されますから、現在実行中のタスクとは、アプリが動作しているターミナルタスクにほかなりません。

ターミナルタスクの ID が取得できたら、`terminals` をその ID で検索することでターミナルオブジェクトへのポインタを得ます。得たポインタを使って `Print()` メソッドを呼び出し、文字列をターミナルへと表示します。

これだけといえばこれだけです。意外と単純。バイト数を指定できる `Terminal::Print()` が無いので実装していきます。

リスト 21.10 `Print()` がバイト数を受け取れるようにする (`terminal.cpp`)

```
void Terminal::Print(const char* s, std::optional<size_t> len) {
    const auto cursor_before = CalcCursorPos();
    DrawCursor(false);

    if (len) {
        for (size_t i = 0; i < *len; ++i) {
            Print(*s);
            ++s;
        }
    } else {
        while (*s) {
            Print(*s);
            ++s;
        }
    }

    DrawCursor(true);
    const auto cursor_after = CalcCursorPos();
```

```

Vector2D<int> draw_pos{ToplevelWindow::kTopLeftMargin.x, cursor_before.y};
Vector2D<int> draw_size{window_->InnerSize().x,
                        cursor_after.y - cursor_before.y + 16};

Rectangle<int> draw_area{draw_pos, draw_size};

Message msg = MakeLayerMessage(
    task_id_, LayerID(), LayerOperation::DrawArea, draw_area);
__asm__("cli");
task_manager->SendMessage(1, msg);
__asm__("sti");
}

```

リスト 21.10 に改造した `Terminal::Print()` を示します。主な改造は 3 つで、引数 `len` を追加し、`len` を用いて文字列を描画するようにし、最後に画面の再描画を行うようにしました。

`len` の型に `std::optional` を採用して「バイト数を指定しない」場合を表現できるようにしました。これは、今までバイト数を指定せずに `Print()` を呼び出していた多くの箇所を修正したくないからです。`len` を `std::optional` 型にしたうえで **リスト 21.11** のようにデフォルト引数を指定すれば、既存の呼び出し箇所を修正する必要がなくなります。

リスト 21.11 `len` にデフォルト引数を指定する (`terminal.hpp`)

```
void Print(const char* s, std::optional<size_t> len = std::nullopt);
```

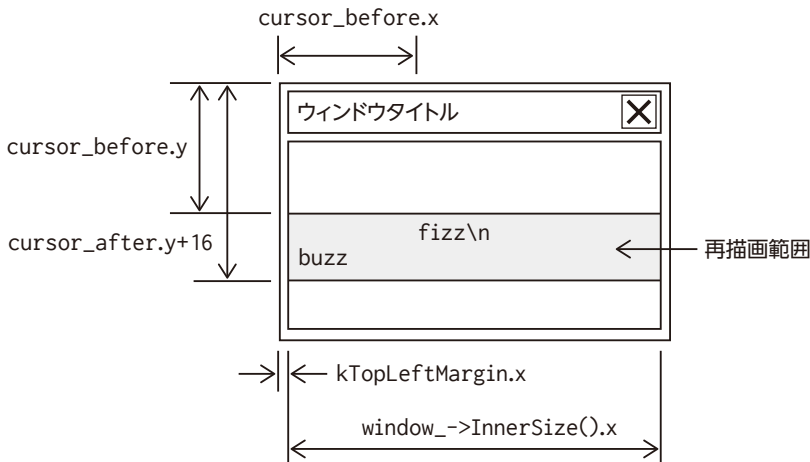
`Print()` の後半で画面の再描画を行います。今まではこの処理がなく、アプリが終了したタイミングで画面を再描画していました。しかし今は、`rpn` コマンドは起動したらずっと終了しないため、それではまずいのです。アプリが終了しなくても文字列をターミナルに表示するために、文字列表示システムコールの中で再描画することにしました。この機能は、長く動くプログラムを作った場合にも役立ちます。

再描画の範囲の計算が複雑なのは、再描画をなるべく狭い範囲に限定するためです。描画範囲を小さくする基本的なアイデアは、文字列表示の前後でのカーソル位置を利用することです。カーソル位置の差を取れば再描画すべき範囲が求まるはずですね。ただ、文字列に改行文字が含まれる場合を考慮しなければならず少し難しくなります。

描画範囲を最小にするのは結構難しいので、上記のプログラムでは妥協して、横方向はウィンドウの幅いっぱい、縦方向はカーソル位置の縦方向の差 +1 行分からなる長方形を再描画範囲としています。

図 21.4 に、`"fizz\nbuzz"` という 2 行の文字列を表示した場合の再描画範囲の計算を示します。行のどこから表示したかに関わらず、2 行全部を再描画範囲としていることが分かりますね。

図 21.4 Print() 内での再描画範囲の計算



21.4 write() の作成

これでシステムコールの準備が整いました。次はアプリ側で `printf()` を呼び出すのに必要な `write()` を実装します。関数のプロトタイプは Linux のターミナルで `man 2 write` として調べるか、`$HOME/osbook/devenv/x86_64-elf/include/sys/unistd.h` に書いてあるのを読んでください。

リスト 21.12 write() はシステムコールを使って文字列を出力する (rpn/newlib_support.c)

```
struct SyscallResult {
    uint64_t value;
    int error;
};
struct SyscallResult SyscallPutString(uint64_t, uint64_t, uint64_t);

ssize_t write(int fd, const void* buf, size_t count) {
    struct SyscallResult res = SyscallPutString(fd, (uint64_t)buf, count);
    if (res.error == 0) {
        return res.value;
    }
    errno = res.error;
    return -1;
}
```

`write()` の実装をリスト 21.12 に示します。システムコールを頑張って作ったおかげで、こちらの関数の実装自体はとても単純です。受け取った 3 つの引数をシステムコールに全部渡し、戻り値のエラーを検査するだけです。`write()` は処理が失敗したら `errno` にエラー値を設定してから `-1` を返すことになっていますのでそんな感じに実装してみました。

`write()` が呼び出している `SyscallPutString()` は、`SyscallLogString()` のコードをコピー

してシステムコール番号を 0x80000001 に変えただけです。ソースコードの紹介はしなくて大丈夫でしょう。

リスト 21.13 `rpn` コマンドで `printf()` を使う (`rpn/rpn.cpp`)

```
long result = 0;
if (stack_ptr >= 0) {
    result = Pop();
}

printf("%ld\n", result);
while (1);
```

リスト 21.13 のように、`rpn` コマンドが `printf()` を呼び出すようにします。もちろん `printf()` ですから様々な書式指定ができます。試しに `long` 型を表示させてみます。

リスト 21.14 `newlib_support.o` をリンクする (`rpn/Makefile`)

```
TARGET = rpn
OBJS = rpn.o syscall.o newlib_support.o
include ../Makefile.elfapp
```

`apps/rpn/Makefile` を編集して、`write()` が定義されている `newlib_support.o` を `rpn` のリンク対象に追加します (リスト 21.14)。

リスト 21.15 C 言語のコンパイルルールを追加 (`Makefile.elfapp`)

```
%.o: %.c Makefile
    clang $(CPPFLAGS) $(CFLAGS) -c $< -o $@
```

このままではうまくコンパイルとリンクができないことが判明したので、`apps/Makefile.elfapp` の方も編集します (リスト 21.15)。さて、ここまでできたらビルドして実行してみます (図 21.5)。ふむふむ、ちゃんと動いているようですね。満足です。

図 21.5 printf() で整数が表示できている様子



21.5 終了システムコール (osbook_day21c)

今の状態ではアプリが終了することができません。main() の処理が終了してしまうと OS 全体が死んでしまうという情けないことになっているためです。よく考えてみると「アプリが終了する」というのは割と難しい処理です。ある CPU 上で動いているアプリが終了するということは、直後からその CPU 上で OS 本体のプログラムが動作を再開するということです。CPU の視点では、リング 3 からリング 0 へと動作モードが変わることにほかなりません。

リング 3 からリング 0 に遷移する仕組みとはシステムコールそのものです。システムコールを使ったアプリの終了処理とは、見方を変えると「永遠に処理が終わらないシステムコールを呼び出すこと」だと言えます。アプリが「終了用システムコール」を呼ぶと、リング 3 からリング 0 に処理が移ったまま、システムコールから処理が返ってくることはありません。そのようなシステムコールを作ればアプリの終了が実現できそうです。この節ではアプリ終了用システムコールを作ってみます。

アプリの終了方法はもう 1 つあります。アプリが CPU 例外が発生させたら、OS 全体ではなくそのアプリだけを終了させるのです。アプリの main() が、終了システムコールではなく return によって終了すると不正なアドレスにジャンプします。そのうち CPU 例外が発生するでしょうから、例外を捕捉したらアプリを止めます。この方法は終了システムコールを呼ばないお行儀の悪いアプリや、意図的に不正な処理をしようとするアプリに対処するためには必要な機能ですが、正常なアプリには不要です。本書では実装せず読者への課題とします。

さて、終了システムコールが他のシステムコールと大きく違うのは、システムコールの処理が終わっても sysret 命令を実行しない点です。その代わりに OS の処理 (Terminal::ExecuteFile()) に実行を戻したいです。実行を戻す、というのはどういうことでしょうか？

ここでアプリが起動されるとき処理を少し思い出してください。アプリ（ELF ファイル）を適切な仮想アドレスへ配置し、エントリポイントへ `retf` 命令によりジャンプするのです。ジャンプの処理は `CallApp()`（リスト 20.4）が担当しています。終了システムコールが無い今の状態では、`CallApp()` を呼び出したら最後、処理が返ってくることはありません。先ほど書いた「処理を戻す」とは、`CallApp()` から戻ってくるようにするという事です。これがこの節の最終目標です。

さて、終了システムコールを作っていきます。3 つ目のシステムコールですので番号は `0x80000002` となります。この番号を検知したら特別な処理へと分岐するように `SyscallEntry()` を改造します（リスト 21.16）。

リスト 21.16 終了システムコール専用の処理へ分岐する (asmfunc.asm)

```
push rax ; システムコール番号を保存

mov rcx, r10
and eax, 0x7fffffff
mov rbp, rsp
and rsp, 0xfffffffffffffff0

call [syscall_table + 8 * eax]
; rbx, r12-r15 は callee-saved なので呼び出し側で保存しない
; rax は戻り値用なので呼び出し側で保存しない

mov rsp, rbp

pop rsi ; システムコール番号を復帰
cmp esi, 0x80000002
je .exit
```

システムコール番号は EAX レジスタにより渡されてきます。しかし EAX レジスタは `call` 命令で呼び出された関数の中で変更されてしまいますから、いったんスタックに保存しておきます。`call` で呼び出したシステムコールの処理が終わったらスタックに保存してある番号を取り出します。このとき、システムコールの戻り値が格納されている RAX と RDX を変更しないように注意します。番号が `0x80000002` であるかをチェックし、等しければ終了システムコール専用の処理 `.exit` へ分岐します。

リスト 21.17 アプリを終了させて OS に戻る (asmfunc.asm)

```
.exit:
mov rsp, rax
mov eax, edx

pop r15
pop r14
pop r13
pop r12
pop rbp
pop rbx

ret ; CallApp の次の行に飛ぶ
```

リスト 21.17 に `.exit` の処理を紹介します。先頭の 2 つの `mov` 命令は、スタックをアプリ用から OS 用に切り替え、`CallApp()` の戻り値を設定しています。これだけ見ても謎だと思いますから後で詳しく説明します。結論を言えば、`.exit` に処理が来た時点で `RAX` には `CallApp()` で保存した OS 用スタック領域のポインタが入っています。したがって、1 つ目の `mov` 命令により、スタックポインタが OS 用スタックを指すようになります。

6 つの `pop` 命令は、各レジスタを `CallApp()` 呼び出し時の状態へと復帰します。RSP は OS 用スタックを指していますから、各 `pop` 命令はアプリ用ではなく OS 用のスタックから値を読み取ることに注意してください。これに合わせて、`CallApp()` の先頭にレジスタ値を保存する処理を追加しました (**リスト 21.18**)。 `pop` と `push` がちょうど逆順に並んでいるのが分かるでしょうか。

リスト 21.18 `CallApp()` の先頭でレジスタ値を保存する (`asmfunc.asm`)

```
global CallApp
CallApp: ; int CallApp(int argc, char** argv, uint16_t ss,
        ;          uint64_t rip, uint64_t rsp, uint64_t* os_stack_ptr);
        push rbx
        push rbp
        push r12
        push r13
        push r14
        push r15
        mov [r9], rsp ; OS 用のスタックポインタを保存

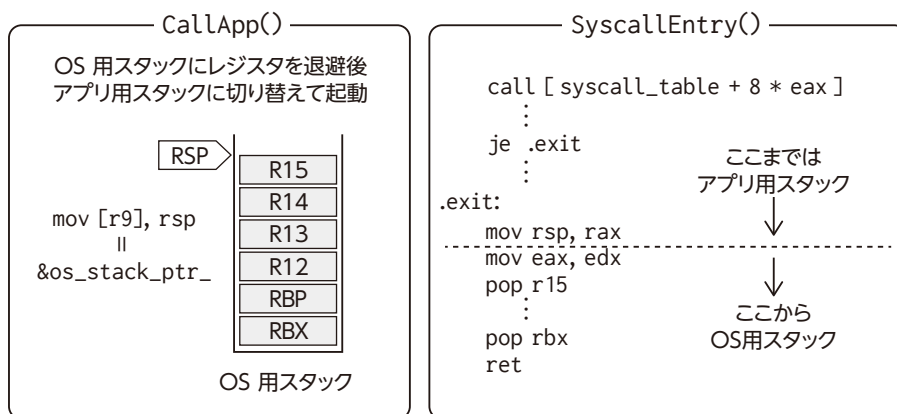
        push rdx ; SS
        push r8  ; RSP
        add rdx, 8
        push rdx ; CS
        push rcx ; RIP
        o64 retf
        ; アプリケーションが終了してもここには来ない
```

`CallApp()` と `SyscallEntry()` の処理を、アプリの起動と終了に絞って時系列でまとめると次の通りです。

- `CallApp`: OS 用スタックにレジスタ群を保存し、
- `CallApp`: アプリ用スタックに切り替え、
- `CallApp`: アプリを起動し、
- `SyscallEntry.exit`: OS 用スタックに切り替え、
- `SyscallEntry.exit`: OS 用スタックからレジスタ群を復帰し、
- `SyscallEntry.exit`: `CallApp()` 呼び出しの次の行に戻る。

`CallApp()` を呼び出した時点のレジスタ値をスタックから復元することで、OS は `CallApp()` の次の行に戻ることができます。図で説明すると **図 21.6** のような仕組みになっています。

図 21.6 OS スタックの保存と復帰



CallApp() の `mov [r9], rsp` に注目します。コメントにあるように、ここで OS 用スタックを指すポインタ (OS 用スタックポインタ) をメモリに保存しています。R9 レジスタには CallApp() の第 6 引数、`os_stack_ptr` が割り当てられていますから、C++ 風にと次のような処理に相当します。

```
*os_stack_ptr = RSP;
```

CallApp() の第 6 引数にスタックポインタを保存するための変数を指定するように、呼び出し側を改造しました (リスト 21.19)。

リスト 21.19 CallApp() にスタックポインタを保存する変数を指定する (terminal.cpp)

```
__asm__("cli");
auto& task = task_manager->CurrentTask();
__asm__("sti");

auto entry_addr = elf_header->e_entry;
int ret = CallApp(argc.value, argv, 3 << 3 | 3, entry_addr,
                 stack_frame_addr.value + 4096 - 8,
                 &task.OSStackPointer());

char s[64];
sprintf(s, "app exited. ret = %d\n", ret);
Print(s);
```

OS 用スタックのスタックポインタを保存するための変数は Task クラスに持たせることにしました。Task クラスはもともと、そのタスク用のスタックポインタを保存するための変数 `Task::stack_ptr_` を持っています。この変数に並べて `Task::os_stack_ptr_` を持たせました。追加した変数は CallApp() 実行時の RSP の値を保存する専用の変数です。

リスト 21.20 OSStackPointer() は変数への参照を返す (task.cpp)

```
uint64_t& Task::OSStackPointer() {
    return os_stack_ptr_;
}
```

Task::OSStackPointer() の実装を **リスト 21.20** に示します。os_stack_ptr_ への参照を返すだけの単純なメソッドです。

Task::stack_ptr_ は、タスク切り替えのたびに SwitchContext() によりその時点の RSP の値が保存されます。そのため、CallApp() によりアプリが起動した後の初めての SwitchContext() 実行時に、アプリ用スタックのスタックポインタが保存されてしまいます。ですから OS 用スタックを復元する値としては使えないのです。

ちなみに CallApp() はもともと、アプリ用コードセグメントとスタックセグメントを別々に指定できました。しかし、syscall を使う以上は 2 つを別々に指定できる意味がありませんので、スタックセグメントのセクタ値だけを指定できるようにしました。これで 1 つ引数を削減でき、6 個目の引数として &task.OSStackPointer() を渡せます。System V AMD64 ABI では引数が 6 個を超えるとちょっと面倒なので、助かりました。ほっ。

21.6 スタックポインタの復帰

スタックポインタを保存する仕組みは分かりました。次は復帰の仕組みを見てみます。OS 用スタックポインタの復帰は SyscallEntry.exit の最初の処理、mov rsp, rax です。といっても、なぜ RAX に OS 用スタックポインタが入っているのか謎ですね。SyscallEntry() のどこをみても、Task::os_stack_ptr_ から値を読み取って RAX に書き込むような処理は見当たりません。その謎を解くカギは 0x80000002 で呼び出される C++ 側の関数 syscall::Exit() にあります。

リスト 21.21 Exit() は OS 用スタックポインタと終了コードを返す (syscall.cpp)

```
SYSCALL(Exit) {
    __asm__("cli");
    auto& task = task_manager->CurrentTask();
    __asm__("sti");
    return { task.OSStackPointer(), static_cast<int>(arg1) };
}
```

syscall::Exit() の実装を **リスト 21.21** に示します。処理はとても単純で、現在のタスクが保持する OS 用スタックポインタの値 (CallApp() 実行時に保存された値) と、システムコールの第 1 引数を返すだけです。

これら 2 つの戻り値をアセンブリ言語の視点から見ると、RAX に task.OSStackPointer() が、RDX に static_cast<int>(arg1) が対応します。基本的に関数の戻り値は RAX で表現するのですが、1 つのレジスタでは足りない場合には RDX が追加で使われることが System V AMD64 ABI で決まっています。

リスト 21.22 アプリを終了させて OS に戻る (asmfunc.asm)

```
.exit:
    mov rsp, rax
    mov eax, edx

    pop r15
    pop r14
    pop r13
    pop r12
    pop rbp
    pop rbx

    ret ; CallApp の次の行に飛ぶ
```

リスト 21.22 に SyscallEntry.exit の処理を再掲します。直前にある call 命令と je .exit の間では RAX と RDX を変更していないため、.exit に来たときには RAX:RDX に syscall::Exit() の戻り値が設定されたままの状態になっています。ということで、2 つの mov により、スタックポインタが OS 用に復元され、EAX には syscall::Exit() の第 1 引数が設定されることになります。

リスト 21.23 アプリから終了システムコールを呼び出す (rpn/rpn.cpp)

```
long result = 0;
if (stack_ptr >= 0) {
    result = Pop();
}

printf("%ld\n", result);
SyscallExit(static_cast<int>(result));
```

リスト 21.23 にアプリから終了システムコールを呼び出している箇所を示します。終了システムコールの引数はアプリの終了コード (exit code) です。多くの OS では終了コードが 0 なら正常終了、0 以外ならエラー終了、という意味になります。MikanOS では今のところアプリの終了コードに意味はなく、ただ「app exited. ret = %d」と表示されるだけです。終了コードが変化することを確認するため、逆ポーランド記法の計算結果を終了コードに指定してみました。

Linux で C/C++ でプログラミングするときは main() の中で return 0; とするだけでアプリを正常に終了することができます。exit(0); を書く必要はありません。しかし MikanOS のアプリでは必ず SyscallExit(0); の実行が必要です。さもなくば OS 全体が死んでしまいます。

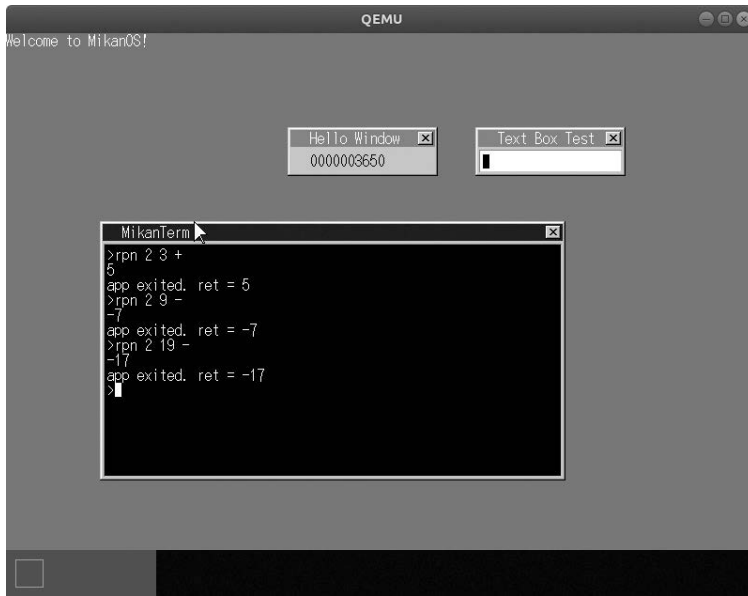
この差は main() が呼ばれる仕組みの差から来ています。MikanOS では main() が最初に呼ばれるため、必要な初期化や後処理は其中で完結させねばなりません。Linux などでは最初にスタートアップルーチン^{*1}というものが呼ばれ、その中から main() が呼び出されます。スタートアップルーチンには main() から戻ってきたときに exit() を呼び出す処理が書かれているのです。

MikanOS では main() が return 文で終了してはいけません。そのため main() の戻り値の型を int にする意味がないどころか、return 文で値を返さない場合に警告が出てしまい邪魔です (コンパイラは SyscallExit() を呼ぶとそこでアプリが終了することなど知らないのです)。そのため、戻り値型

* 1 典型的には crt0.o というファイルにスタートアップルーチンが含まれます。

を void に変更しておきました。

図 21.7 rpn コマンドが様々な終了コードで終了する様子



これで終了システムコールが完成したはずですが、rpn コマンドの動作確認をしました（図 21.7）。アプリがきちんと終了してターミナルに処理が戻ってくるようになりました。OS を再起動させなくても何度もコマンドを実行できます。終了コードもちゃんと計算結果が反映されているようです。大成功！

21.7 コード整理 (osbook_day21d)

せっかく終了システムコールを作りましたが rpn コマンドしか改造していませんでした。large コマンド^{*2}の方もちゃんと終了システムコールを呼ぶようにしましょう。そのためには apps/rpn/syscall.asm に含まれる、システムコールと C++ を結びつけるための関数群が large の方にも必要です。ということでそのファイルを apps/syscall.asm へと移動します。

移動にともなって apps/rpn/Makefile の OBJS から syscall.o を抜き、アプリ間で共通の apps/Makefile.elfapp をリスト 21.24 のように変更しました。

リスト 21.24 アプリ共通で syscall.o をリンクする (Makefile.elfapp)

```
OBJS += ../syscall.o ../newlib_support.o
```

*2 large コマンドはページングの検証のために作った、とても大きなグローバル変数を利用するアプリです。

Makefile.elfapp は ELF アプリのビルドで共通に利用されます。ここで OBJS を変更するようにすれば、今後アプリが増えても自動的に syscall.o がリンクされるようになります。Newlib をアプリで使うために必要になる newlib_support.o もついでに、apps/rpn/ の中から apps/ へと移動しておきました。

リスト 21.25 システムコール関数の宣言集 (syscall.h)

```
#include <cstdint>
#include "../kernel/logger.hpp"

extern "C" {

struct SyscallResult {
    uint64_t value;
    int error;
};

SyscallResult SyscallLogString(LogLevel level, const char* message);
SyscallResult SyscallPutString(int fd, const char* s, size_t len);
void SyscallExit(int exit_code);

} // extern "C"
```

さらについで作業として、システムコールと C++ を結びつける関数群のプロトタイプ宣言を集めたヘッダファイル syscall.h を作りました (リスト 21.25)。このように共通化しておけば、アプリやシステムコールが今後増えたときに各アプリのソースコードで個別にプロトタイプ宣言を書く必要がなくなります。

リスト 21.26 large コマンドのソースコード (large/large.cpp)

```
#include <cstdlib>
#include "../syscall.h"

char table[3 * 1024 * 1024];

extern "C" void main(int argc, char** argv) {
    SyscallExit(atoi(argv[1]));
}
```

syscall.h を利用して large コマンドで終了システムコールを呼ぶように改造しました (リスト 21.26)。main() はもはや値を返しませんから、rpn のときと同じように戻り値型を void にしました。

21.8 ウィンドウを開く (osbook_day21e)

前置きが意外と長くなってしまいました。やっと本章でやりたかったことに手を付けられそうです。それはウィンドウを開き、絵を描くことです。この節ではまずウィンドウを開くシステムコールを作ります。ウィンドウを開く事自体は今までもやっています。それをシステムコールとして実装するだけで良いはずです。

リスト 21.27 ウィンドウを開くシステムコール (syscall.h)

```

SyscallResult SyscallLogString(LogLevel level, const char* message);
SyscallResult SyscallPutString(int fd, const char* s, size_t len);
void SyscallExit(int exit_code);
SyscallResult SyscallOpenWindow(int w, int h, int x, int y, const char* title);

```

リスト 21.27 にウィンドウを開くシステムコール `syscall::OpenWindow()` のプロトタイプ宣言を示します。引数 `w` と `h` はウィンドウの横幅と高さです。 `x` と `y` はウィンドウの初期位置です。いずれもピクセル単位で指定します。 `title` はウィンドウのタイトル文字列です。

リスト 21.28 ウィンドウを開くシステムコール (syscall.cpp)

```

SYSCALL(OpenWindow) {
    const int w = arg1, h = arg2, x = arg3, y = arg4;
    const auto title = reinterpret_cast<const char*>(arg5);
    const auto win = std::make_shared<ToplevelWindow>(
        w, h, screen_config.pixel_format, title);

    __asm__("cli");
    const auto layer_id = layer_manager->NewLayer()
        .SetWindow(win)
        .SetDraggable(true)
        .Move({x, y})
        .ID();
    active_layer->Activate(layer_id);
    __asm__("sti");

    return { layer_id, 0 };
}

```

リスト 21.28 に実装を示します。引数にしたがってウィンドウ (`ToplevelWindow`) を生成し、レイヤマネージャに登録します。新しく開いたウィンドウはアクティブ化することにしました。ウィンドウを作った後にウィンドウを操作できるよう、システムコールの戻り値としてレイヤ ID を返します。今後増やすウィンドウ関係のシステムコールにはこのレイヤ ID を指定するようになるつもりです。

リスト 21.29 winhello コマンドのソースコード (winhello/winhello.cpp)

```
#include "../syscall.h"

extern "C" void main(int argc, char** argv) {
    SyscallOpenWindow(200, 100, 10, 10, "winhello");
    SyscallExit(0);
}
```

リスト 21.29 には、新しく増やしたコマンド winhello のソースコードを示します。単にウィンドウを開くだけの単純なアプリです。このコマンドを起動すると図 21.8 のようになりました。ウィンドウがちゃんと表示されていますね。よしよし、完成です。

図 21.8 winhello コマンドがウィンドウを開いた様子



21.9 ウィンドウに文字を書く (osbook_day21f)

ウィンドウが開けるようになりましたが、ウィンドウに文字さえ書けないのはつまらないです。絵を描くのが最終的な目標ですが、とりあえず文字を出せるようにしてみます。

リスト 21.30 WinWriteString() はウィンドウに文字を描く (syscall.cpp)

```
SYSCALL(WinWriteString) {
    const unsigned int layer_id = arg1;
    const int x = arg2, y = arg3;
    const uint32_t color = arg4;
    const auto s = reinterpret_cast<const char*>(arg5);

    __asm__("cli");
    auto layer = layer_manager->FindLayer(layer_id);
    __asm__("sti");
    if (layer == nullptr) {
        return { 0, EBADF };
    }

    WriteString(*layer->GetWindow()->Writer(), {x, y}, s, ToColor(color));
    __asm__("cli");
    layer_manager->Draw(layer_id);
    __asm__("sti");

    return { 0, 0 };
}
```

ということでウィンドウに文字を描くための `syscall::WinWriteString()` システムコールを作りました (リスト 21.30)。`layer_id` は描画先のウィンドウを含むレイヤ ID、`x`、`y` はウィンドウ左上を基準とする描画位置です。`color` が文字色で `s` が文字列へのポインタです。

指定されたレイヤ ID を使ってレイヤを取得して、そのレイヤが持つウィンドウに文字列を描画します。描画結果を画面に反映するために、`layer_manager->Draw()` を用いてレイヤを再描画しておきます。

システムコール本体を作るほかに、`syscall_table` への登録や `apps/syscall.asm` への関数追加も行いました。他のシステムコールと同じ作業なので紙面では紹介しません。

リスト 21.31 winhello コマンドの中で文字を描画する (winhello/winhello.cpp)

```
#include "../syscall.h"

extern "C" void main(int argc, char** argv) {
    auto [layer_id, err_openwin]
        = SyscallOpenWindow(200, 100, 10, 10, "winhello");
    if (err_openwin) {
        SyscallExit(err_openwin);
    }

    SyscallWinWriteString(layer_id, 7, 24, 0xc00000, "hello world!");
    SyscallWinWriteString(layer_id, 24, 40, 0x00c000, "hello world!");
}
```

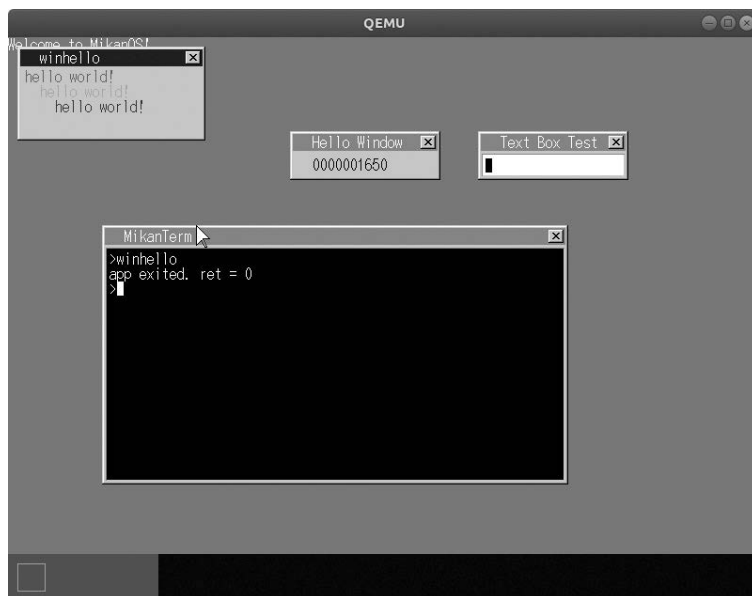
```

SyscallWinWriteString(layer_id, 40, 56, 0x0000c0, "hello world!");
SyscallExit(0);
}

```

作成したシステムコールを呼び出すように winhello コマンドを改造しました（リスト 21.31）。まず、SyscallOpenWindow() の戻り値を保存するように改造しました。なぜなら、戻り値（レイヤ ID）が SyscallWinWriteString() の実行に必要なからです。エラーなくウィンドウを開けたら、位置や色を変えて 3 つの文字列を描画します。

図 21.9 ウィンドウに文字列を描画してみた様子



改造したコマンドを実行した結果を図 21.9 に示します。ちゃんと 3 つの文字列が色違いで表示されました。こんなに少ない変更で文字列が表示できるようになって、筆者は満足です。

第22章

グラフィックと イベント (1)

ウィンドウを開いてやりたかったのはグラフィカルな処理です。ウィンドウに自由に絵が描けるようになると表現力が高まり、見た目に楽しいものが作れるようになります。モチベーションの維持にも効果的です。実はウィンドウに絵を描くのはかなり単純な処理なのでさくっとやってみます。

この章の後半ではアプリケーションがキーボードの入力を受け付ける仕組みを作ろうと思います。この仕組みによってアプリはキー入力があるまで待機し、適切に終了処理ができるようになります。キーボード入力の仕組みを使うとマウスやタイマのイベントも受け取れるようになり、様々な応用ができるようになります。

22.1 exit()を使う (osbook_day22a)

この章の本題に入る前に、ちょっと気になった部分を直したいと思います。それはアプリの終了方法です。今は `SyscallExit()` を呼び出してアプリを終了させていますが、C 標準ライブラリにある `exit()` を使うようにしたいです。なぜなら `exit()` は C 言語の規格にしたがって正しく終了処理を行ってくれるからです。

C 言語の規格にはプログラム終了時に関数を呼び出す仕組みがあります。 `atexit()` を使って関数を登録しておくと、 `exit()` の実行時 (`main()` の中で呼ぶ場合に加え、スタートアップルーチンが呼ぶ場合も含む) にそれらの関数が呼ばれます。どこでプログラムが終了しても必ず実行したい後処理用の関数を登録しておけば実行漏れがなくなります。 `atexit()` を明示的に使っているアプリは今のところありませんが、標準ライブラリ関数の中から呼ばれないとも限りません。きちんと `exit()` を使うようにしておきます。

リスト 22.1 exit() を使ってアプリを終了する (winhello/winhello.cpp)

```
SyscallWinWriteString(layer_id, 40, 56, 0x0000c0, "hello world!");
exit(0);
```

早速、**リスト 22.1** のように `SyscallExit()` の代わりに `exit()` を使うように改造しました。特に難しいことはないのでソースコードは紹介しませんが、 `winhello` 以外のアプリでも同様の改造をしておきました。

リスト 22.2 _exit() を用意する (apps/newlib_support.c)

```
void _exit(int status) {
    SyscallExit(status);
}
```

ただ、この改造をただけではリンク時に `_exit` が無いと怒られてしまいます。Newlib では `exit()` を使うためには `_exit()` を作る必要があるのです。**リスト 22.2** のように定義しました。中身が単純すぎてちょっと拍子抜けしちゃいましたか？

そうそう、あんまり本質ではないのですが重要な変更を行いました。それは `apps/newlib_support.c` から `apps/syscall.h` をインクルードする変更です。今まではインクルードせず、必要になったシステムコールのプロトタイプ宣言をその都度書いていました。しかし、今後もっとシステムコールを `newlib_support.c` の中から呼び出すことになると思いますから、今の段階で `syscall.h` をインクルードする方式にしておきたいと思いました。

ただインクルードするだけじゃないのか、という声が聞こえてきそうです。ここでわざわざ取り上げたのは、C と C++ から共通にインクルードできるヘッダファイルの書き方を紹介しようと思ったからで

す。リスト 22.3 に `syscall.h` の全体を示しました。

リスト 22.3 C と C++ から共通に使えるヘッダファイル (`apps/syscall.h`)

```

#ifdef __cplusplus
#include <cstdint>
#include <cstdint>

extern "C" {
#else
#include <stddef.h>
#include <stdint.h>
#endif

#include "../kernel/logger.hpp"

struct SyscallResult {
    uint64_t value;
    int error;
};

struct SyscallResult SyscallLogString(enum LogLevel level, const char* message);
struct SyscallResult SyscallPutString(int fd, const char* s, size_t len);
void SyscallExit(int exit_code);
struct SyscallResult SyscallOpenWindow(int w, int h, int x, int y, const char* title);
struct SyscallResult SyscallWinWriteString(unsigned int layer_id, int x, int y,
                                           uint32_t color, const char* s);

#ifdef __cplusplus
} // extern "C"
#endif

```

C と C++ で共通に使うヘッダファイルを作る際に問題になるのが、C++ の特徴である関数名マングリングです。C++ では引数の型や個数が異なる同名の関数を定義できるように、純粋な関数名に引数の型名（と対応する呪文のような文字列）をくっつけたようなシンボルに変換されます。この機能はCにはありませんから、ソースコード上は同じプロトタイプ宣言でもCとC++で意味が変わってしまうのです。それを防ぐのが `extern "C"` の効果なのです。これを関数プロトタイプの前に置くか `extern "C" {}` として囲むことで、関数名がマングリングされるのを防げます。

`extern "C"` という書き方はC++でしかできませんので、`syscall.h` がC++からインクルードされたときだけ `extern "C"` を有効化する、というような仕掛けを入れなければなりません。これは条件コンパイルという仕組みでできます。そのときに使うのが `__cplusplus` というマクロです。このマクロはC++としてコンパイルするときに定義され、Cのコンパイルでは定義されませんから、マクロが定義されていたら `extern "C"` を有効化すればいいことになります。マクロが定義されていたら、というチェックは `#ifdef` でできます。

それから、Cでは列挙体の型名の前に `enum` を、構造体の型名の前に `struct` を付けなくてははいけません。C++では付けても付けなくてもいいことになっていますので、Cに合わせて `enum` と `struct` を書き足しました。

リスト 22.4 enumを付け足す (logger.hpp)

```
int Log(enum LogLevel level, const char* format, ...);
```

apps/syscall.hの中でLogLevelの定義を使いたかったので、kernel/logger.hppを少し改造してC言語からもインクルードできるようにしました（リスト22.4）。Log()などの関数がCから使える必要はありませんからextern "C"を付けたりする作業は不要です。

さて、このくらいで改造は終了です。お疲れ様でした。見た目の変化は何もありませんので動作写真は載せないことにします。

22.2 点を描く (osbook_day22b)

この章ではウィンドウに絵を描く機能を付けていきますよ。まず最も単純な点の描画からやってみましょう。指定した座標に指定した色の点を打つだけです。点の縦横の大きさも指定できるようにしましょうか。点というよりは長方形を描くシステムコールになりますね。

リスト 22.5 長方形を塗りつぶすシステムコール (syscall.h)

```
struct SyscallResult SyscallWinFillRectangle(  
    unsigned int layer_id, int x, int y, int w, int h, uint32_t color);
```

リスト22.5にシステムコールのプロトタイプ宣言を示します。何も難しいことはありません。長方形の左上の座標をx、yで指定し、横と縦の大きさをw、hで指定します。colorは塗りつぶす色です。

システムコール本体はsyscall::WinWriteString()とほぼ同じ中身になりそうです。関数全体をコピーして少し改造すれば簡単ですね。レイヤIDからウィンドウを持ってくる処理まで共通で、ウィンドウに文字列を書く処理（WriteString()）を、長方形を描く処理（FillRectangle()）に書き換えるだけです。

というように、同類の処理を増やすときにコピペするのは楽なのですが欠点が多いので避けたいところです。共通な処理はなるべく共通化しながら、差分だけ別に作れる構造にしたいです。共通な処理のバグを直したり機能変更したりする際に、共通化してあることで同じ修正をいくつかの関数に適用する必要がなくなります。素敵ですね^{*1}。

もし同じようなコピペを他にする予定がないなら無理に共通化しなくてもまあ良いかなと思います。ですが、ウィンドウに何かを描画するシステムコールは今後も増やす予定ですので、この機会に共通化を行います。

*1 すべてのコピペが悪ではありません。処理が似ているからといって必ずしも共通化すべき、というわけでもないのがプログラミングの難しいところ。

リスト 22.6 シンプルになったウィンドウ描画システムコール群 (syscall.cpp)

```
SYSCALL(WinWriteString) {
    return DoWinFunc(
        [](Window& win,
           int x, int y, uint32_t color, const char* s) {
            WriteString(*win.Writer(), {x, y}, s, ToColor(color));
            return Result{ 0, 0 };
        }, arg1, arg2, arg3, arg4, reinterpret_cast<const char*>(arg5));
}

SYSCALL(WinFillRectangle) {
    return DoWinFunc(
        [](Window& win,
           int x, int y, int w, int h, uint32_t color) {
            FillRectangle(*win.Writer(), {x, y}, {w, h}, ToColor(color));
            return Result{ 0, 0 };
        }, arg1, arg2, arg3, arg4, arg5, arg6);
}
```

リスト 22.6 に 2 つのウィンドウ描画システムコールの定義を示します。1 つは共通な処理を `DoWinFunc()` に抜き出したことでシンプルになった `syscall::WinWriteString()` です。もう 1 つは新たに付け足した長方形描画用の `syscall::WinFillRectangle()` です。ちょっとコードが分かりにくいかもしれませんが詳しく説明します。

どちらの関数も 1 つの `return` 文だけを持ちます。この `return` 文では `DoWinFunc()` という関数を呼び出します。この関数はレイヤ ID からウィンドウを取得するといった共通の処理を行う関数として新たに作ったものです。実装は後で紹介します。

システムコール毎の差分（文字列を書くのか長方形を描画するのか）は `DoWinFunc()` の第 1 引数で与える関数で表現します。`DoWinFunc()` の第 1 引数に何を指定しているかなーと見てみると、どちらのシステムコールでもラムダ式を与えています。ラムダ式は「6.3 PCI デバイスの探索」でも出てきた、無名の関数を定義する C++ の機能でしたね。そのラムダ式は第 1 引数に渡されてきたウィンドウに対し、残りの引数を利用して文字や長方形を描画します。

リスト 22.7 ウィンドウ描画処理の共通部分を担当する `DoWinFunc()` (syscall.cpp)

```
namespace {
    template <class Func, class... Args>
    Result DoWinFunc(Func f, unsigned int layer_id, Args... args) {
        __asm__("cli");
        auto layer = layer_manager->FindLayer(layer_id);
        __asm__("sti");
        if (layer == nullptr) {
            return { 0, EBADF };
        }

        const auto res = f(*layer->GetWindow(), args...);
        if (res.error) {
            return res;
        }
    }
}
```

```

__asm__("cli");
layer_manager->Draw(layer_id);
__asm__("sti");

return res;
}
}

```

リスト 22.7 は共通処理を担当する `DoWinFunc()` の定義です。関数テンプレートとして作ってみました。この関数は、第 2 引数 `layer_id` で指定されたレイヤを検索し、そのレイヤが持つウィンドウを取得します。第 1 引数で指定された関数 `f` に、取得したウィンドウと第 3 以降の引数を与えて呼び出します。最後にウィンドウを再描画して処理を終えます。

この関数定義の大きな特徴は可変引数テンプレート^{*2}という、引数の数が可変の関数を定義するための C++ の機能を使っているところです。`DoWinFunc()` の第 3 引数に書いてある `Args... args` は、0 個以上の任意の数の引数に対応します。`DoWinFunc()` に指定した引数は、1 つ目が `f` に、2 つ目が `layer_id` に、残りが `args` に割り当てられるという仕組みです。例えば `DoWinFunc(arg1, arg2, arg3, arg4);` と呼び出したとすると、`f=arg1`、`layer_id=arg2`、`args={arg3,arg4}` となります。

そして、中央にある `f` の第 2 引数に渡している `args...` は、`args` を展開する書き方です。`args={arg3,arg4}` のとき、`f(*layer->GetWindow(), args...)` は `f(*layer->GetWindow(), arg3, arg4)` という意味になります。

リスト 22.8 星空を描く stars コマンド (stars/stars.cpp)

```

#include <cstdlib>
#include <random>
#include "../syscall.h"

static constexpr int kWidth = 100, kHeight = 100;

extern "C" void main(int argc, char** argv) {
    auto [layer_id, err_openwin]
        = SyscallOpenWindow(kWidth + 8, kHeight + 28, 10, 10, "stars");
    if (err_openwin) {
        exit(err_openwin);
    }

    SyscallWinFillRectangle(layer_id, 4, 24, kWidth, kHeight, 0x000000);

    int num_stars = 100;
    if (argc >= 2) {
        num_stars = atoi(argv[1]);
    }

    std::default_random_engine rand_engine;
    std::uniform_int_distribution x_dist(0, kWidth - 2), y_dist(0, kHeight - 2);
    for (int i = 0; i < num_stars; ++i) {
        int x = x_dist(rand_engine);
        int y = y_dist(rand_engine);
        SyscallWinFillRectangle(layer_id, 4 + x, 24 + y, 2, 2, 0xffff100);
    }
}

```

```

    }

    exit(0);
}

```

長方形を描くシステムコールを使って何かアプリを作ってみましょう。「30日でできる! OS自作入門」(参考文献 [1]) で紹介されている stars コマンドが描く星空が素敵だったので、真似して作ってみました (リスト 22.8)。このコマンドは、黒い夜空に輝くたくさんの星を描きます。本家のコマンドは 50 個固定で星を描画していましたが、本書の stars コマンドはコマンドライン引数で星の数を調整できます。

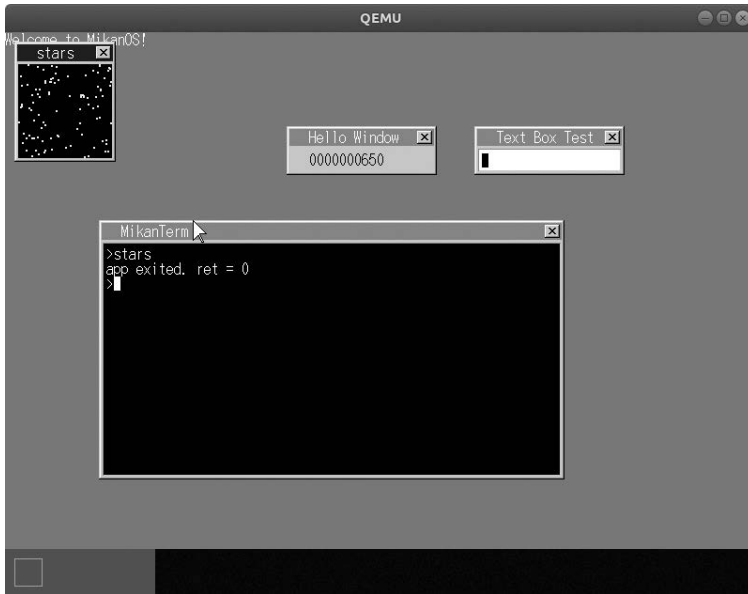
実装を紹介します。まずウィンドウを開き、先ほど作ったシステムコールを使って黒く塗りつぶします。これが夜空となります。単なる点の描画システムコールではなく、長方形描画ができるように作ったのが早速役に立ちましたね! 夜空を準備できたら、次にコマンドライン引数を読み取って星の数 num_stars を設定します。コマンドライン引数を省略したらデフォルトで 100 個の星を描きます。最後に指定された数だけループを回し、星を描いていきます。

ループの前に作っている 3 つの変数 rand_engine、x_dist、y_dist は、星をまばらに散らすのに使う **乱数** (random number) を得るための準備です。乱数とは、規則性がない (ように見える) まばらの数値のことです。サイコロは身近にある乱数生成器の例です。サイコロを振って出る目はまばらですよ。それを乱数と呼びます。

計算機の中でサイコロを振ることはできませんが、計算によって擬似的に乱数を作り出すことができます。rand_engine はまさに乱数を作り出す疑似乱数生成器です。乱数生成器から得られる値の範囲を決めるのが x_dist と y_dist です。x_dist は星の横方向の位置を決めるもので、0 以上 kWidth-2 以下の範囲を表します。y_dist は縦方向についてのものです。乱数生成器 rand_engine と値の範囲を決める x_dist を組み合わせて使うと、希望した値の範囲の乱数が手に入ります。

ループの中ではまず、横と縦の位置を乱数で決めます。次に、長方形描画システムコールを使ってその位置に星を描きます。星の大きさは 2×2 としてみました。

図 22.1 stars コマンドで星空を描いてみた



このコマンドを実行した様子を図 22.1 に示します。おお、なんと美しい星空！ 擬似乱数生成器の意思により描かれた世界で唯一の星空です。すみません大げさですね。とりあえず長方形を描画するシステムコールがうまく動くことが分かったのでこの節はおしまいです。

22.3 タイマ値の取得 (osbook_day22c)

星を描くプログラムの動きを考えると、とても非効率な気がしてきました。なぜかという、星を 1 個描くたびに画面を再描画しているためです。星をすべて描き終えて最後に 1 回だけ再描画させる方が圧倒的に速いはず。ウィンドウに描画を行うシステムコールを改造して、再描画しないオプションを指定できるようにしようと思います。

「最適化を行うにはまず計測」の原則にのっとり、システムコールを改造する前に時間を計れるようにします。タイマ値 (timer_manager->CurrentTick()) を取得するためのシステムコールがあればアプリ側で処理時間を計れますね。ということで、さくっと作ってみます。

リスト 22.9 タイマ値と周波数を取得するシステムコール (syscall.cpp)

```
SYSCALL(GetCurrentTick) {
    return { timer_manager->CurrentTick(), kTimerFreq };
}
```

リスト 22.9 に作成したシステムコールを示します。非常に簡単な実装ですね。他のシステムコールは主となる値とエラー値の組を返しますが、このシステムコールは必ず成功するためエラー値の代わりに周波数を返すことにしました。周波数だけを取得するシステムコールを別に作るよりすっきりすると

思います。

リスト 22.10 星を描く時間を計る (stars/stars.cpp)

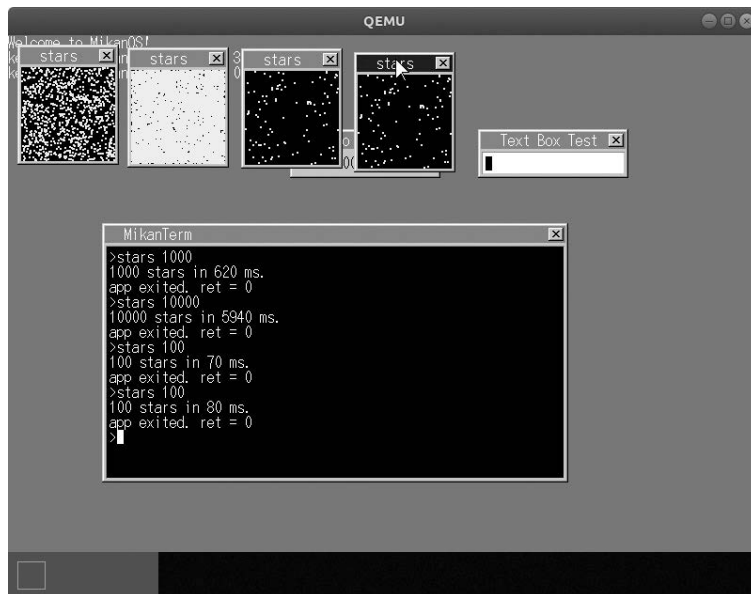
```
auto [tick_start, timer_freq] = SyscallGetCurrentTick();

std::default_random_engine rand_engine;
std::uniform_int_distribution x_dist(0, kWidth - 2), y_dist(0, kHeight - 2);
for (int i = 0; i < num_stars; ++i) {
    int x = x_dist(rand_engine);
    int y = y_dist(rand_engine);
    SyscallWinFillRectangle(layer_id, 4 + x, 24 + y, 2, 2, 0xffff100);
}

auto tick_end = SyscallGetCurrentTick();
printf("%d stars in %lu ms.\n",
      num_stars,
      (tick_end.value - tick_start) * 1000 / timer_freq);
```

作成したシステムコールを使い、星を描くループにかかる時間を計てみます。リスト 22.10 のようにループの前後でタイマ値を取得し、その差をミリ秒に変換して表示するようにしました。

図 22.2 星の数を変えて時間を計る



筆者の QEMU 環境で星の数を変えて何度か実行した結果を図 22.2 に示します。100 個の星を描画するのにかった時間は約 0.08 秒、1000 個では約 0.62 秒、10000 個では約 5.94 秒となりました。大体、星の数と時間が比例しているようです。予想通りです。再描画にかかる時間がどの程度なのか、次の節で最適化した後で再計測すれば分かるでしょう。ちなみに、ミリ秒の下 1 桁がすべて 0 できれいに揃っているのは、`timer_manager->CurrentTick()` が 10 ミリ秒単位でしか時間を計れないからです。

22.4 ウィンドウ描画の最適化 (osbook_day22d)

時間を計れるようにしたので最適化をやっていきます。ウィンドウに文字や絵を描くすべてのシステムコールに、統一的に再描画をしないオプションを付け、再描画だけするシステムコールを作ることになります。bool no_redraw みたいな引数を各システムコールに追加すれば簡単にできそうですね。と思ったのですが、syscall::WinFillRectangle() は 6 つの引数をすべて使ってしまっており、追加する余裕がありません。引数を追加せずになんとか出来ないでしょうか？

layer_id は運の良いことに 32 ビット整数です*³。引数の受け渡しに使うレジスタは 64 ビット幅ですから上位 32 ビットが空いていることになります。この部分をフラグとして使えば、引数を追加しなくても対応可能です。さらに良いことに、引数を追加しないので改造部分が少なくて済む気がします。

リスト 22.11 レイヤ ID の上位 32 ビットをフラグに使う (syscall.cpp)

```
template <class Func, class... Args>
Result DoWinFunc(Func f, uint64_t layer_id_flags, Args... args) {
    const uint32_t layer_flags = layer_id_flags >> 32;
    const unsigned int layer_id = layer_id_flags & 0xffffffff;

    __asm__("cli");
    auto layer = layer_manager->FindLayer(layer_id);
    __asm__("sti");
    if (layer == nullptr) {
        return { 0, EBADF };
    }

    const auto res = f(*layer->GetWindow(), args...);
    if (res.error) {
        return res;
    }

    if ((layer_flags & 1) == 0) {
        __asm__("cli");
        layer_manager->Draw(layer_id);
        __asm__("sti");
    }

    return res;
}
```

リスト 22.11 に改造版の DoWinFunc() を示します。レイヤ ID を受けとる引数を uint64_t layer_id_flags として、フラグが付くことを明確にしました。その上位 32 ビットをフラグとして変数 layer_flags に取り出します。

そして、今回の改造の主目的は再描画を抑制することです。layer_flags のビット 0 が 1 なら再描画をしないという if 文を追加しました。この仕様であれば互換性を保てますからね。逆に、ビット 0 が 1 なら再描画する、という仕様にしてしまうと、ウィンドウに描画するアプリをすべて改造しないと正しく表示されなくなってしまいます。

*3 正確に言えば unsigned int が 32 ビット幅とは限りません。ただ仮に 64 ビット幅だったとしても 32 ビットで収まる数のレイヤが生成できれば十分なので、上位 32 ビットをフラグに使って問題ありません。

リスト 22.12 再描画システムコール (syscall.cpp)

```
SYSCALL(WinRedraw) {  
    return DoWinFunc(  
        [](Window&) {  
            return Result{ 0, 0 };  
        }, arg1);  
}
```

再描画だけを行うシンプルなシステムコール `syscall::WinRedraw()` を追加しました（**リスト 22.12**）。実はこんなシステムコールを新しく作らなくても、`WinFillRectangle()` に長方形の大きさとして 0×0 を指定すれば代用可能です。ですが、分かりやすさのために再描画専用のシステムコールを新設することにしました。

リスト 22.13 再描画しないオプションの定数化と再描画システムコール (syscall.h)

```
#define LAYER_NO_REDRAW (0x00000001ull << 32)  
struct SyscallResult SyscallWinWriteString(  
    uint64_t layer_id_flags, int x, int y, uint32_t color, const char* s);  
struct SyscallResult SyscallWinFillRectangle(  
    uint64_t layer_id_flags, int x, int y, int w, int h, uint32_t color);  
struct SyscallResult SyscallGetCurrentTick();  
struct SyscallResult SyscallWinRedraw(uint64_t layer_id_flags);
```

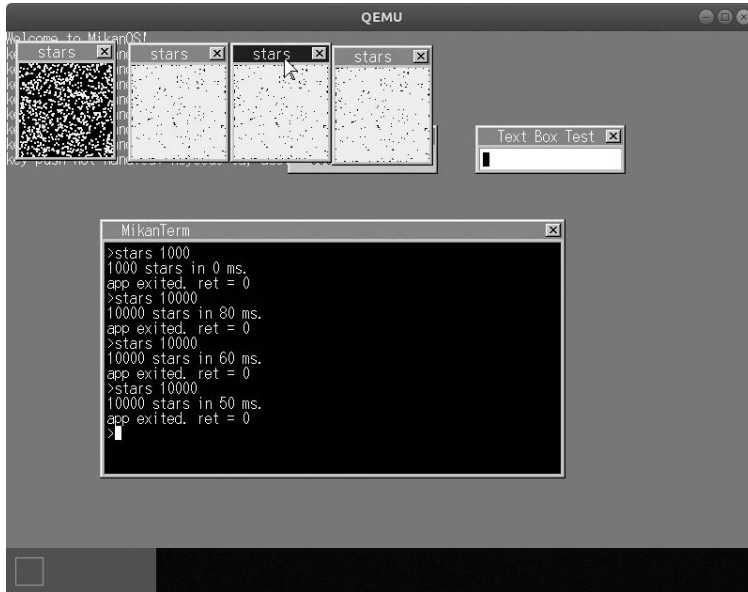
リスト 22.13 に示すように、再描画をしないオプションを定数マクロ `LAYER_NO_REDRAW` として用意しました。`0x00000001ull << 32` というような、一見して意味が分からないリテラル（マジックナンバー）には、このように名前を付けておくのが良いですね。それから、新しく追加した再描画システムコールのプロトタイプ宣言も追加しておきました。

リスト 22.14 再描画をしないオプションを付けて星を描く (stars/stars.cpp)

```
auto [tick_start, timer_freq] = SyscallGetCurrentTick();  
  
std::default_random_engine rand_engine;  
std::uniform_int_distribution x_dist(0, kWidth - 2), y_dist(0, kHeight - 2);  
for (int i = 0; i < num_stars; ++i) {  
    int x = x_dist(rand_engine);  
    int y = y_dist(rand_engine);  
    SyscallWinFillRectangle(layer_id | LAYER_NO_REDRAW,  
        4 + x, 24 + y, 2, 2, 0xffff100);  
}  
SyscallWinRedraw(layer_id);  
  
auto tick_end = SyscallGetCurrentTick();  
printf("%d stars in %lu ms.\n",  
    num_stars,  
    (tick_end.value - tick_start) * 1000 / timer_freq);
```

再描画をしないオプションを使って星を描画するように改造しました（リスト 22.14）。星をすべて描画し終わったら、最後に再描画システムコールを呼び出して 1 回だけ再描画を行います。この改造でどのくらい高速化するでしょうか？ わくわく。

図 22.3 再描画をオフにして星を描画する



いろいろな星の数で実行した様子を図 22.3 に示します。なんと、1 万個の星を描画しても 100 ミリ秒未満！ これはすごい効果です。平均 60 ミリ秒とすると、毎回再描画するバージョンに比べて 99 倍速です。今回の改造は、変更量は少ないのに、ウィンドウへの描画に関するすべてのシステムコールに機能を追加できました。これは DoWinFunc() として処理の共通化をしていたおかげです。

22.5 線を引く (osbook_day22e)

長方形を描画できるようになりました。長方形さえ描画できれば原理的にどんな複雑な絵でも描けます。細かい部分も 1 ピクセルずつ点を打っていけばいいのですから。しかし、それでは斜めの線を引いたり、円を描いたりするのがとても面倒です。この節では基本図形である直線を簡単に引けるシステムコールを作ってみます。

直線の開始点 (x_0, y_0) と終了点 (x_1, y_1) の座標が与えられたとき、直線を表す式は次の通りです。（ちょっと難しい数式かもしれませんが。分からない場合は読み飛ばして大丈夫です。）

$$y = \frac{y_1 - y_0}{x_1 - x_0} (x - x_0) + y_0$$

$\frac{y_1 - y_0}{x_1 - x_0}$ の部分は直線の傾きを表し、 m という文字を当てることが多いと思います。これを C++ のプログラム風に表すと次のようになります。


```
double m = (y1 - y0) / (x1 - x0);
for (int x = x0; x <= x1; ++x) {
    int y = m*(x - x0) + y0;
    // (x, y) の位置に点を打つ
}
```

実際にはこのままだと不完全です。最もダメなのは、直線の傾き m が 1 より大きい（45 度より垂直に近い、とも言える）場合に点線になってしまうことです。ループが 1 周して x が 1 増えるたびに y 座標は m だけ増えます。 m が 1 より大きいと、前回打った点と次の点が離れてしまうのです。ということで、直線の傾きが 1 以下の場合と 1 より大きい場合で条件分岐して直線を描きます。

リスト 22.15 指定された 2 点間に直線を描く (syscall.cpp)

```
SYSCALL(WinDrawLine) {
    return DoWinFunc(
        [](Window& win,
            int x0, int y0, int x1, int y1, uint32_t color) {
            auto sign = [](int x) {
                return (x > 0) ? 1 : (x < 0) ? -1 : 0;
            };
            const int dx = x1 - x0 + sign(x1 - x0);
            const int dy = y1 - y0 + sign(y1 - y0);

            if (dx == 0 && dy == 0) {
                win.Writer()->Write({x0, y0}, ToColor(color));
                return Result{ 0, 0 };
            }

            const auto floord = static_cast<double*>(double)>(floor);
            const auto ceild = static_cast<double*>(double)>(ceil);

            if (abs(dx) >= abs(dy)) {
                if (dx < 0) {
                    std::swap(x0, x1);
                    std::swap(y0, y1);
                }
                const auto roundish = y1 >= y0 ? floord : ceild;
                const double m = static_cast<double>(dy) / dx;
                for (int x = x0; x <= x1; ++x) {
                    const int y = roundish(m * (x - x0) + y0);
                    win.Writer()->Write({x, y}, ToColor(color));
                }
            } else {
                if (dy < 0) {
                    std::swap(x0, x1);
                    std::swap(y0, y1);
                }
                const auto roundish = x1 >= x0 ? floord : ceild;
                const double m = static_cast<double>(dx) / dy;
                for (int y = y0; y <= y1; ++y) {
                    const int x = roundish(m * (y - y0) + x0);
                    win.Writer()->Write({x, y}, ToColor(color));
                }
            }
        })
}
```

```

    return Result{ 0, 0 };
}, arg1, arg2, arg3, arg4, arg5, arg6);
}

```

直線の引き方の基本的なアイデアが分かったところで実際のシステムコールの実装に取り掛かります。**リスト 22.15** に直線を引くシステムコール `syscall::WinDrawLine()` を示します。

システムコールの先頭で 2 つの定数 dx と dy を計算しています。これらはそれぞれ、直線の横方向の変位と縦方向の変位です。言い換えると直線の開始点 (x_0, y_0) から見た終了点 (x_1, y_1) の座標を求めています。

ラムダ関数 `sign()` は、与えられた値の符号を 1、0、-1 で返す関数です。 dx 、 dy の値を補正するのに使います。`WinDrawLine()` システムコールは 2 つの座標を与え、両方の点を端点とした直線を引きます。`sign()` を使って dx 、 dy を補正しておかないと、点 (x_1, y_1) を含まないような直線になってしまうのです。とても細かいことですが、なるべく正確に直線を引きたかったので面倒ですがこのようなプログラムになりました。

dx 、 dy を計算したら実際の描画に入ります。最初に、実は直線ではなく点の描画である場合 (2 つの端点と同じ座標の場合) を特別に処理します。この場合を除いておかないと、後で直線の傾きを計算するときに分母が 0 になって困ってしまいます。

点の描画である場合を除けたら、次は本当に直線の処理になります。まずは後で使う定数 `floord` と `ceild` を定義しておきます。これらは、引数の型によって定義が異なる複数の `floor()` と `ceil()` の中から、`double` 型のバージョンを選び出したものです。C++ での関数オーバーロードは便利ですが、引数を与えない状態で関数を選ぶとするとちょっと面倒です。

さて、直線の描画ですが、最初に説明したように水平に近い場合と垂直に近い場合に分けます。水平に近い (傾きが 1 以下) というのは $\text{abs}(dx) \geq \text{abs}(dy)$ が真であることと同値です。`abs()` という関数は絶対値 (absolute value) を計算する C 標準ライブラリの関数です。

水平に近い場合と垂直に近い場合の処理はとても良く似ています。前者では X 軸に沿って変数を動かして点を打っていきます。後者では Y 軸に沿って処理をしますが、X と Y の違いだけで処理の形はまったく一緒です。ということで、前者のプログラムだけを説明します。

まず最初に dx が負の場合、すなわち点 (x_0, y_0) より左に点 (x_1, y_1) がある場合に 2 つの端点を入れ替えています。こうすることで、後の for 文実行時に必ず $x_0 < x_1$ が成り立った状態にしようということです。この処理がないと $x_0 > x_1$ のときに for 文が実行されませんからね。

端点を入れ替え終わったら直線の傾き m を求めます。 dx と dy はどちらも整数型です。割り算する前に浮動小数点数型に変換することで、割り算によって小数点以下が切り捨てられることを防ぎます。

ループの中では求めた傾きを使って X 座標から Y 座標を計算します。Y 座標を整数に変換する際は、Y 座標が大きくなる方向の直線なのか、小さくなる方向の直線なのかで切り捨て (`floor`) と切り上げ (`ceil`) を切り替えます。微妙な差ですが、この処理をサボると 1 ピクセルのズレが発生することがあります。X 方向の長さ分だけ点を打つ処理を繰り返せば線を引く処理は完了です。

リスト 22.16 カラフルな線を描く lines コマンド (lines/lines.cpp)

```
#include <cmath>
#include <cstdlib>
#include <random>
#include "../syscall.h"

static constexpr int kRadius = 90;

constexpr uint32_t Color(int deg) {
    if (deg <= 30) {
        return (255 * deg / 30 << 8) | 0xff0000;
    } else if (deg <= 60) {
        return (255 * (60 - deg) / 30) << 16 | 0x00ff00;
    } else if (deg <= 90) {
        return (255 * (deg - 60) / 30) | 0x00ff00;
    } else if (deg <= 120) {
        return (255 * (120 - deg) / 30) << 8 | 0x0000ff;
    } else if (deg <= 150) {
        return (255 * (deg - 120) / 30) << 16 | 0x0000ff;
    } else {
        return (255 * (180 - deg) / 30) | 0xff0000;
    }
};

extern "C" void main(int argc, char** argv) {
    auto [layer_id, err_openwin]
        = SyscallOpenWindow(kRadius * 2 + 10 + 8, kRadius + 28, 10, 10, "lines");
    if (err_openwin) {
        exit(err_openwin);
    }

    const int x0 = 4, y0 = 24, x1 = 4 + kRadius + 10, y1 = 24 + kRadius;
    for (int deg = 0; deg <= 90; deg += 5) {
        const int x = kRadius * cos(M_PI * deg / 180.0);
        const int y = kRadius * sin(M_PI * deg / 180.0);
        SyscallWinDrawLine(layer_id, x0, y0, x0 + x, y0 + y, Color(deg));
        SyscallWinDrawLine(layer_id, x1, y1, x1 + x, y1 - y, Color(deg + 90));
    }
    exit(0);
}
```

作成した直線描画システムコールを用いてたくさんの直線を引くコマンド lines を作りました（**リスト 22.16**）。main() ではウィンドウを開き、そのウィンドウに線を引きます。線を引く方法が結構工夫してあって、1 点から放射状に線が伸びるように 5 度ずつ回転させながら線を引きます。ループに使っている変数 deg が度数を保持する変数で、これが 5 度ずつ 90 度まで変化します。C 標準ライブラリの三角関数 cos() と sin() を使い、角度と半径から X 座標と Y 座標を計算します。

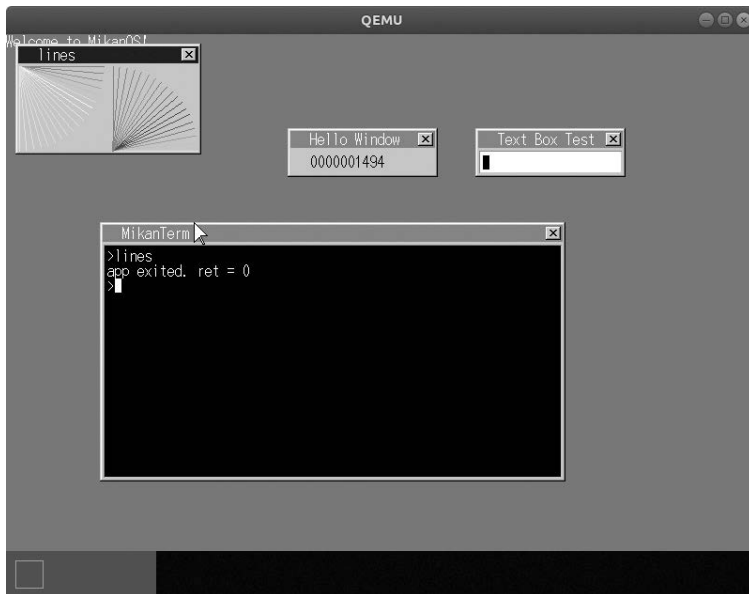
色が黒だけだとつまらないので、角度に応じてグラデーションがかかるように工夫してみました。角度から色を計算する関数 Color() を定義しました。この関数は 0 度から 180 度までの値を受け取って、角度に応じた色（RGB 値）を返します。

リスト 22.17 数学ライブラリをリンクする (Makefile.elfapp)

```
$(TARGET): $(OBJS) Makefile
    ld.lld $(LDFLAGS) -o $@ $(OBJS) -lc -lc++ -lc++abi -lm
```

このプログラムをリンクするには `cos()` と `sin()` が必要です。そのため、リスト 22.17 に示す通り、数学ライブラリ (libm) をリンクするようにしました。

図 22.4 lines コマンドが放射状の線を描き出す様子



ビルドして実行した様子を図 22.4 に示します。どの角度でも点線になってしまうことなく、きれいな直線になっています。条件分岐がちゃんと効いている証拠です。そして、いろんな色の線が広がっていて綺麗ですよ。

22.6 ウィンドウのクローズ (osbook_day22f)

ふと立ち止まってみるといろいろな描画系コマンドが揃ってきました。カラフルな文字列、いろいろな大きさの四角形、直線。これらのコマンドを起動して遊んでいると重大なことに気がきました。アプリが終了してもウィンドウが閉じないで残ったままになっているのです。ということで、この節ではウィンドウを閉じるシステムコールを作ります。

リスト 22.18 RemoveLayer() は指定されたレイヤを削除する (layer.cpp)

```
void LayerManager::RemoveLayer(unsigned int id) {
    Hide(id);

    auto pred = [id](const std::unique_ptr<Layer>& elem) {
        return elem->ID() == id;
    };
    EraseIf(layers_, pred);
}
```

今まで、一度作ったレイヤやウィンドウを閉じる仕組みを作っていませんでした。そこで、まずは指定したレイヤを削除するための関数 RemoveLayer() を新たに作りました(リスト 22.18)。この関数は、レイヤ ID で指定されたレイヤを layer_stack_ および layers_ から削除します。layer_stack_ からの削除は Hide() メソッドに任せます。久々にレイヤマネージャに手を入れましたね。レイヤマネージャの処理を覚えていましたか？

EraseIf() は与えた条件 (pred) が真となる要素を削除する関数です。実装は Erase() (リスト 14.25) とほぼ同じですので、紹介しません。

リスト 22.19 レイヤを削除するシステムコール (syscall.cpp)

```
SYSCALL(CloseWindow) {
    const unsigned int layer_id = arg1 & 0xffffffff;
    const auto layer = layer_manager->FindLayer(layer_id);

    if (layer == nullptr) {
        return { EBADF, 0 };
    }

    const auto layer_pos = layer->GetPosition();
    const auto win_size = layer->GetWindow()->Size();

    __asm__("cli");
    active_layer->Activate(0);
    layer_manager->RemoveLayer(layer_id);
    layer_manager->Draw({layer_pos, win_size});
    __asm__("sti");

    return { 0, 0 };
}
```

追加した RemoveLayer() を利用してウィンドウ (とそのレイヤ) を閉じるシステムコール syscall::CloseWindow() をリスト 22.19 に示します。特に難しいところはないと思います。指定されたレイヤ ID のフラグ部分は使わないので無視すれば良いです。

1 つ工夫したのは再描画の処理です。具体的には layer_manager->Draw({layer_pos, win_size}) の部分です。RemoveLayer() を使ってレイヤを削除した後はレイヤを指定した再描画ができなくなりますから、レイヤ削除前にレイヤの位置とウィンドウの大きさを取得しておき、それらを使って再描画します。

リスト 22.20 アプリ終了前にウィンドウを閉じる (winhello/winhello.cpp)

```

SyscallWinWriteString(layer_id, 7, 24, 0xc00000, "hello world!");
SyscallWinWriteString(layer_id, 24, 40, 0x00c000, "hello world!");
SyscallWinWriteString(layer_id, 40, 56, 0x0000c0, "hello world!");

SyscallCloseWindow(layer_id);
exit(0);

```

winhello コマンドを改造して、コマンド終了前にウィンドウを閉じるようにしました（**リスト 22.20**）。`exit(0);` の前に作成したシステムコールの呼び出しを加えただけです。

改造版の winhello を実行してみます。あれ？ ターミナルに `app exited. ret = 0` と出るだけで何も変化が見えませんか。これはまあ当たり前前で、アプリが起動してウィンドウを開いた後、一瞬でウィンドウを閉じてしまうからです。これではウィンドウを閉じない方がまだマシですね。

これを解決するには、winhello コマンドが何らかのイベントを待つようにすれば良いです。例えばタイマのタイムアウトを待ったり、キー入力があるまで待ったり、マウスで閉じるボタンがクリックされるまで待ったり、などです。これらは統一的に「イベントを待つ」システムコールを作ることに対応ができそうです。次の節で作ってみます。

22.7 キー入力待つ (osbook_day22g)

ウィンドウが一瞬で閉じないようにするために、何らかのイベント発生を待つシステムコールを作ります。最初からすべてのイベントに対応するのは大変ですので、まずは `Ctrl-Q` が押下されたらアプリを終了することを目標にします。

大枠の構造はこんな感じにしたいと思います。まず、アプリから「何かイベントがあれば教えてね」とシステムコールを使って OS 本体へ依頼します。依頼を受けた OS 側は、そのアプリ向けのイベントを拾ってアプリに返します。アプリは返ってきたイベントを調べて、それが `Ctrl-Q` に相当するものだったらウィンドウを閉じてアプリを終了させます。

システムコールを実行したときにそのアプリ向けのイベントが 1 つ以上存在すれば、即座にシステムコールを終了させてイベントをアプリ側に送り返します。一方、まだイベントが何もないときの挙動は大きく 2 つ考えられます。1 つはイベントが何もなくあったことを即座に送り返す方法、もう 1 つはイベントが来るまで待つ方法です。

前者の挙動では、アプリ側はイベントを取得できるまでシステムコールを繰り返し呼び続けることになります。ポーリングと呼ばれる方法ですね。もし全力でシステムコールを呼び続けると CPU 使用率が 100% 近くになり、本来なら他のタスクが使えていた CPU の能力が無駄になってしまいます。そのため、ポーリングをする際はループするたびに少しスリープするというような工夫が必要になります。ただ、スリープするためにもスリープするためのシステムコールが必要となるため、作るのが大変です。

後者の挙動では、アプリ側はシステムコールが必ず 1 つ以上のイベントを返してくれることを期待できます。イベントが来るまではアプリは OS によって自動的にスリープ状態となります。そのため、アプリは何もしなくても CPU が無駄になることがありません。この節では後者の挙動を目指してシステムコールを作ってみようと思います。

リスト 22.21 イベント取得システムコールのプロトタイプ (syscall.h)

```
struct SyscallResult SyscallReadEvent(struct AppEvent* events, size_t len);
```

リスト 22.21 に今回実装するシステムコールのプロトタイプ宣言を示します。アプリ側がイベントを読み取るためのデータ領域 `events` を与え、システムコールがそのデータ領域にイベントを書き込む、という仕様を想定しています。一度に複数のイベントを読み取ることができるよう、データ領域は配列としておき、配列の要素数を `len` で指定します。システムコールは実際に読み取ったイベントの数を戻り値として返すことにしましょう。

リスト 22.22 AppEvent は 1 つのイベントを表す (app_event.hpp)

```
#pragma once

#ifdef __cplusplus
extern "C" {
#endif

struct AppEvent {
    enum Type {
        kQuit,
    } type;
};

#ifdef __cplusplus
} // extern "C"
#endif
```

イベントを表す構造体 `AppEvent` の定義は**リスト 22.22** のようにしてみました。今のところ、イベント種別を表す `type` フィールドだけを持った単純な構造体です。`kQuit` はアプリを止める (quit) ことを表すイベント種別です。このイベントがアプリに送られてきたらアプリは終了処理を行う、ということにしましょう。

リスト 22.23 終了イベントが来たらアプリを終了させる (winhello/winhello.cpp)

```
AppEvent events[1];
while (true) {
    auto [ n, err ] = SyscallReadEvent(events, 1);
    if (err) {
        printf("ReadEvent failed: %s\n", strerror(err));
        break;
    }
    if (events[0].type == AppEvent::kQuit) {
        break;
    } else {
        printf("unknown event: type = %d\n", events[0].type);
    }
}
SyscallCloseWindow(layer_id);
exit(0);
```

winhello コマンドをリスト 22.23 に示すように改造し、終了イベントが来るまでアプリを終了させず待機するようにしました。終了イベントを受け取るか SyscallReadEvent() が失敗するまでずっとループします。現段階ではイベント種別として終了イベントしかなく、else 節 (printf("unknown event:".....の行) が実行されることはありませんが、将来的にイベント種別が増えたときのために書いておきました。

SyscallReadEvent() は1つ以上のイベントが来るまで待機（ブロック）します。します、というか、そうなるようにこれから作ります。ですので、上記のプログラムは一見すると全力で SyscallReadEvent() を呼び出しまくるように見えるかもしれませんが、実際にはイベントが来るまで待機するため、CPU の使用率が 100% になるようなことはありません。

さて、これからシステムコールの中身を実装していくのですが、どのように実装したらいいでしょうか？ やりたいのは、Ctrl-Q が入力されたらシステムコールの引数 events にイベントを書き込み、システムコールから復帰することです。このためには、アプリのウィンドウに対して入力されたキーを受け取る必要があります。実は、この仕組みはすでに「16.1 ターミナルでキー入力」で作成してあるのですが、覚えていますか？

そのとき作ったのは、レイヤIDとタスクを紐付けるための layer_task_mapを用いた仕組みでした。キーが押下されたときにアクティブだったウィンドウのレイヤIDからタスクを検索し、そのタスクへキー入力メッセージを送ります。この仕組みを利用するとイベント取得システムコールを簡単に作れそうです。アプリはターミナルタスク上で起動しますから、アプリに対するキー入力を受け取るにはターミナルタスクのメッセージキューを見れば良いでしょう。

リスト 22.24 アプリへのキー入力をターミナルタスクから受け取る (syscall.cpp)

```
SYSCALL(ReadEvent) {
    if (arg1 < 0x8000'0000'0000'0000) {
        return { 0, EFAULT };
    }
    const auto app_events = reinterpret_cast<AppEvent*>(arg1);
    const size_t len = arg2;

    __asm__("cli");
    auto& task = task_manager->CurrentTask();
    __asm__("sti");
    size_t i = 0;

    while (i < len) {
        __asm__("cli");
        auto msg = task.ReceiveMessage();
        if (!msg && i == 0) {
            task.Sleep();
            continue;
        }
        __asm__("sti");

        if (!msg) {
            break;
        }

        switch (msg->type) {
            case Message::kKeyPush:
```



```

        if (msg->arg.keyboard.keycode == 20 /* Q key */ &&
            msg->arg.keyboard.modifier & (kLControlBitMask | kRControlBitMask)) {
            app_events[i].type = AppEvent::kQuit;
            ++i;
        }
        break;
    default:
        Log(kInfo, "uncaught event type: %u\n", msg->type);
    }
}

return { i, 0 };
}

```

`syscall::ReadEvent()` の実装を **リスト 22.24** に示します。このシステムコールの主な処理は、ターミナルタスクのメッセージキューからメッセージを1つずつ取り出し、`AppEvent` 型に変換してシステムコールの引数 `events` に書き込んでいくことです。上から順番に処理を説明します。

まず、第1引数 `events` の値をチェックします。配列へのポインタとして無効な値であればエラーです。アプリが使ってよいメモリ領域は仮想アドレス空間の後半だけですから、もし前半を指すポインタが指定されていたらエラーにします。わざと小さい値を指定することで OS 本体のメモリ領域を破壊しようとする悪いアプリがあるかもしれませんからね。

第1引数の値が正常であれば、次に実行中のタスクを取ってきます。実行中のタスクとはアプリを実行しているターミナルタスクですね。

次に `while` ループです。ループの先頭で `task.ReceiveMessage()` として、タスクのメッセージキューからメッセージを1つ取得します。1つもメッセージがなければ `msg` は空 (`std::nullopt` という値) になります。空だったらタスクをスリープさせます。ここでタスクをスリープさせることにより、イベントが何も無いときに `ReadEvent()` システムコールが待機するという挙動になります。もし、ここでスリープさせずにシステムコールを終了させるようにするとポーリング方式となりますね。

さて、メッセージキューに1つ以上のメッセージがあれば `msg` が空ではなくなります。後は `msg` を `AppEvent` 型に変換し、`events` 配列の先頭から書き込むだけです。

USB HID の仕様書 (HID Usage Table、**参考文献 [13]**) によれば Q キーのキーコードは 20 です。そして Ctrl キーはモディファイアキーですので、Ctrl-Q の判定は「キーコードが 20、かつモディファイアキーの Ctrl ビットが立っている」という条件になりますね。Ctrl キーは左右ありますからどちらにも対応できるようにしておきました。Ctrl-Q が入力されたことが分かったら、`events` に `kQuit` イベントを書き込みます。

イベントを1つ以上 `events` に書き込めたらカウンタ `i` が 0 より大きな値になります。その状態で処理すべきメッセージがなくなれば `while` ループを抜け、システムコールの処理を終えます。システムコールがエラー無く終了すれば、`events` には必ず1つ以上のイベントが書き込まれていることになります。

リスト 22.25 layer_task_map にアプリのウィンドウを登録する (syscall.cpp)

```
__asm__("cli");
const auto layer_id = layer_manager->NewLayer()
    .SetWindow(win)
    .SetDraggable(true)
    .Move({x, y})
    .ID();
active_layer->Activate(layer_id);

const auto task_id = task_manager->CurrentTask().ID();
layer_task_map->insert(std::make_pair(layer_id, task_id));
__asm__("sti");
```

これでシステムコールが完成かと思ったら忘れていた処理がありました。それは、アプリのウィンドウを layer_task_map に登録することです。登録しておかないと、アプリのウィンドウに入力したキーがターミナルタスクへと送られず、ReadEvent() システムコールで何のイベントも取得できません。アプリがウィンドウを開いたとき (syscall::OpenWindow()) に、そのウィンドウを含むレイヤを layer_task_map に登録するように改造しました (リスト 22.25)。

リスト 22.26 アプリがウィンドウを閉じたら登録を解除する (syscall.cpp)

```
__asm__("cli");
active_layer->Activate(0);
layer_manager->RemoveLayer(layer_id);
layer_manager->Draw({layer_pos, win_size});
layer_task_map->erase(layer_id);
__asm__("sti");
```

さらに、アプリがウィンドウを閉じたら layer_task_map の登録を解除するようにしておきます。リスト 22.26 に示すように、syscall::CloseWindow() に layer_task_map->erase(layer_id); の 1 行を追加しておきました。

さて、ここで重大な問題に気付きました。syscall::ReadEvent() の中で task.Sleep() を実行していることです。task.Sleep() はその中で SwitchContext() を呼び出します。システムコール実行時には RSP はアプリ用のスタックを指しているため、「21.1 IST を設定しよう」で説明したのと同様の問題が起きるのです。SwitchContext() (から続いて実行される RestoreContext()) はスタックに iret 命令のためのフレームを積み、その後 CR3 レジスタの値を復帰させます。iret 命令はアプリ用スタックに積まれたフレームを読み取れず、ページフォルトが起きてしまいます*4。

タイマ割り込みの内部で RestoreContext() を実行する際のページフォルトは IST の仕組みにより回避しました。しかし、システムコールは割り込みの仕組みを経由せずに呼び出されるため、別の回避策が必要です。ここでは、システムコール実行時にスタックを入れ替えることにします。

* 4 紙面の都合でこのバグをここで紹介、修正しますが、実はこのバグが顕在化するの
は「24.3 複数アプリの同時起動」の時点です。そこで行う、アプリごとに PML4
を切り替える改造によって iretq がページフォルトを発生させるようになります。

リスト 22.27 システムコール実行時にスタックを入れ替える (asmfunc.asm)

```

extern GetCurrentTaskOSStackPointer
extern syscall_table
global SyscallEntry
SyscallEntry: ; void SyscallEntry(void);
    push rbp
    push rcx ; original RIP
    push r11 ; original RFLAGS

    push rax ; システムコール番号を保存

    mov rcx, r10
    and eax, 0x7fffffff
    mov rbp, rsp

    ; システムコールを OS 用スタックで実行するための準備
    and rsp, 0xfffffffffffffff0
    push rax
    push rdx
    cli
    call GetCurrentTaskOSStackPointer
    sti
    mov rdx, [rsp + 0] ; RDX
    mov [rax - 16], rdx
    mov rdx, [rsp + 8] ; RAX
    mov [rax - 8], rdx

    lea rsp, [rax - 16]
    pop rdx
    pop rax
    and rsp, 0xfffffffffffffff0

    call [syscall_table + 8 * eax]
    ; rbx, r12-r15 は callee-saved なので呼び出し側で保存しない
    ; rax は戻り値用なので呼び出し側で保存しない

    mov rsp, rbp

```

ということで、`syscall_table` に登録された関数を実行する前にスタックを OS 用に入れ替えるように改造しました (リスト 22.27)。やった改造は、「システムコールを OS 用スタックで実行するための準備」というコメントに続く 14 行を追加しただけです。この 14 行はちょっと処理が難しいと思いますので、上から見ていきます。

まず RSP の値を 16 バイト境界に揃えているのは、C++ で実装した関数 `GetCurrentTaskOSStackPointer()` を安全に呼び出すためです。8 バイトの `push` 命令を 2 回実行しているのでアラインメントは崩れません (奇数回の実行だと崩れてしまいます)。この関数の実行中に割り込みがあるとまずいですから、呼び出しの前後で割り込み禁止、許可をしておきます。

リスト 22.28 現在のタスクの OS 用スタックポインタを取得する (task.cpp)

```
__attribute__((no_caller_saved_registers))
extern "C" uint64_t GetCurrentTaskOSStackPointer() {
    return task_manager->CurrentTask().OSStackPointer();
}
```

GetCurrentTaskOSStackPointer() は、現在実行中のタスクの OS 用スタックポインタの値を取得する関数です（リスト 22.28）。CallApp() で保存したスタックポインタの値が返されます。この関数をアセンブリ言語の中から利用するために、extern "C" を付けて名前がマングリングされないようにします。

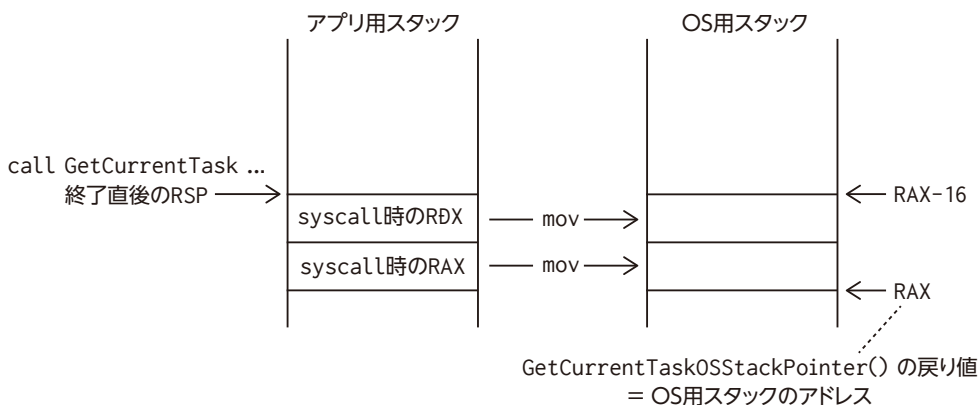
この関数は syscall 命令のすぐ後という、各レジスタの値が壊れてほしくない場面で呼び出されます。関数は一般にレジスタの値を変更してしまいますから、安全に関数を呼び出すには大量のレジスタの保存と復帰をしなければなりません。実際に関数内で変更されるレジスタは少ないかもしれませんが、どれが変更されるかは C++ コンパイラの気まぐれで決まりますから、勝手に仮定するのは良くないです。

そのための __attribute__((no_caller_saved_registers)) です。これを付けて関数を定義することで、レジスタの保存と復帰をその関数の責任でやってくれるようになります。Caller とは呼び出し側という意味で、「no caller saved registers」は「呼び出し側ではどのレジスタも保存しませんよ」ということです。だから呼び出された方でレジスタの保存と復帰をやっねと。もちろん、戻り値用レジスタ（RDX と RAX）だけは呼び出し側で保存、復帰しなければなりませんけど。call 命令の前にある 2 つの push 命令はこのためだったわけです。

GetCurrentTaskOSStackPointer() の戻り値が RAX レジスタに格納されるんだ、というのを念頭に置いて、SyscallEntry() の処理の続きを見ていきます。RAX に書かれた OS 用スタックを指すアドレスを RSP に設定する、というのが最終的なゴールです。

syscall_table に登録された関数を呼び出す前に RDX の値を復帰させる必要があります。RDX にはシステムコールの第 3 引数が設定されている必要がありますからね。これをやるために、図 22.5 のようにアプリ用スタックに保存した 2 つのレジスタの値を OS 用スタックにコピー (mov) するようにしました。

図 22.5 OS 用スタックに値をコピーする様子



OS 用スタックに値をコピーした後、RSP を OS 用スタックに切り替えて pop 命令を 2 回実行することで、2 つのレジスタを復帰させることができます。スタックの切り替えが終わったら、改めて OS 用スタックのスタックポインタを 16 バイト境界に揃えてシステムコール関数を呼び出します。

図 22.6 Ctrl-Q により winhello コマンドが終了したところ



ちょっと長かったですがキー入力を待つ仕組みが完成しました。それではビルドして実行してみます。
図 22.6 に、winhello コマンドを実行して Ctrl-Q を入力した直後の様子を載せます。写真では分かりにくいですが、Ctrl-Q を押下した直後にウィンドウが閉じてアプリがコード 0 で終了しました。よしよし、期待通りです。アプリでキー入力を受け取れるようになってかなりテンションが上がってきましたが、長くなってしまったので、この章はこの辺で終わりにしましょう。

第23章

グラフィックと イベント(2)

この章ではイベント処理の仕組みにマウスを対応させます。マウス入力ができるとマウスの軌跡を取れるということです。簡単なお絵描きアプリを作ってみようかと思います。本書では扱いませんが、お絵描きアプリの延長で、設計図などを描くためのCADと言われるソフトウェアも作れるようになるはずです。夢が広がりますね。マウス入力ができるようになったら、次はアプリケーションが時間経過を待つ仕組みを作ります。これもキー入力の仕組みに相乗りすることで簡単に作れるんじゃないかなーと思っています。指定した時間だけ待てるようになると、処理をゆっくり進めることができるようになり、アニメーションを実現できます。

23.1 マウス入力 (osbook_day23a)

アプリがマウスの動きを受け取れるようにします。具体的には、マウスが動くたびに縦横の移動量が記録されたイベントが生じ、ReadEvent() システムコールで取得できるようになれば良いでしょう。マウスの動きに応じてタスクのメッセージキューにどんどんマウス移動通知が来るということです。

リスト 23.1 マウス移動イベント (app_event.hpp)

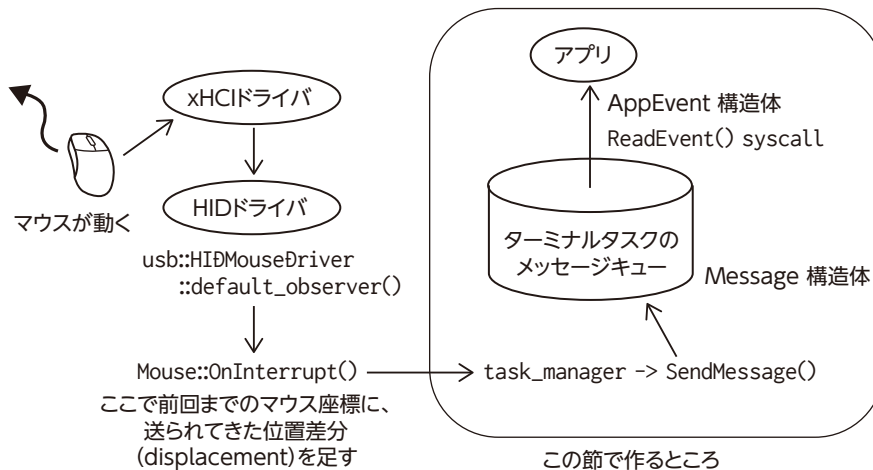
```
struct AppEvent {
    enum Type {
        kQuit,
        kMouseMove,
    } type;

    union {
        struct {
            int x, y;
            int dx, dy;
            uint8_t buttons;
        } mouse_move;
    } arg;
};
```

リスト 23.1 に示す通り、AppEvent 構造体にマウス移動を通知するイベント種別を追加しました。イベントの引数はウィンドウ左上を基準としたマウス座標 x、y、直前からの移動量 dx、dy、そのときのマウスのボタン押下状況 buttons です。ReadEvent() システムコールを実行するとこのイベントが読み取れるようにするのが、以降の目標です。

イベントが発生してから ReadEvent() を使ってアプリが読み取るまでの流れが少し複雑ですので、ここでしっかり確認しておきます。マウスが動いて USB HID クラスドライバが反応してからアプリまでイベントが伝わる経路を図 23.1 に示します。

図 23.1 マウス移動イベントの伝搬



マウスが動くと筆者が作った USB ドライバが作動します。この USB ドライバは、最終的に `usb::HIDMouseDriver::default_observer` に登録された関数を呼び出します。ここには `Mouse::OnInterrupt()` が登録してあって、その関数はいまのところ、ウィンドウのドラッグを担当しています。この節では `Mouse::OnInterrupt()` に機能を追加して、マウスが移動したという情報をタスクへ送信しようと思います。

アプリが `ReadEvent()` システムコールで読み取るときには `AppEvent` 構造体を使いますが、タスクへメッセージを送信する際は `Message` 構造体を使う必要があります。そのため、`AppEvent` に行った変更と同様のものを `Message` 構造体にもする必要があります。本当に同じ改造をただけなので、変更点の紹介は省略します。

さて、マウスが移動したイベントはどのタスクに送るべきでしょうか？ タスクが複数起動していたら全部のタスクに送るのがいいのでしょうか？ それとも、アクティブウィンドウ（を持つタスク）にだけ送るようにしましょうか？ 戦略はいろいろ考えられます。ここではアクティブウィンドウにだけ送ることにします。全部のタスクに送ってしまうと処理が重くなりそうですからね。

リスト 23.2 `SendMessage()` はアクティブウィンドウにマウスイベントを送る (`mouse.cpp`)

```
void SendMessage(Vector2D<int> newpos, Vector2D<int> posdiff,
                  uint8_t buttons) {
    const auto act = active_layer->GetActive();
    if (!act) {
        return;
    }
    const auto layer = layer_manager->FindLayer(act);

    const auto task_it = layer_task_map->find(act);
    if (task_it == layer_task_map->end()) {
        return;
    }

    if (posdiff.x != 0 || posdiff.y != 0) {
        const auto relpos = newpos - layer->GetPosition();
        Message msg{Message::kMouseMove};
        msg.arg.mouse_move.x = relpos.x;
        msg.arg.mouse_move.y = relpos.y;
        msg.arg.mouse_move.dx = posdiff.x;
        msg.arg.mouse_move.dy = posdiff.y;
        msg.arg.mouse_move.buttons = buttons;
        task_manager->SendMessage(task_it->second, msg);
    }
}
```

リスト 23.2 に新しく作った関数 `SendMessage()` の定義を示します。この関数は、マウスの位置、直前からの移動量、ボタン押下状況を受け取って、それをアクティブウィンドウをもつタスクに送信します。マウスの座標はウィンドウの左上を基準とした値に変換してからタスクへ送信してあげることになりました。その方が、受け取ってからの処理がやりやすいでしょう。

リスト 23.3 Mouse::OnInterrupt() から SendMouseEvent() を呼ぶ (mouse.cpp)

```
if (drag_layer_id_ == 0) {
    SendMouseEvent(newpos, posdiff, buttons);
}
```

マウスが移動したりボタンが押されたりしたときに呼び出される Mouse::OnInterrupt() に、先ほど作った SendMouseEvent() を呼び出す処理を追加しました(リスト 23.3)。1 つ工夫したのは、ウィンドウをドラッグしているときにはマウスイベントを発生させないことです。ウィンドウのドラッグ中は、posdiff は 0 にならないものの、ウィンドウ左上からのマウス座標は変化しませんので、アプリにイベントを送っても無駄になってしまいます。

リスト 23.4 ReadEvent() をマウス移動イベントに対応させる (syscall.cpp)

```
case Message::kMouseMove:
    app_events[i].type = AppEvent::kMouseMove;
    app_events[i].arg.mouse_move.x = msg->arg.mouse_move.x;
    app_events[i].arg.mouse_move.y = msg->arg.mouse_move.y;
    app_events[i].arg.mouse_move.dx = msg->arg.mouse_move.dx;
    app_events[i].arg.mouse_move.dy = msg->arg.mouse_move.dy;
    app_events[i].arg.mouse_move.buttons = msg->arg.mouse_move.buttons;
    ++i;
    break;
```

タスクのメッセージキューにマウス移動イベントが送信されるようになりました。次は syscall::ReadEvent() にマウス移動イベント用の処理を付け加えます(リスト 23.4)。app_events[i].arg.mouse_move(AppEvent 内にある構造体)と msg->arg.mouse_move(Message 内にある構造体)は同じメンバを同じ順序で持つ構造体ですが、あくまでも両者は違う型ですのでまるごとの代入ができません。そのため、要素毎に代入しています。

2 つの構造体が同じ定義なのに型を共通化していないことについて、ちょっと脱線しますが説明します。型を共通化していないのは意図的です。もちろん、やろうと思えば 1 つのヘッダファイルに構造体の定義を置き、AppEvent と Message でその型を使うことができます。そうすれば app_events[i].arg.mouse_move = msg->arg.mouse_move; というように構造体まるごとの代入ができます。しかし、両者は本質的に違う型なのです。Message は OS 本体の内部でイベントを送受信するためのもの、AppEvent はアプリがイベントを受け取るためのものです。このように、本来は違うのだけど「たまたま似ている」ものを共通化してしまうと後で面倒なことになり得ます。そのため、ここではあえて両者を異なる構造体型として定義してみました。

話をもとに戻します。以上の改造で ReadEvent() システムコールでマウスの動きを読み取れるようになったはずですが。早速、マウスの動きを使って何かアプリを作りたいです。簡単に作れて、マウスの動きが取得できていることが分かりやすいアプリ……。あ、そうだ！ 目玉アプリを作ってみることにします。

リスト 23.5 マウスの動きを追いかける目玉を描くアプリ (eye/eye.cpp)

```
#include <cmath>
#include <cstdio>
#include <cstdlib>
```

```

#include <algorithm>
#include "../syscall.h"

static const int kCanvasSize = 100, kEyeSize = 10;

void DrawEye(uint64_t layer_id_flags,
             int mouse_x, int mouse_y, uint32_t color) {
    const double center_x = mouse_x - kCanvasSize/2 - 4;
    const double center_y = mouse_y - kCanvasSize/2 - 24;

    const double direction = atan2(center_y, center_x);
    double distance = sqrt(pow(center_x, 2) + pow(center_y, 2));
    distance = std::min<double>(distance, kCanvasSize/2 - kEyeSize/2);

    const double eye_center_x = cos(direction) * distance;
    const double eye_center_y = sin(direction) * distance;
    const int eye_x = static_cast<int>(eye_center_x) + kCanvasSize/2 + 4;
    const int eye_y = static_cast<int>(eye_center_y) + kCanvasSize/2 + 24;

    SyscallWinFillRectangle(layer_id_flags, eye_x - kEyeSize/2, eye_y - kEyeSize/2, kEyeSize,
                             kEyeSize, color);
}

extern "C" void main(int argc, char** argv) {
    auto [layer_id, err_openwin]
        = SyscallOpenWindow(kCanvasSize + 8, kCanvasSize + 28, 10, 10, "eye");
    if (err_openwin) {
        exit(err_openwin);
    }

    SyscallWinFillRectangle(layer_id, 4, 24, kCanvasSize, kCanvasSize, 0xffffffff);

    AppEvent events[1];
    while (true) {
        auto [n, err] = SyscallReadEvent(events, 1);
        if (err) {
            printf("ReadEvent failed: %s\n", strerror(err));
            break;
        }
        if (events[0].type == AppEvent::kQuit) {
            break;
        } else if (events[0].type == AppEvent::kMouseMove) {
            auto& arg = events[0].arg.mouse_move;
            SyscallWinFillRectangle(layer_id | LAYER_NO_REDRAW,
                                    4, 24, kCanvasSize, kCanvasSize, 0xffffffff);
            DrawEye(layer_id, arg.x, arg.y, 0x000000);
        } else {
            printf("unknown event: type = %d\n", events[0].type);
        }
    }
    SyscallCloseWindow(layer_id);
    exit(0);
}

```

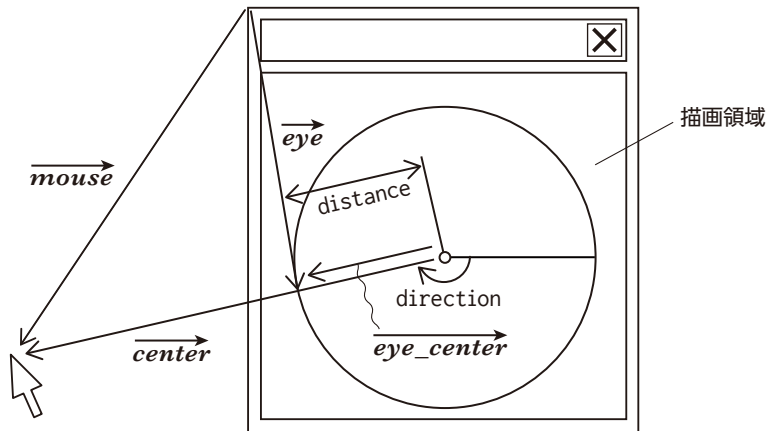
目玉アプリとは、マウスカーソルを見つめるように目玉がぐりぐり動くアプリのことです。Linuxなどで使われる X Window System 向けの有名なアプリで xeyes というものがあります。これを真似て eye というコマンドを作ってみました。実装をリスト 23.5 に示します。

eye コマンドは正方形の描画領域を持つウィンドウを使用します。kCanvasSize はその描画領域の 1 辺の大きさを表します。kEyeSize は目玉の大きさです。本当は円の目玉を描きたかったのですが、円の描画は面倒なので正方形の目玉です。だからこの定数も目玉の 1 辺の大きさを表します。

関数 DrawEye() は、指定したウィンドウに目玉を描きます。目玉はマウスカーソルを見つめる（追いかける）位置に描画します。マウスカーソルが描画領域の外にあるときは描画領域から飛び出ない位置に、描画領域内にあるときはマウスカーソルの真下になります。

DrawEye() の最後に SyscallWinFillRectangle() を呼び出すまでの計算はすべて目玉の位置を計算するためのものです。計算の流れを以下で説明していきますが、理解しなくても本書全体の理解には影響がありません。全部読み飛ばしても大丈夫です。

図 23.2 目玉の位置の計算



さて、目玉の位置計算を説明しますよ。説明用の図は図 23.2 です。この図の $\overrightarrow{mouse}$ は関数の引数で与えられたマウスカーソルの座標 $mouse_x$ 、 $mouse_y$ をベクトル表記で表したものです。この座標はウィンドウ左上を基準としているので、ベクトルの始点もウィンドウ左上になっています。 $\overrightarrow{center}$ は座標 $center_x$ 、 $center_y$ のベクトル表記です。マウスカーソルの座標 $\overrightarrow{mouse}$ を、描画領域の中心から見た座標に変換したものです。

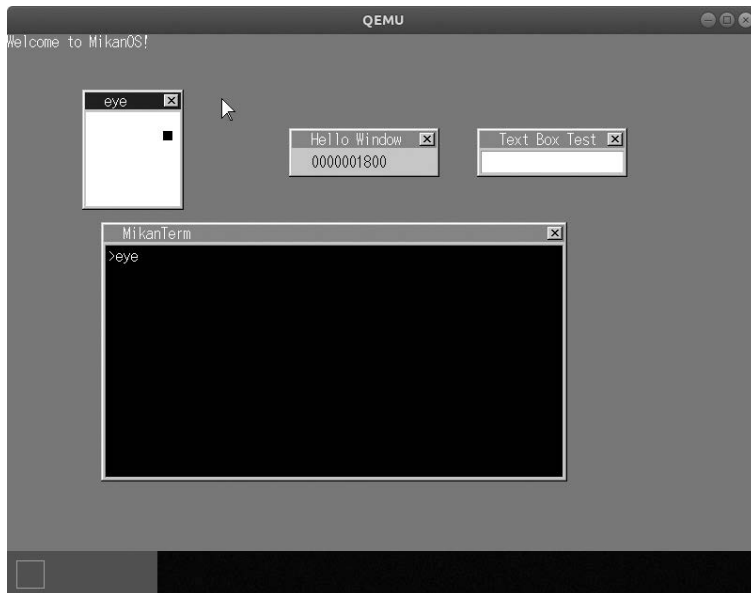
次に、描画領域の中心から見たマウスカーソルの方向（角度） $direction$ と距離 $distance$ を計算します。 $atan2()$ は X 方向と Y 方向の変位から角度を求める C 標準関数です。距離 $distance$ は三平方の定理を使って求めます。

距離が描画領域の半径 ($kCanvasSize/2 - kEyeSize/2$) より長くなならないように $std::min$ を使って制限します。この処理のおかげで、目玉が描画領域の中心から半径 $kCanvasSize/2$ の円に収まります。

$\overrightarrow{eye_center}$ は描画領域中心から見た目玉の位置で、 $\overrightarrow{eye}$ はそれを、ウィンドウ左上を基準とした座標に変換したベクトルです。最終的に $\overrightarrow{eye}$ の座標を使って目玉を描画すれば完了です。

main() の方で目新しいのは SyscallReadEvent() を使って受け取ったイベントが AppEvent::kMouseMove のときの処理を書いてあるところですね。マウスが動くときのイベントが発生しますから、そのときに DrawEye() を呼び出して目玉を描けばオッケーですね。目玉を描く前に描画領域を白で塗りつぶしておかないと目玉が何個も描かれ、やがて太い円になってしまいます。

図 23.3 マウスカーソルを目玉が追いかけている



アプリを実行した様子を図 23.3 に示します。どうですか？ ちゃんとマウスカーソルを追いかけるように目玉（といってもただの四角）が動きますよ！ 楽しい！ ぐりぐり～！

23.2 お絵描きソフト (osbook_day23b)

マウスの動きが取れるようになったので、調子に乗ってボタンのクリックも取りたくなってきました。ボタンの状態が取れるとなると、マウスをクリックしたまま動かして線を引くお絵描きソフトのようなものが作れるはずですよ。

リスト 23.6 ボタンクリックに対応するイベント種別を追加 (app_event.hpp)

```
enum Type {
    kQuit,
    kMouseMove,
    kMouseButton,
} type;
```

早速、マウスのボタンクリックに対応するイベントを追加していきます。まずは AppEvent 構造体と Message 構造体にイベント種別を追加します。AppEvent のイベント種別の定義はリスト 23.6 のようになりました。

リスト 23.7 ボタンクリックイベントの引数 (app_event.hpp)

```
struct {
    int x, y;
    int press; // 1: press, 0: release
    int button;
} mouse_button;
```

イベント種別 kMouseButton に対応する引数も追加します。リスト 23.7 に引数用の構造体定義を示します。もちろん、これも AppEvent と Message の両方に追加します。press は、本来は真偽値を表すために bool を使いたところですが、C 言語では bool が使えないので整数型で代用します。

さて、ウィンドウのどこかをクリックしたときにクリックイベントを発生させたいです。しかし、今の実装ではウィンドウのどの部分をクリックしてもドラッグモードになってしまうことを思い出しました。お絵描きソフトを作るためには「クリックしたままマウスを動かして線を引く」という処理をやりたいので、クリックしたときにドラッグモードになってしまうと困ります。ドラッグモードを開始するのを、タイトルバーをクリックしたときに制限しましょう。

リスト 23.8 タイトルバーをクリックしたときだけドラッグモードに入る (mouse.cpp)

```
const bool previous_left_pressed = (previous_buttons_ & 0x01);
const bool left_pressed = (buttons & 0x01);
if (!previous_left_pressed && left_pressed) {
    auto layer = layer_manager->FindLayerByPosition(position_, layer_id_);
    if (layer && layer->IsDraggable()) {
        const auto y_layer = position_.y - layer->GetPosition().y;
        if (y_layer < ToplevelWindow::kTopLeftMargin.y) {
            drag_layer_id_ = layer->ID();
        }
        active_layer->Activate(layer->ID());
    } else {
        active_layer->Activate(0);
    }
} else if (previous_left_pressed && left_pressed) {
```

リスト 23.8 に示すように、タイトルバーがある部分 (Y 座標が 0 から ToplevelWindow::kTopLeftMargin.y の間のどこか) をクリックしたときだけ drag_layer_id_ 変数を設定することにしました。drag_layer_id_ が 0 以外のときだけドラッグ処理が行われますから、この改造によってタイトルバーの上で左ボタンを押したときだけドラッグモードが開始されるようになるはずです。

リスト 23.9 SendMessage() に 1 つ前のボタン状態も渡す (mouse.cpp)

```
if (drag_layer_id_ == 0) {
    SendMessage(newpos, posdiff, buttons, previous_buttons_);
}
```

リスト 23.9 に示す通り、SendMessage() の第 4 引数に 1 つ前のボタン押下状態 previous_buttons_ を渡すようにしました。新旧のボタン押下状態を比べることで、ボタンを押したり離したりした瞬間を検知できます。

リスト 23.10 ボタンの押下状態が変化したときにイベントを送る (mouse.cpp)

```
if (previous_buttons != buttons) {
    const auto diff = previous_buttons ^ buttons;
    for (int i = 0; i < 8; ++i) {
        if ((diff >> i) & 1) {
            Message msg{Message::kMouseButton};
            msg.arg.mouse_button.x = relpos.x;
            msg.arg.mouse_button.y = relpos.y;
            msg.arg.mouse_button.press = (buttons >> i) & 1;
            msg.arg.mouse_button.button = i;
            task_manager->SendMessage(task_it->second, msg);
        }
    }
}
```

SendMessage() を改造し、ボタンの状態が変化したときにイベントを送信する処理を加えました (リスト 23.10)。diff 変数には新旧のボタン状態の差分を表す値が入ります。^ は XOR 演算子です。previous_buttons と buttons はマウスの各ボタンの状態のビットマップになっているため、ビット単位の XOR を計算すれば変化があるビットが 1、変化がないビットが 0 となり、変化があるボタンを検出できます。

i 番目のボタンの押下状態が変化すると、diff 変数の i 番目のビットが 1 になります。すると (diff >> i) & 1 が真になって if 文が実行されます。このようにして、変化したボタンについてだけメッセージが生成されてタスクに送られます。メッセージには、ボタンが変化した際のマウスの座標 (ウィンドウ左上が基準)、ボタンが押されたのか、離されたのかを区別する値、ボタンの番号 (0 始まり) が含まれます。

リスト 23.11 ReadEvent() でボタンイベントに対応する (syscall.cpp)

```
case Message::kMouseButton:
    app_events[i].type = AppEvent::kMouseButton;
    app_events[i].arg.mouse_button.x = msg->arg.mouse_button.x;
    app_events[i].arg.mouse_button.y = msg->arg.mouse_button.y;
    app_events[i].arg.mouse_button.press = msg->arg.mouse_button.press;
    app_events[i].arg.mouse_button.button = msg->arg.mouse_button.button;
    ++i;
    break;
```

次に改造するのは syscall::ReadEvent() です。リスト 23.11 のように、ボタンイベント kMouseButton に対応する処理を追加しました。いつものように Message 構造体を AppEvent 構造体に変換するだけです。

リスト 23.12 ボタンを押したままドラッグすると線を引けるアプリ (paint/paint.cpp)

```

#include <stdio>
#include <stdlib>
#include <string>
#include "../syscall.h"

static const int kWidth = 200, kHeight = 130;

bool IsInside(int x, int y) {
    return 4 <= x && x < 4 + kWidth && 24 <= y && y < 24 + kHeight;
}

extern "C" void main(int argc, char** argv) {
    auto [layer_id, err_openwin]
        = SyscallOpenWindow(kWidth + 8, kHeight + 28, 10, 10, "paint");
    if (err_openwin) {
        exit(err_openwin);
    }

    AppEvent events[1];
    bool press = false;
    while (true) {
        auto [n, err] = SyscallReadEvent(events, 1);
        if (err) {
            printf("ReadEvent failed: %s\n", strerror(err));
            break;
        }
        if (events[0].type == AppEvent::kQuit) {
            break;
        }
        if (events[0].type == AppEvent::kMouseMove) {
            auto& arg = events[0].arg.mouse_move;
            const auto prev_x = arg.x - arg.dx, prev_y = arg.y - arg.dy;
            if (press && IsInside(prev_x, prev_y) && IsInside(arg.x, arg.y)) {
                SyscallWinDrawLine(layer_id, prev_x, prev_y, arg.x, arg.y, 0x000000);
            }
        }
        if (events[0].type == AppEvent::kMouseButton) {
            auto& arg = events[0].arg.mouse_button;
            if (arg.button == 0) {
                press = arg.press;
                SyscallWinFillRectangle(layer_id, arg.x, arg.y, 1, 1, 0x000000);
            }
        }
        else {
            printf("unknown event: type = %d\n", events[0].type);
        }
    }

    SyscallCloseWindow(layer_id);
    exit(0);
}

```

追加したシステムコールを使って作ったお絵描きソフトの実装を**リスト 23.12**に示します。この paint コマンドはウィンドウを開き、マウスの操作を待ちます。左ボタン（ボタン0）を押したままマウスを動かすことで線を引けます。

重要な変数は press です。この変数はマウスの左ボタンが押されているときに真となり、押され

ていないときに偽となります。この変数が真のときに限って線を引けば絵が描けるというわけです。kMouseButton イベントが発生した場合にこの変数が更新されます。

マウス移動イベント (kMouseMove) が発生したら実際に線を引きます。press が真となっていて、かつマウスカーソルがウィンドウの描画領域の内側にある (IsInside()) が真) ときに SyscallWinDrawLine() を使って線を引きます。マウス移動イベントにはマウスの移動量 dx、dy が含まれるので、それを使うと簡単に線を引けます。

図 23.4 お絵描きソフトでお絵描きしてみる



paint コマンドで絵を描く様子を図 23.4 に示します。スクリーンショットではよく分かりませんが、とてもスムーズに違和感なく線を引けました。トラックパッドで描いているのでブレブレですが、特に処理が引かかるようなことがなく、操作自体はとても快適です。

23.3 タイマコマンド (osbook_day23c)

本題である時間待ちの機能を作ります。OS 本体にあるタイマ機能を活用すればそれほど難しくならないと思います。

タイマ機能を見直していたら、タイムアウトしたことを通知するメッセージの送り先タスクが 1 に固定されていることが分かりました。TimerManager::Tick() の中で task_manager->SendMessage(1, m); としています。この仕組みのまま、アプリがタイマを設定したらタイムアウトがメインタスクに通知されます。メインタスクは、それがアプリによって設定されたタイマであることを認識したら、ターミナルタスクへと通知メッセージを転送する必要があります。

転送するにしても、タイマクラスの中に「転送先のタスク ID」を含めておかねばなりません。今はターミナルが 1 つだから良いものの、将来的にターミナルを増やそうと思ったときにどこに再転送

すればいいか判断するのに必要だからです。ということで**リスト 23.13**に示すようにタスク ID を表すフィールド `task_id_` と、取得するためのメソッド `TaskID()` を追加しました。

リスト 23.13 タイマクラスにタスク ID を持たせる (timer.hpp)

```
class Timer {
public:
    Timer(unsigned long timeout, int value, uint64_t task_id);
    unsigned long Timeout() const { return timeout_; }
    int Value() const { return value_; }
    uint64_t TaskID() const { return task_id_; }

private:
    unsigned long timeout_;
    int value_;
    uint64_t task_id_;
};
```

タイマクラスのコンストラクタに引数を追加しましたので、タイマを生成しているところをすべて修正する必要があります。例えば**リスト 23.14**のように `task_id` に数値を指定します。1 を指定することで今までと同じ挙動になります。その他の箇所も同様に修正します。

リスト 23.14 `task_id` にメインタスクの番号である 1 を指定する (main.cpp)

```
const int kTextboxCursorTimer = 1;
const int kTimer05Sec = static_cast<int>(kTimerFreq * 0.5);
timer_manager->AddTimer(Timer{kTimer05Sec, kTextboxCursorTimer, 1});
bool textbox_cursor_visible = false;
```

最終的にやりたいのは、アプリが設定したタイマのタイムアウト通知メッセージを、そのアプリが属するタスク（ターミナルタスク）に送信することです。これを実現するために、先ほど追加したタスク ID の使い方が 2 通り考えられます。

1 つはタイムアウト通知メッセージをメインタスク（タスク 1）に送る際、タスク ID をメッセージに含める方法です。メインタスクはメッセージに記されたタスク ID を見て、適切なタスクにメッセージを転送できます。もう 1 つはタイムアウト通知メッセージをそもそも当該タスクへと送ることです。この方法はメッセージがメインタスクを経由しないため効率的です。以降の改造ではこちらの方法を採用します。

リスト 23.15 タイマに記されたタスクへタイムアウト通知を送る (timer.cpp)

```
Message m{Message::kTimerTimeout};
m.arg.timer.timeout = t.Timeout();
m.arg.timer.value = t.Value();
task_manager->SendMessage(t.TaskID(), m);
```

`TimerManager::Tick()` の修正を**リスト 23.15**に示します。改造したのは 4 行目でタスク ID を指定する部分です。定数 1 の代わりに `t.TaskID()` を使うようにしました。

リスト 23.16 タイマ生成システムコール (syscall.cpp)

```

SYSCALL(CreateTimer) {
    const unsigned int mode = arg1;
    const int timer_value = arg2;
    if (timer_value <= 0) {
        return { 0, EINVAL };
    }

    __asm__("cli");
    const uint64_t task_id = task_manager->CurrentTask().ID();
    __asm__("sti");

    unsigned long timeout = arg3 * kTimerFreq / 1000;
    if (mode & 1) { // relative
        timeout += timer_manager->CurrentTick();
    }

    __asm__("cli");
    timer_manager->AddTimer(Timer{timeout, -timer_value, task_id});
    __asm__("sti");
    return { timeout * 1000 / kTimerFreq, 0 };
}

```

アプリがタイマを生成するシステムコール `syscall::CreateTimer()` を作りました。実装をリスト 23.16 に示します。第 1 引数はタイマの動作モード、第 2 引数はタイムアウト時に通知される任意の値です。第 3 引数はタイムアウト時刻を指定するためのミリ秒単位の値です。正常終了時の戻り値はタイマにセットされたタイムアウト時刻となります。OS 起動時を 0 とした時刻をミリ秒で表した値です。

動作モード `mode` の各ビットに意味を持たせることにしました。ビット 0 が 1 なら相対 (relative) モードです。相対モードでは現在時刻を基準として、`arg3` で指定された時間だけ後にタイマをセットします。ビット 0 が 0 なら絶対 (absolute) モードで、`arg3` はタイマがタイムアウトする時刻をミリ秒で表します。

最後にタイマを追加するときに `-timer_value` として符号を反転しているのは、OS 本体とアプリが生成したタイマを区別するためです。ターミナルタスクには、カーソル点滅のためにメインタスクから 0.5 秒おきにタイムアウト通知が送られてきます。このメッセージはアプリ起動中かどうかに関わらず送られてきますから、アプリが生成したタイマのタイマ値が被ってしまうと困ります。タイムアウトしたのが、アプリが設定したタイマか、カーソル点滅用タイマかが分からないからです。そこで、OS 本体が生成するタイマの値は 0 以上、アプリが生成するタイマの値は負の数として区別します。

リスト 23.17 タスク切り替え用タイマの値を正の数に修正する (timer.hpp)

```

const int kTaskTimerPeriod = static_cast<int>(kTimerFreq * 0.02);
const int kTaskTimerValue = std::numeric_limits<int>::max();

```

そういえばタスク切り替え用のタイマにはどんな値を使っていたかなーと思って調べてみると、`std::numeric_limits<int>::min()` を使っているではありませんか。これは負の値ですから良くないです。他の場所で使わなそうな値なら何でもいいので、**リスト 23.17** のように正の最大値にしておきました。(厳密に考えると、実はアプリは `std::numeric_limits<int>::min()` をタイマ値として指定できませんから、今のままでも大丈夫なんですけどね)

ここまでの改造で、アプリが `CreateTimer()` システムコールを使ってタイマを生成し、タイムアウト時にそのアプリが動くターミナルタスクへと通知メッセージが来るまでは出来ました。次に、アプリ自身がタイムアウトに気付く仕組みを作ります。これは `ReadEvent()` システムコールに機能を追加するのが楽でしょう。

リスト 23.18 AppEvent 構造体にタイムアウト通知イベントの引数を追加する (app_event.hpp)

```
struct {
    unsigned long timeout;
    int value;
} timer;
```

リスト 23.18 に AppEvent 構造体に追加した引数を載せました。タイムアウト時刻とタイマ値を記載します。このほか、イベント種別として `kTimerTimeout` も追加しておきましたよ。

リスト 23.19 ReadEvent() をタイムアウトイベントに対応させる (syscall.cpp)

```
case Message::kTimerTimeout:
    if (msg->arg.timer.value < 0) {
        app_events[i].type = AppEvent::kTimerTimeout;
        app_events[i].arg.timer.timeout = msg->arg.timer.timeout;
        app_events[i].arg.timer.value = -msg->arg.timer.value;
        ++i;
    }
    break;
```

リスト 23.19 に `syscall::ReadEvent()` の改造部分を示します。他のイベントと同じように、Message 構造体と AppEvent 構造体の変換をしています。他と違うのは、タイマ値が 0 以上の場合はイベントを無視していることです。タイマ値が 0 以上、すなわちアプリが生成したものではないタイマのタイムアウトイベントはアプリが取得できるべきではありません。

リスト 23.20 タイマの動作モードを表す定数を準備する (syscall.h)

```
#define TIMER_ONESHOT_REL 1
#define TIMER_ONESHOT_ABS 0
struct SyscallResult SyscallCreateTimer(
    unsigned int type, int timer_value, unsigned long timeout_ms);
```

リスト 23.20 のように、`syscall.h` に `SyscallCreateTimer()` のプロトタイプ宣言と、必要な定数の定義を追加しました。2 つの定数はタイマの動作モードを決めるものです。今は単発 (oneshot) モードしかなく、その中で時刻の設定が相対か絶対かを選ぶことができます。

リスト 23.21 指定されたミリ秒を待つタイマアプリ (timer/timer.cpp)

```
#include <stdio>
#include <stdlib>
#include <string>
#include "../syscall.h"

extern "C" void main(int argc, char** argv) {
    if (argc <= 1) {
        printf("Usage: timer <msec>\n");
        exit(1);
    }

    const unsigned long duration_ms = atoi(argv[1]);
    const auto timeout = SyscallCreateTimer(TIMER_ONESHOT_REL, 1, duration_ms);
    printf("timer created. timeout = %lu\n", timeout.value);

    AppEvent events[1];
    while (true) {
        SyscallReadEvent(events, 1);
        if (events[0].type == AppEvent::kTimerTimeout) {
            printf("%lu msec elapsed!\n", duration_ms);
            break;
        } else {
            printf("unknown event: type = %d\n", events[0].type);
        }
    }
    exit(0);
}
```

追加したシステムコールを使って時間待ちするだけの timer コマンドを作ってみました (リスト 23.21)。コマンドライン引数で待ち時間を指定すると、その時間だけ待った後に「xx msec elapsed!」というメッセージを出力します。久しぶりにウィンドウを開かないタイプのコマンドを作った気がしますね。

図 23.5 timer コマンドを使って 3 秒待っているところ



作ったコマンドの動作の様子を図 23.5 に示します。1000 と入力すると 1 秒が、3000 で 3 秒が計れています。うん、成功ですね。

23.4 アニメーション (osbook_day23d)

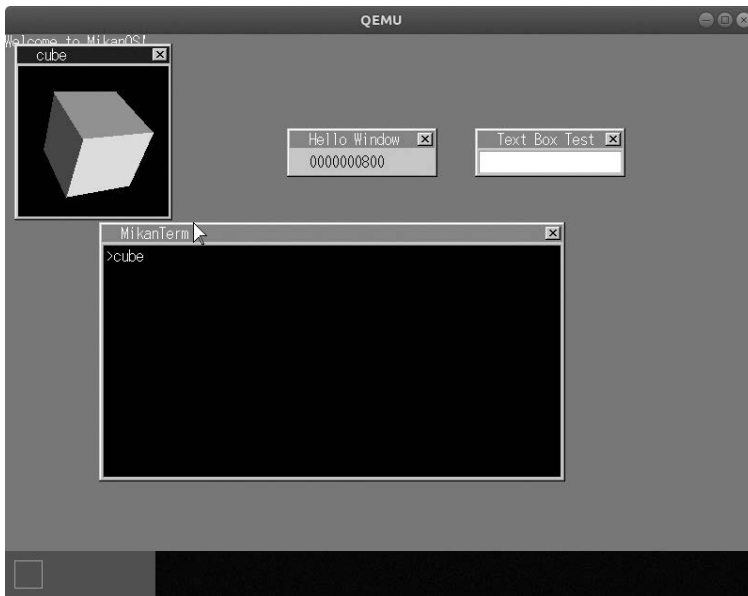
読者の皆さんはアニメを見たことがあると思いますが、アニメはどうしてキャラクターなどが動いているように見えるのでしょうか？ アニメーション、略してアニメは何枚もの静止画を連続して切り替えることで動きを表現します。コンピュータでも絵を短時間で切り替えていくことで動きを表現することができます。

アニメで大切なのは絵と絵の切り替えです。切り替えが速すぎても遅すぎても正しい動画になりません。そこで登場するのが先ほど作ったタイマです。タイマを使うと、絵の切り替え間隔を自由に制御できますから、ちょうどいい速度で再生されるアニメを実現できます。

この節では動きのある表現をするアプリを作りたいです。といっても、大量の絵を用意するのはとても大変ですので、計算によって生成できるような絵を用いてアニメを実現しようと思います。うーん、そうですね、立方体を回転させてみるというのはどうでしょうか？ 3次元の立方体（サイコロ）をX軸、Y軸、Z軸まわりにちょっとずつ回転させて、それを画面に表示するのです。

ということで cube というアプリを作りました。ソースコードは apps/cube/cube.cpp です。プログラムを理解するには三角関数、ベクトル、回転行列などの知識が必要ですが、本書では扱いません。なぜなら、それらの数学の知識は本書で扱うレベルの OS の作成には不要だからです。でも、改造して遊べるように主要な定数や変数については後で簡単に説明します。このアプリは参考文献 [1] の著者の川合氏が作ったプログラム (<http://k.osask.jp/wiki/?p20191125a>) を改造して作りました。

図 23.6 カラフルな立方体が回転している



cube アプリを起動すると図 23.6 のようになりました。6 面それぞれの色を変えているので回転の様子がよく分かります。正方形の描画領域に、立方体がちょっとずつ回転しながら描かれます。

リスト 23.22 cube アプリの定数や変数 (cube/cube.cpp)

```
using namespace std;

template <class T>
struct Vector3D {
    T x, y, z;
};

template <class T>
struct Vector2D {
    T x, y;
};

void DrawObj(uint64_t layer_id);
void DrawSurface(uint64_t layer_id, int sur);
bool Sleep(unsigned long ms);

const int kScale = 50, kMargin = 10;
const int kCanvasSize = 3 * kScale + kMargin;
const array<Vector3D<int>, 8> kCube{{
    { 1, 1, 1}, { 1, 1, -1}, { 1, -1, 1}, { 1, -1, -1},
    {-1, 1, 1}, {-1, 1, -1}, {-1, -1, 1}, {-1, -1, -1}
}};

const array<array<int, 4>, 6> kSurface{{
    {0,4,6,2}, {1,3,7,5}, {0,2,3,1}, {0,1,5,4}, {4,5,7,6}, {6,7,3,2}
```

```

});
const array<uint32_t, kSurface.size()> kColor{
    0xff0000, 0x00ff00, 0xffff00, 0x0000ff, 0xff00ff, 0x00ffff
};

array<Vector3D<double>, kCube.size()> vert;
array<double, kSurface.size()> centerz4;
array<Vector2D<int>, kCube.size()> scr;

```

リスト 23.22 に定数やグローバル変数を定義している箇所を示します。この中で特に重要なものを説明します。説明を理解しなくても本書の続きを読むには困りませんから、難しいと思ったら次の節までばっさり飛ばしてください。

まずは定数から。kScale は立方体の大きさに影響します。この値を N 倍にすればウィンドウや立方体が N 倍の大きさになります。kMargin は立方体と描画枠の余白の大きさです。kCanvasSize はピクセル単位で描画領域の縦および横の大きさを表します。

kCube は立方体の頂点座標の定義です。実際の立方体の大きさは kScale によって決まりますので、ここでは中心が原点の単位円みたいな感じに定義しておきます。kSurface は立方体の面を定義します。例えば先頭の要素 {0,4,6,2} は頂点 0、4、6、2 を順に巡ってできる面、という意味になります。kColor は各面の色を表します。

次に変数です。vert は回転後の立方体の頂点座標です。頂点座標なので要素の数は kCube と同じ 8 個です。centerz4 は立方体の各面の真ん中（重心）の Z 座標を格納する変数です。立方体の各面を、最も奥にある（=Z 座標が最大）面から描画するのに利用します。scr は画面（screen）座標を格納する変数です。3 次元空間で回転した立方体の各頂点を画面に射影したときの座標です。平面座標であり、単位はピクセル（整数）なので、要素の型は Vector2D<int> としています。

最後に関数の概要です。main() は立方体を回転させ、回転後の頂点座標を vert に書き込みます。同時に各面の Z 座標を求めて centerz4 に書き込みます。回転処理が終わったら DrawObj() を呼びます。

DrawObj() は回転後の立方体を layer_id で指定されたウィンドウに描画します。この関数はまず 3 次元空間の座標 vert を画面に射影して scr を設定します。次に、立方体の 6 面の中で最も奥にある面から順に DrawSurface() を呼びます。

DrawSurface() は指定された面をウィンドウに描画します。描画のやり方の基本的なアイデアはこんな感じ：立方体の各面は、画面に射影されると歪んだ四角形になります。その四角形を 1 ピクセル幅の横線になるように細切りにします。細切りの横線を、1 本ずつ SyscallWinFillRectangle() で描画します。

23.5 ブロック崩しゲーム (osbook_day23e)

OS の作成というより、OS の機能を使ったアプリの作成が続いてしまいましたが、次章からはまたちょっと難しい内容になると思うので、今のうちに楽しんでしまいましょう。

さて、そういえばまだ任意のキーを受け取れるようなイベントがありませんでした。任意のキーとタイマが使えるようになるとゲームを作ることができるようになります。この節では任意のキーを受け取れるように ReadEvent() システムコールを改造し、ゲームの例としてブロック崩しゲームを作って

みようと思います。

リスト 23.23 AppEvent 構造体にキーイベントを追加 (app_event.hpp)

```
struct {
    uint8_t modifier;
    uint8_t keycode;
    char ascii;
    int press; // 1: press, 0: release
} keypush;
```

リスト 23.23 のように、AppEvent 構造体にキー押下イベント用の引数 keypush を追加しました。ゲームを作るにあたり、キーが押された (press) ときに加え離された (release) ときもイベントを取れると嬉しいので、区別できるように press というメンバを入れてあります。紙面には載せませんが AppEvent::type に kKeyPush という定数を追加しました。

キーが押されると、最初に反応するのが xHCI (USB ホストコントローラ) ドライバです。xHCI ドライバは HID キーボードに対応するドライバである HIDKeyboardDriver の OnDataReceived() メソッドを呼び出します。このメソッドが、最終的に HIDKeyboardDriver::default_observer に登録されている関数を呼び出すという流れです。

この関数は今までキーが押されたときだけ呼び出される仕組みでした。これを、キーが押されたときに加えて離されたときにも呼び出されるように改造しなくてははいけません。

リスト 23.24 キー押下イベントオブザーバの型を修正 (usb/classdriver/keyboard.hpp)

```
using ObserverType = void (uint8_t modifier, uint8_t keycode, bool press);
```

まずはリスト 23.24 に示すように、HIDKeyboardDriver::ObserverType の定義を変更しました。キーを押したのか離したのかを区別できるように press という引数を追加しただけです

リスト 23.25 オブザーバに追加の引数を渡す (usb/classdriver/keyboard.cpp)

```
Error HIDKeyboardDriver::OnDataReceived() {
    std::bitset<256> prev, current;
    for (int i = 2; i < 8; ++i) {
        prev.set(PreviousBuffer()[i], true);
        current.set(Buffer()[i], true);
    }
    const auto changed = prev ^ current;
    const auto pressed = changed & current;
    for (int key = 1; key < 256; ++key) {
        if (changed.test(key)) {
            NotifyKeyPush(Buffer()[0], key, pressed.test(key));
        }
    }
    return MAKE_ERROR(Error::kSuccess);
}
```

それに合わせて、オブザーバに追加の引数を渡すように OnDataReceived() 側も改造します (リスト 23.25)。キーを離れたことを検出する機能を付け加えるために、処理のアルゴリズムを大きく変更しました。ちょっと詳しく処理を追います。

表 23.1 R をずっと押しながら、Q を離して S を押した場合のビット集合の値

	ビット 0:19	ビット 20:22 (QRS)	ビット 23:255
prev	0..0	110	0..0
current	0..0	011	0..0
changed	0..0	101	0..0
pressed	0..0	001	0..0

std::bitset という、ビット集合を扱う標準クラスを用いてキーの変化を計算します。2 つのビット集合 prev と current はどちらも、キーコード (0 ~ 255 の値) がビットの位置と対応します。キーコード N のキーが押されていたらビット N の値が 1 となります。例えば、R キーをずっと押しながら、Q キーを離して S キーを押した場合のビット集合の値は表 23.1 のようになります。

prev は直前のキーの押下状況を、current は現在のキーの押下状況を表します。ですので、両者を比べて変化があったビットを抽出すれば変化したキーだけを検出できます。それが changed です。このビット集合は、変化があったキーに対応するビットだけが 1 になっています。ビットが 0 → 1 または 1 → 0 に変化したことは、排他的論理和 (^) を使えば一発で求まります。変化したキーのうち、押された (0 → 1) キーを表すのが pressed です。

ビット集合の指定したビットの値を得るには test(ビット位置) というメソッドを使います。for ループを使って、changed の中で 1 になっているビット、すなわち変化したキーの番号を探します。変化したキーがあれば、押されたのか離されたのかに関係なく NotifyKeyPush() を呼び出します。キーが押されたかどうかという情報は、先ほど改造した ObserverType に適合するように第 3 引数に渡します。

リスト 23.26 実際のオブザーバが press を受け取るようにする (keyboard.cpp)

```
void InitializeKeyboard() {
    usb::HIDKeyboardDriver::default_observer =
        [](uint8_t modifier, uint8_t keycode, bool press) {
            const bool shift = (modifier & (kLShiftBitMask | kRShiftBitMask)) != 0;
            char ascii = keycode_map[keycode];
            if (shift) {
                ascii = keycode_map_shifted[keycode];
            }
            Message msg{Message::kKeyPush};
            msg.arg.keyboard.modifier = modifier;
            msg.arg.keyboard.keycode = keycode;
            msg.arg.keyboard.ascii = ascii;
            msg.arg.keyboard.press = press;
            task_manager->SendMessage(1, msg);
        };
}
```

`InitializeKeyboard()` で登録しているオブザーバの改造もします。**リスト 23.26** に示すように、オブザーバに第 3 引数を追加してキーの押下状態を受け取れるようにします。ああそうだ、紙面では紹介していませんでしたが `Message::arg::keyboard` に `press` フィールドを追加しています。この追加した `press` フィールドに対して、引数で受け取った `press` をコピーするというのが今回の主な改造ポイントです。

リスト 23.27 `ReadEvent()` システムコールでキーイベントを扱う (`syscall.cpp`)

```
case Message::kKeyPush:
    if (msg->arg.keyboard.keycode == 20 /* Q key */ &&
        msg->arg.keyboard.modifier & (kLControlBitMask | kRControlBitMask)) {
        app_events[i].type = AppEvent::kQuit;
        ++i;
    } else {
        app_events[i].type = AppEvent::kKeyPush;
        app_events[i].arg.keypush.modifier = msg->arg.keyboard.modifier;
        app_events[i].arg.keypush.keycode = msg->arg.keyboard.keycode;
        app_events[i].arg.keypush.ascii = msg->arg.keyboard.ascii;
        app_events[i].arg.keypush.press = msg->arg.keyboard.press;
        ++i;
    }
    break;
```

次に `syscall::ReadEvent()` も改造し、キーイベントをちゃんと扱えるようにしましょう。`Ctrl-Q` 以外のキーが入力されたとき、`kKeyPush` タイプのイベントを送信します (**リスト 23.27**)。

リスト 23.28 ターミナルにキーが 2 重に入力されないように修正 (`terminal.cpp`)

```
if (msg->arg.keyboard.press) {
    const auto area = terminal->InputKey(msg->arg.keyboard.modifier,
                                         msg->arg.keyboard.keycode,
                                         msg->arg.keyboard.ascii);
```

この状態で実行してみるとターミナルに文字が 2 重に入力されてしまうことが分かりました。ab とキーを押すと aabb と入力されてしまうのです。これは、ターミナルがキー押下イベントを処理する際に `press` を見ていないからです。**リスト 23.28** に示すように、キーが押された (`press` が真) ときだけキーの入力処理をするように `if` 文でガードするようにしました。

そういえば「Text Box Test」ウィンドウの方も 2 重入力されちゃいますね。同様に修正しておきます (**リスト 23.29**)。

リスト 23.29 テキストボックスも 2 重に入力されないように修正 (`main.cpp`)

```
if (auto act = active_layer->GetActive(); act == text_window_layer_id) {
    if (msg->arg.keyboard.press) {
        InputTextWindow(msg->arg.keyboard.ascii);
    }
} else {
```

ここまで作成したキーイベントを利用してブロック崩しゲームを作ってみました。例によってソースコードの全体的な説明はしませんので、興味のある方は `apps/blocks/blocks.cpp` を読んでください。

図 23.7 ブロック崩しゲームを遊んでいる様子



作成した `blocks` コマンドを起動すると図 23.7 のような画面が出ます。左右キーでバーを動かし、スペースキーでボールを発射します。ライフや得点機能、全部のブロックを消したときのクリア判定などは実装していませんが、とりあえずブロックを消す遊びはできます。MikanOS 初のゲームが作れました。やったね！

第 24 章

複数のターミナル

現状だとアプリケーションを 1 つ起動するとターミナルが待機状態になってしまい、複数のアプリケーションを同時に起動できません。せっかく複数のウィンドウを開けるしマルチタスクの機能もあるので、複数同時に起動してみたいですね。この章ではそれを実現してみたいと思います。

24.1 ターミナルを増やす (osbook_day24a)

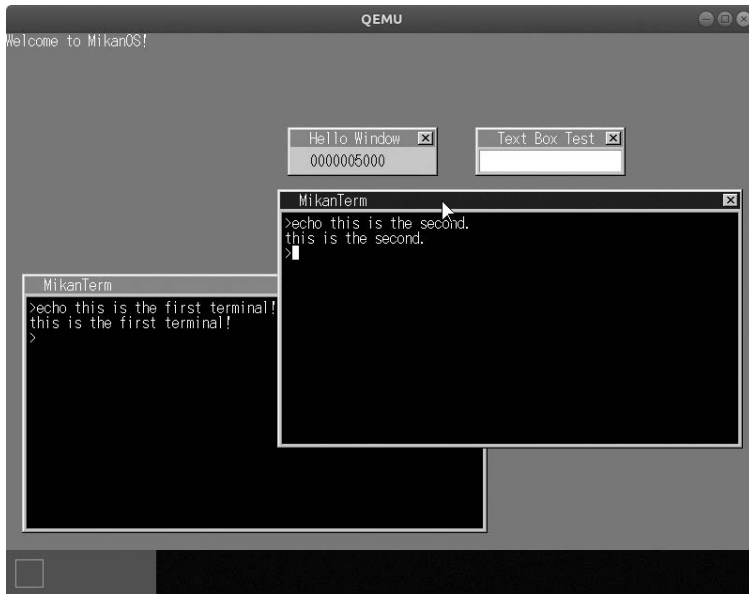
複数のアプリを同時に起動させようというのがこの章の目標です。さて、どうやりましょうか……。アプリを起動している仕組みを振り返ってみると、Terminal::ExecuteFile() の中で ELF ファイルをメインメモリに読み込んで起動しているのです。ターミナルを複数起動することができれば、ターミナルの数だけアプリを起動できるような気がします。ということで、まずは複数のターミナルを開けるようにしてみます。

リスト 24.1 F2 キーが押されたらターミナルを生成する (main.cpp)

```
case Message::kKeyPush:
    if (auto act = active_layer->GetActive(); act == text_window_layer_id) {
        if (msg->arg.keyboard.press) {
            InputTextWindow(msg->arg.keyboard.ascii);
        }
    } else if (msg->arg.keyboard.press &&
               msg->arg.keyboard.keycode == 59 /* F2 */) {
        task_manager->NewTask()
            .InitContext(TaskTerminal, 0)
            .Wakeup();
    } else {
```

リスト 24.1 のように F2 キーが押されたときにターミナルタスクを生成してみることにしました。とても安直なアイデアですが、うまくいくでしょうか？

図 24.1 複数のターミナルを開いてみた様子



F2 キーを押したところ、図 24.1 のように 2 つめのターミナルが開きました。2 つのターミナルのアクティブ、非アクティブを切り替えられますし、キー入力をそれぞれに振り分けることもできています。とりあえずは成功かなと思いますが、でもよく観察すると、2 つめのターミナルではカーソルが点滅しないことが分かります。また、片方のターミナルで `cube` を起動したまま、もう片方のターミナルで `lines` を起動してみたところ、`lines` のウィンドウは開くものの #UD 例外が発生してしまいました。次の節からはこれらの不具合に対処していきます。

24.2 カーソル点滅を自分で (osbook_day24b)

まずは 2 つめのターミナルでカーソルが点滅しない問題に対処します。今のカーソル点滅の仕組みは、メインタスクが管理するタイマがタイムアウトしたときに、そのタイムアウトメッセージをターミナルタスクにコピーして送信することで実現しています。この仕組みのままでは、ターミナルが増えたときにどのターミナルへメッセージを送るかをメインタスクが判断する必要があり、コードが複雑になりそうです。今の仕組みには、ターミナルが非アクティブなときにもカーソルが点滅してしまうという微妙な問題もありますね。

そこで、ターミナル自身がカーソル点滅を制御することにします。つまりターミナルタスクが起動したときにタイマを作動させ、独自に 0.5 秒を計るのです。そうしておけば、自動的にすべてのターミナルでカーソルが点滅するでしょう。

また、ターミナルのウィンドウが非アクティブになったらカーソルを点滅させなくするという機能もターミナル自身が実現できるようにしてみます。このためには、ターミナルタスクがウィンドウのアクティブ / 非アクティブの切り替わりを知る必要があります。ActiveLayer クラスを改造して、アクティブ状態が切り替わったらメッセージをタスクへ送信するようにします。

リスト 24.2 ターミナルタスク自身がタイマを制御する (terminal.cpp)

```
auto add_blink_timer = [task_id](unsigned long t){
    timer_manager->AddTimer(Timer{t + static_cast<int>(kTimerFreq * 0.5),
                                1, task_id});
};
add_blink_timer(timer_manager->CurrentTick());

bool window_isactive = false;

while (true) {
    __asm__("cli");
    auto msg = task.ReceiveMessage();
    if (!msg) {
        task.Sleep();
        __asm__("sti");
        continue;
    }
    __asm__("sti");

    switch (msg->type) {
    case Message::kTimerTimeout:
        add_blink_timer(msg->arg.timer.timeout);
        if (window_isactive) {
```

```

const auto area = terminal->BlinkCursor();
Message msg = MakeLayerMessage(
    task_id, terminal->LayerID(), LayerOperation::DrawArea, area);
__asm__("cli");
task_manager->SendMessage(1, msg);
__asm__("sti");
}
break;

```

< 中略 >

```

case Message::kWindowActive:
    window_isactive = msg->arg.window_active.activate;
    break;
default:
    break;
}
}

```

リスト 24.2 をみると今回の変更内容の全体像が分かります。ポイントは2つ。timer_manager->AddTimer() をターミナルタスク自身が呼び出すようにしたことと、kWindowActive というメッセージによりウィンドウのアクティブ状態をターミナルタスクに通知するようにしたことです。カーソル点滅用タイマを 0.5 秒間隔で常に動作するようにしつつ、ウィンドウがアクティブのとき (window_isactive が真のとき) だけ実際にカーソルを点滅させるようにしてみました。

期待する動作はこんな感じです。最初、ターミナルではカーソルが点滅しています。他のウィンドウをクリックするなどしてターミナルウィンドウが非アクティブになると、その時点の点滅状態のままカーソルの点滅が止まります。再度アクティブになったら、またカーソルの点滅が再開します。うまくいくかどうかは後で確認しましょうね。

リスト 24.3 アクティブウィンドウが切り替わったことをタスクに通知する (layer.cpp)

```

void ActiveLayer::Activate(unsigned int layer_id) {
    if (active_layer_ == layer_id) {
        return;
    }

    if (active_layer_ > 0) {
        Layer* layer = manager_.FindLayer(active_layer_);
        layer->GetWindow()->Deactivate();
        manager_.Draw(active_layer_);
        SendWindowActiveMessage(active_layer_, 0);
    }

    active_layer_ = layer_id;
    if (active_layer_ > 0) {
        Layer* layer = manager_.FindLayer(active_layer_);
        layer->GetWindow()->Activate();
        manager_.UpDown(active_layer_, manager_.GetHeight(mouse_layer_) - 1);
        manager_.Draw(active_layer_);
        SendWindowActiveMessage(active_layer_, 1);
    }
}

```


リスト 24.3 に改造した `ActiveLayer::Activate()` を示します。改造といっても2行追加しただけです。`SendWindowActiveMessage()` という関数を使って、ウィンドウのアクティブ状態そのウィンドウ（レイヤ）が属するタスクに通知します。

リスト 24.4 指定されたレイヤが属するタスクに `kWindowActive` メッセージを送る (`layer.cpp`)

```
Error SendWindowActiveMessage(unsigned int layer_id, int activate) {
    auto task_it = layer_task_map->find(layer_id);
    if (task_it == layer_task_map->end()) {
        return MAKE_ERROR(Error::kNoSuchTask);
    }

    Message msg{Message::kWindowActive};
    msg.arg.window_active.activate = activate;
    return task_manager->SendMessage(task_it->second, msg);
}
```

`SendWindowActiveMessage()` の実装を **リスト 24.4** に示します。指定されたレイヤが属するタスクを探してきて、そのようなタスクがあれば `kWindowActive` タイプのメッセージを送信します。とっても簡単な関数ですね。

リスト 24.5 `kWindowActive` タイプのメッセージの引数 (`message.hpp`)

```
struct {
    int activate; // 1: activate, 0: deactivate
} window_active;
```

`kWindowActive` タイプのメッセージの引数を表す構造体は **リスト 24.5** のようになっています。`activate` はウィンドウがアクティブかどうかを表すフィールドです。

紙面では紹介しませんが、`main.cpp` のメインループからカーソル点滅用タイマを他のタスクに送る処理を削除しました。もはや不要なので。

図 24.2 複数のターミナルで別々にカーソルが点滅する様子

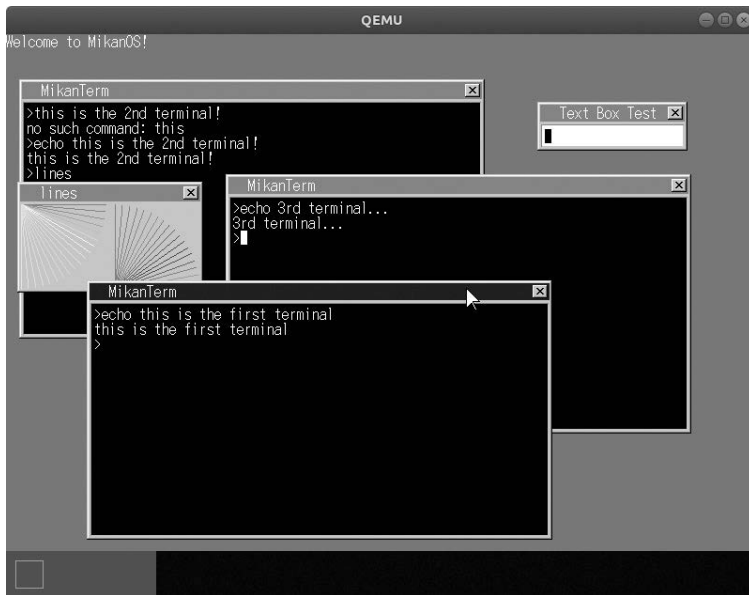
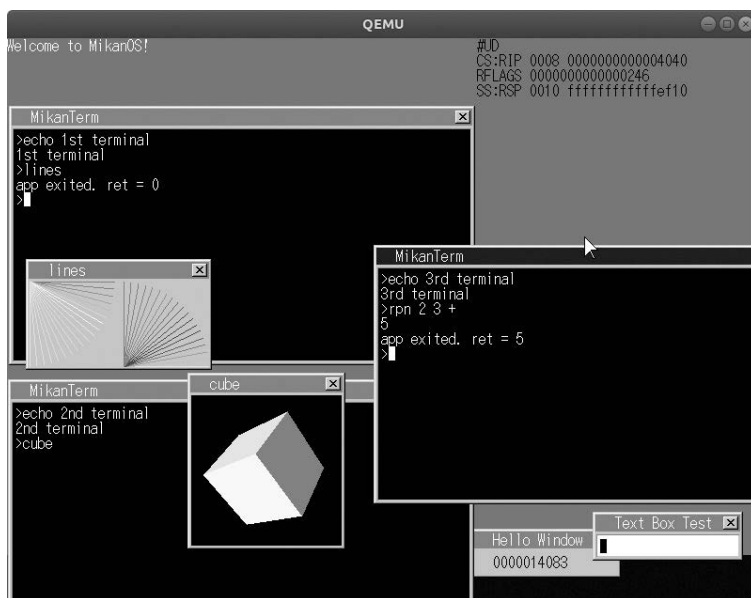


図 24.2 が実行した様子です。複数のターミナルを立ち上げると、それぞれ別々にカーソルの点滅が始まります。タスク A のウィンドウにあるカーソルと、「this is the first terminal」とあるターミナルのカーソルをよく見ると、両者は同期していませんね（タスク A の方はカーソルが表示されていますが、ターミナルのカーソルは消えています）。スクリーンショットだと分かりづらいですが、ウィンドウが非アクティブになるとカーソルの点滅が止まることも確認できました。

24.3 複数アプリの同時起動 (osbook_day24c)

F2 キーで複数のターミナルを出せてとっても楽しいのでいろいろ遊んでいたら、図 24.3 のように CPU 例外が発生して OS 全体がフリーズしてしまいました。状況はこんな感じ。まずデフォルトのターミナル（「1st terminal」と表示したターミナル）で `lines` アプリを実行しました。このアプリはウィンドウを閉じることなくすぐに終了します。その後 2 つ目のターミナルで `cube` アプリを立ち上げたまま、3 つ目のターミナルで `rpn` コマンドを実行しました。その直後に例外が発生しました。

図 24.3 複数のアプリを起動したら暴走して例外が発生した様子



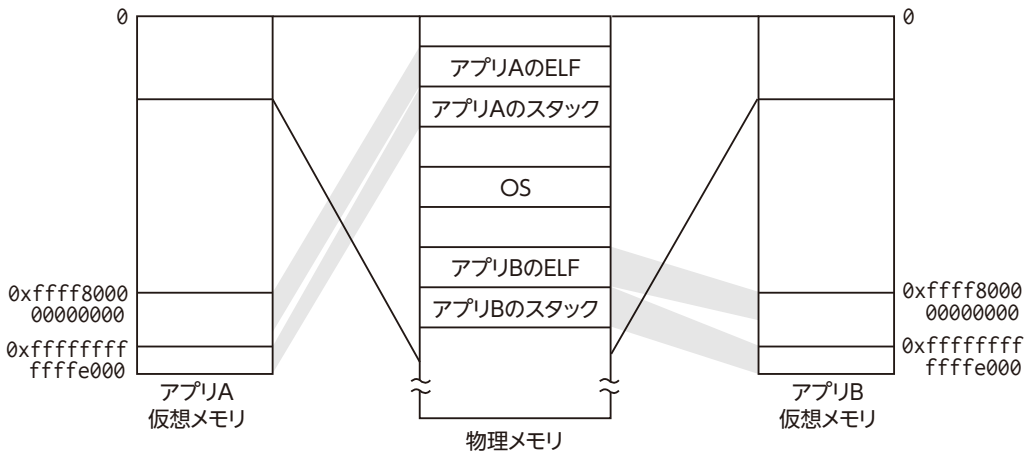
複数のアプリを同時に動かすと例外が出てしまう原因は何だと思いますか？ 筆者はなんとなく想像できました。そう、仮想メモリ領域が競合することです。それぞれのアプリは `0xfffff80000000000` からの仮想メモリ領域に自身の実行可能ファイルを配置したり、`0xffffffffffffe000` からの 4KiB をスタック領域として使ったりします。これらの領域が、複数のアプリ間で競合するのですね。

この問題はすでに「19.2 アドレス変換」で議論したのでした。そこでは、競合しないための方法として、リンクアドレスをずらす方法、アプリ読み込み時に再配置する方法、アドレスを変換する方法、という 3 種類を紹介しました。リンクアドレスをずらす方法では、たくさんの開発者がアプリを自由にならんと調整が難しくなります。仮にアプリ間で調整できたとしても、同じアプリを複数起動することは依然としてできないわけです。再配置は難しいので、本書では採用しません。

本書で採用する方法は、階層ページング構造の切り替えをすることです。アプリは専用の階層ページング構造を持つことにして、アプリを切り替えるときに CPU が参照する階層ページング構造も切り替えます^{*1}。そうすれば同じ仮想アドレスを異なる物理アドレスへと対応付けることが可能になります(図 24.4)。

* 1 この方法は、アプリ間でアドレス空間を分離できるため、あるアプリから他のアプリのデータを読めなくなり、セキュリティが向上するというメリットがあります。他の 2 つの手法にはないメリットです。

図 24.4 アプリ毎に独自の仮想アドレス空間を持つ様子



階層ページング構造の最上位のデータ構造はPML4です。CPUのCR3レジスタが、PML4が置いてある物理アドレスを保持していて、CPUはメモリアクセスをする際にこのPML4を参照して仮想アドレスから物理アドレスへ変換するのです。このあたりの話を忘れてしまった方は「**19.3 アプリのロードと実行**」を読み返してみてください。

まとめると、アプリ毎にPML4を頂点とする階層ページング構造を持たせ、アプリ切り替え時にCR3レジスタを書き換える、というのがやるべきことです。以降、プログラムに落とし込んでいきます。

実は、すでにTaskContext::cr3としてPML4のアドレスをタスク毎(=アプリ毎)に持ち、タスク切り替えのたびにCR3レジスタを書き換える仕組みは実装してあります。そのため、やるべきはタスク毎にPML4を生成してTaskContext::cr3に設定することですね。

リスト 24.6 実行可能ファイルをロードする前にPML4を設定する (terminal.cpp)

```
__asm__("cli");
auto& task = task_manager->CurrentTask();
__asm__("sti");

if (auto pml4 = SetupPML4(task); pml4.error) {
    return pml4.error;
}

if (auto err = LoadELF(elf_header)) {
    return err;
}
```

Terminal::ExecuteFile()の中の、アプリの実行可能ファイルのロード処理を変更します(リスト24.6)。アプリ独自の仮想アドレスに実行可能ファイルをロードするのが目標です。主な変更は、LoadELF()を呼ぶ前にSetupPML4()を用いてアプリ独自のPML4を構築するようにしたことです。

リスト 24.7 SetupPML4() は新しい PML4 を構築して有効化する (terminal.cpp)

```

WithError<PageMapEntry*> SetupPML4(Task& current_task) {
    auto pml4 = NewPageMap();
    if (pml4.error) {
        return pml4;
    }

    const auto current_pml4 = reinterpret_cast<PageMapEntry*>(GetCR3());
    memcpy(pml4.value, current_pml4, 256 * sizeof(uint64_t));

    const auto cr3 = reinterpret_cast<uint64_t>(pml4.value);
    SetCR3(cr3);
    current_task.Context().cr3 = cr3;
    return pml4;
}

```

SetupPML4() の実装をリスト 24.7 に示します。やることは大きく 3 つ、PML4 として用いるデータ構造を生成し、OS 用のページマップ設定をコピーし、構築した PML4 を CR3 と現在のタスクに設定します。注目していただきたいのが memcpy() の行です。この行は、現在 CR3 に設定されている PML4 から新しく生成した PML4 へ値をコピーしています。よく見ると、コピーする要素数が 256 個となっています。PML4 は 64 ビットの要素が 512 個並んだ配列ですから、前半だけコピーしていることになります。

PML4 の前半だけコピーしている理由は、OS 用のメモリマッピングだけをコピーしたいからです。PML4 の前半は仮想アドレスの前半に、PML4 の後半は仮想アドレスの後半に対応します。したがって PML4 の前半には OS 用の設定が、後半にはアプリ用の設定があるのです。OS 用のメモリマッピングはすべてのアプリで共通ですから、このように前半をそのままコピーしちゃえば良いのです。

前半だけ OS 用のマッピングが設定された PML4 を CR3 および現在のタスクに設定して SetupPML4() の処理は完了です。CR3 を変更した直後から CPU は新しい PML4 を使い始めます。そのため、そのままアプリの実行可能ファイルをロードすることで、新しい PML4 を使ってロード処理が実施されます。

構築した PML4 を CR3 に設定するだけでなく、current_task.Context().cr3 = cr3; のように現在のタスクのコンテキスト構造体にも設定していますね。これはタスク切り替えが起きたときのためです。タスク切り替えはいつ起きるか分かりません。タスク切り替えが起きて、再度このタスクに戻ってきたときに、ちゃんとこのタスク用の PML4 が CR3 に設定されていて欲しいのです。

リスト 24.8 アプリ終了後に PML4 も削除する (terminal.cpp)

```

const auto addr_first = GetFirstLoadAddress(elf_header);
if (auto err = CleanPageMaps(LinearAddress4Level{addr_first})) {
    return err;
}
return FreePML4(task);
}

```

次はアプリが終了した際のお片付けです。**リスト 24.8**に示すように、アプリが終了したら `CleanPageMaps()` で PML4 より下層の階層ページング構造を削除します。これは今まで通りです。今回変えたのはその後に `FreePML4()` を呼び出すようにした部分です。今までは PML4 はもともと OS 用として作ったものを使いまわしていたので削除してはいけませんでした。今はアプリ用に生成した PML4 を使っているため、削除する必要があるのです。

リスト 24.9 `FreePML4()` はアプリ用 PML4 を登録解除してから削除する (terminal.cpp)

```
Error FreePML4(Task& current_task) {
    const auto cr3 = current_task.Context().cr3;
    current_task.Context().cr3 = 0;
    ResetCR3();

    const FrameID frame{cr3 / kBytesPerFrame};
    return memory_manager->Free(frame, 1);
}
```

`FreePML4()` の実装を**リスト 24.9**に示します。この関数はまず、タスク構造体から PML4 の登録を解除し、CR3 をリセットして OS 用の PML4 を指すようにします (`ResetCR3()` の実装は後で紹介します)。これらの処理を行うことで、このタスク用の PML4 が CPU から参照されなくなります。

タスク構造体から登録を解除する処理と CR3 のリセットの順序は重要です。もし逆にしてしまったらどうなるかを考えます。仮に CR3 をリセットした直後にタスクスイッチが発生し、再度このタスクに処理が戻ってきた場面を想像してください。タスクスイッチによりこのタスクに戻ってくるとき、このタスク構造体にはまだ有効な PML4 がセットされています。したがって、その PML4 が CR3 に設定されてしまいます。せっかく CR3 をリセットしたのに、その効果がなくなってしまうわけです。

リスト 24.10 `ResetCR3()` は CR3 レジスタに OS 用の PML4 を設定する (paging.cpp)

```
void ResetCR3() {
    SetCR3(reinterpret_cast<uint64_t>(&pml4_table[0]));
}
```

`ResetCR3()` の実装は**リスト 24.10**のようにとっても簡単です。CR3 に、OS 用の PML4 を設定するだけです。`pml4_table` は `kernel/paging.cpp` の中で定義された静的変数で、OS 起動時に CR3 に設定されるものです。

ここまで改造してきて、肝心の実行可能ファイルの読み込み処理を担当する `LoadELF()` は変更していませんが大丈夫なのでしょうか？ はい、大丈夫です。なぜなら、`LoadELF()` を実行する前に CR3 を更新し、新しく作った PML4 を指すようにしているからです。`LoadELF()` は、そのときに CR3 に設定されている階層ページング構造に対して新しくエントリを追加します。ですので、自動的に新しく作った PML4 (とそこから指されている下層のページング構造) に設定を書いてくれるのです。だから `SetupPML4()` と `LoadELF()` を呼び出す順序を逆にするとうまく動きません。

図 24.5 複数のアプリを同時に起動した様子



改造は終わりました。複数のアプリが起動できるようになったはずですが。実行すると図 24.5 のようになりました。これは感動です。複数のアプリが**同時に**動いています。すごくないですか！？ いよいよマルチタスク OS と呼べるようになってきたのではないのでしょうか。

24.4 ウィンドウの重なりのバグ修正 (osbook_day24d)

たくさんターミナルやアプリを起動していて気づいたのですが、新しくウィンドウを開いたときにアクティブにはなるものの、重なり順が最上位にならず他のウィンドウの下に隠れちゃうのです。この動作のままでは、たくさんアプリを起動するときに地味に面倒ですし、新しく開いたウィンドウが一番上になるのが自然だと思いますので、ここで修正してしまいましょう。

まずはどこにバグがあるか調査です。ActiveLayer::Activate() の中に調査用のコードを仕込んで、このメソッドが呼ばれるたびに LayerManager::layer_stack_ の変化を観察してみました。このメンバ変数は LayerManager の中でプライベートとして宣言されていますので、そのままではクラス外から観察できません。そこで、次のように ActiveLayer クラスをフレンド宣言し、ActiveLayer クラスから LayerManager のプライベートなメンバを見るようにしました。

```
class LayerManager {
    .....
private:
    .....
    std::vector<Layer*> layer_stack_{};
    unsigned int latest_id_{0};
```

```
friend class ActiveLayer;
};
```

追加したのは `friend class` の行です。フレンド宣言とは `friend` キーワードを使って関数やクラスを指定することです。クラスのプライベートなメンバはそのクラスの外側から通常は見えませんが、フレンド宣言された関数やクラス（ここでは `ActiveLayer` クラス）からは見えるようになります。フレンド宣言はクラスのカプセル化の機能を壊すため濫用すべきではありませんが、一時的なデバッグで便利に使う分には問題ないと思います。

リスト 24.11 新規に作成したレイヤは最背面に表示してから最前面に移動する (layer.cpp)

```
manager_.UpDown(active_layer_, 0);
manager_.UpDown(active_layer_, manager_.GetHeight(mouse_layer_) - 1);
```

`stack_layer_` 変数の観察の結果、新規に作ったレイヤを `ActiveLayer::Activate()` により最前面に持ってくる処理のところに問題があることが分かりました。新規に作ったレイヤは初期状態で非表示（高さ -1）です。非表示の状態から一気にトップに持ってくるとき、表示中レイヤの数が 1 枚増えるため高さの計算が狂うようです。リスト 24.11 のように、いったん再底面（高さ 0）にしてから、改めてトップに持ってくるように改造しました。

リスト 24.12 マウスレイヤを作ってからターミナルを作る (main.cpp)

```
usb::xhci::Initialize();
InitializeKeyboard();
InitializeMouse();

task_manager->NewTask()
    .InitContext(TaskTerminal, 0)
    .Wakeup();
```

デバッグをしていたら気になるところを見つけたので直しておきました。リスト 24.12 の通り、ターミナルタスクの生成処理を `InitializeMouse()` の後に移動したのです。ターミナルタスクの最初の方で実行する `ActiveLayer::Activate()` は、その前に `ActiveLayer::SetMouseLayer()` でマウスレイヤが設定してあることを前提としています。マウスレイヤの設定は `InitializeMouse()` の中で行っていますから、ターミナルタスクの起動より前に呼び出しておく必要があった、ということです。普段は運良く問題がありませんが、タイミングによってはエラーになる潜在的なバグですね。

以上の修正を行ってビルドし、実行してみました。いろいろなアプリを起動させたり、F2 でターミナルを増やしてみたりしたときのウィンドウの重なりがきちんと修正されているようです。スクリーンショットは載せませんが、気になる人は手元で動かして確認してみてください。

24.5 ターミナル無しのアプリ起動 (osbook_day24e)

アプリを同時に起動できるようになり、調子に乗ってたくさんアプリを出して遊んでいると気が付きませんか？ ターミナル画面が無駄にたくさん出てしまっていて邪魔なことに。しかも、大半のターミナルはただアプリを起動して、そのアプリの終了を待つだけの役目しかありません。とても無駄な感じがします。

そこでこの節ではターミナルを開くことなくアプリを起動できる仕組みを作ろうと思います。そのために、アプリをどうやって起動させていたか簡単におさらいしておきましょう。

1. ユーザーがターミナルにアプリ名を入力する
2. ターミナルは、入力されたアプリ名が内蔵コマンド名と一致するかを調べ、一致していなければ `ExecuteFile()` メソッドを呼ぶ
3. `ExecuteFile()` がアプリ名に対応する実行可能ファイルをメモリに読み込み、エントリポイントを呼ぶ

このように、アプリの起動処理の大部分はターミナルが担当しています。ターミナルを開かずにアプリを起動できるようにする方法を考えてみると、ターミナルを使わないのではなく、ターミナルを裏で動かすけれども画面を表示しない、というようにすると楽な気がします。その方針で実装してみますね。

リスト 24.13 noterm 組み込みコマンドを増設 (terminal.cpp)

```
} else if (strcmp(command, "noterm") == 0) {
    task_manager->NewTask()
        .InitContext(TaskTerminal, reinterpret_cast<int64_t*>(first_arg))
        .Wakeup();
} else if (command[0] != 0) {
```

ターミナルの組み込みコマンドとして `noterm` を追加してみました (リスト 24.13)。このコマンドは `noterm <コマンドライン>` のように使います。指定したコマンドラインを、画面を表示しないモードで起動したターミナル上で実行します。F2 キーを押したときとほぼ同じようにターミナルタスクを起動します。1つ違うのは、`TaskTerminal()` の引数 `data` に、コマンドライン文字列へのポインタ `first_arg` を渡していることです。起動直後に表示するターミナルや F2 キーを押したときに開くターミナルでは `data` に 0 を指定していました。

リスト 24.14 コマンドラインが渡されたら画面を開かず実行する (terminal.cpp)

```
void TaskTerminal(uint64_t task_id, int64_t data) {
    const char* command_line = reinterpret_cast<char*>(data);
    const bool show_window = command_line == nullptr;

    __asm__("cli");
    Task& task = task_manager->CurrentTask();
    Terminal* terminal = new Terminal{task_id, show_window};
    if (show_window) {
        layer_manager->Move(terminal->LayerID(), {100, 200});
        layer_task_map->insert(std::make_pair(terminal->LayerID(), task_id));
        active_layer->Activate(terminal->LayerID());
    }
```

```

}
(*terminals)[task_id] = terminal;
__asm__("sti");

if (command_line) {
    for (int i = 0; command_line[i] != '\0'; ++i) {
        terminal->InputKey(0, 0, command_line[i]);
    }
    terminal->InputKey(0, 0, '\n');
}

```

そのようにして受け取ったコマンドライン引数を受け取る側は**リスト 24.14** のようになっています。引数 data は 0 か、もしくはコマンドライン文字列を指すポインタを整数に変換した値になっているはずです。ポインタ型に変換し直して変数 command_line に入れておきます。

その次の行は、ターミナル画面を表示するかどうかを表す変数 show_window を設定しています。コマンドライン文字列が指定されなかったとき、すなわち command_line がヌルポインタのときは画面を表示するので show_window を真とします。この変数が偽、すなわち画面を表示しないモードのときはレイヤ関連の操作をしないように、layer_manager->Move() などの処理は if 文で囲いました。

上記のプログラムの最大の特徴は、後ろの方にある for 文です。この for 文は、command_line で指定されたコマンドライン文字列をターミナルへ入力する役目があります。人間がキーを押して文字列を入力するのではなく、command_line の文字列を自動入力するということですね。キーが押されたとき用のメソッド terminal->InputKey() を利用すると、あたかもキー入力があったかのようにターミナルへ入力できます。

リスト 24.15 画面描画に関する処理を if 文で囲む (terminal.cpp)

```

switch (msg->type) {
case Message::kTimerTimeout:
    add_blink_timer(msg->arg.timer.timeout);
    if (show_window && window_isactive) {
        const auto area = terminal->BlinkCursor();
        Message msg = MakeLayerMessage(
            task_id, terminal->LayerID(), LayerOperation::DrawArea, area);
        __asm__("cli");
        task_manager->SendMessage(1, msg);
        __asm__("sti");
    }
    break;
case Message::kKeyPress:
    if (msg->arg.keyboard.press) {
        const auto area = terminal->InputKey(msg->arg.keyboard.modifier,
                                            msg->arg.keyboard.keycode,
                                            msg->arg.keyboard.ascii);

        if (show_window) {
            Message msg = MakeLayerMessage(
                task_id, terminal->LayerID(), LayerOperation::DrawArea, area);
            __asm__("cli");
            task_manager->SendMessage(1, msg);
            __asm__("sti");
        }
    }
}

```

```

        break;
    case Message::kWindowActive:
        window_isactive = msg->arg.window_active.activate;
        break;
    default:
        break;
}

```

TaskTerminal() の後半はリスト 24.15 のようにしました。大きくは変わっていません。show_window が真のときのみ画面描画に関する処理をするように変えただけです。kKeyPush を受け取ったときの処理で、terminal->InputKey() の呼び出しも含め全体を if (show_window) で囲めばいいと思うかもしれませんが。今のところはその通りです。そうしていないのは拡張性を高めたかったからです。terminal->InputKey() の呼び出しを残しておくことで、ターミナルに対してキー入力メッセージを送りつけ、ターミナルを外部のプログラムから操作するというような処理ができるようになります。

リスト 24.16 Terminal クラスのコンストラクタに画面表示の有無を指定 (terminal.cpp)

```

Terminal::Terminal(uint64_t task_id, bool show_window)
    : task_id_{task_id}, show_window_{show_window} {
    if (show_window) {
        window_ = std::make_shared<ToplevelWindow>(
            kColumns * 8 + 8 + ToplevelWindow::kMarginX,
            kRows * 16 + 8 + ToplevelWindow::kMarginY,
            screen_config.pixel_format,
            "MikanTerm");
        DrawTerminal(*window_->InnerWriter(), {0, 0}, window_->InnerSize());

        layer_id_ = layer_manager->NewLayer()
            .SetWindow(window_)
            .SetDraggable(true)
            .ID();

        Print(">");
    }
    cmd_history_.resize(8);
}

```

先ほどのプログラムで紹介しませんでした。Terminal クラスのコンストラクタに引数を追加しました(リスト 24.16)。画面表示の有無を表す引数 show_window です。この引数が真のときのみ、ウィンドウの生成とレイヤの追加を行います。show_window の値はコンストラクタに加え、その他のメソッドでも利用します。メンバ変数 show_window_ を新たに作りそこにコピーしておきます。

リスト 24.17 show_window_ が真のときのみカーソルを描く (terminal.cpp)

```
void Terminal::DrawCursor(bool visible) {
    if (show_window_) {
        const auto color = visible ? ToColor(0xffffffff) : ToColor(0);
        FillRectangle(*window_ -> Writer(), CalcCursorPos(), {7, 15}, color);
    }
}
```

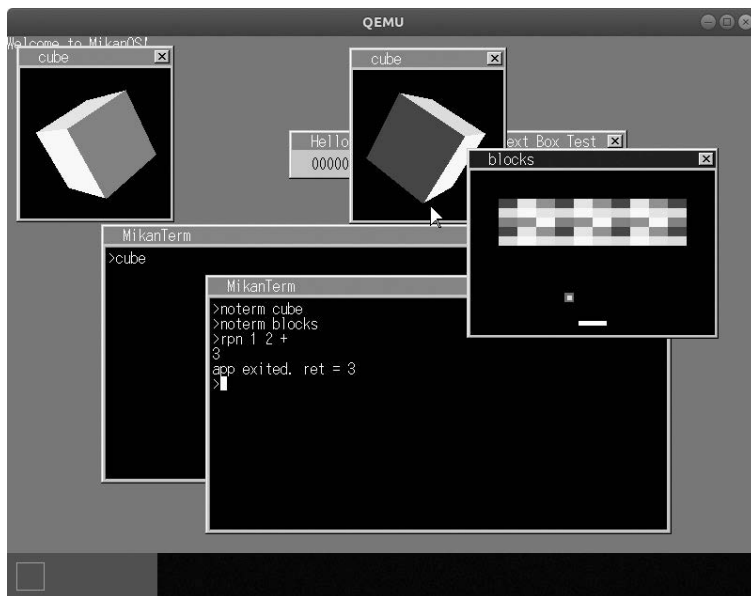
show_window_ は、例えば**リスト 24.17** のように利用します。show_window_ が真のときのみカーソルを描画します。show_window_ が偽のときはウィンドウが無い、すなわち window_ がヌルポインタになりますから、もし FillRectangle(*window_ -> Writer(), ……を実行してしまうとバグになります。ということで、show_window_ の値で条件分岐が必要になってくるのです。

リスト 24.18 show_window_ が真のときのみ描画する (terminal.cpp)

```
} else if (ascii != 0) {
    if (cursor_.x < kColumns - 1 && linebuf_index_ < kLineMax - 1) {
        linebuf_[linebuf_index_] = ascii;
        ++linebuf_index_;
        if (show_window_) {
            WriteAscii(*window_ -> Writer(), CalcCursorPos(), ascii, {255, 255, 255});
        }
        ++cursor_.x;
    }
} else if (keycode == 0x51) { // down arrow
```

ほかにもいくつか画面描画を行っている箇所があります。すべてを示すのはちょっと長いので、あと1つだけ例を示します。**リスト 24.18** を見てください。window_ を使っている行を if 文で囲っています。このように、window_ を使っている行をすべて if 文で囲みます。画面描画に関係ない部分まで if 文で囲んでしまうと、必要な処理が実行されなく可能性がありますから注意しましょう。

図 24.6 noterm コマンドを使ってアプリを起動した様子



作成した noterm コマンドを実行した様子を図 24.6 に示します。最初のターミナルでは noterm を使わずに cube を起動してみました。このときの挙動は従来と変わっていないように見えます。期待通り。次に F2 で新しいターミナルを立ち上げ、noterm cube を実行してみました。すると、cube アプリが起動しつつ、ターミナルでは次のプロンプト(>)が表示されるではありませんか！ ターミナルを次々に開くことなく、いくつでもアプリを起動できます。大成功ですね！

24.6 OSをフリーズさせるアプリ (osbook_day24f)

本章で、いくつもターミナルを開けるようになり、快適に複数のアプリを動かせるようになりました。機能としては申し分ないのですが、でも前から気になっていた重大な問題が 1 つ残っています。それは、アプリが OS 全体をフリーズさせられることです。

今の MikanOS では、アプリが CPU 例外を発生させた場合、それがわざとではなくても、例外ハンドラが呼び出されて OS 全体がフリーズしてしまいます。CPU 例外は、例えば不正なメモリアドレスに対して読み書きするとか（ヌルポインタアクセスなどが代表的）、hlt 命令を無理やり実行しようとするとか、そういう原因で発生します。そのようなバグをすべて無くすことは難しいですし、悪い人がわざとそういうアプリを作ることも考えられます。いずれにせよ、問題があるアプリによって OS 全体がフリーズさせられるのは嬉しくないですね。

本章の残りでアプリが発生させた例外で OS 全体がフリーズしない仕組みを作ろうと思います。その前に、今の段階でアプリが OS をフリーズさせられることを証明するために、わざと例外を起こすアプリを作ってみます。

リスト 24.19 fault コマンドの実装 (fault/fault.cpp)

```

#include <stdio>
#include <stdlib>
#include <string>
#include "../syscall.h"

extern "C" void main(int argc, char** argv) {
    const char* cmd = "hlt";
    if (argc >= 2) {
        cmd = argv[1];
    }

    if (strcmp(cmd, "hlt") == 0) {
        __asm__("hlt");
    } else if (strcmp(cmd, "wr_kernel") == 0) {
        int* p = reinterpret_cast<int*>(0x100);
        *p = 42;
    } else if (strcmp(cmd, "wr_app") == 0) {
        int* p = reinterpret_cast<int*>(0xffff8000ffff0000);
        *p = 123;
    } else if (strcmp(cmd, "zero") == 0) {
        volatile int z = 0;
        printf("100/%d = %d\n", z, 100/z);
    }

    exit(0);
}

```

リスト 24.19 にわざと例外を起こす `fault` コマンドの実装を示します。コマンドライン引数でいくつかの種類の例外を選ぶように工夫しました。`hlt` はその名の通り `hlt` 命令を実行しようとしています。`hlt` は特権命令ですのでアプリからは実行できず、一般保護例外が出るはずですが。`wr_kernel` は OS 用のメモリ領域（仮想アドレス空間の前半に OS 用領域があったね）に書き込みをしようとしています。`wr_app` はアプリ用のアドレス空間ではあるものの、ページエントリが存在しない領域へ書き込みを試みます。いずれもページフォルトが発生するでしょう。`zero` はゼロ除算例外を発生させます。

リスト 24.20 ExitApp() はアプリを強制終了する (asmfunc.asm)

```
global ExitApp ; void ExitApp(uint64_t rsp, int32_t ret_val);
ExitApp:
    mov rsp, rdi
    mov eax, esi

    pop r15
    pop r14
    pop r13
    pop r12
    pop rbp
    pop rbx

    ret ; CallApp の次の行に飛ぶ
```

SyscallEntry.exit の処理をほとんどコピペして作った ExitApp() の実装をリスト 24.20 に示します。この関数は第 1 引数に OS 用スタックポインタを、第 2 引数に CallApp() の戻り値となる値を取ります。処理の詳しい説明は「**21.5 終了システムコール**」にあります。コメントにあるように、最後の ret が実行されると Terminal::ExecuteFile() の CallApp() の次の行に戻ります。

リスト 24.21 KillApp() は現在実行中のアプリを強制終了する (interrupt.cpp)

```
void KillApp(InterruptFrame* frame) {
    const auto cpl = frame->cs & 0x3;
    if (cpl != 3) {
        return;
    }

    auto& task = task_manager->CurrentTask();
    __asm__("sti");
    ExitApp(task.OSStackPointer(), 128 + SIGSEGV);
}

#define FaultHandlerWithError(fault_name) \
    __attribute__((interrupt)) \
    void IntHandler ## fault_name (InterruptFrame* frame, uint64_t error_code) { \
        KillApp(frame); \
        PrintFrame(frame, "#" #fault_name); \
        WriteString(*screen_writer, {500, 16*4}, "ERR", {0, 0, 0}); \
        PrintHex(error_code, 16, {500 + 8*4, 16*4}); \
        while (true) __asm__("hlt"); \
    }

#define FaultHandlerNoError(fault_name) \
    __attribute__((interrupt)) \
    void IntHandler ## fault_name (InterruptFrame* frame) { \
        KillApp(frame); \
        PrintFrame(frame, "#" #fault_name); \
        while (true) __asm__("hlt"); \
    }
```

リスト 24.21 は `ExitApp()` を利用してアプリを強制終了させる `KillApp()` と、それを呼び出す例外ハンドラの装束です。今回追加した `KillApp()` はスタックフレームに記録されている CS の値、言い換えると例外の原因となったプログラムを実行していたときの CS の値を見て、その例外がアプリ起因かどうかを調べます。CPL=3 以外のときはアプリ起因の例外ではないため、アプリの強制終了は行いません。CPL=3 であれば `ExitApp()` を使ってアプリを強制終了させます。

`ExitApp()` を呼ぶ前に `sti` 命令を実行しています。理由は、アプリを強制終了するケースでは明示的に IF=1 に設定する必要があるからです。普通、割り込みや例外により呼び出された割り込みハンドラから元のプログラムに戻る際は `iret` 命令を使います*¹。iret 命令はスタックに保存されている RFLAGS の値を書き戻すことで IF=1 とします*²。しかし、アプリを強制終了するケースでは `ExitApp()` を呼び出したきり割り込みハンドラには戻ってきません。そのため `iret` が実行されず、IF の値が元に戻らないのです。そのため `sti` 命令を使って明示的に IF=1 に変え、次なる割り込みや例外が受け付けられる状態にしておきます。

アプリを起動してから例外が発生し、強制終了するまでの流れを簡単にまとめてみます。

1. `CallApp()` によりアプリが起動する。このとき、RSP にアプリ用スタックが設定され、CS はアプリ用 (CPL=3) になる。
2. アプリが CPU 例外を起こす。
3. CPU は IDT を調べ、適切な割り込みハンドラを呼び出す。CPU は CS に OS 用の値 (kKernelCS) を、IF ビットに 0 を設定する。
4. 割り込みハンドラは `KillApp()` を呼び出す。スタックフレームに保存されている CS の CPL フィールドは CPL=3 なのでアプリ実行中の例外と判断される。
5. `sti` により IF=1 となり、次の割り込みが許可される。
6. `KillApp()` は `ExitApp()` を呼び出す。`ExitApp()` は RSP および各レジスタを OS 用に復元し、`CallApp()` の次の行にジャンプする。

リスト 24.22 フラットバイナリ形式のアプリを実行させない (terminal.cpp)

```
auto elf_header = reinterpret_cast<Elf64_Ehdr*>(&file_buf[0]);
if (memcmp(elf_header->e_ident, "\x7f" "ELF", 4) != 0) {
    return MAKE_ERROR(Error::kInvalidFile);
}
```

さて、これで必要な改造は終わりなのですが、最後にフラットバイナリ形式のアプリ実行機能を削除しておきます。ELF 形式に対応する前にフラットバイナリ形式のアプリに対応したことを覚えていますか？ フラットバイナリ形式のアプリは CPL=0、つまり OS モードのまま実行するように作っていたため、今回追加した保護機能が動きません。もうフラットバイナリ形式に対応している必要性もなくなったと思いますから思い切って機能を削除しておきましょう。**リスト 24.22** のように、ELF 形式以外のファイルを実行しようとするエラーを出すようにしました。

*1 そんな命令は書いた記憶がないよ、と思うでしょう。それはその通りで、`__attribute__((interrupt))` を付けた関数ではコンパイラが自動的に `iret` 命令を使うようにしてくれるため、自分では書かなくて済んでいるのです。

*2 もちろん、スタックに保存されている RFLAGS の IF ビットの値が 0 だったら `iret` 命令が RFLAGS を復元したところで IF=1 とはなりません。

図 24.8 例外によりアプリが強制終了させられた様子



図 24.8 に前節と同じように `fault` コマンドを実行した様子を示します。同じコマンドを実行したのですが、今回の改造によって OS がフリーズすることなくアプリが強制終了されたことが分かります。アプリの終了コードは 139 となっています。これは `ExitApp()` の引数に `128 + SIGSEGV` を設定したためです。いまのところ、どの例外で終了したとしても同じエラーコードになります。

第 25 章

アプリで ファイル読み込み

アプリの同時起動も快適にできるようになり、とても OS っぽくなってきました。見た目だけなら立派な OS だと感じるレベルではないでしょうか。しかし、機能の面から見るとアプリができる操作がまだまだ少なく未熟です。一般的な OS であれば、アプリがメモリを確保したりファイルを読み書きしたりする機能は最も基本となる機能として搭載されています。この章では、MikanOS にアプリによるファイル読み込み機能を追加してみます。ファイル書き込みは次章で取り上げます。

25.1 ディレクトリ対応 (osbook_day25a)

ファイル読み込みを試すために新しいアプリを作ろうとしてふと気付きました。アプリの数が増えすぎて、ls コマンドで全体が表示しきれなくなっています。これはちょっと困ります。これ以上アプリが増えたら、何のファイルがあるか思い出すために ls を実行しても一部しか分からなくなってしまうから。これを解決するために、この節ではディレクトリに対応してみたいと思います。

ディレクトリ (directory) とは複数のファイルをしまっておく場所のことです。OS によっては「フォルダ」と呼ぶこともあります。ディレクトリの中にディレクトリを入れることができる場合が多く、FAT ファイルシステムも例外ではありません。ディレクトリの中にディレクトリを入れられるということは、ディレクトリの階層構造を作ることができます。この階層構造をディレクトリツリーと呼んだりします。

図 25.1 ディレクトリツリー



OS 起動用の USB メモリを例にすると、ディレクトリツリーは図 25.1 のようになります。図の中にルートディレクトリというものがありますね。これはディレクトリツリーの最も上位に位置するディレクトリです。ルートディレクトリだけは他のディレクトリに格納されておらず、単独で存在しています。その他のディレクトリはすべて、いずれかのディレクトリに格納されます。

ディレクトリツリーの中で特定のディレクトリやファイルを指定する便利な方法に**パス** (path) があります。例えばブートローダのファイル BOOTX64.EFI を指定するパスは /EFI/BOOT/BOOTX64.EFI となります。パスは一般的に「道」という意味で、ファイルシステムの分野でのパスは、あるディレクトリからあるディレクトリやファイルまでの道のことです。パスはディレクトリ名を / で区切って並べたものです。区切り文字は OS によって変わりますが、Linux では / を使いますので、MikanOS もそれに conforms。

先頭を / で書き始めたパスのことを特に**絶対パス**と呼びます。先頭の / はルートディレクトリを表します。ルートディレクトリには名前がありませんから、先頭に / を書いたらそれがルートディレクトリだということになっているのです。絶対パスに対して、ルートディレクトリ以外を起点として書いたパスを**相対パス**と呼びます。例えば BOOT/BOOTX64.EFI は相対パスです。絶対パスは 1 通りの解釈になりますが、相対パスはどこを起点とするかで指すファイルが異なるという特徴があります (良い悪いではなく、そういう特徴があるよということです)。

「17.3 ディレクトリエントリ」で説明したように、FAT ファイルシステムではディレクトリを特殊なファイルとして表現するのです。ルートディレクトリもその他のディレクトリも、ディレクトリエントリの配列を中身として持つファイルです。ファイルなのかディレクトリなのかは、そのファイルの属性 DIR_Attr (表 17.3 FAT ファイルシステムのディレクトリエントリの構造) で決まります。DIR_Attr の値が ATTR_DIRECTORY だったらそのファイルはディレクトリを表します。ディレクトリだということが分かったら、ファイルの内容をディレクトリエントリの配列だと思って読み取ることで、そのディレクトリに含まれるファイルの一覧を得ることができます。

リスト 25.1 ls コマンドをディレクトリに対応させる (terminal.cpp)

```

} else if (strcmp(command, "ls") == 0) {
    if (first_arg[0] == '\\0') {
        ListAllEntries(this, fat::boot_volume_image->root_cluster);
    } else {
        auto [ dir, post_slash ] = fat::FindFile(first_arg);
        if (dir == nullptr) {
            Print("No such file or directory: ");
            Print(first_arg);
            Print("\\n");
        } else if (dir->attr == fat::Attribute::kDirectory) {
            ListAllEntries(this, dir->FirstCluster());
        } else {
            char name[13];
            fat::FormatName(*dir, name);
            if (post_slash) {
                Print(name);
                Print(" is not a directory\\n");
            } else {
                Print(name);
                Print("\\n");
            }
        }
    }
}
} else if (strcmp(command, "cat") == 0) {

```

早速 ls コマンドを改造してディレクトリに対応させてみます。**リスト 25.1** は改造後の ls コマンドの実装です。ls コマンドはコマンドライン引数としてパス文字列を受け取ることにしました。Linux の ls コマンドもパス文字列を受け取ることができて、例えば foo/bar というパスを指定すると、foo ディレクトリの中にある bar というファイルを表示する、という意味になります。もし bar がディレクトリだったら bar 自身ではなく中にあるファイルが一覧されます。

処理を大きく変えました。今まではルートディレクトリ内のファイル一覧を出力する処理を直接書いていました。改造後は fat::FindFile() を使うようにしたのが大きな変更です。後で紹介しますが、今まではルートディレクトリの中に入っているファイルを検索することしかできなかった FindFile() を改造し、指定されたパスを解釈してディレクトリツリーを潜ってファイルを検索する機能を追加しました。

改造後の FindFile() は 2 つの値の組を返します。1 つは検索して見つけたファイルまたはディレクトリを表すディレクトリエントリ (DirectoryEntry) です。もう 1 つは / 記号がその直後にあるかどうかを表す真偽値です。戻り値の具体例を表 25.1 に示します。

表 25.1 `fat::FindFile()` の引数と戻り値の例

引数	戻り値 1	戻り値 2
"memmap"	memmap を表すエントリ (ファイル)	false
"memmap/hoge"	memmap を表すエントリ (ファイル)	true
"efi/boot"	boot を表すエントリ (ディレクトリ)	false
"efi/boot/"	boot を表すエントリ (ディレクトリ)	true
"hoge/"	nullptr	true

`FindFile()` の戻り値を利用して場合分けをしています。指定されたパスに該当するエントリが無い場合は `dir` が `nullptr` となります。その場合は「No such file or directory」と表示して処理を終えます。指定されたパスに該当するエントリがディレクトリであれば、そのディレクトリの内容を一覧表示します (`ListAllEntries()` が一覧表示を担当する関数です)。

`FindFile()` の戻り値が `nullptr` でもなくディレクトリでもない場合、2 つ目の戻り値 (`post_slash`) でさらに場合分けをしています。ちょっと説明が細くなる割にはあんまり重要な処理でもありませんから、詳しい説明は省略します。簡単に言えば、`ls memmap` と指定したら MEMMAP と、`ls memmap/hoge` と指定したら MEMMAP is not a directory と表示するための場合分けとなっています。

リスト 25.2 `FindFile()` をディレクトリ階層に対応させる (fat.cpp)

```
std::pair<DirectoryEntry*, bool>
FindFile(const char* path, unsigned long directory_cluster) {
    if (path[0] == '/') {
        directory_cluster = boot_volume_image->root_cluster;
        ++path;
    } else if (directory_cluster == 0) {
        directory_cluster = boot_volume_image->root_cluster;
    }

    char path_elem[13];
    const auto [ next_path, post_slash ] = NextPathElement(path, path_elem);
    const bool path_last = next_path == nullptr || next_path[0] == '\0';

    while (directory_cluster != kEndOfClusterchain) {
        auto dir = GetSectorByCluster<DirectoryEntry>(directory_cluster);
        for (int i = 0; i < bytes_per_cluster / sizeof(DirectoryEntry); ++i) {
            if (dir[i].name[0] == 0x00) {
                goto not_found;
            } else if (!NameIsEqual(dir[i], path_elem)) {
                continue;
            }

            if (dir[i].attr == Attribute::kDirectory && !path_last) {
                return FindFile(next_path, dir[i].FirstCluster());
            } else {
                // dir[i] がディレクトリではないか、パスの末尾に来てしまったので探索をやめる
                return { &dir[i], post_slash };
            }
        }
    }
}
```

```
        directory_cluster = NextCluster(directory_cluster);
    }

not_found:
    return { nullptr, post_slash };
}
```

`fat::FindFile()` の実装をリスト 25.2 に示します。この関数はもともとリスト 18.3 で作りました。当時は 1 つのディレクトリ内を検索することしかできませんでした。今回の改造によってディレクトリを潜ってファイルを探索ようになりました。while ループの中央を見ると分かりますが、この関数は自分自身を呼び出す再帰関数になっています。ls `efi/boot` を実行したとして、`FindFile()` の大雑把な動作は表 25.2 のようになります。

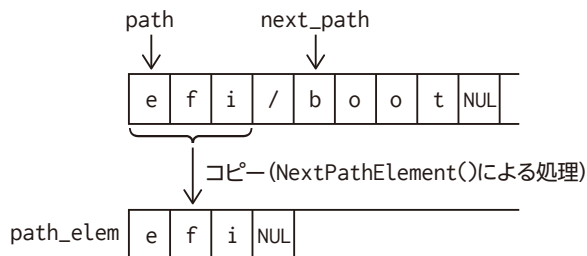
表 25.2 `FindFile()` の動作例

呼び出し	引数 path	path_elem	path_last	動作
1 回目	"efi/boot"	"efi"	false	ルートディレクトリの中の efi を開く
2 回目	"boot"	"boot"	true	boot に対応するエントリと false を返す

ざっと先頭から処理を眺めてみます。まずは `path` と `directory_cluster` の値を調整します。`path` の先頭が `/` であれば絶対パスですから、`directory_cluster` にはルートディレクトリを表す値を入れます。`directory_cluster` が省略されていた場合もルートディレクトリを表す値を入れることにします。

次に `NextPathElement(path, path_elem)` という処理を行います。この関数は、`path` で指定したパス文字列を `/` で区切って、その先頭要素を取り出して変数 `path_elem` にコピーします。`FindFile("efi/boot")` の 1 回目の呼び出しにおける `path` と `path_elem`、`next_path` の関係を図 25.2 に示します。

図 25.2 `path` と `path_elem`、`next_path` の関係



変数 `path_last` は、`path_elem` にコピーされた文字列がパスの末尾かどうかを表します。真ならパスはそれで終わり、偽ならまだ続きがあることを示します。

`FindFile()` の中央にあるループは `path_elem` と一致する名前のファイルまたはディレクトリを探索処理です。`directory_cluster` で指定されたディレクトリ内のエントリを 1 つずつ調べていっ

て、目的の名前と一致するエントリを探します。名前が一致するエントリが最後まで見つからなかったら検索失敗となり、`not_found:` の処理によりヌルポインタを返します。

目的の名前を持つエントリが見つければその種類に応じた処理をします。そのエントリがディレクトリであり、パスの末尾でなければディレクトリツリーを1段潜ります。ディレクトリでないとき、またはパスの末尾に到達したときは見つけたエントリをそのまま返します。

実装を紹介していない関数はいくつかあります。まず `fat::FormatName()` です。これは引数で渡したファイル名（基本名と拡張子）を整形して配列にコピーするだけの関数です。実装は単純なので紹介は割愛します。次に `ListAllEntries()` ですが、これはもともとの `ls` コマンドの実装とほぼ同じです。単に指定されたディレクトリの中身を一覧表示するだけですから、これも紹介は割愛します。最後に `NextPathElement()` です。これはちょっと重要な関数ですので**リスト 25.3** に実装を示します。

リスト 25.3 `NextPathElement()` はパスを / で区切った最初の要素を取得する (terminal.cpp)

```
std::pair<const char*, bool>
NextPathElement(const char* path, char* path_elem) {
    const char* next_slash = strchr(path, '/');
    if (next_slash == nullptr) {
        strcpy(path_elem, path);
        return { nullptr, false };
    }

    const auto elem_len = next_slash - path;
    strncpy(path_elem, path, elem_len);
    path_elem[elem_len] = '\0';
    return { &next_slash[1], true };
}
```

`NextPathElement()` は多目的な関数です。目的は大きく3つあります。1つ目は `path` を / で区切った最初の要素を `path_elem` にコピーすることです。2つ目は最初の要素の末尾に / があるかどうかを返すことで、3つ目はパスの次の要素を指すポインタを返すことです。

`NextPathElement()` の処理を理解するキモは `strchr(path, '/')` です。これは `path` の先頭から '/' を検索し、見つけた位置を返します。 `path` が `"ab/cd"` だったとすると、戻り値は `path[2]` を指すポインタとなります。 `path` が `"abc/d"` なら戻り値は `path[3]` を指すポインタです。もし `path` の中に '/' が1つもなければヌルポインタとなります。

リスト 25.4 `FindFile()` を使っている箇所を修正する (terminal.cpp)

```
} else if (command[0] != 0) {
    auto [ file_entry, post_slash ] = fat::FindFile(command);
    if (!file_entry) {
        Print("no such command: ");
        Print(command);
        Print("\n");
    } else if (file_entry->attr != fat::Attribute::kDirectory && post_slash) {
        char name[13];
        fat::FormatName(*file_entry, name);
        Print(name);
        Print(" is not a directory\n");
    } else if (auto err = ExecuteFile(*file_entry, command, first_arg)) {
```



```

    Print("failed to exec file: ");
    Print(err.Name());
    Print("\n");
}

```

FindFile() の戻り値型を変えましたから、この関数を使っている場所の修正が必要となります。**リスト 25.4** に修正例を示しました。このプログラムはターミナルの中で外部コマンドを起動するところを抜き出したものです。ls コマンドのときと似たような感じで、3つの場合に分けて処理します。

1. ファイルもディレクトリも見つからなかったとき
2. 見つかったエントリはディレクトリではない、にもかかわらず末尾に / があるとき
3. その他（ファイルが見つかったとき）

cat コマンドでも FindFile() を使っていたので、同様に修正しておきました。同じような修正ですので説明は省きます。

さて、これで一通り説明が終わりました。改造した ls コマンドで遊んでみましょう。あ、そうそう。もともと ls コマンドをディレクトリに対応させたかった理由はアプリが増えすぎて ls の結果で画面が溢れてしまうからでした。ということで、今回からはアプリを apps というディレクトリの中に入れることにしました。そのためには、次のように MikanOS のビルドスクリプトを呼ぶときに環境変数 APPS_DIR を設定します。APPS_DIR に何らかの文字列を設定して build.sh を呼び出すことで、ディスクイメージにその名前前でディレクトリが作られる仕組みになっています。

```
$ APPS_DIR=apps ./build.sh run
```

ls コマンドを、引数を変えて実行した様子を **図 25.3** に示します。

図 25.3 ls コマンドをいろいろな引数で実行する



1つ目は引数なしですが、これは従来通りルートディレクトリのファイル一覧が出力されました。今までと違って随分と出力されるファイル数が減ったことが分かります。よく見ると APPS という名前が見えますね。これが APPS_DIR 変数の効果で作られたアプリ用のディレクトリです。2つ目は apps ディレクトリ内のファイル stars を指定しました。apps 内にはたくさんのアプリが入っているはずですが、名前が一致するアプリだけが表示されています。これも成功。3つ目は efi ディレクトリを指定しました。efi ディレクトリ内のファイルが全部見えます。ということで、いろんなパターンでちゃんと動くようですね。大成功！

ちなみに「.」はそのディレクトリ自身を、「..」は1段上のディレクトリ（親ディレクトリ）を表す特殊なファイルです。この2つの特殊ファイルをうまく使うとさらに OS っぽい動作を実現できるようになります。例えば、Linux では次のようなコマンドはエラーなく実行できます。

```
$ ls ../
```

これは、カレントディレクトリの1つ上のディレクトリに含まれるファイルを表示するためのコマンドです。特殊ファイル .. を使うことで「1つ上」を表現しています。ちなみにカレントディレクトリとは、ユーザーが今居るディレクトリということです。ターミナルで作業するとき、ユーザーはどこかのディレクトリに「居る」ことになっているのです。ターミナル上で相対パスを指定すると、それはカレントディレクトリからの相対パスとして解釈されます。起動直後はホームディレクトリ (\$HOME) がカレントディレクトリになる設定が一般的でしょう。そして、カレントディレクトリを切り替えるコマンドが cd です。Change Directory の略です。MikanOS にはカレントディレクトリの概念がありませんが、興味があればぜひ実装してみてください。

ちなみに、FindFile() を使う処理全部がディレクトリに対応したため、cat コマンドでディレクトリ内のファイルを表示することもできますし、もちろん apps 内のアプリを起動することもできます。めでたし、めでたし！

25.2 ファイル読み込み (osbook_day25b)

ここからが本題で、アプリからファイルを読み書きする機能を開発していきます。ファイル読み込みの方が簡単そうなのでまずはそちらからやります。

さて、ファイルを読むために最低限必要なことは何でしょうか？ ターミナルに備わる cat コマンドの実装を見ながら考えてみると次の3つです。

- ファイルを表すパスからファイルのディレクトリエントリを特定する
- ディレクトリエントリを読み、ファイル内容が置いてあるクラスタ番号を得る
- クラスタ番号からブロック番号を得る

ブロック番号が得られればファイル内容を実際に読み出すことができます。本格的な OS であれば、ブロック番号を計算した後に実際にデータが記録された記憶装置（HDD や SSD など）にアクセスしてデータを読む処理も必要です。MikanOS では、ブートローダであらかじめ記憶装置のデータをメモリ上に読み出してあるため、いちいちハードウェアにアクセスしなくて済みます。

アプリがファイルを読む機能を設計するとしてぱっと思い付くのは、上記の仕組みをそのままアプリ

が使えるようにすることです。システムコールとして、パスを指定してディレクトリエントリを得るシステムコールと、クラスタ番号を指定してデータを読み出すシステムコールがあれば十分でしょう。この方法ではアプリがディレクトリエントリやクラスタという概念を扱う必要があります。

しかし、一般的な OS ではアプリに対してディレクトリエントリを直接見せたりはしません。なぜならファイルシステムを抽象化するのが OS の役目だと一般的に考えられているからです。現在普及しているファイルシステムは FAT 以外にもたくさんあります。FAT ファイルシステムに固有のデータ構造であるディレクトリエントリを直接アプリに公開してしまうと、FAT 上のファイルしか扱えないアプリになってしまいます。

C 言語の標準関数に `fopen()` と `fread()` があります。これらは `FILE*` 型の値を使ってファイルを扱います。また、Linux などの POSIX 系 OS では、アプリはファイルディスクリプタ (`fd`) と呼ばれる整数値を使ってファイルを扱うこともできます。いずれにせよ、対象のファイルがどんなファイルシステムにあるかをアプリが気にする必要はありません。MikanOS で採用している標準 C ライブラリ Newlib は POSIX に準拠していて、整数値でファイルを扱います。ですから、それにならってアプリからは整数値としてファイルを操作できるようなシステムコールを作ることになります。

OS にぜひ持たせたい機能にオフセット（読み込み位置）の管理機能があります。アプリはファイルを読む際に、一気に全体を読むのではなく少しずつ読み進めることがよくあります。大きなファイル全体を一気に読むには大量のメモリが必要になってしまうことがあるからです。そこで、OS 側でオフセットを管理してくれていると、アプリは先頭から一定のバイト数ずつデータを読んでいく処理が簡単になります。OS 側がオフセットを管理する仕組みには、通常、オフセットを変更するためのシークと呼ばれる操作もセットで提供されます。

もちろん、ファイル先頭からのオフセットと読み込むバイト数を毎回必ず指定するようなシステムコールも考えられます。アプリが自分でオフセットを管理するわけです。しかしこの方法はファイル読み込みの効率を下げるかもしれません。例えば FAT のクラスタチェーンは先頭からたどることしかできませんから、毎回オフセットが指定される形式では、毎回クラスタチェーンを先頭からたどらねばなりません。ファイルが大きいほど、ファイルの後ろの方を読もうとするほど、効率が落ちてしまいますよね。OS 側がオフセットを管理する方式なら、シーク操作が指示されない間はクラスタチェーンをたどらなくて済みます。

前置きが長くなってしまいましたがアプリがファイルを読める仕組みを作ります。目標は `fopen()` と `fread()` でファイルを読めるようになることです。これを検証するためにアプリを新しく作りました。**リスト 25.5** に紹介する `readfile` コマンドです。

リスト 25.5 ファイルの先頭 3 行を表示するコマンド (`readfile/readfile.cpp`)

```
#include <stdio>
#include <stdlib>

extern "C" void main(int argc, char** argv) {
    const char* path = "/memmap";
    if (argc >= 2) {
        path = argv[1];
    }

    FILE* fp = fopen(path, "r");
    if (fp == nullptr) {
        printf("failed to open: %s\n", path);
    }
}
```

```

    exit(1);
}

char line[256];
for (int i = 0; i < 3; ++i) {
    if (fgets(line, sizeof(line), fp) == nullptr) {
        printf("failed to get a line\n");
        exit(1);
    }
    printf("%s", line);
}
printf("----\n");
exit(0);
}

```

`fopen()` は指定されたファイルを開き、以後ファイルを操作するのに必要となる `FILE*` 型の値を返す C 標準ライブラリ関数です。第 1 引数にはファイルへのパスを与えます。第 2 引数はファイルを開く際のモードです。読み込みなら `r`、書き込みなら `w`、ファイル末尾への追加なら `a` を指定します。ほかにも読み書きできるモードなどがあります。詳しくは C 言語の教科書を参照してください。

このプログラムでは `fread()` の代わりに `fgets()` を使っています。テキストファイルを 1 行単位で読むのに便利だからです。`fgets()` は改行文字 ('`\n`') に到達するまで、またはバッファが満タンになるまで読み込んでくれます。改行文字も含めてバッファに格納するので、その後の `printf()` では改行文字を追加で出力する必要はありません。`fgets()` も `fopen()` と同様、多くの C 言語の教科書で説明されているでしょう。

`fopen()` と `fgets()` の実装の大部分は Newlib に含まれていて、OS 依存となるいくつかの関数をプログラマが提供するだけで使えるようになります。筆者が確認したところ、3 つの関数 `open()`、`read()`、`sbrk()` をきちんと実装する必要があるようです^{*1}。これらは例によって `apps/newlib_support.c` に実装するのがいいでしょう。それぞれの実装を順に紹介します。

リスト 25.6 `open()` は `OpenFile()` システムコールを呼びだけ (`newlib_support.c`)

```

int open(const char* path, int flags) {
    struct SyscallResult res = SyscallOpenFile(path, flags);
    if (res.error == 0) {
        return res.value;
    }
    errno = res.error;
    return -1;
}

```

`open()` の実装を **リスト 25.6** に示します。何も難しいことはありませんね。新しく作った `OpenFile()` システムコールを呼び出すだけの簡単なものです。`open()` の仕様を `man 2 open` で調べてみると、失敗時は `errno` にエラー値を設定したうえで `-1` を返すということになっているため、その通りに実装しています。

*1 `fopen()` と `fread()` が正しく動くのに必要な OS 依存関数群を特定するため、筆者は、`fopen()` と `fread()` を使う検証用アプリを作り、それが正しく動くようになるまで OS 依存関数群を 1 つずつ実装していきました。

リスト 25.7 read() も ReadFile() システムコールを呼ぶだけ (newlib_support.c)

```
ssize_t read(int fd, void* buf, size_t count) {
    struct SyscallResult res = SyscallReadFile(fd, buf, count);
    if (res.error == 0) {
        return res.value;
    }
    errno = res.error;
    return -1;
}
```

続いては read() の実装です (リスト 25.7)。read() は printf() をアプリから使えるように改造する際、エラーを返すだけの関数として定義したものです。今回の改造で、ちゃんとシステムコールを呼び出すようになりました。

ここまでで説明した 2 つの関数 open() と read() は、名前からして使われ方が想像しやすいですね。もちろん fopen() が open() を、fread() が read() を使います。では sbrk() は何に必要なのでしょうか？ これは「9.1 重ね合わせ処理」で説明した通り malloc() が内部で利用する関数です。malloc() は標準的なメモリ確保関数として、その他の標準関数から使われます。Newlib の実装を読んでも fopen() や fread()、fgets() も malloc() を内部で利用することがあるみたいです。ということで、これらが正しく動作するために sbrk() が必須です。

リスト 25.8 非常に簡易的に実装した sbrk() (newlib_support.c)

```
caddr_t sbrk(int incr) {
    static uint8_t heap[4096];
    static int i = 0;
    int prev = i;
    i += incr;
    return (caddr_t)&heap[prev];
}
```

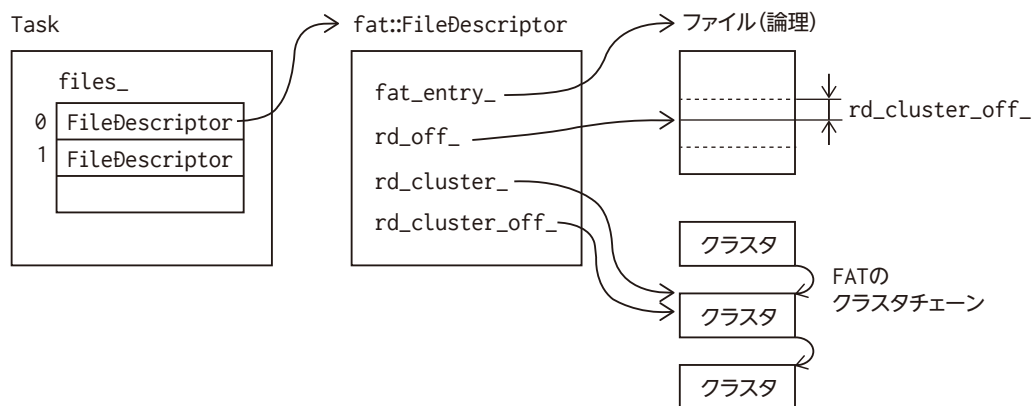
リスト 25.8 に sbrk() の実装を示します。といっても、まだアプリがメモリ確保するためのシステムコールを用意していませんでしたので、システムコールを使わない簡易版の実装としました。「とりあえず fopen() と fgets() が動けばいいや」という発想で、エラーチェックもしていない適当な実装です。メモリ確保のシステムコールを作ったら、改めてちゃんとした実装に差し替えることにしましょう。

newlib_support.c の変更を説明しましたので、次は新規に追加した 2 つのシステムコール OpenFile() と ReadFile() の説明をします。そのために、まずは OS 側でのファイル管理を設計しましょう。

先に書いた通り、アプリが整数値 (ファイルディスクリプタ番号) でファイル进行操作する設計にします。この整数値はアプリに固有で、同時に起動した 2 つのアプリで番号が共有されないようにしたいです。ファイルを扱う 2 つのアプリ上で、たまたま同じ番号のファイルディスクリプタがあったとして、それらに関連性を持たせないということです。

番号がアプリ間で共有される仕組みだと、先に起動したアプリが開いたファイルの番号を用いて後から起動したアプリが (自分ではファイルを開くことなく) 勝手にそのファイル进行操作できてしまうかもしれません。そのような安全性の話を考えないとしても、開かれているファイルの一覧をアプリごとに管理する方が楽に作れそうな気がします。

図 25.4 タスクとファイルディスクリプタとファイルの関係



階層ページング構造をアプリ毎に持たせる改造と同じように、ファイルディスクリプタの配列を Task クラスに持たせるようにします (図 25.4)。ファイルディスクリプタはまた別のクラスとして作り、その中にファイルに紐づく FAT ディレクトリエントリへの参照やファイル先頭からのオフセットなどの情報を持たせます。アプリへ通知するファイルディスクリプタ番号は Task::files_ の添字ということでしょう。

リスト 25.9 Task クラスにファイルディスクリプタの配列を持たせる (task.hpp)

```
private:
    uint64_t id_;

    < 中略 >

    std::vector<std::unique_ptr<fat::FileDescriptor>> files_{};
```

設計にしたがって Task クラスにファイルディスクリプタの配列 files_ を追加しました (リスト 25.9)。型がちょっと長いですが、よく読むとファイルディスクリプタへのポインタの配列であることが分かります。std::unique_ptr は「9.3 重ね合わせ処理の原理」でも登場したスマートポインタです。

それから、クラス外部から配列を利用するために Files() メソッドも追加してあります。Task クラスに対する変更はこれら 2 行の追加だけです。

リスト 25.10 FileDescriptor クラスはファイルやオフセットの情報を持つ (fat.hpp)

```
class FileDescriptor {
public:
    explicit FileDescriptor(DirectoryEntry& fat_entry);
    size_t Read(void* buf, size_t len);

private:
    DirectoryEntry& fat_entry_;
    size_t rd_off_ = 0;
    unsigned long rd_cluster_ = 0;
```

```

    size_t rd_cluster_off_ = 0;
};

```

FileDescriptor クラスをリスト 25.10 のように実装しました。4 つのメンバ変数の役割を表 25.3 に示します。

表 25.3 FileDescriptor クラスのメンバ変数

変数名	役割
fat_entry_	このファイルディスクリプタが指すファイルへの参照
rd_off_	ファイル先頭からの読み込みオフセット (バイト単位)
rd_cluster_	rd_off_ が指す位置に対応するクラスタの番号
rd_cluster_off_	クラスタ先頭からのオフセット (バイト単位)

rd_off_ はファイル先頭からの論理的なオフセットです。FAT ファイルシステムにおいてはファイルデータは連続的に配置されているのではなく、クラスタ単位で分かれて配置されます。実際にファイルの内容を読み取る際は、ファイル先頭からのオフセットではなくクラスタ番号とクラスタ内オフセットの組が必要です。それらを表すのが rd_cluster_ と rd_cluster_off_ です。

リスト 25.11 FileDescriptor クラスのコンストラクタとメソッド (fat.cpp)

```

FileDescriptor::FileDescriptor(DirectoryEntry& fat_entry)
    : fat_entry_{fat_entry} {}

size_t FileDescriptor::Read(void* buf, size_t len) {
    if (rd_cluster_ == 0) {
        rd_cluster_ = fat_entry_.FirstCluster();
    }
    uint8_t* buf8 = reinterpret_cast<uint8_t*>(buf);
    len = std::min(len, fat_entry_.file_size - rd_off_);

    size_t total = 0;
    while (total < len) {
        uint8_t* sec = GetSectorByCluster<uint8_t>(rd_cluster_);
        size_t n = std::min(len - total, bytes_per_cluster - rd_cluster_off_);
        memcpy(&buf8[total], &sec[rd_cluster_off_], n);
        total += n;

        rd_cluster_off_ += n;
        if (rd_cluster_off_ == bytes_per_cluster) {
            rd_cluster_ = NextCluster(rd_cluster_);
            rd_cluster_off_ = 0;
        }
    }

    rd_off_ += total;
    return total;
}

```


FileDescriptor クラスの2つのメソッドの実装を**リスト 25.11**に示します。コンストラクタはまあ簡単ですね。ファイルを表すディレクトリエントリへの参照を受け取り、メンバ変数に設定するだけです。fat_entry_ は参照変数ですからコンストラクタ以外で値を設定することはできません。

Read() は fat_entry_ が指すファイルのデータを、引数 buf が指すメモリ領域へと読み出すメソッドです。ファイルデータのうち、rd_off_ が指す位置から len バイト分のデータを buf へコピーします。このメソッドを実行すると、rd_off_ はメソッド呼び出し前の状態より len バイトだけ進みます。このメソッドは読み込んだデータのバイト数を返します。

while 文でちょっとずつ読み込んでいるのはクラスタ単位で分けて読む必要があるからです。読もうとしているファイルの範囲がクラスタ境界をまたがる場合、memcpy() をクラスタ境界の前と後で分けて実行する必要があります。それをするためにループしながら読み込みをしているのですね。

リスト 25.12 新規に追加したシステムコールのプロトタイプ宣言 (syscall.h)

```
struct SyscallResult SyscallOpenFile(const char* path, int flags);
struct SyscallResult SyscallReadFile(int fd, void* buf, size_t count);
```

FileDescriptor クラスの紹介はこれくらいにして、いよいよシステムコール本体の実装をしていきます。まずはシステムコールのプロトタイプ宣言を**リスト 25.12**に紹介します。いずれも open() と read() と同じ引数にしてあります。

リスト 25.13 OpenFile() システムコール (syscall.cpp)

```
SYSCALL(OpenFile) {
    const char* path = reinterpret_cast<const char*>(arg1);
    const int flags = arg2;
    __asm__("cli");
    auto& task = task_manager->CurrentTask();
    __asm__("sti");

    if ((flags & O_ACCMODE) == O_WRONLY) {
        return { 0, EINVAL };
    }

    auto [ dir, post_slash ] = fat::FindFile(path);
    if (dir == nullptr) {
        return { 0, ENOENT };
    } else if (dir->attr != fat::Attribute::kDirectory && post_slash) {
        return { 0, ENOENT };
    }

    size_t fd = AllocateFD(task);
    task.Files()[fd] = std::make_unique<fat::FileDescriptor>(*dir);
    return { fd, 0 };
}
```

syscall::OpenFile() システムコールを**リスト 25.13**に示します。引数 path は開きたいファイルへのパスが指定されます。引数 flags にはファイルを開く際のモードなどが指定されます。fopen() に指定するモードと意味は似ていますが、fopen() のモードが文字列だったのに対し open() の flags は整数です。

man 2 open など仕様を確認すると flags の仕様が分かります。それによれば、3つの値 O_RDONLY、O_WRONLY、O_RDWR のいずれか 1つだけを必ず指定しなければならないようです。この3つの値のことを**アクセスモード**と呼びます。また、アクセスモードに加えて他の様々な属性を追加することができます。読み込みモードでは関係ありませんが、書き込みモードで開く際に O_CREAT を指定すると「ファイルがなければ新規作成する」という意味になります。O_CREAT 付きの書き込みモードでファイルを開くには open("memmap", O_WRONLY|O_CREAT) のようにビット OR で属性を追加します。

flags & O_ACCMODE というのは、flags の属性ビットを無視してアクセスモードのみを取り出すための書き方です。上記のプログラムでは、取り出したアクセスモードが WRONLY のときにエラーにしています。まだファイル書き込み機能はありませんからね……。

アクセスモードの検査が終わったら path で指定されたファイルを探します。これには fat::FindFile() を使います。前の節でこの関数をディレクトリ対応させておいたため、OpenFile() システムコールも自動的にディレクトリに対応できちゃいます。いいですね。path で指定されたファイルがなかったり、ディレクトリではないにもかかわらず末尾に / があったりする場合はエラー ENOENT を返します。No Entry の略で、指定されたファイルやディレクトリが見つからないことを意味するエラーコードです。

path が指すファイルが見つければ、次はファイルディスクリプタを新規に割り当てます。関数 AllocateFD() は Task::files_ の中の使われていないファイルディスクリプタのうち最も先頭の番号を返します。もし Task::files_ のすべての要素が使用中であれば、末尾に新しいファイルディスクリプタを増やし、その番号を返します。いずれにせよ、AllocateFD() の戻り値は Task::files_ の中の空いている要素を指す添字です。その添字が表す要素に対し、ファイルディスクリプタを生成して代入すれば OK です。

リスト 25.14 AllocateFD() は Task::files_ の空き要素を見つける (syscall.cpp)

```
size_t AllocateFD(Task& task) {
    const size_t num_files = task.Files().size();
    for (size_t i = 0; i < num_files; ++i) {
        if (!task.Files()[i]) {
            return i;
        }
    }
    task.Files().emplace_back();
    return num_files;
}
```

AllocateFD() の実装は**リスト 25.14** のようになっています。与えられた Task クラスの files_ を先頭から調べ、ヌルポインタになっている要素を見つけたらその添字を返します。すべての要素が使用中であれば、末尾に空の要素を追加してその添字を返します。

リスト 25.15 ReadFile() システムコール (syscall.cpp)

```

SYSCALL(ReadFile) {
    const int fd = arg1;
    void* buf = reinterpret_cast<void*>(arg2);
    size_t count = arg3;
    __asm__("cli");
    auto& task = task_manager->CurrentTask();
    __asm__("sti");

    if (fd < 0 || task.Files().size() <= fd || !task.Files()[fd]) {
        return { 0, EBADF };
    }
    return { task.Files()[fd]->Read(buf, count), 0 };
}

```

syscall::ReadFile() の実装をリスト 25.15 に示します。このシステムコールのポイントは第 1 引数で指定されたファイルディスクリプタが files_ の添字であるということですね。それが分かれば簡単です。もし、まだ開いていないファイルディスクリプタを指定された場合、ファイルディスクリプタが無効であることを示す EBADF エラーを返します。

ここまで改造できたらアプリでファイル読み込みができるようになっているはずですよ。ビルドして readfile コマンドを実行してみた様子を図 25.5 に載せました。うん、意図通りに先頭の 3 行が表示されているようです。いいですね！

図 25.5 readfile コマンドで memmap を表示してみた



MikanOS では実装しませんでした。Linux や Windows のような汎用 OS にはファイルを権限で保護する仕組みがあります。Linux ではファイルの属性として所有者および所有グループを設定できます。そして、所有者、所有グループ、その他のユーザーについて個別に読み、書き、実行権限を設定できます。Linux を使っていて `rw-r-xr-x` のような表示を見たことはありませんか？ これが権限です。3 桁ずつ区切って読みます。最初の 3 桁がファイルの所有者、真ん中の 3 桁が所有グループ、最後の 3 桁がその他のユーザーに対する権限です。r は読み込み可能、w は書き込み可能、x は実行可能の意味です。Linux では複数のユーザーが 1 つの OS を使うことを前提として作られていますので、他人のファイルを読むことができないよう、このような仕組みでファイルを保護しています。

25.3 正規表現検索 (osbook_day25c)

ファイル読み込みができるようになりましたので、それを利用して正規表現検索を行うコマンド `grep` を作ってみます。正規表現検索とは「あいまい検索」と呼ばれることもあります。正規表現(regular expression)と呼ばれる、文字列のパターンを表現するための言語があります。正規表現を用いると、文字列の完全一致だけではない検索ができるのです。`grep` はファイルを正規表現で検索するためのコマンドとして非常に有名で、使ったことがある方も多いでしょう。正規表現そのものの説明は専門の書籍に任せます。

リスト 25.16 C++ の正規表現ライブラリを利用して作った `grep` コマンド (`grep/grep.cpp`)

```
#include <cstdio>
#include <cstdlib>
#include <regex>

extern "C" void main(int argc, char** argv) {
    if (argc < 3) {
        printf("Usage: %s <pattern> <file>\n", argv[0]);
        exit(1);
    }

    std::regex pattern(argv[1]);

    FILE* fp = fopen(argv[2], "r");
    if (fp == nullptr) {
        printf("failed to open: %s\n", argv[2]);
        exit(1);
    }

    char line[256];
    while (fgets(line, sizeof(line), fp)) {
        std::cmatch m;
        if (std::regex_search(line, m, pattern)) {
            printf("%s", line);
        }
    }
    exit(0);
}
```

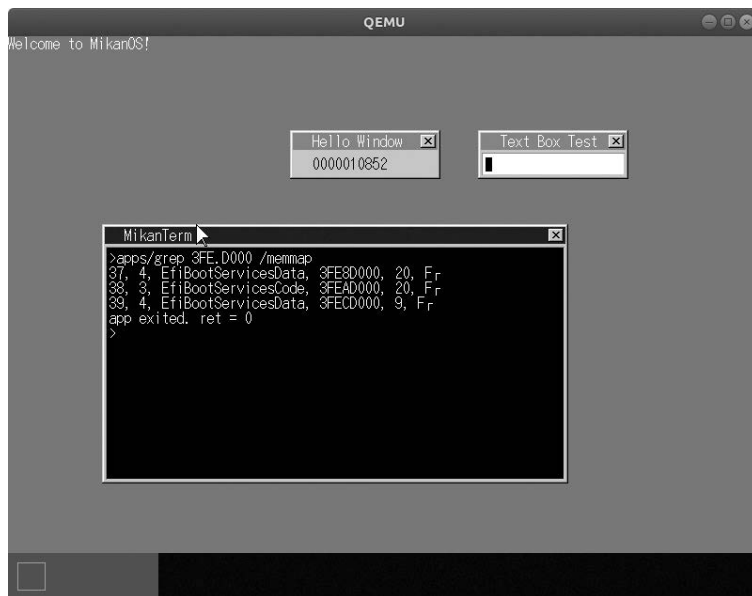
リスト 25.16 に `grep` コマンドの実装を示します。とてもシンプルな作りです。正規表現を解釈するのは本来大変なのですが、C++ に標準で用意されている `<regex>` ライブラリを利用すれば非常に楽です。`std::regex` が正規表現によるパターンを表現するためのクラスです。コンストラクタに正規表現で書かれたパターン文字列を渡します。`std::regex_search()` が与えられた文字列からパターンに一致する部分文字列を検索します。一致する部分があれば真を返します。

リスト 25.17 `posix_memalign()` はアライメントされたメモリ領域を確保する (`newlib_support.c`)

```
int posix_memalign(void** memptr, size_t alignment, size_t size) {
    void* p = malloc(size + alignment - 1);
    if (!p) {
        return ENOMEM;
    }
    uintptr_t addr = (uintptr_t)p;
    *memptr = (void*)((addr + alignment - 1) & ~(uintptr_t)(alignment - 1));
    return 0;
}
```

`<regex>` ライブラリを使おうとしたら `posix_memalign()` という関数が定義されていないというエラーが出ました。この関数は `malloc()` と同様にメモリを確保する関数ですが、アライメントされたメモリ領域を確保する点が `malloc()` と異なります。この関数は正確には Newlib のための関数ではなく、`libc++` (C++ の標準ライブラリ) が依存する関数ですが、面倒なので `newlib_support.c` に定義してしまいました (**リスト 25.17**)。

図 25.6 grep コマンドでパターン検索を試してみる



grep コマンドを使って、memmap ファイルから 3FE.D000 を含む行を検索してみた様子を図 25.6 に示します。正規表現で「.」は任意の 1 文字を表す記号です。スクリーンショットを見てみると、どうやら正しく検索できているらしいことが分かります。

きりがいいのでこの辺りで終わりにして、次章では、ファイル入力の応用技、標準入力の仕組みから始めようかと思います。

第26章

アプリで ファイル書き込み

アプリからファイルが読めるようになると応用範囲がぐっと広がります。実際、grepのような実用的なアプリを実装できることを確認できましたね。計算機の本質は入力を受け取って計算し、結果を出力することです。ターミナルへの印字だけでも出力ではありますが、やはりファイルに書き込む機能はサポートしておきたいところです。この章ではアプリがファイルへ書き込みできる仕組みを作ります。

26.1 標準入力 (osbook_day26a)

ファイル書き込みをやる前に標準入力の仕組みを作ってみます。ファイル書き込みとは直接結びつかない機能ではありますが、標準入力に対応することでファイル操作まわりの仕組みが柔軟になり、今後の開発がやりやすくなると思います。それに、筆者はキーボードやファイルを統一的に扱える標準入力という仕組みにロマンを感じます！

読者の皆さんは標準入力という言葉聞いたことがありますか？ これは、プログラムへ文字列を入力するための標準的なインターフェースのことです。C 言語の `scanf()` がデータを読み取る口が**標準入力** (standard input) です。入力があれば出力もありまして、`printf()` の出力先を**標準出力** (standard output)、`perror()` の出力先を**標準エラー出力** (standard error) といいます。標準入出力は便利なので、C 言語に限らずいろいろなプログラミング言語で簡単に使えるようになっていることが多いです。

C 言語では標準入出力はファイルとして扱えるようになっています。普通のファイルは `fopen()` で開いてから使わなければなりませんが、標準入出力だけはプログラム起動時に自動的に開かれており、`FILE*` 型のグローバル変数 `stdin`、`stdout`、`stderr` として利用可能です。普通のファイルと同様に、`fgets(buf, sizeof(buf), stdin)` のように使えます。

C 言語の標準仕様と同様に、POSIX でも標準入出力をファイルとして扱います。POSIX ではファイルをファイルディスクリプタという整数値で管理します。普通のファイルは `open()` で開くたびに自動的に番号が振られますが、標準入出力には `stdin=0`、`stdout=1`、`stderr=2` という固定の番号が初めから付与されています。Newlib は POSIX に準拠しているため、Newlib の `printf()` は最終的に、標準出力 (`fd=1`) に対して `write(1, ...)` を呼び出します。**リスト 21.7** で `if (fd == 1)` と書いたのは、`fd=1` に対する `write()` が実行された結果、`syscall::PutString()` の変数 `fd` の値が 1 になるからなのです。

Linux のターミナル上でプログラムをただ起動させると、標準入力はキーボードに、標準出力はターミナル画面に接続されます。そのため、標準入力から入力すればキーボード入力を受け取れますし、標準出力へ印字すればターミナル画面に文字列が表示されるわけです。キーボードやターミナル画面をファイルとして見せる仕組みが標準入出力だと思ってください。

この節では MikanOS に標準入力の仕組みを実装します。目標は `readfile` コマンドのソースコードを改造することなく、キーボードからの入力を 3 行分読み取ってターミナルに印字することです。これを実現するためには、`readfile` コマンドの中の `fgets()` が標準入力から文字列を読み取れることが必要です。さて、この仕組みの実現方法を皆さんは思い付くでしょうか？

26.2 ファイルディスクリプタの抽象化

標準入力をファイルとして見せる仕組みを作りたいのですが、ちょっと考えれば FAT ファイルシステム上のファイルとキーボード入力をファイルに見せかけることは根本的に違うなあと思うわけです。FAT のファイルはデータがメモリ（本来ならば HDD や SSD などのストレージ）上に並んだバイト列です。キーボード入力はユーザーがキーを押すたびに 1 文字ずつデータが生まれる装置です。ハードウェアの視点ではまったく異なるものですね。

ただ、文字列（またはバイト列）を読み取れるという性質は共通です。FAT のファイルもキーボード入力も（読み取り速度の差はありますが）1 文字ずつ読み取ることが可能ですね。このように「文字列を読み取る」という性質をもつものであれば何でもファイルとみなせます。「文字列を読み取れる何か」

が基礎にあって、その派生として FAT のファイルやキーボードを実現すれば、その両者を共通に扱えそうですね。C++ でこれを素直に表現する方法があります。そう、継承です。

リスト 26.1 FileDescriptor は文字列を読み取れる何かを表す (file.hpp)

```
#pragma once

class FileDescriptor {
public:
    virtual ~FileDescriptor() = default;
    virtual size_t Read(void* buf, size_t len) = 0;
};
```

リスト 26.1 に基底クラスとなる FileDescriptor^{*1} を紹介します。このクラスを継承し、FAT ファイルシステム上のファイルを扱うクラスとキーボード入力を扱うクラスを作る方針です。C++ のお約束で、継承の基底として使うクラスのデストラクタには必ず virtual を付けるのです。Read() メソッドは子クラスで実装しますので、ここでは純粹仮想関数（＝実装が無い仮想関数）として宣言するのが適しています。

リスト 26.2 fat::FileDescriptor は FAT 上のファイルを扱う (fat.hpp)

```
class FileDescriptor : public ::FileDescriptor {
public:
    explicit FileDescriptor(DirectoryEntry& fat_entry);
    size_t Read(void* buf, size_t len) override;

private:
    DirectoryEntry& fat_entry_;
    size_t rd_off_ = 0;
    unsigned long rd_cluster_ = 0;
    size_t rd_cluster_off_ = 0;
};
```

前章で実装した fat::FileDescriptor クラスを、今作った FileDescriptor クラスを継承するように書き換えました（リスト 26.2）。1 行目に継承の書き方（: public 親クラス）が追加されているのが分かります。クラスの中身はほとんど変えていませんが、たった 1 箇所、Read() の宣言に override を追加しました。親クラスの Read() を上書きすることを明示してミスを防ぎます。この継承により、fat::FileDescriptor のインスタンスは親クラス ::FileDescriptor のポインタ変数を用いて操作できるようになりました。

*1 fat.hpp で定義された FileDescriptor と名前が同じですが異なるクラスです。なぜなら両クラスは異なる名前空間に入っているからです。名前空間を明示した書き方は、それぞれ fat::FileDescriptor、::FileDescriptor となります。

リスト 26.3 TerminalFileDescriptor はキーボードをファイルに見せかける (terminal.hpp)

```
class TerminalFileDescriptor : public FileDescriptor {
public:
    explicit TerminalFileDescriptor(Task& task, Terminal& term);
    size_t Read(void* buf, size_t len) override;

private:
    Task& task_;
    Terminal& term_;
};
```

fat::FileDescriptor と同じく、継承を用いてキーボード入力をファイルにみせかける TerminalFileDescriptor クラスを実装しました (リスト 26.3)。Read() メソッドの実装は後で紹介します。

リスト 26.4 Task::files_ の要素を親クラスのポインタ型に変える (task.hpp)

```
std::vector<std::unique_ptr<::FileDescriptor>> files_{};
```

さて、継承を使って 2 種類の「ファイル」を表現しました。それらを共通に扱えるようにするため、リスト 26.4 のように Task::files_ の要素の型を親クラスのポインタ型に変えました。これにより、Task::files_ を利用する場所 (ReadFile() システムコール) では、実際にその「ファイル」が FAT のファイルなのかキーボード入力をファイル化したものなのか、区別せず利用できます。2 つの異なるものを「ファイル」として抽象化したということです。

26.3 キーボード入力を受け取る

さて、普通のファイルなら ReadFile() の前に OpenFile() を使って開かれるはずです。一方、標準入力アプリ起動時に fd=0 として開かれている必要があります。すなわち、アプリ起動前に Task::files_ の先頭に標準入力を表す値が入っている状態にします。

リスト 26.5 fd=0 に標準入力を設定する (terminal.cpp)

```
task.Files().push_back(
    std::make_unique<TerminalFileDescriptor>(task, *this));

auto entry_addr = elf_header->e_entry;
int ret = CallApp(argc.value, argv, 3 << 3 | 3, entry_addr,
    stack_frame_addr.value + 4096 - 8,
    &task.OSStackPointer());

task.Files().clear();
```

アプリ起動前に標準入力を `fd=0` として設定するプログラムを **リスト 26.5** に示します。このプログラムは `Terminal::ExecuteFile()` からの抜粋です。 `CallApp()` を実行する直前に、`Task::files_` の先頭に標準入力を表す値 (`TerminalFileDescriptor` のインスタンス) を設定しています。

1 つのターミナル上で何回もアプリが起動と終了することを考えると、アプリ終了時に `Task::files_` を空にしておく必要があることに気づきました。 `CallApp()` の後に実行している `task.Files().clear();` はそのためです。これをしないと、アプリを起動するたびにファイルが積み重なってファイルディスクリプタ番号が後ろにずれていってしまいます。

リスト 26.6 `TerminalFileDescriptor::Read()` はキーボード入力を読む (`terminal.cpp`)

```
size_t TerminalFileDescriptor::Read(void* buf, size_t len) {
    char* bufc = reinterpret_cast<char*>(buf);

    while (true) {
        __asm__("cli");
        auto msg = task_.ReceiveMessage();
        if (!msg) {
            task_.Sleep();
            continue;
        }
        __asm__("sti");

        if (msg->type == Message::kKeyPush && msg->arg.keyboard.press) {
            bufc[0] = msg->arg.keyboard.ascii;
            term_.Print(bufc, 1);
            return 1;
        }
    }
}
```

キーボード入力を受け取る `TerminalFileDescriptor::Read()` の実装を **リスト 26.6** に示します。難しいことはあまりないでしょう。タスクのメッセージキューから 1 文字読み取って返せば良いです。通常のファイル読み込みの場合は `len` で指定されたバイト数になるまで読み込みを続けようとしませんが、キーボード入力は 1 文字ずつしか来ませんから 1 文字入力した時点で `Read()` を終わらせるようにします。

`term_.Print(bufc, 1);` はキー入力結果を即座にターミナルに印字するための処理です。この処理を **エコーバック** と呼びます。この効果は、試しにこの行をコメントアウトして実行してみればよく分かります。エコーバックがあることで入力作業がとても快適になります。

`readfile` コマンドは引数としてパスを受け取り、そのパスが指すファイルから 3 行読むコマンドです。すなわち、入力ファイルはパスで指定できる必要があります。今のところ標準入力に対応するパスがありませんから、残念ながら `readfile` が標準入力を読み取ることができません。折角今まで頑張ってきたのに、これは悲しいです……。

リスト 26.7 標準入力にファイル名を与える (syscall.cpp)

```
SYSCALL(OpenFile) {
    const char* path = reinterpret_cast<const char*>(arg1);
    const int flags = arg2;
    __asm__("cli");
    auto& task = task_manager->CurrentTask();
    __asm__("sti");

    if (strcmp(path, "@stdin") == 0) {
        return { 0, 0 };
    }
}
```

実は悲しむ必要はぜんぜんありません！ リスト 26.7 のように、特殊なパスで標準入力を指定できるようにすればいいだけです。これで、`fopen("@stdin", "r")` とすると標準入力を取得できますから、`readfile` コマンドで標準入力を扱えるようになるはずです。標準入力にどんなパスを対応付けるかは OS によって異なり、MikanOS では `@stdin` としてみました。Linux では `/dev/stdin` です。

図 26.1 `readfile` コマンドに標準入力を指定した様子



ここまでの改造を実行した様子を図 26.1 に示します。スクリーンショットだと分かりにくいので動作を説明します。まず、キーボードで「abc……」を入力しているとき、キー入力のたびに入力した文字がターミナルに表示されていきます。エコーバック機能のおかげです。その後、エンターキーを押した瞬間に次の行に「abcdef」が一気に表示されました。これは、`fgets()` により 1 行読み取られた結果が `printf()` でターミナルへ出力されているからです。2 行目も同様に、エンターを入力した瞬間に同じ内容の行が表示されました。3 行目はエンター入力前のタイミングで止めてあります。

`readfile` コマンド側は 1 行も変えていないのにキーボード入力を扱えるようになりました。自分

で言うのも恥ずかしいですが、この仕組み、かなりかっこいいです。

26.4 EOF と EOT (osbook_day26b)

標準入力の仕組みはファイルを開くどのコマンドからも同じように使えます。もちろん `grep` コマンドでも使えますから、実際に `apps/grep foo @stdin` として実行し、キーボードからいろいろ打ち込んで試してみました。すると、ちゃんと `foo` を含む行だけが複製されました。もちろん検索にヒットしない行もエコーバックにより 1 行だけ表示はされますが、検索にヒットする行はさらにもう 1 行追加で表示されるわけですね。良い感じです。

しかし問題があります。`grep` コマンドの入力ファイルに標準入力を指定してしまうと、永久にコマンドの動作が終わらないのです。`grep` はファイルの終端に達するまで動作を続けます。通常のファイルであれば終端がありますが、標準入力では終端がないからです。

ファイルの終端を EOF (End of File) と呼びます。一方、標準入力やネットワーク通信のようなデータ転送における終端のことを EOT (End of Transmission) と呼びます。両者は似たような概念ですが、EOT は特別に仕組みを作らないと実現できません。この節では標準入力において EOT を表現できるようにします。

Linux ではターミナルで `Ctrl-D` を入力すると EOT を表すことになっています。Linux のターミナルでは、`Ctrl` キーを押しながら `A` から `Z`、およびいくつかの記号を入力することで、ASCII の 1 から 31 にある制御文字を入力する機能があります。`A-Z` が順に 1-26 に対応します。`D` は先頭から 4 番目です。実は EOT は ASCII コードの 4 に割り当てられています。そのため `Ctrl` キーを押しながら `D` キーを押すことで EOT を入力できるというわけです。この仕組みを MikanOS にも搭載して `grep` を終了できるようにしてみます。

リスト 26.8 `Ctrl-D` で EOT を入力する (terminal.cpp)

```
size_t TerminalFileDescriptor::Read(void* buf, size_t len) {
    char* bufc = reinterpret_cast<char*>(buf);

    while (true) {
        __asm__("cli");
        auto msg = task_.ReceiveMessage();
        if (!msg) {
            task_.Sleep();
            continue;
        }
        __asm__("sti");

        if (msg->type != Message::kKeyPush || !msg->arg.keyboard.press) {
            continue;
        }
        if (msg->arg.keyboard.modifier & (kLControlBitMask | kRControlBitMask)) {
            char s[3] = "^ ";
            s[1] = toupper(msg->arg.keyboard.ascii);
            term_.Print(s);
            if (msg->arg.keyboard.keycode == 7 /* D */) {
                return 0; // EOT
            }
        }
    }
}
```

```

    continue;
}

bufc[0] = msg->arg.keyboard.ascii;
term_.Print(bufc, 1);
return 1;
}
}

```

TerminalFileDescriptor::Read() をリスト 26.8 のように改造しました。Ctrl が押されていない場合の動作は今までと変わりません。Ctrl が押されていたら制御文字の処理モードになります。制御文字モードでは入力された文字をキャレット (^) 付きで表示し、制御文字が入力されたことが分かるようにしてみました。Ctrl-D 以外の制御文字は今のところ無視することにして、Ctrl-D を検知したら 0 を返します。これはファイル終端に来たときと同じ動作です。ですので、アプリ側は EOF と EOT は区別できず、普通にファイル終端と同じように処理するだけです。問題はないでしょう。

図 26.2 Ctrl-D で grep コマンドを終了させた様子



早速実行してみます (図 26.2)。適当なところで Ctrl-D を入力すると即座に grep コマンドが終了しました。簡単な改造で EOT に対応できて大満足です。

26.5 ファイル書き込み (1) (osbook_day26c)

本章の本题、ファイル書き込みをやってみます。これは大変そうな気がします。なにせ今までアプリ内はもちろん、OS 本体でもファイル書き込みの機能は一切開発していないからです。設計からやっていきますね。ファイル書き込みと言ってもストレージ装置に書き込むのではなく、メインメモリ上に内容を保存するだけにします。MikanOS にはストレージ装置を制御するドライバプログラムは搭載されていませんからね。

ファイルを書き込むというのは、要するに FAT ファイルシステム上のデータを書き換えることです。書き換えるべきデータは大きく 2 つ。1 つはデータ本体です。FAT のクラスタチェーン上にファイルデータを書き込んでいきます。ファイルが大きい場合は 1 つのクラスタでは収まらないでしょうから、クラスタを繋ぐ作業も必要でしょうね。もう 1 つはディレクトリエントリです。ディレクトリエントリにはファイルサイズや書き込み日時が記録されていますから、ファイル書き換えに合わせてそれらを更新します。ファイルを新規作成する場合はディレクトリエントリを新規追加する必要もあります。

- ・ ファイルを新規に作る機能。指定したディレクトリの中に新規のディレクトリエントリを作り、ファイルサイズ 0 で初期化する。
- ・ 既存のファイルにデータを書き込む機能。

作るべき機能を考えてみると、ユーザーが直接欲しい機能はこの 2 つかなと思います。新規ファイルにデータを書く場合でも、サイズ 0 のファイルを新規に作った後は「既存のファイル」と見なせますからね。同時に両方を作ると大変そうですから、この節では空のファイルを新しく作る機能だけを実装してみたいと思います。

リスト 26.9 O_CREAT が指定されたらファイルを新規に作る (syscall.cpp)

```
if (strcmp(path, "@stdin") == 0) {
    return { 0, 0 };
}

auto [ file, post_slash ] = fat::FindFile(path);
if (file == nullptr) {
    if ((flags & O_CREAT) == 0) {
        return { 0, ENOENT };
    }
    auto [ new_file, err ] = CreateFile(path);
    if (err) {
        return { 0, err };
    }
    file = new_file;
} else if (file->attr != fat::Attribute::kDirectory && post_slash) {
    return { 0, ENOENT };
}

size_t fd = AllocateFD(task);
task.Files()[fd] = std::make_unique<fat::FileDescriptor>(*file);
return { fd, 0 };
```

リスト 26.9 に `OpenFile()` システムコールの改造部分とその周りを示します。主な変更は、指定されたパスが見つからなかったとき (`if (file == nullptr)`) の処理です。今までは単に `ENOENT` エラーを返すだけでしたが、`O_CREAT` フラグが指定されていたら空のファイルを新規に作るように処理を変えました。`CreateFile()` という関数が空のファイルを作る関数です。後で実装します。

`O_CREAT` フラグは POSIX の `open()` のフラグとして決められている標準的なフラグです。このフラグは、開こうとしているファイルが無い場合に新規作成する効果を持ちます。筆者が手元の Linux で試したところ、モード (`O_RDONLY`、`O_WRONLY`、`O_RDWR`) に関係なくファイルが無い場合には空ファイルが作成されました。ということで、MikanOS でもそのような実装にしてみました。

リスト 26.10 戻り値を変換する (syscall.cpp)

```
std::pair<fat::DirectoryEntry*, int> CreateFile(const char* path) {
    auto [ file, err ] = fat::CreateFile(path);
    switch (err.Cause()) {
        case Error::kIsDirectory: return { file, EISDIR };
        case Error::kNoSuchEntry: return { file, ENOENT };
        case Error::kNoEnoughMemory: return { file, ENOSPC };
        default: return { file, 0 };
    }
}
```

`O_CREAT` の処理に利用する `CreateFile()` の実装を **リスト 26.10** に示します。実際の処理の大部分は `fat::CreateFile()` に定義していて、ここでは戻り値を `Error` 型からシステムコール用のエラー値に変換しているだけです。

リスト 26.11 `CreateFile()` は指定された名前の空ファイルを作る (fat.cpp)

```
WithError<DirectoryEntry*> CreateFile(const char* path) {
    auto parent_dir_cluster = fat::boot_volume_image->root_cluster;
    const char* filename = path;

    if (const char* slash_pos = strrchr(path, '/')) {
        filename = &slash_pos[1];
        if (slash_pos[1] == '\\0') {
            return { nullptr, MAKE_ERROR(Error::kIsDirectory) };
        }
    }

    char parent_dir_name[slash_pos - path + 1];
    strncpy(parent_dir_name, path, slash_pos - path);
    parent_dir_name[slash_pos - path] = '\\0';

    if (parent_dir_name[0] != '\\0') {
        auto [ parent_dir, post_slash2 ] = fat::FindFile(parent_dir_name);
        if (parent_dir == nullptr) {
            return { nullptr, MAKE_ERROR(Error::kNoSuchEntry) };
        }
        parent_dir_cluster = parent_dir->FirstCluster();
    }
}

auto dir = fat::AllocateEntry(parent_dir_cluster);
if (dir == nullptr) {
```



```

    return { nullptr, MAKE_ERROR(Error::kNoEnoughMemory) };
}
fat::SetFileName(*dir, filename);
dir->file_size = 0;
return { dir, MAKE_ERROR(Error::kSuccess) };
}

```

fat::CreateFile() の実装をリスト 26.11 に示します。この関数は、引数で指定されたパスに空ファイルを作ります。パスにディレクトリ名が含まれていればそのディレクトリに、ディレクトリが含まれていなければルートディレクトリにファイルを作ります。

parent_dir_cluster は重要な変数です。この変数は空ファイルを作るディレクトリを表します。初期値をルートディレクトリとしていますから、パスにディレクトリ名が含まれていなかった場合にはルートディレクトリに空ファイルが作られることになります。

次にパスにディレクトリ名が含まれているかを調べ、含まれていればそのディレクトリを開きます。strrchr() は、指定した文字列の末尾から文字を検索する C 標準ライブラリの関数です。パスを、ディレクトリ名とファイル名に分割するために使っています。パスに複数の区切り文字 (/) が含まれていても、ちゃんと最後の区切り文字を検索するために strchr() ではなく strrchr() (r が 1 つ多い) を使います。パスに / が含まれていれば、fat::FindFile() によりそのディレクトリを開き、クラスタ番号を parent_dir_cluster に設定します。

最後に fat::AllocateEntry() を利用して、指定したディレクトリの中に空ファイルを作ります。ファイル名は fat::SetFileName() で設定します。以降、これら 2 つの関数、および関連する関数を説明していきます。

リスト 26.12 AllocateEntry() は空きエントリを探して返す (fat.cpp)

```

DirectoryEntry* AllocateEntry(unsigned long dir_cluster) {
    while (true) {
        auto dir = GetSectorByCluster<DirectoryEntry>(dir_cluster);
        for (int i = 0; i < bytes_per_cluster / sizeof(DirectoryEntry); ++i) {
            if (dir[i].name[0] == 0 || dir[i].name[0] == 0xe5) {
                return &dir[i];
            }
        }
        auto next = NextCluster(dir_cluster);
        if (next == kEndOfClusterchain) {
            break;
        }
        dir_cluster = next;
    }

    dir_cluster = ExtendCluster(dir_cluster, 1);
    auto dir = GetSectorByCluster<DirectoryEntry>(dir_cluster);
    memset(dir, 0, bytes_per_cluster);
    return &dir[0];
}

```

`fat::AllocateEntry()` をリスト 26.12 のように実装しました。この関数は指定したディレクトリの中を探し、未使用のディレクトリエントリを発見して返します。ディレクトリエントリの `name` の先頭 1 バイトが 0 または 0xE5 であれば、そのエントリは未使用であることを示しますから、そのようなエントリを発見したらそれを返します。

空きエントリが 1 つもない場合がまれに発生します。そんなときはファイル生成をあきらめるのではなく、ディレクトリのデータ領域を 1 クラスタだけ拡張し、空きエントリを確保することにします。そのための処理が `ExtendCluster()` の呼び出しから始まる 4 行です。`ExtendCluster()` はクラスタチェーンを伸ばす (extend) ための関数で、指定した数の空きクラスタをクラスタチェーンの末尾にくっつけ、最後尾のクラスタ番号を返します。ここでは増やすクラスタの数を 1 としていますから、戻り値 `dir_cluster` はまさに今増やしたクラスタの番号になります。増やしたばかりのクラスタはもちろん全体が空ですから、問答無用で先頭のエントリを返せばいいですね。

リスト 26.13 `ExtendCluster()` はクラスタチェーンを延伸する (fat.cpp)

```
unsigned long ExtendCluster(unsigned long eoc_cluster, size_t n) {
    uint32_t* fat = GetFAT();
    while (!IsEndOfClusterchain(fat[eoc_cluster])) {
        eoc_cluster = fat[eoc_cluster];
    }

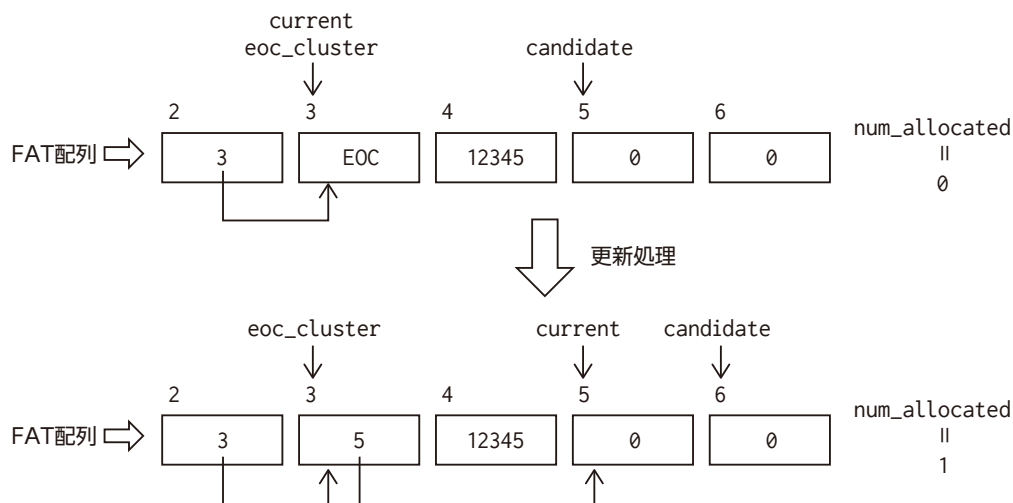
    size_t num_allocated = 0;
    auto current = eoc_cluster;

    for (unsigned long candidate = 2; num_allocated < n; ++candidate) {
        if (fat[candidate] != 0) { // candidate cluster is not free
            continue;
        }
        fat[current] = candidate;
        current = candidate;
        ++num_allocated;
    }
    fat[current] = kEndOfClusterchain;
    return current;
}
```

`fat::ExtendCluster()` はリスト 26.13 のようになっています。この関数は、引数 `n` で指定した数の空きクラスタを集め、`eoc_cluster` で指定したクラスタチェーンの最後尾に追加するのが役割です。

先頭の `while` 文では、引数 `eoc_cluster` で与えたクラスタが属するクラスタチェーンを末尾までたどります。後半にある `for` 文では、ボリューム内の全クラスタを先頭 (クラスタ 2) から調べ、必要な数の空きクラスタを探します。ループ変数 `candidate` は英語で「候補」という意味があります。ここでは、空きクラスタの候補という意味でこの名前にしました。発見した空きクラスタの数は `num_allocated` で管理しています。この値が `n` になるまで空きクラスタを探し続けます。

図 26.3 クラスタチェーンの末尾にクラスタ 5 を追加する様子



for 文では、見つけた空きクラスタをクラスタチェーンの末尾に追加します。図 26.3 は、クラスタ 2→クラスタ 3 というチェーンの末尾にクラスタを追加する例を表します。クラスタ 4 はほかのチェーンの一部として使用中なので、候補としてクラスタ 5 が選ばれます。更新処理によってクラスタ 3 にクラスタ 5 が繋がれ、num_allocated がインクリメントされます。

ボリュームが満杯に近づいてくると、ボリューム内の全クラスタを探したけど十分な空きクラスタが無かった、ということがあり得ます。しかし上記の実装ではそのチェックをさぼっています。それは、十分な空きクラスタが無いパターンはなかなか起きないし、エラーチェックをするコードがかなり大きくなりそうだからです。そもそも、MikanOS のブートローダではボリュームの先頭 16MiB しか読み込んでいません。だから、元のボリュームに存在するはずのクラスタ数よりもずいぶん少ない領域しかメモリ上にありません。そのためクラスタ数の検査は無意味なのです。(ボリュームの先頭から 16MiB を超えそうになったらやめる、というエラーチェックは十分に価値があるでしょう。興味のある方は改造してみてください。)

ExtendCluster() の中で使っている GetFAT() はボリュームの中の FAT 構造 (32 ビット値の配列) の先頭ポインタを得る関数、IsEndOfClusterchain() は与えられたクラスタ番号がチェーン末尾を表す値かどうかを確認するための関数です。詳しい実装の紹介は省略します。

ここまでで、CreateFile() の中で呼び出している fat::AllocateEntry() の説明が終わりました。その後に登場する fat::SetFileName() についても簡単に紹介しておきますね。

リスト 26.14 SetFileName() はディレクトリエントリに短名を書き込む (fat.cpp)

```
void SetFileName(DirectoryEntry& entry, const char* name) {
    const char* dot_pos = strchr(name, '.');
    memset(entry.name, ' ', 8*3);
    if (dot_pos) {
        for (int i = 0; i < 8 && i < dot_pos - name; ++i) {
            entry.name[i] = toupper(name[i]);
        }
    }
}
```

```

    for (int i = 0; i < 3 && dot_pos[i + 1]; ++i) {
        entry.name[8 + i] = toupper(dot_pos[i + 1]);
    }
} else {
    for (int i = 0; i < 8 && name[i]; ++i) {
        entry.name[i] = toupper(name[i]);
    }
}
}
}

```

リスト 26.14 に `fat::SetFileName()` の実装を示します。この関数はやっていることはとてもシンプルです。引数 `name` で指定された短ファイル名 (8+3 形式) を大文字に変換しながらディレクトリエントリの `entry.name` フィールドにコピーするだけです。例えば `"hoge.foo"` というファイル名が与えられたら、`entry.name` には `"HOGEE FOO"` という 11 文字を書き込みます。FAT の仕様でファイル名は全部大文字で書く必要があります (MikanOS だけで読み書きできればいいボリュームならそんな仕様を守る必要はないわけですが)。

ディレクトリエントリには、ほかにもファイル作成日時、ファイル変更日時などの項目がありますが、今は特に変更しないことにします。ただ、ファイルサイズだけは確実に 0 にします。コマンドでファイルを読み取ったときに誤動作しないためです。

`OpenFile()` システムコールに `O_CREAT` を指定したらファイルが作られる仕組みが整ったはずです。ファイルを書き込みモードで開くコマンドを作って検証してみましょう。

リスト 26.15 ファイルをコピーする `cp` コマンド (`cp/cp.cpp`)

```

#include <cstdio>
#include <cstdlib>

extern "C" void main(int argc, char** argv) {
    if (argc < 3) {
        printf("Usage: %s <src> <dest>\n", argv[0]);
        exit(1);
    }

    FILE* fp_src = fopen(argv[1], "r");
    if (fp_src == nullptr) {
        printf("failed to open for read: %s\n", argv[1]);
        exit(1);
    }

    FILE* fp_dest = fopen(argv[2], "w");
    if (fp_dest == nullptr) {
        printf("failed to open for write: %s\n", argv[2]);
        exit(1);
    }

    char buf[256];
    size_t bytes;
    while ((bytes = fread(buf, 1, sizeof(buf), fp_src)) > 0) {
        const size_t written = fwrite(buf, 1, bytes, fp_dest);
        if (bytes != written) {
            printf("failed to write to %s\n", argv[2]);
            exit(1);
        }
    }
}

```

```

    }
}
exit(0);
}

```

ファイルをコピーする標準的なコマンドである `cp` を実装してみました (リスト 26.15)。オリジナルのコマンドより大分機能は少ないですが。このコマンドは `cp <src> <dest>` というように呼び出すと、`src` で指定したファイルを `dest` で指定したファイル名でコピーします。

プログラム自体はとても簡単です。 `src` と `dest` で指定されたファイルをそれぞれ読み込み、書き込みモードで開き、ファイル内容を 256 バイトずつコピーしていただくだけです。 `fopen()` のモードとして `w` を指定することで、 `open()` のフラグに `O_WRONLY|O_CREAT` が指定されます。

図 26.4 `cp` コマンドが新規ファイルを作る様子



図 26.4 に `cp` コマンドの実行の様子を示します。 `memmap` ファイルを `abc` という名前でコピーしようとしています。まだ `fwrite()` が依存する `write()` の実装をしていませんから、データのコピーに失敗して `cp` コマンドは全体としては失敗してしまいます。しかし `fopen()` 自体は成功していて、 `ABC` というファイルが新しく生成されています。良い感じですね。もちろん `ABC` を `cat` コマンドで表示してみると中身は空です。

26.6 ファイル書き込み (2) (osbook_day26d)

空ファイルが作れるようになりました。この節では既存のファイルにデータを書き込む機能を作りま
す。これが完成すると `fwrite()` がきちんと動くようになり、`cp` コマンドでちゃんとファイルをコピー
できるようになるはずです。

リスト 26.16 ファイルディスクリプタに書き込みインターフェースを増やす (file.hpp)

```
class FileDescriptor {
public:
    virtual ~FileDescriptor() = default;
    virtual size_t Read(void* buf, size_t len) = 0;
    virtual size_t Write(const void* buf, size_t len) = 0;
};
```

ファイル書き込みをサポートするにあたり、手始めに `FileDescriptor` クラスに `Write()` メソ
ッドを増やすところから始めました(リスト 26.16)。親クラスに純粋仮想関数を増やすと、子クラスでオー
バーライドしない限りコンパイルエラーが出るようになります。このエラーを解決するには、2 つの子
クラス (`fat::FileDescriptor` と `TerminalFileDescriptor`) に `Write()` メソッドを実装す
る必要があります。

リスト 26.17 `TerminalFileDescriptor::Write()` はターミナルへ出力する (terminal.cpp)

```
size_t TerminalFileDescriptor::Write(const void* buf, size_t len) {
    term_.Print(reinterpret_cast<const char*>(buf), len);
    return len;
}
```

`TerminalFileDescriptor::Write()` は簡単そうだったので先に実装しました(リスト
26.17)。`Read()` に比べて非常に簡単に作れました。受け取った文字列をターミナルに出力するだけで
完了です。

リスト 26.18 先頭 3 つのファイルを標準入出力とする (terminal.cpp)

```
for (int i = 0; i < 3; ++i) {
    task.Files().push_back(
        std::make_unique<TerminalFileDescriptor>(task, *this));
}
```

`TerminalFileDescriptor` が読み込みにも書き込みにも対応しましたので、標準入力 (`fd=0`) だ
けでなく標準出力 (`fd=1`) と標準エラー出力 (`fd=2`) としても使えるようになりました。そこで、リス
ト 26.18 のように `Task::files_` の先頭 3 つをすべて `TerminalFileDescriptor` とするよう
にしました。今までは `fd=0` だけを設定していましたが、これからは `fd=0`、1、2 がターミナルに接続され
ます。

リスト 26.19 PutString() システムコールで fd を使う (syscall.cpp)

```
SYSCALL(PutString) {
    const auto fd = arg1;
    const char* s = reinterpret_cast<const char*>(arg2);
    const auto len = arg3;
    if (len > 1024) {
        return { 0, E2BIG };
    }

    __asm__("cli");
    auto& task = task_manager->CurrentTask();
    __asm__("sti");

    if (fd < 0 || task.Files().size() <= fd || !task.Files()[fd]) {
        return { 0, EBADF };
    }
    return { task.Files()[fd]->Write(s, len), 0 };
}
```

fd=1、2 がきちんと設定されたことを受けて、**リスト 26.19** のように `syscall::PutString()` を改造しました。今までは fd が 1 だったらターミナルへ文字列を出力する、というプログラムになっていました。この改造によって `Task::files_` に登録されたファイルディスクリプタを使うようになります。この変更にとともって、タスク ID からターミナルを検索するための対応表である `terminals` というグローバル変数が不要になりました。変数を利用している箇所をすべて削除しておきましたよ。

リスト 26.20 fat::FileDescriptor に Write() を増やす (fat.hpp)

```
class FileDescriptor : public ::FileDescriptor {
public:
    explicit FileDescriptor(DirectoryEntry& fat_entry);
    size_t Read(void* buf, size_t len) override;
    size_t Write(const void* buf, size_t len) override;

private:
    DirectoryEntry& fat_entry_;
    size_t rd_off_ = 0;
    unsigned long rd_cluster_ = 0;
    size_t rd_cluster_off_ = 0;
    size_t wr_off_ = 0;
    unsigned long wr_cluster_ = 0;
    size_t wr_cluster_off_ = 0;
};
```

`TerminalFileDescriptor` の改造は本当に楽でした。大変なのは `fat::FileDescriptor` の方です。手始めに `Write()` メソッドの宣言と、必要になりそうなメンバ変数を増やしました (**リスト 26.20**)。読み込みに使う 3 つのメンバ変数 (`rd_` から始まる) を真似して、書き込みに使うメンバ変数を 3 つ作ります。 `wr_off_` はファイル先頭からのオフセット、 `wr_cluster_` は書き込み対象のクラスタ番号、 `wr_cluster_off_` はクラスタ内でのオフセットです。

リスト 26.21 FAT ファイルシステム上のファイルに対する書き込み (fat.cpp)

```

size_t FileDescriptor::Write(const void* buf, size_t len) {
    auto num_cluster = [](size_t bytes) {
        return (bytes + bytes_per_cluster - 1) / bytes_per_cluster;
    };

    if (wr_cluster_ == 0) {
        if (fat_entry_.FirstCluster() != 0) {
            wr_cluster_ = fat_entry_.FirstCluster();
        } else {
            wr_cluster_ = AllocateClusterChain(num_cluster(len));
            fat_entry_.first_cluster_low = wr_cluster_ & 0xffff;
            fat_entry_.first_cluster_high = (wr_cluster_ >> 16) & 0xffff;
        }
    }

    const uint8_t* buf8 = reinterpret_cast<const uint8_t*>(buf);

    size_t total = 0;
    while (total < len) {
        if (wr_cluster_off_ == bytes_per_cluster) {
            const auto next_cluster = NextCluster(wr_cluster_);
            if (next_cluster == kEndOfClusterchain) {
                wr_cluster_ = ExtendCluster(wr_cluster_, num_cluster(len - total));
            } else {
                wr_cluster_ = next_cluster;
            }
            wr_cluster_off_ = 0;
        }

        uint8_t* sec = GetSectorByCluster<uint8_t>(wr_cluster_);
        size_t n = std::min(len, bytes_per_cluster - wr_cluster_off_);
        memcpy(&sec[wr_cluster_off_], &buf8[total], n);
        total += n;

        wr_cluster_off_ += n;
    }

    wr_off_ += total;
    fat_entry_.file_size = wr_off_;
    return total;
}

```

本題となる `fat::FileDescriptor::Write()` の実装を**リスト 26.21** に示します。上から順に説明していきます。

`num_cluster` は指定したバイト数を書き込むのに必要なクラスタの数を計算する関数です。`bytes_per_cluster - 1` を足してから `bytes_per_cluster` で割るのは、小数点以下を切り上げるときの書き方です。1 バイトを書くにも 1 つのクラスタが必要になりますから、切り捨てではなく切り上げて計算します。

次の if 文は、初めてファイルへ書き込むときに実行されます。`fat_entry_.FirstCluster()` の値は、新規作成されたばかりの空ファイルであれば 0 となります。この場合、後で実装する `AllocateClusterChain()` を使って必要なだけの空きクラスタを割り当てます。

一方、既存のファイルであれば `fat_entry_.FirstCluster()` は先頭のクラスタ番号となります。この場合、そのクラスタ番号を `wr_cluster_` に設定します。すると、以降の書き込み処理によりファイル内容が先頭から上書きされることになります。なんだかファイル内容が上書きされて壊されちゃうので心配になりますが、`fopen()` の仕様として、“w” モードで開いたファイルの内容はすべて削除されることになっていますから、これでいいのです。

`fopen()` を “a” モード^{*2} で呼び出した場合、ファイルの末尾から追記するモードになるというのが仕様です。しかし、現在の `syscall::OpenFile()` の実装ではそれに対応できていません。このモードに対応するのは読者の課題とします。

次に `while` 文の処理を説明します。読み込みと同じように、指定されたバイト数だけ書き込みが終わるまでループするという構造になっています。全体の流れはファイルの読み込みと似ています。

ループ先頭の処理が特徴的です。ループ先頭では、クラスタチェーンの末尾に来てしまったら空きクラスタを増やす、という処理をやっています。書き込みオフセット (`wr_cluster_off_`) がクラスタ末尾に来たら、チェーンの次のクラスタを `wr_cluster_` に設定しようとしています。チェーンがまだ続く場合はそれで良いのですが、チェーンの末尾だったら次のクラスタというものは存在しません。そこで `ExtendCluster()` により必要なだけのクラスタをチェーン末尾に追加します。残りの処理はファイル読み込みと同じ感じなので、特に難しいことはないでしょう。

リスト 26.22 `AllocateClusterChain()` は指定された数の空きクラスタを確保する (fat.cpp)

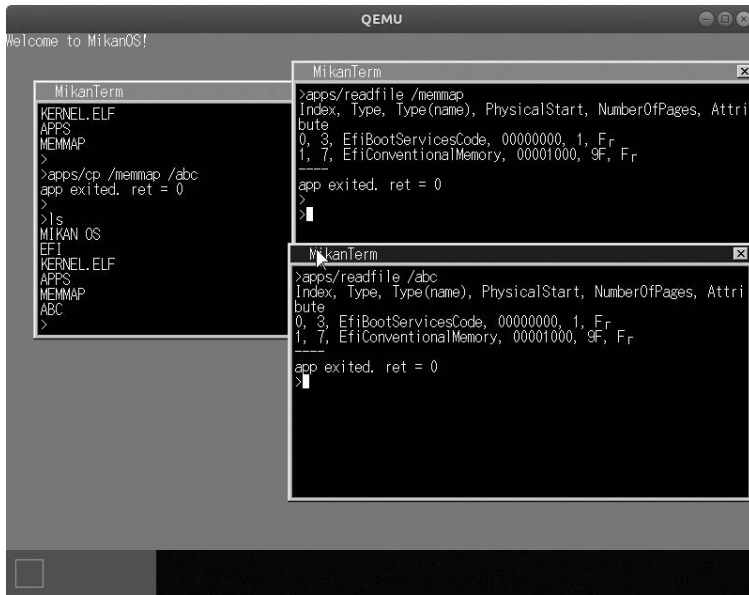
```
unsigned long AllocateClusterChain(size_t n) {
    uint32_t* fat = GetFAT();
    unsigned long first_cluster;
    for (first_cluster = 2; ; ++first_cluster) {
        if (fat[first_cluster] == 0) {
            fat[first_cluster] = kEndOfClusterchain;
            break;
        }
    }

    if (n > 1) {
        ExtendCluster(first_cluster, n - 1);
    }
    return first_cluster;
}
```

リスト 26.22 に、空ファイルに初めて書き込む際に `fat::FileDescriptor::Write()` が使う関数 `fat::AllocateClusterChain()` の実装を示します。この関数は指定された数の空きクラスタからなるクラスタチェーンを作り、先頭のクラスタの番号を返します。

*2 「a」は append モードの頭文字。append とは、英語で「追記」の意味。

図 26.5 cp コマンドでファイルをコピーした様子



さて、改造が一通り終わりましたので cp コマンドを実行してみます。実行の様子を表した図 26.5 を見てください。3つのターミナルが開かれています。左のターミナルでは cp コマンドを起動しました。cp が終了コード 0 で正常終了していることと、ABC という名前のファイルが増えていることが分かります。

右上のターミナルでは memmap の内容を、右下のターミナルでは abc の内容を表示しています。同じ内容になっていることが分かりますね。つまり cp コマンドがちゃんとファイルのコピーをしたということです。筆者は今、結構感動しています。ううう、MikanOS が結構本格的になってきて嬉しいです。

そういえば今回は apps/newlib_support.c の write() は改造しませんでした。にもかかわらず、うまくファイル書き込みができたのは、write() がもともと PutString() システムコールを利用していたからです。今回の改造により PutString() が fd=1 以外のファイルに対しても正しく動くようになったため、write() は改造しなくて済みました。

第 27 章

アプリのメモリ管理

この章の主題はアプリケーションによるメモリ確保です。MikanOS は 64 ビットモードで動きますので、アプリケーションには広大な仮想アドレス空間が提供されています。しかし、今はそれらを活用できていたとは言えません。メモリ確保の方法としてデマンドページングを実装します。デマンドページングはどんなに大きなメモリ領域を確保しても、実際にそのメモリ領域を使用するまでは物理メモリが圧迫されないという不思議な仕組みです。この章ではじめて、ページフォルトを有効に利用したプログラミングが登場します。

27.1 デマンドページング (osbook_day27a)

デマンドページング (demand paging) の言葉の意味をまずは説明します。デマンドは要求、需要というような意味の英単語です。ページングはおなじみ、アドレスマッピングの仕組みのことです。ページングの仕組みを思い出してみると、階層ページング構造に対して設定を行うことで、ページ（固定長の仮想アドレス範囲）をフレーム（固定長の物理アドレス範囲）にマップするものでしたね。デマンドページングとは、初めはどのページに対してもフレームを割り当てないでおき、ページに初めてアクセスされた時点でそのページにだけフレームを割り当てる方式のことです。ページに対して需要が発生した時点で、その需要があったページだけ処理するというのがデマンドページングの名前が意味するところです。

デマンドページングはページフォルトという、普通だったら起きてほしくない CPU 例外を活用して実現する面白い仕組みです。準備としてページテーブルのエントリにある present ビットを 0 にしておきます。といっても初期状態でページテーブルは 0 クリアされますから、これは何もしなくても達成されますね。準備が完了したらそのページにアクセスがあるのを待ちます。そのページがアクセスされたときの処理の流れは次のようになります。

1. ページにアクセスがあると、present ビットが 0 のためページフォルトが発生します。
2. ページフォルトの例外ハンドラは、原因となったページにフレームを割り当てます。
3. 例外ハンドラを終了し、ページフォルトの原因となった命令から再実行します。
4. すると、何事もなかったかのように処理が続行されます。

今までであれば、例外が発生したらアプリを強制終了してしまっていました。しかし、デマンドページングではページフォルトが発生しても物理メモリを割り当てた後でアプリの実行を継続します。面白い仕組みではありませんか？

デマンドページングはいろんなところに活用できます。例えばアプリの実行に際して ELF ファイルをメモリに読み込みますが、デマンドページングを使うと実際のファイル読み込みをほとんどしないうちにアプリを起動できます。

ELF ファイルのヘッダさえ読み込むことができればエントリポイントのアドレスが手に入ります。ELF ファイルの LOAD セグメントをメモリに配置する処理をせず、そのままエントリポイントを call します。当然、呼び出し先のページは present=0 ですからページフォルトが発生しますよね。ページフォルトのハンドラの中で、対象のページにフレームを割り当てます。そして、そのフレームに対し ELF ファイルの対応する部分を読み出します。その状態で例外ハンドラを抜けると call 命令が再実行されます。今度は present=1 になっていて、ELF ファイル（の一部）も読み込まれていますのでエントリポイントが正常に実行されます。

上記はデマンドページングをファイルの遅延読み込みに使った例です。実行ファイルの中で実際に実行される機械語の範囲が限られているような場合、実行ファイル全体をメモリ上に読み込む必要はないわけです。ある短い時間枠で見ると、大きなプログラムでも実行されているのは特定の for 文だけだったりします。これを**参照の局所性**と呼びます。その for 文を構成する機械語さえメモリ上にあれば、CPU はその for 文を実行し続けられるのです。これはプログラムが使う変数についても成り立ちます。そのときに必要な変数だけがメモリ上にあればプログラムは実行が可能です。

デマンドページングを単にメモリ確保の仕組みとして使うのは簡単です。ページフォルトが起きたときにフレームを割り当てるだけでいいですからね。デマンドページングによりファイルの遅延読み

込みをするのは少し難しいです。ページとファイルを対応付ける仕組みが必要になります。そもそも、MikanOS ではファイルが最初からメモリ上に置かれているため、ファイルの遅延読み込みをするメリットがほとんどありません。HDD や SSD からデータを読み出す速度がとても遅いからこそ遅延読み込みが役に立つのです。ということで、この節では純粋にアプリがメモリ確保をする仕組みとしてデマンドページングを実現します。

リスト 27.1 ページフォルトが起きたら `HandlePageFault()` を呼び (interrupt.cpp)

```
__attribute__((interrupt))
void IntHandlerPF(InterruptFrame* frame, uint64_t error_code) {
    uint64_t cr2 = GetCR2();
    if (auto err = HandlePageFault(error_code, cr2); !err) {
        return;
    }
    KillApp(frame);
    PrintFrame(frame, "#PF");
    WriteString(*screen_writer, {500, 16*4}, "ERR", {0, 0, 0});
    PrintHex(error_code, 16, {500 + 8*4, 16*4});
    while (true) __asm__("hlt");
}
```

デマンドページングの核はページフォルト発生時の処理です。リスト 27.1 に示す通り、ページフォルトが起きたときに、新しく作る関数 `HandlePageFault()` を呼び出すようにしました。この関数がエラー無く終了すればデマンドページングの処理が成功したということで、アプリを強制終了させることなく例外ハンドラを抜けます。その場合、アプリは何事もなかったかのように処理を続けます。

ページフォルトは、プログラムが `present=0` のページにアクセスしたり、そのページを読み書きする十分な権限がなかったりすると発生します。ページフォルトの例外ハンドラに渡されるエラーコード(引数 `error_code`)には例外の理由が記録されています。また、例外発生時の CR2 レジスタの値は原因となるメモリアドレスが記録されています。これら 2 つの情報を組み合わせることで、どのページを読み書きしようとして、どのような理由でページフォルトが発生したかを特定できます。

リスト 27.2 `HandlePageFault()` は例外の原因となったページにフレームを割り当てる (paging.cpp)

```
Error HandlePageFault(uint64_t error_code, uint64_t causal_addr) {
    auto& task = task_manager->CurrentTask();
    if (error_code & 1) { // P=1 かつページレベルの権限違反により例外が起きた
        return MAKE_ERROR(Error::kAlreadyAllocated);
    }
    if (causal_addr < task.ÐPagingBegin() || task.ÐPagingEnd() <= causal_addr) {
        return MAKE_ERROR(Error::kIndexOutOfRange);
    }
    return SetupPageMaps(LinearAddress4Level{causal_addr}, 1);
}
```

リスト 27.2 に `HandlePageFault()` の実装を示します。エラーチェックをしているだけの if 文を無視すると、この関数の本質はたった 1 行、`SetupPageMaps()` の呼び出しだけです。この 1 行はページフォルトの原因となったページに物理フレームを割り当てます。デマンドページングというとし

うな響きですが、なんとこれだけなのです。

SetupPageMaps() はもともと terminal.cpp の中の無名名前空間で定義していた関数です。この関数をデマンドページングの処理でも共通に使いたくなりましたが、無名名前空間にある限り他のファイルからは利用できません。そこで、グローバル名前空間に移動させ、terminal.cpp と paging.cpp の両方で使えるようにしました。そして、ページング関連の処理だということで、ついでに paging.cpp に定義を移しておきましたよ。

さて、HandlePageFault() でやっている 2 つのエラーチェックはちょっと重要なので説明します。1 つ目の if 文ではエラーコードのビット 0 が 1 だったらエラーとしています。ページフォルトのエラーコードは表 27.1 に示すビット構成になっています。ビット 0 が 1 というのは、ページ自体は存在するけれども、何らかの権限違反によってページフォルトが起きたことを意味します。デマンドページングは存在しないページにアクセスしたときに物理フレームを割り当てる仕組みですから、P=0 の必要があります。P=1 であるページフォルトはデマンドページングによるページフォルトではなく、本当にダメなページフォルトです。

表 27.1 ページフォルトのエラーコードのビット定義 (抜粋)

ビット位置	ビット名	意味
0	P	0= 存在しないページによって例外が発生した
		1= ページの権限違反によって例外が発生した
1	W/R	0= 読み込みによって例外が発生した
		1= 書き込みによって例外が発生した
2	U/S	0= 例外の原因はスーパーバイザーモードのアクセス
		1= 例外の原因はユーザーモードのアクセス
3	RSVD	0= 予約ビットの違反が例外の原因ではない
		0= 予約ビットが 1 になっているため例外が発生した

2 つ目の if 文ではページフォルトの原因となったアドレスが task.DPagingBegin() から task.DPagingEnd() の間に収まっていることを確認しています。これら 2 つのメソッドはデマンドページングとして有効な仮想アドレスの範囲を表します。このアドレス範囲はアプリが事前に申告しておくものです。この範囲チェックがない場合、アプリのバグ（例えばバッファオーバーラン）により空きアドレスを読み書きしまくって無制限に物理フレームの割り当てが行われ、物理メモリが枯渇するかもしれません。そこで、デマンドページングが可能なアドレス範囲を事前申告した範囲に限ることで、バグによるメモリの枯渇を防ぎます。

リスト 27.3 デマンドページングのアドレス範囲を保持するメンバ変数 (task.hpp)

```
uint64_t dpaging_begin_{0}, dpaging_end_{0};
```

リスト 27.4 デマンドページングのアドレス範囲を取得、設定するメソッド (task.cpp)

```
uint64_t Task::DPagingBegin() const {
    return dpaging_begin_;
}

void Task::SetDPagingBegin(uint64_t v) {
    dpaging_begin_ = v;
}

uint64_t Task::DPagingEnd() const {
    return dpaging_end_;
}

void Task::SetDPagingEnd(uint64_t v) {
    dpaging_end_ = v;
}
```

デマンドページング可能なアドレス範囲を保持するメンバ変数の定義をリスト 27.3 に、これらのメンバ変数を読み書きするメソッドの定義をリスト 27.4 に示します。これらはすべてアプリに紐づく情報ですから、例によって Task 構造体に入れました。このアドレス範囲をアプリから設定できるようにする必要がありますから、次にそのためのシステムコールを作ります。

リスト 27.5 DemandPages() はデマンドページング可能なアドレス範囲を拡大する (syscall.cpp)

```
SYSCALL(DemandPages) {
    const size_t num_pages = arg1;
    // const int flags = arg2;
    __asm__("cli");
    auto& task = task_manager->CurrentTask();
    __asm__("sti");

    const uint64_t dp_end = task.DPagingEnd();
    task.SetDPagingEnd(dp_end + 4096 * num_pages);
    return { dp_end, 0 };
}
```

デマンドページングが可能な仮想アドレスの範囲をアプリが申告するためのシステムコール `syscall::DemandPages()` を新しく作りました (リスト 27.5)。このシステムコールは引数にページ数を取って、その分だけ `Task::dpaging_end_` を後ろにずらします。拡大したアドレス範囲は、以降のページフォルト発生時に物理フレーム割り当てが可能となります。

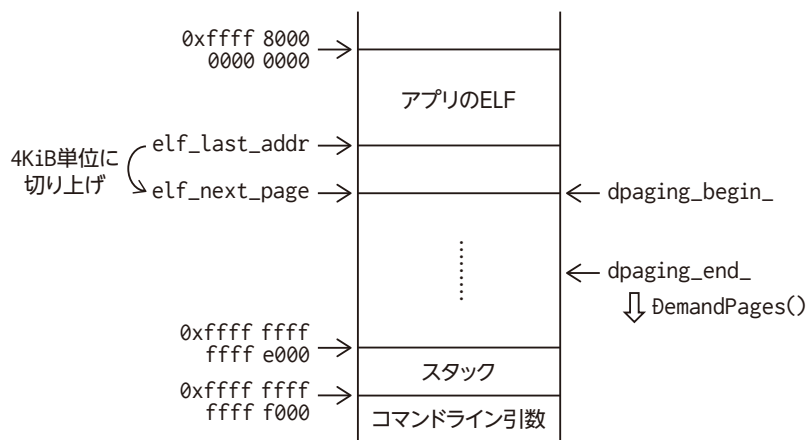
アプリにとっては、このシステムコールでアドレス範囲を拡大することは、実際にメモリを確保することと事実上同じになります。なぜなら、アプリから見ると、拡大した範囲のアドレスに始めから物理フレームが割り当てられているのか、アクセスした瞬間にページフォルトが発生して物理フレームが割り当てられたのが区別できないからです*1。ということは、この仕組みを `malloc()` の実現のために使えるということです。後ほど `sbrk()` を、デマンドページングを使って実装し直してみます。

*1 メモリを読み書きするのにかかった時間を計るなどして推測することはできるかもしれませんが。

アプリが `DemandPages()` システムコールを使う前に、`Task::dpaging_begin_` と `Task::dpaging_end_` を適切な値に初期化しなければなりません。でも、一体どんな値にすればいいでしょうか？

`0xffff800000000000` 以降のアドレス空間の中で使われていない範囲であれば自由です。ただし飛び飛びはだめで、連続している必要があります。`0xffff800000000000` 以降のアドレス空間で最も広い連続区間を探す、というのは愚直にやると大変です（なんたって 128TB 分のアドレス空間がありますからね……）。そこで、アプリの ELF ファイルが配置されている直後を開始点とし、そこから後ろに延ばしていくことにします(図 27.1)。これらを設定する処理を `CallApp()` でアプリを起動する前に追加します。

図 27.1 `DemandPages()` によりアドレス範囲が拡大する様子



リスト 27.6 ELF ファイルの最終アドレスを取得する (terminal.cpp)

```
const auto [ elf_last_addr, elf_err ] = LoadELF(elf_header);
if (elf_err) {
    return elf_err;
}
```

リスト 27.6 に示すように、メモリにロードした ELF ファイルの最終アドレス `elf_last_addr` を取得する処理を追加しました。`elf_last_addr` は、ELF ファイル内のすべての LOAD セグメントのうち、最も後ろに配置された LOAD セグメントの末尾アドレスを表します。ELF ファイルをロードする関数 `LoadELF()` はもともとエラー値だけを返す仕様でしたが、最終アドレスを追加で返すようにしました。LOAD セグメントの `p_vaddr + p_memsz` の最大値を計算するだけの単純な改造です。

リスト 27.7 デマンドページングのアドレス範囲の初期値を設定する (terminal.cpp)

```
const uint64_t elf_next_page =
    (elf_last_addr + 4095) & 0xffff'ffff'ffff'f000;
task.SetDPagingBegin(elf_next_page);
task.SetDPagingEnd(elf_next_page);

auto entry_addr = elf_header->e_entry;
```



```
int ret = CallApp(argc.value, argv, 3 << 3 | 3, entry_addr,
```

取得した ELF ファイルの最終アドレスを利用してデマンドページングのアドレス範囲を初期化する処理をリスト 27.7 に示します。elf_last_addr は 4KiB 単位に揃っているわけではありません。4KiB 単位のアドレスに切り上げた値を elf_next_page としています。デマンドページングが必ずページ単位で処理されるようにするためです。

リスト 27.8 デマンドページングを使ってメモリを確保する sbrk() (newlib_support.c)

```
caddr_t sbrk(int incr) {
    static uint64_t dpage_end = 0;
    static uint64_t program_break = 0;

    if (dpage_end == 0 || dpage_end < program_break + incr) {
        int num_pages = (incr + 4095) / 4096;
        struct SyscallResult res = SyscallDemandPages(num_pages, 0);
        if (res.error) {
            errno = ENOMEM;
            return (caddr_t)-1;
        }
        program_break = res.value;
        dpage_end = res.value + 4096 * num_pages;
    }

    const uint64_t prev_break = program_break;
    program_break += incr;
    return (caddr_t)prev_break;
}
```

さて、最後に Newlib の malloc() が必要とするメモリ確保用の関数 sbrk() を、デマンドページングを使って実装してみます (リスト 27.8)。sbrk() はプログラムブレイクを設定する関数で、もとの実装は「25.2 ファイル読み込み」で作ったものでした。

プログラムブレイクはどんどん後ろに伸びていくメモリ領域の末尾を表す値のことでしたね。これは今回作ったデマンドページングの仕組みとそっくりです。1 つ違うのは、sbrk() の引数はバイト単位で指定される可能性がある一方、デマンドページングによるアドレス範囲の拡大はページ単位でしか起こらないことです。もし sbrk() が引数 128、256 で 2 回呼び出されたとしましょう。初回の呼び出し時は SyscallDemandPages() を 1 回実行します。しかし、sbrk() の 2 回目の呼び出し時は、既に 1 ページ = 4KiB のデマンドページング領域が確保されているため、SyscallDemandPages() を実行する必要はありません。ちょっと複雑な sbrk() の実装は、そんな動きを実現するためのものなのです。

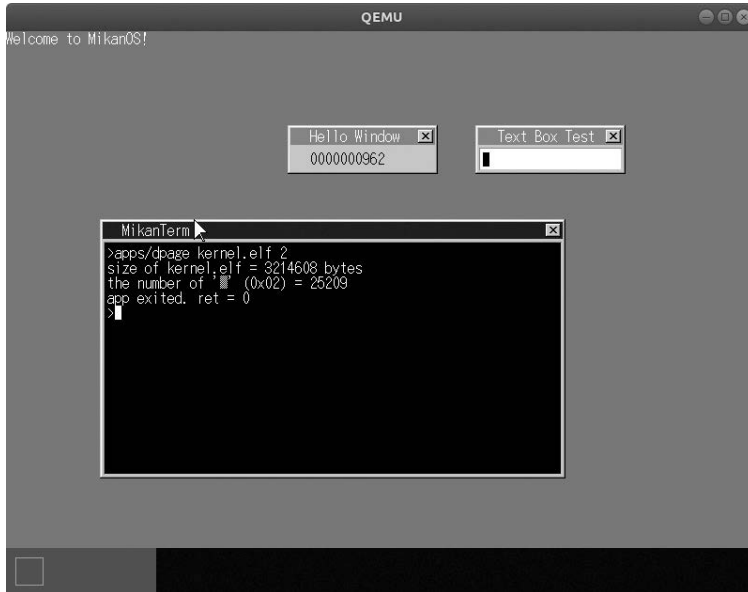
改造が完了しましたので実行してみます。といいつつ、あんまり良いアプリがありませんね。ちょっとだけ malloc() が使われるアプリはあるにはありますが、大量のメモリを必要とするアプリで試してみたいです。ということでデマンドページング実験用のコマンド dpage を作りました。

dpage コマンドは、dpage <path> <num> というようにファイルパスと数値を受け取ります。そして、ファイルの中で、num で指定した値を持つバイトの数を集計します。num に 10 (=0x0a) を指定すると改行文字の数、つまり行数を集計できます。詳しいコマンドの実装は割愛します。

処理の流れとしては、デマンドページングの実験のためにファイルの中身を一旦すべてデマンドページングで確保したメモリ上に読み出したのち、先頭から各バイトの値を調べます。本来ならファイルを

一気に読み出す必要は無く、少しずつ読んでバイトを検索する処理を繰り返せば少量のメモリで処理が完了します。今回はデマンドページングをテストしたいので、ファイル全体をまずメモリ上に読み出すという処理にしているのです。

図 27.2 dpage コマンドで kernel.elf の中を検索する様子



dpage コマンドを使ってメモリを大量に消費する様子は図 27.2 のようになりました。kernel.elf は 3MB 程度の大きさがあり、778 ページの仮想アドレス空間を消費する計算です。何回実行してもエラーになったり OS の動きがおかしくなったりはしませんね。いい感じです！

27.2 メモリマップトファイル (osbook_day27b)

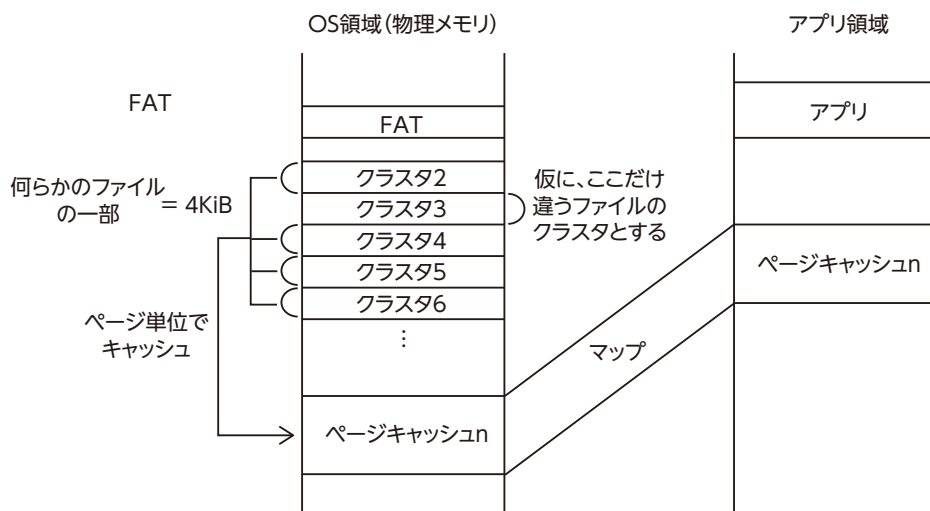
ページングを用いた興味深い仕組みの 1 つにメモリマップトファイルがあります。メモリにマップされた (mapped) ファイル、という意味です。ファイルをメモリアドレスにマップすることで、あたかも普通のメモリであるかのようにファイルを読み書きする仕組みです。似た仕組みに MMIO (メモリマップト IO) がありましたね。MMIO はハードウェアのレジスタをメモリにマップし、普通のメモリであるかのようにレジスタを読み書きします。様々なものをメモリに見せかけることで、特殊な機械語を使うことなく、mov 命令だけで読み書きできるのが魅力です。

ファイルの読み書きは、普通は fread() や fwrite() を使ってファイルの先頭から順に行います。ファイルのデータを単なるバイト列とみなして、その列を先頭から順に読んだり書いたりするので、ストリーム^{*2}方式と呼ぶこともあります。

メモリマップトファイル方式は、ファイルの内容を仮想アドレス空間にマップして読み書きします。あたかも普通のメモリのようにアクセスできますから、ばらばらの場所を読み書きするランダムアクセスがとてもしやすいです。fread() や fwrite() などのストリーム方式の API を使ってランダムアク

セスをするには、1 箇所を読み書きしたら次の目的地にシーク（移動）し、またちょっと読み書きしてシーク、というのを繰り返す必要があります。これはとても面倒ですし、実行効率も悪そうです。

図 27.3 ページキャッシュをアプリの仮想アドレス空間にマップする



メモリマップトファイルは図 27.3 な感じに実現してみようと思います。ポイントは、**ページキャッシュ**という名の物理メモリ領域を介して、アプリの仮想アドレス空間と実際のファイル内容を結びつけているところです。ページキャッシュにファイル内容（の一部）をコピーし、アプリの仮想アドレス空間にそのページをマップすることで、アプリがファイル内容を読み書きできるようになります。メモリマップトファイルというと、HDD や SSD などの記憶装置上にあるファイルがそのまま仮想アドレス空間にマップされているような印象を受けるかもしれませんが、しかし、一旦このように本当のメモリ領域を仲介しなければ、メモリマップトファイルは実現できません（少なくとも普通のパソコンでは）。この仕組みは Linux でのメモリマップトファイルの仕組みと大体同じです。

MikanOS ではファイルがもともとメモリ上に置かれているため、ページキャッシュにファイルデータをコピーしなければならないこの方式は無駄が多いです。ですが、メモリマップトファイルの実現にはこの方式が必須なのです。というのも FAT ファイルシステムのクラスタがメモリページ以上の大きさを持つとは限らないからです。1 クラスタ=2 セクタとして FAT ファイルシステムを作った場合、1 クラスタ=1KiB となります。

複数のクラスタからなるファイルの場合、飛び飛びのクラスタにデータが配置されることがあります。これを**フラグメンテーション**を起こした状態とか、フラグメント化している、などと表現します。アプリの仮想アドレス空間にファイルをマップする際は、ファイルデータの先頭から連続するようにマップしなければなりません。しかし、クラスタが 4KiB より小さい場合にフラグメンテーションが起きると、4KiB ページでは粒度が大きすぎて正しくマップできません。だからページキャッシュを仲介する方法を採用します。

* 2 Stream（小川）。バイト列が、まるでちょろちょろと細く流れる小川に見えるため。

リスト 27.9 アプリ固有のファイルマッピングを表現する構造体 (task.hpp)

```
struct FileMapping {
    int fd;
    uint64_t vaddr_begin, vaddr_end;
};
```

まずはファイルのマッピングを表現する FileMapping 構造体を作りました (リスト 27.9)。どのファイル (fd) を、仮想アドレスのどの範囲 (vaddr_begin から vaddr_end まで) にマップするか、という紐づけを表します。ファイルディスクリプタ番号や仮想アドレス空間というのはアプリ固有ですから (アプリが違えば同じ「3 番」でも違うファイルを指します)、アプリ毎に FileMapping クラスのインスタンスを持つことになります。

リスト 27.10 Task にファイルマッピング情報を持たせる (task.hpp)

```
uint64_t dpaging_begin_{0}, dpaging_end_{0};
uint64_t file_map_end_{0};
std::vector<FileMapping> file_maps_{};
```

アプリ毎に情報を持たせるのはもう慣れっこですよ。そうですね。そうです、Task クラスに情報を持たせれば良いのです。リスト 27.10 のようにファイルマッピングの配列 file_maps_ を追加しました。その直前にある file_map_end_ も今回追加した変数です。これはファイルをマップする仮想アドレス範囲に影響する変数で、詳しくは直後に説明します。

リスト 27.11 追加したメンバ変数を読み書きするメソッド (task.cpp)

```
uint64_t Task::FileMapEnd() const {
    return file_map_end_;
}

void Task::SetFileMapEnd(uint64_t v) {
    file_map_end_ = v;
}

std::vector<FileMapping>& Task::FileMaps() {
    return file_maps_;
}
```

リスト 27.11 に示す通りに、追加した 2 つのメンバ変数を読み書きするメソッドを増設しました。何も説明することはないかと思います。

リスト 27.12 file_map_end_ に初期値を設定する (terminal.cpp)

```
task.SetDPagingEnd(elf_next_page);

task.SetFileMapEnd(0xffff'ffff'ffff'e000);

auto entry_addr = elf_header->e_entry;
int ret = CallApp(argc.value, argv, 3 << 3 | 3, entry_addr,
```

```

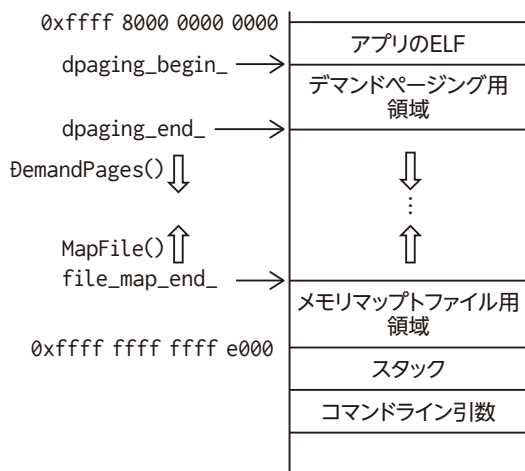
        stack_frame_addr.value + 4096 - 8,
        &task.OSStackPointer());

task.Files().clear();
task.FileMaps().clear();

```

さて、Task::file_map_end_ の役割を説明します。この変数は、メモリマップトファイルに利用するアドレス範囲を決める変数です。リスト 27.12 のように、アプリを起動する前に初期値を設定するようにしました。初期値は 0xfffffffffe000 と、仮想アドレス空間のほぼ末尾に近い値になっています。この理由は、図 27.4 のように仮想アドレスの末尾から 0 方向に向けて、メモリマップトファイル用のアドレス範囲を拡大していこうと考えたからです。

図 27.4 メモリマップトファイルは仮想アドレス末尾に置く



というのも、アプリ用の仮想アドレス空間（0xffff800000000000 から後ろ）の前の方にはアプリの ELF ファイルがあり、その直後からのアドレス範囲はデマンドページングで利用されます。デマンドページングのアドレス範囲はどんどん後ろに伸びていきますから、その後ろに障害物があると困ったことになります。したがって、メモリマップトファイルは末尾の方から前方に向かってアドレス範囲を拡大していくことにしました。これなら広大な仮想アドレス空間を最大限利用できます。

アプリが終了した後で忘れずに Task::file_maps_ を空にしましょう。筆者は当初この処理を書き忘れていて、アプリの挙動が 1 回目と 2 回目で変わるバグに悩まされました。

リスト 27.13 MapFile() システムコールはファイルマッピングを登録する (syscall.cpp)

```

SYSCALL(MapFile) {
    const int fd = arg1;
    size_t* file_size = reinterpret_cast<size_t*>(arg2);
    // const int flags = arg3;
    __asm__("cli");
    auto& task = task_manager->CurrentTask();
    __asm__("sti");
}

```

```

if (fd < 0 || task.Files().size() <= fd || !task.Files()[fd]) {
    return { 0, EBADF };
}

*file_size = task.Files()[fd]->Size();
const uint64_t vaddr_end = task.FileMapEnd();
const uint64_t vaddr_begin = (vaddr_end - *file_size) & 0xffff'ffff'ffff'f000;
task.SetFileMapEnd(vaddr_begin);
task.FileMaps().push_back(FileMapping{fd, vaddr_begin, vaddr_end});
return { vaddr_begin, 0 };
}

```

リスト 27.13 に新しく作ったシステムコール `syscall::MapFile()` の実装を示します。このシステムコールは第1引数にファイルディスクリプタ番号を取り、そのファイルを仮想メモリにマップします。戻り値はファイルの内容をマップしたメモリ領域の先頭アドレスとなります。アプリはそのアドレスから始まるファイルの大きさの分のメモリ領域を読むことで、ファイルデータを読み出せます。そういえばファイルの大きさを得るためのシステムコールがありませんでした。MapFile() の第2引数に `size_t` 型の変数へのポインタを指定することで、ファイルサイズ (バイト数) を取得できるようにしました。

システムコールの処理をちょっと詳しく見てみます。まず、いつものようにファイルディスクリプタ番号が不正な値になっていないことを確認します。確認が終わったら、次に、システムコールの第2引数で指定されたポインタを使い、ファイルサイズを書き込みます。これでアプリはファイルの大きさを知ることができます。

次の行からがこのシステムコールの本質的な処理となっています。まず現在の `Task::file_map_end_` の値を変数 `vaddr_end` に保存します。次に、その値からファイルサイズを引き、4KiB 境界にそろえた値を `vaddr_begin` とします。4KiB 境界にそろえているのは、もちろん 4KiB ページ境界に合わせるためです。そして、MapFile() が再度実行されたときに備えて、`Task::file_map_end_` を `vaddr_begin` の値に更新します。最後に、今作成したファイルマッピング情報を `file_maps_` に追加し、システムコールの処理は終了です。

もう気付いたかもしれませんが、このシステムコールは階層ページング構造を変更したりは一切していません。単に、ある仮想アドレス範囲をこのファイルにマッピングしたいなあ、という希望を登録しているだけです。ファイルが本当に仮想アドレス空間にマップされるようにする処理は、ページフォルトが起きた際にやることにします。このように、本当に必要になるまで処理を遅らせるのを遅延処理といって、不要な処理を省略できる可能性が高くなります。その処理が比較的重たい処理であれば、遅延処理の効果が出やすくなります。それに、遅延処理にはロマンを感じます。

リスト 27.14 ページフォルト発生時にファイルをマップする (paging.cpp)

```

Error HandlePageFault(uint64_t error_code, uint64_t causal_addr) {
    auto& task = task_manager->CurrentTask();
    if (error_code & 1) { // P=1 かつページレベルの権限違反により例外が起きた
        return MAKE_ERROR(Error::kAlreadyAllocated);
    }
    if (task.DPagingBegin() <= causal_addr && causal_addr < task.DPagingEnd()) {
        return SetupPageMaps(LinearAddress4Level{causal_addr}, 1);
    }
}

```

```

    if (auto m = FindFileMapping(task.FileMaps(), causal_addr)) {
        return PreparePageCache(*task.Files()[m->fd], *m, causal_addr);
    }
    return MAKE_ERROR(Error::kIndexOutOfRange);
}

```

リスト 27.14 にページフォルト発生時に呼ばれる `HandlePageFault()` を示します。今回はデマンドページングの処理だけをしていました。今回はメモリマップトファイルの処理を加えています。

ページフォルトの原因となった仮想アドレス `causal_addr` がデマンドページングのアドレス範囲に含まれていなかったとき、メモリマップトファイルの処理が始まります。まず `FindFileMapping()` を使って、そのアドレスが該当するファイルマッピングを探します。 `MapFile()` システムコールで事前に登録されたファイルマッピングのアドレス範囲に該当するものがあれば、そのファイルマッピングが返されます。

`FindFileMapping()` が有効なポインタを返せば `if` 文の中身が実行されます。 `if` 文の中身では `PreparePageCache()` を用いて `causal_addr` を含む 4KiB ページを作成します。このページにはファイルデータの対応する部分（最大 4KiB）がコピーされ、アプリがファイルデータを読めるようになります。

リスト 27.15 指定されたアドレスを含むファイルマッピングを探す (paging.cpp)

```

const FileMapping* FindFileMapping(const std::vector<FileMapping>& fmaps,
                                   uint64_t causal_vaddr) {
    for (const FileMapping& m : fmaps) {
        if (m.vaddr_begin <= causal_vaddr && causal_vaddr < m.vaddr_end) {
            return &m;
        }
    }
    return nullptr;
}

```

`FindFileMapping()` の実装を **リスト 27.15** に示します。与えられた `FileMapping` 構造体の配列から、ある仮想アドレスを含むアドレス範囲を持つものを探し、見つけたらそれを返します。いずれのファイルマッピングも該当しなければヌルポインタを返します。

リスト 27.16 指定されたページを作成しファイルをコピーする (paging.cpp)

```

Error PreparePageCache(FileDescriptor& fd, const FileMapping& m,
                       uint64_t causal_vaddr) {
    LinearAddress4Level page_vaddr{causal_vaddr};
    page_vaddr.parts.offset = 0;
    if (auto err = SetupPageMaps(page_vaddr, 1)) {
        return err;
    }

    const long file_offset = page_vaddr.value - m.vaddr_begin;
    void* page_cache = reinterpret_cast<void*>(page_vaddr.value);
    fd.Load(page_cache, 4096, file_offset);
    return MAKE_ERROR(Error::kSuccess);
}

```

PreparePageCache() の実装をリスト 27.16 に示します。この関数は、ページフォルトの原因となった仮想アドレス causal_vaddr を含む 4KiB ページを作成し、そのページへファイルデータをコピーします。

まず、仮想アドレスのオフセット部分を 0 にして 4KiB 境界に揃えたアドレス page_vaddr を用意します。SetupPageMaps() を使って、そのアドレスから始まる 4KiB ページを 1 つ作成します。SetupPageMaps() は、内部で物理フレームを生成してページに割り当てるところまでやってくれるのでした。ページの割り当てが済んだら、そのページ（が指す物理フレーム）へとファイルデータをコピーします。

図 27.5 ページフォルトを契機としてページキャッシュを作成する様子

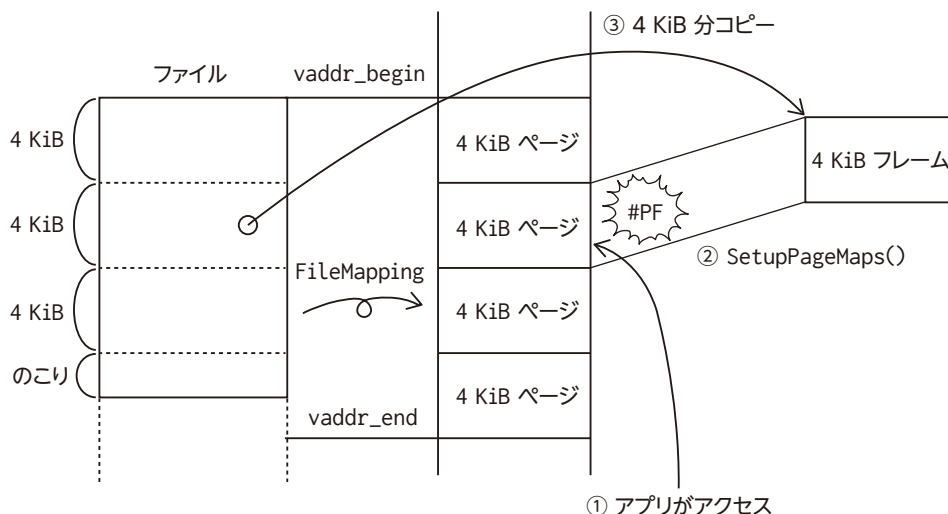


図 27.5 に、ページフォルトが起きてからページキャッシュが作られるまでを示します。まず、ファイルを読み書きしようとしたアプリが `vaddr_begin` と `vaddr_end` の範囲内の仮想アドレス領域にアクセスするとページフォルトが起きます (①)。PreparePageCache() が内部で SetupPageMaps() を呼び出すことでページにフレームを割り当てます (②)。そこに `fd.Load()` を使い、ファイルデータをコピーします (③)。

リスト 27.17 FileDescriptor にメソッドを 2 つ増やした (file.hpp)

```
class FileDescriptor {
public:
    virtual ~FileDescriptor() = default;
    virtual size_t Read(void* buf, size_t len) = 0;
    virtual size_t Write(const void* buf, size_t len) = 0;
    virtual size_t Size() const = 0;

    /** @brief Load reads file content without changing internal offset
```



```
*/
virtual size_t Load(void* buf, size_t len, size_t offset) = 0;
};
```

最後に、FileDescriptor クラスに増やした2つのメソッド Size() と Load() を紹介します (リスト 27.17)。Size() はそのファイルのバイト数を返します。Load() は、内部で管理している読み書きオフセット値を変化させることなく、指定したバッファにファイルデータを読み込みます。

リスト 27.18 Load() は指定した位置からファイル内容を読む (fat.cpp)

```
size_t FileDescriptor::Load(void* buf, size_t len, size_t offset) {
    FileDescriptor fd{fat_entry_};
    fd.rd_off_ = offset;

    unsigned long cluster = fat_entry_.FirstCluster();
    while (offset >= bytes_per_cluster) {
        offset -= bytes_per_cluster;
        cluster = NextCluster(cluster);
    }

    fd.rd_cluster_ = cluster;
    fd.rd_cluster_off_ = offset;
    return fd.Read(buf, len);
}
```

Load() はリスト 27.18 のように実装してみました。ファイルデータをバッファに読み込む処理を複数の場所に書きたくありませんでしたので、Read() を活用する形で作りました。そういえば fat.cpp には LoadFile() という似た関数があったので、今実装した Load() を利用するように改造しておきましたよ。

リスト 27.19 ファイルをメモリにマップして読んでみる (mmap/mmap.cpp)

```
#include <cstdlib>
#include <cstdio>
#include <fcntl.h>
#include "../syscall.h"

extern "C" void main(int argc, char** argv) {
    SyscallResult res = SyscallOpenFile("/memmap", O_RDONLY);
    if (res.error) {
        exit(res.error);
    }
    const int fd = res.value;
    size_t file_size;
    res = SyscallMapFile(fd, &file_size, 0);
    if (res.error) {
        exit(res.error);
    }

    char* p = reinterpret_cast<char*>(res.value);
```

```

for (size_t i = 0; i < file_size; ++i) {
    printf("%c", p[i]);
}
printf("\nread from mapped file (%lu bytes)\n", file_size);

exit(0);
}

```

メモリマップトファイルの OS 側の実装はすべて終わりましたので、検証用アプリを作ります（リスト 27.19）。新たに作ったシステムコールである `SyscallMapFile()` は、典型的には `SyscallOpenFile()` と組み合わせて使います。マップしたいファイルのファイルディスクリプタ番号が必要だからです。

`SyscallMapFile()` が成功すると、戻り値でファイルがマップされた領域の先頭アドレスが返されます。それを `char*` 型のポインタに変換して内容を読むとファイルデータを読み取ることができます。

上記のアプリではファイルの先頭から 1 文字ずつ `printf()` により表示しています。実際には、このような逐次アクセスはストリーム方式の API (`fread()`) を使う方がシンプルに書けます。メモリマップトファイルが本領発揮するのはランダムアクセスのときですが、ランダムアクセスを用いた簡単な検証アプリを思い付きませんでした。ごめんなさい。

図 27.6 mmap コマンドがメモリにマップされたファイルを表示する様子



図 27.6 に `mmap` コマンドを実行した様子を示します。ファイルの中身がちゃんと表示されていることが分かります。全然面白くない例ですみませんが、とりあえず、成功です。

27.3 メモリ使用量を測ろう (osbook_day27c)

MikanOS にデマンドページングやメモリマップトファイルといったメモリ関連の機能が増えてきましたし、メモリ使用量を確認するコマンドがあると面白いかなと思いつきました。Linux には `free` というコマンドがあって、システムの空きメモリ量と使用中のメモリ量を確認できます。MikanOS でも同じようなコマンドを作ってみます。

リスト 27.20 メモリ状態を表す構造体 (memory_manager.hpp)

```
struct MemoryStat {
    size_t allocated_frames;
    size_t total_frames;
};
```

リスト 27.20 に、使用中フレームの数と総フレームの数を表す `MemoryStat` 構造体を作りました。これを戻り値にするメソッドをメモリマネージャに追加します。

リスト 27.21 `Stat()` はその瞬間のメモリ状態を返す (memory_manager.cpp)

```
MemoryStat BitmapMemoryManager::Stat() const {
    size_t sum = 0;
    for (int i = range_begin_.ID() / kBitsPerMapLine;
         i < range_end_.ID() / kBitsPerMapLine; ++i) {
        sum += std::bitset<kBitsPerMapLine>(alloc_map_[i]).count();
    }
    return { sum, range_end_.ID() - range_begin_.ID() };
}
```

その瞬間のメモリ状態を計算して返す関数 `BitmapMemoryManager::Stat()` をリスト 27.21 のように実装しました。メモリマネージャが管理する範囲を先頭から調べていって、使用中フレームの数を数えます。このメモリマネージャは1ビット1フレームで管理していて、ビットが1なら使用中、0なら未使用ということでしたね。 `std::bitset` の `count()` メソッドを使うと、指定した整数値の中で1になっているビットの数を数えられます。

リスト 27.22 `memstat` コマンド (terminal.cpp)

```
} else if (strcmp(command, "memstat") == 0) {
    const auto p_stat = memory_manager->Stat();

    char s[64];
    sprintf(s, "Phys used : %lu frames (%llu MiB)\n",
            p_stat.allocated_frames,
            p_stat.allocated_frames * kBytesPerFrame / 1024 / 1024);
    Print(s);
    sprintf(s, "Phys total: %lu frames (%llu MiB)\n",
            p_stat.total_frames,
            p_stat.total_frames * kBytesPerFrame / 1024 / 1024);
    Print(s);
} else if (command[0] != 0) {
```

リスト 27.22 に memstat コマンドの実装を示します。メモリマネージャにメモリ状態を問い合わせ、それを表示するだけの非常に単純な処理ですね。

図 27.7 memstat を使って cube が消費するメモリ量を測る

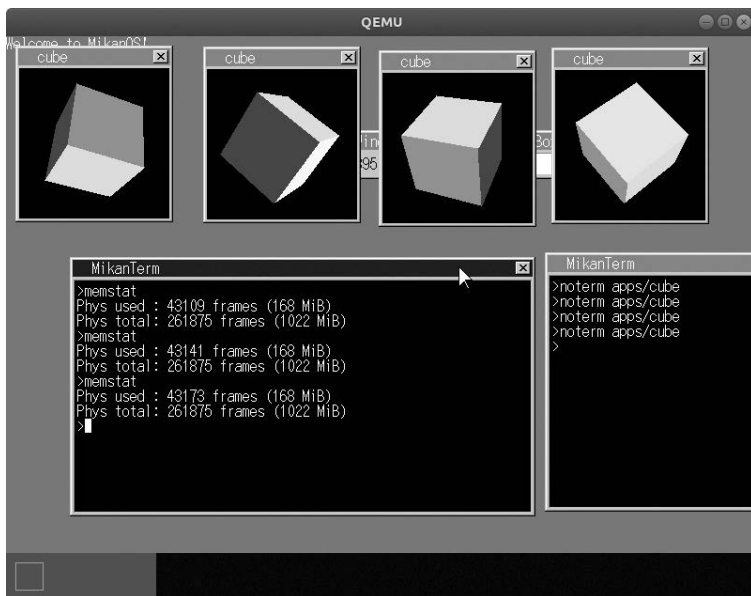


図 27.7 に memstat を 3 回実行した様子を示します。最初はシステム起動直後、2 番目は追加のターミナルを開いて apps/cube を 2 つ起動させたとき、最後はさらに apps/cube を 2 つ起動させた後のメモリ量です。2 つ起動させるたびに 32 フレームが消費されているなあ、ということが分かります。結構簡単に便利なコマンドを作れました。満足！

27.4 コピーオンライト (osbook_day27d)

cube アプリを 1 つ立ち上げるたびにメモリが消費されるのは当然と思うかもしれませんが、でもよく考えてみるとちょっと無駄があります。それは、アプリを実行すると問答無用で ELF ファイルの全体を物理メモリ上に読み込んでしまっていることです。すなわち、同じアプリをいくつも実行すると、メモリ上に同じデータのコピーがたくさん生成されてしまうのです。ELF ファイルの中の機械語や読み込み専用のデータ領域については、同じアプリをいくつ実行してもすべて同一になるはずですが、機械語や読み込み専用データ領域を複数のタスクで共有できればメモリの節約につながりますよね。

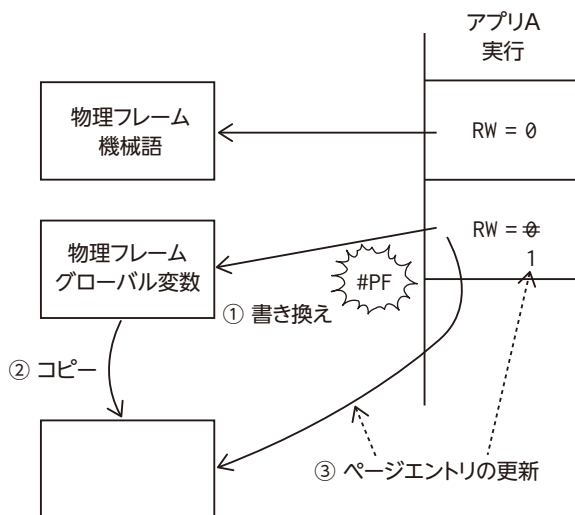
ページングの機能を使えば、複数のタスクで同じ物理メモリを共有することが簡単にできます。2 つのタスクそれぞれの階層ページング構造に、同じ物理アドレスを指すページを設定すれば良いだけです。実は、これは OS 本体用のアドレス領域 (0 から 0x7fffffff まで) では既にやっていることです。アプリを新規に起動する際、アプリ用の PML4 の前半部分に OS 用の PML4 の前半部分をコピーしているため、物理メモリ上の OS 領域はすべてのアプリで共有されます。

最初にアプリを起動したときに ELF ファイルを物理メモリに読み込み、同じアプリを 2 回目以降に

起動する際はその物理メモリを参照するようにページ設定だけ行う、という方法が考えられます。これはうまくいきません。なぜなら ELF ファイルの中にはグローバル変数も含まれていて、それらは書き換えられてしまうからです。複数のアプリが同じ物理メモリを参照していると、1つのアプリでグローバル変数を書き換えると他のアプリにも影響を与えてしまいます。

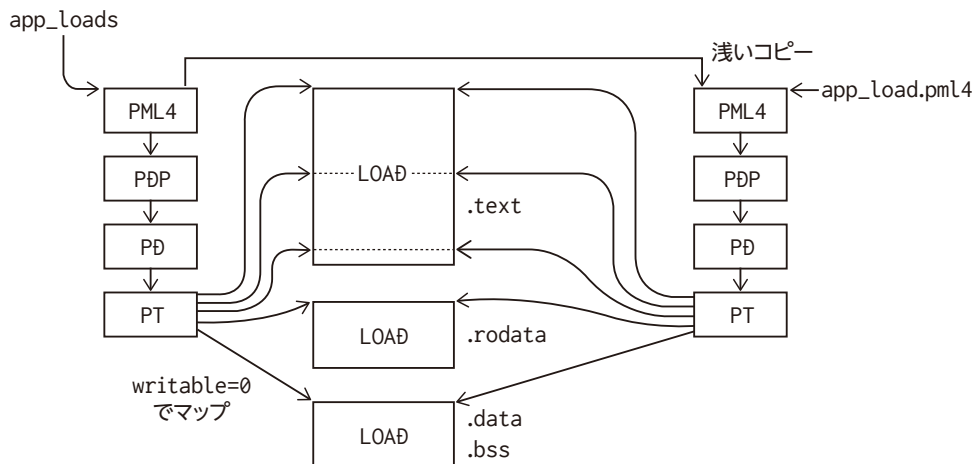
そこで使えるのが、階層ページング構造の各エントリにある R/W ビットです。このビットを 0 にしておくとそのページへの書き込みを禁止できます。アプリがそのページに対し書き込みをしようとするとページフォルトが起きます。ページフォルトのハンドラ内で、当該ページの内容をコピーした物理フレームを新たに確保し、ページがその物理フレームを指すように階層ページング構造を書き換えるのです (図 27.8)。ページフォルトが起きたページの設定を更新し、新しい物理フレームを指すようにして R/W ビットを 1 にします。その後アプリに戻ると、アプリは何事も無かったかのようにグローバル変数へ書き込みを行え、プログラムの実行を続けることができます。

図 27.8 グローバル変数領域がコピーオンライトされる様子



このテクニックを**コピーオンライト** (copy on write, CoW) と呼びます。読み込み専用 (R/W=0) でマップしておいて、初めて書き込まれたときにコピーする、という意味です。コピーオンライトは、コピーする処理に時間がかかり、全体の中で書き込まれる部分が少ない場合に効果を発揮します。ほとんど全体がまんべんなく書き換わるような場合は、結局すべてのページをコピーすることになり、逆に効率が悪くなることもあります。今回、アプリの起動をコピーオンライトで実現しようとしていますが、アプリ起動の場合、物理フレームのコピーは 4KiB ものメモリコピーですから時間がかかりますし、ELF ファイル全体に比べると書き換わる領域は一部ですので、コピーオンライトが効果を発揮するのではないかと思います。

図 27.9 アプリのLOAD セグメントを指す階層ページング構造を複製する



アプリのコピーオンライトを実現するために図 27.9 のようなデータ構造を作ることを目指します。具体的な作り方はこれから考えるとして、最終的にはこの形にしたいということです。2つの階層ページング構造があり、それぞれが同じ物理フレームを指した状態です。各 4KiB ページが読み込み専用 (writable=0) で物理フレームをマップしているのがポイントです。読み込み専用のページに書き込みを行おうとするとページフォルトが発生します。例外ハンドラの中で物理フレームをコピーし、階層ページング構造がコピー先の物理フレームを指すようにすることでコピーオンライトを実現します。

階層ページング構造の2つのうち1つは、次回以降に同じアプリを起動するときのひな型として再利用します。同じアプリを起動するとき、アプリのロード処理 (ELF のプログラムヘッダを見て LOAD セグメントを適切に配置する処理) を省略し、ひな型 (ロード処理済みの階層ページング構造) をコピーするだけで完了とするのです。

リスト 27.23 AppLoadInfo はアプリをロードした後の状態を表現する (terminal.hpp)

```
struct AppLoadInfo {
    uint64_t vaddr_end, entry;
    PageMapEntry* pml4;
};
```

ちょっと長くなりますが実装を始めましょう。まずアプリの実行ファイルをメモリにロードした状態を記憶するための AppLoadInfo 構造体を作りました (リスト 27.23)。この構造体の vaddr_end にはアプリの LOAD セグメントの最終アドレスを記録します。デマンドページングのアドレス範囲の開始点として使うためです。entry にはアプリのエントリポイントのアドレスを記録します。pml4 には階層ページング構造を指すポインタを記録します。

リスト 27.24 app_loads はロード済みアプリの一覧を保持する (terminal.cpp)

```
std::map<fat::DirectoryEntry*, AppLoadInfo>* app_loads;
```

2 回目以降のアプリ起動のために、初回の起動時に情報をメモしておく場所として `app_loads` を作りました (**リスト 27.24**)。このマップを使うと、アプリの実行ファイルを表すディレクトリエントリをキーとして与え、アプリのロード情報を得ることができます。ユーザーが起動しようとしたアプリがこのマップに既に登録されていれば、OS はロード処理を省略して (代わりに階層ページング構造のコピーを行って) アプリを起動します。`app_loads` の初期化はメイン関数の中で、ターミナルタスクを初期化する前にやっています。

リスト 27.25 アプリ起動処理を `LoadApp()` に委譲する (`terminal.cpp`)

```
Error Terminal::ExecuteFile(fat::DirectoryEntry& file_entry, char* command, char* first_arg)
{
    __asm__("cli");
    auto& task = task_manager->CurrentTask();
    __asm__("sti");

    auto [ app_load, err ] = LoadApp(file_entry, task);
    if (err) {
        return err;
    }

    LinearAddress4Level args_frame_addr{0xffff'ffff'ffff'f000};
```

リスト 27.25 に示すように、`Terminal::ExecuteFile()` でのアプリ起動処理を `LoadApp()` という関数に切り出しました。`LoadApp()` の実装は後で紹介しますが、簡単に説明すると、`app_loads` にアプリが登録されていればその情報を使い、登録されていなければ LOAD セグメントのロード処理からちゃんとやる関数です。

リスト 27.26 `LoadApp()` はアプリを `app_loads` から探し、なければロードする (`terminal.cpp`)

```
WithError<AppLoadInfo> LoadApp(fat::DirectoryEntry& file_entry, Task& task) {
    PageMapEntry* temp_pml4;
    if (auto [ pml4, err ] = SetupPML4(task); err) {
        return { {}, err };
    } else {
        temp_pml4 = pml4;
    }

    if (auto it = app_loads->find(&file_entry); it != app_loads->end()) {
        AppLoadInfo app_load = it->second;
        auto err = CopyPageMaps(temp_pml4, app_load.pml4, 4, 256);
        app_load.pml4 = temp_pml4;
        return { app_load, err };
    }

    std::vector<uint8_t> file_buf(file_entry.file_size);
    fat::LoadFile(&file_buf[0], file_buf.size(), file_entry);

    auto elf_header = reinterpret_cast<Elf64_Ehdr*>(&file_buf[0]);
    if (memcmp(elf_header->e_ident, "\x7f" "ELF", 4) != 0) {
        return { {}, MAKE_ERROR(Error::kInvalidFile) };
    }
}
```

```

auto [ last_addr, err_load ] = LoadELF(elf_header);
if (err_load) {
    return { {}, err_load };
}

AppLoadInfo app_load{last_addr, elf_header->e_entry, temp_pml4};
app_loads->insert(std::make_pair(&file_entry, app_load));

if (auto [ pml4, err ] = SetupPML4(task); err) {
    return { app_load, err };
} else {
    app_load.pml4 = pml4;
}

auto err = CopyPageMaps(app_load.pml4, temp_pml4, 4, 256);
return { app_load, err };
}

```

LoadApp() の実装をリスト 27.26 に紹介します。この関数の動作が理解できればこの節の内容を理解したも同然の、重要な関数です。先頭から処理を見ていきます。

SetupPML4() をまず実行します。この関数の動作を覚えていますか？ この関数は、新しく PML4 テーブルとなる 4KiB の物理フレームを生成し、その前半（添字 0 から 255）に OS 用の設定をコピーします。そして、CR3 レジスタに新しく生成した PML4 テーブルを設定します。この関数は新しく生成した PML4 テーブルを返しますので、それを temp_pml4 という変数にメモしておきます。この関数を実行した直後は仮想アドレスのアプリ領域 0xffff800000000000 以降には何も無い、まっさらな設定になっています。

次に app_loads にアプリが登録されているかを調べます。アプリが登録されていれば it != app_loads->end() が真となり、if 文の中身が実行されます。if 文本体の目的は、検索で見つけたアプリ情報が持つ階層ページング構造の設定を temp_pml4 の後半（添字 256 から 511、アプリ領域）にコピーし、それを関数の呼び出し側へ返すことです。if 文本体の処理を順に見ていきます。

まず、it が指すアプリ登録情報を app_load に取り出します。この時点では app_load.pml4 はひな型となる階層ページング構造を指した状態になっています（というか、そうなるようにこの後の処理を作ります）。

CopyPageMaps() によって、ひな型の後半（アプリ領域）を temp_pml4 にコピーします。このコピーにより、2 回目以降のアプリ起動であっても図 27.9 とまったく同じ状態を作り出します。CopyPageMaps() は階層ページング構造の浅いコピーを行う関数です。浅いコピー（シャローコピー）とは、PT が指す先の物理フレームのコピーをしない、階層ページング構造だけのコピーのことです。PML4、PDP、PD、PT については新しいテーブルを作成して値をコピーしますが、PT が指す物理フレームのコピーは行いません。

階層ページング構造のコピーが完了すると、temp_pml4 は前半も後半も設定された完全な状態になります。app_load.pml4 に temp_pml4 を代入することで、app_load.pml4 はひな型ではなく、新たにセットアップされた階層ページング構造を指した状態になります。app_load を返して if 文の処理は終了です。

さて、次は app_loads にアプリが登録されていなかったときの処理です。この部分は、コピーオンライト実装前のアプリのロード処理をほとんどそのままですから説明は不要でしょう。ファイルをバッファに読み込み、ELF ファイルであることを確かめた後、LoadELF() でロードするまでは今までとまっ

たく同じです。その次の処理からは今回新たに付け足しました。

ここで重要なのは、LoadELF() の実行前は temp_pml4 が指す階層ページング構造の後半（アプリ領域）が空だったのに、LoadELF() を実行することによって設定が書き込まれるということです。設定が書き込まれた状態の階層ページング構造を app_loads に登録しておいて、2 回目以降のアプリ起動時にひな型として利用するというのが、コピーオンライトの基本的なアイデアなのです。app_loads->insert() によって、この時点の階層ページング構造が app_loads に登録されます。

この時点では CR3 にはひな型となる階層ページング構造が登録された状態になっています。ひな型が変更されてしまってはまずいですから、SetupPML4() を実行し、新たな階層ページング構造を生成して CR3 を更新します。そして、新たに生成した階層ページング構造を app_load.pml4 に設定しておきます。この時点で app_load.pml4 が指す階層ページング構造の後半（アプリ領域）は空になっていますので、ひな型となる階層ページング構造の後半部分をコピーします。

さて、LoadELF() の処理ですが、本当に今までと同じではダメです。階層ページング構造の writable ビットを 0 にしなければならないからです。従来は writable=1 にしてアプリによる書き込みを許可していましたが、コピーオンライトを実現するには書き込みを禁止しておく必要があります。

リスト 27.27 LOAD セグメントのコピー先ページを読み込み専用とする (terminal.cpp)

```
WithError<uint64_t> CopyLoadSegments(Elf64_Ehdr* ehdr) {
    auto phdr = GetProgramHeader(ehdr);
    uint64_t last_addr = 0;
    for (int i = 0; i < ehdr->e_phnum; ++i) {
        if (phdr[i].p_type != PT_LOAD) continue;

        LinearAddress4Level dest_addr;
        dest_addr.value = phdr[i].p_vaddr;
        last_addr = std::max(last_addr, phdr[i].p_vaddr + phdr[i].p_memsz);
        const auto num_4kpages = (phdr[i].p_memsz + 4095) / 4096;

        // setup pagemaps as readonly (writable = false)
        if (auto err = SetupPageMaps(dest_addr, num_4kpages, false)) {
            return { last_addr, err };
        }

        const auto src = reinterpret_cast<uint8_t*>(ehdr) + phdr[i].p_offset;
        const auto dst = reinterpret_cast<uint8_t*>(phdr[i].p_vaddr);
        memcpy(dst, src, phdr[i].p_filesz);
        memset(dst + phdr[i].p_filesz, 0, phdr[i].p_memsz - phdr[i].p_filesz);
    }
    return { last_addr, MAKE_ERROR(Error::kSuccess) };
}
```

リスト 27.27 に CopyLoadSegments() の全体を示します。改造したのは 1 行だけ、SetupPageMaps() の第 3 引数に false を渡すようにしたことです。今までは 2 引数関数でしたが、今回の改造で writable ビットの値を受け取るようになります。SetupPageMaps() の改造はあとで紹介します。

SetupPageMaps() で LOAD セグメントをコピーする先の 4KiB ページを読み込み専用で設定してしまうと、その後 memcpy() で LOAD セグメントをコピーする時点でエラーになってしまう気がしますね？ というか、エラーにならなかったら、それは「読み込み専用」じゃないですよ。実は、

CPU の仕様で、スーパーバイザーモード (CPL<3) で動いているプログラムは writable ビットの設定に関わらずページに書き込むことができます。ただし、CR0 の WP ビットが 0 であれば。

リスト 27.28 スーパーバイザーによる読み込み専用ページへの書き込みを許可する (paging.cpp)

```
ResetCR3();
SetCR0(GetCR0() & 0xffffefff); // Clear WP
```

ということで WP ビットを 0 にする処理を SetupIdentityPageTable() に追加しておきました(リスト 27.28)。

リスト 27.29 R/W ビットに書く値を指定するように改造した (paging.cpp)

```
WithError<size_t> SetupPageMap(
    PageMapEntry* page_map, int page_map_level, LinearAddress4Level addr,
    size_t num_4kpages, bool writable) {
    while (num_4kpages > 0) {
        const auto entry_index = addr.Part(page_map_level);

        auto [ child_map, err ] = SetNewPageMapIfNotPresent(page_map[entry_index]);
        if (err) {
            return { num_4kpages, err };
        }
        page_map[entry_index].bits.user = 1;

        if (page_map_level == 1) {
            page_map[entry_index].bits.writable = writable;
            --num_4kpages;
        } else {
            page_map[entry_index].bits.writable = true;
            auto [ num_remain_pages, err ] =
                SetupPageMap(child_map, page_map_level - 1, addr, num_4kpages, writable);

```

リスト 27.29 に SetupPageMaps() の処理の大部分を受け持つ SetupPageMap() の改造部分を示します。受け取った writable ビットの設定値を page_map[entry_index].bits.writable に設定するのが主な変更点です。

ただし、writable ビットを引数に応じて変えるのは階層ページング構造の最下層 PT のみです。page_map_level が 1 でない、つまり PML4、PDP、PD ではこれまでと変わらず writable=1 と設定します。ページング機構の仕様で、いずれかの階層で writable=0 となっていれば読み込み専用になりますから、MikanOS では最下層でのみ writable ビットの 0/1 を切り替えることにしました。これで、コピーオンライト発生時に writable を切り替える処理が大分単純になるはずです。

リスト 27.30 ページフォルトが発生したらコピーオンライト処理を実行 (paging.cpp)

```
Error HandlePageFault(uint64_t error_code, uint64_t causal_addr) {
    auto& task = task_manager->CurrentTask();
    const bool present = (error_code >> 0) & 1;
    const bool rw      = (error_code >> 1) & 1;
    const bool user    = (error_code >> 2) & 1;
    if (present && rw && user) {

```

```

    return CopyOnePage(causal_addr);
} else if (present) {
    return MAKE_ERROR(Error::kAlreadyAllocated);
}

if (task.DPagingBegin() <= causal_addr && causal_addr < task.DPagingEnd()) {

```

ページフォルトが発生したときに呼ばれる `HandlePageFault()` の改造部分を **リスト 27.30** に示します。大きな変更点は、エラーコードの各ビットに名前を付けたことと、コピーオンライトを実現するために `CopyOnePage()` を呼び出すようにしたことです。

今まではエラーコードの `present` ビットが 1 のときは必ずエラーになっていました。なぜなら、既に存在するページへのアクセスでページフォルトが起きることを想定していなかったからです。でも今は読み込み専用ページがあります。ページ自体は存在するが、読み込み専用だから書き込み命令が失敗した、ということが起きるのですね。これは、まさにコピーオンライトの本質的な動作です。if (`present && rw && user`) は、ページが存在し (`present=1`)、ページフォルトの原因はユーザーモードの (`user=1`) 書き込み命令である (`rw=1`)、という条件を判定しています。

リスト 27.31 `CopyOnePage()` は 4KiB ページをコピーして書き込み可でマップする (paging.cpp)

```

Error CopyOnePage(uint64_t causal_addr) {
    auto [ p, err ] = NewPageMap();
    if (err) {
        return err;
    }
    const auto aligned_addr = causal_addr & 0xffff'ffff'ffff'f000;
    memcpy(p, reinterpret_cast<const void*>(aligned_addr), 4096);
    return SetPageContent(reinterpret_cast<PageMapEntry*>(GetCR3()), 4,
                          LinearAddress4Level{causal_addr}, p);
}

```

リスト 27.31 に、コピーオンライトの最も核となる関数 `CopyOnePage()` の実装を示します。この関数は **図 27.8** のようなコピーオンライトの動作を実現します。ページフォルトの原因となったアドレス (`causal_addr`) を含む 4KiB ページがマップしている物理フレームの内容をコピーし、書き込み可としてマップしなおします。`SetPageContent()` がページテーブルを書き換えて指定した物理フレームをマップし直す処理を担当します。

リスト 27.32 `SetPageContent()` は物理フレームを書き込み可でマップする (paging.cpp)

```

Error SetPageContent(PageMapEntry* table, int part,
                    LinearAddress4Level addr, PageMapEntry* content) {
    if (part == 1) {
        const auto i = addr.Part(part);
        table[i].SetPointer(content);
        table[i].bits.writable = 1;
        InvalidateTLB(addr.value);
        return MAKE_ERROR(Error::kSuccess);
    }

    const auto i = addr.Part(part);

```

```
return SetPageContent(table[i].Pointer(), part - 1, addr, content);
}
```

SetPageContent() の実装を **リスト 27.32** に示します。この関数は、与えられた仮想アドレス addr を含む 4KiB ページが、content で指定された物理フレームを指すように設定を更新します。addr にしたがって階層ページング構造を降りていき、最下層のページテーブルにたどり着いたら該当する 4KiB ページの設定を更新します。その際、writable を 1 に変更します。この変更を忘れると、ページフォルトの例外ハンドラを抜けてアプリに戻った途端、また書き込み命令によってページフォルトが発生してしまいます。

リスト 27.33 指定されたアドレスを含む TLB の行を無効化する (asmfunc.asm)

```
global InvalidateTLB ; void InvalidateTLB(uint64_t addr);
InvalidateTLB:
    invlpg [rdi]
    ret
```

最後に呼び出している InvalidateTLB() (**リスト 27.33**) は重要です。この関数は TLB の中の、設定変更したページに関する部分だけ無効化 (invalidate) します。invlpg がそのための命令で、これは C++ では書けないのでアセンブリ言語の関数にしています。

TLB (Translation Lookaside Buffer) は仮想アドレスから物理アドレスへと変換する処理を高速化するために CPU に内蔵されている装置です。ページングによるアドレス変換は真面目にやると非常に重たい処理です。例えば、mov 命令を 1 回実行するたびに階層ページング構造をたどるために 4 回のメモリアクセスが発生します。つまりメモリアクセスの回数が 5 倍になります。相当に性能劣化しそうですねーということが分かるかと思います。

これを防ぐためにアドレス変換の結果を TLB に記録しておき、再利用します。TLB はメモリではなく CPU に内蔵された機能ですので、高速に動作します。プログラムがメモリアクセスをしようとしたとき、その仮想アドレスが TLB に登録されていれば、階層ページング構造を調べることなく物理アドレスが得られます。TLB に記録しておけるアドレス変換の数はかなり少ないのですが、通常、プログラムがアクセスするメモリの範囲は（時間を区切ると）狭い範囲で収まるため十分な高速化が望めます。

階層ページング構造はメモリ上のデータ構造ですから、その一部を書き換えても CPU は気づけません。もし階層ページング構造の一部を変更した後に invlpg 命令を実行しなければ、CPU は古くなった TLB を使い続け、本来読み書きすべき物理アドレスとは別の物理アドレスを読み書きしてしまうでしょう*³。invlpg は引数で指定された仮想アドレスを含む TLB の行を無効化します。すると、次にその仮想アドレスに対してメモリアクセスが行われた際に、階層ページング構造をたどる処理が実行され、TLB の情報が最新になります。

* 3 筆者は当初、この処理を入れ忘れていました。QEMU ではうまく動くのに、特定の実機では動かない問題に悩まされ、最終的に invlpg で解決しました。

リスト 27.34 アプリが使っていたページを解放する (paging.cpp)

```

Error CleanPageMap(
    PageMapEntry* page_map, int page_map_level, LinearAddress4Level addr) {
    for (int i = addr.Part(page_map_level); i < 512; ++i) {
        auto entry = page_map[i];

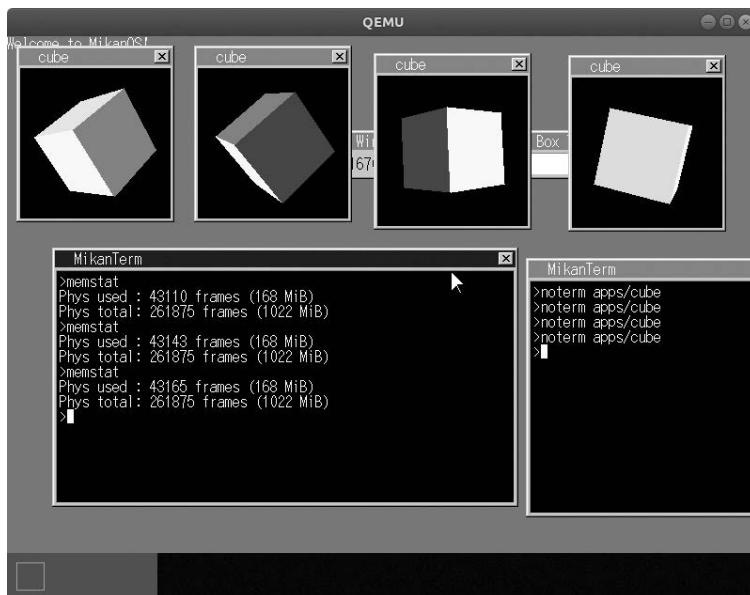
        .....

        if (entry.bits.writable) {
            const auto entry_addr = reinterpret_cast<uintptr_t>(entry.Pointer());
            const FrameID map_frame{entry_addr / kBytesPerFrame};
            if (auto err = memory_manager->Free(map_frame, 1)) {
                return err;
            }
        }
        page_map[i].data = 0;
    }
}

```

最後にアプリ終了後にメモリを解放する CleanPageMap() を修正しておきます (リスト 27.34)。コピーオンライトの処理を実装したため、アプリの機械語 (.text) や読み込み専用データ (.rodata) が含まれる LOAD セグメントは、アプリ終了時点でコピーされずに残っているはずです。CleanPageMap() は今まで、問答無用で 4KiB ページが参照する物理フレームを削除する実装になっていましたが、これからはコピーされていないページについては物理フレームを解放してはいけません。ということで、コピーされたページは必ず writable=1 になるという特徴を利用して、writable=1 のときだけ物理フレームを解放するという処理に変えました。

図 27.10 cube アプリ 1 つあたりのメモリ消費が少なくなっている



説明はこのくらいにして実際に動作させてみます (図 27.10)。前の節で測ったときは 1 つの cube につき 16 フレームを消費していましたが、今は 11 フレームの消費で済むようになりました。cube は約 70KB 程度のアプリですが、もっと大きな実行ファイルならコピーされずに済むページが増え、さらに高い効果を発揮することでしょう。

コピーオンライトは本書で紹介することをちょっと迷うくらい高度な機能なのですが、割とコンパクトに実装できて筆者は感動しています。次章も頑張りましょう！

第 28 章

日本語表示と リダイレクト

英数字はプログラムでの扱いがとても簡単ですから、今まではずっと英数字だけを表示してきました。でも Windows や macOS、Linux などには日本語で文章を読み書きしたりメニューが日本語だったりするわけです。そろそろ日本語表示に挑戦してもよい時期かと思えますから、この章の前半では MikanOS に日本語表示機能を付けます。この章の後半ではコマンドの出力をファイルに保存するためのリダイレクト機能を開発します。今まで、コマンドが出力した文字列はターミナルの画面上に表示されるだけでしたが、リダイレクト機能を使うとファイルに書き出せるようになります。コマンド間でファイルを経由したデータ連携ができるようになります、コマンドの応用範囲が広がります。

28.1 日本語と文字コード (osbook_day28a)

私たちはこれまで ASCII という文字コードを使ってきました。ASCII は計算機の世界で広く普及している文字コードで、英数字と記号類を扱えます。ASCII は 7 ビット、すなわち 0 から 127 の範囲を使う文字コードですので、char 型の変数 1 個で 1 文字を表現できます。128 種類の値があれば、いくつかの制御コード (NUL 文字や EOT、改行、空白など) と英数字、いくつかの記号を表現できます。

一方、世界には英数字以外にたくさんの文字があります。英語以外の言葉を使う国では、これらの文字をコンピュータで扱うためにそれぞれの言語専用の文字コードがたくさん開発されてきました。日本語用では JIS、Shift-JIS、EUC-JP といった文字コードが代表的です。同じ日本語用文字コードであっても、文字と数値の対応関係も違えば、数値をバイト列に直すルール (符号化方式) も異なります。文字コードは「鍵」のようなもので、鍵がなければバイト列を正しい文字に変換できず、読むことができません。

インターネットが発達する前はそれでもそんなに問題はありませんでした。インターネットがない時代は、コンピュータは 1 人、あるいは 1 つのグループが使うものであり、通信するとしても国境を超えることはほとんどありません。ですから、データをやり取りする相手はみんな同じ言語話者です。みんな 1 つの文字コードを使うことを約束しておけば、何も苦労はありませんね。

しかし現代では様々な言語のデータを日常的にやり取りします。英語のウェブサイトを見る人は多いと思いますし、中にはロシア語や中国語などで情報収集する人もいるかもしれません。1 つのウェブサービスに複数の言語が交じることも多いです。SNS には世界中の人が登録していて、日々様々な言語で情報書き込まれます。

このように、様々な言語の文章が入り交じる世界で、各言語を専用の文字コードを使って書くのはとても複雑な処理が必要になります。1 つの方法に、言語を切り替えるマークを配置する方法があります。はい！ ここからは日本語モードです。Shift-JIS として解釈してくださいね。はい！ ここからはロシア語です。KOI8-R として読んでください。はい！ ここからは ASCII です。というように、文字コードを変えるマークが来るたびに解釈する文字コードを切り替えます。うん、考えただけで頭が痛いです。実は JIS コードはこの方法で英語モードと日本語モードを切り替えて英数字、ひらがなやカタカナ、漢字が混じった文章を表現する方式を採用しています。だから、絶対にこの方法がダメ、というわけではありませんが、面倒な方法には違いありません。

Unicode という文字コードがあります。これは全世界の文字を 1 つのコード体系で網羅してしまう、という野心的な文字コードです。英語も日本語もロシア語も中国語も、ぜんぶひっくるめて扱います。文書を Unicode で作成すれば、特殊なマークを挟むことなく、ごく自然に言語が入り混じった文書を作成することができます。SNS でいろんな言語の情報で書き込みがあっても、裏側では Unicode として統一的に扱えるようになります。1 つの文字コードであらゆる言語が表現できるのは、インターネット時代にはとても重要な特徴です。Unicode は現在広く普及していますから MikanOS でも Unicode を用いて日本語対応をしようと思います。

文字コードと言ったときには、文字と数値の対応付けと、数値とビット列の変換方法の 2 つの意味があります。単に Unicode と書くと前者 (文字と数値の対応付け) を意味することが多いと思います。例えば Unicode では「♪」は 0x266a に対応しています。文字に対応付けられた数値を**コードポイント**と呼び、Unicode の世界では U+ を付けて「U+266A」のように書きます。

Unicode で表現された文字列をメモリに書き込んだり、ネットワークで送信したりするには、コードポイントをビット列に変換しなければなりません。コードポイントをビット列に変換するやり方を**符号化方式**と言い、Unicode ではいくつかの符号化方式が定義されています。代表的なものに UTF-8、UTF-16、UTF-32 があります。

最も単純なのが UTF-32 です。これは、コードポイントの数値をそのまま 32 ビットで表すものです。1 文字を 32 ビットで表すのでコードポイントからビット列の変換と、ビット列からコードポイントへの変換がとても簡単です。例えば「A」という文字はコードポイント U+0041 ですから、UTF-32 で符号化すると 32 ビット幅のビット列 0x00000041 となります。変換が分かりやすい一方、明らかな欠点があります。それは、たとえ ASCII コードの範囲内の文字だけで書かれた文章でも、ASCII コードで符号化した場合に比べ、4 倍の大きさになってしまうことです。それから、当然ながら ASCII コードだけに対応しているプログラムでは UTF-32 で符号化されたデータを扱えません。

UTF-16 はコードポイントの数値の下位 16 ビットをそのまま表すものです。UTF-32 よりはマシですが、それでも ASCII コードで符号化した場合に比べ、サイズが 2 倍になってしまいます。また、サロゲートペアと呼ばれる特殊な仕様もあり、意外にも扱いづらい符号化方式です。

UTF-8 はコードポイントを 1 から 4 バイトの可変長で表す符号化方式です。UTF-8 の特徴は何といっても、ASCII コードの範囲 (U+0000 から U+007F) を ASCII コードとまったく同じように符号化することです。例えば「A」という文字は ASCII では 7 ビットの 65 (0b1000001)、UTF-8 では 8 ビットの 65 (0b01000001) で表します。現代の記憶装置のほとんどが 8 ビット=1 バイトを最小単位としていて、ASCII コードの文書もほとんどの場合に 8 ビットで 1 文字を記録しますから、ASCII と UTF-8 で完全に同じバイト列となります。この特徴により、英文であれば ASCII コードと同じサイズになりますし、ASCII コードにしか対応していないプログラムでも扱えます。

また、UTF-8 は UTF-32 や UTF-16 で発生するエンディアンの違いによる問題が起きません。UTF-32 や UTF-16 では、符号化結果のビット列をバイト列に直す順番 (エンディアン) の違いに影響を受けます。例えば 0x00000041 をファイルに書くときに 00 00 00 41 と書くか 41 00 00 00 と書くかが仕様で決まっておらず、ファイルに書いたときと異なるエンディアンで読んでしまうと不正な結果となります。その点、UTF-8 はエンディアンの問題がありません。UTF-8 で 2 バイト以上になるような文字 (ASCII の範囲外の文字) の場合でも、仕様でバイトの記録順が決まっているために混乱が起きないのです。

現状では、筆者が見る限り、テキストファイルの符号化形式として UTF-8 が世界の事実上の標準となっています。OS を含め、ソフトウェアの内部的な表現は UTF-8 ではないかもしれませんが、ファイルやネットワークでやり取りする際の符号化形式としては UTF-8 が強いです。多くのテキストエディタが UTF-8 で文書を読み書きします。そこで、MikanOS でも UTF-8 を基本に日本語機能の実装をしていくことにします。

この節では `Terminal::Print()` と `cat` コマンドを UTF-8 に対応させることを目標にします。これができると、日本語などの ASCII コード範囲外の文字を含む UTF-8 文字列の各文字を正しく認識して、Unicode のコードポイントを計算できるようになります。ただ、コードポイントを計算できても実際に文字列を画面に表示できるわけではありません。そのコードポイントに対応するフォントがあって初めてその文字を画面に表示できます。ということで、次の節では日本語フォントへ対応する予定です。

リスト 28.1 `WriteUnicode()` は与えられたコードポイントに対応する文字を描画する (font.cpp)

```
void WriteUnicode(PixelWriter& writer, Vector2D<int> pos,
                  char32_t c, const PixelColor& color) {
    if (c <= 0x7f) {
        WriteAscii(writer, pos, c, color);
        return;
    }

    WriteAscii(writer, pos, '?', color);
}
```

```
WriteAscii(writer, pos + Vector2D<int>{8, 0}, '?', color);
}
```

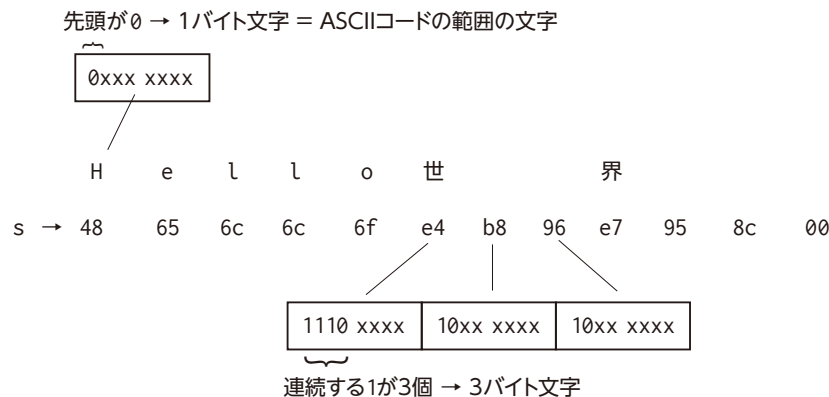
リスト 28.1 に今回の主役となる `WriteUnicode()` を示します。この関数は指定されたコードポイントの文字を指定された位置に描画します。あるいは、UTF-32 で符号化された値を受け取る関数、とも言えます (Unicode コードポイントと UTF-32 で表した値は等しいのでしたね)。今のところ ASCII コード以外に対応したフォントがなく、コードポイントが 0x7f 以下の ASCII コードの範囲内にある文字だけしか表示できません。0x80 以上のコードポイントが与えられたら、仕方がないので ?? と表示することにしておきます。

リスト 28.2 `WriteString()` を Unicode に対応させる (font.cpp)

```
void WriteString(PixelWriter& writer, Vector2D<int> pos, const char* s, const PixelColor& color) {
    int x = 0;
    while (*s) {
        const auto [u32, bytes] = ConvertUTF8To32(s);
        WriteUnicode(writer, pos + Vector2D<int>{8 * x, 0}, u32, color);
        s += bytes;
        x += IsHankaku(u32) ? 1 : 2;
    }
}
```

`WriteString()` を書き換え、新しく作った `WriteUnicode()` を使うようにしました (**リスト 28.2**)。この関数は、与えられた文字列 `s` を UTF-8 で符号化された文字列だと思って先頭から読み、Unicode 文字として 1 文字ずつ表示します。このプログラムを理解するには UTF-8 の構造を知る必要があります。

図 28.1 UTF-8 で符号化された文字列の例



「Hello 世界」という文字列を UTF-8 で符号化したものを図 28.1 に示します。「Hello 世界」は UTF-8 で符号化すると全体で 11 バイトのデータとなります。先頭 5 文字は ASCII コードにある文字ですので 1 文字を 1 バイトで表せます。H=0x48、e=0x65、l=0x6c、o=0x6f です。

「世」と「界」は 1 バイトでは表せない文字で、複数のバイトを組み合わせで 1 文字を表します。UTF-8 では先頭 1 バイトを見ると文字のバイト数が分かります。先頭 1 バイトの最上位ビットから見て、連続する 1 の個数が文字のバイト数を表すのです。「世」の先頭バイトは 0xe4=0b11100100 ですから、最上位ビットから連続する 1 の個数は 3 個ですから「世」は 3 バイト文字ということになります。同様に「界」も 3 バイト文字ですね。2 バイト目以降は、先頭 2 ビットが必ず 0b10 となることが決まっています。

リスト 28.3 CountUTF8Size() は UTF-8 文字のバイト数を求める (font.cpp)

```
int CountUTF8Size(uint8_t c) {
    if (c < 0x80) {
        return 1;
    } else if (0xc0 <= c && c < 0xe0) {
        return 2;
    } else if (0xe0 <= c && c < 0xf0) {
        return 3;
    } else if (0xf0 <= c && c < 0xf8) {
        return 4;
    }
    return 0;
}
```

リスト 28.3 に、UTF-8 文字の先頭バイトから文字のバイト数を得る関数 CountUTF8Size() を示します。ASCII コードの範囲内 (0 ~ 0x7f) であれば 1 バイト文字、連続する 1 が 2 個 (0xc0 ~ 0xdf) なら 2 バイト文字、というようにバイト数を計算します。

リスト 28.4 ConvertUTF8To32() は UTF-8 文字列から 1 文字を取り出す (font.cpp)

```
std::pair<char32_t, int> ConvertUTF8To32(const char* u8) {
    switch (CountUTF8Size(u8[0])) {
        case 1:
            return {
                static_cast<char32_t>(u8[0]),
                1
            };
        case 2:
            return {
                (static_cast<char32_t>(u8[0]) & 0b0001'1111) << 6 |
                (static_cast<char32_t>(u8[1]) & 0b0011'1111) << 0,
                2
            };
        case 3:
            return {
                (static_cast<char32_t>(u8[0]) & 0b0000'1111) << 12 |
                (static_cast<char32_t>(u8[1]) & 0b0011'1111) << 6 |
                (static_cast<char32_t>(u8[2]) & 0b0011'1111) << 0,
                3
            };
        case 4:
            return {
                (static_cast<char32_t>(u8[0]) & 0b0000'0111) << 18 |
                (static_cast<char32_t>(u8[1]) & 0b0011'1111) << 12 |
```

```

        (static_cast<char32_t>(u8[2]) & 0b0011'1111) << 6 |
        (static_cast<char32_t>(u8[3]) & 0b0011'1111) << 0,
        4
    };
    default:
        return { 0, 0 };
    }
}

```

リスト 28.4 に、UTF-8 文字列の先頭から 1 文字を取り出し、その文字のコードポイントを返す関数 `ConvertUTF8To32()` を示します。先頭バイトから文字のバイト数を求め、その数に応じてバイト列を読み進めます。UTF-8 文字からコードポイントに変換するには、**図 28.1** で「x」で示したビットだけを抜き出して結合します。

リスト 28.5 cat を UTF-8 に対応させる (terminal.cpp)

```

    } else {
        fat::FileDescriptor fd{*file_entry};
        char u8buf[4];

        DrawCursor(false);
        while (true) {
            if (fd.Read(&u8buf[0], 1) != 1) {
                break;
            }
            const int u8_remain = CountUTF8Size(u8buf[0]) - 1;
            if (u8_remain > 0 && fd.Read(&u8buf[1], u8_remain) != u8_remain) {
                break;
            }

            const auto [ u32, u8_next ] = ConvertUTF8To32(u8buf);
            Print(u32 ? u32 : U'□ ');
        }
        DrawCursor(true);
    }
}

```

`cat` コマンドのプログラムの中で、実際にファイルからデータを読み込んで表示する処理を UTF-8 に対応させました (**リスト 28.5**)。while 文に注目してください。この while 文では、1 文字に相当するバイト列を `u8buf` に読み込み(最大 4 バイト)、それを `ConvertUTF8To32()` によって Unicode コードポイントに変換し、最後に `Print()` で表示しています。もしうまくコードポイントに変換できなければ `u32` は 0 となります。そのときは「□」を代わりに表示しておきます。

リスト 28.6 Print() を Unicode に対応させる (terminal.cpp)

```

void Terminal::Print(char32_t c) {
    if (!show_window_) {
        return;
    }

    auto newline = [this]() {

```

```

    cursor_.x = 0;
    if (cursor_.y < kRows - 1) {
        ++cursor_.y;
    } else {
        Scroll1();
    }
};

if (c == U'\n') {
    newline();
} else if (IsHankaku(c)) {
    if (cursor_.x == kColumns) {
        newline();
    }
    WriteUnicode(*window_>Writer(), CalcCursorPos(), c, {255, 255, 255});
    ++cursor_.x;
} else {
    if (cursor_.x >= kColumns - 1) {
        newline();
    }
    WriteUnicode(*window_>Writer(), CalcCursorPos(), c, {255, 255, 255});
    cursor_.x += 2;
}
}
}

```

cat の中で使っている `Terminal::Print()` の実装を **リスト 28.6** に示します。今までは `char` 型の引数でしたが、今回の改造で UTF-32 文字を表す型 `char32_t` を受け取るようになりました。`char32_t` は C++11 で導入された型ですから、古い C++ の教科書には載っていないかもしれません。UTF-16 の 1 文字を表す `char16_t` 型もあります。

`U'\n'` というのは、UTF-32 で符号化された改行文字を表します。一般に、`U` を付けない場合は ASCII コード^{*1} 文字リテラルとなり、`U`、`u` を付けるとそれぞれ UTF-32、UTF-16 の文字リテラルとなります。

`IsHankaku()` は新しく作った関数で、指定した文字が半角文字か全角文字かを判定します。実装を **リスト 28.7** に示します。

リスト 28.7 `IsHankaku()` は引数が半角文字であれば真を返す (font.cpp)

```

bool IsHankaku(char32_t c) {
    return c <= 0x7f;
}

```

英数字のように横幅が縦の半分になる文字を半角、日本語などの横と縦が同じ大きさの文字を全角と呼びます。細かく見ればそれぞれの文字の幅は異なりますが(「I」より「M」の方が広いし、「日」より「旧」の方が広い)、処理を簡単にするために半角は 8×16 、全角は 16×16 ピクセルの大きさだとして処理します。このように、文字の幅を同じだとみなすことを「等幅」(とうはば)と言います。対して、文字の幅をちゃんと考慮して処理することは「プロポーショナル」です。MikanOS ではプロポーシ

*1 規格では処理系定義の文字コードが使用されることになっていますが、筆者は ASCII 以外を使う処理系に実際に出会ったことはありません。

ナルは扱いません。

話を `Print()` に戻します。 `IsHankaku()` で与えられた文字が半角か全角かが分かったら、それぞれに応じて改行する位置とカーソルの移動幅を変えます。半角ならターミナル画面の右端ぎりぎりまで表示できますが、全角なら半角 1 文字分だけ手前で改行しなければなりません。カーソルの移動幅は半角なら 1、全角なら 2 とします。ちなみに、文字が半角か全角かというのと、その文字が ASCII コードの範囲内かどうかとは本来は関係ありません。そのため、文字が半角でも全角でも `WriteUnicode()` を使って描画します。

さて、以上で Unicode 対応の改造は完了です。といっても、まだ日本語フォントが無いため、日本語を表示しようとしても「??」になってしまうはずですけどね。それを確かめるために日本語ファイルを用意して実行してみたいと思います。 `resource/jpn.txt` というファイルを作りました (**リスト 28.8**)。

リスト 28.8 日本語表示のテストに使う文書ファイル (jpn.txt)

ようこそ、MikanOS へ！
MikanOS は計算機科学の教材となることを目指して作られています。

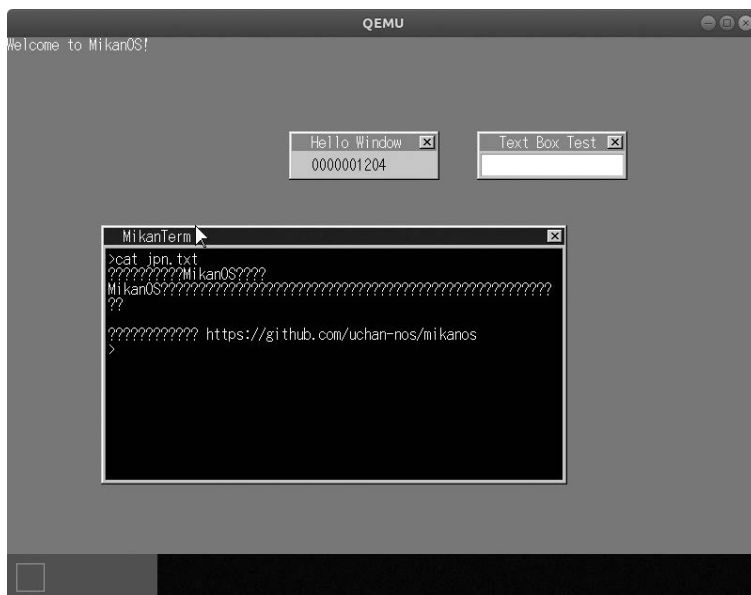
リポジトリ → <https://github.com/uchan-nos/mikanos>

このファイルを MikanOS の起動ボリュームに入れなければなりません。任意のファイルを手軽に起動ボリュームに組み込めるよう、MikanOS 起動スクリプト `$HOME/osbook/devenv/run_mikanos.sh` にある `RESOURCE_DIR` の仕組みを利用します。この仕組みは、`RESOURCE_DIR` で指定したディレクトリにあるファイルを起動ボリュームのルートディレクトリにコピーしてくれます。

```
$ APPS_DIR=apps RESOURCE_DIR=resource ./build.sh run
```

というコマンドラインで QEMU を起動してやってみます。**図 28.2** は `cat jpn.txt` を実行した様子です。日本語の部分だけ「??」になり、ASCII コードはちゃんと表示されているようです。これは期待通りです。次の節で日本語フォントを組み込めば、この「??」もちゃんと表示されるようになるはずです。楽しみです！

図 28.2 日本語を含む UTF-8 文書を表示した様子



28.2 日本語フォント (osbook_day28b)

ASCII コードの描画には OS 本体に組み込んだフォントデータが使われます。東雲フォントを元にしたこのフォントデータは、8 × 16 ピクセルの単純なビット列として表されていて、プログラムから `uint8_t` の配列として見えるようになっています。

日本語を含んだビットマップフォントを探すのは大変です。まったくないわけではありませんが、ファイル形式が特殊だったり Unicode に対応していなかったり等の問題があります。選択肢が少ないので、自分の好きなフォントを使いたいと思っても難しいです。

そこで、この節では FreeType というライブラリを MikanOS に組み込んで、TrueType というファイル形式のフォントを使えるようにします。フォントと言えば TrueType、というくらい普及しているファイル形式ですから、これを扱えるようになれば皆さんが好きなフォントを組み込んで使えるようになります。この節からは、IPA フォント^{*2}のシリーズから取ってきた「IPA ゴシック」を `resource/nihongo.ttf` として追加しました。このフォントファイルはライセンスに従う限り再配布が可能なファイルですので、MikanOS に含めて配布しています。ライセンス全文は `IPA_Font_License_Agreement_v1.0.txt` に載っています。

IPA ゴシックのファイルは約 6MiB 程度ととても大きいです。そのため、今までブートローダで読み込んでいたボリュームサイズ (16MiB) では足りなくなりそうです。そこでリスト 28.9 のように、2 倍の 32MiB に増やしておきました。これで当面、OS 本体が大きくなったりアプリが増えたりしても耐えられるでしょう。忘れずに `$HOME/edk2` ディレクトリで build しておいてくださいね。

*2 <https://moji.or.jp/ipafont/ipa00303/>

リスト 28.9 ブートローダで読み込むサイズを増やす (MikanLoaderPkg/Main.c)

```
EFI_BLOCK_IO_MEDIA* media = block_io->Media;
UINTN volume_bytes = (UINTN)media->BlockSize * (media->LastBlock + 1);
if (volume_bytes > 32 * 1024 * 1024) {
    volume_bytes = 32 * 1024 * 1024;
}
```

FreeType ライブラリの使い方は大雑把に次の流れになっています。

1. ライブラリを初期化する (FT_Init_FreeType())
2. フォントファイルを読み込む
3. フェースオブジェクトを作る (FT_New_Memory_Face())
4. 目的の文字を読み込む (FT_Load_Glyph())

この節の主目標は WriteUnicode() を日本語に対応させることです。前の節で作ったこの関数は、ASCII コードの範囲外の文字が来たら「??」と表示してお茶を濁していました。これを改造し、FreeType ライブラリを利用して正しく文字を描画するということです。まずは FreeType ライブラリを初期化する処理から作っていきましょう。

リスト 28.10 FreeType ライブラリを初期化する (font.cpp)

```
void InitializeFont() {
    if (int err = FT_Init_FreeType(&ft_library)) {
        exit(1);
    }

    auto [ entry, pos_slash ] = fat::FindFile("/nihongo.ttf");
    if (entry == nullptr || pos_slash) {
        exit(1);
    }

    const size_t size = entry->file_size;
    nihongo_buf = new std::vector<uint8_t>(size);
    if (LoadFile(nihongo_buf->data(), size, *entry) != size) {
        delete nihongo_buf;
        exit(1);
    }
}
```

リスト 28.10 に MikanOS のフォント関連の初期化を担当する InitializeFont() の実装を示します。先頭から処理を見ていくと、まず FreeType ライブラリを初期化し、次に nihongo.ttf ファイルを開き、最後にファイルを nihongo_buf に読み込んでいます。ここで登場した2つの変数は**リスト 28.11** のように定義した変数です。font.cpp の中からしか使いませんので無名名前空間の中に定義しました。

リスト 28.11 日本語の描画に必要な 2 つの変数 (font.cpp)

```
FT_Library ft_library;
std::vector<uint8_t*> nihongo_buf;
```

ソースコードは示しませんが、定義した `InitializeFont()` はもちろんメイン関数の中で呼び出すようにしましたよ。日本語表示はなるべく早い段階で初期化した方がいいかなと思い、`fat::Initialize(volume_image);` の直後で `InitializeFont()` を呼び出すようにしてみました。

リスト 28.12 NewFTFace() はフェースオブジェクトを準備する (font.cpp)

```
WithError<FT_Face> NewFTFace() {
    FT_Face face;
    if (int err = FT_New_Memory_Face(
        ft_library, nihongo_buf->data(), nihongo_buf->size(), 0, &face)) {
        return { face, MAKE_ERROR(Error::kFreeTypeError) };
    }
    if (int err = FT_Set_Pixel_Sizes(face, 16, 16)) {
        return { face, MAKE_ERROR(Error::kFreeTypeError) };
    }
    return { face, MAKE_ERROR(Error::kSuccess) };
}
```

フォントの世界で**フェース**とは字形のデザインのことです。フォントファイルには複数のデザインが含まれていることがあり、それぞれをフェースと呼びます。**リスト 28.12** に示す `NewFTFace()` は、フォントファイルから先頭のフェース（1 つしかフェースが含まれないならそれ）を読み込みます。FreeType ライブラリの関数 `FT_New_Memory_Face()` を使うと、メモリ上に読み込まれたフォントファイルからフェースを持つてくることができます。

通常、TrueType フォントに収められている字形は拡大、縮小が自由にできるタイプ（ベクターフォント）ですので、実際に描画する前に目的のサイズを決める必要があります。それをやるのが `FT_Set_Pixel_Sizes()` です。ここでは 16×16 ピクセルで描画する設定にしておきます。

リスト 28.13 指定した文字の字形を読み込む関数 (font.cpp)

```
Error RenderUnicode(char32_t c, FT_Face face) {
    const auto glyph_index = FT_Get_Char_Index(face, c);
    if (glyph_index == 0) {
        return MAKE_ERROR(Error::kFreeTypeError);
    }

    if (int err = FT_Load_Glyph(face, glyph_index,
        FT_LOAD_RENDER | FT_LOAD_TARGET_MONO)) {
        return MAKE_ERROR(Error::kFreeTypeError);
    }
    return MAKE_ERROR(Error::kSuccess);
}
```

手順の4番目「目的の文字を読み込む」ための関数 `RenderUnicode()` をリスト 28.13 に示します。読み込みたい文字の Unicode コードポイントと `NewFTFace()` で作ったフェースオブジェクトを渡すと、その文字の字形を読み込みます。

フォントの分野では字形のことをグリフと呼びます。`FT_Get_Char_Index()` は Unicode コードポイントからフェースオブジェクト内のグリフ番号を得る関数です。フォントファイルの中でグリフが Unicode コードポイントの順に並んでいるとは限らないため、この関数で番号を変換する必要があるのです。

得られたグリフ番号を `FT_Load_Glyph()` に渡すとそのグリフを取得できます。グリフは `face->glyph->bitmap` に取得されます。

リスト 28.14 指定された文字に対応するグリフを描画する (font.cpp)

```
Error WriteUnicode(PixelWriter& writer, Vector2D<int> pos,
                  char32_t c, const PixelColor& color) {
    if (c <= 0x7f) {
        WriteAscii(writer, pos, c, color);
        return MAKE_ERROR(Error::kSuccess);
    }

    auto [ face, err ] = NewFTFace();
    if (err) {
        WriteAscii(writer, pos, '?', color);
        WriteAscii(writer, pos + Vector2D<int>{8, 0}, '?', color);
        return err;
    }
    if (auto err = RenderUnicode(c, face)) {
        FT_Done_Face(face);
        WriteAscii(writer, pos, '?', color);
        WriteAscii(writer, pos + Vector2D<int>{8, 0}, '?', color);
        return err;
    }
    FT_Bitmap& bitmap = face->glyph->bitmap;

    const int baseline = (face->height + face->descender) *
        face->size->metrics.y_ppem / face->units_per_EM;
    const auto glyph_topleft = pos + Vector2D<int>{
        face->glyph->bitmap_left, baseline - face->glyph->bitmap_top};

    for (int dy = 0; dy < bitmap.rows; ++dy) {
        unsigned char* q = &bitmap.buffer[bitmap.pitch * dy];
        if (bitmap.pitch < 0) {
            q -= bitmap.pitch * bitmap.rows;
        }
        for (int dx = 0; dx < bitmap.width; ++dx) {
            const bool b = q[dx >> 3] & (0x80 >> (dx & 0x7));
            if (b) {
                writer.Write(glyph_topleft + Vector2D<int>{dx, dy}, color);
            }
        }
    }

    FT_Done_Face(face);
    return MAKE_ERROR(Error::kSuccess);
}
```

}

WriteUnicode() を改造し、作った 2 つの関数を利用してグリフを描くようにしました (リスト 28.14)。RenderUnicode() によって face->glyph->bitmap に読み込まれたグリフを使い、引数で指定された writer に絵を描きます。この描画プログラムを詳しく説明しようと思うと結構長くなりそうです。その割には OS の作成の本質的な部分ではないと思いますから、説明は省略します。ごめんなさい。

1 点だけ。bitmap.buffer に実際のグリフ画像が読み込まれます。FT_LOAD_TARGET_MONO フラグを指定してグリフを読み込んでいるため、1 ビットが 1 ピクセルに対応する白黒画像としてグリフが読み込まれているはずです。すなわち 1 バイトで 8 ピクセル分のデータが入っています。ビット演算を使って `q[dx >> 3] & (0x80 >> (dx & 0x7))` のように 1 ピクセルずつ取り出し、そのビットが 1 なら描画、0 なら何もしない、というように処理すれば OK です。

最後に FT_Done_Face() を使い、NewFTFace() で生成したフェースオブジェクトを破棄して処理完了です。

リスト 28.15 タスクのスタックを 32KiB に増やす (task.hpp)

```
class Task {
public:
    static const int kDefaultLevel = 1;
    static const size_t kDefaultStackBytes = 8 * 4096;
```

「これで日本語対応が完了したぞ！」と思ったら、筆者の環境で、日本語を表示しようとする OS が強制再起動するという現象が起きました。調べてみると、FreeType がスタックを使いすぎ、スタック領域をはみ出してしまっていることが分かりました。リスト 28.15 のようにタスクのスタックを 4KiB から 32KiB に増やしてみたら、正常に動作するようになりました。

リスト 28.16 アプリ用のスタックも 32KiB に増やす (terminal.cpp)

```
const int stack_size = 8 * 4096;
LinearAddress4Level stack_frame_addr{0xffff'ffff'ffff'f000 - stack_size};
if (auto err = SetupPageMaps(stack_frame_addr, stack_size / 4096)) {
    return err;
}
```

<中略>

```
task.SetFileMapEnd(stack_frame_addr.value);

int ret = CallApp(argc.value, argv, 3 << 3 | 3, app_load.entry,
    stack_frame_addr.value + stack_size - 8,
    &task.OSStackPointer());
```

前の節ですでに WriteString() も Unicode 対応させていますから、何もしなくてもアプリからも日本語が表示できるようになっているはずです。ただ、アプリ用のスタックが小さいままだと誤動作してしまいますから、先ほどの改造と同じようにアプリ用スタックも増量します (リスト 28.16)。

リスト 28.17 FreeType ライブラリをリンクする (Makefile)

```
kernel.elf: $(OBJS) Makefile
    ld.lld $(LDFLAGS) -o kernel.elf $(OBJS) -lc -lc++ -lc++abi -lm \
    -lfreetype
```

最後の改造ポイントです。FreeType ライブラリを使うには、カーネルに FreeType ライブラリもリンクする必要があります。リスト 28.17 のように、リンカのオプションに `-lfreetype` を加えます。

図 28.3 cat で日本語文書を表示する様子



図 28.3 に実行した様子を示します。cat jpn.txt とすると「??」ではなくちゃんと日本語が表示されるようになりました。さらに、winhello をコピーして日本語で挨拶を表示するアプリ winjpn も作って動かしてみました。うんうん、いい感じです。日本語が出るようになって、筆者はちょっと感動しています！

28.3 リダイレクト (osbook_day28c)

アプリケーションは `printf()` などを使って標準出力に文字列を出力します。また、`scanf()` などを使って標準入力からデータを読みます。標準入出力はデフォルトでターミナルへ接続されているため、標準出力に出力された文字列は最終的にターミナルへ表示され、ターミナルへ入力された文字列は最終的に標準入力に渡されます。「ターミナル」ではなく「標準入出力」などと回りくどく呼ぶのは、標準入出力をターミナル以外に振り向けることがあるからなのです。標準入出力をターミナル以外につなぐ機能をリダイレクト (redirect) と呼びます。

Linux などではリダイレクトを使って標準入出力をファイルに接続できます。例えばターミナル上で

echo "hoge" > piyo とすると、piyo というファイルに hoge という文字列が書き込まれます。この仕組みを MikanOS でも真似てみようと思います。この節では、標準出力の文字列をファイルに書き出す、つまり出力方向のリダイレクト機能を作ってみます。

リスト 28.18 ターミナルに標準入出力を表すファイルを持たせる (terminal.hpp)

```
bool show_window_;
std::array<std::shared_ptr<FileDescriptor>, 3> files_;
```

まず Terminal クラスに、標準入出力を表す 3 つのファイルディスクリプタを保持する配列 files_ を追加しました (リスト 28.18)。files_ の添字とファイルディスクリプタ番号が対応していて、0 番が標準入力、1 番が標準出力、2 番が標準エラー出力です。

リスト 28.19 初期設定で標準入出力をターミナルに接続する (terminal.cpp)

```
Terminal::Terminal(Task& task, bool show_window)
: task_{task}, show_window_{show_window} {
    for (int i = 0; i < files_.size(); ++i) {
        files_[i] = std::make_shared<TerminalFileDescriptor>(*this);
    }
    if (show_window) {
```

リスト 28.19 に示すように、ターミナルクラスの初期化時に files_ を設定するようにしました。3 つの要素すべてに TerminalFileDescriptor を設定することで、標準入出力がターミナルに接続された状態にしておきます。

リスト 28.20 アプリ起動時に標準入出力のファイルを設定する (terminal.cpp)

```
for (int i = 0; i < files_.size(); ++i) {
    task.Files().push_back(files_[i]);
}
```

リスト 28.20 は Terminal::ExecuteFile() の一部で、アプリ用のファイルディスクリプタの初期値を設定する部分です。今までは、ここで固定的に TerminalFileDescriptor を設定していましたが、Terminal::files_ をコピーするように変えました。この時点で Terminal::files_[1] がターミナルを表す TerminalFileDescriptor から他のファイルディスクリプタに切り替わっていれば、アプリは自動的にそのファイルに対して出力するようになります。これがリダイレクトのキモとなる部分です。

後は、コマンドラインで > piyo のように入力されたことを検知して、files_[1] を piyo を表すファイルディスクリプタに入れ替える処理を書けばいいことになります。

リスト 28.21 Task::files_ の要素を std::shared_ptr に変更する (task.hpp)

```
std::vector<std::shared_ptr<FileDescriptor>> files_{};
```

細かいですが、**リスト 28.21** に示すように `Task::files_` の要素型を `std::shared_ptr` に変えました。 `std::unique_ptr` のままだと `task.Files().push_back(files_[i])` のコンパイルがエラーになるからです。真の原因は `Terminal::files_` の要素型である `std::shared_ptr` から `std::unique_ptr` への代入がエラーになることです。 `std::unique_ptr` は指し先のオブジェクトを占有するポインタである一方で、 `std::shared_ptr` は指し先のオブジェクトを共有するためのポインタです。 `std::unique_ptr` を `std::shared_ptr` に代入することはできますが、その逆はできません。

リスト 28.22 リダイレクト記号を見つけたら標準出力を切り替える (terminal.cpp)

```
void Terminal::ExecuteLine() {
    char* command = &linebuf_[0];
    char* first_arg = strchr(&linebuf_[0], ' ');
    char* redir_char = strchr(&linebuf_[0], '>');
    if (first_arg) {
        *first_arg = 0;
        ++first_arg;
    }

    auto original_stdout = files_[1];

    if (redir_char) {
        *redir_char = 0;
        char* redir_dest = &redir_char[1];
        while (isspace(*redir_dest)) {
            ++redir_dest;
        }

        auto [ file, post_slash ] = fat::FindFile(redir_dest);
        if (file == nullptr) {
            auto [ new_file, err ] = fat::CreateFile(redir_dest);
            if (err) {
                PrintToFD(*files_[2],
                    "failed to create a redirect file: %s\n", err.Name());
                return;
            }
            file = new_file;
        } else if (file->attr == fat::Attribute::kDirectory || post_slash) {
            PrintToFD(*files_[2], "cannot redirect to a directory\n");
            return;
        }
        files_[1] = std::make_shared<fat::FileDescriptor>(*file);
    }

    if (strcmp(command, "echo") == 0) {
```

リスト 28.22 にリダイレクト記号 `>` を見つけたら標準出力を切り替える処理を示します。コマンドライン文字列の中に `>` がある場合、 `redir_char` が非ヌルの値になり、 `if` 文の中が実行されるという仕組みです。 `if` 文の処理を見ていきます。

`redir_dest` はリダイレクト先のファイルパス (`> piyo` なら `piyo`) を指すポインタです。 `while` 文は `>` 記号の後ろにある空白文字を読み飛ばす処理です。 `while` 文が終わると、 `redir_dest` はファイルパスの先頭を指す状態になります。

`fat::FindFile()` の呼び出しから `files_[1]` への代入までの一連の処理では、`redir_dest` が指すファイルがあればそこをリダイレクト先として使用し、なければ `fat::CreateFile()` を使って空ファイルを作ります。ファイルがある場合、ファイル内容は消されて上書きされます。

新規ファイルを作る処理が失敗したら「failed to create a redirect file」というエラーメッセージを表示します。エラーメッセージは、今までなら `Print()` を使って直接ターミナルに出力していましたが、今回からは標準エラー出力 (`files_[2]`) に出力するようにします。こうしておけば、将来的に標準エラー出力をリダイレクトする機能が付けば、エラーメッセージをファイルに記録できるようになります。このときに使っている `PrintToFD()` は新しく作った関数で、文字列を整形しながらファイルディスクリプタに書き込みます。実装は後で紹介します。

さて、ここまでの改造で、アプリが `printf()` などを使い標準出力に出力した文字列を、ターミナルではなくファイルに書き出せるようになったはずです。ですから、早速次のようなコマンドラインで実行してみたいくなります。

```
>echo deadbeef > piyo
```

上記のコマンドラインの意図は `deadbeef` という文字列を `piyo` というファイルに書き込むことです。しかし、これはまだうまくいきません。なぜなら `echo` は `deadbeef` という文字列を標準出力に出力せず、`Print()` を使ってターミナルに直接出力してしまうからです。echo はアプリではなくターミナル組み込みコマンドであり、文字列を出力する際に `files_` に設定したファイルディスクリプタを使うように作られていないのです。

リスト 28.23 echo コマンドが標準出力に表示するようにする (terminal.cpp)

```
if (strcmp(command, "echo") == 0) {
    if (first_arg) {
        PrintToFD(*files_[1], "%s", first_arg);
    }
    PrintToFD(*files_[1], "\n");
} else if (strcmp(command, "clear") == 0) {
```

この問題を解決するのは簡単です。echo が `Print()` を直接使うのではなく、標準出力にメッセージを出力するようにすればいいのです。ということでリスト 28.23 のように、`Print()` を `PrintToFD()` に切り替えました。同様の修正を、他のターミナル組み込みコマンドにも適用します。

リスト 28.24 エラーメッセージは標準エラー出力へ (terminal.cpp)

```
} else if (strcmp(command, "cat") == 0) {
    auto [ file_entry, post_slash ] = fat::FindFile(first_arg);
    if (!file_entry) {
        PrintToFD(*files_[2], "no such file: %s\n", first_arg);
    } else if (file_entry->attr != fat::Attribute::kDirectory && post_slash) {
        char name[13];
        fat::FormatName(*file_entry, name);
        PrintToFD(*files_[2], "%s is not a directory\n", name);
    } else {
        fat::FileDescriptor fd{*file_entry};
        char u8buf[5];
```

```

    DrawCursor(false);
    while (true) {
        if (fd.Read(&u8buf[0], 1) != 1) {
            break;
        }
        const int u8_remain = CountUTF8Size(u8buf[0]) - 1;
        if (u8_remain > 0 && fd.Read(&u8buf[1], u8_remain) != u8_remain) {
            break;
        }
        u8buf[u8_remain + 1] = 0;

        PrintToFD(*files_[1], "%s", u8buf);
    }
    DrawCursor(true);
} else if (strcmp(command, "noterm") == 0) {

```

例として cat の修正をリスト 28.24 に示します。標準出力(files_[1])と標準エラー出力(files_[2])を使い分けているところがポイントです。cat はファイルの内容を表示する、というのが主となる機能です。したがって、ファイルの内容は標準出力に出力します。一方、ファイルが開けない場合などにエラーメッセージを出すことはcatの主機能ではなくエラー処理です。そのような場合は標準エラー出力に対してメッセージを出力するようにします。出力先を分けておくことで、リダイレクト先のファイルにエラーメッセージが混入することを防げます。

echo と cat 以外の組み込みコマンドについても同様に修正しました。簡単な修正ですので、紹介は省きます。

リスト 28.25 ExecuteLine() の最後に標準出力先を元に戻す (terminal.cpp)

```

    files_[1] = original_stdout;
}

```

リダイレクトの処理で Terminal::files_[1] を変更したまま放置してしまうと、次にコマンドを実行したときにリダイレクト設定が引き継がれてしまいます。リスト 28.25 のように、ExecuteLine() の最後で files_[1] を元に戻しておきましょう。

リスト 28.26 PrintToFD() は指定されたファイルディスクリプタに文字列を書く (file.cpp)

```

size_t PrintToFD(FileDescriptor& fd, const char* format, ...) {
    va_list ap;
    int result;
    char s[128];

    va_start(ap, format);
    result = vsprintf(s, format, ap);
    va_end(ap);

    fd.Write(s, result);
    return result;
}

```

最後に PrintToFD() の紹介をします (リスト 28.26)。printf() の実装とほぼ同じなので難しいことはないと思います。文字列を整形してから fd.Write() を使って出力します。この関数は FileDescriptor のどの subclasses に対しても汎用的に使えるため、terminal.cpp ではなく file.cpp に定義を入れ、file.hpp にプロトタイプ宣言を入れておきました。

図 28.4 組み込みコマンドやアプリの出力をリダイレクトする様子

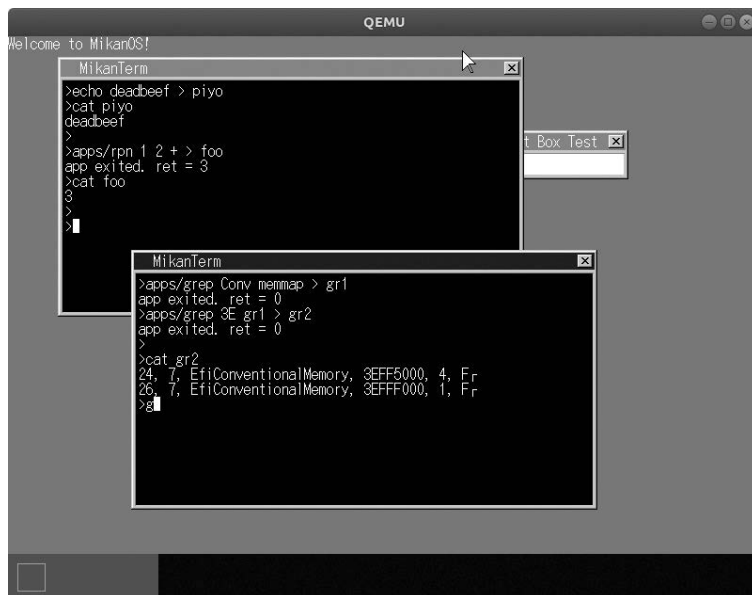


図 28.4 に実行した様子を示します。上側のターミナルでは、echo や rpn コマンドの出力をファイルにリダイレクトしてみました。期待通りにリダイレクトされていることが分かります。下側のターミナルでは grep を 2 回適用し、mmap ファイルの中から Conv と 3E の両方が含まれている行を抜き出してみました。リダイレクトを使えば、このようにコマンド間でデータ連携することができるのです。

本書では標準出力のリダイレクトだけを作りました。Linux などでは標準入力や標準エラー出力のリダイレクトも可能ですが、MikanOS には搭載しないことにします。何故かという、それらに対応しようと思うとコマンドライン引数の解析処理が結構複雑になってしまうからです。具体的には "echo a <in >out" でも "echo a >out <in" でも、標準出力のリダイレクト先が out、標準入力のリダイレクト先が in、として認識する必要があります。面倒ですね。文字列の解析を頑張りさえすれば、リダイレクトの処理自体は Terminal::files_ を入れ替えるだけなので簡単です。文字列の解析はプログラミング力を鍛えるのに役立ちますから、興味のある読者の方はチャレンジしてみると良いのではないのでしょうか。

第29章

アプリ間通信

リダイレクトによってアプリケーション間のデータ連携ができるようになりましたが、世の OS には、ほかにもアプリケーション同士を繋ぐ機能が沢山あります。この章では、それらの中でも代表的なパイプの機能を作ってみようと思います。パイプが使えるようになれば、ファイルを介さずに 1 行で複数のコマンドを連携させられるようになり、とても便利です。実装はしませんが、最後にページングを利用してアプリケーション間で同じメモリ領域を参照する共有メモリという機能も紹介します。共有メモリを使うとパイプより高度なデータ構造をアプリケーション間で使用でき、プログラミングの可能性が広がります。

29.1 終了コード (osbook_day29a)

パイプや共有メモリを作る前に、アプリの終了コードを画面に表示しないようにしたいと思いました。終了コードは `exit()` の引数、すなわちアプリから OS 側に返される戻り値のことです。今まではアプリ終了時に `app exited. ret = 3` のように終了コードが画面に表示されていましたが、毎回表示されるのはちょっと邪魔です。他の OS だと終了コードは画面に表示されません。その代わり、後から取得して利用できるようになっています。目標は、次に示すように `echo $?` で直前のアプリの終了コードを表示できるようになることです。

```
>apps/rpn 1 2 +
3
>echo $?
3
>
```

このように、アプリを実行し、その直後に `echo $?` とすると終了コードを表示するようにしたいです。`$?` はターミナルで使われる変数の一種です。Linux などではターミナル上で `$` に続く名前が変数名として扱われ、その中でも `$?` は直前のアプリの終了コードを表す変数名ということになっています。それを真似しようかなと思います。

リスト 29.1 直前のアプリの終了コードを表すメンバ変数を追加 (terminal.hpp)

```
bool show_window_;
std::array<std::shared_ptr<FileDescriptor>, 3> files_;
int last_exit_code_{0};
```

直前のアプリの終了コードを保持しておく変数 `Terminal::last_exit_code_` を追加しました(リスト 29.1)。

リスト 29.2 `echo $?` は `last_exit_code_` の値を表示する (terminal.cpp)

```
if (strcmp(command, "echo") == 0) {
    if (first_arg && first_arg[0] == '$') {
        if (strcmp(&first_arg[1], "?") == 0) {
            PrintToFD(*files_[1], "%d", last_exit_code_);
        }
    } else if (first_arg) {
        PrintToFD(*files_[1], "%s", first_arg);
    }
    PrintToFD(*files_[1], "\n");
} else if (strcmp(command, "clear") == 0) {
```

リスト 29.2 のように `echo` コマンドを改造し、引数が `$?` のときには `last_exit_code_` の値を表示するようにしました。この時点ではまだ `last_exit_code_` を更新する処理がありませんので、常に 0 が表示されてしまいます。以降ではこの変数を更新する処理を追加していきますよ。

リスト 29.3 アプリの終了コードを保持する変数 exit_code を定義 (terminal.cpp)

```

auto original_stdout = files_[1];
int exit_code = 0;

< 中略 >

last_exit_code_ = exit_code;
files_[1] = original_stdout;
}

```

リスト 29.3 のように、Terminal::ExecuteLine() の先頭の方で exit_code 変数を定義し、最後に last_exit_code_ を更新するようにしてみました。各コマンドの処理の中で exit_code に適切な値を設定します。

リスト 29.4 処理がエラーになったら exit_code を 1 にする (terminal.cpp)

```

} else if (strcmp(command, "cat") == 0) {
    auto [ file_entry, post_slash ] = fat::FindFile(first_arg);
    if (!file_entry) {
        PrintToFD(*files_[2], "no such file: %s\n", first_arg);
        exit_code = 1;
    } else if (file_entry->attr != fat::Attribute::kDirectory && post_slash) {
        char name[13];
        fat::FormatName(*file_entry, name);
        PrintToFD(*files_[2], "%s is not a directory\n", name);
        exit_code = 1;
    } else {

```

リスト 29.4 に cat コマンドの改造例を示します。冒頭で「アプリの終了コード」に対応するのが目標としましたが、ついでにコマンドの終了コードも扱うことにしました。組み込みコマンドとアプリを区別する意味がありませんからね。

多くの OS では、終了コードは 0 が正常終了、非 0 がエラー終了を表すことになっています。そこで、MikanOS でもアプリやコマンドがエラー終了したときには 0 ではない値にしようと思います。上記の改造例では、ファイルが見つからなかったり、ファイルを期待しているのにディレクトリが指定されたりしたら終了コードを 1 にします。このような感じで、他のコマンドでもエラー発生時に exit_code に 1 を代入するようにします。

リスト 29.5 ExecuteFile() が終了コードを返すようにする (terminal.cpp)

```

WithError<int> Terminal::ExecuteFile(fat::DirectoryEntry& file_entry,
                                     char* command, char* first_arg) {
    __asm__("cli");
    auto& task = task_manager->CurrentTask();
    __asm__("sti");

    auto [ app_load, err ] = LoadApp(file_entry, task);
    if (err) {
        return { 0, err };
    }

```

< 中略 >

```
int ret = CallApp(argc.value, argv, 3 << 3 | 3, app_load.entry,
                 stack_frame_addr.value + stack_size - 8,
                 &task.OSStackPointer());

task.Files().clear();
task.FileMaps().clear();

if (auto err = CleanPageMaps(LinearAddress4Level{0xffff'8000'0000'0000})) {
    return { ret, err };
}
return { ret, FreePML4(task) };
}
```

`Terminal::ExecuteFile()` は中でアプリを起動させ、その終了コードを画面に印字するだけで戻り値として返す機能がありませんでした。そこで**リスト 29.5**のように `int` 型で終了コードを返すことにしました。そして、`ExecuteFile()` の中で終了コードを印字する機能は削除しました。

リスト 29.6 `ExecuteFile()` の呼び出し側で終了コードを受け取る (terminal.cpp)

```
auto [ ec, err ] = ExecuteFile(*file_entry, command, first_arg);
if (err) {
    PrintToFD(*files_[2], "failed to exec file: %s\n", err.Name());
    exit_code = -ec;
} else {
    exit_code = ec;
}
```

リスト 29.6 のように `ExecuteFile()` の呼び出し側も改造します。戻り値の終了コード (`ec`) を `exit_code` に代入します。`err` が真 (`Error::kSuccess` 以外のエラー) のときは、アプリ自体が失敗したわけではなく、メモリの確保とか階層ページング構造の設定など、アプリの起動に付随する OS 本体の処理が失敗したことを意味します。そんなときは終了コードをマイナスにした値を真の終了コードとすることにしてみました。何となくそうした程度で、必然性は全然ありませんが……。

図 29.1 様々な終了コード



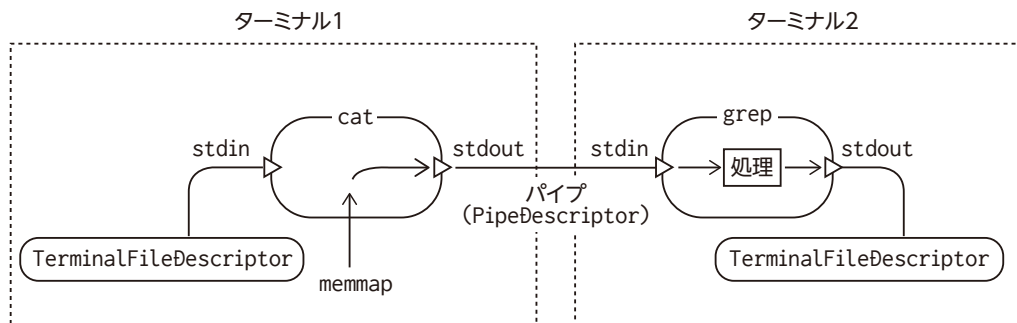
図 29.1 に実行結果を示します。期待通りにアプリの終了コードが表示されるようになりました。完成です。

29.2 パイプ (osbook_day29b)

アプリケーション同士で情報をやり取りする仕組みを追加しようと思います。MikanOS では今のところファイルの読み書きによってアプリ間でデータのやり取りが可能です。しかし、アプリ間で通信するパイプや共有メモリといった便利な機能はまだありません。それらのアプリ間の通信機能が備わることで、他のアプリの出力を加工するようなアプリを作りやすくなります。ちなみに、アプリ間通信はプロセス間通信とも呼びます。

Linuxなどで `grep "hoge" foo | wc -l` というコマンドラインを実行すると、`foo` というファイルから `hoge` が含まれる行数を数えることができます。コマンドライン中にある `|` がパイプを表す記号で、`grep` の標準出力を `wc` の標準入力に接続するという意味になります。`wc -l` は標準入力に与えられた文字列の行数を数えるコマンドです。MikanOS でもこのような処理ができるようにパイプ機能を付けてみます。

図 29.2 cat と grep がパイプで繋がる様子



パイプは図 29.2 のように、片方の標準出力をもう一方の標準入力に接続する仕組みです。図は `cat memmap | grep hoge` の実行に際して、`cat` の標準出力と `grep` の標準入力に接続された状態を表しています。パイプの仕組みは標準入出力を使うものなら対応できるため、アプリでも組み込みコマンドでも区別する必要はありません。この節では、特に断らない限り組み込みコマンドとアプリを含めて「コマンド」と呼ぶことにします。

パイプの実現方法は大きく 2 つ考えられます。1 つは、左側のコマンドの出力を記録しつつ終了を待ち、その後記録しておいたデータを右側のコマンドに流し込む方法です。もう 1 つは、左右のコマンドを両方同時に動かし、繰り返し左側のコマンドの出力を読み取っては右側に送る、という方法です。左側のコマンドが短時間に終わるもので、かつ出力がそれほど大きくなければ前者の方法がシンプルで動作効率も良いと思います。

しかし、一般には後者の方法が好ましいです。左側のコマンドが長時間動く場合、後者の方法なら左側のコマンドの終了を待たずに右側のコマンドを開始できます。左側のコマンドがデータを出力するたびに右側のコマンドへデータが受け渡され、左右の処理が同時並行に動きます。例えば左側のコマンドがネットワークから散発的に受信されるデータを処理するものだったり、人間の入力を待って処理するものだったりした場合に効果を発揮します。

また、左側のコマンドの出力が大きい場合にも左右のコマンドを同時並行に動かすのが有利でしょう。左側のコマンドの出力をちょっとずつ右側のコマンドに流せば、一時的な記憶領域が小さくて済むからです。仮に左側のコマンドが 100GiB の文書データを出力するようなコマンドで、右側のコマンドが行数を数えるコマンドだとしましょう。100GiB の文書データを一時的にメモリに記録するのは大変ですが（少なくとも 100GiB のメモリが必要ですね）、4KiB ずつデータを転送するにすれば 4KiB のメモリ領域で足ります。

この節では左右のコマンドを同時に起動させる方式でパイプを実装してみようと思います。ちょっと複雑になってしまいますが、頑張ります！

リスト 29.7 パイプ記号 | があったらパイプを生成して標準入出力を付け替える (terminal.cpp)

```

void Terminal::ExecuteLine() {
    char* command = &linebuf_[0];
    char* first_arg = strchr(&linebuf_[0], ' ');
    char* redir_char = strchr(&linebuf_[0], '>');
    char* pipe_char = strchr(&linebuf_[0], '|');

    < 中略 >

    std::shared_ptr<PipeDescriptor> pipe_fd;
    uint64_t subtask_id = 0;

    if (pipe_char) {
        *pipe_char = 0;
        char* subcommand = &pipe_char[1];
        while (isspace(*subcommand)) {
            ++subcommand;
        }

        auto& subtask = task_manager->NewTask();
        pipe_fd = std::make_shared<PipeDescriptor>(subtask);
        auto term_desc = new TerminalDescriptor{
            subcommand, true, false,
            { pipe_fd, files_[1], files_[2] }
        };
        files_[1] = pipe_fd;

        subtask_id = subtask
            .InitContext(TaskTerminal, reinterpret_cast<int64_t>(term_desc))
            .Wakeup()
            .ID();
    }

    < 中略 (ここで組み込みコマンドやアプリを実行する) >

    if (pipe_fd) {
        pipe_fd->FinishWrite();
        __asm__("cli");
        auto [ ec, err ] = task_manager->WaitFinish(subtask_id);
        __asm__("sti");
        if (err) {
            Log(kWarn, "failed to wait finish: %s\n", err.Name());
        }
        exit_code = ec;
    }

    last_exit_code_ = exit_code;
    files_[1] = original_stdout;
}

```

今回の改造の全体像を最も良く表す部分をリスト 29.7 に示しました。これはターミナルのコマンドラインを処理する ExecuteLine() の一部です。まずはパイプ記号 | を見つけたとき (if (pipe_char)) の処理を上から順に説明します。

29.3 コマンドラインの解析とタスクの起動

はじめに、ポインタ変数 `subcommand` が右側のコマンド名の先頭を指すように調整します。次に新しいタスクを生成して `subtask` と名付けます。 `subtask` を利用して `PipeDescriptor` 型のオブジェクトを作ります。このクラスは今回新たに作ったもので、パイプ処理の中心的な役割を果たします。

`TerminalDescriptor` 型もまた今回作った構造体です。新しいターミナルを生成する際の初期設定を保持するものです。上記のプログラムでは、表 29.1 の設定で新しいターミナルを起動します。

表 29.1 右側のコマンドを実行するためのターミナル設定

設定項目	設定値	説明
<code>command_line</code>	<code>subcommand</code>	新しいターミナルで <code>subcommand</code> を自動実行する
<code>exit_after_command</code>	<code>true</code>	自動実行後にターミナルを終了する
<code>show_window</code>	<code>false</code>	新しいウィンドウを生成しない
<code>files[0]</code>	<code>pipe_fd</code>	新しいターミナルの標準入力をパイプに接続する
<code>files[1]</code>	<code>files_[1]</code>	新しいターミナルの標準出力を現在のターミナルと同じにする
<code>files[2]</code>	<code>files_[2]</code>	新しいターミナルの標準エラー出力を現在のターミナルと同じにする

`term_desc` を生成した後で `files_[1] = pipe_fd;` を実行しています。これによって、現在のターミナルの標準出力がパイプに接続されます。この行を、`term_desc` を生成する前に置いてしまうと、`term_desc->files[1]` がパイプに接続されてしまい動作がおかしくなります。

最後に、準備したターミナル設定 `term_desc` を使って `TaskTerminal()` を起動します。これで標準入力パイプに接続されたタスクが1つ、新たに動き出します。

パイプを設定したターミナルを起動させた後は、パイプの左側のコマンドをいつも通りに実行します。組み込みコマンドやアプリの実行に関する処理はまったく改造していません。左側のコマンド実行が終わった後に、後片付けの処理 (`if (pipe_fd)`) を追加しました。

後片付けの処理では `pipe_fd->FinishWrite()` を実行して、これ以上データを送信しないよーというのをパイプの受信側に伝えます。これをしないと、パイプの受信側はまだデータが来るかもしれないと思って、ずっと待ち状態になってしまいます。次に `task_manager->WaitFinish(subtask_id)` を実行し、右側のコマンドが終了するのを待ちます。データの送信をやめてしばらくの間は右側のコマンドの処理が続き、そのうち終了するはずですが。

パイプを使った処理のように、2 個以上のコマンドが同時並行に動作する場合、全体の終了コードをどうするかについて選択肢があります。MikanOS では、右側のコマンドの終了コードをコマンドライン全体の終了コードとすることにしました。

29.4 パイプ処理の本体 PipeDescriptor

さて、ここからはパイプの中心的な機能を担う PipeDescriptor の実装を進めていきます。リスト 29.8 にクラス定義を示します。パイプは読み書きができるため、FileDescriptor を継承してファイルの一種として作ります。

リスト 29.8 PipeDescriptor はパイプそのものを表現する (terminal.hpp)

```
class PipeDescriptor : public FileDescriptor {
public:
    explicit PipeDescriptor(Task& task);
    size_t Read(void* buf, size_t len) override;
    size_t Write(const void* buf, size_t len) override;
    size_t Size() const override { return 0; }
    size_t Load(void* buf, size_t len, size_t offset) override { return 0; }

    void FinishWrite();

private:
    Task& task_;
    char data_[16];
    size_t len_{0};
    bool closed_{false};
};
```

Read()、Write()、Size()、Load() は FileDescriptor にあるメソッドですから、その役割は説明不要でしょう。もちろん中身は後で詳しく紹介します。一方で FinishWrite() は PipeDescriptor クラス独自のメソッドです。パイプは普通のファイルと違って末尾がありません。そのため、送信側から「これ以上データを送りません」という意思表示ができる必要があります。FinishWrite() はそのためのメソッドです。ターミナルで Ctrl-D を入力することと似ていますね。

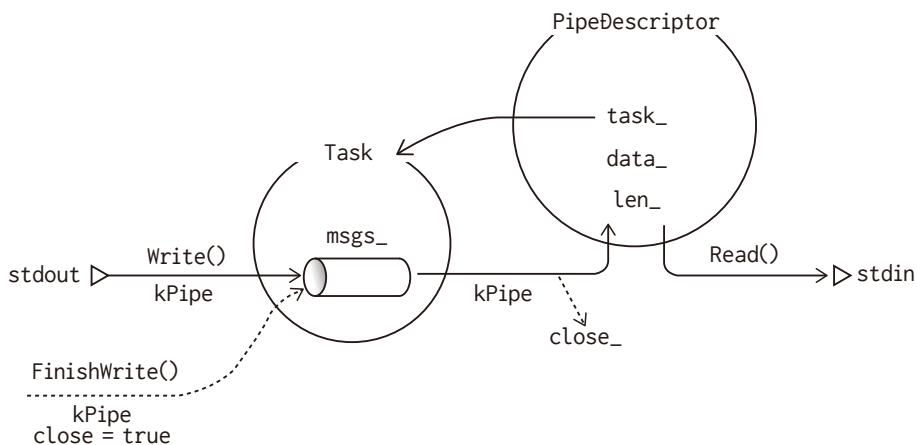
各メンバ変数の役割を簡単に説明します。task_ はデータ送信先のコマンドが動作するタスクを表します。送信元ではなく送信先です。data_ と len_ は、データ送信先のコマンドが Read() を使ってデータを読み取るときに活躍します。closed_ はパイプにこれ以上データが送信されないことを示すフラグです。FinishWrite() を実行すると真となります。

リスト 29.9 Write() は kPipe メッセージを送信する (terminal.cpp)

```
size_t PipeDescriptor::Write(const void* buf, size_t len) {
    auto bufc = reinterpret_cast<const char*>(buf);
    Message msg{Message::kPipe};
    size_t sent_bytes = 0;
    while (sent_bytes < len) {
        msg.arg.pipe.len = std::min(len - sent_bytes, sizeof(msg.arg.pipe.data));
        memcpy(msg.arg.pipe.data, &bufc[sent_bytes], msg.arg.pipe.len);
        sent_bytes += msg.arg.pipe.len;
        __asm__("cli");
        task_.SendMessage(msg);
        __asm__("sti");
    }
    return len;
}
```

リスト 29.9 に、パイプにデータを書きこむためのメソッド `Write()` の実装を示します。このメソッドは受け取ったデータを `PipeDescriptor::task_` のメッセージキューへ送信します (図 29.3)。

図 29.3 `PipeDescriptor` が実現するパイプの仕組みの概略



`Write()` は、受け取ったデータをメッセージ構造体に包んで送信先タスクのメッセージキュー (`msgs_`) に送ります。 `Message::kPipe` の引数型はリスト 29.10 のように定義していて、16 バイトずつデータを送信できるようにしてあります。

リスト 29.10 `kPipe` メッセージの引数型 (`message.hpp`)

```
struct {
    char data[16];
    uint8_t len;
} pipe;
```

`data` を 16 バイトより大きくしすぎるとメッセージ構造体が大きくなり、`kPipe` 以外のメッセージでメモリが無駄になってしまいます。逆に `data` が小さくなると何個もメッセージを送らねばならず、オーバーヘッドが大きくなります。`data` をポインタにしておいて必要な大きさのメモリ領域を割り当てる方法もありますが、ちょっと複雑になるかなと思って今回はデータ領域をメッセージ構造体に埋め込む方法を採用してみました。

リスト 29.11 `Read()` はメッセージキューからデータを受信する (`terminal.cpp`)

```
size_t PipeDescriptor::Read(void* buf, size_t len) {
    if (len_ > 0) {
        const size_t copy_bytes = std::min(len_, len);
        memcpy(buf, data_, copy_bytes);
        len_ -= copy_bytes;
        memmove(data_, &data_[copy_bytes], len_);
        return copy_bytes;
    }
}
```

```

if (closed_) {
    return 0;
}

while (true) {
    __asm__("cli");
    auto msg = task_.ReceiveMessage();
    if (!msg) {
        task_.Sleep();
        continue;
    }
    __asm__("sti");

    if (msg->type != Message::kPipe) {
        continue;
    }

    if (msg->arg.pipe.len == 0) {
        closed_ = true;
        return 0;
    }

    const size_t copy_bytes = std::min<size_t>(msg->arg.pipe.len, len);
    memcpy(buf, msg->arg.pipe.data, copy_bytes);
    len_ = msg->arg.pipe.len - copy_bytes;
    memcpy(data_, &msg->arg.pipe.data[copy_bytes], len_);
    return copy_bytes;
}
}

```

リスト 29.11 に `Read()` の実装を示します。このメソッドはタスクのメッセージキューからメッセージを 1 つずつ受信し、`kPipe` メッセージだったらパイプの処理を行います。

原理が簡単な割に処理が複雑なのは、`Read()` の引数に与えられたバッファ `buf` の大きさが 16 バイト未満の場合を考慮しているからです。`buf` が必ず 16 バイト以上であれば、受信したメッセージに含まれるデータ全体を `buf` にコピーするだけで完了です。しかし `buf` が小さい場合、`Read()` しきれない残りを次回以降の `Read()` で返さなければいけません。そのために `PipeDescriptor::data_` が活躍します。`data_` に残り部分をコピーし、次回以降、`data_` にデータが残っている間はメッセージキューではなく `data_` から読み取るようにします。

リスト 29.12 `FinishWrite()` はこれ以上送信データが無いことを伝える (`terminal.cpp`)

```

void PipeDescriptor::FinishWrite() {
    Message msg{Message::kPipe};
    msg.arg.pipe.len = 0;
    __asm__("cli");
    task_.SendMessage(msg);
    __asm__("sti");
}

```

リスト 29.12 に、送信データがこれ以上無いことを伝えるためのメソッド `FinishWrite()` を示します。処理は単純で、`len` が 0 であるメッセージを送ります。`Read()` メソッドは、`len` が 0 であるメッセージを受け取るとパイプが閉じられたことを認識し、`PipeDescriptor::closed_` を真に設定します。

29.5 ターミナルの起動と終了

ここまでで、パイプを使ったデータの送受信の仕組みを作りました。`kPipe` メッセージをタスクのメッセージキューを経由してやり取りする、パイプの核となる仕組みです。以降では、パイプの右側のタスクの起動と終了に関する処理を実装していきます。

リスト 29.13 `TerminalDescriptor` を受け取るように改造した `TaskTerminal()` (`terminal.cpp`)

```
void TaskTerminal(uint64_t task_id, int64_t data) {
    const auto term_desc = reinterpret_cast<TerminalDescriptor*>(data);
    bool show_window = true;
    if (term_desc) {
        show_window = term_desc->show_window;
    }

    __asm__("cli");
    Task& task = task_manager->CurrentTask();
    Terminal* terminal = new Terminal{task, term_desc};
    if (show_window) {
        layer_manager->Move(terminal->LayerID(), {100, 200});
        layer_task_map->insert(std::make_pair(terminal->LayerID(), task_id));
        active_layer->Activate(terminal->LayerID());
    }
    __asm__("sti");

    if (term_desc && !term_desc->command_line.empty()) {
        for (int i = 0; i < term_desc->command_line.length(); ++i) {
            terminal->InputKey(0, 0, term_desc->command_line[i]);
        }
        terminal->InputKey(0, 0, '\n');
    }

    if (term_desc && term_desc->exit_after_command) {
        delete term_desc;
        __asm__("cli");
        task_manager->Finish(terminal->LastExitCode());
        __asm__("sti");
    }
}
```

リスト 29.13 に `TaskTerminal()` の改造箇所を示します。今までこの関数は第 2 引数としてコマンドライン文字列へのポインタを受け取る設計でした。今回の改造で `TerminalDescriptor` へのポインタを受け取るように変えました。主な変更点は 2 つで、受け取ったポインタ `term_desc` を `Terminal` クラスのコンストラクタに渡すようにしたことと、`term_desc->exit_after_command` が真のときにタスクを終了する処理を加えたことです。

リスト 29.14 TerminalDescriptor を利用してメンバ変数を初期化する (terminal.cpp)

```
Terminal::Terminal(Task& task, const TerminalDescriptor* term_desc)
    : task_{task} {
    if (term_desc) {
        show_window_ = term_desc->show_window;
        for (int i = 0; i < files_.size(); ++i) {
            files_[i] = term_desc->files[i];
        }
    } else {
        show_window_ = true;
        for (int i = 0; i < files_.size(); ++i) {
            files_[i] = std::make_shared<TerminalFileDescriptor>(*this);
        }
    }

    if (show_window_) {
```

まず Terminal クラスのコンストラクタを改造します (**リスト 29.14**)。受け取った term_desc の情報を使って、show_window_ と files_ を初期化します。特に難しいところはないかと思います。最初に起動するターミナルや F2 キーで起動するターミナルの場合、term_desc はヌルポインタとなりますから、そのときはデフォルト値を代入するようにします。

次に term_desc->exit_after_command が真の場合の処理を見ていきます。このフラグが真の場合、term_desc->command_line に設定されたコマンドラインを実行し終わったらターミナルを終了させます。ターミナルを終了させるのに使っている task_manager->Finish() の実装は後で紹介します。

TaskTerminal() の引数を変えたので、ほかにターミナルタスクを起動する部分も改造しなければなりません。調べたところ、幸いにして影響を受ける箇所はnotermコマンドだけでした。このコマンド、覚えていますか? 「**24.5 ターミナル無しのアプリ起動**」で作成した、ウィンドウを開かずにコマンドを別ターミナル上で起動する仕組みです。さくっと改造してしまいます (**リスト 29.15**)。

リスト 29.15 noterm の中で TaskTerminal() に渡す値を変更する (terminal.cpp)

```
} else if (strcmp(command, "noterm") == 0) {
    auto term_desc = new TerminalDescriptor{
        first_arg, true, false, files_
    };
    task_manager->NewTask(
        .InitContext(TaskTerminal, reinterpret_cast<int64_t>(term_desc))
        .Wakeup());
}
```

29.6 タスクの終了

実は今まで、起動したターミナルを終了させることはできませんでした。起動したら起動しっぱなし。noterm はウィンドウを開かずにコマンドを別ターミナル上で起動しますが、コマンド終了後もずっとターミナルが裏で生き続けます。パイプを実現するには右側のコマンドが終了するのを待つ必要があります。そこで、ターミナルを終了できるようにしようと思います。これから作るのはターミナルだけでなく、Task で表されるすべてのタスクで機能する汎用的な終了の仕組みです。

リスト 29.16 Finish() はタスクを終了して finish_tasks_ に終了コードを登録する (task.cpp)

```
void TaskManager::Finish(int exit_code) {
    Task* current_task = RotateCurrentRunQueue(true);

    const auto task_id = current_task->ID();
    auto it = std::find_if(
        tasks_.begin(), tasks_.end(),
        [current_task](const auto& t){ return t.get() == current_task; });
    tasks_.erase(it);

    finish_tasks_[task_id] = exit_code;
    if (auto it = finish_waiter_.find(task_id); it != finish_waiter_.end()) {
        auto waiter = it->second;
        finish_waiter_.erase(it);
        Wakeup(waiter);
    }

    RestoreContext(&CurrentTask().Context());
}
```

リスト 29.16 に Finish() の実装を示します。この関数の役割は現在実行中のタスクを終了することです。主な処理は、現在実行中のタスクをランキュー (running_) とタスク配列 (tasks_) から削除し、終了したことを finish_tasks_ に記録し、終了待ちをしているタスクが finish_waiter_ に登録されていたらそれを起こすことです。上から順に見ていきます。

冒頭では RotateCurrentRunQueue() を使って、ランキューの先頭要素を取り除きます。すなわち、現在実行中のタスク (Finish() を呼び出したタスク) は実行可能状態ではなくなります。この関数の実装は後で紹介します。

次に、tasks_ から現在実行中のタスクを取り除きます。std::find_if() で条件に合致する要素 (タスクオブジェクトへのポインタ) の位置を探し、tasks_.erase() で削除します。こうすることで、完全に現在実行中のタスクがメモリ上から削除されます。

tasks_ の要素はタスクオブジェクトへのポインタです。TaskManager::NewTask() の中で tasks_.emplace_back(new Task{latest_id_}) として生成されたものです (リスト 13.22)。普通、ポインタを削除しただけでは指し示す先のオブジェクトが破棄されず、メモリ上に残ったままになります。そのため、delete 対象オブジェクト ; として明示的に破棄する必要があります。一方、tasks_ の要素はスマートポインタの一種である std::unique_ptr<Task> を使っていますから、要素を削除すると自動的に指す先のオブジェクトも破棄されます。配列の要素としてスマートポインタを使うことで、自分で delete しなくて良いので便利ですね。

細かいですが、tasks_ からタスクを削除する前に current_task->ID() を取得しているのは大

切なことです。tasks_ から削除した後だと、すでに current_task が指すタスクオブジェクトは破棄されてしまっているため、正しくタスク ID を取得できないのです。

話を元に戻します。tasks_ からタスクを削除した後、finish_tasks_ に終了コードを登録します（ここで登録した値は、後で紹介する WaitFinish() が参照します）。次に、finish_waiter_ を参照して、このタスクの終了を待っているタスクがあるかどうかを調べます。そのようなタスクがあれば、Wakeup() を使ってそのタスクを起こします。最後に RestoreContext() を呼び出して、次のタスクに実行を移して処理終了です。RestoreContext() の次の行に戻ってくることはありません。

図 29.4 finish_tasks_ と finish_waiter_ の関係

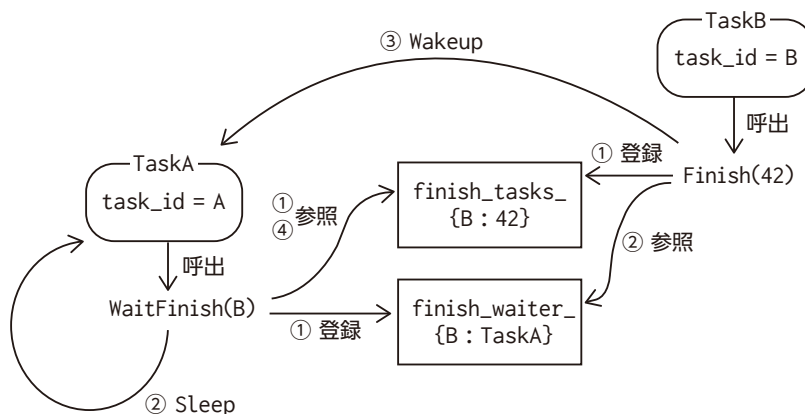


図 29.4 をみながら、finish_tasks_ と finish_waiter_ の使われ方を説明します。パイプの左側のタスクを TaskA、右側のタスクを TaskB とし、TaskB は終了コード 42 で処理を終えるとします。TaskA と TaskB は並行に動いているため、各タスクの中では①②の処理の順序が決まりますが、TaskA の①②と TaskB の①②はどちらが先に実行されるかは分かりません。

TaskB の視点から動作を見てみます。TaskB は渡されたコマンドラインの処理が終わると、Finish() を呼び出してタスクを終了させます。先ほど見たように Finish() はまず finish_tasks_ に自身のタスク ID (B) と終了コード (42) の対応関係を記録します (①)。次に finish_waiter_ を調べ、TaskB の終了を待っているタスクがあるかどうかを調べます (②)。そのようなタスクがあれば起床させます (③)。

今度は TaskA の視点から。TaskA は TaskB の終了を待つために WaitFinish() を呼びます。WaitFinish() は最初に finish_tasks_ を使ってすでに TaskB が終了しているかを調べます。TaskB がまだ終了していなければ、finish_waiter_ に「TaskA が TaskB を待っている」ことを登録し (①)、一旦スリープします (②)。TaskB により起床 (③) した TaskA は、改めて finish_tasks_ を見て TaskB の終了コードを得ます (④)。

リスト 29.17 WaitFinish() は指定したタスクの終了コードを得る (task.cpp)

```

WithError<int> TaskManager::WaitFinish(uint64_t task_id) {
    int exit_code;
    Task* current_task = &CurrentTask();
    while (true) {

```

```

    if (auto it = finish_tasks_.find(task_id); it != finish_tasks_.end()) {
        exit_code = it->second;
        finish_tasks_.erase(it);
        break;
    }
    finish_waiter_[task_id] = current_task;
    Sleep(current_task);
}
return { exit_code, MAKE_ERROR(Error::kSuccess) };
}

```

リスト 29.17 に WaitFinish() の実装を示します。このメソッドは指定したタスクが終了するのを待ち、タスクの終了コードを得ます。このメソッドの処理中にタスク切り替えが起きると酷いことになりますから、このメソッドを実行中は割り込みを禁止しておく必要があります。

酷いことになる例として、TaskA の finish_tasks_.find(task_id) と finish_waiter_[task_id] = current_task; の 2 つの処理の間にタスクが切り替わったときを考えます。前者の処理の後にタスク切り替えが起き、TaskB が実際に終了するとしましょう。TaskB の終了時点では finish_waiter_[task_id] には何も登録されていません。その後、TaskA の実行が再開されると、TaskA はそのまま finish_waiter_[task_id] に自分自身を登録してスリープ状態になります。しかし、その後誰も TaskA を起こさないため、TaskA は永遠にスリープ状態のままです。

SwitchTask() でタスクを切り替えるときと違い、Finish() でタスクを終了するときは「現在のタスク」にコンテキスト保存は不要です。というより、現在のタスクは RestoreContext() が実行される時点 (Finish() の最後) ですでに破棄されており、保存できないというのが正しいですね。

とても長かったですが、パイプに対応するための主な改造を説明し終わりました。これでパイプができるようになっているはずです。実行してみましょう！

図 29.5 パイプを使っていろいろなコマンドを試す

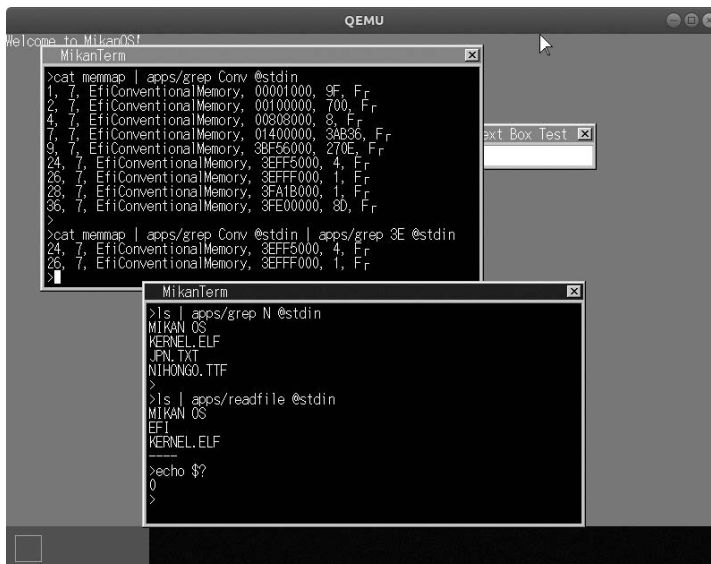


図 29.5 に実行した様子を示します。上側のターミナルは cat の出力を grep で行検索します。2 目のコマンドは cat の結果を 2 回 grep していますが、ちゃんと動いています。このような多段パイプも動くようになっていたとは……。パイプの処理をきちんと実装したおかげですね！ 下側のターミナルでは ls の結果を grep や readfile に渡してみました。うん、これもちゃんと動いているようです。いいですね。

これでパイプの機能については完成なのですが、筆者が使っている USB キーボードでパイプ記号 | が入力できない問題に対処しましたのでご紹介します。筆者のキーボードは日本語配列なのですが、MikanOS では US 配列として認識されます。日本語配列キーボードのエンターキーの真上にある「|¥」というキーは US 配列に存在しません。keycode_map と keycode_map_shifted は今まで、US 配列にあるキーに対応した要素しか値を設定していませんでした。そのため、いくら筆者のキーボードの「|¥」キーを押しても無反応だったのです。

リスト 29.18 手持ちの USB キーボードのパイプ記号に対応する (keyboard.cpp)

```
const char keycode_map_shifted[256] = {
    0,    0,    0,    0,    'A', 'B', 'C', 'D', // 0

    < 中略 >

    0,    '|', 0,    0,    0,    0,    0,    0,    // 136
};
```

「|¥」キーを押してキーコードを調べてみると 137 でした。USB HID の規格（参考文献 [13]）では、137 は Keyboard International3 というキーになっていて、まさに US 配列以外で活躍するキーでした。リスト 29.18 のようにキーコード 137 にパイプ記号を割り当てる改造をすることで、実機でもパイプ記号を入力できるように修正しました。

29.7 sort コマンド (osbook_day29c)

パイプ機能ができたので、新しいコマンドを 1 つ作ってみたいと思います。sort という、文書を行単位でソート（並べ替え）するコマンドです。例えば次のような文書があったときに

```
apple
banana
app
```

次のように並べ替えます。

```
app
apple
banana
```

これだけだとイマイチなんの役に立つかわからないかもしれませんが、汎用的なコマンドなので応用は無限大です。例えば、ある記事に現れる単語の出現頻度を数えたいなーというときに、1 単語 1 行に抽出したファイルを用意して（ここは別コマンドで）、それをソートすれば、ある単語が何回表れたの

かを簡単に数えられますね。まあ、とにかく作ってみます。

リスト 29.19 sort コマンドは行単位で並べ替える (sort/sort.cpp)

```
#include <cstdio>
#include <cstdlib>
#include <string>
#include <vector>

extern "C" void main(int argc, char** argv) {
    FILE* fp = stdin;
    if (argc >= 2) {
        fp = fopen(argv[1], "r");
        if (fp == nullptr) {
            fprintf(stderr, "failed to open '%s'\n", argv[1]);
            exit(1);
        }
    }

    std::vector<std::string> lines;
    char line[1024];
    while (fgets(line, sizeof(line), fp)) {
        lines.push_back(line);
    }

    auto comp = [](const std::string& a, const std::string& b) {
        for (int i = 0; i < std::min(a.length(), b.length()); ++i) {
            if (a[i] < b[i]) {
                return true;
            } else if (a[i] > b[i]) {
                return false;
            }
        }
        return a.length() < b.length();
    };

    std::sort(lines.begin(), lines.end(), comp);
    for (auto& line : lines) {
        printf("%s", line.c_str());
    }
    exit(0);
}
```

リスト 29.19 に sort コマンドの実装を示します。sort はコマンドライン引数でファイル名を取りますが、ファイル名を省略すると標準入力からデータを読み取るように工夫しました。パイプを使うときにいちいち @stdin と書かなくて良くなり、便利です。ファイル名を省略できるのはとても便利なので、よく使う grep も改造しておきました。

図 29.6 sort を試す

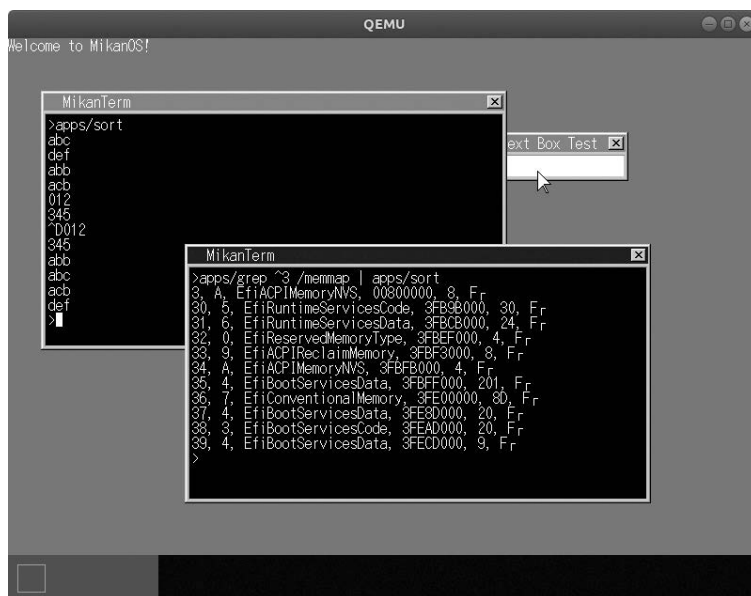


図 29.6 は sort コマンドを使っていろいろな文書を並び替えてみた様子です。左のターミナルでは、キーボードから入力した 6 行を並べ替えています。6 行を入力して Ctrl-D を押すと、その後からソート結果が出力されました。ソート結果は期待通りです。右のターミナルでは、grep と sort を組み合わせ、先頭の文字が「3」の行だけを対象にソートしています。こちらも正しそうです。

Linux などに入っている sort コマンドはずっと多機能です。例えば、文字としてではなく数値として並べ替えるとか、並べ替えた結果、重複した行を削除するといったこともできます。それらを作り始めると紙面が足りませんので本書では作らないことにします。興味があれば読者の皆さんが作ってみてください。

29.8 ターミナルのバグ修正 (osbook_day29d)

パイプを使っていろいろ遊んでいて気が付いたのですが、コマンド実行中にターミナルの最終行しか更新されないように見えます。ターミナルがスクロールするくらいの大きな文書を表示すると、ターミナル全体がスクロールしてほしいところ、最終行以外が静止したままになってしまいます。気持ち悪いのでこのバグだけ直して、この章を終えたいと思います。

リスト 29.20 文字列を表示するたびに再描画する (terminal.cpp)

```
size_t TerminalFileDescriptor::Write(const void* buf, size_t len) {
    term_.Print(reinterpret_cast<const char*>(buf), len);
    term_.Redraw();
    return len;
}
```

ソースコードを眺めて原因を考えていたら見つけました。TerminalFileDescriptor::Write() が文字列を表示した後に画面を再描画していないのです。本来であれば、メインタスクに再描画メッセージを投げないといけません。ということでリスト 29.20 のように修正し、文字列表示後に再描画するようにしました。

リスト 29.21 エコーバックするときも再描画する (terminal.cpp)

```
size_t TerminalFileDescriptor::Read(void* buf, size_t len) {
    char* bufc = reinterpret_cast<char*>(buf);

    while (true) {

< 中略 >

        bufc[0] = msg->arg.keyboard.ascii;
        term_.Print(bufc, 1);
        term_.Redraw();
        return 1;
    }
}
```

これだけでは不十分です。なぜならエコーバック処理の際にもターミナルに対して表示処理が行われるからです。そのときにも画面の再描画を行うようにします (リスト 29.21)。

リスト 29.22 Redraw() はターミナル全体を再描画する (terminal.cpp)

```
void Terminal::Redraw() {
    Rectangle<int> draw_area{TopLevelWindow::kTopLeftMargin,
                             window_->InnerSize()};

    Message msg = MakeLayerMessage(
        task_.ID(), LayerID(), LayerOperation::DrawArea, draw_area);
    __asm__("cli");
    task_manager->SendMessage(1, msg);
    __asm__("sti");
}
```

Terminal::Redraw() の実装はリスト 29.22 のようになっています。ターミナルの領域全体を再描画するようにメインタスクに依頼します。

このままだと cat コマンドの動作がとても遅いことに気づきました。でも、cat memmap | apps/grep . というように、パイプを通すと大幅に高速化します。おそらく、cat が直接表示する際は 1 文字ずつの表示で、パイプを通すと Newlib の機能で 1 行ずつの表示になる^{*1} ことの差だと思います。

1 Newlib の出力ライブラリ (FILE を中心とした機能群) は、表示文字列を一時的にためることで write() の呼び出し、すなわちシステムコールを減らし、高速化する機能を持っています。

リスト 29.23 cat が 1 行ずつ表示するようにする (terminal.cpp)

```
fat::FileDescriptor fd{*file_entry};
char u8buf[1024];
DrawCursor(false);
while (true) {
    if (ReadDelim(fd, '\n', u8buf, sizeof(u8buf)) == 0) {
        break;
    }
    PrintToFD(*files_[1], "%s", u8buf);
}
DrawCursor(true);
```

そこでリスト 29.23 のように cat コマンドを改造して、1 行ずつ表示するようにしました。ReadDelim() を使って '\n' まで読み取り、それを PrintToFD() で出力します。ReadDelim() は、指定したファイルディスクリプタから、指定した文字（ここでは '\n'）に出会うまで 1 バイトずつ読み取る関数です。file.cpp に定義しました。実装は簡単なので説明は省略します。

図 29.7 1 行毎にターミナルが再描画される様子

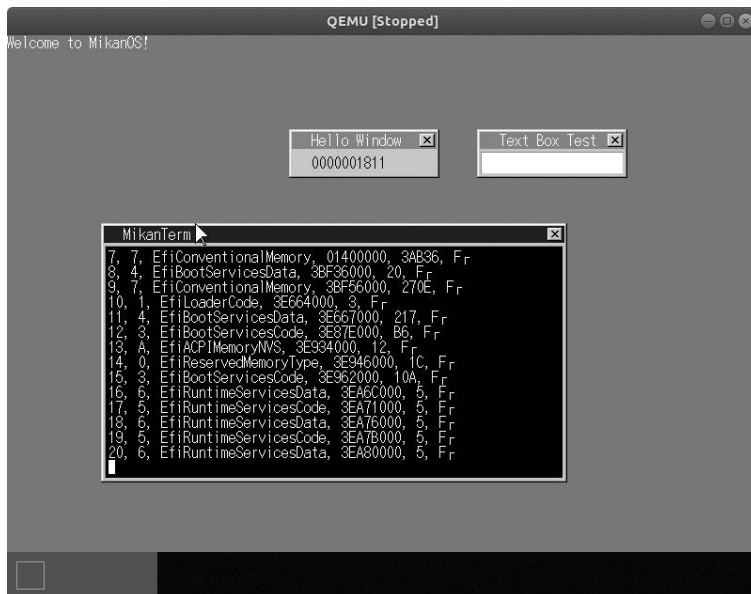


図 29.7 に cat コマンドで memmap を表示している途中の状態を載せました。このスクリーンショットを普通に撮るのは難しいので、GDB を使ってブレークポイントを仕掛けて撮りました。PrintToFD() でブレークしつつ様子を見ると、1、2 行に 1 回、画面が更新されていく様子が分かります。1 行ずつじっくり更新されないのは、画面更新をメインタスクに任せているためでしょう。GDB で止めず普通に実行するとめぬめぬに動きますので、これで完成とします。

29.9 共有メモリ

共有メモリとはページングの仕組みを使ったアプリ間通信の方法です。同じ物理フレームを複数のアプリの仮想アドレスにマップします。すべてのアプリは OS 用のアドレス空間を共有していますが、原則としてアプリ用のアドレス空間（0xffff800000000000 以降）については互いに独立しています。例えば、アプリ A のアドレス 0xffff800000000000 とアプリ B のアドレス 0xffff800000000000 は別の物理フレームにマップされ、共有されません。共有メモリは、例外的に共有できるページを作り出す技術です。

「27.4 コピーオンライト」のように穴をあけることは可能です。コピーオンライトでは、同じ実行ファイルを用いるアプリを複数起動させるときに同じ物理フレームを参照するようにしました。ただ、コピーオンライトでは、そのページに書き込もうとすると別の物理フレームに内容がコピーされてしまいますから、複数のアプリで同じデータを**読み書き**することはできません。

共有メモリはコピーオンライトと似ていますが、物理フレームを仮想ページにマップする際に書き込み許可とする点が違います。書き込み許可でマップすることで、複数のアプリが同じ変数を共有できます。パイプの場合は一方通行の通信でしたが、共有メモリでは通信の方向は決まっています。プログラマの発想次第でどのような使い方もできる、とても汎用的な仕組みです。配列や構造体を共有メモリ上に置くことができますから、パイプが単なるバイト列の送信だったのに比べて高度な表現がしやすいですね。

共有メモリを使うときは**データ競合**（data race）に気を付ける必要があります。これはアプリに限った話ではなく、OS 本体の中であっても起き得ることです。例えば、今までも task_manager を触る間は割り込みを禁止して、タスクが切り替わらないように注意していました。これと同じことが共有メモリでも当然発生するわけです。片方のアプリがデータを変更している最中にタスクが切り替わり他方のアプリが同じ変数を参照すると、不整合なデータが読めてしまいます。

第 30 章

おまけアプリ

いよいよ最終章です。MikanOS はもうすっかり OS っぽくなりました。この章では、いくつかのアプリを作って締めくくりにしようかなと思います。標準出力をページ単位で表示する `more` コマンド、大画面でテキストファイルを閲覧するテキストビューア、画像ファイルを表示する画像ビューアを作ってみようと思います。おまけで3つの改良をしようと思います。1つ目はいちいち `apps/` を付けなくてもアプリを起動できる仕組みです。2つ目は `cat` を標準入力に対応させることです。この機能があると好きな内容のテキストファイルを新規作成できるようになっている便利です。3つ目はウィンドウの右上に表示されている「閉じる」ボタンの有効化です。今まではただの絵でしたが、これを使えるようにしてみます。

30.1 アプリにパスを通す (osbook_day30a)

いちいち apps/grep などと apps/ を付けるのが面倒に思えてきました。Linux ではアプリ名をそのまま書くとアプリを実行できます。同じように、MikanOS でも単に grep と書くだけでアプリを実行できるようにしたら便利そうです。

Linux ではそれを環境変数という仕組みで実現しています。Linux のターミナル（正確にはシェル）には変数が備わっていて、その中でも PATH という特別な変数を利用してディレクトリ名なしでアプリを指定できる仕組みが実現されています。PATH にディレクトリへのパス（良くあるのは /usr/bin など）を設定しておくことで、そのディレクトリに入っているアプリであればアプリ名だけで起動できます。

環境変数の仕組みをちゃんと作り込むのは結構大変な気がしますので、この節では apps/ なしでアプリを起動できることに絞って実装してみます。

リスト 30.1 FindCommand() はアプリを apps から探す (terminal.cpp)

```
fat::DirectoryEntry* FindCommand(const char* command,
                                unsigned long dir_cluster = 0) {
    auto file_entry = fat::FindFile(command, dir_cluster);
    if (file_entry.first != nullptr &&
        (file_entry.first->attr == fat::Attribute::kDirectory ||
         file_entry.second)) {
        return nullptr;
    } else if (file_entry.first) {
        return file_entry.first;
    }

    if (dir_cluster != 0 || strchr(command, '/') != nullptr) {
        return nullptr;
    }

    auto apps_entry = fat::FindFile("apps");
    if (apps_entry.first == nullptr ||
        apps_entry.first->attr != fat::Attribute::kDirectory) {
        return nullptr;
    }
    return FindCommand(command, apps_entry.first->FirstCluster());
}
```

ということで、早速 FindCommand() という関数を作ってみました（リスト 30.1）。この関数は、引数で与えられたアプリをまずはそのまま検索し、見つからなかったら apps の中から探します。

リスト 30.2 FindCommand() を使ってアプリを検索する (terminal.cpp)

```
} else if (command[0] != 0) {
    auto file_entry = FindCommand(command);
    if (!file_entry) {
        PrintToFD(*files_[2], "no such command: %s\n", command);
        exit_code = 1;
    } else {
        auto [ ec, err ] = ExecuteFile(*file_entry, command, first_arg);
    }
}
```

Terminal::ExecuteLine() の中でアプリを起動する部分をリスト 30.2 に示します。先ほど作った FindCommand() を使ってアプリを検索するようにしました。特に難しいところはないかなーと思います。

図 30.1 apps/ なしで各種のアプリを起動している様子



図 30.1 に実行した様子を示します。apps/ を付けても付けなくてもアプリが起動できていることが分かります。

30.2 more コマンド (osbook_day30b)

アプリの一覧を見ようかなーと思って ls apps としてみました。しかし、アプリがたくさんありすぎて始めの方が隠れてしまい、アプリすべてを見ることができませんでした。アプリがたくさんあるのは嬉しいのですが、全部を確認できないのはちょっと不便です。

Linux には more や less というコマンドがあって、ファイルの内容をページ送りしながら読めます。パイプを使うと次のように標準出力をページ送りで読めます。

```
$ ls /usr/bin | more
```

MikanOS でも more コマンドを作ってみることにします。

リスト 30.3 ファイルをページ送りする more コマンド (more/more.cpp)

```

#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <string>
#include <vector>
#include "../syscall.h"

AppEvent WaitKey() {
    AppEvent events[1];
    while (true) {
        auto [ n, err ] = SyscallReadEvent(events, 1);
        if (err) {
            fprintf(stderr, "ReadEvent failed: %s\n", strerror(err));
            exit(1);
        }

        if (events[0].type == AppEvent::kQuit) {
            exit(0);
        }
        if (events[0].type == AppEvent::kKeyPush &&
            events[0].arg.keypush.press) {
            return events[0];
        }
    }
}

extern "C" void main(int argc, char** argv) {
    int page_size = 10;
    int arg_file = 1;
    if (argc >= 2 && argv[1][0] == '-' && isdigit(argv[1][1])) {
        page_size = atoi(&argv[1][1]);
        ++arg_file;
    }

    FILE* fp = stdin;
    if (argc > arg_file) {
        fp = fopen(argv[arg_file], "r");
        if (fp == nullptr) {
            fprintf(stderr, "failed to open '%s'\n", argv[arg_file]);
            exit(1);
        }
    }

    std::vector<std::string> lines{};
    char line[256];
    while (fgets(line, sizeof(line), fp)) {
        lines.emplace_back(line);
    }

    for (int i = 0; i < lines.size(); ++i) {
        if (i > 0 && (i % page_size) == 0) {
            fputs("---more---\n", stderr);
            WaitKey();
        }
    }
}

```

```

    fputs(lines[i].c_str(), stdout);
}
exit(0);
}

```

リスト 30.3 に more コマンドの実装を示します。このコマンドの引数は more [-n] [<file>] となっています。-n はページ送りの行数を指定するオプションで、n には数字が入ります。つまり 3 行ずつページ送りするには -3 と指定します。<file> にはページ送りしたいファイルパスを指定します。省略すると標準入力から読み取ります。

メイン関数の前半ではコマンドライン引数を解析し、ページ送りの行数 page_size とファイル fp を決定します。次にファイル fp からすべての行を読み取り、lines に格納します。最後に一定の行数ずつ表示してキー入力を待つ、という処理を繰り返します。

WaitKey() は ReadEvent() システムコールを使って（久々ですね、これを使うのは）キー入力を待つ関数です。何らかのキーが押された瞬間に WaitKey() から処理が戻り、次の 1 ページの表示処理が再開されます。

これで動くかなーと思って実行してみたところ、いくらキーを押しても無反応です。何が足りないのかちょっと考えてみたら気づきました。パイプを使うと裏でターミナルタスクが起動しますが、キー押下イベントはあくまでもパイプの左側に対応するタスクに送られてしまうからです。この原因は、キー押下イベントを送る先のタスクを登録する表 layer_task_map に、パイプの左側のタスクが登録されてしまっていることです。普段はそれで良いのですが、パイプを使っている間は、各種イベントがパイプの右側のタスクに送信されてほしいです。

リスト 30.4 パイプを使っている間はイベント送信先を変える (terminal.cpp)

```

if (pipe_char) {
    *pipe_char = 0;

    < 中略 >

    subtask_id = subtask
        .InitContext(TaskTerminal, reinterpret_cast<int64_t>(term_desc))
        .Wakeup()
        .ID();
    (*layer_task_map)[layer_id_] = subtask_id;
}

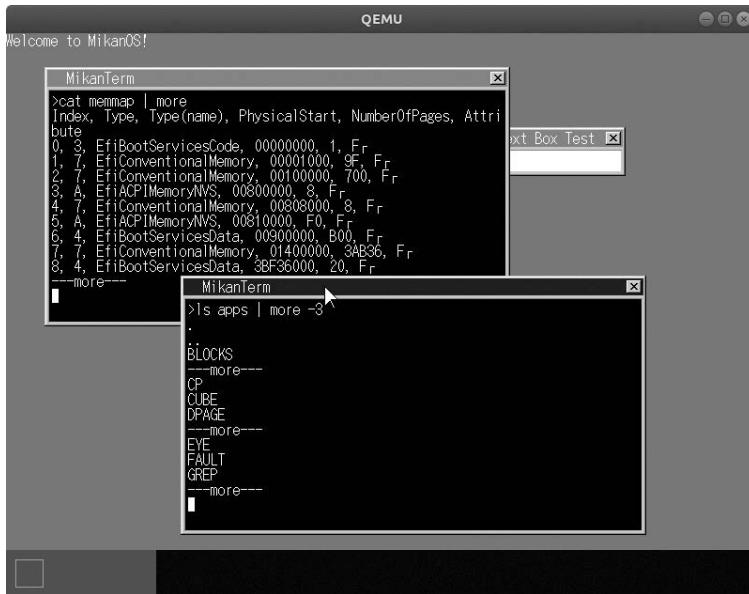
< 中略 >

if (pipe_fd) {
    pipe_fd->FinishWrite();
    __asm__("cli");
    auto [ ec, err ] = task_manager->WaitFinish(subtask_id);
    (*layer_task_map)[layer_id_] = task_.ID();
    __asm__("sti");
}

```

リスト 30.4 のように、パイプを使っている間だけイベントの送信先を切り替える処理を入れてみました。layer_task_map の登録を一時的に更新し、パイプの処理中に送られてきたイベントをすべてパイプの右側のタスク (subtask_id) に送信するようにします。

図 30.2 more コマンドでいろいろな出力をページ単位で見る



more を実行した様子を図 30.2 に示します。cat や ls の出力がページ単位で表示されているのが分かります。

30.3 cat を入力に対応させる (osbook_day30c)

cat コマンドは今のところ標準入力を扱えません。ターミナルの組み込みコマンドですので @stdin という記法も使えませんし、ファイル名を省略したときに標準入力から読み取る機能もありません。

cat が標準入力を扱えるようになると、改行を含むテキストファイルが作れるようになります。cat > foo というように cat の出力をファイルにリダイレクトするのです。テキストファイルが作れるようになると、他のコマンドの動作確認に使うファイルをその場で作れるようになります。いちいち起動ボリュームにファイルを入れて MikanOS を再起動させる必要がありません。

リスト 30.5 cat でファイル名を省略したら標準入力を使う (terminal.cpp)

```

} else if (strcmp(command, "cat") == 0) {
    std::shared_ptr<FileDescriptor> fd;
    if (!first_arg || first_arg[0] == '\0') {
        fd = files[0];
    } else {
        auto [ file_entry, post_slash ] = fat::FindFile(first_arg);
        if (!file_entry) {
            PrintToFD(*files[2], "no such file: %s\n", first_arg);
            exit_code = 1;
        } else if (file_entry->attr != fat::Attribute::kDirectory && post_slash) {
            char name[13];

```

```

        fat::FormatName(*file_entry, name);
        PrintToFD(*files_[2], "%s is not a directory\n", name);
        exit_code = 1;
    } else {
        fd = std::make_shared<fat::FileDescriptor>(*file_entry);
    }
}
if (fd) {
    char u8buf[1024];
    DrawCursor(false);
    while (true) {
        if (ReadDelim(*fd, '\n', u8buf, sizeof(u8buf)) == 0) {
            break;
        }
        PrintToFD(*files_[1], "%s", u8buf);
    }
    DrawCursor(true);
}
} else if (strcmp(command, "noterm") == 0) {

```

リスト 30.5 のように改造しました。コマンドライン引数 `first_arg` が空のときに、標準入力を表すファイルディスクリプタ `files_[0]` を使うようにする、というのが主な改造ポイントです。その他の部分はほとんど改造していません。

リスト 30.6 コマンドと引数の間に空白が 2 文字以上あったときに対応する (terminal.cpp)

```

if (first_arg) {
    *first_arg = 0;
    do {
        ++first_arg;
    } while (isspace(*first_arg));
}

```

いろいろコマンドを実行していて気付いたのですが、コマンド名と引数の間に 2 個以上の空白文字があると引数がうまく認識されません。これは、引数を指す `first_arg` の先頭が空白文字のときに調整する処理が不十分だからです。**リスト 30.6** のように、ループ (do-while 文) で空白を取り除くようにして、この問題を解決しました。

図 30.3 cat を使ってテキストファイル foobar を作る

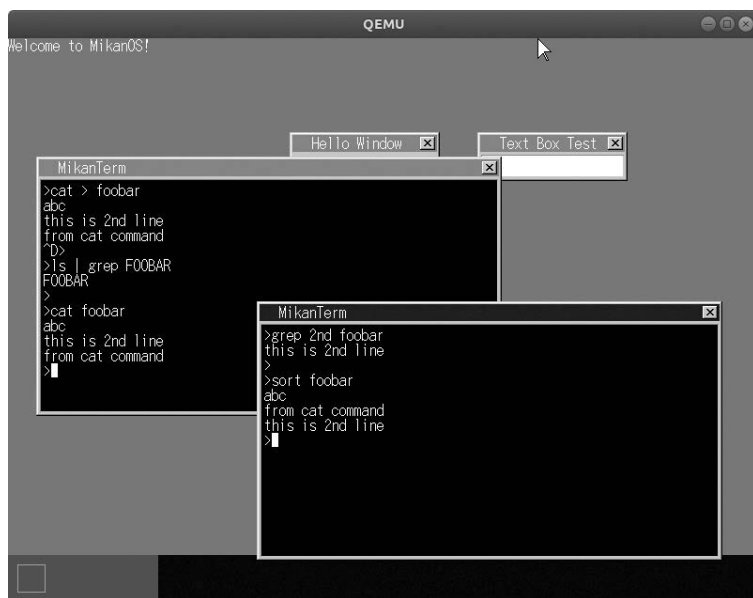


図 30.3 を見てください。左側のターミナルでは、cat とリダイレクトを組み合わせ、foobar という 3 行のテキストファイルを作りました。右側のターミナルでは、作成したファイルを使って、grep と sort の動作確認を行いました。cat が標準入力を扱えるようになり、簡易なテキストが作れるようになりました。テキストエディタのような高度なテキスト作成はできませんし、入力途中で間違えると修正できないなど、相当不便ではありますが、簡易に試す用途には重宝するでしょう。

30.4 閉じるボタン (osbook_day30d)

この節では、今までは単なる絵だった閉じるボタンを、ちゃんと機能するようにしてみます。まず、絶対に必要なのは閉じるボタンがクリックされたことの判定です。ウィンドウの上でマウスの左ボタンがクリックされたとき、座標が閉じるボタンの上にあるかどうかを判定します。この判定処理に加え、実際にウィンドウを閉じる方法は大きく 2 つ考えられます。1 つは外部から強制的にウィンドウを閉じる方法。もう 1 つはそのウィンドウを管轄しているタスクに「ウィンドウを閉じてほしい」というメッセージを送り、自主的に閉じてもらう方法です。

前者（外部からウィンドウを閉じる方法）は危険性があります。ウィンドウが閉じられたことをタスクが知らないまま処理を続けてしまうと、存在しないウィンドウへ絵を書き込もうとしてしまったりするからです。後者の方法なら、タスクが自主的にウィンドウを閉じますから安全です。ということで後者の方法を実装していきます。

手始めに、クリックされた座標が、閉じるボタンの上にあるかどうかを判定する機能を作ります。

リスト 30.7 GetWindowRegion() は座標から領域の種類を判定する (window.hpp)

```
virtual WindowRegion GetWindowRegion(Vector2D<int> pos);
```

Window クラスに、ウィンドウ内の座標からウィンドウ領域の種類を判定するメソッド GetWindowRegion() を実装しました (リスト 30.7)。このメソッドは仮想メソッドとし、子クラスでオーバーライドする想定です。

リスト 30.8 ウィンドウ領域の種類を表す WindowRegion 列挙体 (window.hpp)

```
enum class WindowRegion {
    kTitleBar,
    kCloseButton,
    kBorder,
    kOther,
};
```

このメソッドの戻り値の型である WindowRegion 列挙体の定義をリスト 30.8 に示します。列挙子の名前を見れば大体分かると思いますが、上からそれぞれタイトルバー (ウィンドウのタイトルが表示される部分)、閉じるボタン、ウィンドウの枠線、その他の領域、を表します。GetWindowRegion() にウィンドウ左上を基準とした座標を渡すと、その座標がどの領域に相当するかを判定し、いずれかの値が返されるという仕組みです。

リスト 30.9 Window::GetWindowRegion() の実装 (window.cpp)

```
WindowRegion Window::GetWindowRegion(Vector2D<int> pos) {
    return WindowRegion::kOther;
}
```

Window クラスでは領域の区別はありませんから、GetWindowRegion() メソッドは座標に関係なく常に kOther を返すようにしておきます (リスト 30.9)。

リスト 30.10 ToplevelWindow::GetWindowRegion() の実装 (window.cpp)

```
WindowRegion ToplevelWindow::GetWindowRegion(Vector2D<int> pos) {
    if (pos.x < 2 || Width() - 2 <= pos.x ||
        pos.y < 2 || Height() - 2 <= pos.y) {
        return WindowRegion::kBorder;
    } else if (pos.y < kTopLeftMargin.y) {
        if (Width() - 5 - kCloseButtonWidth <= pos.x && pos.x < Width() - 5 &&
            5 <= pos.y && pos.y < 5 + kCloseButtonHeight) {
            return WindowRegion::kCloseButton;
        }
        return WindowRegion::kTitleBar;
    }
    return WindowRegion::kOther;
}
```

リスト 30.10 に `ToplevelWindow::GetWindowRegion()` の実装を示します。ウィンドウの周囲 2 ピクセルは枠線 (`kBorder`) と判定します。Y 座標が `kTopLeftMargin.y` より小さい範囲をタイトルバー (`kTitleBar`)、その中の右端にある小さな長方形を閉じるボタン (`kCloseButton`) として判定します。

リスト 30.11 ウィンドウの領域の種類に応じて処理を変える (mouse.cpp)

```
const auto pos_layer = position_ - layer->GetPosition();
switch (layer->GetWindow()->GetWindowRegion(pos_layer)) {
case WindowRegion::kTitleBar:
    drag_layer_id_ = layer->ID();
    break;
case WindowRegion::kCloseButton:
    close_layer_id = layer->ID();
    break;
default:
    break;
}
```

リスト 30.11 に、左ボタンをクリックしたときの判定処理を示しました。先ほど追加したメソッド `GetWindowRegion()` の戻り値によって処理を切り替えます。タイトルバーをクリックされた場合はウィンドウのドラッグ処理、閉じるボタンをクリックされた場合は閉じる処理とします。 `drag_layer_id_` と `close_layer_id` のどちらに値を代入するかで、その後の処理を切り替えることにしました。

リスト 30.12 マウスクリックの処理の全体像 (mouse.cpp)

```
unsigned int close_layer_id = 0;

const bool previous_left_pressed = (previous_buttons_ & 0x01);
const bool left_pressed = (buttons & 0x01);
if (!previous_left_pressed && left_pressed) {
    auto layer = layer_manager->FindLayerByPosition(position_, layer_id_);
    if (layer && layer->IsDraggable()) {
        const auto pos_layer = position_ - layer->GetPosition();
        switch (layer->GetWindow()->GetWindowRegion(pos_layer)) {
        case WindowRegion::kTitleBar:
            drag_layer_id_ = layer->ID();
            break;
        case WindowRegion::kCloseButton:
            close_layer_id = layer->ID();
            break;
        default:
            break;
        }
        active_layer->Activate(layer->ID());
    } else {
        active_layer->Activate(0);
    }
} else if (previous_left_pressed && left_pressed) {
    if (drag_layer_id_ > 0) {
        layer_manager->MoveRelative(drag_layer_id_, posdiff);
    }
}
```

```

    }
} else if (previous_left_pressed && !left_pressed) {
    drag_layer_id_ = 0;
}

if (drag_layer_id_ == 0) {
    if (close_layer_id == 0) {
        SendMouseMessage(newpos, posdiff, buttons, previous_buttons_);
    } else {
        SendCloseMessage();
    }
}
}

```

先ほど示した `GetWindowRegion()` を使った条件分岐を含む、マウスクリックに関する処理の全体像を **リスト 30.12** に示します。先ほどの条件分岐処理は、左ボタンが押された瞬間 (`if (!previous_left_pressed && left_pressed)`) の処理に含まれています。

最後の `if` 文で、閉じるボタンがクリックされたのか、それ以外の部分がクリックされたのかで処理を分けます。閉じるボタンがクリックされた場合、新しく作った `SendCloseMessage()` を呼び、それ以外であれば今まで通りの処理を行います。`SendCloseMessage()` は、現在アクティブになっているウィンドウ（に紐づけられているタスク）に対し、閉じるボタンがクリックされた旨を通知します。

リスト 30.13 `SendCloseMessage()` は閉じるボタンのクリックを通知する (mouse.cpp)

```

void SendCloseMessage() {
    const auto [ layer, task_id ] = FindActiveLayerTask();
    if (!layer || !task_id) {
        return;
    }

    Message msg{Message::kWindowClose};
    msg.arg.window_close.layer_id = layer->ID();
    task_manager->SendMessage(task_id, msg);
}

```

リスト 30.13 に `SendCloseMessage()` の実装を示します。この関数は、現在アクティブになっているウィンドウに関連づけられているタスク (`task_id`) に対し、`kWindowClose` メッセージを送信します。この中で使っている `FindActiveLayerTask()` は、現在アクティブなウィンドウを含むレイヤと、そのレイヤが関連付けられているタスクを取得する関数です。**リスト 23.2** の前半部分を単体の関数として抜き出したただけですので、コード紹介は省略します。

リスト 30.14 メッセージ構造体に `kWindowClose` を追加 (message.hpp)

```

struct {
    unsigned int layer_id;
} window_close;

```

メッセージの種別として `Message::kWindowClose` を追加しました。そして、その種別に対応する引数を追加しました (**リスト 30.14**)。引数は、どのウィンドウ (レイヤ) が閉じられたか分かるように、レイヤ ID を持たせてみました。

この引数を作っていてふと思いました。他の引数 (mouse_button や window_active) にもレイヤ ID を持たせるべきだったと。今のところ、1つのアプリは最大1つのウィンドウしか持ちません。なのでレイヤ ID を渡さなくても問題ありませんでした。しかし、もし複数のウィンドウを持つアプリを作るなら、どのウィンドウがクリックされたとか、どのウィンドウがアクティブになったとかを識別できないと困ります。必要に応じて、読者の皆さんの手で拡張してみてください。

さて、ここまでの改造で、閉じるボタンをクリックしたら対応するタスクに kWindowClose メッセージが通知されるようになりました。次は受け側のタスクを改造し、そのメッセージに反応する処理を追加します。

リスト 30.15 kWindowClose を受け取ったらウィンドウを閉じる (terminal.cpp)

```
case Message::kWindowClose:
    CloseLayer(msg->arg.window_close.layer_id);
    __asm__("cli");
    task_manager->Finish(terminal->LastExitCode());
    break;
```

リスト 30.15 にターミナルタスクで受け取ったメッセージを処理する部分に追加した行を示します。kWindowClose を受け取ったらターミナルウィンドウを閉じ、その後ターミナルタスクを終了します。

リスト 30.16 CloseLayer() は指定されたレイヤを閉じる (layer.cpp)

```
Error CloseLayer(unsigned int layer_id) {
    Layer* layer = layer_manager->FindLayer(layer_id);
    if (layer == nullptr) {
        return MAKE_ERROR(Error::kNoSuchEntry);
    }

    const auto pos = layer->GetPosition();
    const auto size = layer->GetWindow()->Size();

    __asm__("cli");
    active_layer->Activate(0);
    layer_manager->RemoveLayer(layer_id);
    layer_manager->Draw({pos, size});
    layer_task_map->erase(layer_id);
    __asm__("sti");

    return MAKE_ERROR(Error::kSuccess);
}
```

CloseLayer() は今回新しく作った関数で、指定したレイヤを閉じます。実装はリスト 30.16 のようになっています。同じような処理を syscall::CloseWindow() でやっていたので、共通化するために関数として切り出しました。

リスト 30.17 ReadEvent() も kWindowClose に対応させる (syscall.cpp)

```

case Message::kWindowClose:
    app_events[i].type = AppEvent::kQuit;
    ++i;
    break;

```

最後に `syscall::ReadEvent()` を改造して、`kWindowClose` を受け取ったときにアプリに `kQuit` イベントを通知するようにしました (リスト 30.17)。これで、`kQuit` イベントを認識するアプリであれば、閉じるボタンに反応するようになるはずです。

図 30.4 ターミナルウィンドウを閉じてみた



初期ターミナルを閉じてみました (図 30.4)。デスクトップにターミナルが1つもないと、何だか寂しいですね……。ウィンドウを閉じる様子はスクリーンショットにし難いのですが、アプリのウィンドウもちゃんと閉じられるようになっていましたよ。

30.5 テキストビューア (osbook_day30e)

今ある OS の機能を利用して役に立つアプリを作ろうと思いました。そうですね……。大きな画面で文書を読むためのアプリ、なんてどうでしょう？ この章の最初の方で `more` コマンドは作りましたが、ページ送りしかできず、また画面の大きさを変えることも不可能です。文書をもっと快適に読めるアプリを作ってみようと思います。文書、特にプレーンテキスト (文字装飾がない、純粋なテキストファイル) を読むためのソフトウェアを「テキストビューア」(text viewer) と呼ぶので、アプリ名を `tview` としてみました。

tview には次の機能を持たせます。

- ・前後にスクロールする機能：上下矢印キーで1行ずつ、PageUp/PageDown キーで画面の半分ずつ、前後にスクロールできる。
- ・画面の大きさを変える機能：コマンドラインで -w 幅 -h 高さ と指定することで、画面の大きさを変えられる。デフォルトは 80 桁×20 行。
- ・タブ幅を変える機能：コマンドラインで -t タブ幅 と指定することで、タブ幅を変えられる。デフォルトは 8 桁。

リスト 30.18 tview のメイン関数 (tview/tview.cpp)

```
extern "C" void main(int argc, char** argv) {
    auto print_help = [argv]() {
        fprintf(stderr,
            "Usage: %s [-w WIDTH] [-h HEIGHT] [-t TAB] <file>\n",
            argv[0]);
    };

    int opt;
    int width = 80, height = 20, tab = 8;
    while ((opt = getopt(argc, argv, "w:h:t:")) != -1) {
        switch (opt) {
            case 'w': width = atoi(optarg); break;
            case 'h': height = atoi(optarg); break;
            case 't': tab = atoi(optarg); break;
            default:
                print_help();
                exit(1);
        }
    }
    if (optind >= argc) {
        print_help();
        exit(1);
    }

    const char* filepath = argv[optind];
    const auto [ fd, content, filesize ] = MapFile(filepath);

    const char* last_slash = strrchr(filepath, '/');
    const char* filename = last_slash ? &last_slash[1] : filepath;
    const auto layer_id = OpenTextWindow(width, height, filename);

    const auto lines = FindLines(content, filesize);

    int start_line = 0;
    while (true) {
        DrawLines(lines, start_line, layer_id, width, height, tab);
        if (UpdateStartLine(&start_line, height, lines.size())) {
            break;
        }
    }

    SyscallCloseWindow(layer_id);
}
```

```
    exit(0);  
}
```

リスト 30.18 に `tview` のメイン関数を示します。メイン関数になるべく短くなるように関数を分けまくっていますので、一見するとシンプルに見えます。でも `tview.cpp` 全体で 250 行程度あり、割と大きいプログラムです。メイン関数の前半では `getopt()` を使ってコマンドライン引数の解析をし、後半ではファイルをメモリマップし、ウィンドウを開き、各行を描画します。後ろの方にある `while` ループでは描画してキー入力イベントを待つというのを繰り返します。

`getopt()` は初登場の関数ですね。これは `unistd.h` に定義されている、POSIX の標準的な関数です。C 言語の標準関数ではありませんが、Newlib に含まれているのでありがたく使わせてもらいました。これはコマンドライン引数のオプションを解析するための関数です。

オプションというのは、指定しなくてもいいし、指定すると何らかの動作を変更できる引数のことです。`tview` の画面の大きさやタブ幅は、デフォルト値のまま満足なら省略したいですし、変えたいときに変えられれば十分です。ということで、指定が必須の引数ではなく、オプションとして作りたいです。例えば `tview hoge` のように、オプションなしで起動するとデフォルトの画面サイズ、タブ幅でテキストを表示してほしいです。また、`tview -w 30 -h 10 hoge` とすれば画面サイズ 30 × 10、タブ幅はデフォルト値で表示するようにしたいです。

ちょっと考えたらこれは相当に面倒な処理です。コマンドライン引数の数や並び方がオプションの指定によって変わりますから、それを考慮して `argv` を扱う必要があります。実は `getopt()` を使うと、そのようなオプションが簡単に実現できるのです。

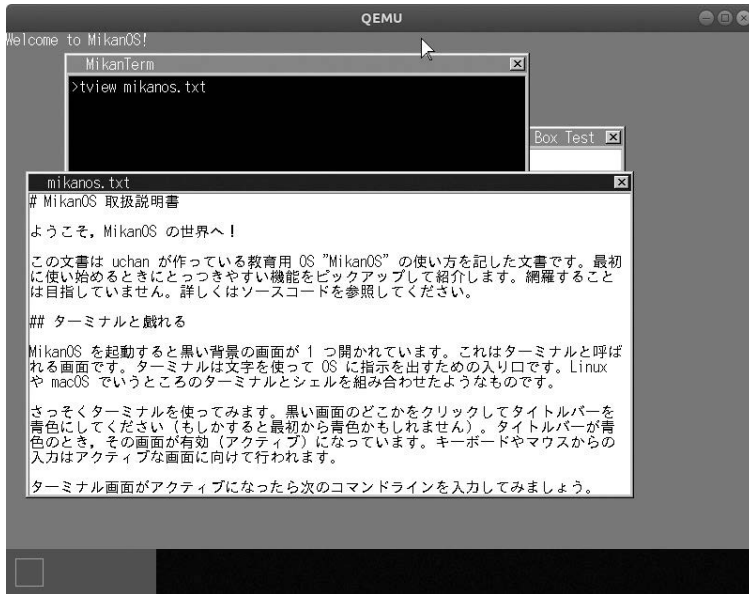
`getopt()` の引数のうち、最も気になるのは第 3 引数に渡している `"w:h:t:"` の意味でしょう。`w` はコマンドラインオプション `-w` を定義していて、その後ろにある `:` はそのオプションが値を取ることを意味します。すなわち `"w:"` は `-w 80` のように値を取るオプション `-w` を作り出します。残りの `"h:t:"` も同様に、値付きの 2 つのオプション `-h`、`-t` を定義しています。

`getopt()` はオプション文字を返します。オプション文字は `w`、`h`、`t` のいずれかですから、`switch` 文で場合分けしてそれぞれの文字に対応した処理を行います。`optarg` は `getopt()` が管理するグローバル変数で、`getopt()` が最後に返したオプションの引数が文字列として格納されます。

`getopt()` が `-1` を返したらそれ以上オプションがないことを意味します。`optind` は、オプションの解析が終わった時点の `argv` の添え字を表す値になっています。この値が `argc` 以上である場合、オプション以外の引数がない、すなわち `tview` にファイル名が指定されなかったことを意味します。そのときはプログラムをエラー終了します。

残りの部分は紙面の都合で説明を割愛します。

図 30.5 tview で MikanOS の取扱説明書を表示した様子



tview 用に resource/mikanos.txt というファイルを追加しました。これは日本語の文書で、MikanOS の使い方が簡単に書いてあります。tview で表示した様子を図 30.5 に示します。きちんと日本語やタブ文字が表示されています。また、上下矢印キーや PageUp/PageDown キーでスクロールもできます。完成です。

30.6 画像ビューア (osbook_day30f)

とうとう、本書の最後の節になってしまいました。最後を飾るのは画像ファイルを表示するソフトウェア「画像ビューア」(graphic viewer) にしようと思います。ビットマップ画像や JPEG 写真などを表示するためのアプリです。

画像 ファイルを画面に表示するには、画像 ファイルを解釈して、最終的に PixelWriter::Write() で画面に表示することになります。つまり、画像ファイルをピクセルの配列に変換できなければなりません。ビットマップ形式だけなら割と簡単だと思います。なぜならビットマップ形式は単純で、ピクセル毎の色情報が配列になっているからです。しかし、その他の画像形式は複雑です。JPEG 形式は離散コサイン変換という数学的な処理が施されていますので、三角関数を用いてデータを復元する必要がありますし、GIF や PNG も圧縮された画像データを展開しなければなりません。自分でこれらを作るのはとても大変ですので既存のライブラリを利用したいです。

画像ビューアは「はりぼて OS」(参考文献 [1]) でも登場するアプリです。「はりぼて OS」の画像ビューアはビットマップと JPEG が表示できます。これは OSASK で使われていた画像ライブラリを使って実現しています。「はりぼて OS」は浮動小数点数の計算ができませんでしたが、OSASK の画像ライブラリは整数だけで JPEG 形式のデータを扱える特殊な作りになっており、そのまま利用できました。このライブラリを MikanOS の画像ビューアに組み込もうとしたところ、なぜか動作しませんでした。とい

うことで、他の画像ライブラリを探します。

なぜ「はりぼて OS」で浮動小数点数の計算ができないかというと、CPU 自体の機能として存在しなかったからです。x86 アーキテクチャでは浮動小数点数の計算はオプションで、特別に設定しないと使えません。一方、x86-64 アーキテクチャでは標準的に浮動小数点数を扱えます。x86-64 は SSE2 命令セットを必ず含むことになっているためです。そのため浮動小数点数計算ができることを前提に作られた普通の JPEG ライブラリが使えます。

JPEG 形式を扱えるライブラリとして stb_image ライブラリ^{*1}を使ってみます。このライブラリはビットマップや JPEG、PNG など、幅広い画像形式に対応していますから、これさえ使えばいろいろな形式に対応した画像ビューアを作れそうです。また、stb_image.h というファイルだけを持ってくれば使えるという手軽さも、このライブラリを採用した理由です。

リスト 30.19 画像ビューア (gview/gview.cpp)

```
#include <stdio>
#include <stdlib>
#include <string>
#include <fcntl.h>
#include <tuple>
#include "../syscall.h"

#define STBI_NO_THREAD_LOCALS
#define STB_IMAGE_IMPLEMENTATION
#define STBI_NO_STDIO
#include "stb_image.h"

std::tuple<int, uint8_t*, size_t> MapFile(const char* filepath) {
    SyscallResult res = SyscallOpenFile(filepath, O_RDONLY);
    if (res.error) {
        fprintf(stderr, "%s: %s\n", strerror(res.error), filepath);
        exit(1);
    }

    const int fd = res.value;
    size_t filesize;
    res = SyscallMapFile(fd, &filesize, 0);
    if (res.error) {
        fprintf(stderr, "%s\n", strerror(res.error));
        exit(1);
    }

    return {fd, reinterpret_cast<uint8_t*>(res.value), filesize};
}

void WaitEvent() {
    AppEvent events[1];
    while (true) {
        auto [n, err] = SyscallReadEvent(events, 1);
        if (err) {
            fprintf(stderr, "ReadEvent failed: %s\n", strerror(err));
        }
    }
}
```

*1 <https://github.com/nothings/stb> の stb_image.h。MIT ライセンス。

```

        return;
    }
    if (events[0].type == AppEvent::kQuit) {
        return;
    }
}

uint32_t GetColorRGB(unsigned char* image_data) {
    return static_cast<uint32_t>(image_data[0]) << 16 |
        static_cast<uint32_t>(image_data[1]) << 8 |
        static_cast<uint32_t>(image_data[2]);
}

uint32_t GetColorGray(unsigned char* image_data) {
    const uint32_t gray = image_data[0];
    return gray << 16 | gray << 8 | gray;
}

extern "C" void main(int argc, char** argv) {
    if (argc < 2) {
        fprintf(stderr, "Usage: %s <file>\n", argv[0]);
        exit(1);
    }

    int width, height, bytes_per_pixel;
    const char* filepath = argv[1];
    const auto [ fd, content, filesize ] = MapFile(filepath);

    unsigned char* image_data = stbi_load_from_memory(
        content, filesize, &width, &height, &bytes_per_pixel, 0);
    if (image_data == nullptr) {
        fprintf(stderr, "failed to load image: %s\n", stbi_failure_reason());
        exit(1);
    }

    fprintf(stderr, "%dx%d, %d bytes/pixel\n", width, height, bytes_per_pixel);
    auto get_color = GetColorRGB;
    if (bytes_per_pixel <= 2) {
        get_color = GetColorGray;
    }

    const char* last_slash = strrchr(filepath, '/');
    const char* filename = last_slash ? &last_slash[1] : filepath;
    SyscallResult window =
        SyscallOpenWindow(8 + width, 28 + height, 10, 10, filename);
    if (window.error) {
        fprintf(stderr, "%s\n", strerror(window.error));
        exit(1);
    }
    const uint64_t layer_id = window.value;

    for (int y = 0; y < height; ++y) {
        for (int x = 0; x < width; ++x) {
            uint32_t c = get_color(&image_data[bytes_per_pixel * (y * width + x)]);
            SyscallWinFillRectangle(layer_id | LAYER_NO_REDRAW,

```

```

        4 + x, 24 + y, 1, 1, c);
    }
}

SyscallWinRedraw(layer_id);
WaitEvent();

SyscallCloseWindow(layer_id);
exit(0);
}

```

リスト 30.19 に画像ビューアのソースコードを示します。かなり長いですが、主な部分を簡単に説明します。

stb_image.h の使い方として、インクルードする前にいくつかのマクロを定義しておく必要があります。STBI_NO_THREAD_LOCALS はマルチスレッド環境に対応するための機能（スレッドローカルストレージ）を無効化します。MikanOS ではマルチスレッドなアプリを作るための機能がまだありませんから、無効化しないとコンパイルできません。STB_IMAGE_IMPLEMENTATION は各種関数の実体を生成します。これを定義しておかないと、リンク時に undefined symbol のエラーが出てしまいます。STBI_NO_STBIO は fopen() などの標準的な入出力機能を使わないことを意味します。一応、MikanOS では fopen() と fread() をサポートしていますが、シークができないなど中途半端ですので stb_image ライブラリからはそれらを使わないようにしておきます。

stbi_load_from_memory() を使うと、メモリ上に配置した画像ファイルから画像を取得できます。引数の content は画像ファイルの内容を指すポインタです。画像ファイルをメモリ上に読み込むには fread() を使うかメモリマップトファイルを使うかの選択肢があります。stbi_load_from_memory() を使うには、画像ファイル全体をメモリ上に読み込んでおく必要がありますから、メモリマップトファイルが便利でしょう。

stbi_load_from_memory() は読み込んだ画像データを返します。この画像データ (image_data) は色情報がピクセルの数だけ並んだ配列になっています。1ピクセルあたりのバイト数は bytes_per_pixel に設定されています。stb_image ライブラリの仕様で、bytes_per_pixel が 1 か 2 ならグレースケール、3 か 4 ならフルカラーです。それらに応じてバイト列からピクセル色を計算する関数を選び、get_color に設定します。

この後はウィンドウを開き、image_data の内容を 1ピクセルずつ描画すれば画像が表示できます。簡単ですね。最後に WaitEvent() を使って、Ctrl-Q が押されるか、閉じるボタンがクリックされるまで待ってからウィンドウを閉じ、アプリを終了させます。

画像ビューアが出来上がったので、いくつか画像を用意して実行してみます。resource ディレクトリに画像を 3 つ追加しました。富士山の写真 fujisan.jpg、東京駅の丸の内駅舎 tokyost.jpg、夜景のイラスト night.bmp です。まず JPEG 画像を表示しようと、gview fujisan.jpg を実行してみました。すると「failed to load image: unknown marker」というエラーメッセージを出し、画像が表示できませんでした。

リスト 30.20 アプリのスタックを増やす (terminal.cpp)

```

const int stack_size = 16 * 4096;
LinearAddress4Level stack_frame_addr{0xffff'ffff'ffff'f000 - stack_size};

```

がんばって解決策を調査した結果、どうやらスタックを増やすとうまくいくようです。**リスト 30.20**のように、アプリ用スタックを 32KiB から 64KiB へ増やしました。じわじわとアプリ用スタックが増えてきてしまいました。少ないスタックで十分なアプリも多いのですが、スタックを最もたくさん消費するアプリに合わせるしかないのが現状です。仮想アドレスの使い方を少し工夫すると、最初はスタックを少しだけ確保しておき、足りなくなったら増やすという手が使えそうです。具体的には、スタック領域のすぐ隣に配置しているメモリマップトファイルのための仮想アドレス空間を、もっと離れた場所に移動しておけばいいですね。仮想アドレス空間は広大ですから、わざわざぎちぎちに詰め込む必要はないのです。

図 30.6 画像ビューアで 3 つの画像を表示した



実行した様子を図 30.6 に示します。良い感じに画像が表示できています。完成です！

第31章

これからの道

ここまで読んでいただいてありがとうございました。とても長い道のりだったと思いますが、楽しんでいただけたら著者としてこれ以上ない喜びです。本書で得た知識、手を動かして得た経験は、今後きっとどこかで役に立つはずです。

OS 自作には終わりがありません。本書で説明したのは OS 自作のほんの一部です。今後も OS 作りを続けたい方のために、方向性をいくつか示します。もちろんこれら以外の方向に進んで頂いて一向に構いません。あくまで、漠然と OS 作りを続けたいと思っているけど具体的に何をやればいいかわからない、という場合の参考として書きます。

まず、素直な方向性として MikanOS を改造するというのがあり得るでしょう。例えば、今はターミナルが OS 本体に入ってしまったっていますが、Linux などと同様にアプリとして作り直すことが考えられます。アプリとしてターミナルを作ろうとすると、アプリが他のアプリを実行する機能が必要になります。Linux などにある `fork` と `exec` というシステムコールで実現されるような機能です。

仮想メモリや共有メモリを実現するのも面白いと思います。仮想メモリとは、搭載された物理メモリの量を超えるメモリをアプリに提供する機能です。アプリがどんどんメモリを要求して物理メモリが枯渇しそうになっても、仮想メモリ機能によってアプリのメモリ要求に応えることができます。具体的には、物理メモリが枯渇しそうなときに物理メモリの一部（使用頻度が低い部分が適する）をストレージ（HDD や SSD）に書き出し、物理メモリに空き領域を作り出します。空いた領域をページングによってアプリの仮想アドレス空間にマップすれば、アプリは何事もなかったかのように処理を続行できます。

メモリ管理のアルゴリズムを改良するというネタもあります。MikanOS のメモリ管理はビットマップ方式の素朴な実装ですから、大きなメモリ要求ほど割り当てや解放に時間がかかる、空き領域が虫食い状態になるフラグメンテーションという現象が起きてしまう——などの欠点があります。Linux が採用するバディシステムやスラブアロケータ等の、高度なメモリ管理アルゴリズムを搭載することで解決できるかもしれません。

次の方向性としてはデバイスドライバの拡充があります。さまざまなハードウェアに対応するので、MikanOS 自体にストレージ読み書きの機能がありません。SATA や NVMe ドライバ（読んでいただいている頃には新しい規格が出ているかもしれませんね）を書くことで、HDD や SSD にアクセスできるようになります。

MikanOS に搭載された USB ドライバを拡張し、マウスやキーボード以外の USB デバイスに対応させるのも楽しいと思います。USB ハブや USB メモリ、プリンタ、ウェブカメラ、ネットワークアダプタなど、USB で接続できる機器はたくさんあります。

別の方向性として、MikanOS の拡張とは関係ない分野に手を出してみるのも楽しいかもしれません。Linux のコードを読んで改造する、C コンパイラを自作する、CPU を自作する——などなど、低レイヤ的な楽しみはたくさんあります。筆者は 2020 年の後半から、OS と言語処理系をすべて自作する OpeLa プロジェクトを始めました。うまくいくかは分かりませんが、目標は自作 OS の上で自作言語処理系を動かし、その言語処理系を使って OS と言語処理系をビルドすることです。つまり OS と言語処理系のセルフホストを目指します。

毎年夏に「セキュリティ・キャンプ全国大会」というイベントが開かれています*¹。このイベントは 4 泊 5 日の合宿形式で、情報セキュリティ分野の第 1 線で活躍する講師が講義を行います。内容はセキュリティやプログラミングに関するもので、ここ数年はハッカソン形式で自作 OS や C コンパイラ、自作言語を開発するゼミが開講されています。今後も同様の講義があるかどうかは分かりませんが、読者の皆さんがセキュリティ・キャンプ全国大会の参加可能年齢（22 歳以下）であれば、参加を検討してみてはいかがでしょうか。参加費、宿泊費、交通費が全額支給されるととてもお得なイベントですからね。

本書冒頭の「OS の作り方」にこんなことを書きました。

OS を作るにはコツがあります。それは、最初から完璧に作ろうとしないことです。最初から完璧を目指す手が止まってしまっても全然前に進まなくなります。

本書の内容はかなり難しかったと思います。できるだけ分かりやすいように説明を加える努力はしましたが、筆者の力には限界があります。もっとよく理解するためには 2 周、3 周と読む必要があるでしょう。2 周目は初回より格段に読みやすさが増しているはずです。余裕があれば Intel SDM (参考文献 [3]) なども併せて読み込んでいくと、深い知識が得られると思います。

困ったときは本書サポートサイトや osdev.jp が良い相談場所になるはずです。本書の内容で分からない部分の質問でも、発展的な内容（「こんな拡張機能を作りたいけどどうやるの？」とか）でも大丈夫です。お気軽に投稿してください。osdev.jp への参加方法は「0.4 OS 自作の楽しみ方」を参照してください。

* 1 2020 年は例外で、新型コロナウイルスの影響により 2 カ月に渡るオンラインでの開催になりました。

付録

付録 A

開発環境のインストール

本書で作る MikanOS をビルドしたり、起動したりする開発環境を整える方法は

<https://github.com/uchan-nos/mikanos-build>

にまとめています。Linux ディストリビューションの 1 つである Ubuntu で開発するのが標準ですが、Windows の WSL (A.01 を参照) 上に用意した Ubuntu を使っても開発可能です。

上記の手順に沿ってインストールを行うと、C++ コンパイラとして Clang が、x86 アセンブラとして Nasm がインストールされます。その他のツールもいくつかインストールされます。そして、ブートローダの開発用に EDK II がダウンロードされます。開発環境が正しくインストールできていれば、次のようなファイルの配置になっているでしょう。そうならないければインストールが失敗しています。

\$HOME/	
edk2/	
MikanLoaderPkg	ブートローダへのシンボリックリンク
osbook/	
day01/	
bin/hello.efi	バイナリエディタ版のハローワールドアプリ
asm/hello.asm	アセンブラ版のハローワールドアプリ (本文中には登場しない)
c/hello.c	C 言語版のハローワールドアプリ
devenv/	
run_qemu.sh	作成した UEFI アプリを QEMU で起動するスクリプト
buildenv.sh	MikanOS ビルド用の環境変数を含んだファイル
x86_64-elf/	MikanOS ビルド用のライブラリ群

A.01 WSL のインストール

Windows PC を使っていて Ubuntu 環境が無い場合、WSL ^{*1} という機能を使って Ubuntu を利用できます。執筆時点で WSL 1 と WSL 2 があり、MikanOS の開発はどちらでも可能です。WSL 2 の方が新しい技術であることと、Microsoft が WSL 2 を推奨していることから、特別な理由がなければ WSL 2 を使えば良いと思います。

<https://aka.ms/wslinstall> に公式のやり方がありますので、そちらに沿ってインストールしてください。以下に WSL 2 のインストール手順の概略をメモしておきます。

1. PowerShell を管理者として開く
2. PowerShell で `dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-Linux /all /norestart` を実行する
3. Windows バージョンが 1903 以降、かつビルド 18362 以上であることを確認する
4. PowerShell で `dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all /norestart` を実行する
5. PC を再起動する

6. x64 マシン用 WSL2 Linux カーネル更新プログラムパッケージをインストールする
7. PowerShell で `wsl --set-default-version 2` を実行する
8. Microsoft Store より Ubuntu をインストールする
9. スタートメニューより Ubuntu を起動する
10. 「Installing, this may take a few minutes...」と表示されるのでしばらく待つ
11. 「Enter new UNIX username」と聞かれたら好きなユーザ名を入力する
12. 「Enter new UNIX password」と聞かれたらパスワードを2回入力する

手順 10 でインストールが「エラー 0x80070003」、または「エラー 0x80370102」で失敗することがあります。このエラーが出た場合、BIOS の設定で CPU の仮想化機能が有効になっていることを確認してください。「Intel Virtualization Technology」のような項目を探し、有効 (Enable) に設定します。この設定の名前や場所は機種により異なりますが、CPU 関連の設定ページにあるかもしれません。

本書の内容の動作確認には Ubuntu 18.04 を使っていますので、Ubuntu のバージョンは 18.04 を選ぶのが最も安心ですが、Ubuntu 20.04 でも大丈夫なはずです。Ubuntu は 2 年ごとに長期サポート版がリリースされていて、18.04 は 2018 年 4 月版、20.04 は 2020 年 4 月版、22.04 は 2022 年 4 月版、……となっています。Ubuntu 22.04 以降は本書の執筆時点ではリリースされていませんから、動作確認が出来ていません。動作確認が取れ次第、mikanos-build リポジトリを更新していこうと思います。

A.02 WSL で QEMU を使う準備

WSL で QEMU を使うには Windows 側に X サーバをインストールしなければなりません。もしかしたら将来的にはこの手順は不要になるかもしれませんが。

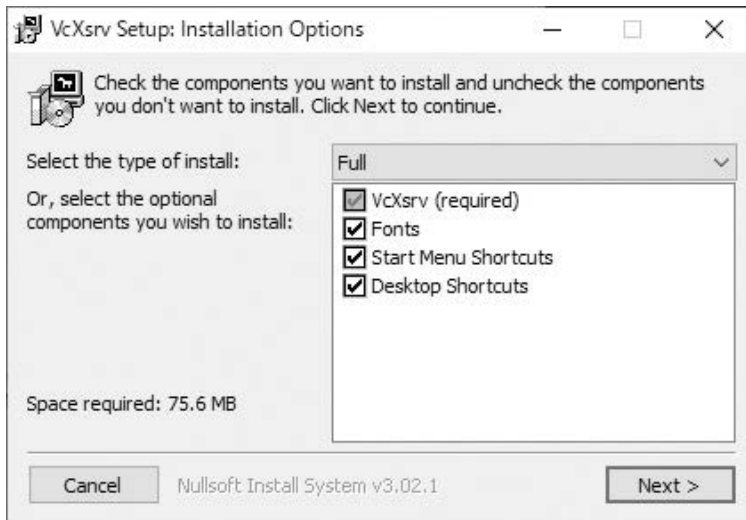
X サーバとは Linux で GUI を扱う中心的な部品です。Linux で GUI (ウィンドウやキーボードなどを中心とした表示、操作の仕組み) を扱うには、伝統的に X Window System という仕組みが使われます。この仕組みを使うと GUI アプリが動くマシンとその画面を表示するマシンを別にすることができます。すなわち、WSL の Ubuntu 上で動くアプリの画面を Windows 側の X サーバに転送し、そちらで描画する、ということが出来ます。WSL そのものには画面描画する機能がありませんので、X Window System による画面転送と描画が、QEMU の動作には必須です。

Windows に対応した X サーバの実装はいくつかあります。VcXsrv、MobaXterm、Cygwin/X、Xming など。筆者が調べたところ、現在人気がある (解説記事をよく見る) のは VcXsrv のようです。筆者もこれを使っています。

VcXsrv を入手するには、公式サイト <https://sourceforge.net/projects/vcxsrv/> にアクセスして Download ボタンより最新版をダウンロードします。本書の執筆時点では 1.20.8.1 が最新でした。ダウンロードが完了すると `vcxsrv-64.1.20.8.1.installer.exe` のような名前のインストーラが手に入りますから、これを起動してください。

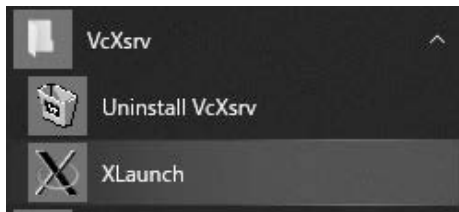
* 1 Windows Subsystem for Linux. Windows 上に Linux 環境を構築できます。

図 A.01 VcXsrv のインストールオプション選択画面



インストーラを起動すると図 A.01 のような画面が表示されます。インストールする部品を選べますが、Full を選んだままで構いません。Next をクリックして次に進みます。次の画面ではインストール先を指定しますが、これも特に理由がなければそのまま Install をクリックします。するとインストールが進行し、しばらく待つと完了します。

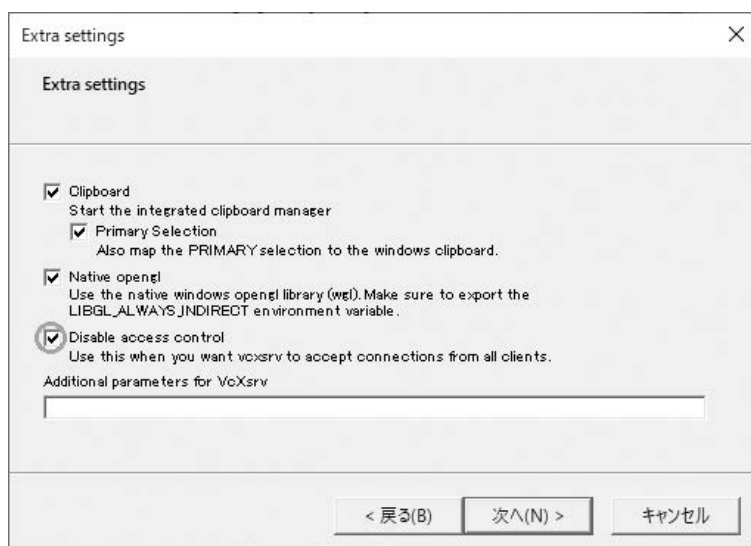
図 A.02 スタートメニューの VcXsrv



インストールが完了すると、Windows のスタートメニューに図 A.02 のような項目が増えているはずです。XLaunch をクリックして、VcXsrv の設定を行う補助アプリを起動します。

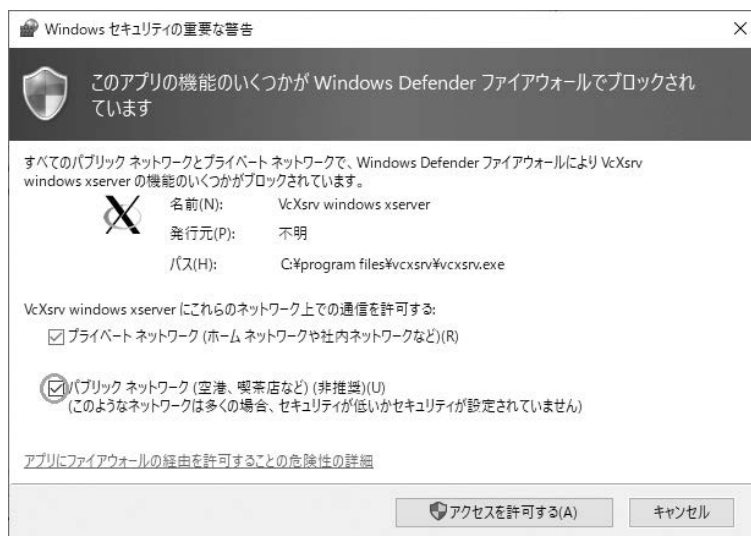
起動後の最初の画面では GUI アプリの画面をどのように表示するかを選べます。とりあえず Multiple windows を選んでおけば良いでしょう。「次へ」をクリックするとクライアント (GUI アプリ) を起動するかどうかを問われます。私たちは後で QEMU を起動するわけですから、今は何もアプリを起動させる必要はありません。Start no client を選びます。

図 A.03 VcXsrv のアクセスコントロールを無効化する



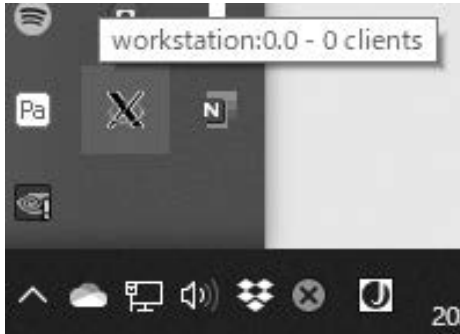
「次へ」をクリックすると「Extra Settings」という画面になります。ここが重要です。「Disable access control」にチェックを入れてください（図 A.03）。この設定をしておかないと、VcXsrv は WSL 上で動く GUI アプリからの接続を拒否してしまいます。この設定をしたら「次へ」をクリックします。最後の画面では、いよいよ VcXsrv を起動しますよ、という英語メッセージが出ますので「完了」をクリックします。

図 A.04 パブリックネットワークからの接続を許可する



VcXsrvを初めて起動するとき、Windowsがネットワーク接続を検出して図 A.04 のようなダイアログを出しますから、追加で「パブリックネットワーク」も選択してください。WSL 2 がパブリックネットワークとして認識されるため、この設定をしておかないと WSL 2 から VcXsrv への通信が遮断されてしまうのです。パブリックネットワークを追加選択したら「アクセスを許可する」をクリックします。

図 A.05 タスクバーに表示された VcXsrv のアイコン



VcXsrvを起動すると、タスクバーに VcXsrv のアイコンが表示されます（図 A.05）。アイコンにマウスを乗せると「PC 名:0.0 - 0 clients」のような表示が出ます。この中の「:0.0」が重要です。「:0.0」以外の数値になっている場合は何かおかしいです。設定が間違っていたり、VcXsrv がすでに起動していたりするかもしれませんので、確認してください。

「Windows セキュリティの重要な警告」ダイアログは初回しか出てきません。初回に「パブリックネットワークを追加し忘れた!」という場合は、ファイアウォールの設定を自分で変更しなければなりません。やり方はいくつかありますが、例えば「ファイアウォールによるアプリケーションの許可」という設定画面で「VcXsrv windows xserver」という項目を見つけ、「パブリック」にチェックを入れるという方法があります（図 A.06）。

図 A.06 VcXsrv へのパブリック接続を許可する



この設定画面は、Windows スタートメニューの検索ボックスに「ファイアウォールによるアプリケーションの許可」と入力すると見つけることができます。

付録 B

MikanOS の入手

MikanOS をビルドするには、開発環境に加えて MikanOS そのもののソースコードも必要です。<https://github.com/uchan-nos/mikanos> から入手できます。開発環境と違い、MikanOS のソースコードはどこに配置しても大丈夫です。例として、`$HOME/workspace/mikanos` に配置するコマンド列を示します。

```
$ mkdir $HOME/workspace
$ cd $HOME/workspace
$ git clone https://github.com/uchan-nos/mikanos.git
```

MikanOS のソースコードには、本書で紹介する各時点のバージョンにタグを付与してあります。タグは `osbook_dayXX` という名前になっています。試しにタグを一覧してみましょう。Git のコマンドを使うには対象のディレクトリに移動しておく必要がありますから、まず `cd $HOME/workspace/mikanos` をしてから `git tag -l` を実行します。

```
$ cd $HOME/workspace/mikanos
$ git tag -l
```

この中から希望する時点のソースコードを得ることができます。例えば「**26.1 標準入力 (osbook_day26a)**」の内容を得るには次のコマンドを実行します。

```
$ git checkout osbook_day26a
```

ファイルに変更があると、`git checkout` コマンドは「`error: Your local changes to the following files would be overwritten by checkout`」というメッセージを出して失敗します。チェックアウト操作によって、ファイルに加えた変更が上書きされ、なくなってしまいますよ、ということです。

変更をすべて破棄して良いなら、次のコマンドを打てばとりあえずチェックアウトできるようになります。変更が永久に失われてしまいますので、注意してください。

```
$ git reset --hard
```

変更を失いたくない場合はコミット作業をしなければなりません。Git の詳しい使い方は他書に譲ります。

B.01 MikanOS のバージョン間の差分確認

Git の機能を使って、2 つのバージョンの完全な差分を確認できます。本文に掲載されたソースコード片だけでは完全ではありませんから、ソースコードを写経するなど完全な差分が必要な際はこの方法を使ってください。例として day26a と day26b の差分を確認するコマンドと、その実行結果を示します。

```
$ cd $HOME/workspace/mikanos
$ git diff osbook_day26a osbook_day26b
diff --git a/kernel/terminal.cpp b/kernel/terminal.cpp
index 9ea0702c..ba04343b 100644
--- a/kernel/terminal.cpp
+++ b/kernel/terminal.cpp
@@ -11,6 +11,7 @@
#include "memory_manager.hpp"
#include "paging.hpp"
#include "timer.hpp"
+#include "keyboard.hpp"

namespace {

@@ -728,11 +729,22 @@ size_t TerminalFileDescriptor::Read(void* buf, size_t len) {
}
__asm__("sti");

- if (msg->type == Message::kKeyPush && msg->arg.keyboard.press) {
-   bufc[0] = msg->arg.keyboard.ascii;
-   term_.Print(bufc, 1);
-   return 1;
+ if (msg->type != Message::kKeyPush || !msg->arg.keyboard.press) {
+   continue;
+ }
+ if (msg->arg.keyboard.modifier & (kLControlBitMask | kRControlBitMask)) {
+   char s[3] = "^ ";
+   s[1] = toupper(msg->arg.keyboard.ascii);
+   term_.Print(s);
+   if (msg->arg.keyboard.keycode == 7 /* ␣ */) {
+     return 0; // EOT
+   }
+   continue;
+ }
+
+ bufc[0] = msg->arg.keyboard.ascii;
+ term_.Print(bufc, 1);
+ return 1;
}
```

行頭にある + は追加された行を、- は削除された行を示します。それぞれ数えてみると、4 行を削除して 16 行を追加することで day26a から day26b へと変化させられるということが分かります。同様の手順で、day26a と day26b のような隣り合う 2 つのバージョンの差分だけでなく、任意の 2 点間の差分を確認することもできます。

B.02 ソースコードの検索

MikanOS のソースコード内を検索するには `git grep` コマンドが便利です。例えば `KernelMain` という文字列を検索するには次のようにします。

```
$ git grep KernelMain
kernel/Makefile:LDFLAGS += --entry KernelMain -z norelro --image-base 0x100000 --static
kernel/asmfunc.asm:extern KernelMainNewStack
.....
```

こんな感じで、指定した文字列を含む行とそのファイル名がすべて表示されます。普通の `grep` コマンドと同じような文法で正規表現を使うことも可能です。

付録 C

EDK II のファイル説明

ソースコード以外で最低限必要なファイルはパッケージ宣言ファイル（.dec）、パッケージ記述ファイル（.dsc）、モジュール情報ファイル（.inf）の3つです。サンプルファイルから一部を引用して説明します。

リスト C.1 MikanLoaderPkg.dec

```
[Defines]
DEC_SPECIFICATION      = 0x00010005
PACKAGE_NAME           = MikanLoaderPkg
PACKAGE_GUID           = 452eae8e-71e9-11e8-a243-df3f1ffdebe1
PACKAGE_VERSION        = 0.1
```

まずパッケージ宣言ファイルを見てみます（**リスト C.1**）。このファイルは、パッケージの名前を決めるのが主な役割で、設定項目が非常に少ないです。PACKAGE_NAME さえ正しい値にすれば、ほかは上記の値を使っておけばとりあえず大丈夫です。このパッケージを広く公開するとか、本家の EDK II に取り込んでもらうつもりなら、設定値には少し気をつける必要があると思います（本書の範囲外なので、各設定項目の意味は解説しません）。

次にパッケージ記述ファイルを見てみます。パッケージ記述ファイルの仕様は「EDK II Platform Description (DSC) File Specification」（**参考文献 [16]**）に詳しく書かれています。このファイルは宣言ファイルに比べて圧倒的に内容が豊富で、見るべき場所がたくさんあります。順番に見ていきましょう。まずは基本的な情報を設定する Defines セクションから始めます。

リスト C.2 MikanLoaderPkg.dsc の Defines セクション

```
[Defines]
PLATFORM_NAME          = MikanLoaderPkg
PLATFORM_GUID          = d3f11f4e-71e9-11e8-a7e1-33fd4f7d5a3e
PLATFORM_VERSION       = 0.1
DSC_SPECIFICATION      = 0x00010005
OUTPUT_DIRECTORY       = Build/MikanLoader$(ARCH)
SUPPORTED_ARCHITECTURES = X64
BUILD_TARGETS          = DEBUG|RELEASE|NOOPT
```

リスト C.2 に Defines セクションを示します。この中で重要なのは SUPPORTED_ARCHITECTURES と OUTPUT_DIRECTORY です。その他の設定項目はあまり重要ではないので説明はしません。

SUPPORTED_ARCHITECTURES にはこの UEFI アプリケーションが対象とするアーキテクチャを指定します。本書では Intel 64 向けの OS を作りますから、ここには X64 と指定しておきます。ほかには IA32 や ARMなどを指定することもできます。

OUTPUT_DIRECTORY にはビルドしたファイル（.efi ファイル）が出力されるディレクトリを指定します。\$(ARCH) の部分は変数になっていて、実際にビルドされたアーキテクチャの名前（Intel 64 向けにビルドした場合は X64）が入ります。後で実際にビルドしてみますから、そのときに確認してみましょう。

リスト C.3 MikanLoaderPkg.dsc の LibraryClasses セクションの一部

```
[LibraryClasses]
  UefiApplicationEntryPoint|MdePkg/Library/UefiApplicationEntryPoint/UefiApplicationEntryPoint.inf
  UefiLib|MdePkg/Library/UefiLib/UefiLib.inf
```

次に LibraryClasses セクション (リスト C.3) に進みます。このセクションでは、UEFI アプリケーションを作成するのに必要となるライブラリの、名前と実際のファイルの対応を設定します。次のように、1 行に 1 つの対応を定義します。

ライブラリ名 | ライブラリのモジュール情報ファイルへのパス

引用した 2 行を見ると、UefiApplicationEntryPoint と UefiLib という 2 つのライブラリを定義していることが分かります。これらのライブラリ名は後で説明するモジュール情報ファイルの LibraryClasses セクションの設定で使います。

リスト C.4 MikanLoaderPkg.dsc の Components セクションの一部

```
[Components]
  MikanLoaderPkg/Loader.inf
```

次に Components セクション (リスト C.4) に進みます。このセクションでは、パッケージを構成するコンポーネント^{*2}を指定します。パッケージをビルドするとき、ここに指定したコンポーネントがビルド対象となります。例えば、EDK II の AppPkg には Hello や Main、Lua といった複数のコンポーネントが含まれていますが、Components セクションで指定しないコンポーネントはビルドされません。

リスト C.5 Loader.inf

```
[Defines]
  INF_VERSION           = 0x00010006
  BASE_NAME             = Loader
  FILE_GUID             = c9d0d202-71e9-11e8-9e52-cfbfd0063fbf
  MODULE_TYPE           = UEFI_APPLICATION
  VERSION_STRING        = 0.1
  ENTRY_POINT           = UefiMain

#  VALID_ARCHITECTURES   = X64

[Sources]
  Main.c

[Packages]
  MdePkg/MdePkg.dec

[LibraryClasses]
  UefiLib
  UefiApplicationEntryPoint
```

[Guids]

[Protocols]

パッケージ記述ファイルは一通り説明しましたので、次にモジュール情報ファイルを見てみます (**リスト C.5**)。モジュール情報ファイルの仕様書は「EDK II Module Information (INF) File Specification」(**参考文献 [17]**) です。1 つのパッケージは複数のモジュールを持つことができます。パッケージ記述ファイルはパッケージ毎に 1 つ、モジュール情報ファイルはモジュール毎に 1 つ作ります。

まず Defines セクション重要なのは `BASE_NAME` と `ENTRY_POINT` です。`BASE_NAME` にはコンポーネントの名前を書けば良いだけなので簡単です。

`ENTRY_POINT` にはこの UEFI アプリケーションのエントリポイント名を書きます。エントリポイントについて詳しくは「**2.2 EDK II でハローワールド**」を参照してください。

Sources セクションではこの UEFI アプリケーションを構成するソースコードを 1 行に 1 ファイルで列挙します。

Packages セクションは、筆者はあまり良く分かっていませんが、このモジュールをビルドするのに必要なパッケージを羅列すれば良いような雰囲気を感じます。とりあえず上記の設定値でうまくいっているため、詳しく探求していません。

LibraryClasses セクションではこの UEFI アプリケーションが依存するライブラリを指定します。ここで指定するライブラリ名はパッケージ記述ファイルの LibraryClasses セクションで定義した名前です。

* 2 「コンポーネント」はひとまとまりのプログラム部品を意味します。パッケージは 1 つ以上のコンポーネントから構成されます。

付録 D

C++ のテンプレート

テンプレートは型を抽象化するための機能です。C++ プログラムでは普通、`int` や `PixelColor` など型を具体的に書く必要がありますが、テンプレートを使えば「何らかの型」に対して共通の実装を書けるようになります。例えば標準ライブラリの `std::vector` はテンプレートになっていて、様々な型を要素とする動的配列を実現しています。要素の追加削除や検索、ソートなどの動的配列の各機能は、要素の型が `int` だろうが `double` だろうが、あるいは自作の構造体だろうが、特定の型に依存せずに実装できます。そのため、テンプレートとして実装するのがぴったりなのです。

リスト D.1 2次元ベクトル `Vector2D` (`graphics.hpp`)

```
template <typename T>
struct Vector2D {
    T x, y;

    template <typename U>
    Vector2D<T>& operator +=(const Vector2D<U>& rhs) {
        x += rhs.x;
        y += rhs.y;
        return *this;
    }
};
```

リスト D.1 に第 6 章で初登場した `Vector2D` の定義を再掲します。最初の行の `template <typename T>` がテンプレートの目印です。この行に続く構造体定義の中では `T` を型として使えるようになります。この `T` をテンプレート引数と呼びます。

`Vector2D` のように、構造体定義がテンプレートになったものを特に構造体テンプレートと呼びます。このほかには関数テンプレートと変数テンプレートがありますが、ここでは説明しません。

コンパイラは構造体テンプレートの使用を検出すると構造体の定義を実体化します。テンプレート実体化 (template instantiation) というのは、構造体定義の中のテンプレート引数 `T` を具体的な型で置き換えることをいいます。`FillRectangle()` の引数で `Vector2D<int>` と書いているため、コンパイラは `T` を `int` で置き換えた構造体の定義を生成します。仮に `Vector2D<double>` という型を使っているコードがあれば、コンパイラは `T` を `double` で実体化したバージョンも生成します。

テンプレートを使うことで、具体的な型にとらわれずに汎用的な構造体を定義できます。`Vector2D<int>` とすれば整数で座標を表せますし、`Vector2D<double>` などとすれば `X`、`Y` 座標を小数で表すこともできます。「2次元ベクトルは `x`、`y` という2つのメンバ変数を持ち、`a+=b` のように加算できる」という特性は、`T` の具体的な型が何であっても変わらないわけです。テンプレートを使えばこのような特性を素直に表現できますね。

付録 E iPXE

iPXE はネットワーク経由で OS を起動するためのソフトウェアです。開発用パソコンから試験用パソコンにネットワーク経由で OS（ブートローダとカーネル）をコピーし、試験用パソコン上で起動させることができます。USB メモリに OS をコピーして抜き差しする手間を省けるため、筆者は開発に活用しています。iPXE の公式ページは <https://ipxe.org/> です。

iPXE を使って OS を起動するには大きく 2 つの準備作業が必要です。順番に説明します。

- ・ iPXE をインストールした USB メモリを用意し、試験用パソコンに挿す
- ・ 開発用パソコンで HTTP サーバを起動する

E.01 iPXE のビルドとインストール

iPXE をビルドするのに必要なツールを導入します。APT ですべて揃いますのでお手軽です。

```
$ sudo apt install build-essential binutils-dev zlib1g-dev libiberty-dev liblzma-dev
```

iPXE のソースコードを取得します。

```
$ git clone https://github.com/ipxe/ipxe.git
```

iPXE の起動スクリプト `load.cfg` を準備します。このスクリプトなしで iPXE を使うこともできますが、毎回 iPXE のコマンドを手で打つ必要があり、面倒です。起動スクリプトは `ipxe/src/load.cfg` という名前で保存しておきます。

```
#!/ipxe

#dhcp
set net0/ip 192.168.0.199
set net0/netmask 255.255.255.0
ifopen net0

prompt Press any key to load the kernel

kernel http://<開発用パソコンの IP アドレス>:8000/EFI/BOOT/BOOTX64.EFI
initrd http://<開発用パソコンの IP アドレス>:8000/fat_disk
initrd http://<開発用パソコンの IP アドレス>:8000/kernel.elf /kernel.elf mode=755

boot
```

この起動スクリプトの前半ではネットワークを設定します。192.168.0.199 は試験用パソコンに割り当てる IP アドレスで、DHCP を回避して起動を高速化するため静的に IP アドレスを設定しています。

実際の IP アドレスは皆さんのネットワーク環境に応じて変更してください。LAN に接続されたその他の機器の IP アドレスと異なるものを使ってください。よく分からなければ DHCP モードにするのも良いでしょう。#dhcp のコメントを外し、その後の 3 行 (set の 2 行と ifopen の 1 行) をコメントアウトすると DHCP モードになります。

<開発用パソコンの IP アドレス> には開発マシンに割り当てられている IP アドレスを書いてください。IP アドレスを調べるには開発用パソコン上で ip a コマンドを実行します。表示された結果の中にある inet a.b.c.d という行が候補となる IP アドレスです。おそらく複数の IP アドレスが表示されますが、その中で inet 192.168.x.y などとなっている行があれば、それが正解である可能性が高いです。

スクリプトの後半では HTTP を使って 3 つのファイルをダウンロードします。kernel と initrd はそれぞれファイルをダウンロードするための iPXE のコマンドです。kernel は、ダウンロードに加えてそのファイルを起動対象として指定します。kernel という名前ですが、あくまでも最初に起動するファイルを指定するコマンドですから、kernel.elf ではなく B00TX64.EFI を指定します。

起動スクリプトが準備できたら iPXE をビルドします。

```
$ cd ipxe/src
$ make bin-x86_64-efi/ipxe.efi EMBED=../load.cfg
```

この手順を実行すると bin-x86_64-efi/ipxe.efi というファイルが生成されます。EMBED=../load.cfg という指定によって、起動スクリプトの中に含んだものになっています。これを USB メモリにインストールします。インストールといっても /EFI/B00T/B00TX64.EFI にコピーするだけです。コピーはどうやってもいいですが、例を 1 つ紹介しておきます。

```
$ sudo mount /dev/sdb1 /mnt/usbmem
$ sudo cp bin-x86_64-efi/ipxe.efi /mnt/usbmem/EFI/B00T/B00TX64.EFI
$ sudo umount /mnt/usbmem
```

この USB メモリを試験用パソコンに挿します。

E.02 HTTP サーバの起動

起動スクリプトは `http://<開発用パソコンの IP アドレス>:8000/...` からファイルをダウンロードしようとしています。そのため、開発用パソコンのポート 8000 に HTTP サーバを立てる必要があります。何を使ってもいいですが、Ubuntu を使っている場合は最もお手軽なのは Python を使った方法だと思いますので、紹介します。

まずは 3 つのファイルが、どこかのディレクトリ (ここでは ipxe_root とする) を基準として ipxe_root/EFI/B00T/B00TX64.EFI、ipxe_root/fat_disk、ipxe_root/kernel.elf として見えるように配置します。シンボリックリンクを使えば簡単にできます。


```
$ cd $HOME/workspace/mikanos
$ mkdir -p ipxe_root/EFI/BOOT
$ cd ipxe_root
$ ln -s $HOME/edk2/Build/MikanLoaderX64/DEBUG_CLANG38/X64/Loader.efi ./EFI/BOOT/BOOTX64.EFI
$ ln -s ../disk.img ./fat_disk
$ ln -s ../kernel/kernel.elf ./kernel.elf
```

ファイルの配置が終わったら Python で HTTP サーバを起動します。

```
$ python3 -m http.server 8000
```

HTTP サーバが起動できたことを確認するには curl コマンドが便利です。-I オプションを使うとファイル本体をダウンロードせず、ファイルがあることを確認できます。

```
$ curl -I http://localhost:8000/kernel.elf
HTTP/1.0 200 OK
Server: SimpleHTTP/0.6 Python/3.6.9
Date: Wed, 02 Dec 2020 07:37:19 GMT
Content-type: application/octet-stream
Content-Length: 4451856
Last-Modified: Tue, 01 Dec 2020 03:08:37 GMT
```

上記のように curl -I の結果が 200 OK となれば HTTP サーバは正常に起動しています。

E.03 ネットワーク起動の実践

iPXE 入り USB メモリと開発用パソコン上の HTTP サーバの準備が整えば、いよいよネットワーク起動ができます。試験用パソコンに iPXE 入り USB メモリを挿し、電源を入れます。BIOS 設定画面を出し、iPXE 入り USB メモリから起動するように調整してください。大抵の機種では Delete キーか F2 キーを連打することで BIOS 設定画面が出てきます。

何も設定をいじらずに iPXE をビルドすると余計な機能が組み込まれてしまい、iPXE 起動時の初期化時間が延びる原因となります。筆者の経験では、「iPXE initialising devices...」というメッセージが出た状態で数十秒の時間がかかる機種がありました。

そんなときは ipxe/src/config/general.h でビルドオプションを調整すると改善する可能性があります。筆者が試した機種では次に示す無線 LAN 関係のオプションの #define を #undef に変更したら非常に高速化しました。

```
#undef CRYPTO_80211_WEP
#undef CRYPTO_80211_WPA
#undef CRYPTO_80211_WPA2
```

その機種には無線 LAN のインターフェースがないため、iPXE が無線 LAN 関連の機能を初期化しようとしてタイムアウトしていた可能性がありますね。

付録 F ASCII コード表

表 F.1 ASCII コード (制御文字)

文字	10 進数	16 進数	エスケープシーケンス
NUL (null character)	0	0x00	\0
SOH (start of header)	1	0x01	
STX (start of text)	2	0x02	
ETX (end of text)	3	0x03	
EOT (end of transmission)	4	0x04	
ENQ (enquiry)	5	0x05	
ACK (acknowledge)	6	0x06	
BEL (bell)	7	0x07	\a
BS (backspace)	8	0x08	\b
HT (horizontal tab)	9	0x09	\t
LF (line feed)	10	0x0a	\n
VT (vertical tab)	11	0x0b	\v
FF (form feed)	12	0x0c	\f
CR (carriage return)	13	0x0d	\r
SO (shift out)	14	0x0e	
SI (shift in)	15	0x0f	
DLE (data link escape)	16	0x10	
DC1 (device control 1)	17	0x11	
DC2 (device control 2)	18	0x12	
DC3 (device control 3)	19	0x13	
DC4 (device control 4)	20	0x14	
NAK (negative acknowledge)	21	0x15	
SYN (synchronous idle)	22	0x16	
ETB (end of transmission block)	23	0x17	
CAN (cancel)	24	0x18	
EM (end of medium)	25	0x19	
SUB (substitute)	26	0x1a	
ESC (escape)	27	0x1b	
FS (file separator)	28	0x1c	
GS (group separator)	29	0x1d	
RS (record separator)	30	0x1e	
US (unit separator)	31	0x1f	
DEL (delete)	127	0x7f	

表 F.1 は ASCII コードのうちの制御文字の部分です。制御文字とは通常は画面に表示されることなく、文書の表示制御などに使われます。非印字可能文字 (non-printable character) と呼ぶこともあります。例えば改行を表す文字 (LF や CR)、タブ文字 (HT や VT) は制御文字です。また、通常の文書では使われませんが、データの区切りを表す文字 (SOH や FS など) などとも規定されています。

ASCII コードは最も代表的な文字コードと言ってもいいでしょう。文字を扱うソフトウェアの多くが ASCII コードに対応しています。我々が普通にプログラムを作成するときは意識せず ASCII コードを使っていることでしょう。近年よく使われるようになった UTF-8 という文字コードは、0x00 ～ 0x7f の範囲において ASCII コードと互換性があります。その範囲の文字だけを使って書かれ、UTF-8 でエンコードされた文書は、ASCII コードでエンコードされた文書とまったく見分けが付きません。

表 F.2 ASCII コード (図形文字 0x20 ～ 0x3f)

文字	10 進数	16 進数	文字	10 進数	16 進数
空白 (' ')	32	0x20	0	48	0x30
!	33	0x21	1	49	0x31
"	34	0x22	2	50	0x32
#	35	0x23	3	51	0x33
\$	36	0x24	4	52	0x34
%	37	0x25	5	53	0x35
&	38	0x26	6	54	0x36
' ('\ ')	39	0x27	7	55	0x37
(	40	0x28	8	56	0x38
)	41	0x29	9	57	0x39
*	42	0x2a	:	58	0x3a
+	43	0x2b	;	59	0x3b
,	44	0x2c	<	60	0x3c
-	45	0x2d	=	61	0x3d
.	46	0x2e	>	62	0x3e
/	47	0x2f	?	63	0x3f

表 F.2 と **表 F.3** は図形文字です。制御文字は画面への表示が意図されていない一方、図形文字 (0x20 ～ 0x7e) は画面に表示するための文字です。そのため、図形文字を印字可能文字 (printable character) と呼びます。

タブ (HT) や改行 (LF) が非印字可能文字なのに空白が印字可能文字なのはどういうことだ、と思うかもしれません。筆者の想像ですが、タブは画面に表示するときは空白文字に変換されてしまい、タブそのものは表示されない、という理屈じゃないかと思います。改行も、次の行に移る効果があるだけで改行文字の形があるわけではありません。ただし、このあたりの分類は場合によって違います。例えば Python というプログラミング言語に用意された定数 `string.printable` には、空白文字に加えて HT、LF、VT、FF、CR が含まれます。

表 F.3 ASCII コード (図形文字 0x40 ~ 0x7e)

文字	10 進数	16 進数	文字	10 進数	16 進数
@	64	0x40	`	96	0x60
A	65	0x41	a	97	0x61
B	66	0x42	b	98	0x62
C	67	0x43	c	99	0x63
D	68	0x44	d	100	0x64
E	69	0x45	e	101	0x65
F	70	0x46	f	102	0x66
G	71	0x47	g	103	0x67
H	72	0x48	h	104	0x68
I	73	0x49	i	105	0x69
J	74	0x4a	j	106	0x6a
K	75	0x4b	k	107	0x6b
L	76	0x4c	l	108	0x6c
M	77	0x4d	m	109	0x6d
N	78	0x4e	n	110	0x6e
O	79	0x4f	o	111	0x6f
P	80	0x50	p	112	0x70
Q	81	0x51	q	113	0x71
R	82	0x52	r	114	0x72
S	83	0x53	s	115	0x73
T	84	0x54	t	116	0x74
U	85	0x55	u	117	0x75
V	86	0x56	v	118	0x76
W	87	0x57	w	119	0x77
X	88	0x58	x	120	0x78
Y	89	0x59	y	121	0x79
Z	90	0x5a	z	122	0x7a
[	91	0x5b	{	123	0x7b
\('\'')	92	0x5c		124	0x7c
]	93	0x5d	}	125	0x7d
^	94	0x5e	~	126	0x7e
_	95	0x5f			

表 F.3 の左右を見比べてみてください。左側のアルファベットは大文字で、右側には同じアルファベットの小文字が載っていますよね。実は、ASCII コードでは大文字に 0x20 を足すと小文字になるのです。この事実を使うと、例えば大文字小文字混じりの文字列をすべて大文字に変換するプログラムは次のように書けます。

```
#include <iostream>

void ToUpper(char* s) {
    while (*s) {
        if ('a' <= *s && *s <= 'z') {
            *s -= 0x20;
        }
        ++s;
    }
}

int main() {
    char greet[] = "Hello, world!";
    std::cout << greet << std::endl;
    ToUpper(greet);
    std::cout << greet << std::endl;
}
```

実際のプログラムでは標準ライブラリ `<cctype>` に定義された `std::toupper` や `std::tolower` を使うことをお勧めします。なぜなら、それらの関数を使っていればプログラムの実行環境が ASCII 以外の文字コードを採用していても対応できるからです。例えばロシアで使うプログラムではキリル文字が入力されるかもしれませんがね。そうすると、ASCII コードを前提とした大文字小文字変換のやり方は使えません。ASCII コードしか使わないことが約束されていれば、上記のプログラムでも問題ありませんけどね。

参考文献 [1]

「30 日でできる！ OS 自作入門」

2006 年、マイナビ出版、川合秀実

ISBN 978-4-8399-1984-9

<https://book.mynavi.jp/ec/products/detail/id=22078>

参考文献 [2]

「Unified Extensible Firmware Interface Specification Version 2.8」

2019 年、Unified EFI Forum, Inc.

<https://www.uefi.org/specifications>

参考文献 [3]

「Intel 64 and IA-32 Architectures Software Developer' s Manual」

2019 年、Intel Corporation

<https://software.intel.com/en-us/articles/intel-sdm>

参考文献 [4]

「QEMU/Monitor」

2018 年、Wikibooks

<https://en.wikibooks.org/wiki/QEMU/Monitor>

参考文献 [5]

「System V Application Binary Interface - DRAFT」

2001 年、The Santa Cruz Operation, Inc.

<https://refspecs.linuxfoundation.org/elf/gabi4+/contents.html>

参考文献 [6]

「System V Application Binary Interface AMD64 Architecture Processor Supplement」

2013 年、Jan Hubicka. et al.

<https://software.intel.com/sites/default/files/article/402129/mpx-linux64-abi.pdf>

参考文献 [7]

「GNU Make 第 3 版」

2005 年、オライリー・ジャパン、Mecklenburg, R.、矢吹／菊池訳

ISBN 4-87311-269-9

<https://www.oreilly.co.jp/library/4873112699/>

※ PDF が無料公開されています。

参考文献 [8]

「リンカ・ローダ実践開発テクニック」

2010 年、CQ 出版、坂井弘亮

ISBN 978-4-7898-3807-8

<https://shop.cqpub.co.jp/hanbai/books/38/38071.html>

参考文献 [9]

「Intel 7 Series / C216 Chipset Family Platform Controller Hub Datasheet」

2012 年、Intel Corporation.
<https://www.intel.com/content/dam/www/public/us/en/documents/datasheets/7-series-chipset-pch-datasheet.pdf>

参考文献 [10]

「The PCI ID Repository」

2019 年、Pool、Albert ほか
<https://pci-ids.ucw.cz/>

参考文献 [11]

「C++ Concurrency in Action, Second Edition」

2019 年、Williams, Anthony
ISBN 978-1-61729-469-3
<https://www.manning.com/books/c-plus-plus-concurrency-in-action-second-edition>

参考文献 [12]

「Advanced Configuration and Power Interface Specification Version 6.3」

2019 年、UEFI Forum, Inc.
<https://www.uefi.org/specifications>

参考文献 [13]

「Universal Serial Bus HID Usage Tables Version 1.12」

2004 年、USB Implementers' Forum
https://www.usb.org/sites/default/files/hut1_21_0.pdf

参考文献 [14]

「Universal Serial Bus Device Class Definition for Human Interface Devices Version 1.11」

2001 年、USB Implementers' Forum
https://www.usb.org/sites/default/files/hid1_11.pdf

参考文献 [15]

「Microsoft Extensible Firmware Initiative FAT32 File System Specification」

2000 年、Microsoft Corporation
<http://download.microsoft.com/download/0/8/4/084C452B-B772-4FE5-89BB-A0CBF082286A/fatgen103.doc>

参考文献 [16]

「EDK II Platform Description (DSC) File Specification」

2017 年、Intel Corporation.
<https://edk2-docs.gitbook.io/edk-ii-dsc-specification/>

参考文献 [17]

「EDK II Module Information (INF) File Specification」

2017 年、Intel Corporation.
<https://edk2-docs.gitbook.io/edk-ii-inf-specification/>

謝辞

この本が出版できたのは、たくさんの人の協力があったからこそです。編集の風穴さんとマイナビ出版の山口さん、制作デザインの Dada House 海江田さんには原稿完成が初期の予定を大幅に過ぎて、延び延びになってしまい多大な迷惑をかけました。辛抱強く待っていただきありがとうございました。その他にもこの本の完成に不可欠の方々がたくさん居ますので、ここで感謝を述べます。

この本を書く最初の、かつ最大のきっかけを与えてくださったのは OSASK であり「30 日でできる！ OS 自作入門」です。OSASK の作者であり、本の著者でもある川合さんには感謝してもしきれません。川合さんが OSASK を作っていなかったら筆者は自作 OS というものに出会わなかったかもしれません。本が出版されていなかったら、それを新しい技術を使って書き直そうと思いませんでした。川合さんが「OS 自作」を一般に広めてくださったおかげで、筆者は OS 自作を趣味として、仕事として楽しめています。川合さんにはさらに、この本の執筆にあたり様々なアドバイスをいただきました。的確なご意見は大変ありがたかったです。

レビューの皆さんのおかげで品質が大変向上しました。東京工業大学情報理工学院教授の権藤先生、セキュリティ・キャンプ全国大会 2017 の受講生だった森真誠さん、同大会 2019 の受講生だった筑波大学の広瀬智之さん、スタンフォード大学数学科・物理学科の佐藤弘崇さん、この本で勉強したいと申し出てくれた中学生代表の平田誠治さん、kaage (@ageprocpp) さん、八木瑛久さん、Osmium_1008 さん。荒削りの原稿にも関わらず辛抱強く読んでくださり、貴重な意見をいただきました。ありがとうございました。

東京工業大学情報理工学院「2020 年度 システム開発プロジェクト応用第二」の受講生の皆様に感謝します。未完成の原稿を読み、数々の機能追加や改造、誤字脱字をレポートしてくれました。本書が大学院レベルの講義で使えるという感触を得るとともに、教科書としての品質が向上しました。

osdev.jp の市川真一さんには様々な機種で MikanOS の動作確認をしていただきました。それだけでなく、貴重な試験機を無償で貸し出していただきました。ありがとうございました。

osdev.jp メンバーの皆さんに感謝しています。皆さんがこのコミュニティを盛り上げてくださっているおかげで、筆者は OS 自作を続けられています。この盛り上がりがなければ、本書を執筆しようという気持ちは湧かなかったでしょう。また、皆さんと交わした技術的な議論も貴重でした。皆さんと深く高度な議論ができたおかげで MikanOS をここまで作れたと思っています。ありがとうございました。

索引

英数字

16進数	30, 69, 320, 396, 477	IDT	165
4階層ページング	442	IF	179, 315, 579
8+3形式	402, 412	Intel 64モード	14
ABI	81, 99, 101, 145, 314, 320, 480, 483, 496	IOアドレス空間	145
ACPI	282	iPXE	19, 729
ACPI PMタイマ	293	IST	81, 488
ADL	155	Linux	14
API	141, 149	LLVM	18
ASCIIコード	39, 732	lspciコマンド	389
BAR0	152	lsコマンド	410
BIOS	4, 20, 40	LVT Timer	227
BIOSパラメータブロック	400	make	94
BPB	400	Message Address	169
BSP	170	Message Data	169
C++	102, 728	MikanOS	3, 722
cat	696	MMIO	152, 628
Clang	18	moreコマンド	693
clearコマンド	388	MSI	169
CLI	23	new演算子	206
COFF形式	45	OHCI	140
CoW	639	OS	3
CPL	191, 462, 577	osdev-jp	19, 107
DDR-SDRAM	20	Panther Point	153
Divide Configuration	227	PBR	400
DPL	166, 191, 460	PCI	141
echoコマンド	384	PDP table	196
EDK II	48, 725	PE形式	43, 45
EHCI	140, 153	PF	478
ELF形式	45, 79, 428	PML4 table	196
EOF	607	pop	173
EOT	607	POSIX	17, 403, 419, 589, 602
exit()	512	printf()	127, 132
FADT	282, 290	printk()	132
FAT	23, 30, 416, 582, 602, 628	public継承	363
FIFO	172	push	173
FILO	173	QEMU	18, 35, 68
FreeType	24, 657	RGB	97
GCC	18	RSDP	282, 290
GDT	185, 187	sortコマンド	685
GP	478	std::string	368
GUI	23	TLB	646
		TrueType	24, 657

TSS	467	インライン変数	147
Ubuntu	18	ウィンドウ	21, 209, 249
UEFI	4, 20, 35, 39, 40	ウィンドウの重なり	569
UEFIメモリマップ	182	ウィンドウ描画	520
UHCI	140	ウィンドウを開く	506
Unicode	39, 650	エコーバック	605, 688
USB3.0ドライバ	14	エミュレータ	18, 28, 35
USB3_PSSSEN	154	エラー処理	86
USB3PRM	154	演算子のオーバーロード	136
USBキーボード	295	エントリポイント	49, 72
USBホストコントローラ	140	お絵かきソフト	543
USBホストドライバ	139	オーバーライド	103
USBメモリ	18	オーバーロード	254
using宣言	327	オフセット	64
UTF-8	652	オペコード	70
VRAM	234	オペランド	70
vtable	108	オペレーティングシステム	3
write()	496	親クラス	103
WSL	18, 716		
x86-64アーキテクチャ	320		
xHCI	140		
XSDT	282, 290		
XUSB2PR	154		
XUSB2PRM	154		

あ行

アイデンティティマッピング	195
アイドルタスク	352
アクティブウィンドウ	360
アセンブリ言語	89
アドレス	54, 71, 87, 110
アドレス演算子	59, 65
アドレス変換	440
アニメーション	552
アプリ間通信	24, 673
アプリケーションのロード	447
アライメント	88, 167, 315, 598
アロー演算子	64
一般保護例外	478
イテレータ	60
イベントループ	180
インクルードガード	52
印字可能文字	733
インスタンス	102
インタフェース	15, 23
インラインアセンブラ	73, 179

か行

カーソル	306
カーソル点滅	561
カーネル	83
カーネルモード	187
階層ベージング構造	197, 441
開発環境	716
重ね合わせ処理	206, 209
仮想アドレス	440, 442
画像ビューア	706
型変換	60
カノニカルアドレス	445
可変引数テンプレート	516
カレントディレクトリ	588
関数のオーバーロード	254, 373
間接演算子	62, 64, 121
キーコード	22, 296
キー入力	380, 528
キーボード入力	604
記述ヘッダ	290
キビバイト	55
逆ポーランド記法	429
キャスト	60, 76, 83, 87
キュー	173, 177
協調的マルチタスク	321
共有メモリ	690
クラス	102

クラスタ	401	純粋仮想関数	102
クラスドライバ	140, 141	省電力化	393
グリフ	660	シリアル	140
クローズ	526	スクロール	130
グローバルディスクリプタテーブル	187	スクロール処理	240
継承	103, 362	図形文字	733
権限レベル	460	スタック	173, 186
構造体テンプレート	153	スタックセグメントレジスタ	193
高速カウンタ	252	スタックポインタ	89, 502
後置記法	429	ストレージ	20, 23, 36, 396, 712
コードセグメント	166	スマートポインタ	214, 225
コードセグメントレジスタ	193	スリープ状態	334
コードポイント	650	正規表現	597
子クラス	103	制御文字	733
コピーオンライト	639	性能測定	344
コマンドライン	676	セキュアブート	31
コマンド履歴	390	セクションヘッダ	111
コンストラクタ	102	セクタ	400
コンソール	129	セグメンテーション	166, 187, 441
コンソールクラス	129	セグメント	110
コンテキスト	310	セグメントディスクリプタ	190
コンテキストスイッチ	311, 321	セグメントレジスタ	193
コンパイル	18, 42, 73, 107, 432, 446	絶対パス	582
		線	522
		全角文字	655
		相対ショートジャンプ	423
		相対パス	582
		ソースコード	18, 42, 73
さ行			
再帰呼び出し	450		
先入れ後出し	173		
先入れ先出し	172		
サブクラス	103		
参照	121		
参照の局所性	622		
シーク	589		
シェバン	45		
シェル	45		
自作OS	3		
システムコール	23, 479, 492		
実行可能状態	334		
実行可能ファイル	36, 42, 43		
実行状態	334		
実行ファイル	438		
写経	19, 40		
シャドウバッファ	231		
シャドウレジスタ	193		
終了コード	670		
終了システムコール	496		
述語	218		
		た行	
		ターミナル	380, 560, 680
		ターミナルウィンドウ	369
		タイマ	21, 162, 226, 259, 270, 290, 547
		タイマ値	518
		タイムアウト	228, 277, 294, 322, 344, 547, 561
		タスク	326
		タスクの終了	682
		チェックサム	30
		抽象化	16, 589, 604, 728
		抽象クラス	250
		抽象データ型	102
		チラチラ解消	253
		定数式	251
		ディスクイメージ	61
		ディスクリプタ	187
		ディスクリプタキャッシュ	193

ディレクトリ	30, 582	番地	54
ディレクトリエントリ	402, 582	反復子	60
データ競合	179, 690	ヒープ領域	105
テキストエディタ	18	非印字可能文字	733
テキストビューア	703	光の3原色	97
テキストボックス	303	ピクセル	20, 21, 82, 83, 97
デストラクタ	102	ビット反転	159
デバイスドライバ	20	ビットフィールド	166
デマングル	458	描画の高速化	374
デマンドページング	622	標準エラー出力	602
点	513	標準出力	602, 663
テンプレート	136, 728	標準入力	602, 663
テンプレートクラス	174	標準ライブラリ	18.4
テンプレート実体化	728	ファームウェア	40
テンプレートの特権化	287	ファイル	23, 36, 41, 396, 582, 602, 628
糖衣構文	65	ファイルアロケーションテーブル	416
同時起動	561	ファイル書き込み	607, 616
閉じるボタン	698	ファイルシステム	14, 23, 196, 396, 416
特権モード	187	ファイルディスクリプタ	24, 187, 589, 602, 630, 663, 697
ドット演算子	64	ファイルヘッダ	111
ドライバ	140, 403, 712	ファイル読み込み	588
ドラッグ移動	261, 266	ブートインタフェース	300

な・は行

中置記法	429	ブートサービス	56
名前修飾	72, 89	ブートルoader	20, 41, 49, 82
偽物のターゲット	95	フェース	658
日本語	650	フォント	20, 24, 119, 124, 242, 477, 651
日本語フォント	657	複数アプリ	561
ヌルディスクリプタ	188	符号化方式	650
		ブッシュ	173
		物理アドレス	195, 440
		フラグメンテーション	628
配置new	104, 206	フラットバイナリ	422, 428
バイト	23, 53	フリーズ	575
バイナリエディタ	28	フリースタンディング環境	74, 432
パイプ	24, 673, 695	プリエンブティブ・マルチタスク	21, 321
パイプ処理	677	フレーム	622
バグ発見	476	フレームバッファ	83, 232
バス	139	プログラムブレーク	206
バス	582, 692	プログラムヘッダ	109, 111
バスドライバ	140, 141	プロセス間通信	25, 673
バックバッファ	259	ブロック崩しゲーム	554
パディング	167	ブロックデバイス	396
パラレル	140	プロンプト	384
はりぼてOS	14	分割コンパイル	122
範囲for文	219	並行処理	311
半角文字	655	並行性バグ	179

並列処理	311
ページ	55, 77, 622
ページキャッシュ	628
ページディレクトリ	196
ページディレクトリポインタテーブル	196
ページテーブル	196
ページフォルト	447, 478, 622
ページフレーム	198
ページマップレベル4テーブル	196
ページング	14, 441
ベースクラス	103
ポインタ	62, 89, 121
ポーリング	151, 156, 271
ホスト環境	74, 432
ホストコントローラドライバ	140
ホストドライバ	140
ホストブリッジ	147
ポップ	173
ボリュームイメージ	403
翻訳単位	122

ま行

マウス	151, 248
マウスカーソル	136
マウス入力	538
マウント	61
マジックナンバー	521
マップ	440
マルチタスク	14, 22, 310, 325
マルチファンクションデバイス	147
マンダリング	72, 513
マンダリ	458
無名関数	144
無名名前空間	188, 624
メインメモリ	20, 21, 36, 41, 53, 69, 164, 182, 440
メソッドチェーン	222, 329
メタデータ	39, 396
メモリアドレス	438
メモリ管理	21, 182, 198
メモリコントローラ	53
メモリ使用量	637
メモリダンプ	69
メモリディスクリプタ	57
メモリマップ	54, 77, 182, 422
メモリマップトIO	152, 628
メモリマップトファイル	628, 709

メモリマネージャ	198
文字	118, 508
文字コード	39, 650
モジュール分割	102
文字列描画	127
モディファイアキー	296

や・ら・わ行

ユーザー定義リテラル	199
ユーザーモード	187
優先順位の逆転	357
優先度	344
呼出規約	101
ラッパ	210, 491
ラムダ式	144, 218, 515
ランキュー	334, 682
乱数	517
ランタイムサービス	56
ランダムアクセス	628
リダイレクト	662
リテラル	199, 227, 370, 439, 521
リニアアドレス	195
リンク	74
リング	460
ルートディレクトリ	582
例外	232
レイヤ	206, 209, 230, 250, 254, 263, 357, 363, 369, 569, 698
レイヤマネージャ	222, 506
レガシーBIOS	40
レジスタ	68
レッドゾーン	81
レポートディスクリプタ	300
ローダ	109
ロード	110, 129, 446, 566, 640
ログ関数	159
ログレベル	159
ワイド文字	50
割り込み	21, 22, 162, 170
割り込み記述子	167
割り込みハンドラ	163
割り込みベクタ	164
割り込み要因番号	164

内田 公太 (うちだ こうた、uchan)

小学生の頃にPICマイコンのアセンブリ言語に出会い、プログラマの道に進む。高校時代は『30日のできる！OS自作入門』(川合秀実、2006年)の校正を手伝う。東京工業大学の計算工学専攻を卒業後、サイボウズ株式会社でインフラ系のプログラミングに従事したり、東京工業大学でソフトウェア系の授業を担当したりしていた。2017年から2020年にかけて、セキュリティ・キャンプ全国大会でOS開発系のゼミを受け持った。2020年にサイボウズ・ラボ株式会社へ転籍し、OSや言語処理系の研究開発をしている。著書に『自作エミュレータで学ぶx86アーキテクチャ』(2015年)がある。

[STAFF]

カバーデザイン：海江田 暁 (Dada House)

制作：Dada House

編集：風穴 江 (windhole)

担当：山口正樹

ゼロからのOS自作入門

2021 年 5 月 14 日 初版第 3 刷発行

2021 年 5 月 20 日 電子版 ver3.02

著 者……………内田 公太

発行者……………滝口直樹

発行所……………株式会社 マイナビ出版

〒101-0003 東京都千代田区一ツ橋 2-6-3 一ツ橋ビル 2F

TEL 03-3556-2736 (編集)

E-mail : pc-books@mynavi.jp

URL : <https://book.mynavi.jp>

©2021 内田 公太, Printed in Japan.

- ・本書掲載内容の無断転載を禁じます。
- ・本書は著作権法上の保護を受けています。本書の無断複写・複製 (コピー、スキャン、デジタル化等) は、著作権法上の例外を除き、禁じられています。
- ・本書についてご質問等ございましたら、マイナビ出版の下記 URL よりお問い合わせください。お電話でのご質問は受け付けておりません。また、本書の内容以外のご質問についてもご対応できません。
https://book.mynavi.jp/inquiry_list/

動画ですいすい学べる
Compass Learning Video

<https://book.mynavi.jp/clv/>



オンラインでプログラミング学習!
「どこでもプログラミング」

<https://book.mynavi.jp/dokopro/>



コンピュータ関連の電子書籍が揃う専門ストア

Tech Book Zone
Manatee
ITを学ぶ電子書籍ストア [マナティ]
<https://book.mynavi.jp/manatee/>



ゼロからの OS自作入門

o p e r a t i n g s y s t e m

ついに OS 自作入門が、著者を変えて進化しました！
著者が変われば視点が変わり、新たな価値が付加されます。
変わらない部分は時代を超えた価値かもしれません。

川合秀実（『30日でできる！OS自作入門』著者）

実機で動作する OS の自作、ワクワクしませんか？
この本を読めば可能です。しかも楽しく！！
手を動かすので OS や周辺技術の理解も深まります。お勧めします。

権藤克彦（東京工業大学 情報理工学院 教授）

読むか悩んでいる人へ。読みましょう。
この本を読む以上に効果的な時間の使い方はないでしょう。
読み終わった人へ。感想を共有しましょう。
同志との出会いや新たな目標ができるかもしれません。

吟遊詩人ヴァネロピ

第 0 章 OSって個人で作れるの？
第 1 章 PCの仕組みとハローワールド
第 2 章 EDK II入門とメモリマップ
第 3 章 画面表示の練習とブートローダ
第 4 章 ピクセル描画とmake入門
第 5 章 文字表示とコンソールクラス
第 6 章 マウス入力とPCI
第 7 章 割り込みとFIFO
第 8 章 メモリ管理
第 9 章 重ね合わせ処理
第10章 ウィンドウ
第11章 タイマとACPI
第12章 キー入力
第13章 マルチタスク(1)
第14章 マルチタスク(2)
第15章 ターミナル

第16章 コマンド
第17章 ファイルシステム
第18章 アプリケーション
第19章 ページング
第20章 システムコール
第21章 アプリからウィンドウ
第22章 グラフィックとイベント(1)
第23章 グラフィックとイベント(2)
第24章 複数のターミナル
第25章 アプリでファイル読み込み
第26章 アプリでファイル書き込み
第27章 アプリのメモリ管理
第28章 日本語表示とリダイレクト
第29章 アプリ間通信
第30章 おまけアプリ
第31章 これからの道

付 録：開発環境のインストール / MikanOSの入手
／ EDK IIのファイル説明 / C++のテンプレート / iPXE / ASCIIコード表

「少し作って
すぐ動かす」
ステップ バイ ステップ
方式！